目 录

VCS篇 3
NCverilog篇 18
verilog代码规范(转) 31
Assert篇 82
V_SVerilog篇 93
nLint篇 133
spyglass篇 143
Verdi 篇 190
Makefile篇 201
Perl篇 227
TCL篇 247
gvim常用小技巧 253
shell(linux)vnc 260
ASIC设计流程简介(转) 265
DC篇 270
Spf文件分析 345
PT/sta篇 351
PT/ptpx（转）- 已用 380
Tmax篇 395
Formality篇 415
Mask简单介绍 430
Synplify知识 439
FPGA知识 467
Synplify_ise串型工作命令批处理运行 502
低功耗篇(转) 505
后端知识 562
SOC产品分析(ppa) 568
芯片设计流程 570
电路设计基础 570
DFT 617
ATPG设计测试相关 622
总线篇 631
验证篇 649
Sdf文件格式分析 892
VCD文件格式分析 895
LIB文件格式 899
Specify用法（转） 903

XA混仿 911
Finesim 混仿 914
常见soc系统架构 919
C8051知识 925
Flip80251知识 950
ARM知识 963
流水线上多周期指令执行时，流水线是否要暂停？ 998
MIPS知识 1073
OTHERS 1100
逻辑分析仪简单使用 1155
常用电路原理及分析 1157
通讯设计注意点(i2c,uart,spi等) 1226

VCS 篇

基本选项及命令介绍

Vcs 选项里面通过 -v 加库，-f file_list(该文件里面为 rtl 代码或测例)，或直接加.v/.sv 文件时，需要注意，文件内容一定为module …end 或 program…endprogram 或 package…endpackage 格式，如果是 task ,function,class 则必须包含在上述模块内，如果是以-v 方式加入文件，无法在 verdi 中 trace 到内部信号

1. 编译项+vcs+initreg+random 可以给所有的寄存器和 memory 赋一个随机的初始值，也可以设置为 0 或 1，指的是变量设置为reg 类型（wire,bit 等不会被初始化），在编译选项需要加上+vcs+initreg+random，运行选项加上+vcs+initreg+0/1/random,最后 reg 类型的变量将是运行选项的值 0/1/random；注意：当使用了上述+vcs+initreg+选项，如果在最开始又对某个 reg 类型的变量赋一个取反的值，则这时会有一个 0 宽度的脉宽
2. +vcs+initmen+random(0|1|x) 初始化存储器和多维数组；--用于前仿真
3. compier 选项先+tc 再加 tb，由于编译的顺序，可以在 tc 中改变 tb 中的一些 define 或预先 define,所以在 tb 中的 define 要求加上 ifndef 的判决
4. 如何单步运行，定位无限 0 延时的循环：当仿真时间没前进，但 Δt 时间在进行，这一般是存在延时为 0 的循环，则可以通过逐行运行来调试使用 ctrl+c 停止仿真，在 run,重复该动作，会显示运行的语句,不能+RI 编译后立即运行，要采用 2 步 法，先 complie 然后在 simv。选项要加

   -debug_all
5. FSDB 转 VCD 的方法

用法： fsdb2vcd fsdb_fname [-o vcd_file_name] [options]

参数选择：

[-bt time[unit]] [-et time[unit]]

[-s [-nocase] {signal_name [-level n] [-verilog|vhdl]}]

[-keep_last_time]

[-map_ns_u [0/x/z]] [-map_ns_y [0/x/z]] [-map_ns_z [0/x/z]]

[-sv] [-v] [-vcs] [-f config_file]

[-bt time[unit]]

指定起始时间，如果不指定，将从 FSDB 的起始时间开始。时间单位可以是 Ms/Ks

/s/ms/us/ns/ps/fs。如果不指定，将采用 ns。

[-et time[unit]]

指定结束时间，如果不指定，将从 FSDB 的结束时间结束。时间单位可以是 Ms/Ks

/s/ms/us/ns/ps/fs。如果不指定，将采用 ns。

-level n

转换所指定的层次和其向下 n 级层次结构的信号。必须和选项-s 连着使用

-o

指定输出VCD 文件的名字

-s

指定需要提取的信号或者层次

例子：

1. 转换所有的信号

%fsdb2vcd verilog.fsdb -o output.vcd

2. 转换指定时间范围和层次的信号

%fsdb2vcd verilog.fsdb -s /system/i_cpu -level 1 -bt 10 -et 1000

更多的解释，请直接在命令行输入 fsdb2vcd，就会有详细参数的解释。

5. 截取 FSDB 中一段波形的方法：

fsdbextract sorce.fsdb -bt (begintime) -et (endtime) -s /tb/udut/tp -level 0 -o destine.fsdb 时间单位为 ns

6. 当开启 -fsdb 命令是，需要设置 debussy 的环境变量，否则会提示 Error-[VPI-LOAD] VPI -load library error 错误

7. DUT_CMP_OPTIONS += +neg_tchk +overlap +sdfverbose -negdelay -sdfretain

neg_tchk 负延迟检查；overlap SDF 反标后的精确时序检查；

#sdfverbose 打印出详细的 SDF 反标信息；negdelay 允许 SDF 文件中负延迟反标

8. DUT_CMP_OPTIONS+=+warn=noTFIPC,noSDFCOM_CFTC,noSDFCOM_NIOD,noSDFCOM_RCI,noSDFCOM_ANICD,noSDFCOM_PONF,noSDFCOM_CFIIS,noSDFCOM_MINF,noSDFCOM_IANE,noSDFCOM_TANE,noSDFCOM_SWC

#不报这些 warning。这是 SDF 反标时出现的，一般都不用管，其中 TFIPC,SDFCOM_CFTC 都表示 message ID，

原句为 +warn=no[ID]|all|none

##DUT_CMP_OPTIONS += +notimingcheck+nospecify+delay_mode_zero

notimingcheck 忽略仿真中的时序检查；把 nospecify 的功能包含进去了，Specify 都是在 rtl 的 module 中

#nospecify ：库中的 cell，会有 specify 语句，指定延时信息的。Nospecify 去除延时信息。注意：如果后仿时有该选项，那么 sdf 文件的反标信息就不能起作用。

delay_mode_zero 去除仿真中延迟信息

9. Using Verilog System Tasks（产生波形的那几个函数）

Compilation

This can be done in following two ways:

•% vcs -fsdb [compile_options] verilog_filelist

•For -P tab flow, replace vcsd.tab with novas.tab, where novas.tab is available in:

<NOVAS_INST_DIR>/share/PLI/VCS/${PLATFORM}/novas.tab

The following is the use model change:

% vcs -debug_pp -P $DEBUSSY_LIB/novas.tab $DEBUSSY_LIB/pli.a [compile_options] verilog_filelist

使用Verilog 系统函数需要使用 -P 那个包

但这时不能用 -fsdb 选项（2010.07 版本支持）它需要做如下环境变量设置

% setenv DEBUSSY_HOME Debussy_installation

% setenv DEBUSSY_LIB $DEBUSSY_HOME/share/PLI/VCS/LINUX

% setenv LD_LIBRARY_PATH ${DEBUSSY_HOME}/share/PLI/lib/LINUX:$DEBUSSY_LIB

% setenv LM_LICENSE_FILE[Debussy_license]:$LM_LICENSE_FILE

down 波形的步骤：

initial begin

$fdsbDumpfile(file_name); //生成波形的名字

$fsdbDumpvars(0,tb)//down 波形的层次，写了这句话，就会开始 down 波形；

//0 表示当前层及所有子层 ,1 表示当前层 ,2 表示往下一层,3,4..依此类推.

$fsdbDumpon;

End

Stopping and resuming the dump ($dumpoff/$dumpon)

fsdbDumplimit - 限制 FSDB 文件 size

-- $fsdbDumpvars([], *)

fsdbDumpfile - 指定 FSDB 文件名

-- $fsdbDumpfile(“”)

fsdbDumpvars - Dump 指定的变量

--

fsdbDumpSingle - Dump 指定的信号

fsdbDumpvariable - Dump 指定的 VHDL 变量

fsdbSwitchDumpFile - 将 dumping 切换到另一个 FSDB 文件

-- $fsdbSwitchDumpFile(“ ”)

fsdbAutoSwitchDumpfile - 限制文件大小并在数据量过大时自动创建新的 FSDB 文件

-- $fsdbAutoSwitchDumpfile(, “ ”,< number of file>)

fsdbDumpflush - Force to Dump Result to FSDB file

fsdbDumpMem - Dump 指定的 memory 的内容

-- $fsdbDumpMem(, [, []])

$fsdbDumpon - 打开 FSDB dumping

$fsdbDumpoff - 关闭 FSDB dumping

当 DUT 的某些信号无法显示在verdi 中时，请注意 fsdb 波形是否配置为最顶层，即注意是否有 down 此模块的波形

10. sva 在波形上显示的方法：在 tb.v 中 dump 波形地方加上：
    $fsdbDumpSVA
11. 如果要增加 assert 语法，需要包含 sva_lib.sv 库
12. 用+incdir 可以改变`include "../../rtl" 语句的相对的路径，但对于没有用`include，而直接使用../../../rtl/这种做法，则相对的路径以 compile 所在的目录为基准
13. DUT_CMP_OPTIONS += +neg_tchk +overlap +sdfverbose -negdelay -sdfretain

neg_tchk 负延迟检查；overlap SDF 反标后的精确时序检查；

#sdfverbose 打印出详细的 SDF 反标信息；negdelay 允许 SDF 文件中负延迟反标

14. DUT_CMP_OPTIONS+warn=noTFIPC,noSDFCOM_CFTC,noSDFCOM_NIOD,noSDFCOM_RCI,noSDFCOM_ANICD,noSDFCOM_PONF,noSDFCOM_CFIIS,noSDFCOM_MINF,noSDFCOM_IANE,noSDFCOM_TANE,noSDFCOM_SWC#不报这些 warning。这是 SDF 反标时出现的，一般都不用管，其中

TFIPC,SDFCOM_CFTC 都表示 message ID，原句为

+warn=no[ID] |all|none

15. ##DUT_CMP_OPTIONS += +notimingcheck+nospecify+delay_mode_zero

notimingcheck 忽略仿真中的时序检查；

#nospecify ：库中的 cell，会有 specify 语句，指定延时信息的。Nospecify 去除延时信息。注意：如果后仿时有该选项，那么 sdf 文件的反标信息就不能起作用。

delay_mode_zero 去除仿真中延迟信息

路径延时说明

```
`timescale 1 ns / 1 ns
module design (out,in);
output out;
input in;
wire int1,int2;
assign #4 out=int2;//这个叫 delay_mode_distributed
buf #3 buf2 (int2,int1),
buf1 (int1,in); //
specify
(in => out) = 7; //这个叫 delay_mode_path
endspecify
endmodule
```

从上面模块可以看出 delay_mode_path 为 7ns, delay_mode_distributed 为 4+3+3=10ns,当编译选项有 delay_mode_path，这个模块的延时为 7ns，如果有 delay_mode_distributed 选项，则这个模块延时为 10ns,如果两个选项都没有，则 vcs 选用最大的数据

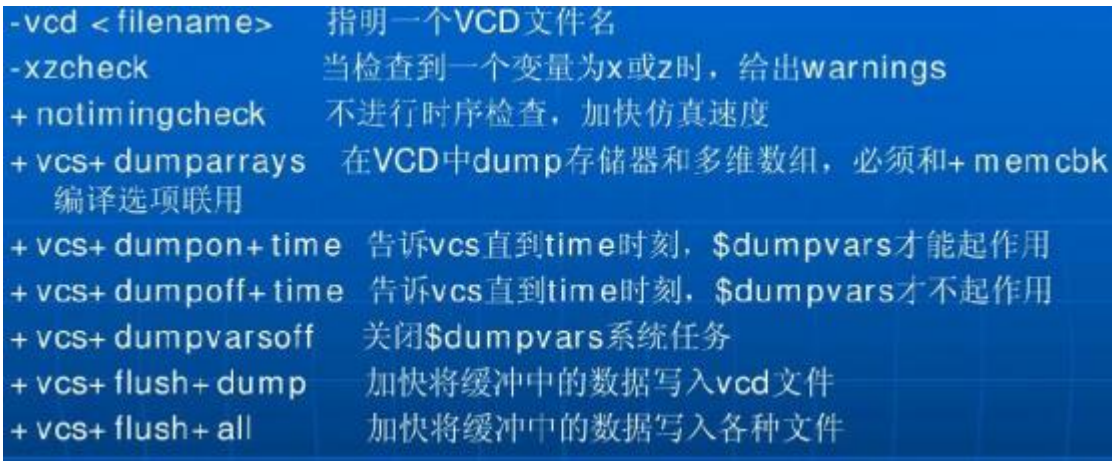
命令	描述
1. +define+<macro_name>=<value>	命令行进行宏定义，在代码中使用了`ifdef 编译指令

2. +mindelays	器件延时使用sdf 文件中的最小值(sdf 文件中的时序(min:typ:max))
3. +maxdelays	器件延时使用sdf 文件中的最大值
4. –v <filename>	导入库文件的 verilog 模型（只编译使用到的 module）,-v 在 verdi 中无法 trace,被当作一个器件,不能追踪到内部 Module 查找路径,在添加module 时,会在此目录下查找,当编译某文件,如果里面有例化某个module,则会编译这个文件下的内容
-y < directory >	Module 查找路径,在添加module 时,会在此目录下查找,当编译某文件,如果里面有例化某个module,则会编译这个文件夹下的内容 与-v 功能差不多,一个是指定文件,一个是指定文件夹
5. +incdir+directory+	指定 VCS 搜索`include 内文件的目录 使用的 include 的文件会在这些目录下查找
6. -full64	支持 64 位模式运行
7. -R	编译完后立刻运行
8. –sverilog	支持 systemverilog 语法(vcs 用,verid 用-sv)
9. +v2k	支持 verilog 2001 特性
10. -lca	Limited Customer Availability，支持动态数据类型 associative Array,dynamic Array,smart Queue,string,event,c
11. –debug_all	支持所有 debug 操作，DVE 选项，可以查看所有断点(line,time.value,event etc) all 就是加权限。 +w 是写 +r 是读+f 是 force +l 是支持ucli 的单步调试。

	+n 是 对网表操作。 具体可以看手册。 +all就是把所有的权限全开
11. –debug_acess+all	在 VCS 编译的时候不用再加-p novas.tab pli.a 来定 verdi 的路径。 直接在 simv 的 ucli 脚本里面 call $fsdbdumpfile 想 dump vpd 的时候至今换 dump file XXX 不用重新编译。版本有差异
12. –fsdb	调用 Verdi PLI 库，支持 fsdb 波形
13. +vcs+lic+wait	等待 vcs license
14. +plusarg_save	将某些 runtime options 加入到编译中，源代码中存在 $test$plusargs(“name”),如果字符匹配则为真，反之为 例：在 tc 中有代码 if($test$plusargs(“testcase0”)) $ 在 Makefile 命令中加入 make testcase0,则 if 为真，打
15. +libext+.v+.sv	在库目录内搜索指定后缀名文件进行编译
16. +verilog2001ext+.v	Specifies a file name extension for Verilog2001 source file.
17. +systemverilogext+.sv	Specifies a file name extension for SystemVerilog source file
18. +warn=[no]ID |none|all…	打开或关闭 warning 信息
19. +optconfigfile+<filename>	定义 VCS 配置文件，第一级同步寄存器不检查看 用一个文件输入
20. -o name	指定运行文件名称
21. -Mdir=directory_path	指定增量编译的目录（默认为 csrc）
22. -l sim.log	生成指定 log 文件
23. +ntb_random_seed=value	指定仿真随机种子
24. +ntb_random_seed_automatic	VCS 自动生成随机种子
25. -timescale=1ns/10ps	命令行定义仿真时间刻度， 若 rtl 中有 timescale 的定义， 则会覆盖 makefile 中的定义
26. –override_timescale=1ns/10ps	覆盖源代码中的仿真时间单位和精度 最好别用，否则会把模型的时间单位覆盖，

	可能导致模型功能错误
27. -pvalue+*tb_top.CYCLE*=value	对指定参数进行赋值 -pvalue+*tb_top.CYCLE*=${period},那么在命令行就可以使用 vcs xxx period=yy 来改变 CYCLE
28. –sv	支持 systemverilog 语法(verdi 有此配置,vcs 没有)
29. –ssf	自动加载 fsdb 文件
30. +neg_tchk	负延迟检查 Pr 后仿需要加上此选项,
31. +delay_mode_zero	去除仿真中的延迟信息，所有的延时为 0 当使用了此选项,specify 中的$setup()检测失败,似乎把 specparam 设置的变量改为 0 了.
32. -notimingcheck	忽略仿真中的时序检查，会忽略 specify 语句中的检查 小心使用,有些模拟如 flash 的时序检查是通过 specify 语句来实现,如果使用了,则无法 check 时序
+nospecify	Specify 里面的内容不起作用,相当于 0 延时和不做时序检查
+no_notifier	时序检查函数不输出变量
33. +maxdelays	选择 sdf 文件中的 max 延迟信息
34. +overlap	SDF 反标后的精确时序检查
35. +sdfverbose	打印出详细的 SDF 反标信息，超过 10 个 warning \error
36. –negdelay	允许 sdf 文件中 iopath 和 interconnect 的负延时反标 Pr 后仿需要加上此选项,很多工艺库的holdtime 为负值
37. -cm line+tgl+cond+fsm	指定覆盖率类型
38. -cm_dir directory_path (default: simv.cm)	指定覆盖率文件名字和路径
39. -cm_hier vcm.cfg (+tree tb_top.dut)	保存模块的层次信息
40. -cm_name test_name	定义覆盖率报告文件名

41. –cm_log	指定覆盖率 log 文件名
42. -assert	定义支持断言
43. +delay_mode_path	模块的延时使用路径延时
44. +delay_mode_distributed	模块的延时使用分布延时,对于分布延时,只要脉冲宽度小直接过滤掉(针对惯性延时)
45. +delay_mode_unit	模块的延时使用所有时间精度中的最小值,specify 中的延迟不起作用,#后的所有数字变为 1,单位使用最小精度，所有 timescale 中的最小精度
+tetramax	用于Tmax 产生的 atpg 验证平台不带时序仿真 Enables simulation of TetraMAX’s testbench in zero delay mode.
46. +delay_mode_zero	模块的延时使用0 延时,不指定延时模式时,vcs 使用路径延时和分布延时中的最大值
47. +loopreport	高版本 vcs 会报 loop
48．+race	自动产生race.out 文件，列出竞争
49．+cli+1,2,3	使用ucli模式，一般为 3，默认模式
50． -ucli	仿真时，进入ucli交互模式



在新版的 atpg 仿真向量中，有一些系统函数，需要在vcs 中加入库，就像 fsdb 波形需要调用函数一样

-P ./verdi201101/share/PLI/VCS/LINUX64/novas.tab

./verdi201101/share/PLI/VCS/LINUX64/pli.a fsdb 波形需要使用的库

-P ./amd64/stildpv/lib/stildpv_vcs.tab

./amd64/stildpv/lib/libstildpv.a atpg 向量中需要调用的库

控制覆盖率 scope：(only need at compile，也可以通过 pragmas to limit condition coverage)

-cm_hier cm_file

cm_file：

+/-tree instance_name [level_number]

+/-module module_name

+/-file file_name

+/-filelist file_name

+/-library

+/-node

支持通配符* ？ +

覆盖率合并：urg [assert minimal]

[-dbname dirname] [-dbname dirname/testname]

[-diff]

[-dir dir1 [dir2 ...]] [-dir dir3 ...]

[-elfile <file>]

...

[-format text/both]

[-hier <file>]

[-log file]

[-metric[+]line+fsm+cond+tgl+branch+assert+group]

[-report dir]

[-show brief/fullhier/...]

参考:$VCS_HOME/doc/pdf/cov_ref.pdf

排除 default case 方法：

urg -line nocasedef

低功耗仿真介绍

1. 命令: vcs mvsim -仿真命令
2. 命令:MVRC—检查命令 , 尽早做MVRC,频繁做MVRC,每次改动都需要做一次 MVRC
3. Checking 包含 Equivalence checking 和 static MV checking

   3.1 Equivalence check 是个 function check,其检查点有:

   a. Extends functional equivalency checking to include power effects on logic function
   b. Connects supply rails to power/ground pins
   c. Powers_down cell outputs and sequential elements
   d. Matches ieee-1801(UPF) simulation semantics
   e. Drives X into circuit using power_down functions triggered power and ground pins

   Equivalence 检查就是 formality,需要 reference and implementation data;

   Reference data 就是 RTL+UPF (经过 simulated with MVSIM 和 Checked with MVRC);

   Implementation data 就是 DC NETLIST 和 UPF’(由 DC 根据 UPF 生成);

   PR 后的 Implementation data 就是 PG_netlist 或 netlist+UPF’’(由 ICC 生成);

   对 technology libraries 要求: all cells require power pins and power_down functions.

   Verify equivalence in all power states

   a. Power on
   b. Power down

   Valid power states are defined in the power state table (PST)

   3.2Static MV Checking 能够检查到上述不能检查到的点

a. Incorrect/missing ISO,LS ,ELS cell

b. Incorrect power connectivity of ELS cell

c. Incorrect supply rail ordering

d. Incorrect power switches

e. Equivalent power connectivity in a PG netlist and corresponding UPF file

Static MV Checking 贯穿整个设计流程

--RTL 阶段:UPF consistency Architectureal checks

--Synthesized design 阶段:Architectural,structural & functional checks

--Power/Ground connected design 阶段:

Architectural,structural,functional,PG checks

4. MVRC flow ---mvgui 以界面图形方式打开工具

4.1 MVRC input

a. RTL --verilog,system verilog,vhdl

b. upf -- capture power-intent

c. ini file(archpro.ini) -runtime config settings for each of the tools

d.liberty (.db) -- latest PG-pin DB’s required,一般的设计 db 库可以不要 PG PIN,但低功耗设计的 DB 库一定需要 PG PIN .

4.2 MVRC FLOW 三步骤:

a. mvcmp -design and upf parsing ,输入 rtl/netlist ,upf,archpro.ini, 产生 PADB 数据包

b. mvdbgen - multi-voltage elaboration,输入 padb 和 liberty,生成新的 PADB 数据包

c. mvrc - support for tcl ;variables support to control tool behavior ; commands for setup ,analysis,query,debug;

4.3 MVRC Setup commands

Read_db

Read_phydb

Remove_db

Remove_phydb

Create_constraint - don't_auto_infer -type <type_name>

Create_constraint - auto_infer -type <type_name>

Create_root -type <typename> -signal <signame>

Create_root -type <typename> -file <filname>

5. 在 testbench 中需要导入 upf 库 :import::*; (与 uvm 相同),upf 库中有各种电源相关函数,如 supply_on("VDD",5.0);supply_off("VDD"); supply_partial_on("VDD",value);使用 import::*时,需要在 sim.setup 中设置路径如 upf:$VCS_HOME/linux/packages/upf

6. LP verification flow
   a. Use mv static verification first and often
   b. Simulate design with upf
   c. Simulatie design with supplies "all on "---for finding static initialization bugs
   d. Boot design starting from "all off" -for validate than lp part of the reset sequence is work properly for all design part
   e. Develop dedicated lp test

Ncverilog 低功耗仿真选项

```
+nclps_cpf+../cadence/cpf/xxx_forsim.cpf +nclps_simctrl_on
+nclps_stime+lps \
+nclps_verbose+1 +nclps_logfile+./log/lps.log
```

如何使用 upf 来实现低功耗流程的仿真验证呢?

第一步，先得有 UPF 文件，根据设计需求，使用TCL 建立脚本，建立和管理独立电压源、确定隔离、建立电平漂移等，一般是设计或者后端人员书写，验证工程师当然也可以写，具体内容参考 IEEE 1801。

第二步，仿真 case 中添加电源上电过程，使用$supply_on 函数给相应 VDD 上电。

第三步，将 UPF 嵌入到仿真命令中，即：

irun -lps_1801 sim.upf -lps_assign_ft_buf -lps_iso_verbose ...

或者

1 ncvlog ...

2 ncelab -lps_1801 sim.upf -lps_assign_ft_buf -lps_iso_verbose ...

3 ncsim ...

相关 options 解释如下：

①-lps_1801 filename: 指定符合 IEEE 1801 标准的 UPF 文件；

②-lps_assign_ft_buf: 指定 assign 赋值被当做 buffer 对待，而非默认的 wire，好处是从 always-on domain 进入和穿过 power-down domain 的信号被 force 成 x，便于 debug；

③-lps_const_aon: 对处在 power-down domain 并且直接和 always-on domain 相连接的 tie-high 或者 tie-low constant，不使能 corruption 功能；默认不使用该功能；

④-lps_enum_rand_corrupt：对于用户定义的 enum 类型数据，在电源关断后，随机从枚举列表中选择一个值作为变量值；和该命令相类似的还有

-lps_enum_right, -lps_implicit_pso 等，因为不常用，就不一一介绍了；

⑤-lps_iso_verbose: 使能 isolation 的 log 功能，这个一般需要加上；

⑥...

其它的 options 请参考 cadence 的 low-power simulation guide。

最后，运行仿真即可。

最后说一下 low power 流程验证正确性和完备性的确认方法。

①增加的 low power 流程不能影响芯片本身功能的正确性，比如通过电源关断来降低芯片功耗，则关断再打开后，芯片还可以正常work。这部分可以利用原有的 self-check 验证环境来确认。

②利用 log 文件分析，上述仿真命令-lps_verbose 和 UPF 文件会为仿真输出与 low power 相关的 warning, error 以及 assertion 信息，通过 log 文件可以 check 流程的正确性；

③自动的 assertion checker 分析，使用-lps_verify 选项可以自动检查电源关断顺序（隔离->复位->断电）和电源打开过程（上电->解复位->解隔离），如不满足该时序要求，会自动报错；

④利用覆盖率保证验证完备性，使用-lps_vplan 选项依据 UPF 文件生成 low power 验证计划，通过 simvision 提供的 coverage 接口，得到 low power 相关的功能覆盖率；

低功耗仿真注意事项

1. 当 power domain 下电时,仿真器会把所有该 domain 的变量赋为 X 态,尤其要注意各个 memory,flash,rom 的模型,由于 flash,rom 是带有记忆功能,所以仿真时在下电前先保存内部内容,然后在上电后再后门赋值回去.
2. 当 force 了某个 power domain 的某个信号,当下电又重新上电时,force 会失效-----此点可能会被改进

NCverilog 篇

基本选项及命令介绍

+access<+/-rwc> Turn on read, write and/or connectivity access

+allow_unused_properties Allow simulator to enable all properties

-allowredefinition Allow mutiple files to define the same object

+amsconnrules+ specify connect rules to use

+amsmatlab Dynamically link vpi code for AMS/Matlab

-amsmt_enable to enable AMS multithread capability

-amsmt_nthreads number of threads for AMS multithreading.

-amsvhdl_ext Override extensions for VHDL AMS sources

+append_key Append keystrokes to existing key file

-arr_access	Allow tf_nodeinfo access to Verilog arrays
-asext	Add extensions to assembly sources
+assert	Enable PSL language features
+assert_count_traces	Use trace-based counting for assertions
+assert_sc	Enable PSL language features
+assert_vhdl	Enable PSL language features
+assert_vlog	Enable PSL language features
-bb_celldefine	Blackbox all verilog modules within `celldefine
-bb_nonsynth	Blackbox unsynthesizable modules in halsynth
-bb_unbound_comp	Ignore unbounded component for synthesis checks
-bb_vital	Blackbox design-units containing VITAL constructs
-c	Parse and elaborate, do NOT simulate
-catcxx	Positional option used to combine C++ sources
-catcxxsize	Sets the CATCXX partition size for large groups
-categories	List the categories and their descriptions
-ccext	Add extensions to C sources
-check	Specify checks and categories of checks
+checkargs	Check command-line arguments for validity
-clean	Deletes previous INCA_libs directory if is exists
-comb_depth	Enable Logic Depth calculation
+compile	Parse only, do NOT elaborate
+controlassert+	Specifies a file containing assertion controls
-cpost	Compile C files after elaboration
+crshell	Create shell files for import mode
-cxxext	Add extensions to C++ sources
-D	Define a macro for the C and C++ compiler
-date	Print date and time when each engine is invoked
-debug	Equivalent to -access +rw, Specman debug
-debugscript	Specify a debug script file name

+define+	Define a macro from command line
-defineall	Define macro from command line for all compilers
+defparam+	Redefine the value of a Verilog parameter
-delay_mode	Delay mode {Zero,PUnit,Unit,Path,Distr,None}
+delay_mode_distributed	Use distributed delay mode
+delay_mode_path	Use path delay mode
+delay_mode_punit	Use precision unit delay mode
+delay_mode_unit	Use unit delay mode
+delay_mode_zero	Use zero delay mode
-design_facts_file	Generate design facts during structural checks
-design_info	Design Information file
-distcomp	Option used to turn distributed compilation on
-distcompargs,	Pass user specified argument to distributed comp
-distcompjobs	Number of parallel distributed compiles to run
-distplat	Specify underlying platform used for distrib comp
-dpi	Add appropriate build options for dpi designs
-dpi_void_task	Return value of export/import tasks will be VOID.
+dut_prof+	Profiler report contains summary for design unit
-dynamic	Build a shared object for simulation
-dynvhpi	Enable user to create VHDL drivers at run time
-efence	Debug ncsim with Electric Fence.
-efenceelab	Debug ncelab with Electric Fence
-end	Terminate the list of files
-extbind	Bind file for binding SV/VHDL to SV/VHDL
-F	Scan file for args relative to file location
-f	Scan file for args relative to irun invocation
-forceelab	Force ncelab to execute
-format	Enables shorter compiling messages from ncsc_run
-g	Turn on C debugging

-gcc_vers 3.2.3 or 4.1 Linux only

-gdb Run ncsim under gdb

-gdbelab Run ncelab under gdb

-gdbpath Use the provided gdb instead of what is shipped

+genassert_synth_pragma Enable generating assertions from synth pragma

-gnu Choose the GNU C and C++ compiler

+gui Invoke the Graphical User Interface

-h Print a minimal help message

-hal Call hal instead of ncsim

-halargs Pass options directly to hal

-halsynth_detailcheck Perform detailed check on unsynthesizable modules

+helpalias Show the different ways to enter an option

+helpall Display all supported option

+helpargs Print help for all the options in use

+helpfileext Show all the file types and their extensions

+helphelp Print out all the options controlling help

-helpncverilog Show the ncverilog form of the options

-helpshowmin Show the minimum characters required for dash opt

+helpshowsubject Show all the subjects for -helpsubject

+helpsubject Display help on the specified subject

+helpverbose Show the verbose help text

+helpwidth+ Set max width for help messages (def 89)

-I Directory to search for C/C++ include files

+ieee1364 Report errors according to IEEE 1364 standards

+import Prepare this verilog design for import to VHDL

+incdir+ Specify directories to search for `include files

-inst_top Specify the top-level instance for HAL analysis

-iusld Prefix `ncroot`/tools/lib path to LD_LIBRARY_PATH

-iusldno Disable the -iusld option

-k Set key file name

-L Directory to search for lib files

-l Set logfile name

-l archive or shared library to be linked in

-level Specify levels in HAL analysis

+libext+ Specify extensions to be used for the -y search

+liborder Library search rule (see documentation)

+librescan Library search rule (see documentation)

+libverbose Print verbose messages about instance binding

-linksysc Use libsystemc.so (dynamic) or libsystemc_ar.a

-lintpragma Process lint pragma in the design

-loadcfc Dynamically load a CFC application

-loadfmi Dynamically load an FMI library

+loadpli1= Specify the PLI1 library_name:boot_routine(s)

+loadsc+ Specify SystemC lib to be dynamically loaded

-loadvhpi Dynamically load a VHPI application

+loadvpi= Specify the VPI library_name:boot_routines(s)

-location Print the location of the installation

-log_amsspice Place amsspice output into the specified logfile

-log_hal Place the hal output into the specified logfile

-logic_depth Enable Logic Depth calculation

-loop_unroll_size Specify the loop unroll limit for halsynth

+max_error_count+ Specify the maximum number of errors processed

+mixesc Handle escaped identifiers in imported model

+multisource_int_delays Make interconnect timing be multisource capable

+name+ Generate snapshot with specified name

-native Use the native C and C++ compiler

+nc64bit Invoke 64bit version

+nca_ext+ Override extensions for archive files

+ncafile+	Specify an access file to be used
+ncams	Force Verilog-AMS and VHDL-AMS compilation
+ncamsfastspice	Enable Fast SPICE simulator (UltraSim)
+ncamslic	Check out an AMS license
+ncamspartinfo+	Mixed-signal partition information
+ncamsv_ext+	Override extensions for Verilog-AMS sources
+ncanalogcontrol+	Specify analog simulation control file
+ncanno_simtime	Enable delay annotation at simulation time
+ncappend_log	Append output log to existing log
-ncb_environment	Specify environment file to be loaded by Ncbrowse
-ncb_file	File for Ncbrowse to load command line arguments
-ncb_filter	Filter for report generation by Ncbrowse
-ncb_format	Set the format of messages in the report
-ncb_nodefaultenv	Prevent Ncbrowse from using default environment
-ncb_order	Set the order in which items are shown
-ncb_report	Specify the report file to be created by Ncbrowse
-ncb_sortby	Specify a sort order to Ncbrowse for report
+ncbatch	Run simulation in batch mode, this is the default
+ncbinding+	Force explicit submodule or unit L.C:v binding
+ncc_ext+	Override extensions for C sources
+ncccargs	Pass arguments to the C compiler
+nccd_lexpragma	Process preprocessor directive before lex pragmas
+nccds_implicit_tmpdir+	Specify location for design data storage
+nccds_implicit_tmponly	Force tools to read design data only from tmpdir
+nccdslib+	Specify a cds.lib file to be used
+ncchecktasks	Check that all $tasks are built-in system tasks
+ncchkdigdisp	Perform digital net's discipline compatibility
+ncconffile+	Generate a configuration file with the given name
+ncconfflat	Requires -CONFFILE, generate a VHDL flat config

+ncconfhier Requires -CONFFILE, gen VHDL hierarchical config

+ncconfname+ Requires -CONFFILE, specify output config name

+nccontrolrelax+ Enable specific relaxed VHDL interpretation

+nccovdesign+ Select coverage design name

+nccovdut+ Select DUT for Coverage

+nccoverage+ Enable coverage instrumentation

+nccovfile+ Specify coverage instrumentation control file

+nccovnomodeldump Disable coverage design database (model) dumping

+nccovoverwrite Enable overwrite of coverage output files

+nccovtest+ Select coverage test name

+nccovworkdir+ Select coverage workdir

+nccpg+ Assigns to all generics/params of this name

+nccpp_ext+ Override extensions for C++ sources

+ncdebug Equivalent to -access +r

+ncdefault_ext+ Override the default extension map

+ncdesign_top+ Specifies top design unit for design-top comp

+ncdisable_enht Disable enhanced timing features

+ncdiscapf Disable the capital F input file mode

+ncdiscipline+ Discipline to use for undisciplined digital wires

+ncdisres+ Set discipline resolution

+ncdpiheader+ Create DPI header file for export functions

+ncdresolution Sets discipline resolution to '-disres detailed'

+ncdumpports_format+ Specify EVCD format flag for $dumpports

+ncdynlib_ext+ Override extensions for dynamic library files

+nce_ext+ Override extensions for e sources

-ncelab_args, Pass arguments to elaborator (ncsc_run compat)

+ncelab_compile requires -CONFILE, compile the configuration file

+ncelabargs+ Pass arguments to elaborator

+ncelabexe+ Specify elaborator with statically linked PLI

+ncelabfile	File for generated elab options from import
+ncendlib	Terminate the list of library files
+ncendstage	Terminate the list of e stage files
+ncerror+	Increase the severity of a warning to an error
+ncescapedname	Print out escaped names in logfile
+ncexit	Exit simulation instead of issuing a TCL prompt
+ncexpand	Force expansion of all vector nets
+ncextassertmsg	Prints Extended Assert message Information
+ncextend_tcheck_data_limit/	Relax timing check data limit
+ncextend_tcheck_reference_limit/	Relax timing check reference limit
+ncfatal+	Increase the severity of a warn/error to fatal
+ncgateloopwarn	Enable potential zero-delay gate loop warning
+ncgenafile+	Generate an access file for PLI and TCL
+ncgeneric+	Associate value with top-level generic
+ncgnoforce	Assigns the value if default value not found
+ncgpg+	Assigns to all generics/params of this name
+ncgverbose	Logs the gpg activity to the ncelablogfile
+nchdlvar+	Specify an hdl.var file to be used
+nciereport	Generate interface element report
+ncignore_defexpr	Ignore default expressions on variable,signal,...
+ncinitbiopz	Initialize boundary inout port to 'Z'
+ncinitialize+	Initialize variables in the design
+ncinitzero	Enable zero initialization of time and integer
+ncinput+	Read TCL commands from file
+ncinsert+	Specify string to be inserted after matching comp
+ncintermod_path	Make interconnects be multisource capable
+nclexpragma	Enable lexical pragma processing
+nclib_binding	Defaults back to the IUS5.4 binding search order
+nclibcell	Mark all cells with `celldefine

+nclibdirname+

Specify directory name to store created library

+nclibdirpath+	Relative path where libraries should be created
+nclibmap+	Specify the library mapping file
+nclibname+	Specify the name of a library to search
+nclicq	Queue simulation till license is available
+nclinedebug	Enable line debugging capabilities
+nclist	Produce a VHDL source listing in specified file
+nclog_ncelab+	Place the ncelab output into specified logfile
+nclog_ncsc_run+	Place the ncsc_run output into specified logfile
+nclog_ncsim+	Place the ncsim output into the specified logfile
+nclog_ncvhdl+	Place the ncvhdl output into specified logfile
+nclog_ncvlog+	Place the ncvlog output into specified logfile
+nclog_svpp+	Place the svpp output into specified logfile
+nclps_assign_ft_buf	Disable continuous assignment as feed through net
+nclps_cpf+	Specify a CPF file for low power simulation
+nclps_iso_off	Turn off port isolation
+nclps_logfile+	Specify a log file for low power simulation
+nclps_off	Turn offflow power simulation
+nclps_rtn_lock	Lock the retained reg value
+nclps_rtn_off	Turn off state retention
+nclps_simctrl_on	Enable runtime control over low power simulation
+nclps_stime+	Specify a time to start low power simulation
+nclps_stl_off	Turn off state loss
+nclps_verbose+	Specify a level of information reporting
+nclps_verify	Enables automatic Low Power verification
+nclps_vhdlpwron	Brings back VHDL objects to a defined state
+ncls_all	Run ncls with the -all option
+ncls_dep	Run ncls with the -dep option

+ncls_so Run ncls with the -source option

+ncls_ss Run ncls with the -snapshot option

+ncmakelib+ Compile HDL files into specified library

+ncmatchinst+ Specify name of instance to match for -INSERT

+ncmaxdelays Select maximum delays for simulation

+ncmindelays Select minimum delays for simulation

+ncmodelincdir+ Specify a list of directories separated by :

+ncmodelpath+ For Verilog-AMS, specify list of source files

+ncmultview Allows selection of arch/config for binding

+ncnamemap_mixgen Do name mapping from VHDL generics to Vlog params

+ncno_cross_def_bind Suppress cross-language default binding

+ncno_notifier Ignore notifiers in timing checks

+ncno_sdfa_header Do not print the SDF annotation header

+ncno_tchk_xgen Turn off X-generation in VITAL timing checks

+ncno_vpd_msg Turn off VITAL pathdelay warnings

+ncno_vpd_xgen Turn off X-generation in VITAL pathdelays

+ncnobinding+ Skip instances of unit given as argument

+ncnobuiltin Do not use any built-in IEEE operators

+ncnocifcheck Disables constraint checking in VDA functions

+ncnocopyright Suppress printing of copyright banner

+ncnodeadcode Turn off dead code optimization

+ncnoesp Disable edge-sensitive iopath delays

+ncnoipd Ignore interconnect delays

+ncnokey Suppress generation of the default keyfile

+ncnolicpromote Do not use a mixed language license

+ncnolicsuspend Disable suspending licenses for SIGTSTP

+ncnoline Do not locate source line on errors

+ncnolog Suppress generation of the default logfile

+ncnomempack Do not pack memories

+ncnomxindr Do not generate NOMXINDR error; split net instead

+ncnontcglitch Suppress delayed net glitch suppression message

+ncnoparamerr Do not flag setting undefined parameters as error

+ncnopragmawarn Disable pragma related warning messages.

+ncnoscynceverydelta Turn Delta cycle accuracy off

+ncnosncomp Do not compile Specman input files

+ncnosource Do not check source file timestamps in update

+ncnospecview Do not invoke the Specview GUI

+ncnostdout Turn off output to screen (terminal)

+ncnotimezeroasrtmsg Suppress printing of time zero assert messages

+ncnovitalaccl Turn off VITAL acceleration

+ncnovitalcheck Suppress VITAL compliance checking

+ncnowarn+ Disable printing of the specified warning

+ncnoxilinxaccl Turn off Xilinx acceleration

+ncntc_level+ Select NTC algorithm 1 or 2 (default is 2)

+ncntc_verbose Display verbose information about NTC process

+ncntcnotchks Generate NTC delay while removing timing checks

+nco_ext+ Override extensions for object files

+ncomicheckinglevel+ Specify OMI checking level {Min, Std, Max}

+ncoverride_precision Override the timescale precision in Verilog

+ncoverride_timescale Override the timescale directives in Verilog

+ncpassword Prompt for sim passwd for SimVis walkup connect

+ncpli_export Export symbols from loadpli, loadvpi

+ncpliverbose Print information for PLI/VPI task registration

+ncppdb+ Invoke the post-processing environment

+ncppe Enter post-processing mode

+ncpragma Enable pragma processing

+ncpreserve Preserves resolution of single-driver sigs

+ncprofile Generate a run-time profile of the design

+ncprofoutput+ Specify an output file for profiling data

+ncprofthread Allow threaded processes to profile

+ncpromt Prompts to select arch/config/view for entity/mod

+ncpropspath+ Specify analog occurrence property database file

+ncrandwarn Enable all SV randomize failure warnings

+ncredmem Use reduced memory image size

+ncreflib+ Add the library to the list of libraries searched

+ncrelax Enable relaxed VHDL interpretation

+ncrun Begin simulation automatically

+ncs_ext+ Override extensions for assembly files

+ncsavechoice+ Specify name of file in which to save bindings

+ncsaveenv Save the shell environment variables

-ncsc_msgs Tell ncsc to run ncsc messages on or off

+ncscargs+ Pass arguments to ncsc_run

+ncscope_discipline+ Specify one scope based discipline

+ncscparameter+ Associates values with top level SystemC params

+ncscprocessorder+ Allow System C process order to vary

+ncscscreateviewables Create ncsc_viewable objs inserted by ncsc_wizard

+ncscynceverydelta+on|off Turn Delta cycle accuracy on

+ncsdf_cmd_file+ Specify file of SDF annotation commands

+ncsdf_nocheck_celltype Do not check the accuracy of CELLTYPE field

+ncsdf_nortis Disable retain input sense

+ncsdf_precision+ Specify precision which SDF data will be modified

+ncsdf_simtime Allow SDF annotation during simulation

+ncsdf_verbose Include detailed information in SDF log file

+ncsdf_worstcase_rounding Truncate SDF min delays, round max

+ncsdfnowarn Do not report SDF warnings

+ncseq_udp_delay+ Specify a constant delay for sequential UDPs

+ncsetdiscipline+ Set discipline for a specified scope.

+ncshare Reuse any available view with NCUID

-ncsim_args, Pass arguments to simulator. (ncsc_run compat)

+ncsimargs+ Pass arguments to simulator

+ncsimcompatible_ams+ Specify compatibility language hspice or spectre

+ncsimexe+ Specify simulator with statically linked PLI

+ncsimfile File for generated sim options from import

+ncsmartlib Specifies multiple library compilation in OIC

+ncsmartorder Order-independent compilation (OIC) for VHDL

+ncsmartscript+

常用命令

1. +nctimescale+1ns/1ps 与 vcs 不同格式
2. +defparam+ 修改 parameter 参数如

   +defparam+M31USBT200HL055P_sim.hs_type=${tc} 可以通过命令行修改 tc 来修改 hs_type 这个 parameter

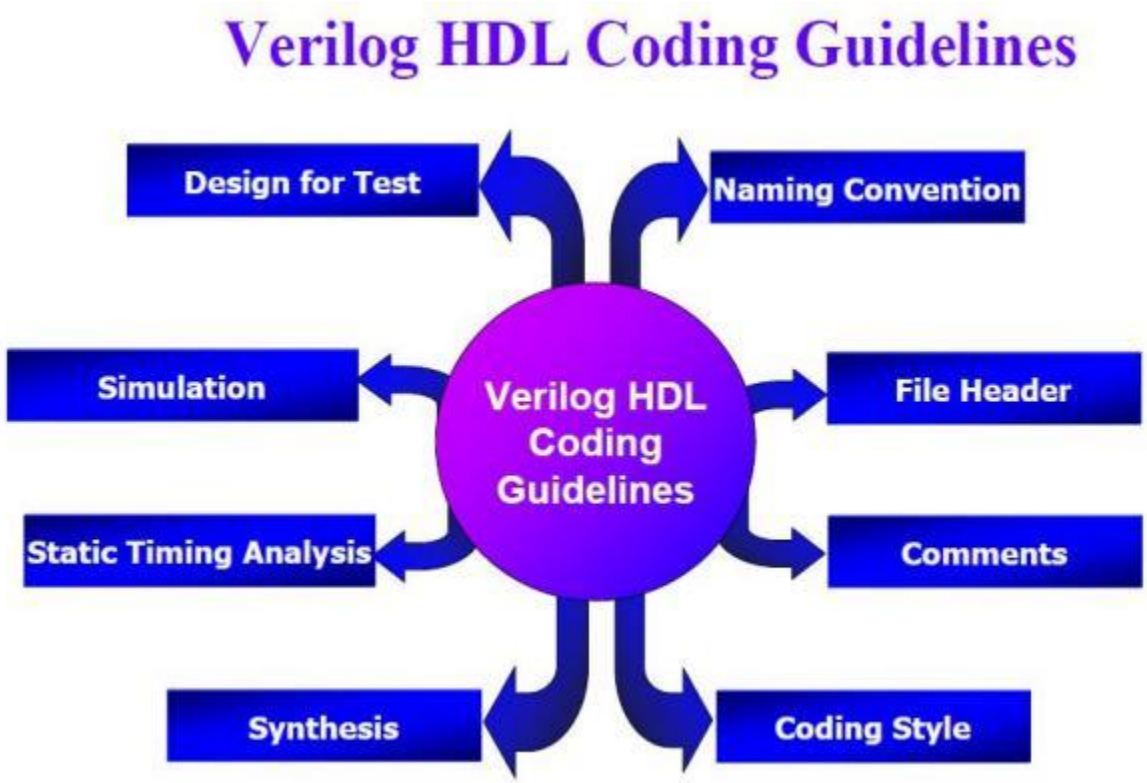
   与 vcs 的-pvalue+选项功能一样
3. +delay_mode_zero 取消所有延时
3. 支持$fsdbDumpfile 等函数需要加

+loadpli1=debpli:novas_pli_boot 命令

verilog 代码规范(转)

本文综述

本文主要从 Naming Convention,File Header,Comments,Coding Style,Synthesis,Static 描述 Timing Analysis,Simulation,Design for Test 等八个方面介绍 verilog 代码设计规范.



术语说明

Severity Levels Definition 严格等级定义

Mandatory 1 (M1) -强制性1

Rules must be followed. If not, design must be fixed.

此等级规则必须遵守,如果不符合,设计必须更改

Mandatory 2 (M2) - -强制性2

Rules should be followed. If not, documentation must be provided.
此等级规则应该遵守,如果不符合,必须提供文档说明

Recommended (R) -建议等级

Rules are recommended to be followed in the design

Why Coding Guidelines 为何要编码指南?

CAD tools is essentiality to be involved in each stage of the ASIC design flow
在ASIC设计过程中,CAD工具非常重要.

However, every CAD tool has its own capability and limitations
然而,每个CAD工具都有它自己的能力和局限性

Designer must be aware of the capability and limitations of each CAD tools he used
设计者必须熟悉每个使用的CAD工具的能力和局限性

Coding Guidelines are required for specific tools to obtain best performance and avoid exhausted iteration
为了获得工具最好的性能以避免疲惫迭代., 要求按照工具的特性进行编码

Verilog 设计规范

Naming Convention Rule Lists 命名约定规则列表

	Naming Convention	Level
[S.NC.1]	A file must contain at most one module unit 一个文件最多只能包含一个 module 模块	M2
[S.NC.2]	File name should be identical to design unit name 文件名应该与设计模块名一致	R
[S.NC.3]	Only alphanumeric and underscore are allowed in naming 只有数字字母和下划线可以用来命名	M1
[S.NC.4]	Names starting with a letter 名字应该以字母开头	M2
[S.NC.5]	Names can not be distinguishable in letter case only 名字不用只用大小写来区分	M1
[S.NC.6]	Use underscores to separate names composed of several words 使用下划线来分开由几个单词构造的名字	R
[S.NC.7]	Use consistent signal names throughout the hierarchy 在整个层次中使用相同的信号名称	M2
[S.NC.8]	Parameters and text macros for constant names are uppercase letters 参数和宏定义使用大写字母	M2
[S.NC.9]	Ports, signals, constructs and instance names are lowercase letters 端口,信号,结构体和实例单元的名字使用小写字母	M2
[S.NC.10]	Use meaningful names 使用能够表示一定含义的单词来名字	R
[S.NC.11]	Consistent clock signal name	M2

一致的时钟信号名称

[S.NC.12]	Consistent reset signal name	M2
	一致的复位信号名称	
[S.NC.13]	Consistent active-low signal name	M2
	一致的低电平信号名称	
[S.NC.14]	For FSM variables, naming in <fsm_cs>, <fsm_ns>	R
	对于状态及变量,使用<fsm_cs>, <fsm_ns>命名	
[S.NC.15]	For latch variables, end in _lat	R
	对于 latch 类型的变量,以_lat 结尾	
[S.NC.16]	For tri-state signals, end in _z	R
	对于三态信号名,用_z 结尾	
[S.NC.17]	For asynchronous signals, end in _a	R
	对于异步信号名,用_结尾	
[S.NC.18]	Verilog and VHDL keywords are prohibited	M1
	命名不能使用Verilog and VHDL 的关键字	
[S.NC.19]	The length of signal names does not exceed 32 characters	M2
	名字的长度不能超过 32 个字母	
[S.NC.20]	Descriptive and consistent abbreviation	R
	描述和缩写需要一致	
[S.NC.21]	Document abbreviations and additional naming conventions	R
	文档缩写和附加命名约定	
[S.NC.22]	Instance names must be identical to the module name	M2
	实例名称必须与模块名相同	
[S.NC.23]	Consistent ordering of bits for describing multibit buses	M2
	要以特定的顺序来描述多位宽总线	

Naming Convention 命名约定

[S.NC.1]	A file must contain at most one module unit	M2

一个文件最多只能包含一个 module 模块

Design structure is more distinct—设计风格更加清晰

[S.NC.2] File name should be identical to design unit name R

文件名应该与设计模块名一致

<design_unit_name>.v, where, <design_unit_name> is the name of the design unit (i.e., module name).

arbiter.v: Verilog design file for module arbiter.

[S.NC.3] Only alphanumeric and underscore are allowed in naming M1

只有数字字母和下划线可以用来命名

[A-Z, a-z, 0-9, _]. However, it is prohibited for using consecutive underscores.

[A-Z, a-z, 0-9, _]可以用来命名,但连续的下划线被禁止使用

[S.NC.4] Names starting with a letter M2

名字应该以字母开头

Names starting with an underscore may cause syntax error and is not allowed with some EDA tools.

在一些 EDA 工具中,以下划线开头的命名可能会有语法错误而不能使用

[S.NC.5] Names can not be distinguishable in letter case only M1

名字不用只用大小写来区分

Do not use ABC, abc, and aBc to represent three different names.

不要使用 ABC, abc, and aBc 来表示三个不同的命名

[S.NC.6] Use underscores to separate names composed of several words R

使用下划线来分开由几个单词构造的名字

RAM address bus can use ram_addr for naming.

RAM 的地址总线可以使用 ram_addr 命名

[S.NC.7] Use consistent signal names throughout the hierarchy M2

在整个层次中使用相同的信号名称

```
block1 u_block1 (
        .rst_n          (rst_n),
        .clk            (clk),
        .addr           (addr),
        .wr_data        (wr_data),
        . . .,
        );
```

```
block1 u_block1 (
        .reset_n        (rst_n),
        .clock          (clk),
        .address        (addr),
        .WR_data        (wr_data),
        . . .,
        );
```

[S.NC.8] Parameters and text macros for constant names are uppercase letters M2

参数和宏定义使用大写字母

```
parameter DATA_WIDTH = 32;
`define AHB_TRANS_SEQ 2'b11
```

[S.NC.9] Ports, signals, constructs and instance names are lowercase letters M2

端口,信号,结构体和实例单元的名字使用小写字母

- **Ports**

```
input           rst_n;
output          wr_data;
```

- **Signals**

```
reg [7:0]       a_reg;
wire            we_n;
```

- **Constructs**

```
task            busy;
function        call_addr;
```

- **Instance names**

```
block1  u_block1 (...
```

[S.NC.10] Use meaningful names R

使用能够表示一定含义的单词来名字

Naming should represent the purpose of the item.

名字应该能够代表一定的含义

[S.NC.11] Consistent clock signal name M2

一致的时钟信号名称

Recommend using clk as a substring for the clock signal naming.

建议使用 clk 作为时钟信号的名字

[S.NC.12] Consistent reset signal name M2

一致的复位信号名称

Recommend using rst as a substring for the reset signal naming.

建议使用 rst 作为复位信号的名字

[S.NC.13] Consistent active-low signal name M2

一致的低电平信号名称

Recommend using _n as the suffix for the active-low signal naming.

建议使用以_n 尾缀来表示低电平

[S.NC.14] For FSM variables, naming in <fsm_cs>, <fsm_ns> R

对于状态及变量,使用<fsm_cs>, <fsm_ns>命名

[S.NC.15] For latch variables, end in _lat R

对于 latch 类型的变量,以_lat 结尾

[S.NC.16] For tri-state signals, end in _z R

对于三态信号名,用_z 结尾

[S.NC.17] For asynchronous signals, end in _a R

对于异步信号名,用_结尾

[S.NC.18] Verilog and VHDL keywords are prohibited M1

命名不能使用Verilog and VHDL 的关键字

[S.NC.19]	The length of signal names does not exceed 32 characters 名字的长度不能超过 32 个字母	M2
[S.NC.20]	Descriptive and consistent abbreviation 描述和缩写需要一致	R
[S.NC.21]	Document abbreviations and additional naming conventions 文档缩写和附加命名约定	R

Exp. recommend using ram_addr instead of ra to represent ram address

建议使用 ram_addr 代替 ra 来表示 ram address

[S.NC.22]	Instance names must be identical to the module name 实例名称必须与模块名相同	M2

Module instantiated once:

```
decoder u_decoder (...);
```

Module with multiple instantiated:

```
mux4 u_mux4_1 (...);
mux4 u_mux4_2 (...);
```

[S.NC.23]	Consistent ordering of bits for describing multibit buses 要以特定的顺序来描述多位宽总线	M2

Recommend using [x:0] in Verilog.

在 verilog 语法中推荐使用[X:0]格式

File Header Rule Lists 文件头规则列表

	File Header	**Level**
[S.FH.1]	Each file must include a file header 每个文件必须有一个文件头	M1
[S.FH.2]	File header must start and end in file header boundary tags 文件头必须有起始和结束标示边界	M1

[S.FH.3]	Necessary fields for a file header 文件头必须包含的内容	
[S.FH.3.1]	Include file name 包含文件名	M2
[S.FH.3.2]	Include author and contact information 包含作者和联系方式	M2
[S.FH.3.3]	Include a release version and version description 包含发布版本和版本描述	M2
[S.FH.3.4]	Include a release date 包含发布日期	M2
[S.FH.3.5]	Include a purpose section 包含目的部分	M2
[S.FH.3.6]	Include a parameter description 包含参数描述	M2
[S.FH.4]	Other header reuse issues 其他头文件重用问题	
[S.FH.4.1]	Include the reset strategy description 包含复位策略描述	R
[S.FH.4.2]	Include the clock strategy description 包含时钟策略描述	R
[S.FH.4.3]	Include the critical timing description 包含关键时序描述	R
[S.FH.4.4]	Include the test feature description 包含测试特性描述	R
[S.FH.4.5]	Include the asynchronous interface description if existed 如果有异步电路,包含异步接口描述	R
[S.FH.4.6]	Include the scan methodology description 包含 scan 扫描策略描述	R
[S.FH.5]	Additional construct should include a header	M2

附加的结构体(函数)也应该包含头文件

[S.FH.6]	Additional construct header must start and end in construct header boundary tags 附加的结构体(函数)头文件应该有起始和结束标志线	M2
[S.FH.7]	Necessary fields for additional construct header 附加的结构体头文件应该包含的内容	
[S.FH.7.1]	Include construct name and data type of return values 包含结构体(函数)名字和返回的数据类型	M2
[S.FH.7.2]	Include construct type 包含结构体(函数)的类型	M2
[S.FH.7.3]	Include a purpose section 包含结构体(函数)目的	M2
[S.FH.7.4]	Include a parameter description 包含结构体(函数)参数描述	M2

File Header

```
//+FHDR----------------------------------------------------------------------
// (C) Copyright Company Industrial technology Research Institute (ITRI)
// All Right Reserved
//---------------------------------------------------------------------------
// FILE NAME: decoder.v
// AUTHOR: phchen
// CONTACT INFORMATION: phchen@itri.org.tw
//---------------------------------------------------------------------------
// RELEASE VERSION: V1.0
// VERSION DESCRIPTION: First Edition no errata
//---------------------------------------------------------------------------
// RELEASE DATE: 10-31-2002
//---------------------------------------------------------------------------
// PURPOSE:  HADDR is used to decoded for generating HSELx signals to
//           select outputs to the AHB system slaves, and controls the read
//           data multiplexer.
//---------------------------------------------------------------------------
// PARAMETERS:
//PARAMETER NAME        RANGE         DESCRIPTION           DEFAULT VALUE
//ADDR_WIDTH            [32,16]       width of the address  32
//-FHDR----------------------------------------------------------------------
```

Additional Construct Header

```
//+CHDR-------------------------------------------------------------------
// CONSTRUCT NAME & RETURN VALUES:
// task_write(haddr, hwrite, hwdata)
//-------------------------------------------------------------------
// TYPE: task
//-------------------------------------------------------------------
// PURPOSE:  input address, write, data signal to execute system write
//-------------------------------------------------------------------
// PARAMETERS:
//PARAMETER NAME        RANGE          DESCRIPTION        DEFAULT VALUE
//DATA_WIDTH           [32,16]         width of the data  32
//-CHDR-------------------------------------------------------------------
```

Comments Rule Lists 注释规则列表

	Comments	Level
[S.COM.1]	Use comments for port declarations 对端口声明使用注释	**M1**
[S.COM.2]	Use comments for internal signal declarations 对内部信号声明使用注释	R
[S.COM.3]	Use comments for functional sections 对函数部分声明使用注释	R
[S.COM.4]	Use one-line comments instead of multiple-line comments 使用单行注释而不用多行注释	R
[S.COM.5]	Use comments for cell instantiations 对实例化单元进行注释	**M2**
[S.COM.6]	Delete unnecessary codes	**M1**

删除不必要的代码

[S.COM.7] Use comments for synthesis directives **M1**

对可综合的指令使用注释

[S.COM.8] Use comments for compiler directives **M1**

对可编译的指令使用注释

Aid IP users to understand the design concept of original designer.

Improve the readability and portability of the code.

Help for IP maintenance and management.

Comments 注释

[S.COM.1] Use comments for port declarations **M1**

对端口声明使用注释

```
input           rst_n; // system reset
input           clk; // system clock
output [31:0]   wr_data; // system write data bus
```

[S.COM.2] Use comments for internal signal declarations R

对内部信号声明使用注释

Internal signal, ex. wire and reg should have descriptive comments.

如,wire,reg 等内部信号应该有描述性的注释

[S.COM.3] Use comments for functional sections R

对函数部分声明使用注释

Functional always blocks or assign statements should have descriptive comments.

Always 块,assign语句应该有详细的注释

[S.COM.4] Use one-line comments instead of multiple-line comments R

使用单行注释而不用多行注释

Use one-line comments // , instead of multiple-line comments /*…..*/.

注释采用单行格式,而不用多行格式

[S.COM.5] Use comments for cell instantiations **M2**

对实例化单元进行注释

```
and u_and1(A_out, SEL_b, A); //explain the reason
```

[S.COM.6] Delete unnecessary codes **M1**

删除不必要的代码

Unnecessary codes represents old code, or any unused code.

Keep the HDL code as clean as possible.

尽可能保持代码的整洁

[S.COM.7] Use comments for synthesis directives **M1**

对可综合的指令使用注释

```
case (select) // synopsys full_case
  3'b000: y = 1'b1;
  3'b001: y = 1'b0;
  3'b101: y = 1'b0;
  3'b111: y = 1'b1;
endcase
```

[S.COM.8] Use comments for compiler directives **M1**

对可编译的指令使用注释

```
`ifdef FOR_SIMULATION // show some test messages for debug
...
`else // NORNAL CONFIGURATION, i.e., for synthesis
   ...
`endif // end FOR_SIMULATION
```

Coding Style Rule Lists 代码风格规则列表

	Coding Style	Level
[S.CS.1]	Code should be aligned in a tabular format 代码应该使用TAB 键对齐	M2
[S.CS.2]	Use space instead of tab stops for code indentation 代码之间应该用空格键而不是 TAB 键	M2
[S.CS.3]	Use a separate line for each HDL statement 每条 HDL 语句使用一行	M1
[S.CS.4]	Use a separate line for each port declaration 每个端口声明使用一行	M1
[S.CS.5]	Preserve port order 保持端口顺序	M2
[S.CS.6]	Declare all internal nets in one section 在同一个地方声明所有的内部 net 变量	R
[S.CS.7]	Keep line length within 72 characters 每行最多 72 个字符	R
[S.CS.8]	Use parameters over the 'define directives to specify configurable constants 使用 parameter 而不是 define 指令来指定配置常量	R
[S.CS.9]	Preserve relationships between constants 保持常数之间的关系	R
[S.CS.10]	Use explicit port mapping instead of positional association in module instantiation 在实例化时,使用显示端口影射,而不用位置关联方式声明	M1
[S.CS.11]	Avoid expression in port connections 在端口连接时避免使用表达式	M1
[S.CS.12]	Use parameters for FSM state encoding	M1

FSM 状态编码使用参数方式

[S.CS.13] Use parentheses in complex equations M2

在复杂的表达式中使用括号

[S.CS.14] Wire must be explicitly declared M1

Wire 变量必须先声明

[S.CS.15] Operand size must match M1

操作数的位宽必须匹配

[S.CS.16] Expression in condition should be 1- bit value R

条件表达式应该只有 1 位

[S.CS.17] Finite State Machine (FSM) coding style R

有限状态机编码风格

Coding Style 代码风格

[S.CS.1] Code should be aligned in a tabular format M2

代码应该使用TAB 键对齐

Aligns the HDL keyword items of the same kind (module/endmodule, always block begin/end, if/else statement, case statement, etc.)

相同类型的关键字需要对齐(module/endmodule 对, always 块的 begin/end, if/else 语句, case 语句等)

[S.CS.2] Use space instead of tab stops for code indentation M2

代码之间应该用空格键而不是 TAB 键

Tabs stops must not be used because they represent different number of spaces from one system to another.

Tab 键不能使用,因为在不同的系统中,它们代表的空格数不一定相同

[S.CS.3] Use a separate line for each HDL statement M1

每条 HDL 语句使用一行

```
Use:
  out1 = a1 & b1; // AND operation
  out2 = a2 | b2; // OR operation
Do not use:
  out1 = a1 & b1; out2 = a2 | b2;
```

[S.CS.4] Use a separate line for each port declaration M1

每个端口声明使用一行

```
Use:
  input A; // port A description
  input B; // port B description
Do not use:
  input A, B;
```

[S.CS.5] Preserve port order M2

保持端口顺序

The order of port declaration keep the same as the port order listed in the module declaration

端口声明顺序必须与在模块内声明的顺序一致

[S.CS.6] Declare all internal nets in one section R

在同一个地方声明所有的内部 net 变量

The declaration of all internal nets should follow the port I/O declaration

所有的内部网表声明都应该在I/O端口声明之后

[S.CS.7] Keep line length within 72 characters R

每行最多 72 个字符

Line length should be limited within 72 characters to reserves space to put the additional line number.

每行的长度应该限制在72个字符内以保留空间来增加行编号

[S.CS.8] Use parameters over the 'define directives to specify R

configurable constants

使用 parameter 而不是 define 指令来指定配置常量

If a `define is used, it should be undefined using `undef. Otherwise, it may cause re-definition error message if other module also exists the same `define constant.

如果使用了define ,那么应该使用undef配套,否则如果其他模块已经定义了相同的define 常量,那么可能会有重复定义

[S.CS.9] Preserve relationships between constants R

保持常数之间的关系

```
parameter       HALFWORD = 16;
parameter       WORD = (2 * HALFWORD);
```

[S.CS.10] Use explicit port mapping instead of positional M1

association in module instantiation

在实例化时,使用显示端口影射,而不用位置关联方式声明

Use:

```
adder u_adder(
      .a    (A),
      .b    (B),
      .sum  (SUM)
        );
```

Do not use:

```
adder u_adder(
       A,
       B,
       SUM
        );
```

[S.CS.11] Avoid expression in port connections M1

在端口连接时避免使用表达式

```
foo u_foo (
     .bad          ((m & n) ^ ((p | q) > 4'hc)),
     .good         (good)); // expressions in port connections
```

[S.CS.12] Use parameters for FSM state encoding M1

FSM 状态编码使用参数方式

```
parameter [1:0] // symbolic state_vector
  ST_IDLE = 2'b00,
  ST_READ = 2'b01,
  ST_WRITE = 2'b10,
  ST_WAIT = 2'b11;
```

[S.CS.13] Use parentheses in complex equations M2

在复杂的表达式中使用括号

```
assign req_out = ((req_ext & en) | req_int);
```

[S.CS.14] Wire must be explicitly declared M1

Wire 变量必须先声明

```
wire int_signal_a; // internal signal for block1 signal_a
wire int_signal_b; // internal signal for block1 signal_b
block u_block(
     .signal_a (int_signal_a),
     .signal_b (int_signal_b
              );
```

Do not count on the language-specific implication.

Follow the strong type-checking principles that are enforced by most modern programming languages.

遵循大多数编程语言都支持的类型检查

[S.CS.15] Operand size must match M1

操作数的位宽必须匹配

```
  wire [32:0] signal_a; // signal_a with 33 bit width
  wire [7:0]  signal_b; // signal_b with 8 bit width
Do not use:
 signal_x <= signal_a & signal_b;
```

Potential error of bit zero-extended

用0扩展的位可能会有潜在的错误

```
module test_width(in, enable, out);
  parameter WIDTH = 33;
  input [WIDTH-1:0] in;
  input enable;
  output [WIDTH-1:0] out;

  assign out = (enable) ? in : 'bz;
endmodule
```

[S.CS.16] Expression in condition should be 1- bit value R

条件表达式应该只有 1 位

```
Do not use:
  if (data)
   data_enable = 1;
Use:
  if (data > 0)
   data_enable = 1;
```

[S.CS.17] Finite State Machine (FSM) coding style R

有限状态机编码风格

Using two-process coding style for Mealy state machine and three-process style for Moore state machine.

采用两段的mealy状态机或三段的moore状态机

Synthesis Rule Lists 综合规则列表

	Synthesis	**Level**
[S.SYN.1]	Use only synthesizable statement Synthesis 只能使用可综合语句	M1
[S.SYN.1.1]	Waveform statement are prohibited 禁止使用波形(仿真)语句	M1
[S.SYN.1.2]	System task and function for simulation (e.g. $display, $monitor,	M1

	$printf, …) are prohibited 系统中的task 和 function 等仿真函数禁止使用	
[S.SYN.1.3]	Wait statement and # delay statement are prohibited Wait 语句及#延时语句禁止使用	M1
[S.SYN.1.4]	Data type of real and event are prohibited 实数和事件数据类型禁止使用	M1
[S.SYN.1.5]	Only one clock per always sensitivity list 每个always 块的敏感列表只能由一个时钟	M1
[S.SYN.1.6]	Loop must be in a static range Loop 只能循环静态范围次数	M1
[S.SYN.2]	Avoid any embedded synthesis scripts 避免任何嵌入式合成脚本	M2
[S.SYN.3]	Avoid full_case and parallel_case synthesis directive statement 避免使用 full_case and parallel_case(全 case 和 并列 case 语句) 综合语句	M2
[S.SYN.4]	Specify code fragment for combinational logic completely 对于组合逻辑需要指明完成的逻辑	M2
[S.SYN.5]	Verilog primitives are prohibited Verilog 原句被禁止使用	M1
[S.SYN.6]	All unused module inputs must be driven 所有未使用的输入信号必须有一个驱动	M1
[S.SYN.7]	All unused module outputs should be connected 所有输出信号都应该连接某个信号	R
[S.SYN.8]	Avoid top level glue logic 避免顶层做铰链逻辑	M2
[S.SYN.9]	Incomplete case statements should have default case assignments 不完整的 case 语句因该有 default 分支语句	M2
[S.SYN.10]	Assign a default state to the state machines	R

状态机因该有一个默认状态

Synthesis 综合

[S.SYN.1] Use only synthesizable statement Synthesis M1

只能使用可综合语句

[S.SYN.2] Avoid any embedded synthesis scripts M2

避免任何嵌入式合成脚本

[S.SYN.3] Avoid full_case and parallel_case synthesis directive statement M2

避免使用 full_case and parallel_case(全 case 和 并列 case 语句)综合语句

```
case (select) // synopsys full_case
  3'b000: y = 1'b1;
  3'b001: y = 1'b0;
  3'b101: y = 1'b0;
  3'b111: y = 1'b1;
endcase
```

```
always @(signal_1 or signal_2
         or signal_3)
begin
  case (1) //synopsys parallel_case
    signal_1: value = 2'b01;
    signal_2: value = 2'b10;
    signal_3: value = 2'b11;
  endcase
```

[S.SYN.4] Specify code fragment for combinational logic completely M2

对于组合逻辑需要指明完成的逻辑

Missing condition:

```
module SYN3_1 (a, data_out);
input a;
output data_out;
always @ (a)
  begin
    if (a == 1'b1)
      data_out = 1'b0;
  end
endmodule
```

Missing assignment:

```
always @(condition)
begin
  case (condition)
    2'b00: data_out = 4'b0000;
    2'b01:
    2'b10: data_out = 4'b0011;
    2'b11:
  endcase
end
```

[S.SYN.5] Verilog primitives are prohibited M1

Verilog 原句被禁止使用

Verilog user-Defined-Primitive (UDP) verilog 用户定义的原语

Verilog built-in primitive verilog 内建原语

[S.SYN.6] All unused module inputs must be driven M1

所有未使用的输入信号必须有一个驱动

Avoid any floated input ports. Floating input ports may result in fatal error of chip.

避免任何悬浮态输入端口,悬浮输入端口可能导致芯片严重的错误

[S.SYN.7] All unused module outputs should be connected R

所有输出信号都应该连接某个信号

Unconnected output ports may not affect the functionality but make users confuse.

没连接的输出端口可能不会导致错误,但会给用户带来困惑

```
Module sub_block
(rst_n, clk, multiplier_out, divider_out);

  input  rst_n, clk;
  output [15:0] multiplier_out;
  output [15:0] divider_out;
```

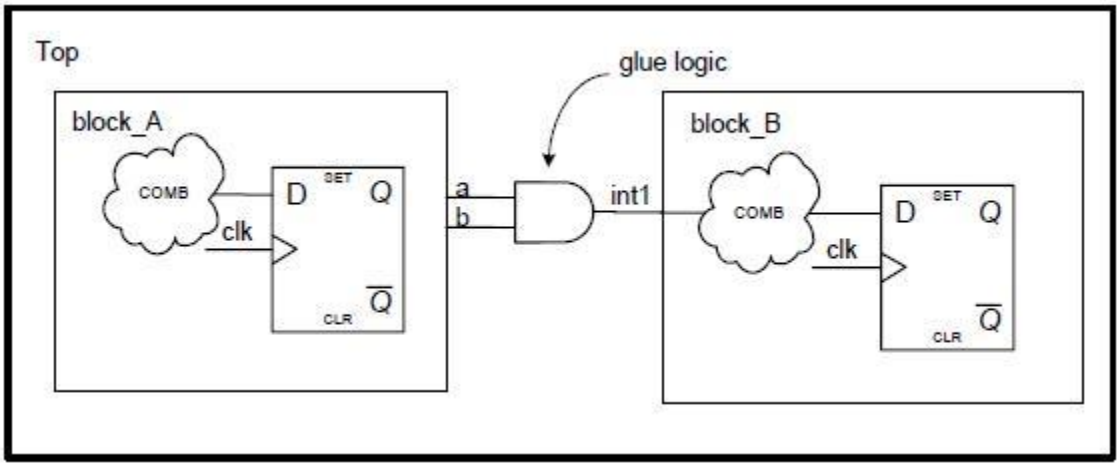
```
module top (rst_n, clk, control,
                top_out);
    input  rst_n;
    input  clk;
    output [7:0] top_out;
    wire   [7:0]  multiplier_out_nc;
    wire   [15:0] divider_out_nc;

sub_block u_sub_block(
       .rst_n              (rst_n),
       .clk                (clk),
       .multiplier_out_nc  ({top_out,
                            multiplier_out_nc}),
       .divider_out_nc     (divider_out_nc)
       );
```

```
module top (rst_n, clk, control,
                top_out);
    input  rst_n;
    input  clk;
    output [7:0] top_out;
    wire   [7:0]  temp;

sub_block u_sub_block(
       .rst_n              (rst_n),
       .clk                (clk),
       .multiplier_out_nc  ({top_out, temp})
       );
```

040318C_IP Qualification -Design Guidelines.ppt P. 40

[S.SYN.8] Avoid top level glue logic M2

避免顶层做铰链逻辑



The top level of the design should contain only at leaf levels of the hierarchy tree.

设计的顶层只能包含层次树的枝叶部分(顶层只能例化个模块)

Synthesis optimization is limited because the top level glue logic can not be merged with the combinational logic inside any modules.

因为顶层的逻辑不能与其它模块的组合逻辑合并而导致不能被优化

[S.SYN.9]	Incomplete case statements should have default case assignments 不完整的 case 语句因该有 default 分支语句	M2

If case statements have no default clause values and not all the cases are covered, unexpected latches may be inferred.

如果case语句没有default值且又没把所有的case覆盖到,那将可能引入不需要的latch.

[S.SYN.10]	Assign a default state to the state machines 状态机因该有一个默认状态	R

Prevent state machines from falling into disorder.

防止状态机陷入到混乱状态

Static Timing Analysis Rule Lists 静态时序分析规则列表

Static Timing Analysis	**Level**

[S.STA.1]	Avoid Combinational feedback loop 避免组合逻辑环路	M2
[S.STA.2]	Using synchronous design practices 使用同步电路设计	M2
[S.STA.3]	Simplify register clock origin 简化寄存器时钟来源	M2
[S.STA.4]	Avoid multicycle path and false path 避免多周期路径和假路径	M2
[S.STA.5]	Avoid clock as data 避免把时钟作为数据	M2
[S.STA.6]	Avoid using latches 避免使用 latch 锁存器	M2

Static Timing Analysis 静态时序分析

[S.STA.1] Avoid Combinational feedback loop M2

避免组合逻辑环路

Combinational feedback loop will cause the problems of unpredictable timing on static timing analysis.

在静态时许分析中,组合反馈回路将会导致一些不可预测的问题

Two ways to handle combinational feedback loop

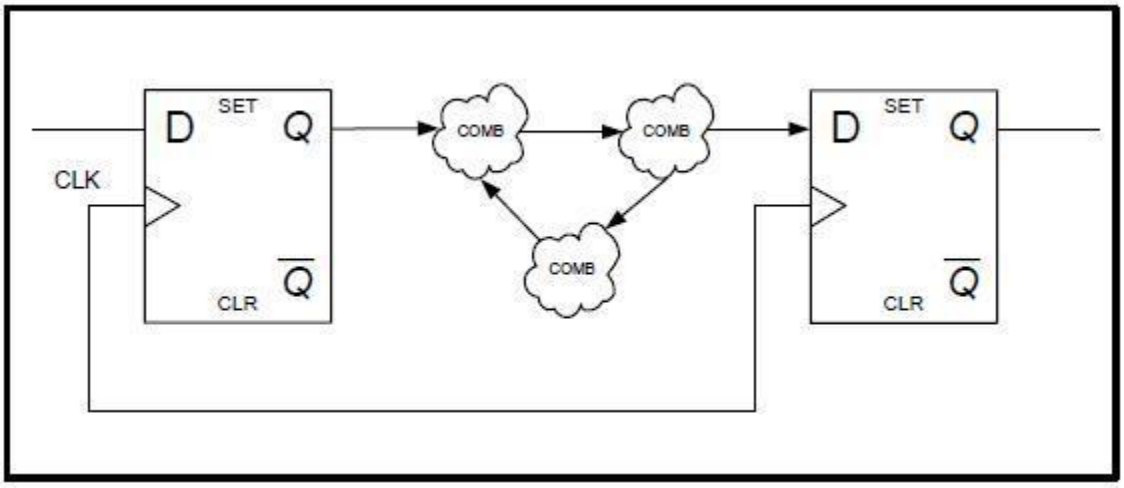
两种方法可以解决组合反馈回路

You can break loops by disabling timing arcs set_disable_timing command

你可以通过设置不使能时序分析命令

STA tool will automatically breaks them.

STA 工具会自动解除回路



[S.STA.2] Using synchronous design practices M2

使用同步电路设计

Asynchronous design should be used only when unavoidable, and should be separated from the synchronous design.

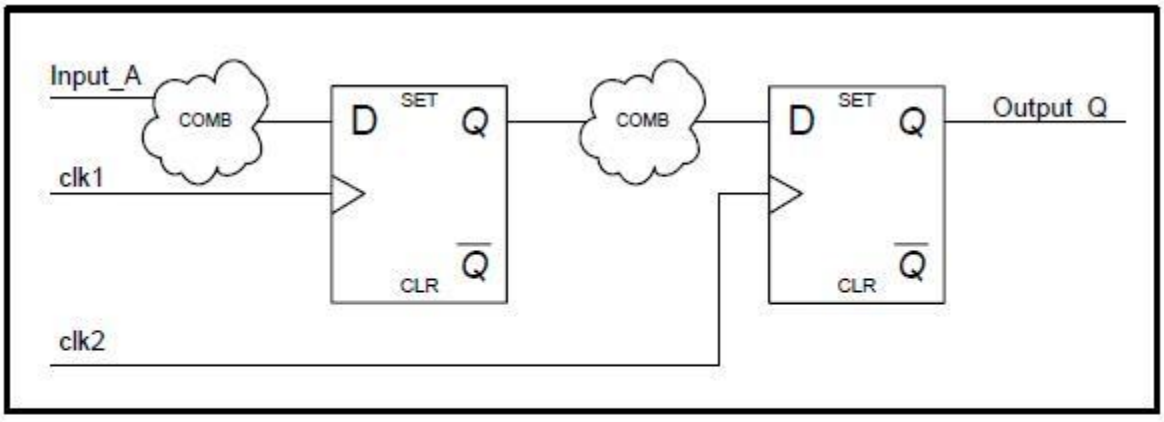
当无法避免使用异步电路时,应该把异步电路分离出来

Some static timing analysis tools do not handle the asynchronous design adequately.

一些静态时许分析工具不能很好的分析异步时序电路

Asynchronous design needs to be verified carefully for its functionality and timing.

异步设计需要非常仔细地验证其功能和时序




```
sta_shell> set_false_path –from clk1 –to clk2
```

[S.STA.3] Simplify register clock origin M2

简化寄存器时钟来源

Avoid connecting internally generated signals or gated clocks to the register clock.

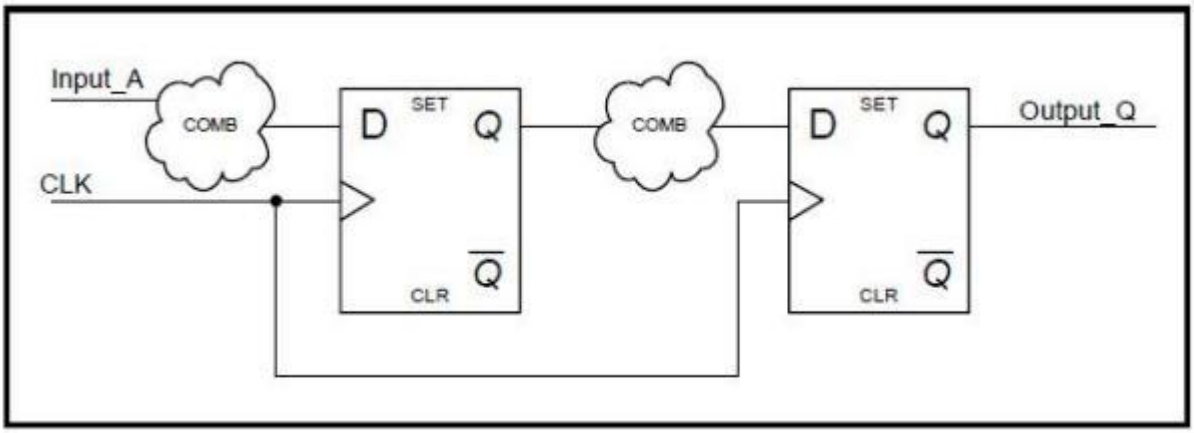
避免使用内部产生的信号和门时钟作为寄存器的时钟

If the internally generated clock or the gated clock is unavoidable in a design, e.g., a low-power design, separate this portion of circuitry in an individual module at the top level of the design.

如果无法避免使用内部产生的时钟或门时钟,如低功耗设计等,那么在顶层把这部分电路分离出来

The register clock origin should be simplified to prevent metastability problems on the data being registered

寄存器的时钟源应该尽量简化来防止出现亚稳态问题



[S.STA.4] Avoid multicycle path and false path M2

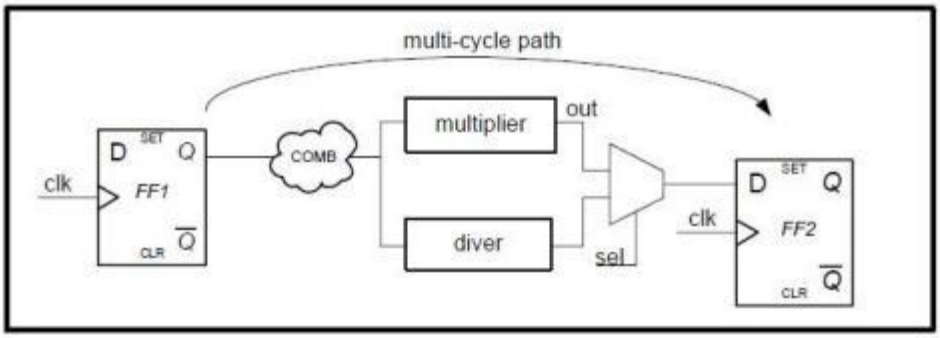
避免多周期路径和假路径

Timing exceptions should be marked by specifying the start and end points of these paths.

特定的时序应该标明路径的起始点和结束点

Timing exceptions lead to longer compile times, try to minimize the total number of timing exceptions

特定的时序分析会导致更长的编译时间,尽量减少特定时序分析的数量



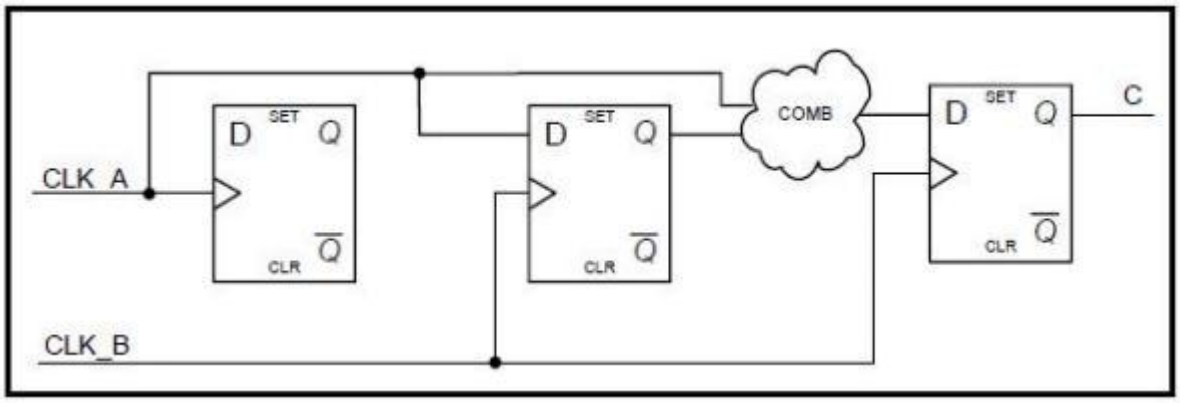

```
sta_shell> set_multicycle_path 3 –from FF1/clk –through multiplier/out –to FF2/D
```

[S.STA.5] Avoid clock as data M2

避免把时钟作为数据

Clocks fed to data inputs of registers directly or indirectly could lead to timing violations and race conditions.

时钟作为数据输入寄存器将会直接或间接导致时序违背或竞争冒险



[S.STA.6] Avoid using latches M2

避免使用 latch 锁存器

Latches are not recommended in your design.

在设计中不推荐使用锁存器

IF any unavoidable latches are inferred in your design, you must provide documentation to describe the special timing requirements of each latch.

如果不可避免地需要在设计中使用锁存器,应该提供文档来描述每个锁存器的特定时序需求

Simulation Rules 仿真规则

	Simulation	Level
[S.SIM.1]	Use non-blocking assignments in sequential always block 在always时序模块中使用非阻塞赋值语句	M1
[S.SIM.2]	Use blocking assignments in combinational always block 在 always 组合逻辑模块中使用阻塞赋值语句	M1
[S.SIM.3]	Avoid missing sensitivity lists in combinational always block 在always组合逻辑模块中避免遗漏敏感列表	M1
[S.SIM.4]	Avoid redundant sensitivity lists 避免冗余的敏感列表	M1
[S.SIM.5]	Initialize control storage elements 初始化存储器控制单元	M2
[S.SIM.6]	Do not assign signals don't care (x) value 不要使用对 x 态不敏感的信号	M1
[S.SIM.7]	Avoid using delay assignment 避免使用延时语句	M2

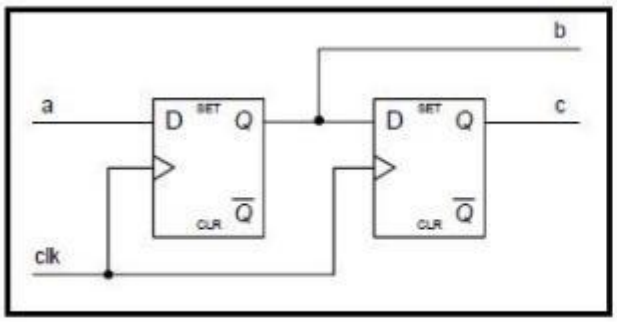
Avoid simulation mismatch

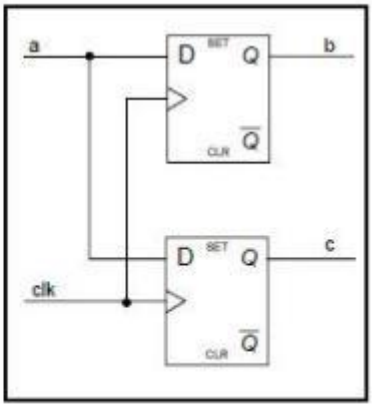
Simulator independent

[S.SIM.1]	Use non-blocking assignments in sequential always block 在always时序模块中使用非阻塞赋值语句	M1

```
always @(posedge clk)
  begin
    b <= a;
    c <= b;
  end
```

```
always @(posedge clk)
  begin
    b = a;
    c = b;
  end
```





[S.SIM.2] Use blocking assignments in combinational always block M1

在 always 组合逻辑模块中使用阻塞赋值语句

Use blocking assignments in always blocks that are intended to generate\ combinational logic.

在always块中使用阻塞赋值将会产生组合逻辑

Non-blocking assignments in combinational always blocks may cause artificial race conditions which will disappear after synthesis.

在组合逻辑always块中使用非阻塞赋值可能会产生认为的竞争冒险,这种竞争冒险在综合后会消失

Blocking assignments ensure the circuits will be evaluated correctly and prevent simulation result mismatches between RTL and gate-level circuits.

阻塞赋值能够保证准确预测电路和防止RTL和门级电路仿真结果不一致的现象

[S.SIM.3] Avoid missing sensitivity lists in combinational always block M1

在always组合逻辑模块中避免遗漏敏感列表

Missing signals in sensitivity list will cause the mismatch of pre-synthesis simulation and post-synthesis simulation.

敏感列表遗漏了信号将会导致前仿和后仿的结果不一致

```
always @ (a)
  begin
    Z = a or b
  end
```

[S.SIM.4] Avoid redundant sensitivity lists M1

避免冗余的敏感列表

The presence of a signal in the sensitivity list that is not used in the process may cause unneeded evaluations in simulation and result in longer simulation times.

在敏感列表中列出了不被使用的信号时,将可能会在仿真时发生不必要的评估而导致仿真时间变长

```
always @ (a or b or c)
  begin
    Z = a or b
  end
```

[S.SIM.5] Initialize control storage elements M2

初始化存储器控制单元

All control storage elements, including all latches and registers, in a control path for control circuitry should be set or reset in the beginning.

所有的存储单元,包含锁存器,寄存器,在开始时,在控制电路中应该被置位或复位.

Control path represents that signal output from register and pass through the conditional logic for control.

控制路径就是从寄存器输出信号和通过条件逻辑控制的信号 ??

The initial value (e.g., “0” or “X”) of each uninitialized latch or register is different from one simulator to another. The simulation result may be simulator dependent.

每个没初始化的锁存器或寄存器的初始值在不同的仿真器中是不一样的,这样它的方针结果取决于仿真器

[S.SIM.6] Do not assign signals don't care (x) value M1

不要使用对 x 态不敏感的信号

Prevent x propagation through the circuitry, especially for the control path.

防止X态在电路中传播,特别是控制路径中

If there exists x value in a control path, it may potentially cause the chain of unknown state for all related circuits in simulation.

如果在控制路径中存在X态,它可能导致在仿真中所有相关电路都在一个不定态状态

```
module control_path (rst_n, clk, select, data_out);
input        rst_n;
input        clk;
input  [2:0] select;
output       data_out;
reg    [2:0] Y;
always @ (select)
   begin
      case (select)
         3'b000 : Y = 3'b001;
         3'b001 : Y = 3'b010;
         3'b010 : Y = 3'b100;
         3'b101 : Y = 3'b101;
         default: Y = 3'bxxx;
      endcase
   end

always @ (posedge clk or negedge
rst_n)
   begin
      if (!rst_n)
         data_out <= 1'b0;
      else if (Y == 3'b001)
         data_out <= 1'b1;
      else
         data_out <= 1'b0;
   end
endmodule
```

[S.SIM.7] Avoid using delay assignment M2

避免使用延时语句

Avoid using both inter- and intra-assignment delays.

避免使用内部和内部赋值延时

Current synthesis tools ignore these delay assignments. Hence, these delays potentially cause the result mismatch between RTL and gate-level simulation.

目前的综合工具会忽略延时赋值,因此.这些延时语句将会导致RTL和门级仿真的结果不一致

```
#1 outp = a & b; // inter-assignment delay
always @(posedge clk or negedge rst_n)
  if(rst_n == 1'b0)
    outp_r <= #1 4'b0; // intra-assignment delay
  else
    outp_r <= #1 outp_nxt; // intra-assignment delay
```

Design for Test Rule Lists 测试规则列表

	Design for Test	Level
[S.DFT.1]	Avoid tri-state devices 避免使用三态器件	M2
[S.DFT.2]	Avoid bi-directional nets 避免使用双向网线	M2
[S.DFT.3]	Avoid latches 避免使用锁存器	M2
[S.DFT.4]	Avoid using both edges of clocks 避免使用双沿时钟	M2
[S.DFT.5]	Avoid gating clocks 避免使用门时钟	M2
[S.DFT.6]	Avoid internal generated clocks 避免使用内部产生时钟	M2
[S.DFT.7]	Avoid internal generated set/reset signals 避免使用内部产生的复位置位信号	M2
[S.DFT.8]	Avoid using clocks and set/reset signals as data 避免把时钟和复位/ 置位信号作为数据	M2
[S.DFT.9]	Avoid combinational loop 避免组合环路	M2

[S.DFT.10] Avoid constant inputs or floating outputs R

避免常量输入和悬浮输出

Be able to do full-scan insertion

尽可能做全扫描插入

Obtain the maximum fault coverage

得到最大的障碍覆盖率

Design for Test

[S.DFT.1] Avoid tri-state devices M2

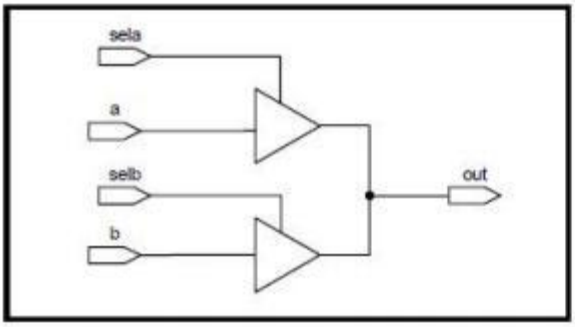
避免使用三态器件

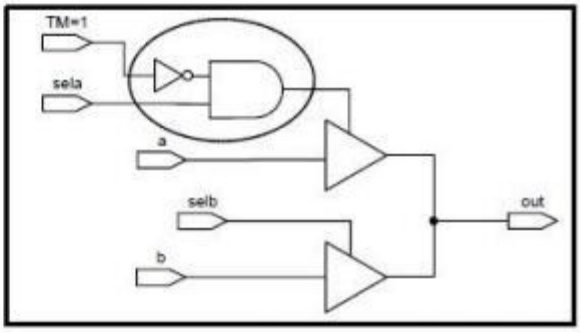
Multiple drivers will possibly cause bus contention, it is recommended to avoid them.

多源驱动可能将会产生总线竞争,不推荐使用

It is recommended to be fixed at RTL.

推荐在RTL中解决此问题





[S.DFT.2] Avoid bi-directional nets M2

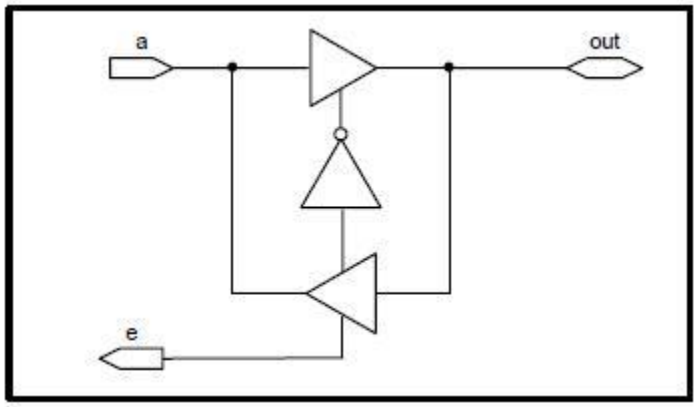
避免使用双向网线

It is recommended to avoid the bi-directional nets for the digital logic design.

在数字逻辑设计中避免使用双向网线

Bi-direction nets make ATPG tools confuse the signal direction and lose the capability to generate the test patterns for the digital circuitry in which the bi-directional nets involve

双向网络会时ATPG工具无法识别信号方向而导致无法对此部分电路产生测试向量



[S.DFT.3] Avoid latches M2

避免使用锁存器

It is recommended to avoid latches. Scannable registers are recommended.

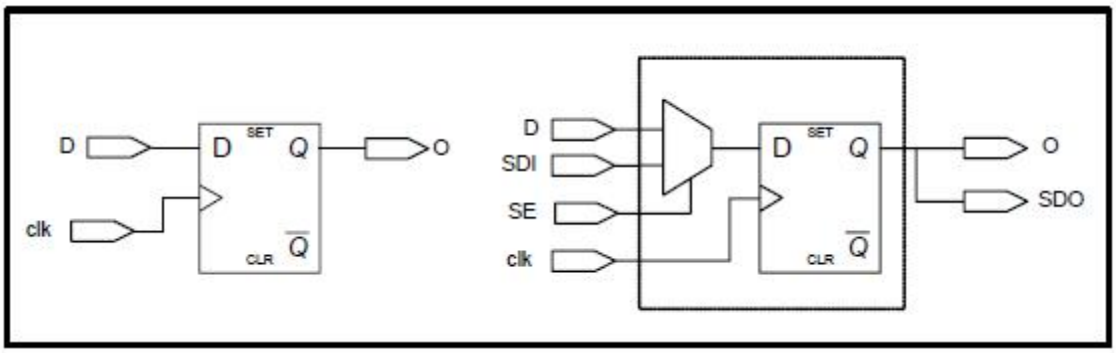
避免使用锁存器,推荐使用可插入扫描链的寄存器

Latches are usually non-scannable and non-transparent without any special control circuit.

锁存器通常是无法插入扫描链中,并且没有特别的控制电路来穿过它

Significant fault coverage might be lost if latches are intensively used.

如果所采取被大量使用,需要重视的障碍覆盖率可能会丢失.



[S.DFT.4] Avoid using both edges of clocks M2

避免使用双沿时钟

Use of mixed clock edges increases the difficulty to chain these DFFs.

使用双沿时钟将会增加插入扫描链的困难

There are two solutions proposed up to now.

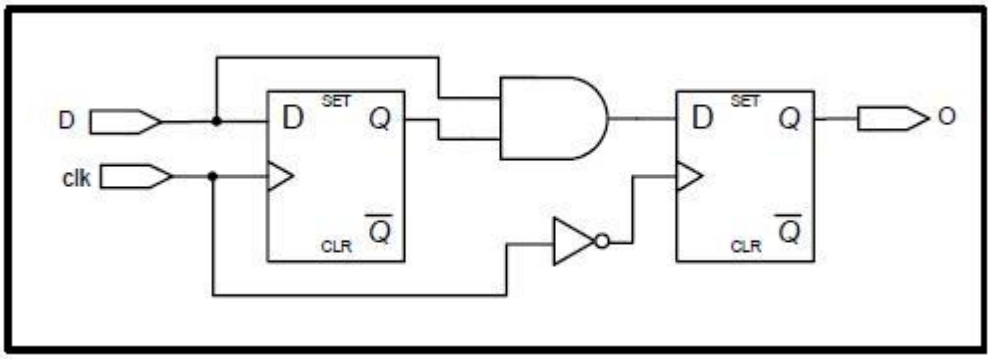
目前有2钟解决方案

One is to insert the protection circuit between the positive-edge and negative edge DFF.

一种是在正沿和负沿DFF之间插入保护电路

The other is to separate positive-edge and negative-edge DFFs into different groups for different scan chains.

另外一种方法就是把正沿和负沿DFF分到不同的组做不同的扫描链



[S.DFT.5] Avoid gating clocks M2

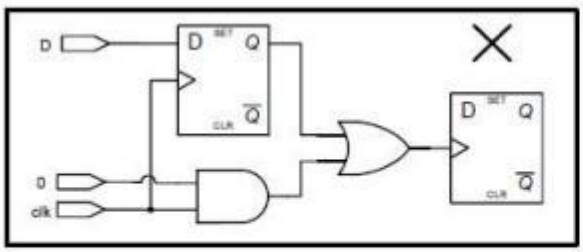
避免使用门时钟

The clock ports of the flip-flops should be controlled directly by one primary clock input.

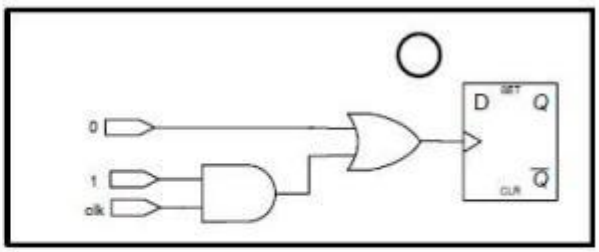
寄存器的时钟端口因该被输入的主时钟控制

The gated clock potentially obstructs the scan shift operations. That is, it's not friendly to DFT.

门控时钟可能会阻碍扫描链的移位操作,因此对于DFT是不推荐使用



Serious gated clock can not be controlled by certain pattern



Gated clock but can be controlled by certain pattern

[S.DFT.6] Avoid internal generated clocks M2

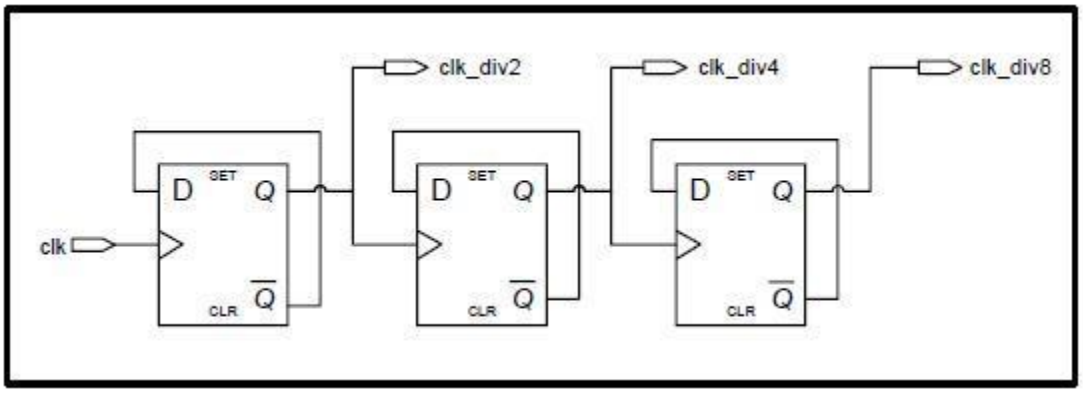
避免使用内部产生时钟

Internal generated clock makes the sequential element un-synchronous and obstructs the scan operation.

内部产生的时钟会使时序单元不同步而导致影响扫描链工作

Hence, this construct significantly reduces the fault coverage.

因此这种电路结构会大大降低障碍覆盖率



[S.DFT.7] Avoid internal generated set/reset signals M2

避免使用内部产生的复位置位信号

The set/reset ports of the flip-flops should be controlled directly by primary inputs to set to the inactive value for the scan operation.

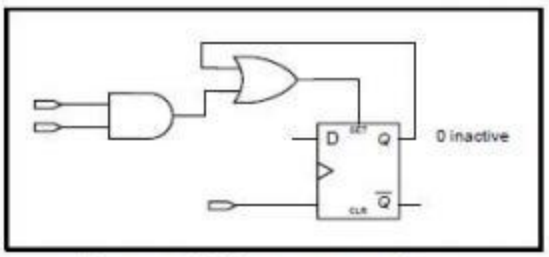
在扫描工作状态,寄存器的复位,置位应该被主输入端来直接控制成失效状态

The internally generated set/reset signal can potentially destroy the integrity of the shift data during the scan shift operation.

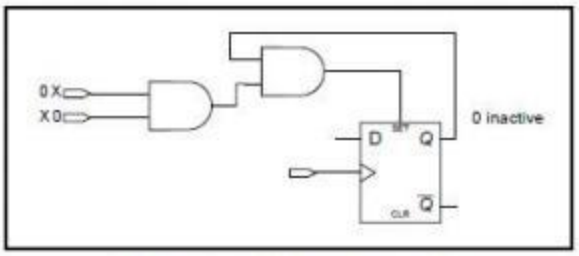
在扫描链移位过程中,内部产生的置位复位信号可能会影响移位数据的完整性

Because the scan operation requires the set/reset ports to be held in inactive states in order to guarantee the shifted data are not destroyed.

因为在扫描过程中要求复位置位端口保持在失效状态以保证移位数据不被破坏.



Uncontrollable preset or clear



Controllable preset or clear

[S.DFT.8] Avoid using clocks and set/reset signals as data M2

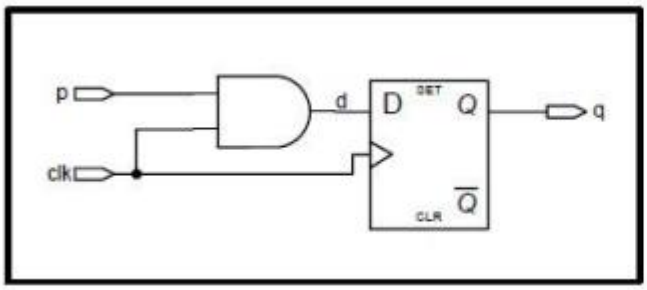
避免把时钟和复位/ 置位信号作为数据

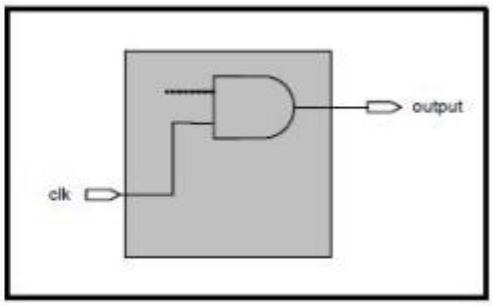
It is recommended that data should not be generated from any clock signal or ser/reset signals.

数据不应该由时钟,置位,复位信号产生

Full-scan can always operate abnormally and unsafely in this construct.

在这种结构中,全扫描电路中经常会工作异常和不可靠.





[S.DFT.9] Avoid combinational loop M2

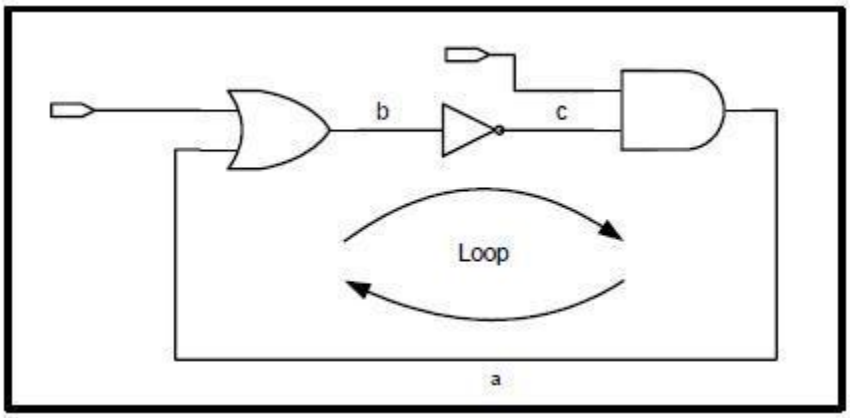
避免组合环路

The combinational loop circuit has the feedback, the ATPG can not determine the logic values in the loop.

因为组合环路电路中有反馈,所以ATPG无法确定环路中逻辑值

The combinational loop is usually considered to be un-testable if the loop is not broken, so the combinational loop will decrease the fault coverage.

如果环路没打开,这个组合逻辑环路被认为是不可测试的,所以它将会降低障碍覆盖率



[S.DFT.10] Avoid constant inputs or floating outputs R

避免常量输入和悬浮输出

It usually occurs when we use the existing functional blocks which are designed and verified, or integrate different cores to synthesize.

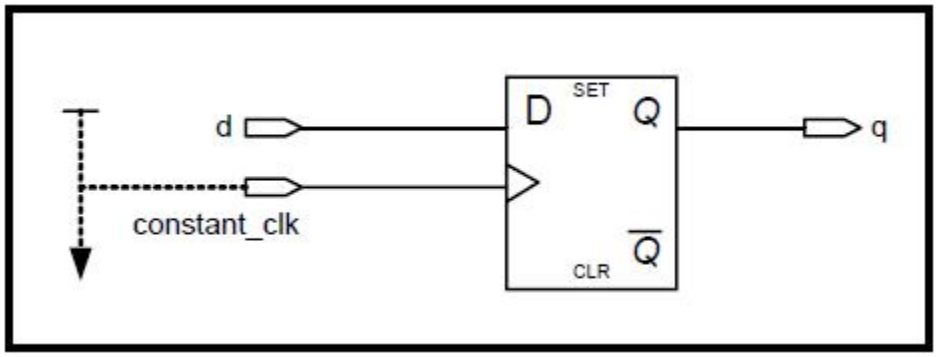
这种情况通常出现在我们使用已经设计验证过的模块或综合集成了不同的核的模块

Because we can not control the constant inputs or observe the floating outputs.

因为我们不能控制内部常量输入和悬浮输出

It will reduce the controllability and the observability respectively, the fault coverage is reduced.

它将会降低各自模块的可控性和观测性 ,因此障碍覆盖率将会降低



Design Style Guidelines Rule Lists 设计风格指南规则列表

Design Style Guidelines	Level

[S.D.1]	IP interface design issue IP 接口设计问题	
[S.D.1.1]	Output signals must be registered 输出端口信号必须是寄存器	M2
[S.D.1.2]	Registers in an IP's interface should be clocked in a single edge of the same clock. 在同一个IP接口的寄存器应该属于同一个时钟的单沿	R
[S.D.1.3]	The interface of an IP should connect to a bus. IP 的接口应该使用总线	R
[S.D.1.4]	The detail information of the input to clock and clock to output signal delay must be planned and provided. 输入到时钟和时钟到输出的延时信息因该被计算出和提供	M1
[S.D.2]	IP should be synchronous. IP 应该使用同步时钟	M2
[S.D.3]	Clock strategy must be documented. 时钟树策略必须有相应文档	M1
[S.D.4]	Avoid handcrafted clock gating. 避免手动门控时钟	M2
[S.D.5]	Rules of reset in the IP IP 的复位规则	
[S.D.5.1]	IP must be able to reset. IP 必须能够被复位	M2
[S.D.5.2]	IP reset strategy must be documented. IP 的复位策略必须有相应文档	M1
[S.D.6]	Avoid any technology-dependent element. 避免使用与工艺有关的元器件	M2
[S.D.7]	Additional functional behavior models should be developed and provided. 应该提供开发使用的功能行为模型	R

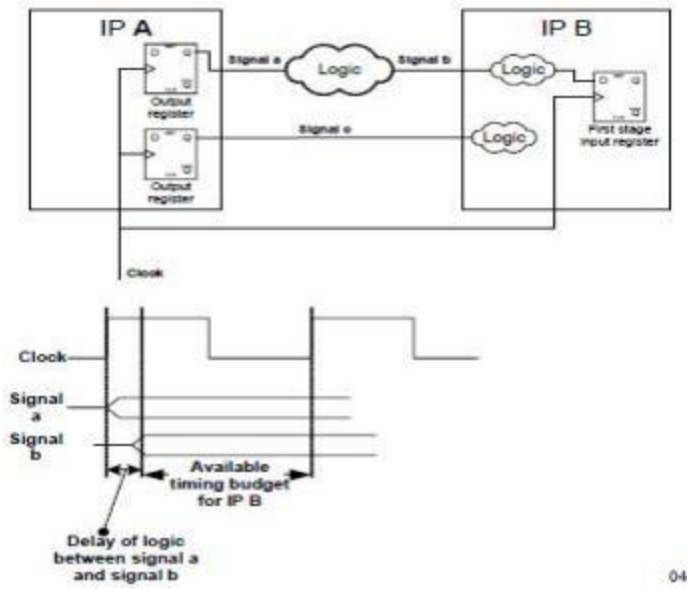
Design Style Guidelines 设计风格指南

[S.D.1.1] Output signals must be registered M2

输出端口信号必须是寄存器

Input delays and output drive strengths are more predictable when registering the output signals from an IP.

当从一个IP输出的是一个寄存器类型.它的输入延时和输出驱动强度是更加可控的.



[S.D.1.2] Registers in an IP's interface should be clocked in a single edge of the same clock. R

在同一个IP接口的寄存器应该属于同一个时钟的单沿

It will reduce the design complexity of the other circuitry that interacts with this IP.

它将会降低与这个IP交互电路设计的复杂度

[S.D.1.3] The interface of an IP should connect to a bus. R

IP 的接口应该使用总线

IP should be connected to the rest of system components through MUX-based bus.

IP应该能够通过复用总线轻松被连接到系统中

When point-to-point scheme (or connection) is unavoidable, IP creators should use an industry standard

当端对端电路无法避免,IP设计者应该使用一个工业标准

[S.D.1.4] The detail information of the input to clock and clock to output signal delay must be planned and provided. M1

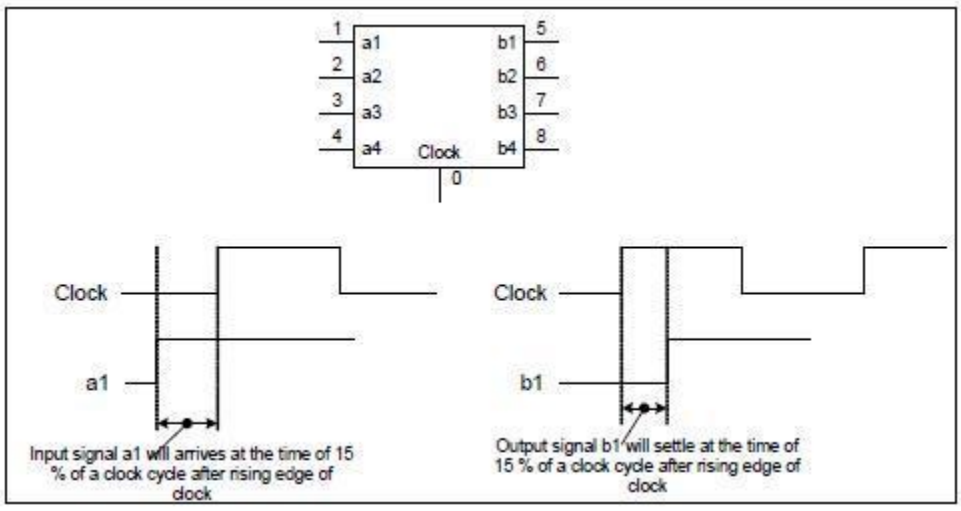
输入到时钟(setup time)和时钟到输出(hold time)的延时信息因该被规划和提供

The default signal delay from the input signal to clock and clock to output signal, must be hand over to IP integrators in document and script template forms.

输入到时钟(setup time)和时钟到输出(hold time)的延时信息应该以文档或脚本\形式交给IP集成者

For a soft IP, this information can be represented in terms of the percentage of the input clock period of this IP

对于软IP,这些信息可以用输入时钟周期的占空比来代替



[S.D.2] IP should be synchronous. M2

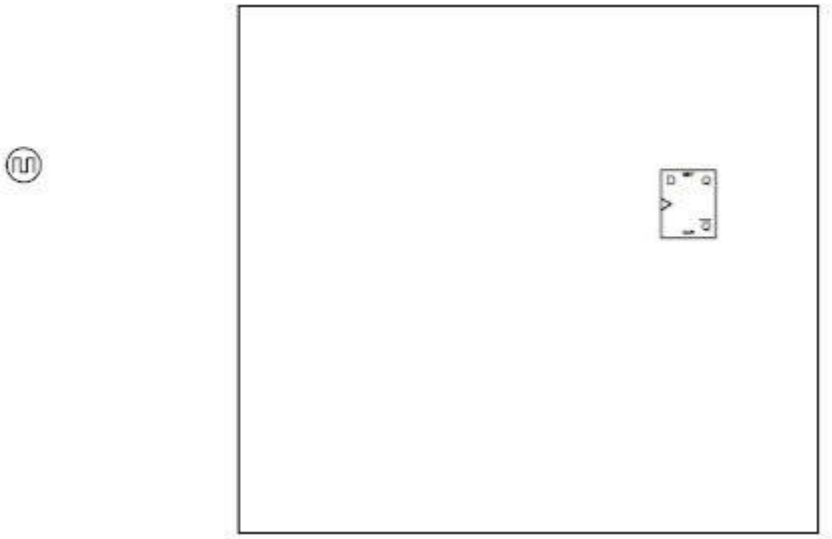
IP 应该使用同步时钟

Although asynchronous designs have advantages in power consumption and performance, they could cause some problems, such as metastability.

尽管异步设计在功耗和性能方面有优势,但它们会产生一些问题,如亚稳态等

[S.D.3] Clock strategy must be documented. M1

时钟树策略必须有相应文档



[S.D.4] Avoid handcrafted clock gating. M2

避免手动门控时钟

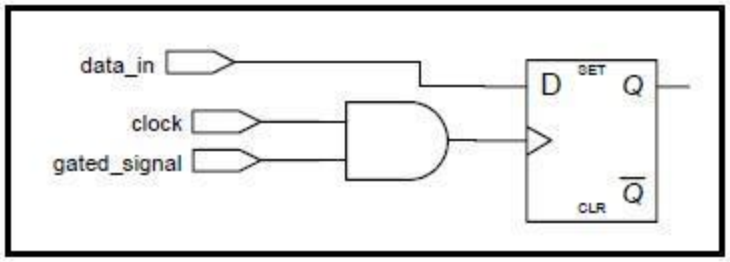
Designers must avoid inserting any handcrafted clock-gating cells to gate their clock signals.

设计者必须避免插入任何手动门控时钟单元来门控时钟信号

If designers need to use clock gating in their IP, they should let power optimization tools to take care this job for them.

如果设计者需要在IP中使用门控时钟,他们应该让功耗优化工具来做这些工作

```
assign gate_clock= clock & gated_signal
always @(posedge gate_clock)
{
  assign data_reg_1 <=data_in;
}
```



[S.D.5.1] IP must be able to reset. M2

IP 必须能够被复位

IP must be designed with reset mechanism to allow IP go back to their initial statuses.

IP设计必须有复位机制来允许IP回到初始状态

Control units in IP must be able to reset to ensure the correctness (controllability) of these IP.

IP的控制单元必须能够复位以保证这些IP的可控制

For the reason of saving area, data path can be an exception of this rule.

对于保留区域,数据路径可能是这个规则的一个例外.???

[S.D.5.2] IP reset strategy must be documented. M1

IP 的复位策略必须有相应文档

Necessary information of IP reset strategies should be delivered is specified in soft IP deliverables guidelines, rule [S.DOC.3].

IP复位策略等应该交付的必要信息应该按照指导规则交付

[S.D.6] Avoid any technology-dependent element. M2

避免使用与工艺有关的元器件

Soft IP should not include any technology-dependent element, such as hard IP or elements that related to any specific technology.
软IP应该与工艺无关,硬IP或器件会与制定的工艺相关

If there are any technology dependent macros (or IP) in a soft IP, it will have problems when porting this soft IP into different technologies and make this IP difficult to reuse.
如果在软IP中有某些单元(或IP)与工艺相关,那么当它被使用在不同工艺时将会导致一些问题,因此会使IP的重用性变得困难.

[S.D.7]	Additional functional behavior models should be developed and provided. 应该提供开发使用的功能行为模型	R

IP providers are encouraged to develop and provide more functional models in higher abstraction levels. For example, bus functional models.
IP提供者被要求开发和提供更好抽象层的功能模型,如总线功能模型

These models can enable IP users to perform more functional verification at higher levels in a very efficient way.
这些模型能够让IP用户有效的在更高抽象层做更多的功能性验证

Synthesis Scripts Rule Lists 综合脚本规则列表

	Design Style	**Level**
[S.SCR.1]	Timing budget must be included in synthesis script template. 时序估算必须包含在综合脚本模板中	M2
[S.SCR.2]	Clock and asynchronous reset networks must be set as ideal nets. 时钟和异步复位网络必须设置为理想网络	M2

[S.SCR.3] Scripts in synthesis script template must include headers. M2

综合脚本模板必须包含文件头

[S.SCR.4] Synthesis scripts must be written with comments. R

综合脚本必须写注释

[S.SCR.5] Keep line length within 72 characters in synthesis script template. R

综合脚本中每行最多72个字符

[S.SCR.6] Hard-coded numeric values, filenames and path names are not allowed. M2

硬编码数字,文件名和路径名不能使用

[S.SCR.7] Keep common constants, parameters, commands and tasks in one or a small number of files, whereas unique ones in individual files. R

保持通用的常数,参数,命令和任务在一个或几个文件中,而自定义的参数放在一个特定的文件中()

Synthesis Scripts 综合脚本

[S.SCR.1] Timing budget must be included in synthesis script template. M2

时序估算必须包含在综合脚本模板中

IP creators must develop timing budget during soft IP creating (authoring) process.

IP设计者在IP设计过程中必须考虑时序裕量

IP creators should create a synthesis script template with adjustable timing budget in terms of the percentage of the clock period.

IP设计者应该提供一个根据时钟周期占空比来调整时序裕量的综合脚本模块

[S.SCR.2] Clock and asynchronous reset networks must be set as M2

ideal nets.

时钟和异步复位网络必须设置为理想网络

Clock and asynchronous reset networks are typically optimized by the backend tools.

时钟和异步复位网络通常由后端工具做优化

Instead, they are handled in physical design by means of inserting balanced clock trees with very low skew.

他们会通过在物理设计中插入平衡时钟树来得到一个非常低的时钟偏斜

[S.SCR.3] Scripts in synthesis script template must include headers. M2

综合脚本模板必须包含文件头

```
// +SynHDR+ ----------------------------------------------------
// File name: Uart_timing.scr
// Author: Bryan Wang
// Release Version: V.1.0
// Release Date: 2003/06/10/
// Version Description: First version of this script:
// Purpose: Timing budget of UART
// Parameter:
//       Paddr: Address bus width. The default value is 3
//       Prdata: Read data bus width. The default value is 32
// -SynHDR ----------------------------------------------------
```

[S.SCR.4] Synthesis scripts must be written with comments. R

综合脚本必须写注释

Using comments will improve readability of scripts in synthesis script template.

使用注释将会提高综合脚本模板的可读性

[S.SCR.5] Keep line length within 72 characters in synthesis script template. R

综合脚本中每行最多72个字符

Line length in synthesis scripts is suggested to be 72 characters or less. Some character space is reserved for the line number.

综合脚本每行的长度建议在 72 个字符内,这些字符空间被用作行号.????

Because some output devices like standard terminals or printers have the maximum character limit per line (e.g., 80 characters).
因为一些输出设备,如一些标准的终端和打印机每行有最大的字符限制(如80 个字符)

[S.SCR.6] Hard-coded numeric values, filenames and path names are not allowed. M2

硬编码数字,文件名和路径名不能使用

Any hard-coded numbers, data values, filenames and path names buried in the body of the script are not allowed.
脚本中硬编码数字,文件名和路径名不能使用

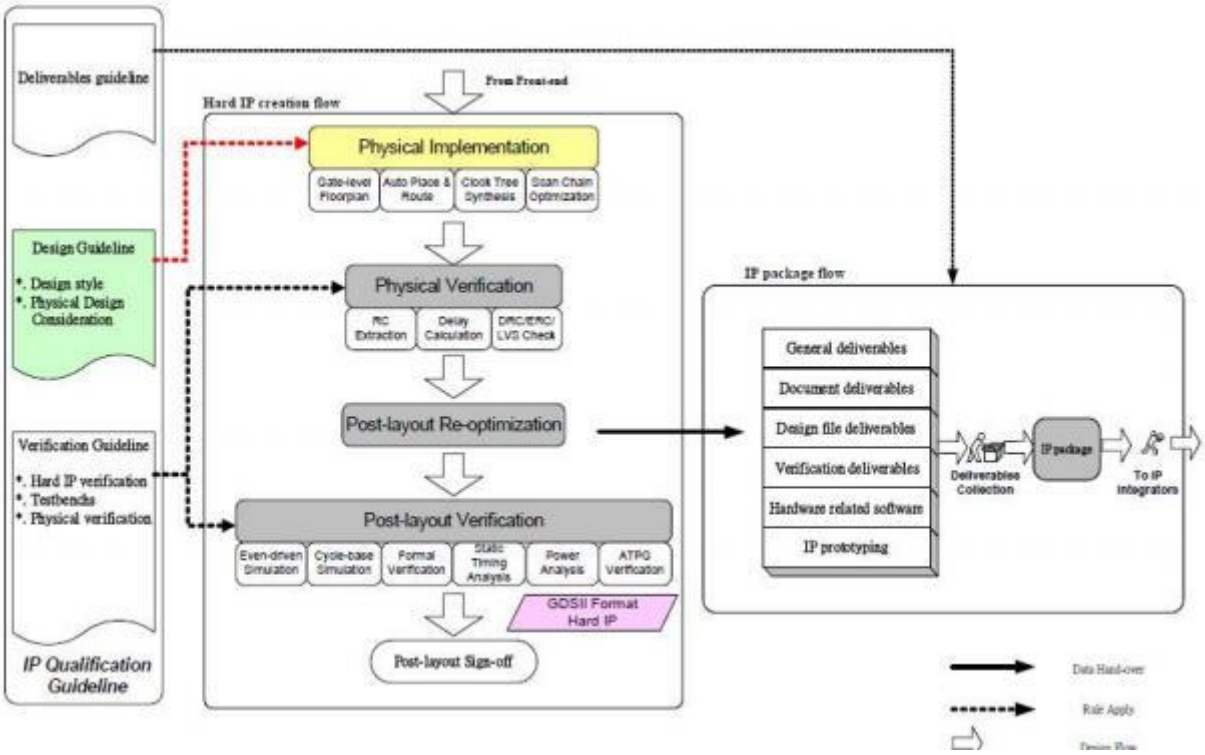
[S.SCR.7] Keep common constants, parameters, commands and tasks in one or a small number of files, whereas unique ones in individual files. R

保持通用的常数,参数,命令和任务在一个或几个文件中,而自定义的参数放在一个特定的文件中()

Common-used scripts, such as settings of the library and search paths are located in a single setup file.
像设置库文件和查找路径等通用脚本可以放在一个单独的配置文件中

Custom-tuned scripts that perform unique tasks like timing constraints are packaged elsewhere.
像时序约束那种定制的与性能相关的脚本打包放在另外一个地方

Hard IP Generic Design Flow 硬 IP 设计流程



	Design Style Guidelines	Level
[H.D.1]	**IP interface design issue** IP 接口设计问题	M2
[H.D.1.1]	Output signals must be registered 输出端口信号必须是寄存器	R
[H.D.1.2]	Registers in an IP's interface should be clocked in a single edge of the same clock. 在同一个 IP 接口的寄存器应该属于同一个时钟的单沿	M1
[H.D.1.3]	The interface of an IP should connect to a bus. IP 的接口应该使用总线	M2
[H.D.1.4]	The detail information of the input to clock and clock to output signal delay must be planned and provided 输入到时钟和时钟到输出的延时信息因该被计算出和提供	M1
[H.D.2]	**IPs should be synchronous.**	R

	IP 应该使用同步时钟	
[H.D.3]	**Clock strategy must be full documented.**	M2
	时钟树策略必须有相应文档	
[H.D.4]	Rules of reset in the IP IP	M2
	IP 的复位规则	
[H.D.4.1]	IPs must be able to reset.	M2
	IP 必须能够被复位	
[H.D.4.2]	IP reset strategies must be documented.	R
	IP 的复位策略必须有相应文档	
[H.D.5]	Additional functional behavior models should be developed and provided.	M2
	应该提供开发使用的功能行为模型	
[H.D.6]	Synthesis models must be developed and provided.	M1
	必须提供使用的综合模型	
[H.D.7]	Power consumption must be analyzed and documented.	M2
	需要作功耗分析并有相关文档	
[H.D.8]	Manufacturing test strategy and information must be documented.	M1
	生产测试策略相关的信息和文档	
	Physical Design Donsideration	**Level**
	物理设计考虑	
[H.D.1]	Aspect ratio of a hard IP should close to 1:1.	R
	硬 IP 的长宽比例应该接近 1:1	
[H.D.2]	Through-cell blockage map and over-cell blockage map must be included.	M1
[H.D.3]	Reserve metal layers for over-IP routing.	R
	给成熟 IP 布线保留一些金属层	
[H.D.4]	Pins of IPs should preserve metal layers with different directions.	R

IP 的管脚应该在不同的方向预留金属层

Hard IP Design Guidelines Rule Lists 硬 IP 设计指南规则列表

[H.D.6] Synthesis models must be developed and provided. M1

必须提供使用的综合模型

The synthesis models must be developed and provided to enable the synthesis process for the chip that utilizes the given IP.

综合模型必须开发以方便启动使用了该IP芯片进行综合处理

[H.D.7] Power consumption must be analyzed and documented. M2

需要作功耗分析并有相关文档

Power consumption including operation condition, operation frequency, and measuring method.

功耗分析包含了工作条件,工作频率和测试方法

[H.D.8] Manufacturing test strategy and information must be documented. M1

生产测试策略相关的信息和文档

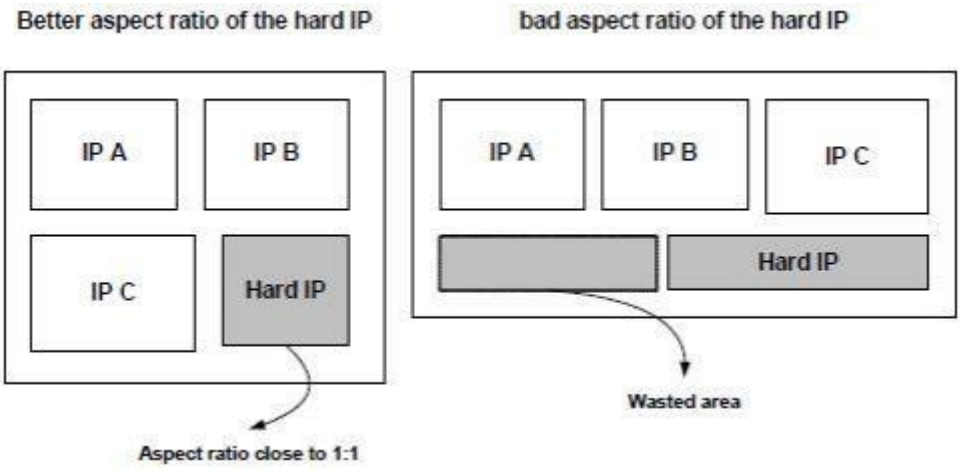
DFT techniques and the related test strategy such as test mode switch, the way to feed the test vectors and observe the results, and the fault coverage must be documented.

DFT技术和相关的测试策略,如测试模式开关,产生测试向量的方法和结果的观测,及障碍覆盖率都必须有相关的文档.

Physical Design Consideration 物理设计考虑

[H.PD.1] Aspect ratio of a hard IP should close to 1:1. **R**

硬IP的长宽比例应该接近1:1



[H.PD.2] Through-cell blockage map and over-cell blockage map must be included. **M1**

必须包含禁止穿过和穿越单元的布局

Hard IP must include these maps to identify areas where through-cell routing (or over-cell routing) is possible and will not cause timing problems. With this information, IP integrators know which parts of this IP are routable.

硬IP必须包含这方面的布局来识别这些区域,在这些区域可以穿过一些cell布线(或穿越一些cell不线),而不会产生时序问题;具有这些信息,IP集成者知道这个IP那些部件是可以布线的

[H.PD.3] Reserve metal layers for over-IP routing. **R**

给成熟IP布线保留一些金属层

Leave several metal layers for over-IP routing in complex design and big hard macros.

在复杂设计和大的硬宏中,给成熟IP布线留出几层金属层

With reserved routing layers, IP integrators have more flexibility in whole chip routing process, especially when the hard IP occupies large area.

在整个芯片布线过程中,如果预留了布线层,IP集成者在布线时会有更大的灵活性.

[H.PD.4] Pins of IPs should preserve metal layers with different **R**

directions.

IP的管脚应该在不同的方向预留金属层

Every hard IP pin needs to have at least one horizontal layer and one vertical layer.

每个硬IP的管脚至少要有垂直方向和水平方向的金属层

The purpose of this rule is to provide more flexibility in whole chip routing process.

这条规则的目的是为了在整个芯片布线时具有更大的灵活性

Assert 篇

基本命令介绍

1. 如果要增加 assert 语法，需要包含 sva_lib.sv 库

sva 在波形上显示的方法：在 tb.v 中 dump 波形地方加上： $fsdbDumpSVA

2. SystemVerilog 添加了两种类型的断言

立即断言：只检查一次

- 执行起来类似于过程块中的语句
- 主要想用于仿真
- 用关键字：assert

并发断言：循环检查

- 基于时钟的语义
- 用于采样变量值
- 用关键字 assert property

3。

3. 1.$rose(signal) 表示 signal 的上升沿

3. 2.$fell(signal) 表示 signal 的下降沿

3. 3.$stable(signal) 表示 signal 不发生变化

3．4. |->交叠蕴含，指如果先行算子匹配，在同一个时钟周期计算后续算子表达式

3．5. |=>非交叠蕴含，指如果先行算子匹配，在下一个时钟周期计算后续算子表达式

3．6. 固定延时用符号 ##

3．7. $ 用于时序的上限，表示时序没有上限，[0:$]则表示任何时间

3．8.$past(signal, number of clock)表示在多少个 clock 之前，signal 的值

3．9.sequence ……endsequence 表示一个序列，可用于 property…… endproperty 中

3．10. not 用来表示属性永远不为真，not s6(s6：序列)，则表示永远不会出现 s6 描述的情况

4. SVA 的插入位置：在一个.v 文件中：

```
module ABC ();
                    rtl 代码
                    SVA 断言
endmodule
```

注意：不要将 SVA 写在 enmodule 外面。

5. 断言编写的一般格式是：

```
【例】断言名称 1：assert property (事件 1)        //没有分号
          $display ("........",$time);              //有分号
else
          $display ("........",$time);              //有分号

断言名称 2：assert property (事件 2)
          $display ("........",$time);
else
          $display ("........",$time);
```

断言的目的是：断定“事件 1”和“事件 2”会发生，如果发生了，就记录为 pass，如果没发生，就记录为 fail。注意：上例中没有 if，只有 else，断言本身就充当 if 的作用。

上例中，事件 1 和事件 2 可以用两种方式来写：

(1) 序列块：sequence name;

。。。。。。。。。。;

endsequence

(2) 属性块：property name;

。。。。。。。。。。;

endsequence

从定义来讲，sequence 块用于定义一个事件（砖），而 property 块用于将事件组织起来，形成更复杂的一个过程（楼）。sequence 块的内容不能为空，你写乱字符都行，但不能什么都没有。sequence 也可以包含另一个sequence,如：

```
sequence s1;
s2(a,b);
                    endsequence  //s1 和 s2 都是 sequence 块
```

sequence 块和 property 块都有 name，使用 assert 调用时都是：“assert property (name);”

在 SVA 中，sequence 块一般用来定义组合逻辑断言，而 property 一般用来定义一个有时间观念的断言，它会常常调用 sequence，一些时序操作如“|->”只能用于 property 就是这个原因。

6. 带参数的 property、带参数的 sequence

property 也可以带参数，参数可以是事件或信号，调用时写成：

```
assert property  (p1(a,b))
```

被主 sequence 调用的从 sequence 也能带参数，例如从 sequence 名字叫 s2，主 sequence 名字叫 s1:

sequence s1;

s2(a,b);

endsequence

7. property 内部可以定义局部变量，像正常的程序一样。

property p1;

int cnt;

.....

Endproperty

8. 一般，断言是基于时序逻辑的，单纯进行组合逻辑的断言很少见，因为太费内存（时序逻辑是每个时钟周期判断一次，而组合逻辑却是每个时钟周期内判断多次，内存吃不消）。

因此，写断言的一般规则是： time + event，要断定发生什么 event，首先要指定发生 event 的时间，例如

某个时钟上升沿 + 发生某事

某信号下降时 + 发生某事

9. 语法 1：信号（或事件）间的“组合逻辑”关系：

(1) 常见的有：&&, ||, !, ^

(2) a 和b 哪个成立都行，但如果都成立，就认为是 a 成立：firstmatch(a||b)，与“||”基本相同，不同点是当 a 和 b 都成立时，认为 a 成立。

(3) a ? b:c —— a 事件成功后，触发b，a 不成功则触发 c

10. 语法 2：在“时序逻辑”中判断独立的一根信号的行为：

@ (posedge clk) A 事件; ——当 clk 上升沿时，如果发生 A 事件，断言将报警。

边沿触发内置函数：(假设存在一个信号a)

$rose(a);——信号上升

$fell(a);——信号下降

$stable(a);——信号值不变

11. 语法 3：在“时序逻辑”中判断多个事件/信号的行为关系：

(1) intersect(a,b)——断定 a 和 b 两个事件同时产生，且同时结束。使用方法 @(posedge clk) a intersect b;

(2) a within b ——断定 b 事件发生的时间段里包含 a 事件发生的时间段。

(3) a ##2 b ——断定 a 事件发生后 2 个单位时间内b 事件一定会发生。

a ##[1:3] b ——断定 a 事件发生后 1~3 个单位时间内b 事件一定会发生。

a ##[3:$] b ——断定 a 事件发生后 3 个周期时间后 b 事件一定会发生。

(4) c throughout (a ##2 b) ——断定在 a 事件成立到 b 事件成立的过程中，c 事件“一直”成立。

(5) @ (posedge clk) a |-> b ——断定 clk 上升沿后，a 事件“开始发生”，同时，b 事件发生。

(6) @ (posedge clk) a.end |-> b ——断定 clk 上升沿后，a 事件执行了一段时间“结束”后，同时，b 事件发生。

注：″a |-> b″ 在逻辑上是一个判断句式，即：

```
if a
b;
else
succeed;
```

因此，一旦 a 发生，b 必须发生，断言才成功。如果 a 没发生，走 else，同样成功。

12. 1. @ (posedge clk) a |=> b ——断定 clk 上升沿后，a 事件开始发生，下一个时钟沿后，b 事件开始发生。即指会在每个 clk 的上升沿检查条件 a 是否成立，如果成立则触发此断言，下面类似。

12. 2 。@ (posedge clk) a |=>##2b ——断定 clk 上升沿后，a 事件开始发生，下三个时钟沿后，b 事件开始发生。 |=>本身就代表了一个时钟

12．3 。@ (posedge clk) $past(a,2) == 1'b1 ——断定 a 信号在 2 个时钟周期“以前”，其电平值是 1。

12．4． @ (posedge clk) a [*3] ——断定“@ (posedge clk) a”在连续 3 个时钟周期内都成立。

12．5． @ (posedge clk) a [*1:3] ——断定“@ (posedge clk) a”在连续 1~3 个时钟周期内都成立。

12．6．@ (posedge clk) a [->3] ——断定“@ (posedge clk) a”在非连续的 3 个时钟周期内都成立。

13．举一个复杂点的例子：

```
property ABC;

int tmp;

@(posedge clk) ($rose(a),tmp = b) |-> ##4 (c == (tmp*tmp+1)) ##3 d[*3];

endproperty
```

上例的一个property 说明：当 clk 上升沿时，断言开始。首先断定信号 a 由低变高，将此时的信号b 的值赋给变量 tmp，4 个时钟周期后，断定信号 c 的值是 4 个周期前b^2+1，再过 3 个周期，断定信号 d 一定会起来，再过 3 个周期，信号 d 又起来一次。。。。。。。只有这些断定都成功，该句断言成功。otherwise，信号 a 从一开始就没起来，则断言也成功。（应该说事件 d 会发生，而非信号）

语法 4：多时钟域联合断言：一句断言可以表示多个时钟域的信号关系，例如：

```
@ （posedge clk1） a |-> ##1 @ (posedge clk2) b
```

当 clk1上升沿时，事件 a 发生，紧接着如果过来第二个时钟 clk2 的上升沿，则 b 发生。“##1”在跨时钟时不表示一个时钟周期，只表示等待最近的一个跨时钟事件。所以此处不能写成##2 或其他。但是可以写成：

```
@ （posedge clk1） a |=> @ (posedge clk2) b
```

语法 5：总线的断言函数

总线就是好多根bit 线，共同表示一个数。SVA 提供了多bit 状态一起判断的函数，即总线断言函数：

(1) $onehot(BUS)　　————BUS 中有且仅有 1 bit 是高，其他是低。

(2) $onehot0(BUS)　　————BUS 中有不超过 1 bit 是高，也允许全 0。

(3) $isunknown(BUS)　————BUS 中存在高阻态或未知态。

(4) countones(BUS)==n ————BUS 中有且仅有 n bits 是高，其他是低。

语法 6：屏蔽不定态

当信号被断言时，如果信号是未复位的不定态，不管怎么断言，都会报告：“断言失败”，为了在不定态不报告问题，在断言时可以屏蔽。

如： @(posedge clk) (q == $past(d))，当未复位时报错，屏蔽方法是将该句改写为：

@(posedge clk) disable iff (!rst_n) (q == $past(d)) //rst 是低电平有效

语法 6：断言覆盖率检测：

name: cover property (func_name)

在 modelsim 中开启断言编译和显示功能：

（1）【编译 verilog 代码时按照 system verilog 进行编译】 vlog -sv abc.v

（2）【仿真命令加一个-assertdebug】 vsim -assertdebug -novopt testbench

（3）【如果想看断言成功与否的分析，使用打开断言窗口的命令】 view assertions

在 VCS 中加入断言编译和显示功能：

在 fsdb 文件中加一句话：$fsdbDumpSVA

在 VCS 编译参数：system "vcs $VCS_SIMULATION" 中加入一些 options:

-assert enable_diag\

```
-assert vpiSeqBeginTime\
-assert vpiSeqFail\
-assert report=路径\
-assert finish_maxfail=100
```

经验】以下是一些编写断言的经验：

1. 断言的目的：传统的验证方法是通过加激励，观察输出。这种方法对案例的依赖严重，案例设计不好，问题不便于暴露。而断言是伴随 RTL 代码的，不依赖测试案例，而是相对“静态”。例如：我们要测试一个串行数据读写单元，数据线只有一根，先传四位地址，再传数据。

（1）案例验证法：写一个地址，再写一段数据，然后读取该地址，看输出的是不是刚才写的数据。

（2）断言法：不需要专门设计地址和数据，当发起写时，在地址传输的时间里将地址存储到一个变量里，在数据传输的时间里将数据存储到一个变量里，观察 RAM 中该地址是否存在该数据就可以了。

断言设计相当于在电脑上把 RTL 实现的功能再实现一遍。

2. 断言中可以包含 function 和 task。而且 function 经常用于断言，因为有的处理很复杂，而断言又是“一句式”的，无法分成好几句进行表达，所以需要 function 替断言分担工作。

3. 断言允许规定同时发生的事件，就是组合逻辑，你可以写成：a &&b，也可以写成 a ##0 b，不能写 ##0.5，不支持小数。

4. 断言是用电脑模仿 RTL 的运行过程，当 RTL 功能复杂时，你必须用到变量。断言中支持 C 语言的 int和数组声明，但在赋值时“不能”写成：##4 var = Signal，其中 var 是断言中的变量，和 RTL 无关，Signal 是 RTL 中的一个信号。本句是想在第 4 周期将Signal 的值赋给var，以便在后面使用该值。但本句只有变量赋值，没有对 RTL 信号的任何断言，就会报错，解决方法是：##4 （“废话”，var = Signal)，一定要有断言的话我们就写“废话”，

例如:data == data 等。如果有多个变量要赋值也可以，##4 （废话，变量 1 赋值，变量 2 赋值.....）

5. 关于断言的表达风格：语法介绍的“a |-> b”，实际上是“if a, then b”的逻辑，当 a 不发生，b 也不会被判断，该断言自然成功。但当我们的逻辑是

```
if a1
        {
if a2
then b
        }
```

该如何用断言表达？？？？或许可以写成：“a1 |-> a2 |-> b”，也可以，但常用的表达是：

“a1 && a2 |-> b”或者“a1 ##3 a2 |-> b”

6. 关于断言的时序：时序逻辑的断言需要注意的一个问题：

例如：假设当 clk 上升沿到来时，b<=a。将上述逻辑写成断言时，如果写成“@(posedge clk) b==a”，看起来和 b<=a 一样，但实际上是错的。因为当时钟上升时，b 还没有得到a 的值，a 还需要一段保持时间。即，断言中的信号值实际上是时钟沿到来之前的值，而不是时钟沿到来后他们将要编程的值。所以，b<=a 逻辑的断言应该是：“@ (posedge clk) (a==a,tmp=a) |=> (b==tmp);”

针对上述几点，举一个复杂的例子：

断言wr 的功能是检查串行地址输入是否正确，串行地址输入线是 DataIn 。$time 返回值以 0.1ns 为单位（因为我在 testbench 中的单位规定是`timescale 1ns/100ps，精度是 100ps = 0.1ns），所以$time/10 才是 ns。

```
////////////////////////////////////////////////////////////////
////////////
    wr: assert property (wr_p)
```

```
        $display ("succeed: ",$time/10);
    else
        $display ("error: ",$time/10);
    ////////////////////////////////////////////////////////////////
///////////////
    //断言可以声明一个 int 数组 arr[4],
    //“@(posedge clk) !vld_pulse_r[0] && !DataIn”是真实的预备条件
    //“##4 (read==read, arr[0] = DataIn)”只是为了在特定时间内赋值，
有用的语句是“arr[0] = DataIn”，//“read==read”是废话，为了编译
通过。
    //arr 赋值完毕后，进入 function 进行处理，判断实际地址 addr 跟 junc
处理过的数据是否相同。
    //“addr == junc(arr[0],arr[1],arr[2],arr[3]);”就是 junction
调用。

    property wr_p;
    int arr[4];
    @(posedge clk)  !vld_pulse_r[0] &&  !DataIn
                ##4  (read==read, arr[0] = DataIn)
                ##1  (read==read, arr[1] = DataIn)
                ##1  (read==read, arr[2] = DataIn)
                ##1  (read==read, arr[3] = DataIn)  |=>
                addr == junc(arr[0],arr[1],arr[2],arr[3]);
    endproperty
    ////////////////////////////////////////////////////////////////
////////////
    function  [3:0] junc;
    input a,b,c,d;
    reg  [3:0] a1;
```

```
reg [3:0] b1;
reg [3:0] c1;
reg [3:0] d1;


        a1 = {3'b0,a};
        b1 = {3'b0,b};
        c1 = {3'b0,c};
        d1 = {3'b0,d};
junc = a1+(b1<<1)+(c1<<2)+(d1<<3);
        $display (junc);
    Endfunction
```

如果想在 SVA 中使用类似 for(){....}的功能，别忘了语法中介绍的[*3]，这是在断言中实现 for 的唯一方式。

```
                ##4 (废话, cnt = 0, arr[cnt] = DataIn, cnt++)   //
```

初始化一下，

```
                ##1 (read==read, arr[cnt] = DataIn, cnt++)[*3]
```

//循环 3 次

8. 每句断言都是一个小程序：如上例，在##4 时间点上，(废话, cnt = 0, arr[cnt] = DataIn, cnt++)就是一个小程序，信号断言必须是第一句，其他运算按照顺序进行。

9. 断言的变量除了可用 C 语言中的 int，float 外，还可以是 reg [n:0]等数字电路类型。

10. 注意：

像这种写法：

```
property ept_p;
@(posedge rd_clk)    ((rd_num == 0)  |-> rd_ept)
&&  (rd_ept  |->  (rd_num == 0));
```

```
endproperty
```

是错误的，写了|->，就不能再用&&等事件组合逻辑了。

解决方法是使用 2 个断言，没更好的方法。

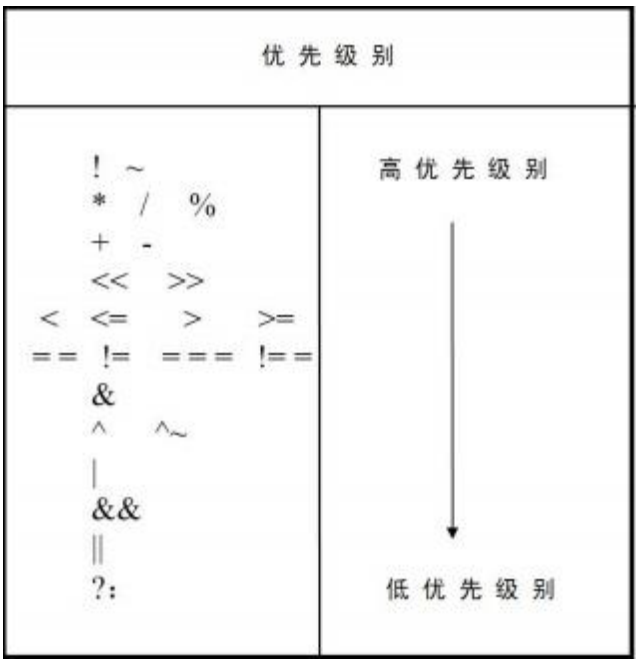
例子讲解

```
property request_2state;
@(posedge clk) disable iff(rst);
   $isunkonwn(request)==0;
endproperty
assert_request_2state:  assert property  (request_2state);
```

上述例子为：复位释放后，每个时钟沿都会检测 request 信号是否为x或 z 态，如果没有则正常运行，否则报错，检测一些关键信号非常有用

V_SVerilog 篇

基本语法介绍



1. 在 sv import C 时，如果在 sv 的 for 循环中有调用 C，那么可能 for 循环的变量被 C 改变清 0，解决办法在外面定义循环变量为 static 类型，（尤其是 C 使用了大量的存储空间）

2. = 阻塞赋值；=>非阻塞赋值

3. Fork…join;fork…join_any;fork…join_none 区别

Fork

Task a;

Task b;

Join

Task c;

只有先 task a,task b 全部执行完，才执行 task c

Fork

Task a;

Task b;

Join_any

Task c;

task a,task b 任何一个执行完，接下来都会执行 task c（a,b 中没执行完的继续执行）

Fork

Task a;

Task b;

Join_none

Task c;

Task d;

……

task a,task b,task c 同时执行（但第一条语句是执行 c 的语句），task c,task d 会顺序往下执行；上面语句相当于

fork

beign

Task c;

Task d;

…

End

begin

Task a;

Task b

end

join

fork…join_none 内部的功能可以用来监控 join_none后面的内容似乎很好用。

可以使用 disable 来停止线程,如果使用了标签(labe),则停止有标签的线程,如果没有使用标签,则关闭所有当前衍生出来的线程

注意使用 disable 来关闭某一个任务,如果在 task 中使用了 disable 本身,那么所有被调用的任务都会被关闭;

Task call; …… disable call; endtask

Initial call(1); call(2); call(3); end,当任何一个 call 中执行了 disable call,那么所有的 3 个 call 都会停止.

4. 在使用== 比较两端数据非 int 类型数据时，位宽需要保持相同

5. Primitives 原语 介绍

Or,buf,nand;bufif0(buf if 0,条件缓冲，0 有效，位置顺序

buf(out,in,enable));

bufif1;notif0,notif1,

buf 有线与功能,如 u1 bufif0(pad,1,0),这个 cell 的 pad 输出 1; u2 bufif0(pad,1,1),这个 pad 输出 z,最终结果 pad 输出 1

MOS 开关：

pmos(out,in,enable),低电平有效;

nmos(out,in,enable),高电平有效;

rpmos, rnmos:信号强度较弱;用于对同一个信号赋值时,如果对方为非高阻态,则以对方为赋值;

如下面 pad 模型:

supply1 my1;

supply0 my0;

buf (C,PAD);//pad 给 c(core)

bufif0 (C_buf, I, OEN);如果是是输入态, C_buf 为 z,否则 c_buf=I

rnmos (C_buf, my1,PU); //如果上拉,c_buf 为弱 1,如果外部 PAD 也有赋值,则以外部 pad 为输出值.

pmos____(PAD,C_buf,my0);_//把 C_buf 给 PAD,如果 c_buf 是强 1/0, 则此处输出的PAD 就是强 1/0, 如果 c_buf是弱 1/0,则此处输出的 PAD 就是弱 1/0,

上下拉：pullup (out);pulldown(out)

双向开关：tran(inout,inout2); tran(inout,inout2)

双向可控开关:tranif0(inout,inout2,ctrl);

tranif1(inout,inout2,ctrl); rtranif0(inout,inout2,ctrl);

rtranif1(inout,inout2,ctrl)

上述可控型的结构，当不导通时，输出为Z 态

门延时说明： 上升延时指!1->1 的延时; 下降延时指!0->0 的延时; 截至延时指的 是 0/1->Z(关闭)的延时，只有三态门会有 3 个延时，其它的都是 2 个延时参数；如果只写了一个延时，则表示上升/下降/截至延时相同；每个延时又分min,type,max；所以在 vcs 选项中需要有 maxdelays,mindelays,typedelays;

`default_nettype nettype; `default_nettype 用来定义没声明但又用到线网的类型，如，`default_nettype wand 表示后面出现的未加说明的线网都是线与线网，直到遇到另外一个`default_nettype 或`resetall 编译指令出现

数据类型转换

1.静态转换不对转换值进行检查，并在需要转换的表达式前加单引号，如 int i; i=int’(10.0-0.1); —强制转换,不会抱错,如果有溢出,则很麻烦,不如$cast 好

2.动态转换函数$cast 会对越界的数值进行检查。

3.流操作符用于把其后的数据打包成一个比特流，如 int h; bit[7:0] j[4]=’{8’ha,8’hb,8’hc,8’hd};h={>>{j}}从前向后打包，h=0a0b0c0d。 h={<<{j}}从后向前打包，h=b030d050。

h = { <<byte{j} };//0d0c0b0a,字节倒序,

操作符>>把数据从左至右变成流，操作符<<把数据从右至左变成流;

不能将比特流结果直接赋给非合并数组,而应该在赋值表达式的左边使用流操作符把比特流才分到非合并数组中;注意使用{}和<< , >>一起使用.

把比特流给队列比较好用,

bit test[$];bit[7:0] data1; test = {>>{data1}};

如果底层是发送数据流,则可以直接使用

foreach(tx[i]) tx_data(tx[i]); 上层把各种格式塞到 tx 这个队列中,drive 再把这个队列发出去.

合并数组和非合并数组

1）合并数组：

存储方式是连续的，中间没有闲置空间。

例如，32bit 的寄存器，可以看成是 4 个 8bit 的数据，或者也可以看成是 1 个 32bit 的数据。

表示方法：

数组大小和位，必须在变量名前指定，数组大小必须是【msb：lsb】

Bit[3:0] [7:0] bytes ;

2）二维数组和合并数组识别：

合并数组：bit [3:0] [7:0] arrys; 大小在变量名前面放得，且降序

二维数组 int arrays[0:7] [0:3] ;大小在变量名后面放得，可降序可升序

位宽在变量名前面，用于识别合并和非合并数组，位宽在后面，用于识别数组中元素个数

6. V 与 SV 最大的区别在于 SV 是面向对象编程（OOP）：类的封装（CLASS），类的扩展（extends），多态（重载功能）（利用虚方法实现，virtual task/function,多个子程序使用一个共同的名字的现象叫做多态“polymorphism”）

7. 类的封装：变量可以使用 local 类型，它定义的变量只能在当前类使用，即使是子类也不能使用，如果想子类使用则需要定义为protected 类型。

Local 与 protected 区别

8. system verilog 允许使用 2 种构造方法来创建一个可以共享的基类;1.抽象类,即可以被扩展但不能被直接实例化的类,它使用"virtual"关键词进行定义.

2.纯虚方法,这是一种没有实体的方法原型

9. 一个由抽象类扩展而来的类只有所有虚方法都有实体的时候才能被例化.关键词pure 表明一个方法声明是原型定义,而不仅仅是空的虚方法.纯虚方法只能在抽象类中定义,但是抽象类中也可以定义非纯方法

10. 简单说:在抽象类中的方法有实体叫虚方法,没实体就是纯虚方法.virtual task/function

11. V 中的 parameter 可以使用选择关系：如 localparam my_vara = (you_vara<=8) ? 1: (you_vara<=8) ?2:8 ;

12. v 用在设计中只有两种电路，组合逻辑电路描述和寄存器描述；变量也只有两种，wire 和 reg, 连线（含 input 默认 wire 类型）和 assign 语句用wire 类型变量，其它的都是用 reg 类型。Wire temp=1;相当于 assign temp=1;reg temp=vary；只是编译时给 temp 赋了一个初始变量 vary，如果 vary 是开始是 x 态，那么 temp 就是 x 态。

13. 控制覆盖率 scope：(only need at compile，也可以通过 pragmas to limit condition coverage)

-cm_hier cm_file

cm_file：

+/-tree instance_name [level_number]

+/-module module_name

+/-file file_name

+/-filelist file_name

+/-library

+/-node

支持通配符* ？ +

覆盖率合并：urg [assert minimal]

[-dbname dirname] [-dbname dirname/testname]

[-diff]

[-dir dir1 [dir2 ...]] [-dir dir3 ...]

[-elfile <file>]

...

[-format text/both]

[-hier <file>]

[-log file]

[-metric[+]line+fsm+cond+tgl+branch+assert+group]

[-report dir]

[-show brief/fullhier/...]

参考:$VCS_HOME/doc/pdf/cov_ref.pdf

排除 default case 方法：

urg -line nocasedef

14. 在由于不同的数据类型而导致变量不能正常赋值的情况下，SystemVerilog 提供了$cast 系统任务来进行赋值操作。$cast 可以作为任务或函数来调用。

$case 的语法如下：function int $cast(singular dest_var, singular source_exp);

或者 task $cast(singular dest_var, singular source_exp);

dest_var 是需要赋值得变量。

sorce_exp 是将要赋值到目的变量的表达式。

将$cast 作为任务还是函数调用确定了无效赋值是如何处理的。

当作为任务调用时，$cast 试图将源表达式赋值给目的变量。如果赋值是无效的，会出现运行时错误并且目的变量保持不变。

当作为函数调用时，$cast 试图将源表达式赋值给目的变量，如果赋值成功则返回 1。如果赋值失败，$cast 不会进行赋值操作而是返回 0。当作为函数调用时，不会出现运行时错误，并且目的变量保持不变。

15. 必须注意：$cast 执行的是运行时检查。除了检查目的变量和源表达式是否为单一类型外，编译器不会进行类型检查。

16. 动态数组 new 可以传递变量，如 bit[7:0] matrix[]; matrix = new[i];i 为传递进来的变量

二维动态数组 new 的方法：

```
bit[7:0]matrix[][];
matrix = new[i];//
foreach(matrix[i,])begin
matrix[i] = new[j] //
end
```

//最后开辟的数组大小为 matrix[i][j]；使用 matrix.size(),能得到 i 的值，使用 matrix[i].size()可以得到 j 的值；

三维动态数组 new 的方法：

```
bit[7:0]matrix[][][];
matrix = new[i];//
foreach(matrix[i,,])begin
matrix[i] = new[j] //
foreach((matrix[i,j,])begin
```

matrix[i][j] = new[k] //

end

end

//最后开辟的数组大小为 matrix[i][j][k]

关联数组采用在方括号中放置数据类型的形式来进行声明：注意与动态数组的区别---就是其它语言的哈希数组

Bit[31:0] assoc[bit[31:0]];

Int switch[string];

17. 在单纯的 .v 环境中，task 默认为静态的，它的变量都是静态的，静态变量指退出了 task,此变量的值还保留着，下次再调用此 task,变量的值不变，如果采用+1，则在原值上+1；如果 task 声明为 auto 类型，那么其里面的变量为动态的，否则为静态变量，但在 sv 中可以在 automatic task 声明 static 变量，同样在 static task 中可以声明 automatic 变量

reg [7:0] value=0;此句表示在编译时给该变量初始值为 0，并不是每次运行时都赋值为 0；与 reg[7:0] value; value=0;是不同的概念

18. V 常用 2种语法 assign 和 always块，assign 的变量为 wire类型，always 的变量为reg 类型，当为时序电路时用《=，当为组合电路时用=；

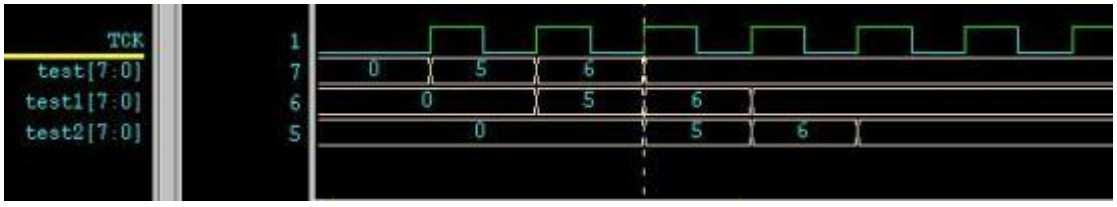
19. V 中的 always @(posedge clk) a<=b;c<=b 与 SV 中 forever begin @(posedge clk) a<=b;c<=b;end 结果一至，不过 forever 需要在 initial begin end 中而已

在验证中<=与=区别

```
initial begin
reg[7:0] test,test1,test2;
test=0;
test1=0;
test2=0;
```

```
    @(posedge TCK)
    test<=5;
    test1<=test;
    test2<=test1;
    @(posedge TCK)
    test<=6;
    test1<=test;
    test2<=test1;
    @(posedge TCK)
    test<=7;
    test1<=test;
    test2<=test1;
    @(posedge TCK)
    test1<=test;
    test2<=test1;
    @(posedge TCK)
    test1<=test;
    test2<=test1;
    @(posedge TCK)
    test1<=test;
    test2<=test1;
    @(posedge TCK)
    test1<=test;
    test2<=test1;
end
```

波形如下：

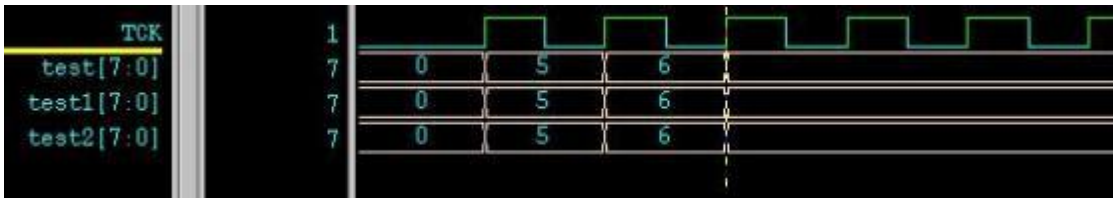


与寄存器的特性一致；

```
initial begin
reg[7:0] test,test1,test2;
test=0;
test1=0;
test2=0;
@(posedge TCK)
test=5;
test1=test;
test2=test1;
@(posedge TCK)
test=6;
test1=test;
test2=test1;
@(posedge TCK)
test=7;
test1=test;
test2=test1;
@(posedge TCK)
test1=test;
test2=test1;
@(posedge TCK)
```

End

波形如下



20. 操作符>>和>>>区别，>>表示逻辑右移，左边补 0，>>>表示数据右移，符号位不变，数据右移；<<在验证中相当于 2**n（2 的 n 次方）

21. 注意，bit,reg 的变量都是无符号数，无法实现有符号数，有符号数的位宽都是固定的，如果 byte signed,short int signed 都是

8bit,interger 是 32bit,time,real 为 64bit;不能定义和为 bit[7:0] signed xx ;或 reg[31：0] signed xx ;

22. i n i t i a l
b e g i n

R e s e t = 0;

R e s e t = #100 1;

R e s e t = #80 0;

R e s e t = #30 1;

e n d
和

i n i t i a l

b e g i n

R e s e t <= 0;

R e s e t <= #100 1;

R e s e t <= #180 0;

R e s e t <= #210 1;

e n d 效果是一样的，原因是=为阻塞（串行），<=为非阻塞（并行）

23. 注意区别&&和&，||和|；&，|表示位与，位或；&&，||表示逻辑与逻辑或，逻辑与/或得到的结果只有真和假（1 和 0，为 1bit 数据）！

表示逻辑取反，~表示位取反，**表示 n 次方，2**3，表示 2 的 3 次方

24. assign #7 B=A;此时表示每隔 7 个单位时间把 A 赋值给 B，如果在 7 个单位之间，A 有变化，B 不会跟着变化

assign B=A;表示B 随着A 的变化而变化，相当于 always @(A) begin B=A end;
只有输入信号 A 变化了，A 才会把值传给 B，试分析 assign B=function(A);

当 A 没变，但 function(A)有变化，此时 B 仍然不会变，因为输入信号 A 没变化，所以 B 就不会变

25. reg，logic,integer 是四态，

bit,int 是 2 态，

四态变量默认初始值为 X,

两态变量默认初始值为 0

wire 当为四态时，默认初始值为Z；当为两态时，默认初始值为 0.

26. a s s i g n # (4,8,6) A r b = & DataBus 上升时延为 4，下降时延为 8，截止时延为 6，传递到 x 的时延为 4 （ 4、8 和 6 中的最小值)； w i r e #5 A r b;这个时延表明 A r b 驱动源值改变与线网 A r b 本身间的时延。考虑下面对线网 A r b 的连续赋值语句：a s s i g n # 2 Arb = Bod & Cap;

27. 可以用$fseek(pt,position,0);定位

1.在 V 中读取文件采用:$readmemb(file.dua,reg),表示把 file.dua 的数据读到 reg 存储器；

读十六进制使用$readmemh(file.dua,reg);每次读取一行，变量的宽度要与每行的向量长度一致，但函数会连续把整个文件读出。对比读出的xxxx 态需要用===

reg [7:0]temp ;fp=$fopen("file.bin","rb");$fread(temp,fp);可以读出二进制 bin 文件;如果 temp 是 32bit 变量，将一次读到 32bit 数据

$readmemb 和$readmemh 读的是整个文件到数组中(**读取对象为文本文件**)，

基本函数:

(1)$readmemb("<数据文件名>",<存储器名>)；

(2)$readmemb("<数据文件名>",<存储器名>,<起始地址>)；

(3)$readmemb("<数据文件名>",<存储器名>,<起始地址>，<结束地址>)；

$fread 每次只读一个变量的长度(**读取的对象为二进制文件**)，读整个文件时需要反复读；它们来源于 C 语言

2.存放的二进制文件都是连续存档的，如果检测到内容为 X 则表示文件结束，注，读出存放的要定义为四态变量，因为四态变量有 X，二态变量只有 0 和

1，这样二态变量无法知道是否文件是否结束;四态变量 logic,reg,二态变量 bit

3.TASK 的输入应该是可以任何类型的数据，如 input string name;定义了变量name 为字符类型，那么它就可以传递一个文件名给该 task;

4.integer file_pr=$fopen(name.f);打开 name.f 这个文件，file_pr 相当于指向该文件的指针。

$fdisplay (file_pr,"Input = %b%b%b, Outpu=t %b%b",A, B, Cin, Sum, Cout “） ; 把""中的内容打印到 name.f 文件中； $fclose(file_pr);关闭这个文件

5.$sdf_annotate("sdf_file",module_instance,"scale_factors");该语句用来读取module_instace(后仿是网表）的反标时序文件（标准延迟文件），scale_factors 为时序的放大倍数，该函数会把网表文件和时序文件结合起来

28. 读取 SDF 文件时$sdf_annotate ("sdf_file", [module_instance, "config_file"," log_file", "mtm_spec","scale_factors"," scale_type"]);

sdf_file：SDF 文件的绝对或相对路径

module_instance：标注范围。缺省为调用$sdf_annotate 所在的范围

config_file：配置文件的绝对或相对路径。缺省使用预设的设置。

Log_file：日志文件名，缺省为 sdf.log。可以用+sdf_verbose 选项生成一个日志文件。

Mtm_spec：选择标注的时序值，可以是{MINIMUM，TYPICAL，MAXIMUM，TOOL_CONTROL}之一。缺省为 TOOL_CONTROL(命令行选项)。这个参数覆盖配置文件中 MTM 关键字。可以在 MAKEFILE 中加入 DELAY_OPT=mindelays 或 maxdelays

Scale_factors：min:typ:max 格式的比例因子，缺省为 1.0:1.0:1.0。这个参数覆盖配置文件 SCALE_FACTORS 关键字。

Scale_type：选择比例因子；可以是{FROM_MINIMUM, FROM_TYPICAL, FROM_MAXIMUM, FROM_MTM}之一。缺省为 FROM_MTM。这个参数覆盖配置文件中 SCALE_TYPE 关键字。

29. 在用$display 时，可以加上宏`__FILE__和`__LINE__,则打印时会显示出是那个文件及那行打印;$display ("string",`__FILE__,`__LINE__)

$display ("%m:xxxxx");%m 表示将会打印出当前打印语句的层次关系，如：cal_pkg::\cal_ref_mdl::ref_mdl_main:xxxx

30. $fprintf,$fscanf

格式打印函数，$fprintf(文件指针，格式字符串，输出表列)；

$fprintf(文件指针，格式字符串，输入表列)；

$fprintf(fp,"%d,%f",I,t);把 I,t 按照格式输出到 fp 指向的文件，

$fscanf(fp,"%d,%f",I, t); 将 fp 指向的文件内容送给变量 I,t，它可以读取文本，但每次都是读取一行，所以变量的长度需要注意

实际读写数据时采用$fread(buff,fp),和$fwrite(buff,fp);

$feof(fp),检查是否到了文件末尾(二进制文件)

$fread 可以读一切 ASIC 文件

31. $write 和$display 一样都是打印语句，只是 display 打印完后自动换行，但 write 不会自动换行，

32. 格式化打印，对于字符类型变量处理很好用，

$sformat(name,"../../aa%0d",i);name 变量可以得到../../aa1, (i=1) 与下面的功能一样

格式化打印: string meger_string=$sformatf("string1_%d",var);可以把 string1_ (var 的值)合并起来给 meger_string;

33. 在一个 CLASS 中，new 函数不能是 virtual 型，

34. 在一个 CLASS 中，如果两个随机数是用randc,那么这两个数不能约束相等，而用 rand，可以约束相等，如果用了 constrain ， 则不要用 randc

35. 数据取反：reg[7:0] datatemp;datatemp.reverse;此时 datatemp 的数据倒转了，注意不能 databuf=datatemp.reverse;

36. 在 TC 中引用 makefile 中定义的宏或其它宏使用 `"`filename`"

37. $test$plusargs(string) 整个函数用来查找命令行的字符串
$value$plusargs(user_string,variable);user_srting 命令行内容
variable 使用时的变量

如+FINISH=10000 +TESTNAME=this_test

real frequency;int stop_clock; reg[8#32:1] testname;

if($value$plusargs) ("FINISH=%d",stop_clock);

if($value$plusargs) ("TESTNAME=%s",testname);

命令行则使用相应的命令 args=+stop_clock=1000;args=+ 是固定用法;

如果使用多个变量则需要使用“ ”，如 args="+stop_clock=1000 +testname="ok""

38. $onehot(表达式) 如果表达式只有 1bit 为 1，则返回真

$onehot0(表达式) 如果表达式至少有 1bit 为 1，则返回真，即表达式不全为 0，及表达是不为 0，返回为真，$onehot0 returns true if at most one bit of expression is high.（表达式至多有一位为 1）,---即表达式中有一个 1 或者全部是 0;

$isunkonwn(表达式) 如果表达式任何一位为X 或 Z，则返回真

$countones(表达式) 返回表达式中 1 的个数

$sampled (expression [, clocking_event]) : is provided to return sampled value of an expression

$rose (expression [, clocking_event])

$fell (expression [, clocking_event])

$stable (expression [, clocking_event])

$past (expression [, number_of_ticks] [, expression2] [, clocking_event])

39. $typeof(表达式) 返回表达式或数据的类型,返回值是不可见的，只能使用；sv 中有一个不常用的数据类型 type，它可以保存变量的类型，如 int busa; type bus_t ; bus = $typeof(busa);

40. 想显示字符可以定义枚举类型数据；格式为

typedef enum {sta0=0,sta1=1,sta2,sta3,...} e_sta;
e_sta e_temp;定义变量 e_temp 为枚举类型

41. $bits(表达式或变量) 返回表达式或变量的有效位数，如 reg[7:0] temp; int long =$bits(temp);//long=8

42. $writememh("file_name", memory_name[, start_addr[, finish_addr]]);

$writememb("file_name", memory_name[, start_addr[, finish_addr]]);

43. 数组查询系统函数

$left shall return the left bound (msb) of the dimension

$right shall return the right bound (lsb) of the dimension

$low shall return the minimum of $left and $right of the dimension

$high shall return the maximum of $left and $right of the dimension

$increment shall return 1 if $left is greater than or equal to $right, and -1 if $left is less than $right

$size shall return the number of elements in the dimension, which is equivalent to $high - $low + 1

$dimensions shall return the number of dimensions in the array, or 0 for a singular object

44. 固定(动态)数组 size 大小用$size(arrary) ，队列的尺寸大小用 q.size();得到

45. reg [8:0] test ; test1=test[8:1]+test[0]; 该方法表示 test/2+余数，如果不+test[0]则表示舍去余数；同理可以推出 /4,/8 对余数的处理方法

46. 用接口时，如果不用时钟，在声明时需要用空括号，不能省略，如 infterface infterface_s();但一般我们都会把一个系统时钟作为输入送给它，下面两个是等效的。 assign infterface_s.clk = clksys; infterface_s(clksys);

47. 联合数组实现了一个所声明类型的元素的查找表。用作索引的数据类型作为查找表的查找键值，并强制了一种顺序

48. 联合数组的声明语法如下：

```
data_type array_id [index_type];
```

其中：data_type 是数组元素的数据类型。固定尺寸数组可以使用的任何类型都可以作为联合数组的数据类型。

array_id 是联合数组的名字。

index_type 是用作索引的数据类型，或者是*。如果指定了*，那么数组可以使用任意尺寸的 integral表达式来索引。采用数据类型作为索引的联合数组将索引表达式限制成一个特定的数据类型。

联合数组声明的例子如下：

```
integer i_array[*];          // 整数联合数组（未指定索引）
bit [20:0] array_b[string]; // 21 位向量的联合数组，使用字符串类型作为索引
event ev_array[myClass];    // 事件类型的联合数组，使用类 myClass 索引
```

49. 4.10.1 num() num()方法的语法如下： num()方法返回联合数组中条目的数目。如果数组是空数组，那么它返回 0。

4.10.2 delete() delete()方法的语法如下：其中 index 是一个可选的适当类型的索引。

如果指定了索引，那么 delete()方法删除特定索引的条目。如果试图删除的条目不存在，该方法不会发布警告信息。

如果没有指定索引，delete()方法删除数组的所有元素。

4.10.3 exists()　exists()方法的语法如下：　exists()函数检查对于指定数组中是否存在指定索引的元素。如果元素存在则返回 1，否则返回 0。

```
protected      uvm_component m_children[string];
m_children.exists(child.get_name()
```

4.10.4 first()　first()方法的语法如下：　first()方法将指定的索引变量赋值为联合数组中第一个（最小的）索引的值。如果数组是空的则返回 0，否则返回 1。

4.10.5 last()　last()方法的语法如下：　last()方法将指定的索引变量赋值为联合数组中最后一个（最大的）索引的值。如果数组是空的则返回 0，否则返回 1。

4.10.6 next()　next()方法的语法如下：　next()方法寻找索引值大于指定索引的条目。如果存在下一个条目，索引变量被赋值为下一个条目的索引，并且函数返回 1。否则，索引不会发生变化，函数返回 0。

4.10.7 prev()　prev()方法的语法如下：　prev()函数寻找索引小于指定索引的条目。如果存在前一个条目，索引变量被赋值为前一个条目的索引，并且函数返回 1。否则，索引不会发生变化，并且函数返回 0。

50.　min,max,unique,reverse,sort,rsort,shuffle 这个几个函数数组队列都可以用

```
int q=’{1,2,6,2,6,8}
```

min:找出数组中最小的值 tq=q.min;则 tq={1};

max:找出数组中最大的值 tq=q.max;则 tq={8};

unique:返回唯一的值 tq=q.unique;则 tq={1,2,6,8};　//min,max,unique 的结果都是一个队列类型

find with:按要求找数 tq=q.find with　(item>3);则 tq={6,6,8};

sun:数组求和 int　tq=q.sum;则　tq=25;

reverse:q.reverse;表示把 q 的数据反向放置；高位在低位，低位在高位；

sort:q.sort;表示把 q 进行排序，小的在 0 位，最大的在最后；

rsort:q.rsort;也是排序，只是最大的在位置 0，最小的在最后，

shuffle:q.shuffle;表示把 q 里面的元素顺序打乱

动态数组使用前需要先 new[];$size 对动态数组很重要，因为动态数组开始并不知道其大小；但在使用前要通过 NEW[5]之类的语句申请空间的大小

数组可以直接比较和赋值

51. 队列与动态数组不同，它不需要通过 NEW[]来分配空间大小，但可以把定宽或动态数组的值复制给队列

队列 int q[$];队列操作：q.insert(1,j);在编号为 1 的数据后插入 j,(数据编号是从 0,1,2 开始)；q.deltet(0);删除第 0 个元素；

q.push_front(6);,在对了 q 的最前面插入数字 6；q.push_back(6);,在对了 q 的最后面插入数字6；

j=q.pop_back;q 从最后弹出一个数给 J，j=q.pop_front;q 从最前面弹出一个数给 J，同时 q 会少一个数

52. 在检测上升沿和下降沿时，由于从 0->!0 表示上升沿，1->!1 表示下降沿，那么也就是说从 0->x 或从 0->z 都会被认为上升沿，从 1->x 或从 1->z 都会被认为下降沿，所以在等待沿是用@不是非常可靠，可以在@语句之后再在相关时钟下用全等===来检测一下是否===0 或是否===1。比较下面语法，===严格相等，结果只有 1 和 0；== 相等，如果遇到其中某一个有 X 态，结果对应位为 X

```
module equalities2();
reg [3: 0] rega, regb, regc;
reg val;
initial begin
rega = 4'b0011;
regb = 4'b1010;
regc = 4'b1x10;
end
```

```
initial fork
      #10 val = rega === regb  ; // val = 0
      #20 val = rega  !== regc;   // val = 1
      #30 val = regb === regc;  // val = 0
      #40 val = regc === regc;  // val = 1
      #50 $finish;
join
Endmodule
module equalities1();
reg [3: 0] rega, regb, regc;
reg val;
initial begin
rega = 4'b0011;
regb = 4'b1010;
regc = 4'b1x10;
end
initial fork
      #10 val = rega == regb  ;  // val = 0
      #20 val = rega  != regc;    // val = 1 ，只要检测到 1bit 不等就成立
      #30 val = regb  != regc;    // val = x  ，
      #40 val = regc == regc;   // val = x
      #50 $finish;
join
Endmodule
```

53. 对于同步寄存器的第一级寄存器的时序检查不必在乎，但实际上会报时序违背，这时需要通过一定的手段让不不报出来干扰。

```
Instance{
   top.udut.reg0,
```

top.udut.reg1,

….

top.udut.regn

} {noTiming};

(上述内容可以写在一个文件中，filename)

只要在 MAKEFILE 中通过选项+optconfigfile+filename，则可以不对上述寄存器做时序检查

其功能等同于

initial begin

force top.udut.reg0.notifier = 1'b0;

…

force top.udut.regn.notifier = 1'b0;

end

notifier 是寄存器的时序方面的一个选项，也可能是其它标志变量，force 它等于 0，相当于屏蔽时序检查的功能，个人倾向于第一种方法

54. 类的 NEW()函数除了分配空间外，还一个就是初始化变量；二值变量默认初始化为 0，四值变量默认初始化为 X。New()返回的是一个指向类对象的句柄。

55. 类里面的静态变量使用范围是这个类的所有实例；而动态变量只使用范围只有当前实例化的实例中，类中默认是动态变量；如下

class Transaction;

static int count=0;//初始化静态变量

int id;

function new();

id=count++;

endfunction

endclass

Transaction t1,t2;

initial begin

```
  t1=new();  //第一个实例，id=0,count=1;
  t2=new();  //第二个实例，id=1,count=2;
$display ("second id=%d,count=%d,t2,id,t2.count");
end
```

在创建类的时候建议使用 ID 来区分；静态变量通常在声明时初始化，而不在 new 函数中初始化

56. this 关键字是一个指针，它指向类一级的变量

```
class Scoping;
string oname;
function new(string oname);
    this.oname = oname;  //类变量 oname = 局部变量 oname;
endfunction
Endclass
```

57. 可以用 typedef 语句声明一个类，然后这个类就可以使用了，然后在后面再定义类;如 typedef love; 然后就可以使用 love 这个变量（，love 可以在其他地方声明

58. 通过方法来创建句柄，方法的参数前需要加上ref，否则创建不了，编译也会出错，因为先要输入变量，然后再返回该变量，只有申明 ref 类型才满足该要求

```
function void create(ref Transaction tr)
tr = new();
   tr.addr = 42;..
Endfunction

transaction t;
initial beign
  create(t);//创建一个 transction
End
```

59. 一个句柄不能传递给 2 个 mailbox();需要拷贝一份分别传递。

60. 流操作符<< （把数据从左至右变成流）和>> （把数据从右至左变成流);后面带表达式，结构或数组。

```
Initial  begin
int h;
bit[7:0] b,g[4],j[4]=`{8'ha,8'hb,8'hc,8'hd};//注意实际是
0a,0b,0c,0d
bit[7:0] q,r,s,t;
h={>>{j}};  //0a0b0c0d--把数组打包成整型
```

61. 基类中的virtual 函数和任务；目的是为了在扩展类中可以重定义，也就是扩展类可以定义一个与基类中同名的方法，但同时在扩展类中可以调用原来的基类中的方法，，注意 new 函数没有virtual 的属性

62. 如果在扩展类定义了一个约束，并且扩展后的约束名和基类里的约束名相同，那么扩展类的约束会替代基类中的约束，这样就可以改变现有约束的行为

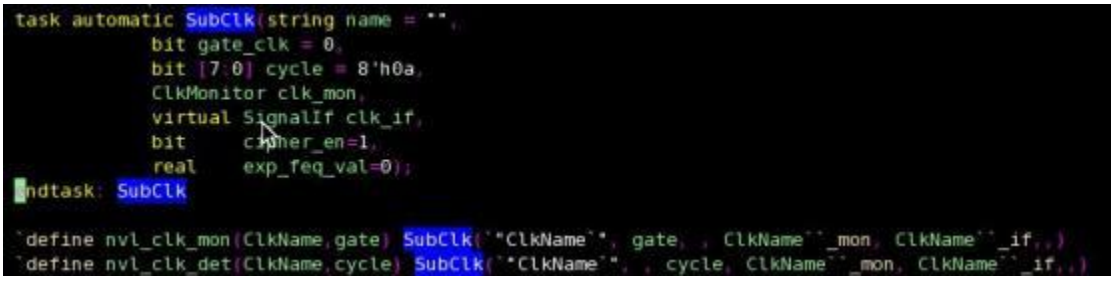
63. 接口信号必须使用非阻塞赋值来驱动；即用<=符号赋值

64. systerverilog 中在 program 中不允许使用 always 块，因为 always 块没有初始化，而验证平台中均需初始化

65. 时钟发生器尽量不要放在程序块（program）中，应该把它放在 module 中，然后例化；主要是避免初始化时的上升沿出现

66. 建议在每个 module 文件顶部加入`timescale 以保证此 module 的单位时间是预期否则会按照编译顺序以前一个`timesacle 定义的单位一致,timescale 只能在 module 外部使用，内部使用的无效

67. 宏可以传递参数，一般都是一个变量或字符，但想传递变量或字符的部分内容，则需要用连接符号``，而连接“只使用`,如



连接下划线 _ 及点 .使用 ``连接“或\使用 `连接。

68. 用 rand_mode(0)可以禁止变量随机化，如

assert(handle.randmize());handle.length.rand_mode(0);hand le.length=55;此时 length 固定为 55；然后再

assert(handle.randmize());此时 length 还是 55，没有随机化，handle 中其它的随机变量都随机了。

69. 用 constraint_mode 打开或关闭约束，可以在随机类中做 2 个约束，一个随机范围广，一个随机几个固定的边界值；这样可以完成先随机再定向测试；

70. 先使用 xxx(class variable).constraint_mode(0);关闭所有约束，再使用 xxx(class variable).xx(constraint

name).constraint_mode(1)打开某个约束

71. randomiz();当（）为空时，默认随机所有的随机变量，当（）为某个随机变量时，则只会随机固定的几个变

量 ;handle.randmize(a,b,c) 只会随机 a,b,c 三个变量

72. constraint 可以使用权重分布 dist 来控制，使用：=或：/;它们有些不同:=表示所有的权重都一样，如[15,19]:=20;则表示出现 15~19 的权重均为 20,；但[15,19]:/20,则表示权重总共为 20，每个值得权重为4；而且权重可以使用变量来控制，在运行中改变此变量，则可以改变权重；如果 0:/w1;1:/w2;

73. ref 端口方向，只有 systerverilog 才有，它是对变量的引用（不能是 NET），它的值是变量最后一次赋的值，如果将一个变量连接到多个 reg 端口，就可能产生竞争，因为多个模块的端口都可能更新同一个变量,当一个 function 改变了 ref 变量时，在 function 外部也可以看的到；ref 与 inout 类似，但 inout 在 function 中时，

function 会先拷贝该变量，当操作完结束返回函数时，再把值拷贝给该变量；简单说只有在函数结束时，此变量才会改变；而 ref 类型时，数据任何时间都可能在改变。

74. 不同的类之间传递信息可以通过mailbox，同一种类不同的对象可以通过静态变量来传递，这个静态变量可以让这个类的所有实例共享，也可以通过 mailbox 来传递

75. systerverilog 提供一种特殊的数据类型 type 它是传递类型的变量，如 type T, T=real;T=int, T=BasrTr(BasrTr 为一个类)等操作，用在参数类比较多

76. package……endpackage SystemVerilog 包提供了额外的机制在多个 SystemVerilog 模块、接口和程序间共享参数、数据、类型、任务、函数、序列以及特性的声明。包是显式命名的作用域，它出现在源文本的最外层（与顶层模块和原语处于同一层次）。类型、变量、任务、函数以及特性都可以在一个包中声明。

这些声明可以在模块、宏模块、接口、程序和其它包中通过导入或完整解析的名字来引用。

位于包内的项目通常是类型定义、任务、以及函数。位于包内的项目不能具有层次化的引用。包还可能包含参数、变量和线网。对于那些不方便通过层次向下传递的全局项目，这种方式可能有用。与在一个编译单元或模块内声明的变量相同，位于包内的变量声明赋值应该出现在任何 initial、always、always_comb、always_latch 或 always_ff 块被启动之前。

77. 随机函数 $urandom_range(min,max); 用在做随机往系统中插入异常，temp =$urandom_range();# $ temp;

78. event 用法：一般是采用 event e;定义事件，-> e;触发事件；@e;等待事件，其中@是等待边沿的触发，而 wait 是等待电平触发，可以采用 wait(e.triggered);

79. 采用@等待的方式，当触发已结束，会引起阻塞，如果用wait 则不会引起阻塞，但要注意 wait 会在单个事件触发器上反复执行.

80. 继承类可以通过与父类使用相同的约束名来改变父类的约束

81. 对象的复制就是采用 copy 函数，比较下面的两个函数

class 中的 copy 函数解释：virtual function transaction copy();

virtual function 表示虚函数；transaction 表示函数返回的数据类型（相当于 int,bit[7:0]之类的数据类型），copy 是函数名(变量)，但因为返回的是一个类，所以在使用时，都需要对 copy 做实例化，即 copy = new();的动作

82.

```
class Transaction
  rand bit[31:0]src,dst;
  virtual function Transaction copy()
      copy=new();
      copy.src=src;
      copy.dst=dst;
  endfuncion
endclass
class BadTr extends Transaction
  rand bit  bad_crc;
  virtual function Transaction copy()
      BadTr bad;   //此处是不是有递归的感觉！！！？？？？
      bad=new();
      bad.src=src;
      bad.dst=dst;
      bad.bad_crc=bad_crc
endfuncion
endclass:BadTr
class Transaction
  rand bit[31:0]src,dst;
  virtual function void copy_data(input Transaction tr);
    copy.src=src;
```

```
  copy.dst=dst;
endfunction
virtual function Transaction copy();
  copy=new();
  copy_data(copy);
endfunction
endclasss
class BadTr extends Transaction
rand bit Bad_crc;
virtual function void copy_data(input Transaction tr);
  BadTr bad;
  super.copy_data(tr);
  $cast(bad,tr);//基类句柄类型转换成扩展类
  bad.bad_crd=crc;
endfunction
virtual function Transaction copy();
  BadTr bad;
  bad=new();
  copy_data(bad);
endfunction
Endclasss
```

83. 扩展类的虚函数必须跟基类的函数相匹配，包括所有的参数和返回类型.

84. $get_coverage 可以得到所有覆盖组的总覆盖率,它返回一个 0~100 的实数,而 get_coverage()和 get_inst_coverage(),可以缩小测量范围

85. CoverGroup::get_coverage()或 cgInst.get_coverage(),

86. get_inst_coverage()返回一个特定覆盖组实例的覆盖率,如:cgInst.get_inst_coverage(),

87. 如果希望得到单个实例的覆盖率,那么需要指定 option.per_instance=1;

88. 变量没在always 敏感列表中，及时变量有变化也不会传输到目的变量，所以可以使用 always(*);来处理

89. class trans;…… endclass trans tr; tr=new(); tr 为句柄，new（）之后产生了一个对象（object）,mailbox 只能传递句柄而非对象，简单点说，对象就是一片存储空间，句柄就是给这片存储空间的名字；

深入分析：tr 这个名字是放在一个存储器中，这个存储器会指向对象这片存储空间，当有 tr=tr1,这种操作时，相当于把 tr1存储器的内容给了 tr，那么 tr 此时便指向了 tr1所指向的存储空间，访问 tr 时，则相当于访问 tr1 的存储空间,但是 tr 和 tr1 是由要求的。

90. 对于静态类属性，无需产生一个该类型的对象就可以直接使用，可以定义为静态方法或静态变量，使用 A::FUNC B::TASK C::variable 方式调用，A,B,C 为一个类，FUNC,TASK 为类里面的静态方法,variable 为类里面的静态变量。

静态类可以理解为在编译时就开辟了存储空间给这个类使用，空间固定在某个存储空间范围；

动态类可以理解为在编译时只是计算出该类需要的存储空间大小，当 new 时，再给这个类开辟空间在某个存储空间范围；

91. 时钟控制块将时序和同步细节与一个测试平台中的结构、功能和过程元素分割开来。因此，采样和驱动时钟控制块信号的时序成为隐式的并且相对于时钟控制块的时钟。这使得我们能够以一个非常简洁的方式编写一组关键的操作，而无需显式地使用时钟或指定时序。这些操作是：

同步的事件

输入采样

同步的驱动

92. clocking bus @(posedge clock1);

```
default input #10ns output #2ns;
input data, ready, enable = top.mem1.enable;
output negedge ack;
input #1step addr;
endclocking
```

在上面的例子中，第一行代码声明了一个称为bus 的时钟控制块，它以信号clock1 的上升沿进行时钟控制。第二行代码通过 default指出：时钟控制块中的所有信号都应该使用一个 10ns 的输入时滞和一个 2ns 的输出时滞。下一行代码为时钟控制块加入了三个输入信号：data、ready 和 enable；最后一个信号指向层次化的信号 top.mem1.enable。第四行代码为时钟控制块加入了一个 ack 信号并过载了缺省的输出时滞从而使 ack 信号在时钟的下降沿被驱动。最后一行代码加入了addr 信号并过载了缺省的输入时滞从而使 addr 在时钟上升沿之前的一个步值时被采样。

除非另有说明，缺省的输入时滞是 1step，缺省的输出时滞是 0。一个步值是一个特殊的时间单位，它的值在 18.10 节中定义。一个 1step 的输入时滞使得输入信号在时钟信号之前的时间步值中采样它们的稳定值（也就是在前一个 Postponed 区域）。与其它代表物理单位的时间单位不同，一个 step 不能被用来设置或修改时间精度或时间单位。

93. 在使用 class 包含功能覆盖率时，注意 class 是无 sample()内建函数，而是 covergroup 有内建 sample 函数；所以在使用时应该是：类例化名.covergroup 名.sample()

```
class transac;
  int x;
  int y;
  covergroup tr_cov;
     coverpoint x;
     coverpoint y {
     bins min = {0};
```

```
    bins max = {10};
    bins mid = {[1,9]};
  }
endgroup
function new();
  tr_cov = new();
endfunction
Endclass
```

94. 在实例化时，当有 inout 时，尽量采$tran()来连接，

```
initial begin
    $deposite(wire var,data) ;
end
```

$deposite 给 wire 赋初始值

95. 对于 task/function 的静态类型变量，每次调用，它都会保留前一次调用的数据，如果设置一个这样的变量，可以知道这个 task/function 调用了多少次

96. 在 program 中，当所有的进程结束后，自动会 finish 退出仿真；而在 module 中必须用 finish 结束仿真；

97. 对于 RTL 中利用真实芯片的随机值（0,或 1），但仿真如果对此信号没有赋初始值，那么仿真将会是 X 态传递；所以在仿真时需要利用 $deposite(wire var,data)赋初始值

98. force wire = 1'b1;在 force 语句的右边只能为固定值，不能为变量，但如果想随着仿真进度变化怎么办？可以设置一个中间wire信号，然后信号 assign wire = reg;,reg 再在测例里面变化就可以

99. 数据处理：有符号数转为无符号数(即取绝对值的意思)：unsigned16[15:0] = signed16[15:0]+16'h8000; 注意有符号的负数为取反加 1,正数为原数

100. $finish(n);

系统任务$finish 的作用是退出仿真器，返回主操作系统，也就是结束仿真过程。$finish 可以带承诺书，根据参数的值输出不同的特征信息。如果不带参数，默认其值为 1；

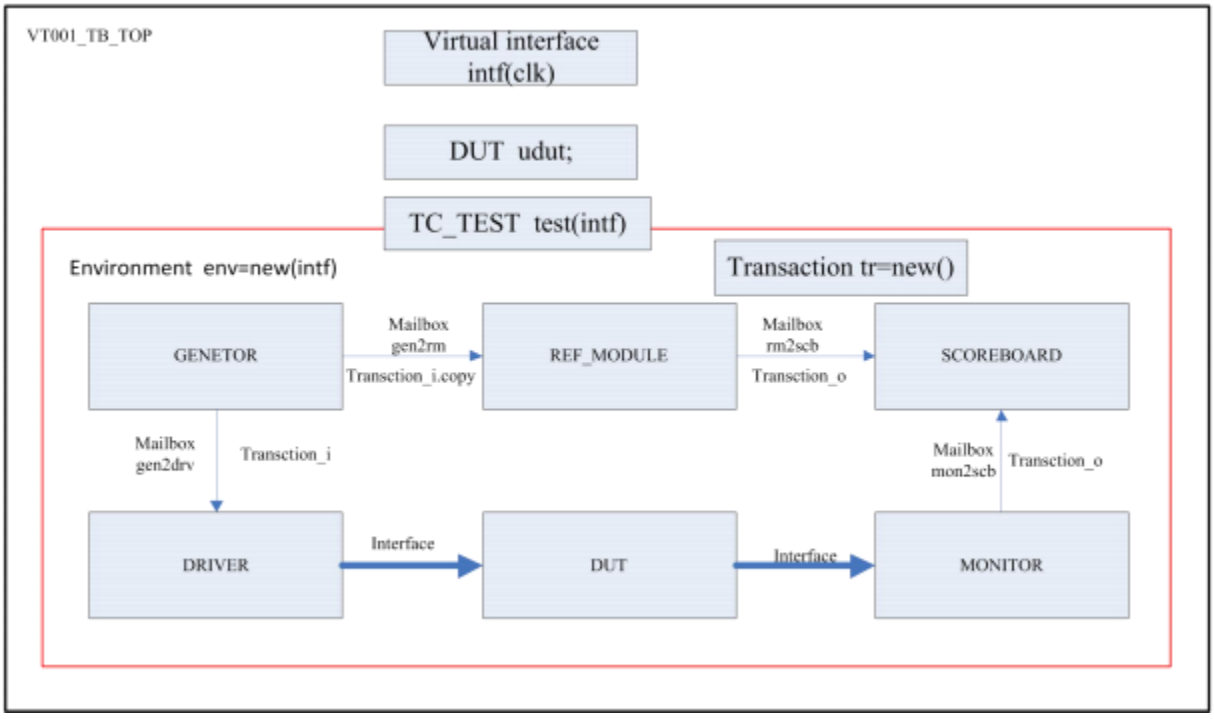
0 不输出任何信息。

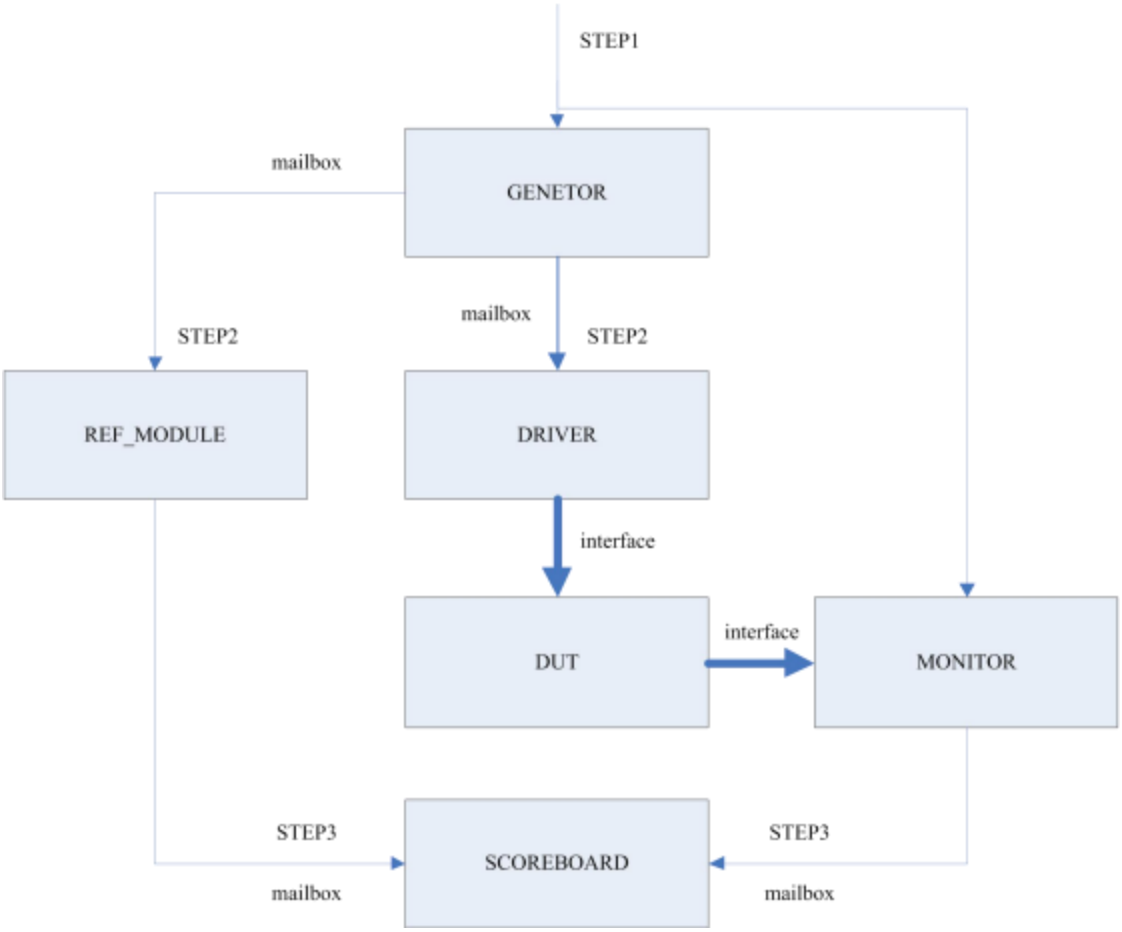
1 输出当前的仿真时间和位置；

2 输出当前的仿真时间、位置和仿真过程中所用memory和 CPU 的时间统计。

101.

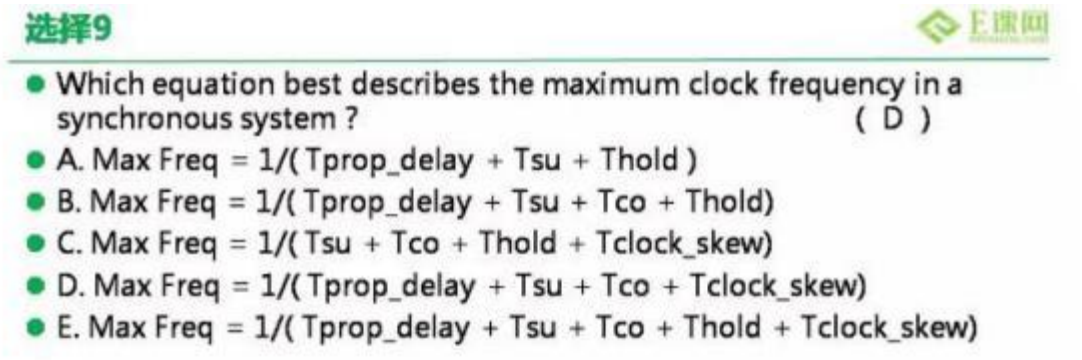
102. SV 标准架构





- 题目：What types list below could be used in packed array? （ A ）
- A. Bit B. int C. logic D. reg
- 解析：合并数组，既可以当作数组，也可以当作单独的数据。例如，有一个32比特的寄存器，有时候希望把它看成4个8比特的数据，有时候则希望把它看成单个的无符号数据。

- 如：bit [3:0] [7:0] barray [3];
  barray[0] = 32'h1234_5678;
  barray[0][3] = 8'h32;
  barray[0][1][6] = 1'b0;



？？ C

Interface 与 virtual interface 的区别:

interface 是静态的，定义后即存在，与原来的 verilog 中的 module 类似。“无论你用或不用，我就在那里，静静等待~~”

而 virtual interface，virutal~意同名字，虚的，就不是真实的，也就是说它根本不是接口，等同于 c 语言中的指针，“我不能告诉你你需要的东西是什么，但是我可以告诉你去哪里找~~”

定义之初，它是空的，即 null,你必须告诉我，我应该指向谁，否则，我一无是处~

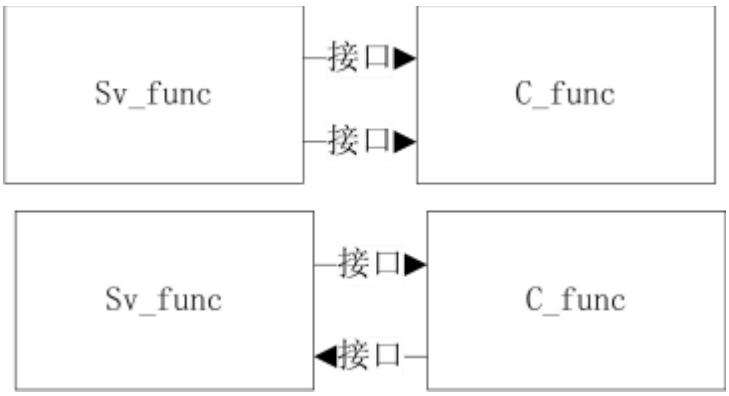
Virtal interface 在 uvm 的 class 经常使用到.

V 中使用 C 模型之 dpi 接口介绍

摘要：现在业界流行的验证语言是 UVM，或者是 SV，但是有些已经熟悉 c 的同事，喜欢直接使用c 语言来写一些函数、参考模型。SV 提供了一个 SV 与 C 的接口，即 DPI。 **什么是 DPI**

DPI，Direct ProgrammingInterface 直接编成接口。它能更加简单的连接 C、C++或者其它非 verilog 编程语言。使用DPI 的准备工作

DPI 简单来说就是把 sv 与其它不同的编程语言连接起来，或者说是交互。这种交互，根据实际的应用，可以单向的，也可以是双向的。



所以需要 sv 函数，还需要 c 函数，另外需要注意的是，两个函数的接口要一致。

另外一个就是导入声明：

Import “DPI-C” function return_type func(type ver);

Sv 与 c 语言之间数据类型的映射

Sv 的数据类型 byte shortint int bit

C 输入的数据类型 char short int int svBit or

C 输出的数据类型char* short int* int* svBit* or

具体可参考 system verilog 这本书

调用 dpi 的步骤

sv 中定义一个结构体

```
typedef struct {
  byte   a;
  int    b;
} RGB_T
```

加入一个导入的声明

```
Import   “DPI-C” function int dpi_function (input RGB_T
  pstruct);
```

定义一个 class 类， 类成员就是我们需要的随机变量（rtl 需要的一些配置信息）；

```
Class cabc_bench;
```

```
  Rand byte  a;
  Rand int   b;
End class
```

上面定义的类 class 中，产生的随机变量要传到共享的结构体中，所以要定义一个函数。

```
Function RTB_T   pack();
  Pack.a =   a;
  Pack.b =   b;
Endfunction:pack
```

c 语言中定义一个共享结构

```
typedef struct {
  unsigned char a;
  int         b;
} *p_rgb
```

定义一个导入的声明函数 ，把共享的结构中的成员传递给 c 中的变量

```
Int   dpi_function(p_rgb rgb)
{
  Cabc_para.a       =       rgb->a  ;
  Cabc_para.b       =       rgb->b  ;
}
```

c 语言中需要删除的部分

由于 c 中本身会产生一些固定的值供函数使用，现在使用了 sv 产生的变量，所以 c 中之前写的产生配置信息的函数要删除。

如何把 SV 中的数组传给 C

数组在 SV 和 C 中的使用方法。

SV:

bit[7:0] array[10],表示具有 10 个元素的一维数组;bit[7:0] Tarray[10][5] 表示 10 行 5 列的二维数组

使用方法: array[i] ; Tarray[i][j]

C:

int array[10] 表示具有 10 个元素的一维数组;int Tarray[4][3] 表示 4 行 3 列的二维数组使用方法: 1.直接使用 array[i]; Tarray[i][j]

2.一维数组使用指针引用: int *p; 定义一个整型指针变量 p=&array[0]; 其 含义是把array 的首地址给 p,在引用数组的内容时,可以用*(p+1), *(p+10),读取 array[1], array[10],的内容,

3.二维数组是用指针引用：int *p; 定义一个整型指针变量 p=&Tarray[0]; 其含义是把 Tarray 的首地址给 p,与一维数组一样，因为P 保存的都是数组的地址而已，无论是一维还是二维数组，都是存放在一片连续的存储空间内，都会有一个首地址存在，只要知道首地址就可以寻找到数组的各个单元,如 Tarray 表示数组首地址，* Tarray 表示第 0 行第 0 列地址，Tarray +1，Tarray[1]表示第 1 行首地址；*（Tarray+1）表示第 1 行第 0 列元素的地址，*（Tarray+1）+2 表示第 1 行第 2 列元素的地址。在引用二位数组时，可以使用如下方法：Tarray [2][3] 表示第二行第三列的值，*(Tarray [2]+3) 表示第二行第三列的值,*(*(Tarray +2)+3) 表示第二行第三列的值,

概括如下：

2000	2002	2004	2006
1	3	5	7
2008	2010	2012	2014
9	11	13	15
2016	2018	2020	2022
17	19	21	23

指针描述如下

表现形式	含义	地址
a	二维数组名，数组首地址	2000

A[0] ,*(a+0), *a	第 0 行第0 列元素地址	2000
A+1	第一行首地址	2008
A[1], *(a+1)	第一行第0 列元素地址	2008
A[1]+2, *(a+1)+2, &[1][2]	第一行第 2 列元素地址	2012
(a[1]+2),(*(a+1)+2),a[1][2]	第一行第 2 列元素的值	元素值为13

SV 把一维数组传递给 C

根据上面 C 的使用方法可以知道,C 引用一个数组,只要知道数组的首地址即可;

当 c 函数的形式参数是数组，如 main(int *p); 因为在在 c 内部引用数组内容时，是使用上述方法，而 sv 中有一个开放数组的类型变量给 C 使用，而不失直接的指针变量，所以此时需要一个数组到指针的变换；变换方法如下：

C 函数形参变为开发数组 main(const svOpenArrayHandle array),函数内部定义 int *p; 再把首地址传给 P 就 OK 了，p = (int *) svGetArrayPtr(array),这样就把数组的首地址给指针变量 P 了；C 其它的方面不需要改动；而对于 SV 方，则只需要定义平常的数组类型变量即可，调用方法如下：

```
Import ” DPI-C ” function void main(input bit[7:0] Carray[]);  //sv
```

中 import 时 Carray[]表示一维动态数组，与 C 里面的 const svOpenArrayHandle array 数据类型对应

```
bit[7:0] array[10]; /sv 中定义的一维数组
foreach(array[i])  array[i]=I;
main(array);  //调用 C 函数
```

在 C 中需要把函数改成如下模式：

```
Main(const svOpenArrayHandle array);
```

SV 把二维数组传递给 C

根据上面 C 对二维数组的引用可知，如果 C 采用了指针变量，那么可以像一维数组那样变换（如果 C 直接使用数组，则不能使用上述方法变换，一维数组使用指针方便），但当C 对二维数组的引用是通过数组来引用，那么则需要使用SV 提供的专用函数来改 C 的内容，而 C 采用一个指针变量来使用二维数组则不如直接使用数组方便，如上面Tarray [2][3] 表示第二行第三列的值，如果使用 p =&Tarray;

则*(p+2*3) 才表示第二行第三列的值；所以二维数组可能直接使用数组会更方便，

在 SV 中有专门函数用于在 C 中对数组的操作 ：

(int*)svGetArrElemPtr2(Tarray,j,i) 表示第 i 行第 j 列元素；

```
Import ” DPI-C ” function void main(input bit[7:0] Carray[][]);  //sv
```

中 import 时 Carray[][]表示二维动态数组，与 C 里面的 const svOpenArrayHandle array 数据类型对应

```
  bit[7:0] array[10][5];     //sv 中定义的二维数组
  for(int  i=0;i<5;i++) begin
      for(int  j=0;j<10;j++) begin
          array[j][i] =  i+j;
      end
  end
  main(array);    //调用 C  函数
```

在 C 中需要把函数改成如下模式：

```
Main(const svOpenArrayHandle array);
```

对于上面 C 所述，二维数组可以使用 *Tarray 来做形参（Tarray 为一个二维数组），而 SV 中没有与之对应的数据类型；而通常 C 会通过一个结构体把所有的变量组合在里面作为一个参数传递，如下：（C 代码）

```
Typedef  c_model_struct
```

{

Int width

Int height;

Int * pixel_R ;//一个二维数组

Int * pixel_G;//一个二维数组

Int * pixel_B ;//一个二维数组

} c_model_struct

不改动 C 代码，C 函数封装一层传递二维数组

Void main(*c_model_struct c_model_struct_ptr);

按照第一节所述，我们在 SV 无法找到与 int *pixel_R 这样的类型变量，sv 中定义 bi t[7:0] pixel_R[10][3]不能与之对应；

从*pixel_R 定义的意义来看，其表示批 pixel_R 的第 0 行第 0 列地址，但 sv 中并没有这样的函数可以取得二维数组的地地址；故行不通，必须改变 C 函数的写法，如上面所述采用SV 提供给 C 的函数；

在 SV 中似乎不能识别上述 C 中的二维数组（Int * pixel_R ;//一个二维数组）！

可以把上面的结构体改为如下：

Typedef c_model_struct

{

Int width

Int height;

Int pixel_R [1920][1080]; //一个二维数组

Int pixel_G[1920][1080];//一个二维数组

Int pixel_B[1920][1080] ;//一个二维数组

} c_model_struct

在 C 再封装一层，通过给二维数组赋值，再传递给原 C 模型；如下操作

c_model_struct c_model_struct_ptr;

c_model_struct *c_model_struct_ptrs;

```
For(j=0;j<1920;j++)
 {
    For(i=0;i<1080;i++)
     {
    c_model_struct_pt. pixel_R[j][i] =
(int*)svGetArrElemPtr2(input_pixel_R,j,i)
  c_model_struct_pt. pixel_G[j][i] =
(int*)svGetArrElemPtr2(input_pixel_G,j,i)
    c_model_struct_pt. pixel_B[j][i] =
(int*)svGetArrElemPtr2(input_pixel_B,j,i)
    }
 }
```

注意，这样相当于开辟了 2 片空间，可能内存不够。。。。。。。

具体函数参考《systemverilog 验证》，P347 页。

可能有其它方法转化， 目前我还没找到方法！大家可以尝试下。

nLint 篇

如何启动 nLint:

1. nLint -gui xx0.v xx1.v xx2.v … & 然后在主界面 run->Lint 进行分析
2. nLint -gui spi_sys.v spi_quad.v
   spi_ctrl.v ../generic/sync_level.v &
3. nLint -gui -f xxxxx & (xxx,为包含所有 rtl 及 define 等)

nLint 检查内容

在 nLint.rs 文件定义,改文件定义了所有会检查到的规则,及当违规时输出严重等级.

如何改变规则

在按照目录下/nlint_J-2014.12/etc/有一个 nLint.rs 文件,这个文件用来配置各个规则;当这个规格的严重等级与自己要求不符合时,可以在工作目录下写一个nLint.rs文件,来修改一些参数.修改方法见下面

RS 文件格式

a.File Version Section

[RuleVersion]

Version = 5004.0

b.Groups Section

[Groups]

1="nLint"

2=RMM

3="IPQ Design Guidelines"

c.Default Checked Group Section

[Default]

Default = "nLint"

d.Group Include Section

[nLint.include]

1 = "Simulation" Enable TRUE

2 = "Synthesis" Enable FALSE

3 = "DFT" Enable

4 = "Design Style" Enable

5 = "Language Construct" Enable

6 = "HDL Translation" Enable

7 = "Coding Style" Disable

8 = "Naming Convention" Disable

9 = "VITAL Compliant" Disable

e.Rules in Sub-group

[Simulation]

22005 = Enable Enable TRUE TRUE

22011 = Enable Enable TRUE FALSE

22013 = Enable Enable FALSE FALSE

...

f.Parameter for Special Rule

[nLint.21013]

vhdl_val="PREFIX","clk_"

vlog_val="PREFIX","clk_"

vlog_severity = Level2

vlog_changeable = TRUE

vlog_TranslateOff = OFF

vhdl_severity = Level2

vhdl_changeable = TRUE

vhdl_TranslateOff = OFF

index_name = "4-3-2-1"

vlog_description = "clock signal prefix"

vhdl description = "clock signal prefix"

举例:

@nLint rc file Version 1.0

[RuleVersion]

Version = 20130.03

[Groups]

1 = "nLint"

2 = "RMM"

3 = "IPQ Design Guidelines"

[Default]

```
Default = nLint
[nLint.21001]
vhdl_val="CASE_LOWER"
vlog_val="CASE_LOWER"
[nLint.21003]
vhdl_val="CASE_LOWER"
vlog_val="CASE_LOWER"
[nLint.21005]
vhdl_val="CASE_LOWER"
vlog_val="CASE_LOWER"
......
[nLint.include]
1 = "Simulation" Enable
2 = "Synthesis" Enable
3 = "DFT" Enable
4 = "ERC" Enable
5 = "Design Style" Enable
6 = "Language Construct" Enable
7 = "HDL Translation" Enable
8 = "Coding Style" Disable
9 = "Naming Convention" Disable
10 = "VITAL Compliant" Disable
11 = "Clock" Enable
12 = "Block Interconnect" Enable
13 = "CTS" Disable

[nLint.Simulation]
22005 =  Enable Enable
22006 =  Enable NONE
```

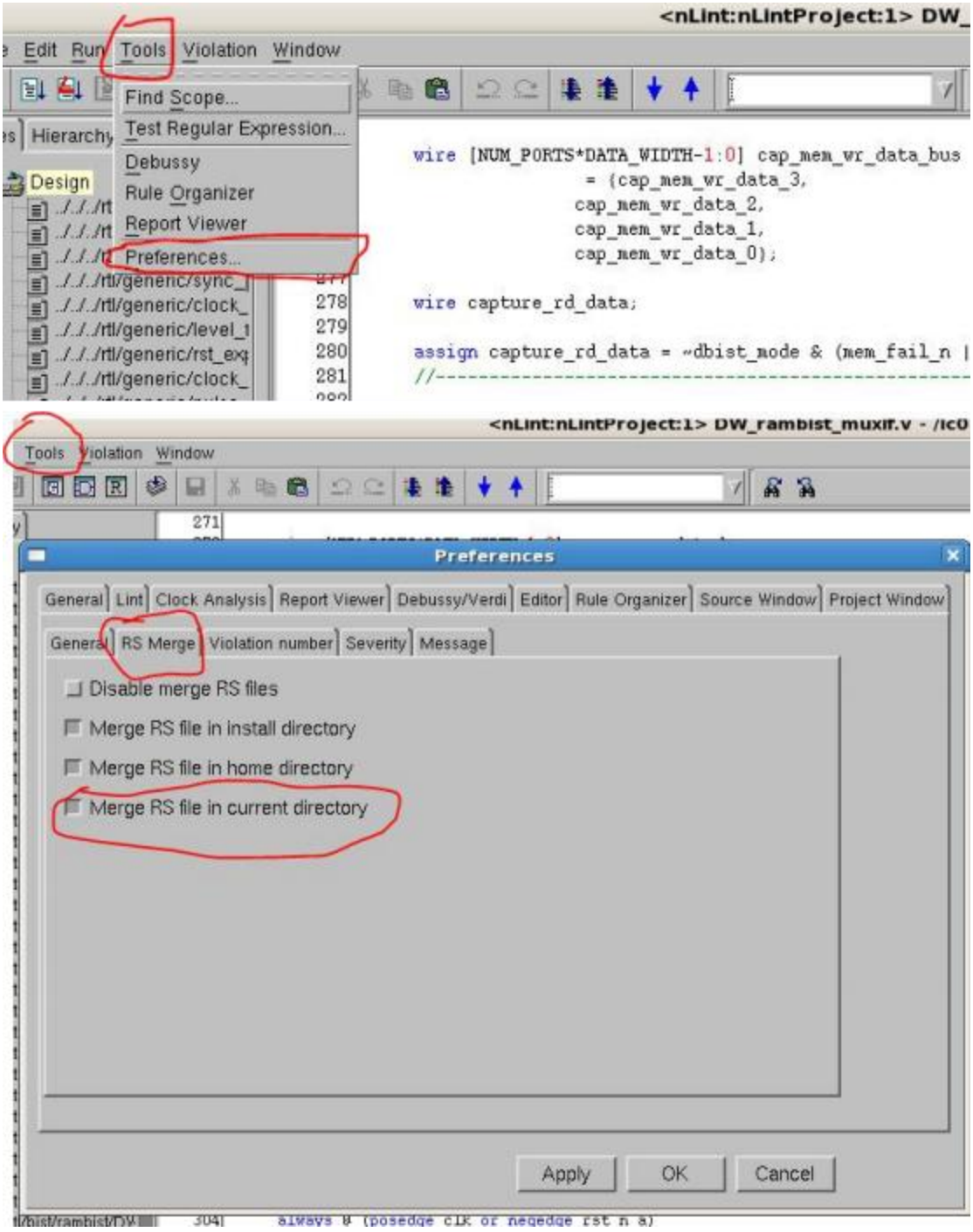
```
    22009 =  Enable NONE
    ......
    [nLint.Simulation]
    22005 =  Enable Enable
    22006 =  Enable NONE
    22009 =  Enable NONE
    22011 =  Enable Enable
    ......
    [nLint.Block Interconnect]
    27355 =  Enable NONE
    27356 =  Disable NONE

    [nLint.CTS]
    25019 =  Disable NONE
    25021 =  Disable NONE

    [RMM.21001]
    vhdl_val="CASE_LOWER"
    vlog_val="CASE_LOWER"
    vlog_indexname = "5.2.1-G1-1"
    vhdl_indexname = "5.2.1-G1-1"
    vhdl_description = "Use lowercase letters for all signal
names"
    vlog_description = "Use lowercase letters for all signal
names"
    [RMM.21003]
    vhdl_val="CASE_LOWER"
    vlog_val="CASE_LOWER"
    ......
```

```
[IPQ Design Guidelines.21001]
vhdl_val="CASE_LOWER"
vhdl_severity=Level2
vlog_val="CASE_LOWER"
vlog_severity=Level2
vlog_indexname = "NC.9-1"
vhdl_indexname = "NC.9-1"
vhdl_description = "Signals names are lowercase letters"
vlog_description = "Signals names are lowercase letters"
[IPQ Design Guidelines.21005]
vhdl_val="CASE_LOWER"
vhdl_severity=Level2
vlog_val="CASE_LOWER"
vlog_severity=Level2
vlog_indexname = "NC.9-5"
vhdl_indexname = "NC.9-5"
vhdl_description = "Port names are lowercase letters"
vlog_description = "Port names are lowercase letters"
[IPQ Design Guidelines.21007]
vhdl_val="32"
vhdl_severity=Level2
vlog_val="32"
vlog_severity=Level2
……
```

如何合并 rs 文件

当需要修改nLint.rs文件的一些参数时,我们不能直接修改安装目录下的nLint.rs文件,我们可以在工作目录下添加一个nLint.rs 文件,环境按照下面要求配置.

Tools ->Preferences ->RS merger



修改输出严重等级范例

如当需要修改编码为 22211 和 22213 的输出严重等级时,可以按照下面方法修改(固定连接为 0 和 1,本来报 error,修改报 warning);23003 为产生 latch,本来报 warning,修改报 error.

@nLint rc file Version 1.0

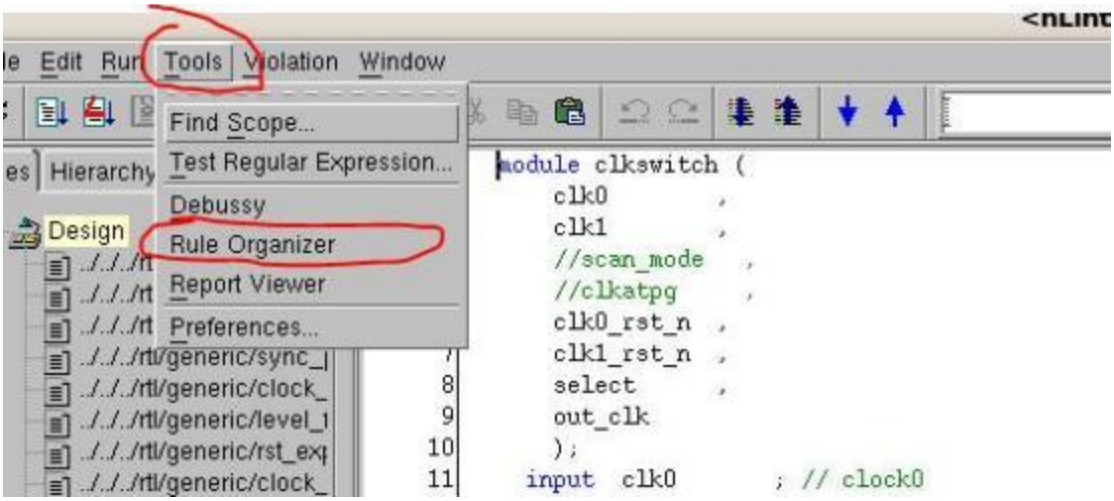
```
[RuleVersion]
Version = 20130.03

[Groups]
1 = "nLint"
2 = "RMM"
3 = "IPQ Design Guidelines"

[Default]
Default = nLint

[nLint.22211]
vhdl_val = "TRUE"
vlog_val = "TRUE"
vhdl_severity = Level2
vlog_severity = Level2
vlog_changeable = TRUE
[nLint.22213]
vhdl_val = "TRUE"
vlog_val = "TRUE"
vhdl_severity = Level2
vlog_severity = Level2
vlog_changeable = TRUE

[nLint.include]
2 = "Synthesis" Enable    //22211,22213 两个错误在 synthesis
部分
[nLint.Synthesis]
22211 =  Enable Enable
```

```
22213 =  Enable Enable
[nLint.23003]
vhdl_val = "TRUE"
vlog_val = "TRUE"
vhdl_severity = Level3
vlog_severity = Level3
vlog_changeable = TRUE

[nLint.include]
1 = "Synthesis" Enable
[nLint.Synthesis]

23003 =  Enable Enable
```

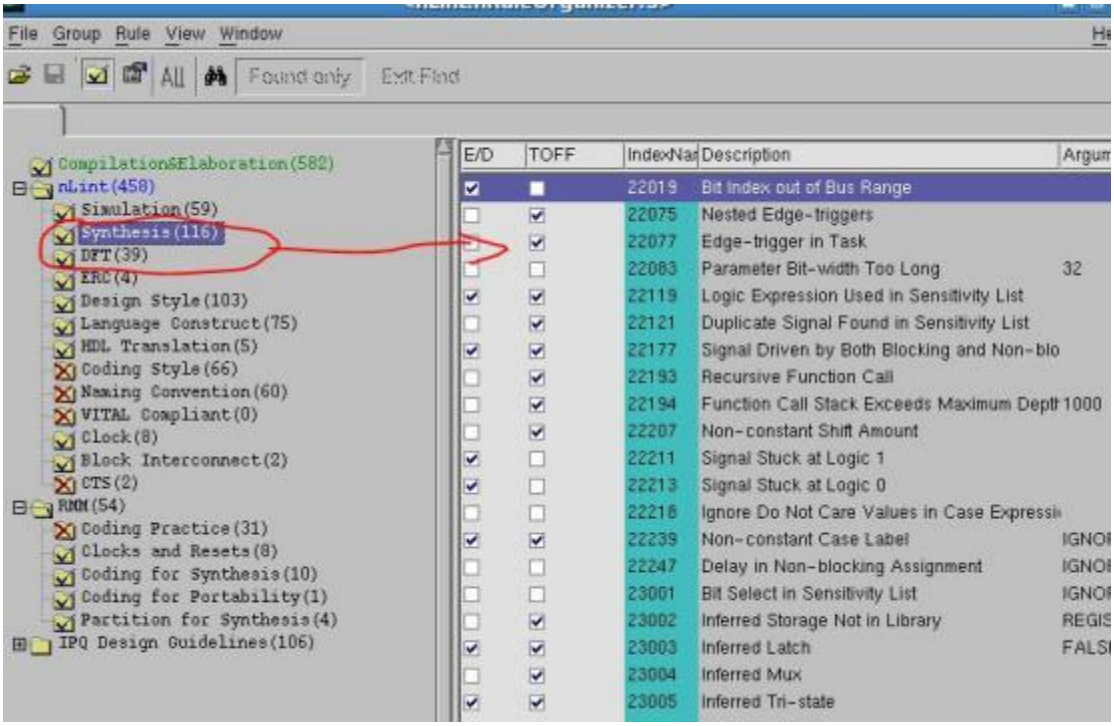
所有的规则编码需要参考相关文档.输出信息分为三个等级，Level 1 为 info，Level 2 为 warning，Level 3 为 error.

修改文件后需要重新启动工具生效!!

如何改变组织规则

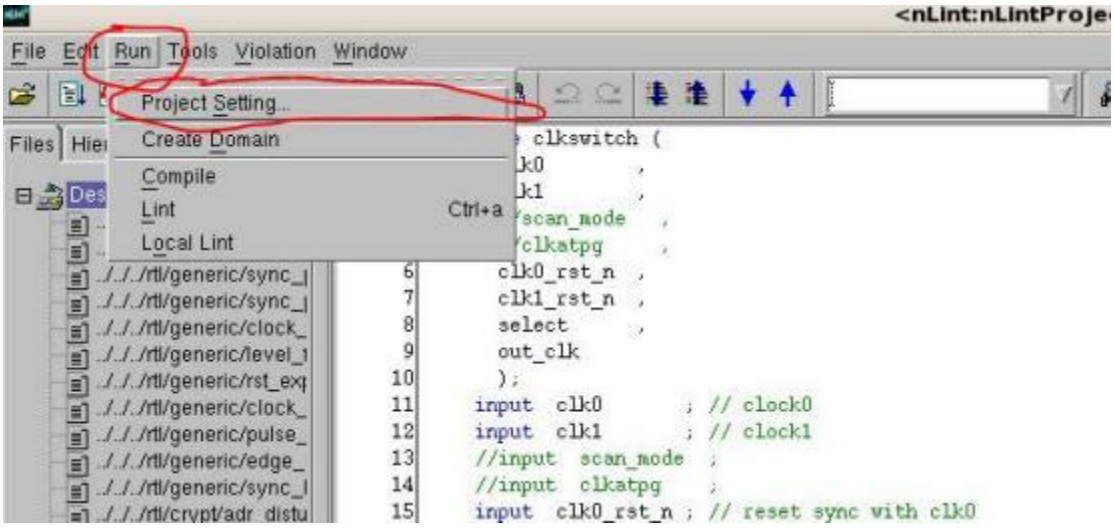


通过下面界面可以把各个规则分类.

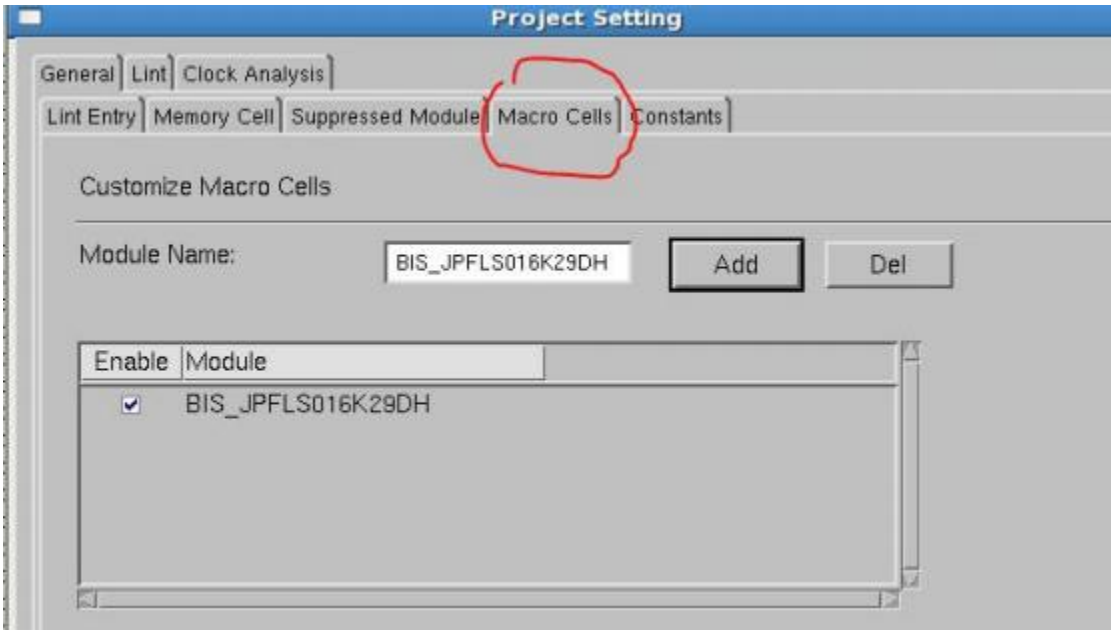


指定某些模块为blackbox

当制定某些模块为黑盒子时,则不会对black box 内部进行规则检查,如下设置:



通过 add 按钮添加 black box



上面这些配置最后会在一个nLint.ds文件中，内容如下

```
-bb DW_rambist
-bb xxx_ipio
-bb BIS_JPFLS016K29DH
-bb mc8051_core
-bb RASP256X9
-bb RASP1536X9
-bs ../../../rtl/bist/rambist/DW_rambist.v
-bs ../../../rtl/flash/BIS_JPFLS016K29DH.v
-bs ../../../rtl/core/mc8051_core.v
-bs ../../../rtl/top/xxx_ipio.v
-partition_top  //如果没有配置，则只会有最后这句。
```

spyglass 篇

1． 启动命令：

```
spyglass
spyglass  -project xxxx.prj
```

spyglass -gui&

xxxx.prj 内容范例

#!SPYGLASS_PROJECT_FILE

#!VERSION 3.0

This is a software generated project file. Manual edits to this file could be lost during the next save operation

Copyright Synopsys Inc.

Last Updated By: SpyGlass SpyGlass_vL-2016.06

Last Updated On Mon Dec 2 17:25:10 2019

##Data Import Section

read_file -type sourcelist cmd_files_ic

read_file -type sgdc xxx.sgdc

##Common Options Section

set_option top xxx

set_option projectwdir .

set_option language_mode mixed

set_option designread_enable_synthesis no

set_option designread_disable_flatten no

set_option enableSV yes

set_option disablev2k yes

set_option 87 no

set_option sdc2sgdc yes

set_option top xxx

set_option active_methodology $SPYGLASS_HOME/GuideWare/latest/block/rtl_handoff

set_option sdc2sgdcfile xxx_sdc2sgdc.sgdc

```
    set_option extractDomainInfo true
    set_option stop { saradc_10bit_5vp15 reg_1p5_5vp15 por_lvd_5vp15
pch_5vp15 sensoradc_5vp15 osc800k_trim_5vp15 crystal_32k16m_5vp15
pll_800kto48m_5vp15 pad_cp pad_vdd pad_vss RASP256X9 RASP1536X9
pfm_1110skh32kx32_v1a0 gpio_ana_pad_5vp15 pad_vpp pad_avdh pad_avs
por2_5vp15 }

    ##Goal Setup Section
    current_methodology
$SPYGLASS_HOME/GuideWare/latest/block/rtl_handoff
    #current_goal cdc/cdc_setup_check -top xxx
    set_parameter enable_generated_clocks yes
    set_parameter write_sdc yes
    set_parameter report_indirect_port_clock yes
    set_parameter report_detail Clock_check10,all
    set_parameter allow_combo_logic yes
    set_parameter debug_proc yes
    set_parameter enable_glitchfreecell_detection yes
    set_parameter show_sdc_progress yes
    set_parameter strict_sync_check no
    set_parameter use_inferred_clocks yes
    set_parameter show_all_sdc_violations yes
    #set_parameter gen_block_sgdc -default
    set_parameter allow_clock_on_output_port yes
    set_parameter clock_gate_cell "ICG_V  ICG"
    set_parameter enable_sync_cell "sync_level, Sync_Pulse"
    ############
```

Cdc 分析流程

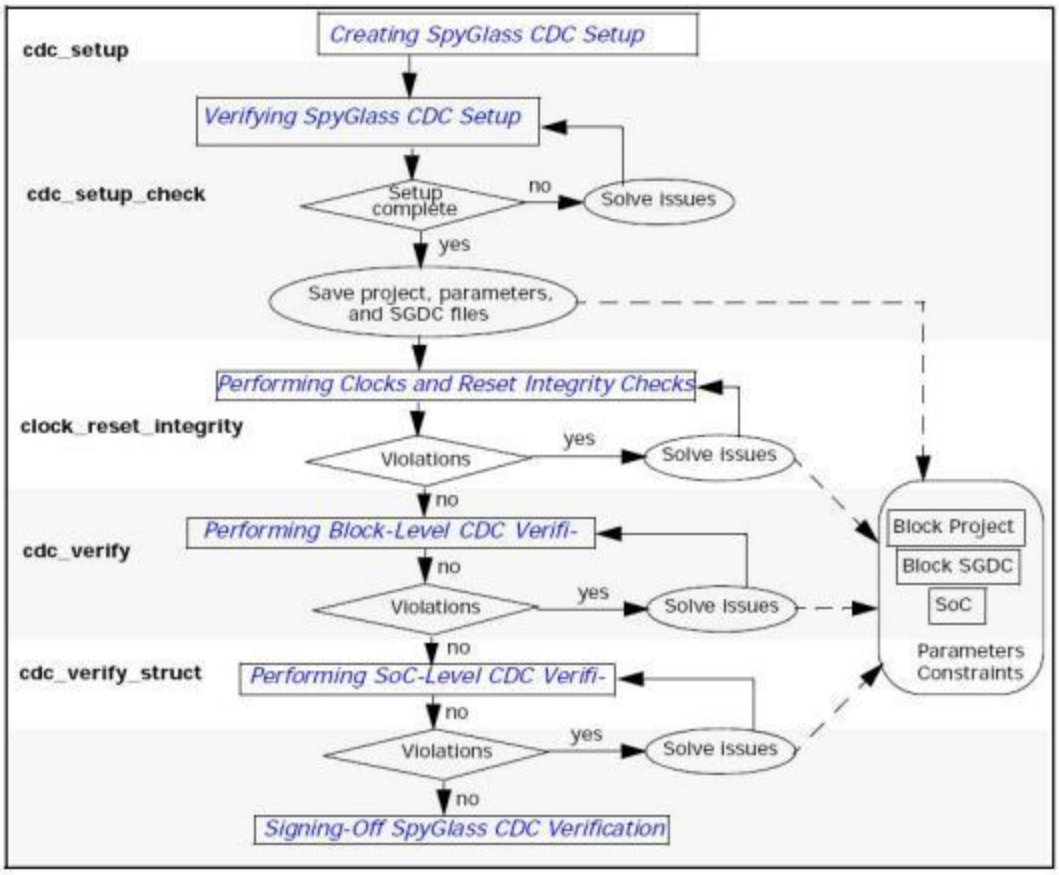


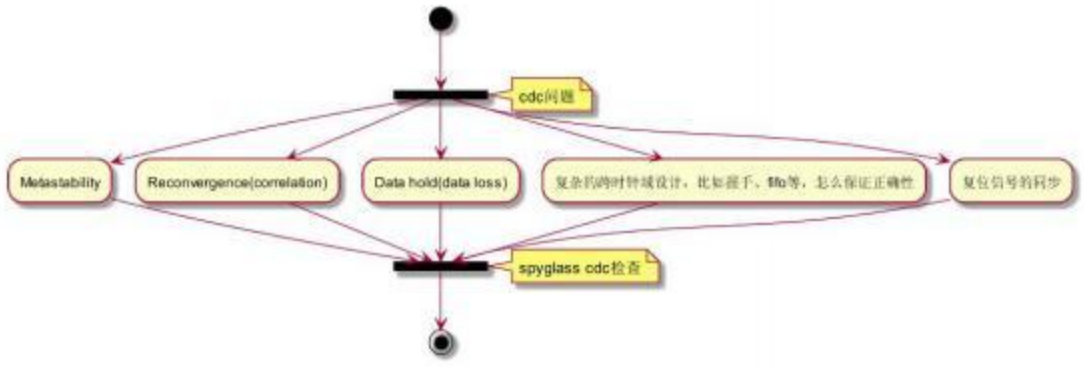
FIGURE 1. The SpyGlass CDC Methodology Flow

各个步骤目的

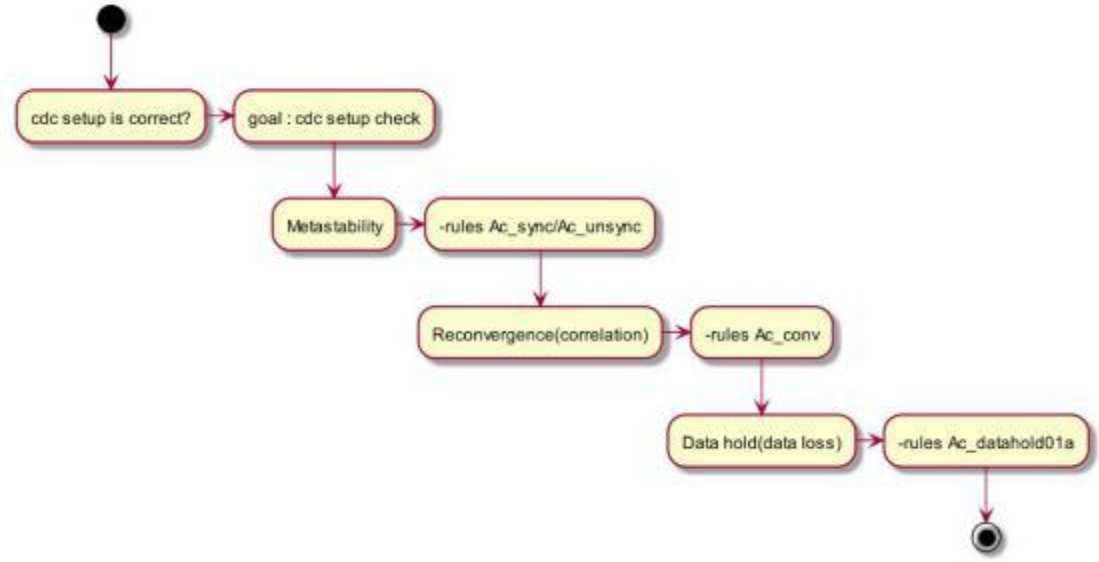
Stage	Summary	Goals
Creating SpyGlass CDC Setup	Specify constraints, parameters, and other design components required for accurate and complete SpyGlass CDC verification.	cdc_setup
Verifying SpyGlass CDC Setup	Check for the correctness and completeness of the setup.	cdc_setup_check
Performing Clocks and Reset Integrity Checks	Fix clock and reset integrity problems	clock_reset_integrity
Performing Block-Level CDC Verification	Fix block-level violations to make the block SpyGlass CDC clean. This stage involves the following tasks: • *Fixing Ac_sync_group Rule Violations* • *Fixing Violations Related to Convergence* • *Fixing Violations Related to Glitches* • *Fixing Violations for Data Hold Checks* • *Fixing Data Hold Issues in Synchronized Data Crossings* • *Fixing Violations Related to Data Correlation and Race Conditions* • *Fixing Violations Related to Reset Synchronization and Deassertion* • *Fixing Violations Related to FIFO Recognition and Verification*	cdc_verify
Performing SoC-Level CDC Verification	Verify the SoC using the verified blocks	cdc_verify_struct
Signing-Off SpyGlass CDC Verification	View reports and sign-off SpyGlass CDC Verification on the SoC	-

2． Spyglass 的作用

检查如下 5 个点



检查方法规则



实际上 spyglass有 ic 设计全套检测方案,其中主要的有:

” SpyGlass DFT Solution, which checks a design for testability issues atthe RT level.

„ SpyGlass Constraints Solution, which checks the suitability of the Synopsys Design Constraints files

„ SpyGlass Power Verify Solution, which checks the design for the structural, architectural, and system level issues in order to reduce power.

„ SpyGlass CDC Solution, which include rules to check clocks, resets, and clock-domain-crossings in the design

„ SpyGlass Power Estimate Solution, which include rules for power estimation and power reduction

„ SpyGlass TXV Solution, which is used to verify correctness of timing exceptions and/or check if critical paths can be ignored or relaxed during timing sign-off.

而且允许客户自定义检查规则.

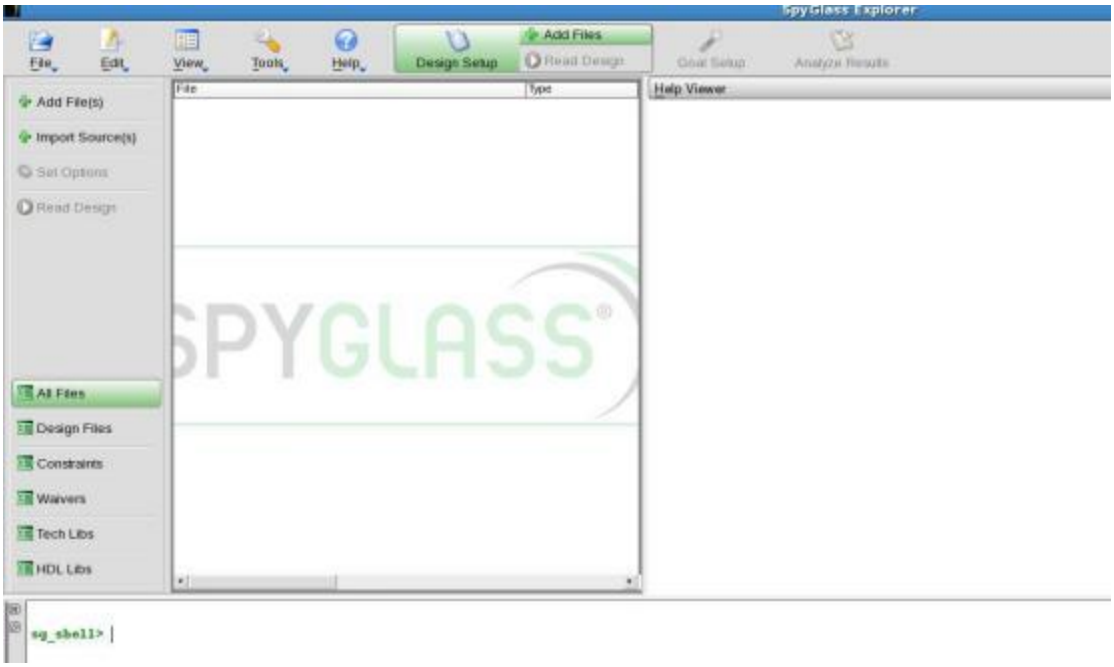
也能够像 nlint 一样改变规则等级---error,worning,info 等级

3． spyglass 工具的引入

spyglass 类似 pt，是静态验证工具。特点就是不需要编写 testbench，就可以验证。

Spyglass 使用步骤:

输入命令进入 gui 界面如下



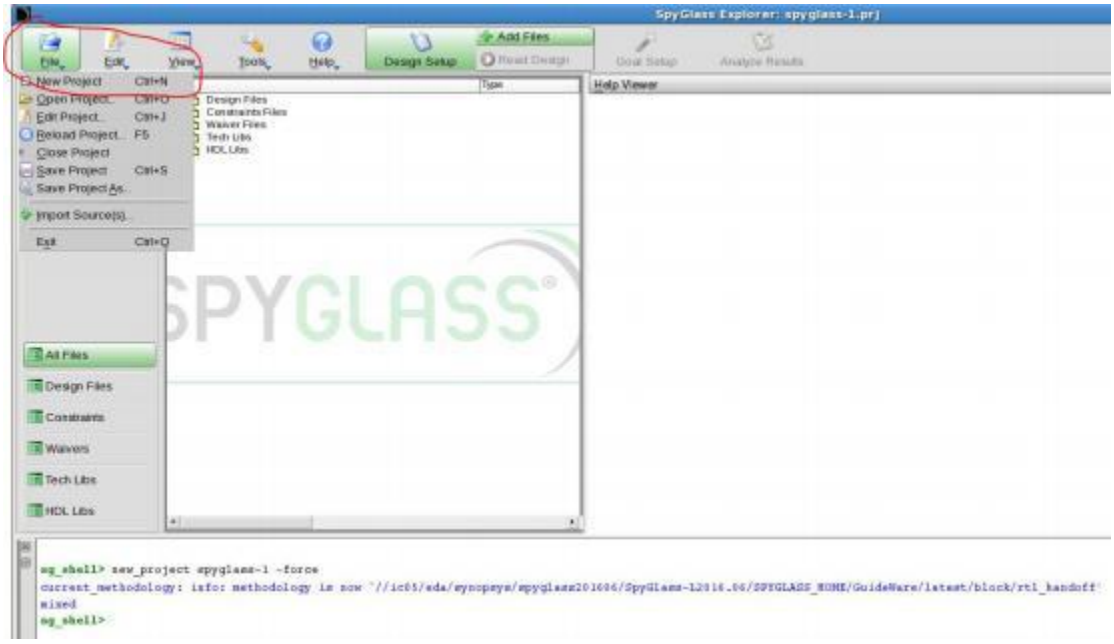
1. Adding Design Files----加入 rtl 和 sgdc 文件

2. Viewing and Changing Design Read Options

3. Running Design Read

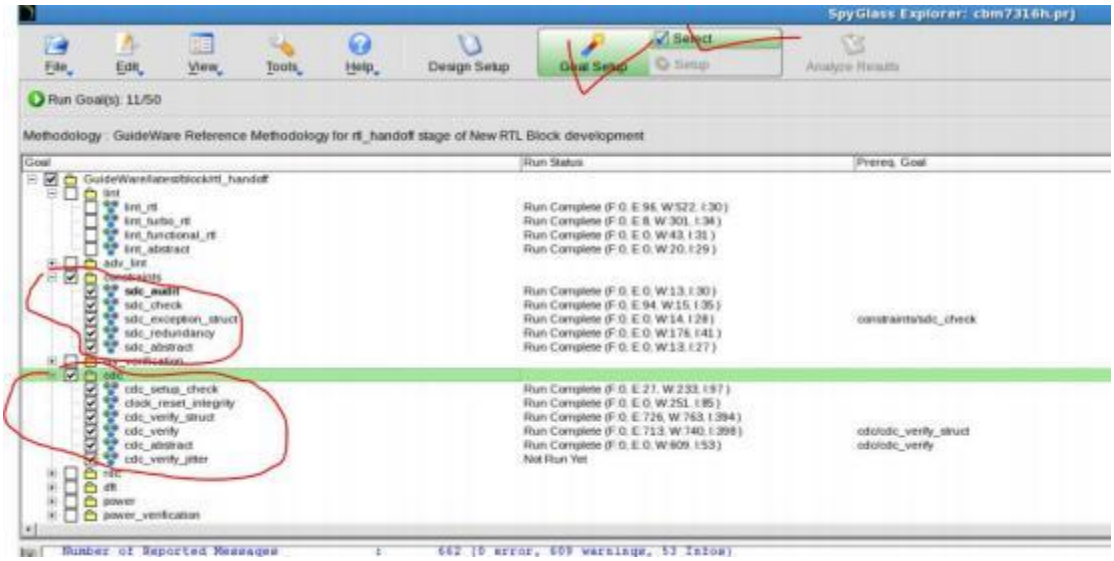
点击 file -> new_project

(对应 sg_shell>new_project spyglass-1 -force 命令)

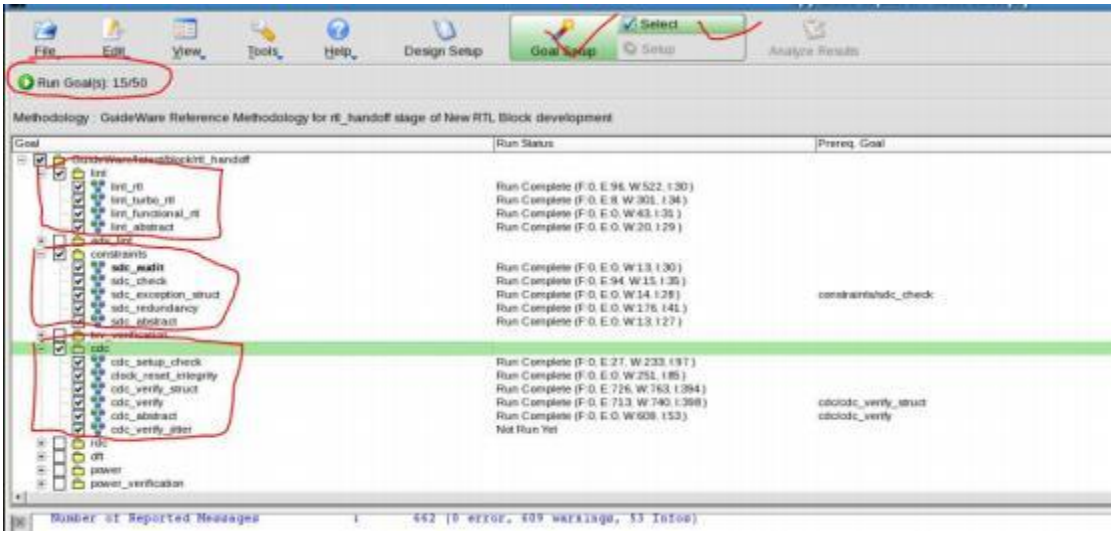


对项目进行 cdc 检查

需要运行 goal setup -> select ->选择运行 constarints & cdc



也可以多加一个 lint,这个可以检查rtl语法.



4. Cdc 使用流程

在做 cdc 检查之前,一般都会先分析 sgdc 约束文件

1. setup cdc -- 配置 cdc 各种命名及参数
2. fix clock and reset problem -- gsdc 定义各种时钟复位关系运行 clock_reset_integrity goal
3. cdc verification --- 执行 cdc 检查
   CDC verification means detecting CDC problems in a design. CDC verification means detecting CDC problems in a design
   1. Set the required parameters.
   2. Run goals, such as cdc_verify and cdc_verify_struct, to detect a large spectrum of CDC problems
4. debug cdc issues --- 定位问题

   主要的问题如下:

   The following issues cover a majority of important violations:

   - *Unsynchronized Crossings Issues*
   - *Convergence Issues*
   - *Reset Synchronization Issues*
   - *Glitch Issues*
   - *Signal Width Errors in Synchronized Control Crossings*
   - *Data Hold Issues in Synchronized Data Crossings*

   For all the other violations, search the SpyGlass CDC documentation for a violation by its rule name. For details, see *Rules in SpyGlass CDC*.

   跨时钟域没做同步问题 reported by Ac_unsync01 & Ac_unsync02
   同步后进入其它时钟域问题

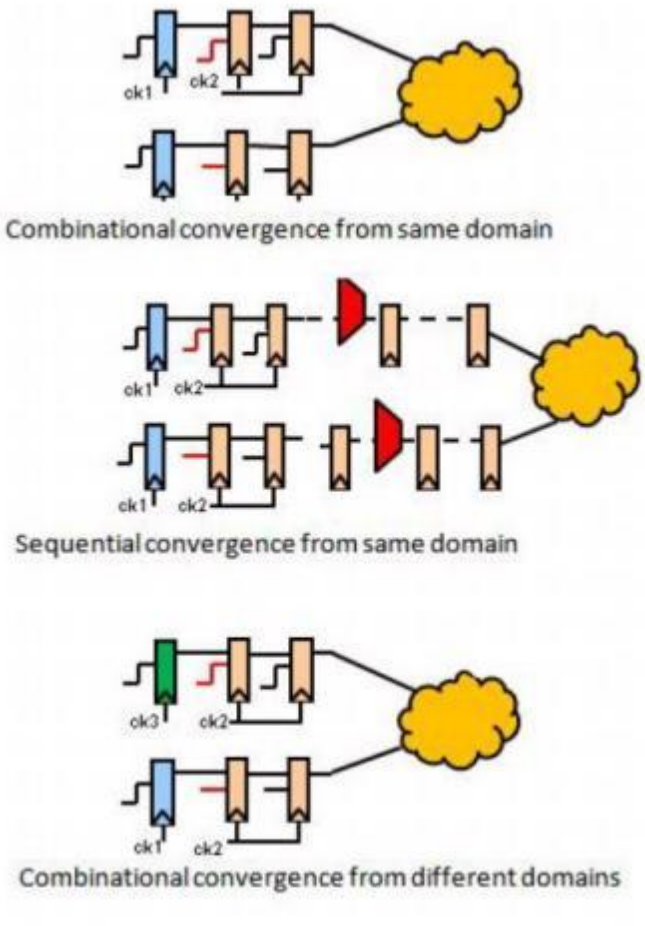
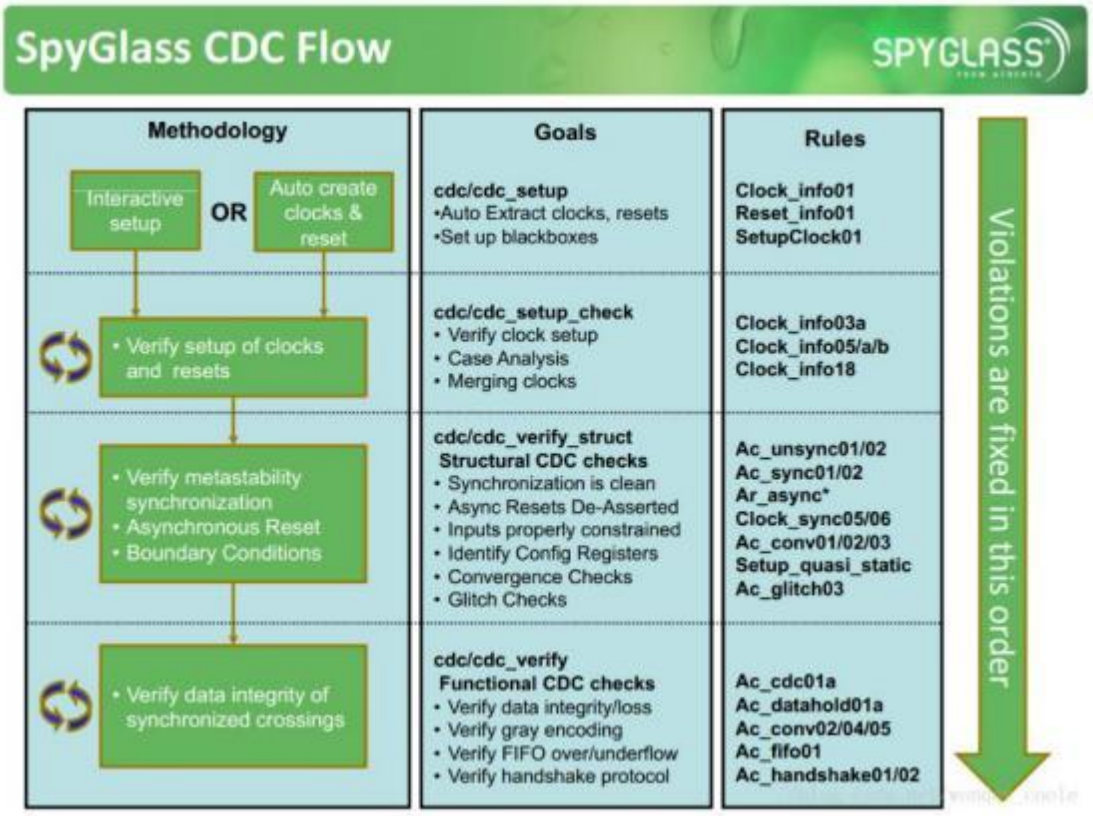


FIGURE 4. Convergence Issues

同步复位问题
毛刺问题
信号宽度问题,主要是快时钟域到慢时钟域问题



5 . Cdc setup

Creating SpyGlass CDC Setup

Creating a setup means specifying design information, such as clocks, resets, and stop modules before *Performing CDC Verification*.

The quality of setup dictates the quality of SpyGlass CDC analysis. A wrong or incomplete setup may result in many false violations or mask a real design bug.

You create a setup by:

- *Specifying Clock Generation Blocks as Black Boxes*
- *Specifying Clocks and Resets for a Design*
- *Generating Clocks and Resets for a Design*
- *Using the Setup Manager*

指定时钟产生模块为bbox——一般时钟源头都为模型，设置为bbox，
为设计指定时钟和复位
为设计指定派生时钟和复位

6． 定位 cdc 问题方式

Debugging CDC Issues

Before debugging, ensure that the analysis is set up correctly. For details, see *Creating SpyGlass CDC Setup*.

If you are running SpyGlass CDC on a reasonable size design, you are likely to see a large number of violations. Most of these violations are because of:

- Incorrect or incomplete setup.
- Configuration signals that should not typically be reported as CDC errors.

You can remove these violations in a systematic way, leaving only a handful of potential real problems that you need to consider.

NOTE: *Never deal with CDC issues by waiving violations. There is a significant danger that you will mask a real problem if you follow that approach.*

You can debug CDC issues by:

- *Using Spreadsheets*
- *Using Incremental Schematic*
- *Viewing Debug Data in Schematic*
- *Filtering Violations Based On Instances*
- *Solving CDC Issues Common to Multiple Violations*

Spreadsheet 表格中的含义如下

	Mnemonic	Constraint	Relationship inferred
1.	SCG	set_clock_group	Aynchronous
2.	SFP	set_false_path	Aynchronous
3.	SCU	set_clock_uncertainity	Synchronous
4.	GEN	parent generated clock	Synchronous
5.	V2R	virtual parent real clock	Synchronous
6.	GRP	set_clock_group (same group)	Synchronou

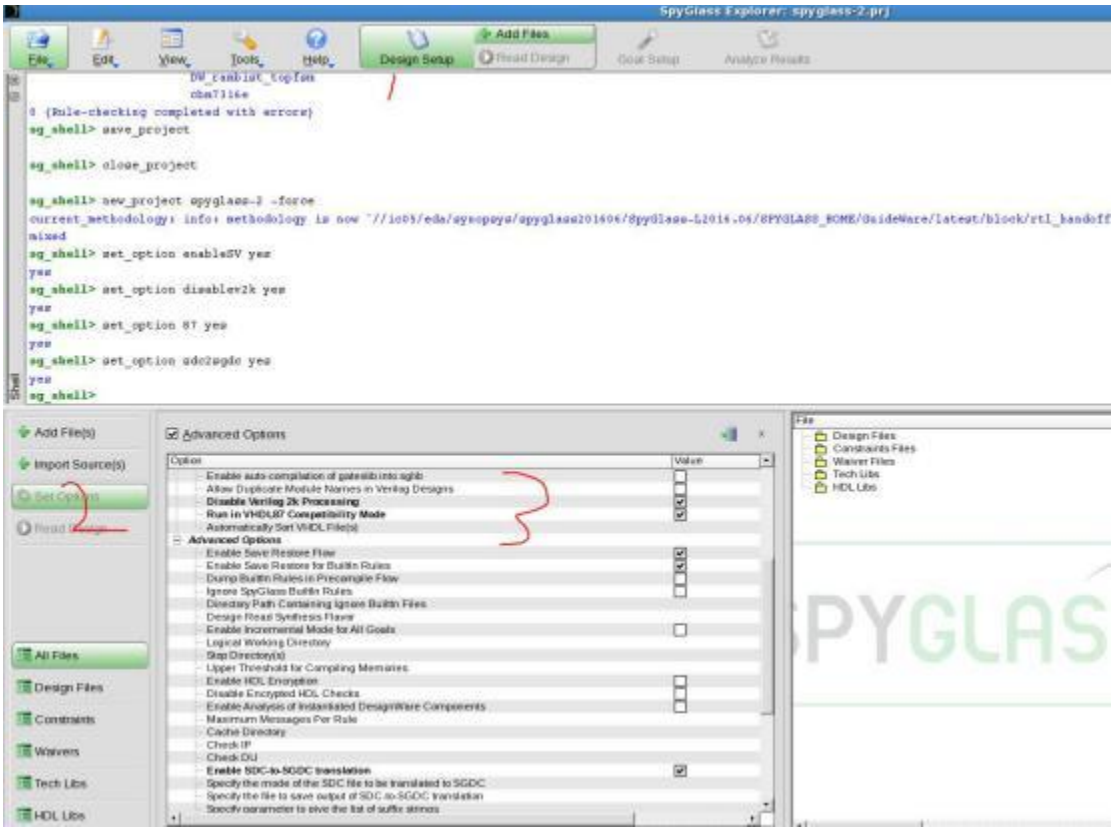
Each cell contains information in the format

1. S (SCG | SFP | SCU | GEN | GRP | V2R)
2. A (SCG | SFP | SCU | GEN)
3. CE - No Domain Assigned because of cyclic Error
   One or both Clock(s) is/are part of cyclic conflict.
   (Refer to Txv_DomainConflict rules for details)
   OR
4. NC - No Domain Assigned since no constraint specified
   (Refer to txv_DomainMissing for details)
5. NA - not applicable - for same clocks

S => Synchronous relation between the clocks.

A => Asynchronous relation between the clocks.

7． 设计配置



当要进入 design setup 界面时,需要点击Add File 按钮

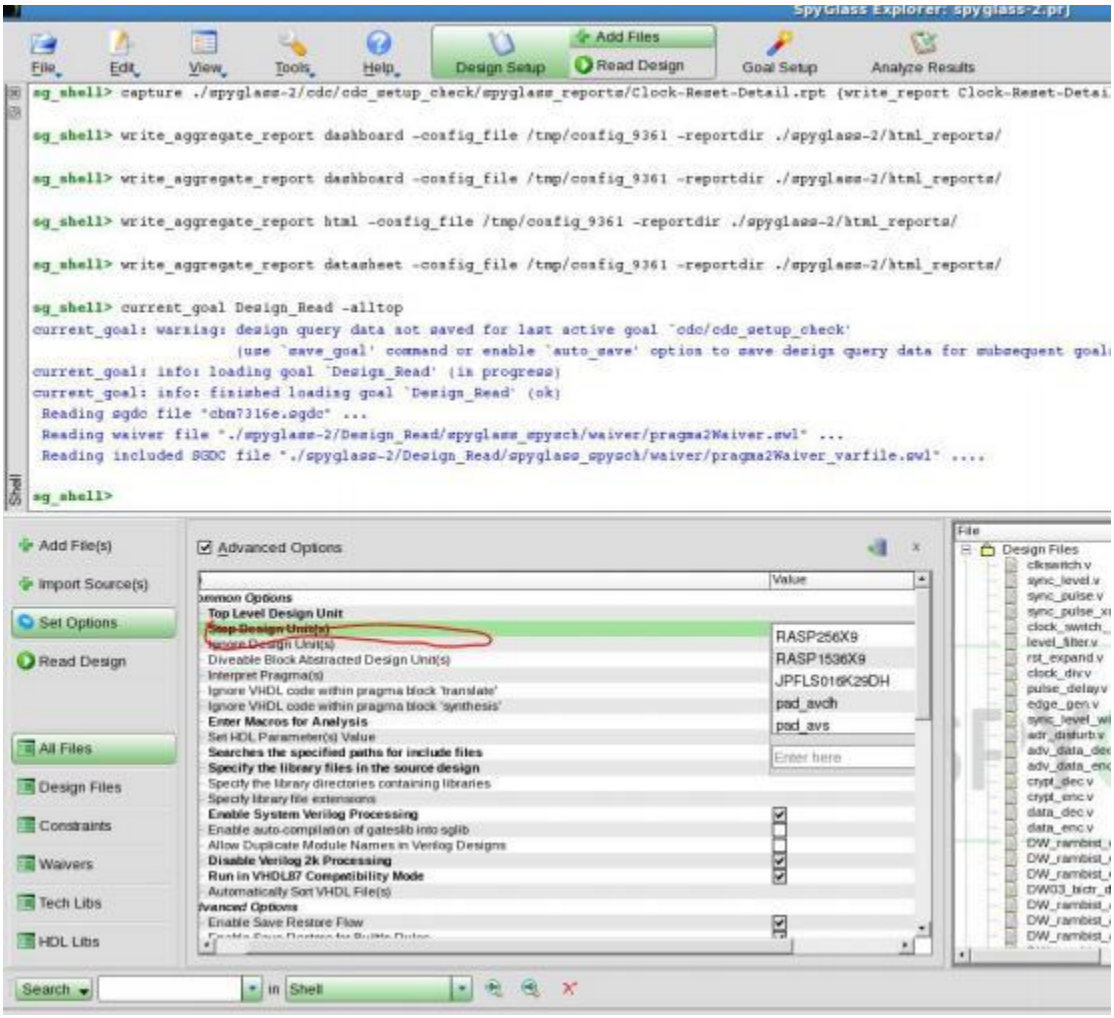
点击保存;(sg_shell> save_project

8． 配置顶层模块

9． Gui 设置 blackbox

点击 add file 进入下面界面,修改 stop design units 的 value,内容填各个 model 的名字;

相当于 shell 命令 set_option stop <module-name>



10． Recognizing Clocks

Different Atrenta standard products process clock information based on their specific rule-checking requirements. Refer to the respective product rules reference document for details.

The following table

Task	SpyGlass lint Solution	SpyGlass STARC Solution	SpyGlass CDC Solution	SpyGlass DFT Solution
Clocks Used For Analysis	Automatic Detection	User-specified (for only two rules)	User-specified clocks and their domains	User-specified
Identification of Clocks[a]	Stops at combinational gates	Stops at combinational gates	Goes beyond combinational gates	Stops at combinational gates
Specification of Clocks	Not possible	Allowed (for only two rules)	Allowed, including internal nodes as clocks	Allowed, but only external pins/ports can be specified
Clock Domain	Same as clock source	Same as clock source	User-specified domain for each clock	Same as clock source
Simple Divider	Divided clock treated as a different domain from the Master clock	Divided clock treated as a different domain from the Master clock	Divided clock treated as a different but related domain from the Master clock	Divided clock treated as a different source and thus domain from the Master clock
Design Constraints	Not supported	Not supported	Supported from 3.2.0	Always supported

a. Clocks are identified by traversing backwards from each flip-flop's clock pin. The identified clocks might be used for all clock-related rules (as in SpyGlass lint Solution), or may just be informative for the users (as in SpyGlass CDC Solution).

层次连接符可以修改

Use the use_hier_sep_slash parameter to change the default hierarchy separator of the SGDC constraints.

By default, the hierarchy separator used in SGDC constraint’s name field is ’.’

11．能够识别的 sdc 的语法

SpyGlass translates the following SDC commands to SGDC commands:

„ create_clock

„ create_generated_clock

This command is not commented out in the generated SGDC file only if the following conditions hold true:

ˆ If generated clocks have domains different than the domains of their source clocks

ˆ If generated clocks are specified on black boxes

说明:generae clock 不能有 mux,它无法使用 set_case_analysis 来指定选择那一个时钟源;

Create_clock 它可以有 mux,使用 set_case_analysis 来指定选择那一个时钟源;也就是说在mux 的位置一定要有一个 Create_clock.

” set_case_analysis

” set_clock_group

” set_clock_sense

” set_disable_timing

” set_false_path

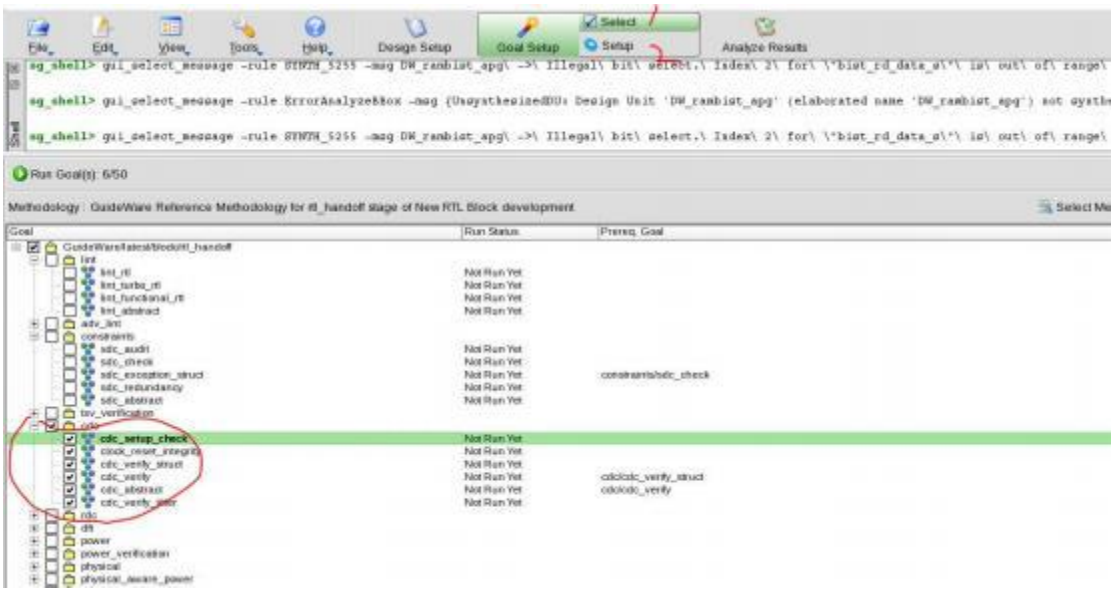
” set_input_delay

” set_mode

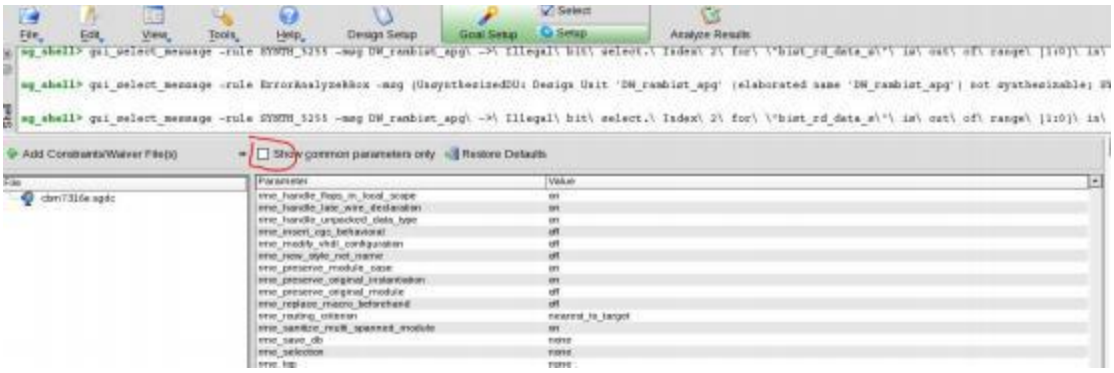
” set_multicycle_path

” set_output_delay

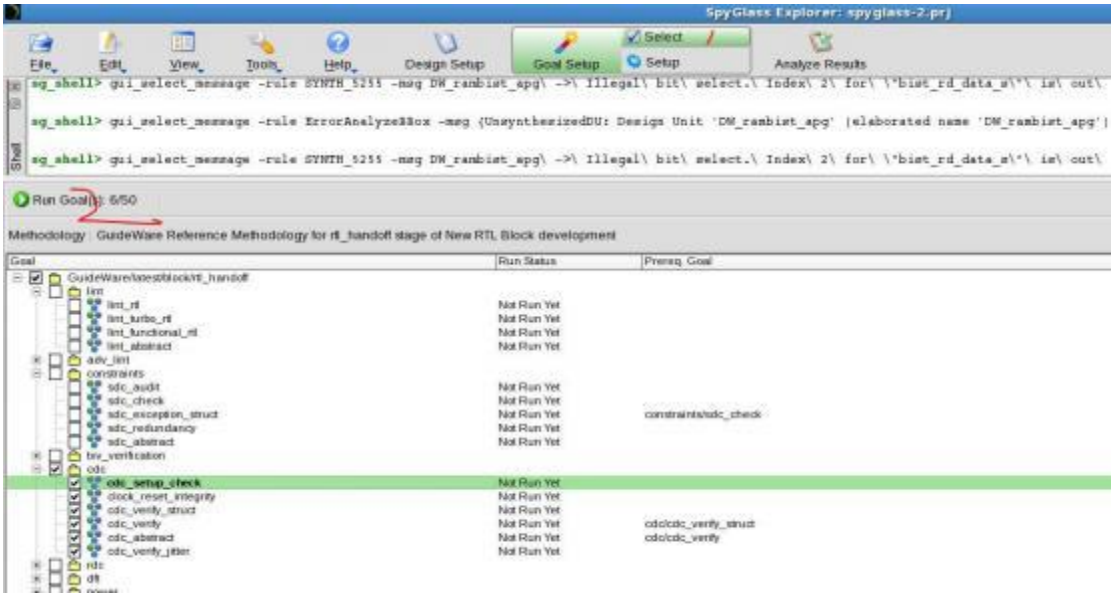
12．Cdc 配置



如上图,需要先选择 cdc,然后再点击 setup,进入下图

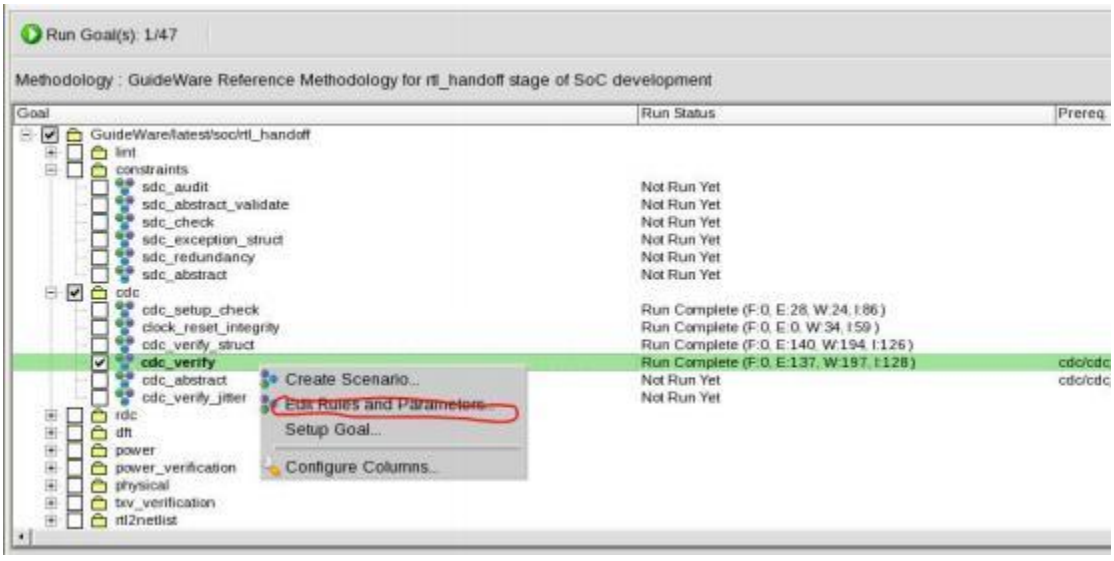


上图的红圈如果钩中,则会显示最基本的配置选项;再次点击 select,进入下图

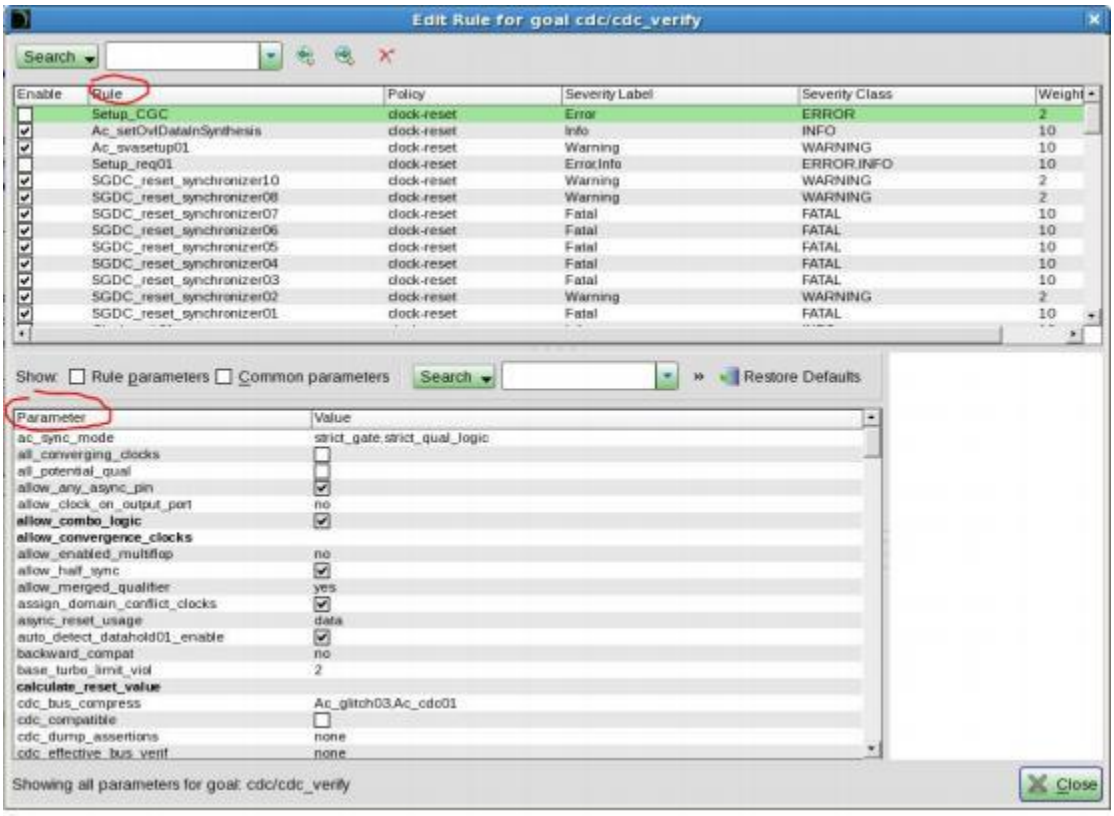


点击 run Goal,则会运行选择的项目,上图是 CDC

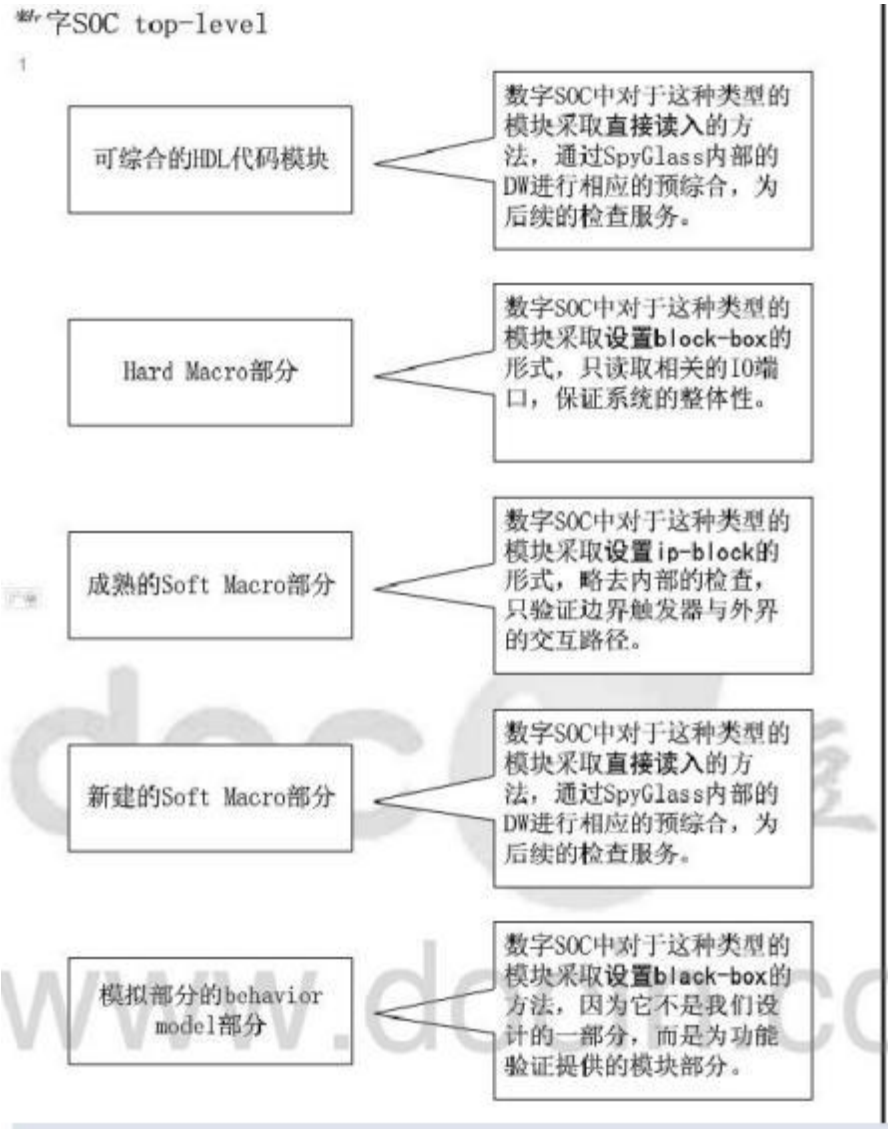
每一个运行项都可以配置规则和参数:通过右键,调出如下界面



再展开



对 rule 点击右键,可以更改规则特性



Spyglass 进行 CDC 检查需要输入的文件：

1. sgdc 文件

   在 sgdc 文件中加入综合的 sdc 文件,如下

   ```
   current_design xxx
   sdc_data  -file xxxx_src.sdc
   ```

2. rtl filelist

13. 控制参数命令

1. 使能 sv 语法:set_option enableSV yes
2. 内部黑盒子:set_option inferblackbox yes

上述命令后会在当前输出目录 spyglass_sch 下,输出一个 sgBlackbox.v 文件,里面记录了 blackbox 信息.

3.set_option handlememory yes

4.使能 sdc2sgdc 格式: set_option sdc2sgdc yes

使用方法,在 sgdc 文件中包含 sdc 文件,如下

current_design <design-name>

sdc_data -file <sdc-file-name>

spyglass 默认打开 DEFAULT_BBOX_MODEL 这个开关,是对黑盒子处理选项.

By default, the DEFAULT_BBOX_MODEL configuration key is set to BBOX_LIBCELL so that only the SpyGlass Library Compiler-generated cells are considered as the BBOX_MODEL type.

这个是在 .spyglass.setup 的配置文件设置,在$SPYGLASS_HOME 目录下有这样一个文件,但这个文件可以覆盖,其优先级如下:

$CWD (highest) > $HOME > $SPYGLASS_HOME (lowest).

如果在工作目录下在 .spyglass.setup 文件中加入

DEFAULT_BBOX_MODEL = NONE

AUTOENABLE_MEMORY_HANDLING = yes

4. 使能 generated_clk:

   set_parameter enable_generated_clocks yes

5. 产生 generated_clk_file:

   Set_parameter use_inferred_clocks yes

6. 配置门控时钟

Set_parameter clock_gate_cell “CG01,CG02”

Set_parameter clock_gate_cell CG*/CG

7. 配置异步电路模式—即如何识别同步异步电路

Set_parameter ac_sync_mode

“soft_gate”/”strict_gate”/”soft_qual_logci”/”strict_qual_logic”

8. 减少 cdc 一些不必要的报告

Cdc_reduce_pessimism

Set_parameter Cdc_reduce_pessimism

mbit_macro,no_convergence_at_syncreset,no_co

nvergence_at_enable,bbox,hanging_net

9. 减少 clock 一些不必要的报告

Set_parameter clock_reduce_pessimism latch_en, mux_sel,

mux_sel_derived, all,

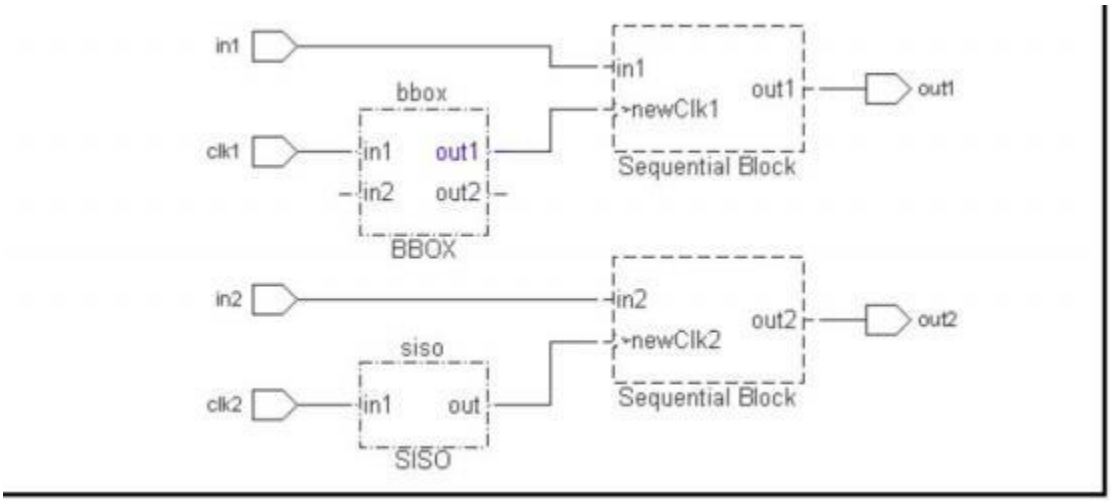
all_potential_clocks, check_enable_for_glitch,

single_input_output_bbox,

stop_derived_at_random_logic, and

ignore_same_domain

如 clock 可以从 bbox 输出等



如上图,如果每配置相关参数,则会报错

set_parameter reset_synchronize_cells “ ”

14. 设置同步器,同 21 项

SpyGlass 中只区分两个类型的 CDC， control 和 data crossing； Qualifier 是指 multi-flop,sync_cell 等 synchronizer。SpyGlass 可以识别所有 CDC 中的 qualifiers。

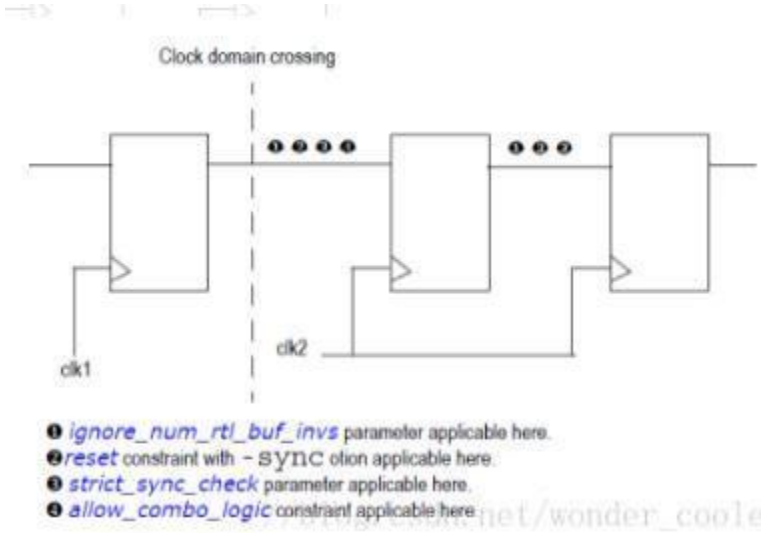
工具检测到了两个时钟路径之间如果有同步电路,则不会报错

Control crossing 可以是：

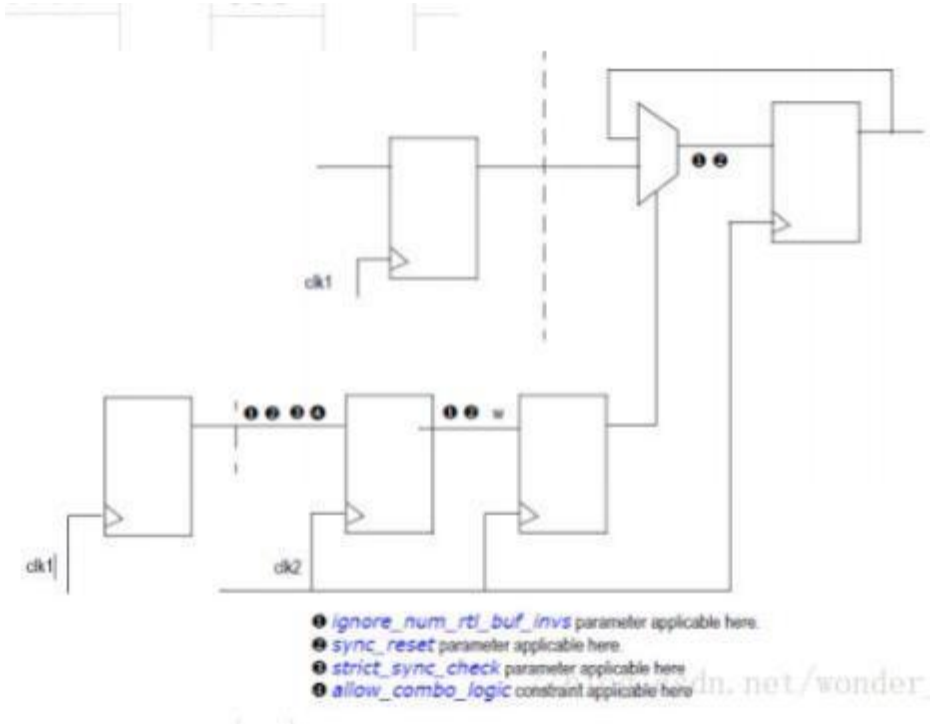
1. RTL multi-flops(auto detected)
2. Liberty cells(require sync_cell constraint)
3. single flops(require sync_cell constraint).

Data crossing 可以识别

1. **data CDC by control crossing:**



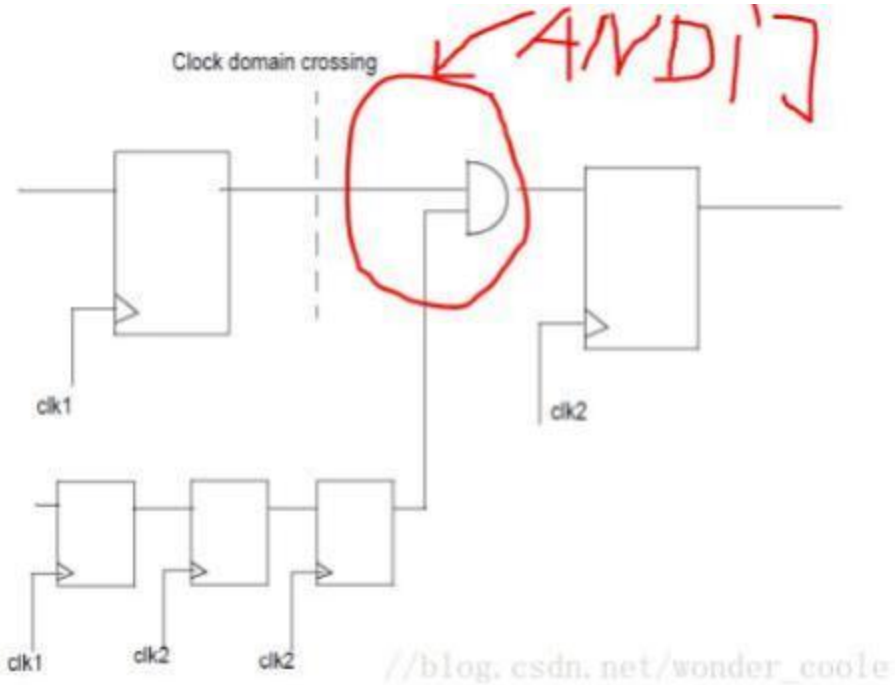
2. **MUX sync data crossing** . 设置参数 *enable_mux_sync*, 可以 diable。



前提是上面的数据在控制选择信号有效期间为稳定数据,

3. AND gate sync 的 data crossing 模式。 设置参数 *enable_and_sync*. AND 门包括 AND/OR/NAND/NOR.

不允许 NOR 门



前提是上面的数据在控制选择信号有效期间为稳定数据,

原文链接：

https://blog.csdn.net/wonder_coole/article/details/79566097

15. 常用的.prj 配置

```
##Data Import Section

read_file -type sourcelist cmd_files_ic.f
read_file -type sgdc prjxxx.sgdc

##Common Options Section

set_option projectwdir .
set_option language_mode mixed
set_option designread_enable_synthesis no
```

```
set_option designread_disable_flatten no
set_option active_methodology
$SPYGLASS_HOME/GuideWare/latest/block/rtl_handoff
set_option top prjxxx
set_option enableSV yes
set_option sdc2sgdc yes
set_option aggregate_reportdir /xxx/spyglass/report/#设置报告路径
set_option define { DBG51 SYN CDC_CHECK }
set_option stop { RASP256X9 RASP1024X9 JLFLS008K29DG pad_avdh pad_avs
pad_vpp pad_vss }
set_option gen_block_options { debug,adc_ctrl } ##这两个模块单独产生报
告

##Goal Setup Section

current_methodology $SPYGLASS_HOME/GuideWare/latest/block/rtl_handoff


current_goal cdc/cdc_setup_check -top prjxxx
set_parameter enable_generated_clocks yes##使能门控时钟
set_parameter_write_sdc_yes
###set_parameter clock_gate_cell ICG_V   ##指定门控时钟
set_parameter ClkGeneratorName "ICG_V  ICG"
set_parameter enable_sync_cell "sync_level, Sync_Pulse" ##指定同步电路
名称
```

16. 设置某个模块不分析,相当于 black_box

```
set_option stop <module-name>

set_option stop {block1 block2}
```

这个设置支持通配符

```
set_option stop {e.*} //Skips all architectures of the e entity

set_option stop {a1*}

set_option stop {\a1\*}
```

Here, the first example matches a11, a12, and a13 while the second example matches \a1*.

当 spyglass 在 design analysis 时,会不分析这个 module.

```
set_option ignoredu { <design-unit-names> }
```

当 spyglass 在 read design 时,会跳过相关设计单元

set_option ignorefile <file-name>

set_option ignorefile myaddlfile.v//注意需要指定相对路径

当 spyglass 在 design analysis 时,会跳过相关文件

注意, set_option nobb yes 这个选项是设计中不能有 black boxes,如果有则退出 spyglass

当某个模块被设置成black boxes 时,虽然端口名称还存在,但都变成了 input 类型,也就是说black box 不存在输出信号,不能驱动其它模块,所以信号无法穿过 black box

17. Sgdc 语法

1.基本语法说明

使用#或//来添加注释;

通过配置把综合的 sdc 转化为 sgdc
Shell :set_option sdc2sgdc yes
在 .sgdc 文件中只需要写入下面语句就可以
current_design ssssss
sdc_data -file sssss_src.sdc
sssss_src.sdc 为项目综合的约束文件.
在目录/constraints/sdc_check/spyglass_reports/sdc2sgdc/会有转化后的 sdgc 文件.
由于在综合
需要使用 set_case_analysis 来处理的时钟
格式: set_case_analysis -name sig1_data3_data4 -value 1

Consequences of Not Configuring MUXes

Consider the scenario in the following figure:

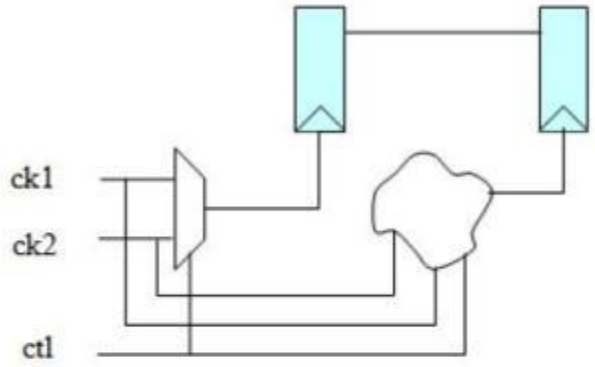


FIGURE 2. Configuring the MUX

In the above scenario, if you do not configure the MUX by applying *set_case_analysis* on its select pin, multiple clocks may drive the same flip-flop. As a result, SpyGlass may infer the path between these flip-flops as asynchronous crossings even if these paths are synchronous. This results in false unsynchronized violations, which results in noise and more time for CDC verification closure.

Fixing Violations for Non Configured MUXes

Check for the *Clock_info05* violations that report cases where you should define *set_case_analysis* on the MUX select pin.

Refer to the *Clock Setup* window that shows MUXes involved in clock paths from where you can interactively define a value for MUX selects.

如果有上面情况,而没有使用set_case_analysis 处理则会在Clock_info05 文件中报告错误.

2. Sgdc 范例

1. Sgdc 语法混合使用
   在 sdc2sgdc 时,sdc 和 sgdc 可以混合,如下

```
current_design sssss
    set_case_analysis -name   "sssss.atpg_test_en" -value 0
    set_case_analysis -name "sssss..osc_sel" -value 1
    //设置路径分析
    sdc_data  -file sssss_src.sdc   //综合时使用的约束文件

    reset  -name "sssss.uAnd2.Z" -async   -value 0
    reset  -name "sssss. POR_SRC" -async   -value 0
    //设置异步复位信号
```

2. Sgdc 单纯的做 cdc 检查,不能识别(推断)sdc 中使用综合后的门级信号.

16. Spyglass 结果分析

Cdc 生成的结果在
/spyglass-1/cdc/cdc_verify/spyglass_reports/clock-reset/目录下.
报告含义

” Ac_unsync01 rule violations: These violations report scalar unsynchronized crossings. Such crossings act as control signals that synchronize complex data crossings.

„ Ac_unsync02 rule violations: These violations report data crossings where no valid synchronization is found.

Fixing Violations Related to Convergence:

Checkfor theAc_conv01, Ac_conv02, Ac_conv03, Ac_conv04, andAc_conv05 violations.

Convergence issues can occur when multiple signals cross from one domain to another but they are separately synchronized.

Fixing Violations Related to Glitches

*Checkfor the Ac_glitch*or Clock_glitch*violations.*

These rules report glitch-prone logic that can lead to problems similar to synchronization issues.

Fixing Violations for Data Hold Checks

Checkfor the Ac_cdc01violations.

Such violations indicate potential problems in signals or data crossing typically from a fast clock domain to a slower clock domain where data sent may have already changed by the time the capturing clock arrives. The Dealing with Functional Checks section provides further detail on how to debug such functional checks.

Fixing Data Hold Issues in Synchronized Data Crossings

Checkfor the Ac_datahold01aviolations.

Such violations report clock domain crossings where data can be unstable while the enable is active. For every data change, the enable should be

activated to capture the new data and should be deactivated before the next data is loaded.

The Dealing with Functional Checks section provides further detail on how to debug such functional checks.

Fixing Violations Related to Data Correlation and Race Conditions

Checkfor gray-codeviolations, suchasAc_cdc08, Ac_conv01, and Ac_conv02.

Convergence of signals, such as control buses can cause major problems if

they are not implemented using approved methods.

Typically, with control buses crossing clock domains, designers implement gray code schemes to handle such issues. Using a gray-encoded implementation for control bus signals ensures that only one bit of the control signal changes during any one clock cycle.

Fixing Violations Related to Reset Synchronization and Deassertion

Checkfor thefollowing ruleviolations:

Rule	Violation Reported
Ar_unsync01	Reports unsynchronized reset signals in the design
Ar_sync01	Reports synchronized reset signals in the design
Ar_asyncdeassert01	Reports if reset signal is asynchronously de-asserted
Ar_syncdeassert01	Reports if reset signal is synchronously de-asserted or not de-asserted at all
Reset_sync02	Reports asynchronous reset signals that are generated in asynchronous clock domain

Fixing Violations Related to FIFO Recognition and Verification

Checkfor the Ac_fifo01and Ac_sync_group ruleviolations.

SpyGlass can automatically identify FIFOs. FIFO recognition may produce following results:

„ Fully recognized FIFOs: This is the case if memory and pointers of a FIFO are identified

„ Partially recognized FIFOs/Memory: A 2-dimensional memory or a lib/

sglib memory identified by SpyGlass for which read/write pointers were not identified.

„ Disabled: When fa_msgmode is set to none.

减少干扰设置:

Reducing Noise

You can reduce noise by:

” Setting Parameters: ” allow_combo_logic; ” cdc_reduce_pessimism; clock_reduce_pessimism: cdc_false_path ; reset -sync ;

” Setting Constraints

” Filtering Violations in a Spreadsheet

17. Spy 各阶段检查的功能点

1.Rules of the cdc_setup Goal

The *cdc_setup* goals runs the following rules:

Rule	Description
Clock_info03a	Reports unconstrained clock nets
Clock_info03b	Reports flip-flops, latches, or clock gating cells whose data pins are tied to a constant value
Clock_info03c	Reports flip-flops or latches where the clock/enable pin is set to a constant
Clock_info05	Reports MUX descriptions where two or more clock signals converge
Clock_info05b	Reports clock signals converging at a combinational gate other than a MUX
Clock_info15	Reports port domain information
Reset_info09a	Reports unconstrained asynchronous reset nets
Info_Case_Analysis	Provides schematic highlight of propagated values.
Clock_check07	Reports clock domains that reach another clock domain
Clock_converge01	Reports a clock signal whose multiple fan-outs converge
Reset_check03	Reports synchronous reset signals that are being used as active high as well as active low
Reset_check10	Reports asynchronous resets used as non-reset signals
Reset_check11	Reports asynchronous resets used as both active-high and active-low
Reset_check12	Reports flops/latches/sequential element that do not get active reset during power on reset
Clock_info18	Reports unconstrained ports summary
Ac_resetvalue01	Reports missing '-value' field in 'reset' constraint

18.Rules of the clock_reset_integrity Goal

The *clock_reset_integrity* goal runs the following rules:

Rule	Description
Clock_info05b	Potential glitch in clock tree due to clocks converging on combination gate (other than a MUX)
Clock_check01	Potential glitch in clock tree due to unexpected gates in clock tree (e.g. XOR gate in clock tree)
Clock_check04	Both positive and negative edges of clocks used in a same design
Reset_check01	Reset usage check against sync/async_set_reset pragma
Reset_check02	Glitches in reset paths due to unexpected gates (e.g. XOR gate in reset tree)
Reset_check03	Both positive and negative edges of synchronous reset used in a same design
Reset_check04	Both positive and negative edges of asynchronous reset used in a same design
Reset_check06	High fan-out reset nets not driven by placeholder cell
Reset_check07	Glitches on reset paths due to combinational logic on reset tree
Clock_Reset_check01	Glitches due to unwanted gates on clock or reset trees
Clock_Reset_check02	Race between flip-flop output and its clock/reset
Clock_Reset_check03	Race between flip-flop clock and reset
Info_Case_Analysis	Information on case-analysis to help debug violations
ClockEnableRace	Race between clock and enable of a same flip-flop
Clock_Reset_info01	Clock and reset usage matrix for information
Clock_glitch02	Gated clocks with improper enable logic
Clock_glitch03	Clock re-convergence at MUX
Clock_glitch04	Glitches due to combination logic driving flip-flops clock pin
Clock_converge01	Reports a clock signal whose multiple fan-outs converge

Rules in the cdc_verify Goal

The goal cdc_verify runs the following rules in addition to the Ac_sync_group rules:

Rule	Description
Clock_sync05	Reports primary inputs that are multi-sampled
Clock_sync06	Reports primary outputs driven by multiple clock domain flip-flops or latches
Clock_sync09	Reports signals that are synchronized more than once in the same destination domain
Ar_unsync01*	Reports unsynchronized reset signals in the design
Ar_sync01*	Reports synchronized reset signals in the design
Ar_asyncdeassert01*	Reports if reset signal is asynchronously de-asserted
Ar_syncdeassert01*	Reports if reset signal is synchronously de-asserted or not de-asserted at all
Reset_sync02	Asynchronous reset should not be generated in asynchronous clock domain
Reset_sync04	Asynchronous resets synchronized more than once in the same clock domain
Ac_cdc01a*	Data hold in multi-flop synchronized fast-to-slow crossing
Ac_datahold01a*	Reports synchronized data clock domain crossings where data can be unstable
Ac_conv01*	Check for sequential convergence of properly synchronized control crossings
Ac_conv02*	Check for combinational convergence of properly synchronized control crossings
Ac_conv03*	Convergence of synchronized signals from different source domains
Ac_cdc08*	Gray encoding of control bus crossing clock domains
Ac_fifo01*	FIFO overflow and underflow checks
Info_Case_Analysis	Provides schematic highlight of propagated values
Ac_clockperiod01*	Reports missing '-period' or '-edge' fields in 'clock' constraint
Ac_clockperiod02*	Reports clocks whose periods are rounded off by SpyGlass for lower design cycle
Ac_clockperiod03*	Reports correlated clocks whose design cycle is greater than the threshold value.
Ac_initstate01*	Reports a valid state of the design from which the formal analysis would actually start.
Ar_syncrst_validation*	Verifies user-defined synchronous resets
Ac_crossing01*	Generate spreadsheet for Crossing Matrix view
Ac_glitch03	Reports clock domain crossings subject to glitches

NOTE: ** means the rules and parameters that require Advanced CDC License.*

The cdc_verify goal also includes all the rules of cdc_setup_check goal. These are added to verify any new constraints, which may be added during verification.

Rules in the cdc_abstract_validate Goal

The cdc_abstract_validate goal runs the following rules:

Rule	Description
SGDC_abstract_port_validation01	Checks that the domain defined for a port is consistent with the domain that drives it from the higher-level block
SGDC_abstract_port_validation02	Verifies that a port with -sync specified is driven by a synchronizer from the higher-level block
SGDC_abstract_port_validation03	Verifies that the clocks of the synchronizer (source and destination) defined in abstract_port match those in the higher-level block
SGDC_abstract_port_validation04	Verifies that the combo parameter specified in abstract_port constraint matches what drives the port from the top-level block
SGDC_cdc_false_path_validation01	Verifies that the -from and -to clocks of a the cdc_false_path constraint are different in the top-level block
SGDC_clock_validation01	Verifies that no clock propagates to a port of the block if no clock constraint is defined in the abstract model
SGDC_clock_validation02	Verifies that a clock propagates to a port of the block if a clock constraint is defined in the abstract model
SGDC_clock_domain_validation01	Verifies that two or more ports that have the same domain in the abstract model receive the same clock from the top-level block
SGDC_clock_domain_validation02	Verifies that two or more ports that have different clocks domain in the abstract model receive different clocks from the top-level block
SGDC_define_reset_order_validation01	Verifies that the resets defined in from and to fields of the define_reset_order constraint are driven by resets in the higher-level block
SGDC_define_reset_order_validation02	Verifies that the resets defined in from and to fields of the define_reset_order constraint are driven by different resets in the higher-level block

Rule	Description
SGDC_input_validation01	Verifies that the domain of the clock defined in input/abstract_port constraint matches the domain of the clock that drives the port in the higher-level block
SGDC_input_validation02	Verifies that if no input/abstract_port constraint is defined, then the port is not driven by a flip-flop in the higher-level block
SGDC_num_flops_validation01	Verifies that the clocks specified in from_clk and to_clk of num_flops constraints are not the same in the higher-level block
SGDC_num_flops_validation02	Verifies that the number of flip-flops in the num_flop constraints for a clock pair in the abstract model matches the number of flip-flops for the corresponding pair in the higher-level block
SGDC_reset_validation01	Verifies that a port with no reset constraint in the abstract model is not driven by a reset in the higher-level block
SGDC_reset_validation02	Verifies that a port with a rest constraint in the abstract model is driven by a reset in the higher-level block
SGDC_reset_validation03	Verifies that top and block level asynchronous and synchronous reset types are not conflicting
SGDC_reset_validation04	Verifies that the active value of a reset for a port defined in the abstract model matches the value of the reset that drives the port in the higher-level block
SGDC_qualifier_validation01	Verifies that the clocks specified in from_clk and to_clk of a qualifier constraint are not the same in the higher-level block
SGDC_qualifier_validation02	Verifies that if a port does not have a qualifier constraint in the abstract model, then no qualifier drives the port in the higher-level block
SGDC_set_case_analysis_validation01	Verifies that the value of a set_case_analysis constraint on a port in the abstract model matches the value propagated to the port in the higher-level block

Rule	Description
SGDC_set_case_analysis_validation02	Verifies that if a port does not have a set_case_analysis constraint in the abstract model, then no constant value is propagate to that port in the higher-level block
SGDC_virtualclock_validation01	Verifies the validity of block-level virtual clock with higher-level clocks

18． Cdc 最终输出的报告

在 spyglass/spyglass-1/consolidated_reports/目录下会有每个步骤的输出报告,其中 cdc_cdc_verify/ CDC-summary-report.rpt 会有跨时钟报告,如下:
(同时在/spyglass-1/ prjxxx/cdc/ 目录下会有相同的整体报告（通过链接到 cdc_cdc_verify/ CDC-summary-report.rpt）及详细的跨时钟报告
 set_option top prjxxx)
其中 Ac_unsync01 为单 bit 未同步数据
其中 Ac_unsync02 为多 bit 未同步数据

……

```
=========================================================================
Section E. CDC Analysis and Verification:
=========================================================================

  Number of Clock Domain Crossings             : 181
    Number of Unsynchronized Crossings         : 147
        Number of scalar  (Single-bit) crossings
(Ac_unsync01)                 : 69
        Number of vector  (Multi-bit) crossings
(Ac_unsync02)                  : 78
    Number of Synchronized
Crossings                                        : 34
        Number of Scalar  (Single-bit) Crossings
(Ac_sync01)                   : 6
        Number of Vector  (Multi-bit) Crossings
(Ac_sync02)                    : 28
  Multi-sampled clock inputs
(Clock_sync05)                                  : 0
  Multi-Transitioned clocks
(Clock_sync06)                                   : 0
  Multi Synchronized control signal
(Ac_coherency06)                          : 4
  Total Convergences                             : 7
     Combinational Convergence of same domain synchronized
signals(Ac_conv02): 1
     Sequential Convergence of same domain synchronized signals
(Ac_conv01) : 3
     Convergence of different domain synchronized signals
(Ac_conv03)        : 3
  Non-converging buses crossing domain
(Ac_conv04)                           : 0
```

```
  Data-loss for fast to slow synchronized control crossings
(Ac_cdc01a)      : 0
  Incorrect data-enable sequence in synchronized data crossings
(Ac_datahold01a): 0
  Glitches in synchronized control crossing paths
(Ac_glitch03)              : 13
  For more details, refer to following files:
  ------------------------------------
   - Ac_sync01.csv
   - Ac_sync02.csv
   - Ac_unsync01.csv
   - Ac_unsync02.csv
   - Ac_coherency06.csv
   - Ac_conv01.csv
   - Ac_conv02.csv
   - Ac_conv03.csv
   - Ac_glitch03.csv
=========================================================================
Section F. Asynchronous Reset Analysis and Verification:
=========================================================================
   Asynchronous resets without reset synchronizers
(Ar_unsync01)             : 4
   Asynchronous resets with reset synchronizers
(Ar_sync01)                  : 6
   Asynchronous resets with asynchronous deassertion
(Ar_asyncdeassert01)    : 0
   Asynchronous resets with synchronous deassertion
(Ar_syncdeassert01)      : 6
   Asynchronous resets synchronized multiple times in same domain
(Reset_sync04)  : 3
   Asynchronous resets generated from a different domain
(Reset_sync02)      : 2

  For more details, refer to following files:
  ---------------------------------------
......
```

The SpyGlass CDC solution generates the following reports:

Report Name	Description
Clock-Reset-Summary	Provides information on clocks, resets, unconstrained clock nets, and clock-domain crossings in a design.
Clock-Reset-Detail	Provides information on the following: • Synchronized and unsynchronized clock domain crossings • Crossings filtered by using the *cdc_false_path* constraint • Flip-flops that have their data pin tied to a constant • Synchronization techniques
CKTree	Shows clock hierarchy in a tree-like format (also called clock tree).
CKCondensedTree	Shows a condensed clock tree in which unlike the *CKTree* report, this report shows the number of leaves instead of actually listing the leaves.
RSTree	Shows set/reset trees in a design.
SyncRstTree	Shows synchronous set/reset trees in a design.
PortClockMatrix	Provides information on constraints coverage of ports.

Report Name	Description
SynchInfo	Provides information on destinations and synchronizers involved in different synchronization schemes.
CrossingInfo	Provides information on source and destination flip-flops for all synchronized and unsynchronized crossings.
CKPathInfo	Provides information on clock cells on clock paths.
CKSGDCInfo	Provides information on user-specified constraints.
CDC-report	Provides summary on design, design setup, and verification results.
CDC-summary	Provides a concise summary of the design, design setup, and verification results. It is similar to The CDC Report with some extra information.
CDC-detailed	Provides a concise summary of the design, design setup, and verification results. It is similar to The CDC Report with some extra information.
adv_cdc	Provides information that helps you to analyze the cause of a bug and helps you to gather functional analysis statistics.
adv_reg	Provides information on clocks, resets, and registers in a design.
NoClockCell-Summary	Provides information on the objects specified by the *noclockcell_start*, *noclockcell_stop_instance*, *noclockcell_stop_module*, and *noclockcell_stop_signal* constraints.
DeltaDelay-Concise	Shows a list of delta delay values for each clock in a design and number of flip-flops and latches for each delta delay value.
DeltaDelay-Detailed	Shows a list of delta delays for each clock, net names of flip-flops for each delay value, and net names of latches for each delay value.
DeltaDelay02-Detailed	Shows a list of flip-flops that can cause simulation problems due to delta delay issues.
DeltaDelay-Summary	Provides information on objects specified by the *deltacheck_start*, *deltacheck_stop_instance*, *deltacheck_stop_module*, *deltacheck_stop_signal* constraints.
Ac_sync_group_detail	Shows details of violations reported by the *Ac_sync02*, *Ac_sync01*, *Ac_unsync02*, and *Ac_unsync01* rules.

Report Name	Description
Ac_sync_qualifier	Shows all the control-crossing synchronizers with their qualifiers usage status in synchronizing data crossings.
Glitch_detailed	Shows a summary of all the sources that are crossing destinations and contain glitch-related issues.
Module_Topology	Shows the dependency of modules in a design.
distributed_time	Shows run time details of SpyGlass CDC rules that are run in parallel on same or different machines.

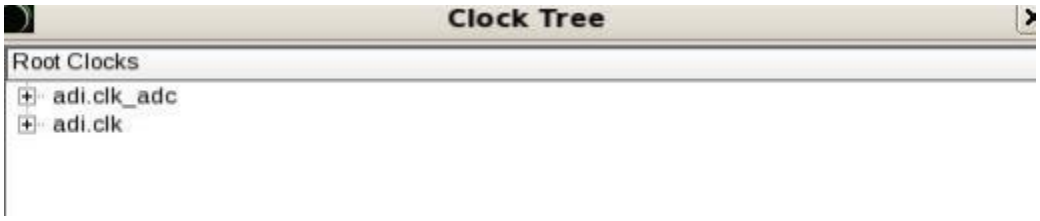
NOTE: *By default, the width of each column in the reports is dependent on the length of information. You can format the display of information by wrapping up the text by setting the format_report parameter to* `yes`.

19． 每步输出的结果

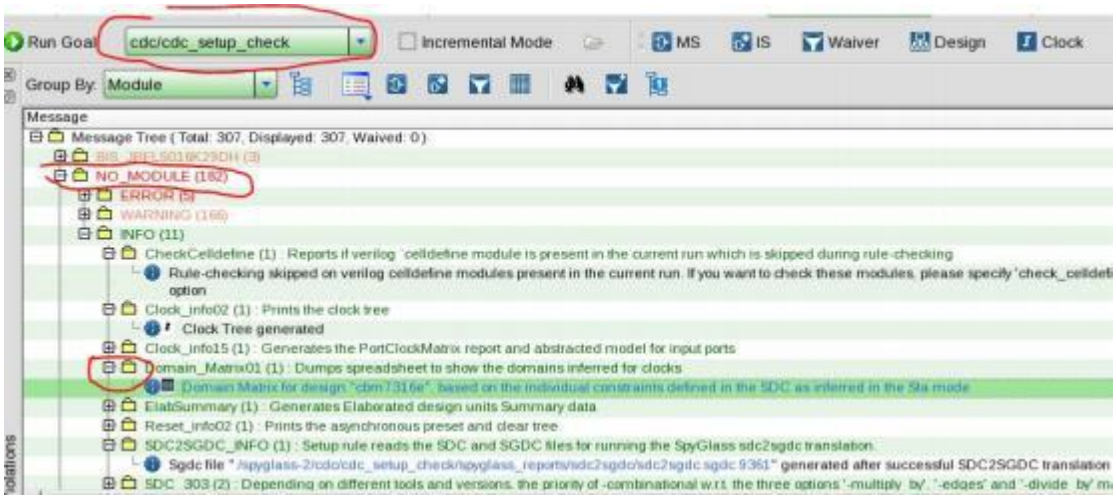
1 在 cdc_check_setup 检查之后,就可以看到各个时钟之间的结果---根据 sdc 产生

1.1 Clock tree

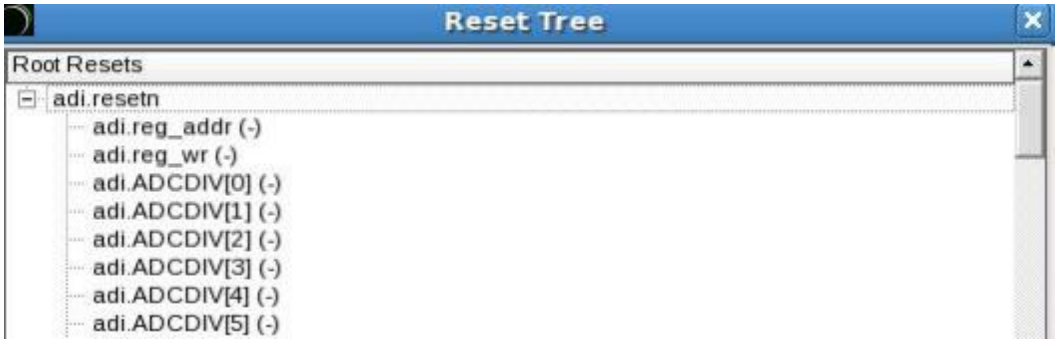
整个时钟树结构,且各个时钟带的寄存器



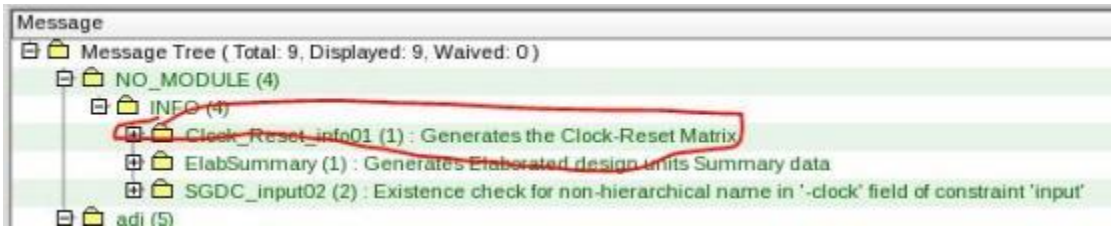
1.2 各个时钟之间的关系—时钟矩阵



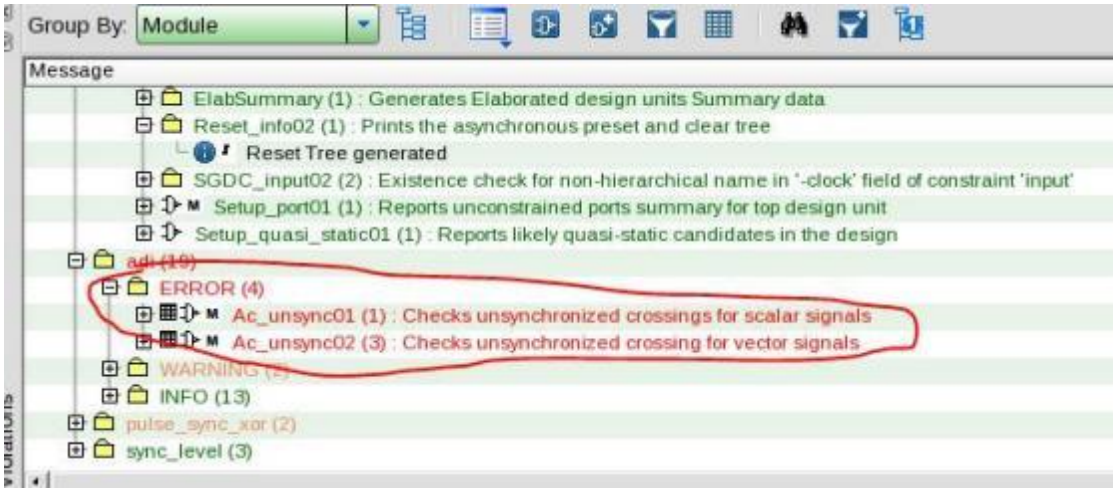
1.2 复位树



2 在 clock_reset_integrity 之后,可以看到 clock_reset 之间的关系



3 cdc_verify_struct 这个动作开始做 cdc 检查



直接双击,可以看到错误地方,

Ac_unsync01:单 bit 信号在两个时钟域没同步

Ac_unsync02:多 bit 信号在两个时钟域没同步

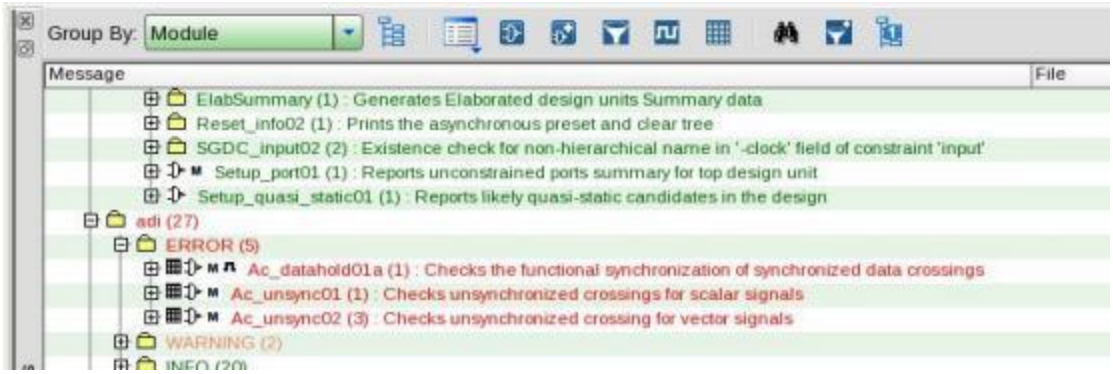
点击上图红圈,可以看到电路结构

Ac_conv02:两个时钟域的信号做了组合逻辑操作

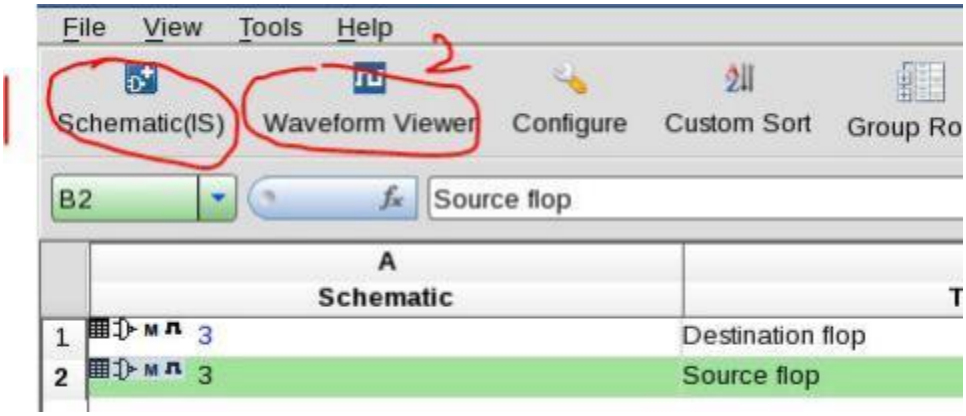
Clock_sync06:输出的信号来自于多个时钟域

Reset_sync02:复位信号不来自某个时钟域(应该是没做同步释放)

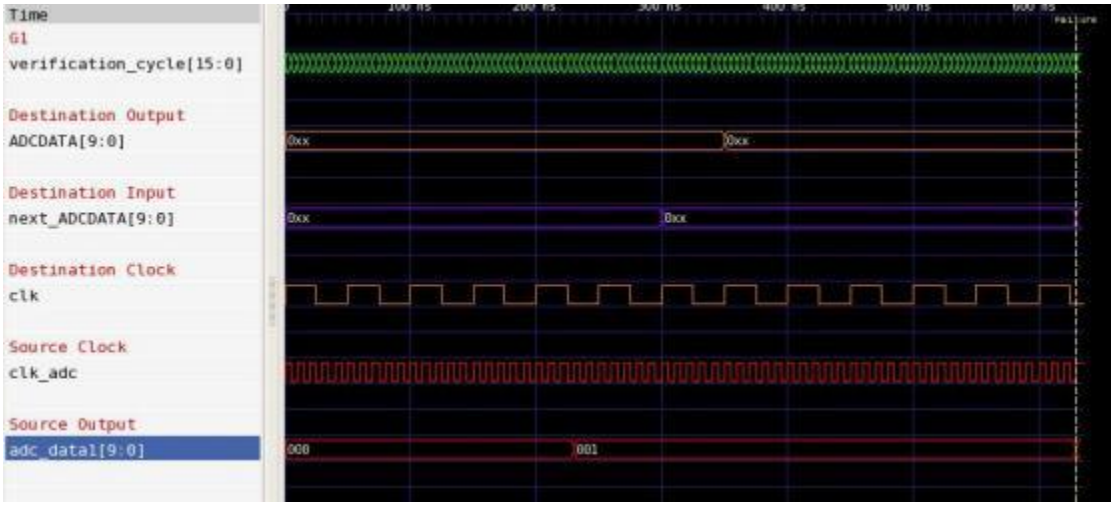
4 4 cdc_verify 这个动作 cdc 检查—内部信号检查



Ac_datahold01a:主要是检查数据类的同步问题
双击进入下图表格描述



上图红圈 1 可以看到电路结构,红圈 2 会看到仿真波形,如下



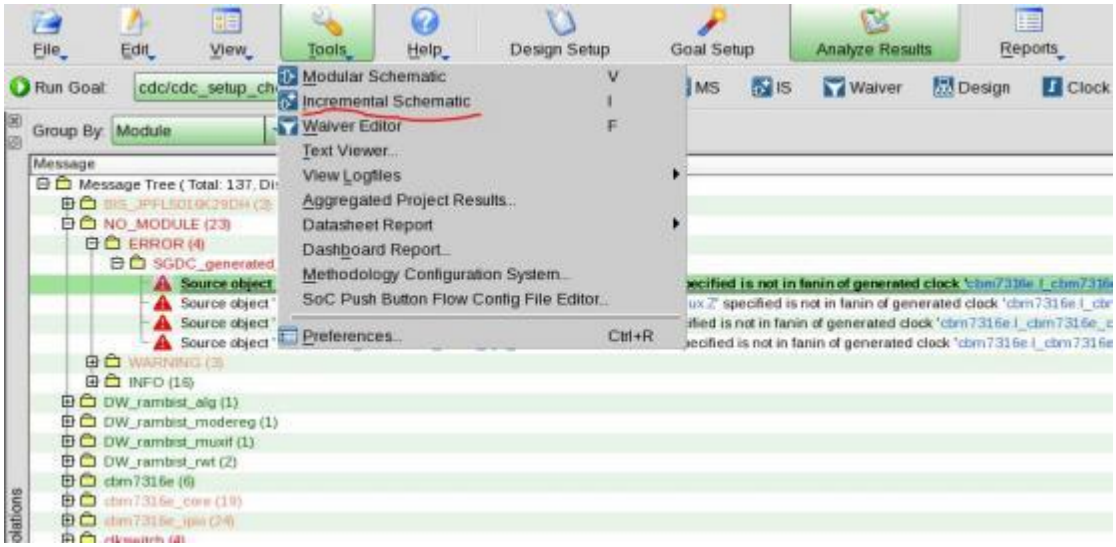
注意:对于数据类的同步,我们可能采用 1bit 同步过的信号来与这组数据来做同步,主要是根据 0&anydata=0,当同步信号为 1 时,数据有效.这样使用的前提是数据在单 bit 信号变化过程中要稳定不变.即在同步信号变高之前已经稳定,在同步信号变低之后才可以变化.

5 cdc_abstract

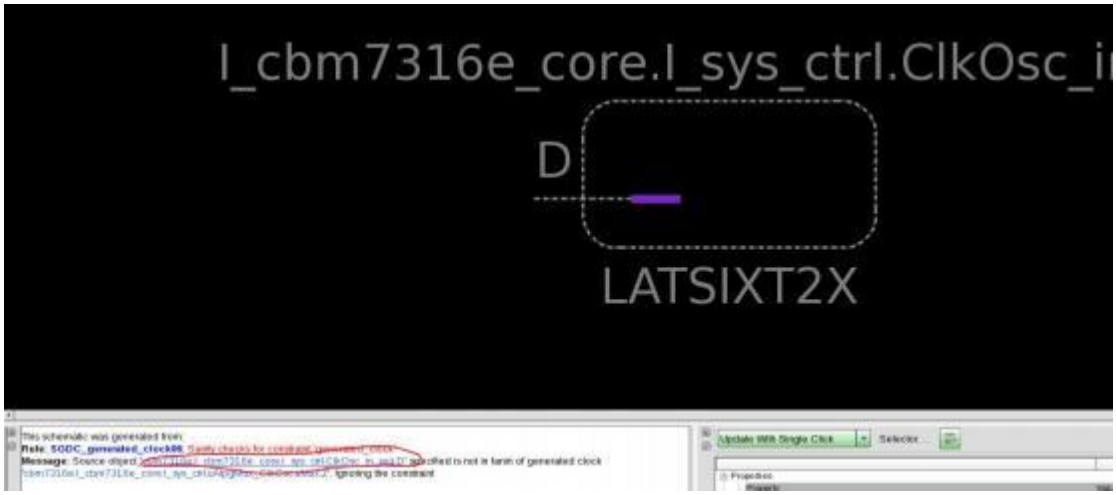
这个动作只是提取端口信号检查,不做内部检查,见上面描述的规则检查

20. 使用电路图分析问题

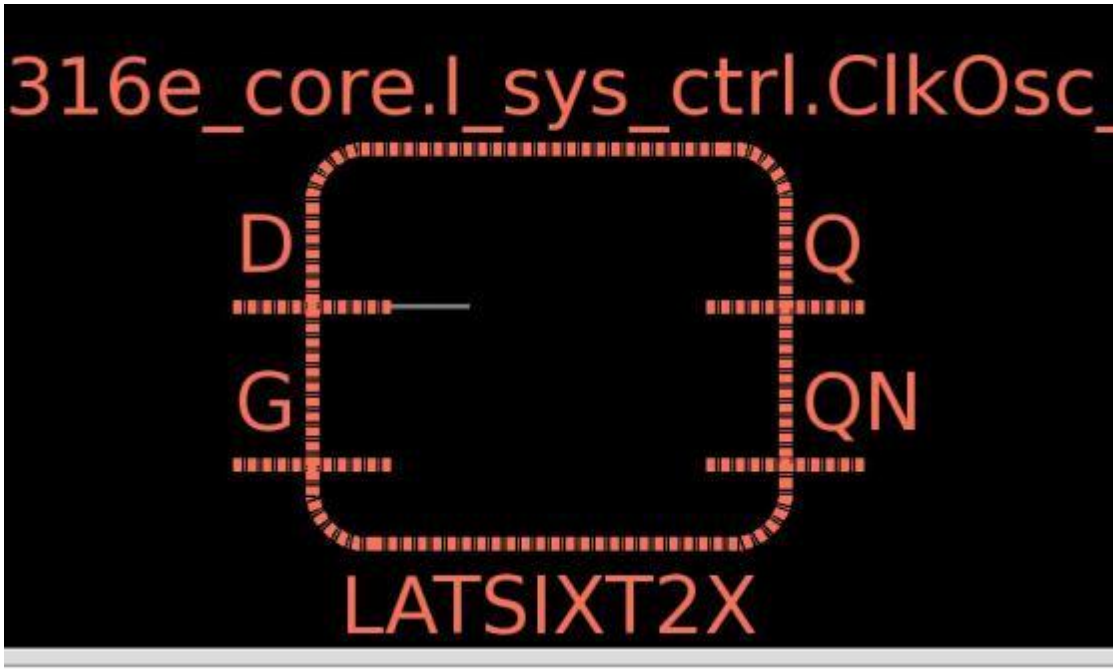
通过 Tools -> incremental schematic,可以进入电路图界面



双击下图红圈,会出现电路图模块



双击图形边框,出显示出 pin



双击 pin,会显示出它的 load 或 driver,已框图方式出现

不断使用上述方法,可以显示所有的电路连接关系

21. CDC Rule:

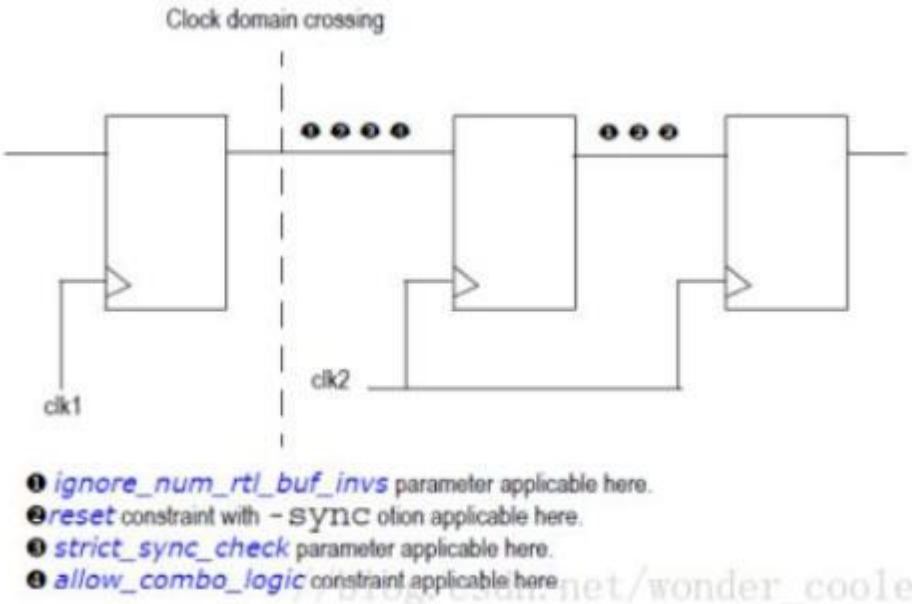
SpyGlass 中只区分两个类型的 CDC， control 和 data crossing； Qualifier 是指 multi-flop,sync_cell 等 synchronizer。SpyGlass 可以识别所有 CDC 中的 qualifiers。

Control crossing 可以是：

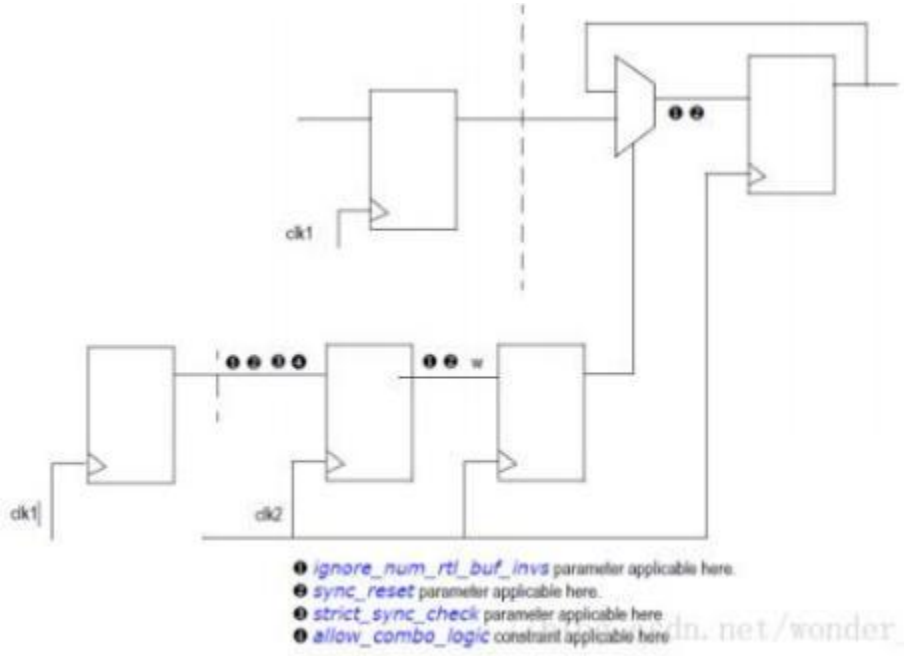
1. RTL multi-flops(auto detected)
2. Liberty cells(require sync_cell constraint)
3. single flops(require sync_cell constraint).

Data crossing 可以识别

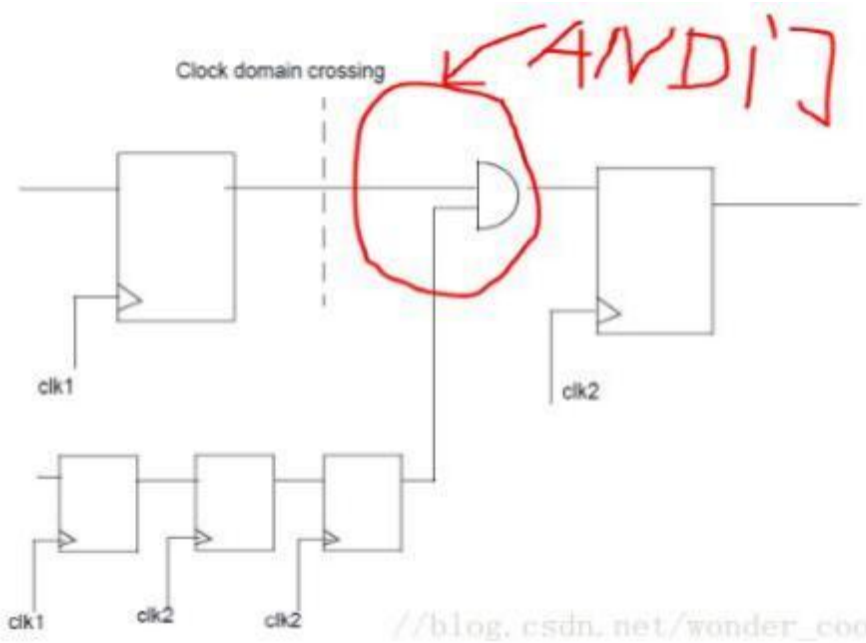
1. **data CDC by control crossing:**



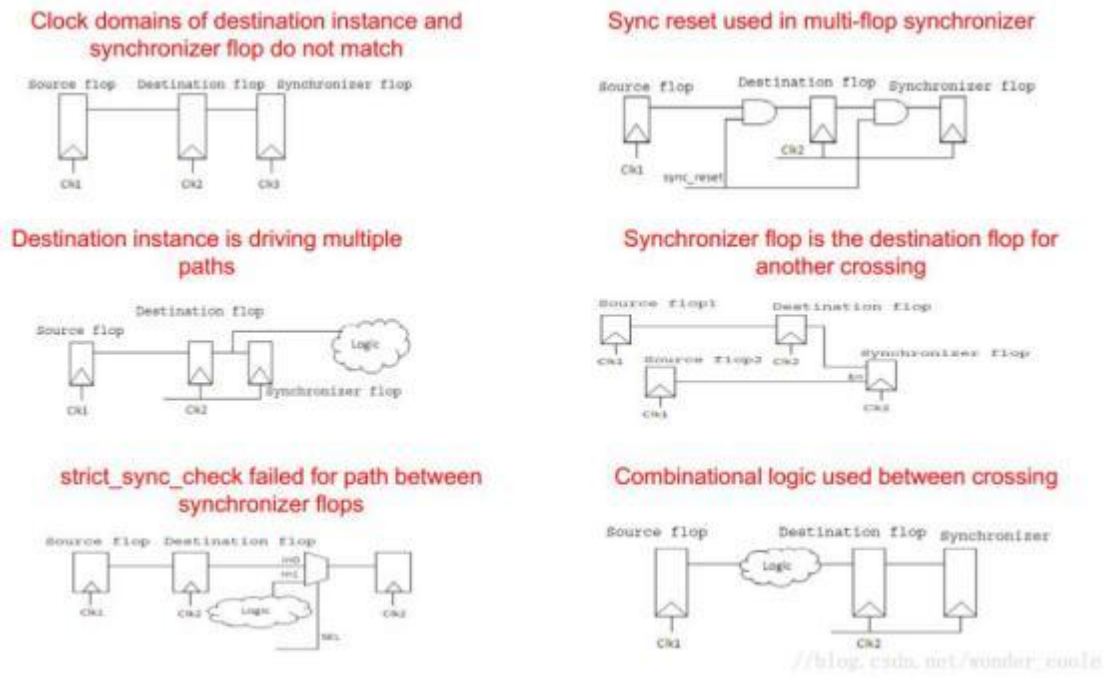
2. **MUX sync data crossing** . 设置参数 *enable_mux_sync*, 可以 diable。



3. **AND gate sync 的 data crossing 模式**。 设置参数 *enable_and_sync*. AND 门包括 AND/OR/NAND/NOR. 不允许 NOR 门。



4. **一些常见的 unsync 的情况：**



5. **处理 violation：**

对应每个 flop，SpyGlass 都会报出结果，结果有三类：synced, unsynced及no analysis(stuch clock or data). 对应每个unsynced crossing，SpyGlass 都会给出 REASON. 要 trace 所以的 source 来确定真正的原因。

- 不要 waive Ac_unsync01/02 violation. 警告：不要靠 waive 来消灭问题，这样会带来潜在的 risk!!!

- 要靠优化 constraint 来消灭 unsync issue。

1）靠 set_case_analysis 来选择 clock 的正确工作模式。

2）靠设置 quasi-static 来确定不会 toggle 的 signal，这样可

以解决很多组合逻辑的问题。

3） reset

4)qualifier， 用在 multi-flop 不存在时，
如 primary input(abstract_port -sync) 或 blackbox。

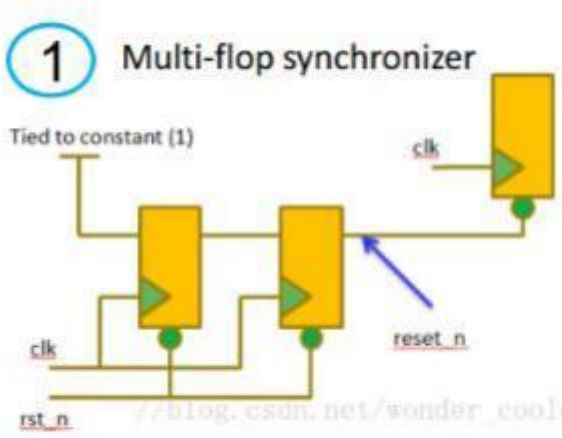
5）sync_cell, 当 SpyGlass 不能识别 sync cell 是用。

6）FIFO constraint.

7) cdc_false_path, 如果 sync 是在外部或靠 SW 控制的时候，例如信号改变前 off clock。

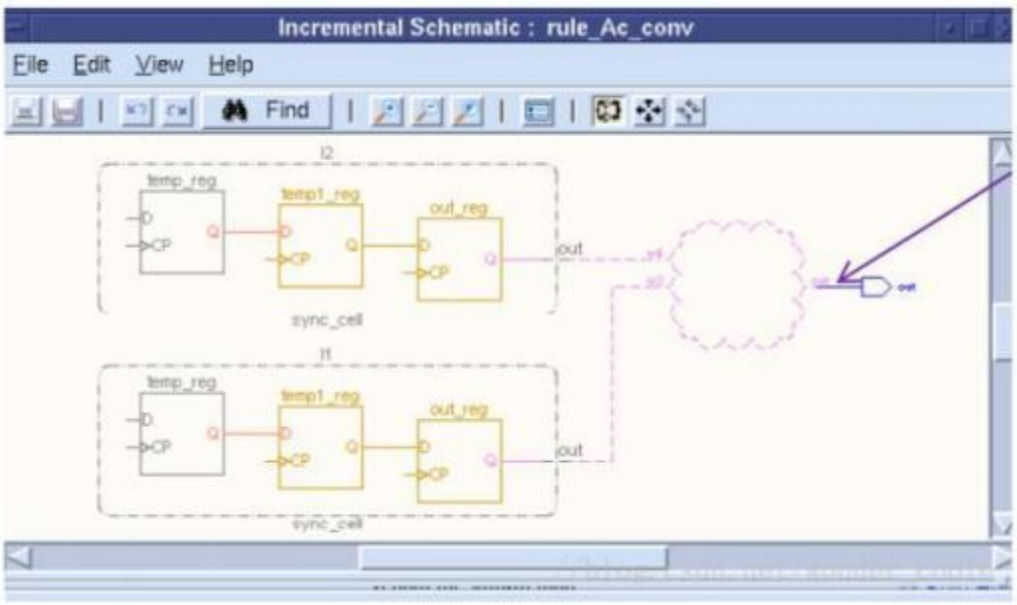
- 如果 violation 太多，应该回头看看 setup 对错。
- 注意 intentionally unsynced crossing. 如配置寄存器，interrupt 信号，GPIO, 及有 SW 控制的 crossing 等。

6. **Reset synchronization：** 常用的 reset synchronizer 如下图



如果 design 中缺少reset synchronizer, 工具会报告reset 源到 flop 端 path 缺少 reset 同步器的问题。出来 design 要有 reset 同步器，在 SGDC 中要合理添加 reset constraint。如果使用了 synchronizer cell, 也要设置 reset_synchronize_cells 参数指明.

7. **Coherency check：** 同步后的两个信号在 destination clock domain 组合使用会造成严重的 function failure！！ 如下面情况：



怎么办呢?
1）source signals 是不是 gray coding?
2) source 信号是不是从来不会同时变化?
3）如果 gate function 是 OR， source 信号同时变化是不是不要紧?
4）使用 waiver 或者添加 cdc_filter_coherency constraint

22． 修改错误等级

See the SpyGlass Policy Customization Guide for more details

23． 自定义规则

See the SpyGlass Policy Customization Guide for more details

24． 注意点

1. 时钟树上不能有mux,否则需要使用set_case_analysis 来指定,注意需要指定当前mux 的 sel 信号
2. 把与时钟复位无关的模拟模块全部设置为黑盒子
3. 可以使用man xxx 来获取各种命令说明,使用 tab 键可以自动匹配,如 man set_parameter ，
   可以通过 help -params 来得到各种 set_parameter xx 中 xx 的内容;help -params xx 能够得到 xx 使用方法

   可以通过 help -option 来得到各种 set_option xx 中 xx 的内容;
   help -option xx 能够得到 xx 使用方法

4.由于每个阶段使用的参数选项不一样,可能在某些阶段会报 warning 不识别某些参数等,这些并不影响结果

Verdi 篇

产生波形函数

Using Verilog System Tasks（产生波形的那几个函数）

Compilation

This can be done in following two ways:

•% vcs -fsdb [compile_options] verilog_filelist

•For -P tab flow, replace vcsd.tab with novas.tab, where novas.tab is available in:

<NOVAS_INST_DIR>/share/PLI/VCS/${PLATFORM}/novas.tab

The following is the use model change:

% vcs -debug_pp -P $DEBUSSY_LIB/novas.tab $DEBUSSY_LIB/pli.a [compile_options] verilog_filelist

使用Verilog 系统函数需要使用 -P 那个包

但这时不能用 -fsdb 选项（2010.07 版本支持）它需要做如下环境变量设置

% setenv DEBUSSY_HOME Debussy_installation

% setenv DEBUSSY_LIB $DEBUSSY_HOME/share/PLI/VCS/LINUX

% setenv LD_LIBRARY_PATH ${DEBUSSY_HOME}/share/PLI/lib/LINUX:$DEBUSSY_LIB

% setenv LM_LICENSE_FILE[Debussy_license]:$LM_LICENSE_FILE

down fsdb 波形的步骤：

```
initial begin
   $fdsbDumpfile(file_name); //生成波形的名字
```

$fsdbDumpvars(0,tb)//down 波形的层次，写了这句话，就会开始 down 波形;

//0 表示当前层及所有子层 ,1 表示当前层 ,2 表示往下一层,3,4..依此类推.

$fsdbDumpon;

End

Stopping and resuming the dump ($dumpoff/$dumpon)

fsdbDumplimit - 限制 FSDB 文件 size

-- $fsdbDumpvars([], *)

fsdbDumpfile - 指定 FSDB 文件名

-- $fsdbDumpfile(“”)

fsdbDumpvars - Dump 指定的变量

--

fsdbDumpSingle - Dump 指定的信号

fsdbDumpvariable - Dump 指定的 VHDL 变量

fsdbSwitchDumpFile - 将 dumping 切换到另一个 FSDB 文件

-- $fsdbSwitchDumpFile(“”)

fsdbAutoSwitchDumpfile - 限制文件大小并在数据量过大时自动创建新的 FSDB 文件

-- $fsdbAutoSwitchDumpfile(file size,“file_name”,< number of file>)

fsdbDumpflush - Force to Dump Result to FSDB file

fsdbDumpMem - Dump 指定的 memory 的内容

-- $fsdbDumpMem(, [, []])

$fsdbDumpon - 打开 FSDB dumping

$fsdbDumpoff - 关闭 FSDB dumping

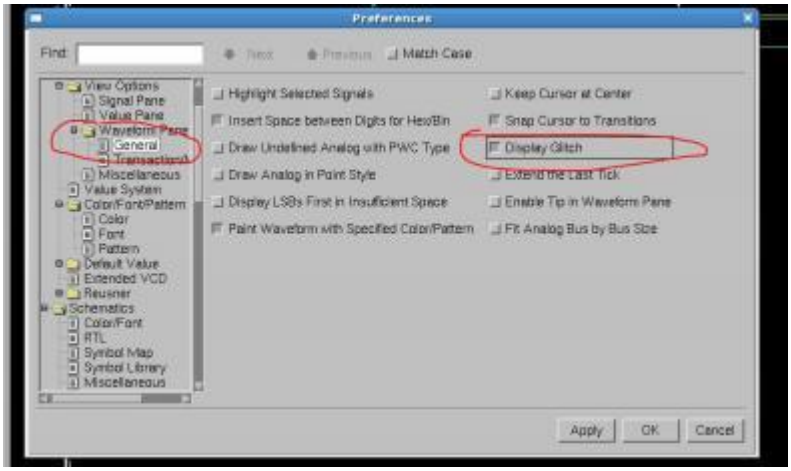
down vcd 波形

```
$dumpfile( "sim_scan_vcd_out");
$dumpvars( 0,top);
```

显示毛刺

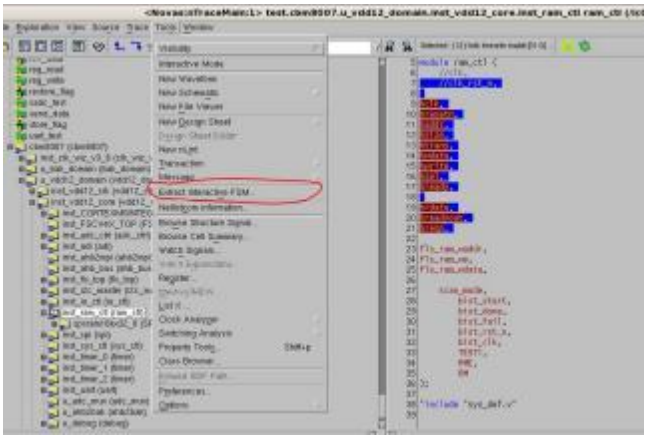
需要在.cshrc 文件中设置环境变量 setenv FSDB_GLITCH 1,

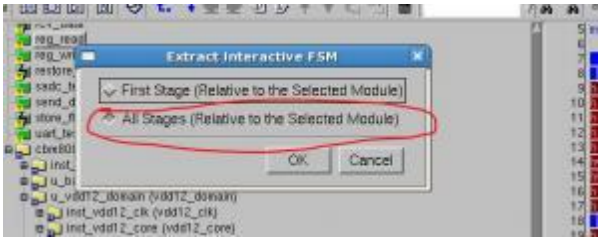
下图中的控制位无效?



如何显示状态机

打开波形,把信号加入,如下步骤





然后波形中的状态机就会以字符的方式显示

如何 dump memory 数据

在仿真中，RTL 中的 Memory模型或寄存器阵列中的数据一般情况下不能直接拉到波形窗口跟踪，通过将 Memory数组先Dump 出来再添加到波形窗口可方便跟踪数组元素的值。

1. 使用该方法 VCS 和 NOVAS 都需要使用新版本才支持

VCS　　E-2011.03-SP1

NOVAS　2011.07

2. 在 dump 波形的地方使用$fsdbDumpMDA()函数把Memory数组Dump 出来。

例如 RTL 中有一数组定义为

reg [7:0] mem[4095:0];

在 dump fsdb 波形的文件中添加

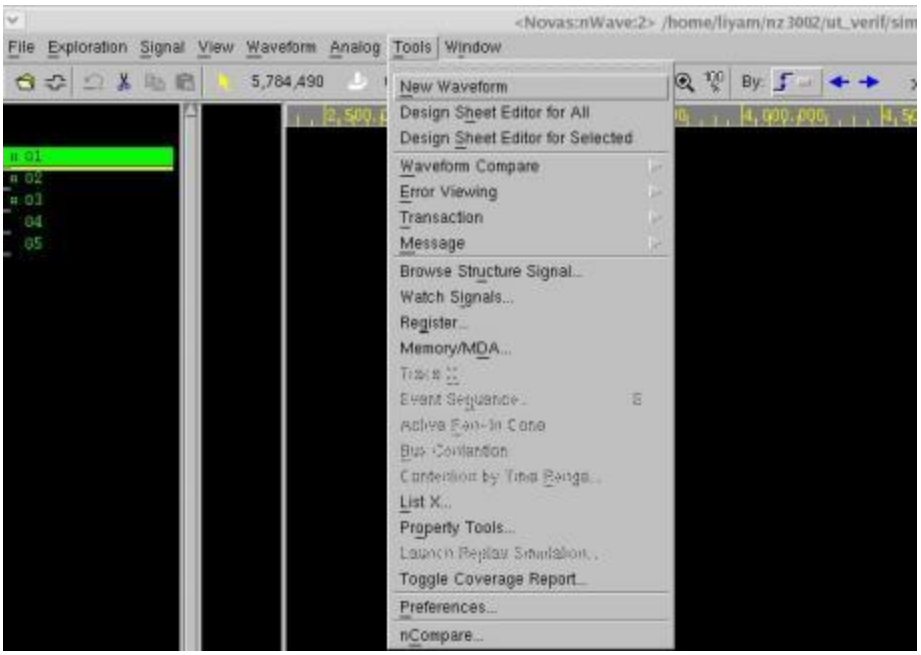
$fsdbDumpMDA(tb_top.uSlave.mem);

1. 关于该函数详细参数用法参考工作站

/opt/othersoft/novas/2011.07/doc/linking_dumping.pdf　　P53

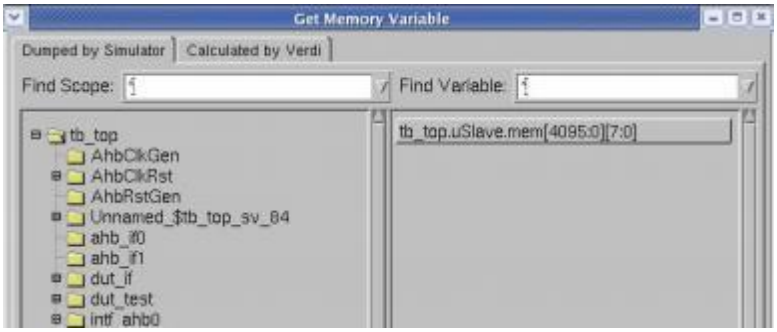
3. 将数组元素添加到 Verdi/Debussy nWave 窗口

3.1 在 nTrace 或 nWave 窗口选择 Tools--Memory/MDA

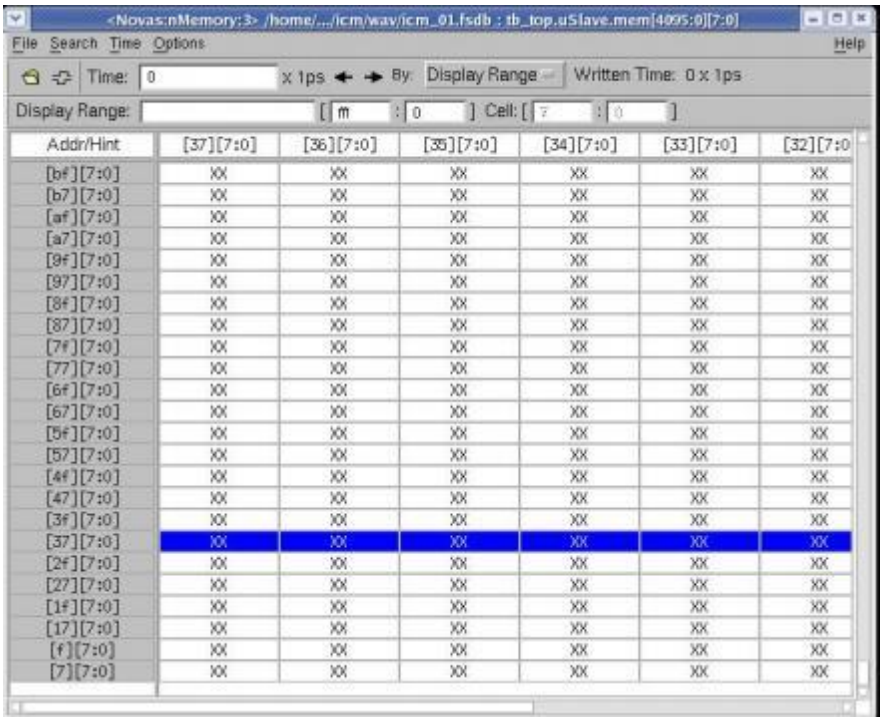


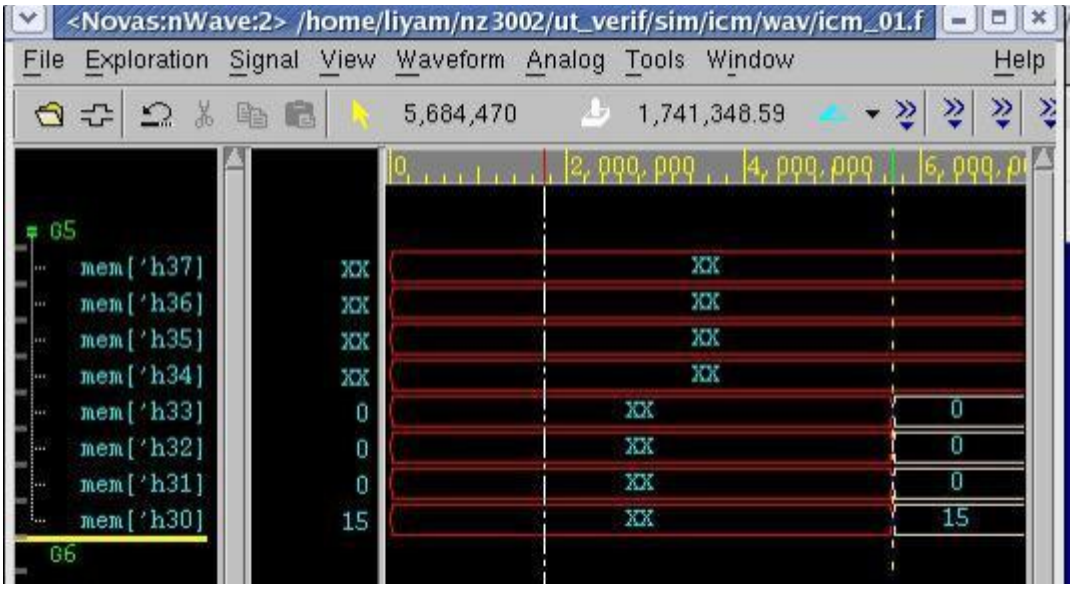
3.2 在弹出的 nMemory 窗口里选择 File—Get Memory Variable

3.3 在 Get Memory Variable 窗口选择之前 Dump 出来的数组



3.4 在 Memory 阵列窗口左键选择需要查看的数组元素，下图中选中了 30h->37h 的元素，按住鼠标的滚轮拖到 nWave 窗口。





4. 该功能可能会使运行速率变慢或运行数据加多，在不需要时，请关闭该功能。另外，可以设置$fsdbDumpMDA( )函数只 Dump 数组中的部分元素，方法自行查看文档研究。

如何設定 Debussy / Verdi 的 port 顏色(转)

1. **Abstract**

在 trace code 時，我們通常會將 input port 與 output port 拉出來 waveform 觀察，了解該 module 的 in out 關係，若有必要再深入到內部的 register, wire 與 state，所以若能將 input / output port 用不同的顏色顯示，有別於 register, wire 與 state，將有助於 trace code 的進行。

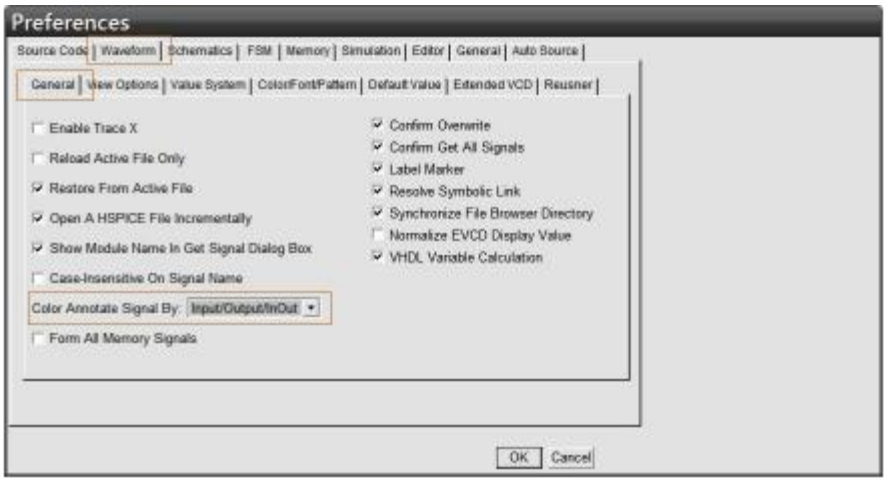
2. **Introduction**

使用環境：Debussy 5.4v9 (Windows 版), Verdi 2010.10 (Linux 版)

3. **Debussy**

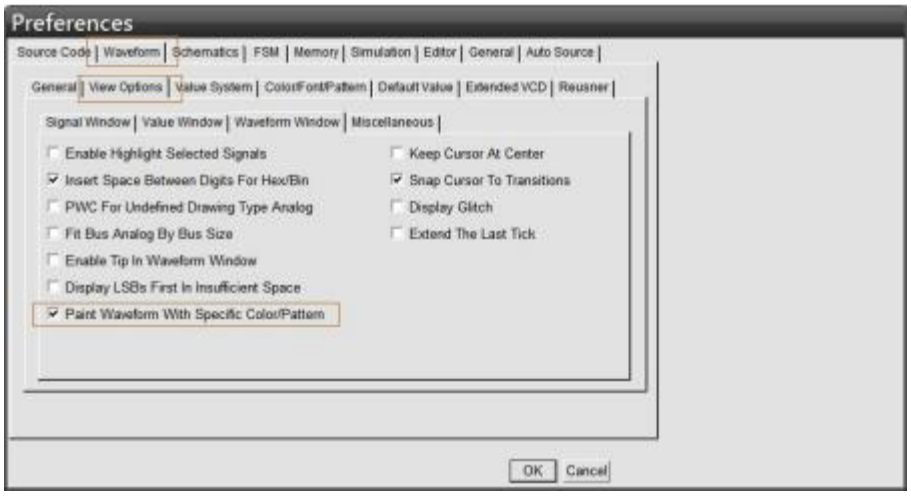
4. **Step 1：設定 Color Annotate Signal By : Input/Output/Inout**

5. Tools –> Preferences –> Waveform Tab –> General Tab

6. 

7. **Step 2：設定 Paint Waveform With Specific Color/ Pattern**

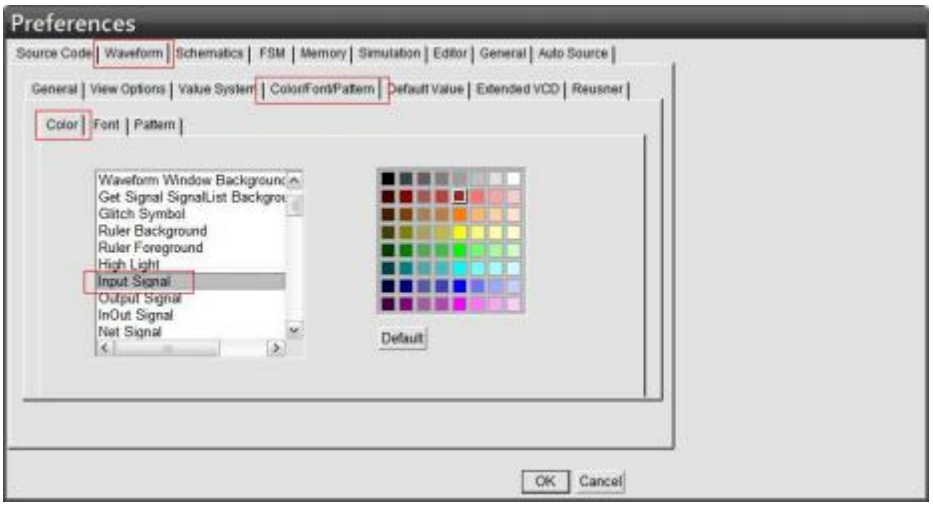
8. Tools –> Preferences –> Waveform Tab –> View Options Tab

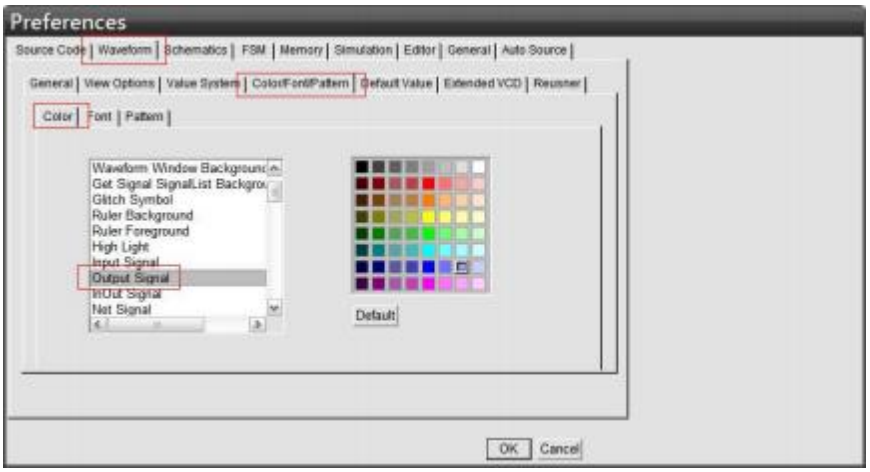
9. 

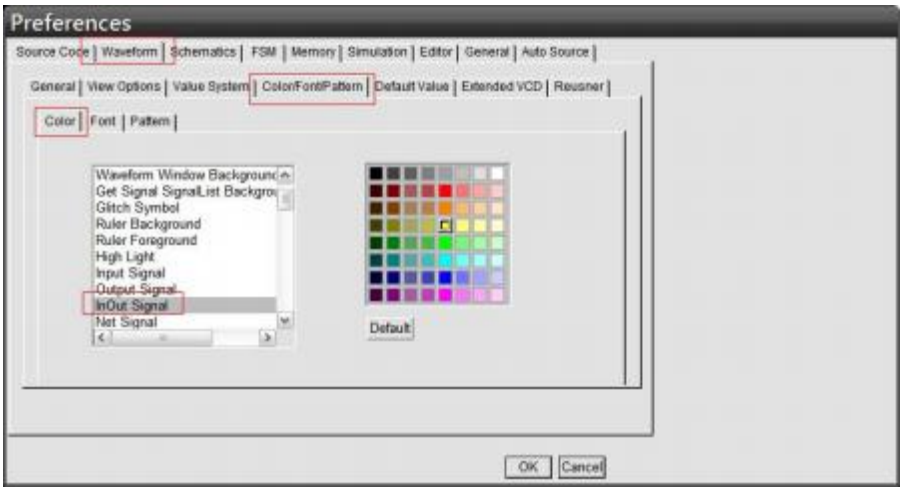
10. **Step 3：設定 input / output / inout port 顏色**

11. Tools -> Preferences -> Waveform Tab -> Color/Font/Pattern Tab –> Color Tab

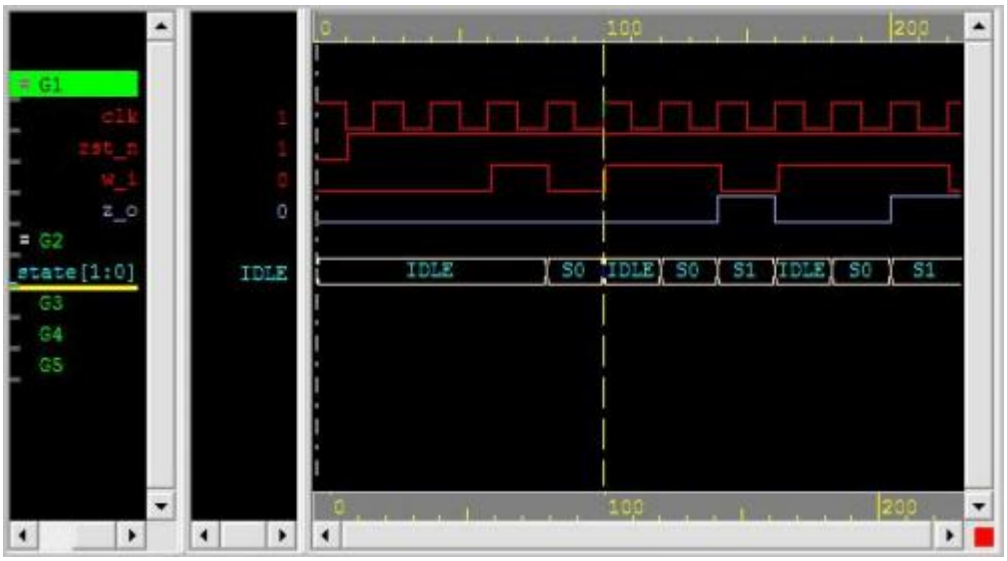
12. 請自行選擇你喜歡的顏色

13. 

14. 

15. 

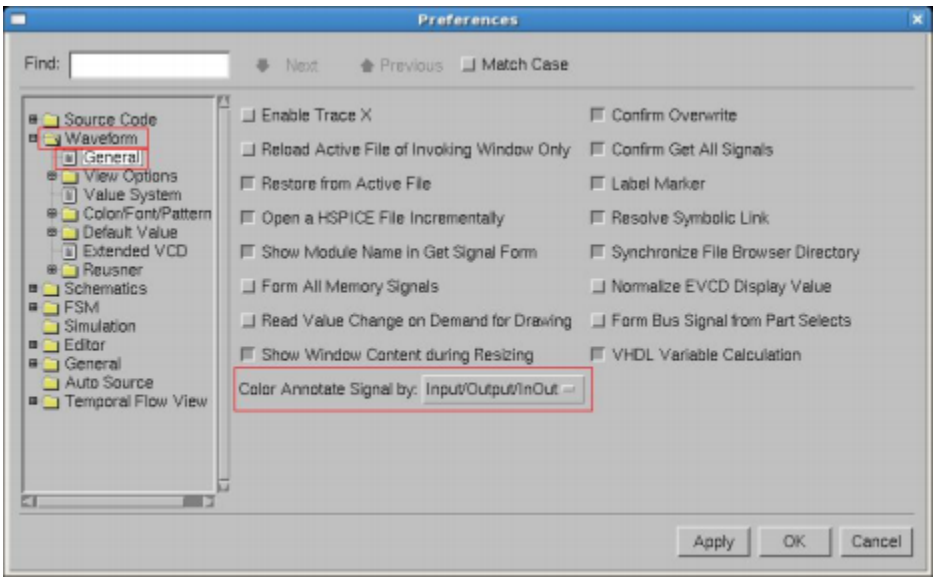
16. **最後結果**

17. 

18. **Verdi**

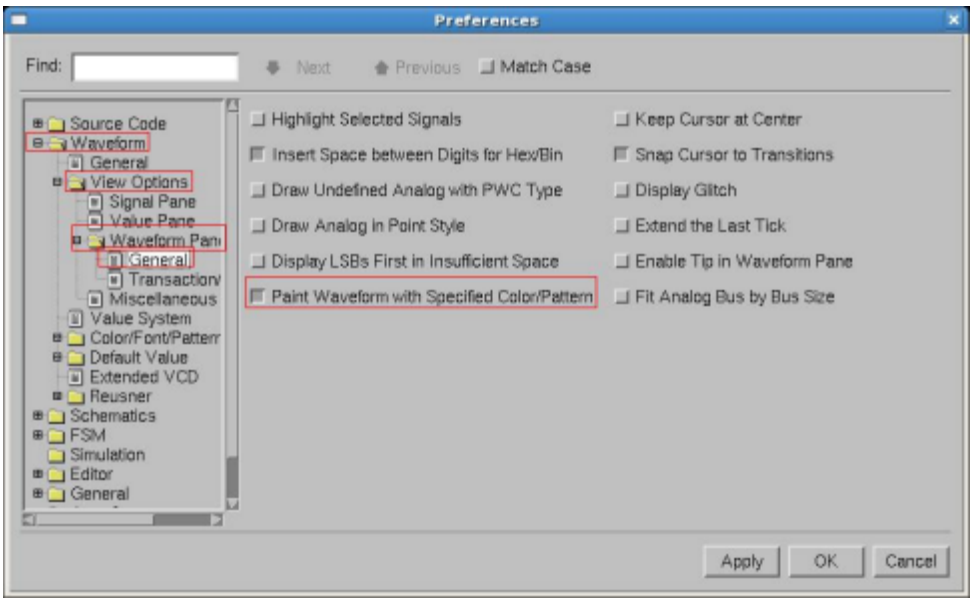
19. **Step 1：設定 Color Annotate Signal By：Input/Output/Inout**

20. Tools –> Preferences –> Waveform –> General

21. 

22. **Step 2：設定 Paint Waveform With Specific Color/Pattern**

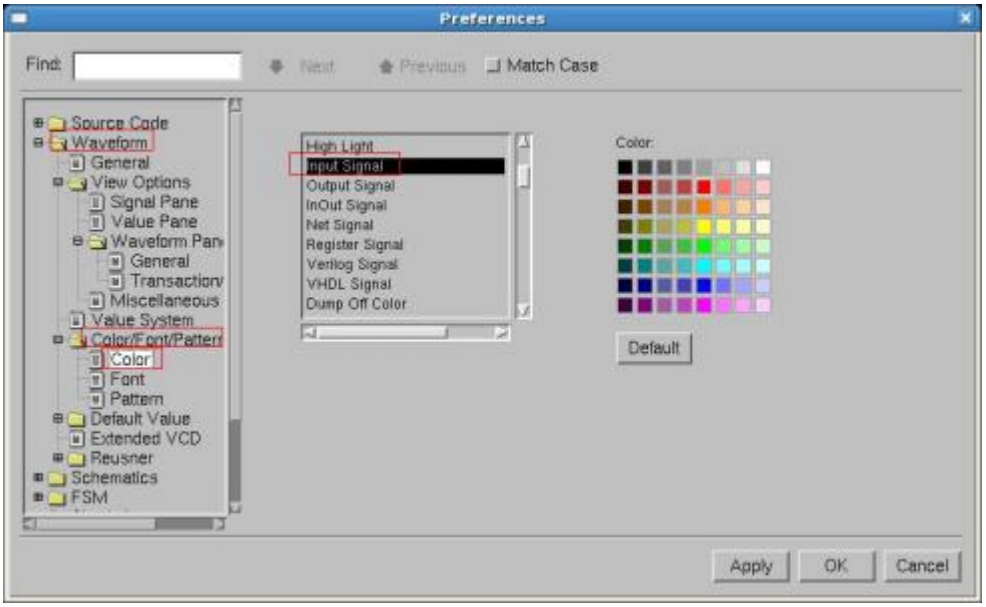
23. Tools –> Preferences –> Waveform –> View Options -> Waveform Panel -> General

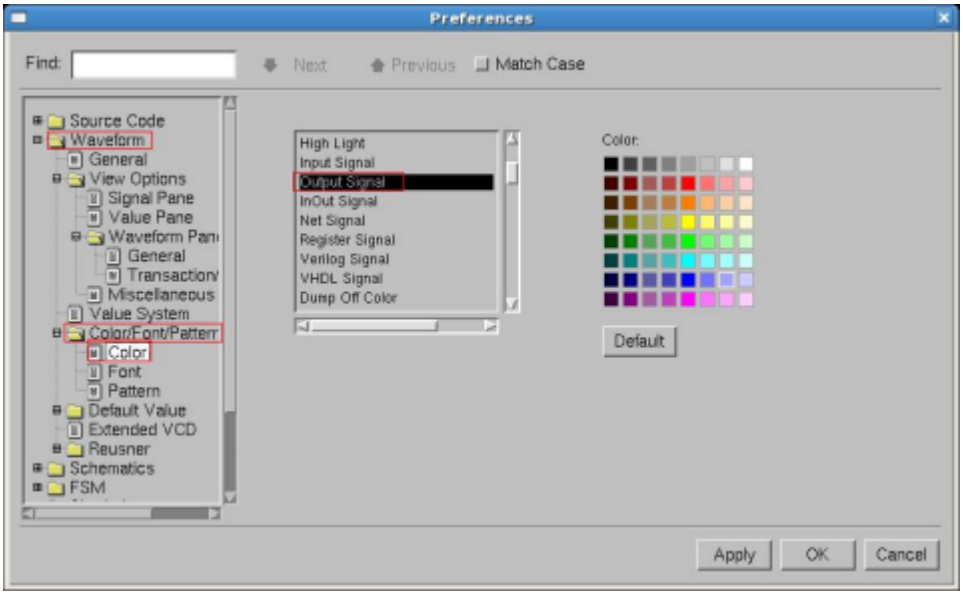
24. 

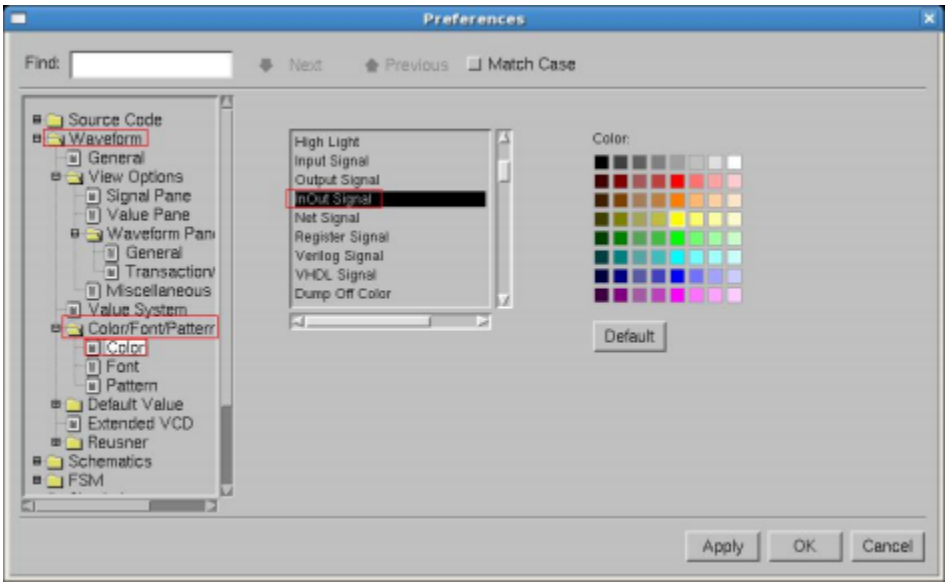
25. **Step 3：設定 input / output / inout port 顏色**

26. Tools -> Preferences -> Waveform -> Color/Font/Pattern–> Color

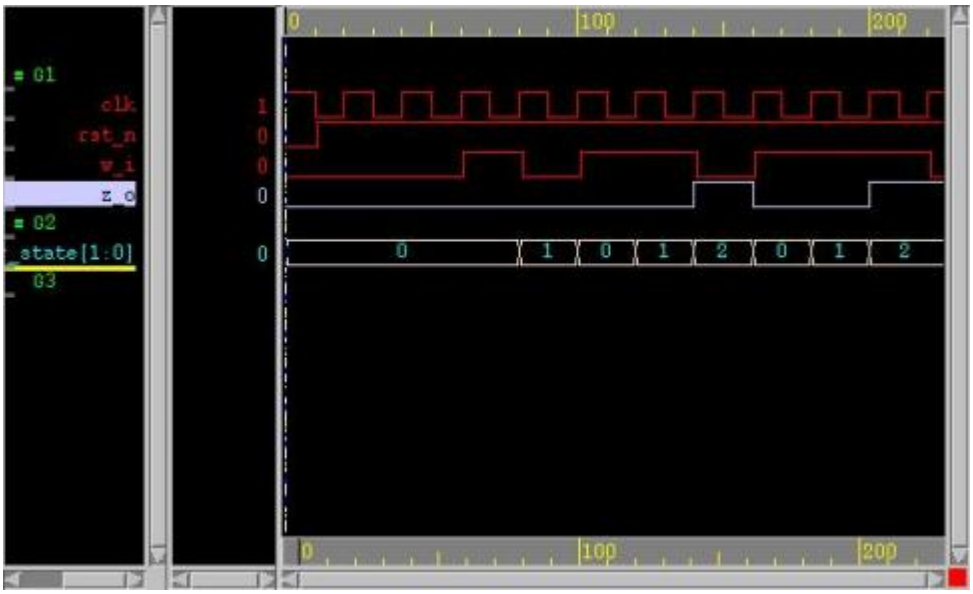
27. 請自行選擇你喜歡的顏色

28. 

29. 
Preferences
Find:
Next
Previous
Match Case
Source Code
Waveform
General
View Options
Signal Pane
Value Pane
Waveform Pan
General
Transaction
Miscellaneous
Value System
Color/Font/Pattern
Color
Font
Pattern
Default Value
Extended VCD
Reusner
Schematics
FSM
High Light
Input Signal
Output Signal
InOut Signal
Net Signal
Register Signal
Verilog Signal
VHDL Signal
Dump Off Color
Color:
Default
Apply
OK
Cancel

30. 
Preferences
Find:
Next
Previous
Match Case
Source Code
Waveform
General
View Options
Signal Pane
Value Pane
Waveform Pan
General
Transaction
Miscellaneous
Value System
Color/Font/Pattern
Color
Font
Pattern
Default Value
Extended VCD
Reusner
Schematics
FSM
High Light
Input Signal
Output Signal
InOut Signal
Net Signal
Register Signal
Verilog Signal
VHDL Signal
Dump Off Color
Color:
Default
Apply
OK
Cancel

31. **最後結果**

32. 

33.

Makefile 篇

1． 变量：

makefile 注释语句的方法是使用##号

无需申明，直接赋值=或追加赋值+=，但使用时需要用$符号；du_cmp = xxx;du_cmp += yyy;ku=$(du_cmp);$符号表示引用程序每行后面没；符号

1. makefile 定义变量直接赋值不需要$符号，当使用赋值给其它变量时需要使用$符号，

name=$(comandname)

myname = $(name)

2. 使用 shell 命令，支持连续命令如 cur_prj = $(shell cd $(cur_dir) ;ls -t *.sv | head -1);head -1 表示显示第一行

3. 也可以使用``来操作 shell 命令如：`cat module_list.f` >module.txt cat：原样显示文件内容

4. 获取路径 PWD

5. 获取时间 var = ″ `data +%Y%m%d%H%M″ 表示年月日时分，注意大小写 M，

6. MAKEFILE 变量赋值时，后面不能带有空格，或许这就是 MAKEFILE 语句后不用符号的原因

7. 当前目录下有可执行文件名与MAKEFILE 的目标名字一致时，会执行当前可执行文件;

8. 如：MAKEFLIE 有命令 regress: $regress_comp $regress_run
当前有可执行文件 regress_run*，这时如果执行 make regress,则 $regress_run 不会执行，而会执行当前目录文件 regress_run;

在 MAKEFILE 的头声明.PHONY:regress_run,可以避免此类问题发生..PHONY 声明了后面的字符为伪命令！！

9. 在 MAKEFILE 中“%”的意思是匹配零或若干字符

dut :compile_01 run_01

comp compile_%   vcs $CMP_OPTIONS

run run_% $RUN_OPTIONS

10. •@开头表示后面的内容不在窗口显示出来

11. makefile 前面的退格不是用空格符而是 TAB 符，必须注意！当 Tab 键被设置为空 4 格时，makefile 中的命令会出错，应该用：set noexpandtab 来取消设置的功能，此时 tab 为机器默认设置

12. 通过在makefile 中添加的文件，它们之间是并行的关系，所以在 tb_top 无法直接链接到 makefile 包含的文件

13. “$@”表示目标的集合，就像一个数组，“$@”依次取出目标，并执于命令。

命令中的“$<”和“$@”是自动化变量，“$<”表示所有的依赖目标集，“$@”表示目标集;（$<对应的是.C 文件，$@对应的是.o 文件）

$@

表示规则中的目标文件集。在模式规则中，如果有多个目标，那么，″$@″就是匹配于目标中模式定义的集合。

$?

所有比目标新的依赖目标的集合。以空格分隔。

$^

所有的依赖目标的集合。以空格分隔。如果在依赖目标中有多个重复的，那个这个变量会去除重复的依赖目标，只保留一份。

```
firmware_update : xxx/rom.code
\ln -sf $^ ./${srom_file};    //$^就是 xxx/rom.code，此处是做一个链接
```

$+

这个变量很像"$^"，也是所有依赖目标的集合。只是它不去除重复的依赖目标。

$(@D)

表示目标文件的目录部分（不包括斜杠）。如果“$@”是“dir/foo.o”，那么“$(@D)”的值为“dir”。如果“$@”不存在斜杠，其值就是“.”（当前目录）。注意它和函数“dir”的区别!

$(@F)

目标文件的完整文件名中除目录以外的部分（实际文件名）。如果“$@”为“dir/foo.o”，那么“$(@F)”只就是“foo.o”。“$(@F)”等价于函数“$(notdir$@)”。

$(*D)

$(*F)

分别代表目标“茎”中的目录部分和文件名部分。

14. $(foreach <var>,<list>,<text>)

这个函数的意思是，把参数<list>中的单词逐一取出放到参数<var>所指定的变量中，然后再执行<text>所包含的表达式。每一次<text>会返回一个字符串，循环过程中，<text>的所返回的每个字符串会以空格分隔，最后当整个循环结束时，<text>所返回的每个字符串所组成的整个字符串（以空格分隔）将会是 foreach 函数的返回值。

所以，<var>最好是一个变量名，<list>可以是一个表达式，而<text>中一般会使用<var>

这个参数来依次枚举<list>中的单词。举个例子：

names := a b c d

files := $(foreach n,$(names),$(n).o)

上面的例子中，$(name)中的单词会被挨个取出，并存到变量“n”中，“$(n).o”每次根

据“$(n)”计算出一个值，这些值以空格分隔，最后作为 foreach 函数的返回，所以，$(f

iles)的值是“a.o b.o c.o d.o”。注意，foreach 中的<var>参数是一个临时的局部变量，foreach 函数执行完后，参数<var>的变量将不在作用，其作用域只在 foreach 函数当中

15. objects = *.o

上面这个例子，表示了，通符同样可以用在变量中。并不是说[*.o]会展开，不！obje

cts 的值就是“*.o”。Makefile 中的变量其实就是 C/C++中的宏。如果你要让通配符在变量中展开，也就是让 objects 的值是所有[.o]的文件名的集合，那么，你可以这样：

objects := $(wildcard *.o)这种用法由关键词“wildcard”指出

16. 文件名操作函数

下面我们要介绍的函数主要是处理文件名的。每个函数的参数字符串都会被当做一个或是一系列的文件名来对待。

$(dir <names...>)

名称：取目录函数——dir。

功能：从文件名序列<names>中取出目录部分。 目录部分是指最后一个反斜杠（“/”）

之前的部分。如果没有反斜杠，那么返回“./”。

返回：返回文件名序列<names>的目录部分。

示例： $(dir src/foo.c hacks)返回值是“src/ ./”。

$(notdir <names...>)

名称：取文件函数——notdir。

功能：从文件名序列<names>中取出非目录部分。非目录部分是指最后一个反斜杠（“
/”）之后的部分。返回：返回文件名序列<names>的非目录部分。

示例： $(notdir src/foo.c hacks)返回值是“foo.c hacks”。

$(suffix <names...>)

名称：取后缀函数——suffix。

功能：从文件名序列<names>中取出各个文件名的后缀。

返回：返回文件名序列<names>的后缀序列，如果文件没有后缀，则返回空字符串。

示例：$(suffix src/foo.c src-1.0/bar.c hacks)返回值是“.c .c”。

$(basename <names...>)

名称：取前缀函数——basename。

功能：从文件名序列<names>中取出各个文件名的前缀部分。

返回：返回文件名序列<names>的前缀序列，如果文件没有前缀，则返回空字符串。

示例：$(basename src/foo.c src-1.0/bar.c hacks)返回值是“src/foo src-1.0/ba
r hacks”。

$(addsuffix <suffix>,<names...>)

名称：加后缀函数——addsuffix。

功能：把后缀<suffix>加到<names>中的每个单词后面。

返回：返回加过后缀的文件名序列。

示例：$(addsuffix .c,foo bar)返回值是“foo.c bar.c”。

$(addprefix <prefix>,<names...>)

名称：加前缀函数——addprefix。

功能：把前缀<prefix>加到<names>中的每个单词前面。

返回：返回加过前缀的文件名序列。

示例：$(addprefix src/,foo bar)返回值是“src/foo src/bar”。

$(join <list1>,<list2>)

名称：连接函数——join。

功能：把<list2>中的单词对应地加到<list1>的单词后面。如果<list1>的单词个数要

比<list2>的多，那么，<list1>中的多出来的单词将保持原样。如果<list2>的单词个数要

比<list1>多，那么，<list2>多出来的单词将被复制到<list2>中。返回：

返回连接过后的字符串。示例：$(join aaa bbb , 111 222 333)返回值是“aaa111 bbb222 333”。

字符串处理函数；其它的见 MAKEFILE 教程

$(subst <from>,<to>,<text>)

名称：字符串替换函数——subst。

功能：把字符串<text>中的<from>字符串替换成<to>。

返回：函数返回被替换过后的字符串。

示例：

$(subst ee,EE,feet on the street),

把“feet on the street”中的“ee”替换成“EE”，返回结果是“fEEt on the

strEEt”。

$(patsubst <pattern>,<replacement>,<text>)

名称：模式字符串替换函数——patsubst。

功能：查找<text>中的单词（单词以“空格”、“Tab”或“回车”“换行”分隔）是

否符合模式<pattern>，如果匹配的话，则以<replacement>替换。这里，<pattern>可以包

括通配符“%”，表示任意长度的字符串。如果<replacement>中也包含“%”，那么，<repl

acement>中的这个“%”将是<pattern>中的那个“%”所代表的字符串。（可以用“\”来转义，以“\%”来表示真实含义的“%”字符）

返回：函数返回被替换过后的字符串。

示例：

$(patsubst %.c,%.o,x.c.c bar.c)

把字符串“x.c.c bar.c”符合模式[%.c]的单词替换成[%.o]，返回结果是“x.c.o bar.o”

17. makefile 支持命令行 shell 命令，如 cur_dir:=$(shell pwd) 其中 pwd 会返回当前的目录路径，表示在命令行输入 pwd 把返回的结果返回给变量 cur_dir;

18. makefile 常用函数：

$(words TEXT) 统计单词数目函数—words 示例： $(words, foo bar) 返回值是“2”

$(word N,TEXT) 取单词函数—word，取字串“TEXT”中第“N”个单词（“N”的值从 1 开始）。$(word 2, foo bar baz) 返回值为“bar”。

$(wordlist S,E,TEXT) 取字串函数—wordlist 从字串“TEXT”中取出从“S”开始到“E”的单词串。“S”和“E”表示单词在字串中位置的数字。$(wordlist 2, 3, foo bar baz) 返回值为：“bar baz”。

$(firstword NAMES…) 取首单词函数—firstword 取字串“NAMES…”中的第一个单词。$(firstword foo bar) 返回值为“foo”。函数“firstword”实现的功能等效于“$(word 1, NAMES…)”。

$(subst a,b,strings) 替换函数把 strings 中的 a 用 b 替换掉，$(subst /, ,user/xxx/svn/ver),返回值为 user xxx svn ver

$(strip STRINT) 函数名称：去空格函数—strip。函数功能：去掉字串（若干单词，使用若干空字符分割）“STRINT”开头和结尾的空字符，并将其中多个连续空字符合并为一个空字符。返回值：无前导和结尾空字符、使用单一空格分割的多单词字符串。函数说明：空字符包括空格、[Tab]等不可显示字符。

示例：STR = a b c LOSTR = $(strip $(STR))结果是“a b c”。

19. 关键字“ifdef”用来判断一个变量是否已经定义。Name = keke;ifdef name ;返回为真；mame = ;ifdef name 返回为假 ifdef 只是检查变量

右边是否有东西，如 myname = ;name = $(myname); ifdef name 返回为真。

Makefile 規則的語法格式：

目標文件列表：依賴文件列表

〈tab 〉命令列表

一個 Makefile 文件主要含有一系列的規則，每條規則包含以下內容。

“ 目標文件列表 " ， 即 make 最終需要創建的文件，如可執行文件和目標文件
; 目標也可以是要執行的動作，如“clean”。

“依賴文件列表 " ，通常是編譯目標文件所需要的其他文件。

“命今列表 " ， 是 make 執行的動作，通常是把指定的相關文件編譯成目標文
件的編譯命令，每個命令占一行，且每個命令行的起始字符必須為 TAB 字符。

除非特別指定，否則make 的工作目錄就是當前目錄。“ 目標文件列表 ″ 是需要創建的二進制文件或目標文件，依賴文件列表是在創建“ 目標文件列表 " 時需要用到的一個或多個文件的列表，命令序列是創建“ 目標文件列表 " 文件所需要執行的步驟，比如編譯命令。

2．范例 1

```
#-------------------------------------------------------------------
# DIRECTORIES
#-------------------------------------------------------------
```

```
VER_DIR              = ../../../..
SIM_DIR              = ../../../sim
TB_DIR               = ../../../tb
TC_DIR               = ../../../tc
RM_DIR               = ../../../rm
SOFT_DIR             = ../../../software

OUTPUT_DIR           = $(SIM_DIR)/$(DUT)/output
COV_DIR              = $(SIM_DIR)/$(DUT)/cover
LOG_DIR              = $(SIM_DIR)/$(DUT)/logs
WAV_DIR              = $(SIM_DIR)/$(DUT)/wav
#--------------------------------------------------------------------------
# CMD OPTIONS
# -------------------------------------------------------------------------
# TEST768
ifeq ($(test768),1)
sftest =1
CFG +=+define+TEST768
endif

fsdb = 1
# fsdb =1, dump the fsdb wave, fsdb=0, don't dump the fsdb wave
ifeq ($(fsdb),1)
CFG +=+define+FSDB
endif
# vcd =1, dump the vcd wave, vcd=0, don't dump the vcd wave
ifeq ($(vcd),1)
CFG +=+define+VCD
endif
# vpd =1, dump the vpd wave, vpd=0, don't dump the vpd wave
```

```
ifeq ($(vpd),1)
CFG +=+define+VPD
endif
# enc=1,the system add the encoder and decoder function, enc=0, don't
enc = 1
ifeq ($(enc},1)
CFG +=+define+ENCDEC
endif
# efs
ifeq ($(efs},1)
fpga =1
CFG +=+define+EFS
endif

# sftest
ifeq ($(sftest},1)
fpga =1
CFG +=+define+SFTEST
endif
# fpga=1, FPGA sim, enc=0, don't
ifeq ($(fpga},1)
CFG +=+define+FPGA
endif

pva = 1
ifeq ($(pva),1)
DBG_CMP_OPTIONS    +=
-pvalue+PRJXXXX_tb_top.uDut.uPrjxxxx_core.uScu.PM_BIST1EXTCYCLE=150
00
CFG +=+define+PVA
```

```
endif
#-------------------------------------------------------------------------
# DUT Files
#-------------------------------------------------------------------------
DUT_RTL_DIR    = $(VER_DIR)/dut/rtl
# SYS DUT
DUT_SRC_RTL       += $(VER_DIR)/dut/rtl/include/Prjxxxx_include_rtl.v
DUT_SRC_SYN       += $(VER_DIR)/dut/synnet/Prjxxxx_include_syn.v
DUT_SRC_PR        += $(VER_DIR)/dut/rtl/include/Prjxxxx_include_rtl.v

SIM_DUT =SIM_$(SIM)

OSC_OPT=FASTOSC#FASTOSC TYPEOSC SLOWOSC
SDF_OPT=WCSDF#WCSDF   TYSDF   BCSDF

ifeq ($(SDF_OPT),BCSDF)
DELAY_OPT =mindelays
else
ifeq ($(SDF_OPT),WCSDF)
DELAY_OPT =maxdelays
else
DELAY_OPT =typedelays
endif
endif

ifeq ($(OSC_OPT),FASTOSC)
DBG_CMP_OPTIONS    +=
-pvalue+PRJXXXX_tb_top.uDut.uPrjxxxx_core.uOSC32M.PERIOD32=29.7
else
ifeq ($(OSC_OPT),SLOWOSC)
```

```
DBG_CMP_OPTIONS    +=
-pvalue+PRJXXXX_tb_top.uDut.uPrjxxxx_core.uOSC32M.PERIOD32=38.0
else
DBG_CMP_OPTIONS    +=
-pvalue+PRJXXXX_tb_top.uDut.uPrjxxxx_core.uOSC32M.PERIOD32=31.25
endif
endif



FPGA_OPT = FPGASIM

ifeq (${SIM_DUT},SIM_SYN)
DUT_SRC_SYS += $(DUT_SRC_SYN)
else
ifeq (${SIM_DUT},SIM_PR)
DUT_SRC_SYS += $(DUT_SRC_PR)
else
DUT_SRC_SYS += $(DUT_SRC_RTL)
endif
endif

DUT_SRC        += $(DUT_SRC_SYS)
DUT_SRC        +=
-v ../../../../../../Lib/Standard_Cell/hhnec/scs8lpa/Design_Kit/scs8lpa_v1.2/
VERILOG/scs8lp.v
DUT_SRC        +=
-v ../../../../../../Lib/IO/hhnec/ef13io5vh4kvinl_poc_v1.2/VERILOG/ef13io5v
h4kvinl_poc.v
```

```
DUT_SRC        +=
-v ../../../../../../Lib/IP/PRJXXX/scs8lp_nand2_1_cap/verilog/scs8lp_nand2_1
_cap.v
DUT_SRC       += -v $(RM_DIR)/lib/verilog/ef13io5vh4kvinl_poc_changed.v
DUT_SRC       += -v $(RM_DIR)/lib/verilog/ClkModel.sv
ifeq (${SIM_DUT},SIM_PR)
DUT_SRC       += -v ../../../../dut/rtl/mesh/Mesh_Analog.v
endif
DUT_INC       +=
+incdir+$(DUT_RTL_DIR)/core/rtl/+$(DUT_RTL_DIR)/include
DUT_SRC      += $(DUT_INC)
#--------------------------------------------------------------------------
# TESTBENCH Files
#--------------------------------------------------------------------------
TB_SRC    += $(TB_DIR)/top/prjxxxx_tb_top.sv
TB_SRC    += $(RM_DIR)/interface/filelist.sv
TB_INC    += +incdir+$(TC_DIR)/$(DUT)/
TB_INC    += +incdir+$(RM_DIR)/interface/usb
TB_INC   += +incdir+$(RM_DIR)/interface/model
TB_INC   += +incdir+$(RM_DIR)/interface/testcase/spi/
TB_INC   += +incdir+$(RM_DIR)/interface/testcase/scc/
TB_SRC    += $(TB_INC)
#--------------------------------------------------------------------------
# SOFTWARE INCLUDE
#----------------------------------------EmfEfc_Crdy-----------------------------
FLASH_HEX     = $(SOFT_DIR)/dua/$(DUT)/flash.DUA

################################################EmfEfc_Crd
y################################
# DUT Compile and run options
```

```
##########################################################
######################
DIR        = $(shell cd ..; pwd)
DUT_SIM_EXEC       += $(OUTPUT_DIR)/$(TC_TEST)_simv

DUT_CMP_OPTIONS    += -full64
DUT_CMP_OPTIONS    += -sverilog +v2k -lca
DUT_CMP_OPTIONS    += -debug_all -fsdb
DUT_CMP_OPTIONS   += +vcs+lic+wait +plusarg_save
DUT_CMP_OPTIONS   += +libext+.sv+libext+.v
DUT_CMP_OPTIONS   += +verilog2001ext+.v +systemverilogext+.sv

ifeq (${SIM_DUT},SIM_SYN)
DUT_CMP_OPTIONS += +define+SYNSIM
else
ifeq (${SIM_DUT},SIM_PR)
DUT_CMP_OPTIONS += +define+PRSIM+$(OSC_OPT)+$(SDF_OPT)
DUT_CMP_OPTIONS += +$(DELAY_OPT) +neg_tchk +overlap +sdfverbose
-negdelay

DUT_CMP_OPTIONS +=
+warn=noSDFCOM_NICD,noSDFCOM_ANICD,noSDFCOM_NL,noSDFCOM_NTC
DTL,noWSUM
DUT_CMP_OPTIONS   += +optconfigfile+$(SIM_DIR)/regfile.config
else
ifdef fpga
DUT_CMP_OPTIONS += +define+$(FPGA_OPT)
else
DUT_CMP_OPTIONS += +warn=noTCL-NTCL
DUT_CMP_OPTIONS += +define+SIM
```

```
endif
endif
endif
DUT_CMP_OPTIONS    += $(CFG)
DUT_CMP_OPTIONS    += -o $(DUT_SIM_EXEC)
DUT_CMP_OPTIONS   += -Mdir=$(OUTPUT_DIR)/$(TC_TEST)_csrc
DUT_CMP_OPTIONS   += -l $(LOG_DIR)/$(TC_TEST).cmp_log


DUT_SIM_OPTIONS   += -l $(LOG_DIR)/$(TC_TEST).run_log
DUT_SIM_OPTIONS   += +ntb_random_seed=$(SEED)

############################################################
######################
# TESTBENCH OPTIONS
############################################################
######################
ifeq ($(SIM),PR)
TC_TEST               =
$(DUT)_$(TC_ID)_$(SIM_DUT)_$(SDF_OPT)_$(OSC_OPT)
else
ifeq ${fpga},1)
TC_TEST               = $(DUT)_fpga_$(TC_ID)
else
TC_TEST               = $(DUT)_$(TC_ID)
endif
endif
TB_CMP_OPTIONS    += -timescale=1ns/10ps
TB_CMP_OPTIONS    += -ntb_opts rvm
# ------------------------------------------------------------
```

```
# DEBUG OPTIONS
#--------------------------------------------------------------------------
VCD_FILE      = $(WAV_DIR)/$(TC_TEST).vcd
VPD_FILE      = $(WAV_DIR)/$(TC_TEST).vpd
FSDB_FILE     = $(WAV_DIR)/$(TC_TEST).fsdb

ifdef vcd
DBG_CMP_OPTIONS  += +define+VCD_FILE=$(VCD_FILE)
endif
ifdef vpd
DBG_CMP_OPTIONS  += +define+VPD_FILE=$(VPD_FILE)
endif
ifdeffsdb
DBG_CMP_OPTIONS  += +define+FSDB_FILE=$(FSDB_FILE)
endif

DBG_CMP_OPTIONS  += +define+FLASH_HEX=$(FLASH_HEX)
DBG_CMP_OPTIONS  += -pvalue+PRJXXXX_tb_top.TESTCASE=$(TC_ID)



#--------------------------------------------------------------------------
# VERDI OPTIONS
#--------------------------------------------------------------------------
VER_CMP_OPTIONS += -sv +v2k
VER_CMP_OPTIONS += +libext+.v +libext+.sv
VER_CMP_OPTIONS += +systemverilogext+.sv +verilog2001ext+.v
VER_CMP_OPTIONS += -ssf $(WAV_DIR)/$(TC_TEST).fsdb
ifeq ($(SIM_DUT),SIM_PR)
VER_CMP_OPTIONS += +define+PRSIM+$(OSC_OPT)+$(SDF_OPT)
VER_CMP_OPTIONS += +$(DELAY_OPT) +neg_tchk +overlap +sdfverbose
```

```
else
ifdef fpga
VER_CMP_OPTIONS += +define+$(FPGA_OPT)
else
VER_CMP_OPTIONS += +define+SIM
endif
endif
VER_CMP_OPTIONS += $(CFG)
#-----------------------------------------------------------------
#    Coverage Options Setting
#-----------------------------------------------------------------
COV_TREE          +='+tree PRJXXXX_tb_top.uDut'
COV_HIER          += $(OUTPUT_DIR)/vcm.cfg
COV_CM_OPTIONS    += -lca
COV_CM_OPTIONS    += -cm line+tgl+cond+fsm
COV_CM_OPTIONS    += -cm_log $(COV_DIR)/$(TC_TEST).cm_log
COV_CM_OPTIONS    += -cm_dir $(COV_DIR)/regress/$(DUT)
COV_CM_OPTIONS    += -cm_name $(TC_TEST)_$(SEED)

#CM Hierrarchy Setting
COV_CMP_OPTIONS += -cm_hier $(COV_HIER)
COV_CMP_OPTIONS += $(COV_CM_OPTIONS)

COV_SIM_OPTIONS += $(COV_CM_OPTIONS)
COV_SIM_OPTIONS += $(COV_CM_REG)
COV_REG_DATA      += -dir $(COV_DIR)/regress/$(DUT).vdb


#-----------------------------------------------------------------
#Simulation Seed Setting
```

```
#-----------------------------------------------------------------
SEED = $(shell date "+%H%M%S")

#################################################################
######################
#
# DEBUG TARGETS
#
#################################################################
######################
$(DUT): $(DUT)_$(TC_ID)

$(DUT)_$(TC_ID): compile_$(TC_ID)   run_$(TC_ID)

comp compile_%:
  vcs $(TB_CMP_OPTIONS) $(DUT_CMP_OPTIONS) $(DUT_SRC) $(TB_SRC)
$(DBG_CMP_OPTIONS)

run run_%:
  $(DUT_SIM_EXEC) $(DUT_SIM_OPTIONS)

gui gui_%:
  $(DUT_SIM_EXEC) $(DUT_SIM_OPTIONS) -gui &

deb_%:
  verdi $(VER_CMP_OPTIONS) $(DUT_SRC) $(TB_SRC) &

deb: deb_%
verdi: deb_%
```

```
pp_%:
  dve -vpd $(WAV_DIR)/$(TC_TEST).vpd
dve: pp_%

urg:
  urg $(COV_REG_DATA) -grade -report $(COV_DIR)/regress/urgReport
  konqueror $(COV_DIR)/regress/urgReport/dashboard.html &
ana:
  analyze_log.pl $(LOG_DIR)/$(TC_TEST).run_log
  @gvim $(LOG_DIR)/$(TC_TEST).run_log.timing &
##########################################################
######################
# REGRESSION TARGETS
##########################################################
######################
regress: regress_compile regress_run

regress_compile:
  @echo $(COV_TREE) > $(COV_HIER)
  vcs $(TB_CMP_OPTIONS) $(DUT_CMP_OPTIONS) $(DUT_SRC) $(TB_SRC)
$(DBG_CMP_OPTIONS) $(COV_CMP_OPTIONS)
#Regress Run Target
regress_run:
  $(DUT_SIM_EXEC) $(DUT_SIM_OPTIONS) $(COV_SIM_OPTIONS)
#Regress Clean Target
regress_clean:
  @\rm -rf $(OUTPUT_DIR)/*

regress_urg:
  urg $(COV_REG_DATA) -grade -report $(COV_DIR)/regress/urgReport
```

```
  konqueror $(COV_DIR)/regress/urgReport/dashboard.html &
##########################################################
######################
#
# ADMINISTRATIVE
#
##########################################################
######################

help:
  @echo
==========================================================
============
  @echo  "                                          "
  @echo  " USAGE: %make target_name_* <SEED=xxx>
"
  @echo  "                                          "
  @echo  " ------------------------ DEBUG TARGETS ----------------------------"
  @echo  " target_name_*    => Compile TB and DUT files, runs the
simulation.   "
  @echo  " clean                => Clean the intermediate files.
"
  @echo  " compile_*            => Compile the TB and DUT.
"
  @echo  " run_*                => Run the simulation.
"
  @echo  " clean_sde            => Clean the intermediate files.
"
  @echo  " gui_*                => Run simulation interactively with DVE.
"
```

```
	@echo "pp_*                    => Post process VPD with DVE.
"
	@echo "urg                     => Make a coverage report for debug runs.
"
	@echo "
"
	@echo "---------------------- REGRESSION TARGETS -----------------------"
	@echo "regress_test_*   => Compile and run with coverage.
"
	@echo "regress_clean    => Remove all coverage files.
"
	@echo "regress_build_* => Build test_*.
"
	@echo "regress_run_*    => Run test* collecting coverage information.
"
	@echo "regress_urg      => Make a coverage report for regression runs.
"
	@echo "
"
	@echo "--------------------- ADMINISTRATIVE TARGETS ----------------------"
	@echo "help                    => Displays this message.
"
	@echo "init                    => Clean all files, including coverage files.
"
	@echo "tar                     => Tar and zip kit and place at ../
"
	@echo "                                                  "
	@echo "e.g.  make usb tc=01
"
```

```
	@echo
==========================================================
============

tar:	clean
	cd ..; \
	tar cvf ${DIR}.tar ${DIR}; \
	rm -f ${DIR}.tgz; \
	gzip ${DIR}.tar; \
	mv ${DIR}.tar.gz ${DIR}.tgz

init: clean
	@mkdir $(DUT)
	@mkdir $(DUT)/cover
	@mkdir $(DUT)/output
	@mkdir $(DUT)/logs
	@mkdir $(DUT)/wav
	@mkdir $(DUT)/run
	@mkdir ../tc/$(DUT)
	@mkdir ../software/dua/$(DUT)
	@cp usb/run/Makefile $(DUT)/run
	@cp usb/run/analyze_log.pl $(DUT)/run
clean:
	@rm -rf $(OUTPUT_DIR)/*
cpana:
	@cp ../../usb/run/analyze_log.pl ./
```

3．范例 2

```
SV_ROOT = $(shell cd ../; pwd)
```

```
SV_SIM_ROOT = $(SV_ROOT)/sim
SV_TC_ROOT = $(SV_SIM_ROOT)/testcases

TC_LIST = $(shell cd $(SV_TC_ROOT); ls -I *.pl)

tc=test_spi_quad
env=ic
post=max
sim=vcs
srom_file=rom.code

help:
  @echo -e "\033[40;32m
**********************************************************************
**********************************************************************
****
 runtest_nc = ncverilog -sv +ncaccess+r+w +nc64bit +notimingchecks
+delay_mode_zero +nowarnCUVWSP +nowarnAGSOPRO
+loadpli1=debpli:novas_pli_boot +nctimescale+1ns/10ps
runtest_nc_lp = irun +ncaccess+r+w +nc64bit +notimingcheck
+delay_mode_unit +ncntcnotchks +nowarnCUVWSP +nowarnAGSOPRO
+ncprofile

runtest_vcs = vcs -R -sverilog  -full64 -debug_all +v2k +lint=TFIPC-L
-timescale=1ns/1ps +delay_mode_zero +plusarg_save -Mupdate -I
+notimingcheck -P
/ic01/eda/novas/verdi201101/share/PLI/VCS/LINUX64/novas.tab
/ic01/eda/novas/verdi201101/share/PLI/VCS/LINUX64/pli.a
runtest_lp = ${runtest_nc} \
```

```
  +nclps_cpf+../cadence/cpf/cbm5072_forsim.cpf +nclps_simctrl_on
+nclps_stime+1ps \
  +nclps_verbose+1 +nclps_logfile+./log/lps.log
runtest_vcs_post = vcs -R -sverilog -full64 -sv +v2k +${post}delays
+optconfigfile+regfile.config -timescale=1ns/1ps +plusarg_save
-Mupdate -I -P
/ic01/eda/novas/verdi201101/share/PLI/VCS/LINUX64/novas.tab
/ic01/eda/novas/verdi201101/share/PLI/VCS/LINUX64/pli.a
runtest_nc_post = ncverilog +ncaccess+r+w +nc64bit +nowarnCUVWSP
+nowarnAGSOPRO +ncprofile


######################################################################
  ./testcases/cbm9999_cociscasm.pl ./testcases/${tc}/firmware/main.
asm;
firmware_update :
/xxx/projects/PROJXXX/lib/mem/srom4kx32_tapeout20180706/rom.code
    \ln -sf $^ ./${srom_file};
######################################################################
##########
cmdfiles_gen : ./testcases/${tc}/verilog/cmd_files_tc
    if [ ${env} == "fpga" ]; then
cat ../filelist/sim/cmd_files_fpga > ./cmd_files ; \
    else cat ../filelist/sim/cmd_files_ic > ./cmd_files ; \
    fi ; \
    cat ./testcases/${tc}/verilog/cmd_files_tc >> ./cmd_files ; \
    chmod 755 ./cmd_files ;

######################################################################
```

```
cmdfiles_gsim_gen :

cat ../filelist/sim/cmd_files_ic_post ./testcases/${tc}/verilog/cm
d_files_tc > ./cmd_files_gsim; \
      chmod 755 ./cmd_files_gsim ;
#####################################################################
pre_sim : firmware_update cmdfiles_gen
  @if [ -e ./testcases/${tc}/presim_script ];
then ./testcases/${tc}/presim_script; fi;

pre_postsim : firmware_update cmdfiles_gsim_gen
  if [ -e ./testcases/${tc}/presim_script ];
then ./testcases/${tc}/presim_script; fi; \

post_sim :
  if [ -e ./testcases/${tc}/postsim_script ];
then ./testcases/${tc}/postsim_script; fi;

post_postsim :
  if [ -e ./testcases/${tc}/postsim_script ];
then ./testcases/${tc}/postsim_script; fi;

#####################################################################
nc : pre_sim
  ${runtest_nc} -f cmd_files -l log/${tc}.log; \
           make post_sim;
    ################################################################
vcs : pre_sim
  ${runtest_vcs} -f cmd_files -l log/${tc}.log; \
```

```
    make post_sim;
vcs_fpga_gate  : pre_sim
  ${runtest_vcs} +define+FPGA_GATE -f
cmd_files_fpga_gate ./model/spim_quad/spim_quad.sv -l log/${tc}.log;
\
    make post_sim;
nc_run  :
  ${runtest_nc} -f cmd_files -l nc.log;
#########################################################################
vcs_post  : pre_postsim
  ${runtest_vcs_post} +define+gate_sim_${post} +define+postsim -f
cmd_files_gsim -l log/${tc}_post_${post}.log; \
    make post_postsim;

#########################################################################
nc_post   : pre_postsim
  ${runtest_nc_post} +${post}delays +define+gate_sim_${post} -f
cmd_files_gsim -l log/${tc}_post_${post}.log; \
    make post_postsim;

#########################################################################
clean  :
  \rm -rf simv simv.daidir/ csrc/ INCA_libs/
#########################################################################
##########makelowpower sim#######################################
#########################################################################
nc_lp_all  : pre_sim
  make pre_sim tc=test_lpall  ; \
```

```
  ${runtest_nc_lp} +define+CPU_PD_EN
-lps_cpf ../cadence/cpf/projxxx_rtl.cpf -lps_simctrl_on -lps_pmode
-lps_stime 10000ns  -lps_verbose 1 -f cmd_files -l log/${tc}.log  ;
\
    make post_sim;

nc_lp_osconly  : pre_sim
  make pre_sim tc=test_lposconly  ; \
  ${runtest_nc_lp} -lps_cpf ../cadence/cpf/projxxx_rtl.cpf
-lps_simctrl_on -lps_pmode -lps_stime 10000ns  -lps_verbose 1 -f
cmd_files -l log/${tc}.log  ; \
    make post_sim;

sim_lp_all  :
  make nc_lp_all tc=test_lpall env=ic  ; \
  \cp cmd_files cmd_files_lpall  ; \
  make nc_lp_osconly tc=test_lposconly env=ic  ; \
  \cp cmd_files cmd_files_lposconly  ;

$(TC_LIST)  :
  $(MAKE) $(sim) tc=$@;

all  : $(TC_LIST)
```

Perl 篇

perl注释语句的方法是使用##号

使用use warnings 会有很好的调试作用.

基础语法

1. 文件头这一行是什么意思？#!/usr/bin/perl 第一个调用的是 usr 下的 bin 文件夹下的 perl ，指明调用 perl 的地址。

通常会在后面加-w 表示使用严格的语法控制，一不小心会出现许多警告，比如变量没有定义、没有初始化等。

chomp 用于去换行符,因为从其它地方获得的字符一般都有换行符号,如下

```
$temp = `pwd`;
chomp ( $temp);
```

2. 子函数参数放在数组@_中,第一个参数放在$_[0]中，第二个参数放在$_[1]中，第三个参数放在$_[2]中，依此类推
3. 获取命令行外部的变量放在数组@ARGV 中第一个参数放在$ARGV[0]中，第二个参数放在$ARGV[1]中，第三个参数放在$ARGV[2]中，依此类推，即在程序中使用$ARGV[0],…来得到外部输入的参数
4. 变量不需要定义，直接使用$temp=xxx,（全局变量）或 my $temp =xxx （局部变量--使用了关键字 my）;@filename；如果要定义则使用

   $temp ;@filename;$符号表示引用，与 MAKEFILE 用法相同；每行语句后面有；符号，与 MAKEFILE 不同
5. 在 perl 脚本中加入 linux shell 执行命令 system，system "gvim test.c";system "cp test.c testcc.c";
6. 使用``来得到内容：$usr_name=`whoami`; $tb_file=`ls *.sv *.v`;
7. 正则表达式贪婪和非贪婪代表什么意思？$s1 = $s2 = "I am very very cold";

```
$s1 =~ s/ve.*y //;尽可能多的匹配
#贪婪式；结果为 I am cold
$s2 =~ s/ve.*?y //; 尽可能少的匹配
#吝啬式；结果为 I am very cold
```

贪婪表达式 (?, *, +, {})

吝啬吝啬表达式(??, *?, +?, {}?)

8. 文件查找关键字：find ../log -name "*.run_log" | xargs grep "ERROR" > ../log/all.log　　find 找文件，grep 找关键字，先找到需要的文件，再在文件中找到关键字

9. 文件文字替换：find -name 'test_rcu_*.sv' |xargs perl -pi -e 's|SFR_Acr_REG|RCUACR_REG|g'

10. 默认变量$_：当前操作的内容默认为放在$_变量中；

11. 数据格式：八进制以 0 开头，十六进制以 0x 开头，二进制 0b 开头；“前置 0”指示符只对数字有效，对由字符串转换过来得数字无效，在本章后面你可以看到。可以利用oct()或 hex()把某个看起来像八

12. 进制或十六进制的数据串转换成数字。虽然没有“二进制”（bin）函数来转换二进制的值，如果某个字符串以0b 开头可由oct()做到。

oct()，hex,oct 分别指示为 16 进制和 8 进制

字符串和数字自动转换时，所有的数字默认为 10 进制，而不会识别为其它进制

13. qw,声明文本列表：my @keys = qw(perls before swine);

my @three_score = ($score{“barney”}, $score{“fred”},

$score{“dino”});

my @three_scores = @score{ qw/ barney fred dino/ };

14. 匹配 m//

在使用正则匹配时，匹配上的内容放在$&中，匹配的前一部分放在$`中，匹配的后一部分放在$'中；

如果匹配多个，多个分别用（）表示，则第 1 个放在$1 中，第 2 个放在$2 中，第 3 个放在$3 中，依次类推,

m/ /i 表示匹配时不区分大小写

m/./s 表示匹配任何字符，包含换行符

/\w{5}/ 表示匹配 5 个字符

/\d{5}/ 表示匹配 5 个数字

^表示补集如/[^\d]/ 表示匹配非数字

15. 替换 s//

s/\s+//;说明 s/表示替换，s+表示 1 个或多个空白，即替换空格

去开头空格 s/^\s+// 说明^表示开头

去结尾空格 s/\s+$// 说明$表示结尾

上述实际的动作是先匹配（符号//）再替换（s）

15. 词匹配 /\blove\b/ 只匹配 love 单词

16. 分割 split /separter/,$string用 separter把$string分割开来；split 默认的操作对象是$_ eg:@temp = split /separter/,$string; 注意有个逗号

17. 结合 join ,$var = join ":" ,1,2,3,4 则 var="1:2:3:4"用：把 1,2,3,4 结合成一个字符串

18. 获取子串 substr($string,$initialpos,$length)； my name = substr("0123456789",5,2);则 name="56",从第 5 位开始取 2 个字符，字符顺序是从 0 编号开始，

19. 子函数参数放在数组@_中第一个参数放在$_[0]中，第二个参数放在$_[1]中，第三个参数放在$_[2]中，依此类推

20. 获取命令行外部的变量放在数组@ARGV 中第一个参数放在$ARGV[0]中，第二个参数放在$ARGV[1]中，第三个参数放在$ARGV[2]中，依此类推，即在程序中使用$ARGV[0],…来得到外部输入的参数

21. 特殊字符：

. 表示除了换行符外的任何一个字符

[]表示括号内中格的任何一个

\d 表示 0~9 中的任何一个字符

\w 表示数字字母和下划线构成的一个单词

\s 表示格式符制表符换行符回车及空格中的任意一个（即为空白）

22. 贪婪匹配 *，+，？

* 表示 0 个或多个

+ 表示 1 个或多个

? 表示 0 个或 1 个

23. 类型变换 hex(temp);

把 temp 转换为 10 进制数

oct(temp);把 temp 转换为 8 进制数

24.结束循环

last;立刻结束循环块，结束这个while

next;立刻结束本次循环，运行到下一次（next）循环；

25.at(实际是@) temp;my @temp;

数组，数组的用法与其它语言一致，push,把一个数放入数组的后面，pop

把一个数从数组中弹出;push(@arry,$temp);把 temp 压入数组 arry 中；

数组时用索引和值构成（index,value）;

访问数组@temp 的一个变量:$temp[1];

给数组@temp 赋值:$temp[0]=5;$temp[3]=8;

unshift 和 shift 对一个数组的开头进行操作

shift ARRAY

这个函数把数组的第一个值移出并且返回它，然后把数组长度减一并且把所有的东西都顺移。如果在数组中不再存在元素，它返回 undef。

如果省略了 ARRAY，那么该函数在子过程和格式的词法范围里移动 @_；

它在文件范围（通常是主程序）里移动 @ARGV。

$testname = shift @ARGV;

大小写字母转换：

函数 uc （uppercase）将所有的小写字母转成大写；

函数 lc （lowercase）将所有的大写字母转成小写；

Perl提供了四个内置的函数类，从而能够轻松地改变字符和字的大小写。函数uc()会用大写返回其所有的参数，而 lc()会用小写返回其参数。如果没有指定参数的话，uc()和 lc()都会对$_变量进行操作。

要改变字的首字母的大小写，就要使用ucfirst()和 lcfirst()。和uc()以及 lc()类似，如果没有指定参数，ucfirst()和 lcfirst()会对$_变量进行操作。

这四个函数都不会改变其参数；它们只是会返回其参数的副本，而这个副本经过了改变。所以，你必须将结果分派给一个变量或者打印它，或者用其他的方式使用返回的值。

26.列表()

数组是由括号括起来并且其元素由逗号分隔开的列表

（“fred”, “barney”, “betty”, “wilma”, “dino”）等同于 qw(fred barney betty wilma dino) #同上，但输入更少；qw 表示“quoted words”

($fred, $barney) = qw <flintstone rubble slate granite>; #两个值被忽略了

($wilma,$dino) = qw[flintstone]; #$dino 为 undef

27.哈希数组%temp

hash 哈希变量,它是有字符和值构成（key,value）;

访问哈希%temp 的一个变量：$temp {key};

给哈希%temp 赋值：$temp {key0}="me";$temp {key1}="you";…

my %last_name = （

“fred” => “flintstone”,

“dino” => undef,

“barney”=> “rubble”;

“betty”=> “rubble”,

);

1.keys 函数会返回此 hash 的所有 keys，values 函数将返回所有的 values。如果hash 中没有元素，则此函数将返回空列表：

my %hash = （“a”=>1, “b”=>2, “c”=>3);

my @k = keys %hash;

my@v = values %hash;

现在，@k 含有“a”, “b”, “c”,而@v 含有 1, 2, 3，其顺序可能不同。

记住，Perl 并不维护 hash 中元素的顺序。但，其中 keys

按照某种顺序存储，则其对应的values 也是这个顺序：

在标量 context 中，这些函数返回 hash 中元素的个数(key/value)。此类操作不需要访问hash 的每一个元素，其效率是很高的：

my $count = keys %hash; #得到 3，是指有 3 个 key/value 对。

2.each 函数，实践中，一般只在while 循环中使用 each：

```
while ($key, $value) = each %hash) {
print “$key => $value\n”;
}
```

上述代码中，首先，each %hash 将从 hash 中返回一个 key/value 对；假设 key 为“c”，value 为 3， 则列表为(“c”,3)。这个
列表赋给($key, $value)，现在$key 为“c”，而$value 为 3 。
，each 返回的 key/vlaue 对，顺序是混乱的（它其顺序和 keys 和 values 函数返回的顺序相同）。如果想将其按序排放，
可以对它们排序（使用 sort），大致如下：

```
foreach $key (sort keys %hash) {
$value =$hass{$key};
print “$key => $value\n”;
#也可以不使用额外的临时变量$value
#print “$key => $hash{key}\n”;
}
```

3.exists 函数，检测某个 KEY 是否存在：

```
if(exists $books{$dino}) {
print “Hey, there’s a libaray card for dino!\n”;
}
```

这即是说，当且仅当 hash 中（keys %books）的 keys 的列表中存在 dino 时，exists $books{“dino”}才返回 true。

4.delet 函数

```
my $person = “betty”;
delete $books{$person}; #将$person 的借书卡删除掉
```

28.数组与哈希的区别

哈希是一种数据结构，和数组类似，可以将值存放到其中，或者从中取回值。但是，和数组不同的是，其索引不是数字，
而是名字。也就是说，索引（这里，我们将它叫key）不是数字而是任意的唯一的字符串使用时[]表示数组， {}表示哈希

29.&subprog　表示子程序

30.文本处理 while(<tp>)或 foreach(<tp>);tp 为文件的指针，这样会循环每行读取文件

@parameter = <tp>;这样 parameter 变量会按行存放 tp 的内容$parameter[0]存放第一行

31.<>为行输入操作;如经常使用<STDIN>

32.Glob

通常，shell 将每个命令行中的任何的文件名模式转换成它所匹配的文件名。这被称作 globbing。

有时在Perl 程序的内部使用像*.pm 这样的模式。我们可以轻易的将它转换为它所匹配的文件名吗？是的，只需要使用 glob

操作：

my @all_files = glob “*”;

my @pm_files = glob “*.pm”;

@all_files 得到了当前目录下的所有文件，这些文件按照字母排序的，不包括由点（.）开头的文件。

许多使用 globbing 的程序中并没有出现 glob 这个字眼。为什么呢？原因是，许多这类代码在 glob 操作被命名前就写好了。它使用一对尖括号（<>），和从文件句柄读入操作类似：

my@all_files = <*>; ##基本上同@all_files = glob “*”一样；

由于尖括号的含义可以指从文件句柄读入或 globbing，Perl 怎么判断使用哪一个操作呢？如果尖括号之间是一个严格意义

上的标识符，则其为文件句柄读入操作；否则，为 globbing 操作。如下例：

my @files = <FRED/*>; ##glob

my @lines = <FRED>; ##文件句柄读入

my $name = “FRED”;

my @files = <$name/*> ##glob

33.localtime 函数

my ($sec, $min, $hour, $day, $mon, $year, $wday, $yday, $isdst)= localtime $timestamp;

$mon 是一个表示月份的数字，范围是 0 到 11，其在月份名字的数组中作为索引值是比较方便的。$year 是指从 1900 到现

在的年份数，因此，需要加上 1900 来得到实际的年数。$wday 的值是从 0（星期天）到 6（星期六），$yday 指一年中的具体天数（从 0（1 月 1 日）到 364 或 365（12 月 31））。如果想从系统中得到当前的时间，可使用 time 函数。localtime 和 gmtime 在默认的情况下都使用 time 的当前值，如果没提供参数：my $now = gmtime; #得到当前的时间

34.改变目录(打开某个文件夹)：chdir　相当于 cd　使用格式 chdir "../../../";

35. 目录句柄 opendir

从给定目录得到其文件名列表的方法还可以使用目录句柄（directory handle）。目录句柄外形及其行为都很像文件句柄。打开（使用 opendir 而非 open），从中读入（使用 readdir 而非 readline），关闭（使用 closedir 而非 close）。不是读入文件的内容，而是将一个目录中的文件名（以及一些其它东西）读入，如下例：

```
my $dir_to_process = “/etc”
opendir DH, $dir_to_process or die “Cannot open $dir_to_process: $!”;
foreach $file(readdir DH) {
print “one file in $dir_to_process is $file\n”;
}
closedir DH;
```

36.删除文件 unlink

```
unlink “slate”, “bedrock”, “lava”;删除三个文件
unlink glob “*.o”; 删除所有.o 后缀文件
```

37.重命名文件 rename：rename “old”, “new”;

38.修改权限 chmod：chmod 0755, “fred”, “barney”;

39.改变所有者 chown：

my $user = 1004;

my $group = 100;

chown $user, $group, glob “*.o”;

40.改变时间戳 utime

my $now = time;

my $ago = $now -24*60*60; #一天的秒数

utime $now, $ago, glob “*”; #设成当前访问的，一天之前修改的第三个时间戳（ctime 的值）永远是“now”，无论什么时候改变的文件，因此不能通过使用utime 函数来设置它（当你设置它后，它会自动重置到“now”）。因为其首要的目的是为了不断的备份：如果文件的 ctime 比备份磁带上的日期更新，则应当进行备份了。

41.index($string,char);返回 char 在 string 字串中的位置

substr($string,$initialpos,$length)，当没有$length 时，默认为后面所有的字符

index 和 substr 可以很好的一起工作。在本例中，我们提取出了字符串中字母 l 后的子串。

my$ $long = “some very very long string”;

my $right = substr($long, index($long, “l”));

42.排序 sort,从小到大排列，reverse sort,从大到小排列

my @numbers = sort { $a <=> $b } @some_numbers; 从小到大

my @numbers = sort { $b <=> $a } @some_numbers; 从大到小；等价于 my @numbers =reverse sort { $a <=> $b } @some_numbers;

43.打开文件写 open

open filefp,">filename"; filefp 表示打开文件的句柄，>表示对文件写，

open filefp,">>filename"; filefp 表示打开文件的句柄，>>表示对文件追加写，如果文件存在，它会在文件后面添加新的内容

open filefp,"<filename"; filefp 表示打开文件的句柄，<表示对文件读，

44.关闭文件 close; close(filefp);

45.打印 print ； print LOG “Captain’s log, stardate 3.14159\n”; # 输出到 LOG 中，LOG 是打开的文件句柄

46.格式化打印 printf;

printf STDERR “%D percent complete.\n”, $done/$total * 100;

%08b:表示前面补 0,8 位 2 进制输出

47.打开文件夹 opendir

my $dir_to_process = “/etc”

opendir DH, $dir_to_process or die “Cannot open $dir_to_process: $!”;

48.一般字符

[A-Za-z]{5} ：用于匹配任何一组 5 个字母字符

[0-9]+ ：用于匹配一个或者多个字符

[*!@#$%&()]：用于匹配这些符号中任何一个

49.特殊字符

\w ：一个单词字符，与[a-zA-Z0-9_]相同

\W ：一个非单词字符，与\w 相反

\d ：一个数字，与[0-9]相同

\D ：一个非数字

\s ：一个白空间字符，与[\t\f\r\n]相同

\S ：一个非白空间字符

50.foreach 循环

foreach 从列表的第一个元素一直循环执行到

最后一个元素，一次迭代一个：

```
foreach $rock (qw/ bedrock slate lava /) {
print “One rock is $rock.\n”; #打印出 3 种 rocks
}
```

控制变量（本例中为$rock）每一次迭代从列表中取出一个新值。第一次为“bedrock”，第三次为“lava”。

```
@rocks = qw/ bedrock slate lava /;
foreach $rocks(@rocks) {
$rock = “\t$rock”; #@rocks 的每一个元素前加入一个 tab
$rock . = “\n”; #每一个元素后加一个换行符
}
print “The rocks are:\n”,@rocks; #每一个元素都被缩进了，并且一个元素占一行
```

51.for 循环

```
for($i =1; $i <=10; $i++){ #从 1 到 1 0
print “I can count to $i;\n”;
}
for($i = -150; $i <= 1000; $i +=3){
print “$i\n”;
}
```

52.While 循环

```
initialization;
while(test){
body;
body;
increment;
}
```

53. 文件检查

检测选项	含义
-r	文件或目录对此（有效的）用户（effective user）或组是可读的
-w	文件或目录对此（有效的）用户或组是可写的
-x	文件或目录对此（有效的）用户或组是可执行的
-o	文件或目录由本（有效的）用户所有
-R	文件或目录对此用户(real user)或组是可读的
-W	文件或目录对此用户或组是可写的
-X	文件或目录对此用户或组是可执行的
-O	文件或目录由本用户所有
-e	文件或目录名存在
-z	文件存在，大小为 0（目录恒为 false）
-s	文件或目录存在，大小大于 0（值为文件的大小，单位：字节）
-f	为普通文本
-d	为目录
-l	为符号链接
-S	为 socket
-p	为管道(Entry is a named pipe(a "fifo"))
-b	为 block-special 文件（如挂载磁盘）
-c	为 character-special 文件（如 I/O 设备）
-u	setuid 的文件或目录
-g	setgid 的文件或目录
-k	File or directory has the sticky bit set
-t	文件句柄为 TTY(系统函数 isatty()的返回结果，不能对文件名使用这个测试)
-T	文件有些像“文本”文件
-B	文件有些像“二进制”文件
-M	修改的时间（单位：天）
-A	访问的时间（单位：天）
-C	索引节点修改时间（单位：天）

54.数字和字符之间自动转换

2．2．4 数字和字符串之间的自动转换

大多数情况下，Perl 将在需要的时候自动在数字和字符串之间转换。它怎样知道什么时候需要字符串，什么时候需要数字呢？这完全依赖于标量值之间的的操作符。如果操作符（如+）需要数字，Perl 将把操作数当作数字看待。如果操作符需要字符串（如 .），Perl 将把操作数当作字符串看待。不必担心数字和字符串的区别；使用恰当的操作符，Perl 将为你做剩下的事。

当在需要数字的地方使用了字符串（如，乘法），Perl 将自动把字符串转换为其等价的数字，就像输入的是十进制浮点数一样◆。因此“12”*“3”将给出 36。后面的非数字部分和前面的空格将被去掉，如“12fred34”*“3”将给出 36 而不会用任何提示◆。在极端情形，当一个不含任何数字的字符串将别转换为 0。如，将“fred”当作数字来使用时。

◆用首字符 0 表示非十进制值对数字有效，对自动转换没有作用，使用 hex() 和 ort() 来转换此类字符串

◆除非你使用了 warnings,我们将很快讨论到。

同样，如在需要字符串的地方使用了数字（如，字符串连接），数字将转换为字符串。例如，如果你想在 Z 后串接 5 乘以 7 的结果◆，可以这样写：

```
"Z" . 5 * 7   #同于 "Z" . 35,或 "Z35"
```

举例

1.Hex2bin

把 keil 根据 8051 核生成的 hex 文件转成文本,无 bank 功能

```
#!/usr/bin/perl
$src_file = $ARGV[0];
@NAME = split/\./,$src_file;
$des_file = join ".",$NAME[0],"dua";
@dua_data;
open src_fp,"<$src_file";
open des_fp,">$des_file";
while(<>) {
    $line_data = $_;
    $data_length = substr($_,1,2);
    $addr = substr($_,3,4);
    $addrh = substr($_,3,2);
    $addrl = substr($_,5,2);
    $data_type = substr($_,7,2);
    $ref_check =
hex($data_length)+hex($addrh)+hex($addrl)+hex($data_type);
for($i=0;$i<(hex($data_length));$i++) {
        $data_temp = substr($line_data,9+$i*2,2);
        $dua_data[hex($addr)+$i] = $data_temp;
        $ref_check = hex($data_temp)+$ref_check;
    }
    $ref_check = $ref_check%256;
    $check = substr($line_data,9+hex($data_length)*2,2);
if(hex($check)!=(256-$ref_check)) {
```

```
printf "this line check error ,expect value is %0x,infact
is %0x\n",hex($check), (256-$ref_check);
    }
if($data_type == 01) {
print "file transfer is last line\n";
    }
}
foreach $data  (@dua_data) {
printf des_fp "%08b\n",hex($data);
 }
close(des_fp);
```

2.analyze_log.pl

```
#! /usr/bin/perl -w
my $file;
my @all_files = glob "*.log";
foreach $file(@all_files) {
printf "file name is %s\n",$file;
}
#$src_path = $ARGV[0];
#$sink_path1 = $ARGV[0].".timing"
$sink_path1 = summy_log;
open(SINK1,">",$sink_path1) or die "can't open $sink_path1 for
writing:$!\n";
print SINK1 "/****************Timing Violation information as
follows***********/\n\n";
foreach $file(@all_files) {
printf SINK1 "/*******now analyze file is %s**********/\n",$file;
printf  "/*******now analyze file  is %s**********/\n",$file;
```

```
    #open(SOURCE,"<",$src_path) or die "can't open $src_path for
reading:$!\n"
open(SOURCE,"<",$file) or die "can't open $file for reading:$!\n";
my $line;
my $jump = 0;
my  $vio_info = "Timing violation";
my  $err_info = "ERROR";
my  $key_info = "the tc is pass";
my @err_reg;
my $string;
my $exist =0;
my $reg;
while(<SOURCE>) {
chomp;
        $line = $_;
if(/\[ERROR\]/) {
print SINK1 "$line\n\n";
        }
if(!$jump) {
if($line =~/(.*$vio_info.*)/) {
                $string = $1;
foreach $reg (@err_reg) {
if($reg eq  $string) {
                        $exist = 1;
                    }
                }
if(!$exist) {
push  (@err_reg,$string);
print SINK1 "$string\n";
```

```
                    $jump = 1;
                }
else{$exist =0;}
            }
        }
else{
print SINK1 "$line\n\n";
            $jump = 0;
        }
    }
close(SOURCE);
}
close(SINK1);
printf "perl analyze log finish \n ";
```

3.delete_cvs_all

删除当前目录下及子目录下所有一个名叫 CVS 的文件夹

```
#!/usr/bin/perl
@dirname = `ls`;
if (-e "CVS") {
    #print "====CVS is EXIT";
    system "rm -rf ./CVS";
}
foreach $dirname( @dirname ){
    print "$dirname";
    chomp ($dirname);
    if (-d $dirname) {
        &sub_dele_cvs($dirname);
```

```
            print "quit &sub_dele_cvs\n";
        }
    }

    sub sub_dele_cvs{
        #print "----$_[0] is para\n";
        chdir "./$_[0]";
        if (-e "CVS") {
            system "rm -rf ./CVS";
        }
        my $pa = `pwd` ;
        #print "----1 $pa is `pwd`\n";
       my  @sub_dirname = `ls`;
       #print "----$pa\n";
        foreach my $sub_dirname( @sub_dirname ){
            chomp($sub_dirname);
            if (-d $sub_dirname) {
                #print "----$sub_dirname is dir\n";
                my $cvs_path = join("/",".",$sub_dirname,"CVS");
                #print "$cvs_path\n";
                system "rm -rf $cvs_path";
                &sub_dele_cvs($sub_dirname);
            }
        }
        chdir "../";
        $pa = `pwd` ;
        #print "----2 $pa is `pwd`\n";
}
```

4.modify_char_all.pl

用于替换当前.sv 文件，中某部分字符

使用 modify_char_all.pl xxxx yyyyy 把所有.sv 文件中 xxxx 替换成 yyyy

```
#!/usr/bin/perl
$src_char = $ARGV[0] ;  ##
$des_char = $ARGV[1] ;
@dirname = `ls`;
foreach $dirname( @dirname ) {
    print "$dirname";
    chomp ($dirname);
    $_ = $dirname;
    if  (m/sv/)  {
        system "mv  $dirname  $`bak";
        open new_file_name,">", "$dirname";
        open file_name, "$`bak";
        while(<file_name>) {
            s/$src_char/$des_char/g  ;
            #print "$_";
            print new_file_name "$_";
        }
        close new_file_name;
        close file_name;
        system "rm -rf  $`bak";
        print "$dirname replace done !!\n";
    }
    if  (-d $dirname)  {
        &sub_modify_char($dirname);
        print "quit &ssub_modify_char\n";
```

```
    }
}

sub sub_modify_char{
    chdir "./$_[0]";
    my $pa = `pwd` ;
    my  @sub_dirname = `ls`;
    foreach my $sub_dirname( @sub_dirname ) {
        chomp ($sub_dirname);
        $_ = $sub_dirname;
        if (m/sv/) {
            open new_sub_file_name,">", "$sub_dirname";
            open sub_file_name, "$`bak";
            while(<sub_file_name>) {
                s/$src_char/$des_char/g ;
                #print "$_";
                print new_sub_file_name "$_";
            }
            close new_file_name;
            close sub_file_name;
            system "rm -rf  $`bak";
            print "$sub_dirname replace done !!\n";
        }
        if (-d $sub_dirname) {
            &sub_modify_char($sub_dirname);
        }
    }
    chdir "../";
    $pa = `pwd` ;
```

}

TCL 篇

1. tcl 程序每行后面没；符号

2. 运行 tcl 脚本:tclsh xxx.tcl

3. 在 tcl 中使用 shell 命令：exec

如：exec find ../log | sort >log.f

4. tcl 常用命令：[],gets,puts

有一个语句：

set Name [gets stdin]

符号对“[]”的意思是：执行里面的命令，用结果替换这个组合，类似于 C 语言中的函数调用。可以使用 shell命令

上文这个句子的含义是：把“gets stdin”的值赋给变量Name。

gets 的语法：

gets channelId ?varName? （注意变量的位置，在后面）

gets 从一个 channel取一个行到指定变量；同时它本身也返回这个行，所以 gets 还有第二种用法：

gets stdin Name

put 语法

Ø puts nonewline fileId string puts 命令把 string 写到 fileId 中，如果没有 nonewline 开关的话，添加换行符。fileId 默认是 stdout。命令返回值为一空字符串。 puts 命令使用 C 的标准 I/O 库的缓冲区方案，这就意味着使用puts 产生的信息不会立即出现在目标文件中。

Ø 如果你想使数据立即出现在文件中，那你就调用 flush 命令： flush fileId 把缓冲区内容写到 fileId 标识的文件中，命令返回值为空字符串。 flush 命令迫使缓冲区数据写到文件中。flush 直到数据被写完才返回。当文件关闭时缓冲区数据会自动 flush。

tcl 数据类型

TCL 注释语句的方法是使用##号

tcl 的数据类型实际上只有一种：字符串

所以在 tcl 中的变量没有类型转换或类型匹配问题，所有的“数据”都是以字符串的形式存在。

那么，tcl如何进行数值计算？我如何计算并输出一个小数？

数值计算

tcl是支持数值计算的（包括数学函数），不过仅能用在 expr 命令中

```
% set x 100
% set y [expr $x + 1]
```

“数据类型”输出

一个命令format，其格式串参数类似 C 语言。

```
% set z [format “This is %-8.2f !” $y]
This is 230.00     !
```

5. 在使用PrimtTime 的过程中，命令的嵌套经常会被用到。嵌套命令时，用方括号([])分隔开每一层的命令，例如：　　命令 1 [命令 2 [命令 3]]

   在这样一个例子中，命令 3 首先被执行，它的结果将被作为命令 2 的一个参数，然后依次执行下去。

6. 在 Tcl 中，可以使用两种方法来引用文本或者说字符串：

   1）弱引用：使用双引号来引用文本。在双引号里出现的变量、命令和反斜杠不会被转义，仍然保持特殊意义。

   2）强引用：使用大括号来引用文本。大括号中的字符串将按照字面上被引用。

   例如：set mydelay 10

   echo "The value of mydelay is $mydelay" 得到的结果将是：The value of mydelay is 10，而　　echo {The value of mydelay is $mydelay} 得到的结果将是：The value of mydelay is $mydelay。

   除此之外，可以使用反斜杠来转义一个单一的特殊字符，以及使用 expr 命令来得到算术表达式的值。

切换目录

Tcl提供 2 个命令来管理当前工作目录:pwd & cd

pwd和UNIX下的pwd命令完全一样，没有参数，返回当前目录的完整路径。

cd 命令也和UNIX命令也一样，使用一个参数，可以把工作目录改变为参数提供的目录。如果cd 没使用参数，UNIX下，会把工作目录变为启动TCL脚本的用户的工作目录，WINDOWS下会把工作目录变为windows操作系统的安装目录所在的盘的根目录(如：C:/)。值得注意的是，提供给cd的参数中路径中的应该用'/'而不是'\'。如 cd C:/TCL/lib。这是UNIX的风格。

文件处理

while 用到较多的地方是处理文件，

```
set fpIn [open $FileName r ]
while { [eof  $fpIn  ] != 1 } {
 set CurLine [gets $fpIn ]
 if { $CurLine == “” } {
  continue
 }
 . . .
```

如果文件已经读完最后一行，继续

while { [eof $fpIn] != 1 } 在 C 和 tcl 里条件都会通过

但在句子 set CurLine [gets $fpIn] 里

tcl 会返回并设CurLine 为文件的最后一行

C 会返回并设 CurLine 为空

所以在 tcl程序里在其后加上了如下判断:

```
if { $CurLine == “” } {
 continue
}
```

子函数 proc{}

proc

说明一个子过程，后面接过程名、参数列表、执行体。需要注意的是，过程的参数列表应该和调用处的参数一致。

但例外的是，如果过程的参数只写一个，它就可以以 list 的形式接收多个入口参数。

7. source xxx.tcl

source

可以把一个外部的 tcl文件在此处展开（执行），我们大家熟知的一个文件就是 etc/aiip.cfg，它里面其实就是一段 tcl 程序，设置了一个名为AIIP 的数组

glob

glob 按返回符合指定条件的文件名 list

主程序入口参数

主程序入口时会带来几个变量，常用的有

数组 env 系统的环境变量

本程序名 argv0

入口参数个数 argc （不包括本程序名，与 C 不同）

入口参数列 argv （不包括本程序名）

如果调用程序：

```
aiobs> hello.tcl Mark John
```

那么在程序里：

```
puts $arg0          ;# hello.tcl
puts $argc          ;# argc = 2
puts $argv          ;# “Mark John”
puts [lindex $argv 0] ;# Mark
puts [lindex $argv 1] ;# John
```

list 函数操作

list 及相关操作

list arg1 … argn 把各个参数合成一个 list

llength $SomeList 返回 list 的长度（元素个数）

lindex $SomeList 0 返回 list 的第一个元素

lindex $SomeList end 返回 list 的最后一个元素 ;# end 是 list 操作的保留字

open 操作

打开文件或管道，简直和 C 一样。相关操作 close 、flush、puts、gets、read、seek、eof、tell

set file [open filename.sv] 打开文件 filename.sv 用 file 指向这个句柄

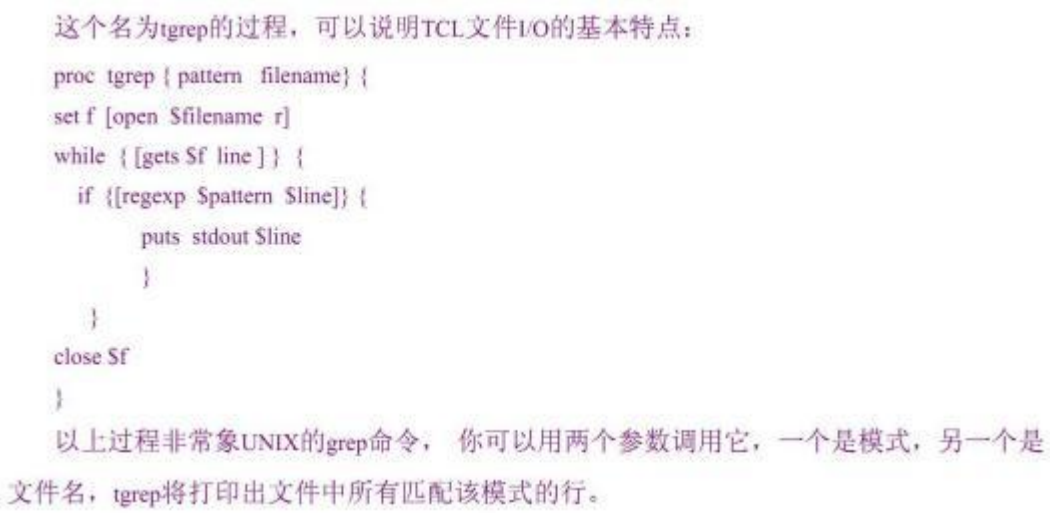
regexp 操作

regexp

对字符串进行正则表达式分析

```
set x 01062501658
regexp “010(.*)” $x y z
puts “$x $y $z
```

打印文件中查找到的行内容


这个名为tgrep的过程，可以说明TCL文件I/O的基本特点：

```
proc tgrep { pattern  filename} {
set f [open $filename r]
while { [gets $f line ] } {
  if {[regexp $pattern $line]} {
        puts stdout $line
        }
   }
close $f
}
```

以上过程非常象UNIX的grep命令， 你可以用两个参数调用它，一个是模式，另一个是文件名，tgrep将打印出文件中所有匹配该模式的行。

上程序中 line 是一个变量,gets $f line 就是从文件 f 中取一行内容到 line 中;

Regexp $pattern $line 就是查看 line 是否能匹配 pattern

举例

```
proc chk_log {} {
set pass_word "pass"
exec find ../log/-name "*.log" | sort >log.f #在某目录下查找.log 文件，排序，并写到 log.f 中
set fileid [open log.f r+] # 只读方式打开文件 log.f，并放入 fileid 句柄
set new_id [open sunmary.log w]
while{gets $fileid line] != -1} {
set tc_pass -1
set logid [open $line r]
while{[gets $logid logline] != -1} {
if{[regexp $pass_words $logline]} {
set tc_pass 1
break
```

```
        }
    }
if($tc_pass == 1){set line [format ""]}
else {}
puts $new_id $line
close $new_id
}
flush $new_id
close $new_id
fulsh $fileid
close $fileid

exec rm -rf log.f
}
chk_log
```

gvim 常用小技巧

.Vimrc 常用设置

```
set nocompatible
"source $VIMRUNTIME/vimrc_example.vim
"source $VIMRUNTIME/mswin.vim
"behave mswin

set number
set ruler
set tags=tags;
set noswf
winpos 400 0
set lines=52
```

```
set columns=150
syntax on
"set sm
set mouse=a
set backspace=indent,eol,start
set clipboard=unnamed
"hi Normal guibg=#d0d0d0
hi Normal guibg=#e0e0e0
"colorscheme desert
color murphy
set ai
set ts=4
set expandtab
set shiftwidth=4
"set autoindent
filetype indent on
"set foldenable
"set foldmethod=indent
"nnoremap <space> @=((foldclosed(line('.')) < 0) ? 'zc' : 'zo')<CR>
set hlsearch
map <C-L> <C-W>l
map <C-TAB> <TAB>
map <TAB> <C-W><C-W>
map f <C-F>
map b <C-B>
```

1.替换一个 word,用\<\>,perl 中用\b\b,

ex command

```
:%s/\<four\>/4/g
```

2.替换多个文件的目标

要对多个文件进行同样的替换操作。显而易见的办法是逐个编辑每个文件，敲入替换命令。不过用宏记录和回放功能就快多了。

假设你有一个目录下有很多C++文件，都以".cpp"为扩展名。现欲将所有名为"GetResp"的函数更名为"GetAnswer"。

vim *.cpp	启动Vim，同时指定了要编辑的文件列表：所有的C++文件。现在你正在编辑的将是第一个文件。
qq	开始宏记录，将后续的操作记录在名为q的宏中。
:%s/\<GetResp\>/GetAnswer/g	在第一个文件中执行替换操作。
:wnext	保存该文件并转到下一个文件进行编辑
q	停止宏记录
@q	执行名为q的宏。它将执行前面记录的替换操作和":wnext"。你可以看看整个过程有没有什么错误。
999@q	对其余文件执行同样的操作[1]

执行到最后一个文件时你会得到一个错误消息，因为":wnext"命令没有文件可以"next"了。错误会让命令停下来，不过有用的操作已经完成了[2]。

> 备注：回放一个宏记录时，任何错误都会导致整个宏记录停止执行。所以你要确保你在记录宏的过程中没有错误消息。

还有一种例外：如果其中一个.cpp文件中连一个"GetResp"都没有，替换操作会引起一个错误，整个宏的执行也会被停下来。标志"e"正是致力于消除这一副作用：

ex command

```
:%s/\<GetResp\>/GetAnswer/ge
```

"e"标志告诉":substitute"命令就算没找到一个匹配的目标你也不要报错。

3.命令的记录与回放

10.1 命令的记录与回放

"."命令可以重复最近一次的编辑动作。但是如果你要做的操作远比这些小儿科复杂呢？那就是Vim为什么要引入命令记录的原因。使用命令记录分三个步骤[1]

1。使用"q{register}"命令开始，后续的动作将被记录进名为{register}的寄存器中。给出的寄存器名字必需是a到z之间的一个字母[2]

2。执行你要执行的操作[1]。

3。按下q以结束对命令的记录(注意仅仅是q一个字符，不要键入多余的字符)。

现在你可以通过"@{recording}"命令来执行刚刚记录下来的记录宏了。

4.移动并执行操作

移动并执行操作

也许实际情况是你在几个不同的地方要执行这些操作(而不象上例中是连续的4行)。这只需要你将光标定位到目标行，然后再执行"@a"命令。如果已经执行过"@a"命令，下次重复执行只需再下"@@"即可。这比"@a"更容易键入。同样，如果你上次执行的是"@b"那么"@@"命令也将重复"@b"的动作。

与"."方法相比，记录回放有几个地方不同，首先，"."命令只能重复一个动作。而在上例中，"@a"重复的是好几个命令，其次，"."命令只重复最近一次改动的命令。而执行一个记录宏允许你随时都可执行同样的操作。最后，你拥有多达26个寄存器可供使用。也就是说，可以同时保存26个不同的命令宏。

5.使用寄存器

使用寄存器

用于命令记录的寄存器与用于yank和删除命令的寄存器是同一个东西。所以还可以多种方式混合操作这些寄存器。

假设你在寄存器n中记录了一些命令。执行时发现有些命令弄错了。当然你可以再重来一遍，但是还有其它办法补救：

List

G	到文件最后
o<Esc>	生成一个新行
"np	将寄存器n的内容置于该行。你会看到这些命令就象你键入的普通文本一样。
{edits}	修改错误的部分。这与编辑普通文本无异
0	到行首
"ny$	将正确的结果回存到寄存器n中
dd	删除这行草稿

现在你可以用"@n"来回放正确的命令宏了。(如果你的宏记录命令中包括有断行，调整上例中最后两个操作以确保将正确的结果存回寄存器n中)

6.重复数字加/减 1

26.2 加与减

需要重复地进行数字的加减时，通常加减量是一样的。比如上例中每个年份都加1。除了象上面一样用替换命令进行修改之外，CTRL-A命令也可以[1]

同样对上例中的内容，使用搜索命令来查找一个年份：

normal mode command

```
/19[0-9][0-9]|20[0-9][0-9]
```

现在按CTRL-A。当前光标下的年份会被加1：

Display

```
The financial results for 2002 are better
than for 2000.  The income increased by 50%,
even though 2001 had more rain than 2000.
                2000            2001
income          45,403          66,234
```

"n"命令查找下一个年份，"."来重复上一次的CTRL-A("."更好键入一些)。不断重复"n"和"."直到所有的年份都修改完毕。

提示：设置'hlsearch'选项可以让你对这些要改动的地方一目了然，做起来信心也更足一些。

加减1的命令CTRL-A也可以前缀以一个数字参数。如果你有下面的列表项：

1. item four

[1]译：替换命令还有一个唯美的方案：

```
:3,5s/19[0-9][0-9]|20[0-9][0-9]/*((submatch(0)+1)/g)
```

直接用 s / / m(submatch(0)+1)/g 比较好

3 CTRL -A 表示数字+3 add 3

2. item five

3. item six

置光标于"1"上。按下:

normal mode command

```
3 CTRL-A
```

"1"将变为"4"。同样你可以用"."来将这一操作施于其它数字上。

另一个例子:

Display

```
006     foo bar
007     foo bar
```

在这两个数字上使用CTRL-A的结果是:

Display

```
007     foo bar
010     foo bar
```

7加1等于10? 原因是Vim将"007"视为一个八进制的数了,因为前面有0嘛。C程序中经常会出现这种记法。如果你不想让此类数字被看作是8进制的,可以改变下面的选项:

ex command

```
:set nrformats-=octal
```

CTRL-X以类似的方式做减法操作。

7.对多个文件做相同的操作

如果你想把好几个C文件中名为"x_cnt"的变量都改为"x_counter"。这就要动到多个文件。

先把所有要改的文件放到参数列表上:

ex command

```
:args *.c
```

该命令会找到所有的C文件并开始编辑第一个。现在你可以对所有的文件都进行同一个替换操作:

ex command

```
:argdo %s/\<x_cnt\>/x_counter/ge | update
```

":argdo"命令以另一个命令为参数。该命令将对所有待编辑的文件都执行一次。

"%s"替换操作将施于所有行上。它通过"\<x_cnt\>"查找"x_cnt"。其中"\<"和"\>"使得只有完整的单词会被匹配，这样"px_cnt"和"x_cnt2"中的x_cnt才可以免遭毒手。

替换操作的标志"g"使得每行中的全部"x_cnt"都被替换。标志"e"则用于避免某些文件中一个"x_cnt"都找不到时的错误消息。否则的话":argdo"命令遇到这些错误就会终止整个操作。

"|"用来分隔两个命令。后面的"update"命令会在文件有改变时进行保存。如果没有一个"x_cnt"被替换为"x_counter"那就不进行任何操作。

类似于":argdo"，命令":windo"会对所有窗口都执行同样的操作。":bufdo"则是对所有的缓冲区执行操作。这个要小心使用，因为你可能想不到缓冲区列表中还有那么多文件。最好使用该命令之前用":buffers"命令(或":ls")看一下就有哪些缓冲区会被改动。

shell(linux)vnc

查看 linx 版本命令:
uname －a （Linux 查看版本当前操作系统内核信息）

cat /proc/version （Linux 查看当前操作系统版本信息

使用vncconfig 命令调出窗口，选中相关选项，不关闭窗口就可以执行复制等功能。---vnc 窗口用法

使用 csh，第一句申明解析器：#!/bin/csh #!/bin/bash

Shell 命令行参数:

$@ 表示所有参数,$1,$2,$3…分别表示第一第二…个参数

$# 表示外部命令行输入参数的个数

变量　直接赋值，=两边不能有空格，使用时采 用$varname;当变量赋值有空格时需要加“ ”

特殊变量

$? :任何Linux命令结束时,都会返回一个代表运行结果的整数值给 shell.

都会放在$?中

杀进程

Kill -p pid

IO 重定向

默认是STDIO，使用< 和 >来改变输入和输出

shell 识别三种命令

1. 内建命令 2.shell 函数 3.外部命令 shell 函数 就是我们通常写的 内建命令集
2. Linux 命令

du -ks 文件夹 查看文件大小

df -k

top

find path -name char 在 path 目录下查找 char 关键字

gunzip 命令解压 foo.gz 文件

unzip 解压 zip 文件

which + tool name 查看工具安装目录

more .svn/entries

acroread pdf 工具

用 gf（gvim file）可以直接在当前 GVIM 中打开 include 等文件，用 ctrl+^(键盘数字 6)可以返回，或

konqueror

有时用 gvim 打不开文件，尝试用 vi 或 vim

GVIM 中用r 命令替换

用 free -g

用 su 名字在其它机器登陆自己的机器，因为成为了超级用户

在 VCS 中，用 ctrl+c 暂停，然后用 run/step 可以继续运行

在 VCS 中，用 ctrl+z 退出

工作站用 lsload -l 可以看到所有服务器的负载

bjobs

桌面任务栏消失：在登录节点的 terminal 中执行/usr/bin/gnome-session 或/usr/bin/gnome-panel

检查机器慢的原因

1、检查是否有大文件拷贝

ps -ef | grep -E '\b cp|\b mv|\b rm|ftp' | grep -v grep

2、检查是否有大量 stopped 进程

ps axo stat,ppid,pid,user,comm | grep ^T | wc -l

12.显示目录树形结构

1、在终端中输入命令 tree + 目录名，直接用命令 ：E

用： E可以在V I M中显示文件目录

用:vsp 可以直接在vim 中把页面分成 2 半

打包的语法：tar -cf foo.tar foo

解包的语法：tar -xf foo.tar

打包并压缩的语法：tar -czf foo.tar.gz foo

rar 解包命令 unrar e

ls -a 显示所有包含隐藏文件 -t 按时间显示

ls -a dir/dir1/dir2 显示指定目录所有包含隐藏文件 -t 按时间显示

diff, diff3,sdiff,comm 用于文本比较，逐行比较

cmp 用于二进制比较，逐字节比较

文件查找关键字 find ../log -name "*.run_log" | xargs grep "ERROR" > ../log/all.log

文件文字替换 find -name 'test_rcu_*.sv' |xargs perl -pi -e 's|SFR_Acr_REG|RCUACR_REG|g'

1.一般以.sh 为文件后缀。没有也能执行。

2.常见的两种执行方式

n./文件名 在当前目录下写明路径执行，要求文件必须有执行权限，如

chmod 0755 run.sh

./run.sh

nsh 文件名

3.文件开头指定一个或多个解释脚本程序的 shell，如 #!/bin/bash

n 如果指定，注意/bin 不要漏了“/”，否则用./方式执行会找不到路径。

n 不指定的话一般默认以/bin/sh 执行 shell 脚本

一个常用的命令 date 可以后门使用+%Y%m%d 来显示年月日：如 20150423，%H%M 显示小时和分钟，可以用在一些结里

`date +%` 格式使用

如何登陆他人服务器：1.go 对方机器，如 go j31, 2.sd 自己电脑机器名 如 sd j172

grep -r "xxxx" dir 会递归搜索 dir 文件夹及子文件所有含"xxxx"字符内容

grep -l -r "xxxx" dir 会递归搜索 dir 文件夹及子文件所有含"xxxx"字符的文件 ，不显示具体的内容

1.批量替换的方法：

文件名批量替换用“rename”命令，如： rename xxx mipi xxx_*.sv （第一个参数：被替换串,第二个参数：替换成的字符串,第三个参数：匹配要替换的文件模式）

文件内容替换可以用 “sed”命令，如 ： sed -i 's/xxx_/mipi_/g' *

find -type d -name "xxx" 可以找到指定的 xxx 文件夹

2.链接

ln -fs myfile hardlink; myfile1 是真实文件路径，hardlink 是一个符号路径

用 shell 写的 script，第一句写的一般是#!/bin/bash

当执行 script 时，可以引入参数，如 script_exe p1 p2 p3 （script_exe 为 script 文件名，p1,p2,p3 为参数，在 script 文件中使用$1,$2,$3 分别可以引用到第 1，第 2，第 3 个参数）

循环使用 for I in{1..10}

do

done

条件语句使用 if[]; then ... else … fi

打印使用 echo "…."

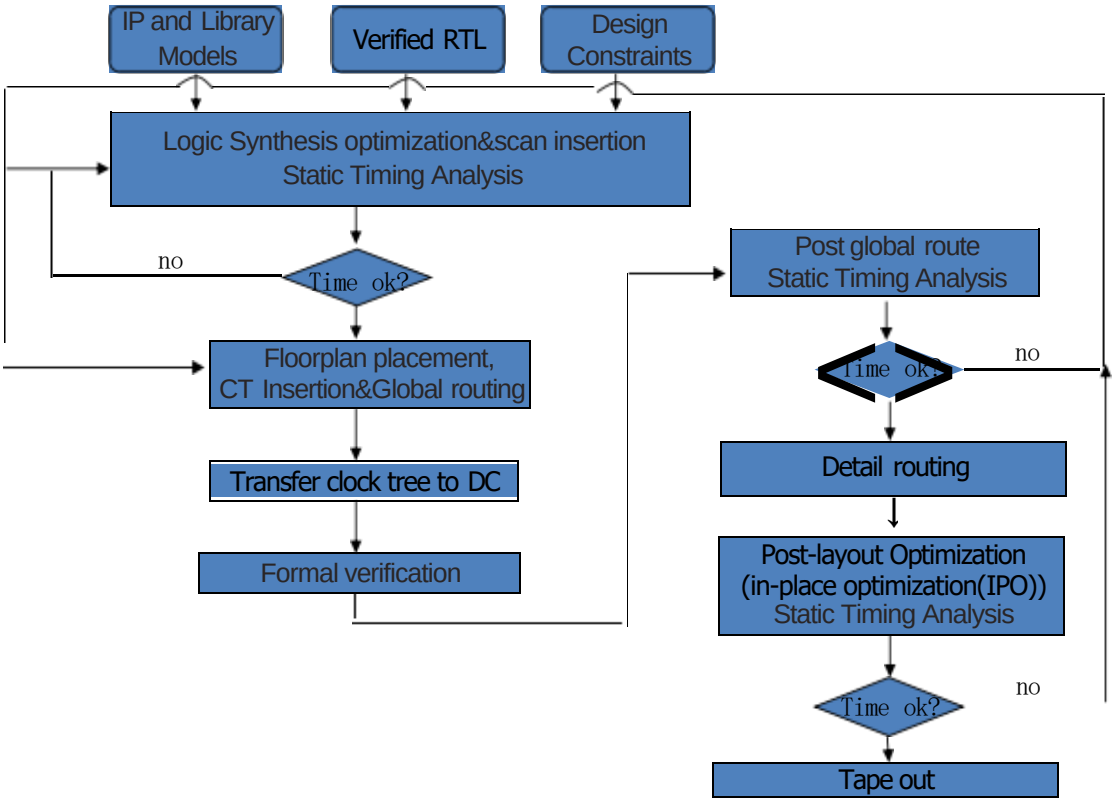
范例

```
open 10.70.53.216          //链接远程服务器
user_name                  //用户名
password                   //密码
cd project/xxxxx /verification/fw/mmu        //打开目录
put .\application_test\debug\dua\ER_BOOT    //上传文件
put .\application_test\debug\dua\ER_XRAM
```

bye

@echo transmit file is sucessfu　　//打印信息

ASIC 设计流程简介(转)



- 设计举例，tap 控制器，已完成代码编写及功能仿真：
- Tap_controller.v
- Tap_bypass.v
- Tap_instruction.v
- Tap_state.v
- 完成全部设计还需经过如下几个步骤：
- Pre_layout
- Synthesis
- STA using PrimeTime
- SDF generation
- Verification

■ Floorolanning and Routing

■ Post_layout

■ 反标来自 layout tool 的信息， STA using PrimeTime

■ Post-layout Optimization

Fix Hold-Time Violation

■ Initial Setup :建立设计环境，技术库文件及其它设计环境设置。

DC .synopsys_dc.setup 文件

```
company   =“zte corporation”;
designer  =“name”;
technology=“0.25 micron”
search_path=search_path+{“.” “/usr/golden/library/std_cells”
\
                          “/usr/golden /library/pads”}
target_library ={std_cells_lib.db}
link_library   ={“*”,std_cells_lib.db,pad_lib.db}
symbol_library ={std_cells.sdb,pad_lib.sdb}
```

■ Synthesis:利用约束完成设计的门及实现及扫描插入

■ Constrain scripts

```
            /* Create real clock  if clock port  is found */
               if  (find(port, clk) == {"clk"})  {
                          clk_name = clk
                create_clock -period clk_period clk
               }
            /* Create virtual clock if clock port is not found */
               if  (find(port, clk) == {})  {
                          clk_name = vclk
               create_clock -period clk_period -name vclk
}
```

```
/* Apply default drive strengths and typical loads for I/O ports
*/
            set_load 1.5 all_outputs()
           set_driving_cell -cell IV all_inputs()
           /* If real clock, set infinite drive strength */
             if (find(port, clk) == {"clk"}) {
            set_drive 0 clk
            }
        /* Apply default timing constraints for modules */
          set_input_delay 1.2 all_inputs() -clock clk_name
          set_output_delay 1.5 all_outputs() -clock clk_name
         set_clock_skew -minus_uncertainty 0.45 clk_name
       /* Set operating conditions */
         set_operating_conditions WCCOM
         /* Turn on Auto Wireload selection Library must support this
feature */
         auto_wire_load_selection = true
```

■ Compile and scan insert 的 scripts,采用 bottom_up 的编译方法

```
set_fix_multiple_port_net  -buffer_constants  -all
compile  -scan
check_test
create_test_pattern  -sample 10
preview_scan
insert_scan
check_test
```

如果模块内的子模块具有 dont_touch 属性需添加如下命令，因要插入扫描

```
remove_attribute find(-hierarchy     design,”*”)dont_touch
```

1. Write netlist

```
remove_unconnected_ports find(-hierarchy cell,”*”)
change_names  -hierarchy  -rules BORG
set_dont_touch current_design
write  -herarchy  -output active_design+”.db”
write  -format verilog  -hierarchy  -output
active_design+”.sv”
```

- Pre_layout 的 STA ：用 DC 的静态时序分析引擎做 block 的 STA,用 PrimeTime 做 full_chip 的 STA。
- Setup_time 分析
- Hold_time 分析

其时序约束和提供给 DC 做逻辑综合的约束相同。静态时序分析同综合一样，是一个迭代的过程，和 cell 的位置及布线关系密切，通常都执行多次，直到满足需要。

- SDF generation,pre_layout 的 SDF 文件，用于 pre_layout timing 仿真，同时还需提供时序约束文件（SDF 格式）给 layout tool 做布局布线。

  script 文件如下：

```
active_design=tap_controller
  read active_design
  current_design active_design
  link
Create_clock  -period 33  -waveform {0 16.5} tck
Set_dont_touch_network {tck trst}
  set_clock_skew  -delay 2.0  -minus_uncertainty 3.0 tck
  set_driving_cell  -cell BUFF1X  -pin Z all_inputs()
  set_drive 0 {tck trst}
  set_input_delay 20.0  -clock tck  -max all_inputs()
  set_output_delay 10.0  -clock tck  -max all_outputs()
  write_timing  -format sdf-v2.1 \
            -output active_design+”.sdf”
```

```
write_constraints  -format sdf  -cover_design\
                   -output constraints.sdf
```

- Verification
- 利用SDF 文件进行动态时序仿真:利用功能仿真时。用 verilog 编写的 test_bench 文件
- 形式验证:利用数学算法检查设计的逻辑的等效性，静态验证，需要的时间比动态仿真少，验证全面。在这里验证 RTL 描述与 gate_level 网表的逻辑等效性。
- Floorolanning and global Routing;
- Estimated 寄生电容和 RC delay 的抽取;
- 利用抽取的参数静态时序分析，若时序不满足要求，生成 custom wire_load 做 incremental synthesis 即 post_layout 的优化,采用 reoptimize_design -in_place 命令，其 script 文件需反标抽取的参数到设计;
- Detail routing;
- real 寄生电容和 RC delay 的抽取;
- 利用抽取的参数静态时序分析，修正 hold_time 如需要做 post_layout 的优化;
- 生成 post_layout 的 SDF 文件,做 gate_level 仿真验证;
- ECO
- Engineering change order,不属于正常的 ASIC 流程,一般只有在 ASIC 设计的后期，需要改变网表，可利用 ECO,例如,在 tape-out(sign-off) 以后，遇见设计的硬件 bug。采用ECO 可仅对设计的一小部分重新布线不影响 chip 其他部分的位置及时序，通常，修改不能大于 10%。
- 最新版本的 DC 提供由 ECO compiler, 可使设计者手工修改网表，节省时间。
- 一些 layout 工具也具有 ECO 功能。

DC 篇

dc_shell -f xxx.tcl

一般需要输出：网表文件 netlist,测试文件 spf（stil Procedures File）文件（用于 tmax 产生测试向量），时序报告

1. 逻辑综合的过程：

首先，综合工具分析 HDL 代码，用一种模型(GTECH) ，对 HDL 进行映射，这个模型是与技术库无关的；

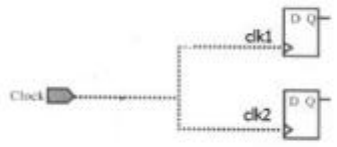
然后，在设计者的控制下，对这个模型进行逻辑优化；

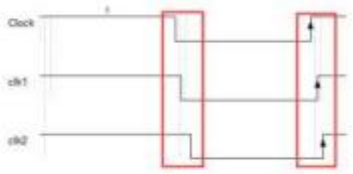
最后一步，进行逻辑映射和门级优化，将逻辑根据约束，映射为专门的技术目标单元库（target cell library）中的 cell，形成了综合后的网表

2. 时钟/时钟树的属性：

偏移(skew)

时钟分支信号在到达寄存器的时钟端口过程中，都存在有线网等延时，由于延时，到达寄存器时钟端口的时钟信号存在有**相位差**，也就是不能保证每一个沿都对齐，这种差异称为时钟偏移（clock skew），也叫时钟偏斜。时钟的偏移如下图所示：

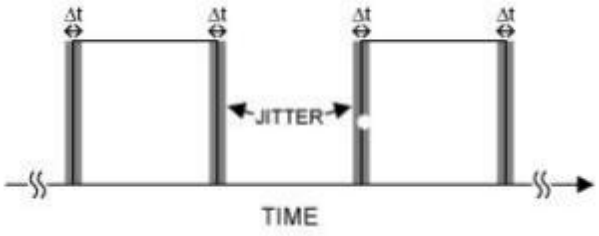




此外，时钟 skew 与时钟频率并没有直接关系，skew 与时钟线的长度及被时钟线驱动的时序单元的负载电容、个数有关

时钟的抖动（jitter）、

相对于理想时钟沿实际时钟存在不随时间积累的、时而超前、时而滞后的偏移称为时钟抖动,简称抖动，如下图所示



时钟的抖动可以分为随机抖动（Random Jitter，简称 Rj）和固有抖动（Deterministic jitter）：

• 随机抖动的来源为热噪声、Shot Noise 和 Flick Noise，与电子器件和半导体器件的**电子和空穴特性**有关，比如ECL 工艺的 PLL 比 TTL 和 CMOS 工艺的 PLL 有更小的随机抖动；

• 固定抖动的来源为：开关电源噪声、串扰、电磁干扰等等，与**电路的设计**有关，可以通过优化设计来改善，比如选择合适的电源滤波方案、合理的 PCB 布局和布线。

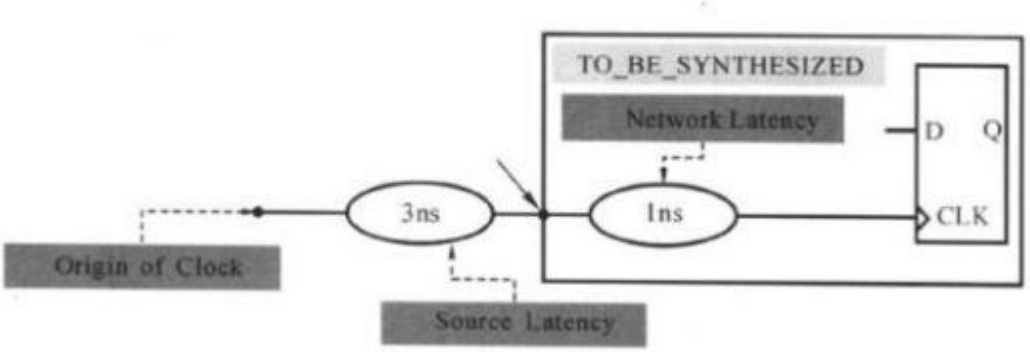
也就是说：**jitter 与时钟频率无直接关系**。

时钟的偏移和时钟的抖动都影响着时钟网络分枝的延迟差异(相位差异)，在 Design Compiler 里面，我们用**时钟的不确定性（uncertainty）**来表示这两种情况的影响。

使用***set_clock_uncertainty命令来设置***

延时（latency）、

时钟从**时钟源**（比如说晶振）出发到达**触发器时钟端口的延时**，称为时钟的延时，包含**时钟源延迟（ source latency）**和时钟**网络的延迟（network latency）**，如下图所示：



时钟源延迟(clock source latency)，也称为插入延迟(insertion delay)，是时钟信号从其实际时钟原点到设计中时钟定义点（时钟的输入引脚）的传输时间，上图是 3ns。

时钟网络的延迟(clock network latency)是时钟信号从其定义的点(端口或引脚)到寄存器时钟引脚的传输，经过缓冲器和连线产生的延迟(latency)，上图是 1ns。

OK，时钟的附加的 buff 属性差不多就是这样了。

转换（transition）时间

时钟的上升沿跳变到下降沿或者时钟下降沿跳变到上升沿的时间，这个时间并不是如左下图所示那样完全没有跳变时钟的，而是像右下图那样，时钟沿的跳变时间就是时钟的转换时间（后面约束的时候会有相关的解释）。

时钟的转换时间与与单元的**延时时间**（也就是器件特性）还有**电容负载**有关。

时序分析基本概念介绍——时钟 sdc 2018-01-25 22:26 来源:数字后端 IC 芯片设计

3. 时序分析时钟 sdc

与时钟相关的命令。主要有以下命令：

create_clock

create_generated_clock

set_clock_uncertainty

set_clock_groups

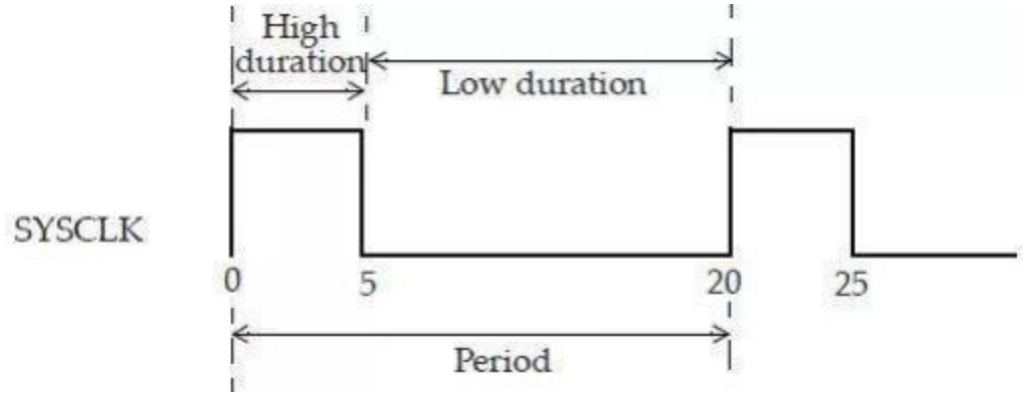
任何 sdc 首先定义的都是时钟，对于一个同步电路而言，缓存器和缓存器之间的路径延迟时间必须小于一个Clock 周期（Period），也就是说，当我们确认了 Clock 规格，所有缓存器间的路径的 Timing Constraint 就会自动给定了。Clock 规格主要包含 Waveform、Uncertainty 和 Clock group 的定

义。我们把它们称为时钟的三要素，当然创建任何时钟都要检查一下这三者有没有正确定义。

create_clock

主要定义一个Clock 的 source 源端、周期、占空比（时钟高电平与周期的比例）及信号上升沿及下降沿的时间点。

来看一个最简单的例子：



这个时钟描述成 sdc 语句就是：

create_clock -name SYSCLK -period 20

-waveform {0 5} [get_ports2 SCLK]

waveform 后面跟上升沿和下降沿的时间

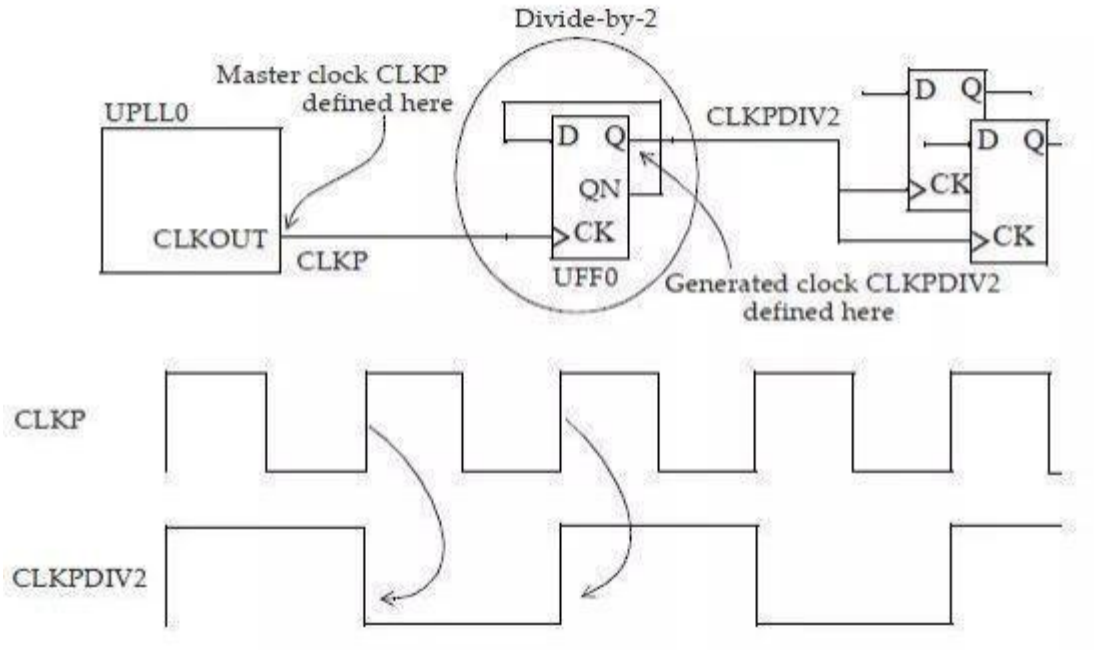
-waveform {time_rise time_falltime_rise time_fall ...}

如果没指定-period,默认的 waveform 为{0, period/2}

create_generated_clock

generated clocks 是另外一个重要的时钟概念

generated clocks 是从 master clock 中取得的时钟定义。master clock 就是指 create_clock 命令指定的时钟产生点，如图所示：



我们可以用如下命令来描述 generated clocks:

#定义 master clock

create_clock -name CLKP -period 10

-waveform {0 5} [get_pins UPLL0/CLKOUT]

#在 Q 点定义 generated clock

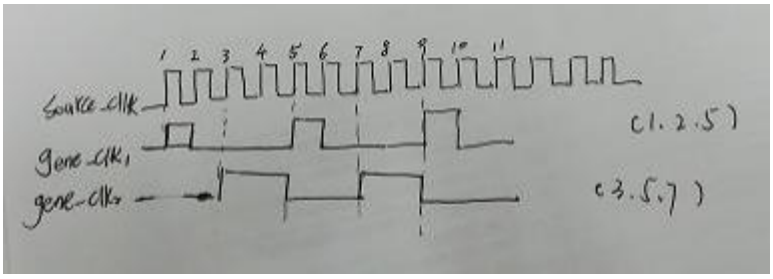
create_generated_clock -name CLKPDIV2

-source UPLL0/CLKOUT

-master_clock CLKP -divide_by 2 [get_pins UFF0/Q]

一般我们把时钟的源头会定义成 create_clock，而分频时钟则会定义为 create_generated_clock. 两者的主要区别在于 CTS 步骤，***generatedclock 并不会产生新的clock domain,*** *而且定义generatedclock后，clockpath 的起点始终位于master clock, 这样source latency并不会重新的计算。这是定义generated clock的优点所在。*

Generate clock 用-edge 产生波形关系.



Virtual clock

这边还有一个经常用的概念就是Virtual Clock，虚拟时钟。

前面介绍的 create_clock， create_generated_clock 都是 real clock。而 virtual clock 则不挂在任何 port 或者 pin 上，只是虚拟创建出来的时钟。如下所示：

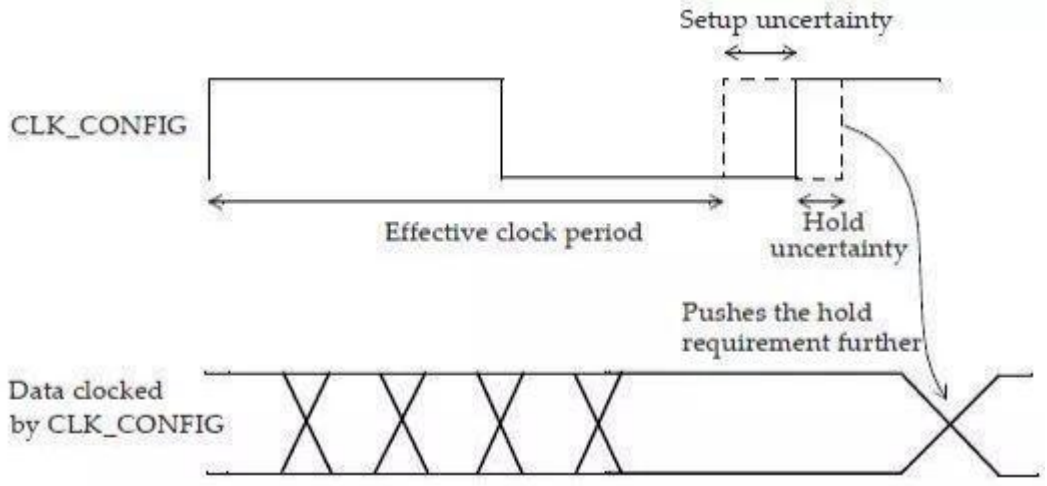
#定义虚拟时钟

create_clock -name VCLK -period 10 -waveform {0 5}

我们通常会把 input/output delay 挂在 virtual clock 上，因为 input/output delay 约束本来就是指片外的时钟，所以挂在虚拟时钟上较为合理。当然如果要省事情，直接挂在real clock 上也是可以的。

*set_clock_uncertainty*主要定义了Clock 信号到时序器件的 Clock 端可能早到或晚到的时间。主要是用来降低 jitter 对有效时钟周期的影响。值得注意的是，在 setup check 中，clock uncertainty 是代表着降低了时钟的有效周期；而在 hold check 中，clock uncertainty 是代表着 hold check 所需要满足的额外 margin。

来看下面一条 reg2reg path. 对照着如下时钟波形图。可以写出下面的约束。



set_clock_uncertainty-from VIRTUAL_SYS_CLK

-to SYS_CLK -hold 0.05

set_clock_uncertainty -from VIRTUAL_SYS_CLK

-to SYS_CLK -setup 0.3

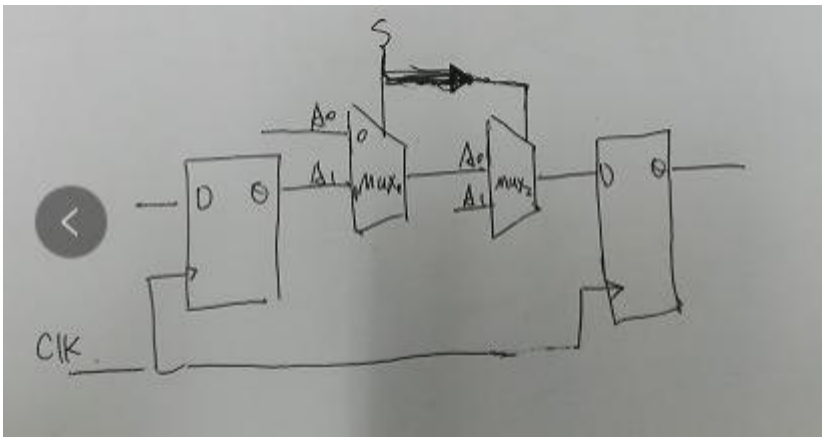
set_clock_uncertainty -from SYS_CLK

-to CFG_CLK -hold 0.05

set_clock_uncertainty -from SYS_CLK

-to CFG_CLK -setup 0.1

set_clock_groups___---效果与 set_false_path 相同,但在同步路径如果设置了 set_false_path,也不会做时序检查,如下



由于 mux1,mux2 的 sel 信号为同一个,那么路径只会有 mux1/A0 到 mux2/A0,mux1/A1 到 mux2/A1,而不会有 mux1/A0 到 mux2/A1,但他们在同一个时钟域,不能使用 set_clock_groups,只能使用 set_false_path;

set_false_path -throgh mux1/A0 -throgh mux2/A1

先通过 mux1/A0 后通过 mux2/A1

set_false_path -throgh mux1/A1 -throgh mux2/A0

定义完时钟后，我们也需要通过设置 clock group来确认各个时钟之间的关系。这是很重要的一步，因为通常我们还需要做 cross domain check，如果 clock group 设错了，会导致整个 STA 检查错误。一般有三个选项：

asynchronous，physically_exclusive 和 logically_exclusive。

asynchronous 代表两个异步的 clock group

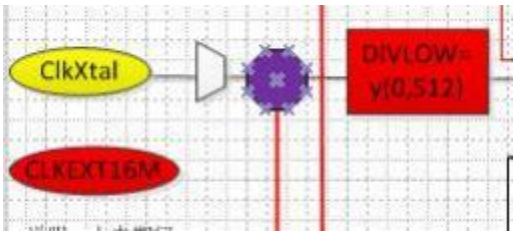
The following example defines two asynchronous clock domains:

```
prompt> set_clock_groups -asynchronous -name g1 \
   -group CLK1 -group CLK2
```

The following example defines a clock named CLK1 as asynchronous with all other clocks in the design:

```
prompt> set_clock_groups -asynchronous -group CLK1
```

physically_exclusive 代表两个 clock group 在物理意义上相互排斥，比如在一个 source pin 上定义了两个时钟。



如上图,clkxtal 可能使用 32k 晶体也可能使用 16m 晶体,如果定义了 2 个时钟,那么在同一个路径上会使用2 个周期之间互相做时序检查,如下 4 种情况

第一个寄存器按照 32k 时钟,输出到第二个时钟按照 16m 时钟来检查;

第一个寄存器按照 16m 时钟,输出到第二个时钟按照 32k 时钟来检查;

第一个寄存器,第二个寄存器按照 16m 时钟检查,

第一个寄存器,第二个寄存器按照 32k 时钟检查,

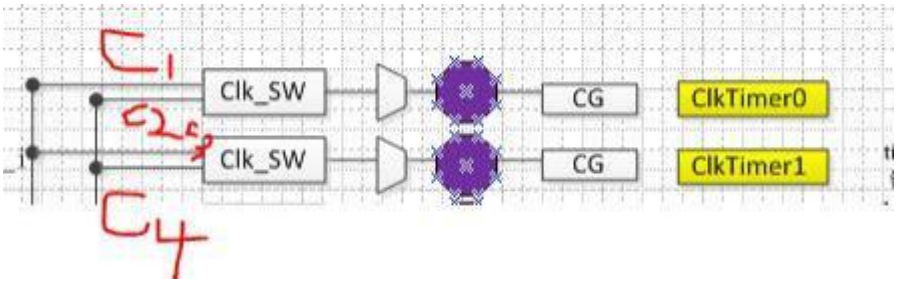
这样是不合理的,实际上只存在后面 2 种情况.

如果按下面设置,2 个寄存器就会按照 16m 和 32k 来检查

```
set_clock_groups -physically_exclusive \
        -group {CLK1} -group {CLK2}
```

但实际上由于只要满足 16m 时序,则会满足 32k,所以只要定义一个 16m 时钟就可以了

logically_exclusive 代表两个 clock group 在逻辑上相互排斥，比如两个 clock 经过 MUX 选择器。



Assume that a design contains two pairs of mutually-exclusive clocks that are multiplexed: CLK1 and CLK2, and CLK3 and CLK4.

If each pair of clocks is selected by a different signal, you must execute two set_clock_groups commands to simultaneously analyze all four clocks:

如上图,如果 1 和 2 的 select 信号是不同的信号,由于 c1和 c2 不能同时存在,c3 和 c4 不能同时存在;如果 clktmer0 与 clktimer1 之间有相关路径,那么理论上(如果没设置),会有 6 种时钟周期检查,如下

第一个寄存器按照 clk1 时钟,输出到第二个时钟按照 clk2 时钟来检查;
第一个寄存器按照 clk1 时钟,输出到第二个时钟按照 clk3 时钟来检查;
第一个寄存器按照 clk1 时钟,输出到第二个时钟按照 clk4 时钟来检查
第一个寄存器按照 clk2 时钟,输出到第二个时钟按照 clk1 时钟来检查;
第一个寄存器按照 clk2 时钟,输出到第二个时钟按照 clk3 时钟来检查;
第一个寄存器按照 clk2 时钟,输出到第二个时钟按照 clk4 时钟来检查

```
prompt> set_clock_groups -logically_exclusive \
   -group CLK1 -group CLK2
prompt> set_clock_groups -logically_exclusive \
   -group CLK3 -group CLK4
```

如果有上面的设置,则会有下面的检查

第一个寄存器按照 clk1 时钟,输出到第二个时钟按照 clk3 时钟来检查;
第一个寄存器按照 clk1 时钟,输出到第二个时钟按照 clk4 时钟来检查
第一个寄存器按照 clk2 时钟,输出到第二个时钟按照 clk3 时钟来检查;
第一个寄存器按照 clk2 时钟,输出到第二个时钟按照 clk4 时钟来检查

If each pair of clocks is selected by the same signal, you must execute only one set_clock_groups command to simultaneously analyze all four clocks:

```
prompt> set_clock_groups -logically_exclusive \
```

-group {CLK1 CLK3} -group {CLK2 CLK4}

如果有上面的设置,则会有下面的检查

第一个寄存器按照 clk1 时钟,输出到第二个时钟按照 clk3 时钟来检查;

第一个寄存器按照 clk2 时钟,输出到第二个时钟按照 clk4 时钟来检查

这样才符合实际情况

一个简单的例子:

set_clock_groups -physically_exclusive

-group {CLK1 CLK2} -group {CLK3 CLK4}

clock group 的定义异常谨慎，需要和前端再三确认。

时钟的定义就到此为止了，只是一些基础的概念，具体命令还有很多延伸扩展的内容，大家要去翻 doc，深入研究下！

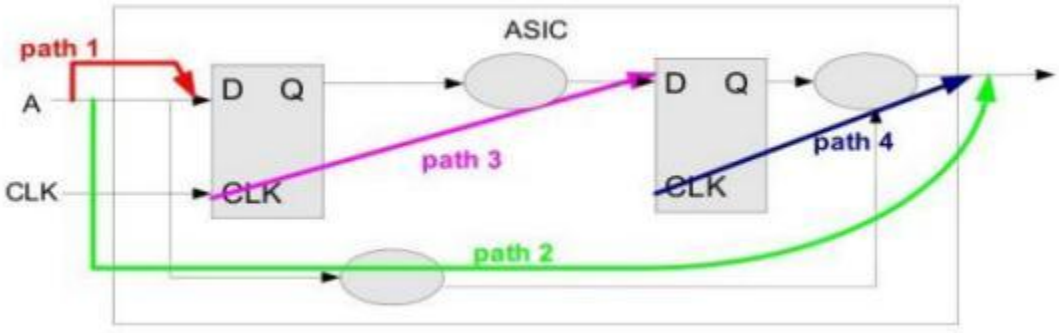
4. 基础概念

所谓 start point 就是：

1. **input port（顶层设计的输入端口）**
2. **clock pin of sequential cell（触发器的 clock pin）**

所谓的 end point 就是：

1. **output port（顶层设计的输出端口）**
2. **data pin of sequential cell（触发器的 data pin）**



1. DC 用于逻辑综合，与布局无关，连线用的是负载模型，输入的是 RTL 代码；PC，物理综合，与布局相关，综合时需要布局信息，输入的是 DC 综合后的门级网标；

2. DC 将设计对象分为 8 类，

Design:具有某种或多种逻辑功能的电路描述；

Cell：设计的 instance；

Reference:cell 或 instance 在库中定义的名字；

Port:design 的输入、输出；

Pin:design 中 cell 的输入、输出；

Net:ports 和 pins 之间或 pins 之间的信号名；

Clock:被定义为时钟源的 pin 或 port；

Library:cell 的集合，如：target_library,link_library；

3. 变量

一种标识符，DC 用于存储信息，可做为命令的操作参数；注：变量值不会被保存在设计数据库中。

4. 属性

DC 用于存储具体的设计对象，如：nets,cells,clocks 的信息，一般来说，属性都是 DC 预定义,命令如下：

set_attribute <object list>get_attribute <object list>

<attribute name><attribute name>

< attribute value>

dc_shell>get_attribute STD_LIB default_max_transition

5. DC 支持的文件格式

*.db 文件，DC 的内部文件格式

Verilog 文件

VHDL

EDIF:electronic design interchange format,用于不同工具间门级网表的移植

6. DC 的文件类型

Script files <filename>.scr

RTL verilog file <filename>.v

RTL VHDL file <filename>.vhd

Synthesized VHDL netlist <filename>.svhd

EDIF file <filename>.edf

Synthesized verilog netlist <filename>.sv

Synopsys database file <filename>.db

report <filename>. rpt

Log files <filename>.log

7. Synopsys technology library

technology library 由半导体制造商提供，定义 cell 的相关信息及设计标准约束，分为两类：

target libraries：在门级优化及映射时，提供生成网表的 cell；

link libraries：提供设计网表中可引用的 cell，DC 不利用 link libraries 的 cell 做综合；

Synopsys technology library 结构组织可看作实际上的技术库标准，它是一个文本文件(扩展名“.lib”），在使用之前，需被 LC 编译，产生一个“.db”文件，其构成如下：

Library Group;

Library level attributes;

Environment description;

Cell description;

8. Environment description:主要包括：

缩放因子（scaling factors)

时序模型（timing rang models)

操作条件（operation condition)

线性负载模型（wire-load models)

缩放因子（scaling factors 或 K-factors）：实际上是一个乘数因子，表示随着 PVT 的变化，对设计延时的修正。例如：

K_process_fall_transition :1.0;

K_process_rise_transition :1.2;

K_temp_fall_transition :0.03;

K_temp_rise_transition :0.04;

K_volt_fall_transition :0.02;

K_volt_rise_transition :0.5;

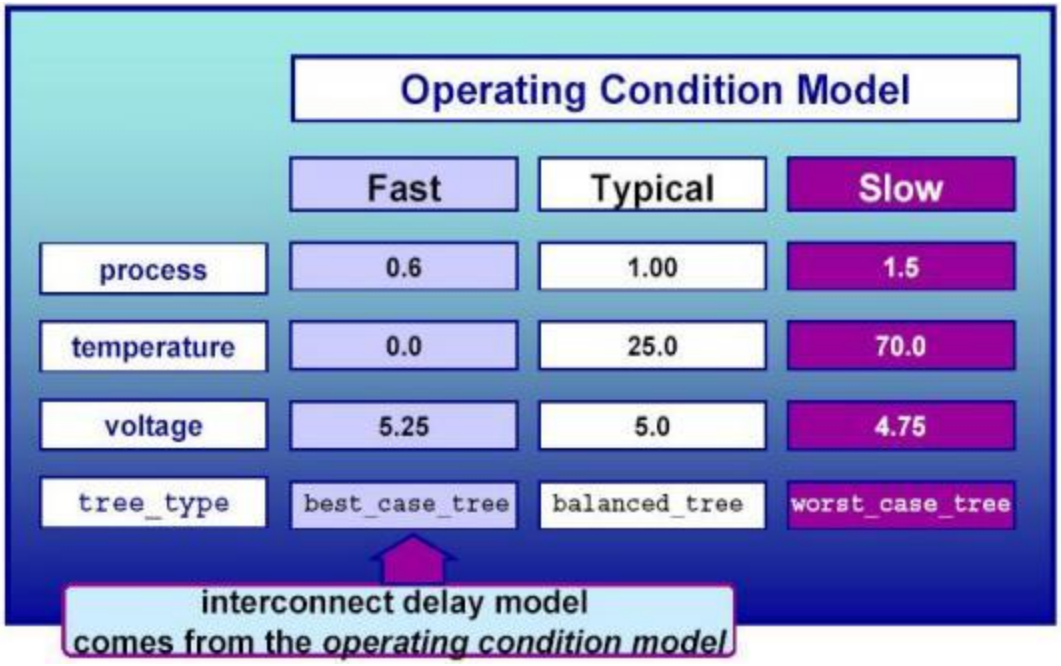
时序模型（timing rang models)：根据操作条件的变化，修正信号到达时间，

如：

```
Timing_range(BEST) {
            faster_factor  :0.5;
            slower_factor  :1.3;
}
```

操作条件（ operation condition)：定义了 PVT 和 RC 树模型。

DC 根据操作条件缩放 cell 和 net 的 delay;



9. 编码指南

RTL 级的 HDL 描述实际是电路结构的文本描述，它是技术无关的，类似于网表。HDL 语言先于综合工具出现，综合工具只能根据 HDL 代码进行逻辑推断，对完成同一功能而书写方式不同的 HDL 代码，综合工具会产生不同逻辑电路。所以不能依靠 DC 去修正不符合规范的代码。写代码时，需清楚描述的硬件电路拓扑结构。

RTL 描述内容如下：

寄存器的结构；

电路的拓扑；

寄存器之间的功能；

DC 仅对寄存器之间的逻辑进行优化,不会对寄存器的排列进行优化。

HDL 代码的编写应注意技术独立性，尽量减少引用在某些库中定义门的hard_coded。若必须引用，将这些引用门放在一个单独的模块内。

Clock 逻辑

Clock 逻辑及reset 生成应放在一个模块中，便于综合时对时钟约束的定义；

Clock 命名在设计的各层次应保持一致性；

在一个模块内不要由多个时钟；

对于时钟源的mux，用于测试时控制时钟，最好人为实例化，便于 set_disable_timing 的使用；

在 top level 不要有粘和逻辑；

一个文件仅包含一个模块，模块名与文件名相同；

I/O pad 应为单独的模块；

最小化不必要的层次，过多的层次会降低电路的性能，因 DC 逻辑优化不能跨越模块的层次；

所有模块的输出直接来自寄存器的输出；

状态机

对 verilog,State 命名使用“parameters” ；

组合逻辑和时序逻辑分开

10.Startup 文件必须定义如下变量：

Search_path:指明库文件的位置

Target_library:既技术库，由生产厂家提供，该库中的 cells,被 DC 用于逻辑映射。 Target library 的文件名应包含在 Link library 的文件清单中，用于 DC 读取门级网表。

Link_ library:该库中的 cells,DC 无法进行映射，例如：RAM,ROM 及 Pad，在 RTL 设计中，这些 cells 以实例化的方式引用。

Symbol_library:该库文件包含技术库中 cells 的图形表示，用于 DA 生成门级示意图。

Target_library 和 Link _library 为设计者提供了将门级网表从一种技术在映射 到另一种技术的方法，将旧的Target library文件名包含在 Link_library 的文件清单中，而 Target_library 包含新的 Target library 文件名，利用 translate 命令实现。

11.RTL design Entry 向 DC 输入 HDL 描述的设计

命令：read 和 analyze&elaborate，推荐用 analyze&elaborate

analyze&elaborate 允许设计者在设计的 GTECH 建立之前，首先去分析设计的语法错误和进行 HDL 代码转换。GTECH 由“soft macros”such as adders, comparators 等组成，这些组件来自 synopsys 的 synthetic lib，每种组件具有多种结构。

Analyze 做语法的检查并产生一个“.syn”文件，存储于 work 路径下的定义的设计库内，可供后来 elaborate 使用。对于一个 analyzed 设备，只需用 elaborate 重新输入，节省时间。Read 就不行。

Environment constraints

功能：定义设计的工艺参数，I/O 端口属性，统计 wire-load 模型。下图解释了描述设计环境约束的 DC 命令：Set_operating_conditions <name of operating conditions>用于描述操作条件：process(电压和温度的比例因数),voltage,temperature, cell 和 wire 的 delay 和操作条件呈线性关系。如：Set_operating_conditions WORST（或 TYPICAL、BEST)

命令 set_operating_conditions -min BEST -max WORST 用于指示 DC 对设计的 WORST 和 BEST 条件，同时优化。

wire_load 模型的选择很重要，太悲观或太乐观的模型都将产生综合的迭带，在 pre-layout 的综合中应选用悲观的模型。命令格式如下：

dc_shell>set_wire_load MEDIUM -mode top

12.design and clock constraints

功能：描述设计的目标，包括时序和面积约束，要注意约束必须是可实现的，否则会导致面积超额，功耗增加或时序不能满足要求。设计约束的 DC 命令如下主要包括两点

约束综合模块的最大面积（ set_max_area ）

约束综合模块 timing path(Create_clock, Set_input_delay, Set_output_delay)

时钟描述

时钟的描述在设计中很关键，传统上，在 clock source 加很大的 buffer 去驱动整个时钟网络，布线时，使时钟网络成鱼骨状，用于减少时钟网络延时和 clock_skew。对于 VDSM，传统的方法已不适用，而是由 layout 工具根据 cell 的位

置综合时钟树，以满足我们对时钟的需求。下面，我们介绍一下描述时钟的 DC 命令。

时钟 DC 命令介绍

Create_clock:用于定义时钟的周期和波形（duty 及起始沿）；

例如：create_clock -period 40 -waveform{0 20} CLK 周期 40ns 上升沿 0 ns, 下降沿 20 ns;

对于仅包含组合逻辑的模块，为了定义该模块的延时约束，需创造一个虚拟时钟定义相对于虚拟时钟的输入输出延时。例如：

create_clock -name vTEMP_CLK -period 20

Set_clock_transition:在pre_layout必须设置一个固定的transition值(由技术库提供)，因为时钟网有很大的 fanout.这样可使 DC 根据该时钟计算实际的延时值。

Set_clock_skew：设置时钟的skew及delay，pre_layout 和 post_layout 命令选项不一样。-propagated 选项让 DC 计算时钟的 skew。

13.Pre-layout 时钟 DC 命令介绍:

估计时钟树的延时和抖动，DC 命令如下:

create_clock -period 40 -waveform {0 20} CLK

Set_clock_skew -delay 2.5 -uncertainty 0.5 CLK

Set_clock_transition 0.2 CLK

set_dont_touch_network CLK

set_drive 0 CLK

考虑到 layout 后时钟网络的变化可若下设置时钟 skew：

set_clock_skew -delay 2.5 -minus_uncertainty 2.0 -plus_uncertainty 0.2 CLK

-minus_uncertainty 用于 setup-time 的计算， -plus_uncertainty 用于 hole-time 的计算.

一个 cell 的 delay 使根据 input signal 的斜率和 output pin 的电容负载决定，对于时钟信号，因为 clock network 的 fanout 很大，从而造成 clock network 末端门的时钟信号的 clock transition time 很慢，使 DC 计算的门延时失真。

14.post-layout 时钟 DC 命令介绍:

这个阶段，用户不需定义时钟的延时和抖动，他们由时钟树决定。 clock transition time 也不需定义。如果 layout 工具提供与 DC 的直接接口，则直接将包含有时钟树的网表回馈给 DC,不需在 script 文件中对时钟的延时和抖动进行描述，如果 layout 工具不能实现此功能，则需用户从 layout 工具提取时钟的延时和抖动信息，描述命令同pre_layout.如果含有时钟树的网表能够移植到 DC，则 clock 的命令描述如下：

create_clock -period 40 -waveform {0 20} CLK

set_clock_skew -propagated -minus_uncertainty 2.0 -plus_uncertainty 0.2 CLK

set_dont_touch_network CLK

set_drive 0 CLK

另外，很小的 clock uncertainty 定义的目的是考虑 process 的变化。

如果无法得到包含有时钟树的网表,只有 SDF 文件，则对原网表只需定义时钟，并将 SDF 文件回馈给原网表，时钟的延时和抖动由SDF 文件决定。

15.输入路径 DC 命令介绍:

Set_input_delay:定义信号相对于时钟的到达时间。指一个信号，在时钟沿之后多少时间到达。

例如：set_input_delay -max 23.0 -clock CLK { dataout }

set_input_delay -min 0.0 -clock CLK { dataout }

输出路径 DC 命令介绍:

Set_output_delay:定义从输出端口数据不可用开始距后一个时钟沿的时间：即时钟周期减去 cell 从上一个时钟沿开始的工作时间。

如：set_output_delay -max 19.0 -clock CLK {dataout}

用该命令对一些信号进行 over-constrain,从而获得最大 setup-time.但可能导致面积和功耗的增加。一个负值（如：-0.5）可在 layout 后，被 in-place optimization 用于为 hold_time 修正提供 timing margin.

16.其它设计 DC 命令介绍

Set_dont_touch_network,常用于 port 或 net 阻止 DC 隔离该 net，和该 net 相连的门具有 dont_touch 属性。常用于 CLK 和 RST

例如：Set_dont_touch_network{CLK,RST} 。

当一个模块例用原始的时钟作为输入，在该模块内部利用分频逻辑产生了二级时钟，则应对二级时钟 output port 上设置 set_dont_touch_network.

当一个电路包含门时钟逻辑时，若在时钟的输入设置 set_dont_touch_network，则阻止 DC 隔离该门逻辑，导致 DRC 发现时钟信号冲突，对门 RESET 同样。

Set_dont_touch，应用于 current_design,cell,net,references.阻止 DC 对模块中的这些元素进行技术映射。

例如： Set_dont_touch find(cell,”sub1”)

Set_dont_use:用于.setup 文件用此命令可将技术库中的某些 cell 滤出，禁止 DC 映射；

例如： Set_dont_use {mylib/SD*},将技术库中名字以 SD 起头的 flip-flops.

17.Advanced constraints

Path:每一条路径都由 startpoint 和 endpoint

statrpoint:input ports 或时序 cell 的 clock pins;

endpoint: output ports 或时序 cell 的 data pins;

Path_delay

Set_false_path：

指示 DC 不要对指定的路径按照时序约束优化，如：异步路径或约束不可实现的路径。false path 路径的鉴别在设计中很关键，如果不对 false path 路径进行标识，DC 会对所有的路径进行优化，从而影响关键时序路径。此命令用于当因为有 false path 关键逻辑时序静态分析失败时。

1.在 clock 域之间的 false path：

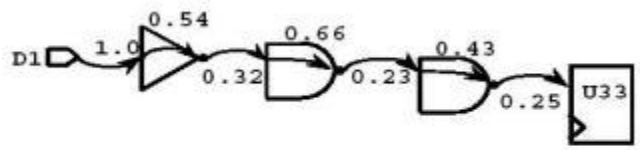
set_false_path -from [get_clock CLKA] -to [get_clock CLKB]

2.logic 的 false path：

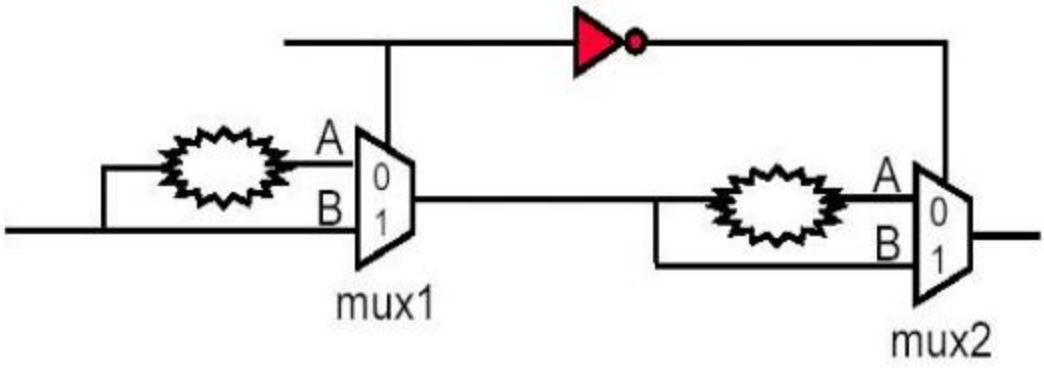
dc_shell>set_false_path -through mux1/A -through mux2/A

dc_shell> set_false_path -through mux1/B -through mux2/B

3. 对含有 tristates 的 path,DC 总是认为 tristates 时能，会产生 false path，推荐将 tristates 最好移致顶层，在子模块中就无 false path



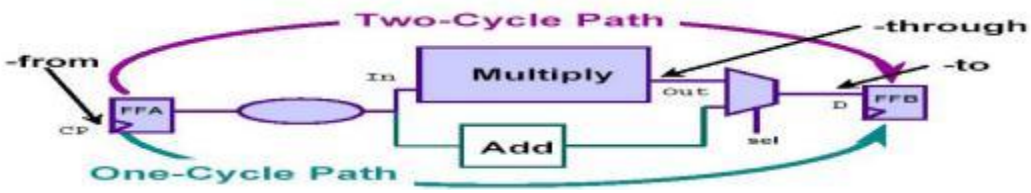
path_delay = (1.0 + 0.54 + 0.32 + 0.66 + 0.23 + 0.43 + 0.25) = 3.43 ns



Set_multicycle_path:因为 DC 假设所有的路径都是单周期的，为了满足时序，对多周期路径会做不必要的优化，从而影响相邻路径或面积。所以这个命令用于隔离多周期路径，通知 DC 通过这条路径所需的周期数。

例如：

```
dc_shell> set_multicycle_path 2 -from FFA/CP \
                                -through Multiply/Out -to FFB/D
```



18.Group_path:将设计中的时序关键路径绑在一起，可以使路径组间具有优先次序，

命令格式：dc_shell>group_path -to {out1 out2} -name grp1;

但添加太多的组会增加编译时间；且会增加最坏违例路径延时。

19.Set_max_delay

对于仅包含组合逻辑的模块，用此命令约束所有输入到输出的总延时。

例如： set_max_delay 5 -from all_inputs() -to all_outputs

对于含有多个时钟的模块，可用通常的方法定义一个时钟，用此命令进行约束定义时钟和其他时钟的关系。

例如： set_max_delay 0 -from CK2 -to all_register(clock_pin)

该命令还是用于包含 gated clocks or resets 的设计。

Set_min_delay，

对于仅包含组合逻辑的模块，定义指定路径的最小延时

例如： Set_min_delay 3 -from all_inputs()

set_fix_hold 一起使用，只是 DC 添加一定延时，满足最小延时的定义。

20.Compilation Strategies

编译就是将 GETCH 逻辑网络映射为技术库的 gate cell，使设计满足用户的约束。

对于一个层次设计分两步实现：

映射各子模块到逻辑门，不考虑约束；

根据时序和面约束优化逻辑，修正模块分界间的违例；

设置编译命令 compile 的选项，进行不同的编译过程

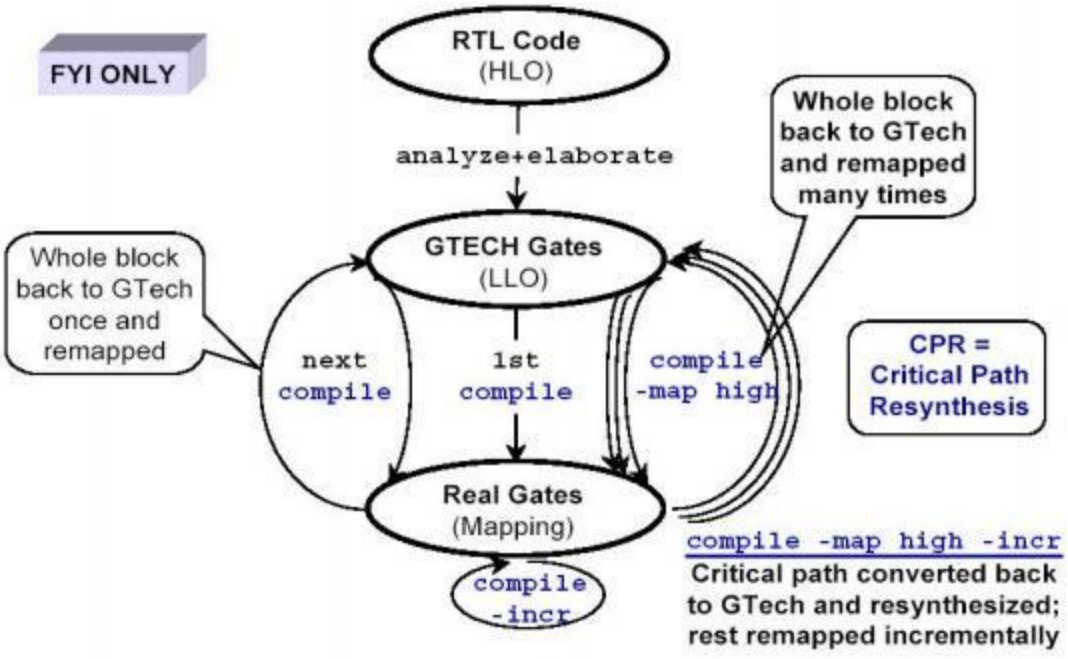
Multiple Instances：在一个模块中，多个实例引用同一个模块定义，会引起 DC 无法绑定各个实例的约束。可采用unquify 命令或编译定义模块时，将其属性设为 dont_touch。

Synopsys 推荐三种编译策略，由设计者根据设计的结构及定义选择。

Top-down hierarchical compile；

Time-budget compile ；

Compile-characterize-write-script-recompile(CCWSR)；



21.Resolving Multiple Instances

Multiple Instances:在一个模块中，多个实例引用同一个模块定义，如下图

如采用 time-budgeting,将 moduleA 分开综合，在编译 moduleB 时，DC 退出编译，显示错误信息。

解决办法：

设置 moduleA 为 dont_touch，编译。

采用unquify 命令使实例 U1,U2 分别对应唯一的定moduleA_U1 和 moduleA_U2。DC 命令为： dc_shell>unquify

建议无论选择何种编译，都采用unquify 命令，唯一化设计，因为 dont_touch 会阻止 DC 对 U1，U2 进行优化。

22.Top-down hierarchical compile：

就是将整个设计作为一个模块进行编译，仅需 top level约束。对一些设计如：单时钟的设计，很适用。允许一次编译大于 100K gate 的设计。对于大型设计，可将子模块合并成一组，然后展开进行编译，可以改善时序。

优点：

仅需 top_level 的约束；

将整个设计作为整体优化，可获得较好的结果；

缺点

编译时间长；

子模块的改变需要整个设计在综合；

如果设计包含多个时钟或生成时钟逻辑，则不能很好执行。

例如： analyze -format vhdl {alu.vhd... risc_core.vhd }

elaborate RISC_CORE

include scripts/top_level. scr

compile

23.Time-budget compile

是一种Bottom-Up 的方法，各自模块可独立编译，适合于划分合理，各自模块的时序约束能够被明确定义的设计。

优点：

每一个子模块都由自己的 Scripts 文件，便于管理；

子模块的改变不需要整个设计全部重新综合；

不需要关心设计类型，如：多时钟或产生时钟；

一般来说，可获得好的结果；

缺点

需要更新维护多个 scripts;

Top 的关键路径也许在子模块并非关键路径；

需要 incrementally 编译，用于 DRC’s;

24.Compile-characterize-write-script-recompile(CCWSR)

适用于不能很好定义子模块内部连结的大型设计，不受硬件存储器的限制。这种方法首先编译每个子模块，需要 top level 约束；然后用 top level 约束 characterize 每一子模块， top level 的时序信息被传播给各个子模块；在执行 write_script 命令生成子模块的约束文件；最后 re-compile 每一格模块。

优点

对存储器不敏感；

由 DC 完成子模块间的优化，获得好的结果；

生成的 scripts 文件可由用户修改；

缺点

生成的 scripts 不易读；

为了各子模块间的接口稳定，需多次综合；

底层模块的改变需要再综合整个设计；

25.Optimizing design

1. Design space exploration 使设计的电路获得最快的逻辑及最小的面积的过程。为了实现

   Design space exploration， 我们假设 HDL 代码被冻结，通过综合与优化来最小化面积及满足时序的要求。

   DC 的编译流程如下：

   delay design rule Fixing design rule Fixing Area

   (delay prioritized) (DRC prioritized)

   现在版本的 DC 编译流程是时序优先，需要加面积约束，进行面积优化。提供给 DC 可实现的约束

   是重要的，如果时序过紧，会生成“vertical logic”,如果时序约束不存在，

   会生成“horizontal logic”,违反时序的定义 。如下图

   而且，约束和延时的关系仅在一定范围内成正比，在这个范围之外，过紧的约束不会减少延时，

   过松的约束也不会增加延时。一般推荐，提供给 DC 的约束比需要的紧 10%，减少综合排版的迭带。

   2.worst Negative slack，对一条指定路径，信号从起点到终点的违例时间

   3.Total Negative Slack

   DC 为了减少模块总的时序违例，需依次减少各个 WNS，因为这个理由，grouping path 和关键

时序段的定义都需被考虑，对于 TNS 为每个终点的 WNS 的和

26.Optimization Techniques

1. Flattening:适合于随机逻辑与控制逻辑，它并不合适包含结构逻辑的设计如：超前进位加法器或乘法器。

目的在于通过移去中间变量，减少输入和输出之间的逻辑层次，提高速度，但会带来面积上的压力。

Flattening 的执行是独立与设计约束的，Flattening 导致在输入输出之间的逻辑是一个 vertical 逻辑。根据设计 flattened 的形式和 effort 的类型，被 flattened 的设计在最终的及术映射优化之前，还要被 structured。用于减少面积。如果用“-map_effort high”,则 DC 不可能 structured 设计，通常用缺省的设置就足够了。对时序不满足的设计对象，可先 flattened,然后 structured(缺省）,若时序还不满足，则关掉 structured。设置 set_flatten 的-phase 为“true”,既先 structured，后 flattened，将产生非凡的结果

。这可使 DC 对两种优化方法产生的逻辑进行比较。对于层次设计，flattened 仅适于当前设计，不被子模块继承。命令如下：

```
set_flatten <true|false>
            -design <list of design>
            -effort <low|medium|high>
            -phase<true|false>
```

2.Structuring:用于结构逻辑，如超前进位加法器， 目的就是通过添加中间变量，使逻辑共享。但会增加逻辑的延时。例如：

before structuring	after structuring
P=ax+ay+c	P=aI+c
Q=x+y+z	Q=I+z
	I=x+y

Structuring 伴随着 timing 约束(缺省）和 Boolean 优化，通 Boolean 优化可减少面积，但会影响时序，

常用于非关键时序电路，如：随机逻辑和有限状态机。

设置 compile_new_boolean_structure= true,使DC采用新的boolean 优化算法，节省时间。命令格式如

下：

```
set_structure <true|false>
```

-design <list of design>

-boolean < true|false >

-timing<true|false>

一般情况，该命令的缺省设置就可产生满意的结果。如果你的设计模块时序并非关键，你可设置

set_max_area 0 并执行 Boolean 优化,否则， structure 仅根据 time 约束进行优化。

Optimizing clock Networks:传统的方法，在时钟源加一个大 buffer，鱼骨结构的时钟网布线（用于减少延时和抖动）仅适用 0.5um 及以上工艺。对于 VDSM 技术，这种方法会增加综合和排版的迭代次数。现在的方法是在排版工具内综合时钟树，时钟树的综合在 cell布局以后布线以前，目的是使排版工具知道寄存器的确切位置，在合适的位置放置buffers，在布局布线中执行时钟树的综合可减少迭带。但在布局布线前的综合的过程中，我们仍然要优化时钟,从而得到更好的结果。方法如下：

在时钟 pin 设置 set_dont_touch_network,使 DC 不会 buffer up 时钟网，这个方法对于不包含门时钟的设计都可满足。对包含门时钟的设计，因 set_dont_touch_network 会沿着时钟网组合逻辑进行传播，直到遇到寄存器，从而使门逻辑具有 dont_touch 属性，会阻止 DC size up 门逻辑，造成 DRC 违例。为了避免这种情况，需移去 set_dont_touch_network 并执行 incremental compilation。

第二种方法就是利用report_net 找出所有的高扇出net,利用 balance_buffer 命令加 buffer ，balance_buffer 并没有考虑时钟抖动，不能代替时钟树的综合。

在执行 in-place-optimization 时，利用 compile-in_place 命令，将 compile_ok_to_buffer_during_inplace _opt 设置为 false,这样 DC 不会再门逻辑上加 buffer，仅 size up 门逻辑。

4.Removing hierarchy：

因为DC 无法跨越模块间优化，一些不好的划分会阻止 DC 获得更好的结果，所以我们移去不必要的层次，使设计划分合理，获得更好的结果。

5.Optimizing for Area

DC 总是优先根据时序来优化设计，对于时序非关键而对面积敏感的模块，可在初始编译中定义面积的需求，不用定义时序的约束。另外，对具有高驱动强度的门设置 dont_use 属性，因为高驱动强度的门虽然加快了逻辑的速度，但具有较大的面积。

6.Incrementally compile:设计映射为门后，时序和面积约束可被再定义，这时设计采用 incrementally 编译。 incrementally 编译确保 DC 维持以前的电路结构，只做门级的优化，可改善设计的时序与性能，不会增加不必要的逻辑。

27.内部扫描：Design For Test

内部扫描插入技术主要指将设计中所有的 flop 用含有 built-in 逻辑的 scan flop 代替，一般 scan_flop 的结构为复用 flop，和普通的 flop 之间有时序的差别，所以，我们应考虑 scan_flop 对设计的影响。

DC 可以通过编译自动实现扫描的插入，从而可在综合设计时直接将 flop 映射为 scan-flop, 也可用其他的方法完成扫描的插入。静态时序分析应采用扫描插入之后的网表，因为 scan-flop 和普通的 flop 之间有时序的差别。实现 test_ready 编译的命令如下：

在编译之前，须通过 set_scan_configuration 命令设置时钟混合、扫描链数、扫描的类型例如： set_scan_configuration -style multiplexed_flip_flop;

命令 compile -scan 指示 DC 直接用 scan_flop 编译，但不链接成扫描链。

命令preview -scan 用于预览由 set_scan_configuration 命令设置的扫描结构。

命令 check -test 推荐在编译后检测设计是否存在违反可测试性设计的有关规定。设计者根据 DC 提供的信息对违例的设计进行修正，推荐修改 RTL 代码

虽然扫描链还没有被插入，但可以通过 ATPG 的产生对设计的故障覆盖率进行评估。命令如下：

create_test_patterns -sample<n>

注意故障覆盖率应按照best case 条件考虑，因为设计如果是一个大设计中的

子模块，该模块嵌入顶层模块时，它的 input ports 的可控性与 output ports 的可观察性可能会发生变化，从而设计整体的故障覆盖率可能比预期的低。

通过对设计某部分的修改，使故障覆盖率达到要求，然后我们用 insert_scan 命令，建立并排序扫描链，优化扫描链，插入额外的测试逻辑，使三态失效。

扫描插入之后，用 report_test 命令获取含有设计的测试信息的报告。

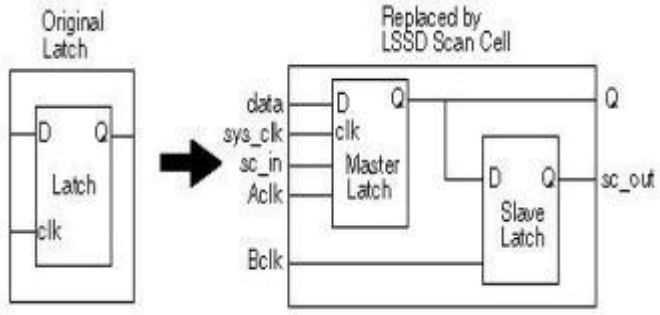
28.DFT 设计指南

1. 内部 tri-state 总线

在扫描的过程中，总线上的驱动也许会同时驱动总线，引起竞争。解决这个问题可以通过添加一个译码逻辑，通过一个mux 去控制每一个 tri-state 驱动的输入使能，mux 用于在普通的操作信号和译码器的输出控制信号之间进行选择，在扫描模式下，选择译码器的输出控制信号，而译码器的输入由外部直接控制，这样，我们可控制 tri-state 驱动的开、和关。

2.Latches

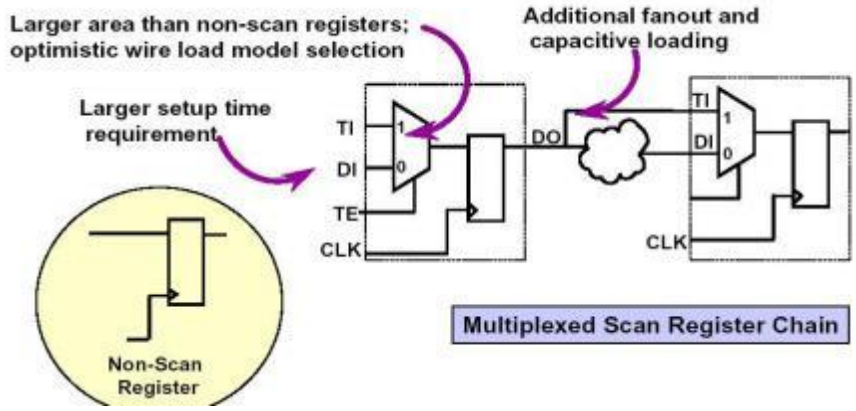
在设计中避免用Latches，因为测试很困难,为使 latch 在扫描模式下透明，需作如下替代。



3.Gated or Generated Clocks

DFT 要求每一个 flop 的 clk 可控，所以对于 Gated or Generated Clocks 的逻辑电路用mux 进行旁路，如下图所示,分频逻辑被旁路，在这种情况下，分频逻辑无法扫描，通过如下命令通知 DC 将此cell从扫描链中排除:

```
dc_shell>set_scan_element false<list of cells or design>
```



4.用时钟的 single Edge

大部分设计都用时钟的 single Edge，但有些情况下，时钟的 rising 和 falling Edge 都要用到，这种情况在测试模式下可能产生时序问题，我们可为这部分电路加一个 mux,是整个设计在测试模式下用相同的时钟边沿。例如：

```
process(clk,test_mode)
begin
    if(test_mode= ‘1’ )then
        muxed_clk_output<=clk;
    else
        muxed_clk_output<=not(clk);
    endif
endprocess
```

5.不可扫描的存储元素

因为存储器(如：RAM）周围无扫描链，所以和它输入输出相连的组合逻辑会产生故障覆盖率的损失，另外，在扫描模式下，RAM 的输出是“unknown”，会影响扫描链故障，解决的方法如下：

6.多时钟域

推荐设计者为不同时钟域安排各自的扫描链路，在一个扫描链内跨接不同的时钟域会因不同的 clock skew 而导致时序问题，但这种方法导致不同长度的扫描链。替代方法是将 flop 根据时钟域分组，然后链接他们成一个扫描链，这需要保证不同时钟域之间的时钟偏移最小。时钟源应可从外部接入，便于在设备测试时可控。另

外一个解决方法是，在时钟源使用时钟复用，从而在测试时只使用用一个时钟。

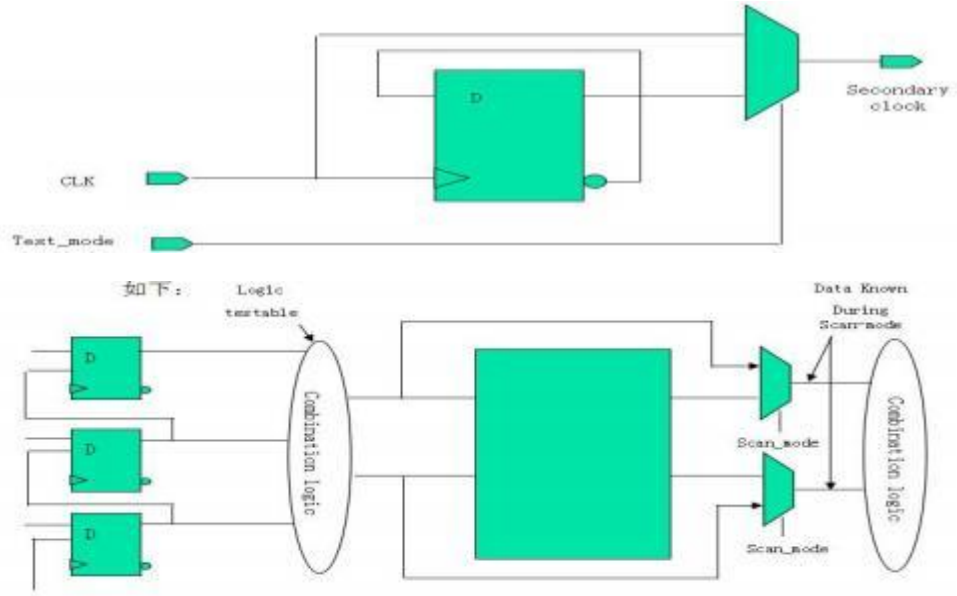
7.排序扫描链最小化时钟偏移

扫描链的时钟偏移会引起 hold-time 违例，虽然扫描时钟的频率较正常工作慢，但 hold-time 与时钟频率无关。我们将具有较大时钟延时的 flop 放在扫描链首，我们将具有较小时钟延时的 flop 放在扫描链尾，从而减少时钟偏移。

8.Gated Reset 和 Preset

Flop 的 reset 和 Preset 必须可控，如果 flop 的 reset 和 Preset 被 gated,则需添加mux，在扫描模式

下旁路门逻辑。



29.LINKS TO LAYOUT ANDPOST LAYOUT OPTIMIZATION

Links to layout 简称 LTL，是 DC 和 layout 工具间的信息交换接口，用于 DC 和 layout 工具之间交换时序信息和布局信息，post-layout 优化，减少综合与排版之间的迭带。我们从以下几个方面介绍如何将综合过程和 layout 过程结合起来，进行 post-layout 优化。

Layout 的网表生成

Layout

参数的抽取

Post-Layout Optimization

30.Layout 的网表生成

许多 layout 工具只接受 Verilog 和 EDIF 网表, EDIF 网表不易读以至在后来执行 ECO 很麻烦，并且EDIF 网表不能仿真从而无法发现在 EDIF 转换过程中出现的错误，所以推荐用Verilog 网表作为 layout 工具的输入。在将网表送往 layout 之前，建议对网表如下处理：

Uniquify 网表

通过改变设计中的 net 名简化网表

移去无连结的 ports

确保所有 cell 的 pin 名都可见

检查 assign 和 tran 语句

检查无意义的 gating clock 或 reset

检查未定义的引用

31.Uniquify

为了在 layout 中进行时钟树的综合，网表在 DC 中必须被 uniquified.所谓 uniquified 就是在设计中使子模块的实例和子模块的定义一一对应，消除一个模块的定义被多次引用的现象。因为大部分 layout 工具以一种flat 的方式完成设计的物理实现。对于多次引用同一定义的各个实例的物理模块的位置是不同的，而这些模块内部的 flop 都需要各自的时钟，从而时钟树上这些模块的clock_net名是不同的。如果Non_ uniquified 网表，来自 layout 工具的时钟树的信息和 DC 中原网表无法对应，因为被多次实例化模块在网表中只有一个定义，所以会出现两个不同的 net 和一个相同的 port 连结，这是不允许的。 Uniquify 的方法如下：

```
dc_shell>remove_attribute find(-hierarchy design,”*”)dont_touch
dc_shell>Uniquify
```

uniquify_naming_style 可以用来控制多次例化子模块每个拷贝的命名方式

32.简化网表

layout 工具在读取一些含有不常见的 net 名如：“*cell*”,”*-return”会存在困难。而且，大部分 layout 工具对 net,port 名称的最大字符数有严格的限制。为了使 DC 按照 layout 工具的要求写出网表，方法如下

可在“.synopsys_dc.setup”文件中进行设置,例如：

```
define_name_rules BORG  -allowed “A-Za-z0-9”  \
```

-first_restricted“_” -last_restricted”_” \

-max_length 30 \

-map {{“*cell*”,”mycell”},{“-*return”,”myreturn”}}

通过命令行，也可指示 DC 遵从以上的规则，例如：

change_names -hierarchy -rules BORGS

通过在 setup 文件中进行设置，DC 允许用户改变bus 的命名类型，例如：

bus_naming_style=%s[%d]

33.移去无连结的 ports

DC 对模块的无连结的 ports 会产生告警，在产生网表以前，移去无连结的 ports，避免 DC 因过多的无连结的 ports 告警而掩盖了真正的警告。方法如下：

remove_unconnected_ports find(-hierarchy cell,”*”)

check_design

34.检查 assign 和 tran 语句

一些布线工具很难读取包含tri wires ,tran 源语，assign 语句的网表。对于“inout”类型的 port,DC 产生 tri wire 语句和 tran 源语，我们可通过在 setup 文件中设置变量 verilogout_no_tri=true,使 DC 将 tri_state net 宣称为 wire。

当设计中出现同一模块 input port 和 output port 直接相连（feedthrough)，output port 连结到 ground 或被一常数（1‘b0,0’b0)驱动，DC 都会在网表中产生 assign 语句。我们可以设置如下变量,在 input port 和 output port 添加 buffer 来解决问题:

set_fix_mulitple_port_nets -feedthroughs

set_fix_multiple_port_nets -all -buffer_constants

如果，以上做法还不能解决问题，则设计中可能存在具有 dont_touch 属性的net，可用如下命令将 net 的 dont_touch 移去。

remove__attribute find(net,<net name>) dont_touch

35.确保所有 cell 的 pin 名都可见

DC 产生不会写出无连结的 ports 名,如：

DFF dff_reg (.D(data),.CLK(m_clock),.Q(data_out));

因 QN port 在设计中没有被使用，所以DC 没有写出 QN port,但在物理实

现上，这个 cell 含有 4 个 port,从而 layout 工具在读取网表时，会产生 cell 的 port 不匹配，可在 setup 文件中设置

变量 verilogout_show_unconnected_pins=true 解决这个问题。现在，一些 layout 工具没有这个限制。

36.检查无意义的 gating clock 或 reset

如果对 clock 或 reset 忘记设置 set_dont_touch_network 属性，则可能在 clock 或 reset 信号线上出现 buffer,可通过如下命令对 clock 或 reset 进行检查：

report_transitive_fanout -clock_tree

report_transitive_fanout -from reset

用 -clock_tree 选项必须提前对 clock 做出定义，-from 选项不需定义，-from 和 -clock_tree 选项不能同时应用。

37.对于未定义的引用，DC 会产生告警。

38.Layout

将优化过的网表转化为物理实现的过程。步骤如下：

Floorplanning;

clock tree insertion;

routing the database;

39.Floorplanning

floorplanning 就是将 cell 和 macros(RAMs,ROMs,sub-blocks)按排在合适的位置，从而减少net 的 RC 延时和布线电容，产生更快的设计。同时获得最小面积，降低布线阻塞。对于大型设计建议采用层次化的布局布线，首先使 sub-blocks 的布局布线符合时序和面积的要求，然后将 sub-blocks 作为一个 macros,和其他的 cell 及 macros 进行布局布线。下面从以下三个方面对 floorplanning 进行介绍：

时序驱动的布局（timing_driven layout(TDL)）;

反标 floorplan 信息;

相关建议;

40. 时序驱动的布局（timing_driven layout(TDL)）

layout 工具利用DC 产生的时序约束文件作为标注，根据设计时序约束信息安排 cell 和macros 的位置,该时序信息由 DC 用下列命令产生一个 SDF 格式的文件提供。

命令格式

write_constraints -format <sdf|sdf-v2.1> //版本 1.0 及 2.1

-cover_design//指示 DC 的时序约束应包含 the worst path 的每一个 drive-load pin 对

-from <from list>

-to <to list>

-through <through list>

-output <output file name>

41.Floorplan 的反标信息

反标信息使 DC 和后端工具集成，可更有效的执行，从而获得时序和面积的收敛。

Floorplan 可提供 PDEF（physical design exchange format)文件、set_load 格式的 net 负载电容文件，SDF 格式的 net 和 cell delay 文件。布局之前，网表根据各逻辑层次的wire_load 模型进行优化，布局之后，物理上设计的层次也许和逻辑上的层次不同，DC 必须根据物理层次划分作调整。

PDEF 文件：子模块的物理位置信息及物理分组和cell 的位置信息。

DC 用如下的命令读取 Layout 工具提供的 PDEF 文件：

read_cluster -design <design name><pdef filename>

优化完成之后， DC 用如下的命令将物理信息被回馈给Layout工具

write_cluster -design <design name><pdef filename>

SDF 文件：net 的 pin_to_pin 的 timing arcs 和 cell delay；

Set_load 文件：net 电容，DC 利用它做精确的 net 和 cell delay 计算及 DRC.格式如下：

```
set auto_link_disable true
 set_load 0.034 [get_nets N3243]
```

42.建议

TDL 适用于各种类型的设计，尤其是高速设计

对大型设计，为了节省时间，可只产生选择net 的时序约束。然而，如果 layout 工具可产生时序约束，则优先考虑 这种方法。

对大型设计采用层次化的布局布线，可获得更好的结果，便于控制。

在 DC 内，用 PDEF 文件包含的物理信息作 post-layout 优化 。

43.Clock tree insertion

控制 clock latency and skew.时钟树的层次和时钟树所用门的驱动强度有关，提供如下建议：

使用平衡结构且级别少的时钟树，级别越多，latency 越大；

在大时钟树，用高驱动强度的buffer，这也有助于减少时钟树的级别；

为了减少不同时钟域的 clock skew,需平衡两个时钟树的级数和 buffer 的驱动强度。

rise 和 fall 平衡的 buffer 不一定比普通的 buffer 快。

一些库提供的 buffer 对信号的 rise time 比 fall time 延迟低，对 positive flop，是很好的选择。

减少时钟 latency，可用大驱动的两极 inverters 器,因一个 buffer 由两个 inverters 组成，两极时钟树由 inverters 组成，可少用一个 buffer.

从 pad source 到一级 buffer 的 wire 需比较宽。

抽取 时钟，对包含 gate 的时钟。但这时，时钟树不能用 inverters;

44.Transfer of clock tree to DC

layout tool 综合了时钟树之后，修改了物理设计，而 DC 中的原网表却没有反映这个更改。随意必须将时钟树的信息传送给 DC.

layout tool 一般会以Verilog 或 EDIF 的格式生成平铺的设计网表，该网表没有层次，不易读，有效的验证方法就是在原来的层次网表和 layout tool 生成的网表之间进行形式（formal）验证。但形式验证对设计的大小和复杂度有限制，一般的形式验证工具善于做模块验证，但不善于对整个芯片进行验证。

以 SDF 文件格式传递从时钟源到 endpoint 的（flop 的 clock pin）点到点的时钟延时信息，DC 将这个 SDF 文件的信息反标到原来的设计上，决定时钟的延时和抖动，这种方法需注意 synopsys lib 和 layout tool lib 应相匹配，保证时序编排的一致性。因为 y 原始网表不包含 clock tree 的 cell 和 net, 根据 layout database 验证原始网表仍存在困难。

一些 layout tool 可创建包含“disconnect_net,create_cell”等命令的 DC scripts 文件，可以使 DC 将时钟树插入原设计数据库，不修原设计的层次，从而更利于形式验证。

将 layout tool 产生的设计改变报告用 script 语言如：Perl 或 Awk 修改，从而直接修改了verilog 网表，修改过的网表和原网表做形式验证。

45.布线

global布线：对设计做出布线计划，安排net 的路径，不进行实际的连接；

detailed 布线：确定实际连结路径的几何形状，不同的net 不能在同一层交叉

46.EXTRACTION 参数的抽取:

参数的抽取:

综合和优化是利用 wire-load 模型实现的，wire-load 模型在对最终的布线电容的统计的基础上建立的，所以 wire-load 模型和实际的线路延时值间有一定的差别，从而导致设计不是最优。 为了进一步优化，我们通过抽取 layout数据库产生用于进一步优化的延时值。抽取的内容：layout tool根据不同的算法抽取如下信息：

DSPF 或 SPEF 格式的详细参数：DSPF 包含布线后网表的分布式 RC 信息，最精确，但不可行，一般用于关键的net 或时钟树

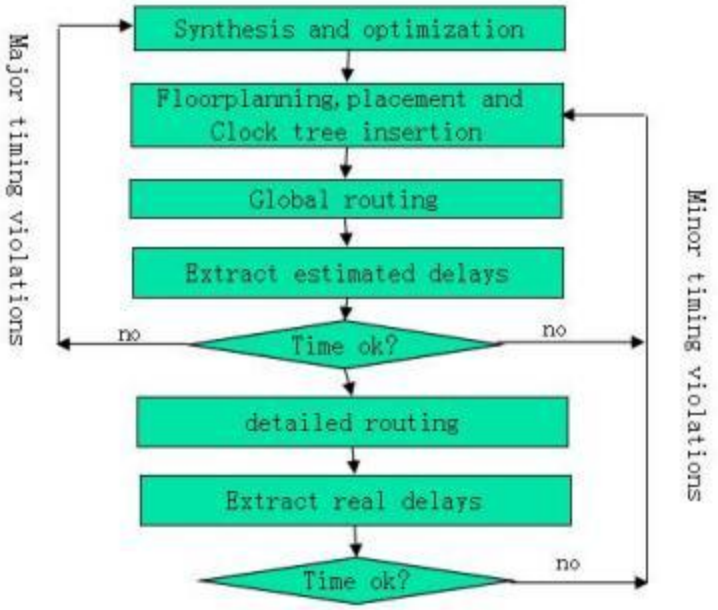
RSPF 或 SPEF 格式的简化参数：用 pi 型模型表示net 的 RC 信息，耗费时间一般用于关键的net或小的设计。

SDF 格式的 net 和 cell 延时：延时计算由 layout tool 执行，DC 使用他们进行优化受到限制，而且因为 cell 的延时固定，PT 的 case 分析失去了意义。

SDF 格式的 net RC 延时+lumped 电容：DC 可以利用 RC 延时和net 的电容负载进行自己的延时计算和post-layout优化。lumped 电容的文件以 synopsys 的 set_load 格式提供。lumped 电容包括 net 的 driving cell 的输出电容, net 的 driven cell 的输入电容，net 的 wire capacitance。

- 参数的抽取的过程

47.



48.Post-Layout Optimization

过程如下：

反标由 layout tool 工具产生的数据信息到 DC 中的设计 ；

根据时序违例的情况采用利用custom wire_load模型重新综合或利用in_place Optimization 技术，

需 layout tool 提供下列数据

SDF 格式的 Net RC 延时；

Set_load 格式的文件，包含容性的 net 负载；

PDEF 格式的物理位置信息；

49.Post-Layout Optimization 的步骤

反标

layout 数据必须反标给 DC,DC 完成 post-layout optimization, Layout tool 可能提供 两套数据文件分别对应 the worst case 和 the best case。则数据文件需和使用相应的 Synopsys lib 的设计对应。

命令，下列命令，反标 layout 产生的信息到 DC 中的设计。

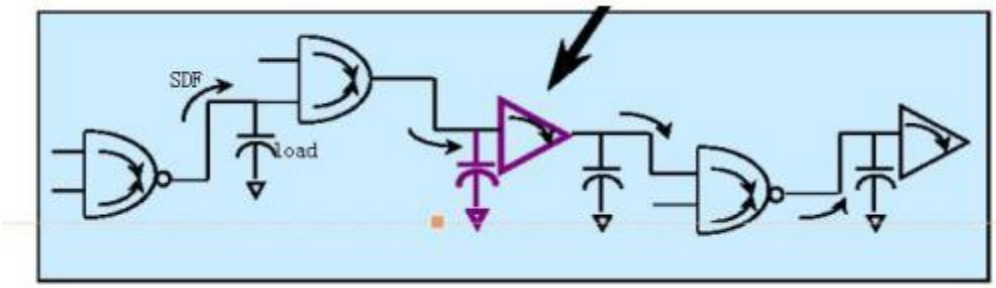
current_design <design name>

include -quiet <set_load file name>

read_timing -f sdf <RC file name in SDF format>

read_cluster <cluster file name in PDEF>

反标结果

50. 

51. custom wire-load 模型生成

反标的 Layout tool 数据对 custom wire-load 模型的影响

利用Net 电容和 SDF 文件生成的 custom wire-load 模型和设计的逻辑层次相对应；

PDEF 文件使 custom wire-load 模型根据物理设计的层次生成；

DC 命令

create_wire_load -design <design name>

-cluster <cluster name>//layout 使用的组名

-trim <trim value> //数据的下限值

-percentile <percentile value> //调整客户 wire-load 的乐观与悲观程度

-output <output file name>

用新的 CWLM(custom wire-load model)取代 layout 前的通用 wire-load model。命令如下：

update_lib <library name><CWLM file name>

该命令不会覆盖 DC 中原来的库，仅用CWLM 更新存储器中的 DC 库.

52. IPO 时维持设计的原结构，仅修改错误部分，对 layout 影响不大。IPO 的一般通过在某一确定部分添加/置换逻辑门达到修正 SETUP_TIME 和 HOLD_TIME 的目的。

IPO 和库相关，限于采用如下方法：

改变 cell 的大小；

插入或移去存在 cell（buffers)

synopsys 的技术库需作如下的设置， 以支持 IPO 操作。

in_place_swap_mode:match_footprint，允许有相同 cell_footprint 值的 cell 进行互换而不影响 layout。

在 setup文件或执行 IPO 之前设置下列变量达到对cell大小的改变

compile_ignore_footprint_during_inplace_opt =true|false;

compile_ok_to_buffer_during_inplace_opt =true|false;

compile_ignore_area_during_inplace_opt =true|false;

compile_disable_area_opt_during_inplace_opt =true|false;

通过如下命令执行 IPO

compile - in_place//不利用物理信息，使用 lib 的通用 wire_load

reoptimize _design - in_place//建议使用,利用物理信息，使用 custom wire_load

53.Location based Optimization

基于 cell 定位的算法

采用更精确的参数模型

将插入的 cell 放在更合适的位置，并通过 PDEF 文件通知 layout tool 例如：

Location based Optimization

LBO 是 IPO 的一部分，当对含反标有 PDEF 的设计利用“ reoptimize _design - in_place ”命令进行 IPO 时，LBO 自动被执行。

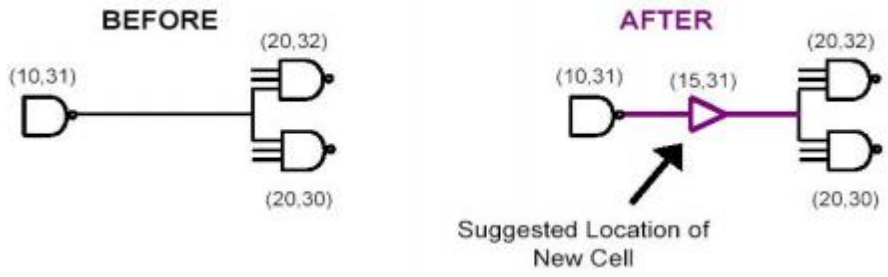
为了在“ reoptimize _design - in_place ”时执行 LBO,下列的变量需和 IPO 的相关变量一起设置

lbo_buffer_removal_enable =true

lbo_buffer_insertion_enable =true

对 IPO 或 LBO 做出的改变，可用下列命令写出：

reoptimize_design_change_list_file_name=<file name>



54.

55.Fix Hold-time Violation

在设计的初步布局布线之后修改hold-time violation，可利用更精确的延时信息。

方法如下：

Synopsys 方法；

人为插入 delays；

用 DC 命令自动插入延时；

56.Synopsys 方法

有两种方法：

two-pass 综合

Single pass 综合，通过 DC 命令

Set_fix_hold

指示 DC 在采用 CLK 做时钟的电路合适的位置插入 buffer 或置换数据路径的 cell 而修正 hold-time。在设计进行 layout 信息反标之后，用 reoptimize _design 命令修正 hold-time,在 layout 之前，采用“compile - incremental”命令修正 hold-time。

利用 reoptimize _design 命令

支持min_max 同时分析和优化；

反标 layout tool 提供的 min 和 max 延时；

用 set_fix_hold <clock name>修正 hold-time；

two-pass 综合

在排版前，用 worst-case lid 编译设计，获得最大 setup-time,在 layout 后用 best-case lib 对设计再映射，从而修正 hold-time，这个方法稳定性好。

57.Single pass 综合

```
set_min_library “<worst case library name>”
                -min_version “<best case library name>
set_operating_conditions  -min BEST  -max WORST
include net_delay.set_load
reading_timing interconnect.sdf
reading_cluster floorplan.pdef
set_input_delay  -max 20.0  -clock CLK {IN1 IN2}
set_input_delay  -min -1.0  -clock CLK {IN1 IN2}
set_output_delay  -max 10  -clock CLK {IN1 IN2}
set_fix_hold CLK
reoptimize_design  -in_place
```

58.人为插入 delay

对于少于 10-20 处的hold-time 违例，可人为插入 delay，一般都是通过加入一串连续的 buffer,但因为 buffer 间距离过近，总的延时由buffer 的 cell 延时决定，连接延时很小从而造成延时不足以修正hold-time。解决的办法是将多个高fanin 的门连结在一起，利用 fanin 电容比单输入的 buffer 大，增加延时

59.用 DC 命令自动插入延时

用于hold-time 违例较多，用 IPO 费很多时间，对时序报告使用的 script 语言（perl 或 Awk)做语法分析，获得时序分析失败路径 setup-time 的 slack 和 hold-time 的违例。根据这些结果，对失败路径，用户生成 disconnect_net,create_cell,connect 的 DC 命令，指示 DC 在合适的位置插入 buffer.

60.SDF 文件的生成

SDF 文件分为如下两种：

pre_layout

post_layout

post_layout 的 SDF 文件由DC 在设计回注了 RC 延时值和 lumped 电容后产生。PT 也 可产生 SDF 文 件。DC 命令如下:

```
write_timing  -format sdf-v2.1  -output<file name>
```

SDF 用于做 gate-level 动态时序仿真。

SDF 包含的时序信息组成如下：

IOPATH delay：cell delay,根据输出的 wire loading 和输入信号的 transition 计算

INTERCONNECT delay：是一条路的 driving cell 的 output pin 和 driven cell 的 input pin RC delay。

SETUP timing check：根据技术库的描述，确定时序 cell 的 setup-time

HOLD timing check：根据技术库的描述，确定时序 cell 的 hold-time

61. 生成 pre_layout 的 SDF 文件

pre_layout 的延时根据通用 wire-load 计算,因为没有时钟树的插入，所以首先需对时钟树延时进行估计，DC 命令如下：

```
create_clock  -period 30  -waveform {0 15} CLK
set_clock_skew  -delay 2.0 CLK
set_clock_transition 0.2 CLK
```

DC 利用以上值做静态时序分析，在生成 SDF 文件时，不会输出。为了使 DC 用设置的 clock delay 代替计算，DC 可对时钟的 input pad (CLKPAD)作如下设置：

```
set_annotated_delay 2.0  -cell \
                         -from CLKPAD/A  -to CLKPAD/Z
```

设置 clock_transition 不仅用于静态时序分析，还用于计算 driven cell 的 delay,将结果写入 SDF 文件中。

62. 生成 post_layout 的 SDF 文件

需生成之前，做如下设置,反标 layout 数据到设计：

```
read_timing  -format sdf<interconnect RC’s in SDF format>
include  -quiet <parasitic capacitances in set_load format>

create_clock  -period 30  -waveform {0 15} CLK
set_clock_skew  -propagated CLK
```

63.SDF 文件的时序检查

部分设计的 setup_time 和 hold_time 的违例常引起仿真出现 X’s，引起仿真失败。但有时，我们想忽略这部分设计的违例验证其它部分，这时我们可以人为的选择伪造 SDF 文件中 setup_time 和 hold_time 的值，使仿真成功。

然而最好使用下列 DC 命令向仿真器注明 setup_time 和hold_time 的值为zero，确保仿真成功。

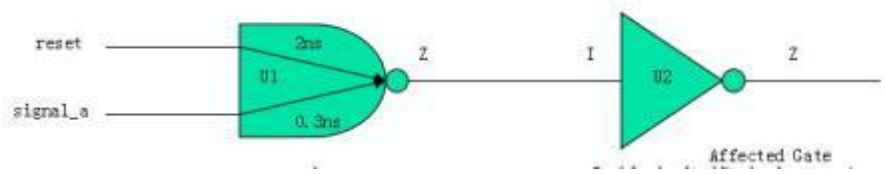
```
set_annotated_check 0  -setup  -hold \
                       -from REG1/CLK \
                       -to REG 1/D
```

---在仿真命令中加选项文件才是正确的做法,+notimingcheck。

64. 阻止错误的时序计算在 SDF 文件的输出

Cell delay 由 input transition 和输出负载决定，而 input transition 又由 driving cell 的 transition delay 决定，所以，若 driving cell 由多个 time arc，产生一个很大的 driven cell delay，为了避免这个问题，阻止阻止错误的时序计算在 SDF 文件的输出，在生成 SDF 文件之前，使用如下 DC 命令：

```
set_disable_timing U1  -from A  -to Z
```



65. 双沿问题

当使用到一个时钟的双沿,只要定义好这个时钟的周期,占空比,dc 约束会自动来约束上升沿->下降沿,下降沿->上升沿符合时序,及作时序检查

5. 命令介绍

读取 ddc文件---此文件为 dc 编译完工程的文件,当产生一些额外报告时,无须重新编译工程,直接通过read_ddc xxxx.ddc 命令后在使用报告命令产生对应的报告
前提是需要先有输出 ddc 文件,不会默认有此文件输出

```
write -f ddc -hier -output $ddc_path/$MNAME.read_rtl.ddc
```

时钟分析设置

set_multicycle_path -setup xx -from xxx/cp through xxx/out -to xxx/D 多周期路径 setup 约束

set_multicycle_path -hold xx -from xxx/cp through xxx/out -to xxx/D 多周期路径 holdup 约束

set_clock_gating_check -setup xx -hold xxx [current_design] 门控时钟 latch 约束

set_disable_timing CLOCK_GEN/U1 -from a -to y 去掉从引脚 a 到引脚 y 的时间弧，即不分析这条路径

set_case_analysis 0 [get_pin xx] 设置某个 PIN 为 0 的情况下分析时序 如可以设置 scan_mode 为 0 来做 function 约束

set_propagated_clock：set_propagated_clock 用来在版图后根据时钟树的实际寄生参数自动计算所有时钟引脚延迟，而无需 set_clock_latency 等命令为时钟建模 set_propagated_clock [list[all_clocks] [get_generated_clock *]]]

set_clock_latency：为时钟建模，指定时钟与源时钟的延时，

eg:set_clock_latency 2 MCLK，设定时钟与主时钟 MCLK 的延时为 2ns

set_clock_uncertainty：为时钟建模，指定各时钟之间的相对延时（uncertainty 指 jitter + skew）,模拟了时钟的抖动和时钟偏差 set_clock_uncertainty 0.5 MCLK

set_clock_transition：为时钟建模，指定各时钟上升和下降时间（电压从 20%~80% 或 80%~20%），如果没加-setup ,-hold,则上升下降时间相同

set_clock_transition 0.25 MCLK

环境属性设置：

set_driving_cell：说明输入端口有一个真实的外部单元驱动

set_driving_cell -lib_cell FD1 -pin Q [get_ports IN1]

set_load：说明端口（输入或输出）的外部电容负载 set_load 5 [get_ports OUT1]

set_operating_conditions：设置工作条件 set_operating_conditions -max "slow_125_1.62"

set_wire_load_model：设置连线负载模型，默认是选择 WLM，相当于

set_wire_load_model WLM

set_wire_load_mode：设置当连线穿越 2 个模块时的连线规则 set_wire_load_mode enclodes/top enclodes 模式比较不悲观 top 模式比较悲观

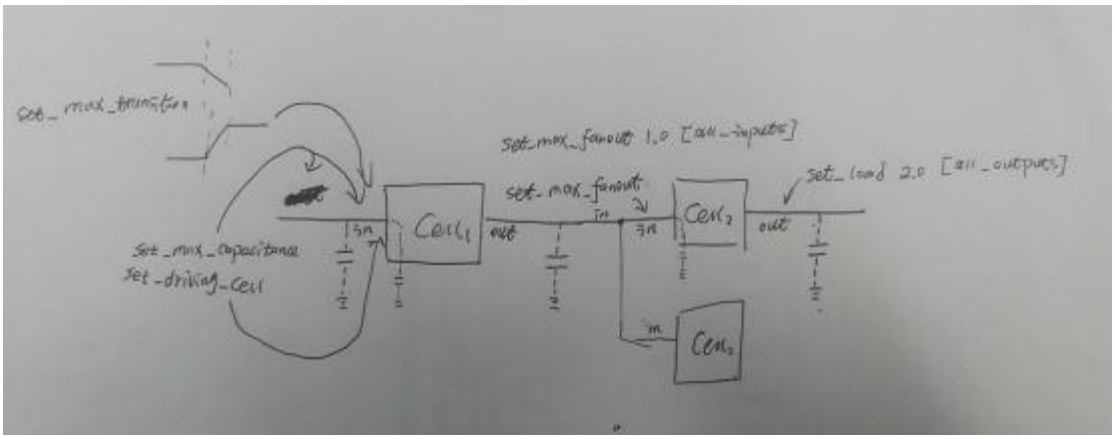
设计规则约束

set_max_capacitance：为设计加入最大电容设计规则，单位为 pF；通过 set MAX_CAP [get_attribute $DRIVE_PIN max_capacitance] (set DRIVE_PIN TECH_LIB/invla27/Y) 可以找到 invla27 最大允许电容负载

set_max_transition：为设计设置最大转换时间;通过 set MAX_TRANS [get_attribute $DRIVE_PIN max_transition] 可以找到 invla27 最大允许转换时间

set_max_fanout：为设计设置最大扇出负载； 通过 set LOAD [get_attribute $DRIVE_PIN fanout_load] 可以找到 invla27 扇出负载

环境变量设置点:



DFT 相关：

在综合时,需要设置做 DFT 的相关管脚,测试时钟 TestClock,测试复位 RST_N,测试使能 Scan_Enable

命令格式为 set_dft_signal -view exist/spec -type TestClock/RST_N/Scan_Enable/TestMode -active_state 0/1 -port port_name -hookup_pin pin_name

exist 表示用本设计的端口用做某一用途,如用作时钟/复位等,spec 表示用设计内部信号用作某一用途,如果作为scan 模式信号, active_state 表示该型号是高或低电平有效,port 表示设计中的端口号,hookup_pin 指定本设计中某个 cell 的 pin 上,

当 TestMode 是用端口控制时,可以使用 set_dft_signal -view exist -type Constatnt -active_state 1 -port port_name 来定义,如果不是,而是设计内部逻

辑控制,则使用如下方法 set_dft_signal -view spec -type TestMode -active_state 1 -hookup_pin I_prjxxxxxxx_core/I_test_ctrl/atpg_test_en_reg/Q

create_test_protocol：

NAME

create_test_protocol

Creates a test protocol based on user specifications.

SYNTAX

status create_test_protocol

[-infer_asynch]

[-infer_clock]

[-capture_procedure single_clock | multi_clock]

ARGUMENTS

-infer_asynch

Infers asynchronous set and reset signals in the design.

-infer_clock

Infers test clocks in the design.

-capture_procedure single_clock | multi_clock

Specifies the capture procedure type. The multi_clock type creates a protocol file that uses generic capture procedures for all capture clocks. The single_clock type creates a protocol file that uses the legacy 3-vector capture procedures for all capture clocks.

The default value is multi_clock.

The create_test_protocol command should be executed before running the dft_drc command because design rule checking requires a test protocol

dc -scan 只是把普通寄存器替换成带有 scan 功能的寄存器

dc -insert_dft 该命令把扫描链链接起来

check_timing 在电路中寻找未加约束的路径：check_timing

report_clock 检查已经定义的时钟：report_clock -attr -skew

report_port　检查所有端口上所加的约束

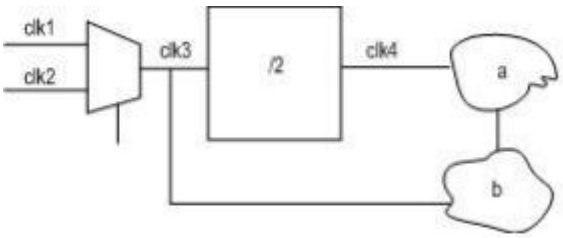
report_lib　把厂商所提供的所有工作条件列出来

report_timing_requirements 报告设计中所有的例外

report_constraint

查看电路是否违反设计规则要求 report_constraint -all violation

report_timing　报告设计的时序是否满足目标



对于上图:包括有时钟选择和时钟分频,以及跨时钟域的问题.

1、 clk3 是由 clk1 和 clk2 产生的

2、 clk4 通过 clk3 的 2 分频产生

3、 clk4 控制的逻辑 a 与 clk3 控制的逻辑 b 之间存在通信，并且没有进行同步处理

对于这种情况首先我们知道，clk1 与 clk2 物理上是异步时钟，所以可以直接用 create_clock 命令创建时钟。

Create_clock -name clk1 -period 10 -waveform {0 5} clk1

Create_clock -name clk2 -period 20 -waveform {0 10} clk2

而 clk3 是经过选择器之后的时钟，我们也希望能够创建这个时钟，并且它来自于 clk1 和 clk2 所以我们使用 create_generated_clock 创建 clk3

Created_generated_clock -name clk3_1 -source [get_ports clk1] clk3 -combinational -divide_by 1

Created_generated_clock -name clk3_2 -source [get_ports clk2] clk3 -combinational -divide_by 1 -add

Clk4 是 clk3经过 2 分频获得的时钟，我们也需要创建

Created_generated_clock -name clk4_1 -source [get_nets clk3] clk4 -divide_by 2 -master_clock [get_clocks clk1]

Created_generated_clock -name clk4_2 -source [get_nets clk3] clk4 -divide_by 2 -master_clock [get_clocks clk2] -add

只有这样做才能够保证 dc 工具能够正确地处理 a,b 两块逻辑之间的数据交互。

在此对 created_generated_clock 进行讲解：

create_generated_clock

[-name clock_name]

[-add]

source_objects

-source master_pin

[-master_clock clock]

[-divide_by divide_factor

| -multiply_by multiply_factor]

[-duty_cycle percent]

[-invert]

[-preinvert]

[-edges edge_list]

[-edge_shift edge_shift_list]

[-combinational]

[-comment comment_string]

1、 默认这个命令产生时钟是和 source 时钟是相关

-name 用来定义时钟名

-add 如果一个时钟被定义了两次，例如上面的 clk3 由于来自两个时钟，所以在这里我们定义了两次，一次是告诉工具 clk3 来自 clk1，所以我们定义的名字是 clk3_1。但是第一次定义的 clk3 不需要添加-add 选项，因为在之

前并没有对 clk3 进行过其他定义。第二次我们定义 clk3 的时钟来自于 clk2，所以名字上我们叫 clk3_2，但是 clk3_1 和 clk3_2 都是 clk3，所以我们通过-add 命令来告诉工具这两个时钟其实是一个时钟线。

source_objects 其实就是你想要定义的时钟所在的点，例如：clk

-source 这个选项是告诉 dc 工具，定义的这个时钟是由哪个源产生的，这里的源时钟产生这个时钟的时钟点。

-master_clock 这个选项是告诉工具，当前的这个时钟的源是由多个时钟定义的，但是我们想要分析这个源的哪一个时钟。

-divide_by 分频参数。

-multiply_by 倍频参数。

-invert 创建时钟和源时钟是反向关系。

-preinvert 创建时钟和源时钟是并非是反向关系，但是想要按照反向的关系进行分析。

-combinational 产生时钟和源时钟之间没有经过寄存器而是经过组合逻辑产生的。

6. 注意点

Dc 读取 db 库除了把逻辑 map 成物理 cell外,还一个目的就是读取时序做约束.

1. 但一个系统中使用了双沿怎么处理？

按正常约束就可以，但要定义好时钟的上下沿的关系，因为DC 工具会自动识别，会把上下沿作为同步时钟处理。或者说当作 2 个时钟处理，但按同步时钟处理

2. 当在做 scan 时,模式寄存器不能插入扫描链中,否则模式寄存器会变化

a.该 scan 模式寄存器的复位端设计成于 scan 的 reset 无关,则综合时不会插入扫描链中

b,综合时把相关寄存器配置为 set_dont_touch_network [get_pins -h "*atpg_test_en_reg/Q"] -no_propagate

3. clock gating check 是以 clock 为参考来约束 enable 信号，所以 clock 跟 enable 信号是工具推断 clock gating check 的基本对象。通常，一个 cell 如果满足如下条件，工具便会对该 cell 做 clock gating check.

Clock pin:

a. 有 clock 传播到该 pin 上；

b. 这个 clock 必须是有效 clock，所谓有效 clock 指 clock 穿过这个 gating cell 后会传到寄存器的 CK pin 或会驱动 ouput/inout port 或做为某个 generated clock 的 master clock. 除此之外都不算有效 clock, 如某个 clock 传过 gating cell 后传播到寄存器的数据端，则该 clock 不算有效 clock.

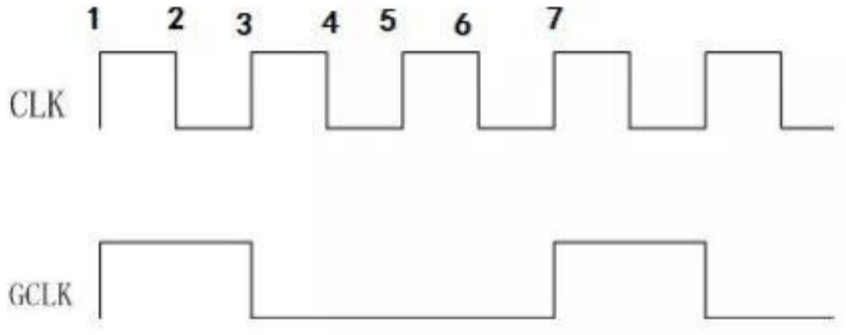
c. 跟 gating pin 都会达到该 cell 的同一个 output, 即 clock pin 跟 gating pin 都有到该 cell 同一个 output pin 的 timing arc.

Gating pin:

a. 跟 clock pin 都会达到该 cell 的同一个 output, 即 gating pin 跟 clock pin 都有到该 cell 同一个 output pin 的 timing arc.

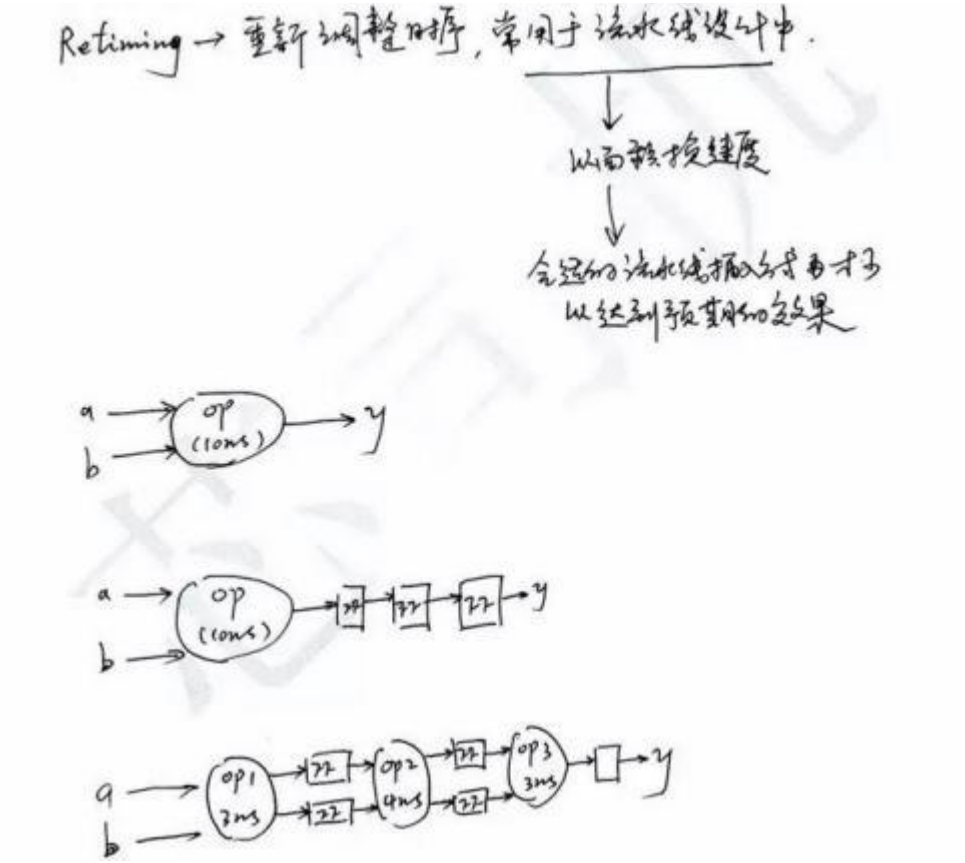
b. 必须没有有效 clock 传播到该 pin 上。

4. create_generate_clcok -edge 的意思是源时钟的第几个沿 。如-edge 1 3 7 的波形如下。另外对 GCLK 的周期进行的修改，是 30ns，并非 80ns。



5. Retiming，顾名思义，就是重新调整时序，作为逻辑综合时序优化的一种典型技术，常用于流水线设计中。如果设计中遇到一个复杂的组合逻辑，可以通过流水线技术，以面积换速度的思路来解决。但流水线的插入要合适，才能达到预期的频率要求。

比如：y=OP(a, b)是一个复杂的逻辑操作，估计延时为 10ns，现采用三级流水线设计来提高频率到 250MHz，如果在 RTL 中插入流水线，可能无法在最大程度上保证均匀的插入流水线。Design Compiler 提供了 Register Retiming 技术，RTL 中只需要在输出 Y 连续打三拍，综合时 compile_ultra-retiming，工具会自动拆分复杂的组合逻辑 OP 为 op1，op2，op3,并插入寄存器，最终达到设计要求的时钟频率（图下图）



6. DC 命令：set_ideal_network、set_dont_touch_network、set_dont_touch 应用

set_ideal_network，set_dont_touch_network 和 set_dont_touch 这三个命令的共同点是，都可以对某些对象(objects)设置 dont_touch 属性(attribute)，从而在使得综合工具在优化阶段不去修改或移除该 object，比如对时钟或复位线设置了 dont_touch 属性，那么就可以避免在综合优化时对时钟或复位线上插入buffer

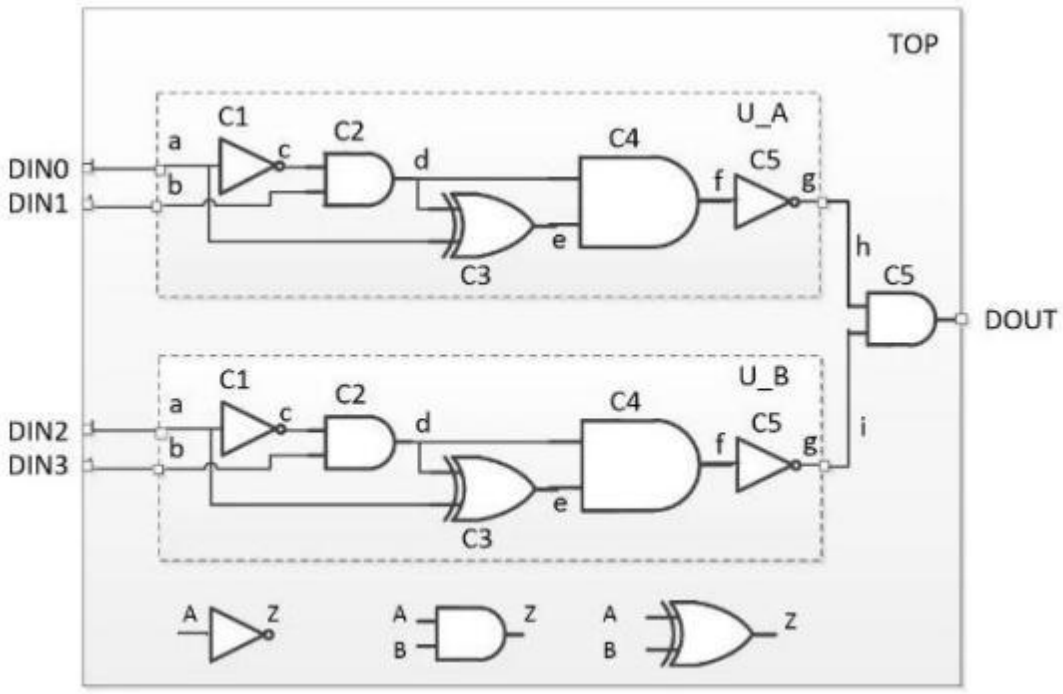


图 1

从设置对象（ object)方面来说。

a） set_ ideal_network 的设置对象是当前设计的 port 或者子设计内部 p in，不能是多级别设计的边界 p in 或者 port。也就是说，只能对顶层设计 port 或者 leaf cell 的 p in 使用 set_ ideal_network 命令来标记其为 ideal source，然后通过连接关系将 ideal_network 属性传递，其中 leaf cell 的含义是类似与非门、或门这样的工艺库中的基本单元。说到这里，很多人会迷惑，不是说设置的是 dont_touch 属性么，怎么又说 ideal_network 属性了呢？其实，如果使用 list_dont_touch_types 或者 report_dont_touch 命令，会发现 dont_touch 是有不同 type 的，set_ ideal_network 命令设置的 dont_touch 属性是 idn 类型的，而 set_dont_touch_network 命令设置的 type 是 dtn 。为了区分，常将这两个命令设置的属性称为 ideal_network 和 dont_touch_network。以图 1 举例说明，在图 1 中，所有非门的输入 p in 脚为 A，输出 p in 脚为 Z，所有与门和异或门的两个输入 p in 脚分别为 A、B，输出 p in 脚为 Z，在顶层设计 TOP 下有两个电路结构完全相同的子设计 U_A 和 U_B，如果对 U_A/a 这一 port 使用 set_ ideal_network 命令，DC 认为该操作是无效的，因为该端口在子设计的边界上；但如果对 C1 这个非门的输入 p in 脚 A 使用 set_ ideal_network 命令，那么该命令成功将输入 p in 脚 U_A/C1/A 标记为 ideal_network source。那 set_ ideal_network 是否可以对 net 设置 ideal_network 属性呢?是可以的，比如 set_ ideal_network

[get_nets U_A/a] -no_propagate，注意使用 set_ ideal_network 命令对 net 设置理想属性时，-no_propagate 选项必须添加上，其作用是理想属性碰到组合逻辑门便不再传递，在执行这条命令的时候，DC 会给出警告，告知综合人员驱动这条 net 的源头 p in 被标记为 ideal source。

b） set_dont_touch_network 命令可设置对象（object）有 p in、port 和 clock，是不能对一条 net 使用该命令的。不过 set_ ideal_network 命令中要求设置对象 p in 和 port 不能在 hierarchical boundary 上，而 set_dont_touch_network 无这一限制。仍以图 1 为例，使用命令 set_dont_touch_network[get_nets U_A/d]是非法的，但是如果对线 d 的驱动 p in 使用命令 set_dont_touch_network [get_nets U_A/C2/Z] 则是可行的。需要说明的是，如果使用 set_dont_touch_network [get_nets U_A/a]，DC 并不报错，因为 U_A 中有同名 p in 脚 U_A/a，但不建议这样使用。

c）set_dont_touch 命令的设置对象(object)最多，有 net、cell、 design、 reference 和 library cell。如果想对线 c 使用 set_dont_touch [get_p ins U_A/C1]命令来设置 dont touch 属性，DC 会报错，因为 set_dont_touch 命令是不能对 p in、port 使用的。这里需要说明的是，如果使用 set_dont_touch[get_p ins U_A/a]，DC 并不报错，因为有和 p in 脚同名的线 U_A/a 存在，不过也不建议这样使用。

从 attribute 的传递规则方面来说。

a） 使用命令 set_ ideal_network 来设置 ideal_network 属性时，该属性传递规则为：

如果驱动一条 net 的所有 p in 都是理想的，那么这条 net 也具有该属性；

如果一条 net 具备 ideal_network 属性，那么所有它的驱动 p in 都被视为理想的；如果一个组合逻辑门的所有输入 p in 都是理想的，那么这个组合逻辑门单元被视为理想的。仍以图 1 举例来说明该属性的传递效果。

set_ ideal_network [get_p ins U_A/C2/A]

set_ ideal_network [get_p ins U_A/C2/B]

那么，该理想属性可以传输到线 d，此时线 e 不是理想的，但再加上 set_ ideal_network [get_p ins u first a/C3/B]命令，结果线 e 也是理想的了。

b） 使用 set_dont_touch_network 命令设置 dont_touch_network 属性，其传递能力最强，传递效果最不可控，它可穿过连接的任何组合逻辑终止于时序逻辑，且具有单向传输的特点。

如果对一个单元的输入 pin 使用 set_dont_touch_network 命令，那么该 cell 被标记上 dont_touch，而单元外连接这个输入 pin 的 net 没有被标记；

如果对一个 cell 的输出 pin 使用 set_dont_touch_network 标记了 dont_touch，那么 cell 外连接该输出 pin 的 net 被标记为 dont_touch，而这个 cell 没有被标记。仍以图 1 举例来说明 dont_touch_network 属性的传递规 。使用命令 set_dont_touch [get_pins U_A/C4/A]，此时，不用对 pin 脚 U_A/C4/B 使用该命令，就已经将 dont_touch_network 属性传递到了线 f、线 g、线 h 和线 DOUT。

c） 使用 set_dont_touch 对 objects 设置 dont_touch 属性，该属性的传递规则为：如果一个多级别单元被设置了该属性了，那么该单元内的所有子单元都具备该属性；

对库中的单元施加了该属性，那么所有该库单元的例化也具备该属性；

如果对一条 net 置该属性，那么该条线连接的所有同级别的 mapped 单元也具备该属性；

对 reference 施加该属性，那么该 reference 中所有 cell 也具备该属性；

对一个设计施加该属性，意味着例化该设计时，该设计内部所有 cell 也都具有 dont_touch 属性。

需要注意的是：

a） 如果一条 net 上具有 unmapped 单元，那么该属性会被忽略，该 net 仍会被优化，但是如果在综合前将变量 enable_keep_signal_dt_net 设置为真，那么该线就会被保留，不过这对 QoR 有影响。

b） set_dont_touch_network 命令可以覆盖命令 set_dont_touch object false 的作用效果；

c） dont_touch 属性会对扫描插入有影响。

d） 如果对 unmapped 单元使用该命令，会造成 compile_ultra 运行失败 。仍以图 1 举例来说明 dont_touch 属性的传递规则 set_dont_touch [get_cells U_A]，综合优化的结果是 U_A 保持原结构且不映射到单元库，而与 U_A 完全相同的 U_B 则优化为一个与非门。

对门控单元的影响来说 。用 set_ ideal_network 或 set_dont_touch_network 命令并不影响时钟门控的修改替换，但对门控单元或者包含门控单元的父级结构使用 set_dont_touch 命令，那么会影响该门控单元的优化和修改。

说完了这个三个命令的相同点和不同点，总结一下这几个命令的使用场景：

a）对复位信号网络设置 dont_touch 属性时，最好使用 set_ ideal_network 命令，因为 set_dont_touch_network 不能 disable 对 DRC 的检查。

b）如果设计中例化了 library cell，也就是说想指定使用某个 library cell 时，对该 library_cell 使用 set_dont_touch 命令。

c）对 CTS 前的时钟网络设置 dont_touch 属性时，最好使用 set_ ideal_network 命令，因为 set_dont_touch_network 不能 disable 对 DRC 的检查；不过使用 set_ ideal_network 命令时，如果遇到将时钟用作数据的情况，那么这时候也影响了该数据线的 DRC 规则修正和延时计算，所以对 CTS 前的时钟网最好使用

set_auto_ disable_drc_nets -on_clock_networktrue 命令，使用该命令既能对时钟网络 disable DRC 检查又不影响时钟用作数据时的延时计算和 DRC 检查，关键使用该命令最简单、不易出错。

d）对 CTS 后的时钟网络，是需要计算时钟线上的延时的，这时应使用 set_dont_touch_network 命令，这样既不影响时钟线上延时计算，也达到了优化时对时钟树的保护效果。

7, 当项目中用到了第三方的网表（加密或没加密），则把它当做一个 cell 看待，综合时，在.setup 文件中加入对应的.db 文件就可以了，一个硬核

如果对方只是提供网表，并没有生成 db 文件，那么就可以把那个模块设置为 don’t touch； set_dont_touch [get_designs -hierarchical "module_name"];这种情况下对方不能加密

8, 在综合时都是使用 worst 的 db 文件，因为 dc 后可以分析 setup 是否满足，如果综合的 setup 不满足，pr 肯定就无法满足

9，在数字给模拟的控制信号，尽量使用寄存器输出，一是为了避免毛刺，二是如果模拟控制信号之间有时序要求比较好约束；工具也比较容易实现.

可以通过 set_max_delay 命令来约束.如下

1.The following example shows how to optimize the design so that any delay path to a port named Y is less than 10 units.

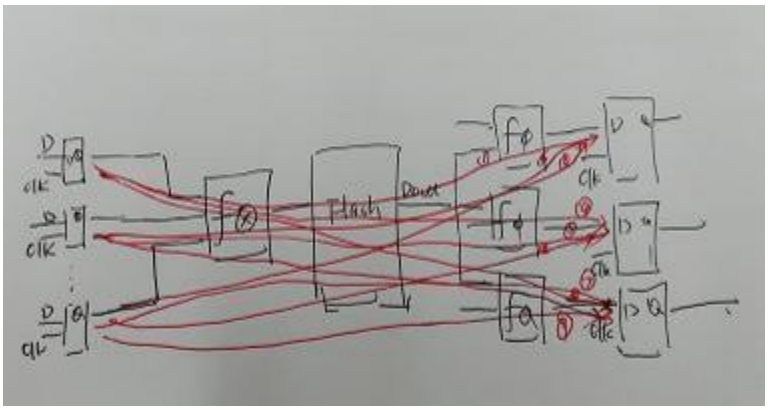
```
prompt> set_max_delay 10.0 -to {Y}
```

2.This example shows how to set a requirement that all paths leading to ports named busA[*] have a delay of less than 5.0 and are included in the path group named busA.

```
prompt> set_max_delay 5.0 -to "busA[*]" -group_path "busA"
```

3.This example shows how to set a maximum delay requirement of 3.0 for all paths that first pass through either pin u1/Z or u2/Z then pass through pin u5/Z or u6/Z.

prompt> set_max_delay 3.0 -through {u1/Z u2/Z} -through {u5/Z u6/Z}
set_max_delay 58 -through xxx/FPFLS032K19DA_i/DOUT 此设置为从 flash dout 输出/通过 到任何一个寄存器的 d 端(d 端前为组合逻辑)或 port 最大延时不大于 58ns,这是当 flash 为无时钟的异步 flash 需要做的约束,



如上,9 条路径的延时都小于 58ns
如果上述 flash 是带有时钟的同步 flash(读),那么其 flash lib/db 库必定会有像寄存器一样的时钟检查 (sethold time check),那么就只需要按照同步时钟/时序逻辑一样约束时钟就可以了.
具体用法在 dc_shell 下使用 man set_max_delay 命令查看.

7. Dc 中对时钟复位的约束

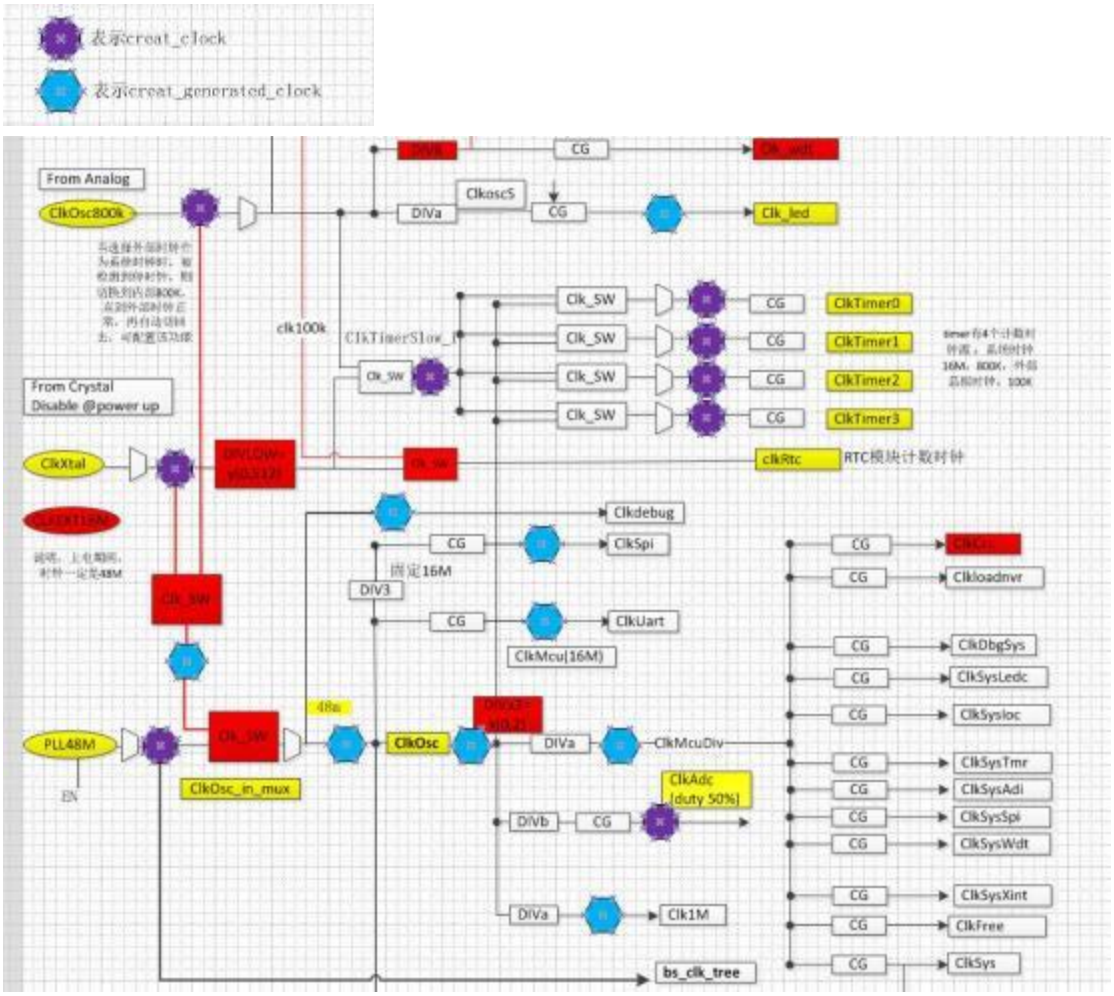
在 CTS 前把复位信号设置为理想信号, 这样 DC 综合时中间不会插 BUF
时钟根据 create_clock, create_generated_clock 生成理想时钟

```
create_clock -name clk_test -period $test_clock_period [get_port P02_PAD]
create_generated_clock -name clk_ram [get_pin xxx/uMux/Z] -source [get_pin
xxx/uClkMcuDiv/uTlat/Z] -master_clock clk_mcu_div -add -divide_by 1
set_clock_uncertainty $clock_skew [all_clocks] #设置时钟模型—特性
```

在 cts 后会根据实际生成的时钟树得到时钟特性

```
if {$con_type == "pre_layout" } {
set_ideal_network -no_propagate [get_nets "xxx/PorAtpg"]
set_ideal_network -no_propagate [get_nets "xxx/ResetnPor_osc1"]
……
set_ideal_network -no_propagate [get_nets "xxx/ResetnMcu"]
set_ideal_network -no_propagate [get_pins "xxx/uAnd_ScanEn/uAnd2/Z"]
}
else ##CTS 后的时钟为实际时钟
{
set_propagated_clock [all_clocks]
set_clock_gating_check -setup 0.5 -hold 0.5 [all_clocks]
}
```

8. 时钟树分析



上图中只有三个真正的源时钟,osc800k, pll48m,外部晶体 xtal 必须使用 create_clock 来定义;

而上面的 clkTimer0, clkTimer1, clkTimer2, clkTimer3,由于是一个专用时钟,不会与其它时钟域交互,有几个交互信号也做了同步处理,所以这里使用 create_clock 来配置,这样比较简单;

ClkTimerSlow 使用 create_clock 是因为它的负载寄存器只有在 clk_sw 中,而这个模块里面的信号都有做同步处理,所以直接使用 create_clock 更方便

ClkAdc 的负载寄存器在同一个时钟域主要用于模拟模块,很少与其它模块交互,即使有交互也做了同步处理.

9. 范例分析

注意在运行目录下需要有一个 .synopsys_dc.setup 文件，它里面配置了综合库，在 dc 安装目录下也有一个，当运行目录下有时，则会覆盖安装目录文件，配置的库使用最差的库综合---用于前期的 setup检查，如下内容
为.synopsys_dc.setup 内容

```
//*************
set DW_PATH  /ic05/eda/synopsys/syn_M-2016.12-SP4/libraries/syn

set search_path [list . $DW_PATH \
/xxxx/lib/flash/HHG_F011Q7P6_JPFLS016K29DHV00K/synopsys_model \
/xxxx/lib/memory/RASP1536X9 \
/xxx/lib/memory/RASP256X9 \
/xxx/lib/io \
/xxxxx/hhgrace/Verisilicon/HG11LP/FEView_STDIO/STD_6T/Synopsys \
]#设置路径

set synthetic_library [list dw_foundation.sldb standard.sldb
dft_lbist.sldb]
##设置综合库
set link_library [concat * \
hg11lp_6T_ss_1p35v_125c.db \
JPFLS016K29DH_max.db \
RASP256X9_ss_1p35v_125c.db \
RASP1536X9_ss_1p35v_125c.db \
SP5VP15_PG_V1p0_max.db \
$synthetic_library]
##设置链接库
set target_library [list hg11lp_6T_ss_1p35v_125c.db]
##设置目标库

set verilogout_higher_designs_first true
set verilogout_no_negative_index true
set verilogout_no_tri true
set verilogout_single_bit false
set verilogout_show_unconnected_pins true
set bus_naming_style %s_%d
//*********
```

下面内容为综合脚本 xxx.tcl 运行命令为 dc_shell -f xxx.tcl

```
 set script_path {./scr}

 set report_path {./results/report}
```

```
set net_path {./results/netlist}
set ddc_path {./results/ddc}
set rtl_list_path {../filelist/asic}

define_design_lib work -path ./work
set hdlin_enable_rtldrc_info true        //使能 drc 检查

set MNAME prjxxxx
set_svf results/report/${MNAME}_syn.svf        //.svf 文件包含了优化信息,formality 用
source -echo  $rtl_list_path/prjxxxx_syn_rtl_read.tcl  //把 rtl 读入

current_design $MNAME    //指定当前设计
link              /链接
uniquify          //统一化,就是会把例化的模块一一对应,

set_dont_use [get_lib_cells "gsmc13_lp_ss_1p35v_125c/NAND6* \
                             gsmc13_lp_ss_1p35v_125c/NOR6* \
                             gsmc13_lp_ss_1p35v_125c/OAI33* \
                             gsmc13_lp_ss_1p35v_125c/PULL* \
                             gsmc13_lp_ss_1p35v_125c/RSLAT* \
                             gsmc13_lp_ss_1p35v_125c/TIE* \
                             gsmc13_lp_ss_1p35v_125c/BUFT* \
                             gsmc13_lp_ss_1p35v_125c/DEL*"]
//指定不用库的某些 cell,可能是因为面积太大了
set_dont_touch [find cell -h "*spare_cell* *_PAD* *AVD* *AVS* *VDD* *VSS*  "]
set_dont_touch [find cell -h "*uAnd*" ]
```

```
set_dont_touch [find cell -h "*uMux*" ]
set_dont_touch [find cell -h "*uAtpgMux*" ]
set_dont_touch [find cell -h "I_prjxxxx_ipio"]
//制定上面器件不动(优化),因为设计时已经确定好了
set_dont_touch_network [get_pins -h "*uAnd_ScanEn/Y*"]
-no_propagate
set_dont_touch_network [get_pins -h "*atpg_test_en_reg/Q"]
-no_propagate
//指定器件及相关的网络不能优化
set auto_wire_load_selection false    //不自动选择网线负载
########################################
####select the mode and type here!!!!
########################################
set     con_type    pre_layout
source -echo $script_path/prjxxxx_src.sdc    //读入时序约束文件
set_max_area 0
set test_mode_port_naming_style scan_mode%s
//设置 scan 中 test_mode 命名格式,在本设计中 test_mode 叫 scan_mode

set_clock_gating_style  \
//设置门控时钟类型,
                    -sequential_cell  latch \
//使用 latch 类型的门控时钟
                     -positive_edge_logic integrated \
 //时钟正沿有效,即会在负沿 gating
                    -minimum_bitwidth 3 \
//只有超过 3 个寄存器的时钟才插入门控时钟
                    -max_fanout  10 \
//最大扇出 10 个门
```

-control_point before \

//控制逻辑在前部,指的是插入测试 mux 在哪个地方插入

-control_signal test_mode

//控制信号为 test_mode,本设计中定义了 test_mode 叫 scan_mode

check_design > ./$report_path/$MNAME.chk //检查设计

compile_ultra -gate_clock -no_autoungroup -scan /编译设计,插入扫描逻辑,注意此处只是把带有scan功能的寄存器代替了普通的寄存器,但并没有链接起来,只有执行了下面的insert_dft才会把链串起来

change_names -rules verilog -hierarchy

//制定命名规则,尤其是总线,网表的命令与rtl命令不一致

write -f ddc -hier -output $ddc_path/$MNAME.read_rtl.ddc

//输出 ddc 文件,此文件包含了所有信息

write_file -f verilog -hierarchy -output $net_path/$MNAME.v

输出综合后的网表,verilog 格式

set_svf -off

check_timing > 1.timing //检查在电路中寻找未加约束的路径：即检查约束是否正确

check_design > 1.chd

report_timing -path full_clock -input -cap -nets -transition -max_path 100 -delay max -nosplit > ./$report_path/$MNAME.time.max.report

report_timing -path full_clock -input -cap -nets -transition -max_path 100 -delay min -nosplit > ./$report_path/$MNAME.time.min.report

//按要求输出时序报告,

report_constraint -all_violators > ./$report_path/$MNAME.vio

//查看电路是否违反设计规则要求,包含时序及其它规则

#quit

```
#########################################################################
#  Compile
#########################################################################
```

#read_ddc ./results/ddc/prjxxxx.read_rtl.ddc//如果是单独做下面的动作,只需要读入上面写出的 ddc 文件,由于这里是一次性运行,所以不用读入

```
set test_default_delay 0
set test_default_bidir_delay 0
set test_default_strobe 40
set test_default_period 100
```

//上面是设置 scan/dft 的参数

```
set_dft_drc_configuration -internal_pins enable  //设置 dft 模式是内部信号,即不是由 外部输入信号控制表示进入 scan_mode,而是内部某个信号表示 scan_mode

set_dft_signal -view existing_dft -type ScanClock  -port [find port P02_PAD] -hookup_pin I_prjxxxx_ipio/uP0_2/C -timing [list 45 55]
```

//设置 dft 时钟从哪输入及时钟特点 45 表示上升沿时刻,55 表示下降沿时刻

```
set_dft_signal -view existing_dft -type Reset    -active_state 0 -port [find port P05_PAD] -hookup_pin I_prjxxxx_ipio/uP0_5/C
```

//设置 dft 复位信号从哪来,因为是端口,已经存在用 existing_dft

```
set_dft_signal -view spec -type TestMode -active_state 1 -hookup_pin I_prjxxxx_core/I_test_ctrl/atpg_test_en_reg/Q
```

//设置 dft 中 testmode 是从哪来,因为是来自内部信号,所以用 spec 指定

```
set_dft_signal -view existing_dft -type ScanEnable -active_state 1
-port [find port P03_PAD] -hookup_pin
I_prjxxxx_core/I_sys_ctrl/uAnd_ScanEn/uAnd2/Z
//设置 dft scanenable(SE)信号从哪来,因为是端口,已经存在用
existing_dft
//上面的-hookup_pin 指出测试信号与那个 pin 相连
set_scan_configuration -add_lockup true -chain_count 3 -clock_mixing
mix_clocks -style multiplexed_flip_flop -internal_clock single
扫描类型配置
set_autofix_configuration -type external_bus -method no_disabling
```
设置自动修复类型 此处为外部总线
```
create_test_protocol
//创建测试协议,生成 spf 文件,此处多余,都没输出 spf 文件—没指出生成
的文件名
dft_drc
//dft 的 drc 规则检查,下面那个就可以了
dft_drc > $report_path/$MNAME.dft_rtl.chk
//输出检测结果,
set_fix_multiple_port_nets -all -buffer_constants [get_designs "*"]
//设置 port 输出都会加入 buffer
#compile_ultra -no_autoungroup -scan

create_test_protocol  //此处多余,都没输出 spf 文件
dft_drc
dft_drc > $report_path/$MNAME.dft_pre_scan.chk

report_timing -path full_clock -input -cap -nets -transition
-max_path 100 -delay max
-nosplit > ./$report_path/$MNAME.pre_scan.time.max.report
```

```
report_timing -path full_clock -input -cap -nets -transition
-max_path 100 -delay min
-nosplit > ./$report_path/$MNAME.pre_scan.time.min.report
report_constraint
-all_violators > ./$report_path/$MNAME.pre_scan.vio

set_dft_signal -view spec -port [find port "P01_PAD"] -hookup_pin
I_prjxxxx_ipio/uP0_1/C -type ScanDataIn
set_dft_signal -view spec -port [find port "P06_PAD"] -hookup_pin
I_prjxxxx_ipio/uP0_6/C -type ScanDataIn
set_dft_signal -view spec -port [find port "P12_PAD"] -hookup_pin
I_prjxxxx_ipio/uP1_2/C -type ScanDataIn

set_dft_signal -view spec -port [find port "P00_PAD"] -hookup_pin
I_prjxxxx_ipio/uP0_0/I -type ScanDataOut
set_dft_signal -view spec -port [find port "P04_PAD"] -hookup_pin
I_prjxxxx_ipio/uP0_4/I -type ScanDataOut
set_dft_signal -view spec -port [find port "P11_PAD"] -hookup_pin
I_prjxxxx_ipio/uP1_1/I -type ScanDataOut
//与其它控制信号一样,制定数据的输入输出管脚
set_dft_signal -view spec -port [find port "P03_PAD"] -hookup_pin
I_prjxxxx_core/I_sys_ctrl/uAnd_ScanEn/uAnd2/Z -type ScanEnable
-active_state 1
//重复定义了
set_scan_path chain1 -view spec -scan_data_in P01_PAD -scan_data_out
P00_PAD -scan_master_clock P02_PAD
set_scan_path chain2 -view spec -scan_data_in P06_PAD -scan_data_out
P04_PAD -scan_master_clock P02_PAD
```

```
set_scan_path chain3 -view spec -scan_data_in P12_PAD -scan_data_out
P11_PAD -scan_master_clock P02_PAD
//配置 3 条链各自的输入输出及时钟
create_test_protocol -infer_clock -infer_asynch
//创建测试协议
dft_drc > $report_path/$MNAME.dft_pre_scan.chk
//dft  drc 检查
preview_dft > $report_path/$MNAME.dft.preview_dft
//察看链的信息,主要有如下的说明
/****************************************
Preview_dft report
For     : 'Insert_dft' command
Design  : prjxxxx
Version: F-2011.09-SP2
Date    : Wed May 31 16:46:53 2017
****************************************
Number of chains: 3
Scan methodology: full_scan
Scan style: multiplexed_flip_flop
Clock domain: mix_clocks
Scan enable: P03_PAD  (hookup pin:
I_prjxxxx_core/I_sys_ctrl/uAnd_ScanEn/uAnd2/Z, sense:
non_inverted)
Scan chain 'chain1'  (P01_PAD --> P00_PAD) contains 2229 cells
Scan chain 'chain2'  (P06_PAD --> P04_PAD) contains 2229 cells
Scan chain 'chain3'  (P12_PAD --> P11_PAD) contains 2229 cells
************ Test Point Plan Report ************
Total number of test points   : 0
Number of Autofix test points: 0
```

Number of Wrapper test points: 0

Number of test modes : 0

Number of test point enables : 0

Number of data sources : 0

Number of data sinks : 0

****

/**********

insert_dft

//插入链,即把所有的链串起来

write_test_protocol -out $report_path/$MNAME.spf

//写测试协议,给 tmax 生成测试向量用

dft_drc > $report_path/$MNAME.dft_post_scan.chk

write_scan_def -output results/netlist/${MNAME}_scan.def

//输出 scan def 描述,它是可测试设计的通用标准,是包含扫描链链接信息的 def 格式的文件,如下:

//**********

- 1_SG6

+ START I_prjxxxx_core/I_sys_ctrl/LOCKUP1 Q

+ FLOATING

I_prjxxxx_core/I_sys_ctrl/uClkTcm3/uClkSw/clk1_sel_reg1_reg (IN TI) (OUT Q)

I_prjxxxx_core/I_sys_ctrl/uClkTcm2/uClkSw/clk1_sel_reg1_reg (IN TI) (OUT Q)

I_prjxxxx_core/I_sys_ctrl/uClkTcm1/uClkSw/clk1_sel_reg1_reg (IN TI) (OUT Q)

```
I_prjxxxx_core/I_sys_ctrl/uClkTcm0/uClkSw/clk1_sel_reg1_reg ( IN
TI ) ( OUT Q )
+ PARTITION
I_prjxxxx_core/I_sys_ctrl/uAtpgMux_ClkTimerSlow/uMux/Z_55_55
+ STOP I_prjxxxx_core/I_sys_ctrl/uClkTcm0/uClkSw/LOCKUP D ;

- 1_SG7
+ START I_prjxxxx_core/I_sys_ctrl/uClkTcm0/uClkSw/LOCKUP Q
+ FLOATING I_prjxxxx_core/I_sys_ctrl/uClkOsc_in/clk1_sel_reg1_reg
( IN TI ) ( OUT QN )
+ PARTITION
I_prjxxxx_core/I_sys_ctrl/uAtpgMux_ClkXtal/uMux/Z_55_55
+ STOP I_prjxxxx_core/I_sys_ctrl/uClkOsc_in/LOCKUP D ;
//*****************************************//

report_scan_path -view existing_dft -chain all >
$report_path/$MNAME.scan.report
//输出链报告
report_scan_path -view existing_dft -cell all >>
$report_path/$MNAME.scan.report
//输出 cell 报告
report_timing -path full_clock -input -cap -nets -transition
-max_path 100 -delay max
-nosplit > ./$report_path/$MNAME.time.max.report
report_timing -path full_clock -input -cap -nets -transition
-max_path 100 -delay min
-nosplit > ./$report_path/$MNAME.time.min.report
report_constraint -all_violators > ./$report_path/$MNAME.vio
```

```
#################output result & report################
#change_names -rules verilog -verbose -hierarchy
change_names -rules verilog -hierarchy
write_file -format verilog -hierarchy -output ./$net_path/$MNAME.sv
//输出带扫描链的网表
write -f ddc -hier -output $ddc_path/$MNAME.ddc
//输出带扫描链的 DDC

#############write out dont-touch list#####################
get_cells -h "*uAnd*" >>  ./$net_path/$MNAME.dont_touch_cell_list
get_cells -h "*uMux*" >>  ./$net_path/$MNAME.dont_touch_cell_list
get_cells -h
"*uAtpgMux*" >>  ./$net_path/$MNAME.dont_touch_cell_list
#################write out func-sdc##########################
reset_design
set con_type post_layout
set CASE_MODE 16M
source -echo $script_path/${MNAME}_src.sdc  //读入综合用的 sdc
write_sdc results/netlist/${MNAME}_func0.sdc //输出 sdc 给 PR 用
check_timing > check_func0.timing

reset_design
set con_type post_layout
set CASE_MODE 24M
source -echo $script_path/${MNAME}_src.sdc
write_sdc results/netlist/${MNAME}_func1.sdc
check_timing > check_func1.timing
```

```
reset_design
source -echo $script_path/${MNAME}_scan_src.sdc
write_sdc results/netlist/${MNAME}_scan.sdc
################ write out bist-sdc ######################
```

相关文件介绍

```
//********************
/prjxxxx_syn_rtl_read.tcl 格式如下：
set search_path "$search_path \  ##设置 include 路径
/xxx/chip/./rtl/misc \
/xxxxx/chip/./rtl/flash/";
analyze -format verilog -define {DBG51 SYN EXPDP4P5 ECO ECO1 ECO2 } { \
/xxx/chip/./rtl/generic/clkswitch.v \
/xxx/chip/./rtl/generic/sync_level.v \
……
/xxx/chip/./rtl/top/spare_cells.v \
}
elaborate module_top_name -architecture verilog -library work
```

xxx.src 时钟约束文见下节“sdc 文件分析“

10. dc 需要输出给后端的文件

--给pr 使用的文件

1. 综合后的网表—插入 dft后的

```
write_file -format verilog -hierarchy -output ./$net_path/$MNAME.sv
```

//输出带扫描链的网表

2. 约束文件—根据不同的模式需要 不同的 sdc

```
write_sdc results/netlist/${MNAME}_func1.sdc
```

3. Scan.def 文件

```
write_scan_def -output results/netlist/${MNAME}_scan.def
```

4. Don’t_touch 文件

```
get_cells -h "*uAnd*" >>  ./$net_path/$MNAME.dont_touch_cell_list
```

11. Sdc 文件分析

Sdc,主要是对时钟和复位信号的设置分析

```
    set osc48m_clock_period     20
    set osc1_clock_period       60
    set osc2a_clock_period      1000
    set crystal_clock_period  62
    set bs_clock_period       90
    set timer_clock_period    60
    set test_clock_period     100
    set adc_clock_period      100
    set sensor_clock_period   18
    set timer_clock_slow_peroid 1250
set clock_skew 0.2
set max_fanout 20
set con_type "pre_layout"
```

上面是设置内部变量参数

```
set_case_analysis  0 [find pin
I_sssss_core/I_sys_ctrl/uAtpgMux_ClkRam/S0]
set_case_analysis 0 [find pin
I_sssss_core/I_test_ctrl/atpg_test_en_reg/Q]
set_case_analysis 0 [find pin
I_sssss_core/I_flash_bist/JPFLS016K29DH_i/READM1]
set_case_analysis 0 [find pin
I_sssss_core/I_flash_bist/JPFLS016K29DH_i/READM0]
set_case_analysis 0 [get_pins I_sssss_ipio/uP*/LDRV*]
set_case_analysis 0 [get_pins I_sssss_ipio/uP*/ST]
```

上面是配置某些信号固定

```
create_clock -name clk_vir_48m -period $osc48m_clock_period
create_clock -name clk_vir_mcudiv -period $osc1_clock_period
create_clock -name clk_vir_800k -period $osc2a_clock_period
```

```
create_clock -name clk48m -period $osc48m_clock_period [get_pin
I_sssss_ipio/uAna_pll/CLKOUT]
```
上面是定义源时钟 0

```
create_generated_clock -name clk_osc48m_0 [get_pin
I_sssss_core/I_sys_ctrl/ClkOsc_in_reg/D] -source [get_pin
I_sssss_ipio/uAna_pll/CLKOUT] -divide_by 1  -combinational

create_generated_clock -name clk_osc48m [get_pin
I_sssss_core/I_sys_ctrl/uAtpgMux_ClkOsc/uMux/Z] -source [get_pin
I_sssss_core/I_sys_ctrl/ClkOsc_in_reg/D] -divide_by 1
```
上面是定义派生时钟---与对应的源时钟为同一个时钟域

```
create_clock -name clk_osc800k -period $osc2a_clock_period [get_pin
I_sssss_ipio/uAna_Osc800k/OSC_800K]
```
上面是定义源时钟 1

```
create_clock -name clk_crystal -period $crystal_clock_period -waveform
{0.0 3.0} [get_pin I_sssss_ipio/uAna_Xtal32k/XTALCLK]
```
上面是定义源时钟 2

```
create_generated_clock -name clk_xtal [get_pin
I_sssss_core/I_sys_ctrl/uAtpgMux_ClkXtal/uMux/Z] -source [get_pin
I_sssss_ipio/uAna_Xtal32k/XTALCLK] -divide_by 1  -combinational

create_clock -name clk_adc -period $adc_clock_period [get_pin
I_sssss_core/I_sys_ctrl/uAtpgMux_ClkAdc/uMux/Z]

……
if {$con_type == "pre_layout" } {
create_generated_clock -name clk_mcu_div [get_pin
I_sssss_core/I_sys_ctrl/uClkMcuDiv/uTlat/Z] -source [get_pin
I_sssss_core/I_sys_ctrl/uAtpgMux_ClkOsc/uMux/Z] -edges {1 2 63}
} else {
if {$CASE_MODE == "24M" } {
set_case_analysis 1 [find pin I_sssss_core/I_misc_reg/MISCREG_reg_3/Q]

create_generated_clock -name clk_mcu_div [get_pin
I_sssss_core/I_sys_ctrl/uClkMcuDiv/uTlat/Z] -source [get_pin
I_sssss_core/I_sys_ctrl/uAtpgMux_ClkOsc/uMux/Z] -edges {1 2 5}
} else {
create_generated_clock -name clk_mcu_div [get_pin
I_sssss_core/I_sys_ctrl/uClkMcuDiv/uTlat/Z] -source [get_pin
```

```
I_sssss_core/I_sys_ctrl/uAtpgMux_ClkOsc/uMux/Z] -edges {1 2 7}
-combinational
}
}
create_generated_clock -name clk_prog    [get_pin
I_sssss_core/I_sys_ctrl/uClk1M/uTlat/Z] -source     [get_pin
I_sssss_core/I_sys_ctrl/uAtpgMux_ClkOsc/uMux/Z] -edges {1 2 97}

……

create_clock -name clk_test -period $test_clock_period [get_port
PO2_PAD]
create_generated_clock -name clk_ram [get_pin
I_sssss_core/I_sys_ctrl/uAtpgMux_ClkRam/uMux/Z] -source [get_pin
I_sssss_core/I_sys_ctrl/uClkMcuDiv/uTlat/Z] -master_clock clk_mcu_div
-add -divide_by 1
```
注意上面派生时钟的写法

```
create_clock -name clk_timer_slow -period $timer_clock_slow_peroid
[get_pin I_sssss_core/I_sys_ctrl/uAtpgMux_ClkTimerSlow/uMux/Z]

set_clock_uncertainty $clock_skew [all_clocks]
```
上面是设置时钟之间的关系

```
if {$con_type == "pre_layout" } {
set_ideal_network -no_propagate [get_nets
"I_sssss_core/I_sys_ctrl/PorAtpg"]
set_ideal_network -no_propagate [get_nets
"I_sssss_core/I_sys_ctrl/ResetnPor_osc1"]
set_ideal_network -no_propagate [get_nets
"I_sssss_core/I_sys_ctrl/ResetnPor_osc2"]
set_ideal_network -no_propagate [get_nets
"I_sssss_core/I_sys_ctrl/ResetnPor_xtal"]
set_ideal_network -no_propagate [get_nets
"I_sssss_core/I_sys_ctrl/ResetnPor"]
set_ideal_network -no_propagate [get_nets
"I_sssss_core/I_sys_ctrl/ResetnAndMcu"]
set_ideal_network -no_propagate [get_nets
"I_sssss_core/I_sys_ctrl/ResetnMcu"]
set_ideal_network -no_propagate [get_pins
"I_sssss_core/I_sys_ctrl/uAnd_ScanEn/uAnd2/Z"]
```
上面这些语句是设置复位信号为理想网络,这样在综合时就不会插入 buff 在里面

```
} else {
```

```
set_propagated_clock [all_clocks]
set_clock_gating_check -setup 0.5 -hold 0.5 [all_clocks]
}
```
上面是配置对时钟,复位网络的处理方式:pr 前都是理想化处理

```
set_input_transition 0.8  [all_inputs]
```
对输出信号的描述:
```
set_load 0.1 [all_outputs]
```
对输出信号的负载定义

```
set gpio_input [find port "P00_PAD P01_PAD P02_PAD P03_PAD P04_PAD
P05_PAD P06_PAD P07_PAD \
                           P10_PAD P11_PAD P12_PAD P13_PAD P14_PAD
P15_PAD P16_PAD P17_PAD \
                           P20_PAD P21_PAD P22_PAD P23_PAD P24_PAD
P25_PAD P26_PAD P27_PAD \
                           P30_PAD P31_PAD P32_PAD P33_PAD P34_PAD
P35_PAD P36_PAD P37_PAD \
                           P40_PAD P41_PAD P42_PAD P43_PAD P44_PAD
P45_PAD P46_PAD P47_PAD "]

set gpio_output [find port "P00_PAD P01_PAD P02_PAD P03_PAD P04_PAD
P05_PAD P06_PAD P07_PAD \
                           P10_PAD P11_PAD P12_PAD P13_PAD P14_PAD
P15_PAD P16_PAD P17_PAD \
                           P20_PAD P21_PAD P22_PAD P23_PAD P24_PAD
P25_PAD P26_PAD P27_PAD \
                           P30_PAD P31_PAD P32_PAD P33_PAD P34_PAD
P35_PAD P36_PAD P37_PAD \
                           P40_PAD P41_PAD P42_PAD P43_PAD P44_PAD
P45_PAD P46_PAD P47_PAD "]

##constraint gpio  interface
set_input_delay -max 20 $gpio_input -clock clk_vir_mcudiv
set_input_delay -min 0  $gpio_input -clock clk_vir_mcudiv

set_output_delay -max 15 $gpio_output  -clock clk_vir_mcudiv
set_output_delay -min 0  $gpio_output -clock clk_vir_mcudiv


set test_input  [find port "P01_PAD P03_PAD"]
set test_output [find port "P00_PAD P04_PAD"]

set_input_delay -max -clock clk_test 40 $test_input
```

```
set_input_delay -min -clock clk_test 20 $test_input

set_output_delay -max -clock clk_test 15 $test_output
set_output_delay -min -clock clk_test 0 $test_output


set_input_delay -max 18 [find pin  "I_sssss_ipio/uAna_POR/LVD_OUT"]
-clock clk_vir_48m
set_input_delay -min 0 [find pin  "I_sssss_ipio/uAna_POR/LVD_OUT"]
-clock clk_vir_48m
set_input_delay -max 8 [find pin  "I_sssss_ipio/uAna_Pch/VOUT"] -clock
clk_vir_48m
set_input_delay -min 0 [find pin  "I_sssss_ipio/uAna_Pch/VOUT"] -clock
clk_vir_48m
```

对输入输出 pad 时序上的约束,相当于约束输入 pad 到第一级寄存器的延时和最后一个寄存器到输出 pad 的延时

```
####exception for the design##########
set CLK_LIST [all_clocks]
foreach_in_collection clk1 $CLK_LIST {
    if {  [get_object_name $clk1] == "clk_test" } {
    } else {
    set_false_path -from [get_clocks clk_test] -to [get_clocks $clk1]
    set_false_path -from [get_clocks $clk1] -to [get_clocks clk_test]
    }
}

set_false_path -from [get_clocks clk_osc800k] -to [get_clocks
clk_timer2]
set_false_path -from [get_clocks clk_osc800k] -to [get_clocks
clk_timer3]
set_false_path -from [get_clocks clk_osc800k] -to [get_clocks clk_uart]
……
```

设置 false path,不做同步分析

```
if {$con_type == "pre_layout" } {
set_multicycle_path 2 -through
I_sssss_core/I_flash_bist/JPFLS016K29DH_i/DOUT -setup
set_multicycle_path 1 -through
I_sssss_core/I_flash_bist/JPFLS016K29DH_i/DOUT  -hold
```

设置多周期路径

```
set_max_delay 58 -through
I_sssss_core/I_flash_bist/JPFLS016K29DH_i/DOUT
}
```

设置组合路径延时

```
set nand_tree_input [find port "P01_PAD P02_PAD P03_PAD P04_PAD P05_PAD
P06_PAD P07_PAD \
                            P10_PAD P11_PAD P12_PAD P13_PAD P14_PAD
P15_PAD P16_PAD P17_PAD \
                            P20_PAD P21_PAD P22_PAD P23_PAD P24_PAD
P25_PAD P26_PAD P27_PAD \
                            P30_PAD P31_PAD P32_PAD P33_PAD P34_PAD
P35_PAD P36_PAD P37_PAD \
                            P40_PAD P41_PAD P42_PAD P43_PAD P44_PAD
P45_PAD P46_PAD P47_PAD "]

set nand_tree_output  [find port "P00_PAD"]
```

上面是设置变量

```
if {$con_type == "post_layout" } {
set_disable_clock_gating_check  I_sssss_core/I_ip ioc/U2166/B
}
```

12. 查看 DC 关键结果

dc 综合后,需要检查的结果有

查看 log:一定要看 log 是否有 Error;

1:设计方面

1.1upf 是否正确

使用 check_mv_design 命令检查

```
check_mv_design  -verbose >
${REPORT_PATH}${top}_check_mv_design.rpt
```

1.2检查设计---多余的线 ,位宽不匹配,cell 无 driver 或 load 等等问题

check_design > ./$report_path/$MNAME.chk

---与 nLint 功能差不多。

1.3检查约束是否完善

check_timing 执行时序检查

---把最差路径（无论是否满足时序）打印出来（设置打印多少条）

1.4查看电路是否违反设计规则要求,包含时序及其它规则

report_constraint -all_violators > ./$report_path/$MNAME.vio

——把所有的时序违背，面积（max_area）违背，负载（max_capacitance）违背，max_transition 违背，功耗（max_leakage_power）违背等信息报告出来

1.5门控时钟插入是否正常

report_clock_gating > results/report/clk_gating.rpt

report_clock_gating -ungated -verbose -nosplit >> results/report/clk_gating.rpt

report_clock_gating -gated -verbose -nosplit >> results/report/clk_gating.rpt

2:时序方面

2.1 报告时序设计违背

report_constraint -all_violators > ./$report_path/$MNAME.vio

2.1 报告时序路径延时

report_timing -path full_clock -input -cap -nets -transition -max_path 100 -delay max -nosplit > ./$report_path/$MNAME.time.max.report

report_timing -path full_clock -input -cap -nets -transition -max_path 100 -delay min -nosplit > ./$report_path/$MNAME.time.min.report

通过 report_timing命令报出两种 case 的最差时序情况,主要是看 setup 是否满足.holdtime 由 pR 后检查

13. Lib 文件转 db 文件

Dc 综合时需要的是 db 文件,我们可以打开查看的为lib 文件,当 dc 综合时需要通过 dc 工具把 lib 文件转化为 db 文件,步骤如下

dc2015.06 版之后, 自身就不再支持 librarycompiler,需要自行安装

打开 dc 工具:dc_shell

使能 write db 功能: enable_write_lib_mode

读取 lib 文件: read_lib RASP1024x9_ff_1p65v_0c.lib

写 db 文件: write_lib -format db -o RASP1024X9_ff_1p65v_0c RASP1024x9_ff_1p65v_0c.lib

做成一个 tcl 脚本如下 lib2db.tcl:

```
date
enable_write_lib_mode
read_lib RASP1024x9_ss_1p35v_125c.lib
write_lib -format db -o RASP1024X9_ss_1p35v_125c
read_lib RASP1024x9_ff_1p65v_0c.lib
write_lib -format db -o RASP1024X9_ff_1p65v_0c
RASP1024x9_ff_1p65v_0c.lib
exit
```

执行命令: dc_shell -f lib2db.tcl | tee lib2db.log

Spf 文件分析

Spf 基础

Spf 就是 stil Procedures File，由 dc 工具生成的文件；tmax 根据综合后的网表和 spf 文件产生 atpg 测试向量，产生的向量可以有多种形式（ 目前只接触到 2 种）；
一种生成的向量在 testbench 文件中，直接以数据形式给出，这种文件就一个 testbench 文件；
另外一种是以.stil 文件格式存在，在 testbench 中用函数 $STILDPV_setup ("e.stil",,," t.dut")来读取.stil 文件，此种方法 vcs 需要增加选项来获取该函数的能力，此种方式有 2 个文件，一个 testbench 文件，一个.stil 文件。其中.stil 文件与 spf 有点相似。因为 stil 文件就是工具 TMAX 从网表文件提取相关信息，根据 spf 格式生成更加详细的文件---spf 文件相当于一个模板

spf 文件格式(转)

stil 文件是 atpg 的基础，用好 atpg 工具当然要熟悉 stil 格式。
下面是基本说明：）

```
STIL 1.0 {
Extension DesignP2000.5;
}
Header {
Title " TetraMAX(TM) 2000.05-i000622_173054 STIL output";
Date "Wed Dec 3117:21:05 1999";
```

```
History { }
}
Signals {
CLK In; RSTB In; SDI2In; SDI1 In; INC In; SCAN In; HACKIN In; si4 In;
six In; DO InOut; D1InOut; D2 InOut; D3 InOut; SDO2 Out; COUT Out;
HACKOUT Out; so4 Out;sox Out;
}
SignalGroups {
_pi = 'DO + D1 + D2 + D3+ CLK + RSTB + SDI2 + SDI1 + INC +
SCAN + HACKIN + si4 +six';
_default_Clk1_Timing_ ='RSTB';
_io = 'DO + D1 + D2 +D3' { WFCMap 0X->0; WFCMap 1X->1; WFCMap
ZX->Z; WFCMapNX->N; }
_po = 'SDO2 + COUT + DO+ D1 + D2 + D3 + HACKOUT + so4 + sox';
_default_In_Timing_ ='DO + D1 + D2 + D3 + CLK + RSTB + SDI2 +
SDI1 + INC + SCAN +HACKIN + si4 + six';
_default_Out_Timing_ ='SDO2 + COUT + DO + D1 + D2 + D3 + HACKOUT
+ so4 + sox';
_default_Clk0_Timing_ ='CLK';
  }

ScanStructures {
// Uncomment and modifythe following to suit your design
// ScanChain chain_name{ ScanIn chain_input_name; ScanOut
chain_output_name; }
}
Timing {
WaveformTable_default_WFT_ {
Period '100ns';
Waveforms {
_default_In_Timing_ { 0{ '0ns' D; } }
_default_In_Timing_ { 1{ '0ns' U; } }
_default_In_Timing_ { Z{ '0ns' Z; } }
_default_In_Timing_ { N{ '0ns' N; } }
_default_Clk0_Timing_ {P { '0ns' D; '50ns' U; '80ns' D; } }
_default_Clk1_Timing_ {P { '0ns' U; '50ns' D; '80ns' U; } }
_default_Out_Timing_ { X{ '0ns' X; } }
_default_Out_Timing_ { H{ '0ns' X; '40ns' H; } }
_default_Out_Timing_ { T{ '0ns' X; '40ns' T; } }
_default_Out_Timing_ { L{ '0ns' X; '40ns' L; } }
}
}
}
PatternBurst _burst_ {PatList {
_pattern_ {
```

```
}
}}
PatternExec {
PatternBurst_burst_;
}
Procedures {
capture_CLK {
W _default_WFT_;
forcePI: V { _pi=\r13 #; _po=\j \r9 X ; }
measurePO: V { _po=\r9 #; }
pulse: V { CLK=P; _po=\j\r9 X ; }
}
capture_RSTB {
W _default_WFT_;
forcePI: V { _pi=\r13 #; _po=\j \r9 X ; }
measurePO: V { _po=\r9 #; }
pulse: V { RSTB=P;_po=\j \r9 X ; }
}
capture {
W _default_WFT_;
forcePI: V { _pi=\r13 #; _po=\j \r9 X ; }
measurePO: V { _po=\r9 #; }
}

// Uncomment and modifythe following to suit your design
// PRE_CLOCK_MEASUREProcedures {
// load_unload {
// W_default_WFT_;
// C { test_so=X;test_si=0; test_si2=0; test_so2=X; clk=0; tclk=0;
reset=1; test_se=1; }
// Shift { W_default_WFT_;
// V { _si=#; _so=#; CLK= P; }
// }
// }
// TMAX GENERATEDPOST_CLOCK_MEASURE (Closer to DFTCompiler Procedures {
// load_unload {
// W_default_WFT_;
// C { test_si=0;test_si2=0; clk=0; tclk=0; reset=1; test_se=1; }
// V { _so=##; }
// Shift { W_default_WFT_;
// V { _si=##; _so=##;clk=P; }
// }
}
MacroDefs {
```

```
test_setup {
W _default_WFT_;
V { CLK=0; RSTB=1;}
}
}
```

开始版本和头不用说。
1.定义设计的信号，即哪些是输入信号，哪些是输出信号，哪些信号是输入输出。
eg.

```
Signals {
CLK In; RSTB In; SDI2In; SDI1 In; INC In; SCAN In; HACKIN In; si4 In;
six In; DO InOut; D1InOut; D2 InOut; D3 InOut; SDO2 Out; COUT Out;
HACKOUT Out; so4 Out;sox Out;
}
```

2.定义信号组：

```
_in = input pins
_out = output pins
   _io = bidirectional pins

   _pi = inputs +bidirectional pins

   _po = outputs +bidirectional pins

   _si = scan chain inputs

   _so = scan chain outputs
```

设置时钟的信号组
eg.

```
   SignalGroups {

   _pi = 'DO + D1 + D2 + D3+ CLK + RSTB + SDI2 + SDI1 + INC' ;

   _io = 'DO + D1 + D2 +D3' ;

   _po = 'SDO2 + COUT + DO+ D1 + D2 + D3 + HACKOUT + so4 + sox';

   _default_ClkO_Timing_ ='CLK';

   }
```

3.定义扫描链结构
eg.

```
   ScanStructures {
```

```
ScanChain "c1"{ ScanIn SDI2; ScanOut SDO2; }
ScanChain "c2"{ ScanIn SDI1; ScanOut D1; }
ScanChain "c3"{ ScanIn DIN; ScanOut YABX; }
ScanChain "c4"{ ScanIn "IRQ[4]"; ScanOut XYZ; }
}
```

对于双向 io 还有额外参数 WFCMap 设置多状态情况下的值

4.定义信号组各种状态的时序

如时钟周期，输入 0,1,Z,N。输出 H,L,T,X。

其中

0，表示输入 0；

1，表示输入 1；

2，Z，表示输入关闭，

3，N，表示输入未知。

H，表示输出为高；

L，表示输出为低；

T，表示输出关闭；

X，表示输出无操作

P 表示脉冲，

D 表示强制为低，

U 表示强制为高

eg.如下，定义了输入的 0,1,N,Z 的时序以及 Clock，reset 的脉冲波形，输出的 H，L，T，X 波形。

```
Timing {
WaveformTable_default_WFT_ {
Period '100ns';
Waveforms {
_default_In_Timing_ { 0{ '0ns' D; } }
_default_In_Timing_ { 1{ '0ns' U; } }
_default_In_Timing_ { Z{ '0ns' Z; } }
_default_In_Timing_ { N{ '0ns' N; } }
_default_Clk0_Timing_ {P { '0ns' D; '50ns' U; '80ns' D; } }
_default_Clk1_Timing_ {P { '0ns' U; '50ns' D; '80ns' U; } }
_default_Out_Timing_ { X{ '0ns' X; } }
_default_Out_Timing_ { H{ '0ns' X; '40ns' H; } }
```

```
_default_Out_Timing_ { T{ '0ns' X; '40ns' T; } }
_default_Out_Timing_ { L{ '0ns' X; '40ns' L; } }
}
}
}
```

5.定义各种操作的时序,如：

```
capture_RSTB {
W _default_WFT_;
forcePI: V { _pi=\r13 #; _po=\j \r9 X ; }
measurePO: V { _po=\r9 #; }
pulse: V { RSTB=P;_po=\j \r9 X ; }
}
```

由于 atpg 是基于周期的，所以一个向量 V 即可表示一个周期，另外

F 表示 force，force 信号为固定值；

C 表示 condition，满足某一条件，向下进行。

定义了一个 capture_RSTB 的操作，这个操作使用_default_WFT_定义的 x 信号组的状态时序，包含 3 个周期：

第一个周期 forcePI 操作，

#表示输入，

\r 表示重复，

\j 表示连接。

所以第一周期表示信号组_pi 输入 13 个数据，信号组_po 的每个信号在原来的值基础上连接上一个x

第二个周期 measurePo 操作，_po 得到 9 个输出.

第三个周期pulse 操作，产生 RSTB 脉冲，信号组_po 重新连接上 9 个x

WFT ：waveform table, 在 timing procedure 定义

调用 WFT 时，用 W 来引用

F：表示 force signal

V：表示一个 cycle，例如，"rst"=1,"clk"=P

C：表示一个条件的判断 conditional

实际 stil 文件说明

Spf 文件没有下面这些内容，这些是 tmax 生成的 pattern

```
Pattern "_pattern_" {
   W "_multiclock_capture_WFT_";
   "precondition all Signals": C { "_pi"=\j \r45 X ; "_po"=\j \r45 X ; }
   Macro "test_setup";
   Ann {* chain_test *}
   "pattern 0": Call "load_unload" {  //调用 load_unload 来输出数据
      "PO1_PAD"=
00110011001100110011001100110011001100110011001100110011001100110011001100110
01100110011001100110011001100110011001100110011001100110011001100110011001100
110011001100110011001100110……
}
Call "multiclock_capture" { //调用 multiclock_capture 来输出数据
      "_pi"=TTHHOHH1H1HHH1111111111111111111111111111111TTT;
"_po"=TTHHOHH1H1HHH1111111111111111111111111111111TTT; }
   "pattern 1": Call "load_unload" { //调用 load_unload 来输出数据

      "POO_PAD"=
HHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLH
HLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHH
LLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHL
LHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHLL
HHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHHLLHH
……
};
}
```

PT/sta 篇

输入文件为：pr 后的网表 netlist 及对应的 spef 文件
输出文件：用于后仿的 sdf 文件

PrimeTime:启动命令 pt_shell -f xxx.src

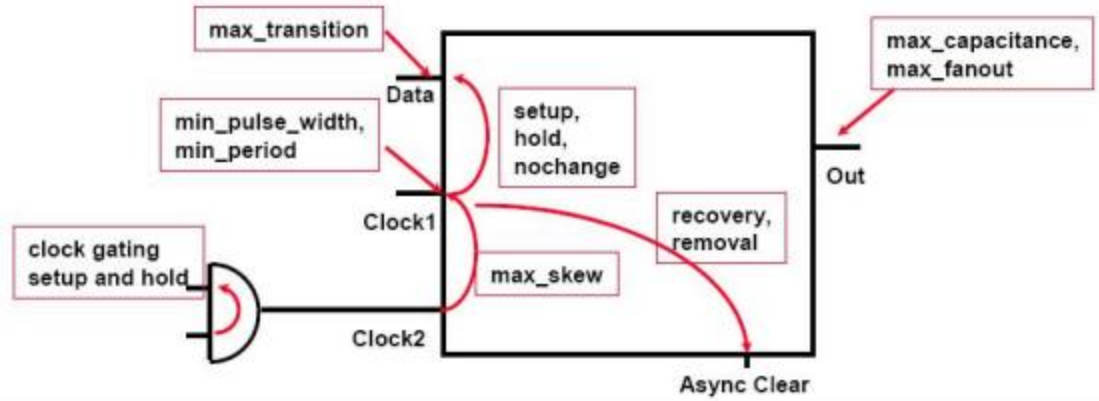
当有警告时，通过man warning_id 可以查阅警告详情

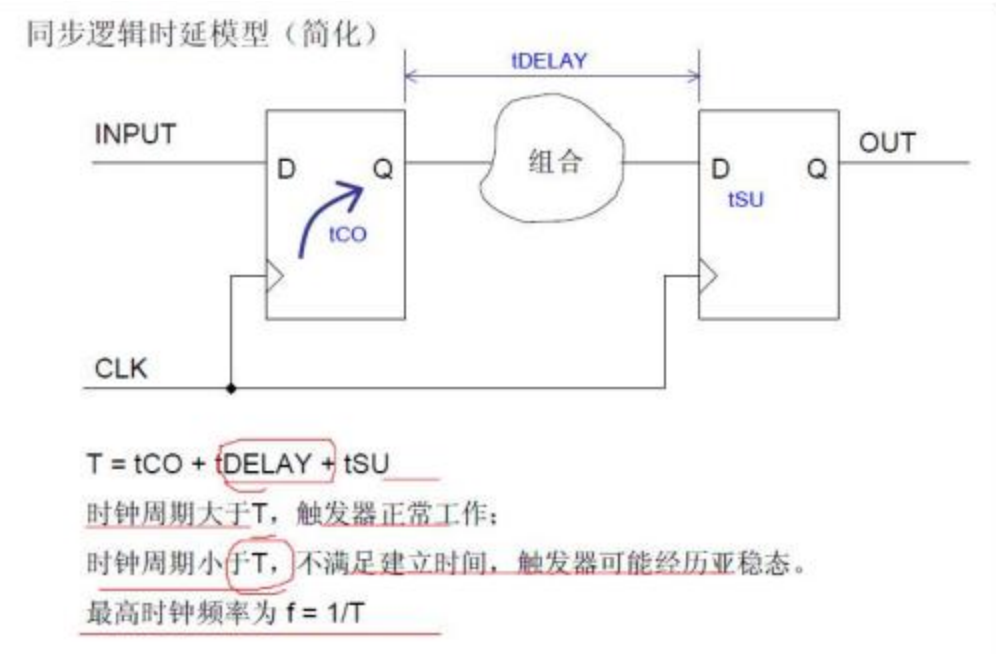
通过 help *keyword*可以查看所有的关键此为 keyword 的命令

通过man *keyword_cmd* 可以查看命令具体用法

时序基础介绍

主要的时序内容



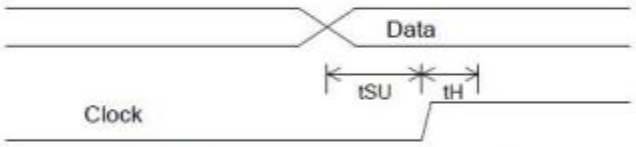


建立时间（Setup time）tSU

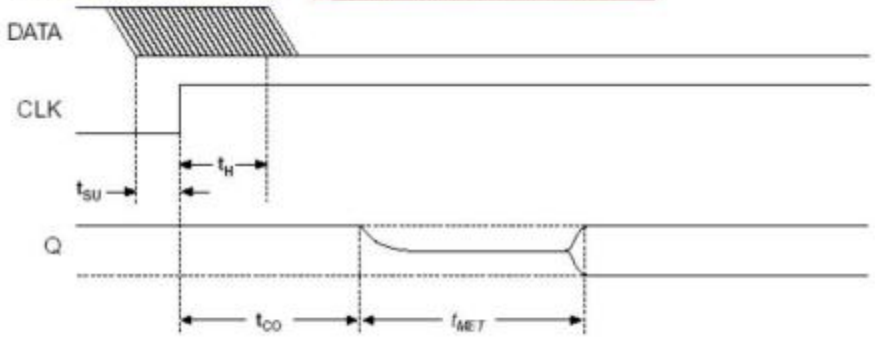
触发器的时钟沿到来之前，输入数据稳定不变的时间。

保持时间（Hold time）tH

触发器的时钟沿到来之后，输入数据稳定不变的时间。



不满足建立/保持时间时，触发器可能会经历亚稳态

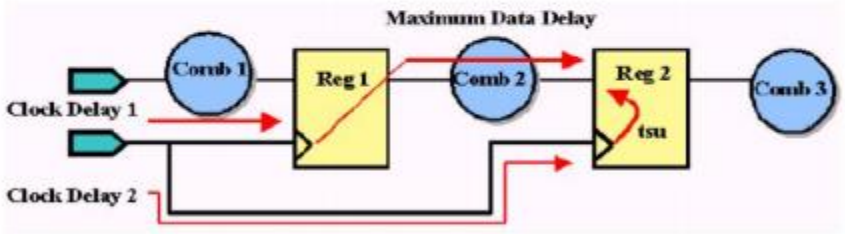


tSU - 建立时间（Setup time），tH - 保持时间（Hold time）

tCO – 时钟到输出时间（Clock-to-Output time）

tMET – 建立时间（Setting time）（亚稳态到稳态的时间，与器件工艺有关）

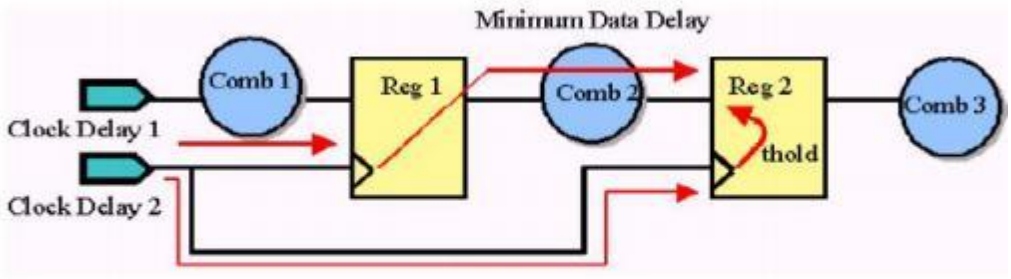
建立时间检查



clock delay 1 + max data path - clock delay 2 + tsu ≤ clock period

max_delay+tsu<=Tclk+△Tclk;　　(△Tclk=tclk2-tclk1)

保持时间检查

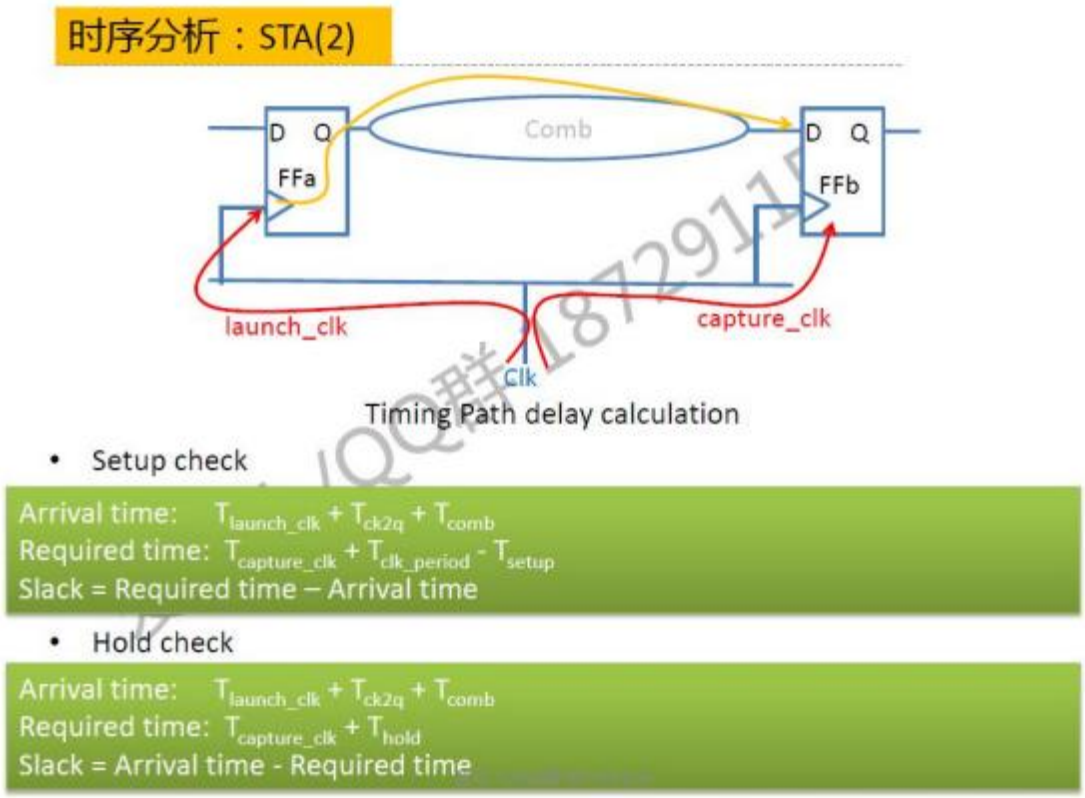


clock delay 1 + minimum data delay – clock delay 2 – th ⩾ 0

min_delay>=△Tclk+th

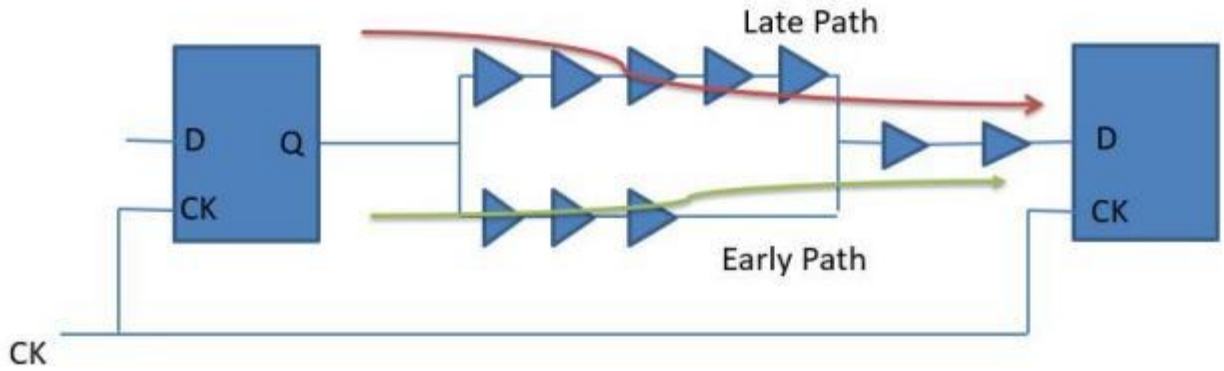
一般来说 clock delay2>clock_delay1

注意,有时会发现 hold time 为负数,这种情况就是因为 clock_delay1- clock delay2 大于 hold time,即上式中△Tclk 为负数,且大于 th



OCV 讲解（转）

今天我们要介绍的时序分析概念是 on chip variations，简称 OCV。OCV 会对时序分析提出更严格的要求。那为什么需要 OCV 呢，因为制造工艺的限制，同一芯片上不同位置的 MOS 晶体管的性能会有一些差异。库中的 PVT 是一个"点"，比如 1.2V，250℃，工艺 1.0。但实际芯片的 PVT 永远不会落在一个点上，而是一个范围；比如说有时序关系的几个 cell，可能这几个 cell 的 PVT 是 1.18V，20℃，工艺 0.98。而那个 cell 的 PVT 是 1.21V，35℃，工艺 1.01。这些 cell 的 PVT 都不在那个点上，怎么去分析呢？这时候就需要 OCV 了。



还是参照 single mode 和 bc-wc 那张图，OCV 的检查方式更加严格，如下所示：

For setup :

Launch clock path :late path from max lib

Data Path :late path from max lib

Capture Clock path :early path from min lib

For Hold:

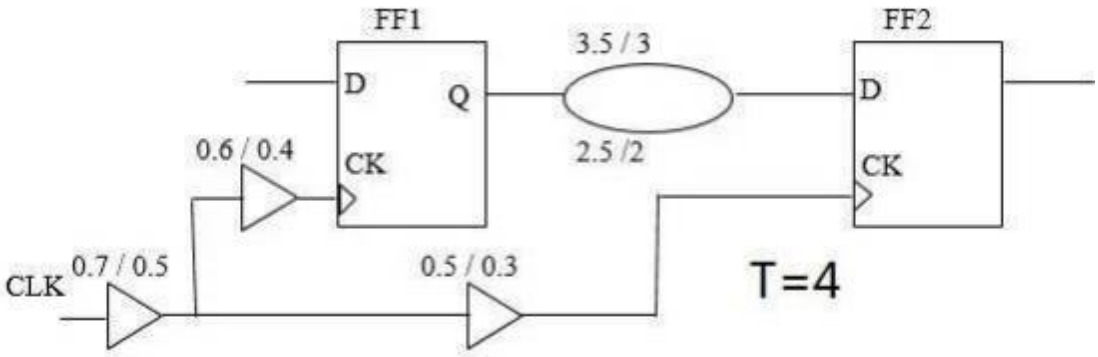
Launch clock path :early path from min lib

Data Path :early path from min lib

Capture Clock path :late path from max lib

OCV for setup check

还是原来 BC-WC 分析模式那张图：



假设 PVT 情况可以在整个 chip 上变化，（暂时不考虑 CPPR）

clock period = 4

Launch clock late path (max) = 0.7 + 0.6 = 1.3

Data late path (max) = 3.5

Capture clock early path (min) = 0.5 + 0.3 = 0.8

Setup = 0.2

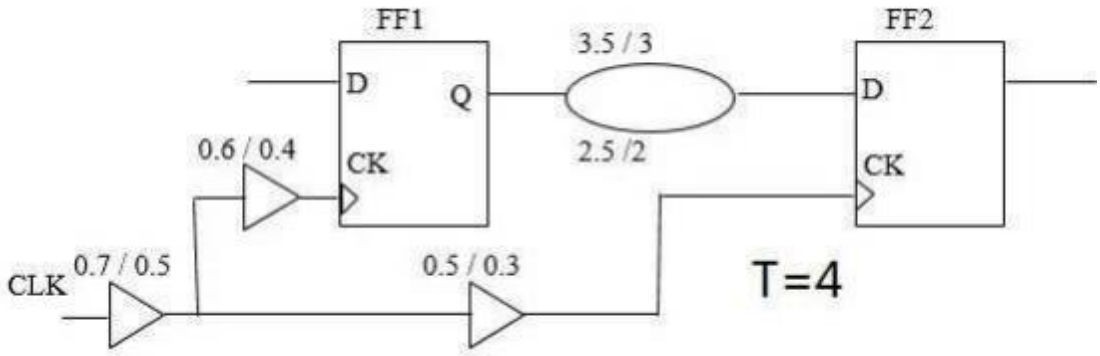
Data arrival time = 1.3 + 3.5 = 4.8

Data required time = 4 + 0.8 - 0.2 = 4.6

Slack =Data required time - Data arrival time=4.6 - 4.8 = -0.2

OCV for Hold check

对于 Hold check, 可以参考下图分析模式：



Launch clock early path (min) = 0.5 + 0.4 = 0.9

Data early path (min) = 2

Capture clock late path (max) = 0.7 + 0.5 = 1.2

Hold = 0.2

Data arrival time = 0.9 + 2 = 2.9

Data required time =1.2+0.2 = 1.4

Slack = Data arrival time - Data required time=2.9 - 1.4 = 1.5

通过上诉的描述，大家对 OCV 的分析模式有一定了解了吧，OCV 其实是一种相对悲观的分析模式，为了使 design 和 fabrication 之间的结果更加接近，现在又诞生了 AOCV，SOCV 等更高级的分析模式。

时序分析概念是 timing derate. 我们可以称为时序增减因子 。我们知道在芯片的生产过程中， 由于刻蚀，不同点的温度，金属不均匀， 串扰，晶体管沟道长度等影响因素，导致片上各个位置单元延迟不一样。因此，我们需要一个缩放因子来让设计更加严格。

timing derate 是计算 OCV 的一种简单方法，在某单一条件（BC-WC）下，把指定 path 的 delay 放大或者缩小一些，这个比率就是 derate。比如说：

-late

setup ==> data path * 1.1

hold ==> clock path * 1.1

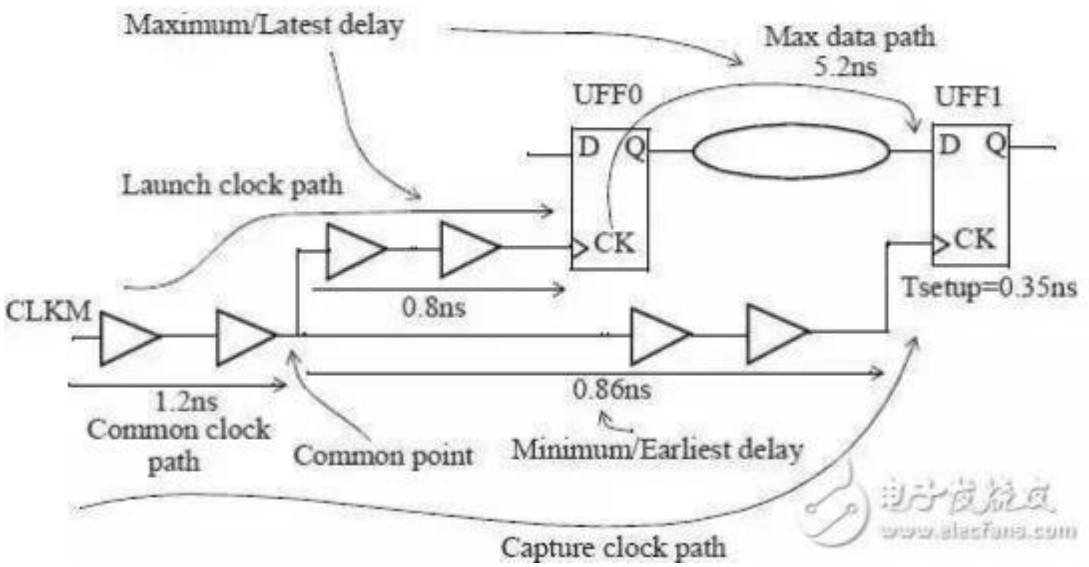
-early

setup ==> clock path * 0.9

hold ==> data path * 0.9

以下图为例：

在 setup check 中，



Date arrival time 即 data path 和 launch clock path 需要使用 -late 选项，使得路径变慢。

Date require time 即 capture clock path 需要使用 -early 选项，加快路径延迟。

需要注意的是：考虑 time derate 需要在某个单一条件下，比如说 BC 或者 WC 条件下，把指定 path 的延迟再放大或者缩小一点，要么是 BC，要么是 WC，不要把 BC 和 WC 混在一起，再 OCV，那样太过于悲观。

setup check 一般是工作在 WC PVT 条件下，因此不需要在 late path 上，即 launch clock path 以及 data path 上再加 time derate，因为在 WC 条件下，launch clock path 以及 data path 上的延迟已经是所有条件下最差的 delay 了，没有必要再加大延迟，但是 WC 条件下 capture clock path 上的 delay 肯定不是最小的，因此需要加快。

所以上面的 timing path 做 setup check，time derate 只需要这样设置：

set_timing_derate -early 0.9

set_timing_derate -late 1.0

我们可以计算一下设了 timing derate 以后 setup check 的变化：

上图中： launch clock path = （1.2+0.8）*1.0 = 2.0

max data path = 5.2 * 1.0 =5.2

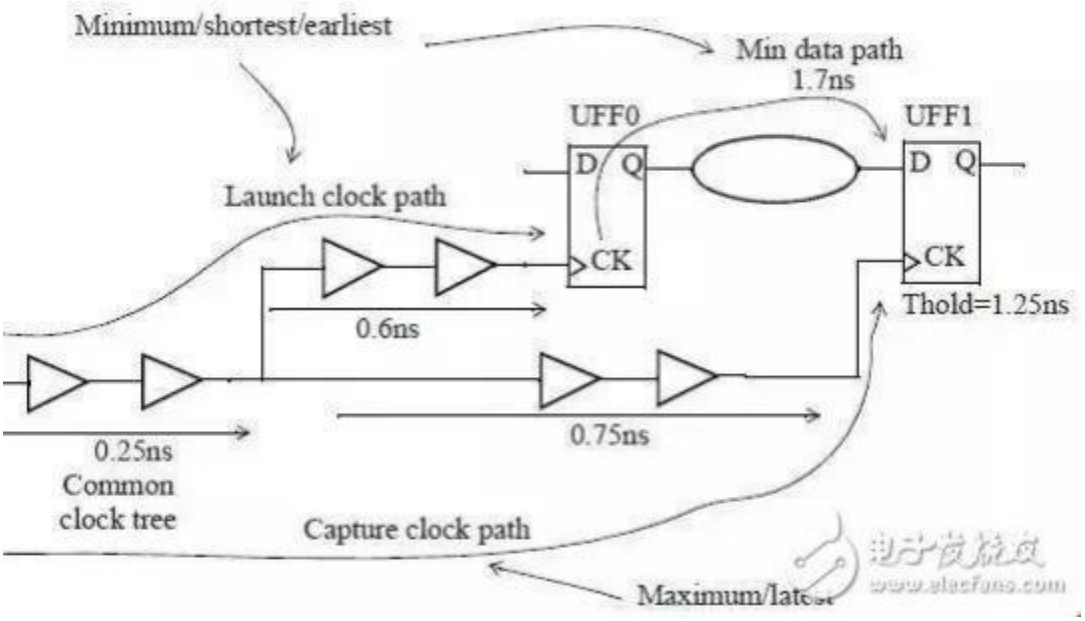
capture clock path = (1.2 + 0.86) *0.9 = 1.854

所以最小时钟周期 T = 2.0 + 5.2 -1.854 + 0.385= 5.731

（0.385=0.35*1.1,setup time:0.35 ？）

可以看到：考虑 timing derate 以后，会降低整个 design 的工作频率。

在 Hold check 中



考虑 time derate 的情况与 setup 正好相反，

Data require time 中的 capture clock path 使用-late 选项，使路径变慢。

Data arrival time 中的 data path 和 launch clock path 使用-early 选项，使路径加快

实际上，Hold check 一般在 BC 条件下，因此，launch clock path 与 data path 不需要再进一步减小 delay，因为已经是最小 delay，但是 BC 条件下的 capture clock path 需要 derate. 可以使用如下设置

set_timing_derate -early 1.0

set_timing_derate -late 1.2

这样添加 time derate 后

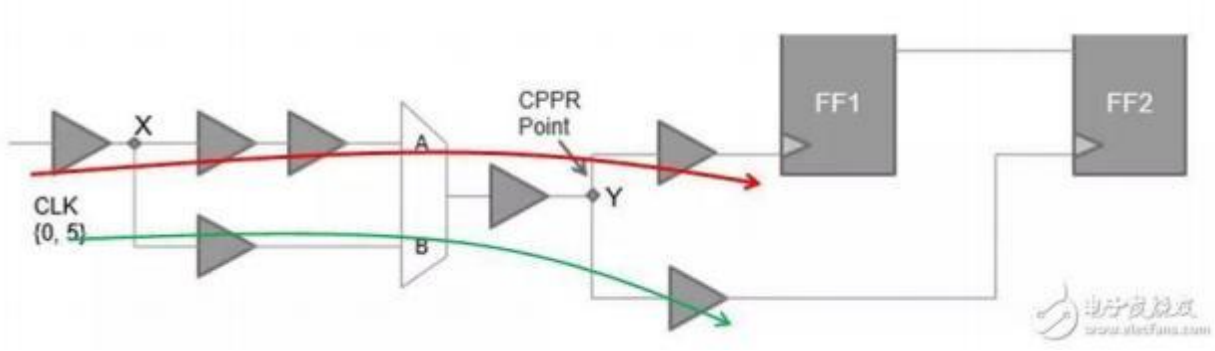
Launch clock path = 0.85 * 1.0 = 0.85

Min data path = 1.7 * 1.0 = 1.7

Capture clock path = 1.0 * 1.2 = 1.2

所以 slack=0.85+1.7-1.2-1.25=0.1

介绍的时序分析概念是 CPPR(CRPR)。全称 Clock Path Pessimism Removal(Clock Reconvergence Pessimism Removal)，中文名“共同路径悲观去除”。它的作用是去除 clock path 上的相同路径上的悲观计算量。如下图所示：



由于 STA 是穷举型的分析路径，在分析 setup timing 时，clock launch path 会选择红色的路径，clock capture path 则会选择绿色的路径，这本身就是不合理的情况，一个 mux 不可能同时存在两条经过的路径，所以我们需要去除这个计算的悲观量。

CPPR 不仅仅存在于 OCV，在以下几种情况它都可以存在：

Single Mode without set_timing_derate

BC-WC Mode without set_timing_derate

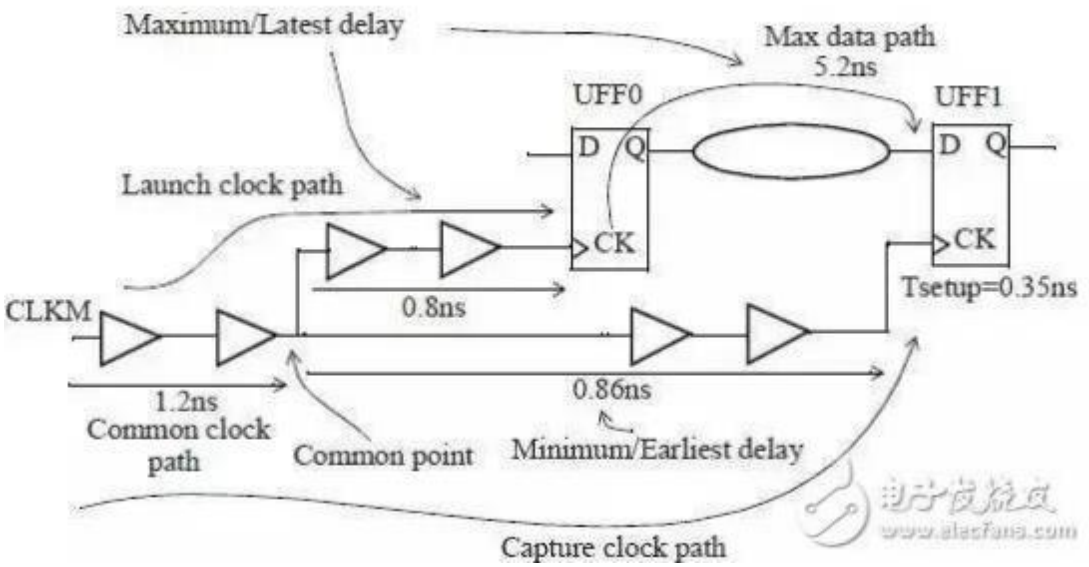
OCV Mode without set_timing_derate

Single Mode with set_timing_derate

BC-WC Mode with set_timing_derate

OCV mode with set_timing_derate

我们再看上一个计算 timing derate 的例子：



大家肯定发现了对于 path 最前端那 1.2ns 的延迟，在 setup 分析时，launch clock path 中没有被 derate, 而在 capture clock path 中被 time derate 1.2*0.9 =1.08. 显然这是相互矛盾的。

对于上诉设计，考虑 CPPR 之后，我们必须减去一个 CPP 因子 =1.2-1.08=0.12

所以最小时钟周期 T=5.731-0.12 = 5.611

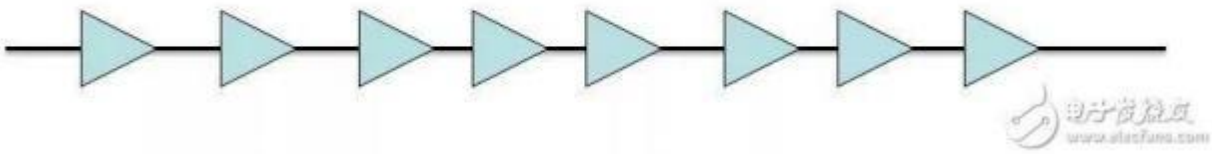
使用方法：

set_analysis_mode -cppr {none|both|setup |hold}

介绍的时序分析概念是 AOCV。全称 Stage Based Advanced OCV。我们知道，在 OCV 分析过程中，我们会给 data path,clock path 上设定单一的 timing derate 值 。随着工艺演变的加速，我们发现这种设置方法是过于悲观的，大家可以想象下，OCV 是片上误差，就代表一条 path 上

有的 cell delay 大于标准值，那也有的 cell delay 会小于标准值。因此不能一味的加大或减小 delay 来模拟片上误差。

如下图所示，对于下面这样一条 buffer 链，假设 8 个 buffer 处于不同的 PVT 条件下，OCV 会将 8 个 buffer 都选用最差的条件来分析（同一 derate 参数），而 AOCV 则会采用不同的 timing derate 值来分析。



AOCV 有它专门的 libary 库，我们称为 AOCV table。按照维度分为两种，一种是一位的只以 stage count 作为计算的表格，如下图所示：

version： 2.0

object_type： lib_cell

object_spec： LIB/BUF1X

rf_type： rise fall

delay_type： cell

derate_type： late

path_type： data

depth： 1 2 3 4 5

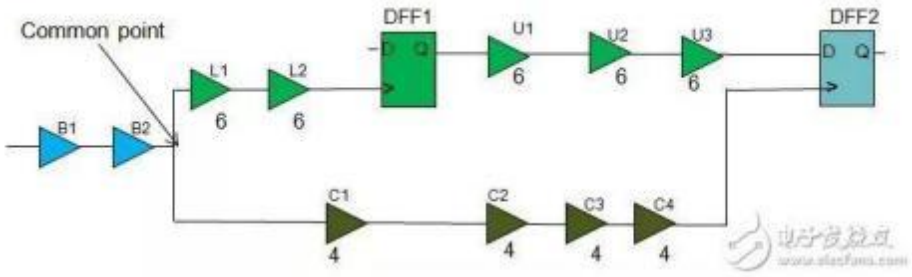
distance :

table : \

1.123 1.090 1.075 1.067 1.062

depth 就代表着 stage count，从表格中我们可以看到随着； 路径的深入，derate 的效应会减小 。那我们怎么来计算 stage count 呢？

通常这个计算方法比较复杂，不同的电路情况对应着不同的 count 计算方式：如下 timing path，我们将 L1，L2，DFF1，U1，U2，U3 的 stage count 设成 6，而 C1，C2，C3，C4 的 stage count 需要设成 4，这边需要说明的是 B1，B2 由于是 common point，所以在计算 stage count 时需要忽略。



另外一种是以 distance 和 stage count 混合组成的二维 AOCV table. 它在计算 derate 时同时考虑了 timing path 的距离因素，当然这个虽然更加精确，但是会增加 runtime，所以一般现在一维表格用的更多。

version : 2.0

object_type : lib_cell

```
object_spec : LIB/BUF1X

rf_type : rise fall

delay_type : cell

derate_type : late

path_type : data

depth : 1 2 3 4 5

distance : 500 1000 1500 2000

table : \

1.123 1.090 1.075 1.067 1.062 \

1.124 1.091 1.076 1.068 1.063 \

1.125 1.092 1.077 1.070 1.065 \

1.126 1.094 1.079 1.072 1.067
```

使用方法：

```
set_analysis_mode -aocv true
```

Pin	Delay	Arrival Time	Aocv Derate	Aocv Stage Count
u_ca53_noram/u_ca53biu/u_ca53biu_data_write_buffer s/wbuf_mbist_odd_bank_sel_rst_1_reg/CLK		289.999		
u_ca53_noram/u_ca53biu/u_ca53biu_data_write_buffer s/wbuf_mbist_odd_bank_sel_rst_1_reg/QN	64.415	354.414	1.031	9.000
u_ca53_noram/u_ca53biu/u_ca53biu_data_write_buffer s/FE_OFC63248_wbuf_mbist_odd_bank_sel_rst_1/Z	16.690	371.105	1.050	9.000
u_ca53_noram/u_ca53biu/u_ca53biu_data_write_buffer s/g62194/Z	26.039	397.144	1.042	10.000
u_ca53_noram/u_ca53biu/u_ca53biu_data_write_buffer s/FE_OFC115335_n_3192/Z	43.976	441.120	1.023	8.000
u_ca53_noram/u_ca53biu/u_ca53biu_data_write_buffer s/FE_OFC169212_n_5388/Z	57.418	498.538	1.039	8.000
u_ca53_noram/u_ca53biu/u_ca53biu_data_write_buffer s/FE_ESOC28471_FE_OFN227666_n_5388/Z	73.850	572.388	1.026	11.000
u_ca53_noram/u_ca53biu/u_ca53biu_data_write_buffer s/g62254/Z	14.334	586.722	1.031	11.000
u_ca53_noram/u_ca53biu/u_ca53biu_data_write_buffer	19.238	605.960	1.018	8.000

范例分析

注意:由于 pt 时序分析分 max,type,min,其使用的库不一样,所以在工作目录下需要配置不同的.synopsys_pt.setup 的文件(.synopsys_pt.setup 就是设置不同的库信息),如下面********部分就是.synopsys_pt.setup 配置内容

```
**********************************************************
set search_path [list . \
/xxx/projects/SSSSS/lib/flash/HHG_F011Q7P6_JPFLS016K29DHV00K/synopsys_model \
/xxx/projects/SSSSS/lib/memory/RASP1536X9 \
/xxx/projects/SSSSS/lib/memory/RASP256X9 \
/xxx/projects/SSSSS/lib/io \
/xxx/projects/lib/p110nm/hhgrace/Verisilicon/HG11LP/FEView_STDIO/STD_6T/Synopsys \
/xxx/eda/synopsys/pt_vJ-2014.06-SP1/libraries/syn \
]

set synthetic_library [list dw_foundation.sldb standard.sldb]

set link_library [concat * \
hg11lp_6T_ss_1p35v_125c.db \
JPFLS016K29DH_max.db \
RASP256X9_ss_1p35v_125c.db \
RASP1536X9_ss_1p35v_125c.db \
SP5VP15_PG_V1p0_max.db \
```

```
$synthetic_library]

set target_library [list hg11lp_6T_ss_1p35v_125c.db]

set verilogout_higher_designs_first true
set verilogout_no_negative_index true
set verilogout_no_tri true
set verilogout_single_bit false
set verilogout_show_unconnected_pins true
set bus_naming_style %s_%d
```

下面是 xxx.src 内容

```
source ./scr/set.scr     //设置变量

set MNAME projects_name

set CASE $DELAY_TYPE

set VER eco7


read_verilog ./results/netlist/${VER}/${MNAME}.HV     //读入网表


current_design $MNAME  //指定当前设计名称

link   //链接


set si_enable_analysis true    //使能 set_case_analysis 命令

set timing_remove_clock_reconvergence_pessimism true  //同一段时钟的误差去除 ,比如寄存器 A 和寄存器 B 的时钟,有一段路径相同一条,那么这部分误差则为 0

set pba_enable_xtalk_delay_ocv_pessimism_reduction true   //使能 OCV 分析

set timing_clock_reconvergence_pessimism same_transition

set_timing_derate -late 1

set_timing_derate -early 1.1
```

read_parasitics

-keep_capacitive_coupling ./results/netlist/${VER}/${MNAME}_post_layout_max.spef　　/读取RC参数用来估算路径延时

set_operating_conditions -analysis_type on_chip_variation ss_1p35v_125c　　/工作条件决定了cell的延时

set con_type post_layout_

source -echo ../syn/scr/${MNAME}_src.sdc　　//执行约束

check_timing　　*/执行时序分析*

source -echo scr/report_timing_func.scr　//时序报告输出脚本

##################analysis spef annotation###################

report_design >

results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.all_vio

report_annotated_parasitics >

results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.all_vio

report_annotated_parasitics -pin_to_pin_nets -list_not_annotated -max_nets 30 >>

results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.all_vio

##################analysis spef annotation###################

report_constraint -all_violators >>

results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.all_vio

report_constraint -all_violators -verbose >

results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.all_vio.ver

#save_session ./session/${MNAME}_func_${DELAY_TYPE}

```
#exit
#################analysis coverage check##################
report_analysis_coverage -significant_digits 4 -status_details
untested -check_type setup -exclude_untested \
 {constant_disabled user_disabled} -nosplit >
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.cross_cl
k

report_analysis_coverage -significant_digits 4 -exclude_untested \
 {constant_disabled user_disabled false_paths} -nosplit >>
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.all_vio
report_analysis_coverage -significant_digits 4 -status_details
untested -check_type setup -exclude_untested \
 {constant_disabled user_disabled false_paths} -nosplit >>
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.all_vio
report_analysis_coverage -significant_digits 4 -status_details
untested -check_type hold -exclude_untested \
 {constant_disabled user_disabled false_paths} -nosplit >>
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.all_vio

#################analysis coverage check##################
##################report clock skew#################----------

if [file exists
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.clk_skew]
 {
file delete
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.clk_skew
}
```

```
set CLK_LIST [get_clocks "bs_clk clk_osc48m clk_osc800k clk_crystal
clk_adc clk_timer0 clk_timer1 clk_timer2 clk_timer3 clk_mcu_div
clk_load_nvr clk_prog clk_spi clk_uart clk_debug clk_led clk_test"]

    report_clock_timing -type skew -clock $CLK_LIST -nworst 5 >>
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.clk_skew

#####################################################################
if [file exists
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.clk_skew.
ver] {
file delete
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.clk_skew.
ver
}

report_clock_timing -type skew -clock $CLK_LIST -nworst 5 >>
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.clk_skew.
ver

########################clock gate report#########################
if [file exists
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.clk_gate]
 {
file delete
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.clk_gate
}
report_constraint -clock_gating_setup -verbose >>
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.clk_gate
```

```
report_constraint -clock_gating_hold -verbose >>
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.clk_gate

report_timing -path full_clock_expanded -input -cap -nets
-transition -nworst 20 -nosplit -delay max -through [get_pins
-hierarchical latch/E] >>
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.clk_gate
report_timing -path full_clock_expanded -input -cap -nets
-transition -nworst 20 -nosplit -delay min -through [get_pins
-hierarchical latch/E] >>
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.clk_gate
```

命令介绍

report_clock 命令检查已经定义的时钟，如 report_clock -attr -skew 将会报告出所有的定义时钟信息，看与自己的约束是否一致

report_port 命令检查所有端口上所加的约束 如 report_port -verbose

报告 setup ： report_timing -delay max -from[] -to []

报告 hold ： report_timing -delay min -from[] -to []

```
NAME
       set_timing_derate
              Sets derating factors on the current design
or specified
              objects.  Derating factors specify the upper and lower
limits on
              delays for a particular operating condition.

SYNTAX
       status set_timing_derate
```

[-rise]

[-fall]

[-min]

[-max]

[-early]

[-late]

[-clock]

[-data]

[-net_delay]

[-cell_delay]

[-cell_check]

value

object_list

Data Types

value	float
object_list	list

DESCRIPTION

The set_timing_derate command sets derating factors for the current design or specified objects. If timing derating factors are specified, some or all path delays are multiplied by derating factors. Derating

factors define the lower and upper ranges of timing for a certain operating condition. The derating factor specified with the -late option multiplies data paths delays for maximum delay (setup) check and clock

paths delays for minimum delay (hold) check. The derating factor specified with the -early option multiplies data paths for hold checks and clock paths for setup checks.

The -net_delay, -cell_delay, and -cell_check options apply the specified derating value to net delays, cell delays, or cell setup and hold time requirements, respectively. The -net_delay option can apply to the

whole design only, so it cannot be used with an object list.

If none of these three options is used and an object list is provided,the derating value applies to cell delay only; if no object list is provided, the value applies to both cell delay and net delay.

The setup timing check is calculated using the maximum operating condition. The hold timing check is calculated using the minimum operatingcondition.

Input delay and ideal clock network latency are not derated.

常用命令

- **List all violations, sorted by clock domain**

1. Syntax: report_constraint <options>

2. -max_delay or -min_delay option: check setup or hold violation

3. -all_violators -verbose option: report all violation in detailed

- **Report all timing information**

1. Syntax: report_timing <options>

2. -delay_type: max, min, max_min, max_rise, min_rise …

3. -from/to/through/exclude, -max_path, -nworst, -path, -path_type …

4. -net, -cap, -input_pins, -slack_lesser_than, -slack_greater_than …

- Design Tip

1. report_constraint -all_violators：报出所有的 violations

2. report_constraint -all_violators -verbose : use long run time

3. report_timing -net -cap -input_pins -delay_type -from/to

PT 产生 sdf 文件

```
source -echo scr/set.scr
set MNAME prjxxxx0
set TYPE ${TYPE}

read_verilog
/xxx/projects/PRJXXXX0/release/backend/output/prjxxxx0.HV

current_design $MNAME
link

if { ${TIMING_MODEL} eq "ccs" } {
set rc_driver_model_mode     advanced
set rc_receiver_model_mode   advanced
set rc_noise_model_mode      advanced
#set power_model_preference   ccs
}

set si_enable_analysis true
set timing_remove_clock_reconvergence_pessimism    true
set pba_enable_xtalk_delay_ocv_pessimism_reduction     true
```

```
set timing_clock_reconvergence_pessimism     same_transition

if { ${TYPE} eq "minmax" } {
read_parasitics
-keep_capacitive_coupling  ./output/${MNAME}_post_layout_max.spef
} else {
read_parasitics -keep_capacitive_coupling
/output/${MNAME}_post_layout_${TYPE}.spef
}###spef 文件是 pr 后端通过 StartRC 抽取的文件

if { ${TYPE} eq "max" } {
set_operating_conditions -analysis_type  on_chip_variation
-library   \
        scc011ums_uhd_hvt_ss_v1p08_125c_basic  ss_v1p08_125c
或者如下:
read_parasitics ./${MNAME}_post_layout_cmax.spef
 set_operating_conditions -analysis_type single SS_1P35V_125C
////////////////
set_timing_derate -late 1
set_timing_derate -early 0.95
}
##    sc7_logic011_base_hvt_ss_typical_max_1p08v_125c
ss_typical_max_1p08v_125c

if { ${TYPE} eq "min" } {
set_operating_conditions -analysis_type on_chip_variation -library
\
        scc011ums_uhd_hvt_ff_v1p32_-40c_basic   ff_v1p32_-40c
set_timing_derate -late 1.05
```

```
set_timing_derate -early 1
}
##    sc7_logic011_base_hvt_ff_typical_min_1p32v_m40c
ff_typical_min_1p32v_m40c

if { ${TYPE} eq "typ" } {
set_operating_conditions -analysis_type on_chip_variation -library
\
        scc011ums_uhd_hvt_tt_v1p2_25c_basic  tt_v1p2_25c
set_timing_derate -late 1.025
set_timing_derate -early 0.975
}
##        sc7_logic011_base_hvt_tt_typical_max_1p20v_25c
tt_typical_max_1p20v_25c

if { ${TYPE} eq "minmax" } {
set_operating_conditions -analysis_type on_chip_variation -library
\
        scc011ums_uhd_hvt_ff_v1p32_-40c_basic   ff_v1p32_-40c
set_timing_derate -late 1.05
set_timing_derate -early 1
}
##    sc7_logic011_base_hvt_ff_typical_min_1p32v_m40c
ff_typical_min_1p32v_m40c

if { ${TYPE} eq "min0" } {
set_operating_conditions -analysis_type on_chip_variation -library
\
        scc011ums_uhd_hvt_ff_v1p32_0c_basic ff_v1p32_0c
```

```
set_timing_derate -late 1.05
set_timing_derate -early 1
}
##     sc7_logic011_base_hvt_ff_typical_min_1p32v_0c
ff_typical_min_1p32v_0c

set con_type post_layout
source -echo ../syn/scr/${MNAME}_src.sdc

foreach_in_collection des [find cell -h "*gray_*_cnt_d1_reg_* \
                                         *wa_gray_p0_reg_* \
                                         *ra_gray_p0_reg_* \
                                          *signal_delay1_reg \
                                          *dqs_d_reg_0_ \
                                          *d1_reg \
                                          *clk40m_cnt_reg_* \

*signal_pulse_delay1*_reg"] {
    set_annotated_check -setup 0 -from [find pin [get_object_name
$des]/CK*] \
    -to [find pin [get_object_name $des]/D]
    set_annotated_check -hold 0 -from [find pin [get_object_name
$des]/CK*] \
    -to [find pin [get_object_name $des]/D]
}


write_sdf -version 3.0 -context verilog
results/netlist/${MNAME}_post_layout_${TYPE}.sdf
```

exit

查看 PT 关键结果

1.PT 主要是关心时序方面的结果,

1.1 通过 report_constraint 命令来得到时序结果

```
report_constraint  -all_violators \
-verbose >
results/report/${MNAME}_post_layout_${CASE}_${CASE_MODE}.all_vio.
ver
```

1.2 需要关心各种 case 下的时序结果

Pt 分析步骤

1. 设置各种环境
2. 吃网表文件
3. 设置顶层 module
4. link
5. 吃 sdc 文件 - source …sdc
6. 根据 sdc 文件检查是否与实际时序相符—时序分析 check_timing
7. 产生所有的时序报告---source …report_time.src

检查时钟

Check_timing

时序报告

Report_timing

报告寄存器之间的延时

```
pt_shell> report_timing -from ff1/CP -to ff3/D -exceptions all
                ****************************************
                Report : timing
                        -path_type full
                        -delay max
                        -max_paths 1
                  ...
              ****************************************
```

使用 man report_timing 在 pt_shell 环境下有详细说明

PT/ptpx（转）--已用

PTPX Power Analysis Flow

功耗分析可以与 sta 在一起分析，只需要加入下面步骤即可：

PrimeTime PX 工具是 PrimeTime 工具内的一个 feature。

PTPX 的功耗分析，可以报告出 chip，block，cell 的各个 level 的功耗。

使用 PTPX 可以分析的功耗的方式：

1)Average power analysis，支持 activity 的 propagation 方式，主要用在项目早期做评估。

可以是 defaults，user_defined，derived from HDL simulation 的 switching 文件。

2)Time-based power analysis，通过 event-drived 算法来统计功耗。

主要用在项目 signoff 时候，加入 IR-drop 的分析。

电路中的功耗：

1)Leakage power，电路处在 inactive 或者 static 时的功耗。

Intrinsic Leakage power，主要是 source-to-drain subthreshold leakage。

diffusion layers 和 substrate 之间的 current leak。

这种 leakage 是 state 和 voltage dependent。

Gate Leakage Power，leakage 的主要来源，有 source to gate 和 gate to drain 的 power。

主要由 gate oxide thickness 和 voltage 来决定。

2)Dynamic Power，电路处在 active 模式下的功耗。即使在 output 不改变的情况下，输入也可能由于 transition 带来功耗。

Internal Power，包括内部电路冲放电的功耗和 short circuit 的功耗。对于 fast transition times，short-circuit 比较低。

对于 slow transition times，short-circuit 比较高。

Switching Power，cell 的 output 的电容冲放电。

set_power_derate：用来设置某个 design，cell，library cell，hierarchical，leaf cell 的 power 的百分比。

report_power_derate

PTPX 可以将某个 design 的 power data 放在一个 power model 中，通过extract_model -power 来得到这个 model，

可以通过在设计中例化的方式，来加快 chip_level 的 power analysis 的进度。(生成格式为.lib 文件)

针对 gate_level 的 design，PTPX 会生成一个带 clock pin 的 power model，可以用在分析一个复杂 macro cell 的功耗。

Power Analysis input：

logic library，包含 timing 和 power 信息，支持 NLPM 和 CCS 类型的 library。

Gate_level netlist，支持 verilog，VHDL，systemverilog 格式。

Design constraints，计算 primary input 的 transition time 和 define clock。

Activity，VCD/ SAIF，default，user_ defined。

net parasitic，SPEF 文件，包含 net RC 参数。

1)set power_enable_analysis 为 true。

2)读入 verilog，VHDL，db，ddc 等格式的 netlist，logic library 为 db 格式。

3)读入 SPEF 文件，包含 WLD 定义，以及 sdf 文件，包含 glitch。

4)set_operating_conditions 设置 PVT。

5)针对某些 fanout 比较高的 net(clock reset)，设置 power_limit_extrapolation_range。

6)当 timing data 还没有 update 的时候，PTPX 会自动首先进行 timing analysis，也可以通过 update_timing 来显式调用。

7)check_power 检查潜在的功耗计算问题，默认有 out_of_table_range 和 missing_table 检查。

8)设置 power analysis 的模式，set_app_var power_analysis_mode average | time_based。

9)指定 switching activity data，主要有 set_switching_activity，set_case_analysis。

read_vcd -time(加入 time window)/-when(Boolean condition)。

如果 vcd 等波形从 RTL 中得到，还需要一个 mapping 文件来进行波形与 netlist 之间匹配。

10)update power 开始进行 power analysis 的分析。

11)report power 打出 power report。

time_base 的报告比 average 报告多出以下几项。

```
-----------------------------------------------
Total Power              = 3.670e-03   (100.00%)
-----------------------------------------------
X Transition Power       = 2.171e-07
Glitching Power           = 3.018e-05

Peak Power                =     0.0968
Peak Time                 =    1956.00
```

12)最后通过 save_session 和 restore_session 来保存以及提取 session。

可以在 sta 脚本中加入下列命令

```
set power_enable_analysis true
#set_app_var power_analysis_mode average | time_based
set_app_var power_analysis_mode average
read_saif  power_time.saif -strip_path xxx_top/u_xxx
# power_time.saif 根据 uinx 命令 vcd2saif  -i xxx  -o xxx 命令获得
check_power
update_power
report_power
```

需要注意通过 vcd2saif 与 pt_shell 版本兼容问题

PTPX 中的 activity 文件以及 mapping 文件

在不同的 simulation 中的 capturing switching activity：

Simulation	Captured	Not captured	Trade-offs
RTL	Synthesis-invariant elements	1. Internal nodes 2. Correlation of non synthesis-invariant elements 3. Glitching 4. State and path dependencies	Fast runtime at expense of some accuracy
Zero-delay and unit-delay gate-level	1. Synthesis-invariant elements 2. Internal nodes 3. Correlation 4. State dependencies 5. Some path dependencies	1. Some path dependencies 2. Glitching	More accurate than RTL simulation, but significantly higher runtime
Full-timing gate-level	1. All elements of design 2. Correlation 3. State and path dependencies	Highest accuracy, but runtime can be very long	Correlation between primary inputs

SAIF：Switching Activity Interface Format，包含 toggle counts 和 static probabilities。

RTL 中的 SAIF 文件包含 primary input，hierarchical port，sysnthesis-invariant 单元如 sequential elements。

block box cells，tristate cell，latches，retention registers，clock-gating cell 等。

不包含 integrated clock-gating cells 和 latch-based isolation cells 等。

VCD：Verilog Change Dump format，是一种 event-based format，包含每个 value change 以及他们的time。

read_vcd -rtl 表示输入的是 rtl 的 activity 文件。进行 average 分析。--需要 map 文件

如何使用的是 netlist 仿真得到的 vcd 就不需要 map_file

-zero_delay 表示输入的是 netlist 文件，但是并没有 sdf 文件的。进行 cycle_accurate 分析。

什么都不加时，表示进行基于 event 的分析。

在进行synthesis 时，可能会使得一些 rtl synthesis invariants 的模块(register outputs，primary inputs，tristate，black boxes)

等的名字被改变，所以需要一个 mapping 文件来进行 match RTL activity 到 gate_level component。

PTPX 提供了三种方式：

1)set_rtl_to_get_name 命令。

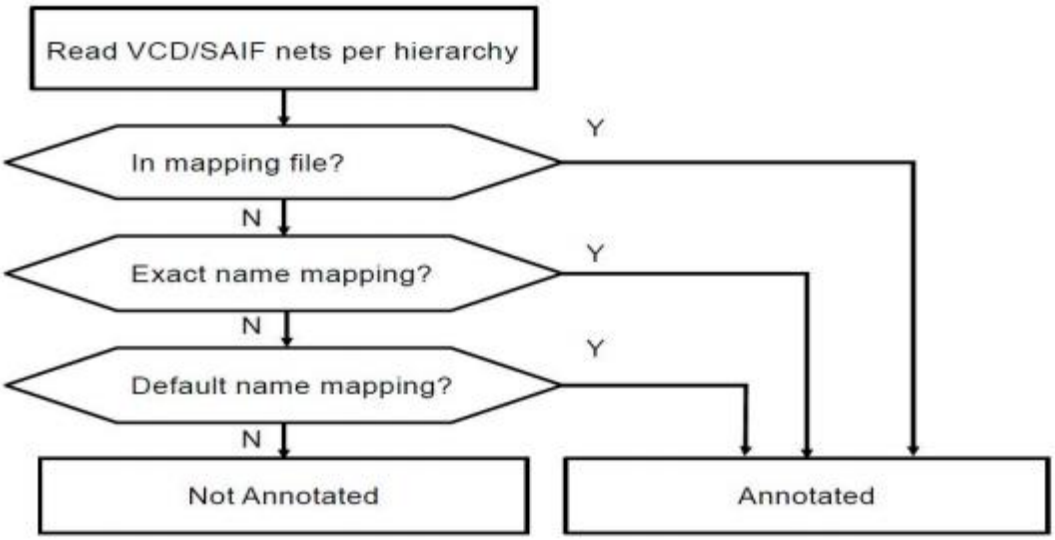
2)exact name mapping

3)default name mapping。

在综合的过程中，如果使用 DC 工具，可以生成一个 map-file，包含很多 set_rtl_to_gate_name 命令来保证 netlist 与 rtl 的一致。

如果没有这个 map-file 就需要人为的设置很多这样的命令，如果工具仍然发现一个 mismatch，那会使用 PTPX 内建的机制，来进行 map。

经过这些操作，仍然 mismatch，PTPX 就不会在 RTL activity file 上反标这个 object。



set_rtl_to_gate_name -rtl{a} -gate{a_reg}

当 map 一个 RTL 的寄存器时，工具实现 Q pin 和 QB pin 的标注。

PTPX 支持one-to-many 的 mapping，如在插入 Clock Tree Synthesis Buffer 时，一个 RTL clock 可以 map 所有的 buffer。

但是不支持多个 RTL object map 到一个 netlist 的 object 上。

Exact name mapping 是区分大小写的，一般情况下，要将 cell 和 pin 的 exact name map 打开，关闭 net 的 exact name map，

因为同一个 net 名字，可能已经经过了 invert。

命令 power_disable_exact_name_match_to_net。

default mapping 方式，

1)将 RTL hierarchy map 到 flatten gate_level object 上，用“_”代替“/”。

2)RTL 的寄存器 map 到 gate_level 的 object 上，用_reg 后缀。

3)bus 上的"[]"号，map 为"_"，a[7]map 为 a_reg_7_。

Name Mapping report：

report_name_mapping，生成一个 RTL 与 netlist 之间的通过 set_rtl_to_gate_name 命令设置的 map 报告。

report_activity_file_check，列出 activity file 中所有的 RTL name。

得到 map 文件方式：在综合时需要加入如下 line6,line15,lne16,line17 语句

```
1 set search_path {. ../src/hdl/rtl ../src/lib/snps ../src/sim}
2 set link_library {* core_typ.db}
3 set target_library {core_typ.db}
4 read_verilog {mac.v}
5 current_design mac
6 saif_map -start
7 link
8 create_clock [get_ports clk]  -period 12  -waveform {6 12}
9 #
10
11 compile
12 write_sdc ./mac.sdc
13 write -hierarchy -format verilog -output ./mac.vg
14 write_parasitics -output ./mac.spf
15 sh vcd2saif -input ../sim/rtlvcd.dump -output ./rtlvcd.saif
16 saif_map -create_map -source_instance tb/macinst -input ./rtlvcd.saif
17 saif_map -write_map ./mac_ptpxmap.tcl -type ptpx
18 exit
```

PTPX 的 average power analysis

在 average power analysis 中，switching activity 被分解为 toggle rate 和 static probabilities 两部分。

annotation 的 source：

Table 5-2 Switching Activity Annotation Sources Using the activity_source Attribute

Attribute value	Annotation source
`file`	Toggle rate derived from a SAIF or VCD file (`read_vcd`, `read_saif` commands)
`set_switching_activity` (Equivalent priority)	Toggle rate derived using the `set_switching_activity` command
`set_case_analysis`	Toggle rate derived using the `set_case_analysis` command
`create_clock`	Toggle rate derived using the `create_clock` command
`implied`	Implied activity without random vector propagation from the annotated design points Q/QB, Buffers/Inverters, Multidriven resolution, Netlist constraints
`default`	Toggle rate value is the tool default
`propagated`	Toggle rate is propagated using random vectors from the annotated design points (zero-delay simulation)
`UNINITIALIZED`	Not user-annotated, implied, or default. The uninitialized nets or pins annotation is derived from cycle simulation, and the `activity_source` attribute value is updated to `propagated` after power analysis

Switching Activity Annotation：(优先级从高到低)

1) 来自 VCD 和 SAIF 这样的波形；

2) 命令 set_switching_activity annotation 某个 individual design；

3) Annotation constant value on set/reset pins，infer_switching_activity –apply 命令；

4) Annotation constant value，用命令 set_case_analysis 命令；

5) Clock net annotation，用命令 create_clock 命令；

set_switching_activity 命令：

可以加入 option，-toggle_rate 和-static_probability。-period 和-base_clock 来指定time unit。

Infer_switching_activity 命令：

在没有提供VCD 和 SAIF 文件时，工具通过 primary input 和 black boxes 的 output 来进行 power 分析，

并根据 default switching activity 来评估 nonannotated pin。

通过 infer_switching_activity 命令来防止 reg 的 reset 和 set 引脚，进行 default 处理。

Estimating Nonannotated Switching activity

1) 对于不能通过 propagation 进行 annotated 的信号，进行 default activity 处理。

2) 通过 buffer 和 inverter 的信号，以及 FF 的 Q/QN 端，进行 implied 处理。

Imply 的方向包括 forward 和 backward，为避免冲突，forward 方向的 imply 优先级较高。

3) Propagating 机制，PTPX 通过一个 zero_delay 的 simulation 来 annotate 一些 net 的 toggle rate 和 static probability。

SAIF 文件的 annotation：

merge_saif 命令，工具默认各个 SAIF 文件分别读入并处理 switching activity，然后乘以各自的 weight。

加入-simple_merge 命令，首先 weight 各自的 activity 文件，后再 propagating activity，并输出到一个 SAIF 文件中。

write_saif 命令，将包含 user-annotated 和 propagated 的 switching activity，输出到一个 saif 文件。

report_switching_activity 命令，将各个 group 的 annotation 的情况报告出来。

PTPX 中的 time_based analysis

根据VCD 文件的type，PTPX 支持 instantaneous peak power analysis 和 cycle_accurate peak power analysis。

Time-Based Power Analysis 支持的 VCD type

VCD types	Name mapping	Event propagation	Enforce cycle accurate peak power
Gate-level VCD using the `read_vcd` command without the `-zero_delay` and `-rtl` options	Off	Off	No
Gate-level zero-delay VCD using the `read_vcd -zero_delay` command	Off	Off	Yes
RTL VCD using the `read_vcd -rtl` command	On	On	Yes

由于 Gate_level 和 zero_delay 的 VCD 不支持 event propagation，所以可以在 read_vcd 之后，通

过命令 report_switching_activity 来报告 switching activity 的 annotation。

在 time_base mode 下，不支持多个 VCD file 的分析。

PTPX 在 testbench 的 level 建立 activity file，在读入 vcd 时，通过 strip_path 的命令来 map 到 design module。

read_vcd –strip_path TB/U1

PTPX 进行 condition power analysis，加入-time 和-when 的 option。

如果vcd 的产生由带systemverilog 的语法的TB 来产生，那么 read_vcd –format systemverilog 命令来读入 vcd。

进行基于 rtl 的 time_based 的分析，命令 read_vcd –rtl 来设置，可以进行 name_mapping 和 event 的 propagate。

进行基于 gate_level 的 peak 分析，必须加入 sdf 文件。

由于 netlist 的 VCD 文件都会比较大，所以 PTPX 工具内部支持 fsdb/gzipped VCD 的解压，

并且可以直接在 PTPX 工具内 invoke simulator。

-pipe_exec "vcs –R –f arguments –l log"

Vector analysis：

用来分析 high activity window。Time_based mode 中 high activity 可能落在不同的 interval，

所以可以通过-peak_window 来设置，最小和最大的 window 来分析 high activity。

write_activity_waveforms –vcd myvcd.vcd –output myvcd.out –interval 0.2 –peak_window 5

生成的 myvcd.out 一般都是 fsdb 格式的，直接通过 nWave 打开。

默认情况下，PTPX 不会 restore state_dependent 和 path_dependent 的 toggle rate，

为了在生成 SAIF 等文件中，能被包含进去，需要设置 write_saif –sdpd_tracking。

set_case_analysis 命令，将某个 port 设置为一个 constant value，优先级低于 VCD 的 annotation，

所以一般来设置 vcd file 中没有的 signal，如 test_enable 信号。

在 time_based 的 analysis 中，

Glitch power 的分析，作为一个 transition 分析，有自己的 ratio 计算方式。

Z State，只有 0-Z-1，1-Z-0 被认为 transition，有 power consume，其他的变化都不做分析。

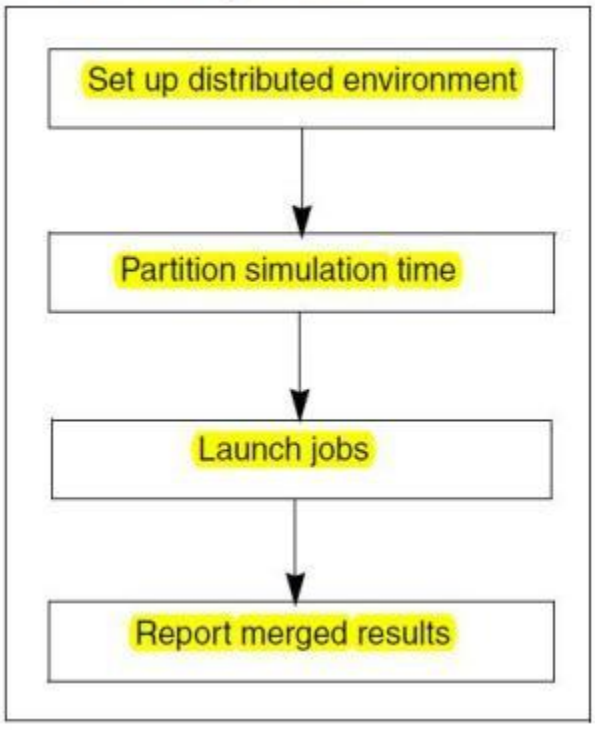
X State，0-X 作为半个 transition，0-x-0 也作为一个 transition。默认情况下，initial 的 X 不做分析。

由于 peak analysis 比较耗时，PTPX 采用 distribute 的方式来进行 multi-scenario analysis。

一个 scenario 是一个 VCD activity file 的 time window，可以并行在 multiple processor 上来运行。

然后将结果 merge 到一起。

Figure 6-7 Distributed Peak Power Analysis Flow



环境设置 flow：

```
set multi_scenario_working_directory ./work
set multi_scenario_merged_error_log ./work/error_log.txt

set_multi_scenario_license_limit -feature PrimeTime 2
set_multi_scenario_license_limit -feature PrimeTime-PX 2

 set_host_options -32bit -num_processes 2 \
          -submit_command /lsf/bin/bsub

 start_hosts


create_scenario -name run_0 -specific_data run_0.tcl \
-common_data common.tcl -common_variables {test_dir}

create_scenario -name run_1 -specific_data run_1.tcl \
-common_data common.tcl    -common_variables {test_dir}

current_session -all

remote_execute { \
   report_power
}
```

PTPX 中的 clock tree 与 LP design

PTPX 在加入 CPF/UPF 这样的文件后，可以分析 multi-voltage，power-gating 这样的设计。

针对某个 power rail 的 cell，PTPX 支持进行 annotate。

set_annotated_power –internal_power 0.1 –leakage_power 0.0 –rail VDDA U1

可以通过 report_annotated_power –rail 的命令来报告出来。

在设置 power_enable_multi_rail_analysis 之后，可以通过 report_power –rails 的命令来报告

某一个 power rail 的 power 消耗。否则可以通过命令
current_power_rail/update_power/report_power 来报告。

每一个 power rail 也有自己的几个 attribute，internal_power，switching_power，
leakage_power，

dynamic_power，total_power。

PTPX 对 clock network 的支持：

在 clock network 还没有 insert 时，进行 estimate；

在 clock network 已经 insert 之后，提供 power analysis；

1) Report：

报告 clock network 部分，report_power –groups clock_network

报告 register 部分，report_power –groups register

Clock network 的 power 总共就这两部分。

2) Annotating:

将 transistor-level 的 simulator 输出的 power 信息，通过 set_annotated_clock_network_power

设置。可以细分为：-internal_power，-switching_power，-leakage_power，-total_power 等 option 来进行标注。

3) Estimating

通过命令 estimate_clock_network_power，PTPX 在内部会建立一个 virtual balanced clock network，

来评估 clock network 的 power。

只支持 average 的 power analysis mode。

PTPX 中的 report 选项

Report 的生成

report_power 表示产生 power report，update_power 表示进行 power analysis。

report_power 命令可以生成四种形式的 report，

```
report_power [-verbose]
      [-cell_power][-net_power]
      [-hierarchy] [-levels level_value]
      [-power_greater_than threshold] [-leaf]
      [-include_boundary_nets]
      [-nworst_number] [-sort_by sort_mode]
      [-clocks clock_list] [-nosplit]
      [-groups group_list]
      [-include_estimated_clock_network]
      [object_list]
```

1) Power group-based，default 的方式。

2) Cell-based，加入-cell_power 的 option

还可以加入-sort_by 的 option 来根据 cell_internal_power(default)，cell_leakage_power，dynamic_power 来生成 report。

3) Net-based，加入-net_power 的 option

还可以加入-sort_by 的 option 来根据 net_static_probability，net_switching_power(default)和 net_toggle_rate 和 total_net_load。

4) Hierarchy_base，加入-hierarchy 的 option

一般只有 hierarchy 的底下一层的 power 被报告出来，可以通过-levels 来进行修改。

还有-group，-rail，-clocks 这样的 option 来进行 filter 进行 report。

在使用-include_estimated_clock_network 之前，必须进行 estimate_clock_network_power 的命令。

PTPX 中预设的 7 中 power groups：

1) Io_pad，包括 I/O pad cells。

2) memory，包括 Memory cell。

3) black_box，除了 io 和 memory 之外没有 functionality 的 cell。

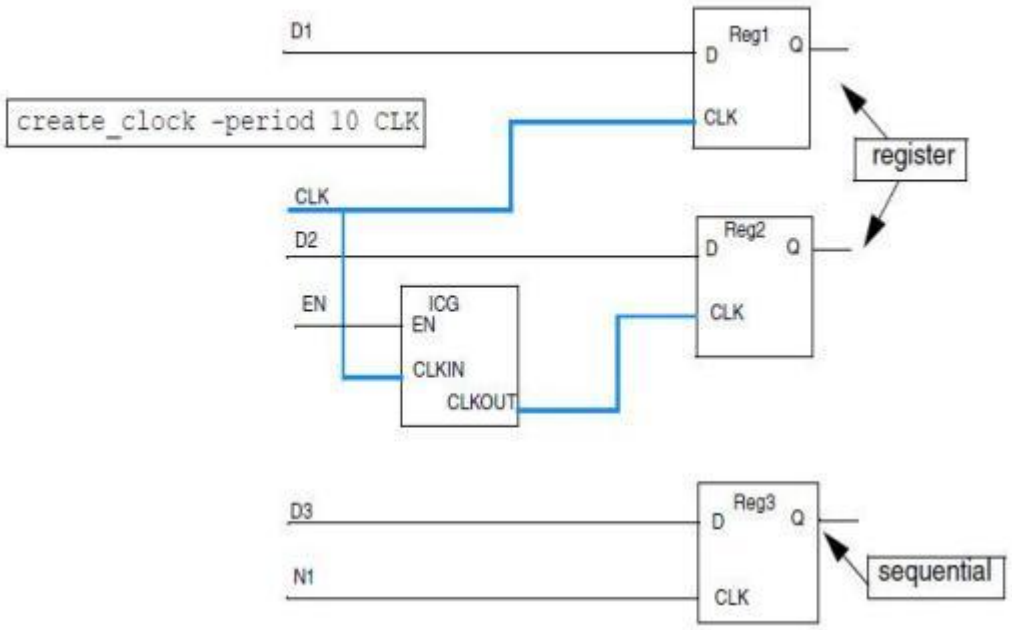
4) clock_network，包括使用 create_clock 的命令定义的 clock 和 clock network 上的 buffer，inverter，clock-gating logic。

5) register，Latches 和 FF，但是不包括由 clock network 驱动的 register 的 clock pin(包含在 clock network group 中)。

6) sequential，Latch 和 FF，但是 clock pin 不连接到 clock network。

7) combinational，剩下的 cell。

Figure 9-1 Difference between Registers and Sequential Power Groups



一般情况下，一个 instance 只会定义在一个 power group，所以这几个 power group 的总和就是这个 design 的 power。

如果 lib 使用 CCS 的 power model，那么可以分别报告出 intrinsic 和 gate leakage 的 power，这时需要设置 power_report_leakage_breakdowns 变量。

通过报告分析 clock_gating 的 efficiency。

report_clock_gate_saving 来报告某个 clock gate 的情况。

1) Toggle Saving，关掉 gate 之后，output 的 clock toggle rate 的减少量。
通过加入-by_clock_gate 的命令来报告每个 clock gate 的 toggle saving。

2) Register number，clock 的 fanout register。

通过加入-sequential 的命令来报告每个 clock gate 的 reg 情况。

越高的 clock freq，越多的 fanout reg，那么该 clock gate 的效率越高。

ptpx 流程

```
set link_library xxxx
read_verilog  xxxxx.v
read_parasitics  xxxx.spef
set power_enable_analysis true
set power_analysis_mode averaged
read_saif  ./xxx_post.saif -strip_path cbm7616
#set_app_var power_analysis_mode time_based
#read_vcd    pd_power.vcd -strip_path xxx_top/u_xxx
check_power
update_power
report_power  --hierarchy  > xxx.power.report
```

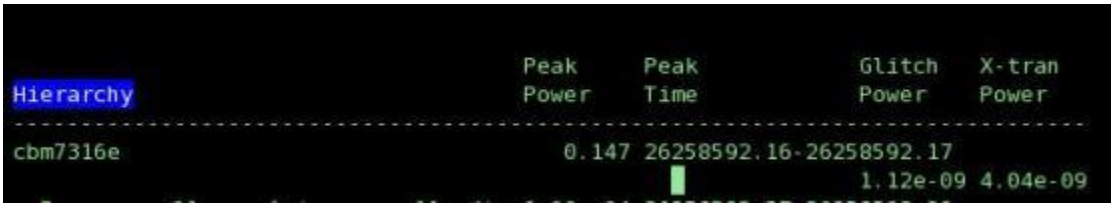
所得到的功耗分析结果在 xxx.power.report 中

上述命令可以在 PT 环境中通过man xxx 命令得到使用方法

如果提供的不是 saif 文件,而是 VCD 文件,则通过 vcd2saif -i vcd_file -o saif_file 转换

如果是 fsdb 波形,则先通过 fsdb2vcd,再用vcd2saif 转化成 saif 文件进行功耗分析

使用 set_app_var power_analysis_mode time_based 模式分析，报告会多一个峰值功耗项，此时必须直接读 vcd 文件；报告如下



警告原因分析

Information: The power model is different from the delay model in the library. (PLIB-161)

工艺库模型类别：

NLDM: 最基本简单的 .lib 非线性线载模型 电压源模型，cap 值是单一值， 90nm 工艺以下，由于晶体管的特性变得很复杂，这种模型不足以表现真实管子行为，不推荐使用这种 lib 

CCSM ： noise，power，timing 于一体的 lib， Synopsys 的复合电流源模型 与 NLDM 不同，它是电流源模型，表现为 lib 中有 Ivtable。

;ECSM ： noise，power，timing 于一体的 lib， Cadence 的有效电流源模型 如果库中的模型不是上述的哪种集成多个功能 模型中既有功耗的信息 也有时序的信息 而是把他拆分了描述 不会有这种警告

---不影响仿真

未解决问题

在仿真过程中发现,leak 功耗严重不合理,总功耗小于单个 cell 的功耗,check_power 结果看,读取的 lib 文件是正确,但有报 out of ramp range 警告,通过 sta 分析,有 transis 和 cap 违背.除非报告的功耗单位是 w 而不是mw!

Tmax 篇

命令 tmax -shell xxx.src 脚本

1. Tmax 流程

- ATPG Design Flow Overview
- Preparing a Netlist
- Reading a Netlist
- Reading Library Models
- Setting Up and Building the ATPG Model
- Performing Test Design Rule Checking (DRC)
- Preparing for ATPG
- Running ATPG
- Analyzing ATPG Output
- Reviewing Test Coverage
- Writing ATPG Patterns

2. TetraMAX 使用注意事项

1.在*.spf 文件中的扫描状态的初始化过程，即是使系统进入到扫描测试状态的 JTAG 信号输入流程必须放在“test_setup”过程中，否则生成的测试激励在测试中可能会产生无法匹配错误，从而导致扫描过程出错。

2.TetraMAX 中的 GSV 分析工具非常有用。使用GSV 可以查看激励运行的各种信息，包括扫描链信息、扫描激励串行移位时 load_unload、shift、capture 各过程的状态信息值。

3.对生成的测试激励的评价主要是故障覆盖率指标。如果故障覆盖率很低的话，可以通过增加测试激励数、去掉不需要测试的电路模块等方法增加覆盖率。

4. 在 Tmax 安装目录下有一个/setup/tmax.rc 文件，里面有一些配置信息

3. TetraMAX 介绍

TetraMAX 是一个高速、高性能的自动测试激励产生工具（ATPG automatic test patterngeneration），它能产生最大测试覆盖的测试激励，同时使用最少的测试向量，可以应用了范围广泛的设计类型和设计流图，也适用于百万门级设计。这是 synopsys 公司的测试工具，它可以和很多EDA 工具一起来完成测试工作。

1.TetraMAX 具有以下几个主要的 ATPG 能力：

能够读入以 verilog、VHDL 和 EDIF 格式的设计网表和 STIL 格式的测试协议信息；

能够生成各种标准和非标准的测试激励文件：WGL、STIL、VHDL、FujitsuTDL、TITDL91、ToshibaTSTL2；

提供 ATPG 模式的选择：Basic-scan ATPG、Fast-sequential ATPG、full-sequential ATPG；

支持以下的 design-for-test（DFT）类型：

ü Various scan flip-flop types (multiplexed flip-flop,master, slave,transparent latch, and so on)；

ü Internal, nondecoded three-state buses

ü Bus keepers

ü RAM and ROM models

ü Proprietary and standard test controllers (such as IEEE1149.1-compliant boundary scan)

产生和验证 ATPG 激励，避免 bus contention 和 float conditions；

提供交互式的分析和调试工具 GSV（graphicalschematic viewer）；

提供连接 verilog 和 VHDL 仿真器；

提供一个整合的故障仿真器支持功能激励的故障仿真；

能够运行 direct automatedtestequipment(ATE)诊断，允许你快速定位设计中的故障位置产生的测试错误；

2. TetraMAX 的测试激励产生支持五种类型的故障模式：

Stuck-at faults:是测试激励产生的标准模式；

IDDQ faults：

Transition delay faults：

Path delay faults：

Bridging faults：

3. TetraMAX 可以兼容大量的 design-for-test 工具

TetraMAX 可以兼容大量的 design-for-test 工具象 DFT compiler。使用了 DFT compiler 和 TetraMAXATPG 的设计流程简便并且结果有质量保证。

ATPG 脚本范例

这是本人参与的一款 CPU 流片测试时所写的 TetraMAX 生成测试激励的教本，仅共参考，呵呵！

```
read netlist smic/lib/*.v
read netlist map/netlist.v
set build -black_box r128x21
set build -black_box r128x22
set build -black_box r256x128_1
set build -black_box dualram128
set build -black_boxS018PLLGS_500
run build_model ppc_top_PAD


add pi constraint 1 PAD_rst_n
add pi constraint 1 PAD_trst_n
add pi constraint 0 PAD_tms
//add pi constraint 0 PAD_tck
```

```
set drc map/750.spf
run drc
report rules -fail
report nonscan cells -summary
report bus -summary
report feedback paths -summary
set atpg -abort 50 -pat 1500 -mergehigh

//add nofaults here for enhance faultsconverage
add faults -all

run atpg -auto
//run atpg basic_scan_only
//run atpg fast_sequential_only
//run atpg full_sequential_only
report summaries
analyze faults -class an
report summaries
report faults -level 5 64 -class au -collapse -verbose

write patterns ./dft/pattern_dft.v -formatVERILOG_Single_file
-serial -replace
write patterns ./dft/pattern_dft.wgl-format WGL -serial -replace
```

ATPG 流程分析(一)

1.读入网表文件

可以使用图形界面设置，或者是命令行设置，具体设置参数见帮助文档。网表文件类型可以是标准ASCII 格式或是GZIP（一般UNIX 的压缩格式），TetraMAX 在读文件前自动检测压缩文件并解压缩。设计中如果有多个文件，

可以多次使用readnetlist命令依次读入文件，同样可以使用通配符（如：./sim/*.v）来代替多个文件，如：BUILD>read netlist/proj1234/shared_verilog/*.v －noabort。当网表文件已经读入了，要想重新读入网表文件，需要使用命令 Readnetlist －delete 或者 Read netlist filename －delete 先清除已有的文件。

2.构造 ATPG 模式

使设计中的各个部分成为 TetraMAX 过程的一部分，所作工作是删除层次，把它们放入 TetraMAX 可以使用的内存映象中。同样可以使用图形界面或是命令行 runbuild_model。

3.执行设计规则检查

这是最重要的一步。DRC(Design RuleChecking)检查包括：

扫描链输入输出是否逻辑连接

是否时钟和异步 set/reset 针与扫描链翻转的连接由唯一的输入端口控制

是否当从正常模式切换到扫描变换模式时，时钟/set/reset 是 off

TetraMAX 完成这些 DRC 检查，你应该提供以下信息：时钟端口，扫描链以及其他控制信息，这些都在 STILtest protocol file（.SPF）中给出.这个文件由DFTCompiler 产生，也可以手动产生这个文件。

ATPG 流程分析(二)

4.开始测试 DRC

使用命令 run drcfilename 运行 DRC 测试。

1) Review DRC 结果

2) 在 GSV 中显示违背规则

在 GSV 中可以直观的看到违背的规则（GSV grahpical schematicviewer）。

3) 测试设计规则目录

测试设计规则被分成了 10 个类别分别为

B (Build rules)

C (Clock rules)

L (LBIST rules)

N (Netlistrules)

P (Path Delayrules)

S (Scan Chainrules)

T (Testerrules)

V (Vectorrules)

X (X-staterules)

Z (Tristaterules)

在线帮助中对每类都有详细的介绍，详见Contents 中的 Error and WarningMessages

TetraMAX 提供命令可改变设计规则严重级别(severity)

每个设计规则都有四个严重级别：Ignore、Warning、Error、Fatal。可以使用命令 set rules 来改变级别。也可以使用命令 reprotrules 决定规则级别的设定以及已经发生的违反的规则数目。例如：set rules B5 warning

4) DRC 执行的详细过程

在执行 DRC 过程中，TetraMAX 做了以下工作：

读.SPF 文件收集信息，检查语法和连接（consistency）故障；

完成总线和线网逻辑的连接能力检查；（Z rules）

在 SPF 中模拟测试过程，看某些情况是否满足要求；

根据定义的测试过程的方向模拟每条扫描链，确保扫描路径对每条扫描链规则来说都是可操作和可靠的。（S rules）

分析所有的时钟和定时设备是否满足 ATPG 规则。（C rules）

分析所有的不需要扫描的设备；

ATPG 流程分析(三)

5.ATPG 准备

初始化故障列表，选择 pattern source. choosesettings for bus contention checking, and specify the patterngeneration effort.

1）使用命令 setfaults 设置故障列表

例如：

TEST> add faults -all 表示包括所有可能的故障。

TEST> read faultsfault_list_filename 表示使用名字为 fault_list_filename 的故障列表

也可以将一些指定的 blocks, instances, gates, orpins 排除到故障列表外，使用下面的方法：

TEST> addnofault /sub_block_A/adder

TEST> addnofault /io/demux/alu

TEST> addfaults -all

表示先去除模块 adder 和 alu，再添加故障列表。

2） 删除故障列表使用命令 removefaults，例如：

TEST> addfaults -all

TEST>remove faults /sub_block_A/adder

TEST>remove faults /io/demux/alu

指定 ATPG 设置，例如：DRC> set atpg-patterns 400 -abort_limit 5

6 运行 ATPG

使用命令 run atpg -random，首先是 Basic-ScanATPG，然后是 Fast-SequentialATPG，和 Full-SequentialATPG。Run atpg -auto_compression

ü Basic-Scan ATPG：

有效的，仅适用于组合电路的全扫描模式。为了得到高的测试覆盖率需要使用该测试模式。

ü Fast-Sequential ATPG

ü Full-Sequential ATPG

ATPG 流程分析(四)

7.ATPG 激励压缩

在测试激励合并的基础上还可以进行测试激励的压缩，使用如下命令：

TEST> run pattern_compress 99 2-noverbose

8.ATPG 激励压缩

在测试激励合并的基础上还可以进行测试激励的压缩，使用如下命令：

TEST> run pattern_compress 99 2-noverbose

9.保存故障列表

TetraMAX 包含了关于设计中潜在的 faults 的列表，可以使用命令 report faults 或者 write faults。Report faults 表示查看故障列表，而 write faults 表示将故障列表写入文件。

例如：

TEST> write faults faults.AU -class au －replace。

下面的命令写所有的故障列表：

TEST> write faults filename -all －replace;

下面的命令只写 undetectable blocked (UB) and undetectable redundant(UR)故障类型：

TEST> write faults filename -class UB -class UR-replace;

10.使用脚本文件

写脚本文件*.tcl 完成 ATPG 操作类似于WINDOWS 中的批处理文件。在我们的 TETRAMAX 环境中采用了脚本运行的方式，所有的上面提到的 ATPG 流程都写到一个脚本文件中（tmax.tcl）。

4. Tmax 常规使用及报告

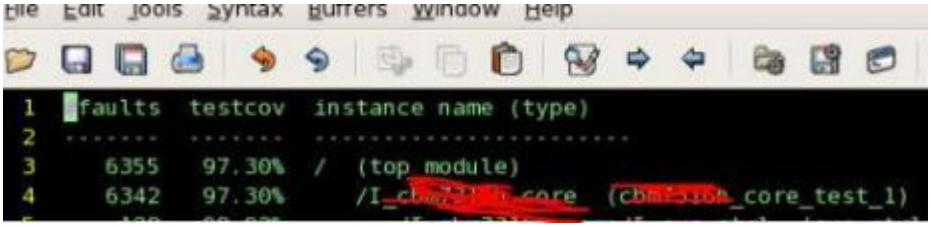
1.在执行生成 atpg 向量前,需要把 soc 中 atpg_en(atpg 模式寄存器)固定为 1, Assign atpg_en=1;否则无法生成 atpg 向量.

考虑在 tmax 脚本中使用 add_pi_constraint 1 atpg_en//此命令只能识别端口

2.对于 port 端口,需要使用 add_pi_constraint 1 p00_pad 固定所有 port 与 scan 模式一直的值(除了atpg 测试时用到的 pad),实际上只需要固定使用的 pad(如复位),因为其它 pad 的输入到输出做了bypass,所以不影响

3. 在仿真 atpg pattern 时,一般不加 sdf 文件,加一个固定延时即可(这样速度会比较快),但如果出错,则必须加 sdf 文件重新仿真---这个是最终能够确定的方法,vcs 有一个专用的选项

4. Scan 覆盖率报告在 report_faults -class AN -level 5 100 > results/gate/xxx.an.where



5. 在 tmax 脚本中，由于模拟模块是不参与插入 scan chain,所以可以设置为 black box,但要注意作为 scan 需要用到的 pad 不能设置为 black box;

6. 如果是使用序列让 rtl 寄存器产生 atpg_en 为 1，则需要修改 netlist，使 assign atpg_en=1;

7. Tmax 使用的是.v 仿真库，而不是综合库，虽然使用的是 read_netlist 命令。

5. 配置信息.rc 文件

```
//
// --- Standard startup file for $SYNOPSYS_TMAX/admin/setup/tmax.rc
//     revision: 23-Jan-2001
// --- common WGL settings
//
set wgl -nomacro -nopad
set wgl -inversion master
set wgl -pre_measured
set wgl -bidi_map 0x 0-
set wgl -bidi_map 1x 1-
set wgl -bidi_map xx x-
set wgl -bidi_map zx -x
set wgl -bidi_map z0 -0
```

```
    set wgl -bidi_map z1 -1
    set wgl -bidi_map zz -z
    set wgl -bidi_map -x --
    set wgl -bidi_map z- --

    //
    // --- color and DFF/DLAT graphics mode
    //
    set environment viewer -dff_dlat_symbol mode3
    set environment viewer -color type_name_text 255 120 0
    set environment gui -end_scroll
    set environment info -save_dimensions
    set environment viewer -design_symbols primitives
    //set environment viewer -primitive
    //set environment viewer -nobuf              // do not use with
also using -primitive

    //
    // --- ALIASES
    //
    unalias -all

    alias read_verilog read_netlist
    alias read_edif read_netlist
    alias read_vhdl read_netlist
    alias read_patterns set patterns external
    alias add_set add_clock
    alias add_reset add_clock
    alias add_set_reset add_clock
```

```
alias report_alias alias
alias history report command -hist
alias hist    report command -hist
alias hi      report command -hist
alias include source
alias do      source

//
// --- for convenience of those familiar with other tools
//
alias check_test           run drc
alias create_test_clock    add clock
alias create_test_patterns run atpg -auto
alias link                 run build_model
alias read_init_protocol   set drc
alias read_test_protocol   run_drc
alias set_test_hold        add_pi_constraint
alias write_test           write_patterns

//
// --- RESET to BUILD MODE
//     Intended use from DRC or TEST modes
//
alias reset_build read net -del ; \
 set build -add -del -reset \
     -mer notie -mer nocascade -mer bus -mer feedback -mer muxp
\
     -mer xorp -merge equiv -merge flip -fault low \
     -hier / -undriv pio ; \
```

```
     set commands history ; \
     set learning -atpg_equivalence -common_input -implication
medium \
         -max_feedback_sources 100 -module -sim_passes 32
-test_passes 5000 -noverbose ; \
     set messages disp -nodouble -transcript ; \
     set netlist -cell -check -dom off -enable_port -escape cond \
         -max_err 10 -nopin -noscalar -noseq -redef last -suppress
-xmodel \
         -dom on ; \
     remove net connections -all ; \
     set rule -reset



    //
    // --- RESET to DRC MODE
    //      Intended use from TEST mode
    //
    alias reset_drc drc -force ; \
     set buses -external z -fault x ; \
     set com history ; \
     set cont bidi bus nodff nofloat noram nowire -atpg -capt -pre
\
         -multi -noretain -sev warn -noverb ; \
     set drc -noallow -nofile -bidi none -clock -any -nogroup -init
X -multi -osc 500 \
         -noremove -shadow -skew 1 -nostore_set -nostore_stab -tla
-notrace ; \
```

```
      set fault -model stuck -rep uncol -sum noverb -pt_credit 50
-au_credit 0 \
          -nofault -noatpg -noequiv -trans any ; \
      set iddq float strong weak write -atpg -notoggle ; \
      set mess display -nodouble -level standard -transcript ; \
      set random -clock -none -length 1024 -observe master ; \
      set rule -reset ; \
      set scan abil off -all ; \
      rem atpg constraints -all ; \
      rem atpg prim -all ; \
      rem pi const -all ; \
      rem pi equiv -all ; \
      rem clock -all ; \
      rem cell const -all ; \
      rem dont -all ; \
      rem nofault -all ; \
      rem po mask -all ; \
      rem capture mask -all


    //
    // --- RESET to TEST mode
    //     Intended use from test mode, clears patterns and gets
ready for
    //     regeneration of patterns
    //
    alias reset_test \
     set pat -del ; \
     set pat int ; \
```

```
     rem faults -all  ; \
     rem nofaults -all  ; \
     rem dont -all  ; \
     set atpg -abo 10 -noallow -cap 0 -nocheckp -cover 100 -dec noran
-di_anal \
         -nofast -full_seq_abort 10 -nofull_seq_atpg
-full_seq_merge off \
         -full_seq_time 10 0 -mer off -pat 0 -prev noran
-random_fill \
         -resim_basic auto -reset_bidis -sto -sum -time 0.0 0.0
-noverb  ; \
     set bus -ext z -fault x  ; \
     set com hist  ; \
     set cont bidi bus nodff_dlat noram nowire -atpg -preclock
-capture \
         -multiple_on -noretain_bidi -severity warning
-noverbose  ; \
     set fault -model stuck -report uncollapsed -summary noverbose
-pt_credit 50 \
         -au_credit 0 -nofault -noatpg -noequiv  ; \
     set iddq float strong weak write -atpg -notoggle  ; \
     set mess display -nodouble -level standard -transcript  ; \
     set prim -max_fanout 2 -time clock -noverb  ; \
     set random -clock -none -length 1024 -observe master  ; \
     set sim -basic -nobidi -data 0 -1 -meas pat -osc 10 2 -words
32 -noverb  \
         -nostore_mem -noverb  ; \
     set wgl -scan_map dash -nomacro -nopad -nolast_scan -nogroup
-inv master \
```

```
        -notester -chain all -force prev


    //
    // --- RESET to ALL
    //     Intended to be used from anywhere and to restore to
similar conditions
    //     as freshly entering the tool.
    //
    alias reset_all build -for ; \
     rea net -del ; \
     set build -add -del -reset \
         -mer notie -mer nocascade -mer bus -mer feedback -mer muxp
\
         -mer xorp -merge equiv -merge flip -fault low \
         -hier / -undriv pio ; \
     set learn -atpg -common -implic med -max_feed 100 -sim 32 -test
5000 -noverb ; \
     set mess disp -tran ; \
     set net -esc cond -noscalar -redefine last -max 10 -celldef
-noseq \
         -enable_port -suppress_faults -xmodel -check_only -nopin
-dom on ; \
     rem net con -all ; \
     set rule -reset ; \
     set bus -ext z -fault x ; \
     set com hist ; \
     set cont bidi bus nodff noram nowire -atpg -capt -pre \
         -multi -noretain -sev warn -noverb ; \
```

```
     set drc -noallow -nofile -bidi none -clock -any -nogroup -init
X -multi -osc 500 \
         -noremove -shadow -skew 1 -nostore_set -nostore_stab -tla
-notrace ; \
     set fault -model stuck -rep uncol -sum noverb -pt_credit 50
-au_credit 0 \
         -nofault -noatpg -noequiv -trans any ; \
     set iddq float strong weak write -atpg -notoggle ; \
     set atpg -abo 10 -noallow -cap 0 -nocheckp -cover 100 -dec noran
-di_anal \
         -nofast -full_seq_abort 10 -nofull_seq_atpg
-full_seq_merge off \
         -full_seq_time 10 0 -mer off -pat 0 -prev noran
-random_fill \
         -resim_basic auto -reset_bidis -sto -sum -time 0.0 0.0
-noverb ; \
     set prim -int 0 -max 2 -time clock -noverb ; \
     set sim -basic -nobidi -data 0 -1 -meas pat -osc 10 2 -words
32 -noverb  \
         -nostore_mem -noverb ; \
     set wgl -scan_map dash -nomacro -nopad -nolast_scan -nogroup
-inv master -pre_meas \
         -chain all -force prev \
         -bi 0x 0- -bi 1x 1- -bi xx x- -bi zx -x -bi z0 -0 -bi z1
-1 -bi zz -z -bi z- --

    //end of file
```

6. 范例分析

运行命令 ： tmax -shell -64bit scr/projects_name_tmax_post.scr | tee log/projects_name_tmax_post.log ###tmax 工具用来生成 ATPG 向量

projects_name_tmax_post.scr 内容如下：

read_netlist ../sim/projects_name.HV.scan 读取综合后的网标

read_netlist

/ic01/projects/lib/p130nm/gsmc/fromVerisilicon/GSMC13LP/FEView_STDIO/STD/Tmax/gsmc13_lp_tmax.v

read_netlist

/xxx/projects/PROJECTS_NAME/revA0/team/xxxxx/chip/rtl/model/RASP1536X9.v.scan

read_netlist

/xxx/projects/PROJECTS_NAME/revA0/team/xxxxx/chip/rtl/model/RASP256X9.v.scan

read_netlist

/xxx/projects/PROJECTS_NAME/revA0/team/xxxxx/chip/rtl/model/FLS024K19DA.v.scan

read_netlist

/xxx/projects/PROJECTS_NAME/revA0/team/xxxxx/chip/rtl/model/io_model.v.scan

read_netlist

/xxx/projects/PROJECTS_NAME/revA0/team/xxxxx/chip/rtl/model/pch_5p15.v.scan

read_netlist

/xxx/projects/PROJECTS_NAME/revA0/team/xxxxx/chip/rtl/model/analog_top.v.scan

read_netlist

/xxx/projects/PROJECTS_NAME/revA0/team/xxxxx/chip/rtl/model/gpio_ana_pad_5vp15.v

read_netlist
/xxx/projects/PROJECTS_NAME/revA0/team/xxxxx/chip/rtl/model/glitch_filter_5vp15.v
read_netlist
/xxx/projects/PROJECTS_NAME/revA0/team/xxxxx/chip/rtl/model/por2_5vp15.v
read_netlist
/xxx/projects/PROJECTS_NAME/revA0/team/xxxxx/chip/rtl/model/ANTFIXHD.v　　上面都是模拟模型----空壳没内容，输出给 Z

###set_build -black_box
set_build -black_box antrevp_5vp15
set_build -black_box saradc_12bit_5vp15
set_build -black_box pad_cp　　设置说明黑盒

report_modules -summary >
results/post/projects_name_tmax.mod_sum.report
report_modules -error >
results/post/projects_name_tmax.mod_err.report
###build_the model
run_build_model projects_name
#--
--
run_drc ../syn/results/report/projects_name.spf　##.spf 文件由 DC 综合生成，包含管脚及扫描链等信息

report_violations -all > results/post/projects_name_drc.vio
add_faults　-all　##增加各种类型的测试缺陷
run_atpg -auto_compression basic_scan_only　##生成扫描压缩向量

write_pattern results/post/projects_name_atpg_single.v -format verilog_s -serial -replace ##按格式输出向量,此文件实际是几个仿真环境,包含 tb,dut,激励及对比,后期仿真就是用该文件

write_pattern results/post/projects_name_atpg_single.stil -format stil -replace ##按格式输出向量

##ite_pattern results/prjxxxx0_atpg_single.stil -format stil99 -vcs -replace -serial 新版工具按此命令生成

analyze_fault -class UD -verb >
results/post/projects_name.undetectable

analyze_fault -class AU -verb >
results/post/projects_name.atpg_untestable

analyze_fault -class ND -verb >
results/post/projects_name.not_detected

report_faults -class AN -level 5 100 >
results/post/projects_name_atpg.an.where ##报告出错位置

analyze_faults -class AN -verbose -max 10 >
results/post/projects_name_atpg.an.why ##报告出错原因

report_summ > results/post/projects_name.sum

report_scan_chains -verb > results/post/projects_name.chain

report_scan_cell -all -verb > results/post/projects_name.cell

#exit

在 2016 版本需要如下命令生成测试平台

write_pattern results/gate/xxx_atpg_single.stil -format stil -cellname type -serial -replace

```
write_testbench -input results/gate/xxx_atpg_single.stil -output
results/gate/xxx_atpg_testbench   -replace
```

7. 其它命令介绍

add_cell_constraints <0 | 1 | x | 0x | 1x | ox | xx>
<[chain_name | instance_name>
[-position {< cell_pos1 | sci> [cell_pos2 | sci]>} | -all>>]
[-index number] 用来在做 scan 时,约束某个 cell 固定为某值,在输出时为 x 态,但同时不影响其它cell正常做 scan;---假如链中有某个cell 固定有问题但又不影响功能,那么加这个选项排除这个 cell,生成测试波形?
Arguments
0 | 1 | x | 0x | 1x | ox | xx
Indicates the type of constraint to be added. The choices 0, 1, or x indicate the constrained value that should be in place at the end of the scan chain load. When a 0, 1, or x constraint is applied, the scan cell is always loaded with this value for every pattern. A capture clock loads new data into the scan cell that might be different from the 0/1/X loaded by a scan load.

When a 0x constraint is specified, the scan cell is always loaded to zero (0) and will always observe X.

When a 1x constraint is specified, the scan cell is always loaded to one (1) and will always observe X.

A ox constraint indicates the observed value is always masked, or considered to be X. The value loaded by a scan load or captured by a capture clock is not affected, and the actual scan cell value continues to affect the fanout cone of the scan cell. A selection of ox affects only the expected value for the scan chain unload. This value is X (or masked). To affect the value loaded by a scan load, use the xx selection. To affect the value loaded by a capture clock, use the add_capture_masks command.

Some compression restrictions may affect the ability to apply cell constraints. In the case of chain tests in compression mode, M177 messages might be reported when attempting to specify constraints, as shown in the following example:
add_cell_constraints 1 8897
add_cell_constraints ox 9299
add_cell_constraints 0 8706

In this case, the -verbose option of the set_contention command is set and the following messages are reported:

```
Warning: Cell constraint 11-15 set to 1 not satisfied for preclock chain
test pattern  (pat1=0).  (M177)
Warning: Cell constraint 25-14 set to 0 not satisfied for preclock chain
test pattern  (pat1=3).  (M177)
Warning: Cell constraint 16-10 set to 1 not satisfied for preclock chain
test pattern  (pat1=0).  (M177)
```

Formality 篇

启动命令：输入 fm_shell -f xxx.tcl

或者 formality -f rtl2gate.fms | tee rtl2gate.fms.log

启动 gui:先 fm_shell 进入 ，再使用 start_gui 进入 gui 界面

他山之石:

1. 如果 DC 的综合脚本中使用了 set_case_analysis 命令,那么在 formality 中使用 set_constant 命令对相应的 port,pin,net 或 register cell 也设置为常数
2. 使用 set_constant 命令将 testmode 设置为无效,从而 disable scan logic
3. 使用 set_constant 命令 disable 边界扫描
4. Register retiming 是为了满足时序或者面积需求,而将寄存器跨组合逻辑移动,这种移动可能活改变寄存器的数量和寄存器的连接关系,进而引起综合出来的 implementation design 和 reference design 比较点匹配不上

1. 形式验证 Formality

Formality 形式验证是一个基于数学意义的验证方法，通过比较两个设计 A,B：如果 A 的逻辑功能被 B 包含，那么形式验证认为是通过的。

需要注意的是并不是说着两个 design 是完全相等的，而是逻辑上具有包含的关系。

注意:netlist 使用插入 scan 后的 netlist,在对比时在脚本中需要 disable scan_en(set constant …);

Clock_Latch 无法匹配

在 IC 的流程中通常用于进行不同流程步骤的netlist 的比较：

逻辑综合 netlist，floorplan netlist， placement netlist ，CTS inserted netlist，P&R netlist

在每一个步骤后都有新的逻辑加入到netlist 中，但是这个新的逻辑的加入不能改变之前 netlist 的功能。

在 dynamic simulation 的流程中，需要开发验证 pattern 作为输入来进行验证，验证的覆盖率取决于 pattern 的开发的完备性；而 formality 是基于数学比较的，不需要任何的输入 pattern。因此相比于动态仿真的优势在于：

不需要开发验证 pattern 速度比较快

覆盖率 100%

纯逻辑上的验证，不考虑物理和 timing信息

缺点在于：

由于不考虑 timing，因此需要配合 STA 工具使用。

formality 包含了 2 个工具：

Equivalence checkers： 逻辑等价性检查,检查两个 design 是否是逻辑上相等的。

Model checkers：检查 design 是否满足某些逻辑属性比如 assert 断言。

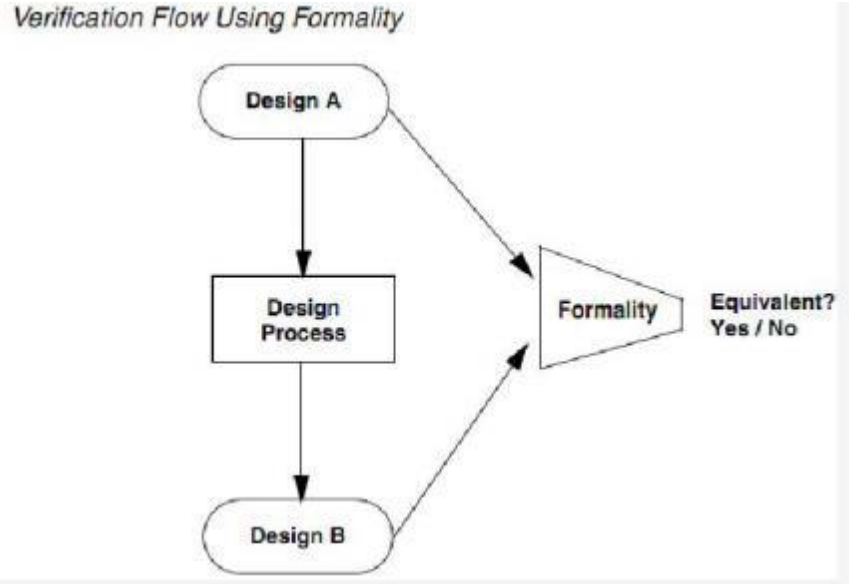
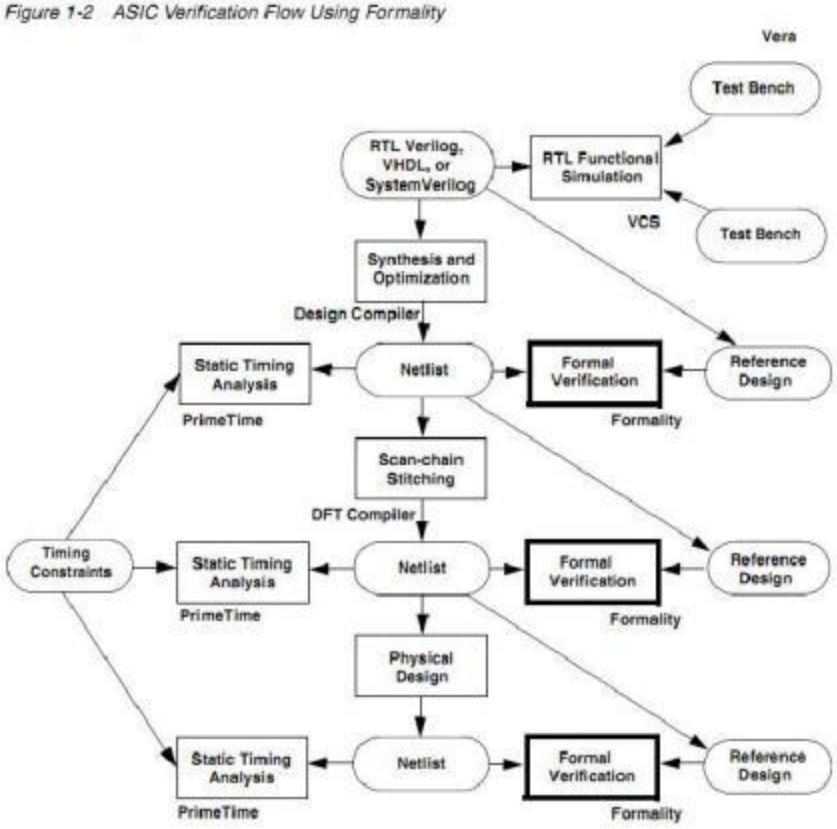


Figure 1-2 ASIC Verification Flow Using Formality



从这个流程中可以看到，第一次比较是原始的 verilog 代码和 DC 综合后的 netlist 的比较；

第二次比较是逻辑综合后的netlist 和 DFT 插入后的netlist 的比较

第三次比较是 DFT 插入后的 netlist 和后端生成的 netlist 的比较

两个术语：

Reference design ： 作为 golden 的 design

Implementation design ： 正在修改的 design，需要证明和参考设计有等价性。

如果 implmentation design 被证明是和 reference design 等价的话，implmentation design 可以成为新的 reference design。

Logic Cones：逻辑锥的概念

logic cones 从一个 design object 作为起始，往回回溯，到某一个 design object 结束的中间所有的组合逻辑。

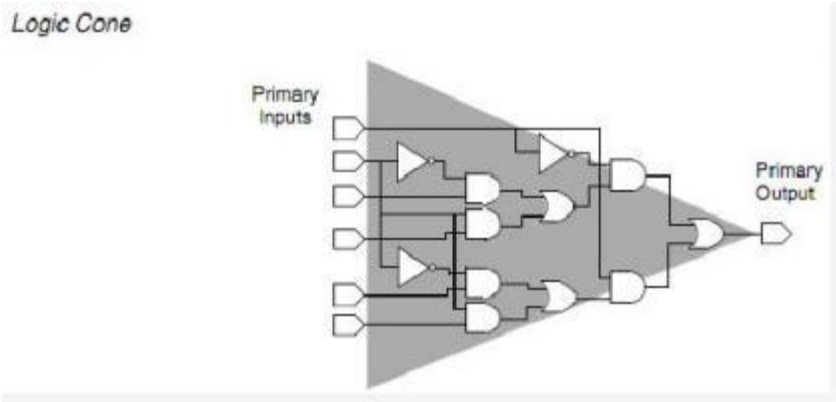
logic cones 的起始点是 formality 中的 compare point：

primary outputs, internal registers, black box input pins,or nets driven by multiple drivers where at least one driver is a port or black box

logic cones 的终止点是：

primary inputs or compare points

因为 formality 中 register 是作为 compare point 的，因此 logic cone 只能是组合逻辑的集合。



上图是一个 logic cone：起点是 primary output，回溯的终点是 primary input。

Black Box 的概念：

A black box is an instance of a design whose function is unknown. Black boxes are

commonly used for components of a design that are not synthesized. Examples of common

black boxes include RAMs, ROMS, analog circuits, and hard IP blocks. The inputs to black boxes are treated as compare points and the outputs of the black boxes are treated as input points to other logic cones

主要是一些内部功能未知的模块。black box 的输入被认为是一个 compare point。

由于 black box 的内部功能未知，因此 black box 的中间逻辑不作为 logic cone。

比较点的概念(compare point):组合逻辑的终点。

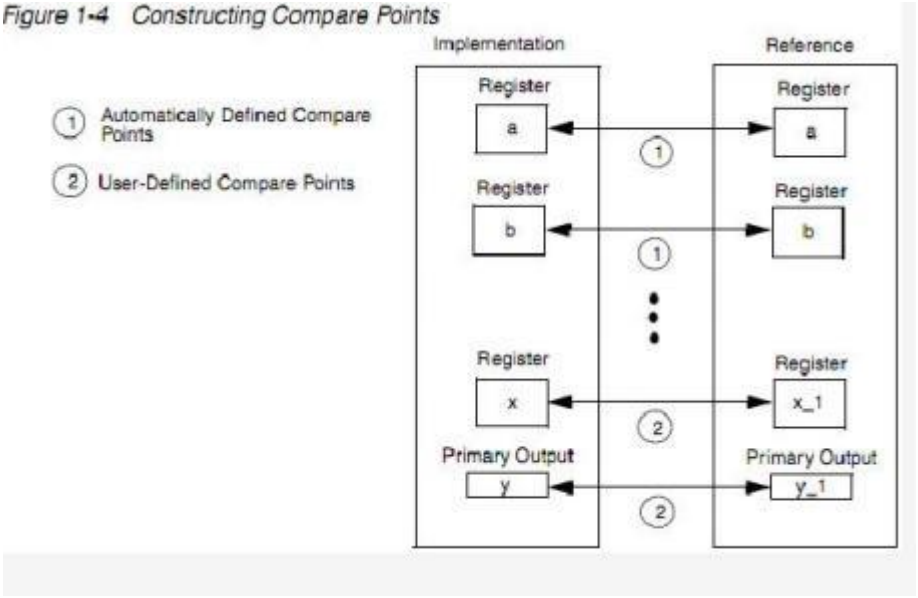
A compare point is a design object used as a combinational logic endpoint during
verification. A compare point can be an output port, register, latch, black box input pin, or net
driven by multiple drivers.

Formality uses the following design objects to create compare points automatically:

- **Primary outputs**
- **Sequential elements 时序单元—寄存器**
- **Black box input pins**
- **Nets driven by multiple drivers**, where at least one driver is a port or black box

formality 依据上面的规则自动创建 compare point。用户也可以自定义的创建一些 compare point

Figure 1-4 Constructing Compare Points



比较算法(matching)：

有了 compare point，需要有一个方法将 reference design 中的 compare point 和 imp lementation design 中的 compare point 能一一对应起来(如果需要进行 complete verification)。

如何进行 compare point 的对应：

(1) 通过 object name 来进行对应，有相同 object name 的 compare point 进行对应。

Name-based matching technique

(2) 如果在其他 design 中 object name 有了改动，导致无法进行对应，那么需要采用其它的方法进行对应。 Non-name-based matching technique 包括 signature analysis 算法

(3) 手工进行对应。User-match

在基于 name-based match 中如果有一些 compare point 没有找到对应，可以通过下面的命令 report：

report_unmatched_points 这种不匹配的问题可能导致的原因是：

object name 被流程的其它部分更改导致无法对应 imp lementation design 中加入了新的逻辑比如 black box 导致有新的 compare point 产生

如下表所示：。

Table 8-1 Unmatched Compare Points Action

Symptom	Possible cause	Action
Same number of unmatched points in reference and implementation designs	Names have undergone a transformation	Use `set_user_match` command Write and test compare rule Modify name match variables Turn on signature analysis For all, see "Report Unmatched Points" on page 8-5
More unmatched points in reference than in implementation design	Unused cells	No action necessary
	full_case directive in RTL code ignored	Set `hdlin_ignore_full_case` to `false`
	Black box created for missing cells	Reread reference design, including the missing cells Make black box in implementation design
More unmatched points in the implementation design than in the reference design	Design transformation created extra logic	Account for design transformation; see "Design Transformations" on page 10-20
	Black box created for missing cells	Reread reference design, including the missing cells Make black box in reference design

compare point 对应的算法和优先级：

The following matching techniques occur by default when you match compare points, and they are executed in this given order:

1. Exact-name matching (name-based matching).
2. Name filtering (name-based matching).
3. Topological equivalence (non-name-based matching).
4. Signature analysis (non-name-based matching).
5. Compare point matching based on net names (name-based matching).

Exact-Name Matching：

包括区分大小写和不区分大小写的对应。

Reference： /WORK/top/memreg (56)

Implementation： /WORK/top/MemReg (56)

ft_shell > set name_match [all | none | port | cell]

后面的参数来指定哪一类的 compare point 使用 exact-name matching 方法：

all: 所有的

None: 只有 primary input 使用

Port: top-level output ports 使用

cell: register,black box input/output pin

Name Filtering：

大小写不敏感的，使用 filter 来进行对应。

Reference： /WORK/top/memreg__[56][1]

Implementation： /WORK/top/MemReg_56_1

Reference： /WORK/top/BUS/A[0]

Implementation： /WORK/top/bus__a_0

以上case 可以形成对应，下面这个不行：

Reference： /WORK/top/BUS/A[0]

Implementation： /WORK/top/busa_0

命令：

ft_shell > set name_match_filter_chars {~ !@#$%^&*()_-+=|\[] {}"' :;<>?,./V}

Reverse the Bit Order in Multibit Registers：

多 bit register 中 bit 位置反一下也认为是一致的。

register a[31:0];

register a[0:31];

Topolog ical Equivalence：

假设两个没有对应的compare point 的 log ic clone 拓扑上是一样的，认为这两个compare point 是对应。 相同的 log ic clone 驱动的 compare point 认为是对应 point。

Signature Analysis：

set signature_analysis_match_compare_points false

Signature analysis is an iterative analysis of the compare points' functional and topolog ical signatures. Functional signatures are derived from random pattern simulation ; topolog ical signatures are derived from fan in cone topology.

Compare Point Matching Based on Net Names：

Matches can be made through either directly attached driven or driving nets。

通过分析 compare point 的驱动 net 或者被驱动的 net 是否相同来进行对应。

set name_match_based_on_nets false

概念：

Design Consistency： 设计一致性，对于相同的 input pattern 产生了相同的输出响应.

但是如果一个存在 don't care (X) 状态，而另外一个 design 的响应为 0、1 都认为是等价的。

Design Equality： 设计相等，同 Design Consistency 类似但是对于 dont care 状态，比较的另外

一方也必须输出 dont care 状态才认为是相等(而不像 Design Consistency 中输出什么值都认为相同)

可见，Design Equality 成立的要求要高于 Design Consistency。

可见，formality 是基于 log ic cone 来做等效性检查的，如果 reference design 的所有 compare point 的

log ic cones 和 imp lementation design 中的 compare point 都是一致的，那么就认为等效；当然 implementation design 中包含的 compare point 可能多于 reference design(有新的逻辑加入).

2. 范例分析

formality 分为三种,RTL VS gate ; rtl vs post ;gate vs post

formality -f scr/prjxxxxx_rtl2gate.fms | tee log/prjxxxxx_rtl2gate.fms.log

下面是 prjxxxxx_rtl2gate.fms 内容

set MNAME prjxxxxx

#Provide setup information to Formality and help Formality

#understand/process design changes by DC

set_svf -append ../syn/results/report/${MNAME}_syn.svf ###GATA 优化信息在此文件中

#if clock gating is inserted by DC

#none default;

#low allows clock gating that holds the clock low when inactive

#high allows clock gating that holds the clock high when inactive

#any allows both high and low styles of clock gating to be considered

set verification_clock_gate_hold_mode any //处理插入门控时钟不一致问题,最后会 unmatch,但 verify 正确

如下 unmatch 报告

#alert the user to possible mismatches between simulation and synthesis

#true error; false warning

set hdlin_error_on_mismatch_message false

#specifies to ignore or not ignore the full_case directive in verilog files

set hdlin_ignore_full_case false

#specifies to ignore or not ignore the parallel_case directive in verilog files

set hdlin_ignore_parallel_case false

#specifies to ignore or not ignore the translate directive in verilog files

set hdlin_ignore_translate false

#specifies how to control black box creation for unresolved design references

set hdlin_unresolved_modules black_box

不识别/无实例（如模拟模块）的 module 设置为 black box

set verification_failing_point_limit 1000

#unread points are those that either do not have other compare points in their fanout

#set search_path

set DW_PATH /ic01/eda/synopsys/syn_vF-2011.09-SP2/libraries/syn

set search_path [list . $DW_PATH]

lappend search_path "../rtl/misc"

#read lib of standard cell;io;memory;analog ip

read_db

/xxx/projects/PROJECTS_NAME/revA0/team/ganlc/test/STD_6T/Synopsys/hg11lp_6T_ss_1p35v_125c.db

read_db

/xxx/projects/PROJECTS_NAME/ip/flash/GRA_F013Q7P6_FLS024K19DA05K/synopsys_model/FLS024K19DA_max.db

read_db

/xxx/projects/PROJECTS_NAME/ip/RASP256X9/RASP256X9_worst.db

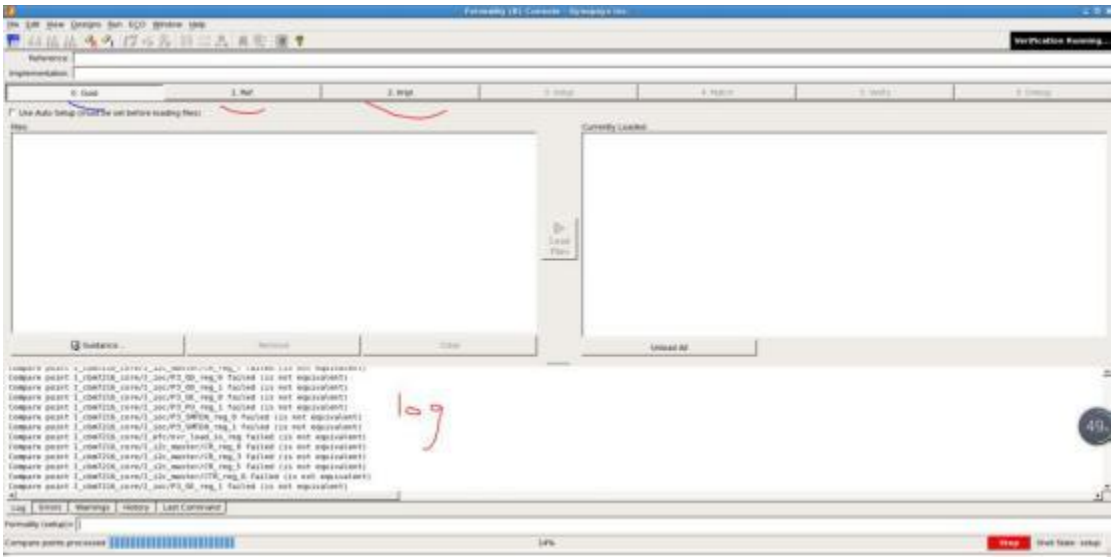
```
read_db
/xxx/projects/PROJECTS_NAME/revA0/team/ganlc/chip/sram_genrate/RA
SP1024X9_worst.db
read_db /xxx/projects/PROJECTS_NAME/ip/analog/projects_name_io.db
read_db
/xxx/projects/PROJECTS_NAME/ip/analog/SP5VP15_PG_V1p0_max.db
read_db
/xxx/projects/PROJECTS_NAME/ip/analog/projects_name_analog_top.db
read_db
/xxx/projects/PROJECTS_NAME/ip/analog/glitch_filter_5vp15/glitch_
filter_5vp15_tt_TYP_max_1p50v_25c.db
read_db /xxx/projects/PROJECTS_NAME/ip/analog/por2_5vp15.db
source -e ../filelist/asic/${MNAME}_fm_rtl_read.tcl  ###读 RTL 文件
---rtl 不需要读模拟模型，见下面格式
set_top r:/WORK/${MNAME}  ###r:refrence 参考标准
current_design ${MNAME}
read_verilog -container i -libname
WORK ../syn/results/netlist/prjxxxxx.v
set_top i:/WORK/${MNAME}
current_design ${MNAME}

set_constant -type port
r:/WORK/${MNAME}/I_prjxxxxx_core/I_test_ctrl/atpg_test_en 0
set_constant -type port
i:/WORK/${MNAME}/I_prjxxxxx_core/I_test_ctrl/atpg_test_en_reg/Q 0
注意此处配置,disable scan chain
match  //formality 第一步
report_unmatched_points >
report/${MNAME}_rtl2gate.unmatched_points.report
```

```
verify  //formality 第二步
report_failing_points >
report/${MNAME}_rtl2gate.failing_points.report
report_unread_endpoints -all >
report/${MNAME}_rtl2gate.unread_points.report //输出结果
#quit

../filelist/asic/${MNAME}_fm_rtl_read.tcl 内容如下
set search_path "$search_path \
/xxx/rtl/misc \
/xxx/rtl/flash/";
read_verilog -container r -libname WORK -define {DBG51 EXPDP4P5 ECO1
ECO2 ECO3 ECO4 ECO5 SYN } { \
/xxx/rtl/generic/clkswitch.v \
/xxx/rtl/generic/sync_level.v \
/xxx/rtl/generic/sync_pulse.v \
/xxx/rtl/generic/sync_pulse_xor.v \
/xxx/rtl/generic/clock_switch_gated.v \
/xxx/rtl/generic/level_filter.v \
。。。。。。。
/xxx/rtl/core/addsub_core.v
}
```

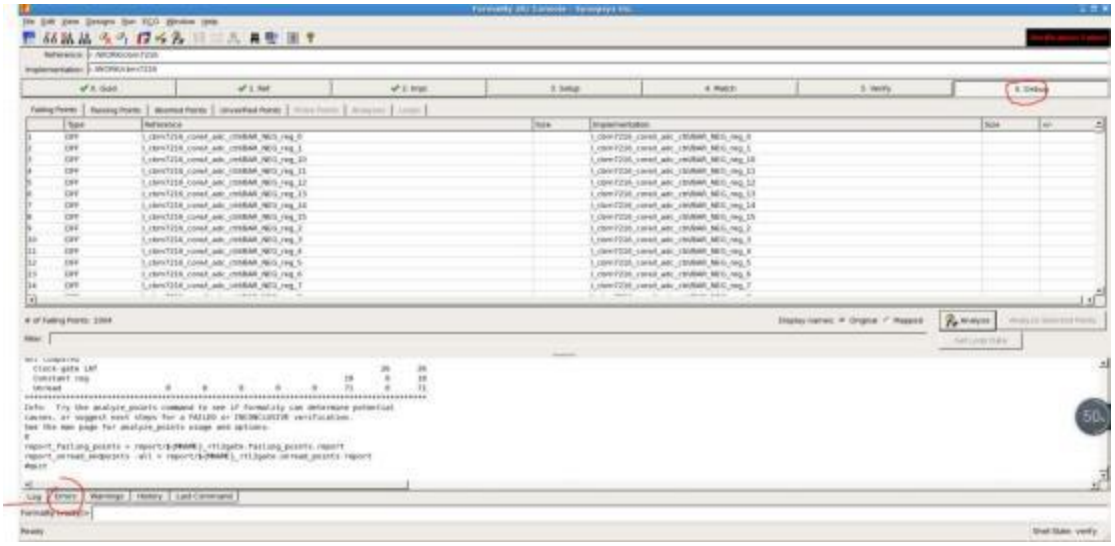
注意：上述 tcl 没有读取模拟部分内容

3. Gui 界面介绍

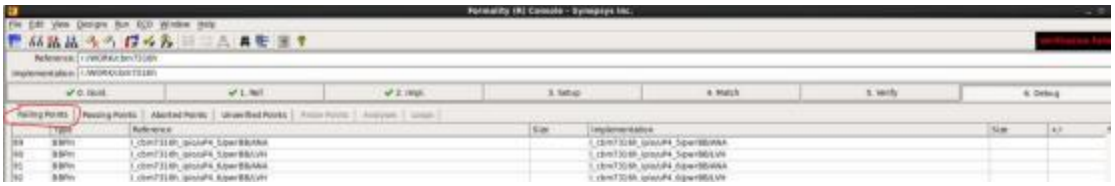
在运行上述脚本会产生如下界面



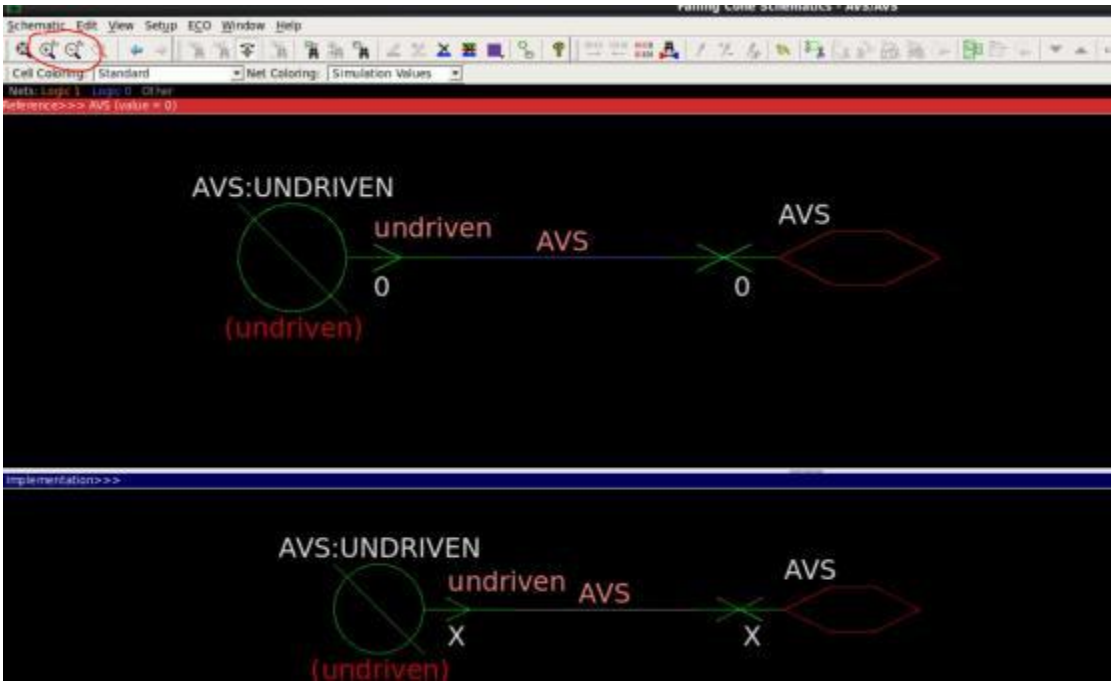
比较结束



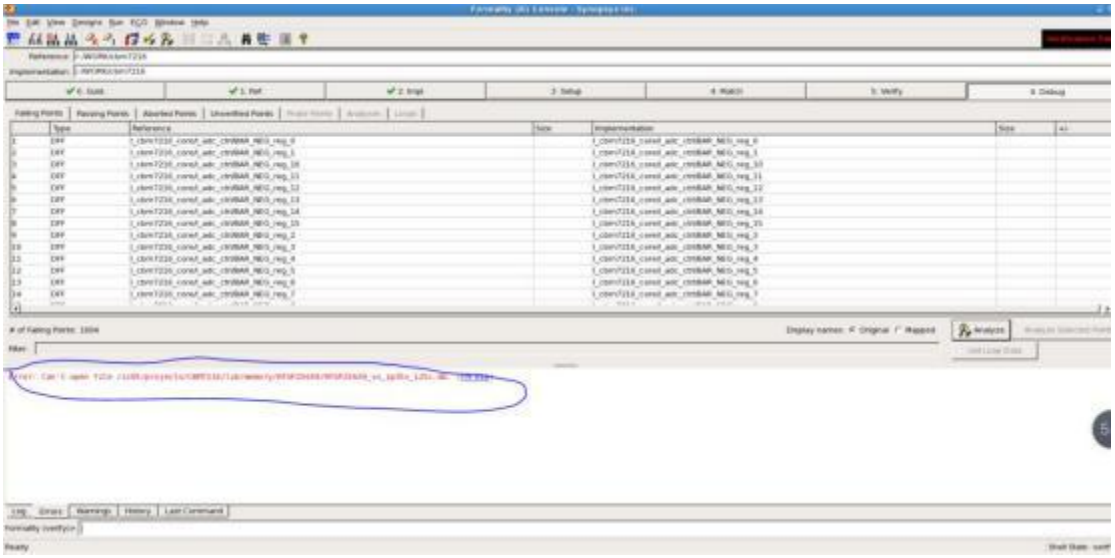
看结果



Fail point 会显示所有对比没通过点,**双击会显示电路及原因**

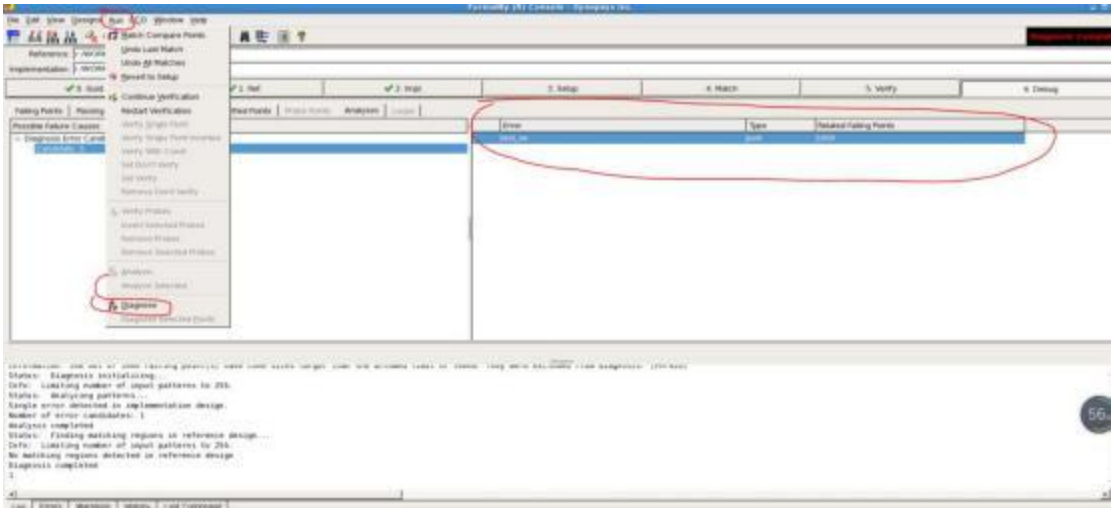


Rtl 的 avs 为 0,pr 后是 undriven,实际这是正常的.



1 提示设计上的违背 ----实际上可能没问题
2 提示找不到模拟模块—综合时没有模拟模块,rtl 中有
Error 一定要 fix

Formality 没通过,如下图



点击 run-→ diagnose 进行诊断,右边显示结果
因为rtl修改后没有重新综合,导致对比失败.

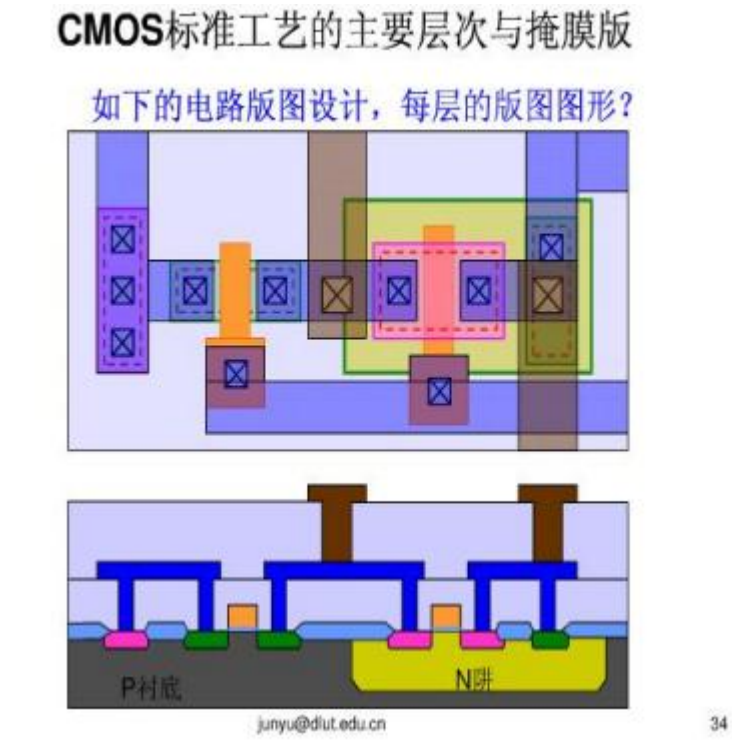
4. 注意事项

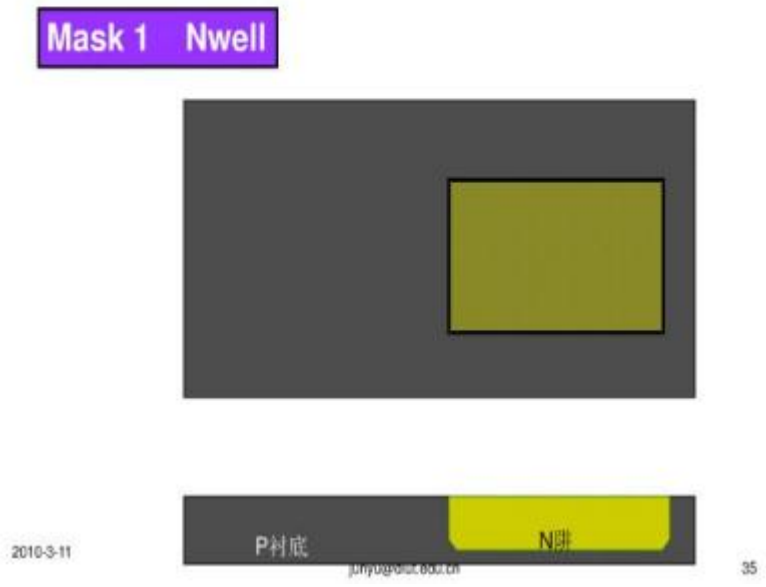
对于悬空的模拟 inout 信号:如果在例化某个 cell 时,如果悬空处理某个 inout 类型的 pin,那么综合时,很可能会被 tie 0,实际综合后的结果就是接 0(综合时没有设置 don't touch),

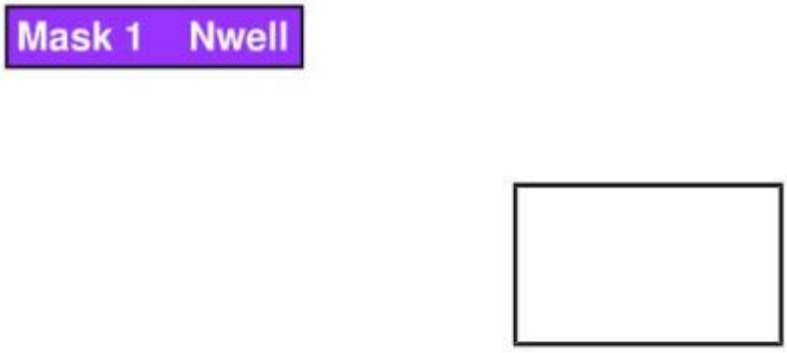
```
pfm_1110skh32kx32_v1a0 PFLS032K190A_1(
   .A        ( A     ),
   .ARRDN    ( ARRDN ),
   .CEb      ( CEb   ),
   .CHIP     ( CHIP  ),
   .CLOCK    ( CLOCK ),
   .CONFEN   ( CONFE ),
   .DIN      ( DIN   ),
   .DOUT     ( DOUT  ),
   .DPD      ( DPD   ),
   .ERASE    ( ERASE ),
   .LCK_CFG  ( LCK_CFG),
   .NVR      ( NVR   ),
   .NVR_CFG  ( NVR_CFG),
   .PORb     ( RESETb  ),
   .PROG     ( PROG  ),
   .PROG2    ( PROG2 ),
   .RDEN     ( RDEN  ),
   .RECALL   ( RECALL ),
   .TM0      ( TM0   ),
   .TM1      ( TM1   ),
   .TMEN     ( TMEN  ),
   .VPP0     ( VPP   ),
`ifdef ECO7
   .VREF     (   ),
`else
   .VREF     ( VREF  ),
`endif
   .WEb      ( WEb   )
);
```

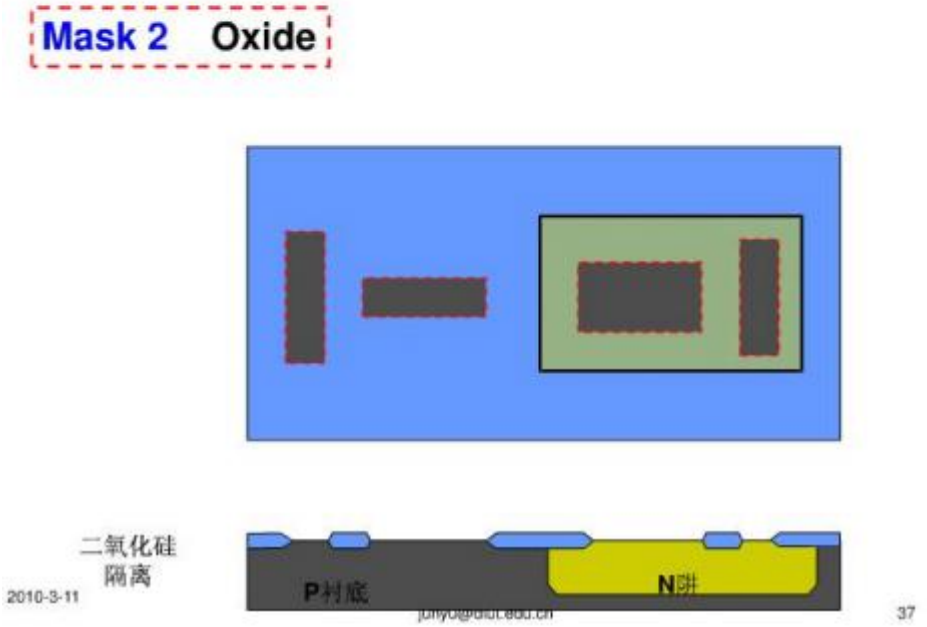
如上,VREF 在综合时被固定接为 0;

Mask 简单介绍

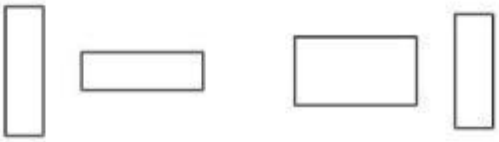

CMOS标准工艺的主要层次与掩膜版
如下的电路版图设计，每层的版图图形？
P衬底
N阱
junyu@dlut.edu.cn
34


Mask 1 Nwell
P衬底
N阱
2010-3-11
35


Mask 1 Nwell
P衬底
N阱

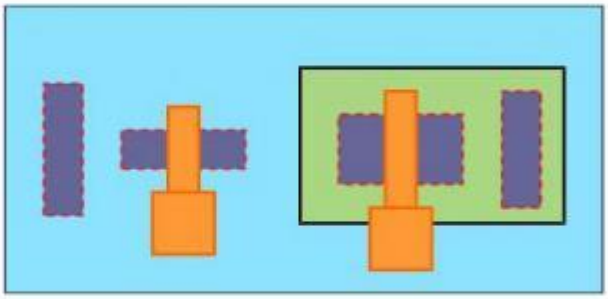

Mask 2 Oxide
二氧化硅
隔离
P衬底
N阱
37

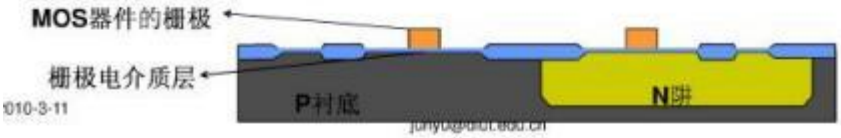
Mask 2 Oxide



010-3-11 38

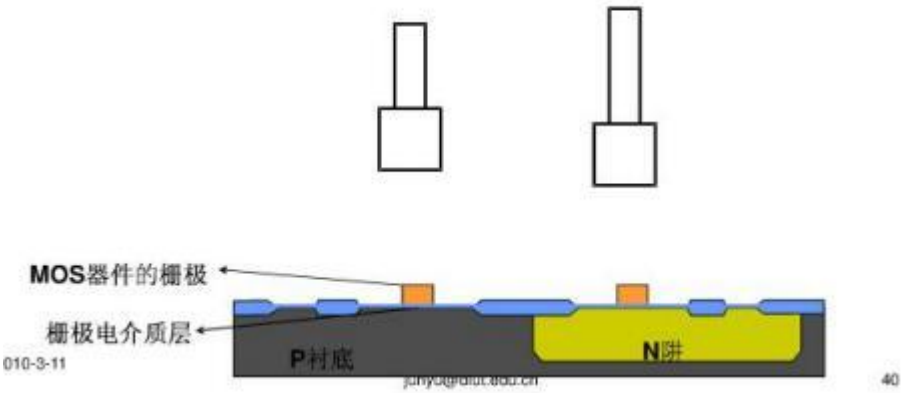
Mask 3 PolyG





010-3-11 39

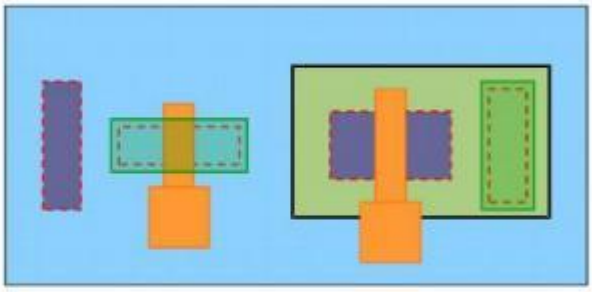
Mask 3 PolyG



010-3-11

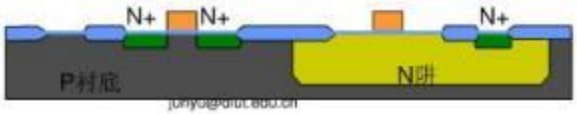
40

Mask 4 nplus



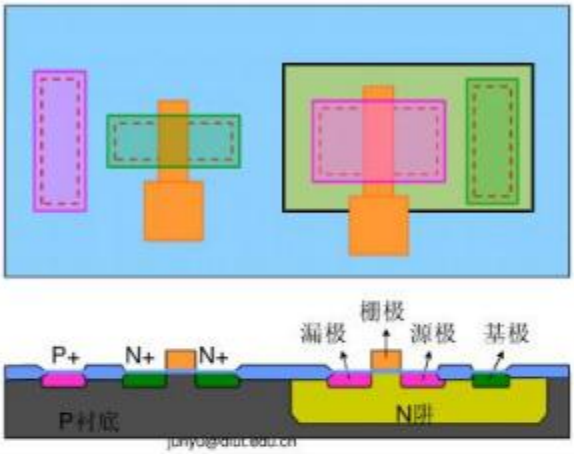
010-3-11 41

Mask 4 nplus



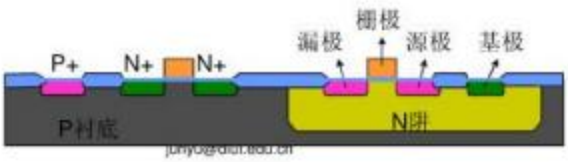
)10-3-11 42

Mask 5 pplus



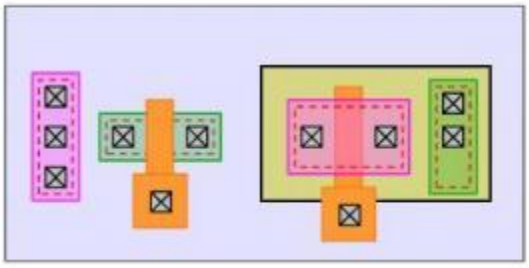
010-3-11 43

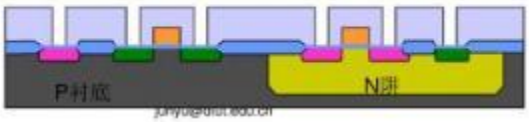
Mask 5 pplus



10-3-11 44

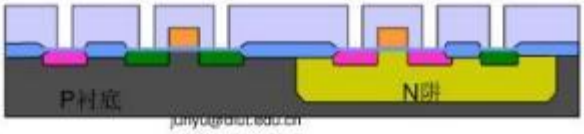
Mask 6 contact



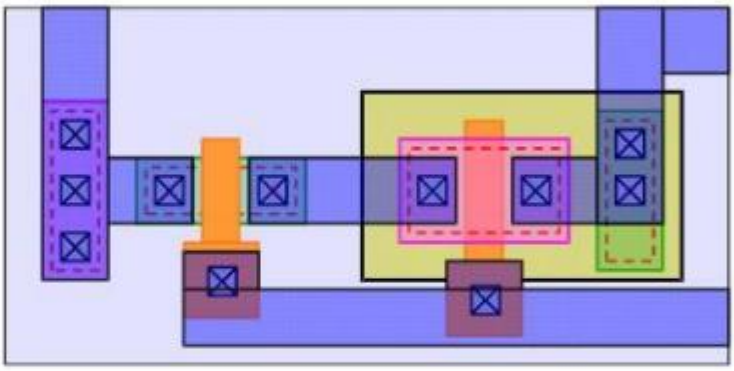


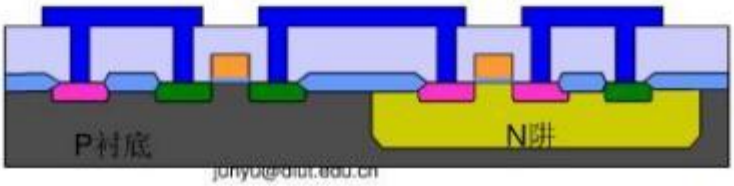
10-3-11 45

Mask 6 contact

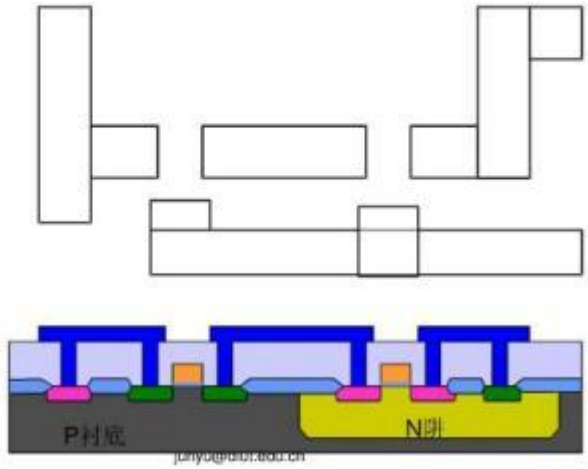


Mask 7 met1



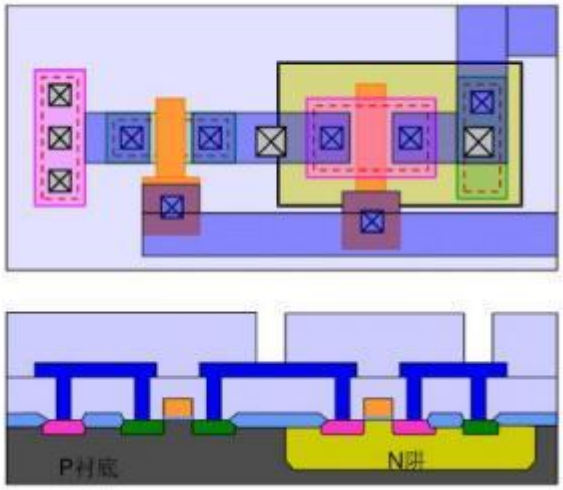


Mask 7 met1



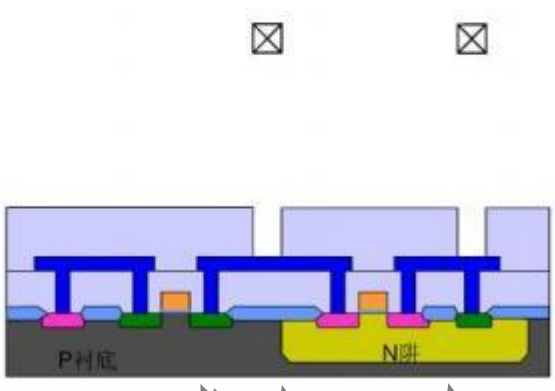
)10-3-11 48

Mask 8 via1



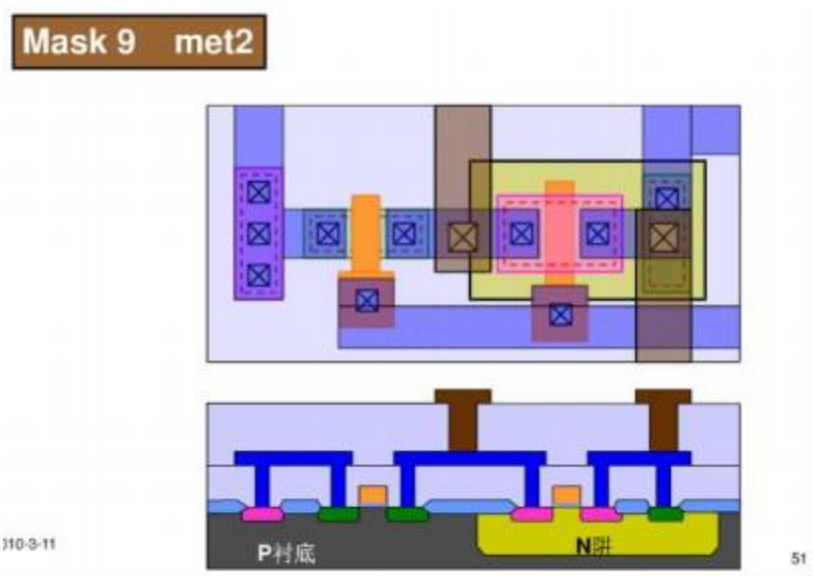
)10-3-11 49

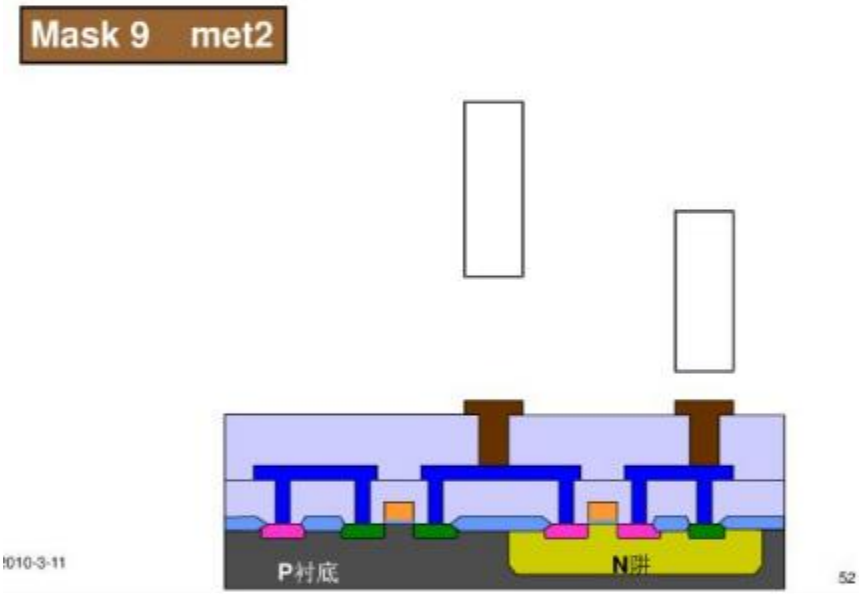
Mask 8 via1

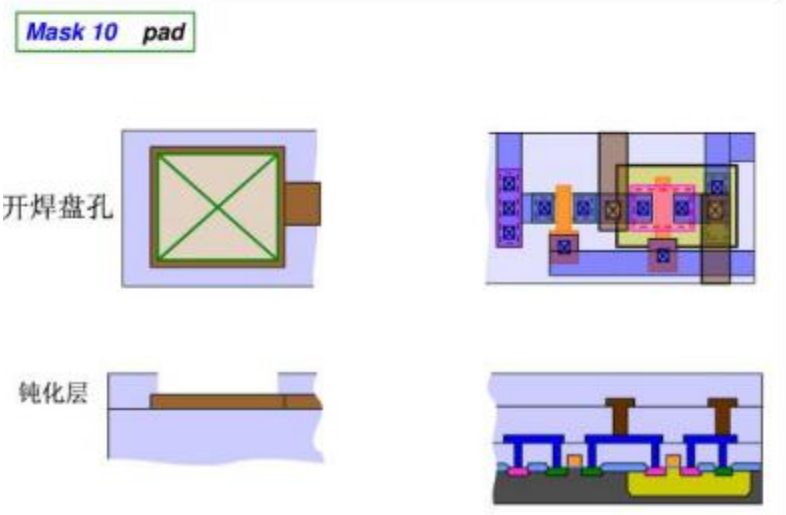


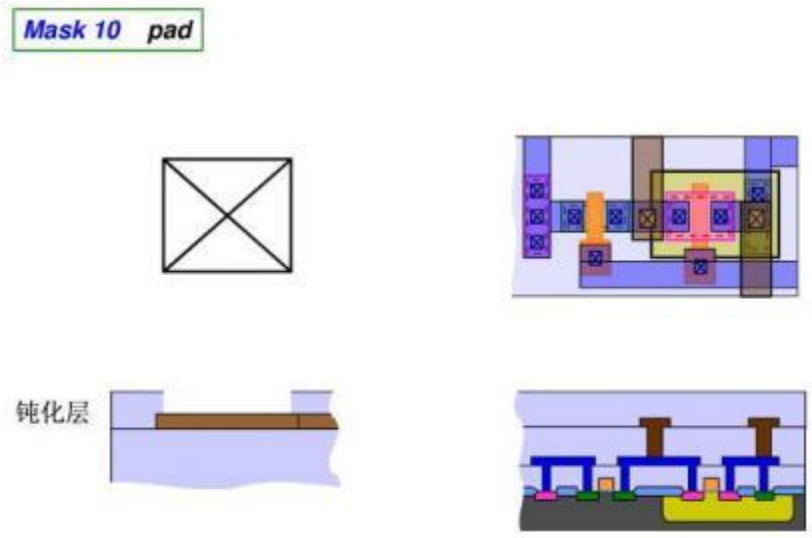
)10-3-11 50

咸鱼:东


Mask 9 met2
P衬底
N阱
51


Mask 9 met2
P衬底
N阱
52


Mask 10 pad
开焊盘孔
钝化层



Synplify 知识

Tcl 使用：

synplify_pro -shell /xxxx_synplify.tcl - 此文件见 linux 下批处理

synplify_pro /xxxx_synplify.tcl --为 gui 界面

Synplify 综合后有几个文件比较重要

.edf 文件 综合出来的网表,给 ISE 使用

Synplify.ucf:根据 sdc 生成的约束文件,给 ISE 使用,如果 synplify 的 sdc 文件中带有管脚约束,那么则 ise 实现时只要 synplify.ucf 一个约束文件就可以了(synplify 使用的约束文件可以把所有的约束条件写入)

.vm 文件:与 edf 文件一样,只是用verilog 生成,可以用来不带时序仿真

Sdc 语法

define_clock {n:clk_osc_fpga_bufg} -name {n:clk_osc_fpga_bufg} -freq {50} -clockgroup {default_clkgroup_0} -rise {0}

define_clock {n:I_xxxx_core.I_sys_ctrl.ClkOsc} -name {n:I_xxxx_core.I_sys_ctrl.ClkOsc} -freq {50} -clockgroup {default_clkgroup_0} -rise {0}

但不在同一个 bufg 上的时钟,它们之间寄存器的的时钟 skew 会很大.

声明端口是上拉或下拉适用于 XC4000 XC5200 和Virtex 其语法如下

.sdc 文件

define_attribute {port_name} xc_pullup {1}

define_attribute {port_name} xc_pulldown {1}

Verilog 中

object /* synthesis xc_pulldown = 1| 0 */ ;

在 Virtex 设计中声明 I/O buffer 的标准 buffer 的可选值如下:BUFGP IBUFG IBUF IOBUF OBUFT OBUF

标准的可选值如下

AGP CTT F_2 F_4 F_6 F_8 F_12

F_16 F_24 GTL GTLP HSTL_I HSTL_III

HSTL_IV LVCMOS2 PCI33_3 PCI33_5 PCI66_3 S_2 S_4

S_6 S_8 S_12 S_16 S_24 SSTL2_I

SSTL2_II SSTL3_I SSTL3_II

其语法如下

.sdc 文件中

define_attribute {port} xc_padtype {buffers_standards}

Verilog 中

object /* synthesis xc_padtype = "buffers_standards" */ ;

define_io_standard {pad} syn_pad_type {lever} syn_io_termination {pullup/pulldown/keeper} 定义 pad 电平及上下拉

在 HDL 源文件中可以加入时间约束，但一般不允许这么做!

// 缩短输入 register_a 的路径延迟 3.0ns

reg register_a /* synthesis syn_reg_input_delay_improve=3.0 */;

// 定义通过 input_a 的路径的输入延迟为 10.0ns

input input_a; //synthesis syn_input_delay=10.0

/* 增加额外的布线延迟 2.0ns 在布局布线中确定的*/

input input_a; //synthesis syn_input_delay_route=2.0

定义时钟：

1 define_clock

语法 define_clock [-disable] clock_name [{-freq MHz|-period ns[{-duty_ns ns| -duty_pct percent}}] [-improve ns] [-route ns]

说明

-improve 用于提高由该时钟控制的寄存器延迟，这个数值可以根据 Synplify 的时间报告中关于相关寄存器的路径延迟的负裕量给出，这是一个高级用户选项

-route 用于提高由该时钟控制的寄存器延迟以满足布局布线，时序报告中的时延与综合时序报告时延的差值，与 Improve 有所不同的是这一栏的数值应填入布局布线工具的时间报告与 Synplify 的时间报告相差的数值，这也是高级用户选项

-duty_ns 和-duty_pct 分别定义用 ns 或百分比说明的时钟占空比

define_clock 的优先级要比 set_option -frequency 高

对内部时钟生成器生成的时钟要用 define_clock 定义时钟名就是输出最终时钟的寄存器

实例名

示例 define_clock clock_a -freq 33.0

define_clock clock_b -period 100.0

define_clock ten_mhz_internal_clock -freq 10.0

2 define_clock_delay

语法 define_clock_delay [-rise | -fall] clock1 [-rise | -fall] clock2 delay

说明：该命令可以定义不同时钟之间的最大延迟，定义两时钟之间是伪路径 delay 的值取

-false 定义占空比不对称即不为 0.5 的时钟

delay 的值是两个沿之间的最小值

//例 2 定义 clka 的上升沿与 clkb 的上升沿的路径为伪路径

define_clock_delay -rise clka -rise clkb -false

3 define_reg_input_delay

语法 define_reg_input_delay register_name [-improve ns] [-route ns]

说明期望输入某寄存器的路径延迟要改善的时间值 register_name 是单比特寄存器而

不能是寄存器总线

例子: define_reg_input_delay reg_a -improve 2.0

4 define_reg_output_delay

语法 define_reg_output_delay register_name [-improve ns] [-route ns]

说明 期望从某寄存器输出的路径延迟要改善的时间值 register_name 是单比特寄存器

而不能是寄存器总线

例子 define_reg_output_delay reg_a -improve 1.0

5 define_input_delay

语法: define_input_delay {input_port_name | -default} ns [-improve ns] [-route ns]

说明定义输入延迟该延迟为信号在片外到达芯片管脚相对于时钟的第一个边沿消耗的时间，无定义时的缺省值是0ns

例子//例 1 对所有的输入端口设置 3ns延迟

define_input_delay -default 3.0

6 define_output_delay

语法 define_output_delay {out_port_name | -default} ns [-improve ns] [-route ns]

说明定义输出延迟该延迟为信号输出芯片管脚到达被驱动的逻辑需要的时间，无定义时的缺省值是0ns

例子 define_output_delay -default 8.0

7 define_multicycle_path -- synpro 不支持

应该使用 set_multicycle_path

语法 define_multicylce_path {-from reg_or_input | -to reg_or_output | -through net } clock_cycles

说明定义多时钟周期路径不要将-through 选项和其他选项一起使用如果用-through 开关选项则需要在源代码使用综合指示 syn_keep 保证关联的信号名始终不会改变

例子 define_multicycle_path -from register_a 2

define_multicycle_path -to register_c 2

8 define_false_path

语法 define_false_path { -from reg_or_input | -to reg_or_output | -through reg }

说明定义在时间分析和优化时可以忽略得路径 不要将-through 选项和其他选项一起使用，如果用-through 开关选项则需要在源代码使用综合指示 syn_keep 保证关联的信号名始终不会改变

例子 define_flase_path -from register_a

define_false_path -to register_c 定义单个寄存器，所有到寄存器 c 的逻辑都是 false path,一般都是定义 2 个时钟域之间为 false path.

9 syn_reference_clock

语法 define_attribute register syn_reference_clock clock

说明该命令用来定义一个参考时钟比如一个寄存器的使能信号每两个时钟周期使能一次则可对该寄存器设置一个参考时钟为原来时钟频率的两倍

例子 define_clock dummy -period 40.0

define_attribute myreg[31:0] syn_reference_clock dummy

对所有使能信号为 en40 的寄存器设置一个参考时钟

define_atrribute {find -reg -enable en40} syn_reference_clock ck2

10. 定义组合逻辑的延时

set_max_delay delay_time -from {} -to {}

set_max_delay

[-from {objectList}]

[-through {objectList} [-through {objectList} ...]]

[-to {objectList}]

delayValue

[-disable]

[-comment commentString]

-to" objects must be of type clock (c: or get_clocks), port (p: or get_ports), inst (i: or get_cells), or pin (t: or get_pins)

所有的组合逻辑到某端口的延时最大为 15ns,约束如下

set_max_delay 15 -to {t: FPFLS032K19DA_i.Imain.wea}

set_max_delay 15 -to {t: FPFLS032K19DA_i.Iinfr.ena}

set_max_delay 15 -to {t: FPFLS032K19DA_i.Imain.addra[*]}

7.2 黑盒时间约束

黑盒是在设计中用综合指示 syn_black_box 声明或实例化的模块对黑盒的时间约束只

能通过在源代码中加入综合指示完成下面是黑盒时间约束

syn_tpdn 定义穿过黑盒的组合逻辑延迟

syn_tsun 定义黑盒的输入延迟

syn_tcon 定义黑盒的输出延迟

n 为 1 10 的整数

用[!]clock 表示 clock 的负沿

例子

```
module ram32x4(z, d, addr, we, clk);
/* synthesis black_box
syn_tpd1="addr[3:0]->z[3:0]=8.0"
syn_tsu1="addr[3:0]->[!]clk=2.0"
syn_tsu2="we->clk=3.0" */
output [3:0] z;
input [3:0] d;
input [3:0] addr;
```

input we;

input clk;

Endmodule

综合属性 Attributes 和指示 Directives)

综合指示用于控制综合中编译阶段的设计分析因而必须加入到源代码中属性是在编译后读入的,因而可以既可以在源程序中说明也可以在约束文件中说明约束文件提供了较大的灵活性可以使得修改约束而不用重新编译源程序因而是强烈推荐采用的方法

在 Verilog 源程序中说明指示或属性采用注释的方法语法如下

// synthesis directive|attribute = "value"

或 /* synthesis directive |attribute = "value" */

XILINX 属性语法---SDC 文件中

syn_noclockbuf :

控制全局时钟Buffer 的自动插入 设属性值为 0 则不插入全局时钟Buffer 其语法如下

.sdc 文件中

define_attribute {clock_port} syn_noclockbuf {1|0}

define_global_attribute syn_noclockbuf {1| 0}

Verilog 中

object /* synthesis syn_noclockbuf = 1| 0 */ ; 不推荐在.v 文件中中添加约束

syn_ pipeline:

在 XC4000 和 Virtex 系列设计中允许将跟在乘法器后面的寄存器移入乘法器实现以提高频率其语法如下

.sdc 文件中

define_attribute {register} syn_pipeline {1|0}

Verilog 中

object /* synthesis syn_pipeline = 1|0 */ ;

syn_ probe：

加入探测点以供测试调试测试点可以使内部信号作为端口在顶层出现

.sdc 文件中

define_attribute {n:netName} syn_probe {0|1}

Verilog 中

object /* synthesis syn_probe = 0|1 */ ;

syn_ ramstyle：

确定设计中RAMs 的实现方法如寄存器或内存块等你可以将这个属性施加于 RAM 驱动的寄存器信号或 RAM 实例名可选值为

registers RAM 由寄存器实现

block_ram 由厂家器件提供的方式实现 RAM

select_ram 由分布在 CLBs 中的 RAMs 实现这是 Xilinx 器件的缺省值

no_rw_check 由 Virtex Block SelectRAM+实现如果是双端口 RAM 将不插入旁路逻辑仅当设计中不会出现同时读写同一地址的情况才可以使用这个属性值在无旁路逻辑的情况下如果同时读写Block SelectRAM+的同一地址输出将是不确定的也会导致 RTL 级仿真和综合后仿真的结果不一致

不过如果确定设计不会出现这种情况可以使用这个值以简化逻辑

这是一个与具体厂家有关的属性参见表 5 其语法如下

.sdc 文件中

define_attribute {signal_name[bit_range]} syn_ramstyle {string}

Verilog 中

object /* synthesis syn_ramstyle = "string" */ ;

syn_reference_clock：

声明一个有别于该寄存器时钟脚上的时钟频率的时钟频率对 XC4000 系列 XC5200 和 Virtex 系列设计有效

syn_useenables：

防止产生带时钟使能端的寄存器语法如下

.sdc 文件中

define attribute {register_instance} syn_seenables {0|1}

Verilog 中

object /* synthesis syn_useenables = 0|1 */ ;

syn_ useioff(Xilinx):

在设计中使用 I/O 触发器只适用于 XC4000 和 Virtex 系列其语法如下

.sdc 文件中

define_attribute {port} syn_useioff {1|0}

define_global_attribute syn_useioff {1|0}

Verilog 中

object /* synthesis syn_useioff = 1|0 */ ;

xc_clockbuftype:

在 Virtex 设计中声明时钟端口使用时钟延迟锁相环 CLKDLL 其语法如下

.sdc 文件中

define_attribute {port} xc_clockbuftype {BUFGDLL}

Verilog 中

object /* synthesis xc_clockbuftype = "BUFGDLL" */ ;

xc_ global_buffers:

控制在设计中使用的全局Buffer 的数量如若在黑盒中使用了全局Buffer 可以用这个属性防止使用的全局 Buffer 数量超过了资源限度，这个属性不等用于源代码中其语法如下

.sdc 文件中

define_global_attribute xc_global_buffers {maximum}

xc_ loc:

声明一个 Module 在 FPGA 上的物理位置用于在模块化流程中综合一个单独的 Module 用行数和列数作为属性值在Virtex2 设计中使用关键字 SLICE 在 Virtex 设计中使用关键字 CLB 语法如下

.sdc 文件中

define_attribute xc_modular_region

{CLB|SLICE_R<n>C<n>:CLB|SLICE_R<n>C<n>}

Verilog 中

object /* xc_modular_region=

"CLB|SLICE_R<n>C<n>:CLB|SLICE_R<n>C<n>" */ ;

xc_ modular_region:

在设计中声明一个设计单元使用 fmap hmap 或 LUT 适用于 XC4000 和 Virtex 系列只能和 xc_uset xc_rloc 一起使用参见 xc_uset 和 xc_rloc 其语法如下

.sdc 文件中

define_attribute {v:primitive_name} xc_map {fmap |hmap |lut}

Verilog 中

object /* synthesis xc_map = "fmap |hmap |lut" */ ;

xc_ map:

声明端口或设计单元的位置关于有效的位置参见Xilinx databook 可以用 xc_loc 声明向量总线端口的片段的位置其语法如下

.sdc 文件中

define_attribute {port_design_name} xc_loc {placements}

Verilog 中

object /* synthesis xc_loc = "placements" */ ;

指定一个总线的所有位的 pad 位置

define_attribute {b[7:0]} xc_loc "P14, P12, P11, P5, P21, P18, P16, P15"

实现对速度的优化

9.1 一般性考虑

在不设置任何约束的情况下进行第一次综合然后

第一步用view log 查看 Timing信息如果与设计目标差距在 5 到 10 可先布局布线看是

否可以满足要求再用 define_reg_input_delay -route

define_reg_output_delay -route 等约束

指导重新综合如果与设计目标差距在 10%到 20%之间可以考虑下面的方法如在 20%以

上的话一般就需要考虑修改设计或是选用更高速的器件了

第二步在工程窗口中设置 synplify 频率设置而不是采用隐含值0MHz

第三步如果设计中存在多个时钟可在约束文件中用 define_clock 定义多个时钟以覆盖

在工程窗口中设置的工作频率

第四步用约束文件适当减少相关路径的fan_out 数目可加快该路径工作速度

第五步如果在设计中用了有限状态机在约束文件中打开 symbolic FSM

Compiler option

第六步用 HDL Analyst 查看综合结果找到关键路径在相应的源代码中加入约束和属

性 define_reg_input_delay -improve define_reg_output_delay -improve

define_input_delay define_output_delay 等对黑匣子进行时间属性约束

syn_tpd

syn_tsu syn_tco 等

9.2 怎样处理关键路径上不满足速度要求的延迟

1 关键路径的起点不是输入端就是寄存器而结束点不是输出端就是寄存器如果起点少

就选择起点添加时间约束和时间属性如果结束点较少就选择结束点添加时间约束和时间属

性

2 对输入端口添加约束如果关键路径的起点是输入端口而且该路径的 slack 时间是

-2.4ns 则可在输入端添加约束 defind_input_delay -improve 2.4

3 如果关键路径的起点是寄存器需对该寄存器添加时间约束和时间属性例如

define_reg_output_delay -improve 2.4

4 如果关键路径的终点是输出端口则可对该该端口添加时间约束

define_output_delay

2.4

5 如果关键路径的终点是寄存器则可添加时间约束

define_reg_input_delay -improve 2.4

6 对添加了约束的设计重新综合查看延迟信息如果时间延迟的超出部分在设计目标的

5 到 10 之间则可考虑布局布线并考虑在布局布线时添加有相应的供应商提供的约束条件

9.3 关于综合约束的建议

设置恰如其分的约束是获得最佳综合结果的关键过严或是过松的约束都达不到最佳的

效果一般可先尝试通用的约束如时钟扇出限制等如果没有达到要求可加入一些严格的

具体约束同时注意放松一些可以放松的约束综合约束的结果是估计值应该以布局布线的

结果为准 Synplify 可以生成布局布线工具使用的约束文件可以充分利用这一功能

9.3.1 时钟

采用同步设计使用尽可能少的时钟可以先尝试使用缺省时钟进行综合如在 project 视窗

或在 project 文件或 Tcl 脚本文件中用 set option -frequency 设定缺省时钟等于最快的时钟若不

能达到要求再使用 define_clock define_clock_delay 详细设置所有时钟同时使用

define_multicycle_path define_false_path syn_reference_clock 放松可以放松的约束使得综合

工具可以充分让关键路径优先使用资源

9.3.2 扇出限制

根据所选器件确定一个恰当的扇出限制扇出过大会使延迟加大扇出过小又会使某些高

扇出网络增加过多的缓冲而造成延迟加大对于高扇出网络综合工具可以通过复制数据源或

插入缓冲降低扇出复制数据源的数量和插入的缓冲数量都会列在 log 文件中最好通过修改设

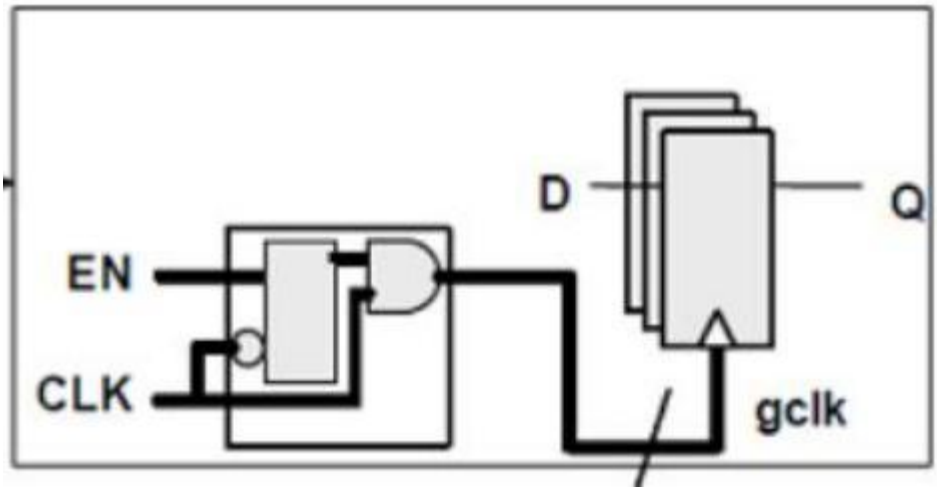
计是负载平衡对于速度要求高又无法平衡的网络可以考虑手工插入全局时钟缓冲作为高扇出缓冲

门控时钟处理

当如此写门控时钟时，FPGA 会自动把把门控移到寄存器的 D 端。下面代码描述的为右边电路

```
module ckgt1d1(CLOCKGATED, ENABLE,  CLOCK);
output CLOCKGATED;
input ENABLE,CLOCK;
wire or_out;
reg EN1;
wire EN1_BUFG;

assign or_out =  ENABLE;

always @(CLOCK or or_out)
 begin
  if(!CLOCK)
   EN1 = or_out;
```

```
 end

assign CLOCKGATED = EN1 & CLOCK;
endmodule
```



对第三方 ip 处理

需要第三方提供对应网表，synplify_pro 配置相关选项
set_option -enable_designware 1
综合工具只有 synplify_premier 支持（据我所知）。其他工具是无法调用 designware 的
designware 在 DC 综合的时候没有多少优化的余地，因为DC 调用的已经是 designware 的版图。仿真的时候都是调用 synopsys 提供的 designware 仿真库，是没办法综合的。

经验总结

如果查看综合后的时钟:
1: 通过 synplify 看电路图---可能有软件版本问题或许不准确
2: 通过时钟报告,里面会报告出有哪些寄存器被优化到时钟树上,那些没优化
3: 通过 synplify 综合后的网表 .vm 文件,查看时钟上的寄存器,看此寄存器是使用哪个时钟,可以查看到生成的时钟是否符合预期

仿真没问题,但实际 fpga 功能却不正常,时钟树又没问题!!这时需要注意端口方向是不是反了,这种情况vcs仿真无法发现.---从生成的.vm 文件是可以看出的可以用 nlint 检查

1．synplify 2012 版本综合问题：时钟识别问题，当前面有一个BUFG时钟，后面又接了BUFG时钟，综合工具会把后面的时钟优化掉，解决方法，在关键真正需要的时钟源作为约束对象，其它的时钟采用组合逻辑实现，BUFG有滤毛刺的功能；

2．当前面一个时钟要作为后面多个时钟的时钟源时，要在每个时钟源前插入一些组合逻辑，比如加 2 级反相器，才能被识别出来，否则将会被优化掉后面的时钟源. ？？

3．在综合工具中，如果约束了串在一起的时钟，而且约束的频率又不同，则会报警告，同时采用最高频率来约束，此时需要在 SDC文件中加入优先级语句来控制，这样才会分段约束

4．当 translate fail 而没任何提示时 ，注意添加 ucf 文件的顺序，可能与添加 ucf 文件的顺序有关，先删除，再一个一个添加。

Fpga 网表仿真

如何仿真 synplify 综合出来的网表 功能性仿真，不带时序？

1.在 VCS 中 需要加入 FPGA 相关的库，一般都是在安装目录下

2.VCS 选项需要+delay_mode_zero,因为 synplify 综合出来的网表.VM 文件有模块内部延时，但却无线延时，如果不加则非常可能会有功能性错误

如下示例

```
ISE_LIB_PATH = /ic01/eda/xilinx/ISE/ISE/verilog/src
FPGA_POST_CMP +=   $(ISE_LIB_PATH)/glbl.v \
                   -y $(ISE_LIB_PATH)/iSE \
                   -y $(ISE_LIB_PATH)/simprims \
                   -y $(ISE_LIB_PATH)/uni9000 \
                   -y $(ISE_LIB_PATH)/unimacro \
                   -y $(ISE_LIB_PATH)/unisims \
                   -y $(ISE_LIB_PATH)/XilinxCoreLib\
                   +delay_mode_zero
```

7.如何把第三方的网表与rtl 一起综合?

需要对方使用对应的库综合出来的网表,与 synpro 综合后的.vm 文件类似.然后与rtl 一起读入,按照正常方法综合就可以了

SDC 常用语法

时钟约束

define_global_attribute syn_noclockbuf {0}　　不自动插时钟buffer—时钟不够时可以指定那些时钟使用时钟buffer

define_clock {n:clk_osc_fpga_bufg} -name {n:clk_osc_fpga_bufg} -freq {48} -clockgroup {default_clkgroup_0} -rise {0}

define_clock {n:x.x.TCK} -name {n:xx.x.TCK} -freq {10} -clockgroup {default_clkgroup_1}

define_clock {n:x.xl.ClkOsc} -name {n:xx.xx.ClkOsc} -freq {48} -clockgroup {default_clkgroup_2} -rise {0}

define_clock_delay -rise {clk_osc_fpga_bufg} -rise {x.x.ClkOsc800k} -false　设置 false path

create_clock -name {inst_clk_wiz_v3_6_clkout2} [get_nets {n:inst_clk_wiz_v3_6.clkout2}] -period {100.0} -waveform {0 50.0}

位置约束

define_attribute {port_design_name} xc_loc {placements}
define_attribute {b[7:0]} xc_loc "P14, P12, P11, P5, P21, P18, P16, P15"

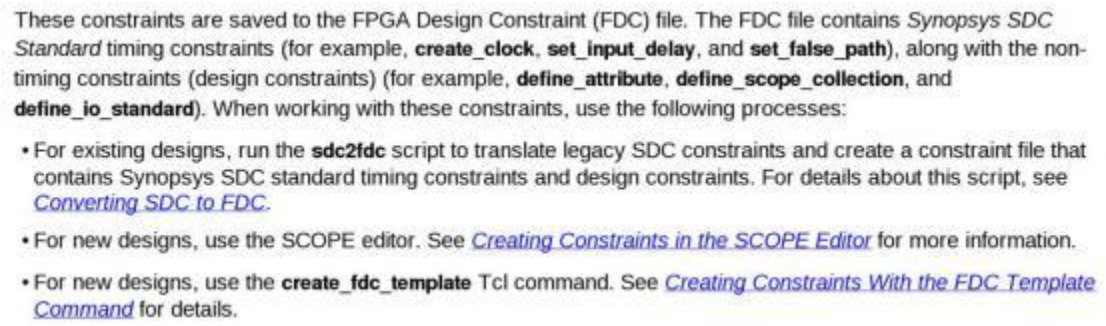
上下拉约束

```
define_attribute {port_name} xc_pullup {1}
define_attribute {port_name} xc_pulldown {1}
```

PAD 类型电平约束

```
define_attribute {port} xc_padtype {buffers_standards}
```

sdc 与 fdc 的关系



These constraints are saved to the FPGA Design Constraint (FDC) file. The FDC file contains *Synopsys SDC Standard* timing constraints (for example, **create_clock**, **set_input_delay**, and **set_false_path**), along with the non-timing constraints (design constraints) (for example, **define_attribute**, **define_scope_collection**, and **define_io_standard**). When working with these constraints, use the following processes:

- For existing designs, run the **sdc2fdc** script to translate legacy SDC constraints and create a constraint file that contains Synopsys SDC standard timing constraints and design constraints. For details about this script, see *Converting SDC to FDC*.
- For new designs, use the SCOPE editor. See *Creating Constraints in the SCOPE Editor* for more information.
- For new designs, use the **create_fdc_template** Tcl command. See *Creating Constraints With the FDC Template Command* for details.

Sdc 是 synopsys 的约束文件，synpro 会自动保存一个叫 fdc 的文件，在 ise 实现时可以加入 fdc 文件
Fpga 约束包含时序约束和设计属性约束（非时序约束），使用 define_attribute，define_xxx 等命令
常用命令如下：

SCOPE Panel	See ...	Tcl Commands
Clocks	Clocks	create_clock set_clock_groups set_clock_latency set_clock_uncertainty
Generated Clocks	Generated Clocks	create_generated_clock
Collections	Collections	define_scope_collection
Inputs/Outputs	Inputs/Outputs	set_input_delay set_output_delay
Registers	Registers	set_reg_input_delay set_reg_output_delay
Delay Paths	Delay Paths	set_false_path set_max_delay set_multicycle_path
Attributes	Attributes	define_attribute define_global_attribute
I/O Standards	I/O Standards	define_io_standard
Compile Points	Compile Points	define_compile_point define_current_design
TCL View	TCL View	--

Linux 下批处理

synplify_pro -batch scr/prjxxx.tcl ;

prjxxx.tcl 内容如下

```
set version [clock format [clock seconds] -format {%Y%m%d}]
set  rtTopDesignName prjxxx
set rtImplName rev_synplify_${version}_v6
## Set up the project and the implementation name.
set rtSynDir [pwd]
project -new [file join . $rtTopDesignName]
```

```
impl -add $rtImplName
impl -active $rtImplName
## Add the source files.
set rtOutputStage [project -dir]
run_tcl -fg /xxxx/prjxxx_fpga_rtl_read.tcl      读取 rtl

## Select the device.
set_option -technology VIRTEX6
set_option -part XC6VLX240T
set_option -speed_grade -1
set_option -package FF1156

# Set the result file and format.
project -result_format  edif
project -result_file  prjxxx.edf

# compilation/mapping options
set_option -top_module ${rtTopDesignName}
set_option -resource_sharing 1

# map options
set_option -pipe 0
set_option -retiming 0
set_option -write_verilog 1
set_option -write_vhdl 0
set_option -maxfan 20

# implementation attributes
set_option -enable_designware 1
```

```
# Add the timing constraint files.
set rtSdcFiles {
  scr/prjxxx_synplify_v6.fdc
}
foreach rtSdcFile $rtSdcFiles { add_file -constraint $rtSdcFile}
//加约束文件
set res [project -run compile]   //编译结果
project -run constraint_check
if {$res != 2} {                 //返回 0，无错误和警告；1，无错误，有警告；2：有错误
  project -run synthesis
}

project -save

cd $rtSynDir
file delete -force latest_v6
exec  ln  -s  $rtImplName latest_v6
puts  stdout  "Thank you"
exit
```

/xxxx/prjxxx_fpga_rtl_read.tcl 格式

```
set_option -hdl_define -set "i2c_inc bb_mul rvt UART UDLY=0 fpga functional FPGA "  //加入 define 宏定义
set_option -include_path "/xxx /verilog/"   增加文件路径
set_option -include_path "/xxxx1/verilog/"
add_file -verilog /xxxx/xx1v.v   增加文件
add_file -verilog /xxxx/xx2.v
```

。。。。。。。。。

```
add_file -verilog /xxxx/xxx.v
```

插入 bufg 与 define_clock

插入 bufg 是一个物理过程，就是把某根信号线连接到bufg（本质是一个时钟输入输出的缓冲器）这个模块上，bufg 的输出可以连接到对应区域的所有寄存器的时钟上，而且到所有的寄存器的 skew 非常小。一般每个 bufg 都会对应一个 define_clock 来约束时序。

最好不要让工具自动插入 bufg，因为它可能把一些低速的时钟也加入 bufg，而导致布线困难

禁止自动插入 bufg 的语法如下：

```
define_global_attribute syn_noclockbuf {0}
  define_attribute {clock_port} syn_noclockbuf {1|0} //指定单个时钟不插入 bufg
  define_global_attribute syn_noclockbuf {1| 0}
```

也可以让工具自动加入 bufg，但不能超过所选 FPGA 拥有的最大 bufg 数量

```
  define_global_attribute xc_global_buffers {maximum}
```

Define_clock 是对某一个时钟约束，也就是对某一个 bufg 上所有连着的寄存器按照 define_clock 的参数做时序检查/约束，如果两个 define_clock 之间有逻辑关系，及 clock a 寄存器的 q 端直接或间接连到了 clock b 寄存器的 d 端，如果这两个时钟没有定义 false path，那么他们就会做时序检查/约束，工具会根据两个时钟的周期，时钟延迟等各种物理信息进行布线来努力满足时序，由于每个 bufg 的时钟之间的 skew 可能会比较大（有些甚至达到 10 几 ns）,那么将很难满足时序，这时工具优化会花很长时间，还可能会导致其它关键路径无法满足时序，所以如果能确定两个时钟是 false path，则在约束文件中指明，那么工具就不会去优化两个时域寄存器中间的组合逻辑了。

定义时钟的语法可以如下：

```
  define_clock {n:clk_osc_fpga_bufg} -name {n:clk_osc_fpga_bufg}
  -freq {48} -clockgroup {default_clkgroup_0} -rise {0}


  define_clock {n:x.x.TCK} -name {n:xx.x.TCK} -freq {10} -clockgroup
  {default_clkgroup_1}
```

工具做时序约束优化与 bufg 无关，只与实际物理连接相关，只要组合逻辑在两个寄存器中间，如果没指明 false path 就会做约束优化，并出具报告，如果没有 define clock（或者是遗漏的寄存器时钟没有定义）,工具会给一个默认的时钟周期。
定义 false path 的语法可以为：
define_clock_delay -rise {clk_osc_fpga_bufg} -rise {x.x.ClkOsc800k} -false

在综合过程中，如果对 bufg 后面的时钟做了分频或者 gate（设计上是作为一个时钟去驱动寄存器），而且没有使用另外一个 bufg 且没有 define_clock，那么在实际综合过程中，后面这些寄存器的时钟会使用bufg 的时钟，而分频或 gate 逻辑会转化为控制寄存器的CE(clock enable)端。本身在 D 端的逻辑还是在 D 端处理。但如果做了如下 syn_seenables 配置，那么情况就不同了。

在综合时，有一个控制选项，可以禁止产生带时钟使能端的寄存器，如下：

syn_seenables:，防止产生带时钟使能端的寄存器，由于 asic 很多 generate clk，它与master 为同步时钟，如果转 FPGA 版本加了此选项，那么master clk 与 generate clk 的 skew 会很大（至少比 asic 的会大），那么要满足时序将会比较困难或者工具需要特别长的时间才能完成布线。

语法如下：

Define_attribute {register_instance} syn_seenables {0|1}
define_global_attribute syn_seenables {0|1}
如果要把 gate clk 和 generate clk 转换为门控时钟，需要在工具上加入下面选项
set_option -fixgatedclocks 3
set_option -fixgeneratedclocks 3

fpga 中寄存器的种类：
FD:基本的寄存器，只有三个端口：Q,D,C
E: CLK ENABLE 时钟使能
C: clr 异步复位
P: PRESET 置位 异步
R: 同步复位 reset
S; 同步置位
FD 没有复位，置位的寄存器

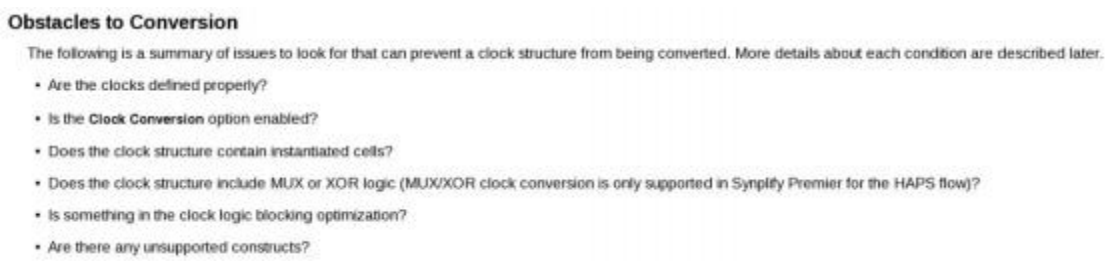
```
  FD \OutReg_0_DOUT_Z[0]  (
    .Q(OutReg_0_DOUT[0]),
    .D(OutReg_0_tmp_d_array_0[0]),
    .C(clk_tcm2)
);
```

FDC 带有 CLR 的寄存器，异步

```
  FDC \slow_sig_Z[2]  (
    .Q(slow_sig[2]),
    .D(slow_sig[1]),
    .C(clk_osc_fpga_bufg),
    .CLR(POR_SRC_Sync_i)
);
FDCE            带有 CE 端的寄存器
  FDCE mcu_in_pd_Z  (
    .Q(mcu_in_pd),
    .D(wakeup_lp_i),
    .C(ClkOscS),
    .CLR(ResetnPor_osc2_i),
    .CE(un2_next_mcu_in_pd_i_0_0)
);
FDCE_1          带有 CE 端的寄存器
  FDCE_1 mcu_pd_neg_Z  (
    .Q(mcu_pd_neg),
    .D(pd_mode),
    .C(ClkOsc_c),
    .CLR(resetn_c_i),
    .CE(ECK_f_f1)
);
FDC_1           带有 CLR 的寄存器，异步复位
  FDC_1 \pch_neg_Z[1]  (
    .Q(pch_neg[1]),
    .D(pch_neg[0]),
    .C(ClkOsc_in_mux),
    .CLR(POR_SRC_Sync2_i)
);
FDE      带有时钟使能寄存器
  FDE \dg_data_Z[12]  (
    .Q(dg_data[12]),
    .D(dg_data_202[12]),
    .C(TCK),
    .CE(resetn_c)
);
FDP             带有 PRE 端口寄存器，异步置位
  FDP \dbg_rstn_reg_Z[1]  (
    .Q(dbg_rstn_reg[1]),
    .D(dbg_rstn_reg[0]),
    .C(ClkOsc_c),
    .PRE(ResetnChip_osc_i)
);
FDPE            带有 PRE,CE 端口寄存器
```

```
  FDPE crc_hold_osc_Z  (
    .Q(crc_hold_osc),
    .D(GND),
    .C(ClkOsc_c),
    .PRE(resetn_c_i),
    .CE(crc_hold_oscce)
);
FDPE_1          带有 PRE,CE 端口寄存器
  FDPE_1 osc_en2a_Z  (
    .Q(osc_en2a),
    .D(next_osc_en2a),
    .C(ClkOsc_c),
    .PRE(ResetnOscEn2_i),
    .CE(ECK_f_f1)
);
FDP_1
  FDP_1 \data1_Z[0]   (
    .Q(data1[0]),
    .D(N_9_i),
    .C(bs_clk),
    .PRE(resetn_c_i)
);
FDR      同步复位
  FDR EN_ff_Z  (
    .Q(ECK_f_f1),
    .D(EN_ffc_i),
    .C(ClkOsc_c),
    .R(en_clk_mcu_i)
);
```

转换门控时钟的条件,要满足下面条件



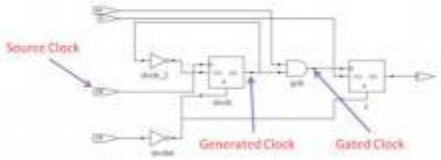
时钟正确定义

Defining Clocks Properly

The most common problems preventing clock conversion from occurring automatically are related to improperly defined clocks. When defining gated and generated clocks:

- Gated clocks should be defined at the nodes to be connected to the clock pins of the sequential cells.
- Use **create_generated_clock** constraints to define the relationship between generated clocks and their sources
- If a clock circuit contains both a clock generator and a gating element, a **create_generated_clock** constraint is needed. Without this constraint, the tool does not know the relationship between the gated signal and the source clock.

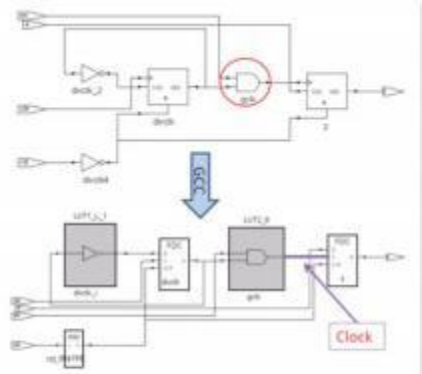
Defining Clocks Example



Using the above example, the first constraint applied is:

```
create_clock -name clk [get_ports clk] -period 10
```

In the above constraint, **get_ports** identifies **clk** as a port. With only the source clock defined, the tool is unable to determine which input of the AND gate is the clock and which input is the enable. Accordingly, no conversion is performed as shown in the following schematic diagram.

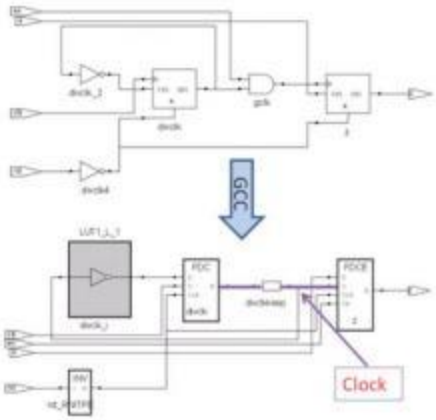


Continuing with the example, a second constraint is added:

```
create_clock -name clk [get_ports clk] -period 10
create_clock -name divclk [get_nets {divclk}] -period 20
```

In the second constraint, **get_nets** identifies **divclk** as a net. With independent clock constraints defining the source and generated clocks, the tool now knows which input of the AND gate is the clock, but still does not know the relationship between the source and generated clocks. In this case, the clock gate and data register 'Z' are converted to an enable flip-flop (FDCE) which is connected to the generated clock as shown in the next schematic.

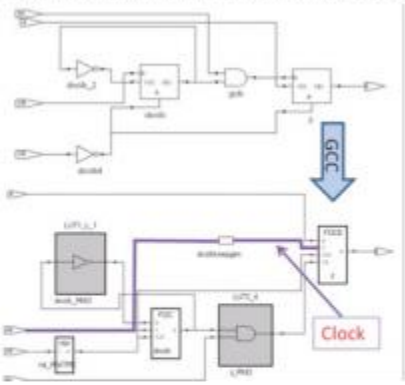
上面的一定都无法正确生成 igc 电路，只有下面的约束才能正确产生预期电路



For the final example, the second clock constraint is replaced with a generated clock constraint.

```
create_clock -name clk [get_ports clk] -period 10

create_generated_clock -name divclk [get_nets {divclk}]
 -source [get_ports clk] -divide_by 2
```

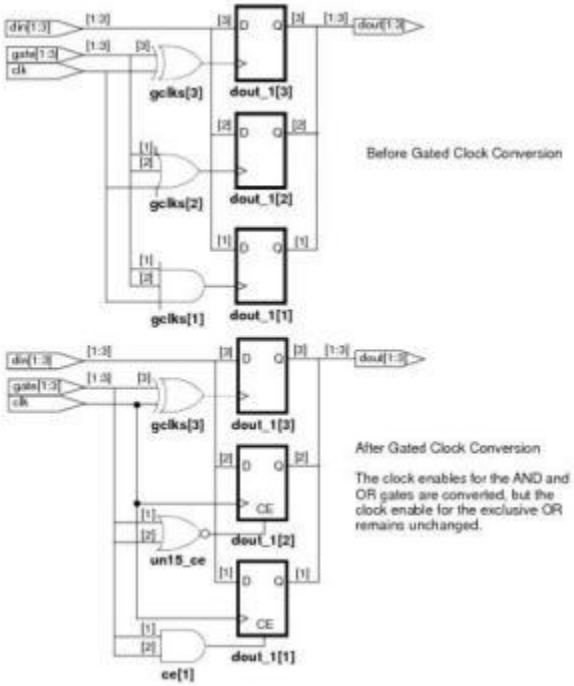
The tool knows which input of the AND gate is the clock and the relationship between the source and generated clocks. For this case, the entire clock circuit is converted to an enable on the 'Z' register, and the 'Z' register clock pin is connected directly to the source clock as shown in the final schematic.



For a gated clock to be converted successfully, the design must meet these requirements:

Condition	Description
Combinational logic only	The gated clock logic must consist only of combinational logic. A derived clock that is the output of a register is not converted.
Single base clock	Identify only one input to the combinational logic for the gated clock as a base clock. To identify a net as a clock, specify a period or frequency constraint for either the gate or the clock in the constraint file. This example defines the clk input as the base clock: `create_clock -name {clk} -freq 10.000 -clockgroup default_clkgroup`
Supported primitives	The sequential primitive clocked by the gated clock must be a supported object. The tools support gated-clock conversion for most sequential primitives. Black-box modules driven by gated clocks can be converted if special synthesis directives are used to define the clock and clock enable inputs to the black box. See *Using Gated Clocks for Black Boxes*.
Correct logic format	See *Correct Logic Format* for an example of the correct logic format.

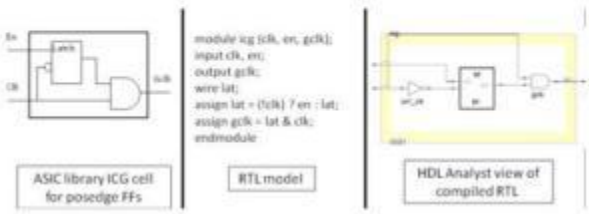
正确是逻辑格式指的是最后于 clk 相关的逻辑必须是与门/或门；



Asic 转 fpga 的 igc 电路设计

Integrated Clock Gating Cells

The Synopsys synthesis tools do not read in ASIC cell libraries. To support conversion of Integrated Clock Gating (IGC) cells, these cells must be modeled in RTL.

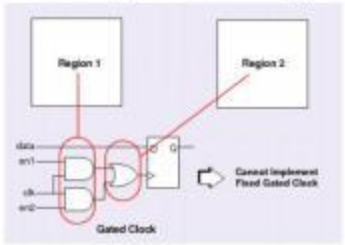


下面情况无法产生 icg 电路

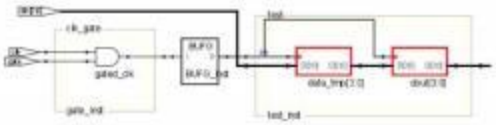
1.

2. Bufg 前面的 gate 不会被转化

- A **syn_keep** attribute attached to a net, which should preserve the net during optimization, is not honored by the **Clock Conversion** option.
- The **Clock Conversion** option cannot be implemented for inferred counters in Altera technologies.
- The **Clock Conversion** option cannot be implemented by the Synplify Premier tool if the gates associated with the gated clock are assigned to different design plan regions (see the following figure). The **Clock Conversion** option *can* be applied if all gates associated with the gated clock are assigned to the same design plan region. Also, the flip-flops can be assigned to any Synplify Premier region.



- A global buffer, when instantiated on a gated clock, blocks gated-clock conversion from being performed (the buffer is viewed as the clock source by the downstream circuitry). In the circuit below, the **clk** input to **test_inst** remains gated.



Similarly, defining a clock constraint at the output of a gated clock prevents conversion from continuing upstream.

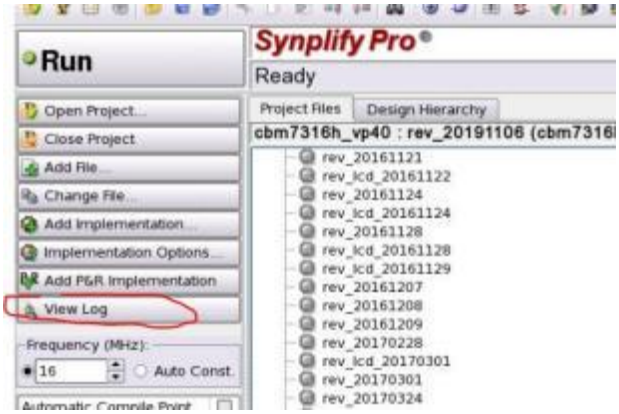
上面只是手册上的说明，但实际在曾经的项目中并没有严格按照要求设计（没有使用 generate clock），也转化成功,但 icg 要按照要求设计，分频时钟被成功转化为寄存器的 CE 端（clock enable）.

看 gate/generate 转化报告

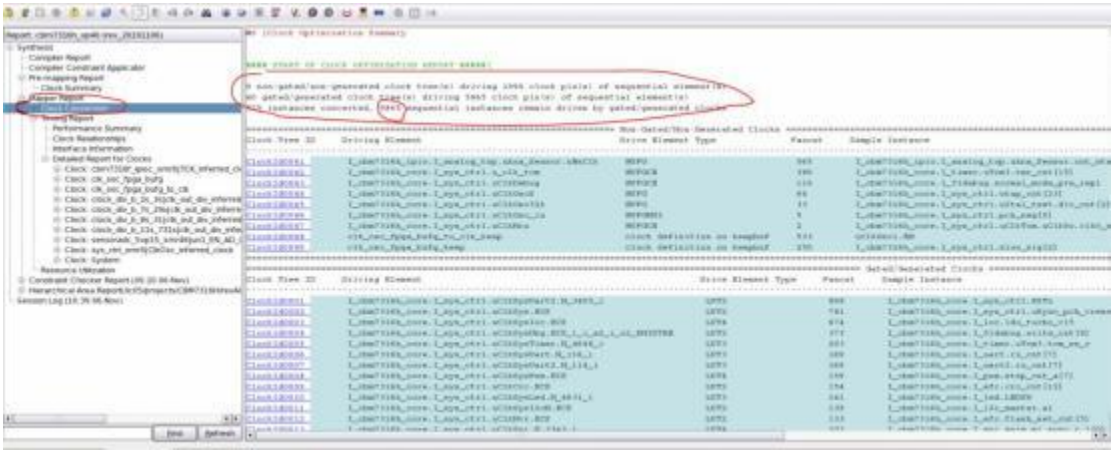
格式如下

When clock conversion is enabled, a typical report resembles the following:

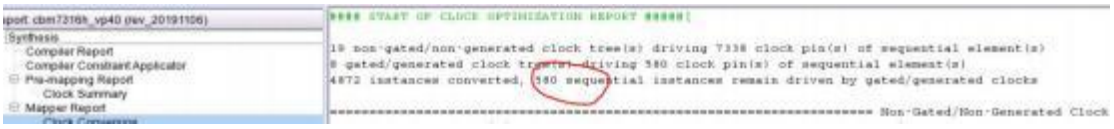
下面实际的报告
在 gui 界面点击 view log



进入下面界面



上面的结果由于约束不正确，有 5869 个寄存器的时钟没转换成功！下面是约束正确只有 580 个没转化成功。



FPGA 知识

由于 FPGA 的时钟树固定,在 pr 时需要避开时钟线,所以pr 后的走线延时会很大,比逻辑延时大很多.

1. 当使用的时钟不是全局时钟时，而 synpilfy 综合时会默认当做全局时钟而插入 IBUFG，则在实现时会报放的位置错误，这时需要在 ucf 文件中声明此时钟不是专用时钟

PIN “CLK_PATH” CLOCK_DEDICATED_ROUTE = FALSE ;

2. 第二全局时钟资源也叫长线资源，它是分布在芯片的行、列的栅栏(Bank)上，一般采用铜、铝工艺，其长度和驱动能力仅次于全局时钟资源。与全局时钟相似，第二全局时钟资源直接同 IOB、CLB、选择性块 RAM 等逻辑单元连接，第二全局时钟信号的驱动能力和时钟抖动延迟等指标仅次于全局时钟信号。Xilinx 的 FPGA 中一般有比较丰富的第二全局时钟资源(很多器件有 24 个第二全局时钟资源)，以满足高速、复杂时序逻辑设计的需要。

在设计中一般将频率较高，扇出数目较多的时钟、使能、高速路径信号指定为第二全局时钟信号。第二全局时钟资源的使用方法比较简单，一般是在约

束编辑器的专用约束(Misc)选项卡中指定所选信号使用低抖动延迟资源“Low Skew”。也可以直接在指导 Xilinx 实现步骤的用户约束文件(UCF)中添“USELOWSKEWLINES”约束命令。上面在约束编辑器中的操作等效于在用户约束文件中添加如下内容：

NET "sum<0>" USELOWSKEWLINES;

NET "sum<1>" USELOWSKEWLINES;

NET "sum<2>" USELOWSKEWLINES;

全局时钟资源是专用布线资源，它存在于全铜布线层上，用户使用全局时钟资源并不会影响芯片的其它布线资源。而相对于全局时钟资源，第二全局时钟资源与之不同，它使用的是芯片内部的布线资源。在设计的密度非常大(面积利用率高于 90%)的情况下，使用所有的第二全局时钟资源会给设计的布局布线带来负面影响，可能导致芯片布线不成功，需要谨慎使用。

3. ASIC 的 PAD 有复杂的控制信号，而 FPGA 的 PAD 都是在实现时设定上下拉，对于单向的 PAD 只需直接赋值即可，双向PAD 需要做一些处理

OUT C;//输出给 FPGA 逻辑

IN I; //来自 FPGA 逻辑信号

IN ENO; //来自FPGA 逻辑信号,1 表示输出使能，0 表示输入

INOUT PAD;//FPGA 端口

Assign C =PAD;

Assign PAD = ENO ?I : Z ;

对于 OD ，OC 之类的控制可以做如下设计

IN OD;

Assign C =PAD;

Assign PAD = ENO&OD&I ? 1'bz: (ENO?I:1'bz);当配置为输出，OD，且输出为 1 时，需要外部上拉，本身 PAD 输出为 Z，否则按照正常输出控制

4. sram 时序

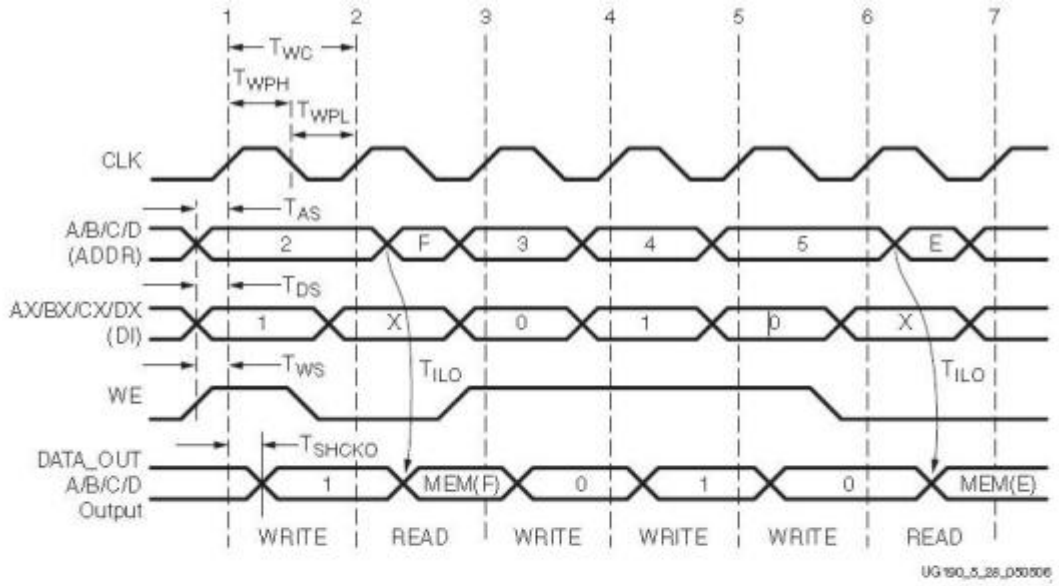


图 5-28: Slice 分布式 RAM 时序特性

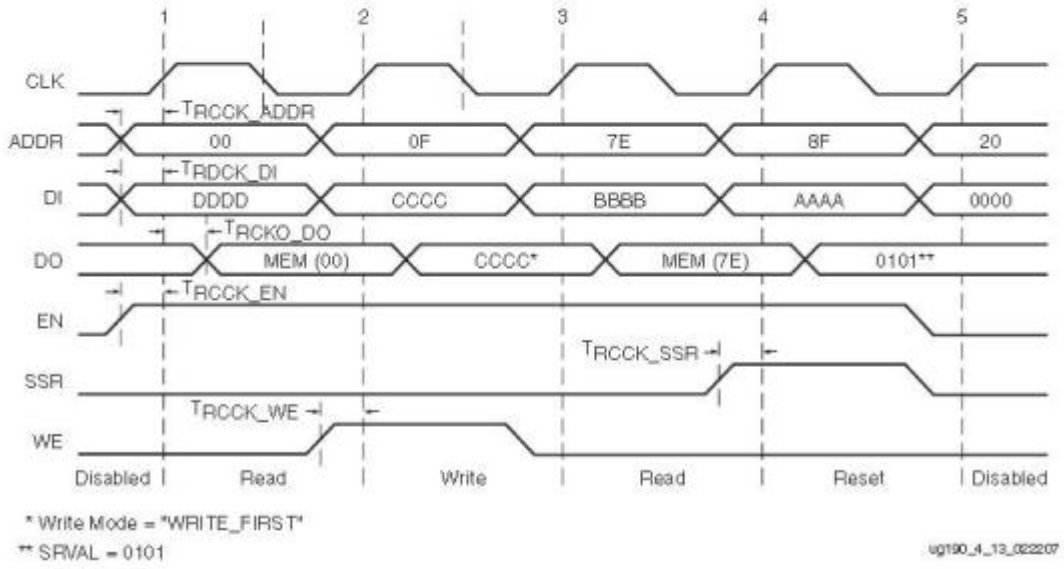


图 4-14: Block RAM 时序图

关于 SRAM 的特性：

1. first_write:写入数据的同时，会把写入的数据存入输出端口；可能出现的问题：当写入数据后，如果不选择此 RAM，再读，现象为读出的数据为写入的数据

2. read_first:写数据时，先把本地址以前的数据先读出到端口，然后在把新数据写入

3. no_change:写数据时，不会影响输出端口数据，输出端口数据一致保持上次读出的数据

可以看出如果需要分布式 RAM 与Block RAM 的时序一致，则分布式 RAM 需要选择 register 输出,分布式具有寄存器的特性,数据当拍给出.

局部时钟的使用:典型时钟区域中有四个 clock-capable I/O 引脚对（中心列有例外）。Clock-capable I/O 对是选择位置中的常规 I/O 对，与附近区域的时钟资源具有特殊硬件连接。有些全局时钟输入也是 clock capable I/O。每个组中有四个专用 clock capable I/O 区。当用作时钟输入时，clock-capable 引脚可以驱动 BUFIO 和 BUFR。这些引脚不能直接连接到全局时钟缓冲器。如果用作单端时钟引脚，则如“全局时钟缓冲器”中所述，必须使用引脚对的 P 端，因为只有这一引脚上存在直接连接。名字如:IO_L9N_CC_5

区域时钟网的传播并非遍及整个 Virtex-5 器件，而是仅限于一个时钟区域。一个时钟区域包含四个独立的区域时钟网。要接入区域时钟网，BUFR 必须例化。一个 BUFR 最多可以驱动两个相邻时钟区域中的区域时钟（图 1-23）。顶部或底部区域中的 BUFR 只能相应接入下边或上边的一个相邻区域。左侧的 BUFR 可以向中心列 I/O 馈送信号。

Clock Capable I/O:输入时钟到局部时钟网络;

I/O 时钟缓冲器 - BUFIO:时钟输入信号通过此缓冲器连接到 IO 时序单元,每个 BUFIO 可驱动同一区域/ 组中的一个 I/O 时钟网络，区域时钟缓冲器 (BUFR) 也是如此。BUFIO 不能驱动逻辑资源（CLB、Block RAM 等），因为 I/O 时钟网络只能覆盖同一组或时钟区域内的 I/O 列。

区域时钟缓冲器 - BUFR:区域时钟缓冲器 (BUFR) 是可以在Virtex-5 器件中使用的另一种时钟缓冲器。BUFR 将时钟信号驱动到时钟区域内一个独立于全局时钟树的专用时钟网。每个 BUFR 可以驱动其所在区域中的四个区域时钟和相邻区域（最多三个时钟区域）中的四个时钟网。与 BUFIO 不同，BUFR 不仅可以驱动其所在时钟区域和相邻时钟区域中的 I/O 逻辑，还可以驱动其中

的逻辑资源（CLB、Block RAM 等）。BUFR 可由 clock capable 引脚或局部互连驱动。另外，BUFR 能够根据时钟输入生成分频时钟输出。分频值为一到八之间的整数。对于要求

跨时钟域或串并转换的源同步应用来说，BUFR 是理想的。典型的时钟区域（四个区域时钟网络）中有两个 BUFR。中心列没有 BUFR。BUFR 中的时钟分频是在软件中通过 BUFR_DIVIDE 属性进行控制;对于

BUFR_DIVIDE = 1 和 BUFR_DIVIDE = BYPASS，通过 BUFR 的传播延迟不同

全局时钟线仅由一个全局时钟缓冲器驱动，即一个 BUFG 驱动一根全局时钟线

一个全局时钟缓冲器常常由一个时钟管理模块 (CMT) 驱动，以消除时钟分配延迟或调整其相对于另一时钟的延迟。全局时钟多于 CMT，但一个 CMT 常常可驱动多个全局时钟。

布线资源：

1． 局部布线: Virtex 系列的局部布线有三种方式的连接,即: a.LUT、触发器、常规布线矩阵(GRM)之间的连接; b.CLB 内部把同一个 CLB 的输出反馈回来作为自身LUT 输入的连接,这种连接提供了最小的延时;c.在相邻的 CLB 间提供水平连接,这种连接消除了信号在 GRM 中的延迟

2.常规布线: Virtex 系列中在 CLB 的通道中,水平与竖直方向各有 24 根单长线连入 GRM 中,通过 GRM 中的可编程开关实现是否互连,以实现 CLB 间的互连;四个方向各有 12 条带缓冲的 16 进制线每跨过 6 个 CLB 从一个 GRM 汇入下一下 GRM;每个 CLB 分别在水平与竖直方向相邻着 12 根带缓冲的双向的长线,长线的长度与 CLB 区域的长或宽相同

2. IO 布线:这是一种在芯片的周围附加的布线资源,它提供了 CLB 阵列与 IOB 的交换界面,称为多功能环,同时,除了能把 IOB 与 CLB 连接起来之外,还可以实现引腿之间信号的交换。

3.直接连线:一些信号需要专门的连线来达到最优性能的实现,直接

连线便实现了这一目的,在 Virtex 芯片中,有两类这样的信号:在两个独立网络中,每个 CLB 传输进位信号给相邻的 CLB;水平布线资源在一个通道内提供 4 条三态总线来与 CLB 交换信号

1. 全局布线:它为时钟及其实遍及整个芯片的多扇出信号提供分配通路。

Virtex 包括两类全局布线资源:主全局布线与次全局布线。主全局布线由 4 个以最小摆率来分配高扇出时钟信号的有专门输入引脚的全局网络构成。次全局布线包括 24 条主干线, 12 条穿过芯片上端, 12 条通过芯片下端,通过这些线,每一个 CLB 列中可以有 12 条信号分布式地通过这列中的 12 条长线。

经验总结

做 fpga 综合前最好做一下 nlint 检查,因为仿真无法发现 input,output 定义反了的错误,但综合会产生错误的电路,在实际的 fpga 板子上功能是错的---此种情况定位非常困难.

0. FPGA 中通过例化 BUFG 把时钟接入全局时钟，通过 BUFR 把时钟接入区域时钟（前提是 fpga 资源右区域时钟）

1. FPGA 中 RAM 块时钟 不会产生门控时钟 ，所以在做 SOC 系统时，它的时钟必须与系统时钟等同，中间不能有任何逻辑 ，否则它们 skew 太大无法满足时序,把门控时钟逻辑转化到 ce/wr/rd 等端口

2. 一个时钟如果来自一个 mux,那么这个时钟所驱动的寄存器只会根据当前的时钟形成门控时钟，不会根据 mux 之前的时钟形成门控寄存器，即，这两类寄存器是两个时钟，且两个时钟有一个 delay,如果这两个寄存器之间做为同步时钟处理，则很麻烦，因为他们两个时钟之间的相位关系无法控制，延时完全根据工具而定，很难约束，ASIC 可以人为产生时钟树，而 FPGA 是固定的时钟树，无法处理

3. 对于分频出来的时钟,,那么这个分频出来的时钟可能并不会优化掉(估计实现不了 使用源时钟作为后面寄存器的时钟,分频逻辑移到寄存器 D 端),而是作为一个单独的时钟出现,需要手动加 BUFG 或设置自动插入 BUFG;----分频时钟会作为一个独立的时钟;

```
clock_div_a #(3) uClkLediDiv  (
        .clk            ( ClkOscS       ),
        .resetn         ( ResetnPor_osc2),
        .div            ( led_div       ),
        .gate           ( gate_clk_led  )
        );
ICG_V uClkLed  ( .E(gate_clk_led &
en_clk_led_sync), .CK(ClkOscS), .TE(scan_mode), .ECK(clk_led) );
```

这种做法的分频时钟,clk_led 所驱动的寄存器,全部会转化为ClkOscS 时钟驱动;

在 fpga 中时钟的 gata 信号会转化在寄存器的 D 端或 ce 端

在 asic 中,寄存器的 D 端逻辑很可能转化到时钟的 gata 信号---看代码风格

3. FPGA 中 ，所有的时钟默认都是同一个时钟域，所以需要根据实际情况来做时钟域分组，指定哪些时钟时异步时钟处理，如果有遗漏则会安装同步来处理;

约束文件定义的时钟，只是用来告诉工具该时钟域的寄存器按照某个频率约束，与同步时钟异步时钟无任何关系，

4. 约束 false path:TIMESPEC TS_clkflash2clkout2=FROM "ref_clk" TO "src_clk" TIG;

5. linux 下 ISE 工具分开使用，fpga_editor 用来拉 probe,coregen 用来产生 CORE，其需要用到 xxx.ncd 和 xxx.pcf 两个文件

关于 USB3.0 接口

1、所有的 data input 和 data output 采用 register in、register out，使用 IOB 中寄存器，125MHz 速度可以保证。

2、PIPE 接口 125MHz，UTMI 接口 60MHz，不建议时钟路径上做 MUX（端口 timing 会难以收敛），建议各个时钟独立，内部添加 FIFO，USB controller 工作在 125MHz 时钟。

3、PIPE 的 tx_clk，建议插入 DCM/PLL，对 tx_clk 的相位进行调整，以收敛 timing。

4、PCB 板级走线需保证均衡性，并建议用地对信号进行彼此区隔。如需要，建议进行板级仿真。

5. 关于 flash 同步接口

1、对于输出：CLK/DQS/数据输出均采用 ODDR，为确保 flash 采用 DQS 的边沿采样数据，送出数据的时钟需相移 90 度。

2、对于输入：因 IDDR cell 需要多几个 cyle 才能将数据输出，如果采用 IDDR cell，需要 flash 主控多送几个 clk，与主控设计不符，不才采用。另一种方案是 DQS 使用区域时钟输入，采用 IDELAY cell 进行处理，使用DQS 上升沿/下降沿各自寄存数据，内部经过FIFO 同步到内部时钟域。数据输入需保证与区域时钟在同一个区域，才能利用区域时钟 skew 小的特性。

关于 SD device 接口

1、CLK/CMD/DAT 接口使用 IDDR/ODDR cell 实现，在 DDR50 场景下 D0/D1 数据不同，在 SD3.0 场景下 D0/D1 数据相同

2、SD3.0 208MHz 时钟可以使用DCM/PLL 对输入时钟相位进行调整，以满足 timing 要求。

关于 SD host 接口

1、对于输出，CLK/CMD/DAT 均使用 ODDR 实现

2、对于输入，因存在 trainning 过程，可以使用 IDELAY 对输入 CMD/DAT 进行调整，也可使用DCM/PLL 对输入数据的采样时钟进行调整，但内部需添加FIFO 进行数据跨时钟域处理。

3）IODELAY 逻辑

V5 系列 FPGA 在每个 IOB 中都有 IODELAY 资源，用于调整信号延时。总共 64 级，每级 75ps 延时。该逻辑必须与 IDELAYCTRL 逻辑一起使用，IDELAYCTRL 的参考时钟为 200Mhz,该 200Mhz 时钟必须来自 PLL 或 DCM，而且必须上 BUFG.

4）可以用 distribute 逻辑去搭 fifo 中的 sram，因为该 sram 很小。

5）对于 DDR 接口的 DQS 信号必须对其进行屏蔽，否则如果 DQS 上有误翻转，则会有错误数据采到。

6）在有 PLL 和 DCM 资源的前提下，尽量先使用 PLL，PLL 的时钟 jitter 比较小，功能强于 DCM。

1. 工具和约束只是起到辅助作用，提高时序还要从代码做起。

2. pll 出来的时钟质量要比 DCM 高，所以在重要的地方尽量使用 pll,pll用完了可以用 DCM

3. FPGA 原型验证的时候，可以在乘法器，加法器和 ram 输出进行寄存输出，这些地方很可能是关键路径。可以用`define 命令和asic 进行区分。

---因为工具会自动进行做流水设计,如后面用寄存器打 2 拍,工具会把寄存器插入组合逻辑中间去,使的延时减低一半.以满足时序

4. 利用区域时钟 pin 往往比全局时钟效果更好，在一个区域内部到寄存器距离更短，skrew 跟小

5. PCB 接插件的信号排布尽量避免干扰，可以用电源信号或者地信号进行隔离。

6. lvcmos 电平的速度比 ttl 电平还要快一些的

7. FPGA 输入和输出延迟是物理特性，是不能完全避免的。

8. 对一些关键或者特殊的逻辑，可以进行区域约束，使布局到固定的位置。

1　同步时钟优先考虑 PLL 而非 DCM

2　PLL 里有 phase shift 选项可以控制相移

3　在 IO 端口处可以用 iddr 和 oddr 制造延迟控制 skew delay

4　FPGA 里有选项可选择数据速度是 fast 还是 slow(默认 slow) --只有 output/inout 有此选项,input 无此选项 eg: NET "X_SPI_HOLD" SLEW = FAST;

1、对于输入延时，LVTTL 和 LCMOS 一致，对于输出延时 LVCMOS 的延时更小。

2、PAD 有 fast/slow 选项，默认是 slow，fast 的速度更快一点。

3、PAD 的输出驱动能力不同，速度也不相同，驱动能力大的速度会快，但过冲也会严重（存在noise 问题）

4、不同的速度等级对 PAD 的延时而言没有太大差别。如果内部时钟跑在很高速（700MHz~800Mhz），不同的速度等级对内部 timing 收敛有意义。

5、DCM 和 PLL 均可以完成的功能，优先使用 PLL，稳定性会更好。

6、区域时钟资源到各个寄存器的 skew 比全局时钟资源的更小。

7、参考时钟为 200MHz 时，IDELAY、ODELAY 每个 step 为 75ps。

8、FIFO 如果不大，建议使用 Distributed SRAM。

对 inout pad 的约束可以使用如下

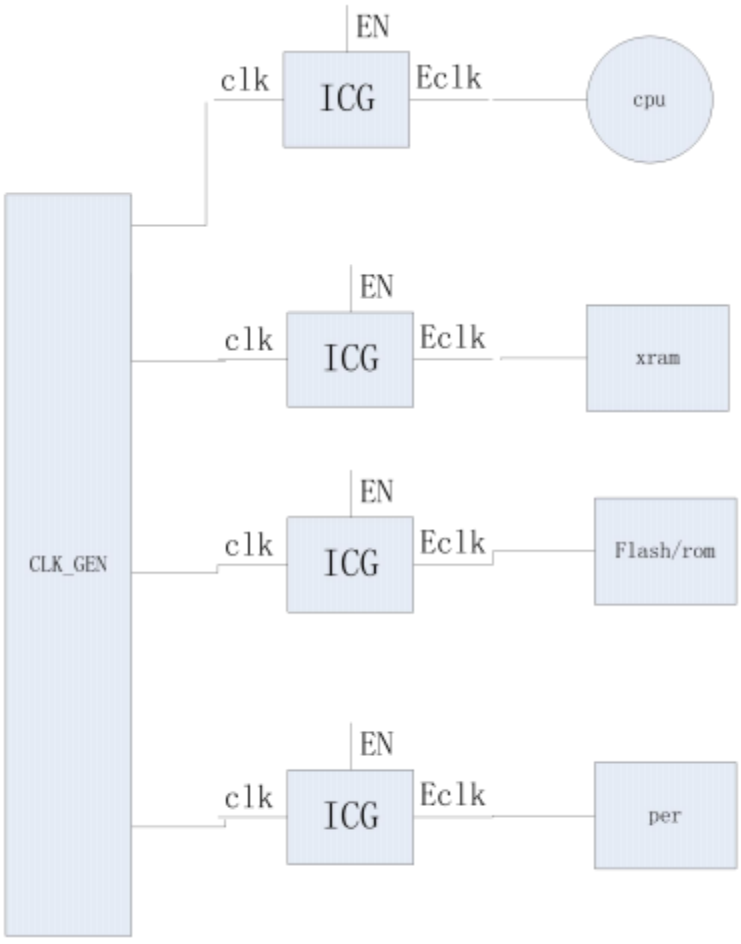
NET "X_SPI_SI" OFFSET = OUT 20 ns AFTER "X_SPI_SCK"; --针对 IO 输出特性约束，X_SPI_SI 在 X_SPI_SCK 变化后 20ns 再变化

NET "X_SPI_SI" OFFSET = IN 1 ns VALID 3 ns BEFORE "X_SPI_SCK" RISING; --针对 IO 输入特性约束，X_SPI_SI 在 X_SPI_SCK 变化前 1ns 变化，维持 3ns，即 setup time 1ns ,hold time:3-1=2ns

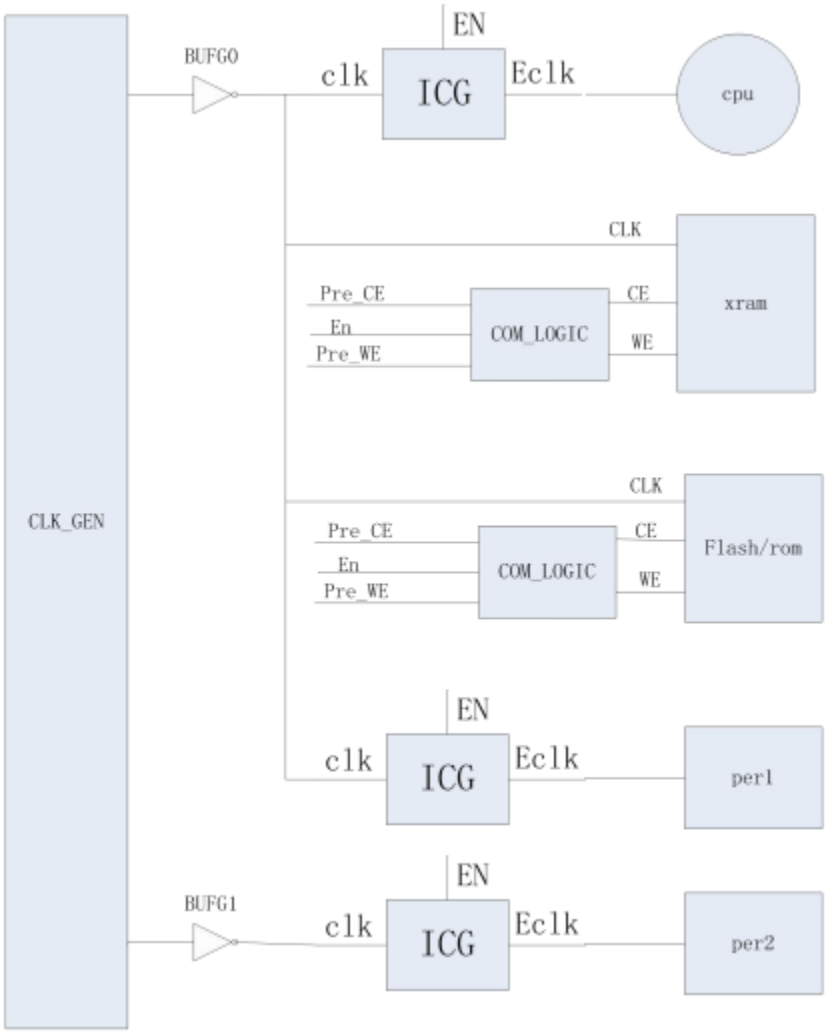
8. Ise 会默认吃一个叫 synplify.ucf 文件（由 synplify 根据约束文件 sdc 生成的时序约束文件），我们只需要加一个 pad 约束文件（其实 pad 约束文件也可以在 synplify 的 sdc 中约束）

9. Soc 系统中时钟处理

   下图是ASIC 中的时钟



10. 在 FPGA 中需要改成如下结构



其中 bufg1,bufg1 要在同一个 group 中,作为同步时钟处理.

上面两图的主要区别是在 xram/flash/rom 时钟处理上不一致,主要是asic 中的时钟树是由自己控制,可以做到 cpu,xram 等时钟为同步时钟;

而 fpga 中,时钟树是固定的,sram 是一个整体,当使用 icg 结构时,由于 sram 不是寄存器,工具无法把使能信号转化到 D 端,时钟没连接到寄存器,所以工具把时钟信号作为数据信号处理,在约束和时序检查时将会忽略此条路径,即 clk 与 sram 的控制信号无时序关系,那么综合的结果将不可控,可能会是时序不满足,但却无法检查而导致 fpga 设计失败.

11. ERROR:Xst:2369 - Empty project file

"E:\design\core\c3\xin\cpu\cpu.prj" 当加了 xco 文件,一直报这个错误,原因是 10.4 版本不支持使用 xco,需要把对应的.ngc 文件放在工程目录下

12.当仿真与实际电路严重不符合时,最好使用nlint 检查是否有

input/output 定义反了,或产生了 latch 等,因为综合器会对 rtl 优化,可能综合出来的电路并不是预期电路 - 绝对是个坑!

COE 文件格式

```
MEMORY_INITIALIZATION_RADIX=2;              //二进制=2，16 进制=16
MEMORY_INITIALIZATION_VECTOR=
00000000000000000000000000000000
。。。。。。
00000000000000000000000000000000
00000000000000000000000000000000
```

tcl 批处理运行

```
 cd ${ISE}/prjxxxx  ;\                          打开工程目录
xtclsh prjxxxx.tcl   run_process  ;\          运行批处理文件 .tcl 中的 run_process
promgen -w -p mcs -c FF -o prjxxxx -u 0 prjxxxx.bit -x xcf128x -data_width 16; 生成 mcs 文件
```

prjxxxx.tcl 文件内容如下(ISE 有标准格式)

```
set myProject "prjxxxx"
set myScript "prjxxxx.tcl"

#
# Main  (top-level) routines
#
# run_process
```

```
# This procedure is used to run processes on an existing project. You
may comment or
# uncomment lines to control which processes are run. This routine is
set up to run
# the Implement Design and Generate Programming File processes by
default. This proc
# also sets process properties as specified in the "set_process_props"
proc. Only
# those properties which have values different from their current
settings in the project
# file will be modified in the project.
#
proc run_process {} {

   global myScript
   global myProject

   ## put out a 'heartbeat' - so we know something's happening.
   puts "\n$myScript: running ($myProject)...\n"

   if { ! [ open_project ] } {
      return false
   }

   set_process_props
   #
   # Remove the comment characters (#'s) to enable the following
commands
   # process run "Synthesize"
```

```
   # process run "Translate"
   # process run "Map"
   # process run "Place & Route"
   #
   set task "Implement Design"
   if { ! [run_task $task] } {
      puts "$myScript: $task run failed, check run output for details."
      project close
      return
   }

   set task "Generate Programming File"
   if { ! [run_task $task] } {
      puts "$myScript: $task run failed, check run output for details."
      project close
      return
   }

   puts "Run completed (successfully)."
   project close

}

#
# rebuild_project
#
# This procedure renames the project file (if it exists) and recreates
the project.
```

```
# It then sets project properties and adds project sources as specified
by the
# set_project_props and add_source_files support procs. It recreates
VHDL Libraries
# as they existed at the time this script was generated.
#
# It then calls run_process to set process properties and run selected
processes.
#
proc rebuild_project {} {

   global myScript
   global myProject

   project close
   ## put out a 'heartbeat' - so we know something's happening.
   puts "\n$myScript: Rebuilding ($myProject)...\n"

   set proj_exts [ list ise xise gise ]
   foreach ext $proj_exts {
      set proj_name "${myProject}.$ext"
      if { [ file exists $proj_name ] } {
         file delete $proj_name
      }
   }

   project new $myProject
   set_project_props
   add_source_files
```

```
   create_libraries
   puts "$myScript: project rebuild completed."

   run_process

}

#
# Support Routines
#
proc run_task { task } {

   # helper proc for run_process

   puts "Running '$task'"
   set result [ process run "$task" ]
   #
   # check process status (and result)
   set status [ process get $task status ]
   if { ( ( $status != "up_to_date" ) && \
            ( $status != "warnings" ) ) || \
          ! $result } {
      return false
   }
   return true
}

#
```

```
# show_help: print information to help users understand the options
available when
#            running this script.
#
proc show_help {} {

   global myScript

   puts ""
   puts "usage: xtclsh $myScript <options>"
   puts "       or you can run xtclsh and then enter 'source
$myScript'."
   puts ""
   puts "options:"
   puts "   run_process       - set properties and run processes."
   puts "   rebuild_project   - rebuild the project from scratch and
run processes."
   puts "   set_project_props - set project properties (device, speed,
etc.)"
   puts "   add_source_files  - add source files"
   puts "   create_libraries  - create vhdl libraries"
   puts "   set_process_props - set process property values"
   puts "   show_help         - print this message"
   puts ""
}

proc open_project {} {

   global myScript
```

```
   global myProject

   if { ! [ file exists ${myProject}.xise ] } {
      ## project file isn't there, rebuild it.
      puts "Project $myProject not found. Use project_rebuild to
recreate it."
      return false
   }

   project open $myProject

   return true

}
#
# set_project_props
#
# This procedure sets the project properties as they were set in the
project
# at the time this script was generated.
#
proc set_project_props {} {

   global myScript

   if { ! [ open_project ] } {
      return false
   }
```

```
   puts "$myScript: Setting project properties..."

   project set family "Virtex6"
   project set device "xc6vlx240t"
   project set package "ff1156"
   project set speed "-1"
   project set top_level_module_type "EDIF"
   project set synthesis_tool "XST (VHDL/Verilog)"
   project set simulator "ISim (VHDL/Verilog)"
   project set "Preferred Language" "Verilog"
   project set "Enable Message Filtering" "false"
}

#
# add_source_files
#
# This procedure add the source files that were known to the project
at the
# time this script was generated.
#
proc add_source_files {} {
   global myScript
   if { ! [ open_project ] } {
      return false
   }

   puts "$myScript: Adding sources to project..."
   xfile add "prjxxxx.edf"
   xfile add "prjxxxx.ucf"
```

xfile add "synplicity.ucf" ##此处如果 synplify 的 sdc 文件把 pin 等信息全部约束完整，那么在 ise 就不需要再添加任何约束，用 synplify 输出的 synplicity.ucf 就可以

```
   # Set the Top Module as well...
   project set top "prjxxxx"

   puts "$myScript: project sources reloaded."

} ; # end add_source_files

#
# create_libraries
#
# This procedure defines VHDL libraries and associates files with those libraries.
# It is expected to be used when recreating the project. Any libraries defined
# when this script was generated are recreated by this procedure.
#
proc create_libraries {} {

   global myScript

   if { ! [ open_project ] } {
      return false
   }

   puts "$myScript: Creating libraries..."
```

```
   # must close the project or library definitions aren't saved.
   project save

} ; # end create_libraries

#
# set_process_props
#
# This procedure sets properties as requested during script generation
(either
# all of the properties, or only those modified from their defaults).
#
proc set_process_props {} {

   global myScript

   if { ! [ open_project ] } {
      return false
   }

   puts "$myScript: setting process properties..."

   project set "Compiled Library Directory"
"\$XILINX/<language>/<simulator>"
   project set "Global Optimization" "Off" -process "Map"
   project set "Use DSP Block" "Auto" -process "Synthesize - XST"
```

project set "DCI Update Mode" "As Required" -process "Generate Programming File"

project set "Enable Cyclic Redundancy Checking (CRC)" "true" -process "Generate Programming File"

project set "Configuration Rate" "2" -process "Generate Programming File"

project set "Pack I/O Registers/Latches into IOBs" "Off" -process "Map"

project set "Place And Route Mode" "Route Only" -process "Place & Route"

project set "Number of Clock Buffers" "32" -process "Synthesize - XST"

project set "Max Fanout" "100000" -process "Synthesize - XST"

project set "Use Clock Enable" "Auto" -process "Synthesize - XST"

project set "Use Synchronous Reset" "Auto" -process "Synthesize - XST"

project set "Use Synchronous Set" "Auto" -process "Synthesize - XST"

project set "Regenerate Core" "Under Current Project Setting" -process "Regenerate Core"

project set "Filter Files From Compile Order" "true"

project set "Last Applied Goal" "Balanced"

project set "Last Applied Strategy" "Xilinx Default (unlocked)"

project set "Last Unlock Status" "false"

project set "Manual Compile Order" "false"

project set "Placer Effort Level" "High" -process "Map"

project set "Extra Cost Tables" "0" -process "Map"

project set "LUT Combining" "Off" -process "Map"

project set "Combinatorial Logic Optimization" "false" -process "Map"

```
project set "Starting Placer Cost Table (1-100)" "1" -process "Map"
project set "Power Reduction" "Off" -process "Map"
project set "Report Fastest Path(s) in Each Constraint" "true" -process "Generate Post-Place & Route Static Timing"
project set "Generate Datasheet Section" "true" -process "Generate Post-Place & Route Static Timing"
project set "Generate Timegroups Section" "false" -process "Generate Post-Place & Route Static Timing"
project set "Report Fastest Path(s) in Each Constraint" "true" -process "Generate Post-Map Static Timing"
project set "Generate Datasheet Section" "true" -process "Generate Post-Map Static Timing"
project set "Generate Timegroups Section" "false" -process "Generate Post-Map Static Timing"
project set "Project Description" ""
project set "Property Specification in Project File" "Store all values"
project set "Reduce Control Sets" "Auto" -process "Synthesize - XST"
project set "Shift Register Minimum Size" "2" -process "Synthesize - XST"
project set "Case Implementation Style" "None" -process "Synthesize - XST"
project set "RAM Extraction" "true" -process "Synthesize - XST"
project set "ROM Extraction" "true" -process "Synthesize - XST"
project set "FSM Encoding Algorithm" "Auto" -process "Synthesize - XST"
project set "Optimization Goal" "Speed" -process "Synthesize - XST"
project set "Optimization Effort" "Normal" -process "Synthesize - XST"
```

project set "Resource Sharing" "true" -process "Synthesize - XST"

project set "Shift Register Extraction" "true" -process "Synthesize - XST"

project set "User Browsed Strategy Files" ""

project set "VHDL Source Analysis Standard" "VHDL-93"

project set "Analysis Effort Level" "Standard" -process "Analyze Power Distribution (XPower Analyzer)"

project set "Analysis Effort Level" "Standard" -process "Generate Text Power Report"

project set "Input TCL Command Script" "" -process "Generate Text Power Report"

project set "Load Physical Constraints File" "Default" -process "Analyze Power Distribution (XPower Analyzer)"

project set "Load Physical Constraints File" "Default" -process "Generate Text Power Report"

project set "Load Simulation File" "Default" -process "Analyze Power Distribution (XPower Analyzer)"

project set "Load Simulation File" "Default" -process "Generate Text Power Report"

project set "Load Setting File" "" -process "Analyze Power Distribution (XPower Analyzer)"

project set "Load Setting File" "" -process "Generate Text Power Report"

project set "Setting Output File" "" -process "Generate Text Power Report"

project set "Produce Verbose Report" "false" -process "Generate Text Power Report"

project set "Other XPWR Command Line Options" "" -process "Generate Text Power Report"

project set "Essential Bits" "false" -process "Generate Programming File"

project set "Encrypt Bitstream" "false" -process "Generate Programming File"

project set "JTAG to System Monitor Connection" "Enable" -process "Generate Programming File"

project set "User Access Register Value" "None" -process "Generate Programming File"

project set "Other Bitgen Command Line Options" "" -process "Generate Programming File"

project set "Maximum Signal Name Length" "20" -process "Generate IBIS Model"

project set "Show All Models" "false" -process "Generate IBIS Model"

project set "Disable Detailed Package Model Insertion" "false" -process "Generate IBIS Model"

project set "Launch SDK after Export" "true" -process "Export Hardware Design To SDK with Bitstream"

project set "Launch SDK after Export" "true" -process "Export Hardware Design To SDK without Bitstream"

project set "Target UCF File Name" "" -process "Back-annotate Pin Locations"

project set "Ignore User Timing Constraints" "false" -process "Map"

project set "Register Ordering" "4" -process "Map"

project set "Use RLOC Constraints" "Yes" -process "Map"

project set "Other Map Command Line Options" "" -process "Map"

project set "Use LOC Constraints" "true" -process "Translate"

project set "Other Ngdbuild Command Line Options" "" -process "Translate"

project set "Use 64-bit PlanAhead on 64-bit Systems" "true" -process "Floorplan Area/IO/Logic (PlanAhead)"

project set "Use 64-bit PlanAhead on 64-bit Systems" "true" -process "I/O Pin Planning (PlanAhead) - Pre-Synthesis"

project set "Use 64-bit PlanAhead on 64-bit Systems" "true" -process "I/O Pin Planning (PlanAhead) - Post-Synthesis"

project set "Ignore User Timing Constraints" "false" -process "Place & Route"

project set "Other Place & Route Command Line Options" "" -process "Place & Route"

project set "BPI Reads Per Page" "1" -process "Generate Programming File"

project set "Configuration Pin Busy" "Pull Up" -process "Generate Programming File"

project set "Configuration Clk (Configuration Pins)" "Pull Up" -process "Generate Programming File"

project set "UserID Code (8 Digit Hexadecimal)" "0xFFFFFFFF" -process "Generate Programming File"

project set "Configuration Pin CS" "Pull Up" -process "Generate Programming File"

project set "Configuration Pin DIn" "Pull Up" -process "Generate Programming File"

project set "Disable JTAG Connection" "false" -process "Generate Programming File"

project set "Configuration Pin Done" "Pull Up" -process "Generate Programming File"

project set "Create ASCII Configuration File" "false" -process "Generate Programming File"

project set "Create Binary Configuration File" "false" -process "Generate Programming File"

project set "Create Bit File" "true" -process "Generate Programming File"

project set "Enable BitStream Compression" "false" -process "Generate Programming File"

project set "Run Design Rules Checker (DRC)" "true" -process "Generate Programming File"

project set "Create IEEE 1532 Configuration File" "false" -process "Generate Programming File"

project set "Create ReadBack Data Files" "false" -process "Generate Programming File"

project set "Configuration Pin HSWAPEN" "Pull Up" -process "Generate Programming File"

project set "Configuration Pin Init" "Pull Up" -process "Generate Programming File"

project set "Configuration Pin M0" "Pull Up" -process "Generate Programming File"

project set "Configuration Pin M1" "Pull Up" -process "Generate Programming File"

project set "Configuration Pin M2" "Pull Up" -process "Generate Programming File"

project set "Configuration Pin Program" "Pull Up" -process "Generate Programming File"

project set "Power Down Device if Over Safe Temperature" "false" -process "Generate Programming File"

project set "Configuration Pin RdWr" "Pull Up" -process "Generate Programming File"

project set "Starting Address for Fallback Configuration" "None" -process "Generate Programming File"

project set "JTAG Pin TCK" "Pull Up" -process "Generate Programming File"

project set "JTAG Pin TDI" "Pull Up" -process "Generate Programming File"

project set "JTAG Pin TDO" "Pull Up" -process "Generate Programming File"

project set "JTAG Pin TMS" "Pull Up" -process "Generate Programming File"

project set "Unused IOB Pins" "Pull Down" -process "Generate Programming File"

project set "Watchdog Timer Mode" "Off" -process "Generate Programming File"

project set "Security" "Enable Readback and Reconfiguration" -process "Generate Programming File"

project set "Done (Output Events)" "Default (4)" -process "Generate Programming File"

project set "Drive Done Pin High" "false" -process "Generate Programming File"

project set "Enable Outputs (Output Events)" "Default (5)" -process "Generate Programming File"

project set "Wait for DCI Match (Output Events)" "Auto" -process "Generate Programming File"

project set "Wait for PLL Lock (Output Events)" "No Wait" -process "Generate Programming File"

project set "Release Write Enable (Output Events)" "Default (6)" -process "Generate Programming File"

project set "FPGA Start-Up Clock" "CCLK" -process "Generate Programming File"

project set "Enable Internal Done Pipe" "true" -process "Generate Programming File"

project set "Allow Logic Optimization Across Hierarchy" "false" -process "Map"

project set "Maximum Compression" "false" -process "Map"

project set "Generate Detailed MAP Report" "false" -process "Map"

project set "Map Slice Logic into Unused Block RAMs" "false" -process "Map"

project set "Perform Timing-Driven Packing and Placement" "false"

project set "Trim Unconnected Signals" "true" -process "Map"

project set "Create I/O Pads from Ports" "false" -process "Translate"

project set "Macro Search Path" " /fpga/coregen" -process "Translate" ##指定需要调用 ip

project set "Netlist Translation Type" "Timestamp" -process "Translate"

project set "User Rules File for Netlister Launcher" "" -process "Translate"

project set "Allow Unexpanded Blocks" "false" -process "Translate"

project set "Allow Unmatched LOC Constraints" "false" -process "Translate"

project set "Allow Unmatched Timing Group Constraints" "false" -process "Translate"

project set "Perform Advanced Analysis" "false" -process "Generate Post-Place & Route Static Timing"

project set "Report Paths by Endpoint" "3" -process "Generate Post-Place & Route Static Timing"

```
   project set "Report Type" "Verbose Report" -process "Generate
Post-Place & Route Static Timing"
   project set "Number of Paths in Error/Verbose Report" "3" -process
"Generate Post-Place & Route Static Timing"
   project set "Stamp Timing Model Filename" "" -process "Generate
Post-Place & Route Static Timing"
   project set "Report Unconstrained Paths" "" -process "Generate
Post-Place & Route Static Timing"
   project set "Perform Advanced Analysis" "false" -process "Generate
Post-Map Static Timing"
   project set "Report Paths by Endpoint" "3" -process "Generate
Post-Map Static Timing"
   project set "Report Type" "Verbose Report" -process "Generate
Post-Map Static Timing"
   project set "Number of Paths in Error/Verbose Report" "3" -process
"Generate Post-Map Static Timing"
   project set "Report Unconstrained Paths" "" -process "Generate
Post-Map Static Timing"
   project set "Add I/O Buffers" "true" -process "Synthesize - XST"
   project set "Global Optimization Goal" "AllClockNets" -process
"Synthesize - XST"
   project set "Keep Hierarchy" "No" -process "Synthesize - XST"
   project set "Register Balancing" "No" -process "Synthesize - XST"
   project set "Register Duplication" "true" -process "Synthesize -
XST"
   project set "Library for Verilog Sources" "" -process "Synthesize
- XST"
   project set "Export Results to XPower Estimator" "" -process
"Generate Text Power Report"
```

```
   project set "Asynchronous To Synchronous" "false" -process
"Synthesize - XST"
   project set "Automatic BRAM Packing" "false" -process "Synthesize
- XST"
   project set "BRAM Utilization Ratio" "100" -process "Synthesize -
XST"
   project set "Bus Delimiter" "<>" -process "Synthesize - XST"
   project set "Case" "Maintain" -process "Synthesize - XST"
   project set "Cores Search Directories" "" -process "Synthesize -
XST"
   project set "Cross Clock Analysis" "false" -process "Synthesize -
XST"
   project set "DSP Utilization Ratio" "100" -process "Synthesize -
XST"
   project set "Equivalent Register Removal" "true" -process
"Synthesize - XST"
   project set "FSM Style" "LUT" -process "Synthesize - XST"
   project set "Generate RTL Schematic" "Yes" -process "Synthesize -
XST"
   project set "Generics, Parameters" "" -process "Synthesize - XST"
   project set "Hierarchy Separator" "/" -process "Synthesize - XST"
   project set "HDL INI File" "" -process "Synthesize - XST"
   project set "LUT Combining" "Auto" -process "Synthesize - XST"
   project set "Library Search Order" "" -process "Synthesize - XST"
   project set "Netlist Hierarchy" "As Optimized" -process "Synthesize
- XST"
   project set "Optimize Instantiated Primitives" "false" -process
"Synthesize - XST"
```

```
   project set "Pack I/O Registers into IOBs" "Auto" -process
"Synthesize - XST"
   project set "Power Reduction" "false" -process "Synthesize - XST"
   project set "Read Cores" "true" -process "Synthesize - XST"
   project set "LUT-FF Pairs Utilization Ratio" "100" -process
"Synthesize - XST"
   project set "Use Synthesis Constraints File" "true" -process
"Synthesize - XST"
   project set "Verilog Include Directories" "" -process "Synthesize
- XST"
   project set "Verilog Macros" "" -process "Synthesize - XST"
   project set "Work Directory" " /fpga/ise/prjxxxx/xst" -process
"Synthesize - XST"
   project set "Write Timing Constraints" "false" -process "Synthesize
- XST"
   project set "Other XST Command Line Options" "" -process "Synthesize
- XST"
   project set "Timing Mode" "Performance Evaluation" -process "Map"
   project set "Generate Asynchronous Delay Report" "false" -process
"Place & Route"
   project set "Generate Clock Region Report" "false" -process "Place
& Route"
   project set "Generate Post-Place & Route Power Report" "false"
-process "Place & Route"
   project set "Generate Post-Place & Route Simulation Model" "false"
-process "Place & Route"
   project set "Power Reduction" "false" -process "Place & Route"
   project set "Place & Route Effort Level (Overall)" "High" -process
"Place & Route"
```

```
project set "Auto Implementation Compile Order" "true"
project set "Equivalent Register Removal" "true" -process "Map"
project set "Placer Extra Effort" "None" -process "Map"
project set "Power Activity File" "" -process "Map"
project set "Register Duplication" "On" -process "Map"
project set "Generate Constraints Interaction Report" "false" -process "Generate Post-Map Static Timing"
project set "Synthesis Constraints File" "" -process "Synthesize - XST"
project set "RAM Style" "Auto" -process "Synthesize - XST"
project set "Maximum Number of Lines in Report" "1000" -process "Generate Text Power Report"
project set "AES Initial Vector" "" -process "Generate Programming File"
project set "HMAC Key (Hex String)" "" -process "Generate Programming File"
project set "Encrypt Key Select" "BBRAM" -process "Generate Programming File"
project set "AES Key (Hex String)" "" -process "Generate Programming File"
project set "Input Encryption Key File" "" -process "Generate Programming File"
project set "Output File Name" "prjxxxx" -process "Generate IBIS Model"
project set "Timing Mode" "Performance Evaluation" -process "Place & Route"
project set "Cycles for First BPI Page Read" "1" -process "Generate Programming File"
```

project set "Enable Debugging of Serial Mode BitStream" "false" -process "Generate Programming File"

project set "Create Logic Allocation File" "false" -process "Generate Programming File"

project set "Create Mask File" "false" -process "Generate Programming File"

project set "Watchdog Timer Value" "0x000000" -process "Generate Programming File"

project set "Allow SelectMAP Pins to Persist" "false" -process "Generate Programming File"

project set "Enable Multi-Threading" "2" -process "Map"

project set "Generate Constraints Interaction Report" "false" -process "Generate Post-Place & Route Static Timing"

project set "Move First Flip-Flop Stage" "true" -process "Synthesize - XST"

project set "Move Last Flip-Flop Stage" "true" -process "Synthesize - XST"

project set "ROM Style" "Auto" -process "Synthesize - XST"

project set "Safe Implementation" "No" -process "Synthesize - XST"

project set "Power Activity File" "" -process "Place & Route"

project set "Extra Effort (Highest PAR level only)" "None" -process "Place & Route"

project set "Fallback Reconfiguration" "Enable" -process "Generate Programming File"

project set "Enable Multi-Threading" "4" -process "Place & Route"

project set "Functional Model Target Language" "Verilog" -process "View HDL Source"

project set "Change Device Speed To" "-1" -process "Generate Post-Place & Route Static Timing"

```
   project set "Change Device Speed To" "-1" -process "Generate
Post-Map Static Timing"

   puts "$myScript: project property values set."

} ; # end set_process_props

proc main {} {

   if { [llength $::argv] == 0 } {
      show_help
      return true
   }

   foreach option $::argv {
      switch $option {
         "show_help"           { show_help }
         "run_process"         { run_process }
         "rebuild_project"     { rebuild_project }
         "set_project_props"   { set_project_props }
         "add_source_files"    { add_source_files }
         "create_libraries"    { create_libraries }
         "set_process_props"   { set_process_props }
         default               { puts "unrecognized option: $option";
show_help }
      }
   }
}
```

```
if { $tcl_interactive } {
   show_help
} else {
   if {[catch {main} result]} {
      puts "$myScript failed: $result."
   }
}
```

Synplify_ise 串型工作命令批处理运行

```
#!/bin/bash
data=$(date "+%Y%m%d")
cd /xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/synplify/
synplify_pro -shell
/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/synplify/scr/xxxx_synpl
ify.tcl
if [ $? -ne 2 ]
then
    echo "**** synplify_pro return value is $? **********"
    cd
/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/xilinx/xxxx_fpga_vp40_r
ev1
    /ic01/eda/xilinx/ISE/ISE/bin/lin64/ngdbuild -ise
"/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/xilinx/xxxx_fpga_vp40_
rev1/xxxx.ise" -intstyle ise -dd _ngo  -sd
"/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/xilinx/xxxx_fpga_vp40_
rev1/" -sd
"/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/coregen/xxxx/" -nt
timestamp -i -p xc2vp40-fg676-5
```

"/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/synplify/latest/xxxx.edf" xxxx.ngd

if [$? -ne 2]

then

echo "**** ngdbuild return value is $? **********"

/ic01/eda/xilinx/ISE/ISE/bin/lin64/map -ise "/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/xilinx/xxxx_fpga_vp40_rev1/xxxx.ise" -intstyle ise -p xc2vp40-fg676-5 -cm speed -ignore_keep_hierarchy -pr off -k 4 -c 100 -tx off -o xxxx_map.ncd xxxx.ngd xxxx.pcf

if [$? -ne 2]

then

echo "**** map return value is $? **********"

/ic01/eda/xilinx/ISE/ISE/bin/lin64/par -ise "/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/xilinx/xxxx_fpga_vp40_rev1/xxxx.ise" -w -intstyle ise -pl high -rl high -xe n -t 20 xxxx_map.ncd xxxx.ncd xxxx.pcf

if [$? -ne 2]

then

echo "**** par return value is $? **********"

/ic01/eda/xilinx/ISE/ISE/bin/lin64/trce -ise "/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/xilinx/xxxx_fpga_vp40_rev1/xxxx.ise" -intstyle ise -e 600 -s 5 -u 100 -xml xxxx xxxx.ncd -o xxxx.twr xxxx.pcf -ucf /xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/synplify/latest/synplicity.ucf

if [$? -ne 2]

then

echo "**** trce return value is $? **********"

```
                    /ic01/eda/xilinx/ISE/ISE/bin/lin64/bitgen -ise
"/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/xilinx/xxxx_fpga_vp40_
rev1/xxxx.ise" -intstyle ise -f xxxx.ut xxxx.ncd
                    if [ $? -ne 2 ]
                    then
                        echo "**** bitgen return value is $?
***************"
                        /ic01/eda/xilinx/ISE/ISE/bin/lin64/promgen -w
-p mcs -c FF -o
/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/mcs/xxxx_$data -u 0
/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/xilinx/xxxx_fpga_vp40_r
ev1/xxxx.bit -x xcf16p
                        if [ $? -ne 2 ]
                        then
                            echo "**** promgen return value is $?
***************"
                            rm -rf
/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/mcs/*.cfi
/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/mcs/*.prm
/xxx/projects/XXXX/revA0/team/ganlc/chip/fpga/mcs/*.sig
                        fi
                    fi
                fi
            fi
        fi
    fi
fi
echo "************************fpga file
done***********************************"
```

当有错误会停下来，如果结果返回 2 表示 error,返回 1 表示 warning。

低功耗篇(转)

1.0 概述

本文从数字集成电路低功耗设计原理下手，对设计中低功耗的实现技术进行描述，包括完成低功耗设计需要的库资料以及常用EDA 工具对低功耗技术实现的方法。

2.0 CMOS 电路的低功耗设计原理

CMOS 电路功耗主要分 3 种:

静态功耗主要与工艺以及电路结构相关;

短路电流功耗主要与驱动电压、p-MOS 和 n-MOS 同时打开时产生的最大电流、翻转频率以及上升、下降时间有关;

开关电流功耗主要与负载电容、驱动电压、翻转频率有关。

做低功耗设计，就必须从这些影响功耗的因素下手。

3.0 低功耗设计手段及 Library 需求

低功耗的设计手段较为复杂，但对于不同的设计，或者不同的工艺，实现的方法却各不相同。

3.1 0.18um 及以上工艺

0.18um 及以上工艺，在低功耗设计手段上较为有限，主要原因在于，静态功耗很小，基本不用关心。

动态功耗方面，主要的功耗来自于 Switching Power，即与负载电容、电压以及工作中的信号翻转频率相关。

减小负载电容，就必须在设计上下功夫，减少电路规模。减少信号翻转频率，除了降低时钟频率外，只有在设计上考虑，能不翻转的信号就不翻转。至于电压，由于 0.18um 及以上工艺的阈值电压有一定的限制，因此，供电电压降低，势必影响工作频率。

一般说来，在 0.18um 工艺下设计电路，主要有以下几种对低功耗设计的考虑。

3.1.1 静态功耗可以忽略

根据现有项目经验可知，利用 0.18um 工艺 Standard Cell 设计出来的某芯片，数字逻辑加上 Ram 和Rom 约 40 万门的电路，在完全静止的状态下，功耗约 200uA 左右（实测数据为 400uA 左右，包括了 50uA Flash，30uA 的 PHY，113uA 的 VR，其他模拟部分漏电不大，因此这里估算为 200uA）。这样的功耗，我们是可以接受的。如果非要减少静态功耗，则可以参照 90nm 工艺的设计思路，专门设计高阈值电压的 MOSFET，或者专门设计切断电源所需的元件，但由此带来设计的复杂性，对 0.18um 工艺的影响还是很大的。如果设计规模没有那么大，且可以满足应用，往往还是可以忽略这个结果的。

3.1.2 时钟门控减小不必要的动态功耗

在寄存器的电路设计中，时钟输入端都会有一个反向器负载，就算输入端不发生变化，时钟的变化也会造成该反向器的变化，由此产生动态功耗。因此在如果该寄存器输入在某种条件下等于输出（即输出保持）时，可以将时钟门控住，以减少无效的时钟翻转。

时钟门控的实现原理如下图所示：

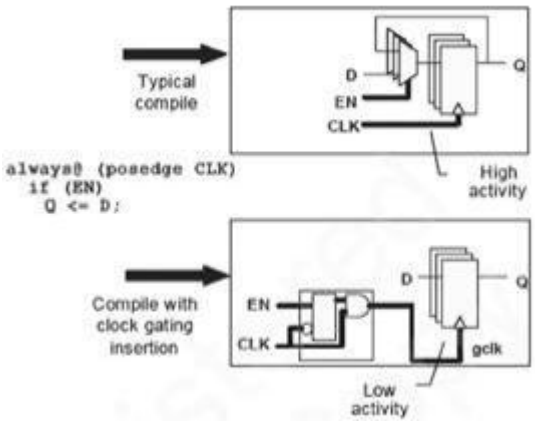


图 3- 1 时钟门控原理图

如上图所示，由于现在的设计方式，大多数是同步设计，设计人员只考虑数据路径，时钟往往是不做处理的。因此如果要实现门控，只需要在设计电路时提供可以识别的控制信号，在综合的时候，EDA 工具就可以自动插入时钟门控。

利用Design Compiler 进行时钟门控单元插入，在读入设计以及时序约束后，需要做以下设置：

1. set_clock_gating_style，设置时钟门控单元插入的约束
2. insert_clock_gating -global，开始插入时钟门控单元；
3. uniquify，将所有时钟门控单元做 uniquify 操作，以便后续 PR
4. hookup_testports -se_port ATPGSE_Pad -se_pin uPad/uATPGSE_Pad/C -verbose，将所有时钟门控单元的 scan_enable 信号与测试用 SE 信号连接起来。如果没有 ATPG，可以不用该句命令。
5. propagate_constraints -gate_clock，将门控单元信息传递给整个电路。
6. report_clock_gating 可以查看时钟门控单元插入的情况，以便做电路修改，或插入时钟门控单元设置的修改。

完成这些设置后，只需要和平常一样做系统综合即可。而在 DC 2008.09 版本以后，第 2~5 的步骤都可以省略，在利用 compile_ultra 进行优化时，第 2、3 步骤都会被自动执行，第 4、5 步骤会在 insert_dft 时被执行。

形式验证工具 Formality，在进行形式验证，需要设置 verification_clock_gatie_hold_mode 为 low、high 或者 any，Formality 就可以识别出时钟门控单元，并与 RTL 进行形式验证了。

3.1.2.1. Clock Gating Cells

这里所说的Clock Gating Cell 是指专门设计的，集成式时钟门控单元（Integrated Clock Gating Cell，简称 ICG），就是利用 Latch 和与门/或门实现的一个独立的 Standard Cell，其优势在于以硬 IP 实现，时序易于掌握，物理实现中对布局布线有帮助。当然如果单元库中不提供专门的时钟门控单元，EDA 工具也可以利用与门、或门、Latch 甚至是寄存器等进行门控单元的实现，但效果都没有 ICG 好用。这里针对 ICG 的插入进行描述。

图 3- 1 中的门控单元是一种典型的，利用负沿使能Latch 以及与门组成的上升沿有效时钟门控单元，只有时钟下降沿后才会将时钟门控住，保证不产生时钟毛刺。

在 Liberty 格式文件中，某个 Cell，需要有 clock_gating_integrated_cell，才能让 EDA 工具认识到，该 Cell 是一种 ICG。不同的 clock_gating_integrated_cell 的设置，需要在 DC 设置 set_clock_gating_style 时做相应的设置，才可能被使用到，下面列举一些常用的设置：

同时，在 ICG 的不同Pin 上，必须有以下属性，来告诉 DC 该 Pin 在 ICG 的使用中是什么功能（这里只列举常用的信息）：

clock_gate_enable_pin	该 pin 是时钟使能控制信号
clock_gate_out_pin	该 pin 是时钟输出信号
clock_gate_clock_pin	该 pin 是时钟输入信号
clock_gate_test_pin	该 pin 是 scan_enable 或 test_mode 信号

3.1.3 使用低电压的库进行设计

由于动态功耗中，驱动电压对功耗的影响也相当大，因此，如果能有一套电压只有 1V 的标准单元库，进行设计，仍然可以达到降低动态功耗的目的。

但电压的降低，势必引起元件延时的增加，且由于 0.18um 工艺下，阈值电压一般在 0.4V 左右，驱动电压的稳定性需求也相当大，否则，可能会导致致命性的错误。

法国的 Dolphin 公司是一家致力于低功耗设计的 IP 提供商，在 TSMC、SMIC 等 Foundry 的 0.18um 工艺下都提供了 1V 的逻辑单元库。下面列出 Dolphin 在 SMIC 0.18um 工艺下设计的一套 1V 逻辑单元库，和 SMIC 0.18um 工艺Metro 标准单元库进行比较。比较中Metro 标准单元库使用 1 个门的 BUFX2M，而 Dolphin 使用最小的 Buffer ni01d1 进行比较。

		area (um*um)	Average leakage(nW)	Rise delay (0.04pf,ns)	Typical Rise Energy (0.04pf,pJ)
SMIC18 METRO	BUFX 2M	8.7808	0.048	0.182256	0.0266
Dolphin 1V for SMIC 0.18um	ni01 d1	10.3488	0.01	0.6	0.005

由比较中可以看出，当电压下降到 1V 后，Rise Energy 下降了 80%以上，除了由于电压下降引起的功耗降低外，Dolphin 应该在电路结构等方面也做了处理，因此不但动态功耗减少了，而且静态功耗也减少了很多。但延时却大了很多，因此如果设计需要翻转的频率不高时，可以考虑利用低功耗的库进行设计，达到降低功耗的目的。如果速度要求很高，这个方法是不可行的。

3.2 90nm 及以下工艺

从 3.1.3 可以知道，降低驱动电压，可以减少动态功耗，但由于电压降低，驱动能力也同时被减弱，因此元件延时较大。为了解决这个问题，工艺尺寸开始减小，以便在减小驱动电压的情况下，增加宽长比（aspect ratio），以达到提高驱动电流的目的，保持元件延时。

同时进入更低尺寸的工艺，氧化层厚度也随之减小，以便减少阈值电压，进一步提高速度。但因为氧化层厚度在减小，漏电电流也变大了。在 90nm 及以下工艺中，漏电电流开始被设计人员关注。

下面对在 90nm 工艺下进行低功耗设计及实现的一些手段结合常用EDA 工具进行描述。

3.2.1 切断未使能电路的电源减小不必要的静态功耗

针对 SMIC 0.18um 工艺 Metro 标准单元库以及 TSMC 90nmLP 工艺高密度标准单元库（dbtcbn90lphdbwptc）进行比较，以一个门的 Buffer 来举例：

		Average leakage(nW)	Incremental	Typical Rise Energe (0.04pf,pJ)	Incremental
SMIC 18 METRO	BUFX2M	0.048	_	0.0266	_
TSMC 90LP Bias	BUFFD1BWP	0.214	345.83%	0.00272	-89.77%

ed Well

可以看出，90nm 工艺下的静态功耗，已经是 0.18um 工艺下功耗的 3.5 倍左右了。根据 3.1.1 可知，利用 0.18um 设计出来的，约 40 万门的电路，静态功耗，大约是 200uA（360uW，0.18um 工艺按 1.8V 供电电压计算）。如果同样规模的电路，放在 90nm 工艺下，则可能达到 1.26mW 左右，即 1.05mA 左右的静态功耗（90nm 工艺按 1.2V 供电电压计算）。

既然，静态功耗这么大，那么在静止时，怎样才能将这些功耗减小呢？一个非常彻底的方法就是将静止状态电路的电源关断。

为了关断电源，就需要在电源网络和电路之间建立一个电源控制电路，他们被称为电源开关单元（Power Switching Cell），在需要关断时，控制 Power Switching Cell 将电路的供电关闭，否则打开，提供电源。

由于电源关断后的电路，其输出信号就没有电路驱动，对于其驱动的电路来说，就会出现输入浮空的状态。为了解决这个问题，就需要在关闭电源的电路输出端添加一个额外的保持电路，当其电源关闭后保持输出，而电源打开时，保持电路则表现的像一个Buffer，输出等于输入即可。同时，如果被关闭电源的电路输入固定电压，也可能产生对地的电流，就需要一个特别的单元对该部分电流进行保护。这样的单元被称为隔离单元（Isolation Cell）。

一般来说 Isolation Cell 的输出部分有较大的电容负载，也就是说 Isolation Cell 的延时将会比较大，对时序有一定的影响，是需要注意的。

当然，对于寄存器来说，如果断电，则原有的数据就无法保存，重新打开电源后，就一定会出现原有数据丢失的情况。因此可以为一些必须保持数据的寄存器建立一个备份设备，电源关闭前，将寄存器的数值保存到备份设备上，电源打开后从备份设备上将数据重新写入寄存器中。这种备份设备叫做保存寄存器单元（Retention Register Cells）。

对于 Power Switching Cell、Isolation Cell 以及 Retention Register Cell，他们在上电之后是不能关闭的，因此使用的电源也和正常功能不同，这些单元被成为常开逻辑单元（Always-On Logic Cells）

下面就各种不同的Cell，描述其原理、库中保存的信息、以及实现流程。

3.2.1. Power Switching Cells

原理上说，Power Switching Cell 结构如下图所示：

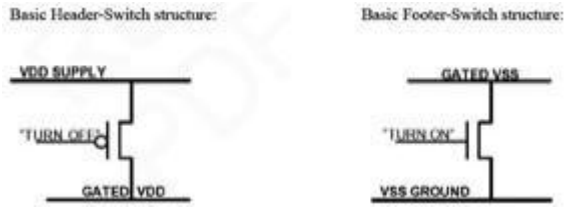


图 3- 2 Power Switching Cell 电路结构示意图

从图中可以看到，Power Switching Cell 的设计原理非常简单，VDD 的控制（被称为 Header Switch），利用一个 P-MOSFET 来控制，当 TURN_OFF 信号为高时表示电路关闭，P-MOSFET 关闭，GATED_VDD 就不供电了，而 TURN_OFF 信号为低时表示电路打开，P-MOSFET 打开，GATED_VDD 等于 VDD_SUPPLY。同理，利用一个 N-MOSFET 来控制 VSS 电源是否供电，被称为 Footer Switch。一般来说，只需要使用 Header Switch 或者 Footer Switch 就可以实现电路关断，其中 Header Switch 结构漏电较小，而 Footer Switch 结构控制效率高，且面积较小。

虽然原理非常容易理解，但电路设计起来非常复杂，需要考虑控制单元对电路供电的能力，考虑不能产生过大的功耗，等等。这里不详细介绍控制单元的设计，详细设计方法可以参看Synopsys 公司和 ARM 公司联合出版的“Low Power Methodology Manual For System on Chip Design.”。

将 Power Switch Cell 设计成单独的器件，在实现时，控制某一块电路，叫做 Coarse Grain Power Switch Cell。

下面是一个 Coarse Grain Power Switch Cell 的 Liberty 格式描述

```
library (<coarse_grain_library_name>) { #library
描述开始
…
lu_table _template （ template_name ） #电压状态
template 描述，dc_current 组中会使用
variable_1 ： input_voltage;
variable_2 ： output_voltage;
```

```
index_1 ( <float>, … );
index_2 ( <float>, … );
}
…
cell(<cell_name>) { #某个 Power Switching Cell 描述开始
switch_cell_type : coarse_grain; #Switching Cell 类型是 coarse_grain，暂时只支持该类型
…
pg_pin ( <VDD/VSS pin name> ) { #申明电源和地的 pg_pin 格式
pg_type : primary_power | primary_ground; #他们是主电源和主地
direction : input ; #方向是输入
…
}
/* Virtual power and ground pins use “switch_function” to describe the logic to shut off the attached design partition */
pg_pin ( <virtual VDD/VSS pin name>) { #申明内部电源和地，这就是输出电源/地的端口
pg_type : internal_power | internal_ground;
direction: output; #方向是输出
…
switch_function : “<function_string>”; #定义开断控制功能，例如 SLEEP
pg_function : “<function_string>”; #内部电源或地功能与输入的 pg_pin 一致，对于 header switch 来说就是 premary_power 的 Pin Name，对于 Footer
```

Switch 来说就是 primary_ground 的 Pin Name

}

dc_current （<dc_current_name>）｛ #定义不同条件下输出Pin 的稳定电流值，EDA 工具利用该数据计算 IR Drop，并进行 Switch 的优化。

related_switch_pin ：<input_pin_name>; #定义控制开断的Pin

related_pg_pin ：<VDD pin name>; #定义可以被控制开断的电源 Pin，如果是 Footer Switch 则是地 Pin

related_internal_pg_pin ：<Virtual VDD>; #定义不会被关闭的内部电源Pin，Footer Switch 则是地Pin

values(”<float>, …”); #定义不同状态下的该 Cell输出的电流值

}

pin （SLEEP）｛ #Pin SLEEP 定义开始，SLEEP 只是举例

direction ： input;

switch_pin ： true; #表示该输入 Pin 是 switch pin，控制电源/地的开断

…

/* The acknowledge output pin uses “function” to represent the propagated switching signal

*/

pin(<acknowledge_output_pin_name>) ｛ #定义应答输出 Pin 开始，完成开断后，与 switch pin 状态一致，有的 Switch Cell 可能没有该 pin

…

function ：“<function_string>”; #功能定义，应该与SLEEP 状态一致

```
power_down_function : “function_string”; #定义关断后电源状态，如对于Header switch 来说可以是!VDD+VSS，而 Footer Switch 来说可以是!VSS+VDD
direction : output;
…
} /* end pin group */
} /* end cell group */
```

另外，为了更好的控制电源通断控制，可以专门设计带有电源控制的逻辑单元，实现时，不需要添加额外的控制电路，这种电源开关结构叫做Fine Grain Power Switch Cell，结构简单，但每个单元都有一个控制器，面积比较大。下面是一个 Fine Grain Power Switch Cell 的 Liberty 格式描述

```
cell(<cell_name>) { #Fine Grain Power Switch Cell 都是某个Cell 内的一部分，不单独出现
is_macro_cell : true; #定义是不是 macro cell
switch_cell_type : coarse_grain | fine_grain; #多数设置为 fine_grain
pg_pin ( <power/ground pin name> ) { #定义电源信号，primary_是可以断开的，backup_是不会断开的。
pg_type : primary_power | primary_ground | backup_power | backup_ground;
direction: input | inout | output;
…
}
/* This is a special pg pin that uses “switch_function” to describe the logic to shut off the attached design partition */
pg_pin ( <internal power/ground pin name>) { #定义内部电源/地
direction: internal | input | output | inout;
```

```
pg_type : internal_power | internal_ground;
switch_function : “<function_string>”;
pg_function : “<function_string>”;
…
}
pin (<input_pin_name>) {
direction : input | inout;
switch_pin : true | false; #如果是 switch pin 就是 true
…
}
…
pin(<output_pin_name>) {
direction : output | inout;
power_down_function : <function_string>;
…
} /* end pin group */
} /* end cell group */
```

3.2.1.2. Isolation Cells

我们知道，在 CMOS 数字逻辑电路当中，当某根信号为 VDD 时，我们认为是逻辑 1，如果为 GND，则认为是逻辑 0。但当某块电路不供电后，其输出就失去了驱动，输出网络如果没有其他信号驱动，其电平就会为高阻态，换句话说，就是不知道电压是多少。

因此，在低功耗设计中，如果需要切断某块电路电源的话，则该电路的输出就会出现不定值，而与这些输出连接的电路也就出现输入浮空的状态，因此就需要在关断电路的输出及其连接的模块之间加一个 Isolation Cell，在电源关闭时，将输出网络固定在某一个电平上，而打开电源后，该电路的输出等于输入，不影响电路功能。

Isolation Cell可以利用逻辑门来实现，利用与门实现可以使输出在关闭电源时为 0，被称为 Low Clamped Isolated Signal，利用或门实现可以使输出在关闭电源时为 1，被称为 High Clamped Isolated Signal，如下图所示，X 是输出信号，“ISOLN”和”ISOL”分别是开关控制信号：

利用 0&x=0; 1| x =1 的特性;

图 3- 3 逻辑门组成的 Isolation Cell

利用逻辑门组成的 Isolation Cell，会产生延时，对速度很快的电路来说可能会有所影响，因此也可以利用一个 N-MOS 或者一个 P-MOS 以 Pull-Up 和 Pull-Down 电路实现，但这会产生多驱动问题，不是常用的方法。

利用逻辑门组成的 Isolation Cell 是常见的 Cell，下面所示是一个 Isolation Cell 的 Liberty 格式描述：

```
   cell(isolation_cell) { is_isolation_cell : true ; #定义该 Cell 是 isolation_cell
…
pg_pin(<pg_pin_name_P>) { #电源 Pin 定义
pg_type : primary_power;
…
}
pg_pin(<pg_pin_name_G>) { #地 Pin 定义
pg_type : primary_ground ;
…
}
pin (data) {
direction : input;
isolation_cell_data_pin : true ; #被控制的输入 Pin
…
}/* End pin group */
```

```
pin (enable) {
isolation_cell_enable_pin : true ; #控制信号
…
}/* End pin group */
…
pin (output) { #输出信号
direction : output;
power_down_function : (!pg_pin_name_P + pg_pin_name_G); #电源关闭后进入为 power down 状态
…
}/* End pin group */
}/* End Cell group */
```

3.2.1.3. Retention Register Cells

对于寄存器来说，当电源关闭后，其保存的数据也就不存在了，如果有些寄存器的数据，希望在断电后继续保持，则需要使用一个专门的Retention Register Cell，在断电前把寄存器的值锁存起来，重新打开电源后将数值存回寄存器去。而该寄存器在断电时，会有一部分电源一直有电。

下图所示，是一种常用的 Retention Register Cell 基础结构：

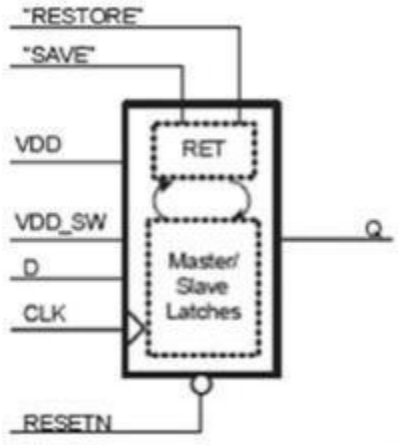


图 3- 4 Retention Register Cell 基础结构图

如上图所示，Master/Slave Latches 是常规的寄存器，工作在 VDD_SW 电压域，也就是可以关断的电压域，D、CLK、RESETN 和 Q 是寄存器的控制和输出信号。RET 电路工作在 VDD 电压域，不会被关闭，当“SAVE”信号有效，RET 会把寄存器的值保存起来，而 RESTORE 有效，RET 将保存的数值写入寄存器中（“SAVE”和“RESTORE”具体时序关系需要参考 IP 提供商提供的数据，这里只是说明一下其功能）。

需要注意的是，一个 Retention Register Cell 比普通寄存器面积要大约 20%，如果设计的鲁棒性好一些，甚至会大超过 50%的面积。

Retention Register Cell 不但是可以保持寄存器的值，还可以保持 Latch 的值，两种不同的结构图如下所示：

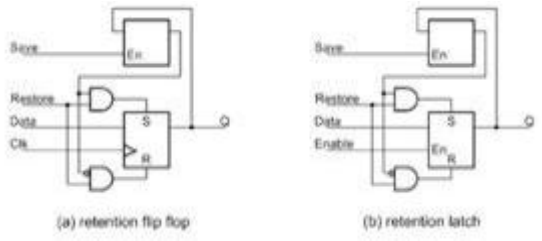


图 3- 5 Retention Cell 的电路结构示意图

图中(a)表示寄存器的 Retention Cell，当 Save 和 Restore 都是 0 的时候，使能保持电路不工作，而寄存器的置位和复位端无效，寄存器正常工作。需要进入 Sleep 模式，首先需要停止寄存器的 Clk（Clk_On 信号关闭），接着准备进入 Sleep 模式。当 Save 为 1 后，Q 端数据被采集进保持电路中，然后可以关闭电源（Power_on 关闭）。打开电源后（Power_on 打开），将 Restore 置 1，如果保持电路输出为 1，则对寄存器做异步置位，否则做异步复位，使寄存器输出与保持电路一致。需要注意的是，Save 为从 0 置 1 后，不可以再有时钟改变 Q 端输出，且 Save 必须有一定的宽度，保证 Q 端数据被记录下来，同时关闭电源一定要等数据被保持下来后才可以进行。打开电源后，Restore 一定要保持一定的时间，使寄存器充分复位或置位，同时在下一个时钟沿到来之前，一定要将 Restore 清 0，否则 Q 端就不会发生变化，电路可能就会出错。当数据被读回后，就可以打开时钟（Clk_On 打开），开始正常工作了。时序示意图如下所示：

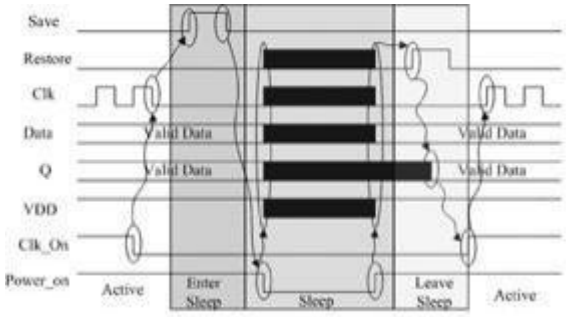


图 3- 6 Retention Register 工作时序示意图

图中(b)是一个 Retention Latch 的电路结构示意图，原理与寄存器的 Retention Cell 类似，这里就不详细描述了。

下面所示是一个 Retention Cell 的 Liberty 格式描述，描述是基于一个常见寄存器或Latch 的格式上进行的：

```
cell(<cell_name>) {
retention_cell : <retention_cell_style>;
pin(<pin_name>) {
retention_pin(<pin_class>, <disable_value>);
…
}
…
}
```

retention_cell_style 主要是定义一个器件识别名，不同功能名字不能一样。这里所说的功能，不止是寄存器或 latch 的功能，还需要考虑到 save 和 restore 过程是否一致。

retention_pin 是的定义表示该 pin 是与 retention cell 功能有关的，其中 pin_class 可以是 restore、save、save_restore，其中 restore 和 save 分表表示该 pin 是 restore 功能和 save 功能，而 save_restore 表示该 pin 根据不同电平为 save 或 restore 功能。disable_value 表示 save 或者 restore 无效时的电平是 1 还是 0。

需要注意的是 retention_pin 定义时一定要有 related_power_pin 和 related_ground_pin 定义其所述的电源域，否则编译时会报 Warning，并为该 Pin 定义 primary_电源域。

同时对于 latch 和寄存器的 Retention Cell 的 latch 和 ff 组描述，会与正常的 latch 和寄存器有所不同，如下，加粗字体为不同的部分：

```
latch (IQ, IQN) {
data_in: D & (SAVE & RESTORE) ;
enable : CLK ;
…
}
```

```
ff(”IQ”, “IQN”) {
next_state : “D & (!SAVE & !RESTORE)” ;
clocked_on : “CP” ;
}
```

3.2.1.4. Always-On Logic Cells

有些 Cell 是不能够被关闭的，如 Power Switch Cell、Retention Register Cells、Isolation Cells，他们就被称为 Always-On Logic Cells。在这些 Cell 的 Liberty 格式描述中就会有一个属性“always-on”是 true。同时对于 Always-On Logic Cells，pg_pin 描述一般都会有两组，primary 和 backup，工具看到该 cell 为 Always On，就会**把 2 组电源地都接到常开的电源/地上**。

```
cell(always_on_cell) {
always_on : true ; #定义该 Cell 是 always_on_cell
…
pg_pin(<pg_pin_name_P>) { #电源 Pin 定义
pg_type : primary_power;
…
}
```

```
pg_pin(<pg_pin_name_G>) { #地 Pin 定义
pg_type : primary_ground ;
…
}
pg_pin(<pg_pin_name_P>) { #电源 Pin 定义
pg_type : backup_power;
…
}
pg_pin(<pg_pin_name_G>) { #地 Pin 定义
pg_type : backup_ground;
…
}
…
}/* End Cell group */
```

3.2.2 利用不同 VT 值的库，实现静态功耗和时序的平衡（Multi-VT）

随着工艺尺寸的减小，Oxide 层厚度的减少，VT 值也一直在减少，这是为了在驱动电压较低的情况下，提高驱动速度。但同时静态功耗也随之增加。

对电路设计来说，并不是每一个部分的电路，路径延时都需要那么小，对于一些非关键路径来说，如果能够使用高 VT 值的元件，则可以在满足时序的前提下减小静态功耗了。

在同一种工艺下，实现不同的 VT 值，可以使用井偏置（Well Bias）技术，使 Substrate 的电压与 Source 的电压存在一定的电压差，就可以改变 VT 值了。使用较多的方法是分别对 N-MOSFET 和 P-MOSFET 增加 1 层 Mask 来提高 VT，或减小 VT。

通常情况下 IP 提供商会提供多套不同的单元库，按照不同的 VT 值进行设计。如 TSMC 90nm LP 工艺的单元库，就会提供普通 VT、High VT、Low VT 以及 Ultra Low VT 四套单元库。下面 TSMC 90nm LP 的四套单元库进行分析：

area

(um*um) Average leakage(nW) Rise delay

(0.04pf,ns) Typical Rise Energe

(0.04pf,pJ) dbtcbn90lphdbwphvttc BUFFD1BWPHVT 2.1952 0.017 0.2444

0.002716 dbtcbn90lphdbwptcBUFFD1BWP 2.1952 0.214 0.237176 0.0027

2 dbtcbn90lphdbwplvttc BUFFD1BWPLVT 2.1952 0.4120.217184 0.002616

dbtcbn90lphdbwpulvttc BUFFD1BWPULVT 2.1952 5.055 0.193556 0.0036

24

表格中 dbtcbn90lphdbwptc 是正常 VT 的库，dbtcbn90lphdbwphvttc 是 High VT 的库，dbtcbn90lphdbwplvttc 是 Low VT 的库，dbtcbn90lphdbwpulvttc 是 Ultra Low VT 的库。分析时使用 1 个门的 Buffer 来进行，面积上都是 2.195um^2。

静态漏电功耗，High VT 库中，1 个门只有 0.017nW，甚至比 SMIC18 METRO 1 个门的漏电功耗还低（参看 3.2.1）。随着 VT 的减小，静态功耗逐渐变大，到 Ultra Low VT 时，其静态功耗，1 个门就有 5.055nW。但也因为 VT 很高，High VT 的 Buffer 延时很慢，比 Ultra Low VT 减慢了约 27%。

从表格看来，延时的变化并不非常明显，如果在时序上要求不是很高，则尽量还是需要使用High VT 的库来实现

3.2.2.1.常用 EDA 工具中 Multi-VT 的实现方法

从 Synopsys Multi-VT 实现过程主要是在逻辑综合（Logic Synthesis）阶段。DC 完成 Multi-VT 的实现，主要是在 target_library 中找出可以使用的所有逻辑单元，并在满足时序约束的情况下，使用最低 leakage power 的单元进行实现。

其实现步骤可以如下：

```
#读入不同 VT 的逻辑单元作为 target library，当 DC 有充分的选择空间
set_target_library {dbtcbn90lphdbwptc.db
dbtcbn90lphdbwphvttc.db \
dbtcbn90lphdbwp lvttc.db
dbtcbn90lphdbwpulvttc.db}
#读入 HDL 代码
read_verilog design_include.v
#link design
current_design design_top
uniquify
link
#读入约束
source design_constraint.tcl
#其他设置
……
#设置最大漏电电流，该设置必须有，否则优化过程不考虑漏电的优化
set_max_leakage_power 0 mw
#开始编译
compile_ultra
```

由于综合时，时序信息并不完全准确，特别是 setup类的时序。因此可以对 setup 时序做稍紧一些的约束，使结果进入 PR 工具后还能够满足时序，如果进入 PR 后仍然无法满足 setup 时序，PR 工具也可以利用类似的原理进行优化。

如在 IC Compiler里，在布线后优化时可以使用如下语句来进行优化：

```
physopt -preserve_footprint -only_power_recovery -post_route
 -incremental
```

这里设置了-preserve_footprint 以及-only_power_recovery，因此只是针对相同footprint 的单元做漏电功耗优化，也就是说如果某个使用了High VT X2 的 Buffer 需要减小延时，则可以替换成 Low VT X2 的 Buffer，这样做可以保持原有的布线结果。

当出现Multi-VT 的单元库后，每个单元库都会多一个 default_threshold_voltage_group 以及 threshold_voltage_group 属性来说明该单元库是 High VT、标准 VT 还是 Low VT。可以利用 report_threshold_voltage_group（DC 和 IC Compiler 都支持）指令报出设计中每种单元库单元占整个设计的百分比，如 High VT 的单元库占60%，标准 VT 占 25%，等。这样做可以使设计者了解不同 VT 对自己设计的影响，如果速度不快，则 Low VT 的库可能占用很少，甚至没有使用，那么完全可以在设计过程中直接不使用该库。

如果库中没有 default_threshold_voltage_group 变量，我们可以在 DC 中设置：

```
set_attr -type string dbtcbn90lphdbwphvtwc.db: dbtcbn90lphdbwphvtwc
default_threshold_voltage_group HVT
```

3.2.3 时钟门控减小不必要的动态功耗

时钟门控单元的插入在 3.1.2 已经有所描述，这里就不多进行描述了。同样的，标准单元库必须提供相应Cell 的库才可以实现。

3.2.4 多供电电压，实现动态功耗与时序的平衡（Multi-Voltage）

我们知道，降低驱动电压 VDD，是减小动态功耗最快的方法，因此在满足时序的情况下，适当降低驱动电压，可以有效的减小动态功耗。而设计中可以使用多驱动电压的设计方法，对于速度要求快的电路，供高一些的驱动电压，如 1.3V，而速度要求不高的模块，则只需要供比较低的驱动电压，如 1.0V。对于逻辑综合来说，DC 中，首先需要对不同电压域的电路设置不同的 operating_condition，综合工具就可以对该电压域电路进行初步分析和优化了。如果使用 UPF，则可以直接使用 load_upf，工具会根据 UPF 的描述自动寻找相应的库文件进行分析。如下所示：

```
set target_library “slow_14V slow 10V
……
read_verilog design_include.v
current_design design_top
link
#set_operating_condition -max slow_10V -max_library
slow_10V
#uDesign/uDMA 速度快，需要使用 1.4V 供电
#set_operating_condition -max slow_14V -max_library
slow_14V -object_list \
#uDesign/uDMA
load_upf power.upf
……
check_mv_design
compile -scan
check_mv_design
……
```

这里 check_mv_design 主要是检查 UPF 对设计的描述是否正确，在 compile 之后再做一次是为了查看 compile后，电路结构域 UPF 的描述是否正确。

接着，需要在不同电压电路之间，添加 Level_shifters，进行电压转换。如下所示：

```
check_level_shifters  insert_level_shifters  -all_clock_nets
-verbose
```

DC 会根据不同电压域设置 operating condition 中 voltage 的数值或者 UPF 中的描述，查找 Level Shifter Cell 中 input_voltage_range 和 output_voltage_range 满足这些电压域需求的元件，在 2 个电压域的数据交互信号上添加 Level Shifter Cells。

3.2.4.1.常用 EDA 工具中 Multi-VT 的实现方法

Level Shifters 主要是在多供电电压设计中，在 2 个不同电压域之间进行电压转换的器件，将某个电压域输出的逻辑电平转换成另外一个电压域可以识别的逻辑电平。

从功能上来看，就像一个Buffer。

下面描述的是 level_shifter 的 Liberty 格式：

```
cell(level_shifter) {
is_level_shifter : true ; #定义为 true，则该 Cell 被认为是 lever shifter cell
level_shifter_type : HL | LH | HL_LH ; #定义电平转换方向，HL 表示高电平转低电平，LH 直低电平转高电平，HL_LH 表示都可以用
input_voltage_range (<float>, <float>); #输入电压范围，指该Cell输入连接的电路，电压工作范围，EDA 工具会根据 Operating Condition 的选择，检查该 Cell 是否满足电路需要，可以在功能 pin 中进行定义，如果在 cell主体定义，则必须与 output_voltage_range 同时存在
output_voltage_range (<float>, <float>) ;#输出电压范围，指该Cell输出连接的电路，电压工作范围，
```

EDA 工具会根据 Operating Condition 的选择，检查该 Cell 是否满足电路需要，可以在功能 pin 中进行定义，如果在 cell主体定义，则必须与

input_voltage_range 同时存在

…

pg_pin(<pg_pin_name_P>) {

pg_type : primary_power;

std_cell_main_rail : true; #该 primary_power 连接在 Cell 设计中的主 rail

…

}

pg_pin(<pg_pin_name_G>) {

pg_type : primary_ground ;

…

}

pin (data) {

direction : input;

input_signal_level : “<voltage_rail_name>”; #输入信号电压环名

input_voltage_range (<float> , <float>) ;

level_shifter_data_pin : true ; #数据功能 Pin

…

}/* End pin group */

pin (enable) {

direction : input;

input_voltage_range (<float> , <float>) ;

level_shifter_enable_pin : true ; #使能 Pin，如果 level shifter 还作为 isolation cell 的时候，使能信号会在电源关闭时停止 level shifter 功能，仅

```
仅作为 isolation cell 存在
…
}/* End pin group */
pin (output) {
direction : output;
output_voltage_range ( <float> , <float>);
power_down_function : (!pg_pin_name_P +
pg_pin_name_G);
…
}/* End pin group */
…
}/* End Cell group */
```

3.2.4.2. Liberty PG Pin 格式

Liberty PG Pin 格式，就是在传统的 Liberty 格式文件中，加上与 Power 有关的信息。在工艺尺寸较大的时候，逻辑单元基本上只工作在一个电源系统中，但在使用多电压设计的小工艺尺寸库文件中，则需要告诉分析工具，该 Cell 的电源和地是接在哪里的，每个 pin 所处的电压域在哪里？（实际上 IO 设计中应该已经有类似的数据，但因为不涉及到多电压设计的方法，在传统的 IO Liberty 中也很少见到 PG Pin 的格式）

下面所示是 PG Pin 格式的描述

```
library (slow) { #library “slow” 描述开
始
⋮
voltage_map (vdd, 1.2); # 有一个 1.2V 的驱动电压，
名为“vdd”
voltage_map (vss, 0.0); # 有一个 0V 的驱动电压，
名为“vss”
…
operating_conditions(slow_12V) { #Operating
```

```
Condition “slow_12V” 定义开始
…
voltage : 1.2; #使用 1.2V 作为该 condition 的电压条件
…
} #Operating Condition 定义结束
…
default_operating_conditions : slow_12V;
#default 使用” slow_12V”
cell(BUFFX1) { # BUFFX1 Cell 描述开始
pg_pin (“VDD”) { # BUFFX1 有一个 Pin “VDD”，定义为 pg_pin
voltage_name : “vdd”; # Pin “VDD” 电压是接在 voltage_map ” vdd” 上的，即 1.2V
pg_type : “primary_power” # Pin “VDD” 是” primary_power” 类型的
}
pg_pin (“VSS”) { # BUFFX1 有一个 Pin “VSS”，定义为 pg_pin
voltage_name : “vss”; # Pin “VSS” 电压是接在 voltage_map ” vss” 上的，即 0V
pg_type : “primary_ground” # Pin “VSS” 是” primary_ground” 类型的
}
…
leakage_power() { #leakage_power 定义开始
related_pg_pin : VDD; # VDD Pin 供电时的 leakage power
…
```

```
}#leakage_power 定义结束
…
pin (A) { #有一个功能 Pin ”A”
direction : input ; #”A”是一个输入
related_power_pin : VDD; #”A”是 VDD 供电电压域的
related_ground_pin : VSS ; #”A”是 VSS 供地的电压域信号
…
}# “A”申明结束
…
pin (Y) { #有一个功能 Pin “Y”
direction : output ; #”Y”是一个输出
power_down_function : (!VDD + VSS) ; #当 VDD 关断，VSS 开着的时候，输出被关断
related_power_pin : VDD; #”Y”是 VDD 供电电压域的
related_ground_pin : VSS ; #”Y”是 VSS 供地的电压域信号
internal_power() { #internal_power 申明开始
related_pg_pin : VDD; #internal power 由 VDD 供电引起
…
} /* end internal_power group */
…
}/* end pin group*/
…
}/* end cell group*/
…
```

```
}/* end library group*/
```

在 PG Pin 格式中，有一个 pg_type 的属性，下表描述该属性的含义：

pg_type	简单描述	具体描述
primary_power	主电源	一般功能信号使用的电源，可以关断
primary_ground	主地	一般功能信号使用的地，可以关断
backup_power	备用电源	Always-on 的电源，主要用于 always-on logic cell 的常开电源
backup_ground	备用地	Always-on 的地，主要用于 always-on logic cell 的常开地
internal_power	内部电源	主要用于 Power Switch cell 的内部电源
internal_ground	内部地	主要用于 Power Switch cell 的内部地

3.2.4.3. 电压和频率的 Scaling 分析

有的时候，IP 提供商可能无法提供足够的单元库，例如提供商可能提供了 1.4V 的单元库和 1.0V 的单元库，但我们希望提供一个 1.2V 的电压，则需要进行电压的 Scaling。

首先我们需要建立一个独立的 operating_conditions，基于 slow_14V 的库，电压是 1.2V：

```
create_operating_conditions  -name slow_12V  -library slow_14V
 -process 1  -voltage 1.2  -temperature 125
```

然后对于需要使用 1.2V 的子电路，需要设置 operating_condition 为 slow_12V。分析和优化工具，会首先根据 slow_14V 提供的model 计算出在 1.4V 驱动下，延时、功耗、电容等信息，然后利用 slow_14V 提供的 k_factor 进行 scaling，计算出 1.2V 条件下的数值。

利用这个方法可以在 IP 提供商没有提供所需分析环境的情况下，利用已有模型估计出在其他环境中出现的情况。但需要注意的是，当条件相差很大时，Scaling 的结果偏差会很大，建议不要使用这种方式进行 sign-off 工作。

至于频率Scaling，则是在应用过程中可以减少频率，以减少翻转率，达到低功耗应用的需求。

Scaling 的使用最好不要超过标准单元库的 10%，否则结果完全不可行，同时如果采用CCS格式的电流源模型Liberty，则Scaling结果会比普通的NLDM格式 Liberty 更精确一些。

3.3 其他未提及工艺

实际上，上述提到的 2 种工艺范围，甚至是现在已有的所有工艺技术，从本质上想进行低功耗设计，其思路都是一致的，方法无外乎关断电源、低压驱动（多电压混合设计）、Multi-VT 等方法，只是当工艺尺寸下降到一定程度后，IP 提供商能提供足够的数据，使 EDA 工具直接使用，减少工程师重复劳动的工作量。

所以低功耗设计，首先应该从系统设计下手，在一个设计中，应该使用怎样的低功耗策略，采用那种工艺尺寸，有没有合适的单元库，如果没有库，是否自己可以设计，这些都必须首先考虑清楚。而工具，只是为了帮助我们实现自己的想法。

4.0 一个低功耗设计实现的例子

这里提供一个 DEMO（pl8051_extend_chip），可以使读者更快的理解低功耗设计的基础。下图所示，是该设计的功能结构图：

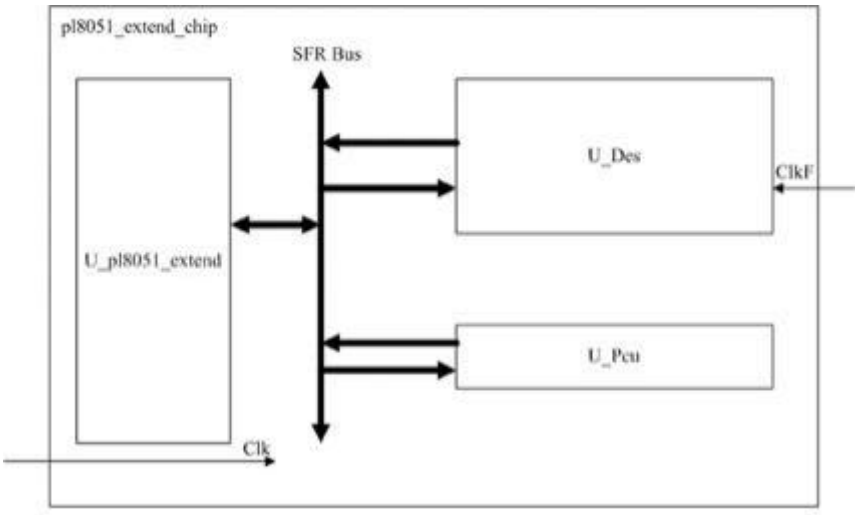


图 4- 1 pl8051_extend_chip 功能结构图

由图 4- 1 可以看出，整个设计主要是由一个 8051 控制器，通过 SFR 总线对其他模块进行控制，U_Des 是一个算法模块，U_Pcu 是功耗控制单元（Power Control Unit）。整个设计都在 Clk 的控制下进行工作，只有 U_Des 工作在 ClkF 下。

这里假设 Clk（28ns）是 ClkF（7ns）的 4 分频时钟，且与 ClkF 同源同向，这样才可以保证 U_Des 控制的正确性。

由于 U_Des 需要在较快的频率下工作，因此需要对 U_Des 进行一些特殊的设计，也就是对整个设计进行低功耗设计，如下所示：

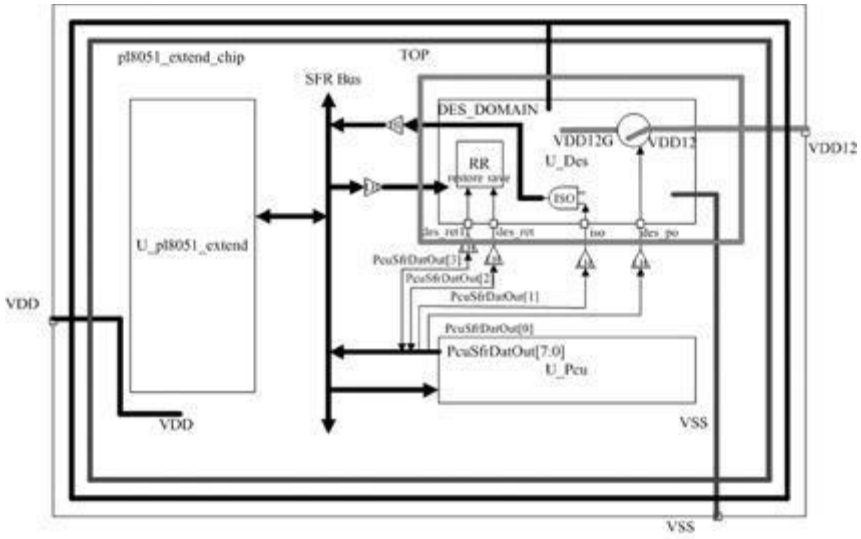


图 4- 2 低功耗设计方案示例

上图主要描述以下信息：

1. 2 个电压域，TOP 以及 DES_DOMAIN。

2. 电源设计外部提供，分 1.2V 的 VDD12 和 1.0V 的 VDD10

3. 设计中除 DES 电路几乎所有电路工作在 1.0V 的 VDD 电压域。

4. U_Des 由于工作速度要求较高，工作在 1.2V 的 VDD12 电压域 DES_DOMAIN，但由于不使用的时候需要关断，以降低静态功耗，因此，通过一组

PowerSwitch 进行控制，控制信号来源为 U_Pcu 的 PcuSfrDatOut[0]输出。该电压域与 TOP 电压域共用VSS 地信号。

5. 由于 U_Des 电路处于 VDD12 电压域，而 U_Pcu 处于 VDD 电压域，因此需要添加 Level Shifter 进行电平转换。

6. U_Des 电路输出信号，需要通过一个 Isolation Cell，在关断电源时提供稳定电平，而该电平为 1.2V，因此还需要利用 Level Shifter 转换成 1.0V 电压域信号，Isolation Cell 的 Enable 信号来自于 U_Pcu 模块的

PcuSfrDatOut[1]信号。

7. U_Des 电路中的寄存器需要使用 Retention Register，save 和 restore 控制信号分别来自于 U_Pcu 输出 PcuSfrDatOut[2]和 PcuSfrDatOut[3]经过 Level Shifter 产生的信号。

Synopsys 公司为低功耗设计提供了一整套的解决方案，即 UPF 设计流程。UPF（Unify Power Format）是一种描述功耗设计思想的文件，Synopsys Galaxy Implementation Platform 只需要读取 UPF 文件，就可以将低功耗设计思路实现，而 Synopsys Discovery Verification Platform 读取 UPF 后，可以对低功耗思路以及最终实现的电路进行验证。

接下来，我们利用UPF 描述该 DEMO 的功耗设计思路。

4.1 申明电压域以及虚拟电压接口

首先申明电压域，并申明 VDD、VDD12 以及 VSS 等电压端口：

```
create_power_domain TOP
create_power_domain DES_DOMAIN_domain
 -elements U_Des
create_supply_port VDD
create_supply_port VDD12
create_supply_port VSS
```

根据上述指令，工具会识别出如下图所示中画圈的信息，电压域及虚拟电压端口开始出现：

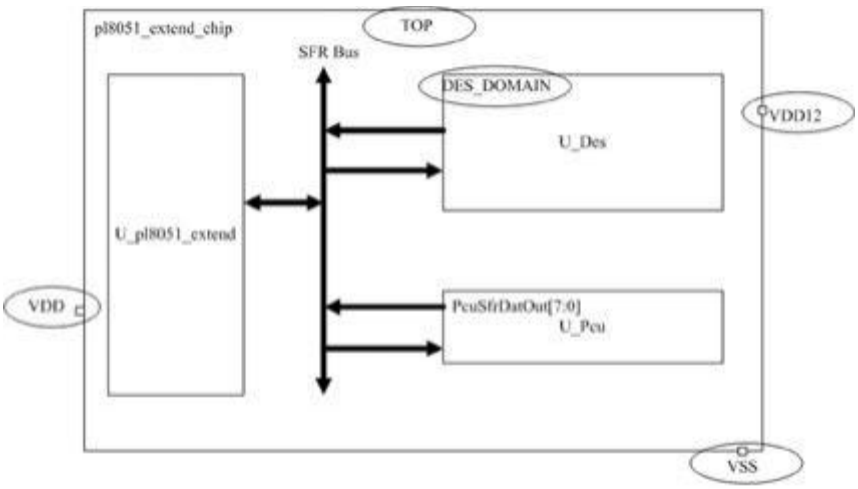


图 4- 3 电压域及虚拟电压端口信息

需要注意的是电压端口只不过是虚拟存在的，并不是一定在顶层连接。

4.2 申明电源网络

接着申明每个电压域中的电源网络，建议从顶层往底层进行申明，如首先申明 TOP 的 VSS，VDD。：

```
create_supply_net VDD
create_supply_net VSS
```

以上语句添加下图中画圈部分：

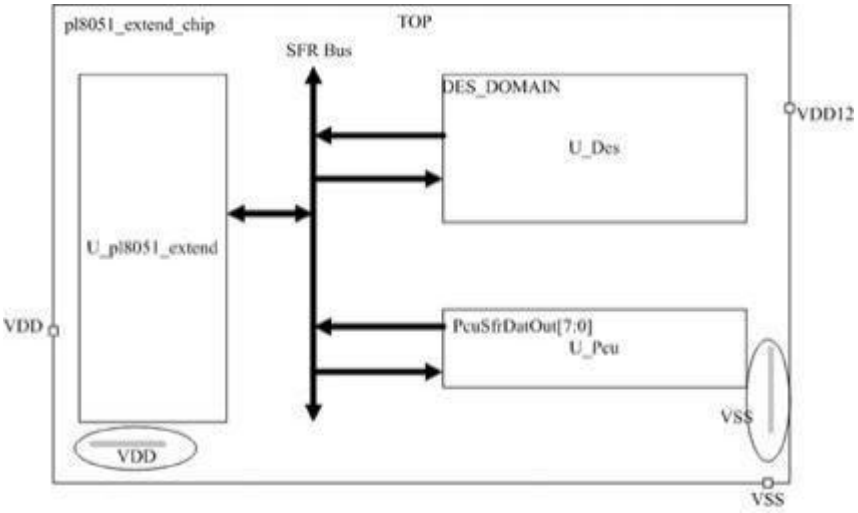


图 4- 4 产生顶层电源网络

接着申明 DES_DOMAIN 的电源网络，但需要注意的是，如果需要插入Level Shifter，最好是在与其有连接关系的电源域（TOP）同时申请电源线，并连接起来：

```
create_supply_net VDD12
create_supply_net VDD12  -domain DES_DOMAIN
 -reuse #DES_DOMAIN 的 VDD12 与 TOP 的 VSS 共用一个网络，因此需要-reuse
create_supply_net VDD12G
create_supply_net VDD12G  -domain DES_DOMAIN
-reuse
create_supply_net VSS  -domain DES_DOMAIN
 -reuse
create_supply_net VDD  -domain DES_DOMAIN
 -reuse
```

以上语句添加下图中画圈部分：

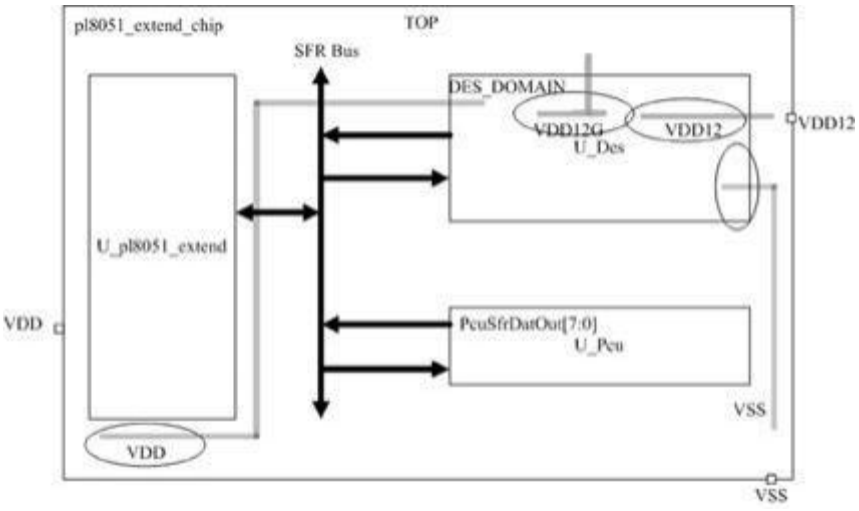


图 4- 5 子电路的电源网路申明

然后将电源网络和电源端口连接起来：

```
connect_supply_net VDD12  -port  {VDD12}
connect_supply_net VDD  -port  {VDD}
connect_supply_net VSS -port  {VSS}
```

以上语句添加下图中画圈部分：

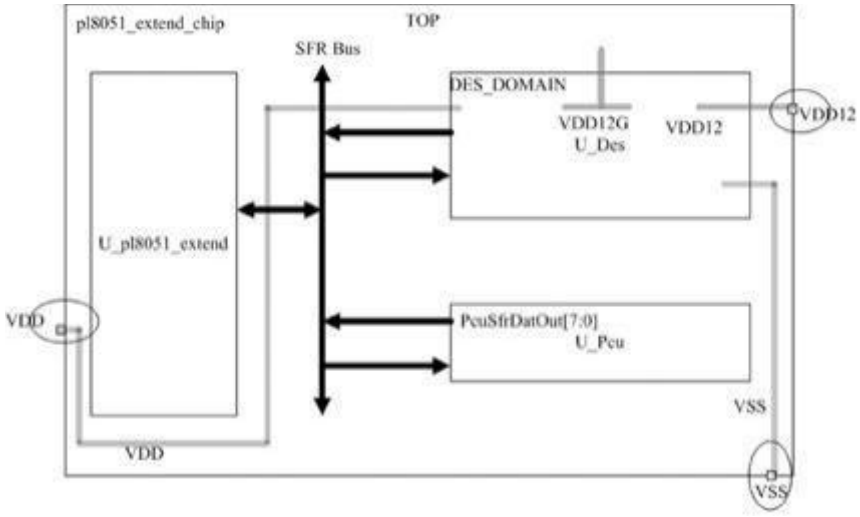


图 4- 6 连接电源网络和电源端口

最后为每个电源域申明主电源网络来源，主电源就是该电压域普通逻辑工作使用的电源：

```
set_domain_supply_net TOP  -primary_power_net
VDD  -primary_ground_net VSS
set_domain_supply_net DES_DOMAIN
 -primary_power_net VDD12G  -primary_ground_net
VSS
```

做完这一步，电源网络申明基本完成

4.3 添加 Power Switch

这添加 Power Switch 主要是 2 步，第一步创建 Power Switch，第二步 map Power Switch。可以参看以下脚本进行：

```
# Power Switch
create_power_switch des_sw \
    -domain DES_DOMAIN \
    -input_supply_port  {in VDD12} \
    -output_supply_port  {out VDD12G} \
    -control_port  {des_po U_Pcu/PcuSfrDatOut[0]} \
    -on_state  {state_on in  {!des_po}} \
    -off_state  {OFF  {des_po}}
```

```
map_power_switch des_sw \
    -domain DES_DOMAIN \
    -lib_cell HDRSIHVTDO
```

create_power_switch 和 map_power_switch 在综合时只会检查语法，不会有实际效果，但在综合时将该 UPF 指令读入系统，可以由DC 输出相应的 UPF 给 Synopsys 的 PR 工具，PR 工具可以自动识别这些指令，到时才会真正添加 Power Switch。2 句命令的结果示意在下图画圈中：

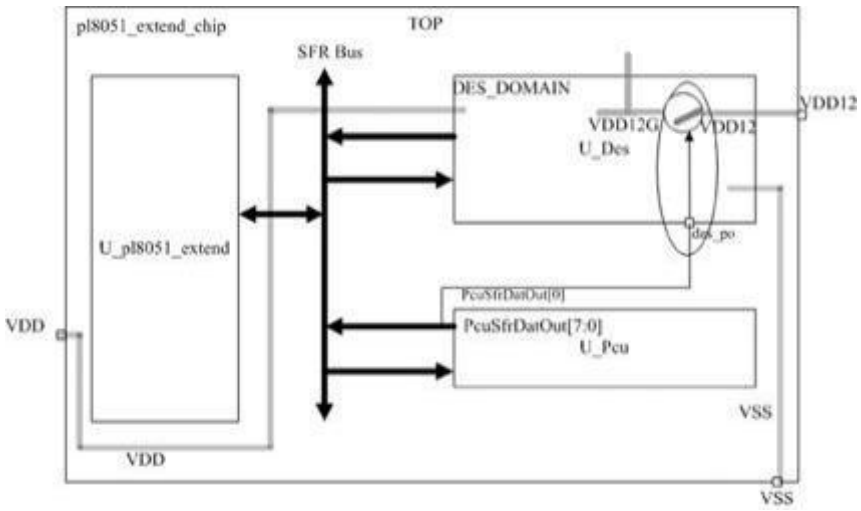


图 4- 7 添加 Power Switch Cell

4.4 建立电源状态表格（Power State Table，PST）

在 Power Switch 插入以后，需要对电压域的有效电压，以及不同电压域之间的关系进行设置，同时建立电压分布及关断的情景（scenarios），可以用于静态电压分析、仿真等。

建立电源状态表格，可以参看以下脚本：

```
add_port_state VDD -state {LV 1.0} #VDD 是 1.0V，属于低电压域（LV）状态
add_port_state VDD12 -state {HV 1.2} #VDD12 是 1.2V，属于高电压域（HV）状态
add_port_state des_sw/out -state {HV 1.2} \ #Power Switch 输
```

```
出是 1.2V，属于高电压域（HV）状态
                    -state {OFF off} #关闭状态（OFF），无电压（off）
add_port_state VSS  -state {ON 0.0} #VSS 处于常开状态（ON），电压 0V
create_pst design_pst  -supplies {VDD VDD12 VDD12G} #建立一个 PST，由 3 个电源状态不同组成
add_pst_state design_work  -pst design_pst  -state {LV HV HV} #design_work 状态下，三组电源都开
add_pst_state stand_by  -pst top_pst  -state {LV HV OFF} #stand_by 状态下，des_sw/out 输出电源为关闭状态
```

需要注意 add_pst_state 的-state 括号里的描述顺序，必须与 create_pst 的-supplies 里一致，选择的状态必须与 add_port_state 的-state 里描述的一致才可以。

这样就建立了下表所示的 PST，design_pst：

	VDD	VDD12	VDD12G
design_work	LV	HV	HV
stand_by	LV	HV	OF

由此建立了 PST，分析工具或仿真工具就知道需要对哪些状态进行分析了。

4.5 插入 Isolation Cell

下面需要为 DES_DOMAIN 输出给 TOP 的输出信号添加 Isolation Cell。脚本如下所示

```
# Isolation Cell
set_isolation des_iso_out \
    -domain DES_DOMAIN \
    -isolation_power_net VDD12 -isolation_ground_net VSS \ #隔离后只有 VDD12 供电
```

```
    -clamp_value 1 \ #关电后输出逻辑 1
    -applies_to outputs #对于所有输出添加

set_isolation_control des_iso_out \
    -domain DES_DOMAIN \
    -isolation_signal U_Pcu/PcuSfrDatOut[1] \
    -isolation_sense high \ #控制信号为高时 ISO 有效
-location self # Isolation Cell 添加在 DES_DOMAIN 里面
```

这样就可以产生如下图所示画圈部分电路：

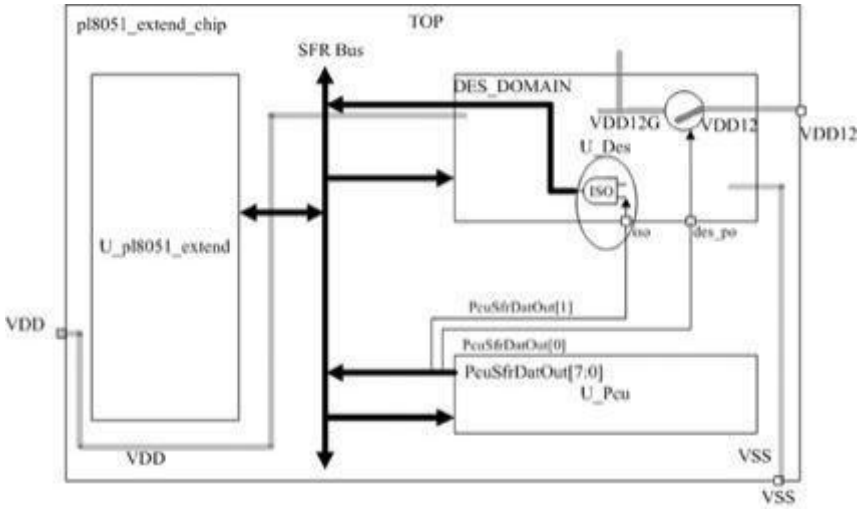


图 4- 8 插入 Isolation Cell

4.6 替换 Retention Register Cell

接着需要使用 Retention Register Cell 将 DES_DOMAIN 中在关断电源后需要保留数据的寄存器替换为 Retention Register Cell，可以使用如下脚本：

```
# Retention Register
set_retention des_ret -domain DES_DOMAIN \
    -retention_power_net VDD12G
-retention_ground_net VSS #retention_power_net 是关电
```

的 Power，表示需要保持 VDD12G 电源域的数据

```
set_retention_control des_ret -domain DES_DOMAIN \
    -save_signal  {U_Pcu/PcuSfrDatOut[2] high} \
    -restore_signal  {U_Pcu/PcuSfrDatOut[3] low}

map_retention_cell des_ret  \
    -domain DES_DOMAIN \
-lib_cell_type RSDFCSD1 #RSDFCSD1 是带有异步置位的
Retention Register Type，可以从 lib 文件中
retention_cell 属性中看到。
```

根据以上描述，会按照下图画圈示意图描述电路：

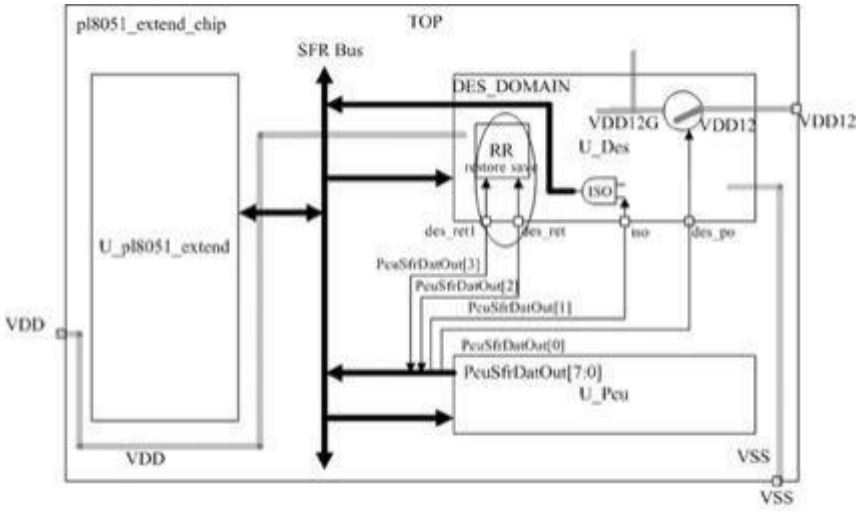


图 4- 9 替换 Retention Register Cell

4.7 插入 Level Shifters

插入 Level Shifters 是根据 PST 的定义进行，根据 PST 定义看来，DES_DOMAIN 和 TOP 之间需要插入 Level Shifters，其中 DES_DOMAIN 输出的 pin（Isolation Cell 的输出）需要插入 H->L 的 Level Shifter，而其输入则需要插入 L->H 的 Level Shifter。

插入 Level Shifter 在 Compile 过程中自动进行，也可以利用 UPF 命令 set_level_shifter 定义插入的规则。

插入Level Shifter 后产生的电路变化示意图如下图画圈处所示：

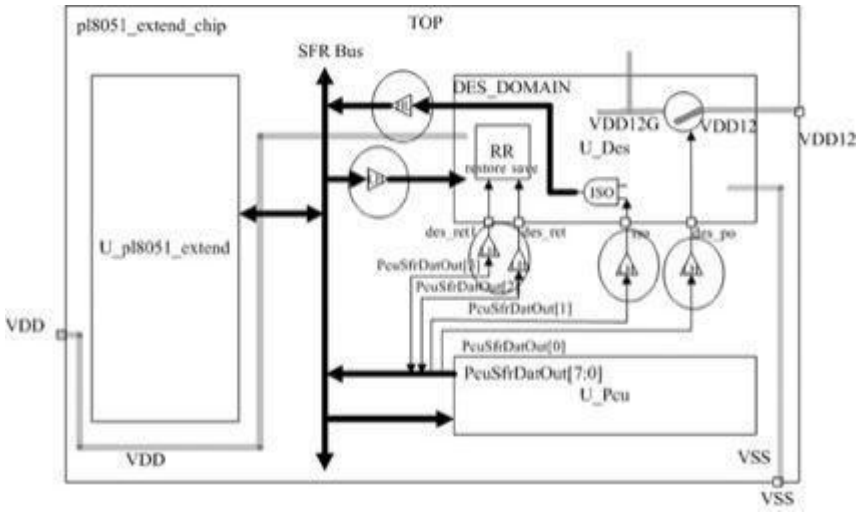


图 4- 10 插入 Level Shifters

注意事项：

level shift & isolation 是在综合时插入,

Power swith 是在 PR 时插入;

5.0 UPF Demo

```
########################
## CREATE POWER DOMAIS
########################
create_power_domain TOP
create_power_domain DES_DOMAIN -elements U_Des

########################
## TOPLEVEL CONNECTIONS
########################
#VDD
```

```
create_supply_port VDD
create_supply_net VDD -domain TOP
create_supply_net VDD -domain DES_DOMAIN -reuse
connect_supply_net VDD -port VDD

#VSS
create_supply_port VSS
create_supply_net VSS -domain TOP
create_supply_net VSS -domain DES_DOMAIN -reuse
connect_supply_net VSS -port VSS

#VDD12
create_supply_port VDD12
create_supply_net VDD12 -domain TOP
create_supply_net VDD12 -domain DES_DOMAIN -reuse
connect_supply_net VDD12 -port VDD12

#VDD12G
create_supply_net VDD12G -domain DES_DOMAIN
create_supply_net VDD12G -domain TOP -reuse

######################
## PRIMARY POWER NETS
######################
set_domain_supply_net TOP   -primary_power_net
VDD   -primary_ground_net VSS
set_domain_supply_net
DES_DOMAIN  -primary_power_net VDD12G
-primary_ground_net VSS
```

```
####################
## DES_DOMAIN SETUP
####################
# Power Switch
create_power_switch des_sw \
    -domain DES_DOMAIN \
    -input_supply_port  {in VDD12} \
    -output_supply_port  {out VDD12G} \
    -control_port  {des_po U_Pcu/PcuSfrDatOut[0]} \
    -on_state  {state_on in  {!des_po}} \
    -off_state  {OFF  {des_po}}

map_power_switch des_sw \
    -domain DES_DOMAIN \
    -lib_cell HDRSIHVTD0

# Isolation Cell
set_isolation des_iso_out \
    -domain DES_DOMAIN \
    -isolation_power_net VDD12 -isolation_ground_net
VSS \
    -clamp_value 1 \
    -applies_to outputs

set_isolation_control des_iso_out \
    -domain DES_DOMAIN \
    -isolation_signal U_Pcu/PcuSfrDatOut[1] \
    -isolation_sense high \
```

```
    -location self

# Retention Register
set_retention des_ret -domain DES_DOMAIN \
    -retention_power_net VDD12G
-retention_ground_net VSS

set_retention_control des_ret -domain DES_DOMAIN \
    -save_signal  {U_Pcu/PcuSfrDatOut[2] high} \
    -restore_signal  {U_Pcu/PcuSfrDatOut[3] low}

map_retention_cell des_ret \
    -domain DES_DOMAIN \
    -lib_cell_type RSDFCRD1

# Level Shifter
set_level_shifter des_ls_lh \
    -domain DES_DOMAIN \
    -applies_to inputs \
    -threshold 0.1 \
    -rule low_to_high \
    -location parent

set_level_shifter des_ls_hl \
    -domain DES_DOMAIN \
    -applies_to outputs \
    -threshold 0.1 \
    -rule high_to_low \
    -location parent
```

```
#####################
# ADD PORT STATE INFO
#####################
add_port_state VDD -state {LV 0.84}
add_port_state VDD12 -state {HV 1.08}
add_port_state VSS -state {ON 0.0}
add_port_state des_sw/out -state {HV 1.08} \
                          -state {OFF off}

#############
## CREATE PST
#############
create_pst design_pst -supplies {VDD VDD12 VDD12G VSS}
add_pst_state design_work -pst design_pst -state {LV
HV HV ON}
add_pst_state stand_by -pst design_pst -state {LV HV
OFF ON}
```

4.9 DC UPF Flow Script Demo

```
source -e ../scr/synopsys_dc.setup
source -e ../scr/ variable.tcl
set_clock_gating_style -min ${CG_MIN_BITWIDTH} -max_fanout
${CG_MAX_FANOUT} \
                              -sequential_cell
${CG_SEQ_CELL} \
                              -positive_edge_logic
${CG_POS_CELL_LIST} \
                              -negative_edge_logic
${CG_NEG_CELL_LIST} \
```

```
                                -control_point
${CG_CONTROL_POINT} \
                                -control_signal
${CG_CONTROL_SIGNAL} \
                                -setup ${CG_SETUP_VALUE}
-hold ${CG_HOLD_VALUE}
set_operand_isolation_style -logic adaptive
set_operand_isolation_slack 0.1
set_svf ${NETLIST_PATH}${top}${VER}.svf
read_verilog ${RTL_INCLUDE}
current_design $top
uniquify
link
load_upf ${SCRIPT_PATH}power.upf
set auto_insert_level_shifters_on_clocks all ;#can be added
in backend flow
set_voltage  1.08 -object_list  {VDD12 VDD12G}
set_voltage 0.84 -object_list VDD
set_voltage 0.0 -object_list VSS
source -e -v ${SCRIPT_PATH}timing.tcl
#M1~M9 are avaliable, M1 will be used by STD
#set_ignored_layers -min_routing_layer M4
set_ignored_layers -max_routing_layer M7
report_ignored_layers >
${REPORT_PATH}${top}_ignored_layers.rpt
set_clock_gating_check -setup ${CG_SETUP_CHECK} -hold
${CG_HOLD_CHECK}  [all_clocks]
set_fix_multiple_port_nets -feedthroughs -outputs
-buffer_constant
```

```
set_attr -type string
tcbn90lphpwc0d70d9_pg.db:tcbn90lphpwc0d70d9
default_threshold_voltage_group SVT_LS_LH
set_attr -type
stringtcbn90lphphvtwc0d70d9_pg.db:tcbn90lphphvtwc0d70d9
default_threshold_voltage_group HVT_LP_LH
set_attr -type string
tcbn90lphpwc0d7_pg.db:tcbn90lphpwc0d7
default_threshold_voltage_group SVT_LV
set_attr -type string
tcbn90lphphvtwc0d7_pg.db:tcbn90lphphvtwc0d7
default_threshold_voltage_group HVT_LV
set_attr -type string tcbn90lphpwc_pg.db:tcbn90lphpwc
default_threshold_voltage_group SVT
set_attr -type string
tcbn90lphphvtwc_pg.db:tcbn90lphphvtwc
default_threshold_voltage_group HVT
set_attr -type string
tcbn90lphphvtcgwc_pg.db:tcbn90lphphvtcgwc
default_threshold_voltage_group HVT_CG
set_attr -type string
tcbn90lphphvtwc0d90d9_pg.db:tcbn90lphphvtwc0d90d9
default_threshold_voltage_group HVT_ISO
set_attr -type string
tcbn90lphphvtwc0d90d7_pg.db:tcbn90lphphvtwc0d90d7
default_threshold_voltage_group HVT_LS_HL
set_attr -type string
tcbn90lphpwc0d90d7_pg.db:tcbn90lphpwc0d90d7
default_threshold_voltage_group SVT_LS_HL
```

```
check_mv_design -verbose
 compile_ultra -scan -no_autoungroup
-no_boundary_optimization
change_names -rules verilog -hierarchy
write -format ddc -hierarchy -output
${DDC_PATH}${top}_noscan${VER}.ddc
write -format verilog -hierarchy -output
${NETLIST_PATH}${top}_noscan${VER}.v
set write_sdc_output_lumped_net_capacitance false
set write_sdc_output_net_resistance false
write_sdc -version 1.5 ${SDC_PATH}${top}_cons${VER}.sdc
check_design >
${REPORT_PATH}${top}_check_design_postcomp.rpt
report_area > ${REPORT_PATH}${top}_area.rpt
check_mv_design  -verbose >
${REPORT_PATH}${top}_check_mv_design.rpt
report_constraint -all > ${REPORT_PATH}${top}_all_vio.rpt
report_clock_gating -gating_elements >>
${REPORT_PATH}${top}_icg.rpt
report_operand_isolation -verbose -isolated >
${REPORT_PATH}${top}_opiso.rpt
report_threshold_voltage_group >
${REPORT_PATH}${top}_VT.rpt
report_threshold_voltage_group -verbose >>
${REPORT_PATH}${top}_VT.rpt
save_upf ${NETLIST_PATH}${top}${VER}_dc.upf #upf’ for next
part of flow
```

6.0 Cpf demo 分析

```
set_cpf_version 1.1          ##cpf 版本
set_hierarchy_separator "/"    ##层次化时用/符号

# define the library sets
define_library_set -name prjxxxx5072_lib_wc -libraries { \    ##定义综合需要使用到的库组合成 set,名字叫 prjxxxx5072_lib_wc
/ic02/projects/PRJXXXX5072/lib/std/syn/ss_1v62_125c.lib     \
/ic02/projects/PRJXXXX5072/lib/io/syn/SP018WP_V0p2_max.lib  \
/ic02/projects/PRJXXXX5072/lib/mem/PRJXXXXDROM16KX8/PRJXXXXDROM16KX8_SS_1.62V_125C.lib     \
/ic02/projects/PRJXXXX5072/lib/mem/PRJXXXXDROM2816X10/PRJXXXXDROM2816X10_SS_1.62V_125C.lib \
/ic02/projects/PRJXXXX5072/lib/mem/spsram1152x32/spsram1152x32_slow_syn.lib \
/ic02/projects/PRJXXXX5072/lib/mem/spsram1856x44/spsram1856x44_slow_syn.lib \
/ic02/projects/PRJXXXX5072/lib/otp/syn/E00010K8PCL3_S90_ss.lib \
/ic02/projects/PRJXXXX5072/lib/audio_dac/syn/SMIC18_DAC_01E_ss.lib \
/ic02/projects/PRJXXXX5072/lib/lp/syn/lp_ss_1v62_125c.lib \
}

# define the isolation cells   定义 isolation 单元使用哪种 cell
define_isolation_cell -cells ISO_AND2X2M -enable A -valid_location to
define_isolation_cell -cells ISO_OR2X2M -enable A -valid_location to
define_isolation_cell -cells ISO_TLATNX2M -enable GN -valid_location to
```

-eable A ,端口A 用来作为使能端,与它相连的信号应该是不可关断的信号,
-valid_location: isolation 应该放在的位置,to 选项,这种 isolation 只能放在与目标电源域兼容的电源域,且用在off to on 的电源域关系,即信号从 A ->B,A 电源关闭,B 电源 ON;不能用于 A 电源开启,B 电源关闭的情况,因为它的基本功能和 isolation 功能都由第二电源产生(目标电源域) ; from 选项, 这种 isolation 只能放在与源电源域兼容的电源域,且用在 off to on 的电源域关系,即信号从 A ->B,A 电源关闭,B 电源 ON;不能用于 A 电源开启,B 电源关闭的情况,因为它的基本功能由源电源域产生,isolation 功能都由第二电源产生(目标电源域) ;

```
# define the power switch cells  定义 power switch cell
define_power_switch_cell -cells POWERSWITCH \
-power_switchable VDD \   ##该 power switch 的输出电源,由
stage_1 enable 控制是否关闭
-power TVDD \   ##该 power switch 的输入电源
-stage_1 enable NSLEEPIN \    ##控制 switch 的开关信号,只有一个阶段
(可以配置 2 个阶段,默认一个阶段)
-stage_1 output NSLEEPOUT \   ##当打开此 switch 时,会输出一个信号,
可以为buffer 输出或反向输出,默认 buffer 输出
-type header    ##关断电源 powr 侧

#####################################################
# Design part of the CPF
#####################################################
set_design prjxxxx5072

# create power domains
create_power_domain -name PD1 \   ##电源域的名字为 PD1
-instances { \   ##instances 包括的模块属于 PD1 电源域
```

I_prjxxxx5072_rtcpon_pad \

I_prjxxxx5072_rtcpon \

I_rtcpon_spare_cells } \

-boundary_ports { \

RTC_WK XI32K XO32K COBOTP_BIST_EN_N} ##只有指定的 port 为该 power domain,没指定的端口则默认为对应的电源域

create_power_domain -name PD2 - default ##设计中没明显划分到某个电源域的模块都默认属于该电源域,其实它是一个pure virtual power domain

#-external_controlled_shutoff -shutoff_condition {!VDDLDO}

create_power_domain -name PD3 \

-shutoff_condition {!I_prjxxxx5072_rtcpon/power_sw_on_last} \ ##此电源域关闭的条件

-instances {I_prjxxxx5072_poff_clk \

I_prjxxxx5072_poff_core \

I_prjxxxx5072_poff_pad \

I_poff_spare_cells_grp0 \

I_poff_spare_cells_grp1 \

I_poff_spare_cells_grp2 \

} \

-base_domains PD2 ##PD3 的 POWER 是从 PD2 通过一个 switch 输出的

注意,power domain 有一个选项 -power_down_states {low|high|random|inverted}

用来控制当 domain 断电时,输出电平状态,默认输出低;

可以理解为只有当load 方需要 drive 方输出为高时才需要加一个isolation 吗?

或者说只要配置了-power_down_states 选项,在某些情况下就可以不需要 isolation?

```
# create nominal conditions
create_nominal_condition -name off -voltage 0
create_nominal_condition -name on  -voltage 1.62  ##nominal 电压为什么是 1.62V 而不是 1.5V?因为使用的wc 条件下的 lib,看下面的 update_nominal_condition 内容.

# create power modes
create_power_mode -name PM1 -domain_conditions {PD1@on PD2@on PD3@on} -default
create_power_mode -name PM2 -domain_conditions {PD1@on PD2@on PD3@off}
create_power_mode -name PM3 -domain_conditions {PD1@on PD2@off PD3@off}

# associate library sets with nominal conditions
update_nominal_condition -name on -library_set prjxxxx5072_lib_wc ##如果 lib set 含 wc,tt.bc,则 create_nominal_condition -name on 选项电压可以有 3 个

# create rules for isolation logic insertion
set pin_list1 { \
I_prjxxxx5072_poff_pad/adc_csn_mux \
I_prjxxxx5072_poff_pad/adc_sh_mux \
I_prjxxxx5072_poff_pad/phy_suspend \
I_prjxxxx5072_poff_pad/rst_dac \
I_prjxxxx5072_poff_pad/dac_scl \
```

I_prjxxxx5072_poff_pad/dac_sda_out \

}

create_isolation_rule -name pd3topd2_iso_high -from PD3 -to PD2 -pins $pin_list1 \##用于 PD3 到 PD2 电源域之间,只用在$ pin_list1 上面,如果没指明那些 pin,则默认所有 pin

-isolation_condition {I_prjxxxx5072_rtcpon/pd3topd2_iso_en_last}

-isolation_output high \##当 xxx 时工作,输出为高,如果没指定条件,则当 PD3 断电时,ISO 自动工作

-isolation_target from ##指定规则适用条件;当为 from 选项时,则当指定的 pin 的 drive 的电源域 off 时,启动工作; 当为 to 选项时,则当指定的 pin 的 load 的电源域 off 时,启动工作;默认为 from.

注意:很多器件有-secondary_domain domain 选项,对应的是-primy_domain domain,其中-primy_domain 是给功能性逻辑部分供电, -secondary_domain 是给 iso 逻辑供电

如果没配置,则表明是用一个电源供电.

set pin_list2 { \

I_prjxxxx5072_poff_pad/power_ctl_last \

I_prjxxxx5072_poff_pad/data_bus_mux_oen[31:0] \

I_prjxxxx5072_poff_pad/data_bus_out_mux[31:0] \

I_prjxxxx5072_poff_pad/real_gpio_pu_n[31:0] \

I_prjxxxx5072_poff_pad/real_gpio_pd_n[31:0] \

I_prjxxxx5072_poff_pad/real_gpio_sr[5] \

I_prjxxxx5072_poff_pad/real_gpio_sr[13:8] \

I_prjxxxx5072_poff_pad/real_gpio_sr[26:16] \

}

create_isolation_rule -name pd3topd2_iso_hold -from PD3 -to PD2 -pins $pin_list2 \

```
-isolation_condition {I_prjxxxx5072_rtcpon/pd3topd2_iso_en_last}
-isolation_output hold \
-isolation_target from

create_isolation_rule -name pd3topd2_iso_low -from PD3 -to PD2
-exclude "$pin_list1 $pin_list2" \
-isolation_condition {!I_prjxxxx5072_rtcpon/pd3topd2_iso_en_n_last}
-isolation_output low \
-isolation_target from

create_isolation_rule -name pd3topd1_iso_low -from PD3 -to PD1 \
-isolation_condition {!I_prjxxxx5072_rtcpon/pd3topd1_iso_en_n_last}
-isolation_output low \
-isolation_target from

set pin_pd2topd1_high {\
I_prjxxxx5072_pon_pad/scan_rst  \
I_prjxxxx5072_pon_pad/rst_delay \
I_prjxxxx5072_pon_pad/int_data_bus_in[7] \
}

set pin_pd2topd1_noiso {\
I_prjxxxx5072_pon_pad/int_test_mode \
I_prjxxxx5072_pon_pad/lowvol_flag_inn\
}

create_isolation_rule -name pd2topd1_iso_high -from PD2 -to PD1
-pins $pin_pd2topd1_high \
```

```
-isolation_condition {I_prjxxxx5072_rtcpon/pd2topd1_iso_en_last}
-isolation_output high \
-isolation_target from

create_isolation_rule -name pd2topd1_iso_low -from PD2 -to PD1
-exclude "$pin_pd2topd1_high $pin_pd2topd1_noiso" \
-isolation_condition {!I_prjxxxx5072_rtcpon/pd2topd1_iso_en_n_last}
-isolation_output low \
-isolation_target from
##表示当指定管脚驱动部分的电源域off 时,PD2 输出给 PD1 的信号(除去指定 PIN)(即 PD1 的输入)为 1

########ISO from on to off should keep low############
create_isolation_rule -name pd1topd2_iso_low -from PD1 -to PD2   \
-isolation_condition {!I_prjxxxx5072_rtcpon/pd2topd1_iso_en_n_last}
-isolation_output low \
-isolation_target to
##表示当 PD2 电源域 off 时,PD1 输出给 PD2 的信号(即 PD2 的输入)为 1


create_isolation_rule -name pd1topd3_iso_low -from PD1 -to PD3   \
-isolation_condition {!I_prjxxxx5072_rtcpon/pd3topd1_iso_en_n_last}
-isolation_output low \
-isolation_target to
##表示当 PD3 电源域 off 时,PD1 输出给 PD3 的信号(即 PD3 的输入)为 1
create_isolation_rule -name pd2topd3_iso_low -from PD2 -to PD3 \
-isolation_condition {!I_prjxxxx5072_rtcpon/pd3topd2_iso_en_n_last}
-isolation_output low \
-isolation_target to
```

##表示当 PD3 电源域 off 时,PD2 输出给 PD3 的信号(即 PD3 的输入)为 1

```
#####################################################
# Additional Information for Logic Synthesis
#####################################################
# specify power targets
set_power_target -leakage 30 -dynamic 250

# specify timing constraints
update_power_mode -name PM1
-sdc_files ../syn/scr/prjxxxx5072_src.sdc
#update_power_mode -name PM2
-sdc_files ../syn/scr/prjxxxx5072_src_PM23.sdc
#update_power_mode -name PM3
-sdc_files ../syn/scr/prjxxxx5072_src_PM23.sdc


# update the rules****
```

如果没使用 update_isolation_rules 来指定使用哪个 cell 及插入位置,工具会根据 create_isolation_rule 自动选择 cell 及插入位置

```
update_isolation_rules -names pd3topd2_iso_high -cells ISO_OR2X2M
-location to
```

##-location to 指明这类 ISO 放的位置,to 放在目标电源域内,from 放在源电源域内,parent,如果在对应的 creat__isolation_rules 中没指定 to 选项,则放在源点源域的上一层次,如果有 to 选项,则放在目的电源域的上一层次

```
update_isolation_rules -names pd3topd2_iso_hold -cells ISO_TLATNX2M
-location to
```

update_isolation_rules -names pd3topd2_iso_low -cells ISO_AND2X2M -location to

update_isolation_rules -names pd3topd1_iso_low -cells ISO_AND2X2M -location to

update_isolation_rules -names pd2topd1_iso_high -cells ISO_OR2X2M -location to

update_isolation_rules -names pd2topd1_iso_low -cells ISO_AND2X2M -location to

update_isolation_rules -names pd1topd3_iso_low -cells ISO_AND2X2M -location to -prefix NOCPF

-prefix 当产生 iso 逻辑时,前缀使用 NOCPF , -suffix xx 当产生 iso 逻辑时,后缀使用xx

update_isolation_rules -names pd1topd2_iso_low -cells ISO_AND2X2M -location to -prefix NOCPF

update_isolation_rules -names pd2topd3_iso_low -cells ISO_AND2X2M -location to -prefix NOCPF

##

Implementation part of the CPF

##

declare power and ground nets

create_power_nets -nets V33_PD1 -voltage 3.3 ##电源网线 V33_PD1 为 3.3V

create_power_nets -nets VDD_PD1 -voltage 1.8

create_power_nets -nets VDDIO -voltage 3.3

##可以这样写 create_power_nets -nets { V33_PD1 VDDIO } -voltage 3.3

create_power_nets -nets VDDCORE -voltage 1.8

create_power_nets -nets VDD_PD3 -internal -voltage 1.8 ##internal 表示 VDD_PD3 一定一个是通过内部 powerswitch 输出的主电源网线;

如果有-external_shutoff_condition 选项,则表示这个电源线来自外部 off chip 电源,并且通过 switch 输出,这 net 必须与外部 IOport or Pad 相连接;
如果既没有-external_shutoff_condition 选项也没有-internal 选项,则表明这个net 为没有开关的主电源net.

```
create_ground_nets -nets VSS_PD1 -voltage 0
create_ground_nets -nets VSS_PD2 -voltage 0
create_ground_nets -nets VSSD_PD2 -voltage 0

# create global connections  用于指定全局网络如何与 PIN 连接
create_global_connection -net V33_PD1 -pins VDD33 -domain PD1
##在 PD1 范围内所有名字为 VDD33 的 PIN 都连接在 V33_PD1 网络上
create_global_connection -net VDD_PD1 -pins VDD   -domain PD1
create_global_connection -net VDD_PD1 -pins VDD18 -domain PD1
create_global_connection -net VSS_PD1 -pins VSS   -domain PD1
create_global_connection -net VSS_PD1 -pins GND   -domain PD1

create_global_connection -net VDDIO    -pins VDD33  -domain PD2
##在 PD2 范围内所有名字为 VDD33 的 PIN 都连接在VDDIO 网络上
create_global_connection -net VDDIO    -pins VDD33C -domain PD2
create_global_connection -net VDDCORE  -pins VDD    -domain PD2
create_global_connection -net VDDCORE  -pins VDD18  -domain PD2
create_global_connection -net VSS_PD2  -pins VSS    -domain PD2
create_global_connection -net VSS_PD2  -pins GND    -domain PD2
create_global_connection -net VSSD_PD2 -pins VSSD   -domain PD2

create_global_connection -net VDD_PD3 -pins VDD  -domain PD3
##在 PD3 范围内所有名字为 VDD 的 PIN 都连接在 VDD_PD3 网络上
```

```
create_global_connection -net VDD_PD3 -pins vdd  -domain PD3
create_global_connection -net VDD_PD3 -pins TVDD -domain PD3
create_global_connection -net VSS_PD2 -pins gnd  -domain PD3
create_global_connection -net VSS_PD2 -pins VSS  -domain PD3
## 如果 create_global_connection -net -port xx，则 xx 为设计顶层的
port
# create rules for power switch insertion
create_power_switch_rule -name PD3_SW -domain PD3
-external_power_net VDDCORE
##表明这个开关在 PD3 域，的输入为PD3 外的电源net
# update the rules**** ##所有的 update xxx 命令表示工具执行该动作，
这时才真正插入相关 cell
update_power_switch_rule -name PD3_SW -cells POWERSWITCH \
-enable_condition_1 {I_prjxxxx5072_rtcpon/power_sw_on_last} \##使
能 power_on
-acknowledge_receiver_1 {I_prjxxxx5072_rtcpon/power_sw_on_ack} \##
一个输入的反馈信号
-prefix CDN_ ##命名规则
##简单理解就是 enable_condition_1 只是让这个 powerswitch 工作，
 acknowledge_receiver_1 检测到真才让 powerswitch 把输入电压输出
# add implementation info for power domains
update_power_domain -name PD1 -primary_power_net VDD_PD1
-primary_ground_net VSS_PD1 ##在 PD1 范围中的 function gate 使用的电
源信号对为 VDD_PD1, VSS_PD1,这两个 net 必须在前面用
create_power_nets -nets VDD_PD1 声明过

update_power_domain -name PD2 -primary_power_net VDDCORE
-primary_ground_net VSS_PD2
```

```
update_power_domain -name PD3 -primary_power_net VDD_PD3
-primary_ground_net VSS_PD2
end_design
```

注意点

level shift & isolation 是在综合时插入,
Power swith 是在 PR 时插入;

后端知识

1. 布局布线：

   即 Place & Route。 把前端提供的 netlist & SDC 通过 后端工具做完各步骤处理后，产生通过了各种物理验证的版图文件 GDS，发给 Foundry 厂家进行 tape out 处理。

   布局即 Place， 把拿到的网表进行 floorplan 处理，把各个模拟模块，memory 模块，IO pad 进行合理摆放，然后通过工具的 place 命令把 Standard cell 放进 Top cell，

   进行 timing 优化，达到 timing 要求。

   布线即 Route。把摆放好的各模块和 Standard cell 按照逻辑连接关系，进行连线处理。

   Route 完以后，亦需进行 Timing 优化，及 DFM 各种处理。

2. CTS

   即时钟树综合。 时序逻辑电路，有各种时钟，在前端部分，时钟作为理想 net 存在，而后端需要对这些时钟 net 进行物理综合，即在时钟 net 上根据 fanout 数量插入各级的buffer，以期达到从时钟源头到各触发器的 CK 端的路径是一样长的。 而由于实际上无法做到一样长，那就把从时钟源头开始算，到各触发器的 CK 端的 timing delay 最大差值，定义为 skew，各种工艺对 Skew 的要求不一样，一般来讲.25/.18um 要求做到 0.3ns 以内，.13um 为 0.2ns，.65um 为 0.1ns。

ECO 技术

所谓 ECO，即前端网表确定以后，不再重新综合提交网表以后，如何在后端实现局部电路或逻辑的修改。 ECO 分为 Tape out 前 ，Tape out 后两种。Tape out 后的 ECO，由于底层模块的 Standard cell 已经固定，不可能再任意添加，所以，只能通过改金属连线来实现逻辑的变化,叫 Metal Fix ECO。为了 Tape out 以后实现的逻辑变化尽可能多，前端在做网表综合的时候可以通过添加冗余逻辑 cell 即 spare cell 来为将来 ECO 提供可供修改的 cell。

在.13um 及以下的工艺，foundry 厂家提供一种用于做 ECO 的通用 cell，可以通过添加 filler cell 的方式加入，此种 cell 具备了metal 以下的层数，但无各层metal 的数据，将来在它上面通过添加metal就可以修改其逻辑。

Tape out之前的 ECO，就是前端的 100%网表提供后，又由于仿真或者固件发现小问题，通过局部电路逻辑的修改即可实现，从后端拿到经过 PR 处理以后的网表，把修改后的网表逻辑再反馈回给后端，后端即可以通过 ECO 步骤把这种逻辑改动反应到后端数据里面去。Tape out之前的 ECO 没有 cell 限制，可以任意添加 cell。如有涉及时钟/reset tree，需找到在同级 buffer tree 上的时钟或者 reset 信号。

3. Power ring

---与 pcb 画电源一个意思

项目经过后端的 floorplan 处理以后，各个模拟模块,memory cell 已经摆放好了位置，此时需要进行Power ring 处理，即为整个芯片做好 power ring 的规划和摆放， 目的是让芯片各个模块的 power pin 得到比较好的连接，芯片各个角落的 standard cell 能得到均匀充足的 power 供应。这种供电的网络结构，叫 Power ring & strap 。

Power ring 即围绕 chip 的一个电源地的环，其宽度由芯片的功耗决定，一般要至少保证 1um/1ma。

Power strap 即条状桥梁，在 power ring 上通过一定的间隔上铺设条状 strap，用来改善电源地网络的供应。

Power ring 铺设的是否好，由 IR drop 检查结果决定。

4. IR drop

IR drop 即芯片的电压经过里面电路的消耗以后，在最严重的地方产生的压降。分电压降和地弹。两者数值相加不能超过供电电压的 5%. 一般我们只做 core 的 IR drop 分析，不做 IO 电压的分析。因为 IO 的电压由 IO pad 电路决定，我们无法干预，一般也不会出问题。

芯片内部各逻辑可以简化为阻容网络，依附在已经布置好的电源地 power ring /Strap 的网格点上，假如 power ring 布的不好，局部地区的 Standard 的供电点未接好，就会在这种地方发现 IR drop 的突变点，IR drop 值会异常增大。就需要结合 IR drop 的结果彩图进行分析并解决。

5. 电源地的排布与个数

由封装决定电源地的排布和个数。在封装管脚数量允许的情况下，尽量放多一些 IO 以及 core 的电源地数量。一般小项目有一对 IO 和一对 core 的电源地就已足够。如果允许各放两对，即最好在芯片对端放置。IO 的电源地是给 IO 供电的，其数量由 IO 的数量决定，一般是 20 个 IO 至少要安排一对 IO 的电源地。IO 的电源地数量越多，其 ESD 保护能力越强，因为 IO 电源地一般包含 ESD 保护电路。

Core 的电源地数量由 Standard 的规模决定，一般 300k 以下规模的电路有一对即可保证供电。500k 的需要两对。 可以通过 IR drop 结果看出供电点是否足够

6. Antenna

天线效应。即连接 MOS 管 Gate 端的金属线，如果很长很长的话，会在金属上收集静电荷，累积到一定的电势后会对管子造成击穿的破坏作用。每一层金属都有长度的限制，

即超过一定的长度就会违背了Antenna 的规定，会产生击穿的风险。Top 层 metal 的可以容忍的长度最长，而且metal 可以逐层传递。即 Metal1 的电荷可以传递给 Metal2,metal2 的可以传递给 metal3，最后累积到最顶层。即底层metal上的antenna 违规可以通过在其中间换线来解决，换到更高层的金属去。

当最高层也有Antenna violation 时，有两种方法可以解决，一个通过添加 Antenna cell，直接把电荷通过新加的 Antenna Diode 泄放掉。另一个方法是通过在此逻辑net 中间上插入 buffer 来降 低连线的长度，或者通过 size 驱动 cell，来增加 Gate 端的面积。Gate 端的面积越大，可容纳的电荷就越多，Antenna 能容忍的长度就越大。

7．物理验证

物理验证主要检查如下 5 方面：

1．DRC， 设计规则检查，主要检查后端设计是否违反工艺厂所要求的设计规则；

2．LVS， layout 与 前端网表的一致性检查；

3．ERC， 电气规则检查。

4．DFM， 针对工艺厂制造过程中的一些保证良率的 rule check。

5．PERC 针对全芯片的 ESD 以及 BLOCK power 连接等详细系统的 ERC check。

9.工艺：几 P 几 M 怎么选择；foundry 怎么一层一层的往上做；固件代码放在哪一层

---几 p 简单理解成普通的 pcb 的单面板/双面板 ，用于放（生成）元器件

---几 M 简单理解成普通的 pcb 的-4/6（几）层板，用于布线

P 指 poly，M 指 Metal;对于几 P 几 M 怎么选择需要考虑因素如下：

1．芯片设计性能方面需求；我们目前所用的都是 1 P 工艺，也就是一层 poly;而对于需要高性能电容电阻的特殊模拟电路可能用到 2p 工艺；

几 Metal选择需要考虑因素：PR 方面阻塞情况；模拟电路大电流走线要求；DUP PAD 由于Metal 层数差别对良率的影响等；

2．其他需要考虑因素：需要与所用第三方 IP 工艺兼容；需要兼顾成本。

对于我们现在所用 N 阱 cmos 工艺而言 主要工艺流程大概如下：

P 衬底 > N 阱扩散 > gate 氧化层 > 有源区注入 > contact 孔 > Metal 1 淀积 > VIA1 > Metal 2 淀积 >VIA2 >。。。 TOP Metal > PAD 开窗

3 固件代码是 01 的组合，通过控制rom 输出 01 来达到设计的目的。

ROM 在内部默认是输出 1 的，当需要有 0 输出的时候，就把一个打开并对地的 nmos 管挂在输出 1 line上，把这个默认的 1 强制拉到 0。 目前我们是通过修改Via1 和 metal来达到这个目的。通过修改其它 layer 也是可以实现的，但是要基于面积和 mask 成本等因素综合考虑。

10. Floorplan

-- 跟普通的 pcb 布局一个概念，摆放好各个元件

即模块摆放，IO 位置调整，power Ring 网络布置。

拿到网表以后，后端需根据前端的数据流框图，及 IO 摆放位置需求，把 IO 的位置安排好，并把各模拟模块及memory 模块根据数据流流向和 IO 的远近关系摆放好，并根据后端经验，把各模块靠角归边，尽量给Standard cell腾出比较方整的位置。再布上合适的电源网络结构。

Floorplan 做完后，一般应该快速走一遍不考虑 timing 的 route，用于验证 Route 的可能性，发现模块摆放不好导致拥挤的地方。

这里提到了拥挤的概念，即阻塞。 及由于局部的地方 standard 太过拥挤或者 floorplan 不合理，不够地方布线，该局部地区的 Route 变得拥挤及阻塞，从而难于布线。需要通过 floorplan 调整，比如把挡着布线的大模块挪走，比如降低局部的Standard cell利用率，让其不再拥挤等手段来修正

11. 噪音即 crosstalk。

即两根平行很长距离的金属线，会产生互相干扰的效应。即噪音现象。表现在 timing 上的话，即会产生一个叠加的 delay，在 min 的时候此叠加 delay 让其变的更小，在 max 的时候让其变的更大。即往更恶劣方向干扰。

修改的话也是有两种方法，一个是让并行过长的两根线的一根进行拐弯，缠绕的处理，让其不再跟人家平行。另一个是用PrimeTime 进行带SI参数的分析，得出互相攻击比较厉害的net 的列表，然后 PT 会产生一个修改的脚本，即在一些关键 cell后面通过插入 buffer,或者修改驱动来增强抗干扰能力，让干扰产生的 delay 变小。

12. PVT 解释

P: Process Corner 如果采用 5-corner model 会有 TT,FF,SS,FS,SF 5 个 corners

v: voltage,电压如：1.0v+10% ,1.0v ,1.0v-10%

T: temperature,温度如：-40C, 0C 25C, 125C。

WCS (Worst Case Slow) : slow process, high temperature, lowest voltage

TYP (typical) : typical process, nominal temperature,nominal voltage

BCF (Best Case Fast) : fast process, lowest temperature, high voltage

WCL (Worst Case @ Cold) : slow process, lowest temperature, lowest voltage

13. Scenarios

Scenarios = Interconnect + operation mode + PVT

1. 内连线情况(interconnect corner)

制造对互连线造成的影响，如：R_typical C_typical, R_max C_max, R_max C_min, R_min C_min

2. 工作模式 (Operation Mode)

如：function mode, scan mode, sleep mode, standby mode, active mode

3. PVT

对多种 scenarios 的综合分析，称之为 MMMC (Multi-Mode Multi-Corner) Analysis。

14. OCV (On-chip Variations)

OCV 可以描述 PVT 在单个芯片所造成的影响。更多的时候，用来考虑长距离走线对时钟路径的影响。在时序分析时引入 derate 参数模拟 OCV 效应，其通过改变时延迟的早晚来影响设计。

15. 三种 STA(Static Timing Analysis)分析方法

1，单一模式，用同一条件分析 setup/hold ；

2，WC_BC 模式，用 worst case 计算 setup，用 best case 计算 hold;

3，OCV 模式，计算 setup 用计算 worst case 数据路径，用 best case 计算时钟路径；计算 hold 用 best case 计算数据路径，用 worst case 计算时钟路径;

16. set_timing_derate

set_timing_derate

[-rise]

[-fall]

[-min]

[-max]

[-early]

[-late]

[-clock]

[-data]

[-net_delay]

[-cell_delay]

[-cell_check]

value

object_list

如果没写-rise 也没写-fall,这表示两者都用,-min(工艺最快),-max(工艺最慢)一样;-early 表示信号比设定的更早,-late 表示信号比设定的更晚;-clock 表示系数加在时钟路径上,-data 表示系数加在数据路径上,如果都没写,这两者都用;

SOC 产品分析(ppa)

1. 功能

1.5.1 通讯接口 --- 通讯速率范围,最快,最慢是否符合需求

1.5.2 定时模块 --- 定时范围,最小,最大定时及定时步长是否符合要求

1.5.3 算法模块 - 计算速度

1.5.4 Ad/da 类 -- 转换速率(采样时钟范围)

1.5.5 IO/PAD - 上下拉,open drain,hi_z,翻转速率等功能是否满足要求

2. 性能

1. **cpu 最快能跑多高速率**
2. cpu 总线类型
3. soc 架构 -- 图文并茂方式提供
4. memory 空间划分,寄存器地址划分 -- 图文并茂方式提供
5. soc 时钟结构 -- 图文并茂方式提供
6. 中断源分配是否合理
7. 复位源分配是否合理
8. dft - 所有模拟部分是否可测,数字 atpg 覆盖率

3. 功耗

1. cpu 跑 nop 指令功耗@最大频率 ---其他模块不工作
2. 系统低功耗电流多大?---所有模块不工作
3. 系统 idle 状态电流多大?---cpu 停止,其它模块根据定义是否工作
4. 待机电流多大

4. 工作模式

正常工作模式
低速工作模式
Idle 工作模式
Pd 工作模式
其它工作模式

5. 芯片面积

1. 内部存储单元多大?—flash/rom,xram,iram 等
2. 模拟部分面积多大?
3. 数字部分多大?

4. 第三方 ip 多大?

芯片设计流程

5. 根据产品分析,定出使用的 cpu,flash,memory 及其它模拟模块
6. 定义好系统架构,预计时钟复位域并划分好功能模块
7. 设计单个模块rtl
8. 对单个 rtl 做 nlint 语法检查
9. 模块验证
10. 设计好所有模块,做系统集成,考虑 dft测试
11. 使用 verdi 做语法检查(注意不能带-sv 选项,sv 选项太宽松了),因为 nlint 检查太广,内容太多,不太好看,容易忽略掉很多东西,通过verdi 语法检查可以很容易检查到一些基本的语法错误
12. 对整体做 nlint 语法检查
13. 用 spyglass 做 cdc 检查 -- 小设计很少做 cdc 检查,由设计保证
14. 系统验证/同时可以做 dc 综合
15. 进行 formality 形式验证
16. 进行 dc 综合,输出给 pr 的 sdc 文件及 atpg 用的 spf 文件
17. 进行 post sim—各个模式都要有测例,无需时序,只是跑个流程,调平台
18. 使用 tmax 生成 atpg 测试平台并仿真,无需时序,只是跑个流程,调平台
19. Pr,提取 sdf 文件及 spef 文件(用于产生 sdf 文件)
20. 使用 sta 工具进行静态时序分析,并产生 sdf 文件
21. 进行带 sdf 文件的 post sim,重复步骤 10,11,此时是 pr 后的网表.
22. 生成 cp 需要的各种 vcd 波形
23. 根据生产厂家的 signoff 文件检查设计是否完毕
24. 输出 gds文件给厂家生产芯片

电路设计基础

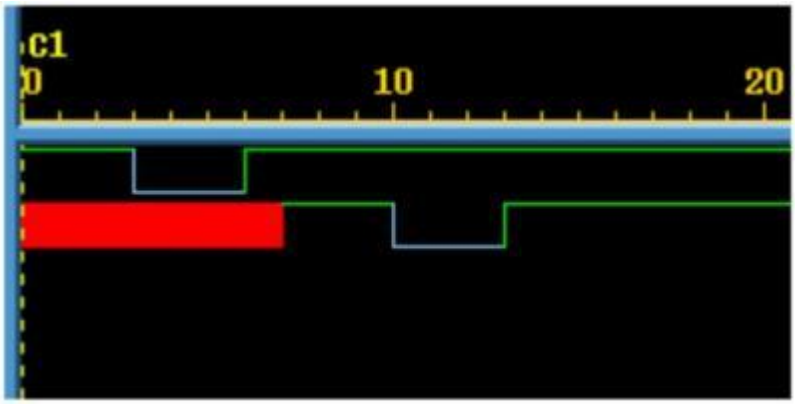
1． 延时信息定义

IOPATH delay：cell delay,根据输出的 wire loading 和输入信号的 transition 计算—一个 cell 从输入到输出的延时,input output path delay

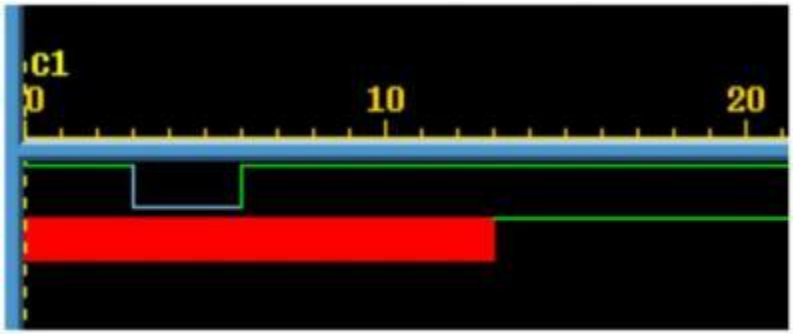
INTERCONNECT delay：path delay,是一条路的 driving cell 的 output pin 和 driven cell 的 input pin RC delay。--两个 cell 之间的延时,互连延时

在 vcs 仿真中，延时分为 2 大类：transport（传输延时） and inertial（惯性延时）

delays：transport：当脉冲小于模块延时，经过模块延时后脉冲会照常输出，



惯性延时：小于模块延时的脉冲会被过滤掉



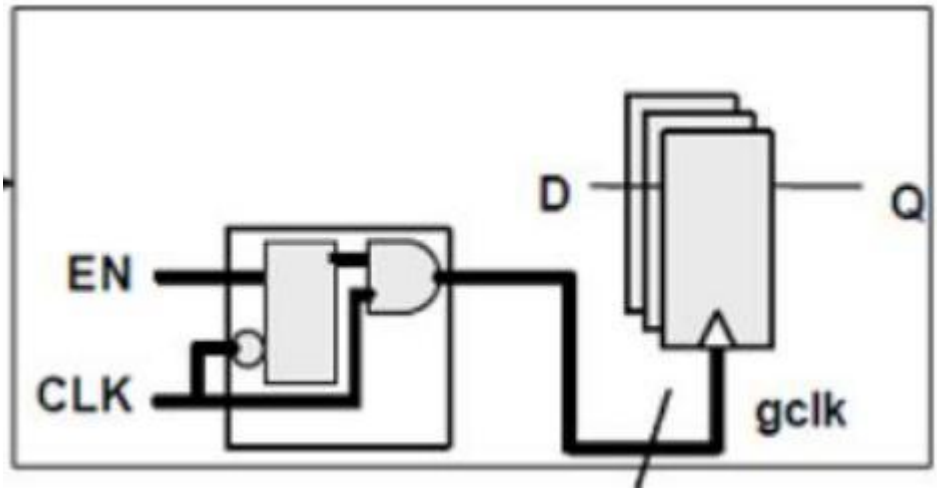
2． 开关管特性

开通比关闭快，所以导致会有瞬间短路功耗;器件的上升沿延时比下降沿大

3． 门控时钟

EN 信号要在 CLK 时钟域

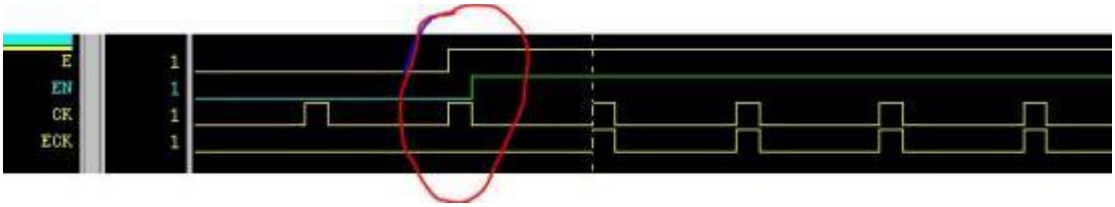
上升沿触发



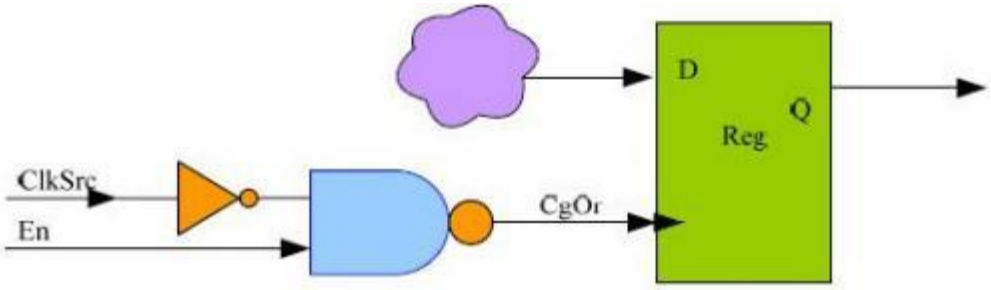
always@(*)

```
    if(~CK)
        EN = E;
    else
        EN = EN;
assign ECK = CK & EN;
```

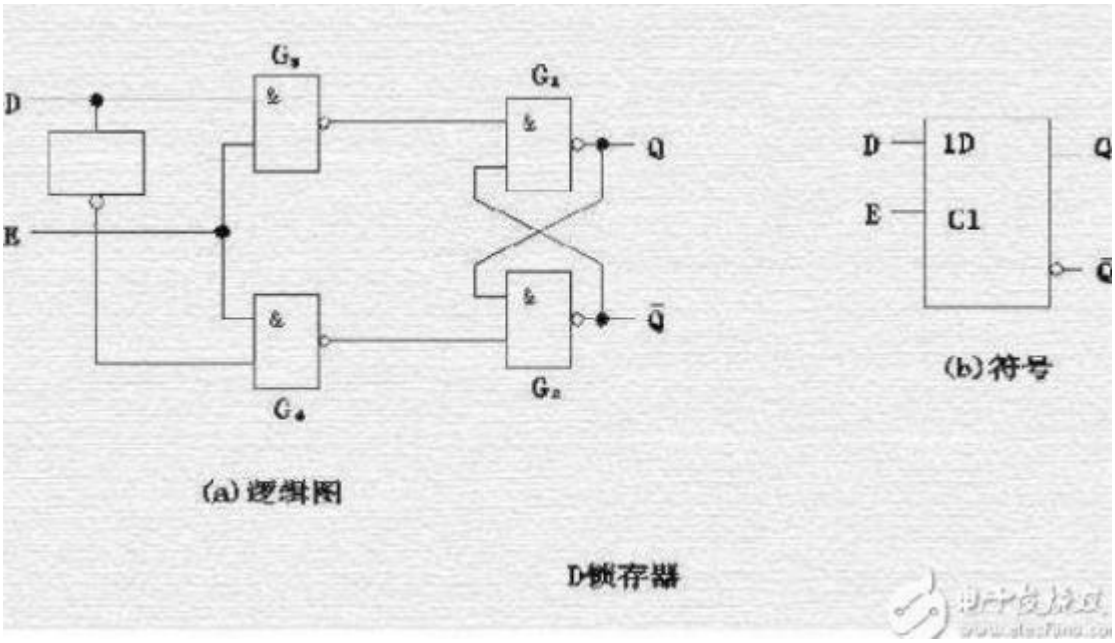
通过 AND 去 gate 时钟,正常情况下是在时钟的下降沿去 gate,EN 信号应该是从时钟下降沿输出,但 ICG 电路对此处做了处理,用了个中间信号,通过 clk 取反来锁存了下,所以 EN 信号还是用上升沿处理.其时序如下,注意第一个时钟丢失,EN 信号是内部信号



下降沿触发



4． 锁存器电路

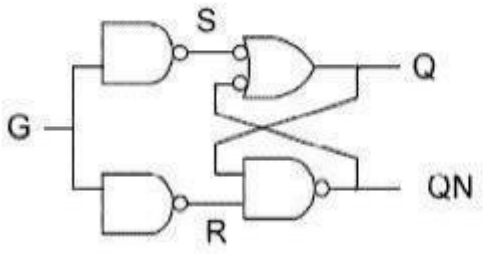


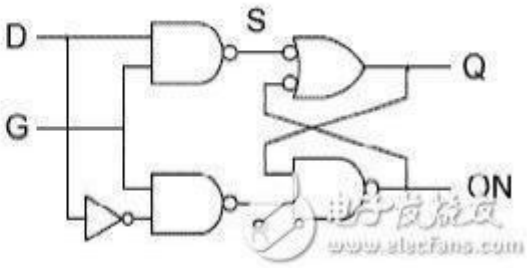
d 锁存器逻辑图详情解析

基本单元是最基础的存储器器件，并在特定的情况下发挥着重要的作用。但是，如果在一个基本单元上再加上两个逻辑门，那么就可以构造出更有用的存储器器件，该器件就称为D 锁存器。

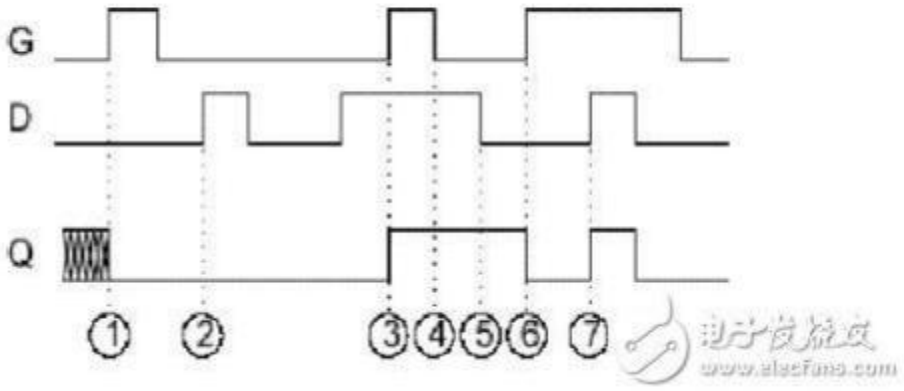
D 锁存器使用基本单元作为存储部件，但它只允许在时序控制信号有效时才能改变（或编程）存储器存储的逻辑值。因此，D 锁存器有两个输入时序控制信号和数据输入。

时序控制信号，通常也称为“门信号”，或“时钟”，或是“锁存使能”，主要用于同步新数据何时可以写入存储器，何时不能写入。如下图左图所示，可以看出当门信号无效时，S 和 R 信号为 1 且输出 Q 由存储在基本单元反馈回路的值决定（所以Q 就是存储的逻辑值）。再看右图，可以看出当门信号有效时，D（数据）输入将 S 和 R 驱动到各自相反的电平，从而在基本单元中强制了一个置位或复位操作。通过组合时序信号和数据输入信号来强制基本单元的置位或复位操作，我们就构造出了一个非常有用的存储器器件。D 锁存器已经广泛使用在所有类型的现代数字电路中了。





如下图所示为D 锁存器时序方框图。注意，当门信号有效时，输出Q 只是简单的“跟随”输入。但是当门信号无效时，输出“记住”了门信号下降沿时的 D 值。



d 锁存器时序方框图

1、 在 G 有效前，Q 未定义；Q 有效时，Q 得到 D 的值

2、 G 无效而 D 有效，Q 不改变

3、 D 和 G 都有效；Q 得到 D 的值

4、 G 下降沿；Q 存储了D 值

5、 D 下降沿，而 G 无效，Q 值不改变

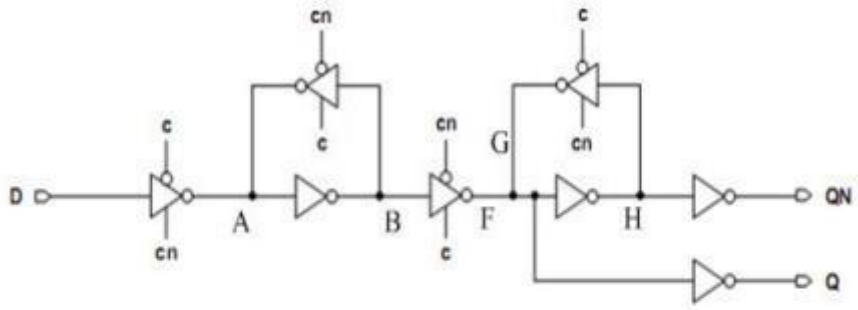
6、 G 有效，Q 得到 D 值

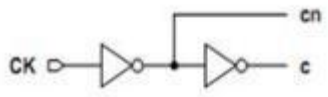
7、 G 有效时，Q 跟随 D 值

5. 时钟电路设计分析

6. 复位电路设计分析

7． 寄存器电路





可以简单的分析一下：

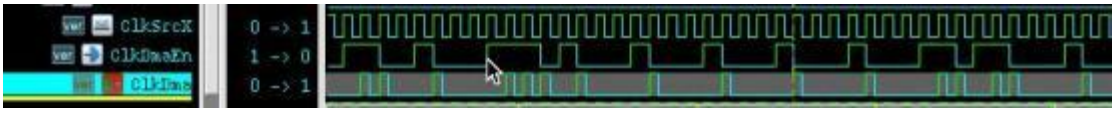
1）D 值变化的情况下，如果 CK 保持低电平不变，采样电路会有一个动态功耗，如果 CK 保持高电平不变，则 D 端的变化不会产生动态功耗。

2）Ck 变化，如果 D 不变化，在一个时钟周期内则会有 4 个传输门的动态功耗。

可见，如果在 D 不变化的时候，如果 CK 也能保持不变，会减少寄存器的动态功耗。

实际上， 目前芯片中时钟信号的功耗占系统功耗的很大一部分（40%左右）占动态功耗的 50%以上。

8． ASIC 版本 ICG 电路 CLK,EN,GCLK 之间的时序关系



GCLK 比 EN 晚一拍出现

主要是因为 icg 电路会在时钟低电平把 en 信号 lath 再 gate 时钟

```
module ICG_V  (E,CK,TE,ECK);
input E,CK,TE;
output ECK;
always@(*)
    if(~CK)
        EN = E;
    else
        EN = EN;
assign ECK = CK & EN;
endmodule
```

FPGA BUFGCE 时序关系

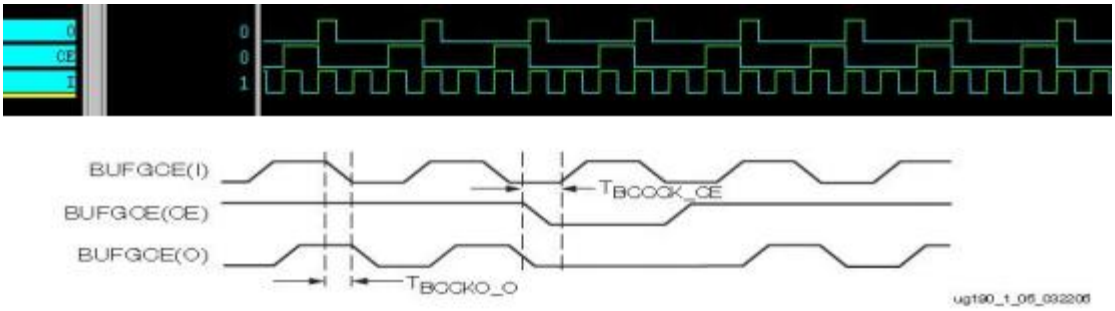
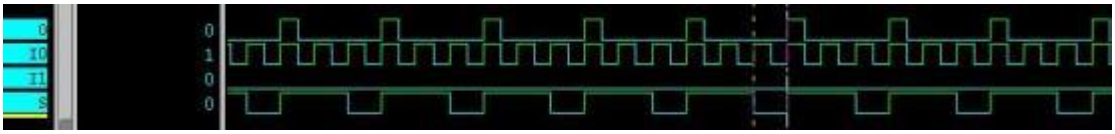
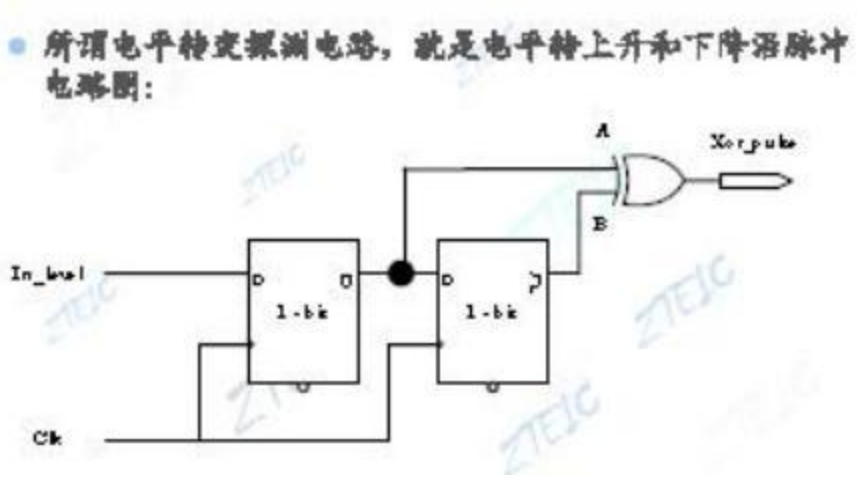


图 1-6: BUFGCE 时序图

BUFGMUX 时序图



9． 电平转变探测电路



10． 电平同步

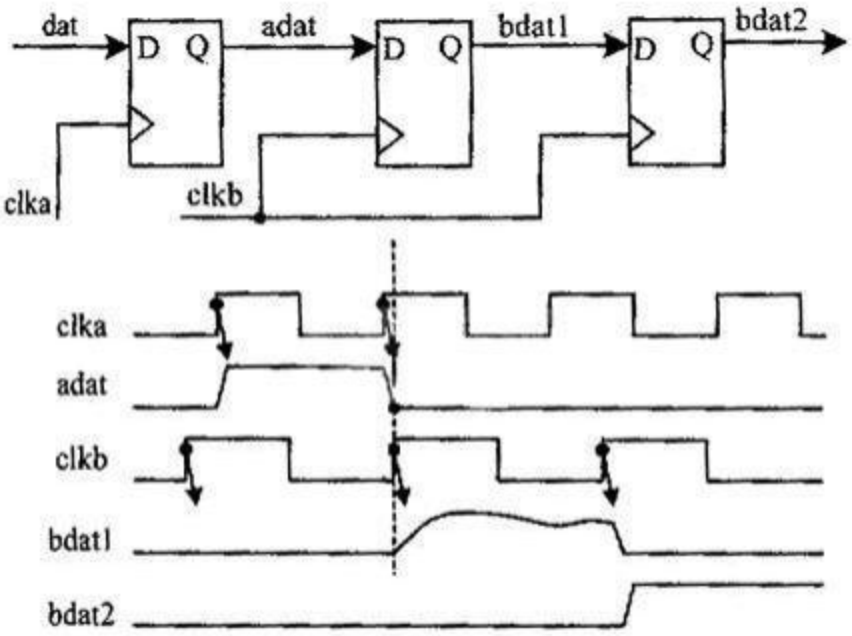


图 3 两级时钟同步消除亚稳态

注意：adat 必须是在 clka 域敲过的信号,必须寄存器输出

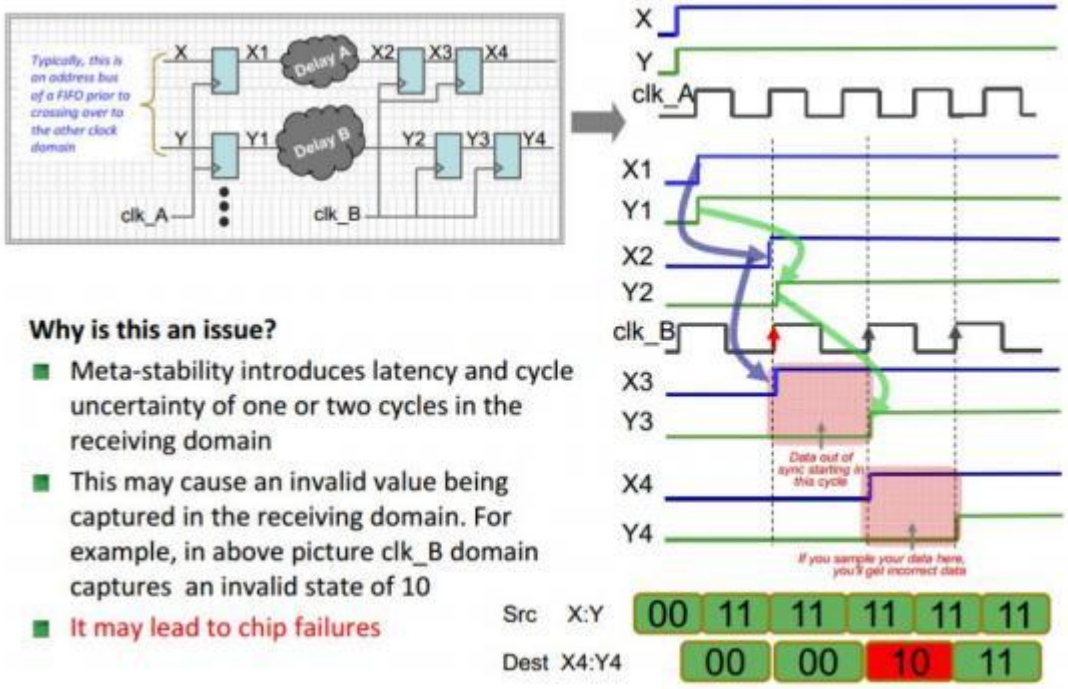
对于打两拍的同步到另外一个时钟域电路，要注意输入应该为寄存器输出的无毛刺信号，否则如果输入为组合逻辑，很可能会采样到毛刺，需要先滤毛刺再同步！！

对于用寄存器可以过滤毛刺是指毛刺脉冲太窄，而寄存器只有在时钟跳变才采样，考虑如果毛刺就是在时钟跳变沿，这时出现 sethold time 不满足，结果如果？

用同步电路只能降低亚稳态的概率，但不能杜绝！一般可以忽略不计，

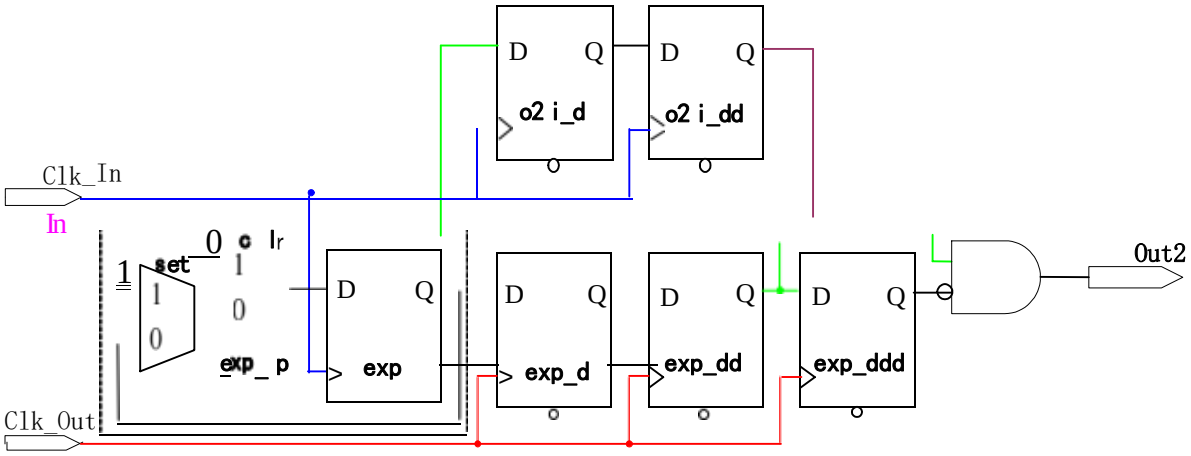
减少亚稳态出现的关键是器件使用比较好的工艺和时钟周期的余量大一些。好器件工艺的 resolution time 会比较短，例如传统的 TTL 电路中，高速的 74F 系列就比 74LS 好；时钟频率低一些，出现亚稳态时提供给输出稳定的时间也会多一些，这样可以减小亚稳态传播的机会。同步系统也存在亚稳态，但是相比异步系统来说，比较容易控制，只要 setup/hold time 满足就可以，而对异步系统，这个简单的要求也不容易满足，这也是同步系统的优点之一。那么究竟如何避免（或者减小)亚稳态的危险呢？注意到亚稳态的触发器继续停留在亚稳态的几率按照指数减少，那么办法就是 等——等足够长的时间，直到这个几率变得小的实际 上不会发生。到底需要有多长呢？有的厂商有一个数据，有的没有，按照普通的做法，至少等一个时钟周期——这也就是所谓的异步数据要用两个触发器打一下。

多比特不能使用打两拍的方式处理,如下图:

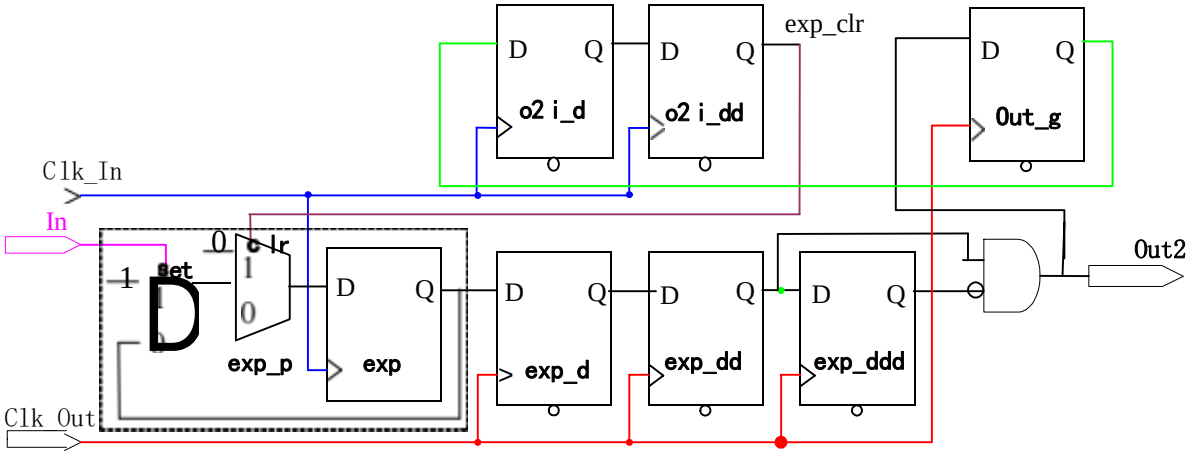


11． 脉冲同步

只有脉冲同步才分快时钟同步到慢时钟或慢时钟同步到快时钟，这个不属于时序概念，而是属于 sequence 概念—即为下一个时钟能否采样到该信号



电平清 0



脉冲清 0

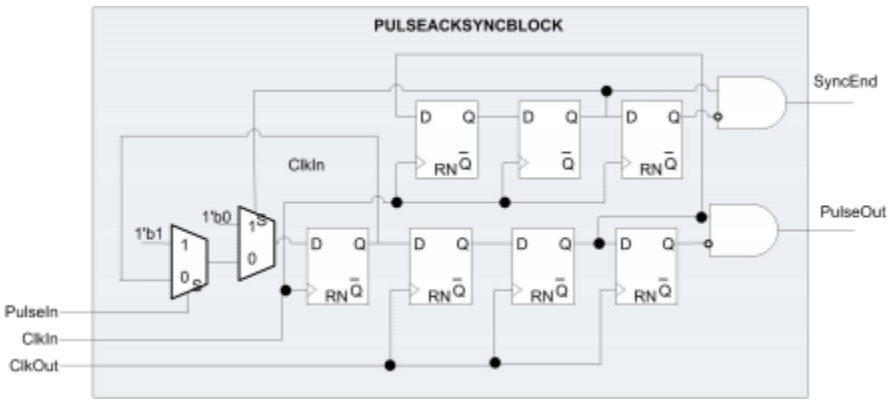
1.脉冲清零电路的 exp_clr 信号采用上升沿脉冲，有死锁的潜在可能：当 Clk_In>Clk_Out，exp_clr N（N<Clk_Out/Clk_In）个快时钟之后的又来一个 In 脉冲,则 exp_clr 后的 In 的下降沿只有 N 个快时钟周期脉宽（小于 1 个慢时钟周期）可能无法被Clk_Out 采样到，无法产生第 2 个 exp_clr 脉冲,从而死锁(后续 In 脉冲无效).

所以使用的前提是：两个脉冲的间隔时间>clk_out 周期

需要注意在 soc 系统中在总线时钟域连续操作 2 个寄存器，如果这两个寄存器都同步到另外一个时钟域去，如果不符合上述间隔条件，这同步后这两个寄存器的操作将会在另外一个时钟域同时操作的现象。

2.电平清零电路的 exp_clr 信号采用电平,可避免死锁，但会漏掉频繁的 In 脉冲.

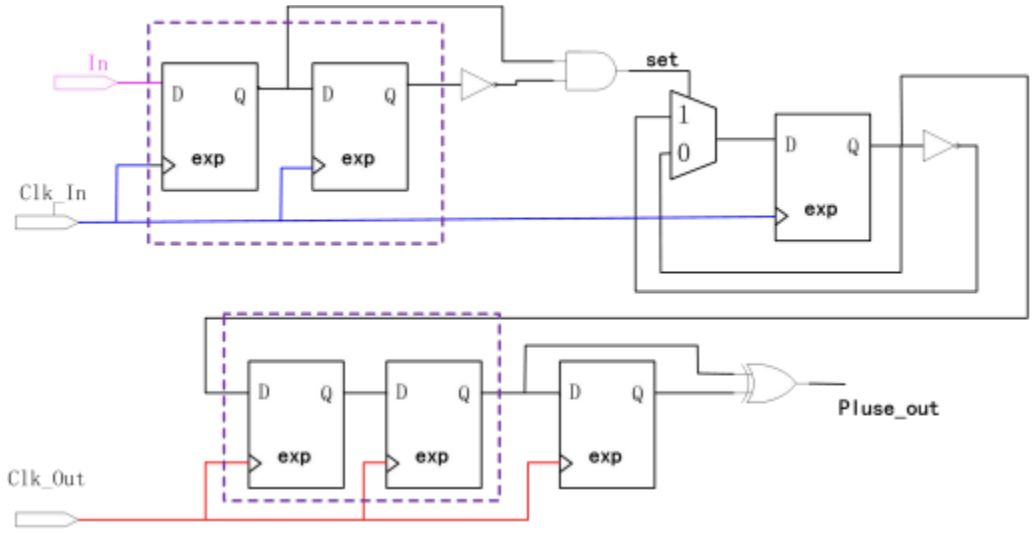
3.对于数据同步，当在a 时钟域有两个控制信号，如 wr_n,cs_n,一起控制数据输出，这时不能分别同步，而需要做一个组合逻辑后再同步 ,如 assign comb=wr_n|wr_n;然后再把 comb 同步到时钟 b

5. 在 SOC 系统中，由于寄存器配置是一个时钟域，各模块的功能可能是另外一个时钟域，只需要把最后一个 start 启动信号同步就可以了，其它的信号在模块时钟域内的 D 端用 start_sync & singal,再在 Q 端输出就可以，原理如上行所述，注意：启动信号之前配置的寄存器不能再发生变化，只有 done 信号同步回去之后才可以重新配置。
6. .对于 counter 计数，使用格雷码
7. 

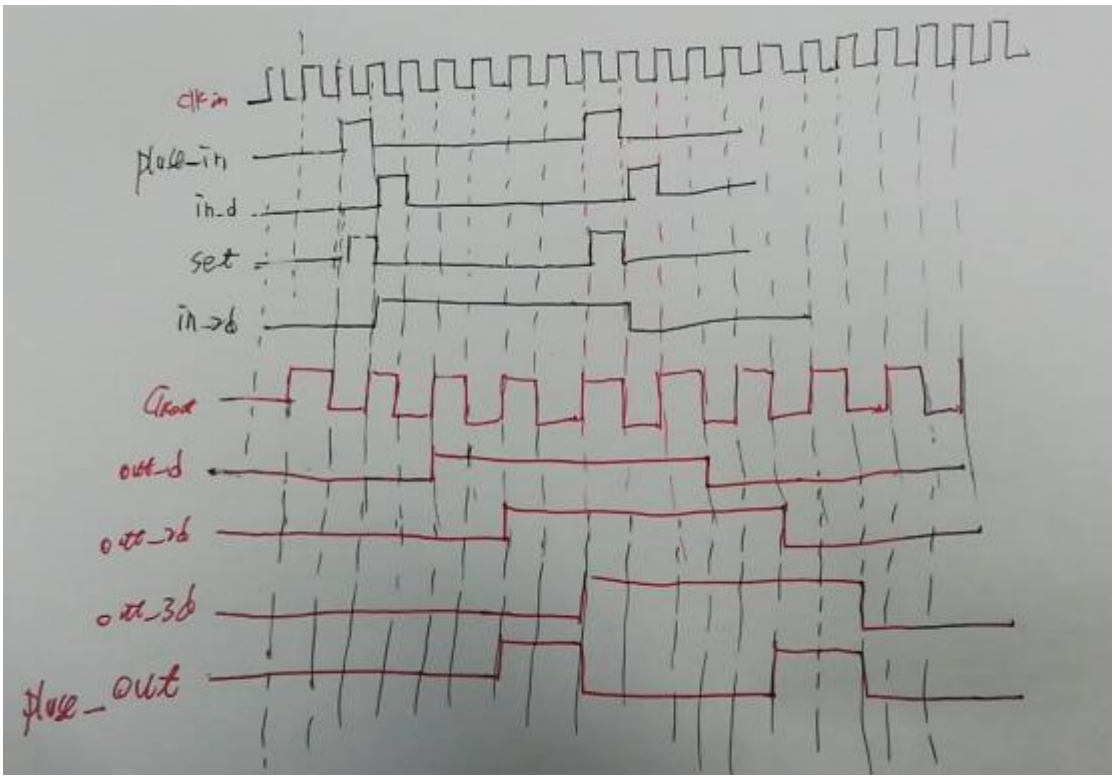
通用脉冲同步电路

下面的这种电路无论是快时钟域到慢时钟域,还是慢时钟到快时钟,都可用

但请注意同步前后脉冲的延时是否符合设计要求.



快时钟到慢时钟域波形图如下



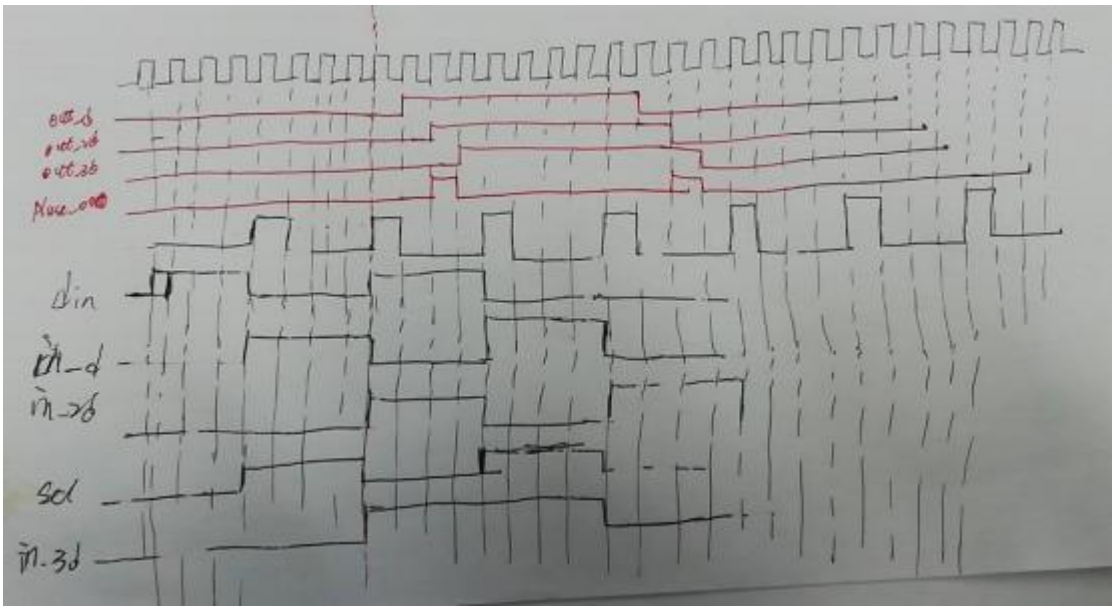
从上面波形可以看出,快时钟的脉冲不能太快,周期必须大于(T 慢/T 快)

比如,快周期为 5M,慢周期为 1M,则脉冲至少要隔 5 个时钟.

如果快时钟的脉冲太快,则无法同步要慢时钟域,无法解决,只能修改设计方案.

慢时钟到快时钟域波形图如下

红色为输出部分时序



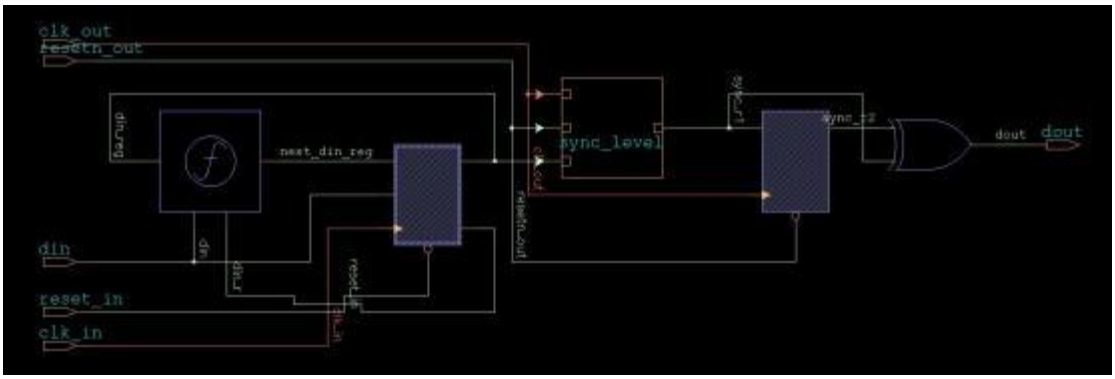
这种时钟关系就不存在上述条件,但注意 2 个脉冲的起始间隔为 2 倍的慢时钟周期+2 倍的快时钟周期,设计时需要考虑这个延时是否符合要求;

如果时间太长,可以考虑用电平同步,再取沿,得到快时钟脉冲,这样只会等待 2 个快时钟周期

下面为rtl代码

用带反馈的电路来指示同步结束

自我清除 pluse 同步电路



使用条件:输入的 2 个 pluse 间隔必须大于输出时钟的 2 倍. sync_level 才能被同步到,

```
assign next_din_reg = (din & ~din_r) ? (!din_reg) : din_reg;
always@(posedge clk_in or negedge reset_in) begin
    if(!reset_in) begin
        din_reg <= 1'b0;
        din_r   <= 1'b0;
    end else begin
        din_reg <= next_din_reg;
        din_r   <= din;
    end
end
sync_level uSync(
    .clk         ( clk_out    ),
    .clk_rst_n   ( resetn_out ),
    .async_sig   ( din_reg    ),
    .sync_sig    ( sync_r1    )
    );

always@(posedge clk_out or negedge resetn_out) begin
    if(!resetn_out) begin
        sync_r2 <= 1'b0;
    end else begin
        sync_r2 <= sync_r1;
    end
end
assign dout = (sync_r2 ^ sync_r1);
```

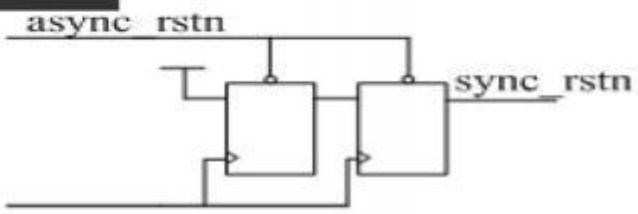
12． 脉冲同步在低功耗设计中注意事项:

由于低功耗设计中会关电源,所以当一个信号从一个电源域同步到另外一个电源域时,可能会导致这个脉冲信号变成电平信号(某个电源关闭且输出维持为 1)而导致同步出现异常,此时可能需要采用电平同步而不是脉冲同步.

13． 异步复位同步释放

（用于异步复位电路中，解决 recover/remove 时序违背

```
always @(posedge clk or negedge async_rstn)
begin
    if(!async_rstn)
    begin
        async_rstn_d <= 1'b0;
        sync_rstn <= 1'b0;
    end
    else
    begin
        async_rstn_d <= 1'b1;
        sync_rstn <= async_rstn_d;
    end
end
```

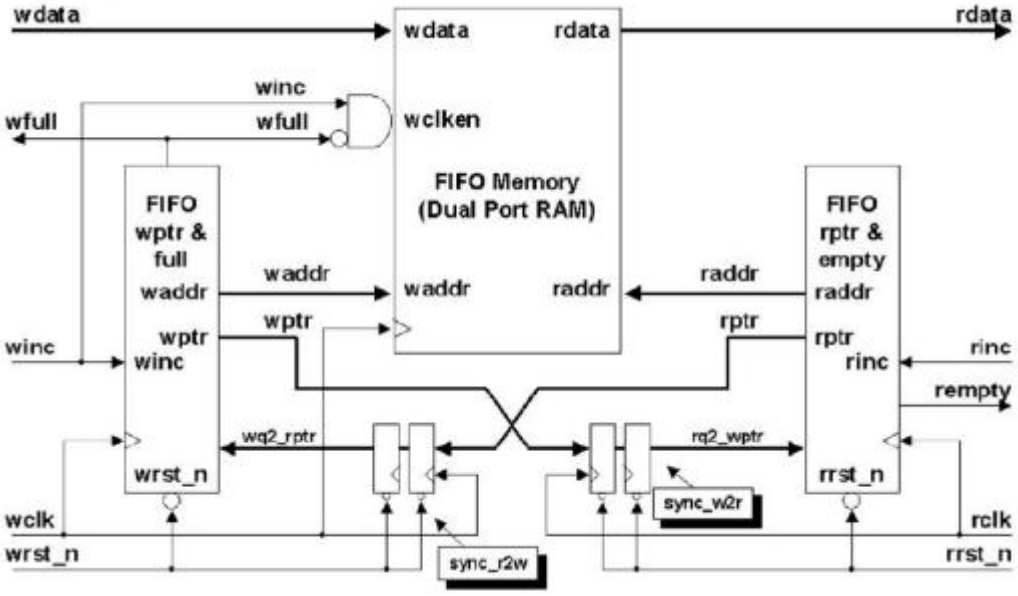


14． 同步 fifo 设计

同步 fifo 设计比较简单，有多维寄存器构成，写入的时候与寄存器相似，与 wr,addr 相关，；读出时则与 rd 无关，与时钟无关，只要给出读地址，数据就输出---读地址与 rd 相关 。

所以与寄存器的操作方法一样。

15． 异步 FIFO 设计



在 FIFO 结构图中，指针同步电路只用了简单的 2 级电平同步电路。而 FIFO 读写双方的时钟在原则上可以是任意关系，为什么这样简单的同步电路不会造成任何问题？

提示：从同步指针的用途来思考这个问题。

使用 fifo 的前提是深度一定要满足读写时钟快慢带来的数据容量,写行为过快,肯定会丢失数据,读时钟快,软件可以等待,如果wclk:rclk 的频率为 2:1,那么深度至少为 2

1. 用作产生空,非空,满非满等标志位sync_r2w 是用来产生满,半满,等标志 在写时钟域产生这些标志---软件写时要先检查这些标志

   Sync_w2r 用来产生空,半空, 非空等标志,在读时钟域产生这些标志位---软件读时要先检查这些标志
2. 在进行读写操作时,一定会先检查标志位.

A,读快写慢: Sync_w2r 可以使用打拍处理; sync_r2w 正常情况下快时钟域同步到慢时钟域,是不能使用打拍方式处理,但由于 sync_r2w 是用来产生满等标志,读之前需要查看非空标志位(非空标志位是 Sync_w2r 来产生,这个信号在此场景为慢时钟域到快时钟域),当为空了,软件不会读,也就是地址会停止不变,直到信号变成非空,在这

段时间 r2w 信号相当与是一个电平信号(电平长度至少超过了2 个慢时钟,所以可以被同步到慢时钟域).

B,读慢写快:因为写之前需要查看满标志,当满标志置起,写动作停止,既写地址停止,原理同上.

3.在异步fifo中只有空(同时可以表示非空含义)和满(同时可以表示非满含义)标志,不存在半空,半满的标志.

注意同步地址必须使用gray 码,因为它每次只有一根地址线改变,如下为 gray 函数,可以直接使用;

```
  function [ADDR_WIDTH:0] bin2gray  ;
    input [ADDR_WIDTH:0]         b;
    begin
      bin2gray  = b ^ {1'b0, b[ADDR_WIDTH:1]};
    end
  endfunction

  function [ADDR_WIDTH:0] gray2bin  ;
    input [ADDR_WIDTH:0]         g;
    reg   [ADDR_WIDTH:0]         b;
    reg   [31:0]          i;
    begin
      b[ADDR_WIDTH] = g[ADDR_WIDTH];
      for (i=0 ; i<(ADDR_WIDTH) ; i=i+1) begin
        b[ADDR_WIDTH- 1 - i] = g[ADDR_WIDTH-1-i] ^
b[ADDR_WIDTH-i];
      end
      gray2bin  = b;
    end
  endfunction
```

直接使用上述函数:

```
assign ra_b          = gray2bin(ra_gray_p1);
assign wa_gray_next = bin2gray (wa_next);
```

注意 FIFO 输出是组合逻辑输出，只跟地址相关 (因为输出就是选择那组 Q 端输出,地址就是选择信号).

一般在做异步fifo 时,写 fifo 需要在时钟上升沿,写信号为高才把数据写入到 fifo 中,但读时,只与地址相关,读信号只会改变地址,而读数据会根据地址直接输出;

fifo 一般是先写后读,如果先读后写处理起来很麻烦,所以读一定要判断非空等标志

二进制于格雷码转换关系:

```
assign wa_gray_next = wa_next ^ {1'b0, wa_next[ADDR_WIDTH:1]};
// -------------
  always @(posedge wclk or negedge wrst_n)
    begin
      if (wrst_n == 1'b0)
        begin
          R0 <= {DATA_WIDTH{1'b0}};
          R1 <= {DATA_WIDTH{1'b0}};
          R2 <= {DATA_WIDTH{1'b0}};
          R3 <= {DATA_WIDTH{1'b0}};
        end
      else if (wr)
        begin
          case (fifo_wa)
            3'd0: R0  <= wdata;
            3'd1: R1  <= wdata;
            3'd2: R2  <= wdata;
            3'd3: R3  <= wdata;
```

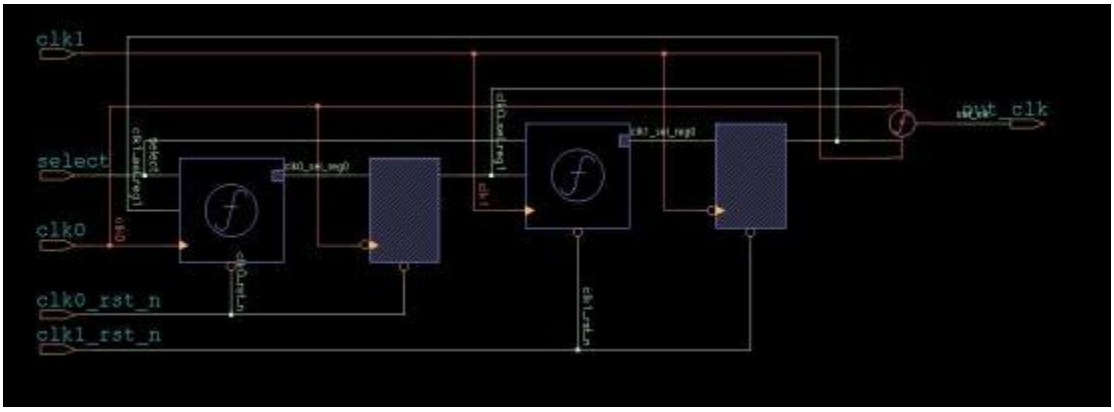
```
        endcase // case(fifo_wa)
      end
  end
always @(*a)
  begin
    case (fifo_ra) // synopsys infer_mux
      3'd0: rdata = R0;
      3'd1: rdata = R1;
      3'd2: rdata = R2;
      3'd3: rdata = R3;
    endcase // case(fifo_ra)
  end
```

16． 静态时钟切换

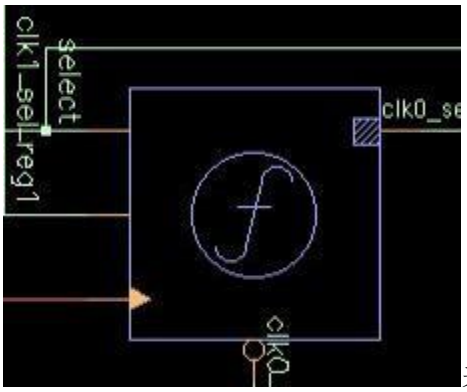
时钟静态切换指两个时钟源由一个选择信号选择去驱动一个时钟网络，但该切换是在应用过程中，一定是切换完成之后,然后再有下一次切换;前提是必须 2 个时钟都存在,

假如某个时钟不存在,需要切换到另外一个时钟,怎么解决?—使用 select 信号去触发停时钟那部分逻辑的复位控制.

其电路结构如下



注意阴影部分是一个纯粹的寄存器,下降沿触发



表示 D 端有组合逻辑的寄存器

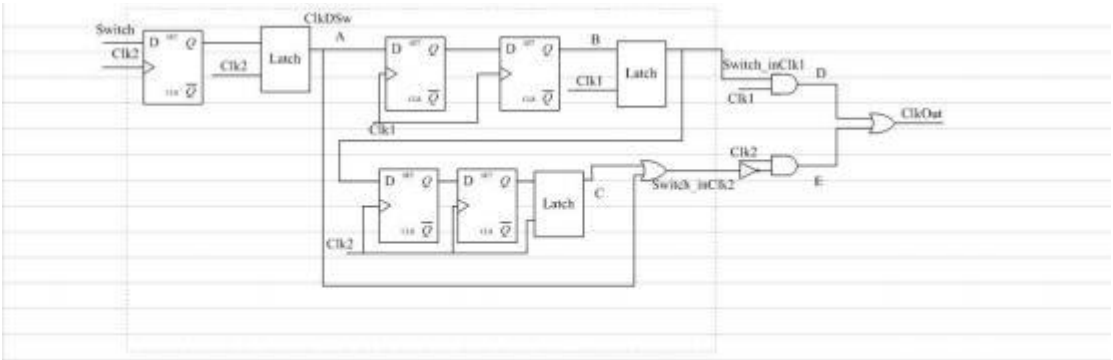
```
always @(posedge clk0 or negedge clk0_rst_n)
    if (!clk0_rst_n)
        clk0_sel_reg0 <= 1'b0;
    else
        clk0_sel_reg0 <= (~select) & (~clk1_sel_reg1);
always @(negedge clk0 or negedge clk0_rst_n)
    if (!clk0_rst_n)
        clk0_sel_reg1 <= 1'b0;
    else
        clk0_sel_reg1 <= clk0_sel_reg0;


always @(posedge clk1 or negedge clk1_rst_n)
    if (!clk1_rst_n)
        clk1_sel_reg0 <= 1'b0;
    else
        clk1_sel_reg0 <= select & (~clk0_sel_reg1);
always @(negedge clk1 or negedge clk1_rst_n)
    if (!clk1_rst_n)
        clk1_sel_reg1 <= 1'b0;
    else
        clk1_sel_reg1 <= clk1_sel_reg0;
```

表示纯粹的组合逻辑

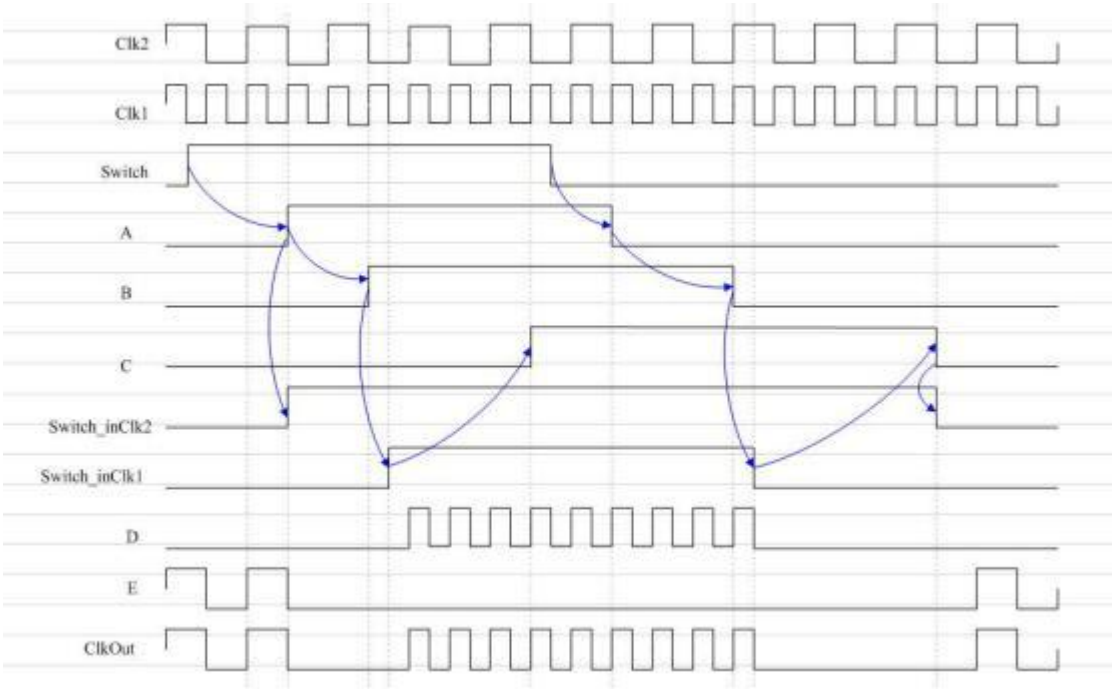
```
assign out_clk = (clk0_sel_reg1 & clk0) | (clk1_sel_reg1 & clk1);
```

17． 动态时钟切换

时钟动态切换指两个时钟源由一个选择信号选择去驱动一个时钟网络，且该切换是在应用过程中较为频繁的，为了保证切换过程中不会出现时钟毛刺等问题，设计该电路保证输出时钟任意时刻都是其中一路时钟的输出，或者没有时钟输出。电路结构及使用方法如下：



从图中可以看出，电路完成的逻辑关系是当Switch为1的时候，ClkOut为Clk1，Switch为0的时候ClkOut为Clk2．整体时序图如下：



当 Switch 从 0 变成 1 的时候，首先会引起 Switch_inClk2 变成 1，以停止 Clk2 通过门控，这时候 ClkOut 输出为 0，直到 Switch_inClk1 为 1，打开 Clk1 的门控，Clk1 输出。

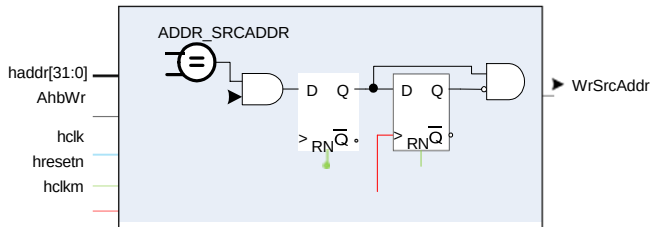
当 Switch 从 1 变成 0 的时候，首先会引起Switch_inClk1 变化以门控住Clk1 的输出，直到 Switch_inClk2 变成 0 之前，ClkOut 没有输出。之后 ClkOut 输出 Clk2。

在这样的时序中可以看出，时钟切换可以做到输出没有毛刺，只是切换时间较长。需要在使用中注意用法。

有的时候，应用该模块切换时钟后可能需要停掉被门控的时钟，这需要确保整个电路都完成切换后在停，否则可能在切换回来的时候发生问题。比方说，从 Clk2 切换到 Clk1 后需要关闭 Clk2，则可以看出 ClkOut 有效输出时 C 点还没有完全同步过来，如果这个时候就停止Clk2，则可能导致下次切换过程出错，因此真正完成 0 到 1 的变化是 Switch、Switch_inClk1 和 Switch_inClk2 相与后在 Clk2 时钟域再打 2 拍后的结果，而真正完成 1 到 0 的变化是三者相或后在Clk2 时钟域再打 2 拍后的结果。

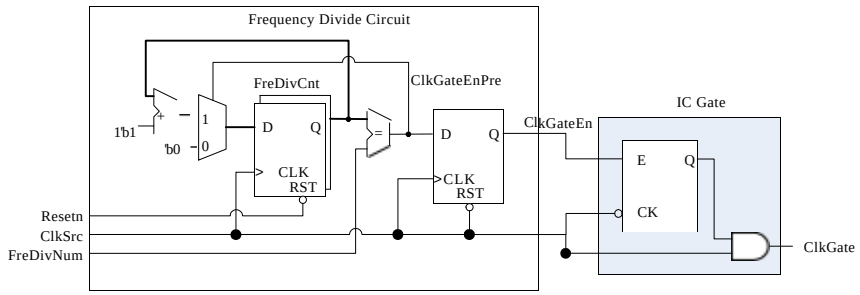
18． 同向不同频

同源时钟同步，一个时钟为另一个时钟的加扰，即在一串脉冲中会突然少几个高电平 ;可以认为是 hclkm>=hclk



分析：hclk 是扣掉时钟个数的时钟，hclkm 为正常时钟；当 AhbWr 为一个脉冲宽度时，被 hclk 采样到，当 hclk 后面的脉冲被扣掉，此时 AhbWr 被采样后的脉冲会变长（扣掉时钟的长度），如果不采用后面的的同步电路，则在 hclkm 域采样时则会认为是多次写动作发生；当采用了后面的同步电路，则会产生已有一个脉冲宽度的高电平。

19． 分频电路




```
reg [DIV_BIT_WIDTH-1:0] div_cnt;
wire [DIV_BIT_WIDTH-1:0] next_div_cnt;
assign next_div_cnt = div_cnt == div ? 0 : div_cnt + 1;
always@(posedge clk or negedge resetn)
    if(~resetn)
        div_cnt     <=  0;
    else
        div_cnt     <=  next_div_cnt;
```

```
wire gate;
assign gate = div_cnt == div;
ICG uClkOut ( .E(gate), .CK(clk), .TE(scan_en), .ECK(clk_out) );
```

注意：div 默认值不要为 0，否则会有一个 clk 的高电平。

如上图所示的分频原理图，ClkSrc 表示被分频的时钟源，FreDivNum 表示分频系数，FreDivCnt 表示分频计数器，ClkGateEn 分频门控信号，ClkGate 表示分频后的时钟输出。ClkGate 和 ClkSrc 之间的频率关系如下公式所示。

$$f_{ClkGate} = \frac{f_{ClkSrc}}{FreDivNum + 1}$$

当 FreDivCnt 的值不为 FreDivNum 时，会一直计数；当 FreDivCnt 计数到 FreDivNum 时，ClkGateEnPre 会置 1，当 ClkSrc 上升沿到来时，FreDivCnt 计数器清 0，同时产生一个 Cycle 的信号 ClkGateEn。ClkGateEn 和 ClkScr 在经过一级 ICG 之后，得到分频后的时钟信号 ClkGate。系统中的 ClkFd、AmbaFd、ApbFd 以及 SacFd 的分频机制均按照此原理进行，便得到相应的时钟使能，并最终经过 ICG 产生相应的时钟。

例如通过软件设置ClkFd 的值，会得到如下的FreDivNum，并最终得到相应分频的时钟。

这样的分频时钟占空比为 1/(FreDivCnt+1),即 只有一个时钟高电平;

(2) 占空比为 1:1 的分频电路

```
always @(posedge clk or negedge resetn)
    if (!resetn) begin
        DIV_SYNCREG0 <= DIV_DEFAULT;
        DIV_SYNCREG1 <= DIV_DEFAULT;
    end
    else begin
        DIV_SYNCREG0 <= div;
        DIV_SYNCREG1 <= DIV_SYNCREG0;
```

```
    end

assign next_div_cnt = div_cnt == DIV_SYNCREG1 ? 0 : div_cnt + 1;
always@(posedge clk or negedge resetn)
    if(~resetn)
        div_cnt     <=  0;
    else
        div_cnt     <=  next_div_cnt;

assign next_clk_out_div = div_cnt == DIV_SYNCREG1 ? ~clk_out_div :
clk_out_div;
always@(posedge clk or negedge resetn)
    if(~resetn)
        clk_out_div <=  1'b0;
    else
        clk_out_div <=  next_clk_out_div;

assign clk_out = clk_out_div;
```

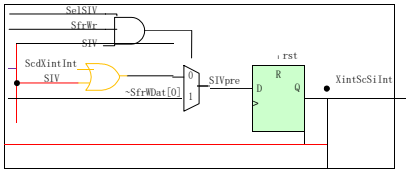
20． 采样判断 0 与 1 的方法：

在 Sample 期间，通过加/减计数器记录采样次数。计数器初值最高位为 1，其余为 0。采样期间，采样数据为 1，计数器+1；为 0，计数器减 1；采样结束，判断计数器最高位是否为 1，如果为 1，表示采样结果为 1，如果为 0，表示采样结果为 0。

如设置计数器为'b1000,则表示采样 7 次，如果 7 次都是 1，则最后固定为'b1111,如果 7 次为 0，则最后结果为'b0001,如果 4 次为 1,3 次为 0，则结果为'b1001,反之为'b0111;注意采样设置肯定是奇数次

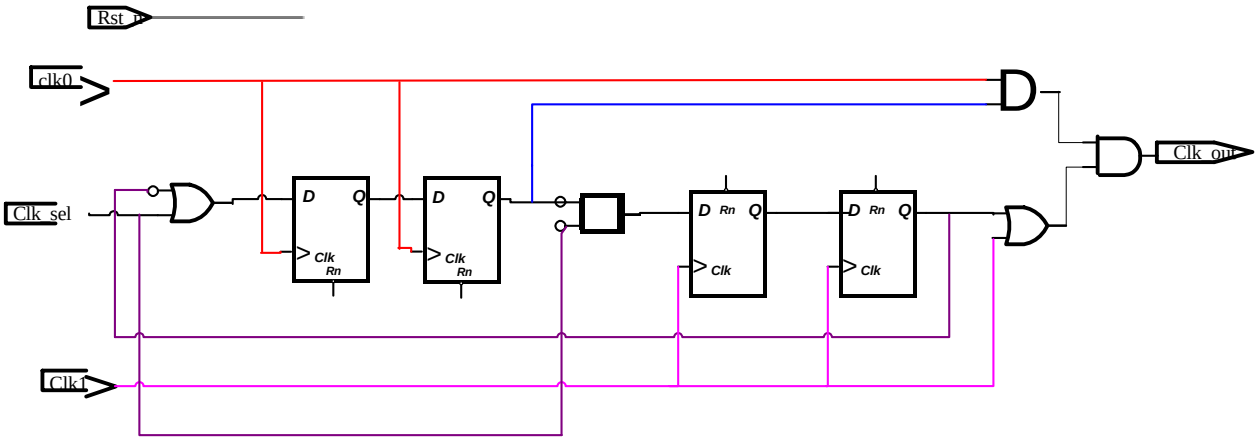
21. 写 1 清 0

寄存器写 1 清 0 并具有保持功能电路，并不影响硬件置 1



22. 时钟切换 2

注意：切换信号 clk_sel 必须是 clk_out 时钟域,所以它需要在两个时钟域同步；当 clk_sel=1,在 clk0 域同步之后与 clk0 相或，一直等于 1，clk_sel 取反后在 clk1 域同步后为 0 与 clk1 相或，得到 clk1 时钟；同理当 clk_sel=0 时，会得到 clk0 时钟



Clk_out=clk_sel?clk1:clk0;

23. 复位

同步复位；没有利用寄存器的 复位端，在 D 端做 清 0 或 1 的处理，通过时钟把 Q 清 0 或 1，有些是在复位端做同步复位

异步复位：利用寄存器的复位端，把 Q 清 0 或 1

24． 复位延时电路设计

利用没有复位端的寄存器来实现延时,其原理是对一组计数寄存器同时进行加和减操作,当加和减的结果相同,则计数完成,但计数长度只有一半,如reg[3:0] 当加减相同时,经过了8 个周期,这种电路设计一般用在上电要求比较严,由于刚上电,复位不稳,寄存器的值是一个随机值---寄存器的电源已经稳定,要不无法工作;当计数不同时,产生复位,计数相同时,则释放.

这个与智力题中：一条粗细不均匀的绳子燃烧需要一个小时，从两头同时点燃则为半个小时的情景是一样的

25． ATPG

在做 ATPG(SCAN)时，寄存器的复位信号要做处理，与正常功能做 MUX，与 ATPG 用的 RST 相连

对于模拟信号，在 ATPG 时，对控制的模拟信号需要给一个固定电平，免得影响模拟电路对整个系统的影响，如电源的开关等

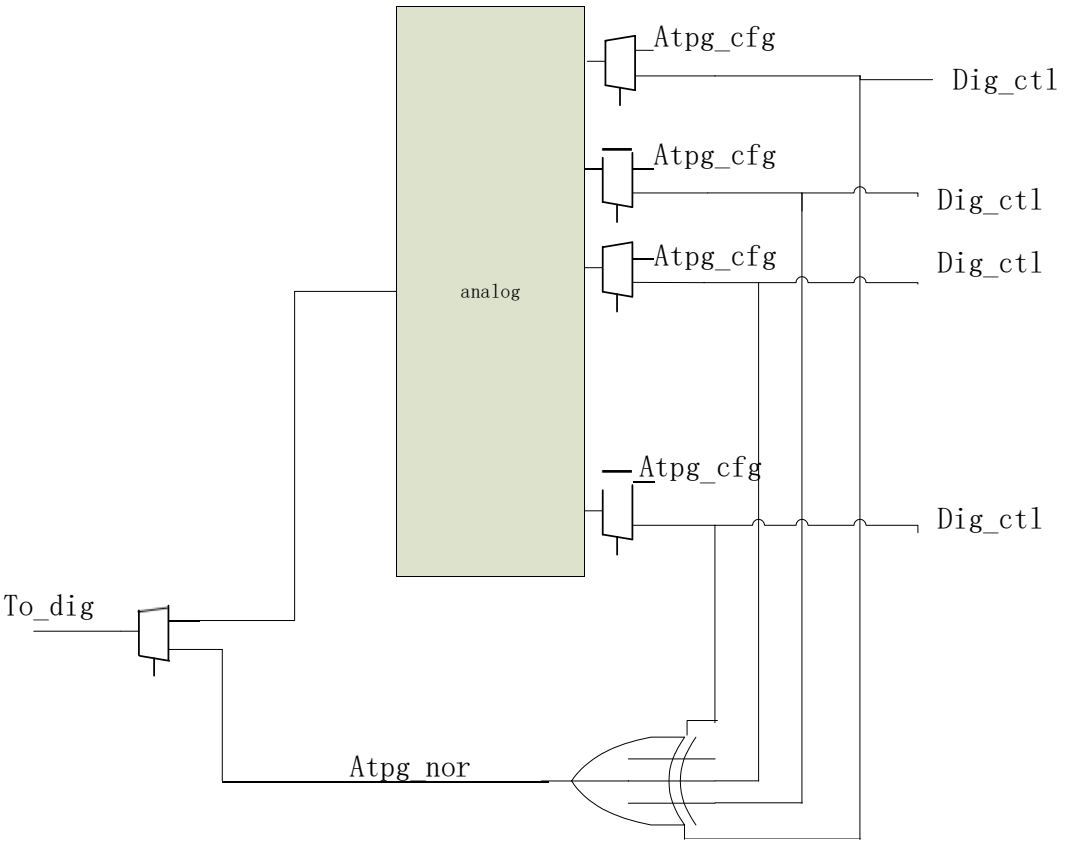
Atpg_mode 信号在整个测试过程中不能发生变化，需要注意此寄存器的设计！

当在做 scan 时,模式寄存器不能插入扫描链中,否则模式寄存器会变化

1.该 scan 模式寄存器的复位端设计成于 scan 的 reset 无关,则综合时不会插入扫描链中

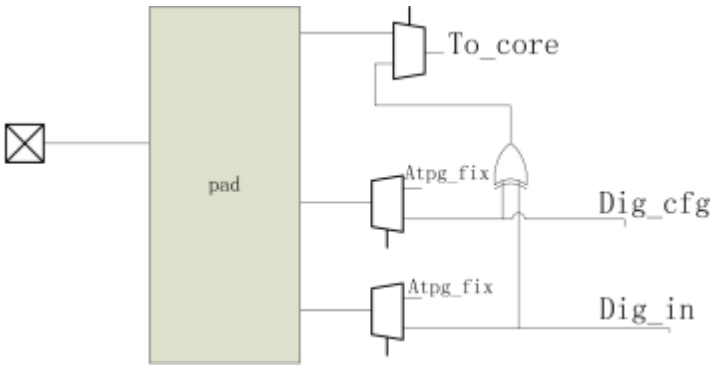
2,综合时把相关寄存器配置为 set_dont_touch_network [get_pins -h "*atpg_test_en_reg/Q"] -no_propagate

在电路设计中一定会有ATPG 模式用来测试数字逻辑,但对于模拟控制部分无法测试到,所以在 atpg 模式下需要修改这部分逻辑,修改防范如下图.dig_ctl表示在function 下的控制信号,atpg_cfg 表示在 atpg 模式的固定的某值,但在 atpg 模式下,使模拟模块不工作,控制逻辑通过异或逻辑产生一个变量通过mux输出到模拟的输出端口;但当该模拟模块没有输出给数字部分,则把这些控制信号与其它模拟模块的控制信号异或后,再送给该模拟输出到数字部分,形成一个完整的数字链



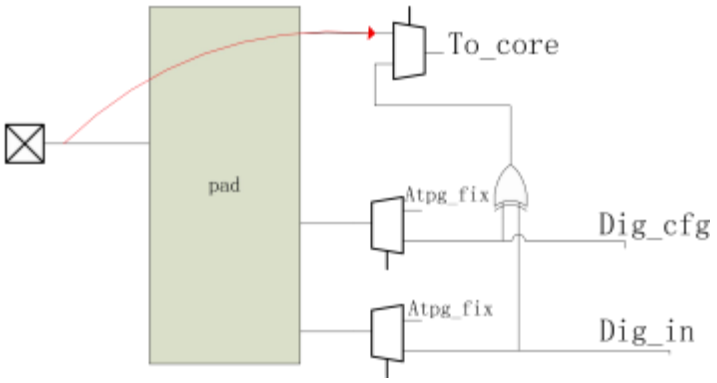
普通 PAD 在 ATPG 下的连接,即不作为atpg 的输入输出口

dig_cfg,指那些上下拉等配制,dig_in 表述需要输出的数字逻辑



PAD 作为ATPG scan_in 的连接方式,即作为第一级寄存器的 D 段直接来自 PAD,其他控制管脚异或到其它管脚去

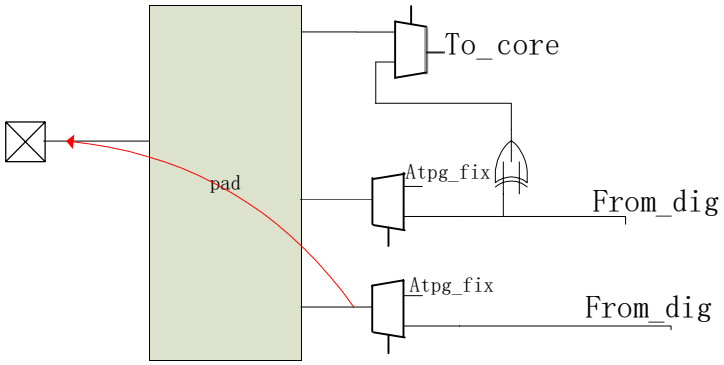
注意该 PAD 在此模式下肯定是要配置为输入模式的,dig_cfg,指那些上下拉等配制,dig_in 表述需要输出的数字逻辑



PAD 作为 ATPG scan_out 的连接方式,即作为最后一级寄存器的 Q 段直接输出到 PAD

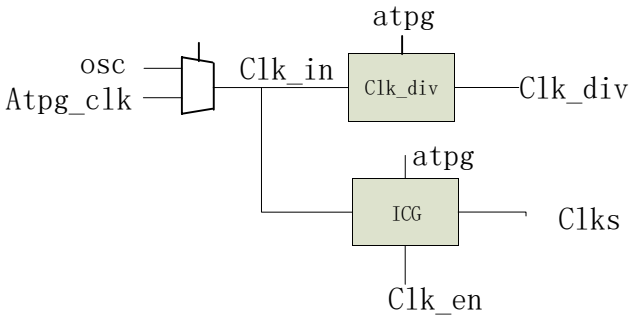
注意该 PAD 在此模式下肯定是要配置为输出模式的

当指定该 PAD 为 scan_out,综合工具会自动对 PAD 的 dig_in 信号做处理,但 PAD 的控制信号还是需要设计者做好



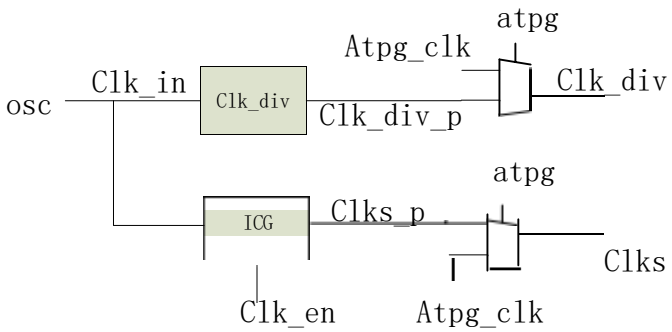
DFT 时钟处理

1).时钟源头做mux



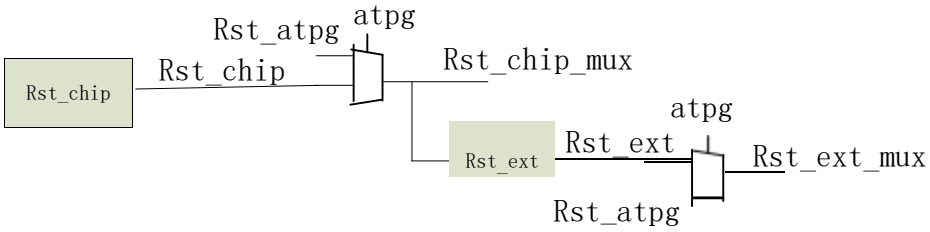
对于 clk_div 模块,当在 atpg 模式下,时钟直通,不分频,同样 ICG 模块中(工艺库会有专门模块),ATPG 模式下,时钟也是直通,实际上 clk_div 模块包含了 icg 模块,就是利用 icg 的直通来处理 clk_div 的直通功能

2).时钟终点做mux



方法 2 于时钟相关的寄存器无法做atpg

DFT 中 reset 信号的处理



每一级的复位信号都做一个mux

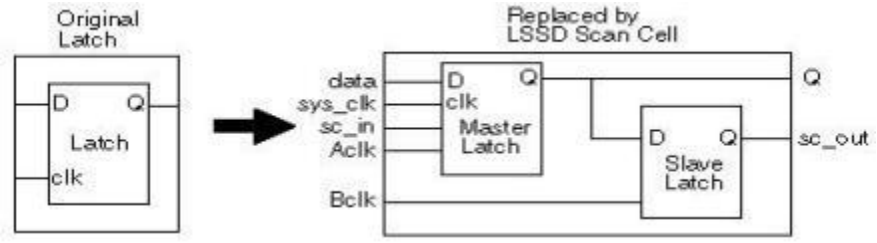
但注意不一定能在最后一级做一个 mux,否则其它复位信号相关的寄存器将无法串到扫描链上

- 内部 tri-state 总线

在扫描的过程中，总线上的驱动也许会同时驱动总线，引起竞争。解决这个问题可以通过添加一个译码逻辑，通过一个mux 去控制每一个 tri-state 驱动的输入使能，mux 用于在普通的操作信号和译码器的输出控制信号之间进行选择，在扫描模式下，选择译码器的输出控制信号，而译码器的输入由外部直接控制，这样，我们可控制 tri-state 驱动的开、和关。

- Latches

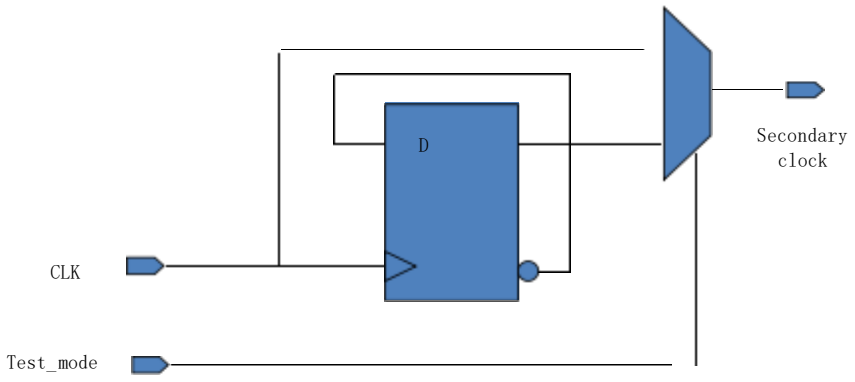
在设计中避免用Latches，因为测试很困难,为使 latch 在扫描模式下透明，需作如下替代。



- Gated or Generated Clocks

DFT 要求每一个 flop 的 clk 可控，所以对于 Gated or Generated Clocks 的逻辑电路用mux 进行旁路，如下图所示,分频逻辑被旁路，在这种情况下，分频逻辑无法扫描，通过如下命令通知 DC 将此 cell 从扫描链中排除:

dc_shell>set_scan_element false<list of cells or design>



■ 用时钟的 single Edge（实际中无需如此）

大部分设计都用时钟的 single Edge，但有些情况下，时钟的 rising 和 falling Edge 都要用到，这种情况在测试模式下可能产生时序问题，我们可为这部分电路加一个 mux,是整个设计在测试模式下用相同的时钟边沿。例如：

```
process(clk,test_mode)
begin
    if(test_mode= ‘1’ )then
        muxed_clk_output<=clk;
    else
        muxed_clk_output<=not(clk);
    endif
endprocess
```

在 ATPG 中分 2 个阶段,一个是 shift 过程,此过程是把数据从端口 shift 到各个寄存器的 D 端,此过程 SE 信号有效,shift 的长度为此条链的数目;然后是 capture 过程,此过程 SE 无效,把 D 端的组合逻辑结果送出给 Q;第一次 capture 输出的结果是最末端寄存器的结果,其它寄存器的结果也展现在 Q 端,即倒数第 2 个寄存器的 Q 端信号进入了组合逻辑到了最后一

个寄存器的 D 端;当第二次来 capture 阶段时,倒数第二个寄存器 Q 端的值输出到了SO 端口;由此可见,在 DFT 中,最初出现的结果是链中最末尾的寄存器相关的结构.

26． 比较电路转化为逻辑与或门电路

对于>,>=.<,<=之类的电路完全可以转化为逻辑门电路，

如：reg[5:0]>=4 可以转化为|reg[5:2]==1'b1;

reg[5:0]>4 可以转化为|[5:3]｜(reg[2]&(|reg[1:0]);即分为》=8 和 7,6,5 两部分

reg[5:0]<=4 可以转化为(|reg[5:2]==1'b0)｜(reg[2]&(!reg[1:0]);即转化为 reg[5:0]<4 和 reg[5:0]==4

reg[5:0]<4 可以转化为|reg[5:3]==1'b0;

27． 乘法电路，转换为左移和加法电路

reg[5:0]*9=>reg[9:0]=reg[5:0]<<3+reg[5:0]

28． 除法电路,转化为左移和右移

结果为近似值,结果越精确，电路会越大

reg[5:0]/7 => reg[5:0]*18/128=> (reg[5:0]<<4+ reg[5:0]<<1) >>7;

或者=》reg[5:0]*146/1024 =》reg[5:0]*(128+16+2)/1024=>

(reg[5:0]<<7+reg[5:0]<<4+reg[5:0]<<1) >>10

29． 时钟 gate 信号处理

组合电路关时钟，注意要在时钟的下降沿关，如果在上升沿关不小心会产生毛刺

30． 自动插 ICG 代码风格

在电路中，很多寄存器都可以在时钟上插 ICG 来停时钟，而非让寄存器的 D 端保持不变，但只有电路是有 Q 反馈到 D 的电路才会自动加 ICG，所有在 RTL 级需要主要书写规格,下面代码省略复位等代码，只列出核心部分

```
always ...
if(counter==8'hff)
   counter= 8'hff;
else
    counter=counter+1;该电路无 Q 反馈到 D，不会插 ICG，应该改为如下电路
if(counter!=8'hff)
  counter=counter+1;无 else 分支，其含义是 else counter = counter;
always ...
if (…)
  a=b;
else
  a =0 ;应该改为
if (…)
  a=b;无 else 分支，其含义是 else a = a;

always ...
if (…)
  a=b;
else if(...)
  a = c;
else if(...)
  a = d ;虽然没写 else,但它仍然不会插 ICG，必须改为
if (…)
  a=b;
```

```
else
   if(...)
      a = c;
   else if(...)
      a = d;        else if 不能超过 2 层，否则都无法自动插入 ICG

always ...
if(rstn==0)  a<=1'b0;
else begin
   case(state_current)
      idle: a <=1'b0;
      ….
   endcase
end
```

需要改写成

```
always ...
if(rstn==0)  a<=1'b0;
else begin
   case(state_current)
      idle: ;
      …
      end_st:begin
         if(state_next!=idle)  a <=1'b0;
      end
   endcase
end
```

31． 乘法电路串行设计

MULA[15:0]*MULB[15:0]

乘法原理：

```
      101
    10110

      000
     101
    101
   000
  101
--------------
```

由此可见（上述 101 代表 mdb,8bit,10110 代表 mda[15:0]）,乘法就是移位相加

```
wire mul_done = (oper_cnt==5'h10) & (state == `DATA_MUL);移 16 次
wire [8:0] add  = mda[23:16]+mdb;
wire [24:0] next_mda_mul =  mul_done ? mda + acc :
                            mda[0] ? {1'b0, add, mda[15:1]} :
                            ~mda[0] ? {1'b0,mda[23:16], mda[15:1]} :
mda;

always@(posedge clk or negedge resetn)
    if(~resetn) begin
        mda     <= 24'habcf;
    end
    else begin
        mda     <= next_mda;
    end
```

32． 除法电路串行设计

```
    00100


101|10110
   |101
   |--------
   |00010
```

由此可见，除法就是移位相减,移的是被除数,而不是除数,相当于上面固定 101,10110 不断向左移动,然后相比较.

方法一:

```
wire        cmpr = {sdr[6:0],mda[23]} < mdb;
wire [23:0] next_mda = start ?  next_mda_div : mda;


wire [23:0] next_mda_div = {mda[22:0],~cmpr};
wire [7:0] next_sdr =  div_start ? 8'h0 :
                       start ? ( cmpr ? {sdr[6:0],mda[23]} :
                                        {sdr[6:0],mda[23]} - mdb ) : sdr ;
always@(posedge clk or negedge resetn)
    if(~resetn) begin
        mda     <= 24'habcf;
        sdr     <= 8'h0;
    end
    else begin
        mda     <= next_mda;
        sdr     <= next_sdr;
    end
```

方法二:

总共 13bit,高 3bit 为整数部分,低 10bit 为小数部分,输入数据为 10bit;下面 coce 条件为 a/b<=5;可以根据下面原理扩展到 32bit 任意除法,这种方法太耗时,需要 64 个时钟周期

```
    assign  div_done =  (oper_cnt==4'd12);
    wire  next_start = div_done ? 1'b0  :
                       div_start ? 1'b1  : start;
    wire  [3:0] next_oper_cnt = div_start ? 4'd0  :
                                start ? oper_cnt + 4'b1  : oper_cnt;

    always@(posedge clk or negedge resetn)
        if(~resetn) begin
            start       <=  1'b0;
            oper_cnt    <=  4'b0;
        end
        else begin
            start       <=  next_start;
            oper_cnt    <=  next_oper_cnt;
        end

    wire  [11:0] mdb =  {adc_data2,2'b00};
    wire  [12:0] cmpr = mda - mdb;

    wire  [12:0] next_mda = div_start ?  {3'b000,adc_data1}  :
                            start ?  (cmpr[12] ?  {mda[11:0],1'b0}  :
{cmpr[11:0],1'b0})  : mda;
    wire  [12:0] next_sdr = div_start ?  13'h0  :
                            start ?   {sdr[11:0],~cmpr[12]}  :  sdr  ;

    always@(posedge clk or negedge resetn)
```

```
if(~resetn) begin
    mda      <= 13'h0;
    sdr      <= 13'h0;
end
else begin
    mda      <= next_mda;
    sdr      <= next_sdr;
end
```

33． 二分法电路设计

使用条件:输出与输入必须具有单调线性关系,即输出随输入增大而增大,或输出随输入增大而减小;此方法最多需要参数位宽的次数;

注意实际应用,如果是在用户模式下对时钟做 trim,不能使用二分法,因为它的变化太大,如果 trim 到过高频率,则电路不正常,系统将会崩溃,千万要小心!可以采用+/-1 的方法,再判断差值是否在允许的误差范围,如果在允许的误差范围内则 trim 结束,否则继续 trim,同时需要考虑 trim 不成功的情况(坏片).

有 2 点需要注意:

1.1当在某时刻,输入参数符合要求,可以立即退出

1.2当在最后还没有符合参数要求,需要根据0bit 为 0 或 1 时,选择离参考值最接近的数. 比较时需要注意输出值与参考值的大小关系,否则会出现负数无法处理.

如下电路: 存在一个小 bug,没有考虑==的情况,但并不会影响最后结果,只是不能快速结束自动 trim;虽然这种情况在本产品中出现的概率很小,

1.3唯一的好处是每次计算的时间是固定的,都是 n 次计算

```
    wire [3:0] next_osc800k_trim_cnt = osc800k_trim_proc &
(osc800k_trim_cnt == 4'd7) ? 4'd0 :
                                        osc800k_trim_proc?
osc800k_trim_cnt+4'd1 : osc800k_trim_cnt;

    reg [15:0] osc800k_trim_dis_6 ;
    reg [6:0] osc800k_trim_6 ;
    always@(posedge OSC800K or negedge rst_sys)
        if(~rst_sys) begin
            osc800k_trim_new      <= 7'h40;
        end
        else if(osc800k_trim_init) begin
            osc800k_trim_new      <= 7'h40;
```

```
        end
        else if(osc800k_trim_proc)begin
            if(osc800k_trim_cnt=='h7)begin
                if(osc800k_cnt>=P_REF_OSC800K)begin //此处应
该分开,考虑 osc800k_cnt==P_REF_OSC800K,此时则 trim 结束,无需做
后面的操作,下面同理

if((osc800k_cnt-P_REF_OSC800K)>=osc800k_trim_dis_6)begin
                        osc800k_trim_new <= osc800k_trim_6;
                    end
                end
                else begin
                    if((P_REF_OSC800K -
osc800k_cnt)>=osc800k_trim_dis_6)begin
                        osc800k_trim_new <= osc800k_trim_6;
                    end
                end
            end
            else begin
                if(osc800k_cnt>P_REF_OSC800K)begin
                    osc800k_trim_new[6-osc800k_trim_cnt] <=
1'b0;
                end
                else begin
                    osc800k_trim_new[6-osc800k_trim_cnt] <=
1'b1;
                end
                if(osc800k_trim_cnt!=6)begin
                    osc800k_trim_new[5-osc800k_trim_cnt] <=
1'b1;
                end
            end
        end
    always@(posedge OSC800K or negedge rst_sys)
        if(~rst_sys) begin
            osc800k_trim_dis_6    <= 'b0;
            osc800k_trim_6        <= 'b0;
        end
        else if(osc800k_trim_proc & (osc800k_trim_cnt == 4'd6))
begin
            osc800k_trim_6 <= osc800k_trim_new ;
            if(osc800k_cnt>=P_REF_OSC800K)begin
                osc800k_trim_dis_6 <=
osc800k_cnt-P_REF_OSC800K;
```

```
                end
                else begin
                    osc800k_trim_dis_6 <=
        P_REF_OSC800K-osc800k_cnt;
                end
            end


        always@(posedge OSC800K or negedge rst_sys)
            if(~rst_sys) begin
                osc800k_trim_cnt    <= 4'd0;
            end
            else begin
                osc800k_trim_cnt    <= next_osc800k_trim_cnt;
        end
```

上面的 osc800k_trim_new[5-osc800k_trim_cnt]这种写法在位宽不大时,可以写成 case 语句来实现,这样在 nlint 检查时不会有警告

34． 无符号数加一个有符号数

用 wire 定义的类型,加法一定是无符号数加法
当用reg 定义变量类型,结果如下:

3. 如果想处理有符号加法,则变量需要全部定义为有符号类型
4. 只要变量(含输入输出)有一个为无符号数,那么将会生成无符号加法器,把符号位当作正常数据处理

如下

```
reg signed [7:0] a;
reg [7:0] b;
reg signed[8:0] c;

 wire [7:0] a1;
 wire[7:0] b1;
 wire[8:0] c1;

assign c1 =a1+b1;
always @(*)begin
    c= a +b;
end

initial begin
    a = -5;
    b = 8'h2;
    force a1 = 8'hf1;
```

```
    force b1 = 8'h2;
    #0;
    $display ("a=%0h,%0d ,b=%0h,%0d ,c=%0h,%0d ,",a,a,b,b,c,c);

$display ("a1=%0h,%0d ,b1=%0h,%0d ,c1=%0h,%0d ,",a1,a1,b1,b1,c1,c1);
end
```

打印结果:

```
a=fb,-5 ,b=2,2 ,c=fd,253 ,
a1=f1,241 ,b1=2,2 ,c1=f3,243 ,
```

```
reg signed [7:0] a;
reg signed[7:0] b;
reg signed[8:0] c;
//////////////////////////
reg signed [7:0] a;
reg signed[7:0] b;
reg signed[8:0] c;

 wire [7:0] a1;
 wire[7:0] b1;
 wire[8:0] c1;

assign c1 =a1+b1;
always @(*)begin
    c= a +b;
end

initial begin
    a = -5;
    b = 8'h2;
    force a1 = -5;
    force b1 = 8'h2;
    #0;
    $display ("a=%0h,%0d ,b=%0h,%0d ,c=%0h,%0d ,",a,a,b,b,c,c);

$display ("a1=%0h,%0d ,b1=%0h,%0d ,c1=%0h,%0d ,",a1,a1,b1,b1,c1,c1);
end
```

打印结果:

```
a=fb,-5 ,b=2,2 ,c=1fd,-3 ,
a1=fb,251 ,b1=2,2 ,c1=fd,253 ,
```

定点数据和浮点数据

数字信号处理的输入源通常都是物理世界的模拟信号，其电平的表示是连续，数字处理会对其进行抽样，在算法阶段，会按照浮点运算的方式进行算法性能仿真，以便于评估最优性能边界。但是由于浮点运算硬件实现代价较大，且算法进行定点化以后的性能劣化通常也在实际使用可以接受范围，因此实际工程实现通常都采用定点化方式，实现算法链路。

浮点格式可以参考 IEEE 754，由于实际工程使用不多，因此这里不做过多叙述，主要介绍定点的方法：

在定点数中，定义小数点的位置，把一个定点数分为两个部分，小数点左边部分的位宽为整数位宽，右边部分为小数位宽，小数点右边为 0~1 之间的小数，小数位宽则代表精度，比如（16,4）表示定点数位宽为 16，整数位宽为 4，小数位宽为 12。当然可能不同公司会有不同定义，但是小数位宽和整数位宽的概念是相同的。

以（4,2） 为例，4 位位宽，2 位小数位

```
     bit3     bit2    bit1      bit0
      0         1        1         0
  整数位    整数位   小数位     小数位
     bit1      bit0    bit-1    bit-2
value(dec) = (2^1) * 0 + (2^0)* 1 + (2^-1) * 1 + (2^-2) * 0 = 1.5
```

```
Reg[15:0] data1  ;---无符号数
Reg signed[7:0]  data2; ---有符号数 -128 ~ 127
Reg[15:0] result;--有符号数
Result = data1 + {8{ data2[7]}, data2} ;因为系统只有 8/18/32/64bit
{8{ data2[7]}, data2} ;为符号位扩展
Data1=0x0100; data2=0xff;//-1 即 256-1=255
Result = 0x0100+0xffff =0x00ff  ;//255

Data1=0x0100; data2=0x7f;//127 即 256+127=383
Result = 0x0100+0x007f =0x017f  ;//383
```

最后结果为表示有符号数的结果

```
Reg[7:0] data1  ;---无符号数
Reg signed[7:0]  data2; ---有符号数 -128 ~ 127
Reg[8:0] result;--有符号数结果比加数至少要多一位
Result = data1 + {{ data2[7]}, data2};
```


咸鱼:东

```
Data1=0xff; data2=0xff;//-1 即 255-1=254
Result = 0xff+0x1ff =0x0fe ;//254
Data1=0xff; data2=0x7f;//127 即 255+127=382
Result = 0xff+0x07f =0x017e ;//382
```

定点减法电路可以转化为+上一个减数的补码（取反+1）：a - b = a + (~b)

7-4 = 111 -100 = 111+（011+1） = 011 =3

35． 减法结果一定为有带符号类型

如下:

```
reg[7:0]a,b;
reg[8:0]c;
reg unsigned [7:0]a2,b2;
reg unsigned [8:0]c2;
wire[7:0]a1,b1;
wire[8:0]c1;
assign c1=a1-b1;
always@(*)begin
    c=a-b;
end
always@(*)begin
    c2=a2-b2;
end

initial begin
#10;
    a=5;
    b=8;
    a2=5;
```

```
    b2=8;


  force   a1=5;
  force    b1=8;
#0;
    $display ("c=%0h,c1=%0h,c2=%0h",c,c1,c2);
End
```

无论被减数和减数是否为有符号类型,但结果于是有符号类型

由于减法结果一定是带符号的,所以注意结果需要多 1bit来表示正负数符号.

输出结果为: c=1fd,c1=1fd,c2=1fd

如果对 c,c1c2 去反+1,则得到结果为 0x3

36. 乘以小数处理方法

对于乘以固定小数办法一般都是采用移位的方法来实现,不过肯定有误差,
如*0.356 可以如下处理
0.356 =356/1000=(356*1.024)/1024 = 365/1024=(256+64+32+8+4+1)/1024
则相当于对数据左移 8bit,6bit ,5,3,2,1bit 再右移 10bit,优化下就是:
(256+64+32+8+4+1)/1024 =
256/1024+64/1024+32/1024+8/1024+4/1024+1/1024,则变成了直接右移
2,4,5,7,8,9bit 后再相加
Din[15:0]*0.356 = din>>>2+din>>>4+din>>>5+din>>>7+din>>>8+din>>>9
注意,上述 356 可以有多种组合,但需要使加法数量最少,每次找到最大的移位数;下面扩展需要根据需求;如上误差计算,设输入的数据是 1000,实际结果为 356,经过逻辑计算结果为 354,误差为 0.0056%;
如果输入为 10,实际结果为 3.56,逻辑处理后=2,误差为50%,所以需要根据实际使用情况来放大.

37. 电路设计总结—注意事项

模块设计:

a. 先把模块功能分解;
b. 梳理各个功能块的关系,需要如何交互;
c. 确定哪些功能块并行处理,哪些功能块串行处理;
d. 确定各个功能块的时钟复位属于哪个域;
e. 异步需要采用何种方式同步;

1. 所有的设计行为都是来源于一个启动/请求信号（start/req）和一个完成/应答信号（done/ack）,只是有些没必要显式来产生这两个信号，比如xram 总线，它是固定的时序，所以没必要弄出这两个信号.

2. 设计主要在组合逻辑即寄存器的 D 端，Q 端只是用来描述寄存器

3. Always 块描述组合逻辑时，默认部分逻辑相当于 else

```
Always(*)begin
    A =b; //相当于下面 if else 逻辑加了句 else a=b;
    If(add)
    A=c+d;
    Else if (min)
    A=c-d;
End
```

注意产生锁存器---如果没有寄存器描述,没有 else 的 always 会产生锁存器电路

4. 在做 atpg 设计时，需要注意相关电源不要被关断---因为寄存器在做 atpg 时会翻转，使用需要注意做 bypass,固定开启电源,但要注意实际测试是否会使用外部电源,模拟设计需要考虑这种情况

5. 在做 atpg 设计时，如果 atpg 模式信号是寄存器产生，需要注意在做 atpg 时会翻转，模式信号可能会变化，要把 atpg_mode（或其它名字）bypass 掉，不能串到链上去，不做 atpg

   1. 该 scan 模式寄存器的复位端设计成于 scan 的 reset 无关,则综合时不会插入扫描链中

   2,综合时把相关寄存器配置为 set_dont_touch_network [get_pins -h "*atpg_test_en_reg/Q"] -no_propagate

6. 做系统设计时，从时钟模块出去的时钟一定要通过atpg mux 后的时钟，同样复位信号也要是通过atpg mux 后的复位，但要注意如 atpg_mode 寄存器的复位是否需要mux。验证也需要注意，否则有些寄存器无法进行 atpg 测试。

7. Memory bist 模块会需要一个 bist_clk,而 memory 本身会需要一个 clk,一般我们不会去做两者的同步，所以当在bist mode 时，需要把两个时钟切换到同一个时钟；一般 bist 模块是第三方 ip，所以必须要注意该点

8. 在做 flash bist 时,由于 flash bist 是一个测试模式,当进入该模式后,最好能返回一个握手行为,表示进入该模式,然后才可以从外面输入各种测试 pattern—主要是为了防止在测试时不正常无法确认是否进入了模式.

9. 所有的模块设计应该有两种归位的方式;
1,复位

2 条件使寄存器在复位状态;如状态机,一个完整的状态机需要考虑下述条件:

` (1) 这个完整状态机在运行过程中,如果再次启动,是立即打断跳回初始态,还是忽略,又或是记录状态,等完成该次运行结束,再重新启动状态机

(2)异常情况处理,其实就是与(1)一样考虑

（3） 电路中的计数电路,要注意计数条件,计数完成是清 0 还是保持会影响到插 icg

10. ASIC 降动态功耗是时钟门控---把在寄存器 D 端逻辑转化到使 clk 的 ICG 的 E 控制上;而 FPGA 优化是把时钟上的控制优化到寄存器的 CE 信号端.

14. 对于数模接口,数字给模拟的信号要通过寄存器;模拟送给数字信号,首先要同步,然后根据模拟特性及项目需求如何采样,要考虑是否有毛刺,毛刺可能的宽度及采样时钟的关系

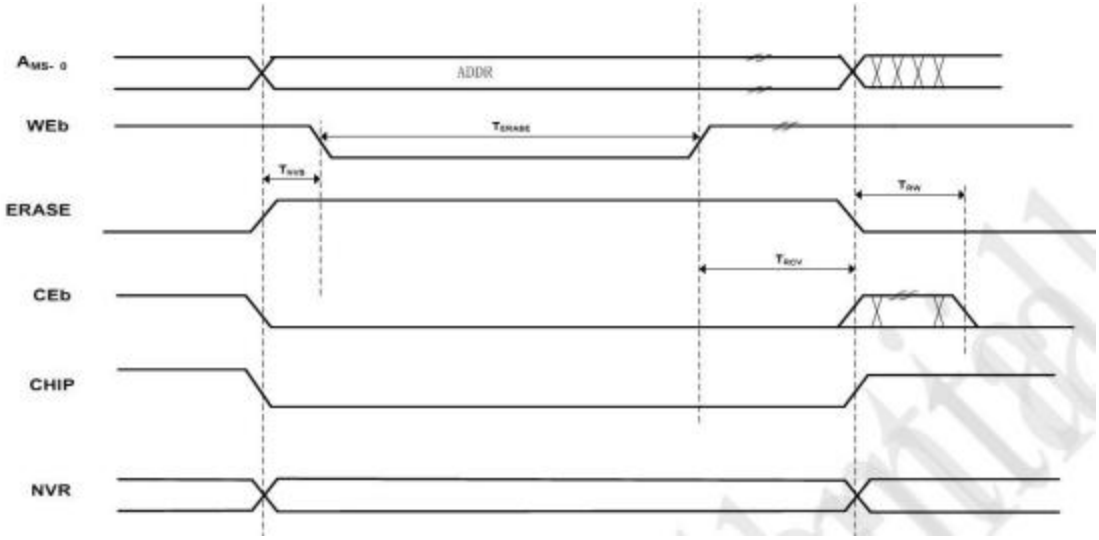
15. 模二加 就是指二进制加法,而不管进位,对应的电路就是异或门

16. 模拟控制信号命名规范,因为模拟模块都会需要测试,所以最好在控制信号上对名字进行规范,如在用户模式出来的控制信号加个后缀 _u(usr)或_r(regiser),在测试模式下出来的控制信号加个后缀 _t(test);经过 mux 连接到模拟端口的信号则不加后缀,这样容易识别

控制信号来源;如模拟模块使能信号analog_en,用户模式下通过寄存器配置出来的则叫analog_en_u,测试模块出来的信号就叫analog_en_t,则 analog_en = test_en ?analog_en_t : analog_en_u;

17. 在例化一些带有复位参数的模块,需要注意是否会输出脉冲,主要是在一些有比较的模块中易忽视,因为复位如果是 0,而计数器一般从 0 开始计数,所以一开始可能会满足条件

18. 对于时序控制一般就是采用计数器或状态机来实现,计数器就是当计数器等于多少,哪些信号置 1/0;状态机则为在某个状态,哪些信号为 0/1;如下 flash 的 erase 接口时序



当启动 erase 命令时,counter 开始计数web,erase 等信号根据 counter 值来变化,设计如下

```
always@(posedge clkim or negedge resetn)
    if(~resetn)
        counter <= 16'd0;
    else
        counter <= next_counter;


wire next_fweb =   (byte_prog_sync | dbg_byte_prog_sync) & (counter == 16'd8)         ? 1'b0 :
                   (sector_erase_sync | dbg_sector_erase_sync ) & (counter == 16'd8)  ? 1'b0 :
                   dbg_chip_erase_sync & (counter == 16'd8)                           ? 1'b0 :
                   fnvr_wr & (counter[5:0] == 6'd8)                                   ? 1'b0 :
                   (byte_prog_sync | dbg_byte_prog_sync) & (counter == 16'd19)           ? 1'b1 : //m
                   //(byte_prog_sync | dbg_byte_prog_sync) & (counter == 16'd22)         ? 1'b1 : /
                   (sector_erase_sync | dbg_sector_erase_sync ) & (counter == 16'd4508)  ? 1'b1 :
                   dbg_chip_erase_sync & (counter == 16'd35000)                       ? 1'b1 :
                   fnvr_wr & (counter[5:0] == 6'd31)                                     ? 1'b1 :
                   fweb;

wire next_fprog = (byte_prog_sync | dbg_byte_prog_sync) & (counter == 16'd28) ? 1'b0 : //+3ms
                   fnvr_wr & (counter[5:0] == 6'd37)                          ? 1'b0 : //+3ms
                  (byte_prog_sync | dbg_byte_prog_sync) & (counter == 16'd1) ? 1'b1 :
                   fnvr_wr & (counter[5:0] == 6'd1)                           ? 1'b1 :
                  fprog;

wire next_fprog2 = (byte_prog_sync | dbg_byte_prog_sync) & (counter == 16'd18)  ? 1'b0 : ////modify b
                   fnvr_wr & ((counter[5:0] == 6'd18) | (counter[5:0] == 6'd30))? 1'b0 : //+3ms
                   (byte_prog_sync | dbg_byte_prog_sync) & (counter == 16'd11)  ? 1'b1 : ////modify b
                   fnvr_wr & ((counter[5:0] == 6'd11) | (counter[5:0] == 6'd23))? 1'b1 : ////modify b
                   fprog2;
```

DFT

Dft 分类

Dft: design for test,可测试性设计,在 soc 产品中,包含了数字部分,模拟部分,及数模混合部分(存储器)

数字部分-----通过 atpg 向量 来测试

模拟部分 --- 需要设计专门的电路来测试所有模拟 ip

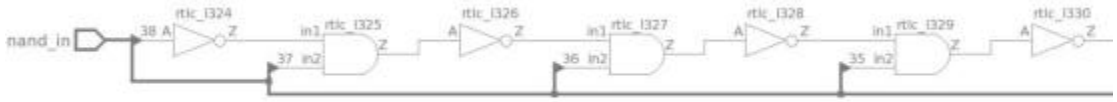
存储器部分 --- memorybist(sram),一般 ip 厂商会提供,flash 部分,也由 ip 厂商提供

管脚 -- 边界扫描,可以使用如下电路代替测试 pad

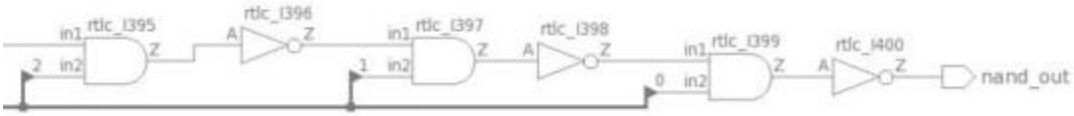
```
module nand_tree (nand_in, nand_out);
parameter NAND_IN_SIZE = 30;
input [NAND_IN_SIZE-1:0] nand_in;
output nand_out;
reg nand_out;
integer i;
always @(nand_in)
begin
    nand_out = 1;
```

```
    for (i=NAND_IN_SIZE-1;i>=0;i=i-1) nand_out = ~(nand_out &
nand_in[i]);
end
endmodule
```

对应电路如下



……



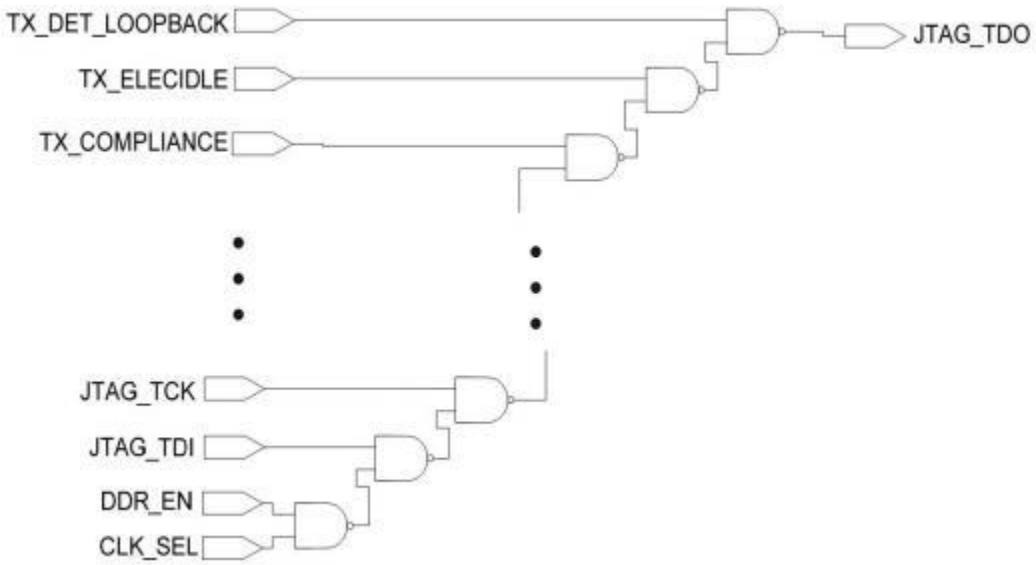
上面电路特点:

1. 初始所有的输入为 0
2. 然后依次按照nand_in 的顺序拉高
3. 此时输出发生翻转

参考电路及时序

1. jtag nandtree

   TX_DET_LOOPBACK 为序号 1,后面的顺序排列



2.utmc 的 nandtree 结构

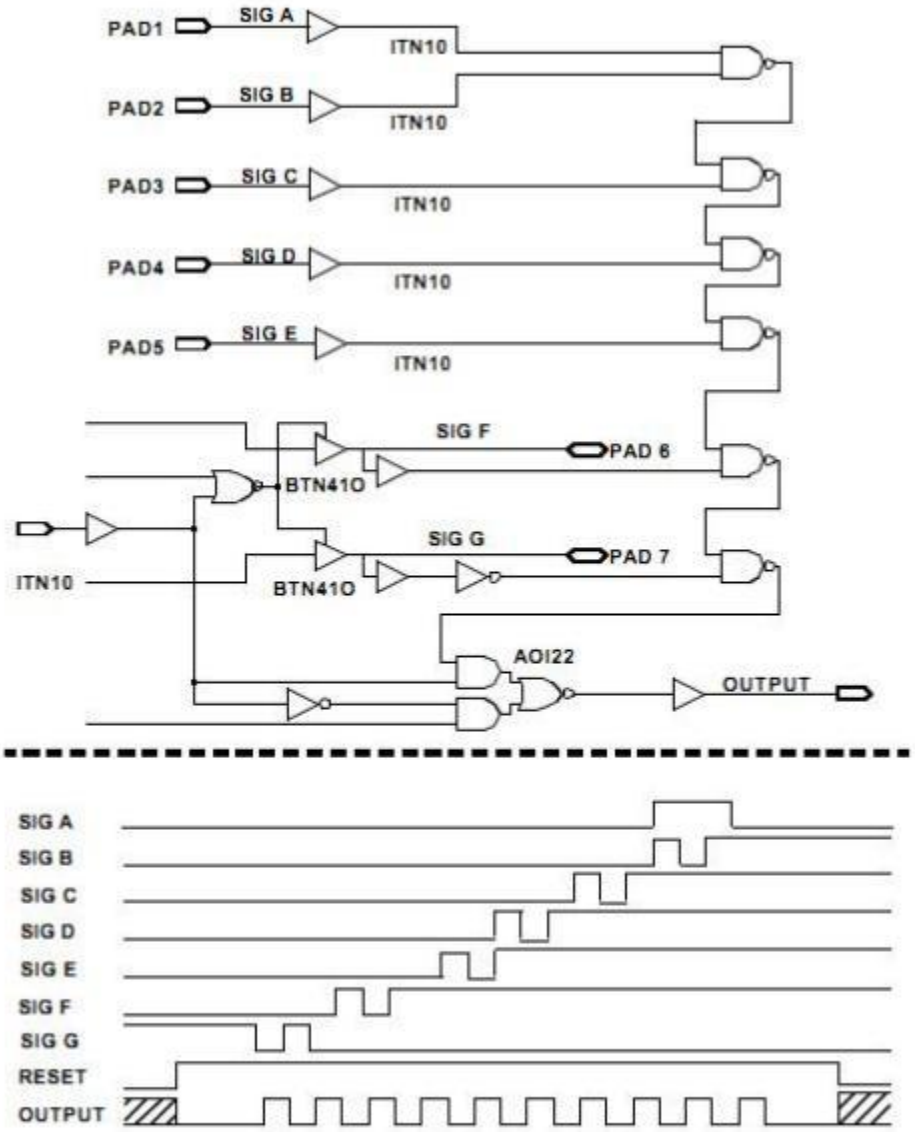


Figure 1. Tree Circuit and Waveforms for DC Testing

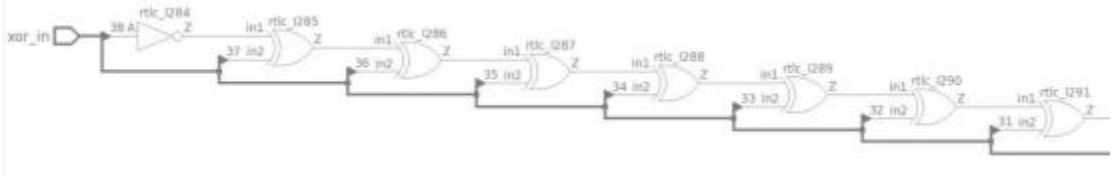
3.其他能够做到的时序

cycle	IN1	IN2	IN3	IN4	IN5	IN6	IN7
00001	0	0	0	0	0	0	0
00002	0	0	0	0	0	0	1
00003	0	0	0	0	0	1	1
00004	0	0	0	0	1	1	1
00005	0	0	0	1	1	1	1
00006	0	0	1	1	1	1	1
00007	0	1	1	1	1	1	1
00008	1	1	1	1	1	1	1
00009	0	1	1	1	1	1	1
00010	0	0	1	1	1	1	1
00011	0	0	0	1	1	1	1
00012	0	0	0	0	1	1	1
00013	0	0	0	0	0	1	1
00014	0	0	0	0	0	0	1
00015	0	0	0	0	0	0	0

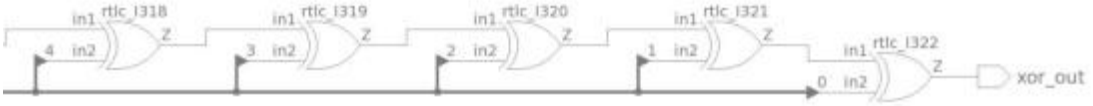
```
module xor_tree  (
       xor_in,
       xor_out);
       parameter XOR_IN_SIZE = 46;
       input  [XOR_IN_SIZE-1:0] xor_in;
       output xor_out;
       reg xor_out;
       integer i;
       always @(xor_in)
       begin
           xor_out = 1'b1;
           for  (i=XOR_IN_SIZE-1;i>=0;i=i-1) xor_out =  (xor_out ^
   xor_in[i]);
```

```
    end
endmodule
```

对应的电路为



……



上面电路特点:

1. 初始所有的输入为 0
2. 然后 xor_in 任何一个 pad 拉高
3. 此时输出发生翻转

可以测试出 pad 是否正常.

芯片测试流程

5. CP 测试 -- 裸 die 测试
   此阶段由专门的测试厂测试,一般测试内容为 scan,厂家的 ip,如 flash,sram 等专用 ip.
   此阶段需要考虑电源如何供电,一般 soc 都含有模拟和数字模块,可能会有一个模拟电源(5v 或 3.3v…)和一个数字电源(1.5v 或 1.2v…),尤其是数字电源为 ldo 输出时,是否需要做 trim,如果机台不提供电源,厂家如何 trim ldo 到要求电压,如果机台供电,如何切换电源;
   由于此阶段是同时测试多个 die,设计时需要考虑信号不能同时到达各个 die,如果在测试阶段需要握手,可能不能同时发生.
   如果 cp 分了几个阶段,需要考虑cp1 的结果在 cp2 的时候能否自动更新,同时也要考虑如果测试过程中需要断电,测试结果如何处理,是只需要保存还是上电需要自动更新.
6. FT 测试 - 封装后测试
   此阶段主要考虑一些不稳定现象,如电源,接触等这些不稳定需要重测解决;
   需要注意一些测试顺序,比如测试电源,时钟,测试完之后再进入其它测试项,这些测试结果是否需要维持以保证后面测试能正常

注意事项:

1.对于模拟 osc 测试需要注意采样频率与待测试试种的关系,由于采样会有电平长度损失,所以会带来误差,会有一个采样时钟内的误差,所以需要考虑能够接受的误差

ATPG 设计测试相关

1． 设计注意事项

1. 在做 atpg 设计时，需要注意相关电源不要被关断---因为寄存器在做 atpg 时会翻转，使用需要注意做 bypass,固定开启电源,--此点无法验证,只能代码设计保证!!!—虽然电源一般会用到复位上去,模型可以检查如果使能没开启复位不释放,但 atpg 的复位是外部输入,所以无法验证.

2. 在做 atpg 设计时，如果 atpg 模式信号是寄存器产生，需要注意在做 atpg 时会翻转，模式信号可能会变化，要把 atpg_mode（或其它名字）bypass 掉，不能串到链上去，不做atpg

2.1.该 scan 模式寄存器的复位端设计成于 scan 的 reset 无关,则综合时不会插入扫描链中

2.2,综合时把相关寄存器配置为 set_dont_touch_network [get_pins -h "*atpg_test_en_reg/Q"] -no_propagate

3. 做系统设计时，从时钟模块出去的时钟一定要通过 atpg mux 后的时钟，同样复位信号也要是通过 atpg mux 后的复位，但要注意如 atpg_mode 寄存器的复位是否需要mux。验证也需要注意，否则有些寄存器无法进行 atpg 测试。

4. 在做 ATPG(SCAN)时，寄存器的复位信号要做处理，与正常功能做 MUX，与 ATPG 用的 RST 相连

5. 对于模拟信号，在 ATPG 时，对控制的模拟信号需要给一个固定电平，免得影响模拟电路对整个系统的影响，如电源的开关等

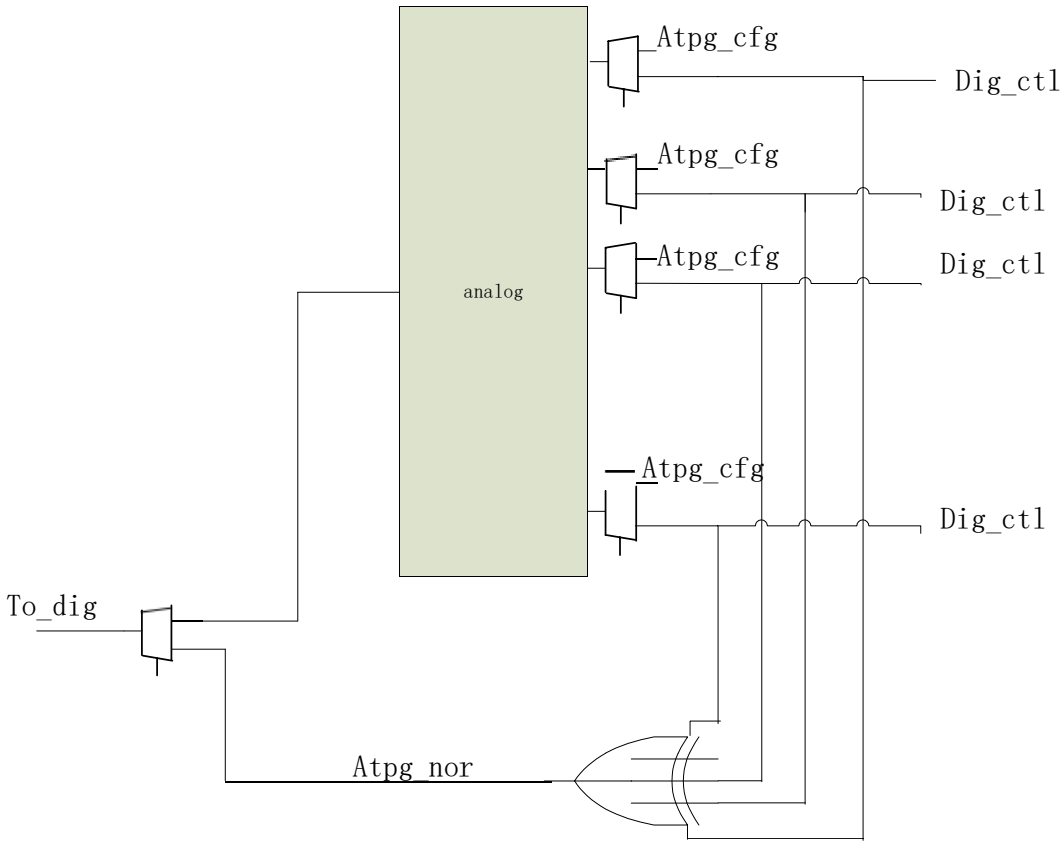
   Atpg_mode 信号在整个测试过程中不能发生变化，需要注意此寄存器的设计!

当在做 scan 时,模式寄存器不能插入扫描链中,否则模式寄存器会变化

1.该 scan 模式寄存器的复位端设计成于 scan 的 reset 无关,则综合时不会插入扫描链中

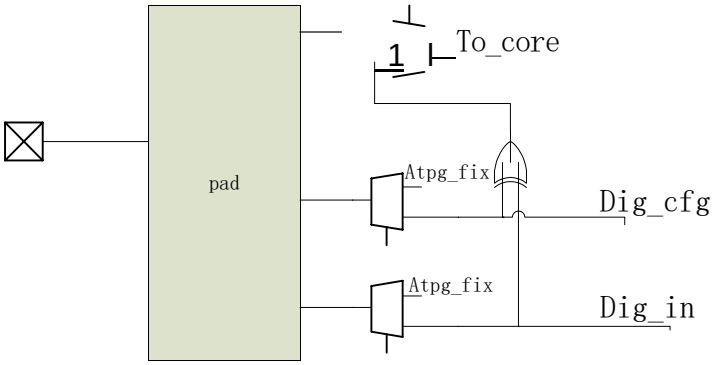
2,综合时把相关寄存器配置为 set_dont_touch_network [get_pins -h "*atpg_test_en_reg/Q"] -no_propagate

6. 在电路设计中一定会有 ATPG 模式用来测试数字逻辑,但对于模拟控制部分无法测试到,所以在 atpg 模式下需要修改这部分逻辑,修改方法如下图.dig_ctl表示在 function 下的控制信号,atpg_cfg 表示在 atpg 模式的固定的某值,但在 atpg 模式下,使模拟模块不工作,控制逻辑通过异或逻辑产生一个变量通过 mux 输出到模拟的输出端口;但当该模拟模块没有输出给数字部分,则把这些控制信号与其它模拟模块的控制信号异或后,再送给该模拟输出到数字部分,形成一个完整的数字链



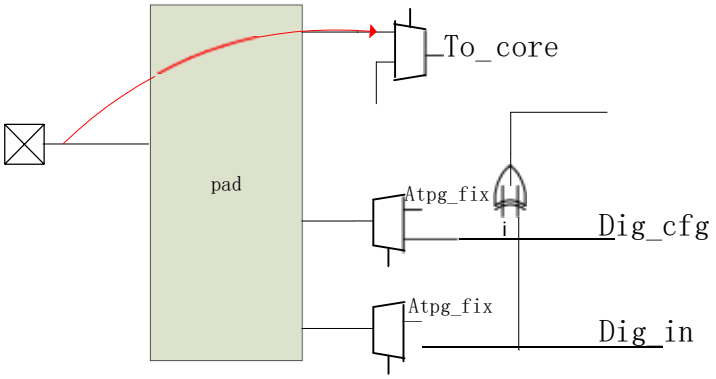
普通 PAD 在 ATPG 下的连接,即不作为atpg 的输入输出口

dig_cfg,指那些上下拉等配制,dig_in 表述需要输出的数字逻辑



PAD 作为ATPG　scan_in 的连接方式,即作为第一级寄存器的 D 段直接来自 PAD,其他控制管脚异或到其它管脚去

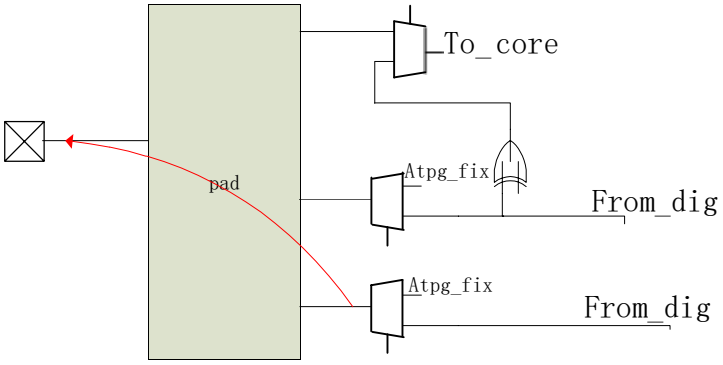
注意该 PAD 在此模式下肯定是要配置为输入模式的,dig_cfg,指那些上下拉等配制,dig_in 表述需要输出的数字逻辑



PAD 作为 ATPG　scan_out 的连接方式,即作为最后一级寄存器的 Q 段直接输出到 PAD

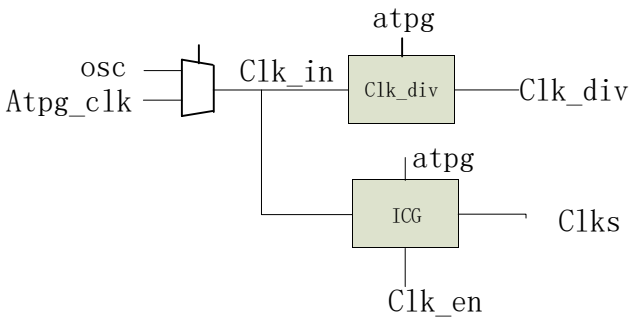
注意该 PAD 在此模式下肯定是要配置为输出模式的

当指定该 PAD 为 scan_out,综合工具会自动对 PAD 的 dig_in 信号做处理,但 PAD 的控制信号还是需要设计者做好



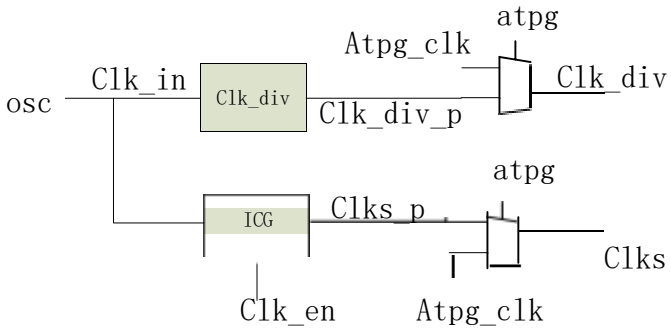
DFT 时钟处理

1).时钟源头做mux

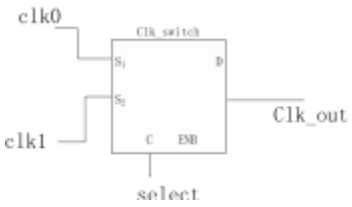


对于 clk_div 模块,当在 atpg 模式下,时钟直通,不分频,同样 ICG 模块中(工艺库会有专门模块),ATPG 模式下,时钟也是直通,实际上 clk_div 模块包含了 icg 模块,就是利用 icg 的直通来处理 clk_div 的直通功能

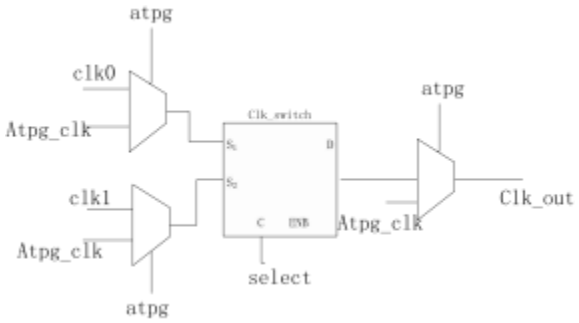
2).时钟终点做mux



3).mux 时钟前后做 atpg 模式下的mux

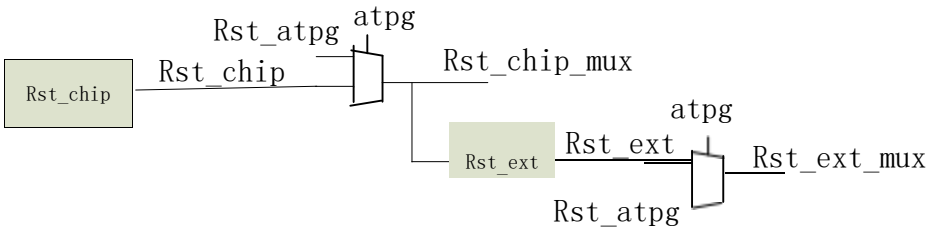


需要设计成如下时钟结果



方法 2 与时钟相关的寄存器无法做atpg

DFT 中 reset 信号的处理



每一级的复位信号都做一个mux

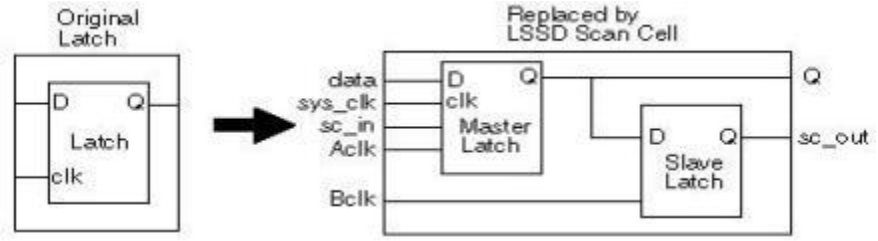
但注意不一定能在最后一级做一个 mux,否则其它复位信号相关的寄存器将无法串到扫描链上

- 内部 tri-state 总线

在扫描的过程中，总线上的驱动也许会同时驱动总线，引起竞争。解决这个问题可以通过添加一个译码逻辑，通过一个mux 去控制每一个 tri-state 驱动的输入使能，mux 用于在普通的操作信号和译码器的输出控制信号之间进行选择，在扫描模式下，选择译码器的输出控制信号，而译码器的输入由外部直接控制，这样，我们可控制 tri-state 驱动的开、和关。

- Latches

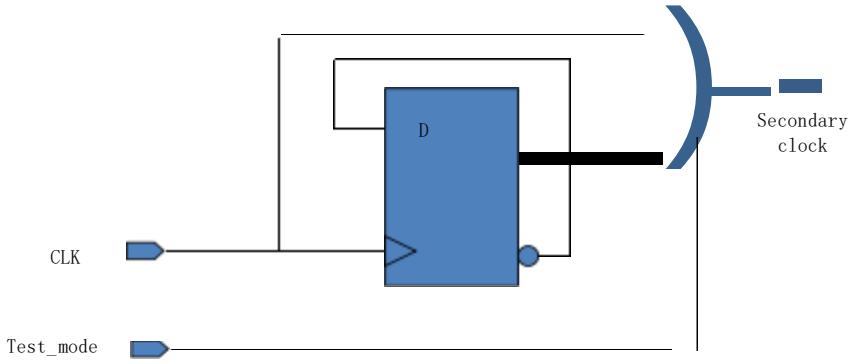
在设计中避免用Latches，因为测试很困难,为使 latch 在扫描模式下透明，需作如下替代。



- Gated or Generated Clocks

DFT 要求每一个 flop 的 clk 可控，所以对于 Gated or Generated Clocks 的逻辑电路用mux 进行旁路，如下图所示,分频逻辑被旁路，在这种情况下，分频逻辑无法扫描，通过如下命令通知 DC 将此 cell 从扫描链中排除:

dc_shell>set_scan_element false<list of cells or design>



■ 用时钟的 single Edge（实际中无需如此）

大部分设计都用时钟的 single Edge，但有些情况下，时钟的 rising 和 falling Edge 都要用到，这种情况在测试模式下可能产生时序问题，我们可为这部分电路加一个 mux,是整个设计在测试模式下用相同的时钟边沿。例如：

```
process(clk,test_mode)
begin
    if(test_mode= ‘1’ )then
        muxed_clk_output<=clk;
    else
        muxed_clk_output<=not(clk);
    endif
endprocess
```

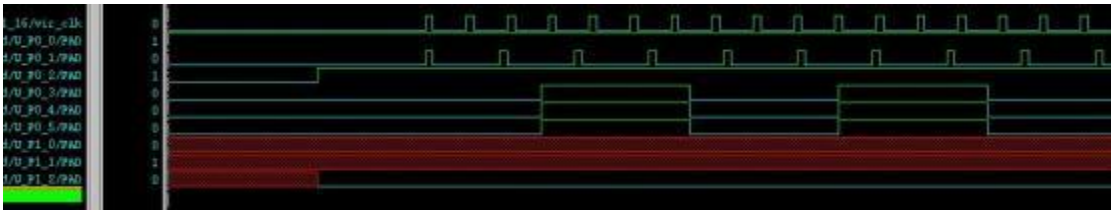
在 ATPG 中分 2 个阶段,一个是 shift 过程,此过程是把数据从端口 shift 到各个寄存器的 D 端,此过程 SE 信号有效,shift 的长度为此条褡的数目;然后是 capture 过程,此过程 SE 无效,把 D 端的组合逻辑结果送出给 Q;第一次 capture 输出的结果是最末端寄存器的结果,其它寄存器的结果也展现在 Q 端,即倒数第 2 个寄存器的 Q 端信号进入了组合逻辑到了最后一个寄存器的 D 端;当第二次来 capture 阶段时,倒数第二个寄存器 Q 端的值输出到了SO 端口;由此可见,在 DFT 中,最初出现的结果是链中最末尾的寄存器相关的结构.

2． 仿真注意事项

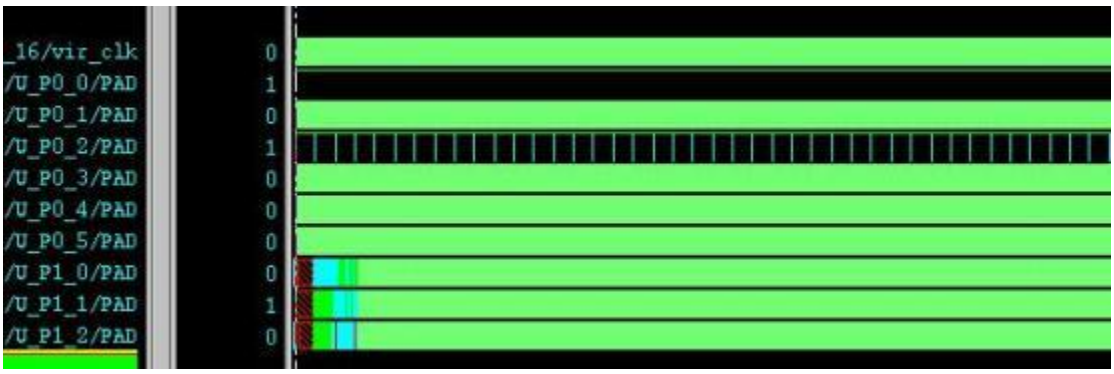
1. 仿真需要使用网表，但如果加了 sdf 文件，速度会非常慢，所以在仿真时可以不加 sdf 文件，而在 vcs 选项中-notimingcheck +delay_mode_zero,vcs 仿真 atpg 向量需要添加一个专门的选项+tetramax；在 nc 中还有一个选项+ncseq_udp_delay+这个选项会给每个寄存器一个固定的延时。
2. ***如果使用不带sdf文件仿真，最好使用nc加上 +ncseq_udp_delay+ 仿真，***当出现 x 态容易定位问题，如果用 vcs 不带延时，相当于一个死循环，所有的 x 态同时起来，无法确定哪个信号为x 态的源头，而且没有延时，在门级仿真+模型可能会导致死循环.
3. Scan 仿真有两种文件格式，一种为普通文件，向量在 tb 里面；另外一种为 stil格式，它是atpg 工具专门生成一个格式文件，用这个格式的向量只能使用vcs 来仿真，并且需要加 -P /xxx/eda/synopsys/syn_vI-2013.12-SP5/amd64/stildpv/lib/stildpv_vcs.tab /xxx/eda/synopsys/syn_vI-2013.12-SP5/amd64/stildpv/lib/libstildpv.a 选项---库文件，里面有读 stil 文件的函数
   目前还没有找到 ncverilog
4. 在机台做 scan 时，在波形文件中需要额外添加一个ref_clk，这个时钟于 scan 时钟相位相同，频率相同（scan 时钟在某个时刻会没有，机台在这个时刻转换时会错误，所以需要这个连续不段的 ref_clk---正常情况是每个信号变化都要采样数据，加了 ref_clk，这只需要监控 ref_clk，只要在它变化时采样数据就可以了，scan_clk 当作普通数据被采样就好了）--根据测试厂家要求是否需要
5. 在机台做测试时，在 se 信号为低时，此时不需要比较输出，但有些工具在此时也会有对比点，这是可能需要人工把此时输出的信号 force 成 x 态，因为工具在检测到 x 态会自动设置成不比较。

6. 在 atpg 测试中，一般会有一个复位信号，但在一些工具生成的 pattern 中却没有使用这个复位信号复位，利用第一个 pattern 给所有寄存器赋初始值，如下图：

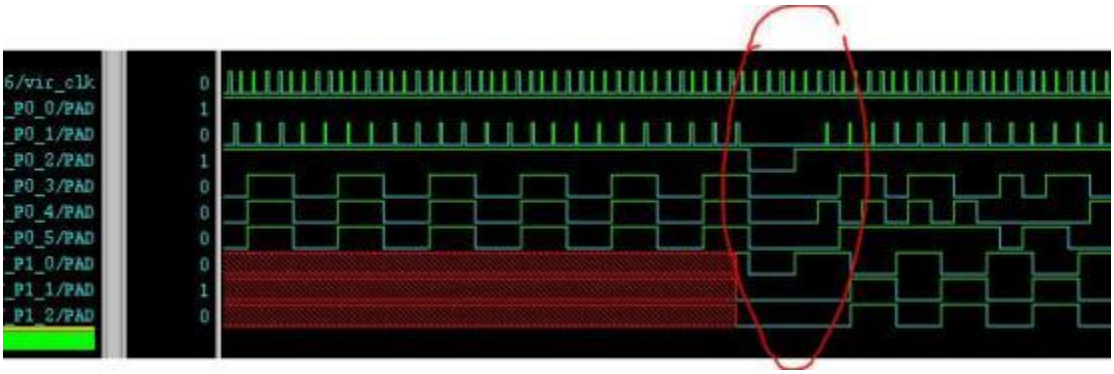
P00:scan_rst ;P01:scan_clk ;P02:scan_se



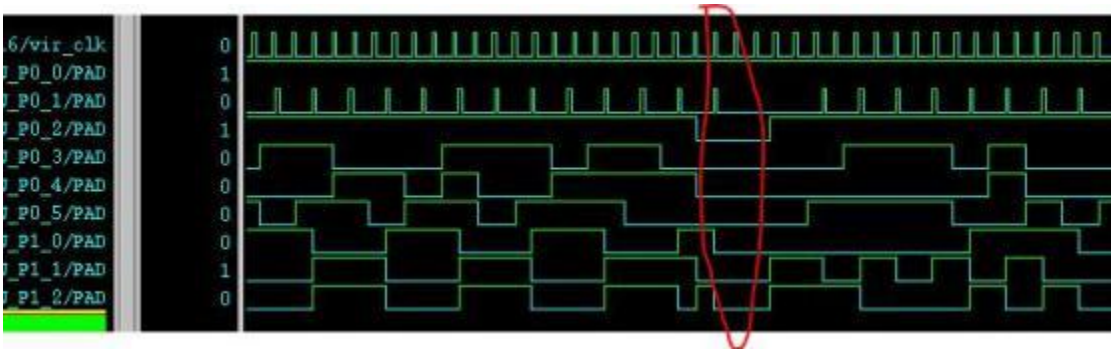
上图为仿真开头部分



上图为全局图，可以看到 P00 根本就没复位



上图为第一个 se 为低图，可以看到没有 scan_clk



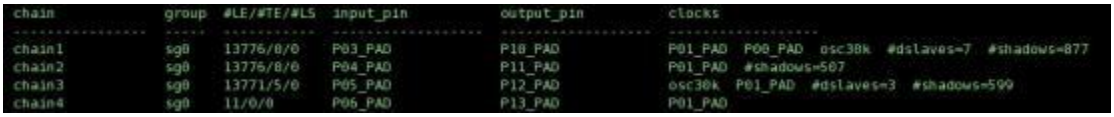
上图为其它 se 为低波形图，可以看到有一个 scan_clk，为的是把 scan 结果 shift 出来。

由此可见，第一个 pattern 起的作用就是给所有的寄存器给各初值，与 reset 的作用一致，只是 reset 逻辑由设计人员给出。而上面做法则有工具生成，更加灵活。所以当在第一个 pattern 的 capure 阶段报告错误可以忽略---它检测的应该是初始值。

4. 在 tmax 生成的结果中，会有一个叫.cell 的文件，里面会纪录详细的 chain 信息，如下：

.sum 文件记录了 chain 的整体信息

.chain 文件记录 chain 的数量等信息



.not_detected，.undetectable 文件记录了那些 cell 无法测试，如一些 pin 没使用，或者 fix 到某些特定值。

3． 生成 atpg pattern 步骤

3.1 屏蔽 atpg_reg 寄存器，使用 assign atpg_reg = 1’b1;注意生成向量后，仿真时需要改回去，通过正常方式进入 atpg 模式

3.2 配置好脚本 xxx.scr,详细见 tmax 篇

3.3 使用 add_pi_constraints 1 P07_PAD 语句对外部模拟信号做实际配置，主要是因为这些模拟信号会进入内部数字电路，如果没影响可以不做处理---注意 pad 复位

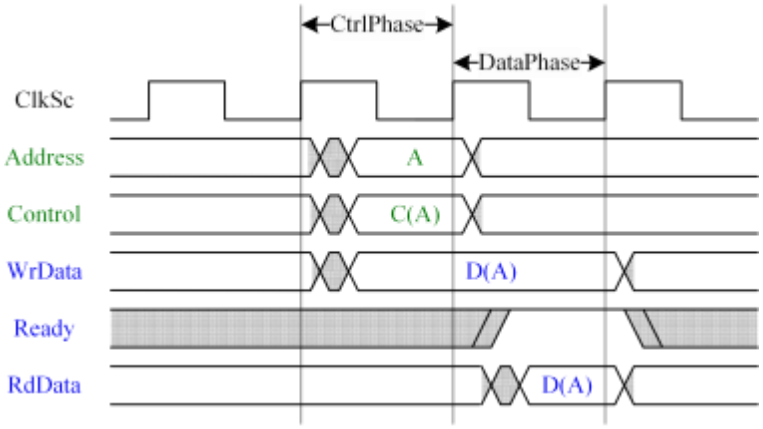
3.3 运行脚本 tmax -shell -64bit scr/xxx.scr | tee log/xxxe.log

总线篇

8051 总线

PBUS&XBUS:

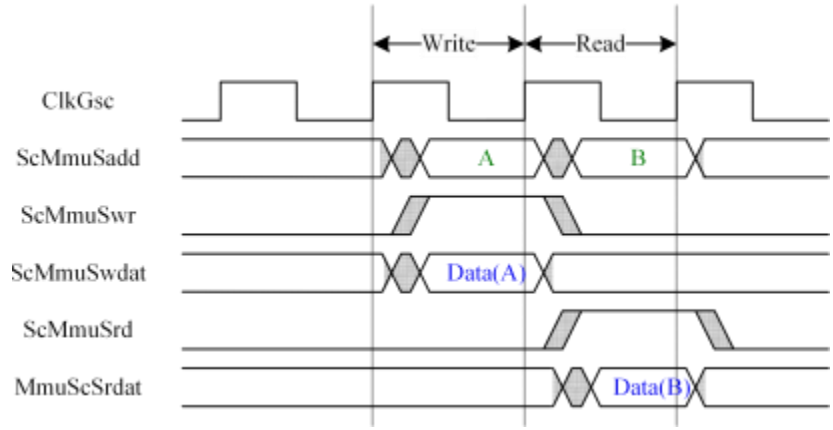
A, PBus 8 位访问时，总线 not ready 会引起停时钟等待：



注意:ready 信号是与返回数据同拍回来! ready 信号复位肯定是 1 的.

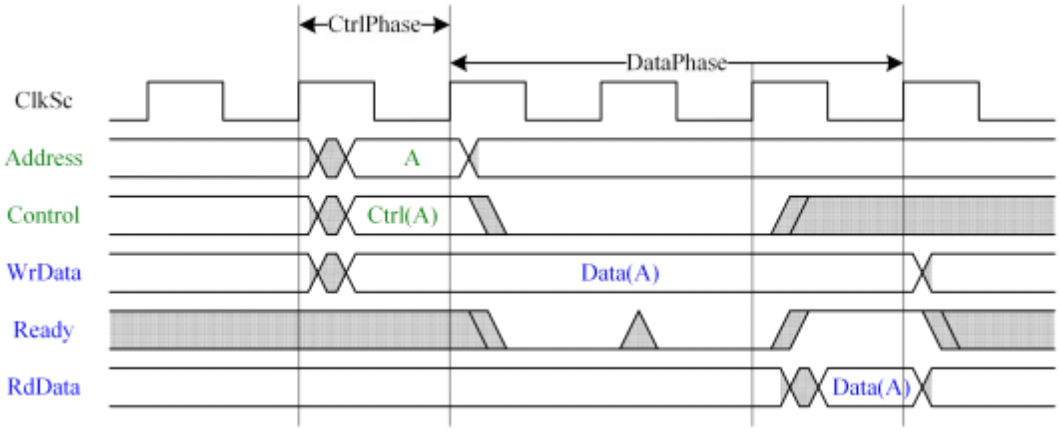
因为 host 是根据 ready 来读取数据.对于 pbus,一般只有 write 信号,所以非 wirte 即为 read,也就是 read 信号永远会比 ready 信号至少提前一拍;

对 SFR 总线的读写操作



8051sfr 读是当拍返回数据

Single with wait:



8051 总线无标准,需要根据 core 实际时序分析

AHB 总线

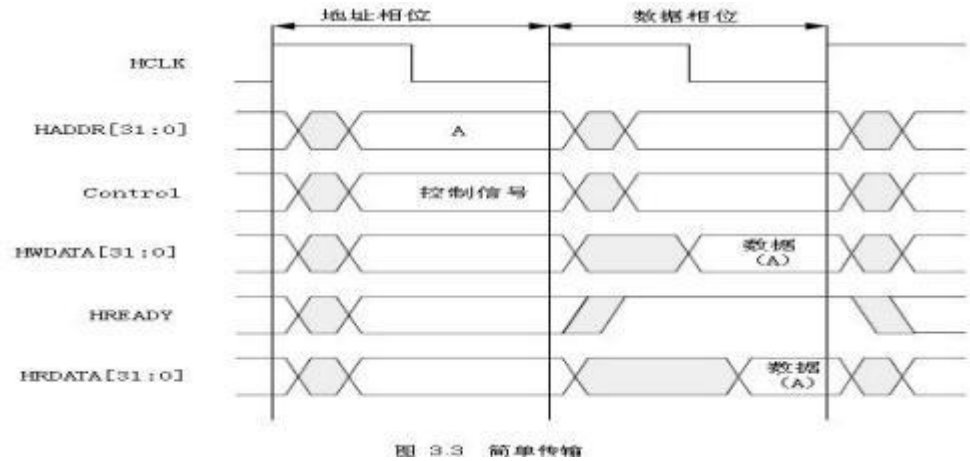


图 3.3 简单传输

注意:ready 信号(置 1)是与返回数据同拍回来!

当 host 发送完地址(ctrl phase),如果 slave 没能及时返回数据,那么此时就需要把 ready 拉低.如果是写行为,ready 信号需要拉低,因为host此时不需要读数据----如果此时 ready 为 1 会怎样?.ready 信号复位肯定是 1 的.因为host 是根据 ready 来读取数据.

Ahb 总线读寄存器和读 code 时序相同(数据相位/下一拍返回数据),与 8051 读 sfr 寄存器不同(当拍返回数据),所以在读数据时会对 rd 信号进行打拍处理.

注意 ahb 和apb 总线时序，ahb 的地址和写数据是分开的 2 拍，apb 写数据和地址同时输出；ahb 支持流水操作，考虑 ahb 写读写读这样连续操作。

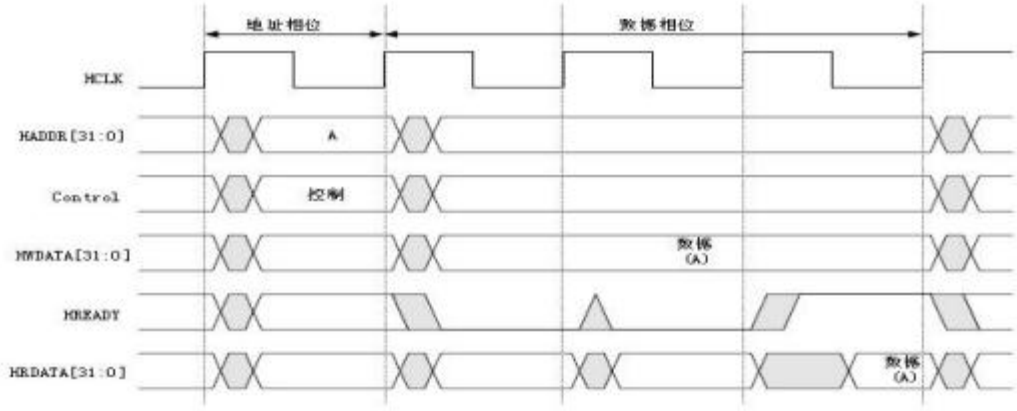


图 3.4 等待状态的传输

考虑对同一个地址进行背靠背操作,即对同一个地址先写后读,如果是操作 ram,那么 ready 信号必须要延时一拍(第三拍需要把数据读出,但实际上第三排才能把数据写入 sram,所以读出的数据并不时刚才写入的数据),这样速度就慢了,为了能够提速,一般在 sram 控制器中用寄存器做一个 buffer,第一次写 sram,并没有真正写入 sram 中,只有第二次有写的行为才会把上一次数据写入 sram 中,由于寄存器是当拍返回,所以能够快速背靠背操作,但要注意如果在写入过程中断电/复位情况(此时 buffer 会清 0)对系统有没有影响.

完整的 AMBA2.0 总线信号

Name	Source	Description
HCLK	Clock source	Bus Clock。All signal timings are related to the rising edge of HCLK。
HRESETn	Reset controller	active LOW。reset whole system。
HADDR[31:0]	Master	32-bit system bus。

HTRANS[1:0]	Master	current transfer type。
HWRITE	Master	HIGH : write transfer。LOW : read transfer。
HSIZE[2:0]	Master	the size of the transfer。
HBURST[2:0]	Master	Indicates if the transfer forms part of a burst。
HPROT[3:0]	Master	Implement some level of protection。
HWDATA[31:0]	Master	write data bus。
HSELx	Decoder	slave select signal。
HRDATA[31:0]	Slave	read data bus。
HREADY	Slave	High : transfer done。LOW : extending transfer。
HRESP[1:0]	Slave	transfer response。
HBUSREQx	Master	bus request。
HLOCKx	Master	Locked transfer。
HGRANTx	Arbiter	Bus grant signal。
HMASTER[3:0]	Arbiter	Indicate granted master number。
HMASTLOCK	Arbiter	Locked sequence。
HSPLITx[15:0]	Slave	Split completion request。

AHB 共支援八種的 transfer size，請參考表 3.2。

HSIZE[2]	HSIZE[1]	HSIZE[0]	Size	Description
0	0	0	8 bits	Byte
0	0	1	16 bits	Halfword
0	1	0	32 bits	Word
0	1	1	64 bits	-
1	0	0	128 bits	4-word line
1	0	1	256 bits	8-word line
1	1	0	512 bits	-
1	1	1	1024 bits	-

5.6 AHB Lite

在某些 AHB 系統中可能只有一個 master，此時系統中仍能使用 arbiter 就顯的不必要了，因此 ARM 在 AMBA 2.0 之外推出 AHB 的變化型- AHB Lite。與 AHB 的不同在於少了 arbiter 與 HGRANT/HBUSREQ 訊號機制，以及 Slave 不能再回應 RETRY/SPLIT response(因爲不可能有其他 master 去 access bus 了)。下圖 5.2 爲 AHB-Lit 系統的 block diagram。

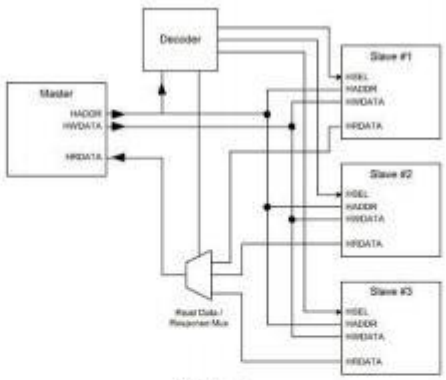


圖 5.2

若原本 master 就設計成 full-AHB 的 interface 與功能，則不需任何改變就可以在 AHB-Lite 與 full-AHB 中應用。相反的若原本是設計給 AHB-Lite 的 master 則需要簡單的 wrapper 才能在 full-AHB 中應用。

若 Slave 原本不具 SPLIT/RETRY 的能力，則在 full-AHB 與 AHB Lite 中皆可應用，但若是具有 SPLIT/RETRY 能力的 slave 則一樣要 wrapper 才能使用在 AHB Lite 中。

Ahb_lite 与 full_ahb 区别:

- 只有一個 master，且不需 Master to Slave Multiplexor。
- 沒有 arbiter。
- Master 沒有 HBUSREQ 訊號，若有則將訊號 floating。
- Master 沒有 HGRANT 訊號，如果有則 tied HIGH。
- Slave 不能產生 SPLIT/RETRY response。
- AHB-Lite 的 lock 訊號，其 timing 要跟 full-AHB 的 HMASTLOCK 的 timing 一樣。

在一般 slave 设计中(使用Ahb_lite 部分),忽略master 的 hbust,hmastlock 信号;返回的 reponse 为 0(表示 okey)

Reponse: 00:okey; 01:error; 10:retry; 11:split;

四個 status 中，OKAY 表示 transfer 成功的完成了，ERROR 表示 transfer 失敗了，失敗的可能原因譬如說企圖寫入 read-only 的 memory location，或讀寫根本不存在的 memory location 等。而 RETRY 和 SPLIT 則是用在當 slave 判斷目前的 transfer 將需要很多的 bus cycle 來完成，爲了避免因爲目前的 transfer 將 bus 一直 lock 住，而回應 RETRY/SPLIT response 給 master，表示目前的 transfer 尚未完成，master 需要重新發出相同的 transfer 再試一次，而此時 arbiter 就能將 bus release 給其他有需要的 master 使用。至於 Retry 和 Split 的差別在於 arbiter 的 master 優先權管理(Priority Scheme)：

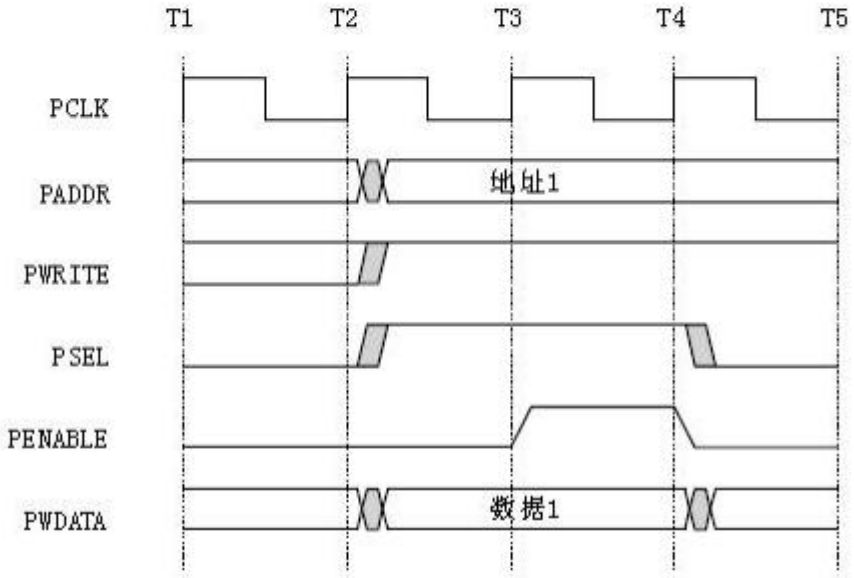
APB 总线

APB 桥既是 APB 总线上唯一的主模块，也是 AHB 系统总线上的从模块，。其主要功能是锁存来自AHB 系统总线的地址、数据和控制信号，并提供二级译码以产生 APB 外围设备的选择信号，从而实现 AHB 协议到 APB 协议的转换。

Apb 完整信号说明

Name	Description
PCLK	Bus clock，rising edge is used to time all transfers.
PRESETn	APB reset，active Low.
PADDR[31:0]	APB address bus.
PSELx	Indicates that the slave device is selected. There is a PSELx signal for each slave.
PENABLE	Indicates the second cycle of an APB transfer.
PWRITE	Transfer direction. High for write access, Low for read access.
PRDATA	Read data bus
PWDATA	Write data bus

图 5.3表示了基本的写传输。



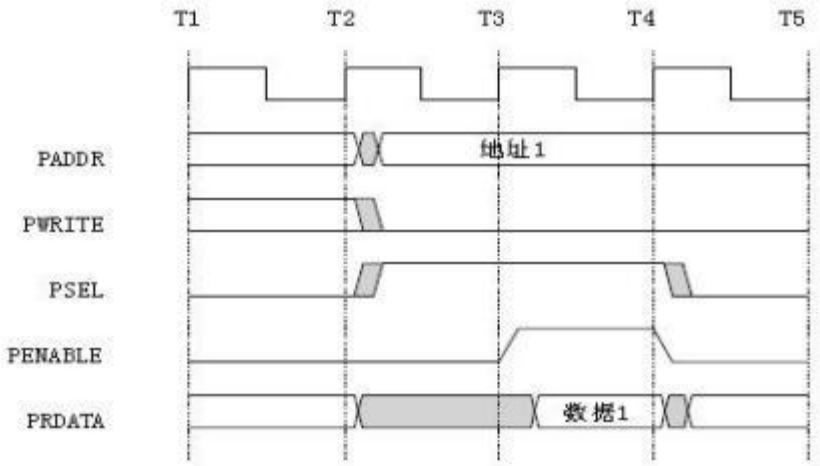
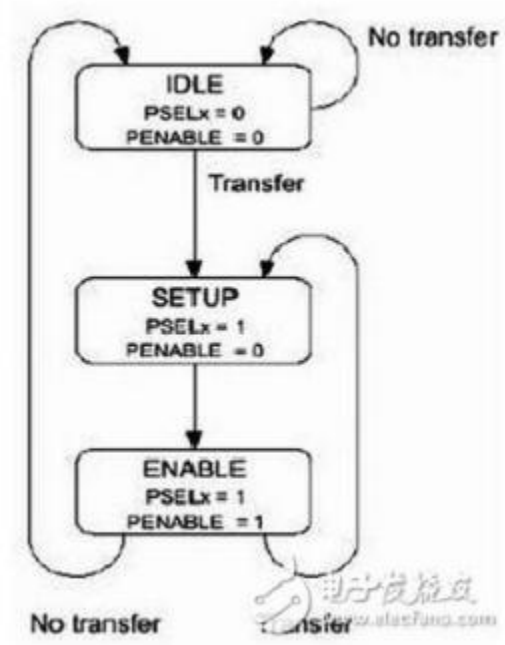


图 5.4 读传输

状态机如下：



APB->AHB,READ

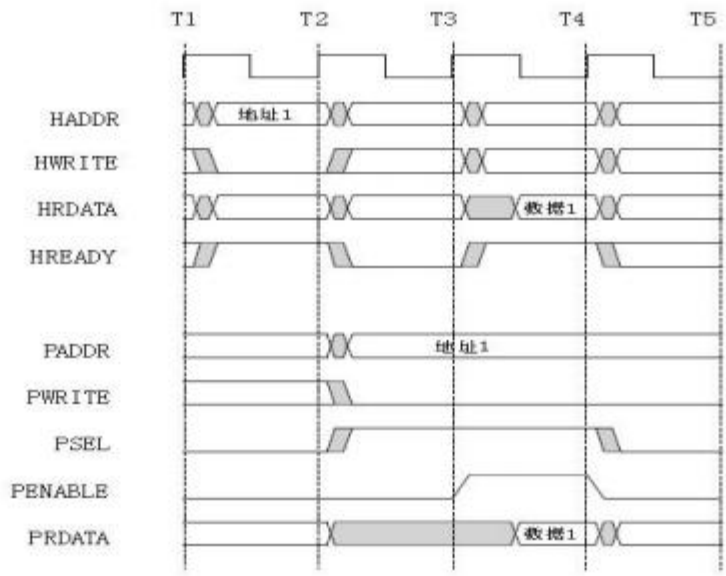


图 5.9 到 AHB 的读传输

传输在 AHB 上的时刻 T1 发起并且地址在 T2 被 APB 桥采样。如果传输是到外设总线那么该地址被广播并且产生合适的外设选择信号。在外设总线上的第一个周期成为 SETUP 周期，当 **PENABLE** 信号被断言后，该周期后跟随着 ENABLE 周期。

在ENABLE周期期间外设必须提供读数据。通常可能会将要读的数据直接发回到AHB，总线主机在ENABLE周期结束时的时钟上升沿采样该数据，在图 5.9中为时间T4。

图 5.11表示了一个写传输

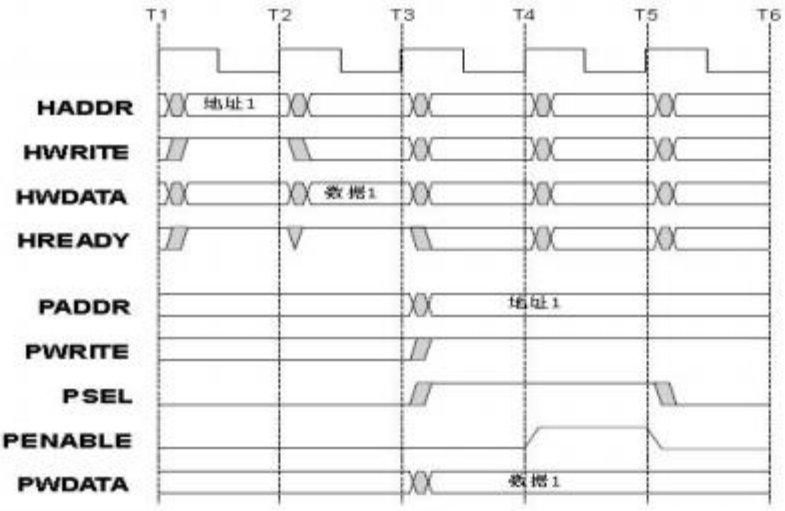


图 5.11 来自 AHB 的写传输

单个到 AHB 的写传输可以零等待状态发生。APB 桥负责采样传输的地址和数据并在 APB 的写传输期间保持这些数据值。

.控制信号

HTRANS[1:0]	传输类型	Description
00	IDLE	主设备占用总线，但没进行传输

		两次 burst 传输中间主设备可发 IDLE 此时就算 slave 被使能，也不会从总线上获取任何的数据信号。如果此时 salve 被选中，那么每一个 IDLE 周期 slave 都要通过 HRESP[1:0]返回一个 OKAY 响应
01	BUSY	主设备占用总线，但是在 burst 传输过程中还没有准备好进行下一次传输 一次 burst 传输中间主设备可发 BUSY 这时 slave 不会从总线上收取数据而是等待，并且通过 HRESP[1:0]返回一个 OKAY 响应。需要注意的是，这个 transfer 需要给出下一拍的地址和控制信号，尽管 slave 不会去采样。
10	NONSEQ	表明一次单个数据的传输或者一次 burst 传输的第一个数据 地址和控制信号与上一次传输无关
11	SEQ	burst 传输接下来的数据 地址和上一次传输的地址是相关的，这时总线上的控制信号应当与之前的保持一致，地址视情况递增或者回环。

HSIZE[2:0]	Size	Description
000	8 bits	Byte
001	16 bits	Halfword
010	32 bits	Word
011	64 bits	-
100	128 bits	4-word line
101	256 bits	8-word line
110	512 bits	-
111	1024 bits	-

. Burst 传输类型

burst 不能超过 1K 地址边界。

HBURST[2:0]	类型	Description
000	SINGLE	Single transfer
001	INCR	Incrementing burst of unspecified length
010	WRAP4	4-beat wrapping burst
011	INCR4	4-beat increment burst
100	WRAP8	8-beat wrapping burst
101	INCR8	8-beat increment burst
110	WRAP16	16-beat wrapping burst
111	INCR16	16-beat increment burst

Description	HPROT[3] cacheable	HPROT[2] bufferable	HPROT[1] privileged	HPROT[0] data/opcode
Opcode fetch	----------	----------	----------	0
Data access	----------	----------	----------	1

指示取的内容是作为 code 还是数据，当地址为 0x~0x1fff_ffff，该 bit 才可能变化

User access	-	-	0	-
Privileged access	-	-	1	-

在一般soc产品正常使用时，该bit一直为1,为特权模式

Not bufferable	-	0	-	-
Bufferable	-	1	-	-

Buffer 指的就是 xram 和 reg 空间，当地址为 0x2000_0000~0x7fff_ffff，该 bit 为 1

Not cacheable	0	-	-	-
Cacheable	1	-	-	-

Cache 含 data cache & code cache,只有操作 code 或 data 区，该 bit 为 1

● HPORT[3:0]：主设备数据传输保护控制信号，该信号为主设备发起的数据传输提供了附加的保护信息，其信号含义如表 3-1 所示：

表 3-1 HPORT 信号含义

Table3-1 HPORT signal encodings

HPORT[3]	HPORT[2]	HPORT[1]	HPORT[0]	功能
—	—	—	0	访问操作码
—	—	—	1	访问数据
—	—	0	—	用户模式
—	—	1	—	特权模式
—	0	—	—	不可缓冲
—	1	—	—	可缓冲
0	—	—	—	不可缓存
1	—	—	—	可缓存

AMBA 总线协议没有强制规定每个主设备都要提供精确的保护信息，除非主设备强制要求，否则从设备可以不设置 HPORT 端口；

3.3.5 Protection Control

HPROT[3:0]可以讓 master 提供額外的 protection information，譬如：指示 transfer 是 opcode fetch 或 data access 等。下表 3.3 爲 AHB 所支援的 protection。由於 AHB Spec.並未規定所有的 master 都要指示精確的 protection information，所以 slave 在設計時若非必須，盡量不要使用 HPROT 訊號。(若 master 沒有 protection transfer 的考慮， HPROT 可以 output 4'b0001 訊號)

Table 3-4 Protection signal encodings

HPROT[3] cacheable	HPROT[2] bufferable	HPROT[1] privileged	HPROT[0] data/opcode	Description
-	-	-	0	Opcode fetch
-	-	-	1	Data access
-	-	0	-	User access
-	-	1	-	Privileged access
-	0	-	-	Not bufferable
-	1	-	-	Bufferable
0	-	-	-	Not cacheable
1	-	-	-	Cacheable

表 3.3

在除了曾经带有 mpu 保护的项目中使用过 hport外,其它的项目均未使用该信号.

```
instr  = ~HPROT[0]&HPROT[1];
rd     =  HPROT[0]&HPROT[1]&read;
```

wr = HPROT[0]&HPROT[1]&write;

HPROT[3:2] 指示指令或数据是从哪里来或读到哪里去，

privilegeded 一般与 cacheable 使用，所以 HPROT 出现的值一般为 a,b,f;

Hport 值常见为,0x7,0xa.0xb,0xf;

Cacheable：跟数据和代码相关

0x7: 表示访问的是周边设备 slave 空间(0x4000_0000 ~ 0x4fff_ffff)

0xa: 表示访问的是代码空间，作为代码处理 (0x0000_0000 ~ 0x1fff_ffff)

0xb: 表示访问的是代码空间，作为数据处理 (0x0000_0000 ~ 0x1fff_ffff)

0xf: 表示访问的是数据 sram 空间（0x2000_0000 ~ 0x2fff_ffff）

下面是具体 core 对应的 hport

```
assign      hprot_o          = { ahb_c, ahb_b, 1'b1, ahb_prot };
```

Ahb_port 用来表示访问数据还是 code;

```
// ------------------------------------------------------------
// Generate AHB cacheable and bufferable sideband
// ------------------------------------------------------------

// the ARMv6-M defined memory map is mapped to HPROT as
// follows:

//   PREFIX     ARMv6-M     CACHEABLE   BUFFERABLE
//  ---------- ----------- ----------- -----------
//  0/1 000x  Normal-WT       yes          no
//  2/3 001x  Normal-WBWA     yes          yes
//  4/5 010x  Device          no           yes
//  6/7 011x  Normal-WBWA     yes          yes
//  8/9 100x  Normal-WT       yes          no
//  A/B 101x  Device          no           yes
//  C/D 110x  Device          no           yes
//  E   1110  SO (PPB)        n/a          n/a
//  F   1111  Device          no           yes

wire        ahb_c = ( ~ahb_addr[31] &  ahb_addr[29] |
                      ~ahb_addr[30] & ~ahb_addr[29] );

wire        ahb_b = ahb_addr[30] | ahb_addr[29];
```

CODE 描述与表格似乎不完全一致,请注意.

HPROT[1]如果没 usr 模式一直为 1, HPROT[0]指示内容为 code 还是 data,

HPROT[3:2]在没有 mpu 的结构时,

其内容如下(M0/M0+文档内容):

Table 4-7 shows the **HPROT[3:2]** memory types.

Table 4-7 Memory types for HPROT[3:2]

Address	Memory type	HPROT[3:2]	Recommended usage
0xF0000000 - 0xFFFFFFFF	Device XN[a]	01	CoreSight ROM tables[b]
0xE0000000 - 0xEFFFFFFF	Reserved	-	System control space
0xA0000000 - 0xDFFFFFFF	Device XN	01	Peripherals
0x80000000 - 0x9FFFFFFF	Normal WT[c]	10	Off chip RAM
0x60000000 - 0x7FFFFFFF	Normal WBWA[d]	11	Off chip RAM
0x40000000 - 0x5FFFFFFF	Device XN	01	Peripherals
0x20000000 - 0x3FFFFFFF	Normal WBWA	11	On chip RAM
0x00000000 - 0x1FFFFFFF	Normal WT[e]	10	ROM or flash

a. XN means execute-never.
b. See *CoreSight ROM table base address* on page 2-8 for more information.
c. WT means write-through.
d. WBWA means write-back-write-allocate.
e. WT means write-through.

AHB-Lite interface transaction types

Table 4-8 shows the Cortex-M0+ processor generated AHB-Lite interface transaction types.

Table 4-8 AHB-Lite interface transaction types

Transaction	HTRANS[1:0]	HPROT[0]	HMASTER[a]	HSIZE[2:0]
Bus idle	00	X	X	0XX
Core instruction fetch[b]	10	0	0	001
				010
Core word load/store	10	1	0	010
Core half-word load/store	10	1	0	001
Core byte load/store	10	1	0	000
Debug word read/write	10	1	1	010
Debug half-word read/write	10	1	1	001
Debug byte read/write	10	1	1	000

a. **HMASTER** can be used by peripherals to distinguish between accesses from the processor core and accesses from the debugger.
b. If you implement the processor with HWF=1, the value of **HSIZE** for core instruction fetches is always b001, indicating 16-bit. If HWF=0, then **HSIZE** uses both values of b010 and b001.

WT,WBWA 含义如下

下面是 M3 的属性

1. 代码区(0x0000_0000- 0x1FFF_FFFF)。该区是可以执行指令的,缓存属性为 WT("写通", Write Through),即不可以缓存。此区亦可写数据。在此区上的数据操作是通过数据总线接口的(读数据使用 D-Code,写数据使用 System),且在此区上的写操作是缓冲的。
2. SRAM 区(0x2000_0000 – 0x3FFF_FFFF)。此区用于片内 SRAM,写操作是缓冲的,并且可以选择 WB-WA(Write Back, Write Allocated)缓存属性。此区亦可以执行指令,以允许把代码拷贝到内存中执行——常用于固件升级等维护工作。
3. 片上外设区(0x4000_0000 – 0x5FFF_FFFF)。该区用于片上外设,因此是不可缓存的,也不可以在此区执行指令(这也称为 eXecute Never, XN。ARM 的参考手册大量使用此术语)。
4. 外部 RAM 区的前半段(0x6000_0000 - 0x7FFF_FFFF)。该区用于片外 RAM,可缓存(缓存属性为 WB-WA),并且可以执行指令。
5. 外部 RAM 区的后半段(0x8000_0000 – 0x9FFF_FFFF)。除了不可缓存(WT)外,同前半段。
6. 外部外设区的前半段(0xA000_0000 – 0xBFFF_FFFF)。用于片外外设的寄存器,也用于多核系统中的共享内存(需要严格按顺序操作,即不可缓冲)。该区也是个不可执行区。
7. 外部外设区的后半段(0xC000_0000 – 0xDFFF_FFFF)。目前与前半段的功能完全一致。
8. 系统区(0xE000_0000 – 0xFFFF_FFFF)。此区是私有外设和供应商指定功能区。此区不可执行代码。系统区涉及到很多关键部位,因此访问都是严格序列化的(不可缓存,不可缓冲)。而供应商指定功能区则是可以缓存和缓冲的。

写通,写回,与写时申请

- 写回(Write Back): 写入的数据先逗留在缓存中,待到必要时再落实到最终目的地,这也是 cache 的最基本职能,用于改善数据传送的效率。
- 写通(Write Through): 写操作"穿透"中途的缓存,直接落入最终的目的地中。可见,写通操作架空了 cache,但它使写操作的结果立即生效。这常用于和片上外设或其它处理器共享的内存中,如显卡的显存,片上外设寄存器,以及双核系统中的共享内存。写通操作和 C 中的"volatile"有密切的联系。
- 写时申请(Write Allocate): 俺也不太清楚~

Bufferable 意思是说不必把结果写进最终目的地而只是写进中间某个buffer 就可以回 response,而 non-bufferable 则是把结果要写进最终的目的地才能回 response。

non-bufferable 一般用于 device 的读写,也就是 SOC 里讲的 MMR。

bufferable 与否区别在于何时回 response。

Cacheable 说的是进行读写的时候要去 cache 里查看(noncacheable 就是不用去 cache 里查看,直接送往 memory)。

WB WT 及 WA 的区别

Write-back:WB; Write-through: WT;write allocate:WA

一、CPU 读 Cache

1. Read through，即直接从内存中读取数据；
2. Read allocate，先把数据读取到 Cache 中，再从 Cache 中读数据。

二、CPU 写 Cache

1. 若 hit 命中，有两种处理方式：

Write-through: write is done synchronously both to the cache and to the backing store。Write-through（直写模式）在数据更新时，把数据同时写入 Cache 和后端存储。此模式的优点是操作简单；缺点是因为数据修改需要同时写入存储，数据写入速度较慢。

Write-back (also called *write-behind*): initially, writing is done only to the cache. The write to the backing store is postponed until the cache blocks containing the data are about to be modified/replaced by new content。Write-back（回写模式）在数据更新时只写入缓存 Cache。只在数据被替换出缓存时，被修改的缓存数据才会被写到后端存储（即先把数据写到Cache 中，再通过 flush 方式写入到内存中）。此模式的优点是数据写入速度快，因为不需要写存储；缺点是一旦更新后的数据未被写入存储时出现系统掉电的情况，数据将无法找回。

2. 若 miss，有两种处理方式：

Write allocate (also called *fetch on write*): data at the missed-write location is loaded to cache, followed by a write-hit operation. In this approach, write misses are similar to read misses.。Write allocate：先把要写的数据载入到 Cache 中，写 Cache，然后再通过 flush 方式写入到内存中； 写缺失操作与读缺失操作类似。

No-write allocate (also called *write-no-allocate* or *write around*): data at the missed-write location is not loaded to cache, and is written directly to the backing store. In this approach, only the reads are being cached。No write allocate：并不将写入位置读入缓存，直接把要写的数据写入到内存中。这种方式下，只有读操作会被缓存。

M0 核存储空间影射

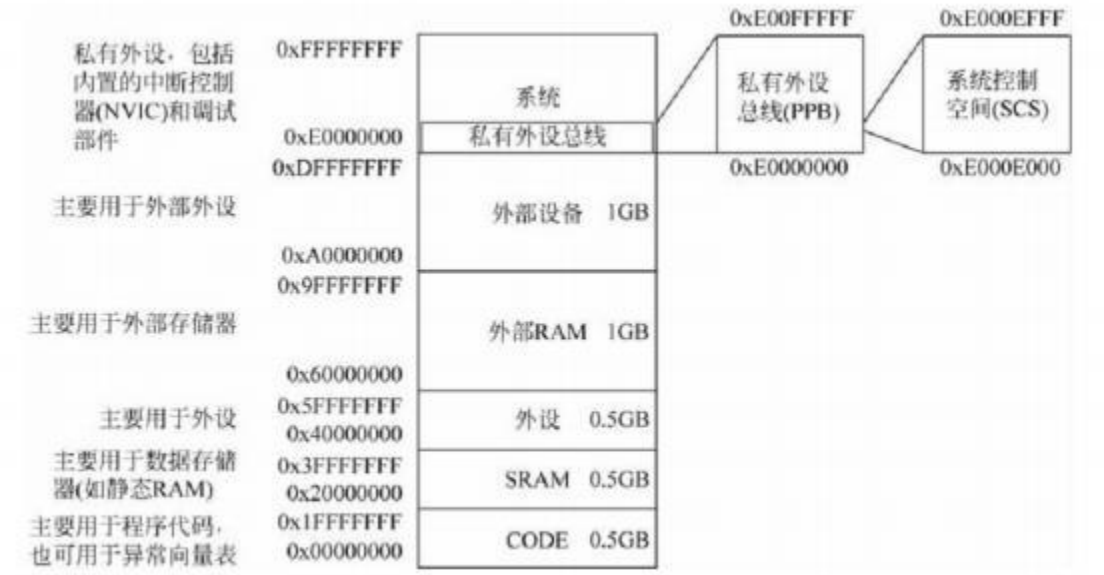


图 3.8 存储器映射

空间特性

Table 2-10 Memory region shareability and cache policies

Address range	Memory region	Memory type[a]	Shareability[a]	Cache policy[b]
0x00000000- 0x1FFFFFFF	Code	Normal	-	WT
0x20000000- 0x3FFFFFFF	SRAM	Normal	-	WBWA
0x40000000- 0x5FFFFFFF	Peripheral	Device	-	-

Address range	Memory region	Memory type[a]	Shareability[a]	Cache policy[b]
0x60000000- 0x7FFFFFFF	External RAM	Normal	-	WBWA
0x80000000- 0x9FFFFFFF				WT
0xA0000000- 0xBFFFFFFF	External device	Device	Shareable	-
0xC0000000- 0xDFFFFFFF			Non-shareable	
0xE0000000- 0xE00FFFFF	Private Peripheral Bus	Strongly- ordered	Shareable	-
0xE0100000- 0xFFFFFFFF	Device	Device	-	-

a. See *Memory regions, types and attributes* on page 2-13 for more information.
b. WT = Write through, no write allocate. WBWA = Write back, write allocate. See the *Glossary* for more information.

The memory types are:

Normal	The processor can re-order transactions for efficiency, or perform speculative reads.
Device	The processor preserves transaction order relative to other transactions to Device or Strongly-ordered memory.
Strongly-ordered	The processor preserves transaction order relative to all other transactions.

AHB 总线和 ASB 总线有下列不同的特点：

● AHB 总线支持分开处理。有很长响应延迟的从机在准备传输的数据时让出总线从事其它传送操作;--所以慢设备挂在apb 总线上，在等待 apb 返回数据时，可以让出总线。

● 使用单一时钟沿控制所有操作，有利于综合和设计验证(通过使用静态时序分析和其他相似工具);

● 使用中心多路器总线方案而不是三态驱动的双向总线;

● 支持更宽的64 位或 128 位数据总线配置。

ASP

ASP 是通用系统总线，是一种微处理器和系统外设的高性能互连，主要特征如下：

● 多控制器;

● 突发数据传输方式;

● 流水线传送;

● 32-128 位总线宽度;

● 包含一种访问保护机制，用来区别特权访问和无特权访问模式，或指令和数据提取等;

● 双向数据总线;

● 提供为较慢设备使用而扼制数据流的机制;

● 仲裁支持 REQ 、GNT 和 LOCK。

APB

APB 是外围互联总线，重点是最小功耗和易于使用，主要特征如下：

- 低性能、低功率外围总线;
- 单控制器;
- 非常简单，只有 4 个控制信号(加上时钟和复位);
- 32 位地址空间;
- 多达 32 位数据总线;
- 分开读和写数据总线。

AHB 总线实现了简单的基于burst 的传输，数据总线带宽可配置 32-1024bit。可以实现简单的 fixed pipeline 在 address/control phase 和 data phase 之间。

典型的 AHB 的 slave 包括：internal memory device，external memory interfaces，high bandwidth peripheral。

AHB 必须是对齐传输，word,addr[1:0]为 0，half-word,addr[0]为 0。包括 wrap burst 传输，也必须是对齐的。

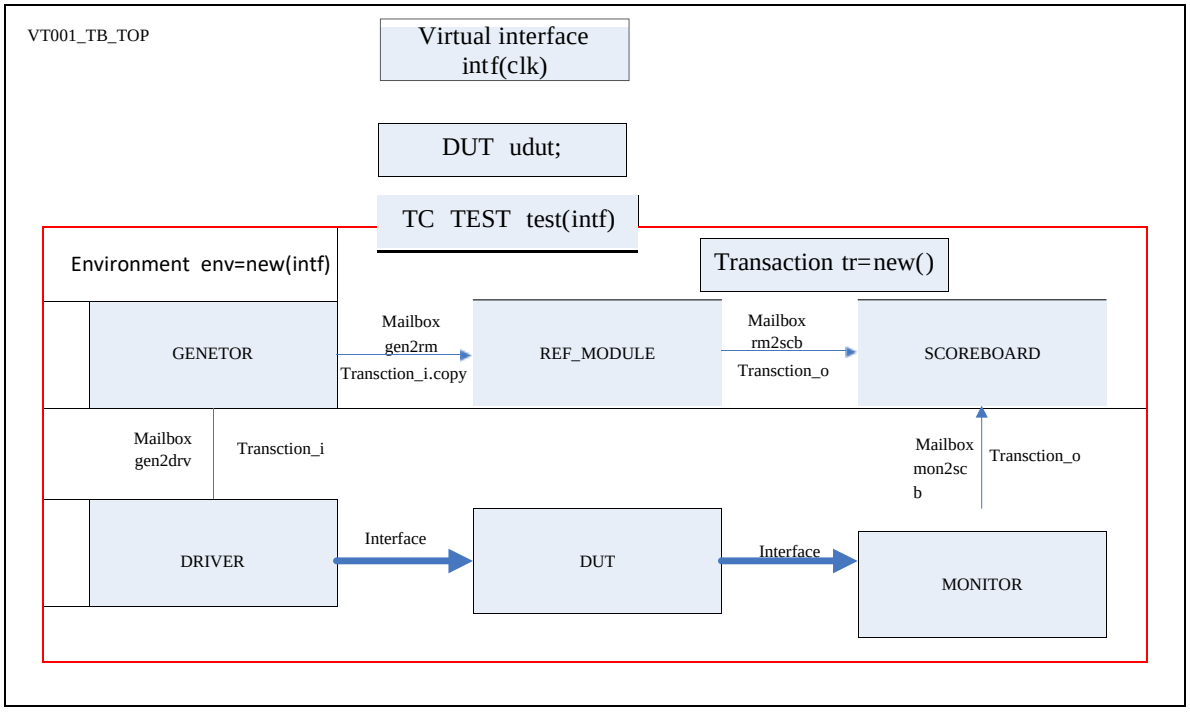
AHB-Lite 协议是整个 AHB 协议的子集，只支持一个总线主设备，不需要总线仲裁器及相应的总线请求/授权协议， 不支持 Retry和 Split 响应。

AHB-Lite 的典型应用结构是一个 AHB-Lite master，一些 AHB-Lite slaves。slave 由 APB 总线实现，HSEL 信号由一个 address decoder 产生。从 slave 到 master 的 rdata，response 由一个 slave-to-master multiplexor 通过选址送给 master。

验证篇

SVTB

Svtb 构架




```
program automatic test;//testcase
  Dut u_dut();
  Virtual interface intf(clk);
  Environment env(clk);
  Initial begin
      Env = new();
      Env.gen_cfg ();
      Env.build();
      Env.run();
      Env.wrap_up ();
  end
endprogram
```

interface

nterface：SV 中新定义的接口方式，用来简化接口连接，使用时注意在 module 或 program 之外定义 interface，然后通过’include 来添加进工程。

```
 interface  arb_if(input bit clk);
//clk 信号，一般单独拿出来
  logic [1:0]grant, request;
//只定义信号类型。类型在不同的modport 中分别定义。
    logic rst;
    clocking cb @(posedge clk);
  //定义时钟块，其中的信号都是 clk 上升沿有效
      output request;
      input  grant;
    enclocking
  modport TEST (clocking cb, output rst);
  //直接引用 clocking, 在定义 rst 的方向---对于 TEST 来说是 OUTPUT
  modport DUT (input request, rst, output grant);  //定义各个信号的方向---是指对于 DUT 的方向
endinterface
```

Program：主要是为了在逻辑和仿真时间上，区分开 RTL 与验证平台。在 SV 搭建的验证环境中，testcase 一般就定义一个program 来开始执行。program 中不能使用 always，因为 program 相比较来说，与 C 语言更靠近一些。所以多用 initial 就可以。program 中的仿真时间与 RTL 中的是有区别的，SV 将同一仿真时刻分为四个区域，Active(design), Observed(assertion), Reactive(testbench), Postponed(sample)。相当于在原 verilog 的基础上又为program 增加了一个执行区间，一个采样区间。所以 clk 的定义不能放在 program 中。当 program 中的 initial 结束时，SV 会调用$finish 完成仿真

SV 搭建验证环境（转）

1)首先定义纯虚类 Sv_object，主要实现下边两个 function：

定义 local static 变量 nextobjectID;虚方法 virtual function void copy (St_object that, CloneType clone_type = DEEP);在基类中复制都可以通过 super.copy 来迭代 copy。

2)在 Sv_object 的基础上，实现对 mailbox 和 event 的封装。产生两个基类 Sv_mailbox_wrapper 与 Sv_event_wrapper。其中 event 多用在进程的控制(wait(event), event(trigger)等形式放在 function 内)，mailbox 主要用在 transaction 的传递。

3)在 Sv_object 的基础上，建立一个 Sv_testobject 的纯虚类，来作为验证环境中各个组件的基类，在该基类中实现 4 个 funciton，

start,stop,main,reset。利用 event 将这几个 funciton 联系起来。这样便有了一个基本的控制流程。

4)Transaction 类的建立，这个类不算验证环境中的构件，所以用 Sv_object 来 extends。不过同样要在 Sv_transaction 这个类中建立基本的流程，比如： generate,execute,check。分别作用在 generator 段，driver,monitor,DUT 段，check 段。同样通过 event 来实现，可以实现三个 function 来控制这三个 event，在上层调用实现 callback。这都是针对一个 tranaction 来说的流程。

5)Generator 基类的建立，在 Sv_testobject 的基础上 extends，要重新定义一个 generate 的 function，来让 testcase 中的 generator 来重定义，完成 randomized 和一些例外的 constrain，或其他的回调函数。

6)Driver 基类的建立，在 Sv_testobject 的基础上 extends，因为需要加入额外额 reset 信号，所以在 Sv_testobject 的基础上，加入另外两个优先级更高的 function，这里的优先级也就是通过 event 阻塞来实现的，wait_for_reset, reset。

7)Monitor 基类的建立，在 Sv_testobject 的基础上 extends，主要是在 Sv_testobject 的基础上定义一个 receive_trans 的 function，来接收从DUT 发出的信号。

8)Check 基类的建立，在 Sv_testobject 的基础上 extends，这个类与具体的 test 相关较大，基类中只需定义一些 event 来表示 start 和 done。

9)Test 基类的建立，在 Sv_testobject 的基础上 extends，这个类主要是完成流程的控制，可以分为 GenConfig (), ConfigDut, Build(), start(), stop, report, 总体先配置好环境，在配置 DUT，而后 clock, reset, 信号进来完成初始化，而后 start 验证平台的各个模块开始工作，在这个阶段再根据 Transaction 的三个 function 来细分控制。

10)test_trans 类的建立，在 Transaction 的基础上 extends，主要是针对一些 DUT 中的信号的 randmize 定义和 constrain，还需要建立一个 param_trans 的 function，主要是传递 transaction，这样可以在最顶层直接例化 function 之后将相应的 mailbox 联系起来。

11)test_generator 类的建立，在 Generator 基类的基础上 extends，主要是重定义 generator 这个 function，也就是完成 randomized 和一些例外的 constrain，或其他的回调函数。同样建立一个 param_trans 的 function，接收来自 test_trans 的 transaction。顶层例化，mailbox 连接。

12)test_driver 类的建立，在 Driver 基类的基础上 extends，主要加入 reset 的控制，重定义 driver 基类中的 wait_for_reset 和 reset 这两个 function。同样建立一个 param_trans 的 function，接收来自 test_generator 的 transaction 和 clk, reset, DUT 的 interface 虚接口。顶层例化，mailbox 连接。

13)test_Monitor 基类的建立，在 Monitor 基类的基础上 extends，主要是收集 DUT 的部分信号。同样建立一个 param_trans 的 function，可以将收集到的信号组成一个 transaction 发送给 test_check，还可以引入一些 clk, reset 的 interface 虚接口。顶层例化，mailbox 连接。

14)test_check 类的建立，在 check 基类的基础上 extends，定义一些很细节的 function，来根据收到的 transaction 与预想的行为进行比对。同样建立 param_trans 的 function，来接受 Monitor 发来的 transaction。顶层例化，mailbox 连接。

15)testcase 类的建立，在 test 基类的基础上 extends，主要是配置验证环境，重定义 GenConfig (), ConfigDut 这两个 function，并且在 build 这个 function 中，例化各个组件的 param_trans 这个 function，来将各个 mailbox 连接起来。

16)定义一个 Testbench_top 的 module，主要例化 DUT_module, DUT_if, clk_if, reset_if 等 RTL 文件。

16)定义一个 program，例化 testcase.sv, top。在 initial 中通过调用一个 testcase 中的 GenConfig ()来开始，整个环境开始执行。

最后还需要加入 cover_file.sv 和 assert_file.sv，这两个文件不需要很复杂的继承，直接按 testcase 定义就好，一般还需要相应的 interface 信号，来与 DUT 进行连接。cover_file_if, assert_file_if。直接通过 bind top 绑定到顶层就好。需要修改覆盖率和 SVA 的时候可以直接修改，他们不影响整个验证环境。额外定义 reset 和 clock 的模块，放在module 中。它们的 cover_file 和 assert_file，也最好单独验证。DUT 中 register 的值得获取，通过 Apb 总线，在 Driver 中进行配置。register 值得获得,可以通过monitor 来得到，其中 test_regif 是通过 bind xxxx 来定位到 DUT 中寄存器所在的那一级 module 。

当然这只是一些最基本的模块，还有 log 文件在这个环境中怎么设计，脚本的问题等。

在 SV 中通过 DPI 来介入 C/C++的函数，

声明：import “DPI-C” function int factorial(input bit i)

```
program automatic test;
initial begin
     for(int  i=1; i<=10; i++)
      $display ("%0d = %0d", i, factorial(i));
  end
```

endprogram

input 表示从 SV 到 C，还支持 output/inout，但是不支持 ref。

对于 C++的引用，SV 只支持静态的方法，所以不能调用 class 例化的对象中的 function

SV 中 OPP 特点

OOP：Object-Oriented Programming，有两点个人认为适合验证环境的搭建：

1)Property (变量)和Method(function/task)的封装,其实是BFM模型更方便的应用。

2)继承(Inheritance)与合成(Composition)非常适合一个架构的搭建。

在 SV 中，类可以定义在 program, module, package 中，但是一般一个类或几个相关的类会单独写在一个文件中。最终在program 中调用。而且在 SV 中，所有成员默认都是 public 类型的，除非显示的用 local/protected 来声明。实际上，SV class 中的变量应该尽可能的用 public 并且 rand，来增加可控性。

local/protected 都可以针对 property 和 method 来说明，local 表示 member 只对该 class 的对象可见，extend 出的 subclass 也是不可见的。 但是该 class 中 non-local 的 method 调用了 local 的 method 或者 property，inherit 之后的 method 也是可以正常执行的。 protected 表示 member 对该 class 和它的 subclass 都是可以见的。 对外部的大环境不可见。

基类的指针可以直接指向扩展类的对象，但是只能调用扩展类中基类的那一部分，

基类的指针也可以直接转换为扩展类的指针，通过$cast 函数，此时的基类指针指向的已经是扩展类对象，

sv 语法

1. 验证中 program 与用 module 的区别

在 program 中，当所有的进程结束后，自动会 finish 退出仿真；而在module 中必须用 finish 结束仿真；

2. Primitives 原语 介绍

Or,buf,nand;bufif0(buf if 0,条件缓冲，0 有效，位置顺序

buf(out,in,enable));

bufif1;notif0,notif1,

MOS 开关：pmos(out,in,enable),低电平有效;nmos(out,in,enable),高电平有效;rpmos;

上下拉：pullup (out);pulldown(out)

双向开关：tran(inout,inout2); tran(inout,inout2)

双向可控开关：

tranif0(inout,inout2,ctrl); tranif1(inout,inout2,ctrl);

rtranif0(inout,inout2,ctrl); rtranif1(inout,inout2,ctrl)

上述可控型的结构，当不导通时，输出为Z 态

门延时说明： 上升延时指!1->1 的延时; 下降延时指!0->0 的延时;截至延时指的 是 0/1->Z(关闭)的延时，只有三态门会有 3 个延时，其它的都是 2 个延时参数；如果只写了一个延时，则表示上升/下降/截至延时相同；每个延时又分min,type,max；所以在 vcs 选项中需要有

maxdelays,mindelays,typedelays;

`default_nettype nettype; `default_nettype 用来定义没声明但又用到线网的类型，如，`default_nettype wand 表示后门出现的未加说明的线网都是线与线网，直到遇到另外一个`default_nettype 或 `resetall 编译指令出现

3. V 与 SV 最大的区别:

在于 SV 是面向对象编程（OOP）：类的封装（CLASS），类的扩展（extends），多态（重载功能）（利用虚方法实现，virtual task/function,多个子程序使用一个共同的名字的现象叫做多态“polymorphism”）

4. 类的封装：变量可以使用 local 类型，它定义的变量只能在当前类使用，即使是子类也不能使用，如果想子类使用则需要定义为protected 类型。

5. V 中的 parameter 可以使用选择关系：

   如 localparam my_vara = (you_vara<=8) ? 1: (you_vara<=8) ? 2:8 ;

6. v 用在设计中只有两种电路，组合逻辑电路描述和寄存器描述；变量也只有两种，wire 和 reg，连线（含 input 默认 wire 类型）和 assign 语句用 wire 类型变量，其它的都是用 reg 类型。Wire temp=1;相当于 assign temp=1;

   验证中 reg temp=vary；只是编译时给 temp 赋了一个初始变量 vary，如果 vary 是开始是 x 态，那么 temp 就是 x 态。

7. system verilog 允许使用 2 种构造方法来创建一个可以共享的基类;1.抽象类,即可以被扩展但不能被直接实例化的类,它使用"virtual"关键词进行定义.2.纯虚方法,这是一种没有实体的方法原型

一个由抽象类扩展而来的类只有所有需方法都有实体的时候才能被例化.关键词pure 表明一个方法声明是原型定义,而不仅仅是空的虚方法.纯虚方法只能在抽象类中定义,但是抽象类中也可以定义非纯方法

简单说:在抽象类中的方法有实体叫虚方法,没实体就是纯虚方法.

8. 对于在环境中使用 fork join 语句导致多线程调用同一个 class 中的 task（多为很多进程调用同一个 drive 的 task），此时将会导致较为复杂的定位，我们可以在 class 中定义对应的 static work 变量，来表明 task 是否被调用，默认设置为 0，进入 task 时，判断是否为 1，否则进入 task,同时work=1,当运行结束，work=0;实际情况是当 work=1,而又进入该 task，应该是错误的行为，不能这样使用，但如果仅是为了仿真，与应用无关，可以在work=1 时，采用wait(work==0)的方法来轮流调用 ；但如果在多个不同类调用同一个 driver 的 task，则无解，只能判断是否被重叠调用。

9. 在由于不同的数据类型而导致变量不能正常赋值的情况下，SystemVerilog 提供了$cast 系统任务来进行赋值操作。$cast可以作为任务或函数来调用。$case 的语法如下： function int $cast(singular dest_var, singular source_exp);

或者 task $cast(singular dest_var, singular source_exp);

dest_var 是需要赋值得变量。

sorce_exp 是将要赋值到目的变量的表达式。

将$cast 作为任务还是函数调用确定了无效赋值是如何处理的。

当作为任务调用时，$cast 试图将源表达式赋值给目的变量。如果赋值是无效的，会出现运行时错误并且目的变量保持不变。

当作为函数调用时，$cast 试图将源表达式赋值给目的变量，如果赋值成功则返回 1。如果赋值失败，$cast 不会进行赋值操作而是返回 0。当作为函数调用时，不会出现运行时错误，并且目的变量保持不变。

必须注意：$cast 执行的是运行时检查。除了检查目的变量和源表达式是否为单一类型外，编译器不会进行类型检查。

$cast 在 system veilog 中被称为向下类型转换（downcasting），类型向下转化是指将一个指向基类的指针转化为指向派生类的指针

$cast 检查把子类赋值给父类,如果是父类赋值给子类运行会报错,如果是用=把父类赋值给子类,编译的时候就会报错.---因为父类的内容,子类一定有,子类的内容,父类不一定有.

$cast 在 system veilog 中被称为向下类型转换（downcasting），类型向下转化是指将一个指向基类的指针转化为指向派生类的指针。那么$cast有什么特别呢？下面以一个例子进行说明。

对于两个具有继承关系的 class，若对应父类的 instance 为 father_tr，而子类的 instance 为 child_tr，那么　如果只是让子类的指针指向父类，有以下几种操作方法：

1、简单赋值：fathrer_tr = child_tr;

2、使用 copy 函数：father_tr = child_tr.copy ();

3、使用$cast 函数：$cast(father_tr, child_tr);

以上 3 种方法均可实现数据的传递，但有本质区别：（1）通过简单赋值操作，使两个 handle father_tr 和 child_tr 指向了同一块 memory，因此，father_tr 的类型也会变成子类的类型。（2）使用 copy 函数　，是将数值进行简单的 copy，即将 child_tr 对应的 memory 中的数据放入 father_tr 对应的memory 中，这种数据的 copy 方式，不会带来类型的变化，因此，

father_tr 仍然是父类的类型，但使用 copy 函数要求 father_tr 得先 new 一块 memory，如果没有 memory，则需要使用 clone 函数。（3）此处的$cast 操作与（1）中的简单赋值很类似，father_tr 的类型也会变为子类的类型。上面子类向父类进行数据传递时，显示不出$cast 操作向下类型转换的优势，但父类向子类传递数据时，无法使用 child_tr = father_tr，则必须要使用 $cast 操作。

使用$cast 操作，直接使用$cast(child_tr, father_tr)是非法的，此时则要求基类的 handle 必须指向派生类的对象，即 father_tr 必须指向child_tr 类型的对象 ，因此一般会有如下几步操作：

1、定义一个派生类类型的对象：child1_tr = new;

2、 father_tr = child1_tr;

3、$cast(child_tr, father_tr);

以上第（3）步，$cast 会做类型检查，若类型兼容返回 1，否则返回 0。

动态数组 new 可以传递变量，如 bit[7:0] matrix[]; matrix = new[i];i 为传递进来的变量

二维动态数组 new 的方法：

```
bit[7:0]matrix[][];
matrix = new[i];//
foreach(matrix[i,])begin
  matrix[i] = new[j] //
end
```

//最后开辟的数组大小为 matrix[i]；使用 matrix.size(),能得到 i 的值，使用 matrix[i].size()可以得到 j 的值；

三维动态数组 new 的方法：

```
bit[7:0]matrix[][][];
matrix = new[i];//
foreach(matrix[i,,])begin
  matrix[i] = new[j] //
```

```
  foreach((matrix[i,j,])begin
    matrix[i][j] = new[k] //
  end
end
```

//最后开辟的数组大小为 matrix[i][j][k]

10. 数据类型强制转换 格式：string’(3*8);注意上撇号是有(″’)这两个符号的按键

11. 在单纯的 .v 环境中，task 默认为静态的，它的变量都是静态的，静态变量指退出了 task,此变量的值还保留着，下次再调用此 task,变量的值不变，如果采用+1，则在原值上+1；如果 task 声明为 auto 类型，那么其里面的变量为动态的，否则为静态变量，但在 sv 中可以在 automatic task 声明 static 变量，同样在 static task 中可以声明 automatic 变量

Sv 中全局变量都是静态变量,局部变量是动态变量.

12. reg [7:0] value=0;此句表示在编译时给该变量初始值为 0，并不是每次运行时都赋值为 0；与 reg[7:0] value; value=0;是不同的概念

13. V 常用 2 种语法 assign 和 always 块，assign 的变量为 wire 类型，always 的变量为reg 类型，当为时序电路时用《=，当为组合电路时用=；

14. 操作符>>和>>>区别，>>表示逻辑右移，左边补 0，>>>表示数据右移，符号位不变，数据右移；<<在验证中相当于 2**n（2 的 n 次方）

15.

```
i n i t i a l
    b e g i n
    R e s e t = 0;
    R e s e t = #100 1;
    R e s e t = #80 0;
    R e s e t = #30 1;
e n d
    和
i n i t i a l
```

b e g i n

R e s e t <= 0;

R e s e t <= #100 1;

R e s e t <= #180 0;

R e s e t <= #210 1;

e n d

效果是一样的，原因是=为阻塞（串行），<=为非阻塞（并行）

16. 注意区别&&和&，||和|；&，|表示位与，位或；&&，||表示逻辑与逻辑或，逻辑与/或得到的结果只有真和假（1和 0，为 1bit数据）！表示逻辑取反，~表示位取反，**表示 n 次方，2**3，表示 2 的 3 次方

17. assign #7 B=A;此时表示每隔 7 个单位时间把 A 赋值给 B，如果在 7 个单位之间，A 有变化，B 不会跟着变化,相当于有滤波功能.

assign B=A;表示 B 随着 A 的变化而变化，相当于 always @(A) begin B=A end;只有输入信号 A 变化了，A 才会把值传给 B，试分析 assign B=function(A);当 A 没变，但 function(A)有变化，此时 B 任然不会变，因为输入信号 A 没变化，所以 B 就不会变，相当于是 always()敏感列表不全。

18. specify 理解 固有延时模型(缺省模型)不传送脉冲宽度小于电路延时的信号。这是开关电路的行为特性

在传输延时模型中，输入上的所有变化在路径延时之后反映到输出上。这是传输线的行为特性。

19. reg，logic,integer 是四态，

bit,int 是 2 态，

四态变量默认初始值为 X,

两态变量默认初始值为 0

wire 当为四态时，默认初始值为Z；当为两态时，默认初始值为 0.

20. 分析

a s s i g n # (4,8,6) A r b = & DataBus 上升时延为 4，下降时延为 8，截止时延为 6，传递到 x 的时延为 4（4、8 和 6 中的最小值）； w

i r e #5 A r b;这个时延表明 A r b 驱动源值改变与线网 A r b 本身间的时延。考虑下面对线网 A r b 的连续赋值

语句：a s s i g n # 2 Arb = Bod & Cap;

21.读取 SDF 文件时

$sdf_annotate ("sdf_file", [module_instance, "config_file","log_file", "mtm_spec","scale_factors"," scale_type"]); sdf_file：SDF 文件的绝对或相对路径

module_instance：标注范围。缺省为调用$sdf_annotate 所在的范围

config_file：配置文件的绝对或相对路径。缺省使用预设的设置。

Log_file： 日志文件名，缺省为 sdf.log。可以用+sdf_verbose 选项生成一个日志文件。

Mtm_spec：选择标注的时序值，可以是{MINIMUM，TYPICAL，MAXIMUM，TOOL_CONTROL}之一。缺省为 TOOL_CONTROL(命令行选项)。这个参数覆盖配置文件中MTM 关键字。可以在MAKEFILE 中加入 DELAY_OPT=mindelays 或 maxdelays

Scale_factors：min:typ:max 格式的比例因子，缺省为 1.0:1.0:1.0。这个参数覆盖配置文件 SCALE_FACTORS 关键字。

Scale_type：选择比例因子；可以是{FROM_MINIMUM, FROM_TYPICAL, FROM_MAXIMUM, FROM_MTM}之一。缺省为 FROM_MTM。这个参数覆盖配置文件中 SCALE_TYPE 关键字。

22.在用$display 时，

可以加上宏`__FILE__和`__LINE__,则打印时会显示出是那个文件及那行打印;$display ("string",`__FILE__,`__LINE__)

$display ("%m:xxxxx");%m 表示将会打印出当前打印语句的层次关系，如：cal_pkg::\cal_ref_mdl::ref_mdl_main:xxxx

$write 和 display 一样都是打印语句，只是 display 打印完后自动换行，但 write 不会自动换行，

格式化打印，对于字符类型变量处理很好用，

$sformat(name,"../../aa%0d",i);name 变量可以得到../../aa1, (i=1)

23.在一个 CLASS 中，new 函数不能是 virtual 型，

24.在一个 CLASS 中，如果两个随机数是用randc,那么这两个数不能约束相等，而用 rand，可以约束相等， 如果用了 constrain ， 则不要用 randc

25.数据取反：reg[7:0] datatemp;datatemp.reverse;此时 datatemp 的数据倒转了，注意不能 databuf=datatemp.reverse;

26.在 TC 中引用 makefile 中定义的宏或其它宏 使用 `"`filename`"

27.$test$plusargs(string) 整个函数用来查找命令行的字符串

$value$plusargs(user_string,variable);user_srting 命令行内容

variable 使用时的变量

如+FINISH=10000 +TESTNAME=this_test

real frequency;int stop_clock; reg[8#32:1] testname;

if($value$plusargs) ("FINISH=%d",stop_clock);

if($value$plusargs) ("TESTNAME=%s",testname);

命令行则使用相应的命令 args=+stop_clock=1000;args=+ 是固定用法;

如果使用多个变量则需要使用 “ ”，如 args="+stop_clock=1000 +testname="ok""

29. 内置函数

$onehot(表达式) 如果表达式只有 1bit 为 1，则返回真

$onehot0(表达式) 如果表达式至少有 1bit 为 1，则返回真，即表达式不全为 0，及表达是不为 0，返回为真，$onehot0 returns true if at most one bit of expression is high.（表达式至少有一位为 1，）

$isunkonwn(表达式) 如果表达式任何一位为X 或 Z，则返回真

$countones(表达式) 返回表达式中 1 的个数

$sampled (expression [, clocking_event]) : is provided to return sampled value of an expression

$rose (expression [, clocking_event])

$fell (expression [, clocking_event])

$stable (expression [, clocking_event])

$past (expression [, number_of_ticks] [, expression2] [, clocking_event])

$typeof(表达式) 返回表达式或数据的类型

$typename 系统函数返回一个代表其自变量的解析类型的字符串

$bits(表达式或变量) 返回表达式或变量的有效位数，如 reg[7:0] temp; int long =$bits(temp);//long=8

固定(动态)数组 size 大小用$size(arrary) ，队列 的尺寸大小用 q.size(); 得到

----在最底层写 driver 或其它方法时，尽量不要用常量，而是根据输入变量的长度来做处理，因为 sv 可以根据变量类型直接或间接探测任何变量的长度。

prev() prev()方法的语法如下： prev()函数寻找索引小于指定索引的条目。如果存在前一个条目，索引变量被赋值为前一个条目的索引，并且函数返回 1。否则，索引不会发生变化，并且函数返回 0。

30. 想显示字符可以定义枚举类型数据；格式为

typedef enum {sta0=0,sta1=1,sta2,sta3,...} e_sta;

e_sta e_temp;定义变量 e_temp 为枚举类型

31. reg [8:0] test ; test1=test[8:1]+test[0]; 该方法表示 test/2+余数，如果不+test[0]则表示舍去余数；同理可以推出/4,/8 对余数的处理方法

32. 用接口时，如果不用时钟，在声明时需要用空括号，不能省略，如

infterface infterface_s();但一般我们都会把一个系统时钟作为输入送给它，下面两个是等效的。 assign infterface_s.clk = clksys; infterface_s(clksys);

33. 联合数组实现了一个所声明类型的元素的查找表。用作索引的数据类型作为查找表的查找键值，并强制了一种顺序

联合数组的声明语法如下：

data_type array_id [index_type];

其中：data_type 是数组元素的数据类型。固定尺寸数组可以使用的任何类型都可以作为联合数组的数据类型。

array_id 是联合数组的名字。

index_type 是用作索引的数据类型， 或者是*。如果指定了*，那么数组可以使用任意尺寸的 integral表达式来索引。采用数据类型作为索引的联合数组将索引表达式限制成一个特定的数据类型。

联合数组声明的例子如下：

```
integer i_array[*];          // 整数联合数组（未指定索引）
bit [20:0] array_b[string]; // 21 位向量的联合数组，使用字符串类型作为索引
event ev_array[myClass];     // 事件类型的联合数组，使用类 myClass 索引
```

num() num()方法的语法如下： num()方法返回联合数组中条目的数目。如果数组是空数组，那么它返回 0。

delete() delete()方法的语法如下： 其中 index 是一个可选的适当类型的索引。

如果指定了索引，那么 delete()方法删除特定索引的条目。如果试图删除的条目不存在，该方法不会发布警告信息。

如果没有指定索引，delete()方法删除数组的所有元素。

exists() exists()方法的语法如下： exists()函数检查对于指定数组中是否存在指定索引的元素。如果元素存在则返回 1，否则返回 0。

first() first()方法的语法如下： first()方法将指定的索引变量赋值为联合数组中第一个（最小的）索引的值。如果数组是空的则返回 0，否则返回 1。

last() last()方法的语法如下： last()方法将指定的索引变量赋值为联合数组中最后一个（最大的）索引的值。如果数组是空的则返回 0，否则返回 1。

next() next()方法的语法如下： next()方法寻找索引值大于指定索引的条目。如果存在下一个条目，索引变量被赋值为下一个条目的索引，并且函数返回 1。否则，索引不会发生变化，函数返回 0。

29. 数组队列函数

min,max,unique,reverse,sort,rsort,shuffle 这个几个函数 数组队列 都可以用

int q=’{1,2,6,2,6,8}

min:找出数组中最小的值 tq=q.min;则 tq={1};

max:找出数组中最大的值 tq=q.max;则 tq={8};

unique:返回唯一的值 tq=q.unique;则 tq={1,2,6,8};

//min,max,unique 的结果都是一个队列类型

fine with:按要求找数 tq=q.find with (item>3);则 tq={6,6,8};

sun:数组求和 int tq=q.sum;则 tq=25;

reverse:q.reverse;表示把 q 的数据反向放置；高位在低位，低位在高位；

sort:q.sort;表示把 q 进行排序，小的在 0 为，最大的在最后；

rsort:q.rsort;也是排序，只是最大的在位置 0，最小的在最后，

shuffle:q.shuffle;表示把 q 里面的元素顺序打乱

动态数组使用前需要先 new[];$size 对动态数组很重要，因为动态数组开始并不知道其大小；但在使用前要通过 NEW[5]之类的语句申请空间的大小

数组可以直接比较和赋值

队列与动态数组不同，它不需要通过 NEW[]来分配空间大小，但可以把定宽或动态数组的值复制给队列

队列 int q[$];队列操作：q.insert(1,j);在编号为 1 的数据后插入 j,(数据编号是从 0,1,2 开始)；q.deltet(0);删除第 0 个元素；

q.push_front(6);,在对了 q 的最前面插入数字 6；q.push_back(6);,在对了 q 的最后面插入数字6；

j=q.pop_back;q 从最后弹出一个数给 J，j=q.pop_front;q 从最前面弹出一个数给 J，同时 q 会少一个数

30. 在检测上升沿和下降沿时，由于从 0->!0 表示上升沿，1->!1 表示下降沿，那么也就是说从 0->x 或从 0->z 都会被认为上升沿，从 1->x 或从 1->z 都会被认为下降沿，所以在等待沿是用@不是非常可靠，可以在@语句之后再在相关时钟下用全等===来检测一下是否===0 或是否===1。比较下面语法

```
module equalities2();
      reg [3: 0] rega, regb, regc;
      reg val;
   initial begin
      rega = 4'b0011;
      regb = 4'b1010;
      regc = 4'b1x10;
   end
   initial fork
      #10 val = rega === regb ; // val = 0
      #20 val = rega !== regc;   // val = 1
      #30 val = regb === regc;  // val = 0
      #40 val = regc === regc;  // val = 1
      #50 $finish;
   join
endmodule
   module equalities1();
      reg [3: 0] rega, regb, regc;
      reg val;
    initial begin
      rega = 4'b0011;
      regb = 4'b1010;
```

```
        regc = 4'b1x10;
    end
    initial fork
        #10 val = rega == regb ;  // val = 0
        #20 val = rega != regc;    // val = 1 ，只要检测到 1bit
不等就成立
        #30 val = regb != regc;    // val = x ，
        #40 val = regc == regc;   // val = x
        #50 $finish;
    join
endmodule
```

31. 对于同步寄存器的第一级寄存器的时序检查不必在乎，但实际上会报时序违背，这时需要通过一定的手段让不不报出来干扰。

```
Instance{
    top.udut.reg0,
    top.udut.reg1,
….
    top.udut.regn
} {noTiming};
```

(上述内容可以写在一个文件中，filename)

只要在 MAKEFILE 中通过选项+optconfigfile+filename，则可以不对上述寄存器做时序检查

其功能等同于

```
initial begin
  force top.udut.reg0.notifier = 1'b0;
…
  force top.udut.regn.notifier = 1'b0;
end
```

notifier 是寄存器的时序方面的一个选项，也可能是其它标志变量，force 它等于 0，相当于屏蔽时序检查的功能，个人倾向于第一种方法

32. 类的 NEW()函数除了分配空间外，还一个就是初始化变量；二值变量默认初始化为 0，四值变量默认初始化为 X。New()返回的是一个指向类对象的句柄。

33. 类里面的静态变量使用范围是这个类的所有实例；而动态变量只使用范围只有当前实例化的实例中，类中默认是动态变量；如下

```
class Transaction;
static int count=0;//初始化静态变量
int id;
function new();
id=count++;
endfunction
endclass
Transaction t1,t2;
initial begin
  t1=new();  //第一个实例，id=0,count=1;
  t2=new();  //第二个实例，id=1,count=2;
$display ("second id=%d,count=%d,t2,id,t2.count");
end
```

在创建类的时候建议使用 ID 来区分；静态变量通常在声明时初始化，而不在 new 函数中初始化

this 关键字是一个指针，它指向类一级的变量

```
class Scoping;
string oname;
function new(string oname);
    this.oname = oname;  //类变量 oname = 局部变量 oname;
endfunction
```

endclass

29. 可以用 typedef 语句声明一个类，然后这个类就可以使用了，然后在后面再定义类;如 typedef love; 然后就可以使用 love 这个变量（love 可以在其他地方声明

30. 通过方法来创建句柄，方法的参数前需要加上ref，否则创建不了，编译也会出错

```
function void create(ref Transaction tr)
   tr = new();
   tr.addr = 42;..
Endfunction
transaction t;
initial beign
   create(t);//创建一个 transction
end
```

31. 一个句柄不能传递给 2 个 mailbox();需要拷贝一份分别传递。

32. 流操作符 << (把数据从左至右变成流) 和>> (把数据从右至左变成流);后面带表达式，结构或数组。

```
Initial  begin
int h;
bit[7:0] b,g[4],j[4]=` {8'ha,8'hb,8'hc,8'hd};//注意实际是 0a,0b,0c,0d
bit[7:0] q,r,s,t;
h={>>{j}};  //0a0b0c0d--把数组打包成整型
```

33. 基类中的virtual 函数和任务；目的是为了在扩展类中可以重定义，也就是扩展类可以定义一个与基类中同名的方法，但同时在扩展类中可以调用原来的基类中的方法，，注意 new 函数没有virtual 的属性

34. 如果在扩展类定义了一个约束，并且扩展后的约束名和基类里的约束名相同，那么扩展类的约束会替代基类中的约束，这样就可以改变现有约束的行为

35. 接口信号必须使用非阻塞赋值来驱动；即用<=符号赋值

36. systerverilog 中在 program 中不允许使用 always 块，因为 always 块没有初始化，而验证平台中均需初始化

37. 时钟发生器尽量不要放在程序块中，应该把它放在module 中，然后例化；主要是避免初始化时的上升沿出现

38. 建议在每个 module 文件顶部加入`timescale 以保证此 module 的单位时间是预期否则会按照编译顺序以前一个`timesacle 定义的单位一致,timescale 只能在 module 外部使用，内部使用的无效

39. 宏可以传递参数，一般都是一个变量或字符，但想传递变量或字符的部分内容，则需要用连接符号``，而连接“只使用`,如

```
task automatic SubClk(string name = "",
            bit gate_clk = 0,
            bit [7:0] cycle = 8'h0a,
            ClkMonitor clk_mon,
            virtual SignalIf clk_if,
            bit     cypher_en=1,
            real    exp_freq_val=0);
endtask: SubClk

`define nvl_clk_mon(ClkName,gate) SubClk(`"ClkName`", gate, , ClkName``_mon, ClkName``_if,,)
`define nvl_clk_det(ClkName,cycle) SubClk(`"ClkName`", , cycle, ClkName``_mon, ClkName``_if,,)
```

```
`define ahb_mst_intf(mx) \
  AhbMasterInterface intf_``mx ( \
      .hclk       ( hclk          ), \
      .hresetn    ( hresetn       ), \
```

Define nvl_clk_mon(ClkName,gate)

SubClk(`"ClkName`test",gate,,ClkName``_mon,ClkName``_if,,)

)

40. 用 rand_mode(0)可以禁止变量随机化，如

assert(handle.randmize());handle.length.rand_mode(0);handle.length=55;此时 length 固定为 55；然后再

assert(handle.randmize());此时 length 还是 55，没有随机化，handle 中其它的随机变量都随机了。

41. 用 constraint_mode 打开或关闭约束，可以在随机类中做 2 个约束，一个随机范围广，一个随机几个固定的边界值；这样可以完成先随机再定向测试；

    先使用 xxx(class variable).constraint_mode(0);关闭所有约束，再使用 xxx(class variable).xx(constraint name).constraint_mode(1)打开某个约束

42. randomiz();当（） 为空时，默认随机所有的随机变量，当（） 为某个随机变量时，则只会随机固定的几个变量 ;handle.randmize(a,b,c) 只会随机 a,b,c 三个变量

43. constraint 可以使用权重分布 dist 来控制，使用：=或：/;它们有些不同:=表示所有的权重都一样，如[15,19]:=20;则表示出现 15~19 的权重均为 20,；但[15,19]:/20,则表示权重总共为 20，每个值得权重为4；而且权重可以使用变量来控制，在运行中改变此变量，则可以改变权重；如果 0:/w1;1:/w2;

44. ref 端口方向，只有 systerverilog 才有，它是对变量的引用（不能是 NET），它的值是变量最后一次赋的值，如果将一个变量连接到多个 reg 端口，就可能产生竞争，因为多个模块的端口都可能更新同一个变量,当一个 function 改变了 ref 变量时，在 function 外部也可以看的到；ref 与 inout 类似，但 inout 在 function 中时，function 会先拷贝该变量，当操作完结束返回函数时，再把值拷贝给该变量；简单说只有在函数结束时，此变量才会改变；而 ref 类型时，数据任何时间都可能在改变。

45. 不同的类之间传递信息可以通过mailbox，同一种类不同的对象可以通过静态变量来传递，这个静态变量可以让这个类的所有实例共享，也可以通过 mailbox 来传递

46. systerverilog 提供一种特殊的数据类型 type 它是传递类型的变量，如 type T, T=real;T=int, T=BasrTr(BasrTr 为一个类)等操作，用在参数类比较多

47. package……endpackage SystemVerilog 包提供了额外的机制在多个 SystemVerilog 模块、接口和程序间共享参数、数据、类型、任务、函数、序列以及特性的声明。包是显式命名的作用域，它出现在源文本的最外层（与顶层模块和原语处于同一层次）。类型、变量、任务、函数以及特性都可以在一个包中声明。

48. 这些声明可以在模块、宏模块、接口、程序和其它包中通过导入或完整解析的名字来引用。

49. 位于包内的项目通常是类型定义、任务、以及函数。位于包内的项目不能具有层次化的引用。包还可能包含参数、变量和线网。对于那些不方便通过层次向下传递的全局项目，这种方式可能有用。与在一个编译单元或模块内声明的变量相同，位于包内的变量声明赋值应该出现在任何 initial、always、always_comb、always_latch 或 always_ff 块被启动之前。

50. 随机函数 $urandom_range(min,max); 用在做随机往系统中插入异常，# $urandom_range() `ms;

51. event 用法：一般是采用 event e;定义事件，-> e;触发事件；@e;等待事件，其中@是等待边沿的触发，而 wait 是等待电平触发，可以采用 wait(e.triggered);

52. 采用@等待的方式，当触发已结束，会引起阻塞，如果用wait 则不会引起阻塞，但要注意 wait 会在单个事件触发器上反复执行.

53. 对象的复制就是采用 copy 函数，比较下面的两个函数

class 中的 copy 函数解释：virtual function transaction copy ();

virtual function 表示虚函数；transaction 表示函数返回的数据类型（相当于 int,bit[7:0]之类的数据类型），copy 是函数名，但因为返回的是一个类，所以在使用时，都需要对 copy 做实例化，即 copy = new();的动作

```
class Transaction
rand bit[31:0]src,dst;
```

```
    virtual function Transaction copy()
    copy=new();
    copy.src=src;
    copy.dst=dst;
    endfuncion
    endclass
    class BadTr extends Transaction
    rand bit  bad_crc;
    virtual function Transaction copy()
    BadTr bad;   //此处是不是有递归的感觉！！！？？？？
    bad=new();
    bad.src=src;
    bad.dst=dst;
    bad.bad_crc=bad_crc
    endfuncion
endclass:BadTr

    class Transaction
    rand bit[31:0]src,dst;
    virtual function void copy_data(input Transaction tr);
    copy.src=src;
    copy.dst=dst;
    endfunction
    virtual function Transaction copy();
    copy=new();
    copy_data(copy);
    endfunction
    endclasss
    class BadTr extends Transaction
```

```
    rand bit Bad_crc;
    virtual function void copy_data(input Transaction tr);
    BadTr bad;
    super.copy_data(tr);
    $cast(bad,tr);//基类句柄类型转换成扩展类,相当于
//bad=tr,此时$cast 作为一个 task,如果出错则会打印信息
    bad.bad_crd=crc;
    endfunction
    virtual function Transaction copy();
    BadTr bad;
    bad=new();
    copy_data(bad);
    endfunction
endclasss
```

54. 扩展类的虚函数必须跟基类的函数相匹配，包括所有的参数和返回类型.

55. $get_coverage 可以得到所有覆盖组的总覆盖率,它返回一个 0~100 的实数,而 get_coverage()和 get_inst_coverage(),可以缩小测量范围; CoverGroup::get_coverage()或

    cgInst.get_coverage(),get_inst_coverage()返回一个特定覆盖组实例的覆盖率,如:cgInst.get_inst_coverage(),如果希望得到单个实例的覆盖率,那么需要指定 option.per_instance=1;

56. 在使用 class 包含功能覆盖率时，注意 class 是无 sample()内建函数，而是 covergroup 有内建 sample 函数；所以在使用时应该是：

    类例化名.covergroup 名.sample()

```
    class transac;
    int x;
    int y;
    covergroup tr_cov;
```

```
        coverpoint x;
        coverpoint y {
            bins min = {0};
            bins max = {10};
            bins mid = {[1,9]};
        }
    endgroup
    function new();
        tr_cov = new();
    endfunction
endclass
```

57. 变量没在always 敏感列表中 ，即使变量有变化 也 不会传输到目的变量，所以可以使用 always(*);来处理

58. class trans;…… endclass trans tr; tr=new(); tr 为句柄 ，new ()之后产生了一个对象（object）,mailbox 只能 传递句柄 而非对象

59. 对于静态类属性，无需产生一个该类型的对象就可以直接使用，可以定义为静态方法或静态变量， 使用 A::FUNC B::TASK

    C::variable 方式调用，A,B,C 为一个类，FUNC,TASK 为类里面的静态方法,variable 为类里面的静态变量。

60. 时钟控制块将时序和同步细节与一个测试平台中的结构、功能和过程元素分割开来。因此，采样和驱动时钟控制块信号的时序成为隐式的并且相对于时钟控制块的时钟。这使得我们能够以一个非常简洁的方式编写一组关键的操作，而无需显式地使用时钟或指定时序。

    这些操作是：

    同步的事件

    输入采样

    同步的驱动

```
    clocking bus @(posedge clock1);
```

```
default input #10ns output #2ns;
input data, ready, enable = top.mem1.enable;
output negedge ack;
input #1step addr;
endclocking
```

在上面的例子中，第一行代码声明了一个称为bus 的时钟控制块，它以信号 clock1 的上升沿进行时钟控制。第二行代码通过default指出：时钟控制块中的所有信号都应该使用一个 10ns 的输入时滞和一个 2ns 的输出时滞。下一行代码为时钟控制块加入了三个输入信号：data、ready 和 enable；最后一个信号指向层次化的信号 top.mem1.enable。第四行代码为时钟控制块加入了一个 ack 信号并过载了缺省的输出时滞从而使 ack 信号在时钟的下降沿被驱动。最后一行代码加入了addr 信号并过载了缺省的输入时滞从而使 addr 在时钟上升沿之前的一个步值时被采样。

除非另有说明，缺省的输入时滞是 1step，缺省的输出时滞是 0。一个步值是一个特殊的时间单位，它的值在 18.10 节中定义。一个 1step 的输入时滞使得输入信号在时钟信号之前的时间步值中采样它们的稳定值（也就是在前一个 Postponed 区域）。与其它代表物理单位的时间单位不同，一个 step 不能被用来设置或修改时间精度或时间单位。

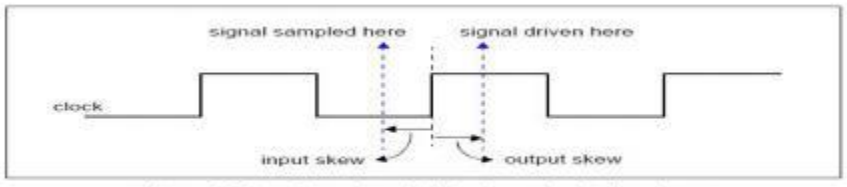


Figure 16-1 — Sample and drive times including skew with respect to the positive edge of the clock.

61. 在实例化时，当有 inout 时，尽量采$tran()来连接，initial begin $deposite(wire var,data) ; end $deposite 给 wire 赋初始值

62. sva 在波形上显示的方法： 在 tb.v 中 dump 波形地方加上：$fsdbDumpSVA

63. force wire = 1'b1;在 force 语句的右边只能为固定值，不能为变量，但如果想随着仿真进度变化怎么办？可以设置一个中间wire信号，然后信号wire = reg;,reg 再在测例里面变化就可以

64. 使用该方法 VCS 和 NOVAS 都需要使用新版本才支持

VCS E-2011.03-SP1

NOVAS 2011.07

2. 在 dump 波形的地方使用$fsdbDumpMDA( )函数把 Memory 数组 Dump 出来。

例如 RTL 中有一数组定义为

reg [7:0] mem[4095:0];

在 dump fsdb 波形的文件中添加

$fsdbDumpMDA(tb_top.uSlave.mem);

关于该函数详细参数用法参考工作站

/opt/othersoft/novas/2011.07/doc/linking_dumping.pdf

P53

3. 将数组元素添加到 Verdi/Debussy nWave 窗口

3.1 在 nTrace 或 nWave 窗口选择 Tools--Memory/MDA

65. 对于需要使用线与功能的 PAD，需要定义 PAD 类型为 triand 类型 ，正常定义为 wire;但 PAD 是用原语来实现则无需如此定义

详解 SV 开源库—svlib（转）

转载 2018-02-04 17:09 :08

最近的一个项目中，使用 UVM 搭建了一个 Testbench，用到字符串的查找与匹配（如：从 128 个寄存器的名字中，分别找出是 2G/3G/4G/5G 的寄存器）。习惯了 perl，python 等脚本的正则表达式，当时的唯一想法就是，SystemVerilog 中怎么使用正则表达式？于是在网上狂搜了一段时间的资料后，终于发现新大陆 -- svlib， 一时忍不住求知的欲望，就把整个 svlib 从头到尾看了一遍，并结合网上找到的一些资料，整理出了本篇博文。

svlib 的问世

相对于 C#， Java 和 Python 等强大的库，SystemVerilog 可能会让你有点失望。正是基于这一点，有一批 verifier 们就局部地联合了起来开始搞事情。他们试图在 SV 基础上开发一些扩展地库，来弥补一些SV 应该具备的底层库； 同时，在这些库后来日益完善的基础上，将这些库开源了出来，并希望不断更新（或者被收入 SV 的标准第三方库）。于是乎开源质量较高且至今保持活力的 SV 的第三方库 svlib 就诞生了。

svlib 简介

Verilog 公司开源了他们之前开发的 SV 的扩展底层库 svlib，详细的资料可以从 http://www.verilab.com/resources/svlib/ 中找到。svlib 分成了若干个分支，不同分支具备相应的功能，这些可以拆解和独立开发的功能包括有：

- 1） 通用的字符串处理和正则表达式功能。
- 2） 对文件和目录操作的功能，包括目录查找、列表、文件属性征询的功能。
- 3） 可以同操作系统互动的功能，包括有查找环境变量、传入命令行、时钟日期信息查询等功能。
- 4） 各种其它方便的功能，例如与枚举类型相关的函数操作。
- 5） 成套的插件用来从.ini 或者 YAML 配置文件读入，继而存储为配置数据格式 DOM；或者反之，从 DOM 格式，存为.ini 或者 YAML 标准格式。

另外，svlib 也有类似 uvm_macros.svh 的宏文件 svlib_macros.svh，该文件也定义了一些易用的宏，方便使用者。

svlib 详解

字符串处理

SV 的 string 有一点尴尬的是，它并不允许直接让用户在 string 类型上面扩展，而尽管它看起来就有点像类，但又无法去继承。

```
string S = '   Some text   ';
int n = S.len();
S = S.toupper();
```

对于上面字符串的操作，跟对象的使用很像，但如果像扩展 string 类型，只能考虑在 svlib 中添加一个新的类 Str，通过类的方法来实现扩展。例如下面的例子：

```
Str ss = Str::create('   Some text   ');
ss.trim(Str::Right);
$display('\'%s\'', ss.get());
```

输出结果：'   Some text'

通过添加新的字符串类Str（内嵌 string 变量并且对其操作），这可以在原有 string 类型的基础上扩展更多需要的方法，例如上面的去除字符串右边的空格。这个新的方法看起来可以带来更多新的特性，但同时也带来了一点麻烦。例如原有的SV 用户已经习惯于对 string 变量本身的操作、通过操作符例如{}来合并字符串、或者直接赋值的形式来获取字符串内容等，这一点同新添加的Str 类通过自带函数的操作有明显的差别。

如果 SV 的用户不习惯新的方式，又或者原有的 SV 代码不想大规模改动的时候怎么办呢？svlib 同时也提供了经典的 package 内建函数的形式，将一些主要的函数也同时按照就有的函数操作的形式来实现。

```
$display('\'%s\'', str_strim(S, Str::BOTH));
```

输出结果：'Some text'

上面这个方式更贴近于原有的字符串操作，用户可以选择任何一种方式来实现字符串的操作。

Str class 提供以下一些方法，实现对 Str object 进行简单操作 ：

```
static function Str Str:: create(string s = "");
function void set (string s);
function string get ();
function Str copy ();
function int len ();
```

first 和 last 是用于找该字符串中的 substring, 如上面介绍，从一个 string array 中找出哪此是 4G，哪些是 5G 的 string。first 返回 substr 在该 Str 中第 1 次搜索成功的 index, 若未搜索成功，则返回-1， first 从前向后搜索，而 last 则从后向前搜索。

```
function int first (string substr, int ignore=0);
function int last (string substr, int ignore=0);
```

正则表达式

对于熟悉正则表达式的读者，尤其是 Python 中正则表达式习惯的读者，对于 svlib 中新添加的正则表达式类 Regex 和其函数处理应该不会感到陌生。

```
Regex re;
re = regex_match('06/07/17', '([0-9]+)/([0-9]+)/([0-9]+)');
if (re != null) begin
    $display('Looks like a data');
    void'(re.subst('$2-$1-20$3'));
    $display('data = %s', re.getStrContents());
end
```

输出结果：

```
Look like a data
07-06-2017
```

svlib 提供非常全面的跟正则表达式相关的匹配和替换的方法，更多的函数可以在 svlib 的文档中找到。

文件和目录操作

文件和目录的操作在脚本语言中是经常使用到的，而 SV 欠缺这一点（或者说现有的文件打开、读写的操作不满足多样需求）。svlib 提供了与路径相关的类、与文本信息相关的类等等。譬如，SV 无法批量地得到相关的文件名，而这一些与目录有关的操作，svlib 提供了方法。下面的例码就可以将当前目录下所有后缀名以 sv 结尾的文件都找到并写入到一个字符串的队列中。

```
string dirList [$];
dirList = sys_fileGlob('*sv');
```

下面通过 Pathname 类来完成文件路径的有关操作，通过自带的函数可以取得文件的扩展名、路径名以及文件名本身。

```
Pathname path = Pathname::create('/home/svlib/src/svlib_pkg.sv');
$display(path.extension()); // .sv
$diaplay(path.dirname()); // /home/svlib/src
$display(path.tail()); // svlib_pkg.sv
```

在得到了文件列表之后，我们可以通过 svlib 提供的更多函数将与文本状态信息获取，继而做更多的操作。例如下面的例码中，通过 file_mTime 来提供任何一种文件最后修改的日期，同时将这一日期按照更易读的形式通过函数 sys_formatTime 取得转化。

```
long int mostRecentTime = sys_dayTime() - 24*60*60;
string mostRecentFile = '';
foreach (dirlist[i]) begin
    long int t = file_mTime(dirlist[i]);
    if (t > mostRecentTime) begin
        mostRecentTime = t;
        mostRecentFile = dirlist[i];
    end
end
if (mostRecentFile != '') begin
    $display('The most recent file is \'%s\'');
    $display('It was modified at %s', sys_formatTime(mostRecentTime, '%c'));
end
```

操作系统互动

svlib 也提供一些函数可以获得当前工作目录下的环境变量、查询用来调用 simulator 的完整命令行参数等。下面这个例码就通过查询当前工作环境变量，来修改一些变量值。

```
string cfgVar = 'SIM_CFG_DIR';
```

```
string cfgDir = '../cfg' ; // default value
if (sys_hasEnv(cfgVar))
    cfgDir = sys_getEnv(cfgVar) ;
...
```

错误处理

所谓精细化的软件编程方式，一定离不开对于各种可能出现的错误代码的排查处理，进而提高代码的健壮性。而 SV 用户还不适应这么做，尽管 Java、Python 的编程种这已经是一种习惯了。svlib 是在充分考虑了用户习惯的基础上，对于错误的处理方式作出了既满足基础用户又满足高级用户的选择：

- 默认情况下，svlib 中的函数当发生错误时，会由内置的 assertion 发起断言错误提醒，继而在仿真器的命令窗口中提示。
- 用户也可以通过$assertoff 的命令来关闭 svlib 中内嵌的与错误处理相关的断言。
- 高级用户也可以从函数的参数列表中取得错误号，继而做精细化的错误处理。

例如上面的错误处理默认情况下，会通过断言的方式来报告错误信息。

```
long int t = file_mTime('MISSING/FILE') ;
```

输出结果： : Failed to stat 'MISSING/FILE', errno=2 (No such file or directory)

或者用户也可以用高级的错误处理方式，通过 error_userHandling(1)来使能这一处理方式，而通过 error_getLast()来取得最近的一次处理结果，如果处理结果非 0 的话，那么可以根据错误号来做具体的处理。下面便是一段高级错误处理代码：

```
error_userHandling(1) ;
t = file_mTime('MISSING/FILE') ;
if (error_getLast() != 0)
$display('whoops, my bad: %s', error_fullMessage()) ;
```

原有随机处理的稳定性

在上面的例子中，svlib 并不建议直接调用 new 构建函数来创建对象，而是通过 create()函数来交由 svlib 内部动态创建对象。这种创建对象的方式会直接影响随机种子的创建，而当 svlib 作为基础库被引入到原有代码时，我们并不希望一些底层函数操作会影响随机数的生成。因此，为了消除 svlib 中动态对象的创建可能对随机处理的影响，**svlib 要求用户只能够使用 create()来创建对象**。下面是 create()函数的实现代码：

```
class svlib_C extends svlib_base;
static function svlib_C create(...) ;
  std::process p = std::process::self();
  string rs = p.get_randstate() ;
```

```
  create = new(...);
  p.set_randstate(rs);
  ... // any other initialization activity here
  endfunction
...
```

例如一个 svlib 中的类 svlib_C，它的 create()函数在创建对象前，首先保存了随机的状态 get_randstate()，而在例化了对象之后又还原了随机状态 set_randstate()。通过这种方式，就可以避免由于 svlib 的介入，创建了新的对象之后，对随机数生成的影响。因此，尽管 svlib 中存在着大量的由对象创建解决问题的方案，但依然可以保证原有仿真环境中随机处理的稳定性。

配置文件解析和输出

在 HDL 设计和验证环境的配置中，通过结构化的文件格式例如YAML 或者. ini 来作为输入继而进行配置的情况很普遍。**也许读者所在的公司还在使用 DOC、CSV 等格式来配置，但结构化、层次化清晰的格式 YAML 或者. ini 更为合适。**这个文件格式需要首选转化为树状的或者队列状的软件对象配置信息，继而保存、修改和反向输出它们。对于 Python，有第三方库给予支持，这就方便文件的解析，而 SV 也参照这一特性，使得可以解析文件、放入配置对象（DOM，Document Object Model），而在需要的时候将 DOM 对象再反向输出为需要的格式，例如 YAML、. ini。下面给出一段综合的例码，来介绍这一过程。这种读取配置文件的格式对于验证环境，可以用来通过文本的形式来解析验证的配置结构、变量等（而不是直接编写配置对象）。

```
largerConfig cfg; // The object to be populated
cfgNode dom; // The DOM root node
cfgFileINI fi; // INI file reader/writer
cfgFileYAML fy; // YAML file reader/writer
// Read the YAML file into a DOM
fy = cfgFileYAML.create();
dom = fy.deserialize('src.yaml');
// Populate cfg from the DOM
cfg = new; // or perhaps use UVM factory
cfg.fromDOM(dom); // also constructs inner object
// Modify the cfg objects
cfg.scalarInt = 42;
cfg.objectSC.scalarString = 'new value';
// Make a new DOM from objects. Name it 'NewCfg'
dom = cfg.toDOM('NewCfg');
```

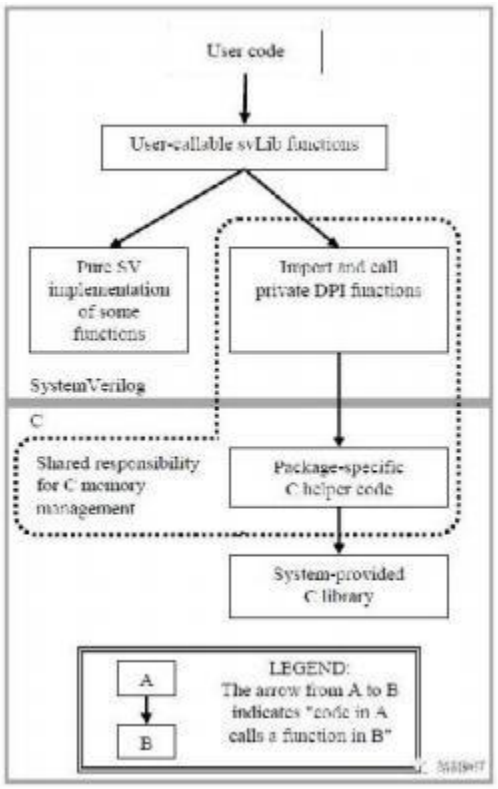
```
// Write the new DOM to a .INI file
fi = new;
fi.serialize('dst.ini', dom);
```

上面的例码首先解析了 YAML 文件格式，继而将配置信息存储到 DOM 结构对象中，稍后对其进行数据修改，并将更新后的数据再输出为.ini 格式。存储到 dst.ini 文件的数据内容如下：

```
scalarInt=42
[objectSC]
scalarInt=1234
scalarString=new value
```

从这里介绍的SV 开源库身上我们可以看出，svlib 的开发者们有着丰富的软件开发经验，而当这些从软件世界迁移过来的开发者在面对 SV 寥寥可数的方法集时，就会不由自主怀念以前软件开发时的各种可用的第三方库。而在 svlib 的开发中，又借用了 C 的一些库和系统接口，例如配置文件 YAML 的解析包是从 C 复用并导入到 svlib 中的。这些分类的方法簇、类簇都可以在 Python、Java 的标准库中找到原型，所以说这些类和函数并不是凭空产生的，而是认识到其它软件编程语言的标准库中一些类和方法对 SV 能够产生帮助时，才移植到 SV 一侧的。又比如上一节在谈论 SV 与其它语言的接口时，我们也谈到了SV 在处理浮点数计算和数学库上面的欠缺，而如果 SV 暂时不提供这样的数学库的情况下，只能建立 SV 通其它支持数学库的语言 Python 或者Matlab 等建立接口，进行混合仿真。

下面这张图给出了 svlib 构建的组织。可以看到 svlib 的一部分函数是由 SV 完全实现的，而另外一部分函数则是间接调用的 DPI-C 的函数，譬如 C 的内存管理方案、复用其它 C 库（YAML）等。



如果想更深入理解 svlib 的 source code, 可在 http://www.verilab.com/resources/svlib/免费下载，通过阅读 svlib 源代码，可以体会到一些开发标准库需要考虑的地方：

. 在对于库、类、函数和参数等命名时需要遵守一定的规则，这不但便于记忆和理解，也便于在文档中查找。

. 考虑到库可能在不同的仿真器和平台上使用，需要尽量在可能的仿真器和平台上测试。

. 最好不要依赖于其它的库。svlib 只基于 SV 的标准库和自带的 DPI C 库，这也使得它可以在其它的方法学库中使用，譬如 UVM、OVM

UVM

1. 基本特性

0. uvm 中框架外的一些变量，如 tc_num,case_name 等一些配置变量，可以在 tc 中通过

uvm_config_db#(int)::set(this, ” path_ ” , ” var_name ” ,var_value);
配置，在对应的模块中通过
uvm_config_db#(int)::get(this, ” path_ ” , ” var_name ” ,var_name);
如：在 monitor 中，不同的测试项监控的对象不一样，不同的测试项，模型也不一样，可以像 sv 一样用 case,if 那样来分支处理，也可以通过重载的方式重写一个 monitor 或 ref_module.

1.任何一个类都可以做一个 get（） 的静态方法，用来创建一个句柄；
做一个 get_type_name()的静态方法来得到类的特性；
通过一个变量存放句柄名字，可以让其它函数访问

```
function new(string name=“ ”)---用于 object 创建实例
       m_inst_id = m_inst_count++;
       m_leaf_name = name;
endfunction
function new(string name=“ ”,uvm_component parent)
---用于 component 创建实例
      set_name(name); // *** VIRTUA
//所以对 componet 可以用 get_name 得到实例化的名字
   m_parent = parent;//可以用 get_parent 得到 parent 的名字
……
endfunction

Class type_class  ;
    Typedef type_class;
    type_class m_inst;
    /*protected*/ type_class m_parent;
    protected     type_class m_children[string];
    protected   type_class m_children_by_handle[type_class];
    local string m_leaf_name; //new 时句柄的名字
    local  int m_inst_id;
   static protected int m_inst_count =0;

    function new(string name=“ ”)---用于 factory 创建实例
       m_inst_id = m_inst_count++;
       m_leaf_name = name;
    endfunction

    virtual function string get_name  ();
       return m_leaf_name;
    endfunction

    virtual function string get_full_name_ ();
      return get_name();
    endfunction
```

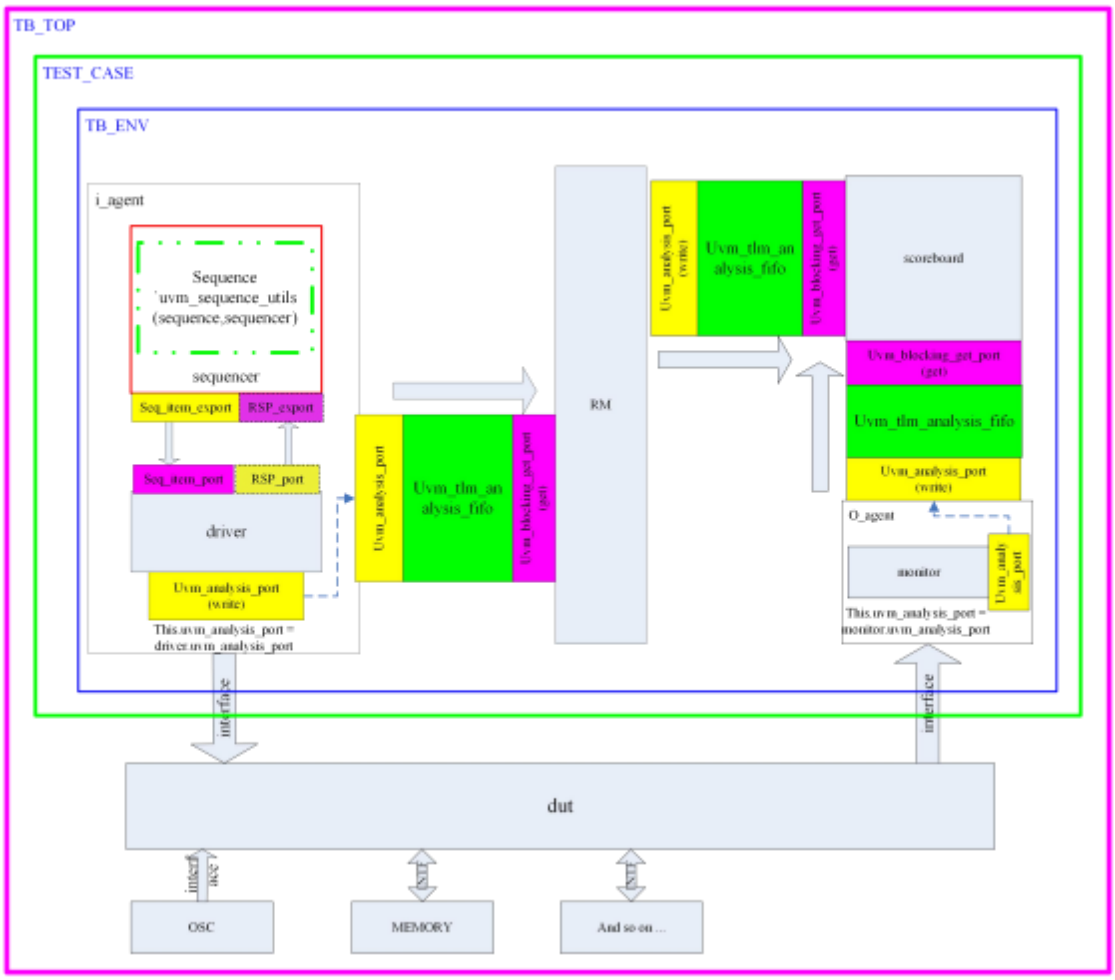
```
    static function type_class get();
        if (m_inst == null) begin
            m_inst = new();
            return m_inst;
    endfunction

    const static string type_name = "type_class";
    virtual  function  string get_type_name ();
        return type_name;
    endfunction
endclass
```

使用时如下

```
Type_class  handle;
String type;
Handle = Type_class::get();
Type = Handle.get_type_name();
```

2.任何一个类都可以有自己的子类—就是类中有另外一个类的变量，可以把这些子类存储起来，做好一些函数备用，就是uvm 中 component 的特性

2.Uvm 常识


TB_TOP
TEST_CASE
TB_ENV
i_agent
Sequence
`uvm_sequence_utils
(sequence,sequencer)
sequencer
Seq_item_export
RSP_export
Seq_item_port
RSP_port
driver
Uvm_analysis_port
(write)
This.uvm_analysis_port =
driver.uvm_analysis_port
Uvm_analysis_port
Uvm_tlm_an
alysis_fifo
Uvm_blocking_get_port
(get)
RM
Uvm_analysis_port
(write)
Uvm_tlm_an
alysis_fifo
Uvm_blocking_get_port
(get)
scoreboard
Uvm_blocking_get_port
(get)
Uvm_tlm_analysis_fifo
Uvm_analysis_port
(write)
O_agent
monitor
Uvm_analy
sis_port
This.uvm_analysis_port =
monitor.uvm_analysis_port
interface
interface
dut
interface
OSC
MEMORY
And so on ...

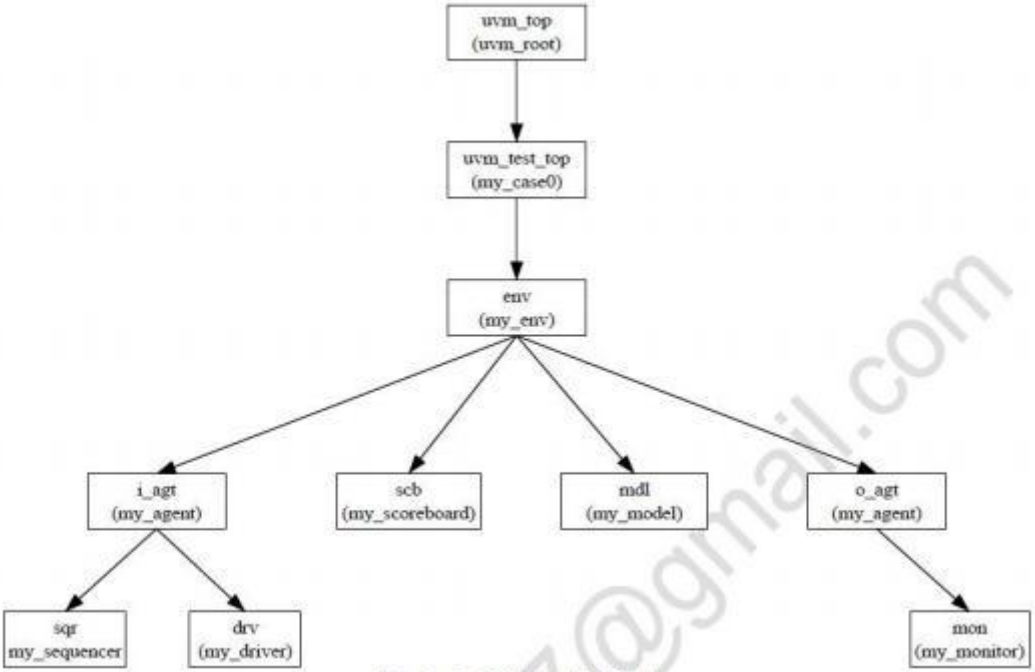


图 2-1 完整的 UVM 树

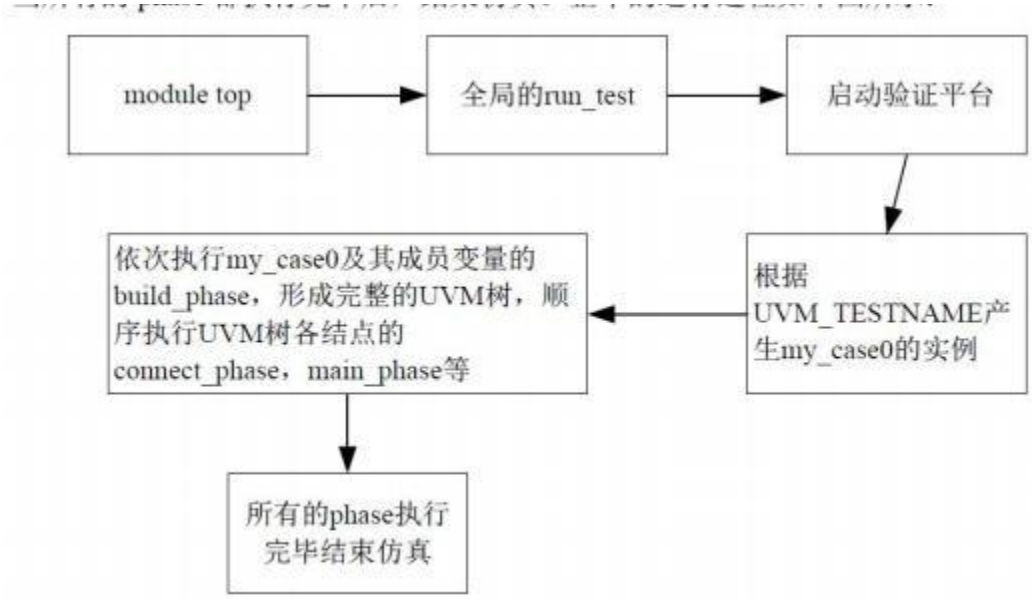


图 1-9 UVM 验证平台执行流程

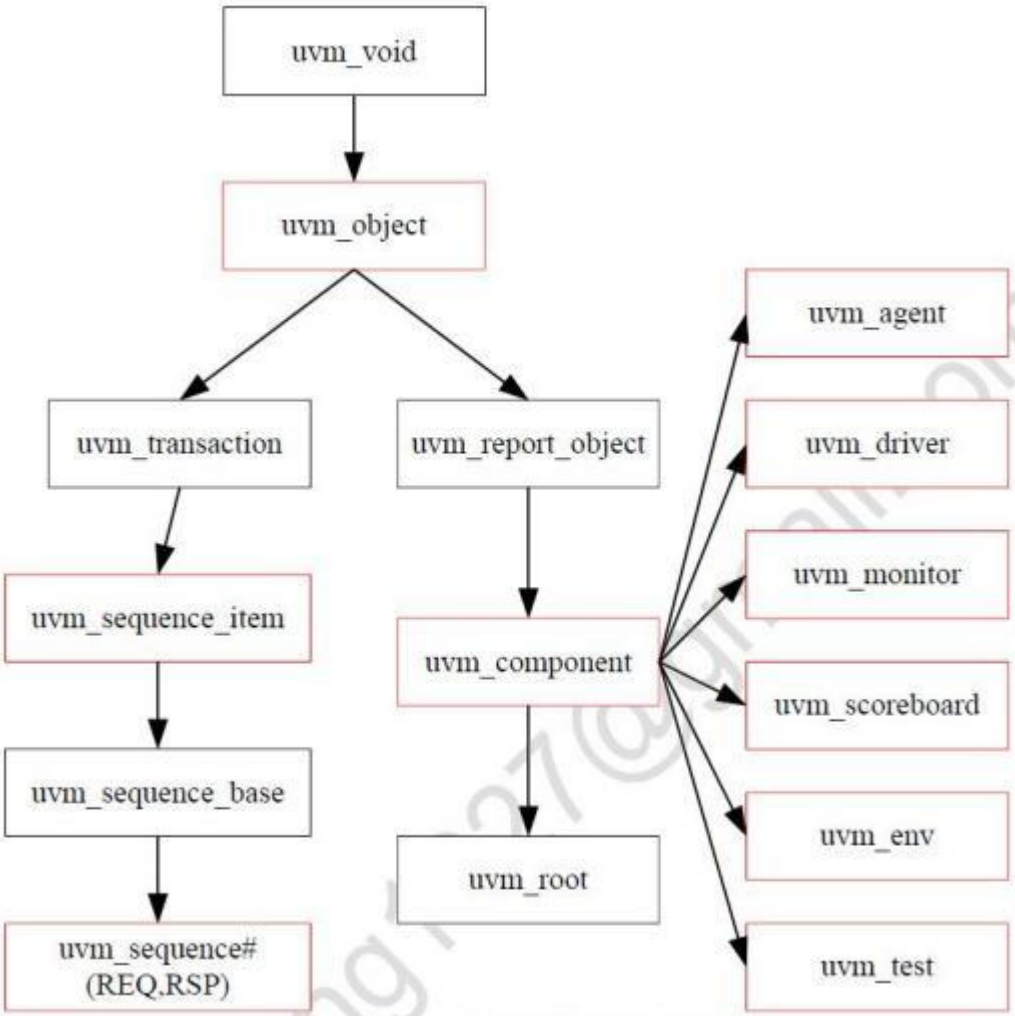


图 2-2 UVM 中常用类的继承关系

红色框框是属于 uvm 成员

```
virtual class uvm_void;
endclass
```

uvm_object 有一个特殊 class ：

```
static /*protected*/ uvm_status_container __m_uvm_status_container
= new;
```

uvm_status_container 有一个特殊类 uvm_scope_stack scope = new;

这些特殊类方在 uvm_misc.svh 文件中

```
class uvm_status_container;
  //The clone setting is used by the set/get config to know if cloning
is on.
```

```
  bit             clone = 1;
  //Information variables used by the macro functions for storage.
  bit          warning    = 0;
  bit          status     = 0;
  uvm_bitstream_t  bitstream  = 0;
  int          intv       = 0;
  int          element    = 0;
  string       stringv    = "";
  string       scratch1   = "";
  string       scratch2   = "";
  string       key        = "";
  uvm_object   object     = null;
  bit          array_warning_done = 0;
  static bit field_array[string];
  static bit print_matches;
  function void do_field_check(string field, uvm_object obj);
   `ifdef UVM_ENABLE_FIELD_CHECKS
    if  (field_array.exists(field))
      uvm_report_error("MLTFLD", $sformatf("Field %s is defined
multiple times in type '%s'",
          field, obj.get_type_name()), UVM_NONE);
    `endif
    field_array[field] = 1;
  endfunction

  function string get_function_type  (int what);
    case  (what)
      UVM_COPY:    return "copy";
      UVM_COMPARE: return "compare";
```

```
      UVM_PRINT:   return "print";
      UVM_RECORD:  return "record";
      UVM_PACK:    return "pack";
      UVM_UNPACK:  return "unpack";
      UVM_FLAGS:   return "get_flags";
      UVM_SETINT:  return "set";
      UVM_SETOBJ:  return "set_object";
      UVM_SETSTR:  return "set_string";
      default:     return "unknown";
    endcase
  endfunction
  // The scope stack is used for messages that are emitted by policy
classes.
  uvm_scope_stack scope  = new;
  function string get_full_scope_arg  ();
    get_full_scope_arg = scope.get();
  endfunction
  //leak.
  bit             cycle_check[uvm_object];
  //These are the policy objects currently in use. The policy object
gets set
  //when a function starts up. The macros use this.
  uvm_comparer    comparer;
  uvm_packer      packer;
  uvm_recorder    recorder;
  uvm_printer     printer;
endclass

class uvm_scope_stack;
```

```
local string m_arg = "";
local string m_stack[$];

// depth
// -----
function int depth();
  return m_stack.size();
endfunction

// scope
// -----
function string get();
  string v;
  if(m_stack.size() == 0) return m_arg;
  get = m_stack[0];
  for(int i=1; i<m_stack.size(); ++i) begin
    v = m_stack[i];
    if(v != "" && (v[0] == "[" || v[0] == "(" || v[0] == "{"))
      get = {get,v};
    else
      get = {get,".",v};
  end
  if(m_arg != "") begin
    if(get != "")
      get = {get, ".", m_arg};
    else
      get = m_arg;
  end
endfunction
```

```
// scope_arg
// ---------
function string get_arg();
  return m_arg;
endfunction

// set_scope
// ---------
function void set  (string s);
  m_stack.delete();
  m_stack.push_back(s);
  m_arg = "";
endfunction

// down
// ----
function void down  (string s);
  m_stack.push_back(s);
  m_arg = "";
endfunction
// down_element
function void down_element  (int element);
  m_stack.push_back($sformatf("[%0d]",element));
  m_arg = "";
endfunction

// up_element
// ------------
```

```
function void up_element  ();
  string s;
  if(!m_stack.size())
    return;
  s = m_stack.pop_back();
  if(s  != "" && s[0]  != "[")
    m_stack.push_back(s);
endfunction

// up
function void up  (byte separator ="." );
  bit found=0;
  string s;
  while(m_stack.size() &&  !found ) begin
    s = m_stack.pop_back();
    if(separator == ".") begin
      if  (s == ""  ||  (s[0]  != "[" && s[0]  != "(" && s[0]  != "{"))
        found = 1;
    end
    else begin
      if(s  != "" && s[0] == separator)
        found = 1;
    end
  end
  m_arg = "";
endfunction

// set_arg
// -------
```

```
  function void set_arg (string arg);
    if(arg=="") return;
    m_arg = arg;
  endfunction
  // set_arg_element
  // ---------------
  function void set_arg_element (string arg, int ele);
    string tmp_value_str;
    tmp_value_str.itoa(ele);
    m_arg = {arg, "[", tmp_value_str, "]"};
  endfunction

  // unset_arg
  // ---------
  function void unset_arg (string arg);
    if(arg == m_arg)
      m_arg = "";
  endfunction
endclass
```

图 3-1 UVM 中的常用 phase

图 3-2 UVM 中所有的 phase

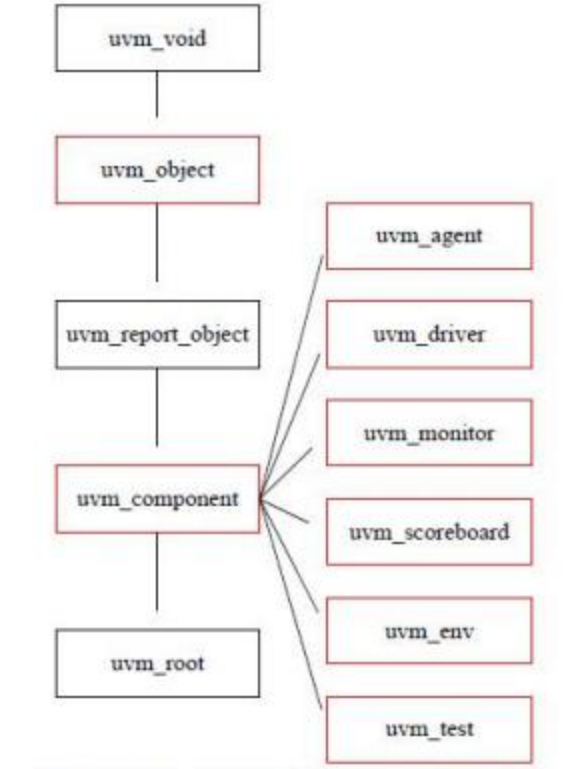


图 10-1 uvm_component 及 uvm_root 类派生图

注意：左边 9 个 phase 是属于 common phase，固定存在，右边 12 个 phase 是属于 uvm phase；在 uvm component 中并没有 run_phase 这个 task,在 uvm 仿真过程中一般不使用 run_phase 这个 task，都是使用右边的 12 个 uvm pahse 的 task,最多的是 main_phase.

1. 为了使用UVM 库，用户需要：

• 编译 UVM 包的顶层文件：uvm_pkg.sv ----这个文件 include 所有 uvm 的文件

• 在所需要的地方导入 uvm_pkg

• 包含 UVM 宏

在 TC 中加入如下语句

```
import uvm_pkg::*;
`include “uvm_macros.svh”
```

---这个文件并不包含在 uvm_pkg 这个包内，所以要单独 include

UVM 在编译时需要加入 uvm_pkg.sv 包，及 uvm_dpi.cc 它们都在安装目录下，同时需要增加-CFLAGS -DVCS 选项

```
UVM_HOME = path/uvm/uvm-1.1d/    #UVM 安装目录
uvm_dir =+incdir+${UVM_HOME}/src+${UVM_HOME}/src/vcs
UVM_COMPILE = ${UVM_HOME}/src/dpi/uvm_dpi.cc\
              ${UVM_HOME}/src/uvm_pkg.sv\
              -CFLAGS\
              -DVCS\
              ${uvm_dir}
```

---因为 uvm_pkg.sv include 的都是相对路径，使用一定要设置uvm 的安装路径

UVM 在运行时需要加入 UVM_TESTNAME= 及 UVM_VERBOSITY= ， seq=

```
SOLVER = 2
plus =
seq = uvm_reg_bit_bash_seq
UVM_RUN = -lca +UVM_TESTNAME=test_case \
```

```
        +ntb_solver_mode=$(SOLVER)\
        +UVM_VERBOSITY=$(verbosity) \
        +${option} +${plus} +seq=${seq}
```

+UVM_TESTNAME=test_case 是因为在 tb 中使用 initial begin run_test(test_case); end 来启动 testcase;

2. uvm 启动：在 tb 中会有 initial run_test();或 run_test("test_case"); end//test_case 为 tc 类；run_test()为 uvm 定义的全局 task，其原型是(uvm_globals.svh)：

```
task run_test(string test_name="");
   uvm_root top;
   top = uvm_root::get();
   top.run_test(test_name);
endtask
```

在 uvm_root(src/base/uvm_root.svh)类中有 run_test(test_name);这样一个 task，其内部有几个关键点如下：

```
task uvm_root::run_test(string test_name="");
  uvm_factory factory= uvm_factory::get();
  bit testname_plusarg;
  int test_name_count;
  string test_names[$];
  string msg;
  uvm_component uvm_test_top;
   ......
     test_name_count =
  clp.get_arg_values("+UVM_TESTNAME=",test_name);//可以解释直接用
  //run_test()+仿真选项+UVM_TESTNAME=test_case；或
  //_test("test_case");两种方法都可以启动 testcase 了。
   ......
  if (test_name != "") begin
```

```
    if(m_children.exists("uvm_test_top")) begin
      uvm_report_fatal("TTINST",
          "An uvm_test_top already exists via a previous call to
run_test", UVM_NONE);
      #0; // forces shutdown because $finish is forked
    end
    $cast(uvm_test_top,
factory.create_component_by_name(test_name,
```

禾口 ```
    "", "uvm_test_top", null)); //通过 factory 模式 create 你
//自己希望运行的 testcase instance，不管你的 testcase 叫什么名字，最
//后 uvm 都会给它重命名为“uvm_test_top”
    if (uvm_test_top == null) begin
      msg = testname_plusarg ? {"command line
+UVM_TESTNAME=",test_name} :
                               {"call to
run_test(",test_name,")"};
      uvm_report_fatal("INVTST",
          {"Requested test from ",msg, " not found." }, UVM_NONE);
    end
  end
  fork begin
      phase_runner_pro = process::self();//得到本进程的句柄
      uvm_phase::m_run_phases(); //关键语句
  end
  join_none;//注意 join_none 的用法,下面的语句会继续与 fork 同时执行
   #0; // let the phase runner start
  wait (m_phase_all_done == 1);
     // clean up after ourselves
   phase_runner_proc.kill();
```
```

```
    report_summarize();
    if (finish_on_completion)
   $finish;
Endtask
```

而在 uvm_phase(src/base/uvm_phase.svh) 类中又有 m_run_phases()这个 task,而 m_run_phases() 又调用了 phase.execute_phase();

```
task uvm_phase::m_run_phases();// (src/base/uvm_phase.svh)
  uvm_root top = uvm_root::get();
  begin
    uvm_phase ph = uvm_domain::get_common_domain();
    void'(m_phase_hopper.try_put(ph));
  end
  forever begin
    uvm_phase phase;
    uvm_process proc;
    m_phase_hopper.get(phase);
    fork
      begin
        proc = new(process::self());
        phase.execute_phase();
      end
    join_none
    m_phase_top_procs[phase] = proc;
    #0;  // let the process start running
  end
endtask
task uvm_phase::execute_phase();//
    uvm_task_phase task_phase;
    uvm_root top;
```

```
    top = uvm_root::get();
    ......
    fork begin
         task_phase.traverse(top.this,UVM_PHASE_EXECUTING);
      end
    join_none
endtask
  virtual function void traverse(uvm_component comp,
                                 uvm_phase phase,
                                 uvm_phase_state state);
  endfunction
  // Provide the required per-component execution flow. Called by
//traverse()
  virtual function void execute(uvm_component comp,
                                uvm_phase phase);
  endfunction
```

uvm_task_phase 类中又有一个关键的 task ; comp.phase_started(phase) ph.execute(comp,phase)//comp 传递进来的是 uvm_root 句柄 top

3. 如何启动 sequence:只需要在某个 componet(如 my_sequencer,my_env)的 main-phase 中启动这个 sequence 即可，这时他们都要在 main_phase 中通过 uvm 的方式实例化 seuence,然后调用 seq.start(i_agt.sqr)来实现（i_agt.sqr 表示送个的对象，如果是在 sequncer 中则为 seq.start()即可）----这时手工启动方式；还一种自动方式启动---default_sqquence 启动，它只需要在某个 component(如 tb_env)的 build_phase 中加入 uvm_config_db#(uvm_object_wrapper)::set(this,"i_agt.sqr.main_phase","default_sequence",my_sequence::type_id::get());采用这种方式 sequence 不用再任何地方实例化

4. 打印层次结构：在 connect之后用函数uvm_top.print_topology ();可以打印出整棵树的结构

5. 在写 TC 时需要导入 `include "uvm_macros.svh" import uvm_pkg::*, ---虽然编译时已经把 uvm_pkg.sv 加入，但是以 package 的形式加入，所以使用时需要 import uvm_pkg::*来导入；`include “uvm_macros.svh”这个可以不要，因为在 uvm_pkg.sv 文件中已经加入了该句，并不在 package 范围内

6. 基本类的写法

```
class trans extends uvm_sequence_item;
    `uvm_object_utils_begin(trans)  //注册进入 factory
     `uvm_object_utils_end;
Endclass
class config extends uvm_object;
    `uvm_object_utils_begin(config)
     `uvm_object_utils_end;
Endclass
class driver extends uvm_driver #(trans) ;
     `uvm_component_utils(driver);
Endclass
class sequence extends uvm_sequence #(trans);
     `uvm_sequence_utils(sequence,sequencer);
Endclass
class sequencer extends uvm_sequencer #(trans);
     `uvm_sequencer_utils(sequencer);
Endclass
class i_agent extends uvm_agent;
     `uvm_component_utils_begin(i_agent)
     `uvm_component_utils_end;
Endclass
```

```
class o_agent extends uvm_sequence_item;
    `uvm_object_utils_begin(o_agent)
    `uvm_object_utils_end;;
Endclass
class ref_mdl extends uvm_component;
    `uvm_component_utils(ref_mdl);;
Endclass
class scorboard extends uvm_scoreboard;
    `uvm_component_utils(scorboard);
Endclass
class env extends uvm_env;
    `uvm_component_utils(env);
Endclass
```

7. uvm_driver 有一个常用的 TLM PORT:seq_item_port,在扩展 driver 类时，经常如下使用

```
task xxx_driver::main_phase(uvm_hase phase)
   i_trans   i_tr;
   super.main_phase(phase);
   while(1) begin
      seq_item_port.get_next_item(i_tr);
      send_data(i_tr);
      seq_item_port.item_done();
   end
endtask
```

uvm driver has 2 TLM ports. 1) Seq_item_port: To get a item from sequencer, driver uses this port. Driver can also send response back using this port. 2) Rsp_port : This can also be used to send response back to sequencer.

8. 在 uvm 中，一般来说，对于 sequenc 和 sequencer 有两种注册方法：

1，sequence： •uvm_object_utils(sequence)

sequencer： •uvm_component_utils(sequencer) 见下面说明 1

2，sequencer： •uvm_sequencer_utils（sequencer）

sequence： •uvm_sequence_utils（sequence，sequencer） 见下面说明 2(常用此方法,但据说不太好，会限制灵活性)

1.every sequencer by default wil have its own sequence library when in the function new(); inside your sequencer use the macro.....`uvm_update_sequence_lib_and_item
all the sequences registered with this sequencer will be updated automatically and the sequencer will know it and update internally in his sequence library

2.First Create a sequencer and register with sequencer utils `uvm_sequencer_utils(sequencer_name)
and now create a sequence and register that sequence with the factory and as well as the sequencer Lib
`uvm_sequence_utils(sequence_name,sequencer_name)

`uvm_sequence_utils(operation_addition, instruction_sequencer),Using the `uvm_sequence_utils macro, register the "operation_addition" sequence with "instruction_sequencer" sequencer.This macro adds the sequence to the sequencer list. This macro will also register the sequence for factory overrides

9. uvm_component 有两大特性，除了上面提到过的在 new 的时候，要指定一个 parent外，另外一个重要的特性就是它具有 phase 自动执行的特性。当整棵树的 build_phase 执行完的时候，会自动执行 connect_phase，以此类推。这样，对于一些有执行顺序要求的语句，可以分别在不同的 phase 完成。这样可以最大程度上避免出错。

uvm_object 是 UVM 中最最基本的类，你能想到的几乎所有的类都是继承自 uvm_object，包括 uvm_component。

uvm_component 有两大特性:

一是通过在new 的时候指定 parent来形成一种树形的组织结构，

二是有 phase 的自动执行特点。

uvm_object 则完全不具有这些概念。所以，你可以想像的出，如果一个类派生自uvm_object，那么这个类的实例不可能以结点的形式出现在 UVM 树上。因为所有的 UVM 树的结点都是由 uvm_component 组成的。

10. 在定义一个类的时候，UVM 强烈推荐使用 uvm_component_utils 或 uvm_object_ utils 宏来把它们注册。通过这两个宏，UVM 就知道我们自己定义一个类，这个类或是派生自uvm_component（使用 uvm_component_utils 注册），要么就是派生自 uvm_object（使用 uvm_object_utils 注册）

那么注册之后有什么用处呢?

```
class A extends uvm_component;
`uvm_component_utils(A)
…
endclass
```

当使用如上代码注册 A 后，那么要创建一个 A 的实例可以这样做：

```
A a;
a = A::type_id::create(“a”, this);
```

11. UVM 验证平台中可以使用 uvm_tlm_analysis_fifo 把 uvm_blocking_get_port 和 uvm_ analysis_port 连接。相当于 mailbox 功能，但机制不一样

```
class uvm_tlm_fifo #(type T=int)
    local mailbox #( T ) m;
    virtual task put( input T t );
        m.put( t );
        put_ap.write( t );
```

```
    endtask


    virtual task get( output T t );
        m_pending_blocked_gets++;
        m.get( t );
        m_pending_blocked_gets- ;
        get_ap.write( t );
    endtask
```

12. 在 UVM 验证平台中使用 virtual interface 来实现 DUT 和验证平台的通信

13. ☆：UVM 验证平台中，各个 component 之间通信使用 TLM 方式，其中要发送信息的一方（如 monitor）一般使用 uvm_analysis_port。

14. 在定义一个派生自 component 的类时，要定义好三个函数(任务)：new，build_phase 和 main_phase。

15. new 函数在定义的时候一般都要写上 super.new(name, parent)这一句，意思就是说执行父类的 new 函数，这里的父类自然是指uvm_driver 了。另外要指定两个参数，一个参数是 string类型的name，就是这个类的实例的名字；一个是 uvm_component 型的 parent，以表明这个 uvm_driver 在 uvm 整个的运行树中处于什么位置。在图 1-4 中，driver 的 parent 就是 In_agent，而 In_agent 的 parent 就是 env。

16. ☆：派生自 uvm_component 的类在其 new 函数中要指明名字（name）和父母（parent），其中 parent 是一个 uvm_component 形的变量，代表此实例在 UVM 树中的父结点。

17. 在类的 build_phase 中，也要写上 super.build_phase(phase)这一句，一般来说，在 build_phase 中要完成类的实例化工作

    Build_phase 是自上而下执行----自上而下为 树图的概念,因为在此过程中,不但实例化各个组件,而且会形成uvm 的树图结构,它们需要有一定的树型关系即,谁是树干,谁是树叶;

    而 connect_phase 是自下而上执行 --- 该过程是一个连接的过程,只有最下面连接好才能向上连接

18. build_phase 中除了要完成实例化之外，还要完成 config_db 机制的 get 行为，即把其它 component 设置给此 component 的一些参数接收过来。如在 my_driver 中就把my_if 给接收了过来（这里把my_if 也当成是一个参数）。

19. 在整个的 UVM 验证平台中，sequence 负责 transaction 的产生，并通过 sequencer 发送给 driver，driver 则根据 transaction 里存储的信息产生激励。要产生不同的 transaction，那么就要在 sequence 中下功夫。验证中要对 DUT 施加以不同的激励，也就是施加不同的 case，在 UVM 中，不同的 case 的差异主要就是体现在 sequence 的不同上。通过控制 sequence，可以完美的产生各种不同的激励。

    注意：uvm_sequence 类没有 build 函数,因为 uvm_sequence 是 object.

20. 通常来说，一个 driver 只能接收一种 transaction，所以在 my_driver 被定义成了一个参数化的类，这个参数就是my_driver 所能接收的 transaction 的类型。

21. ☆：UVM 验证平台中的 agent 实现了 driver，monitor 和 sequencer 的封装，agent 是整个验证平台中唯一直接与 DUT 的物理接口打交道的组件。

    一个 agent 的行为主要是由其封装的 driver，monitor 和 sequencer 实现，所以 agent 的 main_phase 一般不写任何内容。agent 最重要的是其 build_phase 和 connect_phase。

22. uvm_config_db#(config)::set () ;

    uvm_config_db#(config)::get();是一个专门来修改部分变量的在哪里 set 则是谁修改，谁 get 就是修改谁!

23. uvm_config_db#(int)::get(this, ″″, “pre_num_max”, pre_num_max);

24. uvm_config_db 中的 set 和 get 都是静态函数，所以可以用双冒号的形式调用。上面 set 中的第一个参数用以说明是哪个 component 对pre_num_max 进行了设置，一般使用填写 this，第二个参数表示从调用 uvm_config_db::set 的地方看下去，要设置的变量所在的 component 的路径。第三个参数表示一个记号，用以说明这个值是传给 driver 中的哪个

变量的，第四个参数是要设置的值。get 中的第一个参数一般也是 this 即可，第二个参数填写一个空的字符串，第三个参数就是 set 中的第三个参数，这两个参数必须匹配起来，第四个参数则是要设置的变量。这里要说明的是，set和 get 中第三个参数可以跟 get 中第四个参数可以不一样。如第四个参数是pre_num_max，那么第三个参数可以是pre_max，只要保持 set和 get 中第三个参数一致即可。

25. sequence 获得 sequencer 通过

    uvm_config_db#(config)::get(sequencer,"","cfg",m_cfg)

26. driver 通过 uvm_config_db#(virtual xxx_interface)::get(this,"","xxx_intf",xxx_intf) 来得到 interface

    driver 通过 uvm_config_db#(config)::get(this,"","cfg",cfg),来得到配置类

    drive 得到 sequencer 发送过来的 sequence 生成的 tr, 通过 seq_item_port.get_next_item(tr); 得到 。用完后使用 seq_item_port.item_done();来 释放

    drive 通过 uvm_analysis_port #(i_trans) ap2rm ,把 trans 发送给 ref_model;使用方法为 ap2rm.write(i_tr);

27. ref_model 通过 uvm_blocking_get_port #(xxx_i_trans) port;来得到 driver 发送来的 trans,使用方法为：port.get(i_tr);

    ref_model 通过 uvm_analysis_port #(o_trans) ap2scb;来发送 o_trans 给 scoreboard，使用方法为：ap2scb.write(o_tr);

28. monitor 通过 uvm_anlysis_port #(o_trans) ap2scb 来发送 o_trans 给 scoreboard,使用方法为：ap2scb.write(o_tr);

29. i_agent 包含了 sequencer 和 driver ,dut 与 i_agent 交互，i_agent 通过 uvm_analysis_port #(i_trans) 把 trans 送给 ref_module;

30. i_agent 包含了 monitor,o_agent 通过 uvm_analysis_port #(o_trans) 把 o_trans 送给 scoreboard;

31. env 中 通过 uvm_tlm_analysis_fifo #(xxx_x_trans)　来把 o_agent 与 scoreboard,i_agent 与 ref_model,ref_model 与 scoreboard 之间的 port 联接起来，其中xxx_x_trans 表示要传送的类;在使用前需要通过new 函数初始化

使用方法如下：

i_agt.ap2rm.connect(iagt2rm_fifo.analysis_export);//联接到 i_agent 端，ap2rm 是 i_agent 中的一个输出端口

(uvm_anmalysis_port#(i_trans)　ap2rm)

rm.port.connect(iagt2rm_fifo.blocking_get_export);//联接到 ref_model 端，为一个输入端口，与上面组合一对

rm.ap2scb.connect(rm2scb_fifo.analysis_export)

scb.exp_port.connect(rm2scb_fifo.blocking_get_export);

。。。。。。

32. UVM 中使用 connect 函数来建立连接关系。如 A 要和 B 通信（A 是发起者），那么可以这么写：A.connect(B);

但是我们不能写 B.connect(A)。因为在通信的过程中，A 是发起者，B 是被动承担者。这种通信时的主次顺序也适用于连接时，只有发起者才能调用 connect 函数，而被动承担者则做为 connect 的参数。

UVM 的这几种特性非常的实用。事实上，在验证平台上，用的最多的就是 analysis port（analysis export）

33. 如果在编译加了 UVM_TESTNAME=test_case;则测例的名字一定为 class test_case extend uvm_test;　　endclass

34. 那么验证平台又是如何启动的呢？当定义好一个 case 时，我们通常使用如下的方式启动这个 sequence（以 questa 为例）：

vsim +UVM_TESTNAME=my_case0 …

固定的格式

intial begin

uvm_config_db#(virtual xxx_interface)::set(null,"*","xxx_intf",xxx_intf);//设置接口

```
        run_test();//固定启动命令
    end
```

run_test()是 uvm 定义的 task,放在 uvm_pkg 包中

```
task run_test(string test_neme="");
    uvm_root top;
    top = uvm_root::get();
    top.run_test(test_name)
endtask
```

35. UVM 中通过 sequence 产生数据，转交给 sequencer，并最终交给 driver 发送出去。在 sequence 中一般通过 raise_objection 和 drop_objection 来控制仿真平台的关闭。

36. 一个proctected 类型的函数，意味着它要么只能被它本身的成员变量或者成员函数调用，要么被由它派生而来的类的成员变量或者成员函数调用。所以，在外部调用是不可能的

37. 而对于静态函数，并不需要一个类实例化就能调用，而且其调用的方式与非静态函数不一样。非静态函数是用一个点号(.)来调用，而静态函数是用双冒号来调用(::)：

仿真阶段控制方

38. 从高层次来看，已有的阶段方法是（以仿真执行的顺序）：

    new() — 尽管他不是一个 UVM 阶段函数，但是组件的构造函数是组件被创建生成时第一个执行的函数。uvm_component 构造函数包含两个参数：一个字符串name 参数代表组件的名称，以及一个对组件父亲的引用。建议不要使用构造函数参数的缺省值，这样可以保证用户提供有意义的名字和父类。

    build()—第一个 UVM 阶段方法是 build()方法，对每个组件会自顶向下自动调用。此方法可以对子组件进行配置，然后创建子组件。由于构造函数不是完全多态的，因此我们使用此方法来构建在构造函数中没有被创建的子组件。 因为build()方法是自顶向下调用，所以父类的构造配置将在子类 build()方法之前完成。

connect()—此阶段在 build()之后执行。由于所有的子组件在 build()阶段自顶向下都被创建，用户可以认定在 connect()被调用的时候，所有组件的拓扑结构都已经成功创建. 此方法用来做事物级建模(TLM)的连接, 指针引用的赋值，以及虚拟接口(virtual interface)的赋值。

end_of_elaboration()— end_of_elaboration()阶段确保所有的连接，所有的引用都在 connect 阶段完成。在 end_of_elaboration 阶段执行之前，UVM 完成了所有的 TLM 绑定，因此在此阶段，所有的端口绑定都已经建立，可以被访问使用。

start_of_simulation()—此阶段提供了一个机制来避免仿真 0 时刻各个组件之间的互相依赖。组件在此阶段初始化，确保在 run 阶段开始的时候，一切都已经正确初始化。

run()—run()阶段是唯一的一个耗时间的阶段，它定义了组件运行时的主要功能。由于它被定义为任务(task)类型，所以可以消耗仿真时间。当组件从 run 任务返回时，并不代表 run 阶段结束。有可能其内部创建的进程还在继续执行。run 阶段是通过 global_stop_request()任务来完成。此任务在发生超时，或者在对象机制使用的时候调用进程终止方法。

extract()—此阶段用来在仿真结束,，在检查校验阶段之前，提取仿真的结果。此阶段可以用来收集断言错误的个数，覆盖率，以及 DUT 内部讯号和寄存器的值，组件内部变量值，组件的统计信息及其他信息。此阶段执行不耗时间(消耗 0 时间)，是一个函数调用。调用方式是自底向上。(译者补充：从叶子(leaf)组件的 extract 函数开始调用，深度优先)

check()—在前阶段(extract 阶段)得到了重要的仿真结果，check 阶段对结果进行分析判断，以此决定整个仿真的最终结论。此阶段也是一个 0 执行时间的函数调用， 自底向上调用。

report()—此阶段最后执行，用来将结果输出到文件或者屏幕。此阶段是一个 0 执行时间的函数， 自底向上调用。

尽管所有这些钩子方法在uvm_component 以及他的子类中定义，只需要实现用到的相关（阶段）函数/任务（不需要全部实现）. 组件不需要对子组件使用`uvm_field_object 宏，组件在被创建的时候总是被提供其父组件，从而构建了所有组件的层次结构。

39. 有一个特殊的 UVM 组件叫做"uvm_top"。此组件是 uvm_root 类的单一实例，也是由uvm_component 继承而来。uvm_top 组件负责对所有的组件进行阶段控制，并且提供了查找功能（uvm_top.find()和 uvm_top.find_all()），方便定位一些特定的组件。uvm_top 组件还提供了一个存储全局配置的地方。调用 set_config_*()函数相当于调用 uvm_top.set_config_*(). 对消息打印函数也如此，调用 uvm_report_*(或者`uvm_info/warning/error/fatal)等价于调用 uvm_top.uvm_report*函数。
40. callback 翻译为回调，也就是软件编程中回调函数的概念。可以理解为内建一个数组，数组的元素就是函数指针，注册回调函数也就是增加数组元素。在调用的时候，对数组里面每一个元素，都调用其指向的函数。前提是这些函数的原型是一模一样的，也就是返回值和参数是一模一样的。
41. • typedef 来自于 C 语言，用于类型简化声明，这里翻译成类型别名。
42. uvm 打印信息级别设置：

    有 3 种方式来改变冗余级别:

    • 命令行方式(和仿真器相关). 举例如下：

    ```
    % irun … +UVM_VERBOSITY=UVM_NONE
    ```

    • 在 SystemVerilog 中执行命令:

    ```
    set_report_verbosity_level_hier(UVM_MEDIUM);
    ```

    • TCL(运行时)指令:

    ```
    uvm_message uvm_test_top.uart_ctrl_tb.apb0 UVM_FULL
    ```

43. uvm_config_db#(config)::set(this,"*agent.drv","config",cfg);config:传递的类型，*agent.drv:要传送到的目的位置，cfg:类的句柄
44. uvm_config_db#(config)::get(this,"","config",cfg);config:传递的类型，"":由于在本地 get,所以不需要写明，用空白即可，cfg:类的句柄

45. 在 test_case 类的 build 中会固定有

uvm_config_db#(uvm_object_wrapper)::set(this,

"env.i_agt.sqr.main_phase", "default_sequence",

my_sequence::type_id::get());

来启动 sequence;该类型的语法，其含义是通知 env.i_agt.sqr，让其在运行到 main_phase 时自动启动前面定义的my_sequence。这里就是 sequence 的自动启动了。当运行到 main_phase 时，my_sequencer 会自动执行下面的语句：

```
task my_sequencer::main_phase(uvm_phase phase);
    my_sequence my_seq;
    super.main_phase(phase);
    my_seq = new("my_seq");
    my_seq.starting_phase = phase;
    my_seq.start(this);
endtask
```

上面这段代码应该在 uvm_sequencer 这个基类里有，只是“my_seq”这个字符是通过 uvm_config_db#(uvm_object_wrapper)来传递的 -- my_sequence::type_id::get()就会得到 my_seq 这个句柄，所以我们在写 my_sequencer 扩展时，根本不需要写 main_phase();

46. 在 sequence 中有 start 函数来启动，它可以在 env 的 main 中调用 sequence.start 启动，也可以在 sequencer 中的 main 中调用 sequence.start 启动，这两个位置调用都是手动通过 start 函数调用；但实际中，一般使用上述

uvm_config_db#(uvm_object_wrapper)::set(this,

"env.i_agt.sqr.main_phase", "default_sequence",

my_sequence::type_id::get());来启动 sequence；参见《UVM 实战》P46~P50

47. sequence 与 drive 会采用 seq_item_port 同步，在 sequence 中会有 `uvm_send(`uvm_do 包含此动作)，drive 会通过

seq_item_port.get_next_item(tr),获得数据包，当发送完毕后，再通过 seq_item_port.item_done()告诉 sequence 发送完毕，此时 sequence 可以根据实际情况 delay 一些时钟结束此 phase

48. 当把模块验证环境切换到系统验证环境中，注意用控制把 driver 给去掉，因为默认值或复位值会可能会影响 RTL 的行为，尤其是一些采样更会受影响

49. resource_db 机制主要就是用于资源的共享。既然说到共享，那么有三个问题是不得不考虑的：一是资源是存放在什么地方的，比如我们把资源放在一个队列中，也可以放在一个联合数组中，还可以放在一个动态数组中；二是资源是以什么形式存放的，这个问题看起来比较简单，因为我们可以直接存放值就行了，如要共享一个 int 型的资源，那么就直接把这个 int 的值放入一个联合数组就可以了，这个想法固然不错，但是问题在于后面要用到资源的查找等功能，所以仅仅只存放值是不行的；三是资源是如何存取的，即怎么样通过 set 函数把要共享的资源放入要存放的地方，怎么使用 get 函数把资源从存放的地方取出。

50. 在 resource_db 机制中，这三个功能分别通过三个类来实行，实现第一个功能的是 uvm_resource_pool 类，实现第二个功能的是

    uvm_resource#(type T)类，实现第三个功能的是 uvm_resource_db#(type T)类

51. 在 uvm 中可以根据 ralf 文件 ，使用 ralgen 来生成 regmodel,命令如下：ralgen -t syst -uvm system.ralf 把 system.ralf 文件（通过脚本生成的 ralf 文件）内容生成 ral_syst.sv 的文件（我们需要的 reg_model 文件原型）

52. uvm_sequence_item（它扩展自 uvm_transaction,再扩展自 uvm_object）的内建 compare(uvm_object 类里面的)函数，只要 class 里面的元素通过注册进入 factory，就可以通过 xxx_o_tr.compare(yyy_o_tr)来比较 xxx 和 yyy 的 o_tr 所注册的元素

uvm 中 void'(uvm_config_db#(int)::get())的 void'是什么意思

53. 对于有返回值的函数加上void’操作符的意思就是告诉仿真器这个函数虽然有返回值，但是我不需要这个返回值。其实你完全可以不加上void’，

加上这个的唯一好处就在于你可以让仿真器闭嘴：本来函数有返回值，但是你不使用，这时候仿真器会抛出警告。加上 void'可以关闭这个警告，让仿真 log 更干净点而已。

3. 注册机制分析

Factory 工厂的用途:
1. 一个特性类,根据特性产生一个 component 的对象,
2.保存了是否需要重载的特性
Register 的用途
:一个行为类,当需要创建一个类时,需要调用这个类的一个 creat 函数来产生一个对象,并把这个对象的一些特性登记在 factory 中

1 uvm transaction 注册

Transaction 后面使用的 copy,compare,print 函数都是 uvm_object 的函数

```
class my_transaction extends uvm_sequence_item;
            rand bit[47:0] dmac;
            rand bit[47:0] smac;
            rand bit[15:0] ether_type;
            rand byte      pload[];
            rand bit[31:0] crc;
            constraint pload_cons{
            }
            function bit[31:0] calc_crc();
               return 32'h0;
            endfunction
            function void post_randomize();
               crc = calc_crc;
            endfunction
            `uvm_object_utils_begin(my_transaction)
               `uvm_field_int(dmac, UVM_ALL_ON)
```

```
    `uvm_field_int(smac, UVM_ALL_ON)
    `uvm_field_int(ether_type, UVM_ALL_ON)
    `uvm_field_array_int(pload, UVM_ALL_ON)
    `uvm_field_int(crc, UVM_ALL_ON)
  `uvm_object_utils_end
  function new(string name = "my_transaction");
    super.new();
  endfunction
endclass
```

`uvm_object_utils_begin(my_transaction) 原形如下:

```
`define uvm_object_utils_begin(T) \
  `m_uvm_object_registry_internal(T,T)  \
  `m_uvm_object_create_func(T) \
  `m_uvm_get_type_name_func(T) \
  `uvm_field_utils_begin(T)
```

```
`define m_uvm_object_registry_internal(T,S) \
//用来通过字符创建实例
  typedef uvm_object_registry#(T,`"S`") type_id; \
  static function type_id get_type(); \
    return type_id::get(); \
  endfunction \
  virtual function uvm_object_wrapper get_object_type() ; \
    return type_id::get(); \
  endfunction
--- uvm_object_registry 在 uvm_registry.svh 文件中
class uvm_object_registry #(type T=uvm_object, string Tname="<unknown>")
                                    extends uvm_object_wrapper;
---uvm_object_wrapper 在 uvm_factory.svh 文件中
virtual class uvm_object_wrapper;//里面就是 3 个虚函数
  virtual function uvm_object create_object  (string name="");//没有 parent
    return null;
  endfunction
  virtual function uvm_component create_component  (string name,
                                                    uvm_component parent); //需要指明 parent
```

```
    return null;
  endfunction
  pure virtual function string get_type_name() ;//用于扩展，得到 class 的名字
endclass
```

`define m_uvm_object_create_func(T) \ ---实例化一个类,用于 copy 成员,见绿皮书p127 解释

```
    function uvm_object create (string name=""); \
     T tmp; \
     tmp = new() ; \
     if (name!="") \
       tmp.set_name(name); \
     return tmp; \
   endfunction
```

`define m_uvm_get_type_name_func(T) \

```
    const static string type_name = `"T`"; \
   virtual function string get_type_name (); \
     return type_name; \
   endfunction
```

`define uvm_field_utils_begin(T) \

```
    function void __m_uvm_field_automation (uvm_object tmp_data__, \
//object 中的 copy 等函数调用此函数,在 object class 有__m_uvm_field_automation 这样一个虚方法.
                                            int what__, \
                                            string str__); \
   begin \
     T local_data__; /* Used for copy and compare */ \
     typedef T ___local_type____; \
     string string_aa_key; /* Used for associative array lookups */ \
     /* Type is verified by uvm_object::compare() */ \
     super.__m_uvm_field_automation(tmp_data__, what__, str__); \--直接 return
     if(tmp_data__ != null) \
       /* Allow objects in same hierarchy to be copied/compared */ \
       if(!$cast(local_data__, tmp_data__)) return;
```

`uvm_field_int(dmac, UVM_ALL_ON)原型如下:

```
`define uvm_field_int(ARG,FLAG) \
  begin \
    case (what__) \
      UVM_CHECK_FIELDS: \
        begin \
          __m_uvm_status_container.do_field_check(`"ARG`", this); \
        end \
      UVM_COPY: \
        begin \
```

```
          if(local_data__ == null) return; \
          if(!((FLAG)&UVM_NOCOPY)) ARG = local_data__.ARG; \
        end \
      UVM_COMPARE: \
        begin \
          if(local_data__ == null) return; \
          if(!((FLAG)&UVM_NOCOMPARE)) begin \
            if(ARG !== local_data__.ARG) begin \
              void'(__m_uvm_status_container.comparer.compare_field(`"ARG`", ARG,
local_data__.ARG, $bits(ARG))); \
              if(__m_uvm_status_container.comparer.result &&
(__m_uvm_status_container.comparer.show_max <= __m_uvm_status_container.comparer.result))
return; \
            end \
          end \
        end \
      UVM_PACK: \
        if(!((FLAG)&UVM_NOPACK)) begin \
          if($bits(ARG) <= 64) __m_uvm_status_container.packer.pack_field_int(ARG, $bits(ARG));
\
          else __m_uvm_status_container.packer.pack_field(ARG, $bits(ARG)); \
        end \
      UVM_UNPACK: \
        if(!((FLAG)&UVM_NOPACK)) begin \
          if($bits(ARG) <= 64) ARG =
__m_uvm_status_container.packer.unpack_field_int($bits(ARG)); \
          else ARG = __m_uvm_status_container.packer.unpack_field($bits(ARG)); \
        end \
      UVM_RECORD: \
        `m_uvm_record_int(ARG, FLAG) \
      UVM_PRINT: \
        if(!((FLAG)&UVM_NOPRINT)) begin \
          __m_uvm_status_container.printer.print_int(`"ARG`", ARG, $bits(ARG),
uvm_radix_enum'((FLAG)&UVM_RADIX)); \
        end \
      UVM_SETINT: \
        begin \
          bit matched; \
          __m_uvm_status_container.scope.set_arg(`"ARG`"); \
          matched = uvm_is_match(str__, __m_uvm_status_container.scope.get()); \
          if(matched) begin \
            if((FLAG)&UVM_READONLY) begin \
              uvm_report_warning("RDONLY", $sformatf("Readonly argument match %s is ignored",
\
```

```
                __m_uvm_status_container.get_full_scope_arg()), UVM_NONE) ; \
            end \
            else begin \
              if (__m_uvm_status_container.print_matches) \
                  uvm_report_info("STRMTC", {"set_int()", ": Matched string ", str__, " to field
", __m_uvm_status_container.get_full_scope_arg()}, UVM_LOW); \
              ARG = uvm_object::__m_uvm_status_container.bitstream; \
              uvm_object::__m_uvm_status_container.status = 1 ; \
            end \
          end \
          __m_uvm_status_container.scope.unset_arg(`"ARG`") ; \
        end \
    endcase \
  end
```

uvm_object_utils_end 原型如下:

```
`define uvm_object_utils_end \
    end \
  endfunction \
```

object __m_uvm_field_automation 函数如下

```
function void uvm_object::__m_uvm_field_automation (uvm_object tmp_data__,
                                                    int        what__,
                                                    string     str__ ) ;
  return;
endfunction
```

object copy 函数如下

```
uvm_copy_map uvm_global_copy_map = new;
function void uvm_object::copy (uvm_object rhs);
  //For cycle checking
  static int depth;
  if((rhs !=null) && (uvm_global_copy_map.get(rhs) != null)) begin
    return;
  end

  if(rhs==null) begin
    uvm_report_warning("NULLCP", "A null object was supplied to copy; copy is ignored",
UVM_NONE);
    return;
  end

  uvm_global_copy_map.set(rhs, this);
  ++depth;

  __m_uvm_field_automation(rhs, UVM_COPY, "");
```

```
  do_copy (rhs);

  --depth;
  if(depth==0) begin
    uvm_global_copy_map.clear();
  end
endfunction

function void uvm_object::do_copy  (uvm_object rhs);
  return;
endfunction

class uvm_copy_map;
  local uvm_object m_map[uvm_object];
  function void set(uvm_object key, uvm_object obj);
    m_map[key] = obj;
  endfunction
  function uvm_object get(uvm_object key);
    if  (m_map.exists(key))
       return m_map[key];
    return null;
  endfunction
  function void clear();
    m_map.delete();
  endfunction
  function void delete(uvm_object v);
    m_map.delete(v);
  endfunction
endclass
```

do_copy 可以扩展如下

```
// Function: copy
  // The copy makes this object a copy of the specified object.
  // The copy method is not virtual and should not be overloaded in derived
  // classes. To copy the fields of a derived class, that class should override
  // the <do_copy> method.
  // Function: do_copy
  //  // The do_copy method is the user-definable hook called by the copy method.
  // A derived class should override this method to include its fields in a copy
  // operation.
  //
  // A typical implementation is as follows:
  //   class mytype extends uvm_object;
  //     ...
```

```
//     int f1;
//     function void do_copy  (uvm_object rhs);
//       mytype rhs_;
//       super.do_copy (rhs);
//       $cast(rhs_,rhs) ;
//       field_1 = rhs_.field_1;
//     endfunction
//
// The implementation must call ~super.do_copy~, and it must $cast the rhs
// argument to the derived type before copying.
```

```
uvm_copy_map uvm_global_copy_map = new;
class uvm_copy_map;
  local uvm_object m_map[uvm_object];
  function void set(uvm_object key, uvm_object obj);//把对象放到一个关联数组中
    m_map[key] = obj;
  endfunction
  function uvm_object get(uvm_object key);//根据键值,送出内容
    if  (m_map.exists(key))
       return m_map[key];
    return null;
  endfunction
  function void clear() ;//清除关联数组
    m_map.delete();
  endfunction
  function void delete(uvm_object v); //删除键值的关联数组
    m_map.delete(v) ;
  endfunction
endclass
```

在 uvm_object class 中有 do_copy， do_compare， do_print ,do_record， __m_uvm_field_automation function,不过都是空函数—虚函数

2.Component 注册

Component 的 new 函数为 function new (string name, uvm_component parent);
当在使用 AA = TYPE_CLASS::type_id::create("AA");会 new 一个 factory 和一个 register，也就是AA 这个类里面会实例化一个 factory和一个 register,用于记录 AA 的一些信息
一个 component(基类)包含了如下特性

- parent
- children
- 所有 function phase

. 所有 task phase

1.基类包含 parent & children 是因为 uvm 在 run_phase 过程中会根据 create 函数产生的继承关系得到”树关系”(register),使用 fork_join_none 语句让所有组件的 run_phase 同时执行;--此处是一个软件执行过程.

2.基类会有所有的 function & task phase 是因为 uvm 的重载功能是根据面向对象的”多态”特性来实现的(factory)

注册内容包含父类和子类的关系,才能自动执行,uvm环境中是通过从父类找到子类,实现自动执行,如果没注册,就不知道父子类的关系,就无法现实自动执行,具体行为见m_traverse 方法

```
`define uvm_component_utils(T) \
   `m_uvm_component_registry_internal(T,T) \
   `m_uvm_get_type_name_func(T) \
```

```
`define m_uvm_component_registry_internal(T,S) \
   typedef uvm_component_registry #(T,`"S`") type_id ; \   -- uvm_component_registry 是一个类,此类用来根据字符来创建一个句柄,并有一些静态方法(如 creat),注意,这个类命名为 type_id
   static function type_id get_type(); \
     return type_id::get(); \--得到句柄, 此处相当于返回 S
   endfunction \
   virtual function uvm_object_wrapper get_object_type() ; \
     return type_id::get(); \
   endfunction
`define m_uvm_get_type_name_func(T) \ --通过 get_type_name 来得到这个 class 的名字,此处相当于 T
   const static string type_name = `"T`"; \
   virtual function string get_type_name (); \
     return type_name ; \
   endfunction
```

在 uvm_registry.svh 文件中有如下内容:所谓的注册机制就是把把信息放在一个类中--factory,这个类做好了一些 function,用的时候直接调用相关函数

```
class uvm_component_registry #(type T=uvm_component, string Tname="<unknown>")
                                          extends uvm_object_wrapper;
  typedef uvm_component_registry #(T,Tname) this_type ;
  virtual function uvm_component create_component  (string name,
                                                    uvm_component parent) ;
    T obj ;
    obj = new(name, parent) ;
    return obj ;
  endfunction
  const static string type_name = Tname ;
  virtual function string get_type_name() ;
    return type_name ;
```

```
  endfunction
  local static  this_type me = get() ;
  static function this_type get() ; //返回一个对象
    if  (me == null) begin
      uvm_factory f = uvm_factory::get();//创建一个 factory 句柄，此 factory 就是用来保存信息的
      me = new ;  //me 的类型就是 uvm_component_registry
      f.  register(me);
      //把这个类 uvm_component_registry 相关信息写入 factory，是存放在一个叫 m_type_names 的关
      //联数组中，在 factory 中定义为 protected uvm_object_wrapper   m_type_names[string];
    end
    return me;
  endfunction
```

在 factory 中还有一个变量：protected bit m_types[uvm_object_wrapper] ;它是用来指示是否已经登记注册了，用于重载操作标志

protected uvm_factory_override m_type_overrides[$] ;这个队列用于记录重载，uvm_factory_override 这个类用于存储重载的各种信息

```
  static function T create(string name, uvm_component parent, string contxt="");
//产生 2 个类,一个 factor,用于表格记录作用,一个本身的类,得到句柄对象
    uvm_object obj ;
    uvm_factory f = uvm_factory::get();//uvm_factory::get();创建了一个 uvn_factor 句柄并注册进入 factory
    if  (contxt == "" && parent  != null)
      contxt = parent.get_full_name();
    obj = f.create_component_by_type(get(),contxt,name,parent);//创建本身的一个对象,类型为
//uvm_object, component 扩展于 uvm_object; get()到 uvm_component_registry 类型
    if  (!$cast(create, obj)) begin
      string msg ;
      msg = {"Factory did not return a component of type '",type_name,
        "'. A component of type '",obj == null ? "null" : obj.get_type_name(),
        "' was returned instead. Name=",name," Parent=",
        parent==null?"null":parent.get_type_name()," contxt=",contxt};
      uvm_report_fatal("FCTTYP", msg, UVM_NONE) ;
    end
  endfunction
  static function void set_type_override  (uvm_object_wrapper override_type,
                                           bit replace=1);
    factory.set_type_override_by_type(get(),override_type,replace);
  endfunction

  static function void set_inst_override(uvm_object_wrapper override_type,
                                         string inst_path,
                                         uvm_component parent=null);
endclass
```

```
class uvm_factory;
  extern `_protected function new ();
  extern static function uvm_factory get();
  extern function void register (uvm_object_wrapper obj);
  extern function
      void set_inst_override_by_type (uvm_object_wrapper original_type,
                                      uvm_object_wrapper override_type,
                                      string full_inst_path);

  extern function
      void set_inst_override_by_name (string original_type_name,
                                      string override_type_name,
                                      string full_inst_path);
  extern function
      void set_type_override_by_type (uvm_object_wrapper original_type,
                                      uvm_object_wrapper override_type,
                                      bit replace=1);
  extern function
      void set_type_override_by_name (string original_type_name,
                                      string override_type_name,
                                      bit replace=1);
  extern function
      uvm_object    create_object_by_type    (uvm_object_wrapper requested_type,
                                              string parent_inst_path="",
                                              string name="");
   extern function
      uvm_component create_component_by_type (uvm_object_wrapper requested_type,
                                              string parent_inst_path="",
                                              string name,
                                              uvm_component parent);
  extern function
      uvm_object    create_object_by_name    (string requested_type_name,
                                              string parent_inst_path="",
                                              string name="");
  extern function
      uvm_component create_component_by_name (string requested_type_name,
                                              string parent_inst_path="",
                                              string name,
                                              uvm_component parent);
  extern function
      void debug_create_by_type (uvm_object_wrapper requested_type,
                                 string parent_inst_path="",
                                 string name="");
  extern function
```

```
    void debug_create_by_name  (string requested_type_name,
                                string parent_inst_path="",
                                string name="");
  extern function
    uvm_object_wrapper find_override_by_type  (uvm_object_wrapper requested_type,
                                               string full_inst_path) ;
  extern function
    uvm_object_wrapper find_override_by_name  (string requested_type_name,
                                               string full_inst_path) ;
  extern
    function uvm_object_wrapper find_by_name             (string type_name);
  extern function void print  (int all_types=1);
  extern protected
    function void  m_debug_create  (string requested_type_name,
                                    uvm_object_wrapper requested_type,
                                    string parent_inst_path,
                                    string name) ;
    extern protected
    function void  m_debug_display (string requested_type_name,
                                    uvm_object_wrapper result,
                                    string full_inst_path) ;
  static local uvm_factory m_inst;
  protected bit                  m_types[uvm_object_wrapper] ;
  protected bit                  m_lookup_strs[string] ;
  protected uvm_object_wrapper   m_type_names[string];
  protected uvm_factory_override m_type_overrides[$] ;
  protected uvm_factory_queue_class m_inst_override_queues[uvm_object_wrapper];
  protected uvm_factory_queue_class m_inst_override_name_queues[string] ;
  protected uvm_factory_override    m_wildcard_inst_overrides[$];
  local uvm_factory_override     m_override_info[$];
  local static bit m_debug_pass;
  extern function bit m_has_wildcard(string nm) ;
  extern function bit check_inst_override_exists
                                    (uvm_object_wrapper original_type,
                                     uvm_object_wrapper override_type,
                                     string full_inst_path) ;
endclass
const uvm_factory factory = uvm_factory::get();
function uvm_factory uvm_factory::get();
  if  (m_inst == null) begin
    m_inst = new() ;
  end
  return m_inst;
endfunction
```

```
class uvm_factory_override;
  string full_inst_path;
  string orig_type_name;
  string ovrd_type_name;
  bit selected;
  uvm_object_wrapper orig_type ;
  uvm_object_wrapper ovrd_type ;
  function new (string full_inst_path="",
                string orig_type_name="",
                uvm_object_wrapper orig_type=null,
                uvm_object_wrapper ovrd_type);
  if (ovrd_type == null) begin
      uvm_report_fatal ("NULLWR", "Attempting to register a null override object with the
factory", UVM_NONE) ;
    end
    this.full_inst_path= full_inst_path;
    this.orig_type_name = orig_type == null ? orig_type_name : orig_type.get_type_name() ;
    this.orig_type      = orig_type ;
    this.ovrd_type_name = ovrd_type.get_type_name() ;
    this.ovrd_type      = ovrd_type ;
  endfunction
endclass

function uvm_component uvm_factory::create_component_by_type (uvm_object_wrapper
requested_type,
                                                               string parent_inst_path="",
                                                               string name,
                                                               uvm_component parent) ;
  string full_inst_path;
  if (parent_inst_path == "")
    full_inst_path = name;
  else if (name != "")
    full_inst_path = {parent_inst_path,".",name} ;
  else
    full_inst_path = parent_inst_path;
  m_override_info.delete();
  requested_type = find_override_by_type(requested_type, full_inst_path) ;
// find_override_by_type(requested_type, full_inst_path) ;返回的就是 uvm_component_registry,
    //并判断是否重载,如果重载了就使用重载后的类
  return requested_type.create_component(name, parent) ;
//就是调用 registry. create_component(name, parent)
Endfunction
```

```
class uvm_component_registry #(type T=uvm_component, string Tname="<unknown>")
                                        extends uvm_object_wrapper;
  typedef uvm_component_registry #(T,Tname) this_type;
  // Function: create_component
  // Creates a component of type T having the provided ~name~ and ~parent~.
  // This is an override of the method in <uvm_object_wrapper>. It is
  // called by the factory after determining the type of object to create.
  // You should not call this method directly. Call <create> instead.
  virtual function uvm_component create_component (string name,
                                                   uvm_component parent);
    T obj;
    obj = new(name, parent);
    return obj;
  endfunction
.....
endclass
```

总结:

3. TLM 通信—FIFO 分析

3.1 uvm_tlm_analysis_fifo 分析

通过 uvm_tlm_analysis_fifo 在两个 component 之间通信,其实就是通过 mailbox 的方式来处理,只是封装了一层.

```
class uvm_tlm_fifo #(type T=int) extends uvm_tlm_fifo_base #(T);
  const static string type_name = "uvm_tlm_fifo #(T)";
  local mailbox #( T ) m;
  local  int m_size;
  protected int m_pending_blocked_gets;
  // Function: new
  // The ~name~ and ~parent~ are the normal uvm_component constructor
arguments.
  // The ~parent~ should be null if the <uvm_tlm_fifo> is going to be used
in a
  // statically elaborated construct (e.g., a module). The ~size~
indicates the
  // maximum size of the FIFO; a value of zero indicates no upper bound.

  function new(string name, uvm_component parent = null, int size = 1);
    super.new(name, parent);
    m = new( size );
    m_size = size;
  endfunction
```

```
  virtual function string get_type_name();
    return type_name;
  endfunction

  // Function: size
  //
  // Returns the capacity of the FIFO-- that is, the number of entries
  // the FIFO is capable of holding. A return value of 0 indicates the
  // FIFO capacity has no limit.

  virtual function int size();
    return m_size;
  endfunction

  // Function: used
  //
  // Returns the number of entries put into the FIFO.

  virtual function int used();
    return m.num();
  endfunction

  // Function: is_empty
  //
  // Returns 1 when there are no entries in the FIFO, 0 otherwise.

  virtual function bit is_empty();
    return (m.num() == 0);
  endfunction

  // Function: is_full
  //
  // Returns 1 when the number of entries in the FIFO is equal to its
<size>,
  // 0 otherwise.
  virtual function bit is_full();
    return (m_size != 0) && (m.num() == m_size);
  endfunction

  virtual task put( input T t );
    m.put( t );
    put_ap.write( t );
  endtask
```

```
virtual task get( output T t );
  m_pending_blocked_gets++;
  m.get( t );
  m_pending_blocked_gets--;
  get_ap.write( t );
endtask

virtual task peek( output T t );
  m.peek( t );
endtask

virtual function bit try_get( output T t );
  if( !m.try_get( t ) ) begin
    return 0;
  end

  get_ap.write( t );
  return 1;
endfunction

virtual function bit try_peek( output T t );
  if( !m.try_peek( t ) ) begin
    return 0;
  end
  return 1;
endfunction

virtual function bit try_put( input T t );
  if( !m.try_put( t ) ) begin
    return 0;
  end

  put_ap.write( t );
  return 1;
endfunction

virtual function bit can_put();
  return m_size == 0 || m.num() < m_size;
endfunction

virtual function bit can_get();
  return m.num() > 0 && m_pending_blocked_gets == 0;
endfunction
```

```
  virtual function bit can_peek();
    return m.num() > 0;
  endfunction


  // Function: flush
  //
  // Removes all entries from the FIFO, after which <used> returns 0
  // and <is_empty> returns 1.

  virtual function void flush();
    T t;
    bit r;

    r = 1;
    while( r ) r = try_get( t ) ;

    if( m.num() > 0 && m_pending_blocked_gets != 0 ) begin
      uvm_report_error("flush failed" ,
              "there are blocked gets preventing the flush", UVM_NONE);
    end

  endfunction

endclass
```

上面 class 的方法都是mailbox 的操作方法.

```
class uvm_tlm_analysis_fifo #(type T = int) extends uvm_tlm_fifo #(T);.

  function new(string name_, __uvm_component_parent_ = null);
    super.new(name, parent, 0); // analysis fifo must be unbounded
    analysis_export = new("analysis_export", this);
  endfunction

  const static string type_name = "uvm_tlm_analysis_fifo #(T)";

  virtual function string get_type_name();
    return type_name;
  endfunction

  function void write(input T t);
    void'(this.try_put(t)); // unbounded => must succeed
  endfunction
```

```
// try_put—mailbox 的方法
Endclass
```

3.2 blocking_get_export 分析

```
class uvm_blocking_get_export #(type T=int)
            extends uvm_port_base #(uvm_tlm_if_base #(T,T));

`UVM_EXPORT_COMMON(`UVM_TLM_BLOCKING_GET_MASK,"uvm_blocking_get_expor
t")
  `UVM_BLOCKING_GET_IMP  (this.m_if, T, t)
Endclass

virtual class uvm_port_base #(type IF=uvm_void) extends IF;
    ……
endclass

`define UVM_TLM_BLOCKING_GET_MASK禾口     (1<<1)

`define UVM_EXPORT_COMMON(MASK,TYPE_NAME) \
  function new  (string name, uvm_component parent, \
                 int min_size=1, int max_size=1); \
    super.new  (name, parent, UVM_EXPORT, min_size, max_size); \
    m_if_mask = MASK; \
  endfunction \
  `UVM_TLM_GET_TYPE_NAME(TYPE_NAME)

`define UVM_TLM_GET_TYPE_NAME(NAME) \
  virtual function string get_type_name(); \
    return NAME; \
  endfunction
```

uvm_tlm_ if_base 类和 uvm_sqr_ if_base 类定义端口的访问方法。 默认情况下，如果调用每个方法都会发出错误消息。 每个端口都将覆盖下面列出的访问方法的子集。 例如，uvm_blocking_put_port 将覆盖 put 方法（仅）。 这意味着如果你调用 put之外的方法，你会得到在 uvm_tlm_ if_base 类中定义的错误消息

Class \ Method	put	try_ put	can_ put	get	try_ get	can_ get	peek	try_ peek	can_ peek	transport	nb_ transport	write
uvm_blocking_ put_port	X											
uvm_nonblocking_ put_port		X	X									

Class \ Method	put	try_put	can_put	get	try_get	can_get	peek	try_peek	can_peek	transport	nb_transport	write
uvm_put_port	X	X	X									
uvm_blocking_get_port				X								
uvm_nonblocking_get_port					X	X						
uvm_get_port				X	X	X						
uvm_blocking_peek_port							X					
uvm_nonblocking_peek_port								X	X			
uvm_peek_port							X	X	X			
uvm_blocking_get_peek_port				X			X					
uvm_nonblocking_get_peek_port					X	X		X	X			
uvm_get_peek_port				X	X	X	X	X	X			
uvm_blocking_master_port	X			X			X					
uvm_nonblocking_master_port		X	X		X	X		X	X			
uvm_master_port	X	X	X	X	X	X	X	X	X			
uvm_blocking_slave_port	X			X			X					
uvm_nonblocking_slave_port		X	X		X	X		X	X			
uvm_slave_port	X	X	X	X	X	X	X	X	X			
uvm_blocking_transport_port										X		
uvm_nonblocking_transport_port											X	
uvm_transport_port										X	X	

Class \ Method	put	try_put	can_put	get	try_get	can_get	peek	try_peek	can_peek	transport	nb_transport	write
uvm_analysis_port												

3.3 TLM 的通信

首先是 connect,只能有下面几种 connection type:
其次要理解控制流和数据流的方向。
控制流： 控制流永远是 port 到 export 或 imp port 的。也就是说，永远去是 port 作为数据通信的发起者去 call put/get 等 function
数据流：数据流有可能是从 port 到 export/imp port，也有可能从 export/imp port 到 port.
怎么理解呢，如下对比：
component A 有 uvm_put_port，component B 有 uvm_put_export，当 A call put task 时，数据从 A 传递到 B
component A 有 uvm_get_port，component B 有 uvm_get_export，当 A call get task 时，数据从 B 传递到 A
而不管是 put 还是 get task，都会在 component B 中进行定义。有点像是 A 去调了 B 的 task。所以总结起来就是 port 的 task/function 都要落到 B 的 task/function 上去实现
最后一定注意两件事情：
connect 两边的 tlm port 的 type 以及 transaction type 一定要一致，比如 blocking_put_port 要连接 blocking_put_export 或 put_export。
更需要注意，一定要使用 IMP 来终结连接关系， PORT 和 EXPORT 都不能做为连接关系 的终点
analysis port，analysis export， analysis imp
其实实际运用上，还是最常用 analysis port/imp/export（它们有点像是 port/export/imp 的增强版，就个人经验来讲完全已经足够了)。主要有以下区别：
一个 analysis port 可以连接多个 analysis export/imp, 也就是说 analysis port 更像一个广播。
analysis port 只有一个 write function。
analysis port 没有阻塞非阻塞的区分，它本身只是广播，不必等待其它相连的 port 的响应。
既然是广播，那么就有可能一个 analysis port 连接到两个以上的 imp，但是 write function 只有一个，我们怎么办呢？
处理方案：
1.通过宏 uvm_analysis_imp_decl 声明两个后缀不一样的 imp，UVM 会根据两个后缀内建两个新的 imp
`uvm_analysis_imp_decl(_export1)
`uvm_analysis_imp_decl(_export2)
2.声明 port

```
uvm_analysis_imp_export1 #(transaction, scoreboard)    scb_imp1;
uvm_analysis_imp_export2#(transaction, scoreboard)     scb_imp2;
```

3.分别定义两个 port 的 write function

```
function void write_export1(transaction txn);
function void write_export1(transaction txn);
```

特别要注意的一点：

analysis port 广播出去的是 handle，并没有为每一个 export/imp 的 transaction 分配 alloction，如果 export1 对 transaction 进行了修改， 那么 export2 会看到修改后的 transaction.所以好的习惯是在 write function 里面首先 new 一个 transaction，并对传输过来的 transaction 进行 copy 后，再进行后面的操作。

uvm_tlm_fifo,uvm_tlm_analysis_fifo

顾名思义，uvm_tlm_fifo 其实就是能够在 tlm 肚子里面 buffer 很多 transaction，就像 FIFO 一样。

有人可能会这样做，我自己在 scoreboard 里面申明 queue，也一样做到 fifo 的效果。确实，很多情况下，直接把 analysis port 传过来的 transaction 放 queueu 里面，是一个不错的方法。但 fifo 还有以下几个好处：

1. uvm_tlm_fifo 是被动接收数据和被动发送数据，因此可以把发起端和接收端完全分隔开，发数据和收数据的行为完全独立。

2.如果 design 的 transaction 有 outstanding，那么 uvm_tlm_fifo 就直接可以 model design 的 fifo 行为了（当然用 queue 做也不错，甚至碰到 out of order 的时候比 fifo 还好用）。

3.通过 imp 加后缀可以解决多对 port,export 的情况，但如果 port 数据多了呢。比如一个有 16 进 16 出的 AXI interconnect，在 scoreboard 的 port 上去申明 16 个不同名字的 imp 显然不方便。那么这时候只需要申明一个uvm_tlm_fifo 数组就可以了。（IMP 没办法申明数组）

example:

scoreboard:

```
 uvm_tlm_analysis_export  (#sb_item)      in_expor[10];
 uvm_tlm_analysis_fifo(#sb_item)       scb_fifo[10];
 uvm_blocking_get_port #(sb_item)      fecth_port[10];
  ...
  function void connect_phase
    foreach(in_export[i]) begin
      ip_export[i].connect(sb_fifo[i].analysis_export);
     fetch_port[i].connect(sb_fifo[i].blocking_get_export);
    end
  endfunction
```

env:

```
  function void connect_phase
   foreach(mon[i])
    mon[i].op.connect(scb.in_expor[i]);//
```

为什么要有一个 in_export 作桥梁，而不直接用 mon.op 连到 scb 的 fifo 呢。可以参考 uvm_users_guide 2.3.2，这里不作详细说明

endfunction

在 soreboard 中，不必再去写一个 write function 和 get task 了，因为 fifo 里面已经做了。mon.ap.write 直接把数据放到 fifo 里面，而 fetch_port.get 直接出 fifo 头 拿数据。

4.Uvm_root::run_test task 分析

4.1 基本 task 分析

全局函数 run_test

```
task run_test (string test_name="");
  uvm_root top;
  top = uvm_root::get();
  top.run_test(test_name);
endtask
task uvm_root::run_test(string test_name="");

  uvm_factory factory= uvm_factory::get();//get factory 对象
  bit testname_plusarg;
  int test_name_count;
  string test_names[$];
  string msg;
  uvm_component uvm_test_top;//所有的 testcase 都例化为 uvm_test_top

process  phase_runner_proc; //store thread forked below for final
//cleanup

  testname_plusarg = 0;

  // Set up the process that decouples the thread that drops objections
from
  // the process that processes drop/all_dropped objections. Thus, if the
  // original calling thread (the "dropper") gets killed, it does not
affect
  // drain-time and propagation of the drop up the hierarchy.
  // Needs to be done in run_test since it needs to be in an
  // initial block to fork a process.
  uvm_objection::m_init_objections();

`ifndef UVM_NO_DPI
```

```
  // Retrieve the test names provided on the command line.  Command line
  // overrides the argument.通过命令行得到 testcase 名字,名字放在
//test_names 变量中
   test_name_count = clp.get_arg_values("+UVM_TESTNAME=", test_names);

  // If at least one, use first in queue.
  if (test_name_count > 0) begin
    test_name = test_names[0];
    testname_plusarg = 1;
  end

  // If multiple, provided the warning giving the number, which one will
be
  // used and the complete list.
  if (test_name_count > 1) begin
    string test_list;
    string sep;
    for (int i = 0; i < test_names.size(); i++) begin
      if (i != 0)
        sep = ", ";
      test_list = {test_list, sep, test_names[i]};
    end
    uvm_report_warning("MULTTST",
      $sformatf("Multiple (%0d) +UVM_TESTNAME arguments provided on the
command line.  '%s' will be used.  Provided list: %s.", test_name_count,
test_name, test_list), UVM_NONE);
  end

`else

     // plusarg overrides argument
  if ($value$plusargs("UVM_TESTNAME=%s", test_name)) begin
    `uvm_info("NO_DPI_TSTNAME", "UVM_NO_DPI defined--getting
UVM_TESTNAME directly, without DPI", UVM_NONE)
    testname_plusarg = 1;
  end

`endif
//上面的内容就是得到 testcase 这个 class
  // if test now defined, create it using common factory
  if (test_name != "") begin
    if(m_children.exists("uvm_test_top")) begin
      uvm_report_fatal("TTINST",
```

```
          "An uvm_test_top already exists via a previous call to
run_test", UVM_NONE);
      #0; // forces shutdown because $finish is forked
    end
    $cast(uvm_test_top, factory.create_component_by_name(test_name,
          "", "uvm_test_top", null));
//此处实现把任何 testcase 都例化成 uvm_test_top

    if (uvm_test_top == null) begin
      msg = testname_plusarg ? {"command line
+UVM_TESTNAME=",test_name} :{"call to run_test(",test_name,")"};
      uvm_report_fatal("INVTST",
          {"Requested test from ",msg, " not found." }, UVM_NONE);
    end
  end

  if (m_children.num() == 0) begin
    uvm_report_fatal("NOCOMP",
          {"No components instantiated. You must either instantiate",
           " at least one component before calling run_test or use",
           " run_test to do so. To run a test using run_test,",
           " use +UVM_TESTNAME or supply the test name in",
           " the argument to run_test(). Exiting simulation."},
UVM_NONE);
    return;
  end

  uvm_report_info("RNTST", {"Running test ",test_name, "..."},
UVM_LOW);

  // phase runner, isolated from calling process ,下面的 fork
//join_none 是主体
  fork begin
    // spawn the phase runner task
    phase_runner_proc = process::self(); //process 系统内置函数，此处
控制本进程
和uvm_phase::m_run_phases();//内部有一个 forever 语句
  end
  join_none
  #0; // let the phase runner start

  wait (m_phase_all_done == 1);// m_phase_all_done 是一个 bit 变量
  // uvm_phase.svh: top.m_phase_all_done=1,uvm_phase 的 execute_phase
     task 会赋值给它,top 是一个 root
```

```
  // clean up after ourselves
  phase_runner_proc.kill();//杀死本进程，run_test 结束

  report_summarize();//产生结果报告

  if (finish_on_completion)
    $finish;
Endtask
```

m_run_phases

Uvm_phase.svh 有 m_run_phases 这样一个静态 task

```
task uvm_phase::m_run_phases();
  uvm_root top = uvm_root::get();
  // initiate by starting first phase in common domain
  begin
    uvm_phase ph = uvm_domain::get_common_domain();
    // uvm_domain 扩展于 uvm_phase, get_common_domain()会把那 9 个 phase
    //组成结构图;
    void'(m_phase_hopper.try_put(ph));
    // m_phase_hopper 是一个 mailbox,
  end

  forever begin   //循环得到各个 phase，并执行，直到结束
    uvm_phase phase;//每循环一次声明一个句柄
    uvm_process proc;
    m_phase_hopper.get(phase);//从 execute_phase get phase,第一次得到
                              //是 common_domain 这个 uvm_phase,它得到
                              //是 9 个 phase
    fork
和begin
      proc = new(process::self());
      phase.execute_phase();//在 uvm_phase 有这样一个 task,和
        //execute_phase 有 wait 语句,不同的 phase, execute_phase
        //的行为不一样,由于是在 forever语句中,使用相当于很多个
        //phase.execute_phase()并行运行
    end
    join_none
    m_phase_top_procs[phase] = proc;//把目前进程的句柄传递出去
    #0;  // let the process start running
  End//  end forever
Endtask
```

Uvm 的 9 个 phase 就是在此处得到.

```
  static function uvm_domain get_common_domain() ;
    uvm_domain domain;
    uvm_phase schedule;
    if  (m_common_domain  != null)
      return m_common_domain;
    domain = new("common");
    domain.add(uvm_build_phase::get());
    domain.add(uvm_connect_phase::get());
    domain.add(uvm_end_of_elaboration_phase::get());
    domain.add(uvm_start_of_simulation_phase::get());
    domain.add(uvm_run_phase::get());
    domain.add(uvm_extract_phase::get());
    domain.add(uvm_check_phase::get());
    domain.add(uvm_report_phase::get());
    domain.add(uvm_final_phase::get());
    m_domains["common"] = domain;

    // for backward compatibility, make common phases visible;
    // same as uvm_<name>_phase::get().
    build_ph                = domain.find(uvm_build_phase::get());
    connect_ph              = domain.find(uvm_connect_phase::get());
    end_of_elaboration_ph   =
                domain.find(uvm_end_of_elaboration_phase::get());
    start_of_simulation_ph =
                domain.find(uvm_start_of_simulation_phase::get());
    run_ph                  = domain.find(uvm_run_phase::get());
    extract_ph              = domain.find(uvm_extract_phase::get());
    check_ph                = domain.find(uvm_check_phase::get());
    report_ph               = domain.find(uvm_report_phase::get());
    m_common_domain = domain;

    domain = get_uvm_domain();
    m_common_domain.add(domain,
                        .with_phase(m_common_domain.find(uvm_run_phase::
get())));
    return m_common_domain;
  endfunction

  // Function: add_uvm_phases
  //
  // Appends to the given ~schedule~ the built-in UVM phases.
  //
  static function void add_uvm_phases(uvm_phase schedule);
```

```
    schedule.add(uvm_pre_reset_phase::get());
    schedule.add(uvm_reset_phase::get());
    schedule.add(uvm_post_reset_phase::get());
    schedule.add(uvm_pre_configure_phase::get());
    schedule.add(uvm_configure_phase::get());
    schedule.add(uvm_post_configure_phase::get());
    schedule.add(uvm_pre_main_phase::get());
    schedule.add(uvm_main_phase::get());
    schedule.add(uvm_post_main_phase::get());
    schedule.add(uvm_pre_shutdown_phase::get());
    schedule.add(uvm_shutdown_phase::get());
    schedule.add(uvm_post_shutdown_phase::get());
  endfunction
  // Function: get_uvm_domain
  // Get a handle to the singleton ~uvm~ domain
  //
  static function uvm_domain get_uvm_domain();
    if (m_uvm_domain == null) begin
      m_uvm_domain = new("uvm");
      m_uvm_schedule = new("uvm_sched", UVM_PHASE_SCHEDULE);
      add_uvm_phases(m_uvm_schedule);
      m_uvm_domain.add(m_uvm_schedule);
    end
    return m_uvm_domain;
  endfunction
```

execute_phase

```
task uvm_phase::execute_phase();
  uvm_task_phase task_phase;
  uvm_root top;
  top = uvm_root::get();
  // If we got here by jumping forward, we must wait for
  // all its predecessor nodes to be marked DONE.
  // (the next conditional speeds this up)
  // Also, this helps us fast-forward through terminal (end) nodes
  foreach (m_predecessors[pred])// predecessors:前辈,前任
// uvm_phase 中有这样一个变量 protected bit  m_predecessors[uvm_phase];
    wait (pred.m_state == UVM_PHASE_DONE);
    /* typedef enum { UVM_PHASE_IMP,
                      UVM_PHASE_NODE,
                      UVM_PHASE_TERMINAL,
```

```
                    UVM_PHASE_SCHEDULE,
                    UVM_PHASE_DOMAIN,
                    UVM_PHASE_GLOBAL
  } uvm_phase_type;
  typedef enum { UVM_PHASE_DORMANT      = 1,
                 UVM_PHASE_SCHEDULED    = 2,
                 UVM_PHASE_SYNCING      = 4,
                 UVM_PHASE_STARTED      = 8,
                 UVM_PHASE_EXECUTING    = 16,
                 UVM_PHASE_READY_TO_END = 32,
                 UVM_PHASE_ENDED        = 64,
                 UVM_PHASE_CLEANUP      = 128,
                 UVM_PHASE_DONE         = 256,
                 UVM_PHASE_JUMPING      = 512
                 } uvm_phase_state;
  */
// If DONE (by, say, a forward jump), return immed
if (m_state == UVM_PHASE_DONE)
  return;
// SYNCING:
// Wait for phases with which we have a sync()
// relationship to be ready. Sync can be 2-way -
// this additional state avoids deadlock.
if (m_sync.size()) begin
  m_state = UVM_PHASE_SYNCING;
  foreach (m_sync[i]) begin
    wait (m_sync[i].m_state >= UVM_PHASE_SYNCING);
  end
end
m_run_count++;
 // If we're a schedule or domain, then "fake" execution
if (m_phase_type != UVM_PHASE_NODE) begin
  m_state = UVM_PHASE_STARTED;
  #0;
  m_state = UVM_PHASE_EXECUTING;
  #0;
end
else begin // PHASE NODE
  // STARTED:
  m_state = UVM_PHASE_STARTED;
  m_imp.traverse(top,this,UVM_PHASE_STARTED);
  #0; // LET ANY WAITERS WAKE UP
  //if (m_imp.get_phase_type() != UVM_PHASE_TASK) begin
  if (!$cast(task_phase,m_imp)) begin// m_imp 是 uvm_phase
```

```
      //_EXECUTING:_(function_phases)
和  m_state = UVM_PHASE_EXECUTING;
      #0; // LET ANY WAITERS WAKE UP
      m_imp.traverse(top,this,UVM_PHASE_EXECUTING);
和end
    else_begin//_EXECUTING:_(task_phases)
        fork : master_phase_process
          begin
              m_phase_proc = process::self();
                  // EXECUTING: (task phases)
              m_state = UVM_PHASE_EXECUTING;
和和  task_phase_traverse(top,this,UVM_PHASE_EXECUTING);
和和  wait(0); // stay alive for later kill
          end
        join_none
          uvm_wait_for_nba_region();//no blocking active=nba
           //Give sequences, etc. a chance to object
          // Now wait for one of three criterion for end-of-phase.
        fork
        begin // guard
          do begin
           // if looped back from READY_TO_END, change state
           m_state = UVM_PHASE_EXECUTING;
           #0;
           fork
             // WAIT_FOR_ALL_DROPPED
             begin
         // OVM semantic: don't end until objection raised or stop request
               if (phase_done.get_objection_total(top) ||
                   m_use_ovm_run_semantic && m_imp.get_name() == "run")
               begin
                 if (!phase_done.m_top_all_dropped)
                   phase_done.wait_for(UVM_ALL_DROPPED, top);
                 `UVM_PH_TRACE("PH/TRC/EXE/ALLDROP","PHASE EXIT
ALL_DROPPED",this,UVM_DEBUG)
               end
               else begin
                  if (m_phase_trace)
                    `UVM_PH_TRACE("PH/TRC/SKIP","No objections raised,
skipping phase",this,UVM_LOW)
               end
             end
             // TIMEOUT
             begin
```

```
                if  (top.phase_timeout == 0)
                  wait(top.phase_timeout  != 0);
                `uvm_delay (top.phase_timeout)
                if  ($time == `UVM_DEFAULT_TIMEOUT) begin
                  `uvm_error("PH_TIMEOUT",
                     $sformatf("Default phase timeout of %0t hit. All
processes are waiting, indicating a probable testbench issue. Phase '%0s'
ready  to  end",
                               top.phase_timeout, get_name()))
                end
                else begin
                  `uvm_error("PH_TIMEOUT",
                     $sformatf("Phase timeout of %0t hit, phase '%0s'
ready  to  end",
                               top.phase_timeout, get_name()))
                end
                phase_done.clear(this);
                `UVM_PH_TRACE("PH/TRC/EXE/3","PHASE EXIT
TIMEOUT",this,UVM_DEBUG)
              end
            join_any
            disable fork;
            phase_done.clear(this);
            // If jump is pending, do not allow prolonging of phase
            if(!m_jump_fwd && !m_jump_bkwd) begin
              //--------------
              // READY_TO_END:
              //--------------
              `UVM_PH_TRACE("PH_READY_TO_END","PHASE READY TO
END",this,UVM_DEBUG)
              //m_state = UVM_PHASE_READY_TO_END;
              m_ready_to_end_count++;
              if  (m_ready_to_end_count < max_ready_to_end_iter) begin
                if  (m_phase_trace)
                  `UVM_PH_TRACE("PH_READY_TO_END_CB","CALLING
READY_TO_END CB",this,UVM_HIGH)
                if  (m_imp  != null)
                  m_imp.traverse(top,this,UVM_PHASE_READY_TO_END);
              end
              #0; // LET ANY WAITERS WAKE UP
              // WAIT FOR PREDECESSORS
              if  (!phase_done.get_objection_total(top)) begin
                m_state = UVM_PHASE_READY_TO_END;
                m_wait_for_pred();
```

```
            end
          end
        end
        while (phase_done.get_objection_total(top));
        end
      join // guard
    end
  end
```

traverse

uvm_task_phase.svh 内容

```
  virtual function void traverse(uvm_component comp,
                                 uvm_phase phase,
                                 uvm_phase_state state);
    phase.m_num_procs_not_yet_returned = 0;
    m_traverse(comp, phase, state);
  endfunction
```

m_traverse

task_phase.svh 有 m_traverse 这样一个静态 task

```
  function void m_traverse(uvm_component comp,
                           uvm_phase phase,
                           uvm_phase_state state);
    string name;
    uvm_domain phase_domain =phase.get_domain();
    uvm_domain comp_domain = comp.get_domain();
    if (comp.get_first_child(name))
      do
禾口    m_traverse(comp.get_child(name), phase, state);//递归调用,同时
          //执行所有组件
      while(comp.get_next_child(name));
//把所有的”孩子”的”孩子”执行完,在执行第二个孩子,依次类推----深度
优先
    if (phase_domain == uvm_domain::get_common_domain() ||
        phase_domain == comp_domain) begin
      case (state)
        UVM_PHASE_STARTED: begin
          comp.m_current_phase = phase;
          comp.m_apply_verbosity_settings(phase);
          comp.phase_started(phase);
         //component 类中有一个叫 phase_started 没有主体的虚 function ,
```

```
        //uvm_root 扩展自 component,它有实体 phase_started function
          end
        UVM_PHASE_EXECUTING: begin
          uvm_phase ph = this;
          if (comp.m_phase_imps.exists(this))
          //component 类中有 uvm_phase  m_phase_imps[uvm_phase]变量
            ph = comp.m_phase_imps[this];
          ph.execute(comp, phase);//do … while 用了 fork … join_none
                  //相当于自下而上并行执行所有组件,
          //uvm_phase 中的 execute 是一个没有主体的虚函数,在
          //  uvm_task_phase 中有实体
          end
        UVM_PHASE_READY_TO_END: begin
          comp.phase_ready_to_end(phase);
          //component 类中有一个叫 phase_ready_to_end 的虚 function
          end
        UVM_PHASE_ENDED: begin
          comp.phase_ended(phase);
          //component 类中有一个叫 phase_ended 的虚 function
          comp.m_current_phase = null;
          end
        default:
          `uvm_fatal("PH_BADEXEC","task phase traverse internal error")
      endcase
    end
  endfunction
```

execute

uvm_task_phase.svh 有 execute 这个函数

```
  protected virtual function void execute(uvm_component comp,
                                          uvm_phase phase);
    fork
      begin
        uvm_sequencer_base seqr;// uvm_sequencer_base 扩展自 component
        phase.m_num_procs_not_yet_returned++;
        if ($cast(seqr,comp))
          seqr.start_phase_sequence(phase);
          //内部有 fork  seq.start(this); join_none 语句
          // seq.start 是启动 se
          exec_task(comp,phase);
        phase.m_num_procs_not_yet_returned--;
      end
```

```
    join_none
  endfunction
```

exec_task

virtual task exec_task(uvm_component comp, uvm_phase phase); endtask
----这个 task 属于 uvm_task_phase 类,无主体.

在 uvm_common_phase.svh 和 uvm_runtime_phase.svh(所有的类均扩展自 uvm_task_phase)分别存放了两大类 task/function; uvm_runtime_phase.svh 各个阶段的 task 名字都叫 exec_task,只是每个阶段的 exec_task 调用的 phase task 不一样,(component class 定义了各个 phase 的 task)

```
class uvm_run_phase extends uvm_task_phase;
   virtual task exec_task(uvm_component comp, uvm_phase phase);
      uvm_component comp_;
      if ($cast(comp_,comp))
        comp_.run_phase(phase);
   endtask : exec_task
   local static uvm_run_phase m_inst;
   static const string type_name = "uvm_run_phase";
   static function uvm_run_phase get();
      if(m_inst == null) begin
        m_inst = new;
          end
      return m_inst;
   endfunction : get
   `_protected function new(string name="run");
      super.new(name);
   endfunction : new
   virtual function string get_type_name();
      return type_name;
   endfunction : get_type_name
endclass : uvm_run_phase
```

exec_func

```
class uvm_extract_phase extends uvm_bottomup_phase;
   virtual function void exec_func(uvm_component comp, uvm_phase phase);
      uvm_component comp_;
      if ($cast(comp_,comp))
        comp_.extract_phase(phase);
   endfunction : exec_func
   local static uvm_extract_phase m_inst;
```

```
  static const string type_name = "uvm_extract_phase";
  static function uvm_extract_phase get();
     if(m_inst == null) begin
        m_inst = new();
     end
     return m_inst;
  endfunction : get
  `_protected function new(string name="extract");
     super.new(name);
  endfunction : new
  virtual function string get_type_name();
     return type_name;
  endfunction : get_type_name
endclass : uvm_extract_phase
```

class uvm_process

```
  class uvm_process;
  protected process m_process_id;
  function new(process pid);
    m_process_id = pid;
  endfunction

  function process self();
    return m_process_id;
  endfunction

  virtual function void kill();
    m_process_id.kill();
  endfunction

`ifdef UVM_USE_FPC
  virtual function process::state status();
    return m_process_id.status();
  endfunction

  task await();
    m_process_id.await();
  endtask

  task suspend();
   m_process_id.suspend();
  endtask
```

```
  function void resume();
   m_process_id.resume();
  endfunction
`else
  virtual function int status();
    return m_process_id.status();
  endfunction
`endif

Endclass
```

4.2 总结

Uvm 自动运行的过程大致如下:

1. Tb 环境调用全局函数 run_test();

2. 全局函数 run_test 调用root:: run_test 函数, 它有 2 种方法得到 testcase;

3. root:: run_test使用 fork…join_none调用一个进程函数process::self 和 uvm_phase::m_run_phases(),并等待进程结束

4. uvm_phase::m_run_phases()内部一个 forever,永远执行 phase,直到执行完所有phase,然后一直被一个从mailbox 的 get 函数卡住——因为没有phase 了.

5. m_run_phases()内部有一个 forever,里面使用 fork join_none,包含了 phase.execute_phase(),这个函数会对 mailbox put 内容,同时会保留一个句柄, 不同的 phase, execute_phase 的行为不一样,由于是在 forever 语句中,使用相当于很多个 phase.execute_phase()并行运行

6. phase.execute_phase()函数主要是用来执行 function 和 task,根据 phase 的性质来调用 function 还是 task,当是 function 时,顺序执行 m_imp.traverse(top,this,UVM_PHASE_EXECUTING), traverse 函数就是采用递归的方式把所有 children 执行一边(其实就是除了 run_phase/mian_phase 的其它阶段);当是 task 时,则采用fork…join_none 的方式,相当于所有的 children 同时执行(其实就是指 run_phase/mian_phase 阶段).

7. traverse(top,this,UVM_PHASE_EXECUTING)内部就是一个递归函数,里面有一个 do m_traverse while();从上到下,把所有的组件执行一遍.根据不同的 phase 执行不同的行为,其中有一个是 UVM_PHASE_EXECUTING 状态,它会调用 ph.execute(comp, phase)这个 function,而这个 function 又会使用 fork…join_none 调用 exec_task(comp,phase)这个 task,再根据上面的 while 循环,则是所有的组件都会执行 exec_task(comp,phase);

8. _exec_task_是 uvm_run_phase 类中的一个 task,它通过组件句柄指向的方式指向各个组件的 run_phase(uvm_phase)这个 task(内部又调用了一个 run()task); exec_task 在 uvm_runtime_phase 文件中的各个 class 中都有这个 task,根据不同的 phase 调用不同的内容

5.raise_objection 机制分析

在 execute_phase 函数中有下面的判断

```
if (phase_done.get_objection_total(top) ||
    m_use_ovm_run_semantic && m_imp.get_name() == "run")
     begin
          if (!phase_done.m_top_all_dropped)
               phase_done.wait_for(UVM_ALL_DROPPED, top);
          `UVM_PH_TRACE("PH/TRC/EXE/ALLDROP","PHASE EXIT
           ALL_DROPPED",this,UVM_DEBUG)
     end
 else begin
      if (m_phase_trace)
         `UVM_PH_TRACE("PH/TRC/SKIP","No objections raised, skipping
           phase",this,UVM_LOW)
 end
```

分析:

phase_done.wait_for(UVM_ALL_DROPPED, top);这句就是一直等待所有的 drop_object,只要有 raise_object,就一定会等到 drop_object,否则就卡死 .

phase_done.get_objection_total(top)这句就是判断是否有 raise_object;

在 raise_objection 函数中,会对内部 m_total_count[obj]变量自加.

6. sequence 机制分析

sequence 不属于 env 中的 component,sequencer 才是 env 中的 component;sequencer 中有一个 sequence 的句柄,sequencer 把 sequence 产生的内容发送给 driver,可见 sequence 是一个专门产生激励的 class;

sequence 的继承关系:

```
virtual class uvm_sequence #(type REQ = uvm_sequence_item,
                             type RSP = REQ) extends uvm_sequence_base;
```

virtual 这种类必须扩展才能使用,不能直接使用.

```
class uvm_sequence_base extends uvm_sequence_item;
class uvm_sequence_item extends uvm_transaction;
```

注意 uvm_sequence 请求和响应的类型都是 uvm_sequence_item;
uvm_sequence_base 中有下面一个 task;

```
virtual task start  (uvm_sequencer_base sequencer,
                     uvm_sequence_base parent_sequence = null,
                     int this_priority = -1,
                     bit call_pre_post = 1);
```

uvm_sequencer 继承关系如下:

```
class uvm_sequencer #(type REQ=uvm_sequence_item, RSP=REQ)
                                    extends uvm_sequencer_param_base
#(REQ, RSP);
class uvm_sequencer_param_base #(type REQ = uvm_sequence_item,
                                  type RSP = REQ) extends
uvm_sequencer_base;
class uvm_sequencer_base extends uvm_component;
```

uvm_sequencer 如何启动 sequence

在 uvm_sequencer 的基类 uvm_sequencer_base 中的 run_phase 阶段会启动 start_default_sequence()task 如下:

```
task uvm_sequencer_base::run_phase(uvm_phase phase);
  super.run_phase(phase);
  start_default_sequence();
endtask
```

start_default_sequence();它会调用 sequence 的 start 虚 task.
Start 有会调用一系列 task,其中主要一个是 body ();task

```
  virtual task body ();
    uvm_report_warning ("uvm_sequence_base", "Body definition
undefined");
    return;
  endtask
```

所以在我们使用时,需要在 sequence 中重载 body 这个函数.
sequencer 如何启动 sequence:

```
   uvm_config_db#(uvm_object_wrapper)::set(this,
                                           "env.i_agt.sqr.main_phase",
                                           "default_sequence",
                                           case0_sequence::type_id::get());
```

sequencer 在 build 阶段会去判断 default_sequence 内部的字符变量 default_sequence 是否为" default_sequence";

为什么是 env.i_agt.sqr.main_phase,而不是 env.i_agt.sqr.run_phase ?
虽然在 uvm_sequencer_base 是一个 run_phase,但 uvm_sequencer_base 继承于 uvm_component,在 uvm_component 中是 main_phase ;在 uvm_phase 机制中 main_phase 和 run_phase 是同时启动的.

7. set_config 机制分析

简单理解就是在一个参数类中有一个变量 val,uvm_config_db::set 把要发送的内容放在uvm_resource 的 val变量,然后其他的组件有可以通过调用 uvm_config_db::get 从 uvm_resource 中把 val 中的内容取走;当然这里面它们的路径要相当(就好像是密码一样)

```
class uvm_resource #(type T=int) extends uvm_resource_base;
  typedef uvm_resource#(T) this_type;
  // singleton handle that represents the type of this resource
  static this_type my_type = get_type();
  // Can't be rand since things like rand strings are not legal.
  protected T val;
function void write(T t, uvm_object accessor = null);

    if(is_read_only ()) begin
      uvm_report_error("resource", $sformatf("resource %s is read
      only -- cannot modify", get_name()));
      return;
    end
    record_write_access(accessor);
    // set the value and set the dirty bit
    val = t;
    modified = 1;
  endfunction

  function T read(uvm_object accessor = null);
    record_read_access(accessor);
    return val;
  endfunction
.....
endclass
```

8.factory override 重载机制分析

使用方法如下:

```
    原形
    set_type_override_by_type  (uvm_object_wrapper original_type,
                                uvm_object_wrapper override_type,
```

bit replace=1);

在 my_case0.sv 有如下内容

Function void my_case0::build_phase(uvm_phase phase);

Set_type_override_by_type(bird::get_type(),parrot::get_type());

Bird_inst = bird::type_id::create(“bird_inst”);

parrot _inst = parrot::type_id::create(“parrot _inst”);

endfunction

最后真正使用的是 parrot _inst;

Set_type_override_by_type 是 uvm_component 的一个 function,这个 function 调用了 factory 中的 Set_type_override_by_type 函数;testcase 最终的扩展来自于 uvm_component;

Factory 重载的三个条件:

. 两个类都必须注册到 factory 中.
. 被重载的类在实例化时,要使用 factory机制式的实例化.
. 重载的类和被重载的类有派生关系;被重载的必须是重载类的父类.
. 注意;component 和 object 类型的不能互相重载.

Set_inst_override_by_type 可以重载类中类,如 agt 中的 monitor;

Set_inst_override_by_type(“env.o_agt.mon”,
my_monitor::get_type(),
new_monitor::get_type());

最终使用的 new_monitor;

重载的行为(语句)放在什么位置:一般都是放在 testcase 的build语句中,因为 build_phase 是从上而下执行---先执行 tc build->env build->agent build …;其实所有不耗时间的 phase 都是从上到下执行;

对于同一层次的 component 则是按照实例化名字的顺序先后执行,--因为同一层次的 component 的句柄是放在一个关联数组中,而下标则是使用new 时的名字,对于字符为下标的关联数组,其排列顺序是按字母顺序排列—可以通过 foreach()语句打印查看顺序.

对于 run_phase 等需要时间的 task,这是通过 fork join_none 来实现同时执行.

那么重载的过程为:在 tc build 通过 Set_type_override_by_type 语句告诉 factory 有一个重载行为,当在 component build(eg,driver)时,进行 creat 产生对象时,实际是把重载的对象指向 driver 的句柄,其实质就是通过 $cast(drv,override_drv)的方式改变句柄的指向,当在调用 drv.main()函数时,实际是调用 override_drv.main()函数---main 函数必须是虚属性,否则无法重载,利用的是 class 的多态特性.

注意的是在重载 transaction 时,继承类不能多出变量,因为在使用时句柄是父类的,父类没有的变量,在使用时会报错

9.在 soc 系统中运用 uvm 方法学

Soc 系统验证与普通的模块验证最大的不同是,soc 是通过配置寄存器来控制各个模块工作，不存在显式的 driver,也就是不存在 driver 通过 PORT 向 sequence 请求产生 sqeuence_item ，而 sequence 通过 sequencer 发送 item 给 driver。

实际在做 soc 验证是通过一种通用固件的方式来实现 tc 与 c 的交互，在 uvm 中的 driver 基本就是用在通讯接口验证上，非接口模块基本不用 driver,只会用到 ref_mode,scb;’

在 soc 验证中，配置寄存器的行为与控制仿真平台的行为在一起，

starting_phase.raise_objection(this);

starting_phase.drop_objection(this);也就是谁执行这两句话就放在谁一起

如下是放在 sequence 中

case0_sequence

```
class case0_sequence extends uvm_sequence #(my_transaction);
   my_transaction m_trans;
   function  new(string name= "case0_sequence");
      super.new(name);
   endfunction
   virtual task body();
      if(starting_phase != null)
         starting_phase.raise_objection(this);
         #100;
      repeat (1) begin
         `uvm_do(m_trans) //如果外接 driver,则此处 m_trans 是给 driver 的内容,此处无 driver，所以可以借用下
         `uvm_info("case0_sequence","uvm_do first",UVM_MEDIUM)
        reg_Bitwrite1(`RTCC1_ADDR,m_trans.tsen_en<<4);
```

```
        reg_Bitwrite1(`RTCC2_ADDR,m_trans.time_wake_en<<11);
        reg_write(`SACTRL_ADDR,m_trans.wdata);
      end //上面就是配置寄存器的行为
      #5_000_000;
      if(starting_phase != null)
         starting_phase.drop_objection(this);
   endtask
   `uvm_object_utils(case0_sequence)
Endclass
```

my_sequencer

```
`ifndef MY_SEQUENCER__SV
`define MY_SEQUENCER__SV
class my_sequencer extends uvm_sequencer #(my_transaction);
   function new(string name, uvm_component parent);
      super.new(name, parent);
   endfunction
   `uvm_component_utils(my_sequencer)
endclass
`endif
```

my_if

```
`ifndef MY_IF__SV
`define MY_IF__SV

interface my_if(input clk, input rst_n);
```

```
    logic [7:0] data;
    logic valid;
endinterface

`endif
```

my_transaction

```
`ifndef MY_TRANSACTION__SV
`define MY_TRANSACTION__SV

class my_transaction extends uvm_sequence_item;
    rand bit[47:0] dmac;
    rand bit[47:0] smac;
    rand bit[15:0] ether_type;
    rand byte      pload[];
    rand bit[31:0] crc;
    constraint pload_cons{
        pload.size >= 46;
        pload.size <= 1500;
    }
    function bit[31:0] calc_crc();
        return 32'h0;
    endfunction
    function void post_randomize();
        crc = calc_crc;
    endfunction
    `uvm_object_utils_begin(my_transaction)
        `uvm_field_int(dmac, UVM_ALL_ON)
```

```
    `uvm_field_int(smac, UVM_ALL_ON)
    `uvm_field_int(ether_type, UVM_ALL_ON)
    `uvm_field_array_int(pload, UVM_ALL_ON)
    `uvm_field_int(crc, UVM_ALL_ON)
  `uvm_object_utils_end
  function new(string name = "my_transaction");
    super.new();
  endfunction

endclass
`endif
```

my_driver

```
`ifndef MY_DRIVER__SV
`define MY_DRIVER__SV
class my_driver extends uvm_driver#(my_transaction);
  virtual my_if vif;
  `uvm_component_utils(my_driver)
  function new(string name = "my_driver", uvm_component parent = null);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    if(!uvm_config_db#(virtual my_if)::get(this, "", "vif", vif))
      `uvm_fatal("my_driver", "virtual interface must be set for
vif!!!")
  endfunction
```

```
    extern task main_phase(uvm_phase phase);
    extern task drive_one_pkt(my_transaction tr);
endclass

task my_driver::main_phase(uvm_phase phase);
    vif.data <= 8'b0;
    vif.valid <= 1'b0;
    while(!vif.rst_n)
        @(posedge vif.clk);
    while(1) begin
        seq_item_port.get_next_item(req);
        drive_one_pkt(req);
        seq_item_port.item_done();
    end
endtask

task my_driver::drive_one_pkt(my_transaction tr);
    byte unsigned     data_q[];
    int  data_size;

    data_size = tr.pack_bytes(data_q) / 8;
    `uvm_info("my_driver", "begin to drive one pkt", UVM_LOW);
    repeat(3) @(posedge vif.clk);
    for ( int i = 0; i < data_size; i++ ) begin
        @(posedge vif.clk);
        vif.valid <= 1'b1;
        vif.data <= data_q[i];
    end
```

```
    @(posedge vif.clk);
    vif.valid <= 1'b0;
    `uvm_info("my_driver", "end drive one pkt", UVM_LOW);
endtask
`endif
```

my_agent

```
class my_agent extends uvm_agent  ;
    my_sequencer  sqr;
    my_driver     drv;
    my_monitor    mon;
    uvm_analysis_port #(my_transaction)  ap;
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
    extern virtual function void build_phase(uvm_phase phase);
    extern virtual function void connect_phase(uvm_phase phase);
    `uvm_component_utils(my_agent)
endclass

function void my_agent::build_phase(uvm_phase phase);
    super.build_phase(phase);
    if  (is_active == UVM_ACTIVE) begin
        sqr = my_sequencer::type_id::create("sqr", this);
        drv = my_driver::type_id::create("drv", this);
    end
    mon = my_monitor::type_id::create("mon", this);
```

```
endfunction

function void my_agent::connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    if  (is_active == UVM_ACTIVE) begin
        drv.seq_item_port.connect(sqr.seq_item_export);
    end
    ap = mon.ap;
endfunction
`endif
```

base_test

```
class base_test extends uvm_test;
    my_env          env;
    function new(string name = "base_test", uvm_component parent = null);
        super.new(name,parent);
    endfunction

    extern virtual function void build_phase(uvm_phase phase);
    extern virtual function void report_phase(uvm_phase phase);
    `uvm_component_utils(base_test)
endclass


function void base_test::build_phase(uvm_phase phase);
    super.build_phase(phase);
    env  =  my_env::type_id::create("env", this);
endfunction
```

```
function void base_test::report_phase(uvm_phase phase);
   uvm_report_server server;
   int err_num;
   super.report_phase(phase);

   server = get_report_server();
   err_num = server.get_severity_count(UVM_ERROR);

   if (err_num != 0) begin
      $display("TEST CASE FAILED");
   end
   else begin
      $display("TEST CASE PASSED");
   end
endfunction

`endif
```

my_case0

```
class my_case0 extends base_test;
   function new(string name = "my_case0", uvm_component parent = null);
      super.new(name,parent);
   endfunction
   extern virtual function void build_phase(uvm_phase phase);
   `uvm_component_utils(my_case0)
endclass
function void my_case0::build_phase(uvm_phase phase);
   super.build_phase(phase);
```

```
    uvm_config_db#(uvm_object_wrapper)::set(this,
                "env.i_agt.sqr.main_phase", "default_sequence",
                                            case0_sequence::type_id::get());
endfunction

`ifndef MY_AGENT__SV
`define MY_AGENT__SV
```

my_monitor

```
`ifndef MY_MONITOR__SV
`define MY_MONITOR__SV
class my_monitor extends uvm_monitor;
    virtual my_if vif;
    uvm_analysis_port #(my_transaction)  ap;
    `uvm_component_utils(my_monitor)
    function new(string name = "my_monitor", uvm_component parent = null);
        super.new(name, parent);
    endfunction
    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if(!uvm_config_db#(virtual my_if)::get(this, "", "vif", vif))
            `uvm_fatal("my_monitor", "virtual interface must be set for
vif!!!")
        ap = new("ap", this);
    endfunction

    extern task main_phase(uvm_phase phase);
    extern task collect_one_pkt(my_transaction tr);
endclass
```

```
task my_monitor::main_phase(uvm_phase phase);
   my_transaction tr;
   while(1) begin
      tr = new("tr");
      collect_one_pkt(tr);
      ap.write(tr);
   end
endtask

task my_monitor::collect_one_pkt(my_transaction tr);
   byte unsigned data_q[$];
   byte unsigned data_array[];
   logic [7:0] data;
   logic valid = 0;
   int data_size;

   while(1) begin
      @(posedge vif.clk);
      if(vif.valid) break;
   end

   `uvm_info("my_monitor", "begin to collect one pkt", UVM_LOW);
   while(vif.valid) begin
      data_q.push_back(vif.data);
      @(posedge vif.clk);
   end
   data_size  = data_q.size();
   data_array = new[data_size];
```

```
    for ( int i = 0; i < data_size; i++ ) begin
        data_array[i] = data_q[i];
    end
    tr.pload = new[data_size - 18]; //da sa, e_type, crc
    data_size = tr.unpack_bytes(data_array) / 8;
    `uvm_info("my_monitor", "end collect one pkt", UVM_LOW);
endtask
`endif
```

my_model

```
`ifndef MY_MODEL__SV
`define MY_MODEL__SV
class my_model extends uvm_component;
    uvm_blocking_get_port #(my_transaction)  port;
    uvm_analysis_port #(my_transaction)  ap;
    extern function new(string name, uvm_component parent);
    extern function void build_phase(uvm_phase phase);
    extern virtual  task main_phase(uvm_phase phase);
    `uvm_component_utils(my_model)
endclass
function my_model::new(string name, uvm_component parent);
    super.new(name, parent);
endfunction

function void my_model::build_phase(uvm_phase phase);
    super.build_phase(phase);
    port = new("port", this);
    ap = new("ap", this);
endfunction
```

```
task my_model::main_phase(uvm_phase phase);
   my_transaction tr;
   my_transaction new_tr;
   super.main_phase(phase);
   while(1) begin
      port.get(tr);
      new_tr = new("new_tr");
      new_tr.copy (tr);
      `uvm_info("my_model", "get one transaction, copy and print it:",
UVM_LOW)
      new_tr.print();
      ap.write(new_tr);
   end
endtask
`endif
```

my_scoreboard

```
`ifndef MY_SCOREBOARD__SV
`define MY_SCOREBOARD__SV
class my_scoreboard extends uvm_scoreboard;
   my_transaction  expect_queue[$];
   uvm_blocking_get_port #(my_transaction)  exp_port;
   uvm_blocking_get_port #(my_transaction)  act_port;
   `uvm_component_utils(my_scoreboard)

   extern function new(string name, uvm_component parent = null);
   extern virtual function void build_phase(uvm_phase phase);
   extern virtual task main_phase(uvm_phase phase);
```

```
endclass

function my_scoreboard::new(string name, uvm_component parent = null);
   super.new(name, parent);
endfunction

function void my_scoreboard::build_phase(uvm_phase phase);
   super.build_phase(phase);
   exp_port = new("exp_port", this);
   act_port = new("act_port", this);
endfunction

task my_scoreboard::main_phase(uvm_phase phase);
   my_transaction  get_expect,  get_actual, tmp_tran;
   bit result;

   super.main_phase(phase);
   fork
      while (1) begin
         exp_port.get(get_expect);
         expect_queue.push_back(get_expect);
      end
      while (1) begin
         act_port.get(get_actual);
         if(expect_queue.size() > 0) begin
            tmp_tran = expect_queue.pop_front();
            result = get_actual.compare(tmp_tran);
            if(result) begin
```

```
                `uvm_info("my_scoreboard", "Compare SUCCESSFULLY",
UVM_LOW);
            end
            else begin
                `uvm_error("my_scoreboard", "Compare FAILED");
                $display("the expect pkt is");
                tmp_tran.print();
                $display("the actual pkt is");
                get_actual.print();
            end
        end
        else begin
            `uvm_error("my_scoreboard", "Received from DUT, while
Expect Queue is empty");
            $display("the unexpected pkt is");
            get_actual.print();
        end
    end
  join
endtask
`endif
```

my_env

```
`ifndef MY_ENV__SV
`define MY_ENV__SV
class my_env extends uvm_env;
  my_agent   i_agt;
  my_agent   o_agt;
  my_model   mdl;
```

```
    my_scoreboard scb;
    uvm_tlm_analysis_fifo #(my_transaction) agt_scb_fifo;
    uvm_tlm_analysis_fifo #(my_transaction) agt_mdl_fifo;
    uvm_tlm_analysis_fifo #(my_transaction) mdl_scb_fifo;
    function new(string name = "my_env", uvm_component parent);
        super.new(name, parent);
    endfunction
    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        i_agt = my_agent::type_id::create("i_agt", this);
        o_agt = my_agent::type_id::create("o_agt", this);
        i_agt.is_active = UVM_ACTIVE;
        o_agt.is_active = UVM_PASSIVE;
        mdl = my_model::type_id::create("mdl", this);
        scb = my_scoreboard::type_id::create("scb", this);
        agt_scb_fifo = new("agt_scb_fifo", this);
        agt_mdl_fifo = new("agt_mdl_fifo", this);
        mdl_scb_fifo = new("mdl_scb_fifo", this);
    endfunction
    extern virtual function void connect_phase(uvm_phase phase);

    `uvm_component_utils(my_env)
endclass

function void my_env::connect_phase(uvm_phase phase);
    super.connect_phase(phase);
    i_agt.ap.connect(agt_mdl_fifo.analysis_export);
    mdl.port.connect(agt_mdl_fifo.blocking_get_export);
    mdl.ap.connect(mdl_scb_fifo.analysis_export);
```

```
    scb.exp_port.connect(mdl_scb_fifo.blocking_get_export);
    o_agt.ap.connect(agt_scb_fifo.analysis_export);
    scb.act_port.connect(agt_scb_fifo.blocking_get_export);
    uvm_top.print_topology ();
endfunction
`endif
```

Top_tb

Top_tb.sv 内容

```
`timescale 1ns/1ps
`include "uvm_macros.svh"     //此句可以不用，在 uvm_pkg.sv 中有此句
import uvm_pkg::*;和//导入 uvm 内容--全部是 class，原文件在 makefile 中加入编译

`include "./env/my_if.sv"
module top_tb;

`include "../sim/reg_addr.sv"
`include "./env/project_define.sv"
`include "../sim/testcases/common_file/bus_opr.sv"
//下面这些 uvm 类需要自己编写，通用格式，不同的 tc 可以通过重载方式修改
`include "./env/my_sequencer.sv"
`include "./env/my_driver.sv"
`include "./env/my_monitor.sv"
`include "./env/my_agent.sv"
`include "./env/my_model.sv"
`include "./env/my_scoreboard.sv"
`include "./env/my_env.sv"
`include "./env/base_test.sv"
```

```
//下面这个文件根据 tc 修改
`include "./env/my_transation.sv"
//下面这个文件内包含了 tc 类和 sequence,根据 tc 修改
`include "./env/my_case.sv"

//--------------clk--------------------------------
……
//--------------set parameter----------------------

initial begin
    while(1) begin
        #1000000;
        $display(" --------- %6d ms ----------", $time/1000000 );
    end
end

my_if input_if(clk, rst_n);
my_if output_if(clk, rst_n);

dut i_dut (

            .P13_PAD(P13_PAD),
……
            .P17_PAD(P17_PAD)
);

initial begin
   clk = 0;
   forever begin
```

```
      #100 clk = ~clk;
   end
end

initial begin
   rst_n = 1'b0;
   #1000;
   rst_n = 1'b1;
end

initial begin
   run_test();
end
my_if vif() ;
assign vif.hreset =
top_tb.I_cbm8007.u_vdd12_domain.inst_vdd12_core.hresetn;
initial begin
   uvm_config_db#(virtual my_if)::set(null,
"uvm_test_top.env.i_agt.sqr", "vif", input_if);
   uvm_config_db#(virtual my_if)::set(null,
"uvm_test_top.env.i_agt.drv", "vif", input_if);
   uvm_config_db#(virtual my_if)::set(null,
"uvm_test_top.env.i_agt.mon", "vif", input_if);
   uvm_config_db#(virtual my_if)::set(null,
"uvm_test_top.env.o_agt.mon", "vif", output_if);
end
    initial begin
       $fsdbDumpfile("uvm_test.fsdb");
       $fsdbDumpvars(0,top_tb);
```

```
        $fsdbDumpon();
    end
endmodule
```

10. run_test()运行过程分析(转)

我相信很多朋友都在坛子里下载过一份《UVM1.1 应用指南和源代码分析》的资料，我很佩服这位前辈，我也从中收益匪浅，但是可惜在讲解 phase 的时候对一些初学者来说有些东西跳跃性有点大，更主要的是没有用一个稍微全面而复杂的例子来进行进一步的总结，让读者知道在实际项目中如何构建一个 user-defined 的 phase 组织架构图。我在这里也是志在狗尾续貂吧，废话少说，开始吧。

一、什么是 phase?

phase 翻译成中文就是相、阶段的意思。

在 UVM 中，官方的说法是：phase 是使 tb 中各种各样的 component 按照各自的需求可以阶段性执行的一种自动化的机制。

简单的说就是使验证组件能够按需自动化执行的一种机制。

二、OVM 有 phase 的概念吗?

有。

那为什么 OVM 中没有看到在各个 function 或者 task 的形参表里看到（ovm_phase phase）呢？

因为 OVM 已经为用户提供了现成的、固定的几个 phase，它们是 new phase（对一个合格的 tb 来说，new phase 是被 ovm root component 在执行 run_test()的时候通过 factory 机制调用的，也就是通过静态函数 create 调用的）、build phase、connect phase、end_of_elaboration phase、 start_of_simulation phase、run phase、extract phase、check phase、report phase。

OVM 没有提供 API 给用户进行一些操作，比如增加一个 phase、让不同的 component 的同一个 phase 异步的运行等等.....

三、UVM 为什么在 OVM 的基础上扩展、增强这个功能?

因为不够用所以才需要扩展，因为不完美用才需要增强。

不完美体现在哪里？我提一点，其余的大家可以挖掘。

一个正常的 DUT，从上电到正常工作到关闭是一个相对比较通用的流程，所以把这些比较通用的流程放在一个 run phase 里面可以但不完美，那我把 run phase 按照这些通用流程拆分成几个子的阶段不是更好么？比如拆分成 reset phase、configure phase、main phase、shutdown phase。

不够用体现在哪里？我同样提一点，其余的大家继续挖掘、跟帖。

一个大的 chip 里面有很多功能性比较独立的模块，这些功能性独立意味着一个模块 A 在 run 的时刻另一个模块 B 可以不 run，也可以 run，B 运行不运行和A 运行不运行关联度不大甚至没有关联，比如A 是只负责处理发通路的而 B 只负责收通路；但是

另一方面，功能性独立并不意味着什么都独立，举例来说，A 模块和B 模块功能性很独立，比如 A 是一个 clock generation 模块，B 是一个 processor 模块，那在 A 没有正常work 的前提下 B 是不能正常工作的。

由此我们至少可以提出两点需求：

(1)能不能让 B 的 reset phase 发生在 A 的 reset phase 之后，这样等待 clock 都稳定了再对 B 做 reset 操作或者 release reset 操作？

(2)能不能让 A 的 main phase 和 B 的 main phase 异步的运行？

上面两点至少需要用到 UVM 中的的如下机制：新建一个 domain、给不同的 component 设置不同的 domain、不同的domain 之间 phase 的同步和异步。

再如，功能更复杂的 tb 可能需要新建一个 user-defined 的 phase，让它在某一需要个时刻点运行，可以是 function 性质的或者task 性质的。

四、UVM phase 的组织架构

（1）在大学学习数据结构的时候，我们知道有几种基本的数据结构，其它的结构都可以分解成这几种的组合；它们分别是线性数据结构（比如链表）、树形结构（比如二叉树）、图形结构（有向图和无向图）；UVM 的 phase 的组织结构也不例外，都可以分解成上述三种。总体上来说，是按照有向图结构进行组织的。

（2）既然是有向图形结构，那么这个数据结构中自然有若干节点和连接这些节点的有向边。另外，一个图也可以分解为若干个子图。

在默认情况下，这些节点就像铁路的一个个站点，每个节点都有自己的属性；这里所说的属性就是指的是 UVM 库中定义的如下几种：

UVM_PHASE_DOMAIN：它的含义是从我开始，到后面的某个节点为止，这期间经过的所有节点都是我的管辖范围。

UVM_PHASE_SCHEDULE：它的含义和 UVM_PHASE_DOMAIN 相同，唯一的区别就是它不具有独立行动的权利，它的外面至少需要套一 UVM_PHASE_DOMAIN；形象点说，UVM_PHASE_DOMAIN 可以代表整个图形结构或者代表某个子图结构，但是，UVM_PHASE_SCHEDULE 则只能代表某个子图结构，它必须属于某个 DOMAIN，也就是它必须被某个 DOMAIN wrapper 起来！

UVM_PHASE_NODE：它的含义是图形结构中的某一个节点，它是一个句柄，它自己不干具体的活（比如main phase 具体要干什么）。

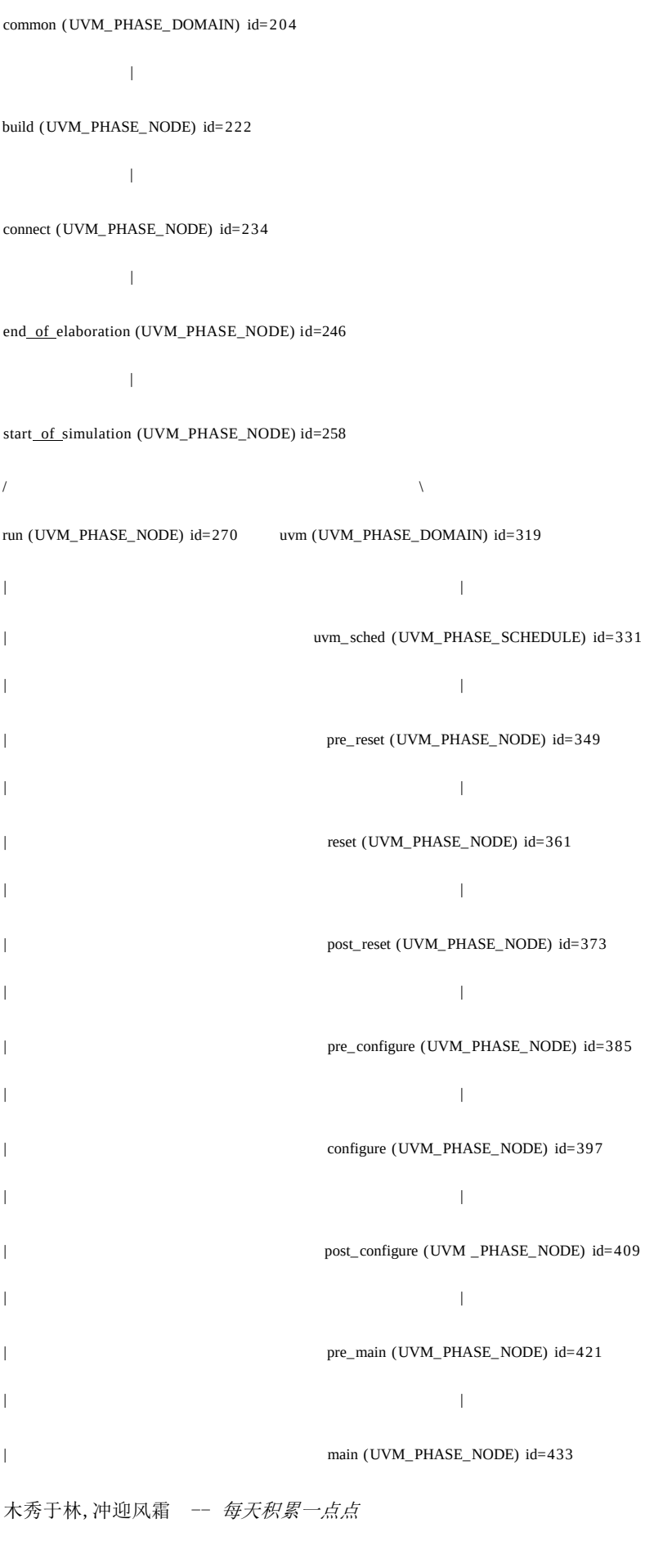
UVM_PHASE_IMP：它的含义就是说它所代表的就是具体干什么活，上述的 UVM_PHASE_NODE 就是会指向某一个 UVM_ PHASE_ IMP。

UVM_PHASE_TERMINAL：它是用来标定 UVM_PHASE_DOMAIN 和 UVM_PHASE_SCHEDULE 的势力范围的！就是说我后面的节点就不是你们的管辖范围了，只有一个 domain 或者 schedule 才有这个东西。

UVM_PHASE_NODE 和 UVM_PHASE_IMP 的关系如果不好理解的话，你可以对照 TLM 中的 port、export、和 imp 来理解，这样就容易些了；像 port、export 只负责传话或者发号施令，而 imp 才比较苦逼，是真正干活的。

有向图中的边在 UVM phase 里则代表了运行的顺序，就是说这个节点过了，下个节点是哪些，我想这个应该比较好理解。

（3）默认 UVM phase 的具体的图形结构：



```
|                                                    |
|                                post_main (UVM_PHASE_NODE) id=445
|                                                    |
|                                pre_shutdown (UVM_PHASE_NODE) id=457
|                                                    |
|                                shutdown (UVM_PHASE_NODE) id=469
|                                                    |
|                               post_shutdown (UVM_PHASE_NODE) id=481
|                                                    |
|                                uvm_sched_end (UVM_PHASE_TERMINAL) id=337
|                                                    |
|                                uvm_end (UVM_PHASE_TERMINAL) id=325
\                                                   /
extract (UVM_PHASE_NODE) id=277
                    |
check (UVM_PHASE_NODE) id=289
                    |
report (UVM_PHASE_NODE) id=301
                    |
final (UVM_PHASE_NODE) id=313
                    |
common_end (UVM_PHASE_TERMINAL) id=210
```

这个图的每个节点是通过调用 m_print_successors () 函数得到的，当然中间的“|”这些符号是我自己编辑的，这个函数在 UVM class reference 的 pdf 资料里没有提到，应该是 UVM 开发人员自己调试用的，但是如果你自己阅读代码就知道有这个函数，因为它不是 local 的，所以我们可以随时调用它。

每个节点的名字，属性和 id 号都一清二楚，细心的朋友可能已经注意到了这些id 号不是随机的，而是有顺序的，是的，它是按照这些节点的建立的先后顺序分配的，id 越大表示越晚建立，实际上这个 id 号就是通过调用 get_inst_id()得来的，这个函数是 uvm_object 的一个基本函数，我就不多讲这个了。

从这个图中，我们可以得到如下几个重要的信息：

（1）这张大的图形结构中有两个domain，一个叫做 common，一个叫做 uvm，这个概念很重要，以后的分析中会反复用到；

（2）UVM 中新加的 12 个 phase（不包括 final phase），从 pre reset 到 post shutdown 都被一个叫做 uvm_sched 的 SCHEDULE 包起来了，作为一个子图隶属于 uvm domain；

（3）OVM 和 UVM 中的 run phase 属于 common domain；

（4）start_of_simulation phase 节点有两个后继节点：run phase node 和 uvm domain；

五、UVM 的 phase 是如何自动运行起来的

从 build phase 到最后的 final phase，自动运行会经过以下几个主要步骤：

1、在 tb_top.sv 中的某个地方调用 run_test（），这个函数在 uvm_root.sv 中，是整个 tb 树形组织结构的根节点，这个函数主要是干三件事情：

（1）通过 factory 模式 create 你自己希望运行的 testcase instance，不管你的 testcase 叫神马名字，最后 uvm 都会给它重命名为“uvm_test_top”。

（2）调用 uvm phase 的一个函数，叫做 m_run_phases()，等下再讲这个函数主要干的事情。

（3）等待（2）结束，kill 所有和（2）有关的进程，如果允许则调用$finish 结束仿真，退出仿真器。

2、m_run_phases()主要干这么两件事情：

（1）创建 UVM 默认的所有 phases，即我们熟知的 build phase、connect phase、run phase、main phase 等等，并将其组织成我上面所讲的那个有向图结构；实现这一步主要是靠调用 uvm_domain class 中的 get_common_domain（）。

（2）开始运行（1）创建组织好的 phases，当然如果有用户自定义的 phase，也会在规定的点运行起来；实现这一步主要是靠调用对应的 phase execute_phase（）函数和一个无限循环的监控进程。

3、uvm_domain 中的 get_common_domain（）主要是干这么三件事情：

（1）创建我上述 phase 有向图结构的第一部分，即 build->connect->end_of_elaboration->start_of_simulation->run->extract->check->report->final。很自然的实现这个功能需要用到两个函数：创建它们用 new（）函数，组织它们让它们有如此先后顺序用add（）函数。

（2）创建我上述 phase 有向图结构的第二部分，即 pre_reset->reset->post_reset->pre_configure->configure->post_configure->pre_main->main->post_main->pre_shutdown->shutdown->post_shutdown。很自然的实现这个功能需也要用到两个函数：创建它们用 new（）函数，组织它们让它们有如此先后顺序用add（）函数。

（3）正如 phase 有向图结构所示，（2）搞出来的东西是作为一个子图呈现在（1）所搞出来的结构中的；那么怎嘛让（2）搞出来的东西就成了（1）所搞出来的东西的一部分了呢？而且还要和 run phase 并行的运行呢？还是要靠这个add（）函数！只不过需要用到 add（）函数形参表中一个叫做 with_phase 的东西。

显而易见，这个定义在 uvm phase class 中的 add（）函数真是无比的重要，正是它让我们可以根据需要把一个个 phase 节点组织成一个任意的被 UVM 规则所允许的图形结构，所以既然它这么重要，我们下一贴就准备专门来谈谈它！

4、phase 的 execute_phase（） 函数主要干哪些事情呢？

简单点说就是按照 phase 的有向图结构一个一个节点的走下去，知道结束；当然这个走的过程是复杂的、曲折的，呵呵，此函数也是比较 复杂、 曲折的；但是有一点，也是最主要最重要的一点，那就是它会调用每个节点所拥有的 traverse（） 函数！正式通过这个函数，我们在各个 phase 中所写的自己的代码才得意被调用、执行！

既然这个函数也是如此的重要，那我将用另一个帖子来专门讨论它一下。

5、traverse（） 函数主要干的事情就是根据每个节点的属性来调用我们自己重载的各个 phase。

主要是两个函数一个是 exec_func（uvm_component comp, uvm_phase phase），一个是 exec_task（uvm_component comp, uvm_phase phase）。很自然的我们想到，执行 build phase 这种 function 性质的就会调用 exec_func（），执行如 main phase 这种 task 性质的就会调用 exec_task（），没错，的确如此！

注意到了这两个函数的形参表了么？我想大家都猜出它们是干什么用的吧。 比如 component A 的 build phase 在执行，那么形参 comp 就是 A，形参 phase 就是 build。

六、UVM phase class 中的 add（） 函数

1、函数原型

```
function void uvm_phase::add(uvm_phase phase,
uvm_phase with_phase=null,
uvm_phase after_phase=null,
uvm_phase before_phase=null);
```

比如调用是 A.add（B，C）就表示把 B 加入到 A 的大家庭中，在 A 这个大家庭中的位置如何取决于后面的三个参数，根据它们的名字我想不难知道它们的意思。

2、函数主体一：参数有效性判断

（1）

```
if (phase == null)
  `uvm_fatal("PH/NULL", "add: phase argument is null");
```

这个含义很明显，如果你想加的这个 phase 还没有创建，你做这个操作是毫无意义的，simulator 会直接退出。

（2）

```
if (with_phase != null && with_phase.get_phase_type() == UVM_PHASE_IMP)
begin
string nm = with_phase.get_name();
with_phase = find(with_phase);
if (with_phase == null)
```

```
`uvm_fatal("PH_BAD_ADD", {"cannot find with_phase '",nm,"' within node
'",get_name(),"'"});
 end
```

这个检查理解起来也不困难，以 A.add(B，C)来说，意思就是说你想把 B 加入到 A 中来，并且想让 B 和 C 可以并行的运行，首先要确保在 A 中确实有C 这个人才行，不然这个操作肯定也会有问题，而且也是fatal。

（3）

```
 if (before_phase != null && before_phase.get_phase_type() == UVM_PHASE_IMP)
begin
 string nm = before_phase.get_name();
before_phase = find(before_phase);
4if (before_phase == null)
`uvm_fatal("PH_BAD_ADD", {"cannot find before_phase '",nm,"' within node
'",get_name(),"'"});
end
```

（4）

```
 if (after_phase != null && after_phase.get_phase_type() == UVM_PHASE_IMP)
begin
string nm = after_phase.get_name();
 after_phase = find(after_phase);
 if (after_phase == null)
`uvm_fatal("PH_BAD_ADD", {"cannot find after_phase '",nm,"' within node
'",get_name(),"'"});
 end
```

有了（2）的解释，（3）和（4）想必就不用再解释了吧。

（5）

```
 if (with_phase != null && (after_phase != null || before_phase != null))
`uvm_fatal("PH_BAD_ADD", "cannot specify both 'with' and 'before/after' phase
relationships");
```

这个含义就是说，你说明了用 with_phase（或者说用 after_phase/before_phase）这个参数已经可以确定你想要放置的位置了，不需要也不允许在用后面两个参数进行进一步框定了，这个主要是为了防止一些低级失误。这个就好比说，数值 A 的值等于圆周

率，并且数值 A 介于 3 和 4 之间；前半句已经可以确定 A 是多少了，后面那句是废话；万一我后半句由于笔误写成了并且数值 A 介于4 和 5 之间，那不是麻烦了。

（6）

```
if (before_phase == this || after_phase == m_end_node || with_phase == m_end_node)
  `uvm_fatal("PH_BAD_ADD","cannot add before begin node, after end node, or with end nodes");
```

这个检查的意思是说：before_phase == this，表示你想插入到我当前这个 domain 或者 schedule 的祖先节点的前面，那是不允许的，this 表示了这个 domain 或者 schedule 的树根。

after_phase == m_end_node，表示你想插入到我当前这个 domain 或者 schedule 的最后一个终结节点的后面，m_end_node 表示了这个 domain 或者 schedule 的终结节点，注意这个终结节点的类型是 UVM_PHASE_TERMINAL 而不是 UVM_PHASE_NDOE，

with_phase == m_end_node，表示你想和我的终结节点并行运行也是不行的，为什么？ 因为 UVM_PHASE_TERMINAL 类型节点（和 UVM_PHASE_DOMAIN 类型节点以及 UVM_PHASE_SCHEDULE 类型节点）都是不运行的，也就是说它们三个的运行时间是 0。

为什么是这样，这个会在后面的 execute_phase（） 函数中看到 UVM 是这么做的。

3、函数主体二：根据插入节点的类型来进行插入位置的框定

```
if (phase.get_phase_type() == UVM_PHASE_IMP) begin
  new_node = new(phase.get_name(),UVM_PHASE_NODE,this);
  new_node.m_imp = phase;
  begin_node = new_node;
  end_node = new_node;
end
else begin
  begin_node = phase;
  end_node = phase.m_end_node;
phase.m_parent = this;
end
```

这个 if else 看似不多，也不复杂，其实需要你对 UVM phase 的组织流程图、UVM phase 的各种类型的节点如 NODE、DOMAIN、TERMINATER 等真正含义的准确理解，不然的话你会觉得这个if else 的内容很奇怪！

其实我个人觉得这个 if else 是这个函数的难点，其它的很容易懂！

以 A.add(B,C)来解释。

（1）首先解释几个变量的意思

begin_node，是这个函数的局部变量，它表示了B 这个子结构中的开始节点是哪一个。

end_node，是这个函数的局部变量，它表示了B 这个子结构中的结束节点是哪一个。

phase 就是这里 B。

m_parent 是 uvm phase class 的一个变量，它表示了包含 B 的更大一层范畴的结构，注意不是父节点的意思；比如说在图形结构中，子图A 属于更大一点的子图 B，而 B 又属于整个图 C 的一部分，那么 A 的 m_parent 就是 B，B 的 m_parent 就是 C，再次提请注意，是范畴的概念不是单纯的父节点的概念。

m_imp 也是 uvm phase class 的一个变量，我们知道在 uvm 节点图形结构中的每个节点都有一个属性，或者是 UVM_PHASE_DOMAIN，或者是 UVM_PHASE_NODE，或者是 UVM_PHASE_TERMINAL 等，但不会是 UVM_PHASE_IMP 类型；这其中的 UVM_PHASE_NODE 就是一个指针，它所指向的那个东西的类型如果是 UVM_PHASE_IMP 的话，那么这个指针的 m_imp 就是那个 UVM_PHASE_IMP 类型的节点；那么有人可能会问，我也问过的一个问题就是，为什么要这么做，我直接把 UVM_PHASE_IMP 类型的节点插进来不就得了，为什么要插入它的指针，然后需要的时候再用这个指针索引？岂不麻烦和多此一举？我个人觉得可能这样做以后的灵活性和可扩展性会更好，因为我们知道面向对象的最大的威力是组合而不是继承，我们可以想想模式设计中的 brige 模式和 strategy 模式（当然还有很多），就是用这种类似的机制完成了一些继承不能做到或者大大增强了以后的可扩展性和灵活性。

（2）if 分支

如果插入进入的子图（注意我改用子图而不是节点了）类型是 UVM_PHASE_IMP，那么 new 一个指向它的句柄，句柄本身的类型设定成 UVM_PHASE_NODE，然后有三句赋值语句，第一句的意思前面已经在解释变量的时候解释过了，那么第二句和第三局是神马意思？其实意思很简单，只要你清楚一件事实，那就是只有一个节点的东西也叫做子图或图，而不是说非要有很多节点，节点之间或串行或并行的那种复杂的结构才叫子图或者图；只有一个节点的结构是最简单的图形结构。在次情况下，它既是 begin 节点也是 end 节点，这样那两句赋值语句就了然了。

（3）else 分支

有了对（2）的理解，这个分支就不难理解了，能进入 else 分支就表示要进入的这个东西或者说子图肯定不是 if 分支中的简单情况，至少有两个节点或者以上。那好了，begin 节点就是这个子图的句柄，end 节点就是这个子图的end 节点，你加入了我，那么我当然是你的上级，所以 m_parent=this 就是这个意思。

有人读上面（3）的某些话可能觉得读不懂，特别是“begin 节点就是这个子图的句柄”这句话，对这句话的理解涉及到对 uvm phase 图形结构的理解，比如说有个图形结构叫做 A，它有两个节点 B 和 C，按道理来说这个图形结构就是 A->B 就完了，但是 UVM 不是这么做的，它为了处理的方便会在生成图形结构的时候多搞两个节点：一个开始节点，一个终结节点；也就是 uvm 会生成

如下图形结构：

A->B->C->终结节点。

如果觉得还不好理解的话，我再举个特别形象例子，C 语言中的字符串。

比如说一个“hello”的 C 字符串，

上面的 A 就相当于取“hello”的地址，即 A=&“hello”,

上面的终结节点就相当于 C 的 '\0',

上面的 B 和 C 就相当于字符串的真正的内容hello。

4、函数主体三：默认情况

```
if (with_phase == null && after_phase == null && before_phase == null) begin
before_phase = m_end_node;
end
```

就是说，如果你就是用 A.add(B)这种形式的话，那么 B 会加到 A 的最后面，就相当于C 语言字符串中加入字符，如果 A=“hello”，B=“uvm”，那么A.add(B)执行时，上面的before_phase = m_end_node;就相当于before_phase=‘\0 ’，执行完成之后变成了 "hellouvm"。

5、函数主体四：add 操作的真正部分

这部分代码我就不贴了，这个 add 函数剩余的部分都是的，反而我觉得这部分最没什么好讲的，难的反而是类型检查啊、位置定位啊这些前戏。

这部分代码无非就是一些后继和前驱的更新而已，如果你了解数据结构中的链表的插入和删除操作，那么这段代码无非就是 C 换成了 SV，没其他别的东西。如有一个链表 L，内容是 A->B->C，如果想把 D 插入到 B 的前面 A 的后面，那么需要

step1：temp = A.后继

step2：D.后继 = temp

step3：D.前驱 = A

step4：A.后继 = D

如此而已。

add（） 函数讨论到这里我觉得对我们以后应用来说也差不多够了，到此为了吧。

这一贴就讨论到这里，很多细节我没有讲，我觉得抓住主线了，那么枝节的东西不难理解，而且 UVM class 的源代码随处可下，要想弄个清楚明白，读源代码是必不可少的。如果设计到这一贴中的某些细节需要了解，可以联系我，我们私下讨论。

11.UVM 验证平台规范(转)

UVM 验证平台的组成

UVM 预先定义了一个类uvm_component，所有验证组件都由此类派生而来，UVM 使用“树”来组织管理各组件，因此整个 UVM 验证平台如倒置的树，如图 1 所示：

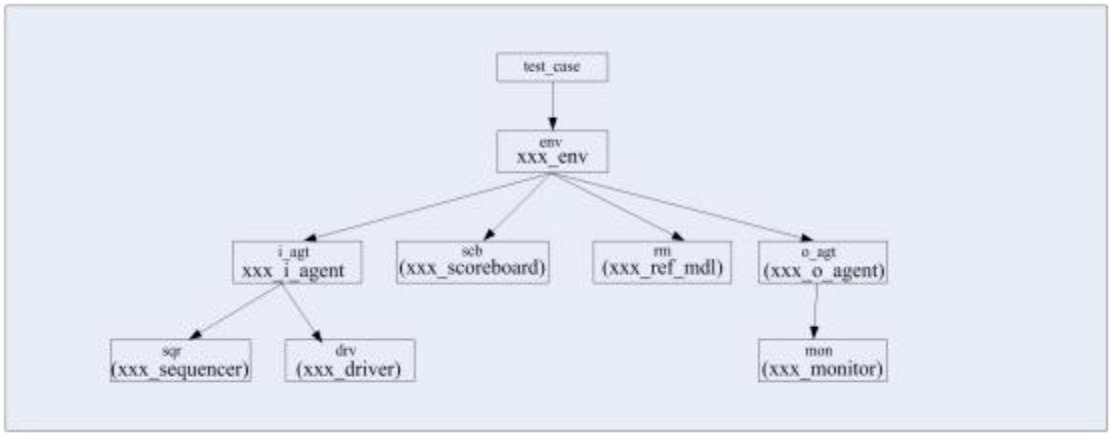


图 1 UVM 验证平台树形结构

本验证平台包含的 UVM component 及其父类如表 1 所示：

表 1 验证平台的 component 及其父类

本验证平台的 UVM component	父类
test_case	uvm_test
xxx_env	uvm_env
xxx _i_agent	uvm_agent
xxx _o_agent	uvm_agent
xxx _scoreboard	uvm_scoreboard
xxx _ref_mdl	uvm_component
xxx _sequencer	uvm_sequencer
xxx _driver	uvm_driver
xxx _monitor	uvm_monitor

验证平台中用到的 component 及 object

本平台涉及的 component 有：

1. test_case：此 component 公共部分在 template.sv 中，trans 和 cfg 约束等放在 test.sv。
2. xxx_env：此 component 把验证平台中固定要使用的 component 都封装起来，当跑不同 case 时，只需在 case 中实例化一个 env 即可。
3. xxx_i_agent：为实现验证平台可重用，xxx_i_agent 将 xxx _driver 和 xxx _sequencer 封装起来。
4. xxx_o_agent：此 component 同 xxx_i_agent 一样，但是只实例化 xxx _monitor。
5. xxx_scoreboard：此 component 接收并比较来自 reference model 和 monitor 的数据，通过比较结果来判断 DUT 是否正确工作。
6. xxx_ref_mdl：此 component 用 systemverilog 语言模拟 DUT 行为，也可以通过DPI接口调用C 语言的函数。
7. xxx_sequencer：此 component 用于组织 sequence，当 driver 要求数据时，它把 sequence 生成的 sequence_item(transaction)转发给 driver。
8. xxx_driver：此 component 向 sequencer 请求 sequence_item 后，将 sequence_item 中的信息按时序要求驱动给 DUT 接口。
9. xxx_monitor：此 component 接收 DUT 端口信息，并将其转换成 transaction 级别的 sequence_item，发送给 scoreboard。

本平台涉及的 object 有：

1. xxx_trans：此 object 派生自 uvm_sequence_item，封装了一定信息。在本平台中针对 DUT 输入输出，分别定义了两个类xxx_i_trans和 xxx_o_trans。
2. xxx_sequence：此 object 派生自 uvm_sequence，是 sequence_item 的组合，也控制验证平台数据流的启动与关闭。
3. xxx_config：此 object 派生自 uvm_object，用于验证平台参数的配置，控制 driver 等 component 的行为。

4. phase：本平台常用的 phase 包括 build_phase，connect_phase，main_phase，run_phase 等，控制 component 的工作流程。

命名规范

本平台遵循以下命名规范：

1. 一般，一个文件只定义一个 class、interface 或者 module，以 class，interface 或 module 名命名文件，后缀为.sv。
2. 各 component/object/interface 类名及其实例化名如表 2 所示：

表 2 文件名、类名及实例化名

文件名	类名	实例化名
test.sv	test_case	
xxx_env.sv	xxx_env	env
xxx_i_agent.sv	xxx_i_agent	i_agt
xxx_o_agent.sv	xxx_o_agent	o_agt
xxx_scoreboard.sv	xxx_scoreboard	scb
xxx_ref_mdl.sv	xxx_ref_mdl	rm
xxx_sequencer.sv	xxx_sequencer	sqr
xxx_sequence.sv	xxx_sequence	seq
xxx_driver.sv	xxx_driver	drv
xxx_monitor.sv	xxx_monitor	mon
xxx_config.sv	xxx_config	cfg
xxx_i_trans.sv	xxx_i_trans	i_tr
xxx_o_trans.sv	xxx_o_trans	o_tr
xxx_pkg.sv	xxx_pkg	pkg
xxx_interface.sv	xxx_interface	xxx _intf

3. xxx_ref_mdl 中各 function/task 必须独立封装，具有形参 input/output，function/task 内部使用局部变量，尽量少使用类成员变量。
4. 除 xxx_config/xxx_i_trans/xxx_o_trans 之外的类，其成员变量命名，需加前缀“m_”，类的对象不需要加前缀。

5. 全局变量命名，需加前缀“g_”。请谨慎使用全局变量，尽量少用。
6. 定义枚举类型，需加后缀“_e”，但枚举型变量不加后缀。
7. interface 声明的变量名与 DUT 端口名保持一致；
8. begin 放在行尾，end 单独一行，如下实例：

```
for(int i=0; i<8; i++) begin
  $display (“like this!”);
end
```

9. 编码使用TAB 键缩进。

port/fifo 命名及连接规则

1. 本平台仅使用了两种类型的 port，uvm_analysis_port 和 uvm_blocking_get_port。类 xxx_driver/xxx_i_agt/xxx_ref_mdl 实例化了 uvm_analysis_port，类 xxx_scoreboard/xxx_ref_mdl 实例化了 uvm_blocking_get_port。
2. 本平台仅使用了一种类型的 fifo，uvm_tlm_analysis_fifo。其命名规则为 A2B_fifo，其中 A 表示 transaction 源，B 表示 transaction 目的。
3. port 与 fifo 连接规则：uvm_analysis_port 连接 fifo 的 analysis_export，uvm_blocking_get_port 连接 fifo 的 blocking_get_export。
4. 本平台中 fifo 与 port 的连接示意图如图 2：方框表示 port，圆表示 export。

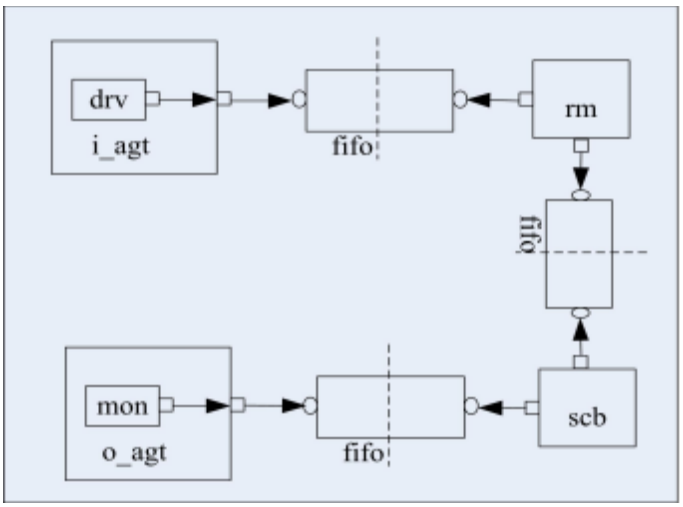


图 2 fifo 与 port 连接示意图

打印信息

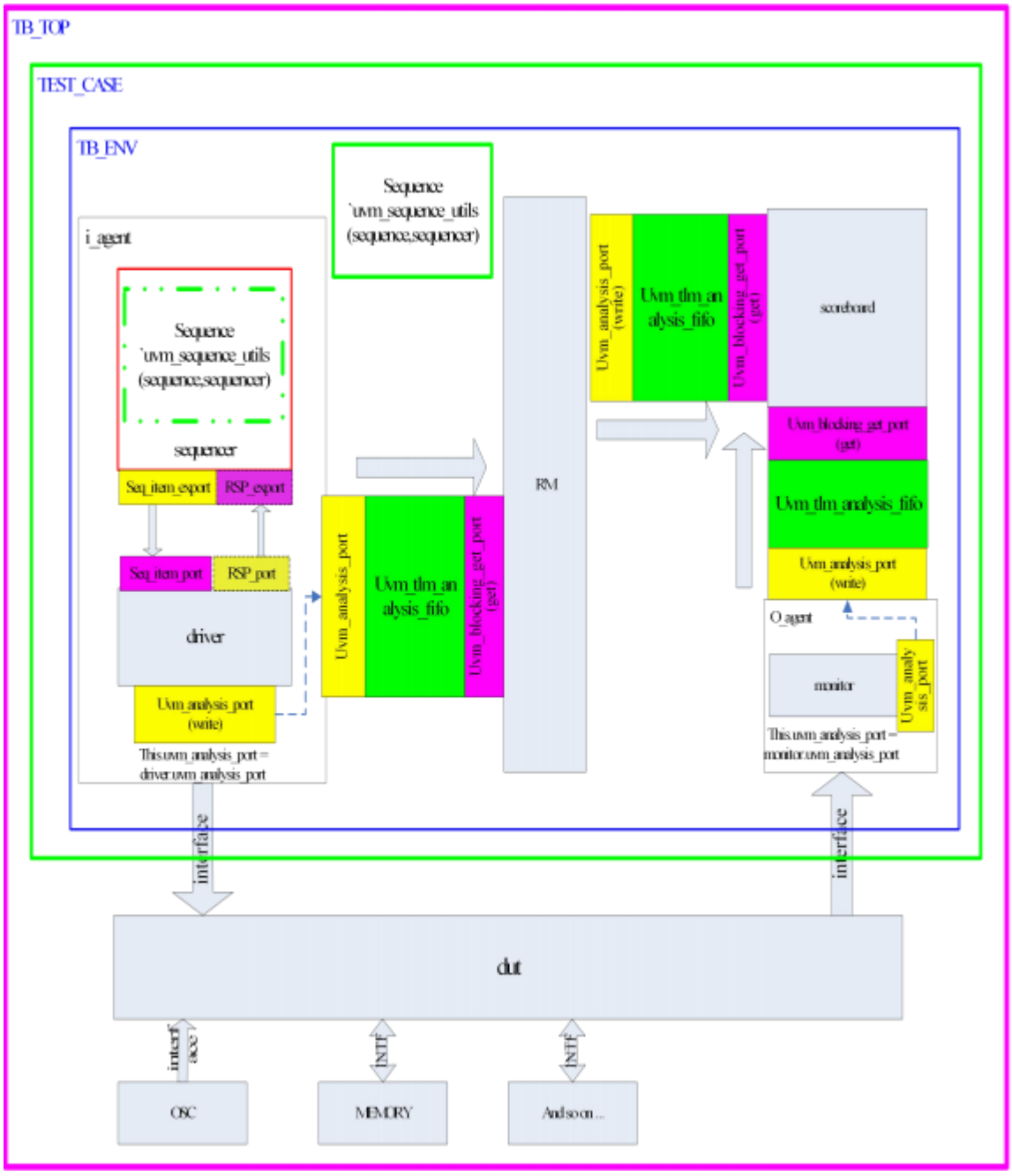
本平台打印信息规则如下：

1. 使用宏打印，即使用`uvm_info，`uvm_warning，`uvm_error，和`uvm_fatal。
2. 默认将 warning/error/fatal 信息打印到 log 中。
3. 有关仿真进程的提示信息，如“reset start”“reset release”等，用`uvm_info 打印，其参数 verbosity 设置为 none，其他 debug 信息，如打印中间变量值等，也用`uvm_info，但参数 verbosity 设置为medium。
4. 打印数据时，若为 16 进制表示，则需加“0x”前缀；若为 2 进制表示，则需加“0b”前缀；若为 10 进制，则不需加前缀，以示区别。

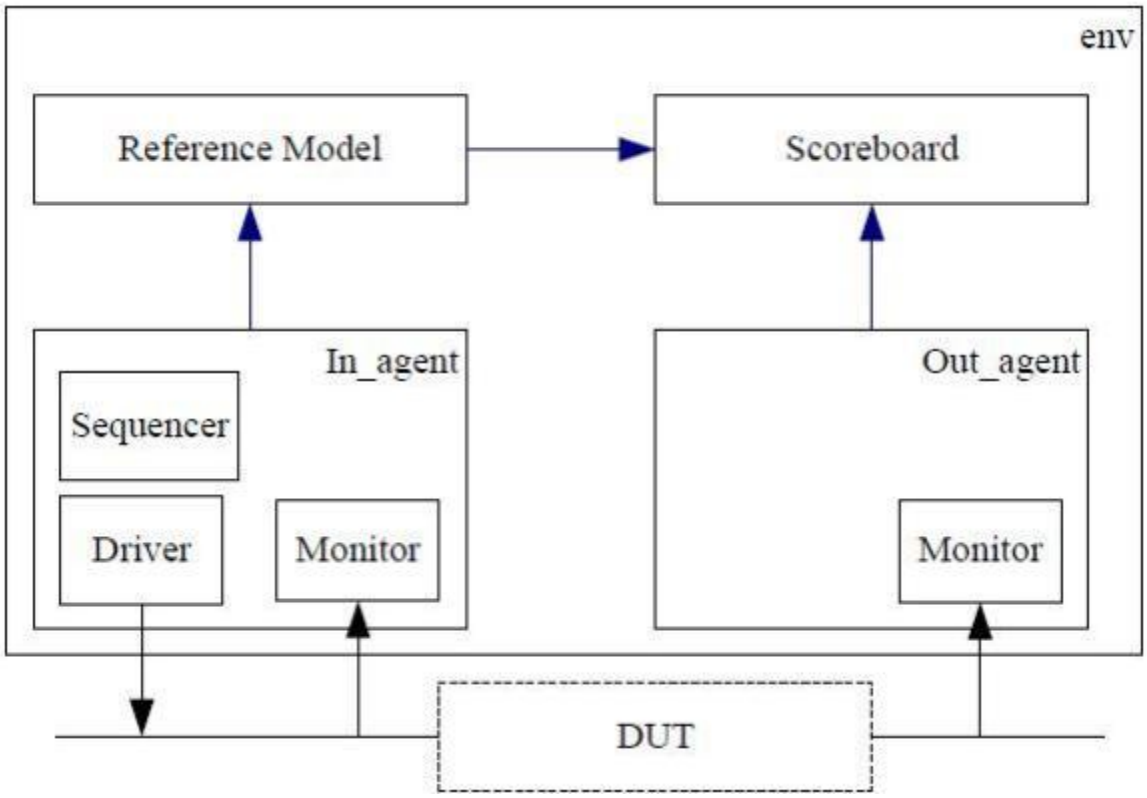
12.UVM 寄存器扫描环境分析

通用UVM 验证环境介绍

验证平台总图



UVM 环境：



UVM 树形层次关系 ：

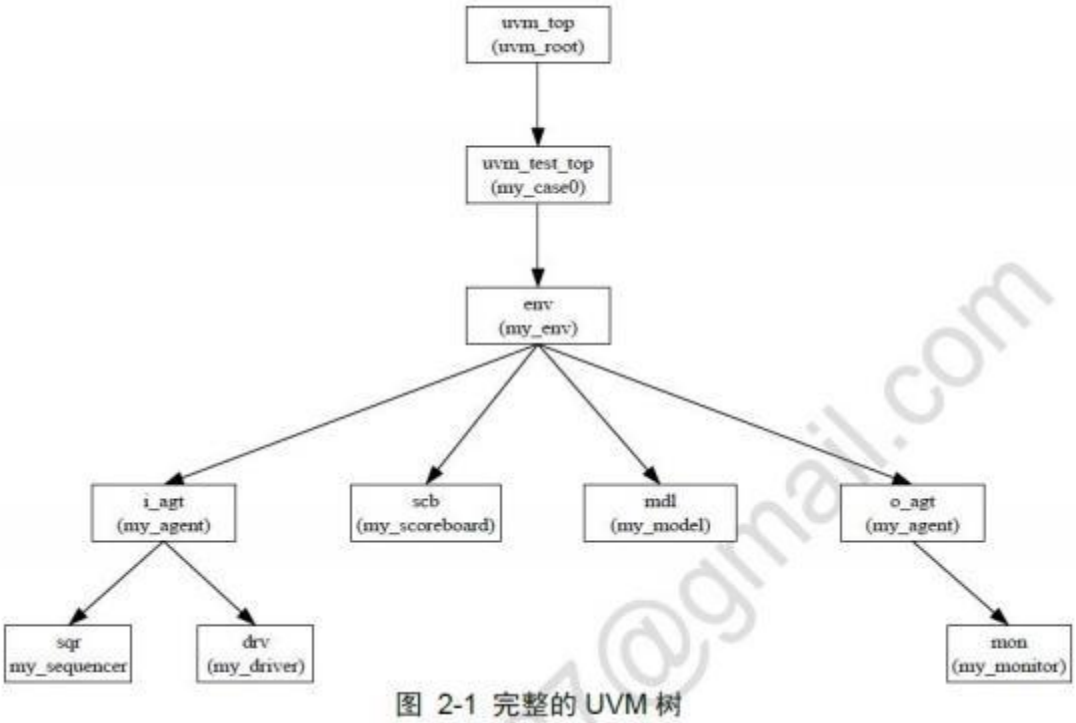


图 2-1 完整的 UVM 树

验证平台中各个模块作用简介:

Transaction:此 class 用来封装一个对象的参数,其扩展自 uvm_sequence_item;

Sequence:此 class 用来产生一系统的 transaction,它为扩展自 uvm_sequence 的参数类,参数种类为 transaction;

上面两个类实际上是 object 类型,他们不属于 env 的 componet

Sequencer:此 class 是把 sequence 产生的 transaction 给 driver;

Driver:此 class 是根据 transaction 的特性对 dut 进行驱动

Ref_mdl:此 class 是 dut 的功能模型

Monitor:此 class 是对 dut 数据的采集,

Scorebord:此 class 是用来对比 ref_mdl 和 monitor 数据,来断定结果.

UVM 启动过程

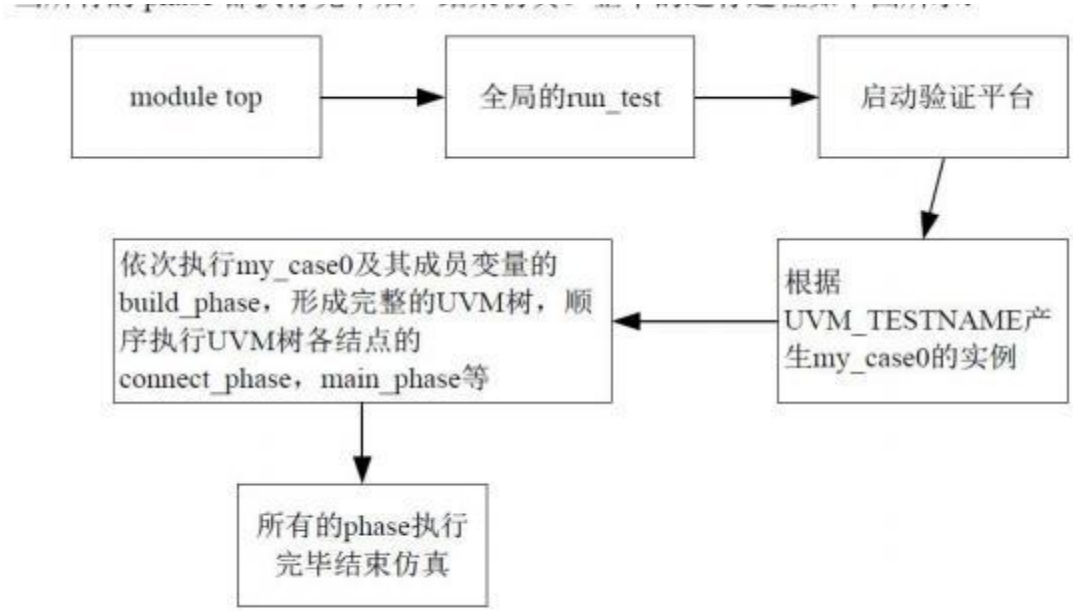


图 1-9 UVM 验证平台执行流程

启动过程分析:

在 module_top 中有

```
Initial begin
  Run_test(test_case);
End
```

Run_test();此为全局函数,在 uvm_root.svh 中可以找到,在 run_test 函数中有几个关键语句如下:

1.1test_name_count =

clp.get_arg_values(“+UVM_TESTNAME”,test_names); 该语句能够采用在 MAKEFILE 中增加定义+UVM_TESTNAME 来启动需要的 test_case;

1.2uvm_phase::m_run_phase 调用 uvm_phase 中的 m_run_phase task,

1.3m_run_phase 调用 execut_phase();此时进入执行各个 phase

由于 sequence 不属于 env 的组件,正常情况下运行 run_phase,sequnce 是不会运行的,也就是无法产生 transaction;所以在环境中需要主动调用 sequence 的 start 函数;调用方法有多种,参考 6.1.2sequence 的启动与执行;

其实 sequence 与 drive 会采用 seq_item_port 同步，在 sequence 中会有 `uvm_send(`uvm_do 包含此动作)，drive 会通过 seq_item_port.get_next_item(tr),获得数据包，当发送完毕后，再通过 seq_item_port.item_done()告诉 sequence 发送完毕，此时 sequence 可以根据实际情况 delay 一些时钟结束此 phase;

所以,UVM run 的关键点之一是,在那里启动 sequence 的 start 函数来产生 transaction.

在调用 sequence 的 start 时要穿入 sequencer 参数,即 sequence.start(sequencer);

项目模板中是在 tc 中设置 default_sequence,然后 sequencer 在 main 中启动 default_sequence ;

Uvm_sequencer 最终扩展的基类是 uvm_sequencer_base(uvm_sequencer_base 扩展自 uvm_component),其 run_phase 的原形如下:

```
Task  uvm_sequencer_base::run_phase(uvm_phase phase);
  Super.run_phase(phase) ;
  Start_default_sequence();
Endtask
```

Testcase 用途

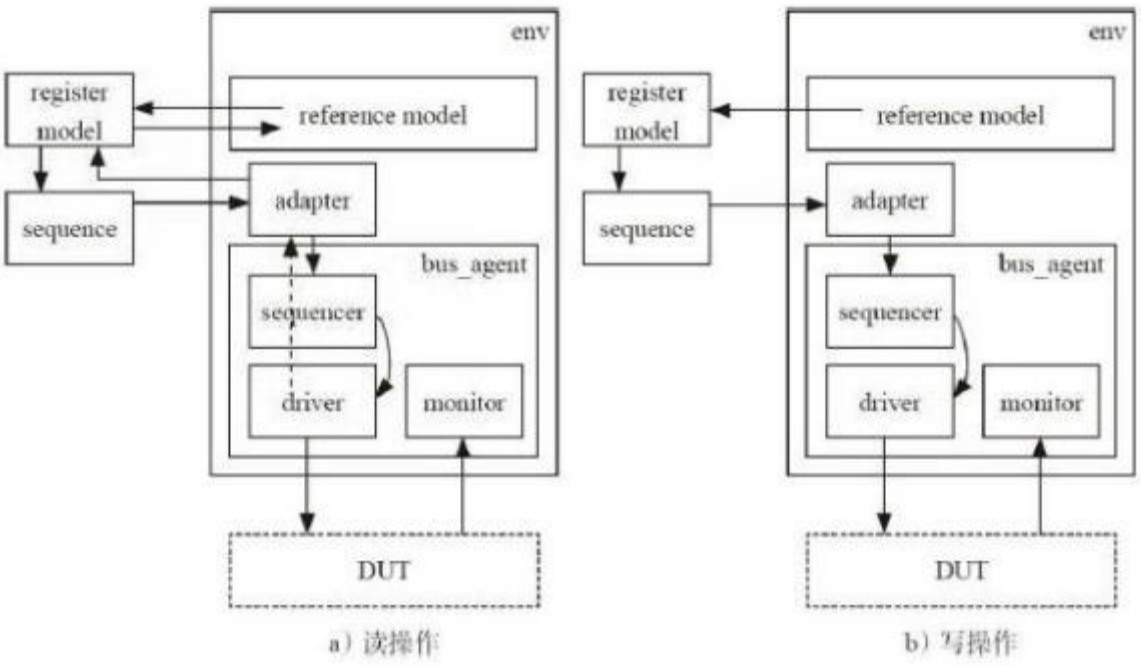
上面分析 uvm 启动过程是基于 uvm环境建立好及引导 run_test 工作以准好好的条件下实现的,所以要实现这些过程还需要一个类来配合使用,这个类就是扩展于 uvm_test 的各种 test_case;

Test_case 有以下几个用途:

1. 与 run_test 对接来启动整个验证环境的运行
2. 通过 factory 机制产生 env
3. 设置本 testcase 需要的 sequence 为 default_sequence
4. 根据需要通过 override 的方式改变 env 中的各个组件

UVM 寄存器验证环境介绍

寄存器扫描验证环境总图



根据项目中的寄存器验证环境来看,实际上不存在referance model;

register model 把 reference mode 和 scoreboard 的工作全部完成,在 UVM 电子书上的构架如下:

环境中各模块的作用:

Register_model:寄存器模型,里面包含了寄存器所有的信息

Sequence:它扩展自 uvm_reg_sequence#(uvm_reg_item)主要是用来产生寄存器操作的参数,如读写,地址,数据等

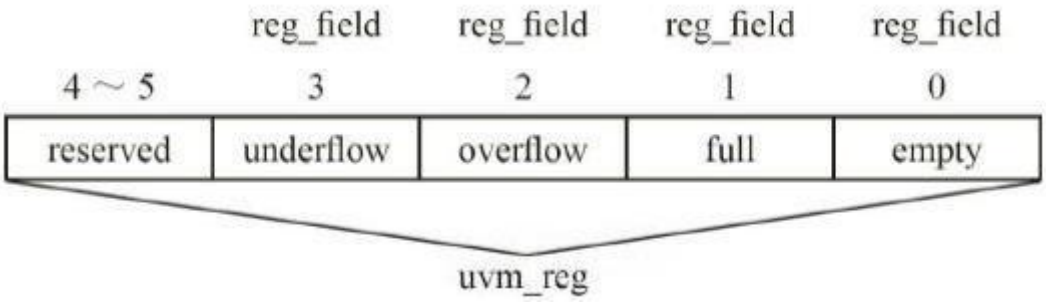
Adapter:它是一个转换器,在寄存器模型侧,其操作的参数时固定的,但 DUT 寄存器的总线各式各样,所以需要做一个转换

Sequencer,driver,monitor 的作用与通用 UVM 验证平台一致.其中在读返回数据的时候,并未通过 sequencer.因为 sequencer 只是发送 Sequence 产生的 transaction(uvm_reg_item),而不会回收数据,只发不收.

关键组件分析

Register_model

1.uvm_reg_field：寄存器模型中的最小单位。



2.uvm_reg：比 uvm_reg_field 高一个级别，但依然是比较小的单位。一个寄存器中至少包含一个 uvm_reg_field。

3.uvm_reg_block：比较大的单位，可以加入许多 uvm_reg，也可以加入其它的 uvm_reg_block。一个寄存器模型中至少包含一个 uvm_reg_block。

4.uvm_reg_map：每个寄存器加入模型时都有地址，uvm_reg_map 存储这些地址，并转换成可以访问的物理地址（因为寄存器模型中的寄存器地址一般

是偏移地址，不是觉得地址）。使用前门时，uvm_reg_map 会将地址转换成绝对地址。每个 reg_block 内部，至少有一个 uvm_reg_map（通常也只有一个）。

Uvm_reg_field:它扩展于 uvm_object, Uvm_reg_field class 内部定义了寄存器位域的所有特性,如值,读写属性,尺寸,复位值,期望值……等;同时也定义了几个关键方法

```
Function void configure(…);
Virtual function void reset(…)
Virtual task write(….)
Virtual task read(….)
Virtual task poke(….)
Virtual task peek(….)
Virtual task mirror(….)
Funciont void set_compare(….)
Funciont uvm_check_e get_compare(….)
Funciont bit predict(….)
Funciont uvm_reg_data_t  XpredictX(….)
Funciont uvm_reg_data_t  XupdateX(….)
Funciont bit Xcheck_accessX(….)
Virtual task do_write(…)
Virtual task do_read(…)
Virtual Funciont void do_predict(….)
```

uvm_reg:它也是扩展自 uvm_object, uvm_reg class 内部定义了寄存器的所有特性,其中包括定义了 uvm_reg_field 和 uvm_reg_block 变量;同时也定义了几个关键方法:

```
Function void configure(…);
Virtual Function void offset(…);
Virtual Function void add_field(…);
Virtual Function void add_map (…);
```

Function void Xlock_modelX(…);

….

Virtual task mirror(output uvm_status_e status,

Input uvm_check_e,……)//此函数是个关键,它会用来做各种操作及对比

Virtual Function uvm_reg_map get_default_map (…);

```
class reg_invert extends uvm_reg;

    rand uvm_reg_field reg_data;

    virtual function void build();
        reg_data = uvm_reg_field::type_id::create("reg_data");
        // parameter: parent, size, lsb_pos, access, volatile, reset value,
       // has_reset, is_rand, individually accessible
        reg_data.configure(this, 1, 0, "RW", 1, 0, 1, 1, 0);
    endfunction

    `uvm_object_utils(reg_invert)

    function new(input string name="reg_invert");
        //parameter: name, size, has_coverage
        super.new(name, 16, UVM_NO_COVERAGE);
    endfunction
endclass
```

Uvm_reg_block: 它也是扩展自uvm_object,内部定义了block,reg,mem,map等数组及其它变量, 同时也定义了几个关键方法:

Function void configure(…);

Virtual Function void uvm_reg_map create_map (…);

Function void set_default_map (…)

Function void add_block(…)

Function void add_map (…)

Function void add_reg (…)

Function void add_vreg (…)

Function void add_mem(…)

Virtual function void lock_model()

.....

```
class reg_model extends uvm_reg_block;
   rand reg_invert invert;

   virtual function void build();
      default_map = create_map("default_map", 0, 2, UVM_BIG_ENDIAN, 0);

      invert = reg_invert::type_id::create("invert", , get_full_name());
      invert.configure(this, null, "");
      invert.build();
      default_map.add_reg(invert, 'h9, "RW");
   endfunction

   `uvm_object_utils(reg_model)

    function new(input string name="reg_model");
        super.new(name, UVM_NO_COVERAGE);
    endfunction

endclass
```

Uvm_reg_map: 它也是扩展自 uvm_object,它内部定义寄存器所有的信息,包含地址,位宽,大小端等;其包含了如下一些方法

Function void configure(…);

Virtual Function void add_reg (…);

Virtual Function void add_mem(…);

Virtual Function void set_sequencer(…);

….

Virtual Function void set_base_addr(…);

…….

Virtual task do_bus_write(…)

Virtual task do_bus_read(…)

Virtual task do_write(…)

Virtual task do_read(…)

……….

由上述可见,每个层次都需要手动添加,它们都扩展于 uvm_object,没有自动调用的功能,需要手动调用.reg_model 不能成为 env 的组件,在 tc 或 env 中例化

通过分析上述 reg_model,它是个 object,没有 phase 概念,不能自动运行,需要在其他的组件中调用,通用的 uvm 平台可以在 sequence 中产生

transaction;uvm_reg 平台也可以在 sequence 中产生 uvm_reg_item;正常情况下,需要手动调用很多reg_model 的方法来产生uvm_reg_item,但uvm 有几个重要的 sequence 来完成寄存器的扫描工作;

uvm_sequencer #(rw)

uvm_sequencer 的原形是

```
class uvm_sequencer
#(type REQ=uvm_sequence_item,RSP=REQ)  extends
uvm_sequencer_param_base #(REQ,RSP);
  ……..
Extern virtual task get_next_item(output REQ t);
Extern virtual task tryt_next_item(output REQ t);
Extern virtual function void item_done(RSP item = null);
Extern virtual task put(RSP t);
Extern virtual task get(output REQ t);
……..
endclass
```

从其内容可以看出, uvm_sequencer 只是提供了对端口的操作,都是被 uvm_reg_sequence 调用来把 uvm_reg_item 传给 driver,并作应答机制

Adapter:

这个class 比较简单,就是用来做两边协议的转化,不同的项目需要做不同的处理

```
class my_adapter extends uvm_reg_adapter;
    string tID = get_type_name();

    `uvm_object_utils(my_adapter)

   function new(string name="my_adapter");
      super.new(name);
   endfunction : new

   function uvm_sequence_item reg2bus(const ref uvm_reg_bus_op rw);
      bus_transaction tr;
      tr = new("tr");
      tr.addr = rw.addr;
      tr.bus_op = (rw.kind == UVM_READ) ? BUS_RD: BUS_WR;
      if (tr.bus_op == BUS_WR)
         tr.wr_data = rw.data;
      return tr;
   endfunction : reg2bus

   function void bus2reg(uvm_sequence_item bus_item, ref uvm_reg_bus_op rw);
      bus_transaction tr;
      if(!$cast(tr, bus_item)) begin
          `uvm_fatal(tID,
           "Provided bus_item is not of the correct type. Expecting bus_transaction")
           return;
      end
      rw.kind = (tr.bus_op == BUS_RD) ? UVM_READ : UVM_WRITE;
      rw.addr = tr.addr;
      rw.byte_en = 'h3;
      rw.data = (tr.bus_op == BUS_RD) ? tr.rd_data : tr.wr_data;
      rw.status = UVM_IS_OK;
   endfunction : bus2reg

endclass : my_adapter
```

但 Adapter 怎么用呢? 在 tb_env 的 connect_phase 中有这样一句话,regmodel.default_map.set_sequencer(apb.sqr, reg2apb);把Adapter 与 sequencer 关联起来

综合所述:上面完成了寄存器和 driver 两边协议的转化.

Sequence:

uvm_reg_sequence#(uvm_reg_item);其内部定义了

Class uvm_reg_sequence #(type BASE=uvm_sequence #(uvm_reg_item))

extends BASE

uvm_reg_block model;

uvm_reg_adapter adapter;

uvm_sequencer #(uvm_reg_item) reg_seqr;

virtual task body ();

……..

Forever begin

```
        Uvm_reg_item  reg_item;
        Reg_seqr.peek(reg_item);
        Do_reg_item(reg_item);
        Reg_seqr.get(reg_item)
    end
endtask
endclass
```

其中uvm_reg_item 定义了寄存器操作相关的变量,详细参看 uvm_reg_item.svh

从上面 forever 循环中可以看到 sequence 的动作:1 产生一条 reg_item -> reg_seqr peek 此条reg_item -> 根据uvm_reg_item 中的kind产生 adapter 定义的操作 driver 的变量-> uvm_sequencer get 出当前的 uvm_reg_item;

关键 Sequence 说明

uvm_reg_hw_reset_seq

这个类扩展自 uvm_reg_sequence,主要是用来检查上电复位后寄存器模型与 DUT 寄存器的默认值是否相同

1).对于 DUT，复位完成后，就是默认值。但是对寄存器模型，如果只是集成到验证平台，不做任何处理，所有值都是 0，此时要调用reset 函数使其寄存器的值变为默认值：

```
function void base_test::build_phase(uvm_phase phase);
…
   rm = reg_model::type_id::create("rm", this);
…
   rm.reset();
…
endfunction
```

2).这个 sequence 检查前会调用 model 的 reset 函数，所以即使 user 没有调用 reset，这个 sequence 也能正常工作（它会自动调）。

3).除了reset 外，这个 sequence 使用前门读取所有寄存器的值，与寄存器模型比较。

4).此 sequence 也要指定 model 变量。

5).如果想跳过某个寄存器的检查，可在启动此 sequence 之前用个 resource_db 设置。

6).resource_db 与 config_db 底层机制实现是一样的，uvm_config_db 实参 uvm_resource_db 派生来的。

使用 resource_db 进行设置：

```
class case0_cfg_vseq extends uvm_sequence;

   `uvm_object_utils(case0_cfg_vseq)
   `uvm_declare_p_sequencer(my_vsqr)

   function  new(string name= "case0_cfg_vseq");
      super.new(name);
   endfunction

   virtual task body();
      uvm_reg_hw_reset_seq ckseq;
      if(starting_phase != null)
         starting_phase.raise_objection(this);
      ckseq = new("ckseq");
      ckseq.model = p_sequencer.p_rm;
      ckseq.start(null);
      if(starting_phase != null)
         starting_phase.drop_objection(this);
   endtask

endclass
```

```
function void my_case0::build_phase(uvm_phase phase);
   super.build_phase(phase);

   uvm_config_db#(uvm_object_wrapper)::set(this,
                                           "v_sqr.configure_phase",
                                           "default_sequence",
                                           case0_cfg_vseq::type_id::get
   uvm_config_db#(uvm_object_wrapper)::set(this,
                                           "v_sqr.main_phase",
                                           "default_sequence",
                                           case0_vseq::type_id::get());
   uvm_resource_db#(bit)::set({"REG::",rm.invert.get_full_name(),".*"},
                              "NO_REG_TESTS", 1, this);
   uvm_resource_db#(bit)::set({"REG::",rm.invert.get_full_name(),".*"},
                              "NO_REG_HW_RESET_TEST", 1, this);

endfunction
```

uvm_reg_bit_bash_seq

前门读写逐个 bit 扫描

uvm_reg_access_seq

寄存器,存储器读写属性测试

下面是与 memory 相关的 sequence

uvm_do_mem_access_seq

存储器读写属性测试

Uvm_do_shared_access _seq

共享存储器读写属性测试

uvm_do_mem_walk_seq

存储器读写逐个测试

uvm_reg_mem_hdl_paths_seq

检查hdl 路径的正确性

启动此 sequence 时必须给model 赋值。在任意的 sequence 中，可以启动此 sequence：

```
class case0_cfg_vseq extends uvm_sequence;

    `uvm_object_utils(case0_cfg_vseq)
    `uvm_declare_p_sequencer(my_vsqr)

    function  new(string name= "case0_cfg_vseq");
       super.new(name);
    endfunction

    virtual task body();
       uvm_status_e   status;
       uvm_reg_data_t value;
       uvm_reg_mem_hdl_paths_seq ckseq;
       if(starting_phase != null)
          starting_phase.raise_objection(this);
       ckseq = new("ckseq");
       ckseq.model = p_sequencer.p_rm;
       ckseq.start(null);
       if(starting_phase != null)
          starting_phase.drop_objection(this);
    endtask

endclass
```

1）start 传入的 sequencer 为 null。因为不依赖这个 sequencer，而依赖于 model 变量。

2）这个 sequence 会试图读取hdl 所指向的寄存器，如果无法读取，报错。

3）还可以检查存储器。

4）如果某个寄存器，存储器在加入寄存器模型时没有指定其hdl 路径，那么检查时会跳过这个寄存器，存储器

8 uvm_reg_mem_shared_access_seq

uvm_reg 关键概念

基本概念：

1、值

1） DUT 值： 寄存器在 DUT 中的值。

2) mirrored 值： 在 reg model 中维护的一个值，尽可能保持与 DUT 中寄存器的值保持同步。（其实是通过 write/read、peek/poke 时都更新 mirrored 来做到的）

3) desired 值： 渴望值。

2、前门 vs 后门

这两个是 write/read、update、mirror 操作的两种模式。

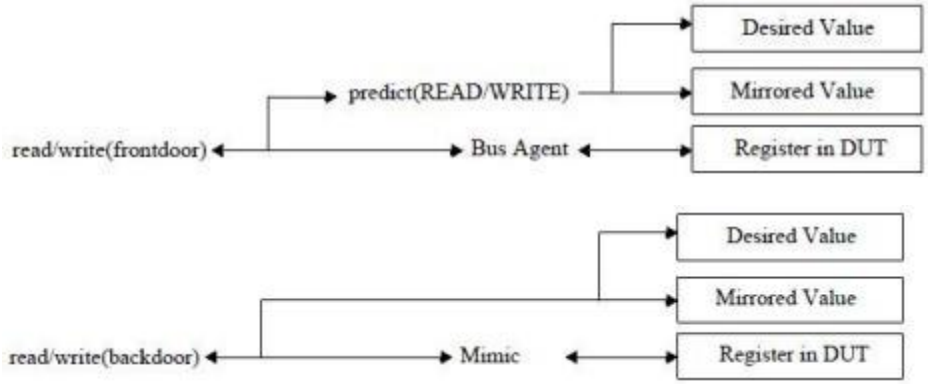
1) 前门：通过物理时序对 DUT 进行操作。占用仿真时间（不是服务器 CPU 时间）

2) 后门：依据寄存器类型，模仿其行为，通过层级引用直接操作 DUT 中的值，不会产生时序。不占用仿真时间。使用后门操作得到的 desired 值、mirrored 值、DUT 值，和使用前门操作得到的值，是一致的。也就是说，仅仅从这三个值的角度来说，前门操作和后门操作没有区别。

基本操作：

1、read/write

普通的读取/写入。分为前门/后门。每次操作时，除了改变 DUT 值之外，还会根据寄存器类型来预测并更新 mirrored、desired 值。数据流图如下（下同）。



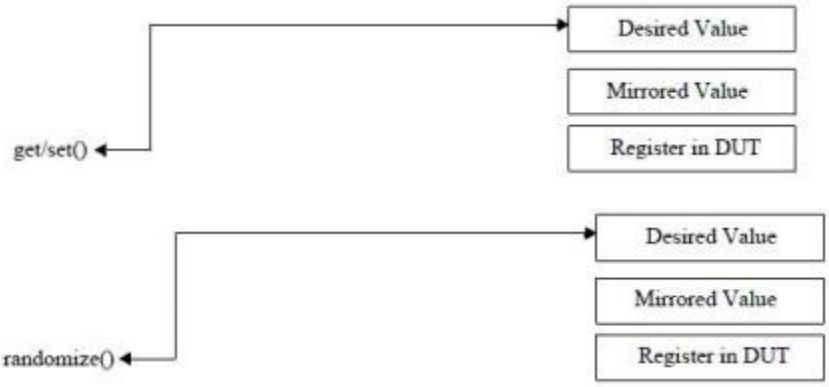
2、peek/poke

通过层级引用直接读取/写入 DUT 中的值。不占用仿真时间。同时直接将值更新到 desired 值、mirrored 值、DUT 值。这两个操作是相当暴力的，它

们并不会受寄存器类型（功能）的影响，只是将寄存器看成一个空间的读写，不会有寄存器本身功能的副作用（如读后清零、读后置位之类）。

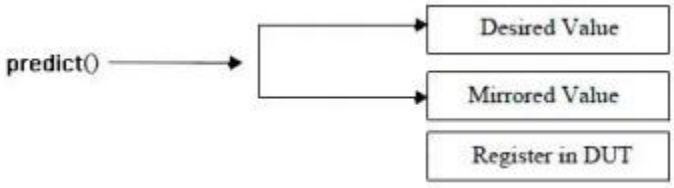
3、get/set/randomize

只对 desired 值进行操作，并不会影响 mirrored 值和 DUT 值。randomize 是随机化后改变 desired 值。



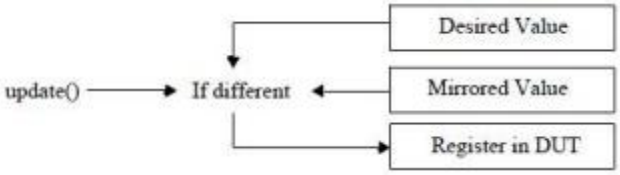
4、predict

对 desired 值和 mirrored 值进行操作，并不会影响 DUT 值。



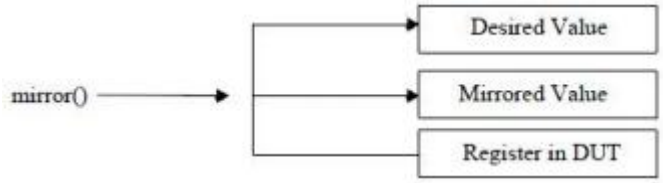
5、update

检查 desired 值与 mirrored 值是否一致，如不一致，则将渴望值写入 DUT，并更新 mirrored 值。模式分为前门/后门。



6、mirror

读取 dut 值，并更新 desired 值和 mirrored 值。模式分为前门和后门。



基本操作之间的比较

1、write(backdoor)和 poke

相同：都是通过层级引用直接写入 DUT 中的值。不占用仿真时间。

不同：write(backdoor)会依据寄存器类型，模仿其行为，而 poke 不会。

例子：暂无~

2、read(backdoor)和 peek

相同：都是通过层级引用直接读取 DUT 中的值。不占用仿真时间。

不同：read(backdoor)会依据寄存器类型，模仿其行为，而 peek 不会。例子：对于读清类型(RC)的寄存器，使用 read(backdoor)操作后，寄存器 DUT 值会清零（相应 mirrored 值也会清零）；而使用 peek 操作后，寄存器 DUT 值不会清零（相应 mirrored 值也不会清零）。

各个基本操作对三个值的影响汇总

操作		相关的值		
		desire	mirror	DUT
write	frontdoor	√	√	√
	backdoor	√	√	√
read	frontdoor	√	√	√

	backdoor	√	√	√
peek		√	√	√
poke		√	√	√
get	update 前	√	×	×
	update 后	√	√	√
set	update 前	√	×	×
	update 后	√	√	√
randomize	update 前	√	×	×
	update 后	√	√	√
predict		√	√	×
update	frontdoor	√	√	√
	backdoor	√	√	√
mirror	frontdoor	√	√	√
	backdoor	√	√	√

寄存器扫描工作流程

在某个 component 中启动某类 sequence,Sequence 调用 register_model 的 read,write 等操作,把 reg_item 传给 sequencer,而 sequencer 调用 adapter,把 reg_item 转化为 driver 需要的变量,再发送给 driver;当为读操作时,返回的数据直接从 driver 返回给 adapter,在到 register_model;进行相关操作及对比,并给出比较结果.

13.uvm 中基类中已有常用变量汇总

uvm_driver

1.seq_item_port

```
uvm_seq_item_pull_port #(REQ, RSP) seq_item_port;
```

此端口为 driver 和 squencer 连接的一个端口,常用的方法为:
seq_item_port.get_next_item(req);
作用为从 squencer 得到一个 item(transaction),req 为 transaction 的句柄,在 squence 中定义的一个变量;
seq_item_port.item_done();完成了发送 item,其实就是一个握手信号,等待当前 item 发送完,然后再去取得下一个 item,用于while 中的握手行为;
2.req

```
REQ req; //REQ 来自于 class uvm_driver #(type REQ=uvm_sequence_item,
                    type RSP=REQ) extends uvm_component;
```

sequence

starting_phase
这个变量是在 uvm_sequence_base 声明的一个变量/句柄
uvm_phase starting_phase;用于控制进程

14. transaction 中常用宏介绍

`uvm_create:实例化 transacton,相当于 new
`uvm_send:把 transacton 发送出去
`uvm_rand_send:对 transaction 随机化后发送出去
在 soc 系统中,一般是先`uvm_create,然后赋值,再`uvm_send.

低功耗验证

1.Cpf+ncverilog 低功耗验证

2.upf+ncverilog 低功耗验证(转)

本文是记录项目过程中遇到的奇巧淫技，如有遗漏或者不足，请大家改正和补充，谢谢。

随着深亚微米技术的普及与发展，leakage 功耗在整个功耗中的比重越来越大，比如45nm 下，已经占到了60%以上，所以低功耗解决方案应运而生。目前已经有一套标准的低功耗设计流程，流程有 CPF（cadence 主导）和 UPF（synopsys 主导）两种，但技术趋势是 UPF 会大一统，所以本篇将为那些仍旧使用ncverilog 而不是vcs 仿真工具的苦逼们提供一些参考。

目前常用的降低低功耗的方法有四种：多电压域、时钟门控、电源关断和动

态电压频率调整。其中的时钟门控对验证影响较小，大家应该都接触过，而剩下的三个对验证工作影响较大，需要用到各个 EDA 厂商的低功耗解决方案。闲话就说这么多，那么 cadence 如何使用 upf 来实现低功耗流程的仿真验证呢？

第一步，先得有 UPF 文件，根据设计需求，使用TCL 建立脚本，建立和管理独立电压源、确定隔离、建立电平漂移等，一般是设计或者后端人员书写，验证工程师当然也可以写，具体内容参考 IEEE 1801。

第二步，仿真 case 中添加电源上电过程，使用$supply_on 函数给相应 VDD 上电。

第三步，将 UPF 嵌入到仿真命令中，即：

irun -lps_1801 sim.upf -lps_assign_ft_buf -lps_iso_verbose ...

或者

1 ncvlog ...

2 ncelab -lps_1801 sim.upf -lps_assign_ft_buf -lps_iso_verbose ...

3 ncsim ...

相关 options 解释如下：

①-lps_1801 filename: 指定符合 IEEE 1801 标准的 UPF 文件；

②-lps_assign_ft_buf: 指定 assign 赋值被当做 buffer 对待，而非默认的 wire，好处是从 always-on domain 进入和穿过 power-down domain 的信号被 force 成 x，便于 debug；

③-lps_const_aon: 对处在 power-down domain 并且直接和 always-on domain 相连接的 tie-high 或者 tie-low constant，不使能 corruption 功能；默认不使用该功能；

④-lps_enum_rand_corrupt：对于用户定义的 enum 类型数据，在电源关断后，随机从枚举列表中选择一个值作为变量值；和该命令相类似的还有 -lps_enum_right, -lps_implicit_pso 等，因为不常用，就不一一介绍了；

⑤-lps_iso_verbose: 使能 isolation 的 log 功能，这个一般需要加上；

⑥...

其它的 options 请参考 cadence 的 low-power simulation guide。

最后，运行仿真即可。

最后说一下 low power 流程验证正确性和完备性的确认方法。

①增加的 low power 流程不能影响芯片本身功能的正确性，比如通过电源关断来降低芯片功耗，则关断再打开后，芯片还可以正常work。这部分可以利用原有的 self-check 验证环境来确认。

②利用 log 文件分析，上述仿真命令-lps_verbose 和 UPF 文件会为仿真输出与 low power 相关的 warning, error 以及 assertion 信息，通过 log 文件可以 check 流程的正确性；

③自动的 assertion checker 分析，使用-lps_verify 选项可以自动检查电源关断顺序（隔离->复位->断电）和电源打开过程（上电->解复位->解隔离），如不满足该时序要求，会自动报错；

④利用覆盖率保证验证完备性，使用-lps_vplan 选项依据 UPF 文件生成 low power 验证计划，通过 simvision 提供的 coverage 接口，得到 low power 相关的功能覆盖率；

验证总结

验证思路

1.信号验证的对象分为数据流和控制流

(1)数据流包含,总线数据,flash,memory之类,它们需要验证是否错位,移位等失误,最大值,最小值,尤其要注意是否会溢出.

(2) 控制流包含各种使能信号,启动信号等,它们需要严正各个对应的功能点是否正确,包含对错两个面

2. 整个模块验证的对象一般为某个流程或通讯,它们需要验证的有

(1)正常流程是否能工作,

(2)当在流程中又来一个启动信号,该如何处理

(3)当在流程中出现一个错误该如何处理

(4)测试这个流程结束之后的各种状态能否保持或自动清 0 或归位等行为是否符合预期

(5)在整个流程工作中,如果配置出现变化,是该忽略还是立即更改，设计是否符合预期

3. 串行协议类验证

(1) 按照格式把数据把送完毕后，需要注意验证到最后一个数据是 0 和 1 两种情况。

(2) 串行时钟和数据最后状态的组合情况需要测试到。--最后时钟是高电平数据时 0 或 1，时钟是低电平数据是 0 或 1 等情况，主要是验证会不会影响下次通讯。

4. 多时钟模块验证

(1) 如果某个模块有多个时钟，如计数器，定时器,uart等模块，他们又一个系统时钟，还一个计数用的时钟，这类设计需要注意验证到他们时钟之间的关系，系统时钟大于，等于，小于计数时钟等几种情况都要注意

(2) 在模块运行过程中，在两个时钟不同关系下，改变配置，如何影响结果，与 2 述相同

系统验证方案

此文档需要列出整个系统的功能点，如电源的控制，时钟的控制，复位的控制及其它异常的控制，系统上电，系统启动等等方面，这个文档可以根据系统设计方案来实现

1. 把所有需要验证的点在验证前尽量提取全，尽量做到”事前做，而非事后补“；
2. 特别需要注意时钟复位模块对时钟的使能，因为模拟晶振，电源都有一段稳定时间，所以验证的时候需要注意上电复位阶段的稳定时间与模拟的 spec 是否一致
3. 写文档要采用 top_down 结构，同时需要图文并茂的方式描述数据流，在开始需要介绍各种概念，定义等名词，或说是总-》分的方式展现系统的构架

功能点测试

每个模块都是为了实现某些功能而存在，所有的功能有各个小的功能点构成；这些小的功能点要运行起来都有其触发条件，及运行结果，所以在设计和验证时都需要考虑两个方面：

b) 所有的 soc 产品基本都会有如下几个模式

正常模式: 正常工作流程,也是我们验证的主要点,其它下面各模式统称为 DFT 模式(design for test);

◎Scan/atpg 模式:主要是通过扫描链来测试数字部分逻辑

需要注意验证的地方有：

1. scan 模式寄存器独立扫描系统，寄存器复位，移位，该寄存器一直为 1
2. Scan_rst 信号能够复位除 scan 模式寄存器外的所有寄存器
3. Scan_clk 能够控制所有寄存器的时钟

◎Mbist 模式:主要是用来测试 soc 中所有的memory

需要注意验证的地方有：

1. 能够报告错误情况，即正反验证
2. 注入错误的时候需要注入边界错误，这样才表示mbist有对整个选择的 ram 进行扫描
3. 注入错误的时候需要对每个 ram 进行注入错误

◎模拟测试模式:主要是用来测试 soc 中各个模拟模块的功能或性能

◎Debug模式:主要是 soc 产品基本都需要软件开发,则需要在线调试,其测试的内容于正常模式的内容一致

◎flash 测试模式:只有 soc 产品用到了 flash 才会有,用来专门测试 flash.

1.每个功能点的触发条件有哪些?

2.该功能实现后会产生哪些影响?

3.需要理解每个模块的功能及作用，即需要知道为什么做这些功能，为了什么。

测试点

对于 SOC 即为某个功能的寄存器配置，对于其它产品即为某个功能的输入控制信号

需要知道模块的那些端口信号会影响该功能点

此点由具体验证人员提取

验证策略

指的是验证某个功能或某个点采用的方法，比如对某些控制信号的验证，可以转化为数据，最后采用这个过程的数据对比，或者采用监控的方法对每次信号变化做监控，又或断言的方法来监控

当 FPGA 验证有问题，但 fpga 版本 RTL 仿真没问题，则可以用 synpro 综合后的网表文件仿真（不加时序），其文件以.vm 后缀存在

验证需要 makefile 加载的文件

验证中包含的文件如下:

1. TB :TESTBENCH,测试平台,属于最高顶层
2. DUT :RTL 代码,验证的对象
3. ENV:验证环境,属于验证中公用的一部分,里面为SV 标准的构架;
4. TC:TESTCASE,具体的测试用例,其会使用到 ENV 里面的内容
5. 模拟 lib

下面两项不一定会使用到

6. 如果使用 UVM,还需要增加 UVM 的相关包
7. 如果使用了C 模型,还需要增加.C 文件

加载文件主要有通过 makefile 方式把文件加载到编译,仿真选项中;其中 makefile 显式的会包含以下内容:

1. TB - makefile 中使用了$(tb_list)变量
2. DUT - makefile 中使用了$(dut_list)变量

3. TC,同时 ENV 采用了在 tc 中使用 include “../../tb_list.sv”的方式加入环境,其中 tb_list.sv 又 include 了 env 中所有的内容;

 - makefile 中使用了$(tc_name)变量

4. lib - makefile 中使用了$(pre_lib)变量,可以前仿后仿使用相同的库

5. uvm - makefile 中使用了$(VUM_COMPILE)变量

6. c 模型 - makefile 中使用了$(c_file)变量

无法验证的点

1. 跨时钟没做同步无法验证到
   ---对于分频时钟,但综合又不作为同源时钟处理的时钟,在 rtl 中最好有注释,然后输出时#1输出,这样容易验证到.
2. 异步复位没做异步复位同步释放，无法验证到
3. 模拟部分无法真实验证到，只能靠模型
4. 时序方面无法验证（上述 1，2）--后仿也无法保证验证的到，靠 pt/spyglass 发现
5. Atpg 状态下对模拟的控制无法验证,atpg 模式寄存器的特性无法验证,见”atpg 测试相关章节”
6. 组合电路的毛刺rtl无法仿真的到，也无法约束，只能靠网表仿真

只能通过后仿验证的点

模拟电路端口的时序(以绝对时间来表示信号之间的关系,如 A 信号上升沿到 B 信号的上升沿需要 100us 等)通过工具无法验证到,如 flash 的控制信号;

因为这些信号虽然通过同一个时钟产生,但每个寄存器的 Tco 不同,再加上clk skew,在输出的信号之间会有不同的延时,所以如果在设计时,控制信号之间的时序设计的太紧,则后仿可能会通不过.而在模拟接口端,这两个信号工具又不会检查它们之间的关系,所以只能靠后仿来保证---如果前仿不过则为设计错误

易遗漏验证点

模拟 ip 测试时，由于是不断电测试，所以在验证时，需要把各个测试串起来跑一遍，避免由于 io 方向切换导致无法进入下一个 ip 的测试。

前仿注意事项

1.由于系统一般都会有一个上电复位过程，有些复杂，有些简单，我们可以在验证环境中监控与上电复位过程相关的关键信号，如果出现问题这样就很容易判断它处于某个阶段

2.有些问题很怪异，从波形上看不到异常，很有可能是毛刺引起，因为一般我们会在verdi上关掉毛刺，但实际上是存在的，

3 还有的是发现有些信号明明是应该等于 0 却发现等于 1，这很可能是在验证环境中有 if(xx==0) xx=1;这样的语句，没有延时，直接改了变量

4 仿真器对于同步电路和非同步电路处理不一样,如下图

电路设计如下

```
Always @(posedge osc800k or negedge xxx)
  If(xxx)  xxxx
  Else if(delay_done) pad_cur_state <=pad_cur_state+3’h1;
```

当 delay_done 是在 osc800k 时钟域时,波形如上图,在虚线部分时钟是可以采样到 delay_done 为高电平;

当 delay_done 不是在 osc800k 时钟域时,在虚线部分时钟上升沿采样到的 delay_done 信号为低电平.----这种情况主要是在验证平台写测试时会存在,如这样写:

```
….
Osc800k=1’b0;
Delay_done=1’b1;
#100;
Delay_done=1’b0;
Osc800k = 1’b1;
#100;
Osc800k=1’b0;
```

Delay_done=1’b1;

….

或者写成

@(posedge Osc800k);

Delay_done=1’b1;

……

都不行

表面上看波形在虚线部分一模一样,但实际含义不一样.

因为仿真器会对寄存器做 Δ延时处理,所以如果需要实现rtl 中寄存器一样的时序,可以采用 interface 中的 clock block 来实现这个时序.其实用<=赋值也可以实现，见前面的 sv 语法部分分析。

如果采用统一固件方法，地址可以使用枚举类型来定义，这样可以打印字符出来，容易识别，同时也可以打印具体地址出来；

```
typedef enum {clk_addr=16'h3aff,
              rst_addr=16'h3bff,
              xxx_addr=16'h96} e_reg_addr;


 task reg_read(input e_reg_addr addr,output [`DATAWIDE-1:0] rdata);
 //寄存器读操作
          wait((`STATUS==8'h00)&(!stop));
          `KINDS = 8'h55;
          `ADDRL = addr[7:0];
          `ADDRH = addr[15:8];
          `START = 8'h22;
          wait((`STATUS==8'h11)&(!stop));
          rdata = `RDATA;
          `STATUS = 'h0;
          `START  = 'h0;
          `KINDS  = 'h0;
```

```
            `ADDRL  = 'h0;
            `ADDRH  = 'h0;
      endtask
```

如果使用 class来封装 task,则可以使用参数化类;如下

```
class bus_opr #(type T=bit[15:0]);
    static  task reg_read(input T addr,output [7:0] rdata,input stop
= stop_interrupt);
    endtask
endclass
```

使用方法: bus_opr #(e_reg_addr)::reg_Bitwrite1(IRQOEN_ADDR,8'h10);

对于寄存器的内容可以用下面 方式处理

```
class CKDIVCTRL;
    rand bit[1:0] SLOW_DIV  ;
    rand bit[5:0] SYS_DIV   ;
    function new();
        SLOW_DIV = 2'h3;
        SYS_DIV  = 2'h3;
    endfunction

    function bit[7:0] CKDIVCTRL_REG;
        CKDIVCTRL_REG =  {SLOW_DIV,SYS_DIV};
    endfunction
endclass
```

使用方法:

```
CKDIVCTRL  CKDIVCTRL_R;
CKDIVCTRL_R = new();
CKDIVCTRL_R.SYS_DIV=2;//定向或随机
```

bus_opr#(T_REG_ADDR)::reg_write(CKDIVCTRL_ADDR,CKDIVCTRL_R.CKDIVCTRL_REG);

后仿注意事项

1. 由于电路中会有同步电路，所以需要设计者提供复位电路的第一级寄存器，做 notimingcheck;如果每配置,会导致后面引入 x 态,致使仿真结果错误

Instance{ cbm8007.u_vdd12_domain.inst_vdd12_clk.inst_mclk_dft_rstn_sync_level.signal_delay1_reg} {noTiming};

后仿加 delay 选项

后仿需要 2 个地方与 delay 选项(max,min,type)相关

1. 读 sdf 文件需要根据 vcs 选项,来选择那种条件下的 sdf,至少会有 worst/best 两个 sdf 文件
2. 在 vcs 选项上+maxdelays/,这个选项是用来器件延时使用 sdf 文件中的最大值,主要是因为$sdf_annotate ("sdf_file", [module_instance, "config_file"," log_file", "mtm_spec","scale_factors"," scale_type"]); 这个系统函数的"mtm_spec"我们一般都是缺省值,那么就需要在 vcs 选项中加 max/min 等选项,如果在此处有配置,那么将会忽略 vcs 的配置.

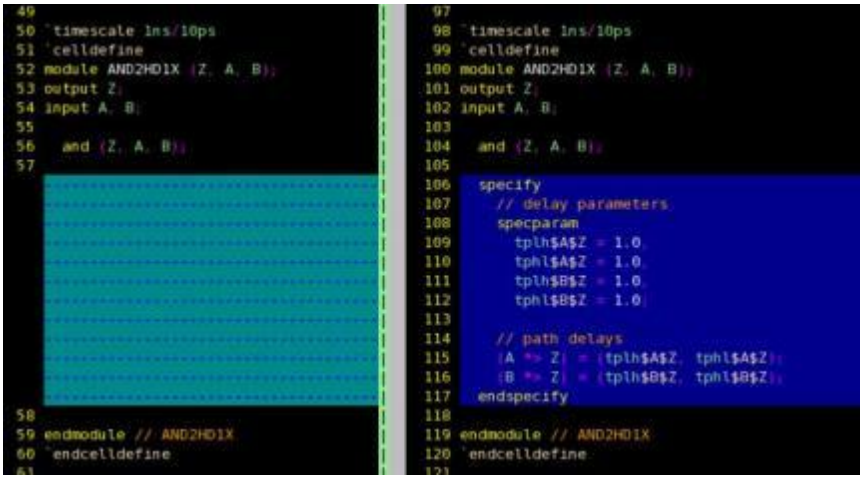
3.对于有负延时的检查,一般都是hold time,需要注意库有没有区别,据网上介绍,有的厂家对于负延时的检查使用了不同的库.

定位后仿 x 态原则

2. 当发现异常，一般都是找最开始出现 x 态的地方
3. 当输出为x 态，输入有多个信号为x 态，此时找最后变成x 的信号（实际上是根据 sdf 文件，查看延时信息，看哪个 x 态引起的），同时需要注意延时是否合---查看 sdf 文件得出结论，
4. 其中x 态会根据 1 或上任何值（含 x 态）等于 1，也就是说当一个输入为 1 的或门，另外一个 x 态不影响输出，都是输出 1；0&任何值（含 x 态）等于 0，也就是说当一个输入为 0 的与门，另外一个 x 态不影响输出，都是输出0；

后仿无延时,报 IOPATH Annotation Not Enabled

当有好多(数十万个,看规模大小)Warning-[SDFCOM_IANE] IOPATH Annotation Not Enabled 时,请注意仿真加载的综合库是否正确,正常情况下每个后仿库的 module 里面需要包含 specify endspecify 语句,否则即使是使用了 $sdf_annotate(″...);也无法读入反标延时
如下图



如果使用左侧的无 specify 的库,sdf 报的 log 结果如下

```
        Total errors: 0
        Total warnings: 499199
***     SDF annotation completed.
```

:

使用右边有 specify 的库, sdf 报的 log 结果如下:

```
        Total errors: 0
        Total warnings: 4760
***     SDF annotation completed.
```

sdf_annotate 与 specify 有什么关系,反标 SDF 文件的过程，也算是更新 specify block 相对应信息的过程，如果 SDF 文件没有包含某些信息，则参考 specify block 中的相应信息。
使用了 specify 的库,还会报 Warning-[SDFCOM_INF] IOPATH not found;

```
29232 Warning-[SDFCOM_INF] IOPATH not found
29233 /ic[redacted]/revA0/team/[redacted]/chip/sta/results/sdf/[redacted]_post_layout_min1.sdf, 791716
29234 module: FAHDUX, "instance: AAA_tmax_testbench_1_16.dut.I[redacted]_alu.i_alumux.U192"
29235   SDF Warning: IOPATH from A to S is not found.
29236
```

对应的 sdf,791716 行(791717, 791719, 791720 行都报相同 warning)如下:

```
791711 (CELL
791712   (CELLTYPE "FAHDUX")
791713   (INSTANCE I_cbm7316f_core/I_51core/i_mc8051_alu/i_alumux/U192)
791714   (DELAY
791715     (ABSOLUTE
791716     (CONDELSE (IOPATH (posedge A) S (0.237:0.237:0.237) (0.331:0.331:0.331))
791717     (CONDELSE (IOPATH (negedge A) S (0.311:0.311:0.311) (0.281:0.281:0.281)))
791718     (CONDELSE (IOPATH A CO (0.354:0.354:0.354) (0.314:0.314:0.314)))
791719     (CONDELSE (IOPATH (posedge B) S (0.306:0.306:0.306) (0.387:0.387:0.387)))
791720     (CONDELSE (IOPATH (negedge B) S (0.397:0.397:0.397) (0.447:0.447:0.447)))
791721     (CONDELSE (IOPATH B CO (0.411:0.411:0.411) (0.400:0.400:0.400)))
791722     (CONDELSE (IOPATH (posedge CI) S (0.123:0.123:0.123) (0.155:0.155:0.155)))
791723     (CONDELSE (IOPATH (negedge CI) S (0.190:0.190:0.190) (0.136:0.136:0.136)))
791724     (CONDELSE (IOPATH CI CO (0.158:0.158:0.158) (0.160:0.160:0.160)))
791725     (COND B==1'b0&&CI==1'b0  (IOPATH A S (0.237:0.237:0.237) (0.281:0.281:0.281)))
791726     (COND B==1'b1&&CI==1'b1  (IOPATH A S (0.237:0.237:0.237) (0.281:0.281:0.281)))
```


咸鱼:东

分析原因估计是组合逻辑单元不认 posedge 语句 ,这个并不影响仿真,因为相当于是多余的描述,仿真并没有该路径(实际的路径包含在描述文件中).

正向点测试

测试每个功能是否都有，有控制位的是否都有

反相点测试

是否有不存在的效果，而该效果会影响系统的正常工作

交叉点测试（铰链出错）

功能与控制位是否一一对应，是否存在一对多，或多对一的控制

异常保护测试

主要是指 PD,IDLE，保护类的测试，因为所有的芯片都会有一些异常保护动作，在做异常等测试时，要考虑时钟等一些动作能否恢复到异常前的状态或记录相关状态

模块验证

可以覆盖所有控制流和数据流，白盒验证，侧重于模块基本功能

1.配置待测 DUT，只需关注当前 DUT 的输出是否满足设计需求

2.需要注意所有的输入信号对输出的影响

3.模块验证的前提是假定输入时序是正确的，所以需要做系统级模块验证

任何一个模块都需要考虑其时钟在什么时候开启，什么时候关闭，复位信号什么时候复位，什么时候释放

系统级模块验证

可以覆盖所有控制流和数据流，黑盒验证，侧重于模块在系统环境中的功能是否能实现，其它模块是否影响它的功能

1.主要是配置 DUT 模块，配置的 DUT 最终将会影响整个系统或结果，需要监控最终结果或关键时序。

2.需要知道如何配置系统才会影响到端口的信号变化，可能会有多种配置只影响一根信号的变化

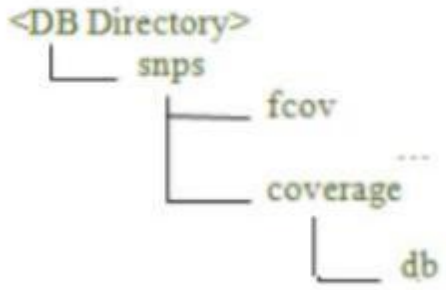
3.如果模块间的时序定义不完全一致，会导致模块间无法正常工作

子系统验证

把几个关了性相对紧密的几个模块在一起验证

功能覆盖率&代码覆盖率

1.功能覆盖率产生步骤：
当我们使用-cm 编译选项时，vcs 工具会在当前目录下自动创建一个 simv.vdb 的目录（可以使用-cm_dir 改变目录，命令为 vcs -cm line+tgl+cond+fsm -cm_dir ）；如下结构



〇在编译阶段，vcs 会向 snps/coverage/db/design 目录写一些关于设计的数据进去
〇在仿真阶段，vcs 会向 snps/coverage/db/testdata 目录写一些覆盖率相关的数据进去
一般会使用-cm_name 选项来保存不同的配置 case，当收集覆盖率的时候，工具会把当前目录下所有的数据进行分析---命名要不同，否则会被覆盖，则永远是分析一个 case 的覆盖率。
分析覆盖率的方法：

使用 urg，命令如下　　urg　xxxx.vdb -full64 -grade -report　……/regress/urgReport,则在 urgReport 目录下会有相关报告
当不同的模块在不同的目录下分别做覆盖率统计，但又想作为一个整体统计，怎么办？---告诉 urg 相关目录，让工具统计。
urg -dir simv1.vdb simv2.vdb simv3.vdb -dbname mergedir/merged
urg　-dir ./codeCov/simv.vdb　-dir ./assert/simv.vdb -dbname merged

SV 采用 CRT 的激励形式，而判断验证进度的标准也就是覆盖率(coverage)。

覆盖率的两种指定形式：

显式的，直接通过 SV 来指定出的，如 SVA，covergroup。

隐式的，在验证过程中，随"register move"就可以由 simulator 得到的，如代码覆盖率等。

覆盖率类型：代码覆盖率---由仿真器直接提供，只能表示设计的冗余度，与 spec 关系不大。

(toggle/expression/block)功能覆盖率---与 spec 比较来发现，design 是否行为正确，需要按 verification plan 来比较进度。

(assertion/covergroup)断言覆盖率---用于检查几个信号之间的关系，常用在查找错误。

漏洞率-------同一功能验证，运行多个 seed，寻找 bug

功能覆盖率的两个重要部分是采样的数据和数据被采样的时刻。数据被采样的时刻可以是 1)使用wait 或@来阻塞赋值，2)使用 sample 的方法。采样的数据则需要在代码中加入覆盖组(Covergroup)和仓(Bin)，覆盖组可以定义在类中，程序块，模块中。**使用覆盖组之前必须先进行实例化。所以一般在类中使用覆盖组时，将覆盖组的实例化加在类的构造函数 new()中**。

Covergroup 和 Coverpoint 的命名应当尽可能的明确，这样方便检查覆盖率文件。

功能覆盖率:根据验证者的理解设置好目标

它只能保证验证者理解的功能都被设计者实现，但无法保证验证者遗漏的验证点和冗余设计;

代码覆盖率:设计者设计的功能是否被验证到

它只能保证设计者所设计的功能被覆盖，但不能验证其遗漏的设计功能点;所以在验证中需要结合两者的覆盖率来检验一个设计是否符合需求，但如果设计者和验证者遗漏了同一个功能点，则无法验证出来，必定是个 bug,只能靠 review 来 cover

验证常用手段

控制类比较

主要是来比较各个控制信号的状态，可以把所有的控制信号用一个变量来保存，然后在通过各种系统函数或其它手段来比较这个变量,尤其是用于控制模拟调整的信号，它们只单纯的让控制信号，提前或滞后某个信号，验证时只能看相关信号的延时信息，可能不能考虑系统是否正常，因为在系统环境中有些信号不能随机配置，具有约束关系，否则系统会出错，此种情况需要把系统对比检测关闭。

通过这个变量的监控，可以监控所有的信号变化情况

@ (express)

if(express===xxx)

可以用于两种方法

1.用 forever 方法，当有变化时，可以逐位判断，再倒推是由谁引起，种方法用在比较简单的条件

2.当发起了某动作，在@该表达式，当变化后在判断是否是需要的信号起来，否则有问题。才用该方法可以检测到控制的对象是否正确及连接性

如果是一一对应的关系，还可以使用上面的$onehot 系统函数,$isunknow 等系统函数

3.需要考虑测例中的变量于 RTL 中变量位宽是否一致，否则当RTL 位宽大于测例中的变量位宽，则可能会有遗漏地方，在对比是使用$bits()函数对比下位宽，可以加强验证的可靠性

4.控制类一定要注意反向测试和干扰验证(既 a 控制 a 事件,b 控制 b 事件,在验证a 时,要注意 b 事件不能发生)

采用宏定义比较方便,并且可以在各个项目中复用

```
`define judge_eq (x,y)\
    if((x!=y) | ($bits(x)!=$bits(y)))begin\
        $display ("ERROR:x,y not equ,x,%0d bits:%0h;
                y,%0d bits:%0h",$bits(x),x,$bits(y),y,$time);\
        err=1;\
     end
```

上述中变量 err 为全局变量,也可以用做参数的方式,这样复用度更高,只是每次引用需要多写一个变量

```
`define judge_eq (x,y,err)\
    if((x!=y) | ($bits(x)!=$bits(y)))begin\
        $display ("ERROR:x,y not equ,x,%0d bits:%0h;
                y,%0d bits:%0h",$bits(x),x,$bits(y),y,$time);\
        err=1;\
     end


`define judge_neq (x,y,err)\
    if((x==y) | ($bits(x)!=$bits(y)))begin\
        $display ("ERROR:x,y  equ,x,%0d bits:%0h;
                y,%0d bits:%0h",$bits(x),x,$bits(y),y,$time);\
        err=1;\
     end
```

连通性测试

正反测试，连通和不连通,主要用于第三方 ip 集成,而且第三方无法提供 case,则只能测试相关端口/pad 于 ip 的连接性,如 flash bist 等,各通讯模块到 pad 端的连接 .

连通性测试需要注意反向测试,如 pad 为入时,需要施加输出激励,验证是不是只有输入功能;其实所有的验证项目都需要做排斥性验证,从配置项来说就是如果一个功能由 1 个 bit来控制,则需要需要验证当配置为 0 时,不能有 1 的功能;如果是 2 个 bit 来表示,则配置为 0 时不能有配置 1,2,3 的功能.

```
`define connect_judge(send_sign,get_sign,delay)\
    #(delay);\
    force send_sign = 1'b0;\
    #(delay);\

if((send_sign!=get_sign)|($bits(send_sign)!=$bits(get_sign)))begin\
        $display("ERROR:send_sign,get_sign not equ,send_sign,%0d
bits:%0h;get_sign,%0d
bits:%0h",$bits(send_sign),send_sign,$bits(get_sign),get_sign,$time)
;\
        err=1;\
    end\
    #(delay);\
    force send_sign = 1'b1;\
    #(delay);\

if((send_sign!=get_sign)|($bits(send_sign)!=$bits(get_sign)))begin\
        $display("ERROR:send_sign,get_sign not equ,send_sign,%0d
bits:%0h;get_sign,%0d
bits:%0h",$bits(send_sign),send_sign,$bits(get_sign),get_sign,$time)
;\
        err=1;\
    end\
    #(delay);\
   release send_sign;
```

```
`define disconnect_judge(send_sign,get_sign,fix_value,delay)\
    #(delay);\
    force send_sign = 1'b0;\
    #(delay);\

if((fix_value!==get_sign) | ($bits(fix_value)!==$bits(get_sign)))begi
n\
        $display ("ERROR:send_sign,get_sign  equ,send_sign,%0d
bits:%0h;get_sign,%0d
bits:%0h",$bits(send_sign),send_sign,$bits(get_sign),get_sign,$time)
;\
        err=1;\
    end\
    #(delay);\
    force send_sign = 1'b1;\
    #(delay);\

if((fix_value!==get_sign) | ($bits(fix_value)!==$bits(get_sign)))begi
n\
        $display ("ERROR:send_sign,get_sign  equ,send_sign,%0d
bits:%0h;get_sign,%0d
bits:%0h",$bits(send_sign),send_sign,$bits(get_sign),get_sign,$time)
;\
        err=1;\
    end\
    #(delay);\
    release send_sign;
```

上述 err 变量可以按照相同方式处理

时序类比较

1.尽量使用断言来实现监控

2.可以用$width(posedge clk,glitch_width);来检查时钟的宽度是否为毛刺

放在 specify …endspecify 中，而 specify 又可以放在 module 中，如下：

```
specify
    $width(posedge    ENOSC_15,         `glitch_width);
endspecify
```

3. 自己用 sv 来实现时序的监控

数据比较，一般用于存储类验证

1.常用的方法是用一个数组来保存结果，再与源数据对比;

数据的选取尽量不重复，可以写的数据就是地址，或者在某些地址加入特定数据以测试特定功能. 如 flash 有对原数据为F F的地址跳过不编程，此时就需要加入数据为 F F的内容测试.

2.异常验证时，可以对某些特定地址取反或+1之类的操作

通过随机数给数组赋值，然后在通过验证环境把数据传给系统，再用另外一个数组存储接收的数据，最后比较两个数组的数据是否一致

2. 在存储类验证时一定要注意边界和前门/后门对比.
3. 对于存储类接口协议等数据严整，需要考虑地址对齐，但验证需要考虑不对齐的 case,地址要在对齐范围内开始，如以 word 对齐，但验证需要验证到启始地址偏移量为 1，2，3 的情况。

验证注意事项

变量初始化

在做验证时需要把所有的变量都做一个初始化，犹如 RTL 中的复位一样。在验证中经常使用队列,数组(动态数组,合并数组),流操作等特性,有利于验证提高效率

Interface 的用处

在做验证时，激励与 DUT 为什么要用 interface,在控制 SRAM 时为什么除了时钟外，addr,wr,rd 需要#1:因为我们在写激励的时候，一般是这样写@（posedge clk）wr=1'b1;@（posedge clk）wr=1'b0 之类的，这样写在第二个时钟沿其实不能采样到 wr=1,除非用 assign #1 wr=wreg; wreg=1,之类的用法，而 interface 的时钟块 output 默认是#1，所以它很好地解决了这个问题，同样在 RAM 的模型也是类似的写法，所以需要#1,；而 RTL 中仿真不需要#1，那是因为 RTL 中是用 always 块来做寄存器的，VCS 编译时能自动解决该问题，对于 interface 的输入，我们采用@（posedge clk） tb= dutin;这样是无法采样到真实时刻的 dutin,而应该@（posedge clk） int.tb<= dutin;它因为接口时钟块块会把输入信号延#1,

RAM 类的模型都是用 task,function 来写的，所以要注意此问题，他们的接口需要延时处理，如果 RAM 模型没使用@(posedge clk) 这样来做数据处理，则不会有上面的问题

interface 信号必须使用非阻塞赋值来驱动，即<=符号;

当 task 中用<=时，也可以让 DUT 采样到数据，但在读数据时需要延时处理

使用 rtl 内部信号处理

在书写验证代码时，当给 RTL 中某个信号赋值时，不能直接使用..module.a.b 的方式引用，必须采用宏定义，这样容易管理及维护

当要监控 RTL 中某个信号时，采用接口或probe 的方式，引出该信号，原因同上

时钟产生

在验证时，系统时钟最好写成一个module 的方式，例化，而不在 top 中写用 forever 去做,这样比较好管理而已。

坚持一种书写规范

类名全部大写不加下划线；常数采用全部大写可用下划线，如 parater CELL_SIZE=8;变量全部使用小写可用下划线

前仿与后仿延时处理

前仿对信号或数据的对比,由于无延时,

所以在 DUT 正确接受数据后#0,#1 都可以正确比较 ,但后仿会有延时,#0,#1 就可能不够,

所以前仿就需避免这样的条件比较 ,根据需要可以延延时一个或半个 clk,因为rtl本来就是下一拍采数据.

后仿存储模型 warning

对于后仿，经常会有存储单元的warning,因为那是用模型导致，存储单元没用网表

验证版本管理

对于验证环境必须记录每次更改信息及原因,尤其是最终版,更需要把相关脚本,命令做详细的说明

把 C 中的打印显示在 TB 环境中

修改 putchar（c）函数，因为 printf 函数最终是调用 putchar()函数把字符输出到终端 ，我们可以把putchar 输出到块存储空间里，

而 TB 环境不断地检查 该存储空间的变化，只要有变化就在 TB 采用$write()的方式打印出来，

同时printf 函数最后一个字符要不是换行键（10），要不是回车键（13）

C 中的 putchar()它是发送一个字符如下：

```
char putchar(char c)
{
sync_message =c & 0x7f;
sync_message |= 0x80;
sync_message &= 0x7f;
}
```

volatile unsigned char xdata sync_message _at_ 0x10 定义变量在一个固定的 XRAM 空间

最高位产生一个0->1->0 的跳变动作

在 TB 中注意需要检测到字符串的结束符后再开始打印所有信息，结束符为 4，换行键为 10,回车键符号是 13（十进制）

$display 和$write 的唯一区别是$display 在文本的结束写一个换行字符而$write 不写

打印小数方法

下面的 task 是一个输入 32bit,高 16bit 为整数部分，低 16bit 为小数部分，通过此 task 可以转化成显示整数 . 小数的格式打印,精度为小数点后 7 位：
如 32’h28000 将会打印出 2.5；

```
task print_decimal(input[31:0] datain);//7 bit decimal
    int datatemp;
    datatemp =0;
    for(int i=0;i<16;i++)begin
```

```
        datatemp = datain[15-i]*(5000000>>i) +datatemp;
    end
    if(datatemp>=1000000)
        $display ("%0h changto
is  %0d.%0d",datain,datain[31:16],datatemp);
    else if(datatemp>=100000)
        $display ("%0h changto
is  %0d.0%0d",datain,datain[31:16],datatemp);
    else  if(datatemp>=10000)
        $display ("%0h changto
is  %0d.00%0d",datain,datain[31:16],datatemp);
    else  if(datatemp>=1000)
        $display ("%0h changto
is  %0d.000%0d",datain,datain[31:16],datatemp);
    else  if(datatemp>=100)
        $display ("%0h changto
is  %0d.0000%0d",datain,datain[31:16],datatemp);
    else  if(datatemp>=10)
        $display ("%0h changto
is  %0d.00000%0d",datain,datain[31:16],datatemp);
    else
        $display ("%0h changto
is  %0d.000000%0d",datain,datain[31:16],datatemp);
endtask
```

RAM 初始化问题

在做 SOC 验证时，由于核预取指令及预译码—如果不会预取指令，则不会有问题，如果对 RAM 没有初始化，系统默认为X 态，当运行到最后一条指令时，核会再取下一个地址的内容，这是取到的为X 态，当译码时

则可能导致 X 态的传递，而导致系统出错；最好的方法是：根据不同的 RAM 进行不同的初始值，如果出厂时 FF，那么对他们进行 FF 初始化，如果是随机数，则用随机数填充！！

寄存器初始化

在基类里面的 new 函数给变量配置默认值，扩展类根据每个不同的测例采用约束的方式改变变量值，进行测试验证

如何验证时钟模块

时钟模块主要是用来管理时钟，其包括时钟的开关，分频，切换；这些动作在不同场景下有不同的要求，所以在验证前需要先把各个场景下的时钟列出来，进行分类，然后再写出相关场景测例，并使用各种监控，一定要使用$width()语法来检测时钟的宽度！---有些设计者会有简单的选择电路去 gate 时钟,但 gate 信号却是上升沿实现,--这样做会有毛刺（正确做法是使用时钟下降沿去 gate 时钟）.

此模块验证需要非常熟悉系统，其不仅需要验证时钟的开启关闭是否正常，关键需要验证在需要关闭的时候不能开启，以达到低功耗的需求，而此点不会引起功能上的异常，最难验证

实现手段：需要系统架构工程师给出 gold，使用assert 或其它监控手段来验证各个时钟在各个场景的开关状态。但一定需要使用$width()来检测时钟是否有毛刺。

如何验证复位模块

复位模块主要是用来管理整个系统的复位功能，需要知道有多少个复位源，输出多少种复位，每个复位信号的复位条件和释放条件，同时需要知道各个复位信号复位哪些模块，在验证前需要整理出来；

实现手段：采用监控方式（断言可能不是太合适，需要使用可变参数），比较每级复位的时间，看时间是否符合预期，同时要检测复位模块输出

的复位信号控制到其它模块的连接是否正确（此 gold 需要系统架构工程师给出）--可以采用assert 或单独测试其连通性。

tb 与 c 交互

当 memory 正常是，能否考虑把寄存器配置值后门放到memory 中，然后通过 C 自动按照顺序读取数据写到相关模块的寄存器中做随机冲击，这样可以节约 TC 交互的配置时间，在这样做时，需要在keil 中把xdata 配置空余空间，让编译后的变量空间不与配置空间重叠，只需要在每个模块配置完后握手一次就够了，当然用假核冲击更快，如果与总线打断等方面验证就只能用真核

启动与打断场景验证

任何模块都存在这样一个场景，就是本模块在工作的状态 ，这是又来一次启动，此时模块是继续以前的工作还是打断重新开始工作,此点需要特别注意是否吻合产品需求,与设计者本身代码无太大关系,可能是理解不一致导致.

对于 sfr 相同地址监控的方法

用断言监控模块的片选信号（或类似片选的信号），它们不能同时起来，同时起来的断言语句是 intersect(a,b)———— 断定 a 和b 两个事件同时产生，且同时结束。

但由于模块太多，则只能通过组合后用$onehot 语句来监控

3. 对于<=赋值和=赋值，在仿真中也不完全一致---<= 非阻塞赋值，=阻塞赋值

```
static  task reg_write(input[15:0] addr,input  [7:0] wdata);
        wait(`STATUS==8'h00);
        `KINDS <= 8'haa;
        `ADDRL <= addr[7:0];
```

```
            `ADDRH <= addr[15:8];
            `WDATA <= wdata;
            `START <= 8'h22;
            wait(`STATUS==8'h11);
        //   $display("reg_write add is %0h,wdata
is %0h",addr,wdata,$time);
            #1;
            `STATUS <= 8'h0;
            `START  <= 8'h0;
            `KINDS  <= 8'h0;
            `ADDRL  <= 8'h0;
            `ADDRH  <= 8'h0;
        endtask
```

以上写法完全没问题

```
    static  task reg_write(input[15:0] addr,input  [7:0] wdata);
            wait(`STATUS==8'h00);
            `KINDS = 8'haa;
            `ADDRL = addr[7:0];
            `ADDRH = addr[15:8];
            `WDATA = wdata;
            `START = 8'h22;
             $display ("%0h",`KINDS);//---1
            wait(`STATUS==8'h11);
        //   $display("reg_write add is %0h,wdata
is %0h",addr,wdata,$time);
            #1;
            `STATUS = 8'h0;
            `START  = 8'h0;
            `KINDS  = 8'h0;
```

```
        `ADDRL  = 8'h0;
        `ADDRH  = 8'h0;
    endtask
```

以上写法一般没问题，但有时给 KINDS 等几个握手空间赋值会不成功，必须加上打印---1才成功，而且打印变量必须为握手空间才 OK，而改成左侧<=赋值则无需这些动作,此处猜想是因为仿真器在使用<=赋值时会有 Δ 延时处理---寄存器仿真特性，而打印正好是 Δ 延时；

仔细阅读《systemverilog 验证—测试平台编写指南》P76,P77.

原因简单分析，以上代码是在测试 tb 使用module 中，因为module endmodule 属于 verilog 范畴，它只有一个 active region，rtl,tb 混合执行，会产生竞争，如果上述代码放在 program endprogram 中，应该不会有这现象，因为 program endprogram 属于 system verilog 范畴，而 sv 分得更细，分为下面 4 步（同一时刻按下列顺序执行）：

a. Active ---仿真模块中的设计代码

b. Observed ---执行 system verilog 断言

c. Reactive---执行程序中的测试平台部分

d. Postponed---为测试平台的输入采样信号

所以，如果测试平台使用module endmodule 结构，在与 rtl 有关联时，赋值使用<=较好。

PAD 验证处理

在环境中，如果 PAD 悬空，仿真可能会导致 X 态，实际芯片悬空相当于 Z 态，所以可以初始化 PAD 为 Z 态---通过 pad 模型解决

如果 pad 有复用功能,需要注意验证其唯一性(只能选择一条通路)及oe 的方向(当配置输出功能,要验证没有输入功能,反之相同验证)

模拟（文档）接口

需要注意复位默认值 ,需要模拟设计者提供准确的默认值,一定要提供避免双方理解偏差带来致命的错误

模拟文档一定要有复位值及含义,格式建议写成

1: xxx;

0:xxx

分行写,不要连着写 1:xxx;0:xxx,或文字连着描述,因为这样容易眼睛看错.

模拟建模原则

除了电源，地，参考电压，参考电流等与电源相关的端口外，其它所有的端口都必须用到模型中，以防止端口信号没有被控制而无法验证到;

输出必须与所有的输入相关，当某个输入为x 态，则输出异常;

1．当模型为时钟类，异常输出为固定 0 或 1，最好不要输出x/z 态；因为时钟一般为沿使用，当为x/z态，上下沿检测会正常输出，不看波形可能会掩盖错误;

2．当模型为非时钟类，异常输出为x 态，因为无论是作为数据还是控制信号使用，此时 x 态会传递，对比很容易检测到错误。

3．当模型是电源时，且带有使能控制位，可以把相关使能位与 por 复位信号关联起来，也就是如果没有使能，复位一直不释放，任何情况（模式）下工作都不正常

4. 由于一般模拟模块都需要测试，在模拟端口很可能是组合逻辑输入，在综合时做 formality 检查，只能是功能性检查，但由于各个信号到达模拟端口的延时不一样，可能会产生毛刺，所以需要在模型中加入毛刺检测$width();对所有的模拟输入控制信号都需要加上检测。--主要用于后仿。一般模拟端口最后一个控制组合逻辑是mux，需要检查该mux 的电路是否为简单的mux 电路，如果是复杂的多输出组合逻辑构成，很可能会产生毛刺。

有符号数与无符号数区别

在 sv 环境中，所有变量默认为 unsigned—无符号数，当定义有符号数时，需要特别指明，：byte signed ,shortint signed ,int signed,longint signed;分别表示 8bit 有符号数，16bit 有符号数，32bit，64bit 有符号数。可以这样自己定义非 常规有符号数: bit signed [4:0] temp;

不管是有符号数，还是无符号数，在 sv环境中都是 2 进制的一个运算，只是代表的含义不同，在底层不区分，只是按照最基本的 1+1=10这种操作处理;
负数代表内容为符号数+（取反+1）：10000001=-（111_1110+1）=-111_1111=-127
如下

```
initial begin  //test ,no use
    byte signed testdata0;
    byte signed testdata1;
    byte signed testdatax;
    byte signed testdatay;
    byte signed testdata2;
    byte unsigned testdata3;
    testdata0 = 8'h81;
    testdata1 = 8'h2;
    testdatax = testdata0<<<1;
    testdatay = testdata0<<1;
    testdata2 = testdata0+testdata1;
    testdata3 = testdata0+testdata1;
    $display ("testdata0=%0h,%0d",testdata0,testdata0);
    $display ("testdata1=%0h,%0d",testdata1,testdata1);
    $display ("testdata2=%0h,%0d",testdata2,testdata2);
    $display ("testdata3=%0h,%0d",testdata3,testdata3);
    $display ("testdatax=%0h,%0d",testdatax,testdatax);
    $display ("testdatay=%0h,%0d",testdatay,testdatay);
End
```

输出的结果如下

```
testdata0=81,-127
testdata1=2,2
testdata2=83,-125
testdata3=83,131
testdatax=2,2
testdatay=2,2
```

对比

```
testdata2 = testdata0+testdata1;
testdata3 = testdata0+testdata1;
```

它们计算的结果都是 0x83,在最底层就是普通的二进制运算，而不管符号，只是代表的意义不同而已,一个代表-125，一个代表 131;
当 testdata0 = 8'h81;

testdata1 = 8'h2;
计算
testdata2 = testdata0+testdata1;
testdata3 = testdata0+testdata1;
的结果为
testdata2=3,3
testdata3=3,3
因为最底层益出了，所以出来的结果就是正数

1.如何复用各个模块的寄存器配置

1.每个模块有一个寄存器的 cfg 类，类名字和例化名字确定好就坚决不变，考虑复位值，可以基类的约束为默认值，然后扩展类的约束与基类约束的名字相同，这样可以改变配置值

2.每个模块有一个单独的文件进行寄存器配置

3.在各自的tc 中都 include 各个模块的 cfg 类文件和寄存器配置文件

4.当在各自的 tc 中需要修改配置时，只要扩展相关模块的基类，然后约束相关配置，然后再把扩展类的句柄给基类句柄就 OK 了，如果系统不大，可以让人专门维护上述工作

5.为了让含义易理解，cfg 中的变量全部与 RTL 中一直，或加上 1~2 个字母区分

寄存器的配置，可以使用 NEW 函数，设置为 RTL 的默认值，当要验证默认值时，只使用 new，而不进行 randomize

2.寄存器地址维护

1. 寄存器这样便于打印字符串出来,这样就只需要维护一个叫 reg_addr.sv 的文件.

如下 xxx_addr.sv 文件

```
typedef enum {
  P0OIOCFG_ADDR =  16'h1000
```

```
, P01IOCFG_ADDR =  16'h1001
, P02IOCFG_ADDR =  16'h1002
, P03IOCFG_ADDR =  16'h1003
, P04IOCFG_ADDR =  16'h1004
, STATU_ADDR      =    16'h0E06 } E_REG_ADDR;
```

3.寄存器维护

一个 soc 所有的寄存器用类封装起来,然后用 package 打包,这样就只需要维护一个叫 reg_package.sv 的文件

如下 xxx_reg.sv 文件:

```
class CKDIVCTRL;
    rand bit[1:0] SLOW_DIV  ;
    rand bit[5:0] SYS_DIV   ;
    function new();//复位值
        SLOW_DIV = 2'h3;
        SYS_DIV  = 2'h3;
    endfunction

    function bit[7:0] CKDIVCTRL_REG;//组合成寄存器
        CKDIVCTRL_REG = {SLOW_DIV,SYS_DIV};
    endfunction
    function print_reg(input string names="all");  //用来打印成员变量
        int assoc[string]  ;
        string str  ;
        assoc["SLOW_DIV"] = SLOW_DIV  ;
        assoc["SYS_DIV"] = SYS_DIV;
        if(names=="all")begin
            foreach(assoc[str])begin
```

```
                $display (" %s = 'h%0h",str,assoc[str]);
            end
        end
        else if(assoc.exists(names))begin
            $display (" %s = 'h%0h",names,assoc[names]);
        end
        else begin
            $display (" there is no this unit: %s",names);
        end
    endfunction
endclass
```

采用宏定义更好,如下

```
`define print_begin\
    function print_reg (input string names="all");\
        int assoc[string] ;\
        string str ;

`define add_member(str)    assoc["str"] = str ;

`define print_end\
        if(names=="all")begin\
            foreach(assoc[str])begin\
                $display (" %s = 'h%0h",str,assoc[str]);\
            end\
        end\
        else if(assoc.exists(names))begin\
            $display (" %s = 'h%0h",names,assoc[names]);\
        end\
        else begin\
```

```
                $display(" there is no this unit: %s",names);\
            end\
        endfunction

    class CKDIVCTRL;
        rand bit[1:0] SLOW_DIV  ;
        rand bit[5:0] SYS_DIV   ;
        function new();
            SLOW_DIV = 2'h3;
            SYS_DIV  = 2'h3;
        endfunction
        function bit[7:0] CKDIVCTRL_REG;
            CKDIVCTRL_REG =  {SLOW_DIV,SYS_DIV};
        endfunction
        `print_begin
            `add_member(SLOW_DIV)
            `add_member(SYS_DIV)
            `add_member(CKDIVCTRL_REG)
        `print_end
    endclass
```

上面组合起来使用

```
CKDIVCTRL_R = new();
 CKDIVCTRL_R.SYS_DIV=2;
 $display("%0h",CKDIVCTRL_R.CKDIVCTRL_REG);
bus_opr#(E_REG_ADDR)::reg_write(CKDIVCTRL_ADDR,CKDIVCTRL_R.CKDIVCTRL_REG);
CKDIVCTRL_R.SYS_DIV=9;
 $display("%0h",CKDIVCTRL_R.CKDIVCTRL_REG);
CKDIVCTRL_R.print_reg ();
```

```
CKDIVCTRL_R.print_reg ("SLOW_DIV");
CKDIVCTRL_R.print_reg ("SYS_DIV");
bus_opr#(E_REG_ADDR)::reg_write(CKDIVCTRL_ADDR,CKDIVCTRL_R.CKDIVCTRL_REG);
```

方法 2,仿 uvm 方法做 my_copy,my_compare,my_print 函数

```
class reg_base;
    virtual function base_function( reg_base reg_t,string func);
        return ;
    endfunction
    function my_copy (reg_base rhs);
         base_function(rhs,"copy");
    endfunction
    function my_compare(reg_base rhs);
         base_function(rhs,"compare");
    endfunction
    function my_print();
         base_function(null,"sprint");
    endfunction

endclass: reg_base

`define add_filed_begin(class_type)\
    function base_function(reg_base tmp_type ,string what_);\
        class_type  local_data ;\
        if(!$cast(local_data,tmp_type)) return;\

`define add_member_to_filed(arg)  ;\
        case(what_)\
            "copy":arg = local_data.arg ;\
            "compare":begin if(arg !== local_data.arg) begin;\
                        $display ("arg:%0h <=>
local_data.arg:%0h",arg,local_data.arg); end\
                      end\
            "sprint":begin \
                    $display ("arg:'h%0h",arg);\
                end\
            default:begin\
                $display ("sorry,this options isn't supported now"); \
            end\
        endcase

`define add_filed_end   endfunction
```

在定义寄存器时需要 extands reg_base;

```
class CKDIVCTRL extends reg_base;
    rand bit[1:0] SLOW_DIV  ;
    rand bit[5:0] SYS_DIV   ;
    function new();
        SLOW_DIV = 2'h3;
        SYS_DIV  = 2'h3;
    endfunction

    function bit[7:0] CKDIVCTRL_REG;
        CKDIVCTRL_REG =  {SLOW_DIV,SYS_DIV};
    endfunction
    `add_filed_begin(CKDIVCTRL)
        `add_member_to_filed(SLOW_DIV)
        `add_member_to_filed(SYS_DIV)
    `add_filed_end
Endclass
```

使用方法如下:

```
        CKDIVCTRL CKDIVCTRL_R  ;
        CKDIVCTRL CKDIVCTRL_R2  ;
        CKDIVCTRL_R = new();
        CKDIVCTRL_R2 = new();
        CKDIVCTRL_R.SYS_DIV=2;
        $display ("%0h",CKDIVCTRL_R.CKDIVCTRL_REG);

bus_opr#(E_REG_ADDR)::reg_write(CKDIVCTRL_ADDR,CKDIVCTRL_R.CKDIVCTRL_
REG); //xram parity enable
        CKDIVCTRL_R.SYS_DIV=9;
        $display ("%0h",CKDIVCTRL_R.CKDIVCTRL_REG);
        CKDIVCTRL_R2.my_copy (CKDIVCTRL_R);
        CKDIVCTRL_R2.my_print();
        CKDIVCTRL_R.my_compare(CKDIVCTRL_R2);
```

上面的方法是按照寄存器的定义来组合 class 的数据结构，但实际上按照某个模块的功能来定义 class 的数据结构更好，如，某个模块在系统中有以下特点：系统时钟可以分频—也可以把系统相关的配置单独作为一个 class 的数据机构，使能位，配置寄存器，参数选择寄存器等，则把这些相关的数据作为一个 class。

通用固件验证方法

使用该方法验证，只需要写一套固件，则可以供整个项目使用，稍作修改可以供整个系列产品使用；尤其是给设计人员做自验证，非常方便；

但使用该方法也有其缺点，由于采用的是 sv 与 C 握手交互的方式，会占用部分 XRAM 空间，仿真时间比单纯的 C 会更长些；对于寄存器操作有特殊要求的情况也不实用，如要求在几个 cycle 内就要完成几个寄存器的操作，则可能无法实现；对于速度快的通讯数据也不合适，主要是 C 每个动作耗时太长，可能 会导致丢失数据。

该方法会占用XRAM 空间的用途如下：

#define OPR_ADDR_BASE　1000　定义 XRAM 使用的基地址

#define START XBYTE[OPR_ADDR_BASE+0]

用来保存变量指示 SV 有操作指令；C 会一直循环，当检测到该变量的值为0x22 则表示有操作，程序进入，否则退出一直查询

#define KINDS XBYTE[OPR_ADDR_BASE+1]

用来保存变量指示读写操作；0x55:读，0xaa:写

#define ADDRL XBYTE[OPR_ADDR_BASE+2]

#define ADDRH XBYTE[OPR_ADDR_BASE+3]

用来保存变量表示读写操作的地址

#define WDATA XBYTE[OPR_ADDR_BASE+4]

用来保存变量表示写操作的数据

#define RDATA XBYTE[OPR_ADDR_BASE+5]

用来保存变量表示读操作获得的数据

#define STATUS　XBYTE[OPR_ADDR_BASE+6]

用来保存变量表示 C 读写操作结束，当 C 完成一次操作，最后会对该空间写0x11,表示操作完成，SV 在发送完命令后，会一直等待该标志

#define INTFLAG　XBYTE[OPR_ADDR_BASE+7]

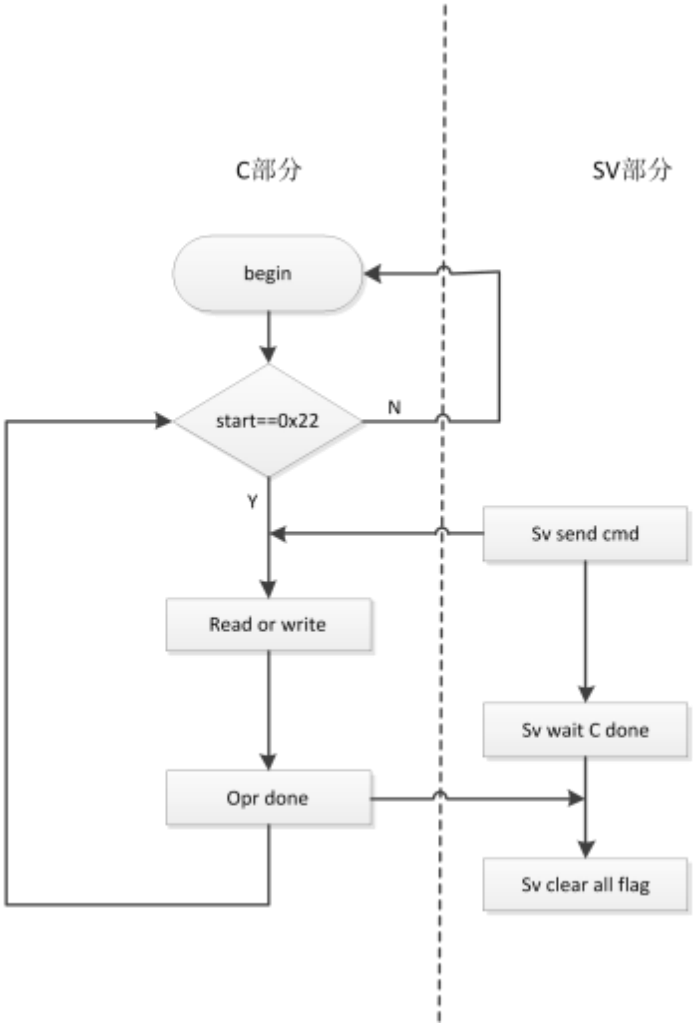
用来保存变量表示 SV 中断函数是否结束，SV 在中断函数最后会对该空间写中断入口地址，C 的中断函数会一直查询该空间，如果没结束，则一直在该中断函数中循环（无其它中断打断时）。

#define BACKFLAG XBYTE[OPR_ADDR_BASE+8]

用来保存变量表示 C 中断函数是否结束，C 在中断函数最后会对该空间写中断入口地址，SV 的中断函数最后等待该空间的值，如果等到预期结果，SV 会恢复上述空间为进入中断前的内容

c) 操作流程介绍

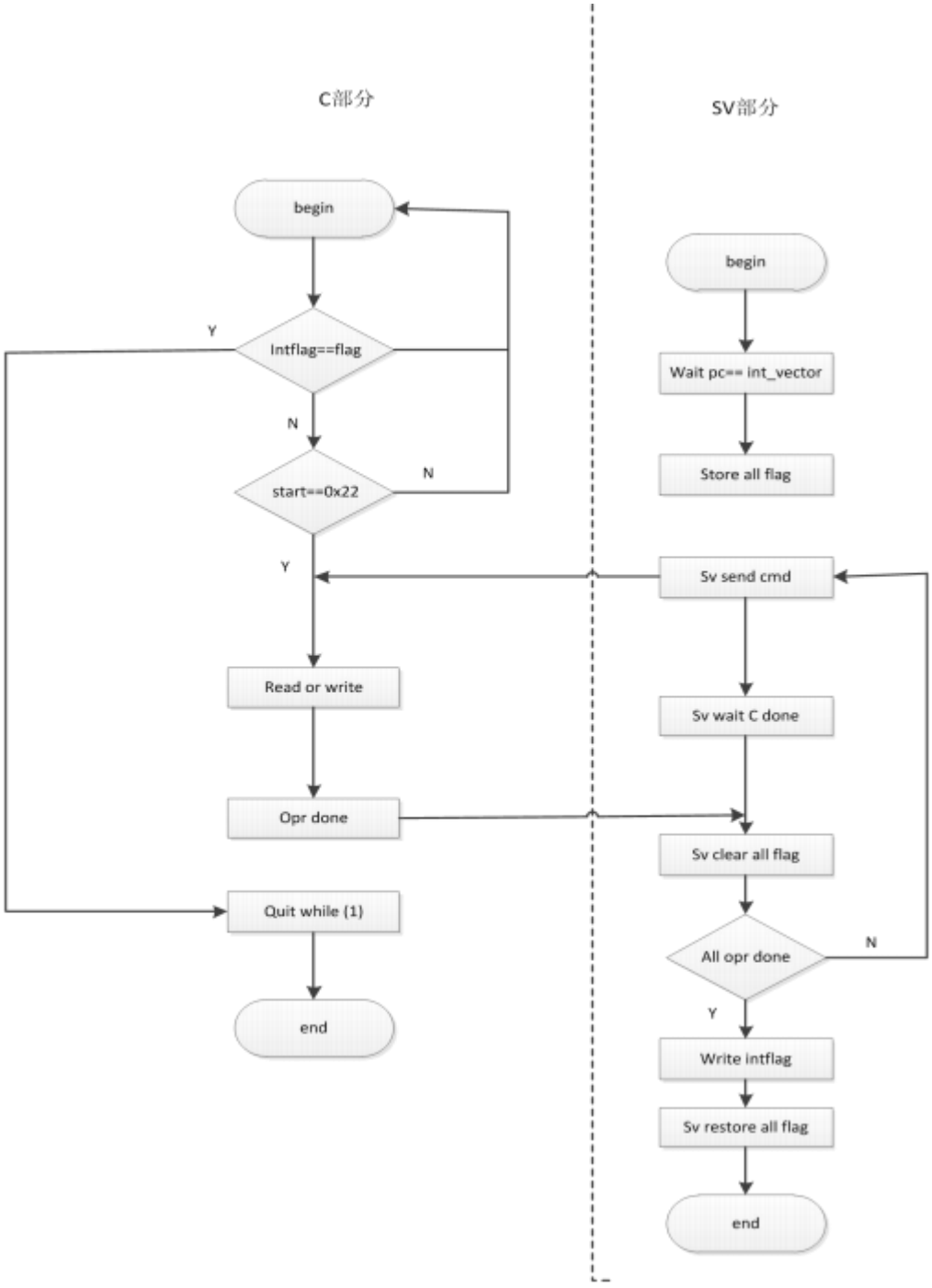
Main 函数与 SV 交互如下



C 程序只有开始，没有结束，永远在 while 循环内；

程序开始，查询 start 是否为 0x22，如果不是则一直循环，如果 SV 发送了命令后，SV 会一直等待 statu，则 start 为 0x22,C 程序则会再判断 kind 等变量，完成此次操作，完成后给 statu 写 0x11;SV 等到 statu=0x11 后，则一次交互完成；

C 中断函数与 SV 中断函数交互如下：



当发生中断时，C 进入中断函数，查询 intflag 是否为 flag 值（中断地址向量，由 SV 写入），如果不是则查询 start 值，与 main 函数步骤一致，如果是，写标志，表示 sv 部分中断操作已完成，则退出中断函数；

当发生中断时（根据 PC 值判断），SV 的 while 退出，同时备份交互状态内容，对所有其它的控制位 stop 写 1（此时其它所有的SV OPR 操作都会在wait

条件）；完成上述动作，则可以使用该中断的 OPR 操作各种寄存器，当完成了中断动作时，对 intflag 空间些中断向量，告诉 C，SV 中断操作已完成，C 可以退出中断函数；同时恢复 OPR 所有的握手空间内容，并清除所有的stop 位。

上述方法能够实现，主要利用了MAIN 函数和所有的中断函数中的 OPR 握手空间为相同空间，同时采用类的特性来实现；如下

```
`define START  `XRAM[`OPR_BASE_ADDR+0][7:0]
`define KINDS  `XRAM[`OPR_BASE_ADDR+1][7:0]
`define ADDRL  `XRAM[`OPR_BASE_ADDR+2][7:0]
`define ADDRH  `XRAM[`OPR_BASE_ADDR+3][7:0]
`define WDATA  `XRAM[`OPR_BASE_ADDR+4][7:0]
`define RDATA  `XRAM[`OPR_BASE_ADDR+5][7:0]
`define STATUS `XRAM[`OPR_BASE_ADDR+6][7:0]
`define INITFLAG `XRAM[`OPR_BASE_ADDR+7][7:0]
`define BACKFLAG `XRAM[`OPR_BASE_ADDR+8][7:0]
```

定义握手空间

```
wire[7:0] debug_START = `START;
wire[7:0] debug_KINDS = `KINDS;
wire[7:0] debug_ADDRL = `ADDRL;
wire[7:0] debug_ADDRH = `ADDRH;
wire[7:0] debug_WDATA = `WDATA;
wire[7:0] debug_RDATA = `RDATA;
wire[7:0] debug_STATUS = `STATUS;
wire[7:0] debug_INITFLAG = `INITFLAG;
wire[7:0] debug_BACKFLAG = `BACKFLAG;
```

debug 时，观察波形用

d) Tb 部分内容

```
`define DATAWIDE  8
```

```
`define ADDRWIDE  16
```

Bus_opr 部分，含各种寄存器操作

可以用在寄存器扫描中,下面 bus_opr 写了 2 种方式，功能差不多，后一种简单点，不带中断功能。

Bus_opr 方式一

```
Class bus_opr;
      string handle_name;  //debug 时，知道 task 是哪个句柄调用的
    bit stop;  //中断时，控制位
    bit stop_bak[];
        bit[`DATAWIDE-1:0] start_bak;
        bit[`DATAWIDE-1:0] kinds_bak;
        bit[`DATAWIDE-1:0] addr_bak;
        bit[`DATAWIDE-1:0] wdata_bak;
        bit[`DATAWIDE-1:0] rdata_bak;
        bit[`DATAWIDE-1:0] status_bak;
    //发生中断时备份握手值
      function new(input int numb,string handle);
          stop = 1'b0;
          stop_bak = new[numb];
          handle_name = handle;
      endfunction   //
      task initial_flag;   //初始化变量
          `START = 'h0;
          `KINDS = 'h0;
          `ADDRL = 'h0;
          `ADDRH = 'h0;
          `WDATA = 'h0;
          `RDATA = 'h0;
```

```
        `STATUS = 'h0;
    endtask
    task store_flag; //保存变量
        #1;
        start_bak = `START ;
        kinds_bak = `KINDS ;
        addrl_bak = `ADDRL ;
        addrh_bak = `ADDRH ;
        wdata_bak = `WDATA ;
        rdata_bak = `RDATA ;
        status_bak = `STATUS;
    endtask
    task restore_flag; //恢复变量
        `START = start_bak;
        `KINDS = kinds_bak;
        `ADDRL = addrl_bak;
        `ADDRH = addrh_bak;
        `WDATA = wdata_bak;
        `RDATA = rdata_bak;
        `STATUS = status_bak;
    endtask

    task reg_read(input[`ADDRWIDE-1:0] addr,output [`DATAWIDE-1:0]
rdata);   //寄存器读操作
         wait((`STATUS==8'h00)&(!stop));
         `KINDS = 8'h55;
         `ADDRL = addr[7:0];
         `ADDRH = addr[15:8];
         `START = 8'h22;
```

```
        wait((`STATUS==8'h11)&(!stop));
        rdata = `RDATA;
        `STATUS = 'h0;
        `START  = 'h0;
        `KINDS  = 'h0;
        `ADDRL  = 'h0;
        `ADDRH  = 'h0;
    endtask

    task reg_write(input[`ADDRWIDE-1:0] addr,input [`DATAWIDE-1:0]
wdata);  //寄存器写操作
        wait( (`STATUS==8'h00)&(!stop) );
        `KINDS = 8'haa;
        `ADDRL = addr[7:0];
        `ADDRH = addr[15:8];
        `WDATA = wdata;
        `START = 8'h22;
        wait( (`STATUS==8'h11)&(!stop) );
        `STATUS = 'h0;
        `START  = 'h0;
        `KINDS  = 'h0;
        `ADDRL  = 'h0;
        `ADDRH  = 'h0;
  endtask
      task reg_Bitwrite1(input[`ADDRWIDE-1:0]
addr,input[`DATAWIDE-1:0] bit_pos);
          reg[`DATAWIDE-1:0] datatemp;
          reg_read(addr,datatemp);
```

```
            //$display ("datatemp is %0h,, (~bit_pos&datatemp) |bit_pos
is %0h",
            datatemp, (~bit_pos&datatemp) |bit_pos);
            reg_write(addr,  ((~bit_pos)&datatemp) |bit_pos);
            $display ("reg_Bitwrite1 add is %0h,,valid bit  is
8'b%8b",addr,bit_pos);
        endtask

        task reg_Bitwrite0(input[`ADDRWIDE-1:0]
addr,input[`DATAWIDE-1:0] bit_pos);
            reg[`DATAWIDE-1:0] datatemp;
            reg_read(addr,datatemp);
            reg_write(addr, datatemp&(~bit_pos));
            $display ("reg_Bitwrite0 add is %0h,,valid bit  is
'b%8b",addr,bit_pos);
        endtask

        task reg_rw_judge(input  [`ADDRWIDE-1:0]
addr,input[`DATAWIDE-1:0] valid,ref int fail);
            int fail_temp;
            reg[`DATAWIDE-1:0] default_value;
            reg[`DATAWIDE-1:0] read_value;
            fail_temp = fail;
            reg_read(addr,default_value);
            for(reg[`DATAWIDE:0]
i='h1;i<{1'b1, {`DATAWIDE{1'b0}}};i=i<<1) begin
                if((valid&i)!=0) begin
                    reg_write(addr,i|default_value);  //write 1
                    reg_read(addr,read_value);
```

```
                    if(read_value!==(i|default_value))begin
                        fail++;
                        $display ("[ERROR:]write 1,addr %0h quote is
err,
                        postion is %h,write data is %h,read&valid data
is %h ",
                        addr,i,i,read_value&valid);
                    end
                    reg_write(addr, (~i)&default_value); //write 0
                    reg_read(addr,read_value);
                    if(read_value!=((~i)&default_value))begin
                        fail++;
                        $display ("[ERROR:]write 0,addr %0h quote is
err,
                        postion is %h,write data is %h,read data is %h
",
                        addr,i,~i,~i|read_value);
                    end
                end
            end
            if(fail_temp==fail) $display ("reg addr:%0h test is
ok",addr);
            else $display ("ERROR:reg addr:%0h test is err",addr);
        endtask
      task wr_wfe;
            begin
                wait(`STATUS=='h00);
                `KINDS = 'hbb;
                `START = 'h22;
```

```
                $display ("write wfe",$time);
            end
        endtask

        task wr_wfi;
            begin
                wait(`STATUS=='h00);
                `KINDS = 'hcc;
                `START = 'h22;
                $display ("write wfi",$time);
            end
        endtask
    ......
Endclass
```

Bus_opr 方式二

下面的代码根据 8051 总线实现，分为读写 code,xram,iram,sfr，对应的 c 也要做相应的改变，使用的格式如下：
这种操作方式是在第一种方式作了扩展，功能更强，可以实现所有的 8051 操作，尤其是用在寄存器扫描上有优势。见下面的 8051 系统,ibus,xbus,pbus,sfr 统一固件源码

3. C 部分内容分析如下：

下面是 51 系统，所以每个交互时间特别长，如果是ARM 那类统一编址的CORE，则速度会快非常多，C 语句也就 10 来行搞定，下面内容没有把 CODE 当作数据读的功能，

Main 函数方式一：

```
void main(void)
 {
```

```
UINT addrtemp;
while(1) {
    if(START==0x22){   //如果 SV 没发送操作，则退出一直循环
        if(KINDS==0x55)     //read
        {
            if(ADDRH!=0x0)    {   //xbus
                addrtemp=(ADDRH<<8)+ADDRL;
                RDATA = XBYTE[addrtemp]  ;
            }
            else if((ADDRL&0x80)==0x0) {   //addr is 0~7f
                RDATA = DBYTE[ADDRL];
            }
            else{        //sfr
                switch(ADDRL) {
                    case 0x80:  {RDATA = P0;  break;}
                    case 0x90:  {RDATA = P1;  break;}
                    ……
                    default: {printf("[C:ERROR:] \n");break;}
                }
            }
        }
        else if(KINDS==0xaa) {        //write
            if(ADDRH!=0x0)    {   //xbus
                addrtemp=(ADDRH<<8)+ADDRL;
                XBYTE[addrtemp] = WDATA  ;
            }
            else if((ADDRL&0x80)==0x0) {   //addr is 0~7f
                DBYTE[ADDRL]= WDATA;
            }
```

```
            else{
                switch(ADDRL) {
                    case 0x80:  { P0 = WDATA;break;}
                    case 0x90:  { P1 = WDATA;break;}
                    ……
                    default:{printf("[C:ERROR:] \n");break;}
                }
            }
        }
        else{
            printf("[C:ERROR:] BUS OPER TYPE  IS ERR \n") ;
        }
        STATUS = 0x11;    //告诉 SV 此次操作完成
    }
  }
}
```

如果是 cotexm0 系统等统一编址的内核,则 c 代码更为简单

```
#include <core_cmInstr.h>
int main(void)
{
  while(1) {
      if(START==0x22) {       //start_flag
          if(KINDS==0x55)     //read
          {
              RDATA =  (*((unsigned int  *)  (ADDR)));
          }
          else if(KINDS==0xaa) {       //write
              (*((unsigned int  *)  (ADDR))) = WDATA  ;
          }
```

```
          else if(KINDS==0xbb){
             __WFE();
          }
          else if(KINDS==0xcc){
             __WFI() ;
          }
          STATUS = 0x11;    //end_flag
      }
  }
}
```

如果带有中断功能，中断部分 c 程序如下

```
void isr_int0(void) interrupt 0
{
  UINT addrtemp;

  while(1){
      if (INTFLAG==0x03){    //sv 环境中再中断中对此变量写 0x3，表示退出中断，0x3 是约定的值，可以更改，建议与中端向量相关
          //BACKFLAG = 0x03;  //放到最后更好
          break;
      }
      else if(START==0x22){
          if(KINDS==0x55)     //read
          {
              if(ADDRH!=0x0)    {   //xbus
                  addrtemp=(ADDRH<<8)+ADDRL;
                  RDATA = XBYTE[addrtemp]  ;
              }
```

```
            else if((ADDRL&0x80)==0x0) {   //addr is 0~7f
                RDATA = DBYTE[ADDRL];
            }
            else{       //sfr
                switch(ADDRL) {
                    case 0x80:  {RDATA = P0;  break;}
                    case 0x90:  {RDATA = P1;  break;}
                    。。。。。。
                    default: {printf("[C:ERROR:] SFR READ ADDR IS ERR
\n");break;}
                }
            }
        }
        else if(KINDS==0xaa) {       //write
            if(ADDRH!=0x0)    {   //xbus
                addrtemp=(ADDRH<<8)+ADDRL;
                XBYTE[addrtemp] = WDATA  ;
            }
            else if((ADDRL&0x80)==0x0) {   //addr is 0~7f
                DBYTE[ADDRL]= WDATA;
            }
            else{
                switch(ADDRL) {
                    case 0x80:  { P0 = WDATA; break;}
                    case 0x90:  { P1 = WDATA; break;}
                    。。。。。。
                    default: {printf("[C:ERROR:] SFR WRITE ADDR IS
ERR \n");break;}
                }
```

```
            }
        }
        else{
            printf("[C:ERROR:] BUS OPER TYPE  IS ERR \n") ;
        }
        STATUS = 0x11;    //
    }
  }
  BACKFLAG = 0x03; //告诉 sv 退出中断，握手信号
}
```

4. SV 环境

```
    initial begin
        bit[7:0] statu;
        bit[7:0] rdata;
        int i;
        bus_opr reg_opr_main ;
        bus_opr reg_opr_int0;
        bus_opr reg_opr_int1;
        bus_opr reg_opr_int2;
        bus_opr reg_opr_int3;
        reg_opr_int0 = new(5,"reg_opr_int0"); //5 为中断个数+1 个
main 函数
        reg_opr_int1 = new(5,"reg_opr_int1");
        reg_opr_int2 = new(5,"reg_opr_int2");
        reg_opr_int3 = new(5,"reg_opr_int3");
        reg_opr_main = new(5,"reg_opr_main") ;
        $display ("reg_opr_main.stop = %0h",reg_opr_main.stop);
        initial_xram;
```

```
        reg_opr_main.initial_flag;
        @(posedge `ResetnMcu);
fork
    begin
        reg_opr_main.reg_Bitwrite1(`MISCR_ADDR,8'h08); //开启系统
时钟
        $display ("========tc=%0d=====",tc_num);
        case(tc_num)
            6:begin
                timer_test(reg_opr_main);  //待测试的 tc
              end
            99:#100_000_000;
            default:$display ("tc number is error ,tc is
0d%0d",tc_num);
        endcase
    end

interrupt0(reg_opr_main,reg_opr_int0,reg_opr_int1,reg_opr_int2,re
g_opr_int3,);

interrupt1(reg_opr_main,reg_opr_int0,reg_opr_int1,reg_opr_int2,re
g_opr_int3);

interrupt2(reg_opr_main,reg_opr_int0,reg_opr_int1,reg_opr_int2,re
g_opr_int3);

interrupt3(reg_opr_main,reg_opr_int0,reg_opr_int1,reg_opr_int2,re
g_opr_int3);
join_any
```

```
disable fork//根据实际情况考虑是否需要
        #10_000_000;
        if(pass==1'b1)begin

$display("////////////////////////////////////////////////////////
/////");
            $display("////////////////this is tc(%0d) is pass
////////////////////",tc_num);

$display("////////////////////////////////////////////////////////
/////");
        end
        else begin

$display("////////////////////////////////////////////////////////
/////");
            $display("////////////////this is tc(%0d) is fail
////////////////////",tc_num);

$display("////////////////////////////////////////////////////////
/////");
        end
        $finish();
    end
```

SV 环境中，中断函数分析

```
`define INT0_ADDR   8'h03
`define INT1_ADDR   8'h0b
`define INT2_ADDR   8'h13
`define INT3_ADDR   8'h1b
```

```
`define INT4_ADDR   8'h23
`define INT5_ADDR   8'h2c
// 上述定义指示握手标志，内容不定
task interrupt0(bus_opr
reg_opr_main,reg_opr_int0,reg_opr_int1,reg_opr_int2,reg_opr_int3)
;
    bit[7:0] rdata;
    while(1) begin
        while(1)begin
            @(posedge `core_fclk);
            if(`PC==`INT0_ADDR) begin   //enter into int0
                `INITFLAG = 8'h0;  //清除握手信号
                break;
            end
        end
        $display("now enter into int0",$time);
//////////上面部分是进中断判断，////////
        reg_opr_int0.store_flag;   //保护 SV 现场
        reg_opr_int0.stop_bak[0]= reg_opr_main.stop;
        reg_opr_int0.stop_bak[1]= reg_opr_int0.stop;
        reg_opr_int0.stop_bak[2]= reg_opr_int1.stop;
        reg_opr_int0.stop_bak[3]= reg_opr_int2.stop;
        reg_opr_int0.stop_bak[4]= reg_opr_int3.stop;
     ///////上述部分 相同 ，可以用宏的方式来 实现
        reg_opr_main.stop = 1'b1;//
        reg_opr_int0.stop = 1'b0;//
        reg_opr_int1.stop = 1'b1;//
        reg_opr_int2.stop = 1'b1;//
        reg_opr_int3.stop = 1'b1;//
```

```
        #1;
        reg_opr_int0.initial_flag;//初始化握手信号
///////////////上面保护现场，基本相同////////
        reg_opr_int0.reg_read(`IRQ0_ADDR,rdata);
        if((rdata&8'h01)==8'h01) begin
        end
        if((rdata&8'h02)==8'h02) begin
            reg_opr_int0.reg_Bitwrite0(`TOSR_ADDR,8'h01);
        end
        if((rdata&8'h04)==8'h04) begin
        end
        if((rdata&8'h08)==8'h08) begin
        end
        if((rdata&8'h10)==8'h10) begin
            reg_opr_int0.reg_Bitwrite0(`ESDSR_ADDR,8'h40);
        end
///////////////上面部分是中断函数的主体////////
        `INITFLAG = `INT0_ADDR;  //表示中断函数结束
        wait(`BACKFLAG==`INT0_ADDR);//等待 C 运行到最后
        reg_opr_int0.restore_flag;   //恢复退出前握手内容
    reg_opr_main.stop = reg_opr_int0.stop_bak[0];//
        reg_opr_int0.stop = reg_opr_int0.stop_bak[1];//
        reg_opr_int1.stop = reg_opr_int0.stop_bak[2];//
        reg_opr_int2.stop = reg_opr_int0.stop_bak[3];//
        reg_opr_int3.stop = reg_opr_int0.stop_bak[4];/所有函数正常
运行
        #1;  //为了 debug 能够看到`BACKFLAGK 变化
        `BACKFLAG=0;
/////////上面部分基本相同，只需要改动握手值//////
```

end

endtask

其它的中断与这个相似的做法 。上面有些 地方可以验证 工程师封装好，设计工程师 只需要修改中间部分即可

对于中断的优先级及打断等情况，SV 会根据 C 自动完成。

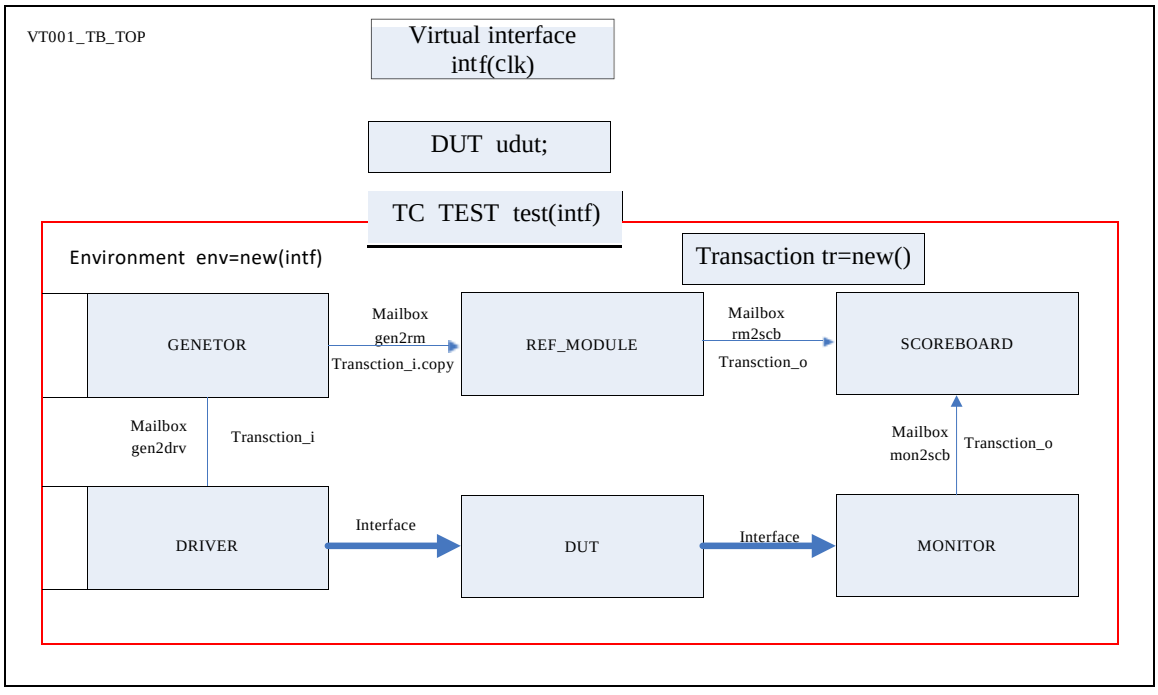
其它 SV 的中断函数与此相似；只需要改动少部分；

注意：对于高速通讯接口，很难适应，因为此种方式，C 耗时太长，有可能会丢失数据；

上述是在传统的验证上使用该方法,在实际项目中可以使用；

5. SVTB 构架使用（理论分析，没实际使用）

下面是 SVTB 架构



SVTB 标准模板如下：

```
Class Environment;
  Generator  gen;
  Agent  agt;
```

```
    Driver  drv
    ……
    Mailbox gen2agt,agt2drv,mon2chk;
    Extern function new();
    Extern function void gen_cfg ();
    Extern function void build();
    Extern task run();
    ------
Endclass
Function void environment ::build();
  Gen2agt =new();
  ……
    Gen = new(gen2agt)
  ……
Endfunction
Task environment::run();
  Fork
    Gen.run();
    Agt.run();
    Drv.run();
    Mon.run();
    ……
  join
endtask

program automatic test;
  Environment env
  Intial begin
    Env = new();
```

```
    Env.gen_cfg ();
    Env.build();
    Env.run();
    Env.wrap_up ();
  end
endprogram
```

SVTB 验证基本运行过程如下：

1. New 验证环境
2. 产生环境配置
3. Build 函数把各个模块连接起来
4. 环境运行起来

环境运行的基本过程是：

Generator 产生 transaction 内容通过 mailbox put 到 driver，同时 put 一份给 refrence，refrence 根据此 transaction 内容得到 gold data,并通过 mailbox put 到 scoreboard,;

Driver 通过 mailbox get 到 taransation, 对 DUT 产生激励;

Monitor 根据条件对 DUT 采集数据，通过 mailbox put 到 scoreboard;

Scoreboard 在 get 到 monitor 内容后再与 refrence put 过来的内容比较，得到结果;

根据上面分析，需要对上述组件进行修改：

1. Transaction ： 因为 soc 产品都是配置一些寄存器来让某个模块工作，所以，我们的 transaction 可以使用队列，把所有需要配置的寄存器地址，数据放在队列中，再一次性put给 driver;同时 ref 也根据配置来产生 gold 值;

2. Driver:需要 new 相关 bus_opr 的句柄，来接收传递过来的参数，并根据 transaction 的内容来调用 bus_opr 的 write 函数;

3. Monitor: 此类需要 new 相关 bus_opr 的句柄，来接收传递过来的参数，主要是中断用的 bus_opr;在该环境中，monitor 有 driver 的功能，当发生中断时，monitor 的 run 函数会运行起来，读标志位，清标志位 ，读数据，并把标志位，及数据打包通过 mailbox put 到 scoreboard 对比；

所以需要修改的地方如下：

1. Environment 类

```
Class Environment;
      Bus_opr  reg_opr_main;
      Bus_opr  reg_opr_int0;
      Bus_opr  reg_opr_int1;
      // reg_opr_intx 根据 CORE 特性来确定有多少路中断
      Generator  gen;
     Agent   agt;
      Driver  drv
      ……

Endclass
Function build();
      reg_opr_main = new(3,“reg_opr_main”);
      reg_opr_int0 = new(3, “reg_opr_int0”);
      reg_opr_int1 = new(3, “reg_opr_int1”);
      gen = new();
      drv = new(agt2drv, reg_opr_main);
      mon = new(mon2chk, reg_opr_main, reg_opr_int0, reg_opr_int1);

  ……;
endfunction
```

driver class 需要做如下改动：

```
class driver;
  mailbox gen2drv;
  bus_opr  reg_opr_main  ;//增加部分
       function  new(input mailbox gen2drv, bus_opr  reg_opr_main)
          this. gen2drv = gen2drv;
          this. reg_opr_main = reg_opr_main;
       endfunction
       task run;
          transaction tr;
          forever begin
              gen2drv.get(tr);
              for(int i=0;i<tr.addr.size(),i++)begin
reg_opr_main(tr.addr[i],tr.data[i]);//其实就是调用 C 中的 main 函数，当进入中断时，此处会自动进入 wait 语句,等待中断函数处理完成再往下继续
              end
       end
       endtask
endclass
```

monitor class 需要做如下改动：

```
class monitor;
  mailbox mon2scb;
bus_opr  reg_opr_int0 ;//增加部分,当待测模块只有一个中断时，则需要一个
bus_opr  reg_opr_int1 ;//增加部分,当待测模块只有一个中断时，则需要一个
```

， 根据实际情况增加多路中断

```
    function  new(input mailbox mon2scb, bus_opr  reg_opr_int0, reg_opr_int1)
        this. mon2scb = mon2scb;
        this. reg_opr_int0 = reg_opr_int0;
        this. reg_opr_int1 = reg_opr_int1;
    endfunction
    task run;
      while(1)begin
      while(1)begin
                @(posedge `core_fclk);
                if(`PC==`INT0_ADDR) begin   //enter into int0
                          `INITFLAG = 8'h0;  //清除握手信号
                          break;
                end
            end
            $display("now enter into int0",$time);
    ///////////上面部分是进中断判断，////////
            reg_opr_int2.store_flag;   //保护 SV 现场
    reg_opr_int0.stop_bak[0]= reg_opr_main.stop;
             reg_opr_int0.stop_bak[1]= reg_opr_int0.stop;
            reg_opr_int0.stop_bak[2]= reg_opr_int1.stop;
            reg_opr_main.stop = 1'b1;// SV  main 函数停止
            reg_opr_int0.stop = 1'b0;//
            reg_opr_int1.stop = 1'b1;// SV  其它中断函数停止
            reg_opr_int0.initial_flag;//初始化握手信号
    ///////////////上面保护现场，基本相同////////
            reg_opr_int0.reg_read(`IRQ0_ADDR,rdata);
            if((rdata&8'h01)==8'h01) begin
            end
```

```
            if((rdata&8'h02)==8'h02) begin
            reg_opr_int0.reg_Bitwrite0(`TOSR_ADDR,8'h01);
            end
            if((rdata&8'h04)==8'h04) begin
            end
            if((rdata&8'h08)==8'h08) begin
            end
            if((rdata&8'h10)==8'h10) begin
               reg_opr_int0.reg_Bitwrite0(`ESDSR_ADDR,8'h40);
            end
      ///////////////上面部分是中断函数的主体,////////
            `INITFLAG = `INT0_ADDR;  //表示中断函数结束
            wait(`BACKFLAG==`INT0_ADDR);//等待 C 运行到最后
            reg_opr_int0.restore_flag;   //恢复退出前握手内容
         reg_opr_main.stop = reg_opr_int0.stop_bak[0];//
            reg_opr_int0.stop = reg_opr_int0.stop_bak[1];//
            reg_opr_int1.stop = reg_opr_int0.stop_bak[2];//
            #1;  //为了 debug 能够看到`BACKFLAGK 变化
            `BACKFLAG=0;
      /////////上面部分基本相同，只需要改动握手值//////
         mon2scb.put(xx);
         end
      endtask
endclass
```

注意：如果是多路中断，想把所有的 monitor 写一起，则需要用 fork join 语句来完善。

根据实际情况使用虚方法及派生类的特点来实现或许是个更好的策略。

```
program automatic test;
  Environment env;
```

```
  Intial begin
      Env = new();
      Env.gen_cfg ();
      Env.build();
      Env.run();
      Env.wrap_up ();
  end
endprogram
```

根据上述实现方法 ，应该也可以在 UVM 环境中实现；

6. 总结：

上述方法主要是利用了中断函数进入和退出执行需要时间的特性来实现，当进入中断到执行中断函数需要一个过程，SV 在此过程做好一系列保护现场工作；

当中断执行最后一条语句到真正退出到 MAN 函数也需要一个过程，SV 这时可以进行现场恢复动作；

当进入中断时，MAIN 函数暂停，SV 本来是不会停止，但在 SV 的每个 bus_opr 的 task 中加了wait()来控制，所以当一进入中断地址，SV 的中断就控制住 SV main 中断的 wait 语句，因为在监控进入中断是一个 cyc le 级的行为，所以响应能够及时；因为所有的函数的进入及退出都需要好多个 cycle 才能完成。

利用上述方法，做一般的系统级模块验证都可以实现，而根本不需要写各种 C 程序，大大简化了设计工程师工作的复杂程度，

7. 8051 系统,ibus,xbus,pbus,sfr 统一固件源码

---此方案代码多些,运行速度比上面的 sfr 操作慢,但功能齐全

C 部分:

```
///KINDS =0X55  :READ;  KINDS=0XAA  :WRITE;
#define OPR_ADDR_BASE  1023
//1023 为 xram 最大地址
#define START   XBYTE[OPR_ADDR_BASE-0]
#define KINDS   XBYTE[OPR_ADDR_BASE-1]
#define ADDRL   XBYTE[OPR_ADDR_BASE-2]
#define ADDRH   XBYTE[OPR_ADDR_BASE-3]
#define WDATA   XBYTE[OPR_ADDR_BASE-4]
#define RDATA   XBYTE[OPR_ADDR_BASE-5]
#define STATUS  XBYTE[OPR_ADDR_BASE-6]
#define XBUS8(addr)      (*((unsigned char volatile xdata *) (addr)))
#define IBUS8(addr)      (*((unsigned char volatile data *) (addr)))
#define PBUS8(addr)      (*((unsigned char volatile code *) (addr)))
void main(void)
 {
  UINT addrtemp;
  UINT datatemp;
  printf("C:start c code \n");
  VTOR = 0x08;
  while(1) {
      if(START==0x22) {
          if((KINDS&0xf)==0x5) {
              if(KINDS==0x15) {
//0x15 read code ,0x25 read ixram,0x35 read xram  0x45 read sfr
                  addrtemp=(ADDRH<<8)+ADDRL;
                  RDATA=PBUS8(addrtemp);
              }
              else if(KINDS==0x25) { // iram add is 0x00 ~ 0xff
```

```
        RDATA=IBUS8(ADDRL);
    }
    else  if(KINDS==0x35) {
        addrtemp=(ADDRH<<8)+ADDRL;
        RDATA=XBUS8(addrtemp);
    }
    else  {
           switch(ADDRL) {
               case 0x80:  {RDATA = P0;  break;}
               case 0x90:  {RDATA = P1;  break;}
               case 0xa0:  {RDATA = P2;  break;}
               case 0xb0:  {RDATA = P3;  break;}
               case 0xc0:  {RDATA = P4;  break;}
               case 0xc8:  {RDATA = P5;  break;}
               case 0xd0:  {RDATA = PSW;break;}
               case 0xE0:  {RDATA = ACC;break;}
               case 0xF0:  {RDATA = B;   break;}
               case 0x81:  {RDATA = SP;  break;}
               case 0x82:  {RDATA = DPL; break;}
               case 0x83:  {RDATA = DPH; break;}

               case 0x86:  {RDATA = PSEQ;    break;}
               case 0x87:  {RDATA = PCON;    break;}
               case 0x88:  {RDATA = VTOR;    break;}

               case 0xA8:  {RDATA = IE;  break;}
               case 0xB8:  {RDATA = IP;  break;}

//                 case 0x89:  {RDATA = ADCDIV;   break;}
```

```
//                      ---------
//                      case 0xAD:  {RDATA = MDBH;   break;}
                        default: {printf("[C:ERROR:] SFR READ ADDR IS
ERR \n");break;}
                    }
                }
            }

        else if((KINDS&0xf)==0xa) {      ////0x1a write code ,0x2a
write ixram,0x3a write xram  0x4a .. xfa write sfr
            if(KINDS == 0x1a) {
                printf("[C:ERROR:] write code need special function
\n");
            }
            else  if(KINDS == 0x2a) { //IRAM
                IBUS8(ADDRL) = WDATA  ;
            }
            else  if(KINDS == 0x3a) { //xRAM
                addrtemp=(ADDRH<<8)+ADDRL;
                XBUS8(addrtemp) = WDATA  ;
            }
            else{
                switch(ADDRL) {
                    case 0x80:  { P0 = WDATA;break;}
                    case 0x90:  { P1 = WDATA;break;}
                    case 0xa0:  { P2 = WDATA;break;}
                    case 0xb0:  { P3 = WDATA;break;}
                    case 0xc0:  { P4 = WDATA;break;}
                    case 0xc8:  { P5 = WDATA;break;}
```

```
                    case 0xd0:  { PSW = WDATA;break;}
                    case 0xE0:  { ACC = WDATA;break;}
                    case 0xF0:  { B = WDATA;  break;}
                    case 0x81:  { SP = WDATA; break;}
                    case 0x82:  { DPL = WDATA;   break;}
                    case 0x83:  { DPH = WDATA;   break;}

                    case 0x86:  { PSEQ = WDATA;  break;}
                    case 0x87:  { PCON = WDATA;  break;}
                    case 0x88:  { VTOR = WDATA;  break;}

                    case 0xA8:  {IE = WDATA;  break;}
                    case 0xB8:  {IP = WDATA;  break;}

//                      case 0x89:  { ADCDIV = WDATA;   break;}
//.........
//                     case 0xAD:  { MDBH = WDATA;  break;}
                    default:{printf("[C:ERROR:] SFR WRITE ADDR IS
ERR \n");break;}
                }
            }
        }
        else{
            printf("[C:ERROR:] BUS OPER TYPE  IS ERR \n") ;
        }
        STATUS = 0x11;    //
        datatemp = PBUS8(11);
    }
  }
```

```
}
```

Sv 部分

下面内容包含了各种寄存器操作，基本的读写，读写权限，只读，只写，写 1 清 0，写 0 清 0 等行为

```
`define OPR_BASE_ADDR  1023
//1023 是 xram 最大地址  - 如果系统的xram 更大，可以更改，实际使用最好不要定义到 1023，定义到 1022，最高地址采用扫描的方式，确认边界没问题。
`define XRAM   uXRAM.mem_array
`define START  `XRAM[`OPR_BASE_ADDR-0]
`define KINDS  `XRAM[`OPR_BASE_ADDR-1]
`define ADDRL  `XRAM[`OPR_BASE_ADDR-2]
`define ADDRH  `XRAM[`OPR_BASE_ADDR-3]
`define WDATA  `XRAM[`OPR_BASE_ADDR-4]
`define RDATA  `XRAM[`OPR_BASE_ADDR-5]
`define STATUS `XRAM[`OPR_BASE_ADDR-6]
`define INITFLAG `XRAM[`OPR_BASE_ADDR-7]

`define PRINTF(str) \
    if(p_lever == 1) $display(str,$time);\
    else $display(" ",$time);
//kind : 0x1a write code ,0x2a write ixram,0x3a write xram  0x4a .. xfa write sfr
//0x15 read code ,0x25 read ixram,0x35 read xram  0x45 ... 0xf5 read sfr

wire[7:0] debug_START = `START;
wire[7:0] debug_KINDS = `KINDS;
wire[7:0] debug_ADDRL = `ADDRL;
```

```
wire[7:0] debug_ADDRH = `ADDRH;
wire[7:0] debug_WDATA = `WDATA;
wire[7:0] debug_RDATA = `RDATA;
wire[7:0] debug_STATUS = `STATUS;
wire[7:0] debug_INITFLAG = `INITFLAG;
 //16377 has been used
class bus_opr #(width=8);
    static int p_lever = 1 ;
    static bit stop_interrupt = 1'b0;
    bit[7:0] start_bak;
    bit[7:0] kinds_bak;
    bit[7:0] addrl_bak;
    bit[7:0] addrh_bak;
    bit[7:0] wdata_bak;
    bit[7:0] rdata_bak;
    bit[7:0] status_bak;
    static  task initial_flag;
        `START = 'h0;
        `KINDS = 'h0;
        `ADDRL = 'h0;
        `ADDRH = 'h0;
        `WDATA = 'h0;
        `RDATA = 'h0;
        `STATUS = 'h0;
    endtask
    task store_flag;
        start_bak = `START ;
        kinds_bak = `KINDS ;
        addrl_bak = `ADDRL ;
```

```
        addrh_bak = `ADDRH ;
        wdata_bak = `WDATA ;
        rdata_bak = `RDATA ;
        status_bak = `STATUS;
        $display ("store data is
start_bak=%0h,kinds_bak=%0h,addrl_bak=%0h,addrh_bak=%0h,wdata_bak
=%0h,\
        rdata_bak=%0h,status_bak=%0h
",start_bak,kinds_bak,addrl_bak,addrh_bak,wdata_bak,rdata_bak,sta
tus_bak,$time);
        $display ("store data is
start_bak=%0h,kinds_bak=%0h,addrl_bak=%0h,addrh_bak=%0h,wdata_bak
=%0h,\
        rdata_bak=%0h,status_bak=%0h
",`START,`KINDS,`ADDRL,`ADDRH,`WDATA,`RDATA,`STATUS,$time);
        $display ("store data is
start_bak=%0h,kinds_bak=%0h,addrl_bak=%0h,addrh_bak=%0h,wdata_bak
=%0h,\
        rdata_bak=%0h,status_bak=%0h
",debug_START,debug_KINDS,debug_ADDRL,debug_ADDRH,debug_WDATA,deb
ug_RDATA,debug_STATUS,$time);
    endtask
    task restore_flag;
        `START = start_bak;
        `KINDS = kinds_bak;
        `ADDRL = addrl_bak;
        `ADDRH = addrh_bak;
        `WDATA = wdata_bak;
        `RDATA = rdata_bak;
```

```
        `STATUS = status_bak;
    endtask

    static  task bus_read(input string bus_type ="xbus",input[15:0]
addr,output  [width-1:0] rdata,input stop = stop_interrupt);
        fork
            begin
                wait(`STATUS==8'h00);
                //`KINDS =  {1'b0,8'h35};
                case(bus_type)
                    "pbus" :  `KINDS =  {1'b0^1'b0^1'b0^1'b1  ^
1'b0^1'b1^1'b0^1'b1,8'h15};
                    "ibus" :  `KINDS =  {1'b0^1'b0^1'b1^1'b0  ^
1'b0^1'b1^1'b0^1'b1,8'h25};
                    "xbus" :  `KINDS =  {1'b0^1'b0^1'b1^1'b1  ^
1'b0^1'b1^1'b0^1'b1,8'h35};
                    "sfrbus" :`KINDS =  {1'b0^1'b1^1'b0^1'b0  ^
1'b0^1'b1^1'b0^1'b1,8'h45};
                 endcase

                `ADDRL = addr[7:0];
                `ADDRH = addr[15:8];
                `START = 8'h22;
                wait(`STATUS[7:0]==8'h11);
                rdata = `RDATA[7:0];
                //$display ("reg_read is over,addr is 0x%0h,rdata is
0x%0h",addr,rdata,$time);
                //`PRINTF($sformat("reg_read is over,addr  is
0x%0h,rdata  is 0x%0h",addr,rdata));
```

```
                #1;
                `STATUS = 'h0;
                `START  = 'h0;
                `KINDS  = 'h0;
                `ADDRL  = 'h0;
                `ADDRH  = 'h0;
            end
            begin
                wait(!stop);
            end
        join
    endtask


    static  task bus_write(input string bus_type ="xbus",input[15:0]
addr,input  [width-1:0] wdata,input stop = stop_interrupt);
        fork
            begin
                wait(`STATUS==8'h00);
                //`KINDS = 8'h3a;
                case(bus_type)
                    "pbus" :  `KINDS = {1'b0^1'b0^1'b0^1'b1 ^
1'b1^1'b0^1'b1^1'b0,8'h1a};
                    "ibus" :  `KINDS = {1'b0^1'b0^1'b1^1'b0 ^
1'b1^1'b0^1'b1^1'b0,8'h2a};
                    "xbus" :  `KINDS = {1'b0^1'b0^1'b1^1'b1 ^
1'b1^1'b0^1'b1^1'b0,8'h3a};
```

```
                    "sfrbus" :`KINDS = {1'b0^1'b1^1'b0^1'b0 ^
1'b1^1'b0^1'b1^1'b0,8'h4a};
                endcase
                `ADDRL = addr[7:0];
                `ADDRH = addr[15:8];
                `WDATA = wdata;
                `START = 8'h22;
                wait(`STATUS[7:0]==8'h11);
                //$display("reg_write add is %0h,wdata
is %0h",addr,wdata,$time);
                // `PRINTF($sformat("reg_write add is %0h,wdata
is %0h",addr,wdata));
                #1;
                `STATUS = 'h0;
                `START  = 'h0;
                `KINDS  = 'h0;
                `ADDRL  = 'h0;
                `ADDRH  = 'h0;
            end
            begin
                wait(!stop);
            end
        join
    endtask


    static  task reg_read(input[15:0] addr,output [width-1:0]
rdata,input stop = stop_interrupt);
        fork
```

```
            begin
                wait(`STATUS==8'h00);
                `KINDS = {1'b0,8'h35};
                `ADDRL = addr[7:0];
                `ADDRH = addr[15:8];
                `START = 8'h22;
                wait(`STATUS[7:0]==8'h11);
                rdata = `RDATA[7:0];
                //$display("reg_read is over,addr is 0x%0h,rdata is
0x%0h",addr,rdata,$time);
                //`PRINTF($sformat("reg_read is over,addr is
0x%0h,rdata is 0x%0h",addr,rdata));
                #1;
                `STATUS = 'h0;
                `START  = 'h0;
                `KINDS  = 'h0;
                `ADDRL  = 'h0;
                `ADDRH  = 'h0;
            end
            begin
                wait(!stop);
            end
        join
    endtask

    static  task reg_write(input[15:0] addr,input [width-1:0]
wdata,input stop = stop_interrupt);
        fork
            begin
```

```
                wait(`STATUS==8'h00);
                `KINDS = 8'h3a;
                `ADDRL = addr[7:0];
                `ADDRH = addr[15:8];
                `WDATA = wdata;
                `START = 8'h22;
                wait(`STATUS[7:0]==8'h11);
                //$display("reg_write add is %0h,wdata
is %0h",addr,wdata,$time);
                // `PRINTF($sformat("reg_write add is %0h,wdata
is %0h",addr,wdata));
                #1;
                `STATUS = 'h0;
                `START  = 'h0;
                `KINDS  = 'h0;
                `ADDRL  = 'h0;
                `ADDRH  = 'h0;
            end
            begin
                wait(!stop);
            end
        join
    endtask


    static  task reg_Bitwrite1(input[15:0] addr,input[width-1:0]
bit_pos,input stop = stop_interrupt);
            reg[width-1:0] datatemp;
```

```
            reg_read(addr,datatemp,stop);
            //$display ("datatemp
is %0h,, (~bit_pos&datatemp) |bit_pos
is %0h",datatemp, (~bit_pos&datatemp) |bit_pos);
            reg_write(addr,  ((~bit_pos)&datatemp) |bit_pos,stop);
            $display ("reg_Bitwrite1 add is %0h,,valid bit  is
8'b%8b",addr,bit_pos);
    endtask

    static  task reg_Bitwrite0(input[15:0] addr,input[width-1:0]
bit_pos,input stop = stop_interrupt);
            reg[width-1:0] datatemp;
            reg_read(addr,datatemp,stop);
            reg_write(addr, datatemp&(~bit_pos),stop);
            $display ("reg_Bitwrite0 add is %0h,,valid bit  is
8'b%8b",addr,bit_pos);
        endtask



    static task reg_rw_judge(input  [15:0] addr,input[width-1:0]
valid);
            reg[width-1:0] default_value;
            reg[width-1:0] read_value;
            reg_read(addr,default_value);
            for(reg[width:0] i='h1;i<('h1<<width);i=i<<1) begin
                if((valid&i)!=0) begin
                    reg_write(addr,i);  //write 1
                    reg_read(addr,read_value);
```

```
                    if((read_value&valid)!==i)begin
                        pass =0;
                        $display ("[ERROR:]write 1,addr %0h quote is
err,postion is %h,write data is %h,read&valid data is %h
",addr,i,i,read_value&valid);
                    end
                    reg_write(addr,~i); //write 0
                    reg_read(addr,read_value);
                    if((~i|(read_value&valid))!=~i)begin
                        pass =0;
                        $display ("[ERROR:]write 0,addr %0h quote is
err,postion is %h,write data is %h,read data is %h
",addr,i,~i,~i|read_value);
                    end

                end
            end
    endtask

        static task reg_ro_judge(input [15:0] addr,input[width-1:0]
valid,ref  backdoor_variable);
            reg[width-1:0] default_value;
            reg[width-1:0] read_value;
            fork
                begin
                    while(1)begin
                        backdoor_variable = 1'h1 ;
                        #1;
                    end
```

```
                end
                begin
                    reg_write(addr,~valid);  //frontdoor write 0
                    reg_read(addr,read_value);
                    if(read_value&valid!=valid) begin
                        $display ("[ERROR:]read only,addr %0h quote
is err,backdoor write  is %0h,front read
is %0h",addr,backdoor_variable,read_value,$time);
                    end
                end
            join_any
            disable fork;
            fork
                begin
                    while(1)begin
                        backdoor_variable = 1'h0 ;
                        #1;
                    end
                end
                begin
                    reg_write(addr,valid);  //frontdoor write 1
                    reg_read(addr,read_value);
                    if((read_value & valid)!='h0) begin
                        $display ("[ERROR:]read only,addr %0h quote
is err,backdoor write is %0h,front read:read_value & valid
is %0h",addr,backdoor_variable,read_value & valid,$time);
                    end
                end
            join_any
```

```
            disable fork;
    endtask

    static task reg_wo_judge(input [15:0] addr,input[width-1:0]
valid,ref [0:0] backdoor_variable);
            reg[width-1:0] default_value;
            reg[width-1:0] read_value;
            reg_write(addr,~backdoor_variable);  //write 1
            reg_read(addr,read_value);
            if(backdoor_variable==(~default_value&valid)) begin
                $display("[ERROR:]write only,addr %0h quote is err
--1,backdoor_variable is %0h",addr,backdoor_variable,$time);
            end
            backdoor_variable = 1'h0 ;
            reg_write(addr,valid);  //write 1
            if(backdoor_variable != 1'b1) begin
                $display("[ERROR:]write only,addr %0h quote is err
--2,backdoor_variable is %0h",addr,backdoor_variable,$time);
            end
            reg_read(addr,read_value);
            if((read_value&valid)!='h0) begin
                $display("[ERROR:]write only,addr %0h quote is err
--3,read_value is %0h",addr,read_value,$time);
            end
            backdoor_variable = 1'h1 ;
            reg_write(addr,~valid);  //write 1
            if(backdoor_variable != 1'b0) begin
                $display("[ERROR:]write only,addr %0h quote is err
--4,backdoor_variable is %0h,",addr,backdoor_variable,$time);
```

```
            end
    endtask

    static task reg_w1c0_judge(input[15:0] addr,input[width-1:0]
valid,ref [0:0] backdoor_variable);
        reg[width-1:0] default_value;
        reg[width-1:0] read_value;
        backdoor_variable = 1'b1;
        reg_write(addr,valid);  //write 1
        if(backdoor_variable != 1'b0) begin
            $display("[ERROR:]write1c0 addr %0h quote is err
--1,backdoor_variable is %0h,",addr,backdoor_variable,$time);
        end
        reg_read(addr,read_value);
        if((read_value&valid)!='h0) begin
            $display("[ERROR:]write1c0,addr %0h quote is err
--2,read_value is %0h",addr,read_value,$time);
        end
        backdoor_variable = 1'b1;
        reg_write(addr,~valid);  //write 0
        if(backdoor_variable == 1'b0) begin
            $display("[ERROR:]write1c0,addr %0h quote is err
--3,backdoor_variable is %0h",addr,backdoor_variable,$time);
        end
        reg_read(addr,read_value);
        if((read_value&valid)=='h0) begin
            $display("[ERROR:]write1c0,addr %0h quote is err
--4,read_value is %0h",addr,read_value,$time);
        end
```

```
    endtask

    static task reg_w0c0_judge(input[15:0] addr,input[width-1:0]
valid,ref [0:0] backdoor_variable);
        reg[width-1:0] default_value;
        reg[width-1:0] read_value;
        backdoor_variable = 1'b1;
        reg_write(addr,valid);  //write 1
        if(backdoor_variable == 1'b0) begin
            $display ("[ERROR:]write0c0,addr %0h quote is err
--1 ,backdoor_variable is %0h",addr,backdoor_variable,$time);
        end
        reg_read(addr,read_value);
        if((read_value&valid)=='h0) begin
            $display ("[ERROR:]write0c0,addr %0h quote is err
--2,read_value is %0h",addr,read_value,$time);
        end
        backdoor_variable = 1'b1;
        reg_write(addr,~valid);  //write 0
        if(backdoor_variable !== 1'b0) begin
            $display ("[ERROR:]write0c0,addr %0h quote is err
--3,backdoor_variable is %0h",addr,backdoor_variable,$time);
        end
        reg_read(addr,read_value);
        if((read_value&valid)!=='h0) begin
            $display ("[ERROR:]write0c0,addr %0h quote is err
--4,read_value is %0h",addr,read_value,$time);
        end
    endtask
```

```
endclass
```

上述 stop 输入也可以不使用.

在 sv 中使用方法如下:

```
bus_opr::bus_read("pbus",16'd124*512+i,rdata);
```

Sv 环境打印 C 环境 printf 内容

a) Sv 环境添加内容

```
`ifdef FPGA
    wire[7:0]    msg_reg  =  xxx.uXRAM.inst.memory[10];
`else
    wire[7:0]    msg_reg  =  xxx.uXRAM.mem_array[1008];
`endif
bit [7:0] strobe_msg;
bit clk_msg;
bit rstn_msg;
reg[7:0] char_buff[$];
reg[7:0] char_msg='h0;
initial begin
    forever begin
        clk_msg =1'b0;
        #10;
        clk_msg = 1'b1;
        #10;
    end
end
initial begin
rstn_msg = 1'b0;
```

```
#100;
rstn_msg =1'b1;
end
always @(posedge clk_msg or negedge rstn_msg)begin
    if(!rstn_msg)
       strobe_msg <=8'h0;
    else
       strobe_msg <= msg_reg;
end

wire strobe_msg_edge = strobe_msg[7]&(!msg_reg[7]);
always @(posedge clk_msg or negedge rstn_msg)begin //store char to string
    if(!rstn_msg)
       char_msg <=8'h0;
    else if(strobe_msg_edge) begin
        char_msg <= msg_reg;
        char_buff.push_back(char_msg);
    end
end

always @(posedge clk_msg )begin
    if((char_msg==10)|(char_msg==13))begin   //10 or 13 is enter key
        foreach(char_buff[i])begin
            if(i==char_buff.size()-1)
            $display("%s",char_buff[i]); //print as change line
            else
            $write("%s",char_buff[i]); //print as no change line
        end
        char_buff.delete();
```

```
    end
end
```

b) c 环境修改内容

在 c 环境中需要替换 putchar()函数—keil51 的 printf 函数是调用 putchar 函数

```
#include "absacc.h"
 volatile unsigned char xdata sync_message _at_ 10   ;
char putchar(char c)
{
    sync_message = c&0x7f;
    sync_message  |= 0x8f;
    sync_message &= 0x7f;
}
//因为ascii 只有 0~127，最高位产生跳变来表示需要打印字符
```

方法 2:来源于 m0+测试平台,原理与上面一样

```
  wire [7:0]  chardata = {1'b0, EXTGPIO[14:8]};
  wire        charstrobe = EXTGPIO[15];
  always@(charstrobe)
     if(charstrobe == 1'b1)
        $write ("%c", chardata);
```

Keil5,arm 是修改 fputc()函数---printf 调用的是 fputc 函数

```
fputc;
#define sync_message  (*((unsigned int  *) 0x2003FFD4))
int fputc(int c)
{
    sync_message = c&0x7f;
```

```
    sync_message  |= 0x80;
    sync_message &= 0x7f;
}
```

如果是要接收数据者是需要修改 fgetc 函数.

putc 和 fputc 基本是一样的，他们的原型是 int putc （ int character, FILE * stream ）；

int fputc （ int character, FILE * stream ）;可以看出他们本质是一样的。你所指的终端和文件都是输出对象，由 FILE * stream 这个参数指定。如果是文件就是文件。putchar(int character)等价于 putc(character, stdout). stdout 是标准输出流，一般是屏幕，也就是终端的意思。

Debug 小手段

1．Debug　loop 问题

当遇到 loop 时，在 ucli 环境下，使用 ctrl+c 暂停，然后 run，再 ctrl+c 暂停，然后 run，可以看到当前运行的代码；

如果是在 cli 环境下，使用 ctrl+c 暂停，然后 step，可以单步运行，使用 continue 会连续运行，可以看到当前运行的代码；

以上命令可以暂停后通过 help 命令看到

验证中常用的对比等方法

```
class backdoor_opr #( width = 16);
    task write(output [width-1:0] variable,input[width-1:0] datain);
        variable =  datain;
    endtask
    task read(input [width-1:0] variable,output[width-1:0] dataout);
        dataout = variable ;
    endtask
endclass


class cmp_judge #( width = 8);
```

```
    static task judge(input string flag= "",input[width-1:0]
gold_data,input[width-1:0] cmp_data,input bit norm_judge=1'b1,ref bit
noneed);
        if($bits(cmp_data)!= width)begin
            $display ("[ERROR:] cmp_judge width is no match ",$time);
        end
        if(^cmp_data == 1'bx)begin
            $display ("[ERROR:]  %s ,there are some x  ;GOLD_DATA
is %0h,infact data is %0h",flag,gold_data,cmp_data,$time);
            pass = 1'b0;
            #1000;
            $finish;
        end
        if(norm_judge)begin
            if(gold_data!=cmp_data)begin
                $display ("[ERROR:]  %s ,GOLD_DATA is %0h,infact data
is %0h",flag,gold_data,cmp_data,$time);
                pass = 1'b0;
                #1000;
                $finish;
            end
            else begin
              //  $display (" re_judge , %s,GOLD_DATA is %0h,infact data
is %0h",flag,gold_data,cmp_data,$time);
            end
        end
        else begin
            if(gold_data==cmp_data)begin
                $display ("[ERROR:] re_judge, %s  :GOLD_DATA
is %0h,infact data is %0h",flag,gold_data,cmp_data,$time);
                pass = 1'b0;
                #1000;
                $finish;
            end
            else begin
               // $display ("  %s  :GOLD_DATA is %0h,infact data
is %0h",flag,gold_data,cmp_data,$time);
            end
        end
    endtask
endclass
```

Sdf 文件格式分析

Sdf:Standard Delay File

1. 文件生成方法

Sdf 文件由 sta 工具（pt）根据后端提供网表和 spef 文件生成。
write_sdf -version 3.0 -context verilog file_name.sdf
见”PT/sta 篇/PT 产生 sdf 文件”章节

2. sdf 文件作用

When VCS reads an SDF file it \u201cback-annotates\u201d delay values to the
design, that is, it adds delay values or changes the delay values specified in the source files.
就是用 sdf 文件的延时信息覆盖 specify 设置的延时，timecheck 等信息。

3. 读取 SDF 文件命令

读取 SDF 文件时$sdf_annotate ("sdf_file", [module_instance, "config_file"," log_file", "mtm_spec","scale_factors","scale_type"]);

sdf_file：SDF 文件的绝对或相对路径

module_instance：标注范围。缺省为调用$sdf_annotate 所在的范围

config_file：配置文件的绝对或相对路径。缺省使用预设的设置。

配置文件一般不使用，缺省，其内容如下

```
SDF Configuration File: sdf.cfg
PATHPULSE=IGNORE;
INTERCONNECT_MIPD=MAXIMUM;
MTM=TOOL_CONTROL;
SCALE_FACTORS=100:0:9;
SCALE_TYPE=FROM_MTM;
MTM = TYPICAL;
SCALE_TYPE=FROM_MINIMUM;
SCALE_FACTORS=1.1:2.1:3.1;
```

Log_file：日志文件名，缺省为 sdf.log。可以用+sdf_verbose 选项生成一个日志文件。

Mtm_spec：选择标注的时序值，可以是{MINIMUM，TYPICAL，MAXIMUM，TOOL_CONTROL}之一。缺省为 TOOL_CONTROL(命令行选项)。这个参数覆盖配置文件中 MTM 关键字。可以在 MAKEFILE 中加入 DELAY_OPT=mindelays 或 maxdelays

Scale_factors：min:typ:max 格式的比例因子，缺省为 1.0:1.0:1.0。这个参数覆盖配置文件 SCALE_FACTORS 关键字。

Scale_type：选择比例因子；可以是{FROM_MINIMUM, FROM_TYPICAL, FROM_MAXIMUM, FROM_MTM}之一。缺省为 FROM_MTM。这个参数覆盖配置文件中 SCALE_TYPE 关键字。

除了上述的$sdf_ annotate task 外，还可以在命令行加入选项配置
-sdf min|typ |max:instance_name:file.sdf
如下：Compilation% vcs -sdf min|typ |max:instance_name:file.sdf

3. 文件分析

```
(DELAYFILE
(SDFVERSION "OVI 3.0")
(DESIGN "xxxxe")  //module name
(DATE "Tue Apr 30 08:53:49 2019")
(VENDOR "M31HDSC700HL055PR_125CSS1P08_cworst_ccs SP55MV_PG_V1p2_max
M31USBT200HL055P_125CSS1P08_post spsram5632x32_lib
CBMDROM10KX32_SS_1.08V_125C")
(PROGRAM "Synopsys PrimeTime")
(VERSION "M-2017.06")
(DIVIDER /)
// OPERATING CONDITION "125CSS1P08::125CSS1P08" //工作条件下
(VOLTAGE 1.08::1.08)
(PROCESS "1.000::1.000")
(TEMPERATURE 125.00::125.00)
(TIMESCALE 1ns)
(CELL
  (CELLTYPE "top_module_name")//把最顶层做为一个 cell
  (INSTANCE)//实例化路径，空表示最顶层/当前文件
  (DELAY
    (ABSOLUTE
    (INTERCONNECT bist_clk__L4_I8/O bist_clk__L5_I7/I  (0.012::0.013)
(0.011::0.011))//表示连线的延时
    。。。。。。。。。
```

```
)
。。。。。。。。
(CELL
  (CELLTYPE "NOR3X1AR7")  //cell 名
  (INSTANCE I_cbm2199e_core/I_test_ctrl/U7) //例化名，路径
  (DELAY
    (ABSOLUTE
    (IOPATH I1 O  (0.153::0.166)  (0.065::0.074))
    (IOPATH I2 O  (0.162::0.177)  (0.060::0.068))
    (IOPATH I3 O  (0.184::0.209)  (0.074::0.089))
   )
  )
)
```

上面表示 io 延时，就是 cell 延时，

IOPATH I1 O (0.153::0.166) (0.065::0.074))表示输入端口 I1 到输出端口 O 的延时；

(0.153::0.166) (0.065::0.074))分别表示上升沿延时和下降沿延时

(0.153::0.166)中的 0.153 表示最小延时，0.166 表示最大延时，正常情况下应该是(min:type:max)格式

```
(CELL
  (CELLTYPE "DFFSBQX05AR7")
  (INSTANCE I_cbm2199e_core/I_test_ctrl/ext_cmd_s_reg_1_)
  (DELAY
    (ABSOLUTE
    (IOPATH (posedge CK) Q (0.499::0.539) (0.391::0.422))
//时钟上升沿到输出的延时，其实就是寄存器的输出延时
    (CONDELSE (IOPATH (negedge SB) Q (0.626::0.678) ()))
//除了下面条件外复位到输出的延时，其实就是寄存器的输出延时
    (COND CK==1&&D==0  (IOPATH (negedge SB) Q (0.626::0.678) ()))
    (COND CK==0&&D==0  (IOPATH (negedge SB) Q (0.610::0.659) ()))
    (COND CK==1&&D==1  (IOPATH (negedge SB) Q (0.626::0.678) ()))
    (COND CK==0&&D==1  (IOPATH (negedge SB) Q (0.614::0.663) ()))
   )//在各种条件下复位到输出的延时
  )
  (TIMINGCHECK//时序检查，不满足抱 warning 及输出 x 态
    (WIDTH    (COND adacond4  (negedge CK))  (0.240::0.247))
    (WIDTH    (COND adacond5  (negedge CK))  (0.247::0.254))
    (WIDTH    (COND adacond5  (posedge CK))  (0.232::0.291))
    (WIDTH    (COND adacond4  (posedge CK))  (0.252::0.310))
    (REMOVAL  (posedge SB)    (COND D==0  (posedge CK))
(0.000::0.000))
    (RECOVERY  (posedge SB)    (COND D==0  (posedge CK))
(0.192::0.206))
```

```
        (WIDTH     (COND adacond0   (negedge SB))  (0.533::0.595))
        (WIDTH     (COND adacond1   (negedge SB))  (0.533::0.595))
        (WIDTH     (COND adacond2   (negedge SB))  (0.552::0.612))
        (WIDTH     (COND adacond3   (negedge SB))  (0.552::0.612))
        (SETUP  (posedge D)      (COND SB==1   (posedge CK))
(0.144::0.149))
        (SETUP  (negedge D)      (COND SB==1   (posedge CK))
(0.331::0.343))
        (HOLD  (posedge D)      (COND SB==1   (posedge CK))  (0.000::0.000))
        (HOLD  (negedge D)      (COND SB==1   (posedge CK))  (0.000::0.000))
      )
    )
```

VCD 文件格式分析

1. 打开方式

.vcd 文件可以直接用 gvim 打开,查看文本内容.波形文件
文件里的时间是绝对时间。

2. 文件分析

下面是某个 vcd 文件内容:

```
$timescale
 1ps
$end
//定义时间单位
$scope module vdd12_domain_test $end  //进入 vdd12_domain_tes 层
$scope module dut $end //进入 vdd12_domain_tes/dut 层
$scope module u_vdd12_domain $end
//进入 vdd12_domain_tes/dutu_vdd12_domain 层
$scope module inst_vdd12_pad $end
//进入 vdd12_domain_tes/dutu_vdd12_domain/inst_vdd12_pad 层
$scope module U_PO_0 $end  //进入 inst_vdd12_pad/ U_PO_0 层
$var wire       1 ″    PAD $end //有一个变量叫 PAD,用符号″表示
$upscope $end   //返回上一层即 inst_vdd12_pad 层
//下面类推
$scope module U_PO_1 $end
$var wire       1 #    PAD $end
$upscope $end
// U_PO_1 有一个变量波形,用符号#表示,位宽为 1
$scope module U_PO_2 $end
```

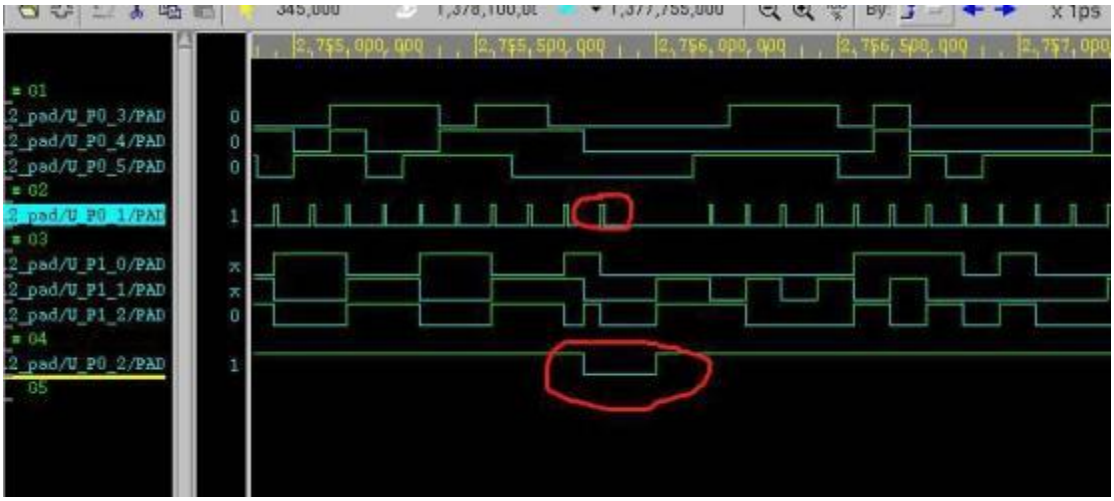
```
$var wire        1 $     PAD $end
$upscope $end

$scope module U_P0_3 $end
$var wire        1 %     PAD $end
$upscope $end

$scope module U_P0_4 $end
$var wire        1 &     PAD $end
$upscope $end

$scope module U_P0_5 $end
$var wire        1 '     PAD $end
$upscope $end

$scope module U_P1_0 $end
$var wire        1 (     PAD $end
$upscope $end

$scope module U_P1_1 $end
$var wire        1 )     PAD $end
$upscope $end

$scope module U_P1_2 $end
$var wire        1 *     PAD $end
$upscope $end//返回上一层即
//vdd12_domain_test/dut/u_vdd12_domain/inst_vdd12_pad
$upscope $end//返回上一层 vdd12_domain_test/dut/u_vdd12_domain
$upscope $end//返回上一层 vdd12_domain_test/dut/
$upscope $end//返回上一层 vdd12_domain_test
$upscope $end//返回上一层,当前层次
$enddefinitions $end//波形符号定义结束了

#0      //0 ps 时刻
$dumpvars  //各个波形的值
1"    // "表示 p00 ,即 P00 在 0ps 时刻为 1
0#
0$
0%
0&
0'
x(
x)
x*  //* 表示 P02,即 P02 在 0ps 时刻为 X
```

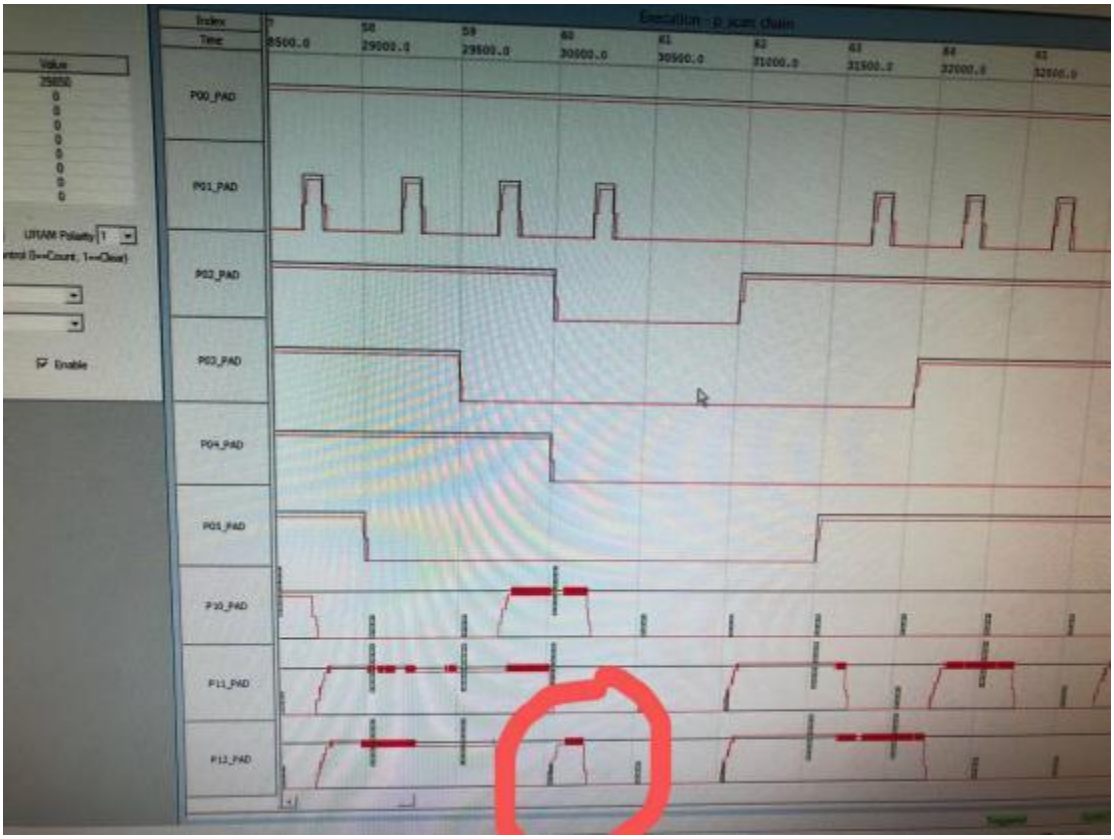
$end
#200000 //表示 200000ps,这些信号至少有一个有变化,下面列出变化信号的值
1$ //$表示 p02,则意味着 p02 此时变成了 1
0* //*表示 p12,则意味着 p12 此时变成了 0
//其它没列出的信号意味保持值不变,下面的含义类推,直到文件结束
#300000
0’
0&
0%
#345000 此时间表示绝对时间
……
#500000
1’
1&
1%

3. Scan 测试注意事项

在机台在做数据对比时,由于需要工具对 vcd 波形转换,所以需要注意如何确定比较点,数据输出都是寄存器输出,由于仿真波形,输出的数据变化是在时钟的上升沿变化(实际上肯定是有延时的),如果转换工具是在每个输出数据变化时比较可能就会有问题.

,



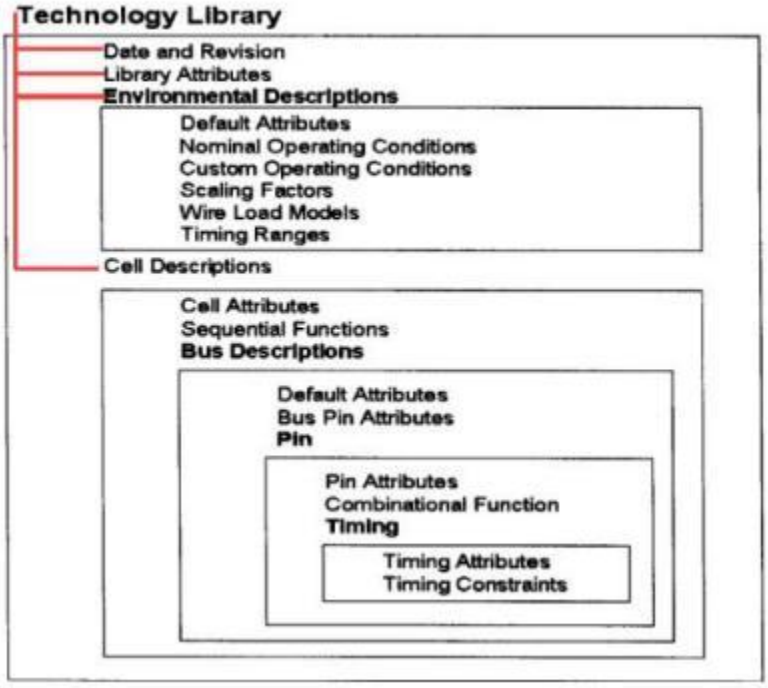


不同的转化工具,其比较点可能也不相同,如上图， 它的比较点不在变化点,而是在时钟的中心位置.理论上应该不会有问题，正常情况下时钟的中心位置是不会变化，数据稳定，是个很好的比较点，但此 pattern 恰好 atpg_se 的变化点在时钟的中心位置，输出会在此时变化，所以不是一个很好的比较点，或者通过手段屏蔽掉此时的比较点（atpg_se 的输出数据不用比较）

LIB 文件格式

1. 文件格式

工艺库的结构如下所示：



2. 用途

可以用来做后仿真，与 db 库对应，db 库用来做综合用，lib 可以通过 dc 工具产生 db 文件。

3. LIB 头说明

下面只做一些基本的说明：

```
    library (hg1llp_6T_ss_1p08v_125c) { //库文件名
      /* general attributes */
      technology ("cmos"); //CMOS 工艺
      delay_model  : table_lookup  ;
      in_place_swap_mode  : match_footprint  ;
      library_features(report_delay_calculation,
report_power_calculation);
      /* documentation attributes */
      revision  : 0.507  ;
      date  : "Wed Nov 04 12:44:07 CST 2015"  ;
      comment  : "Copyright  (c) 2001 - 2015 VeriSilicon Inc. All
Rights Reserved."  ;
      /* unit attributes */单位说明
      time_unit  : "1ns"  ;
```

```
voltage_unit : "1V" ;
current_unit : "1mA" ;
pulling_resistance_unit : "1kohm" ;
leakage_power_unit : "1mW" ;
capacitive_load_unit(1, pf);
/* operation conditions */工作条件说明

power_supply() {
  default_power_rail : VDD ;
  power_rail(VDD, 1.08);
  power_rail(GND, 0.0);
}
nom_process : 1 ;
nom_temperature : 125 ;
nom_voltage : 1.08 ;

operating_conditions(ss_1p08v_125c) {
  process : 1 ;
  temperature : 125 ;
  voltage : 1.08 ;
  tree_type : balanced_tree ;
  power_rail(VDD, 1.08);
  power_rail(GND, 0.0);
}
default_operating_conditions : ss_1p08v_125c ;
/* threshold definitions */阈值定义
slew_lower_threshold_pct_fall : 30 ;
slew_upper_threshold_pct_fall : 70 ;
slew_lower_threshold_pct_rise : 30 ;
slew_upper_threshold_pct_rise : 70 ;
input_threshold_pct_fall : 50 ;
input_threshold_pct_rise : 50 ;
output_threshold_pct_fall : 50 ;
output_threshold_pct_rise : 50 ;
slew_derate_from_library : 0.5 ;
/* default attributes */
default_leakage_power_density : 0 ;
default_cell_leakage_power : 0 ;
default_fanout_load : 1 ;
default_output_pin_cap : 0 ;
default_inout_pin_cap : 9999 ;
default_input_pin_cap : 9999 ;
default_max_transition : 2.64 ;
/* templates */表格格式说明
```

```
        lu_table_template(minpw-active_template_1x3) {
          variable_1 : constrained_pin_transition ; 变量含义
          index_1("1, 2, 3");根据 variable_1 找值
        }

        lu_table_template(setup_template_3x3) {
          variable_1 : constrained_pin_transition ;
          variable_2 : related_pin_transition ;
          index_1("1000, 1001, 1002");
          index_2("1000, 1001, 1002");
        }

        lu_table_template(hold_template_3x3) {
          variable_1 : constrained_pin_transition ;
          variable_2 : related_pin_transition ;
          index_1("1000, 1001, 1002");
          index_2("1000, 1001, 1002");
        }

        lu_table_template(recovery_template_3x3) {
          variable_1 : constrained_pin_transition ;
          variable_2 : related_pin_transition ;
          index_1("1000, 1001, 1002");
          index_2("1000, 1001, 1002");
        }

        lu_table_template(removal_template_3x3) {
          variable_1 : constrained_pin_transition ;
          variable_2 : related_pin_transition ;
          index_1("1000, 1001, 1002");
          index_2("1000, 1001, 1002");
      }
    。。。。。。。。。
```

2. 举例说明

```
cell(INVSIXTPX) {    //cell 名称
    cell_footprint : inv ; 版图名称
    area : 3.2856 ;  //cell 面积

    pin(A) {
      direction : input ;
      capacitance : 0.002928 ;
```

} 输入管脚 ，电容特性为 0.002928

```
    pin(Z) {
      direction : output ;
      capacitance : 0 ; //输出管脚
      function : "(!A)" ; //功能为非门

      internal_power() { //功耗说明
        related_pin : "A" ;

        rise_power(energy_template_6x6) { //A 从 0->1 的瞬态功耗
          index_1("0.02, 0.09216, 0.3454, 0.8291, 1.583, 2.64");
//这个变量为输入转换时间
          index_2("0.001, 0.00413, 0.01512, 0.0361, 0.06879, 0.1147");
//这个变量是输出负载
          values("0.003084, 0.003163, 0.0032, 0.003225, 0.003209,
0.003248",\
                 "0.002905, 0.003003, 0.003102, 0.00317, 0.00322,
0.00326",\
                 "0.002809, 0.002839, 0.002969, 0.003077, 0.003139,
0.003146",\
                 "0.002822, 0.00284, 0.00284, 0.002946, 0.003032,
0.003097",\
                 "0.002934, 0.00289, 0.002905, 0.002846, 0.002936,
0.002961",\
                 "0.003121, 0.003033, 0.002946, 0.002875, 0.002802,
0.002924");
        }
//上表为上升是的功耗，根据 index_1（行号）, index_2（列号）查找上表，
如：A 的转换时间为 0.3454 则行号对应为 3，Z 输出的负载为 0.0361,则对应的
列好为 4，则此时可以知道这个变化过程功耗为 3 行 4 列的数值（0.003077）；
上面所有的单位，index 代表的含义在 lib 的开始会指明。
        fall_power(energy_template_6x6) { //A 从 1->0 的瞬态功耗
          ……
        }
      }

      timing() { //时序说明
        related_pin : "A" ;
        timing_sense : negative_unate ;

        cell_rise(delay_template_6x6) {
          index_1("0.02, 0.09216, 0.3454, 0.8291, 1.583, 2.64");
          index_2("0.001, 0.00413, 0.01512, 0.0361, 0.06879, 0.1147");
```

```
          values("0.06388, 0.1062, 0.2497, 0.5207, 0.9428, 1.538",\
                 "0.09521, 0.138, 0.2821, 0.5541, 0.9773, 1.57",\
                 "0.195, 0.2546, 0.4023, 0.6765, 1.098, 1.694",\
                 "0.3171, 0.422, 0.6301, 0.9061, 1.33, 1.922",\
                 "0.4481, 0.6086, 0.9231, 1.267, 1.693, 2.291",\
                 "0.5845, 0.8058, 1.245, 1.717, 2.204, 2.802");
        }
       方法与功耗查找方法相同
        rise_transition(delay_template_6x6) {
           ……
        }

        cell_fall(delay_template_6x6) {
           ……
        }

        fall_transition(delay_template_6x6) {
        ……
        }
      }
      max_capacitance : 0.1147 ;
    }
    cell_leakage_power : 5.987e-08 ;

    leakage_power() {  2 种状态下的漏电流（静态电流）
      when : "!A" ; A 等于 0 时的电流
      value : "5.41402384e-08" ;
    }

    leakage_power() {
      when : "A" ; A 等于 1 时的电流
      value : "5.98652351e-08" ;
    }
}
```

Specify 用法（转）

一般都是在例化的库单元出现。

specify block 用来描述从源点（source：input/inout port）到终点（destination：output/inout port）的路径延时（path delay），由 specify 开始，到 endspecify 结束，并且只能在模块内部声明，具有精确性（accuracy）和模块性（modularity）的特点。specify block 可以用来执行以下三个任务：

一、描述横穿整个模块的各种路径及其延时。（module path delay）
二、脉冲过滤限制。（pulse filtering limit）
三、时序检查。（timing check）
specify block 有一个专用的关键字 specparam 用来进行参数声明，用法和 parameter 一样，不同点是两者的作用域不同：specparam 只能在 specify block 内部声明及使用，而 parameter 只能在 specify block 外部声明及使用。
//
////////

第一个任务：模块路径延时（module path delay）

一条模块路径可以是一条简单的路径（simple path），或者是一条边缘敏感的路径（edge sensitive path），或者是一条状态依赖的路径（state dependent path）。
//---

都是指明 input->output 的 io 延时

一、simple path，可以由以下两种格式中的任意一种来声明：
1)、并行连接（patallel connection）：source => destination
2)、全连接（full connection）：　　source *> destination
例：　　(a, b => q, qn) = 1; 等价于：
　　(a => q) = 1; (b => qn) = 1;　　-- 一 一对应
而(a, b *> q, qn) = 1; 等价于：
　　(a => q) = 1; (b => q) = 1; (a => qn) = 1; (b => qn) = 1;

//---

二、edge sensitive path，是那些源点（source）使用边沿触发的路径，并使用边缘标示符指明触发条件（posedge/negedge），如果没有指明的话，那么就是任何变化都会触发终点（destination）的变化。
例 1：(posedge clk => (out +: in)) = (1,2);
在 clk 的上升沿，从 clk 到 out 的模块路径，其上升延时是 1，下降延时是 2，从 in 到 out 的数据路径是同向传输，即 out = in。
例 2：(negedge clk => (out -: in)) = (1,2);
在 clk 的下降沿，从 clk 到 out 的模块路径，其上升延时是 1，下降延时是 2，从 in 到 out 的数据路径是反向传输，即 out = ~in。
例 3：(clk => (out : in)) = (1,2);
clk 的任何变化，从 clk 到 out 的模块路径，其上升延时是 1，下降延时是 2，从 in 到 out 的数据路径的传输是不可预知的，同向或者反向或者不变。

Note：模块路径的极性（module path polarity）：未知极性（unknown polarity，无），正极性（positive polarity，+），负极性（negative polarity，-）。

//--

三、state dependent path，是那些源点（source）以来指定条件状态的路径，使用 if 语句（不带 else）。在条件=1 or X or Z 的情况下，认为条件成立。如果有一条路经，存在多个条件同时成立的情况，那么使用延时最小值的那条限制。

例 1：

```
specify
    if(a)     (b => out) = (1,2);
    if(~a)    (b => out) = (2,3);
    if(b)     (a => out) = (1,2);
    if(~b)    (a => out) = (2,3);  //上升沿：2，下降沿：3
endspecify
```

例 2：

```
specify
    if(rst)   (posedge clk => (q +: data)) = (1,2);
    if(~rst)  (posedge clk => (q +: data)) = (2,3);
endspecify
```

需要注意的是，所有输入状态（每个信号都有为 0，为 1 两种状态）都应该说明，否则没有说明的路径使用分布延时（distributed delay），如果也没有声明分布延时（distributed delay）的话，那么使用零延时（zero delay）。如果路径延时和分布延时同时声明的话，则选择最大的延时作为路径延时。另外，也可以使用 ifnone 语句，在其它所有条件都不满足的情况下，说明一个缺省的状态依赖路径延时。

例 3：

```
specify
    (posedge clk => (q +: data)) = (1,2);
    ifnone (clk => q) = (2,3);
endspecify
```

//

第二个任务，脉冲过滤限制（pulse filtering limit）

由于每条传播路径都具有一定的电容性和电阻性，电荷无法在一瞬间积累或消散，所以信号变化的物理特性是具有惯性的。为了更准确地描述这种能力，使用惯性延时（inertial delay），它可以抑制持续信号比传播延时短的输入信号的变化。

例 1：1ns 宽度的窄脉宽通过一个传输延时为 2ns 的 BUFFER

两个脉宽限制值：e-limit（error limit）和 r-limit（rejection limit），并且要求 e-limit >= r-limit，否则报错。当 pulse width >= e-limit 时，输出相应

的逻辑值；当 e-limit > pulse width >= r-limit 时，输出 X 值；当 r-limit > pulse width 时，输出不发生变化。默认情况下，e-limit = r-limit = module transition delay，也可以使用以下3 种控制方式中的任意一种改变路径脉冲限制值：

1、使用 veri log 提供的 PATHPULSE$参数，有些仿真器还要求同时使能相应的选项：比如 VCS，添加+pathpulse 选项。

```
PATHPULSE$ = (<reject-limit>, <error-limit>);
PATHPULSE$<path_source>$<path_destination> = (<reject-limit>, <error-limit>);
```

例：

```
specify
  (en => q) = 12;
  (data => q) = 10;
  (clr, pre *> q) = 4;
  specparam
  PATHPULSE$ = 3, PATHPULSE$en$q = (2,9), PATHPULSE$clr$q = 1;
endspecify
```

en => q 的路径：reject-limit = 2, error-limit =9；

clr => q 和 pre => q 的路径：reject-limit = error-limit = 1；

data => q 的路径：reject-limit = error-limit = 3；

2、使用仿真器专用的编译指导。比如 VCS，+pulse_r/20（取 20%）和+pulse_e/80（取80%），需要注意的是，这个选项要求放在读入 RTL 网表文件之后，否则设置无效。即网表仿真才有效

3、使用SDF 文件反标，并且 SDF 文件中的延时信息具有最高的优先级。SDF 文件格式将在后面介绍。

脉冲过滤限制的默认格式存在两个缺点：

1、X 状态的持续时间比较短。

2、在上升延时和下降延时不相等的情况下，如果脉冲过窄，那么可能出现跟随边缘（trailing edge）先于或等于导引边缘（ leading edge）的现象，这时就会淹没 X 状态。

可以通过修改默认格式，加以改善，具体如下：

on-event vs on-detect:

negative width pulse detection:

需要注意的是，showcancelled list_path_of_outputs，必须在模块路径之前使用，才可以约束到该模块路径。

//
////////

第三个任务，时序检查（timing check）

描述设计要求的时序性能，所有的时序检查有一个参考事件（reference event）和一个数据事件（data event），它们通过一个布尔表达式相联接，还包括一个可选的 notifier 寄存器选项，这个寄存器用来打印错误信息或者传播 X 状态(如果设置为 0，则不会打印错误和传播 x 态)。

这里把时序检查分成两组来说明：

第一组，检查时序窗口的稳定性，包括：setup、hold、recovery 和 removal。

setup：$setup (data_event, reference_event, limit, notifier)；

当 reference_event time - limit < data_event time < reference_event time 时，就会报告 setup time violations。

hold： $hold (reference_event, data_event, limit, notifier)；

当 reference_event time < data_event time < reference_event time + limit 时，就会报告 hold time violations。

setup/hold：$setuphold (reference_event, data_event, setup_limit, hold_limit, notifier)；

$setuphold 是$setup 和$hold 两者的联合。例如：

$setuphold (posedge clk, negedge d, 2, 1, notifier)；等于

$setup (negedge d, posedge clk, 2, notifier)；和 $hold (posedge clk, negedge d, 1, notifier)；

数据事件常常是数据信号，而参考事件常常是时钟信号，如下图：

recovery：$recovery (reference_event, data_event, limit, notifier)；

当 data_event time - limit < reference_event time < data_event time 时，就会报告 recovery time violations。

removal： $removal (reference_event, data_event, limit, notifier)；

当 data_event time < reference_event time < data_event time + limit 时，就会报告 removal time violations。

recovery/removal：$recrem (reference_event, data_event, recovery_limit, removal_limit, notifier)；

$recrem 是$recovery 和$removal 两者的联合。

$recrem (posedge clr, posedge clk, 2, 3, notifier)；等于

$recovery (posedge clr, posedge clk, 2, notifier)；和 $removal (posedge clr, posedge clk, 3, notifier)；

数据事件常常是时钟信号，而参考事件常常是控制信号， 比如清除信号或者置位信号，如下图：

$setuphold 和$recrem 可以接受负值，同时需要激活仿真器的负值时序检查选项（比如 VCS：+neg_tchk），同时还有一个限制：

setup_ limit + hold_ limit > 仿真精度（simulation unit of precision），

recovery_ limit + removal_ limit > 仿真精度（simulation unit of precision），否则仿真器会把负值当成 0 处理。

所以 timescale 设置为 1ns/1ps 比较方便

第二组，检查时钟和控制信号在指定事件之间的时间间隔，包括：skew、width、period 和 nochange。

skew：$skew （reference_event, data_event, limit, notifier）； 限制最大偏斜

$skew （posedge clk1, posedge clk2, 1, notifier）；

当 data_event time - reference_event > limit，则会报告 skew time violations。$skew 是基于事件（event-based）的，如果监测到一个 reference_event，那么就开始评估脉宽，只要监测到一个data_event，就会生成相应的报告，直到监测到下一个 reference_event，才重新开始新的监测。如果在监测到一个 data_event 之前，又监测到一个 reference_event，那么就放弃本次评估，重新开始新的评估。

width：$width （controlled_reference_event, limit, threshold, notifier）； 限制最小脉宽

$width （posedge in, 2, notifier）；

这里 data_event 是隐含的，它等于 reference_event 的相反边缘，当 width < limit 时，就会报告 width time violations。

period：$period （controlled_reference_event, limit, notifier）； 限制最小周期

$period （negedge clk, 10, notifier）；

这里 data_event 是隐含的，它等于 reference_event 的相同边缘，当 period < limit 时，就会报告 period time violations。

nochange：$nochange （reference_event, data_event, start_edge_offset, end_edge_offset, notifier）；

当 leading reference event time - start_edge_offset < data_event < trailing reference event time + end_edge_offset 时，就会报告 nochange time violations。例如：

$nochange （posedge clk, data, 0 , 0）；

当在 clk 高电平期间，data 发生任何变化，就会报告 nochange time violations。

有时候，路径上的时序检查是在一定条件成立的前提下进行的，这就需要引入条件

操作符：&&&。需要注意的是，当存在两个及以上的条件时，要求这些条件首先在 specify 块外部经过适当的组合逻辑产生一个新的控制信号，然后再引入到 specify 块内部使用。

例如：

```
and u1 (clr_and_set, clr, set);
specify
  $setup (negedge data, posedge clk &&& clr_and_set, 3, notifier);
endspecify
```

//

SDF 文件简述：

SDF 文件包含指定路径延时（specify path delay），参数值（specparam values），时序检查约束（timing check constraints），互连线延时（interconnect delay），以及一些和仿真不相关的说明信息。反标 SDF 文件的过程，也算是更新 specify block 相对应信息的过程，如果 SDF 文件没有包含某些信息，则参考 specify block 中的相应信息。

SDF 时序信息在 CELL 内部描述，可以包含一个或多个 DELAY、TIMINGCHECK 和 LABEL。DELAY 部分包含指定路径的传播延时（specify path delay）和互连线延时（interconnect delay）；TIMINGCHECK 部分包含时序检查约束信息（timing check constraint）；LABEL 部分包含新的参数值（specparam）。

DELAY 部分：

例 1：SDF 文件：(IOPATH in out (1.1::1.3) (1.5::1.7));

verilog specify path: (in => out) = (2, 3);

例 2：SDF 文件：(COND en==1'b1 (IOPATH in out (1.2) (1.6));

verilog specify path: if (en) (in => out) = (1, 2);

例 3：互连线延时：(INTERCONNECT source_port load_port delay_values)

SDF 文件：(INTERCONNECT u1/out u2/in (1.2::1.4) (1.4::1.6));

TIMINGCHECK 部分：

例 4：SDF 文件：(SETUP (posedge data) (posedge clk) (3::4));

(HOLD (posedge data) (posedge clk) (1::2));

verilog timing checks: $setup (posedge data, posedge clk, 1);

$hold (posedge clk, posedge data, 2);

例 5： SDF 文件： (SETUP (posedge data) (COND rb==1'b1 (posedge clk)) (3::4));

(HOLD (posedge data) (COND rb==1'b1 (posedge clk)) (1::2));

verilog timing checks： $setup (posedge data, posedge clk &&& rb, 1);

$hold (posedge clk &&& rb, posedge data, 2);

LABEL 部分：

例 6：

SDF 文件：(LABEL

(ABSOLUTE

(dh 60)

(dl 40)))

verilog 文件：specparam dh = 60, dl=40;

SDF 文件反标是一个有序的过程，这就意味着对于同一对 source/load，后续的信息可能修改（INCREMENT）或者覆盖（ABSOLUTE）前面已经声明过的信息，这在反标（$sdf_annotate）多个 SDF 文件时，就很可能发生的。

例 7：覆盖前面的延时信息

(DELAY

(ABSOLUTE

(IOPATH A Z (1) (2))

(IOPATH A Z (2) (3))))实际中 setuphold 可能有合并的情况，如下

```
specify
        (posedge CK => (Q+:D)) = 0;
        if (CK == 1 && D == 1)
            (negedge RB => (Q+:1'b0)) = 0;
        if (CK == 1 && D == 0)
            (negedge RB => (Q+:1'b0)) = 0;
        if (CK == 0 && D == 1)
            (negedge RB => (Q+:1'b0)) = 0;
        if (CK == 0 && D == 0)
            (negedge RB => (Q+:1'b0)) = 0;
        ifnone (negedge RB => (Q+:1'b0)) = 0;
        $setuphold (posedge CK &&& RB == 1, posedge D &&& RB == 1, 0, 0,
notifier,,, delayed_CK, delayed_D);
        $setuphold (posedge CK &&& RB == 1, negedge D &&& RB == 1, 0, 0,
notifier,,, delayed_CK, delayed_D);
        $recrem (posedge RB &&& D == 1, posedge CK &&& D == 1, 0, 0,
notifier,,, delayed_RB, delayed_CK);
        $width (posedge CK &&& adacond4, 0, 0, notifier);
        $width (negedge CK &&& adacond4, 0, 0, notifier);
        $width (posedge CK &&& adacond5, 0, 0, notifier);
        $width (negedge CK &&& adacond5, 0, 0, notifier);
```

```
    $width  (negedge RB &&& adacond2, 0, 0, notifier);
    $width  (negedge RB &&& adacond3, 0, 0, notifier);
    $width  (negedge RB &&& adacond1, 0, 0, notifier);
    $width  (negedge RB &&& adacond0, 0, 0, notifier);
endspecify
```

XA 混仿

混仿可以用来观察上电情况,非常有用

4. XA 混仿四要素

模拟网表

包含模拟工艺库，模拟基本单元，模拟激励，一些控制option。

数字 module

所有的.v module。

接口控制文件

xa 命令以及数模接口转换说明。

VCS 脚本

Vcs 的编译运行命令，指定与xa 的接口控制文件。

1.1模拟网表

Pwl 指上电形式：从 0v 到 1.8v 需要 1us

.probe 命令

也可以使用.print 命令

1.2接口控制文件

```
choose xa -hspice ./spice.sp -o xa_wave/xa -c xa_cmd -wavefmt fsdb;

d2a lov=0v hiv=1.8v rise_time=1ns fall_time=1ns node=test.uSeqGen.Clkin;
d2a lov=0v hiv=1.8v rise_time=1ns fall_time=1ns node=test.uSeqGen.Reset;
a2d hith=1.26v  loth=0.54v  node=test.uSeqGen.Clkout ;
a2d hith=1.26v  loth=0.54v  node=test.uSeqGen.xinv.Clkout1 ;
```

use_spice -cell PBCSUDSL8_UFLASH_P55;//如果 xx.sp 有很多 cell，则在此指定 cell

choose xa -hspice ./x.sp -o xa_wav -c xa_cmd -wavefmt fsdb//会生成xa_wav.log 和xa_wav.fsdb波形,如果没有-o选项，则默认xa.log,xa.fsdb

d2a lov=0v hiv=1.8v rise_time=1ns fall_time=1ns node=tb.u.clk;

a2d hith=1.26 loth=0.54 node test.u.clkout;

d2a rf_time=0.1n hiv=1.08 lov=0.0 node=sim_top.rom.AD*; //xa_cmd 使用 set_wildcard_rule -match* all 语句时，对总线比较好用

bus_format <%d>; //当有 bus 总线时，需要制定模拟 subket bus 格式

指定模拟部分的仿真工具为xa，模拟的网表文件为 spice.sp，输出波形为 fsdb 格式。

约定数模接口以及高低电压，上升时间，下降时间。

```
1 use_spice -cell gpio_ana_pad_5vp15_20181205b;
2 choose xa -n ./tran.sp -o gpio_ana_pad_5vp15 -c xa.cmd -wavefmt fsdb;
3
4 d2a lov=0v hiv=1.8v rise_time=1ns fall_time=1ns node=sim_top.uPad_0.ST;
5 d2a lov=0v hiv=1.8v rise_time=1ns fall_time=1ns node=sim_top.uPad_0.I;
6 d2a lov=0v hiv=1.8v rise_time=1ns fall_time=1ns node=sim_top.uPad_0.OEN;
7 d2a lov=0v hiv=1.8v rise_time=1ns fall_time=1ns node=sim_top.uPad_0.PU;
8 d2a lov=0v hiv=1.8v rise_time=1ns fall_time=1ns node=sim_top.uPad_0.OD_EN;
9 d2a lov=0v hiv=1.8v rise_time=1ns fall_time=1ns node=sim_top.uPad_0.LDRV*;
10 d2a lov=0v hiv=1.8v rise_time=1ns fall_time=1ns node=sim_top.uPad_0.PUR_SEL*;
11 d2a lov=0v hiv=1.8v rise_time=1ns fall_time=1ns node=sim_top.uPad_0.MODE*;
12 bus_format <%d>;
13
```

1.3 xa_cmd 内容

```
1 set_dc_option -report fail
2 set_sim_level 3
3 set_message_limit 10000
4 set_oscillator -subckt OSC_TOP
5 set_sim_level 4 -subckt VR_OSC_TOP
6 probe_waveform_voltage *.* -limit 2
```

set_dc_option -report fail

set_sim_level 3

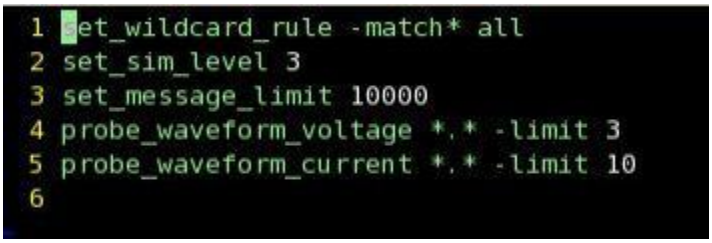
set_wildcard_rule -match* all //用于对总线的匹配

set_message_limit 10000

set_oscilator -subckt osc_top

set_sim_level 4 subckt vr_osc_top

probe_waveform_voltage *.* -limit 2



1.

2.设定仿真等级

3.设定信息限制

4.设定子模块

5.设定子模块的仿真等级

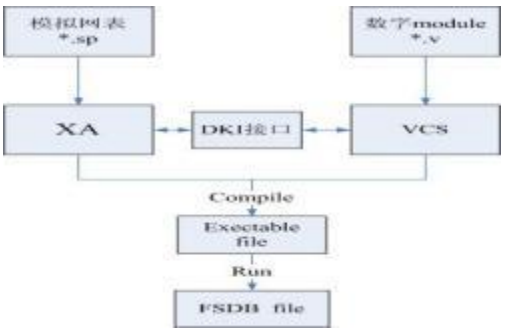
6.设定观察电压probe 的层次为 2 层

1.4 VCS 脚本

```
VCS -f filelist.v \
  -full64 -sverilog -timescale=1ns/1ps \
  -lca -Mupdate +vpdports +v2k -Mdir=xaCsrc \
  -o xaVcs -ad=xaVcsMSV.ctrl -l xaVcsLog/xaVcsElab.log \
  xaVcs -l xaVcsLog/xaVcsRun.log
```

注：运行脚本中约定与 XA 配合的 option 为 -ad=xaVcsMSV.ctrl（此为接口控制文件）

5. XA 混合仿真流程



XA 与 VCS 配合编译网表文件以及数字module

运行生成的可执行文件

可以通过Verdi 打开 Fsdb 文件，观察波形

脚本如下：

```
vcs  -lca -R -sv -sverilog   -debug_all +v2k  +define+sim
+lint=TFIPC-L -timescale=1ns/1ps   +plusarg_save -Mupdate -I  -P
/ic01/eda/novas/verdi201101/share/PLI/VCS/LINUX/novas.tab
/ic01/eda/novas/verdi201101/share/PLI/VCS/LINUX/pli.a
+ad=vcsAD.init -f cmd_files   -l compile.log
```

cmd_files 内容如下：

```
sim_top.sv   //测试 tb top
-v ./cbm_io_model.v    //数字模型，只要有端口，内容可以为空
```

Finesim 混仿

1. finesim 混仿要素

1.1.sp 文件　--由模拟方面提供

1.2 Rtl/netlist 文件

2. Ncverilog 运行命令

ncverilog +ncaccess+rwc +loadvpi=finesim.so:finesim_startup +nc64bit +ncntcnotchks +nowarnCUVWSP +nowarnAGSOPRO +ncprofile +loadpli1=debpli:novas_pli_boot +nctimescale+1ns/1ps -f cmd_files -l sim.log

说明：

+loadvpi=finesim.so:finesim_startup:nc 与 finesim 接口，仿真过程中遇到模拟模块则调用 finesim 工具

-f cmd_files :加载 RTL 或 netlist,含 testbench 及相关 model

3. testbench 需要增加的点

下面都是 finesim 工具的配置及连接关系

```
initial begin
    $finesim_config(,".resistance r_model 1 3k 4k 5k 50k 70k 90k 1g");
    $finesim_config(,".a2d v15_a2d vl=0.15 vh=1.35"); //指定数字模拟接口的电压阈值
    $finesim_config(,".d2a v15_d2a vl=0 vh=1.5 vx=0.75 tr=0.3n tf=0.3n r=r_model");

    $finesim_config(,".output sim_top.uAna_Ldo.VTEST xldo.VTEST          d2a=v15_d2a");
    $finesim_config(,".output sim_top.uAna_Ldo.VBG_TRIM[1] xldo.VBG_TRIM<1>    d2a=v15_d2a");
    $finesim_config(,".output sim_top.uAna_Ldo.VBG_SEL xldo.VBG_SEL        d2a=v15_d2a");
    $finesim_config(,".output sim_top.uAna_Ldo.TURBO_V15 xldo.TURBO_V15      d2a=v15_d2a");
```

$finesim_config(,".input sim_top.uAna_Ldo.CMP_OUT1 x1do.CMP_OUT1 a2d=v15_a2d");

分析

$finesim_config(,".input sim_top.uAna_Ldo.CMP_OUT2 // uAna_Ldo ,testbench 中例化 ldo 的名称

x1do.CMP_OUT2 a2d=v15_a2d"); // x1do 为.sp 文件中例化 ldo 的名称

//用于指定数字和模拟的接口

end

initial begin

$finesim_module("reg_1p5_5vp15_20170622");

End

4. Sp 文件格式

```
.options post node list probe brief
.prot
.lib '/ana /lib/gf013lp.l' tt
.lib '/ana /lib/gf013lp.l' tt_hv
.lib '/ana /lib/gf013lp.l' res_tt
.lib '/ana /lib/gf013lp.l' cap_tt
.lib '/ana/lib/gf013lp.l' bip_tt
.lib '/ana/lib/gf013lp.l' dio_tt  //模拟的基本单元库
.option post
.option finesim_mode=fast
.option finesim_tflush=10u   //各种选项配置

*.option SIM_ACCURACY=100   //配置精度

*.param vd12=vd
*.param vd33=`vd*5/1.5`//配置参数  *用来屏蔽

.inc "/ana /netlist/fnet/reg_1p5_5vp15/netlist"   //模拟模块网表
x1  //例化名称
+ CMP_OUT1 CMP_OUT2 VREF_1V VREF_ADC AVS ENREG
```

+I1UA I1UB I1UC I1UD I1UE I1UF I1UG ICOP5UA ICOP5UB IC1UA IC4UA VBG VDD15 VDD50
+VSS VTEST ENADCBUF ENTRIMBUF1 ENTRIMBUF2 ENVREFTEST LDO_M_V15<1> LDO_M_V15<0>
+VREF_TRIM<4> VREF_TRIM<3> VREF_TRIM<2> VREF_TRIM<1> VREF_TRIM<0>
+ reg_1p5_5vp15_20170622 //模拟模块名称
//接口列表
.inc "/ana /netlist/fnet/saradc_10bit_5vp15/netlist" //模拟模块网表
x2 //例化名称
+ DOUT<9> DOUT<8> DOUT<7> DOUT<6> DOUT<5>
+DOUT<4> DOUT<3> DOUT<2> DOUT<1> DOUT<0> AVS IB_ADC VDD VDD50 VREF VSS
+ saradc_10bit_5vp15_20160823//模拟模块名称

VDD50 VDD50 0 5 //给 VDD50 施加 5v 电源激励

vss vss 0 0 //给 VDD50 施加 5v 电源激励
VAVS AVS 0 0 //给 VDD50 施加 5v 电源激励

CVDD15 VDD15 0 1000p //给 VDD15 接一个 1000p 的电容

*VREF VREF 0 pwl(0 0 10n 5)
IB_ADC VDD50 IB_ADC 1u

rload VDD15 0 50 //给 VDD15 接一个 50 欧的负载

VANA_INPUT0 ANA_INPUT0 0 1.0 //给 ANA_INPUT0 接一个 1v 的电压，注意电压前需要字符”v”
.probe v(x1.*)
.probe v(*)
.probe v(x1.*.*)
.probe v(x2.*) //dump 时的内容

Finesim 用户手册介绍的很详细。

5. 注意点

a.只能有一个 sp 文件，通过$finesim_config (," .finesim -o saradc_10bit_5vp15_20160823 ./sp/saradc_10bit_5vp15/tran.sp");来关联例化与模拟网表之间的连接
b. 多个模拟网表则把它们合并到一个 sp 文件中，如下
.inc "/ project_path
/project/circuit/CBM7316/netlist/fnet/ldo/netlist"
x1
+ CMP_OUT1 CMP_OUT2 VREF_1V VREF_ADC AVS ENREG

+I1UA I1UB I1UC I1UD I1UE I1UF I1UG ICOP5UA ICOP5UB IC1UA IC4UA VBG VDD15 VDD50
+VSS VTEST ENADCBUF ENTRIMBUF1 ENTRIMBUF2 ENVREFTEST LDO_M_V15<1> LDO_M_V15<0>
+LDO_S_V15<1> LDO_S_V15<0> TURBO_V15 V27_V15 VBG_POR VBG_SEL VBG_TRIM<2>
+VBG_TRIM<1> VBG_TRIM<0> VO1_TRIM<4> VO1_TRIM<3> VO1_TRIM<2> VO1_TRIM<1>
+VO1_TRIM<0> VO2_TRIM<4> VO2_TRIM<3> VO2_TRIM<2> VO2_TRIM<1> VO2_TRIM<0>
+VREF_TRIM<4> VREF_TRIM<3> VREF_TRIM<2> VREF_TRIM<1> VREF_TRIM<0>
+ reg_1p5_5vp15_20170622

.inc "/project_path /netlist/fnet/saradc/netlist"
x2
+ DOUT<9> DOUT<8> DOUT<7> DOUT<6> DOUT<5>
+DOUT<4> DOUT<3> DOUT<2> DOUT<1> DOUT<0> AVS IB_ADC VDD VDD50 VREF VSS
+ANA_INPUT0 ANA_INPUT1 ANA_INPUT2 ANA_INPUT3 ANA_INPUT4 ANA_INPUT5 ANA_INPUT6
+ANA_INPUT7 ANA_INPUT_SEL<9> ANA_INPUT_SEL<8> ANA_INPUT_SEL<7> ANA_INPUT_SEL<6>
+ANA_INPUT_SEL<5> ANA_INPUT_SEL<4> ANA_INPUT_SEL<3> ANA_INPUT_SEL<2>
+ANA_INPUT_SEL<1> ANA_INPUT_SEL<0> CLK I_TRIM PD RST SH_EN VREFS VREF_ADC
+ saradc_10bit_5vp15_20160823

c. 虽然合并到一个 sp 文件，但在 tb 需要有各自的 finesim_module

```
initial begin
  $finesim_module("reg_1p5_5vp15_20170622");
End
```

d. dump 波形时，$finesim_config (,".finesim -o reg_1p5_5vp15_20170622__./sp/ldo/tran.sp"); 自动会生成 reg_1p5_5vp15_20170622.fsdb 文件，其 dump 波形的内容在 sp 中通过 probe 指定，如下

```
.tran 10p 1000000u    //精度及仿真结束时间
.probe v(x1.*)
.probe v(*)
.probe v(x1.*.*)
.probe  v(x2.*) //dump 波形的信号
```

e. 当两个模拟模块模拟信号要连接时，可以通过数字定义 wire 变量把他们连接起来，然后在配置中通过 a2d,d2a 来实现

```
$finesim_config (,".inout sim_top.uAna_ldo.VDD15
x1.VDD15     d2a=v15_d2a");
      $finesim_config (,".inout sim_top.uAdc.VDD15
x2.VDD       a2d=v15_a2d");
```

其中 uAna_ldo.VDD15 和.uAdc.VDD15 在 sim_top 中通过一个 wire vdd15 连接起来

f. 不支持总线方式，需要按照下面方法书写

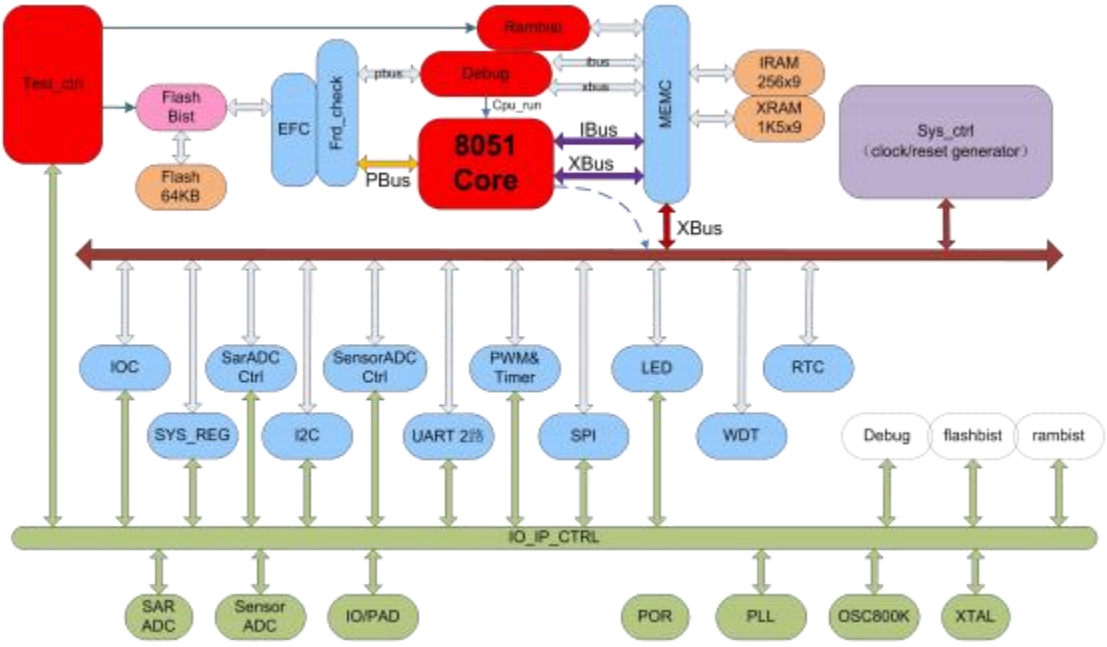
```
saradc_10bit_5vp15_20160823 uAdc(
        .VDD50          (                   ),
        .VDD15          (    vdd15          ),
       .VSS             (                   ),
       .DOUT            ( {
                            adc_out_o[4],
                           adc_out_o[3],
                           adc_out_o[2],
                           adc_out_o[1],
                           adc_out_o[0]
                           })
    );
```

这是因为在 fine_config 中是单线连接方式，adc_out_o[4]，adc_out_o[3]代表是 2 根 wire；

g. 例化的模拟模型最好使用空模型，这样就不会引起一些干扰—主要是防止 verilog 模型写的时序不对而导致一些异常

常见 soc 系统架构

1. C8051 架构

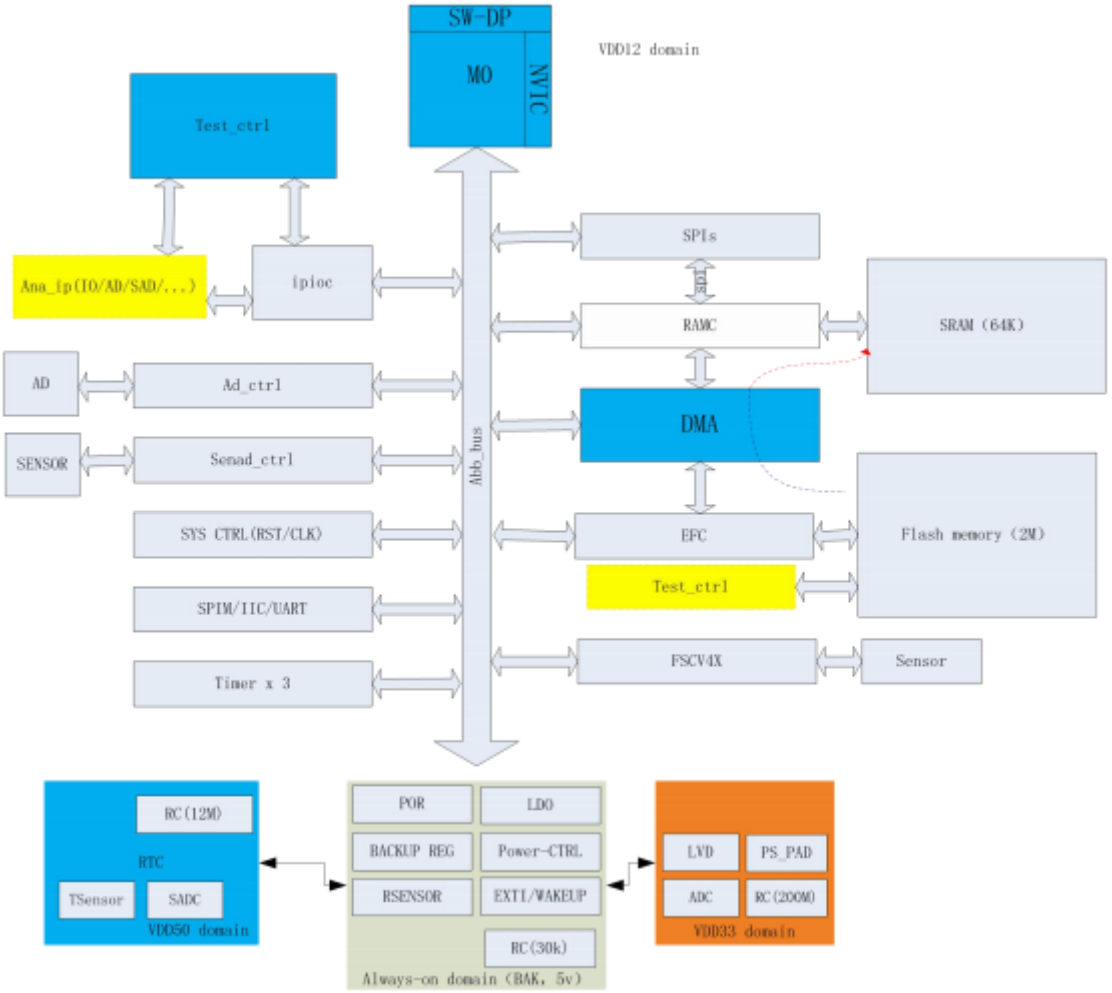


说明：

1. 上图构架一个主 host（8051），三个副 host(debug,test_ctrl)；

8051 核主要用于在用户模式下执行程序；
debug 是在调试模式控制 8051 运行，但 debug 时的一些读写功能（通过memory 窗口操作）是通过自己的 xbus,ibus，pbus 总线控制对应的控制单元；
memory bist 主要用于在测试模式下测试memory，它只要被使能，内部自动产生 memory 端口的时序，产生各种测试花样；
test_ctrl 用于测试模式测试所有的模拟 ip,其中 flash，memory 有专门的测试模块，test_ctrl 只要按要求提供控制时序即可，其他 的普通模拟 ip，则需要直接控制模拟端口，产生时序，测试模拟 ip 测试是否正常。
2. 上图中的架构与正常的 8051 系统有些不同，主要是外设的寄存器全部挂在 xbus 总线上，而且中间通过 ramc 做了mux.
3. flash 包含在 flash bist 中，bist 内部做了测试模式于用户模式的 mux
4. 上图中的 ip_io_ctrl 就是专门来控制普通模拟模块（不像 adc 那样需要时序的模拟模块），它主要是根据模式来 mux 控制源
5. 注意一点的是在控制时钟 osc 等模块时，使能位需要使用时钟的下降沿处理，而不能是上升沿

2. Arm 架构



说明：

上图中的上半部分为常见的 arm 架构，下半部分为低功耗设计部分（不分析）。

其中 arm m0 为主 host，通过 ahb_lite（ahb 简化版，m0 核常用此总线）总线控制所有 slave;并具有调试功能。
Test_ctrl 用于在测试模式测试所有的模拟 ip;
DMA 在此系统中用于根据系统配置把 flash 的数据搬移到 sram 中，其过程为控制 efc 读取 flash，再控制 ramc 把数据写入 sram 中

除了总线不一样，8051 与 arm m0 的架构差不多。

3. 系统在线升级考虑

---触发升级条件 1),硬件管脚触发 2).通过串口(或其它接口)接收到特殊指令触发
思路如下:
对于 8051 核:
升级程序存放在 flash 最后一个 page;当触发了升级,则调用升级程序,此时程序在最后 1page 运行,可以开始擦除 flash->接收新 code 数据,写入 flash,循环这个过程,效验程序没有问题,则触发软复位,进行新程序运行.
--注意,在擦除编程 flash 时,code 总线是 hold 住的,系统不会操作 flash.
还一种方法就是 2 片 flash 乒乓操作,轮流当着升级区域,设置时注意地址影射.
对于 M0 核:
对于 m0 系列,可以把升级程序搬移到 xram,然后程序空间重影射到 xram—M0 核需要硬件做相应处理(增加一个系统寄存器来配置重影射功能),M0+以上的核因为有中断向量重影射,可以直接影射过去.

4. Cpu 操作存储器概念(转)

计算机的存储系统采用 Register，Cache，Memory 和 I/O 的方式来构成存储系统，无疑是一个性能和经济性的妥协的产物。Cache 和 Memory机制是计算机硬件的基础内容，这里就不再啰嗦。下面重点说明 Write-back，Write-through 及 write allocate 这三种操作的区别。

一、CPU 读 Cache

1. Read through，即直接从内存中读取数据；
2. Read allocate，先把数据读取到 Cache 中，再从 Cache 中读数据。

二、CPU 写 Cache

1. 若 hit 命中，有两种处理方式：

Write-through: write is done synchronously both to the cache and to the backing store。Write-through（直写模式）在数据更新时，把数据同时写入 Cache 和后端存储。此模式的优点是操作简单；缺点是因为数据修改需要同时写入存储，数据写入速度较慢。

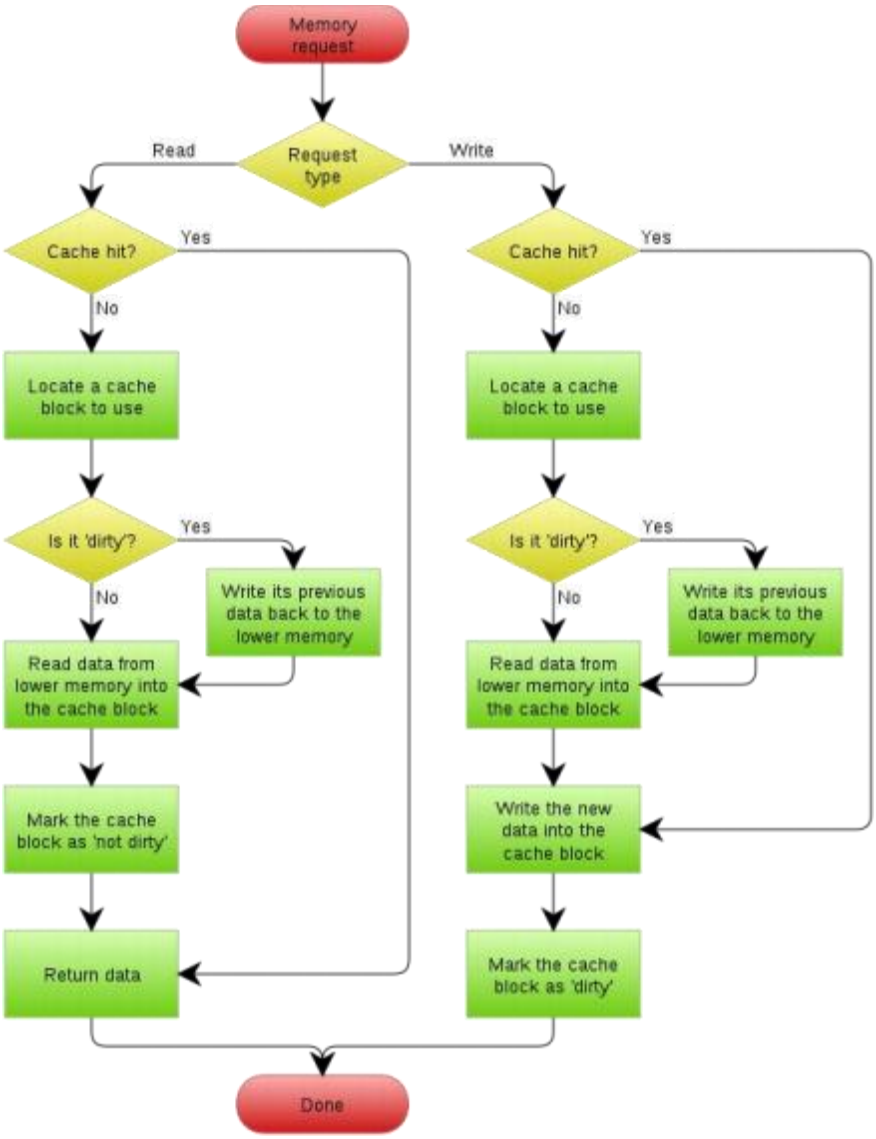
Write-back (also called *write-behind*): initially, writing is done only to the cache. The write to the backing store is postponed until the cache blocks containing the data are about to be modified/replaced by new

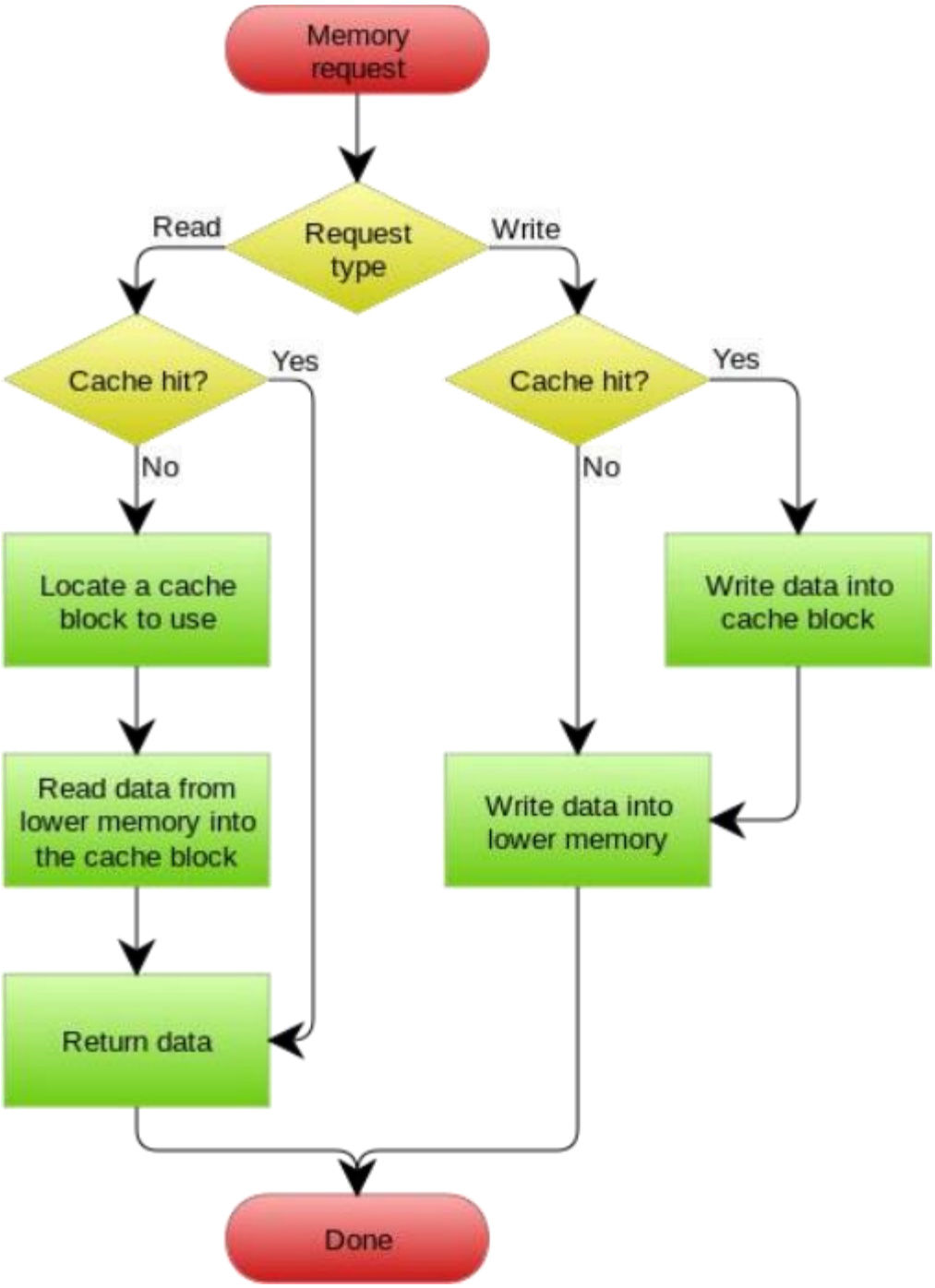
content。Write-back（回写模式）在数据更新时只写入缓存 Cache。只在数据被替换出缓存时，被修改的缓存数据才会被写到后端存储（即先把数据写到 Cache 中，再通过 flush 方式写入到内存中）。此模式的优点是数据写入速度快，因为不需要写存储；缺点是一旦更新后的数据未被写入存储时出现系统掉电的情况，数据将无法找回。

2. 若 miss，有两种处理方式：

Writeallocate (also called *fetch on write*): data at the missed-write location is loaded to cache, followed by a write-hit operation. In this approach, write misses are similar to read misses.。Write allocate：先把要写的数据载入到Cache 中，写 Cache，然后再通过 flush 方式写入到内存中； 写缺失操作与读缺失操作类似。

No-writeallocate (also called *write-no-allocate* or *writearound*): data at the missed-write location is not loaded to cache, and is written directly to the backing store. In this approach, only the reads are being cached。No write allocate：并不将写入位置读入缓存，直接把要写的数据写入到内存中。这种方式下，只有读操作会被缓存。


Memory request
Request type
Read
Write
Cache hit?
Yes
No
Locate a cache block to use
Is it 'dirty'?
Yes
No
Write its previous data back to the lower memory
Read data from lower memory into the cache block
Mark the cache block as 'not dirty'
Return data
Cache hit?
Yes
No
Locate a cache block to use
Is it 'dirty'?
Yes
No
Write its previous data back to the lower memory
Read data from lower memory into the cache block
Write the new data into the cache block
Mark the cache block as 'dirty'
Done



注、本文两幅图也来自wiki。

计算机的 Cache 和 Memory 访问时 Write-back，Write-through 及 write allocate 的区别。
原文地址：http://www.cnblogs.com/guojingdeyuan/p/7626983.html

C8051 知识

1. 基础知识

1. IDLE 是 CPU 停止工作，周边模块工作正常，中断唤醒 ；
2. PD 是 CPU 和周边模块全部停止工作，其唤醒实际是对 PD 寄存器复位;所以 idle 能停系统时钟，但不能停总时钟,模块时钟根据需求决定是否停止;

PD 可以停总时钟;

不同的 8051 设计对退出 pd 的方式不完全一样,如下

在 idle 时,sysclk 会停止,但 fclk 不会停,所以各个中断可以退出idle.

在 pd 时,会关闭所有时钟,所以只能通过复位来改变寄存器

```
assign next_pcon[0] = s_intpre ? 1'b0 :
                          s_regs_wr_en[2] & ~s_regs_wr_en[1] &
(s_adr==8'h87) & ~s_atri_sp_pre & (pseq[1:0]==2'b01) ?
s_data[0] : pcon[0];
assign next_pcon[1] = s_regs_wr_en[2] & ~s_regs_wr_en[1] &
(s_adr==8'h87) & ~s_atri_sp_pre & (pseq[3:2]==2'b01) ?
s_data[1] : pcon[1];

always@(posedge fclk or negedge resetn)//系统复位
begin
    if(!resetn) begin
        pcon[0]     <=  1'b0;
        pseq[1:0]   <=  2'b0;
    end
    else begin
        pcon[0]     <=  next_pcon[0];
        pseq[1:0]   <=  next_pseq[1:0];
    end
```

```
    end
    always@(posedge fclk or negedge resetn_pd)//专用复位
    //resetn_pd= resetn & pd 专用复位
    begin
        if(!resetn_pd) begin
            pcon[1]      <=  1'b0;
            pseq[3:2]    <=  2'b0;
        end
        else begin
            pcon[1]      <=  next_pcon[1];
            pseq[3:2]    <=  next_pseq[3:2];
        end
end
```

3. 对于 C 程序中的定义的 CODE 区常量，它们是在编译时产生，还是在启动时产生？？是在编译时产生，当做代码写入 code 区

4. 对于 C 程序中的定义的非 CODE 区常量，它们是在编译时产生，还是在启动时产生？？编译时，把内容编译到 CODE 区，但它们是在启动时产生写 XRAM 的动作，在进入 MAIN 函数之前会有 XBUS 或 IBUS 动作，把数据写入相应的数据区，简单说就是启动时从 code 读出要写的内容写到 RAM 去

5. 在很多 8051 核设计中，0x00~0x1f（r0~r7 四个区），0x20~0x2f（位寻址区）用内部寄存器实现，其它空间用RAM 实现；

6. 也有些时只把四个区的 R0,R1(因为这两个最常用)，0x20~0x2f（位寻址区）用内部寄存器实现 其它空间用RAM 实现

7. 程序的0 地址放的都是跳转指令（0x2），后 2 字节放跳转地址

8. 短调子函数 SP<-SP+1; (SP)<-PCL;SP<-SP+1; (SP)<-PCH;
   PC[10:0]<-addr11;

9. 长调子函数 SP<-SP+1; (SP)<-PCL;SP<-SP+1; (SP)<-PCH;
   PC[10:0]<-addr16;

10. 从子函数返回程序（子函数的最后一条指令）

PCH<-(SP),SP<-SP-1;PCL<-(SP),SP<-SP-1;

11. 从中断函数返回程序（中断函数的最后一条指令）

PCH<-(SP),SP<-SP-1;PCL<-(SP),SP<-SP-1;

12. Sp 是一个向上增长的行为，即 sp 指针始终指向栈顶，压栈 sp 加 1，出栈 sp 减 1。

13. 中断

外部中断 0 向量:0x03;

定时器中断 0 向量:0x0b:

外部中断 1 向量:0x13;

定时器中断 1 向量:0x1b:

串口中断向量:0x23

以上地址可以在KEIL 中设置偏移量，见下图红圈

注意:这个只是编译器有这个功能,如果硬件没这个寄存器,则修改此处程序在进入中断时会出错,需要硬件支持,在程序启动时配置对应寄存器修改中断偏移.

14. 对于 setb,CLR,SHL,SHR 等对寄存器的操作都是先读取，在修改，再写三个步骤；与我们自己用C 操作的方法一致，不同的是，专用指令只要取指一次，接着产生 3 个动作，如果自己写则每个动作需要取指一次，会多几次取指译码的过程。

15. &a,表示变量 a 的地址，*p,指针变量 p 所指向的变量，其中 p 是指针变量名，即 p 才是变量，而不是*p,*p 是一个动作或一种关系（指向），整体可以作为一个普通的变量；*p 代表一个 int 类型的变量，p 为一个指针变量（指针变量只能存储地址）； int a,int tp;则可以如下使用 tp=*p1; *p1=tp;*p1=*p2;p=&a; st1 *p;则 p 为结构体指针，存储

的是结构体的起始地址； 访问结构体成员的方法有 3 种方法：

1.st1.a;2.p.a;3.p->a;

16. 只能使用 p=&st1 获得指针地址

10. 在 C 中规定数组名代表数组的首地址，int *p;时，可以使用 p=a;或 p=&a;*(p+1)为 a[1]的值；

#define base_adress 0x3000

int *pwm =(int*) base_adress（与 int *pwm; pwm = (int*) base_adress; 含义相同）;把 pwm 指向 base_adress 基地址；使用时可以

pwm[0]=,pwm[1]= …等方法（应该也可以采用 *(pwm),*(pwm+)的方法来做 ）；考虑为什么不可以 pwm = base_adress;因为 pwm 是指针变量，只能用指针赋值 ！

理解#define IBUS8(addr) (*((unsigned char volatile data *) (addr)))：(unsigned char volatile data *) (addr)这段相当于一个指针变量 pwm，(*((unsigned char volatile data *) (addr)))则相当于 *pwm,可以认为是一个变量，可以对它进行读和写；

11. 一个函数在编译时被分配给一个入口地址，这个入口地址就称为函数的指针 int(*p) ()表示 p 是一个指向函数的指针变量

int max();//函数 max

int (*p) ();//函数指针变量 p

p=max;//把函数max 的入口地址给指针变量 p

12. 8051 进中断前会进行现场保护，把 PC 的内容压入堆栈，堆栈的地址由 SP 指定，PC 指针指向中断向量，退出时会把堆栈的内容弹出给 PC

13. 8051 在 LCALL 或 ACALL 子函数时，会把 PC 内出压入堆栈，堆栈的地址由 SP 指定，当子程序结束时会出栈把堆栈的内容返回给 PC;

子函数会通过下列几种方法传递参数：1.累加器和工作寄存器

2.操作数地址传递参数：子程序中所需操作数存放在数据存储 RAM 中，调用子程序之前的入口参数为 R0，R1 或 DPTR 间接指出的地址；出口参数（操作结果）仍由 R0，R1 或 DPTR 间接指出的地址。一般内部 RAM 由 R0，R1 作地址指针，外部 RAM 由 DPTR 作地址指针。

3.堆栈传递参数：在调用子程序前，先把参与运算的操作数压入堆栈。转入子程序后，可按堆栈指针 SP 间接访问堆栈中的操作数，同时又可以把运算结果推入堆栈中，返回主程序后，可用POP 指令获取运算结果。注意：转子程序时，主程序的断点地址也要压入堆栈，占用堆栈两个字节，弹出参数时要用两条 DEC SP 指令修改 SP 指针，以便使 SP 指向操作数。另外在子程序返回指令 RET 之前要加两条 INC SP 指令，以便使 SP 指向断点地址，保证能正确返回主程序。【简单说就是进入子程序前先把参数压入堆栈，再把主程序断点地址压入堆栈，子程序用参数时，先把堆栈地址-2，指向参数地址指针；退出时，把 SP+2，把主程序断点地址弹出，】

现场保护：在转入子程序时，特别是进入中断服务子程序时，要特别注意现场保护，即主程序使用的内部 RAM，各工作寄存器内容，累加器 A 内容和 DPTR 以及 PSW 等寄存器内容，都不因转子程序而改变。如果子程序所使用的寄存器与主程序使用的寄存器有冲突，则在转入子程序后首先要采取现场保护措施。方法是将要保护的单元推入堆栈，而空出的这些单元供子程序使用。返回主程序之前要弹出到原工作单元，恢复主程序原来的状态。

14. 51 单片机的存储空间分为三部分

程序存储区　此区域通过 PBUS 总线访问，对应汇编指令它能通过如下方式寻址：

立即数寻址：MOV A ,#3OH;(编译后#3OH 放在程序存储区)

基址寄存器加变址寄存器：MOVC A,@A+DPTR;

相对寻址:JNZ 23H;(累加器 A 不等于 0，跳到 PC+23H 的地址执行)

对应 C 语言的变量可以如下设置：

```
#define PBUS8(addr)       (*((unsigned char volatile code *) (addr)))
```

内部低 128byteRAM 区　此区域通过 IBUS 访问，对应汇编指令它能通过如下方式寻址：直接寻址：MOV A,3OH;

寄存器寻址: MOV A, R7;

寄存器间接寻址: MOV @RO，A;

位寻址（只能是 2OH~2FH 地址）;

对应 C 语言的变量可以如下设置：

#define IBUS8(addr)　　(*((unsigned char volatile data *) (addr)))

unsigned char volatile data DBGFLAG　_at_ 10　; DBGFLAG 可以在程序中直接引用。

SFR 寄存器区

此区域通过 SFR 总线访问，对应汇编指令只能通过

直接寻址的方式访问:MOV A,80H;

位寻址（SFR 中 12 个能被 8 整除的字节地址）：

对应 C 语言的变量可以如下设置：

sfr MPUCR　　= 0xF8;MPUCR 在程序中可以直接引用

如果是访问 IRAM 的高 128byte 则通过间接寻址 ，用 IRAM 总线访问

外部 RAM 存储区　　此区域通过 XBUS 访问，对应汇编指令它只能

寄存器间接寻址:MOVX A ,@DPTR;

对应 C 语言的变量可以如下设置：

#define XREG8(addr)　　(*((unsigned char volatile xdata *) (addr))),

#define MESHERR3　XBYTE[0x1C29], MESHERR3 可以在程序中直接引用

unsigned char volatile xdata DBGFLAG　_at_ 4095　;; DBGFLAG 可以在程序中直接引用。

如果把变量声明为 pdata,汇编为 MOVX @Ri

如果把变量声明为 xdata,汇编为 MOVX @DPTR

6.5. 关于 C51 编译器的声明标识符

C51 编译器的声明识别符与 **MA82G5A64** 存储空间的对应关系如下：

data

128 字节的内部数据存储空间 (00h~7Fh)。使用除 MOVX 和 MOVC 以外的指令，可以直接或间接的访问。全部或部分的堆栈可能保存在此区域中。

idata

间接数据。256 字节的内部数据存储空间 (00h~FFh) 使用除 MOVX 和 MOVC 以外的指令间接访问。全部或部分的堆栈可能保存在此区域中。此区域包括 data 区 和 data 区以上的 128 字节。

sfr

特殊功能寄存器。CPU 寄存器和外围部件控制/状态寄存器，只能通过直接地址访问。

xdata

外部数据或片上的扩展 RAM (XRAM)：通过"MOVX @DPTR"指令访问标准 80C51 的 64K 存储空间。**MA82G5A64** 有 **5120** 字节的片上 xdata 存储空间。

pdata

分页的外部数据(256 字节) 或片上的扩展 RAM（XRAM）：重叠的 256 字节的存储器地址通过"MOVX @Ri"指令访问。**MA82G5A64** 有 256 字节片上 pdata 存储器它与片上 xdata 存储器共享。

code

64K K 程序存储空间。通过"MOVC @A+DTPR"访问，作为程序部分被读取。**MA82G5A64** 有 **64K** 字节的片上程序存储器。

IDATA 区（52 系统中的内部 256byte RAM,高 128byte 与 SFR 地址重叠） 此区域通过 IBUS 访问，对应汇编指令它只能通过间接寻址 访问，对应 C 语言的变量可以如下设置：

```
#define IBUS8(addr)     (*((unsigned char volatile idata *) (addr)))
```

15. 当使用C 与汇编交互时，它们需要传递参数关系如下：

	Parameter Type			
Parameter #	char, 1 byte ptr	int, 2 byte ptr	long, float	generic ptr
1	R7	R6, R7	R4...R7	R1, R2, R3
2	R5	R4, R5	R4...R7	R1, R2, R3
3	R3	R2, R3	N/A	R1, R2, R3

16. 说明：当汇编需要用 C 函数传来的参数，根据参数类型与个数会选择不同的 Rx 来传递参数，如果参数为 char,则第一个参数通过 R7 传递，第二个参数通过 R5 来传递，第三个参数通过 R3 来传递，剩余的参数将在默认的存储器段中进行。

当函数返回值时，按照下面的存储单元来返回

Return Type	Register(s) Used
bit	carry flag
(unsigned) char	R7
(unsigned) int	R6...R7
(unsigned) long	R4...R7
float	R4...R7
pointer	R1...R3

17. 在 C 程序中插入汇编需要用#pragma asm #pragma endasm 把汇编程序包含起来,同时需要对插有汇编的该文件右键进行设置，如下图

```
void main()
{
ma1();
ma2();
ma3();
ma4();
#pragma asm
MOV DPTR,#08000H
#pragma endasm

}
```

18. 在 Options for Target 菜单的 BL51 Locate 选项卡的 Code: 中输入:?PR?myTest1?MAIN(0x3900), ?PR?myTest?MAIN(0x4000)，把函数 myTest1 定位在程序区的 0x3900 处，把函数 myTest 定义在程序区的 0x4000 处，重新编译就可以了.在 M51 文件中可以找到下面的内容:

19. KEIL51 编译后的结果在 lst/.m51 文件中，包含了 CODE，变量所放的地址 ?PR?开头的表示程序；CODE 001CH 0007H UNIT ?PR?CLOSEDSEND60?SCD 表示文件名为 SCD 里面的 CLOSEDSEND60 的函数放在 CODE 0x1c 开始的地方

?CO?开头的表示定义在 CODE 区的常数；CODE 0026H 0515H UNIT ?CO?SCDTEST 表示文件名为 SCDTEST 里面的常量放在 CODE 的 0x26 开始的地方

还有一些是启动代码的汇编程序的位置

20. KEIL51 启动代码放在 STARTUP.A51 文件中;如果采用了 bank 技术，将会用 L51_BANK.lst 代替 A51 文件

21. 写 C 程序时,源文件最好分三个文件夹,一个为 include,放头文件,一个为 drive 放各模块的驱动,一个为 main,放主程序，KEIL 中 C 程序不区分大小写，

22. struct *p; struct t ;引用是 p->a;t.a;
struct student {

```
 long int num;
 char name[20];
 char sex;
 float score;
}

struct student stu_1 ;
struct student *p ;
p=&stu_1;
```

使用方法：stu_1.name 与 p->name 等价，也可以使用(*p).name;

24. Keil C51 中变量和函数的绝对地址定位问题:

1. 变量绝对地址定位

1) 在定义变量时使用 _at_ 关键字加上地址就可.

e.g. unsigned char idata myvar _at_ 0x40;

把变量 myvar 定义在 idata 的 0x40 处, 在 M51 文件中可以找到这麼一行

IDATA 0040H 0001H ABSOLUTE

表示有变量在 idata 的 0x0040 处绝对地址定位.

2) 使用 KeilC 编译器定义绝对地址的变量, 方法待查.

2. 函数绝对地址定位

1) 在程序中编写一函数 myTest

```
void myTest(void)
{
    // Add your code here
}
```

2) 使用 KeilC 编译器定位绝对地址的函数, 打开 Project -> Options for Target 菜单,

选中 BL51 Locate 选项卡, 在 Code: 中输入: ?PR?myTest?MAIN(0x4000)

把函数 myTest 定位到程序区的 0x4000 处, 再次编译就可以了.

3)　一次定位多个函数的方法

同样地, 在程序中再编写另外一个函数 myTest1

```
void myTest1(void)
{
   // Add your code here
}
```

在 Options for Target 菜单的 BL51 Locate 选项卡的 Code: 中输入:

?PR?myTest1?MAIN(0x3900), ?PR?myTest?MAIN(0x4000)

把函数 myTest1 定位在程序区的 0x3900 处, 把函数 myTest 定义在程 序区的 0x4000 处,

重新编译就可以了.

在 M51 文件中可以找到下面的内容:

```
>> 3.obj TO Reader RAMSIZE (256) CODE (?PR?MYTEST1?MAIN
(0X3900), ?PR?MYTEST?MAIN (0X4000))
                    3665H     029BH                  *** GAP ***
            CODE    3900H     0014H
UNIT          ?PR?MYTEST1?MAIN
                    3914H     06ECH                  *** GAP ***
            CODE    4000H     0014H
UNIT          ?PR?MYTEST?MAIN
```

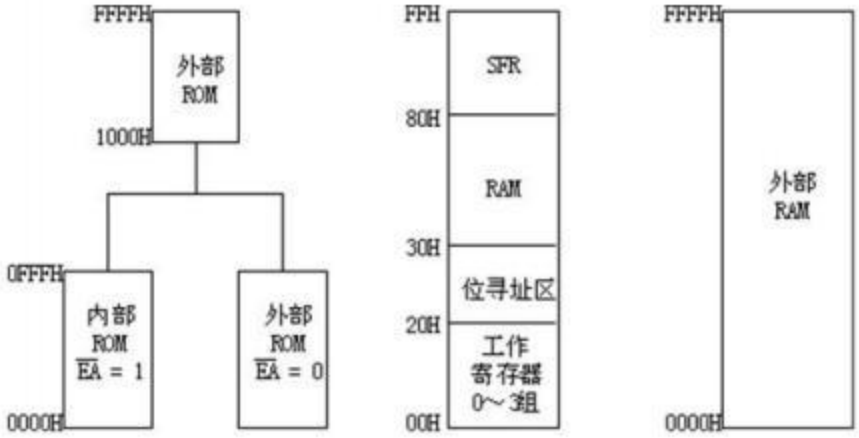
2．Cpu 寄存器

占用了 sfr 寄存器地址

0F8H									0FFH
0F0H	B 00000000								0F7H
0E8H									0EFH
0E0H	ACC 00000000								0E7H
0D8H									0DFH
0D0H	PSW 00000000								0D7H
0C8H	T2CON 00000000	T2MOD XXXXXX00	RCAP2L 00000000	RCAP2H 00000000	TL2 00000000	TH2 00000000			0CFH
0C0H									0C7H
0B8H	IP XX000000								0BFH
0B0H	P3 11111111								0B7H
0A8H	IE 0X000000								0AFH
0A0H	P2 11111111		AUXR1 XXXXXXX0				WDTRST XXXXXXXX		0A7H
98H	SCON 00000000	SBUF XXXXXXXX							9FH
90H	P1 11111111								97H
88H	TCON 00000000	TMOD 00000000	TL0 00000000	TL1 00000000	TH0 00000000	TH1 00000000	AUXR XXX00XX0		8FH
80H	P0 11111111	SP 00000111	DP0L 00000000	DP0H 00000000	DP1L 00000000	DP1H 00000000		PCON 0XXX0000	87H

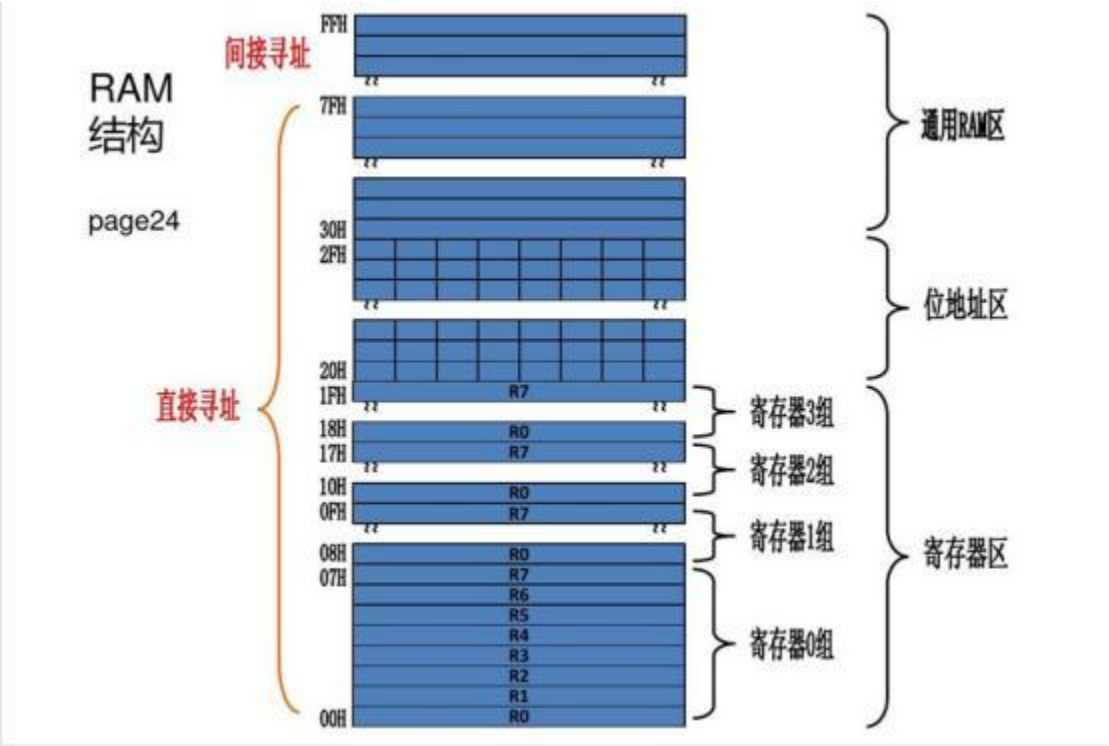
3． Ram 空间分布

（1）MCS-51单片机的存储器名称及容量划分



程序存储器(ROM或EPROM)　内部数据存储器(RAM)　外部数据存储器

图2 MCS-51单片机存储器名称及容量划分



4． 中断系统

标准的 51 系统有 5 个中断（52 系统为 6 个中断），分别为：

外部中断 0（中断向量 0x03），

定时器中断 0（中断向量 0x0b），

外部中断 1（中断向量 0x13），

定时器中断 1（中断向量 0x1b），

串口中断（中断向量 0x23），

定时器中断 2（中断向量 0x2b）---52 系列;

实际的 soc 产品一般都在这几个中断上进行扩展。

中断响应是在取指阶段发生，也就是一定会完成当前指令才响应中断。

在中断程序退出时 （执行RETI指令），不响应中断；如果退出后是在操作 IE,IP 寄存器，这时也不会响应中断。

中断执行过程：

2．发生中断

3. 硬件完成保存当前 pc 的动作,并根据中断，把对应的中断向量送给 pc 寄存器

4. 根据 pc，程序跳转到指定位置

5. 执行保护现场动作---由编译器根据程序会在用户中断函数前插入一段汇编来保存 cpu 的寄存器及用户用到的变量内容。

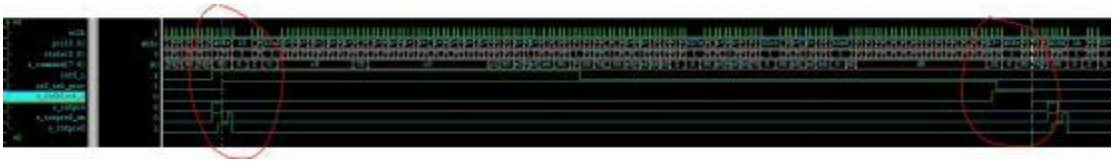
6. 执行用户 c 程序

7. 执行返回现场动作—同样由编译器根据程序会在用户中断函数后插入一段汇编来恢复 cpu 的寄存器及用户用到的变量内容。

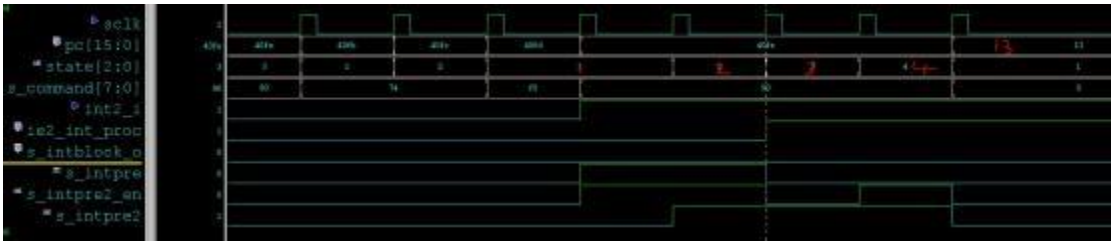
8. 执行第 2 步 pc 的内容

中断信号的要求： 外部中断信号（指过滤后到达 8051 核的信号）需要大于 2 个系统时钟宽度。---或许与 8051 核的设计相关；

下图是一个进入中断和退出中断的波形



进入中断波形：



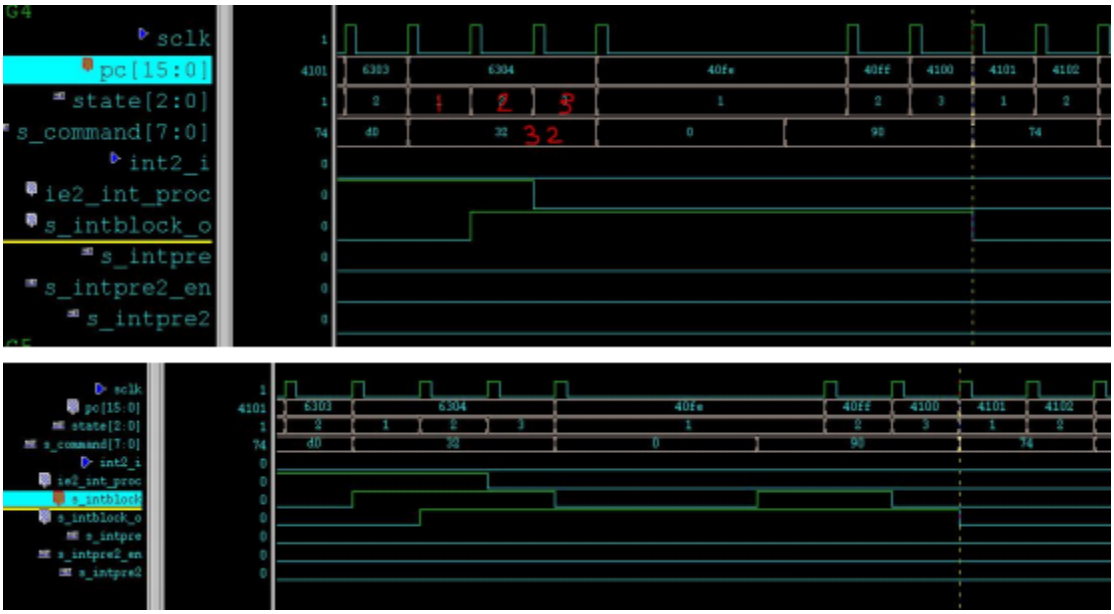
信号说明：

int2_i 是同步过的中断信号，即中断信号源

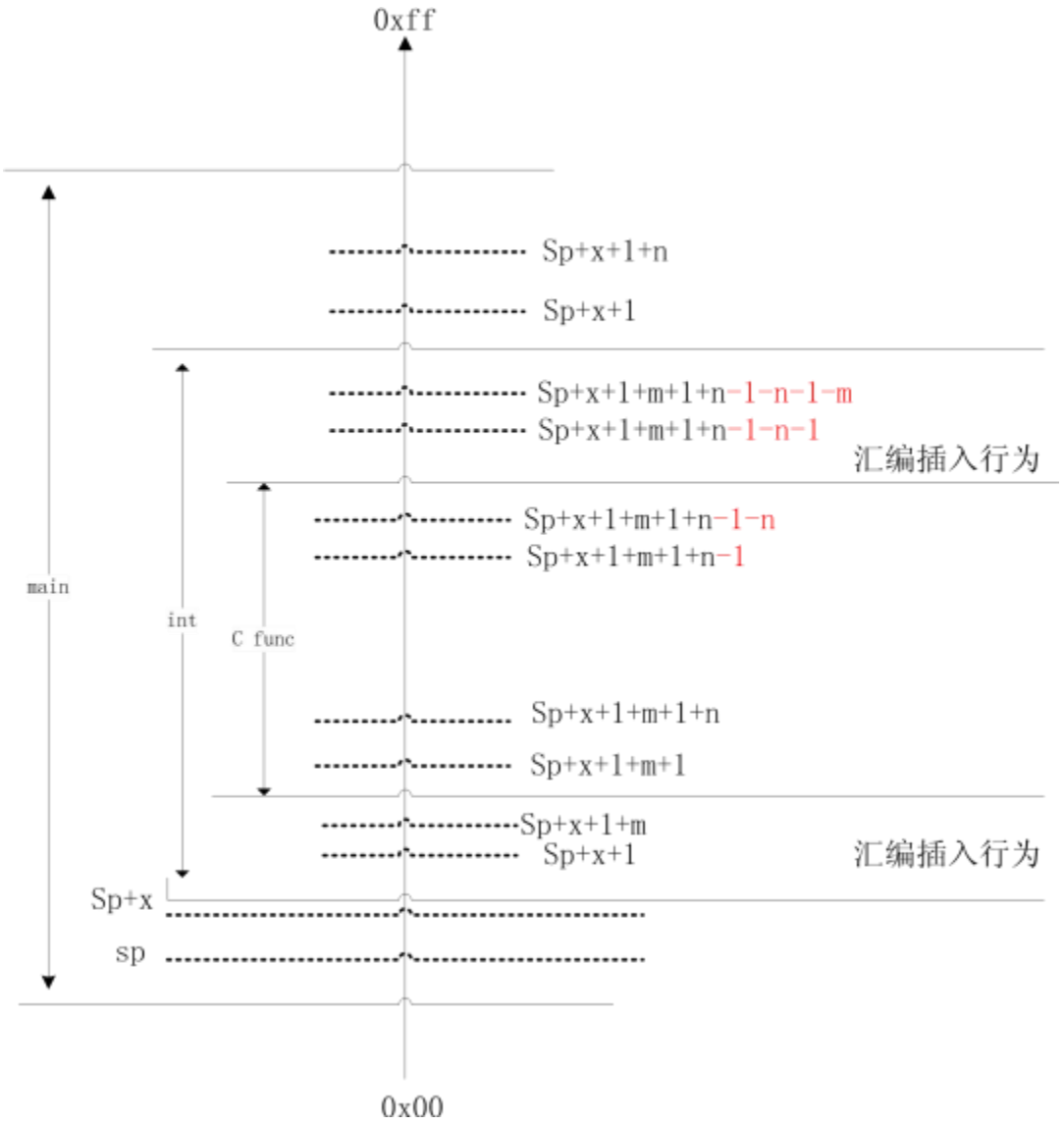
s_intPre2 是经过优先级等判断后的中断信号

ie2_int_proc 是真正指示中断程序过程，后面如果没有优先级更高的中断，则要执行完这个中断程序，pc 等于 13 是在经过当前行为 1，2，3，4 阶段后才进入的中断向量。

退出中断波形



当执行到最后 RETI(上图中 s_command==0x32)，ie2_int_proc 撤消，也 Sp 及变量变化过程如下图：



过程分析： main 程序先使用了一些堆栈 x 个，然后进入中断，堆栈增长，保护现场，用了堆栈 m 个，进入用户程序，使用了n 个堆栈；当用户程序退出时，sp 减小 ，依次释放空间

5． 程序执行过程

8051 总共有 111 条指令：

49 条单字节指令，45 条双字节指令，17 条三字节指令。

8051 系统程序执行分 2 步：

9．取指令

10. 执行指令—根据指令不同，需要 1~3 个阶段（一个阶段一个时钟），其实不同的内核，其执行周期并不完全一样。

如三周期指令有：INC DPTR ；MUL AB ;XCH A,Direct;DIV A,B;等

二周期指令有：XCH A Ri 等

单周期指令有：SUB A @Ri 等

注意:上面并没有 core 发送地址的阶段,所以发送地址阶段是与 EXECx 重叠的,而第一个 fetch 指令的地址为 core 默认发送的 0 地址,但返回给core 的 ready 信号一定比第一个 fetch 提前一拍.而由于相当于地址要提前准备,那么在 fetch 阶段就需要分析指令,以确定需要发送的地址.

fetch 时发生中断时，当前 fetch 到的指令不会执行，接下来有 3 个 exec 动作，这三个动作为：

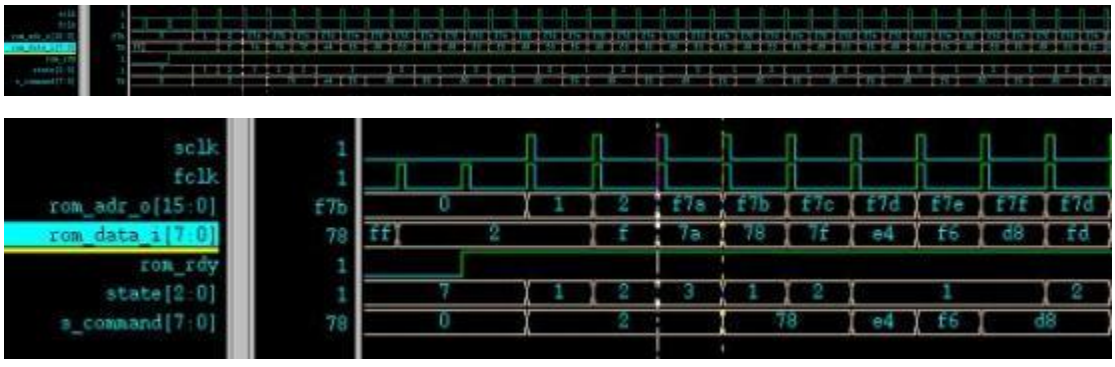
EXEC1: 给 pc 指针中断向量，

EXEC2:保存 pc 指针低 8bit，sp 地址+1，

EXEC3:保存 pc 指针高 8bit，sp 地址+1，

上面这 4 个动作为进入中断行为，在这个过程中，如果经过了 EXEC1，则不会被其它中断打断。---此处与 8051 核设计相关，不同的设计估计情况不一样。

在完全退出中断后会把保存的数据返回给 pc，重新取指，执行的动作。这个保存 pc 的动作为硬件完成。



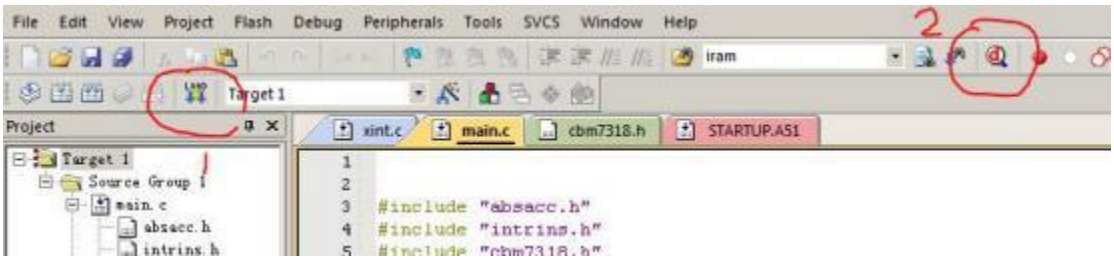
如上:state 含义:7 表示初始状态;1 为 fetch,取指令 ;2,3,4 表示执行的动作;

Rom_rdy 肯定是先为 1,core 才可以动作,上图可以看到提前 sclk 一拍,当第一个 sclk 时,core 读到了第一条指令(0x2),并分析命令 s_command,得出要发送的地址,当在 sclk 的第三拍,执行跳转.core 发送地址 0xf7a;

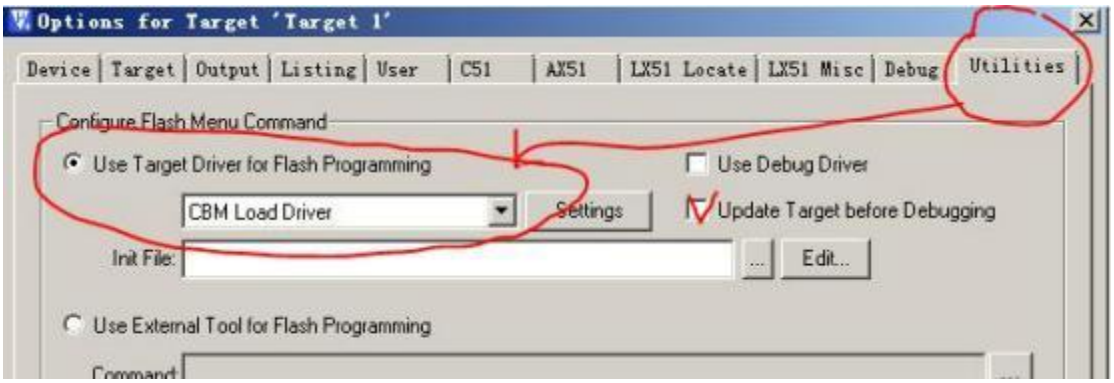
一般的 8051 单片机都不具有流水线操作.直接用状态机完成一条指令的的行为!!

6. Keil 下载

点击下图 2 个地方都会引起下载程序的行为

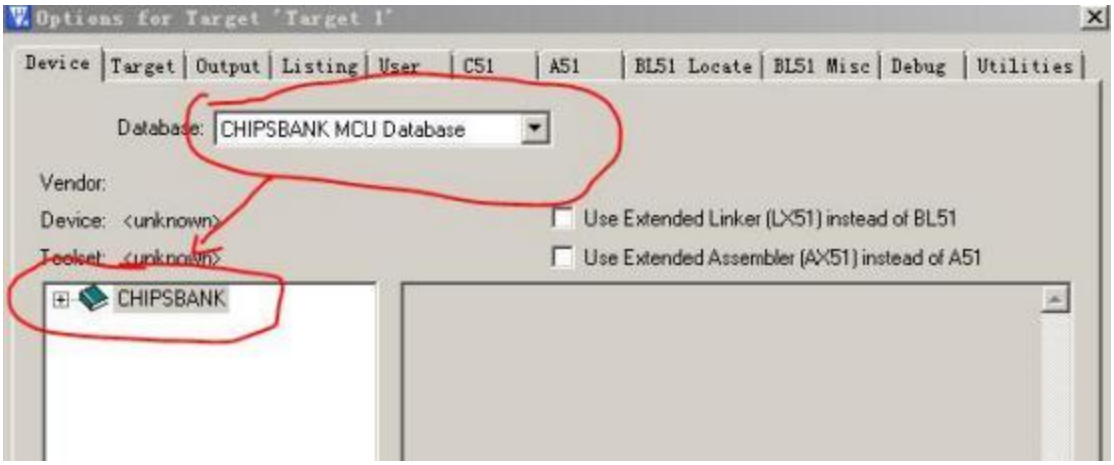


使用 1 下载需要做如下配置

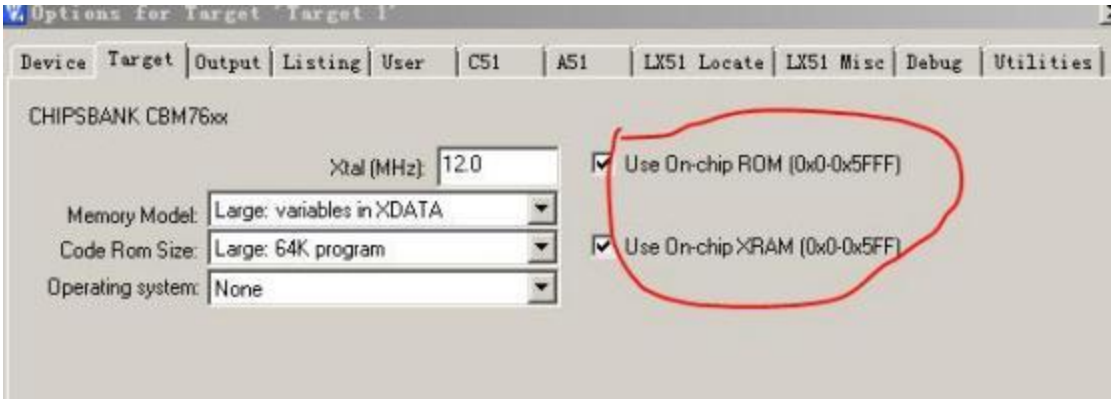


这个驱动需要修改keil 的插件，如果使用 2 下载，则需要勾上 updata target before debugging,这样它们将使用同一个驱动，设置上面下载的驱动，在安装文件需要做如下更改--在安装目录下的 TOOLS.INI文件中指明安装包.CDB 和驱动.dll 文件(放在 c51\bin\目录下)。---dll 文件用上作上图的下载驱动，CDB 包用于下图的器件选择。

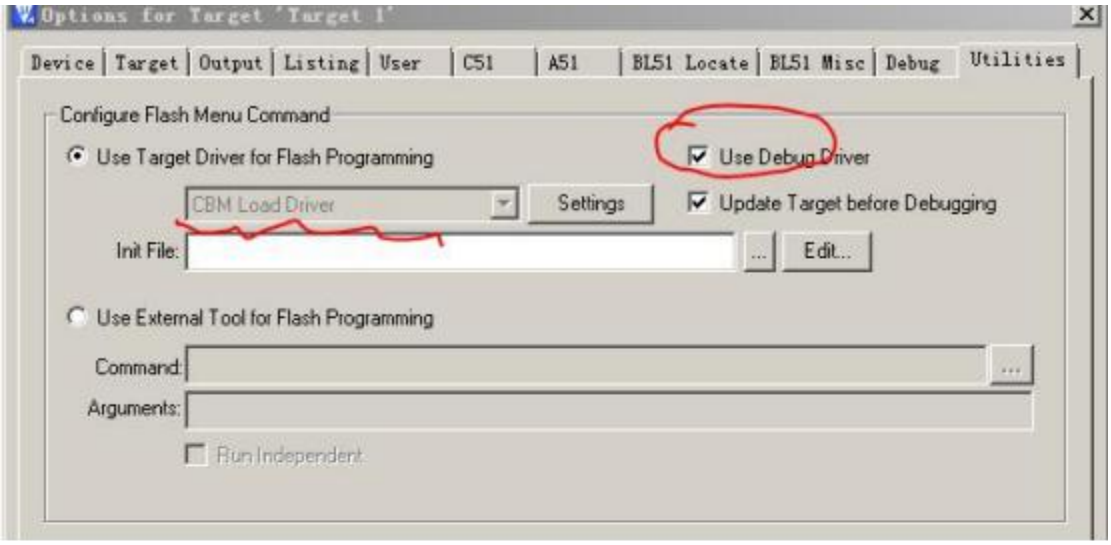
动态链接库英文为 ***DLL***，是 **Dynamic Link Library** 的缩写。DLL 是一个包含可由多个程序，同时使用的代码和数据的库。通过使用 DLL，程序可以实现模块化，由相对独立的组件组成。例如，一个计帐程序可以按模块来销售。可以在运行时将各个模块加载到主程序中（如果安装了相应模块）。因为模块是彼此独立的，所以程序的加载速度更快，而且模块只在相应的功能被请求时才加载。

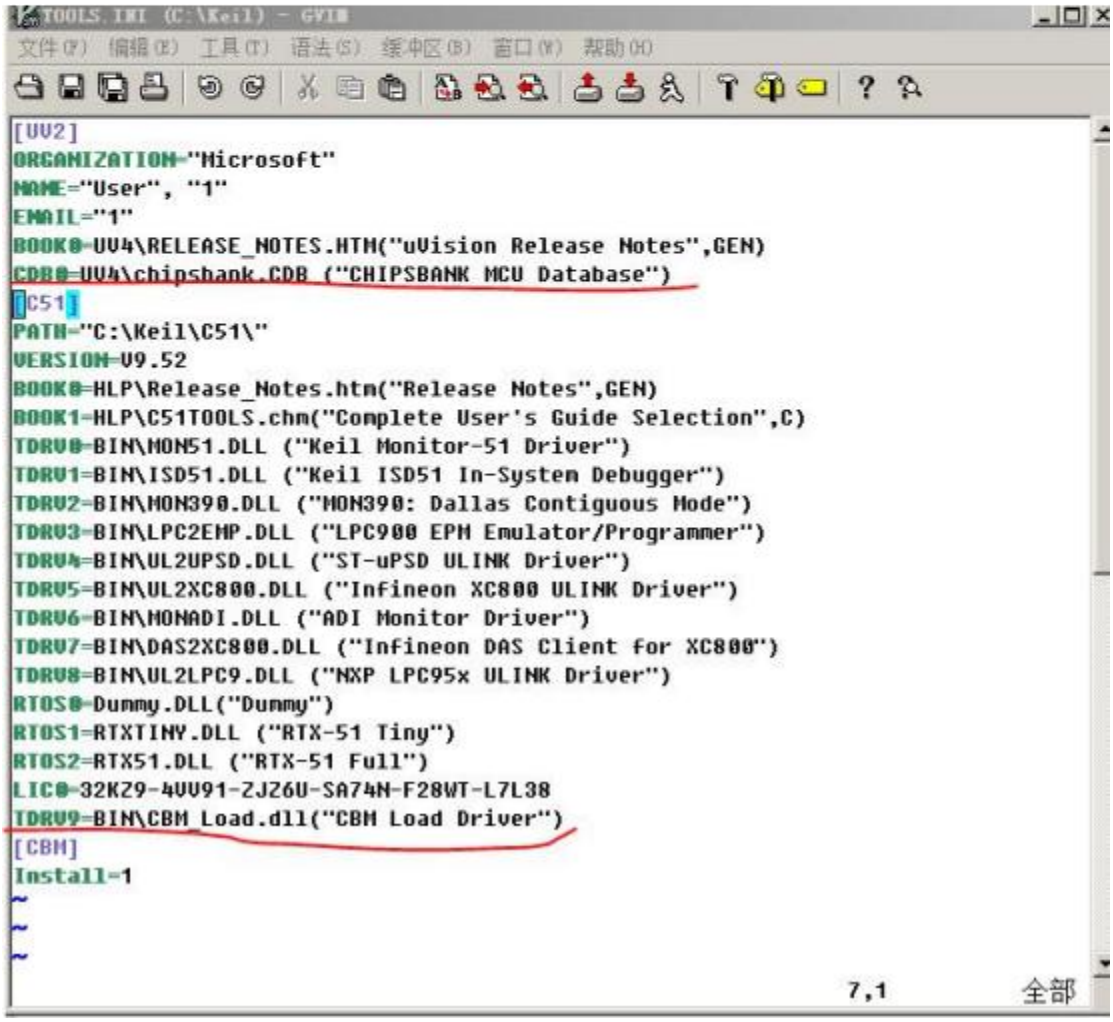


Cdb 文件就是包含上述信息，里面有 rom 大小，xram 大小。如下图所示

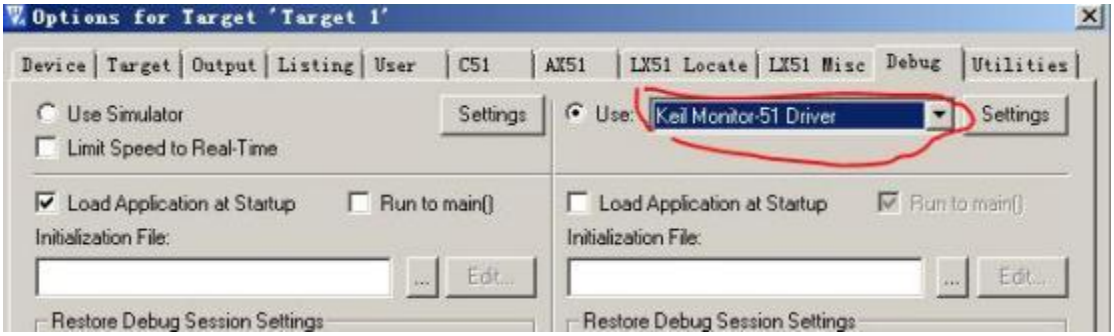


如果选择了 Use Debug Driver，则使用 mon-51 编程 flash,那么对于 flash 控制器需要符合mon-51 的要求





注意在 debug 时，需要设置如下 keil monitor-51 driver,这个东西应该是发送 mon51 命令的 driver;



7． Debug 过程

Debug 的行为分为三类：

5. 配置类：设置断点
6. 操作类：读写各种数据，sfr,xram,iram,code
7. 控制类：全速运行，单步运行，运行到断点，复位

Debug 的行为过程应该为如下几个阶段：

1. 点击 debug 按钮 ---download 程序进入 flash，进入调试状态，keil 发送 0x11 与芯片握手，芯片交替返回 0x00,0xff,如果界面有 sfr 等其它内容，keil会发送相关读取指令，把读取的数据显示在界面；---0x11握手是私有握手命令

   注意:keil 工具本身会下发指令下载程序,如果有私有下载程序插件,那么将会下载 2 遍程序.

   a. 此时程序没有运行，51 内核应该停止运行，一直处于取指状态；

      --指令操作一直在取当前的指令，core 内部所有寄存器不能改变，只能通过调试命令修改；

   b. 当 keil 暂停时，其一定是在完成当前指令后暂停，然后再处于取指状态；

      注意:通过 debug 下载程序时,debug会通过分析代码,进行某种算法,然后再根据算法把代码下载进 rom/flash;也就是它并不是一定按照 flash/rom 的顺序地址下载.

2. 用户配置 keil 界面
3. 用户通过keil 界面仿真—单步或全速运行

4. 复位，复位的行为就是设置 pc 等于 0，也就是点击复位按钮，keil 发送一个写 pc=0 的命令，但芯片需要做一个复位的动作，只是复位芯片，注意不能让keil 与芯片断开，所以调试模块的复位不在此复位域。

由于指令执行有 1，2，3 个不同的执行周期，那怎么找到什么时候执行完当前指令呢？看下面的做法。

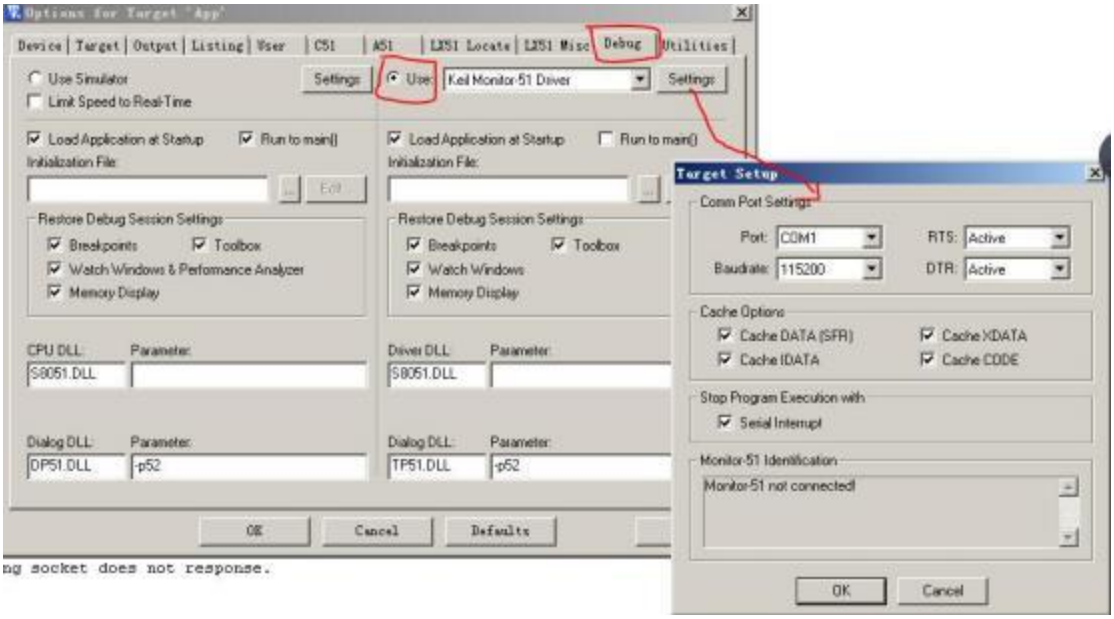
```
always @(posedge sclk or negedge resetn)
    begin
        if (~resetn ) state <= 3'h7;
        else state <= s_nextstate;
`ifdef DBG51
   assign instr_end = cpu_run & (s_nextstate==`FETCH) & rom_rdy;
`endif
```

因为 51 无流水操作,当下一个状态为取指,则当前就一定是指令执行完的状态.

8． Debug 图形界面介绍

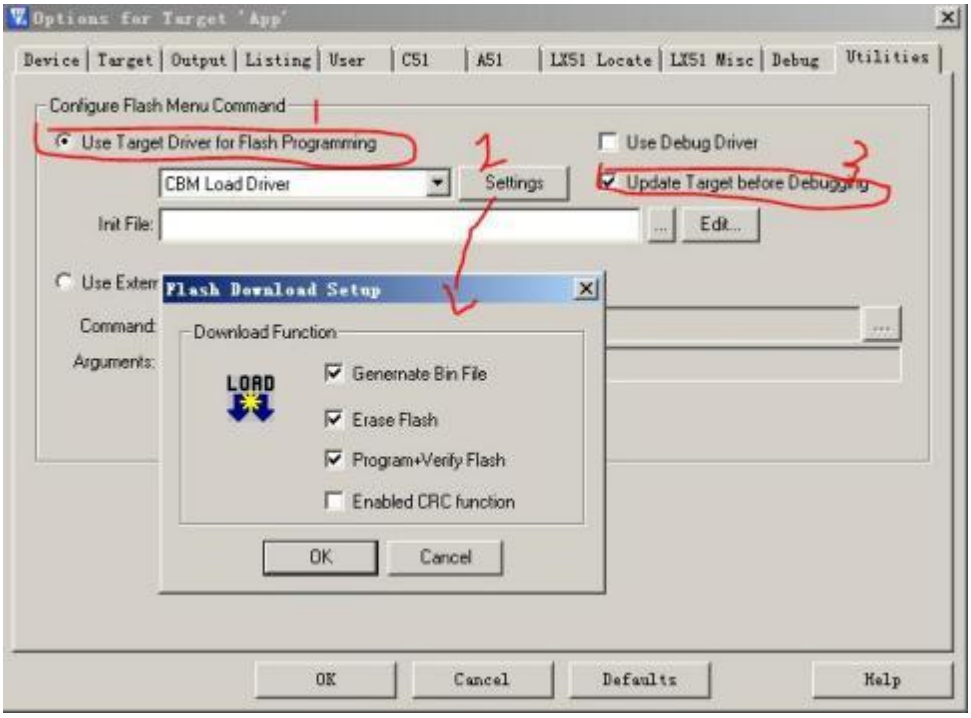
Debug 工具

a.



上图此处配置的是 debug使用的驱动,用的mon51 标准驱动,并设置串口号及波特率;

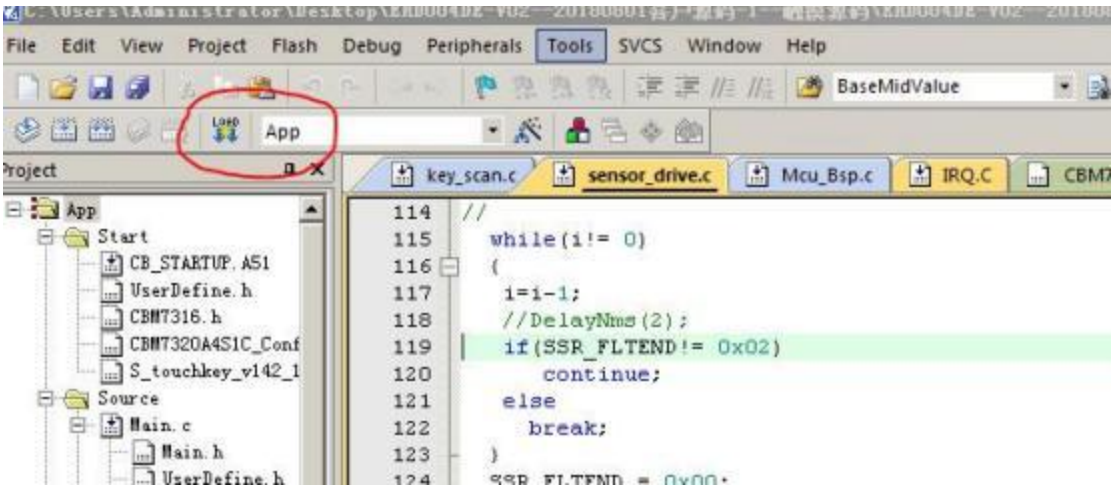
b.



上图配置的 1:下载程序所用到的驱动一此插件为私有插件,2:setings 按纽可以配置下载的行为;3:指明在调试前先会调用 flash download 更新程序,如果没选择 3，则直接进入 debug进程，而不会下载程序。

Debug 的过程就是先 download 程序，然后再调试，也可以分步执行（3 不选择）。

c.



点击上图红色按钮，则会调用b 部分说的驱动来执行产生下载文件及下载

9． Hex 格式讲解（转）

Hex 文件是可以烧录到 MCU 中，被 MCU 执行的一种文件格式。如果用记事本打开可发现，整个文件以行为单位，每行以冒号开头，内容全部为 16 进制码（以 ASCII码形式显示）。Hex 文件可以按照如下的方式进行拆分来分析其中的内容：

例如 “ :1000080080318B1E0828092820280B1D0C280D2854”可以被看作

“0x10 0x00 0x08 0x00 0x80 0x31 0x8B 0x1E 0x08 0x28 0x09 0x28 0x20 0x28 0x0B 0x1D 0x0C 0x28 0x0D 0x28 0x54”

第一个字节 0x10 表示本行数据的长度；

第二、三字节 0x00 0x08 表示本行数据的起始地址；

第四字节 0x00 表示数据类型，数据类型有：0x00、0x01、0x02、0x03、0x04、0x05。

’00’ Data Rrecord：用来记录数据，HEX 文件的大部分记录都是数据记录

’01’ End of File Record: 用来标识文件结束，放在文件的最后，标识 HEX 文件的结尾

’02’ Extended Segment Address Record: 用来标识扩展段地址的记录

’03’ Start Segment Address Record:开始段地址记录

’04’ Extended Linear Address Record: 用来标识扩展线性地址的记录

’05’ Start Linear Address Record:开始线性地址记录

然后是数据，最后一个字节 0x54 为校验和。

校验和的算法为：计算 0x54 前所有 16 进制码的累加和(不计进位)，检验和 = 0x100 - 累加和

在上面的后 2 种记录，都是用来提供地址信息的。每次碰到这 2 个记录的时候，都可以根据记录计算出一个“基 ”地址。对于后面的数据记录，计算地址的时候，都是以这些“基 ”地址为基础的。

HEX 文件都是由记录（RECORD）组成的。在 HEX 文件里面，每一行代表一个记录。记录的基本格式为：

Record mark ‘ :’

Length

Load offset

Record type

INFO or DATA

CHKSUM

1 byte

1 byte

2 bytes

1 byte

n bytes

1 byte

看个例子：

:020000040008F2

:10000400FF00A0E314209FE5001092E5011092E5A3

:00000001FF

对上面的 HEX 文件进行分析：

第 1 条记录的长度为 02，LOAD OFFSET 为 0000，RECTYPE 为 04，说明该记录为扩展段地址记录。数据为 0008，校验和为 F2。从这个记录的长度和数据，我们可以计算出一个基地址，这个地址为(0x0008 << 16)。后面的数据记录都以这个地址为基地址。

第 2 条记录的长度为 10（16），LOAD OFFSET 为 0004，RECTYPE 为 00，说明该记录为数据记录。数据为 FF00A0E314209FE5001092E5011092E5，共 16 个BYTE。这个记录的校验和为 A3。此时的基地址为 0X80000，加上 OFFSET，这个记录里的 16BYTE 的数据的起始地址就是 0x80000 + 0x0004 = 0x80004.

第 3 条记录的长度为 00，LOAD OFFSET 为 0000，TYPE ＝ 01，校验和为 FF。说明这个是一个 END OF FILE RECORD，标识文件的结尾。

在上面这个例子里，实际的数据只有 16 个 BYTE：

FF00A0E314209FE5001092E5011092E5，其起始地址为 0x0004.

Flip80251 知识

CPUINFO (S:96h) read only

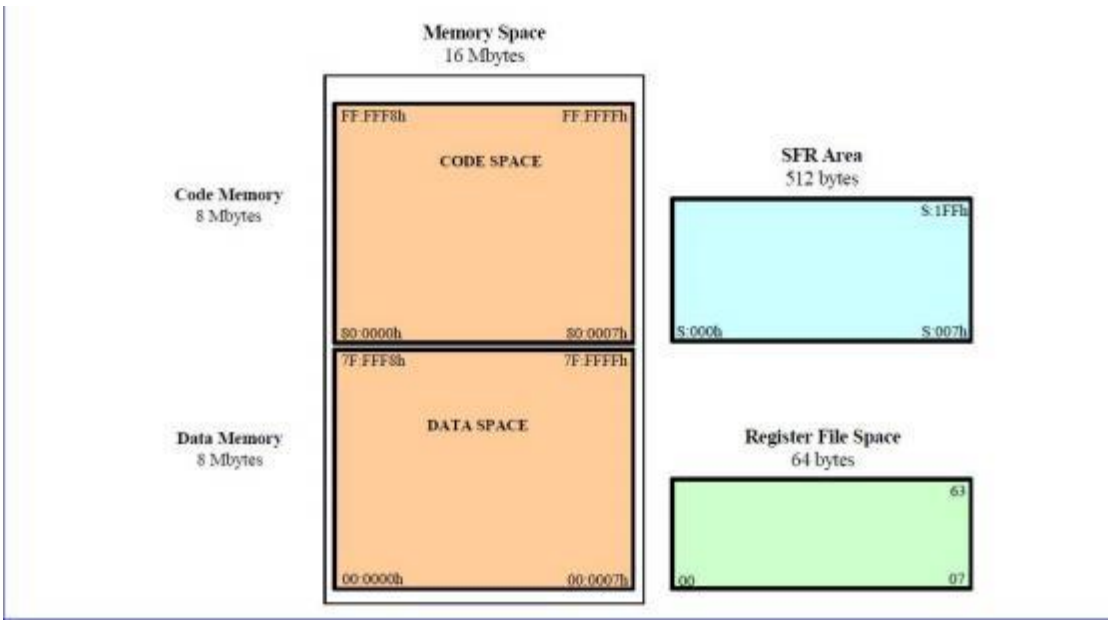
(BIRD) CPU Information Register

BIT	7	6	5	4	3	2	1	0
FIELD	--	--	MONI	SRC	INTR	TMS2	TMS1	TMS0
RESET	0000 0000b							

Bit Number	Bit Mnemonic	Function
7:6	--	**Reserved** The value read from these bits is indeterminate
5	MONI	**Monitor program mapping information bit** Enables to read back the value of the *moninuser* pin
4	SRC	**Source mode / binary mode select** Enable to read back the opcode mode of the Flip80251-Hurricane When high, it indicates that the core is delivered in *source mode* Otherwise, it indicates that the core is in *binary mode*
3	INTR	**Interrupt mode.** Enable to read back the value of *intrmode* pin
2:0	TMS2:0	**Trace Memory size control bus.** Enable to read back the value of the *tmsize[2:0]* bus

Figure 101: CPU Information register (CPUINFO)

地址空间分布



Memory Type	80C51 Architecture			8xC251 Architecture
	Size	Location	Data addressing	Location
Code	64 Kbytes	[0000h-FFFFh]	Indirect using MOVC	[FF:0000h-FFFFFh]
External Data	64 Kbytes	[0000h-FFFFh]	Indirect using MOVX	[01:0000h-01:FFFFh]
Internal Data	128 bytes	[00h-7Fh]	Direct, Indirect	[00:0000h-00:007Fh]
	128 bytes	[80h-FFh]	Indirect	[00:0080h-00:00FFh]
SFRs	128 bytes	[S:80h-S:FFh]	Direct	[S:080h-S:0FFh]
Register	8 bytes	[R0-R7]	Register	[00:0000h-00:001Fh]

Table 16: Address Mappings

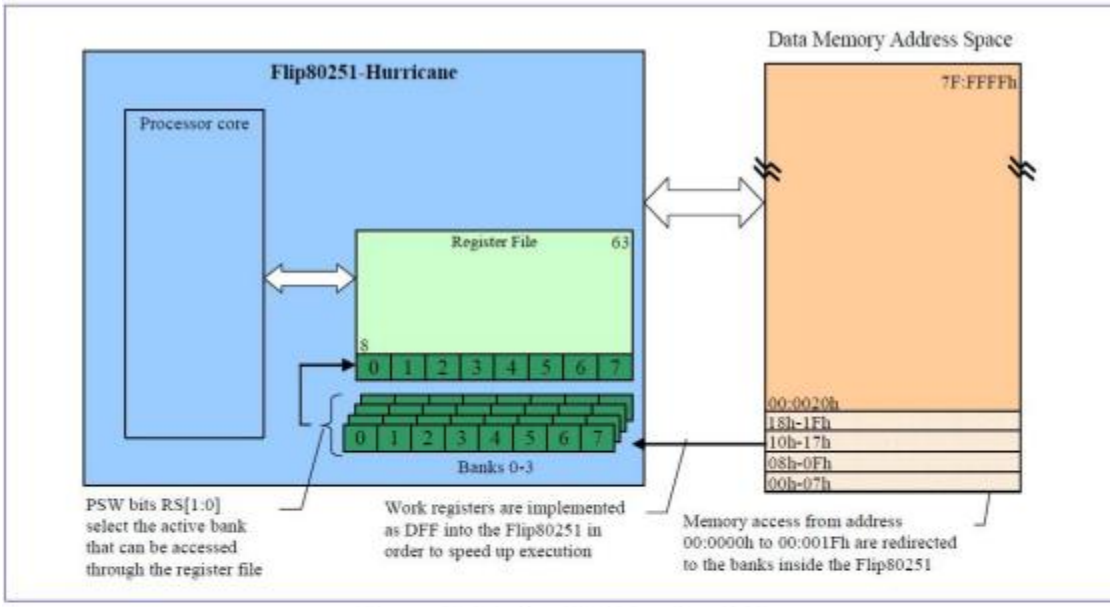


Figure 7: Register File Location 0-7

Segment Prefix	Memory Class	Memory Type	Description
?PR?	CODE or ECODE	program	Executable program code
?CO?	CODE	code	Constant data in program memory
?BI?	BIT	bit	Bit data in internal data memory
?BA?	BDATA	bdata	Bit-addressable data in internal data memory
?DT?	DATA	data	Internal data memory with 8-bit addressing
?ED?	EDATA	near	Variables with 16-bit addressing
?EA?	EBDATA	ebdata	Extended 251 bit-addressable variables
?EB?	EBIT	bit ebdata	Extended 251 bits
?FD?	HDATA	far	Far memory (RAM space)
?FC?	HCONST	const far	Far memory (constant ROM space)
?HD?	HDATA	huge	Huge variables (no size limitation)
?HC?	HCONST	const huge	Huge constants (no size limitation)
?ID?	IDATA	idata	Indirectly addressable internal data memory
?NC?	NCONST	const near	Constants with 16-bit addressing
?PD?	PDATA	pdata	Paged data in external data memory
?XD?	XDATA	xdata	Xdata memory (RAM space)
?XC?	XDATA	const xdata	Xdata memory (constant ROM space)

The HDATA memory class is typically stored in external memory that is usually off-chip or on-chip.

HDATA memory can only be accessed via the registers DR0 .. DR28 and DR56. Up to 16MB of memory can be accessed using these registers.

The EDATA memory is only available in the 251 architecture.

In the 251, EDATA memory can be accessed with direct memory addressing or indirect via the registers WR0 .. WR30. Also the memory class IDATA and DATA can be access with this addressing mode.

基础知识

请注意程序使用的逻辑地址和实际物理地址的关系，由于是 16bitMCU，所以程序一般使用的地址应该是偶数即 2byte;

Binary mode: providing full binary code compatibility with the legacy 80C51/52

Source mode: providing an enriched instruction set with 16/32-bit capability

两套指令集 MAPing:

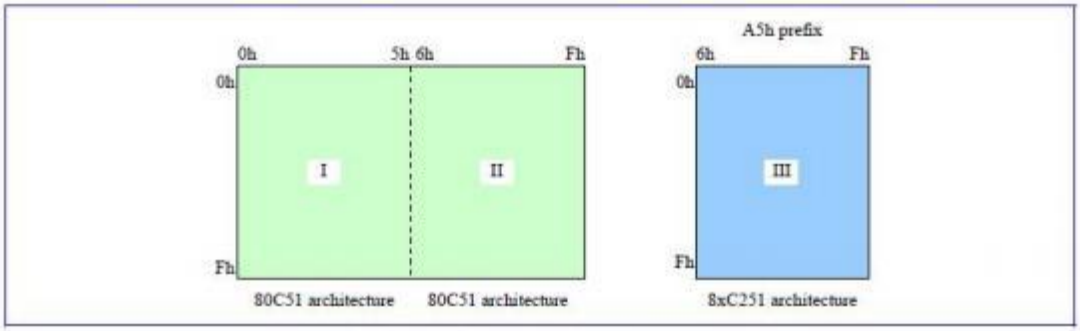


Figure 9: Binary mode opcode map

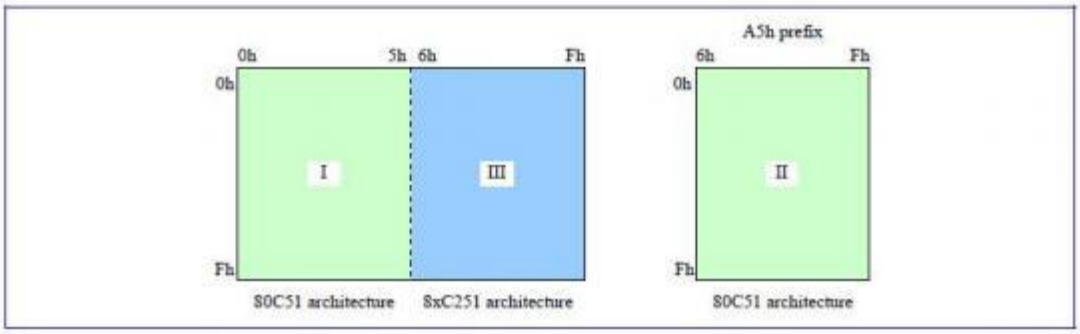


Figure 10: Source mode opcode map

如果想直接使用 80c51 的代码在 Flip80251 系统中运行，则必须选择 Binary mode;

可以实现流水操作:

1. 复位地址指向 0xff0000
2. 取指只需一个 clock;
3. 每次取指 16bit，为 2 条指令
4. .对于程序区支持写操作，实际上是通过间接寻址指令写，与 XBUS 总线的指令一致，MOVX @RKG,Rn

80251 XBUS 空间寻址方式及范围

Mode	Address range of operand	Assembly language Reference	Comments
Register	[00:0000h-00:001Fh]	R0-R15, WR0-WR30, DR0-DR28, DR56, DR60	R0-R7, WR0-WR6, DR0-DR4 are in the register bank currently selected by the PSW and the PSW1
Immediate 2 bits	(N.A) Operand in Instruction	#short = 1, 2 or 4	Used only in increment and decrement instruction
Immediate 8 bits	(N.A) Operand in Instruction	#data8 = #00h-#FFh	
Immediate 16 bits	(N.A) Operand in Instruction	#data16 = #0000h-#FFFFh	
Direct, 8 address bits	[00:0000h-00:007Fh]	dir8 = 00:0000h-00:0007Fh	Internal RAM
	SFRs	dir8 = S:080h-S:0FFh or SFR mnemonic	SFR address
Direct, 16 address bits	[00:0000h-00:FFFFh]	dir16 = 00:0000h-00:FFFFh	
Indirect, 16 address bits	[00:0000h-00:FFFFh]	@WR0-@WR30	
Indirect, 24 address bits (*)	[00:0000h-FF:FFFFh]	@DR0-@DR28, @DR56, @DR60	
Displacement, 16 address bits	[00:0000h-00:FFFFh]	@WRj+dis16 = @WR0+0h through @WR30+FFFFh	Offset is signed, address wraps around in region 00:
Displacement, 24 address bits (*)	[00:0000h-FF:FFFFh]	@DRk+dis24= @DR0+0h through @DR28+FFFFh, @DR56+(0-FFFFh), @DR60+(0-FFFFh)	Offset is signed.

Table 38: Addressing Modes for Data instruction in the Flip80251 architecture

Register file 可以 byte,word,dword 访问

3.3 Register File

The Flip80251-Hurricane register file consists of 40 byte locations: 0-31 and 56-63. These locations are accessible as bit, bytes, words, and dwords. Several locations are dedicated to special registers; the others are general-purpose registers.

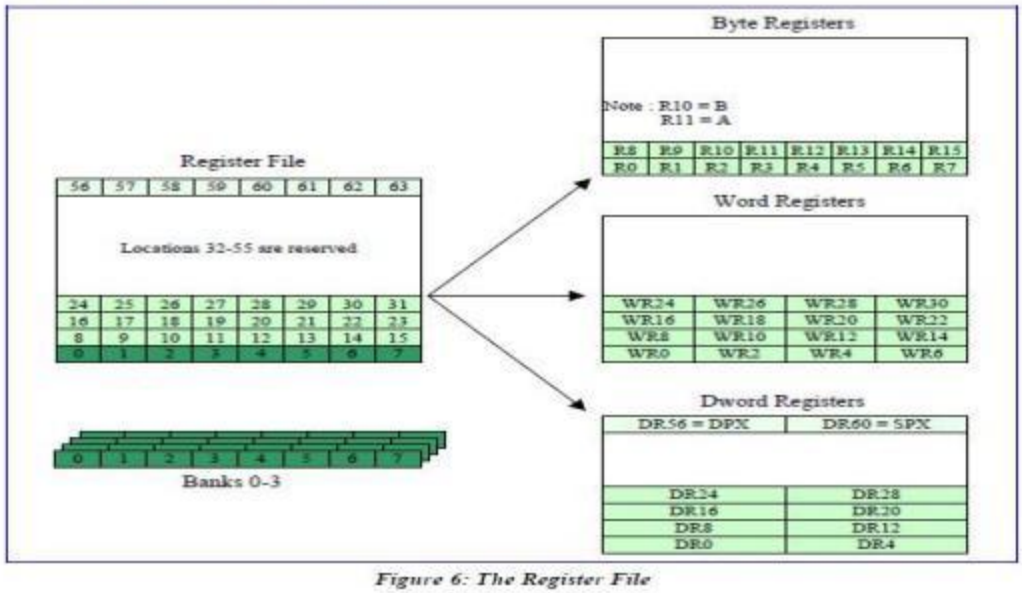


Figure 6: The Register File

register file 可以做为中间临时变量，可以用三种方式操作，byte,word,dword,

当操作 8bit 数据可以使用 Rm(0,1,2...)

当操作 32bit 数据时，可以使用 DRk(k=0,4,8...)

对应关系如上图

中间空白处用于固定的特殊功能寄存器，如左图所示

DPXL 默认值为 01，所以默认 xram 总线为 0x01xxxx;标准 8051 没有

Register File					SFRs	
Name			Mnemonic	Reg.	Location	Mnemonic	Address
Stack Pointer (SPX)	-		-	DR60	60	-	-
	-		-		61	-	-
	Stack Pointer, High		SPH		62	SPH	S:0BEh
	Stack Pointer, Low		SP		63	SP	S:081h
Data Pointer (DPX)	-		-	DR56	56	-	-
	Data Pointer, Extended Low		DPXL		57	DPXL	S:084h
	DPTR	Data Pointer, High	DPH		58	DPH	S:083h
		Data Pointer, Low	DPL		59	DPL	S:082h
Accumulator (A register)			A	R11	11	ACC	S:0E0h
B Register			B	R10	10	B	S:0F0h

Table 18: Dedicated Registers in the Register file and their corresponding SFRs

Flip80251 核的 3 类时钟

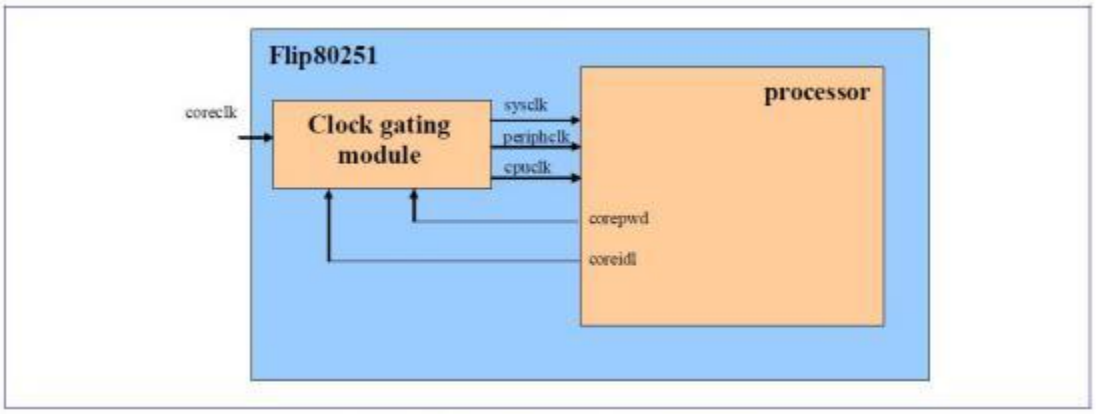


Figure 81: Internal clock management

注意 PD 和 IDL 退出的条件不一致：PD 一定要外部中断才能退出，而 IDLE 只要有中断就可以退出

PD 和 IDLE 状态主要不同点：PD 状态所有时钟停止，IDLE 状态中断等模块的时钟不停止

sysclk:与 coreclk 同源，但没有门控，它用于 8051 的寄存器和同步 PD 模式下的唤醒信号，PD 只能外部中断 int0,int1,NMI三种中断唤醒，属于异步信号

periphclk:8051 外设和中断控制模块时钟，该时钟只在 PD 状态停止，IDLE 状态不停止

cpuclk:cup 时钟，当 idle & pd 状态，该时钟停止

中断让时钟运行起来，但标志位的释放是根据程序运行到RETI这条指令来做的

由于中断唤醒，所以在进入 PD 前必须配置相关中断使能

SFR 时序

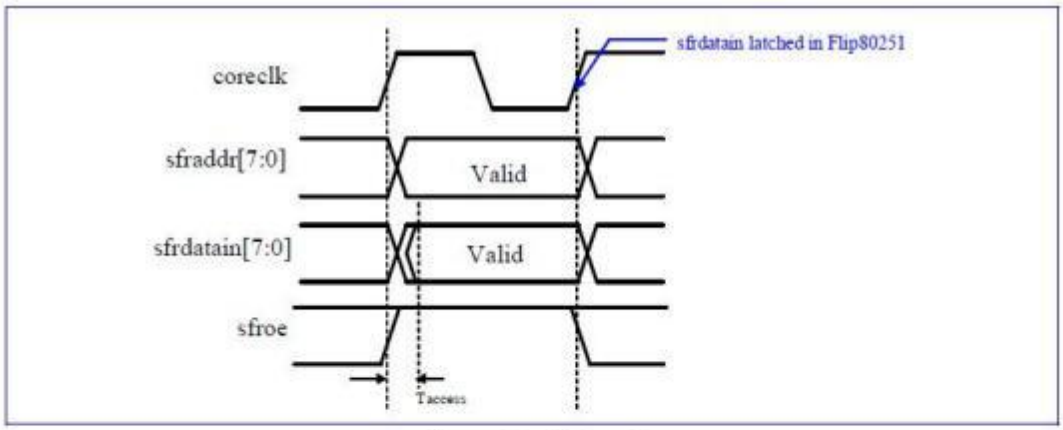


Figure 82: SFR read

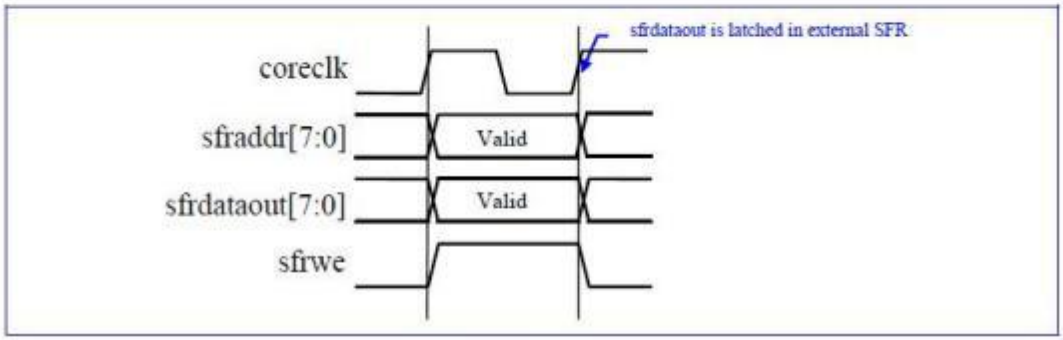


Figure 83: SFR write

PBSU 时序

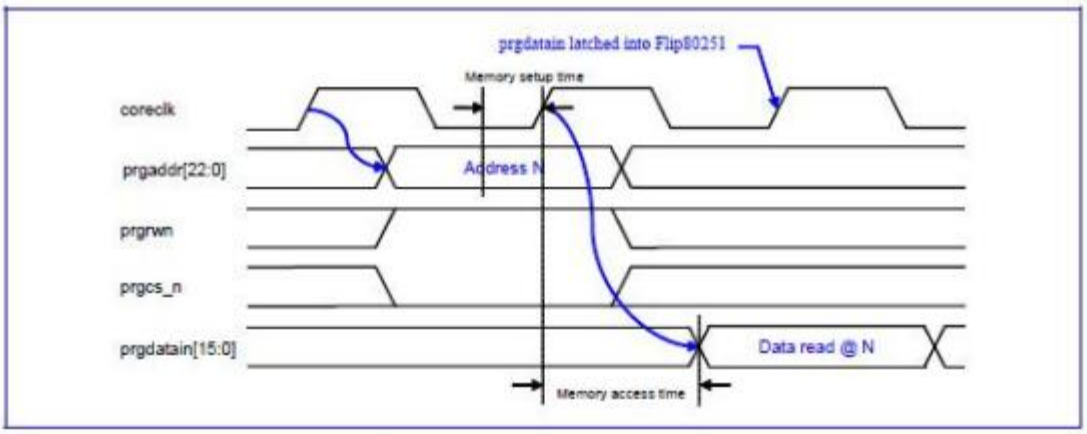


Figure 84: Program memory fetch or read

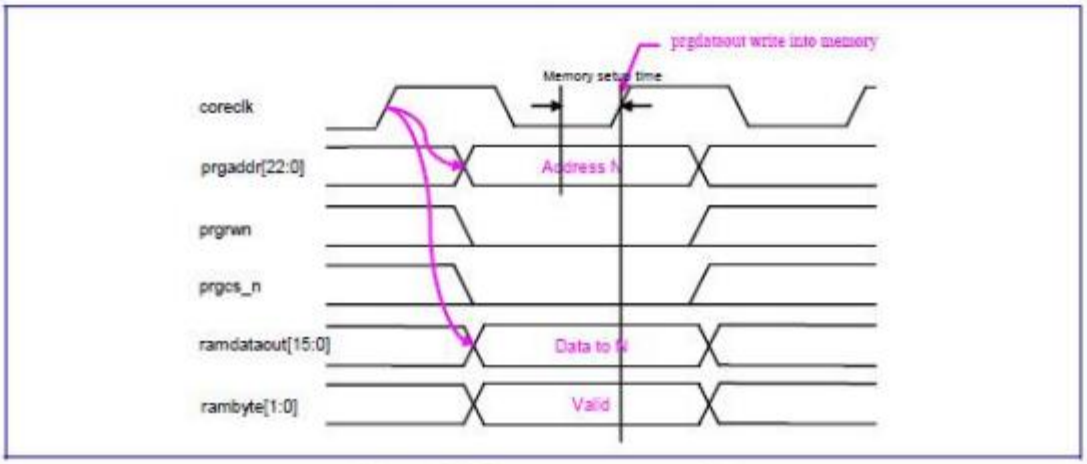


Figure 85: Program memory write

XBUS 时序

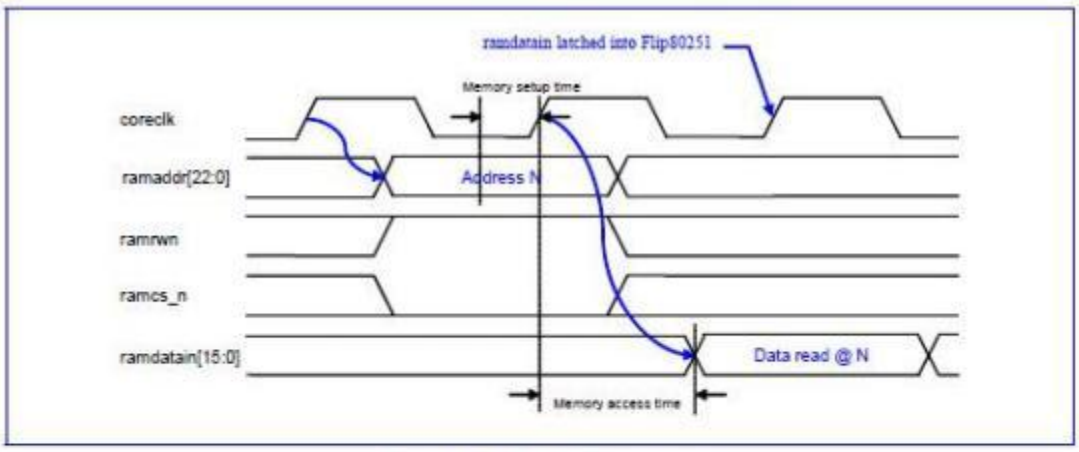


Figure 86: Data Memory read

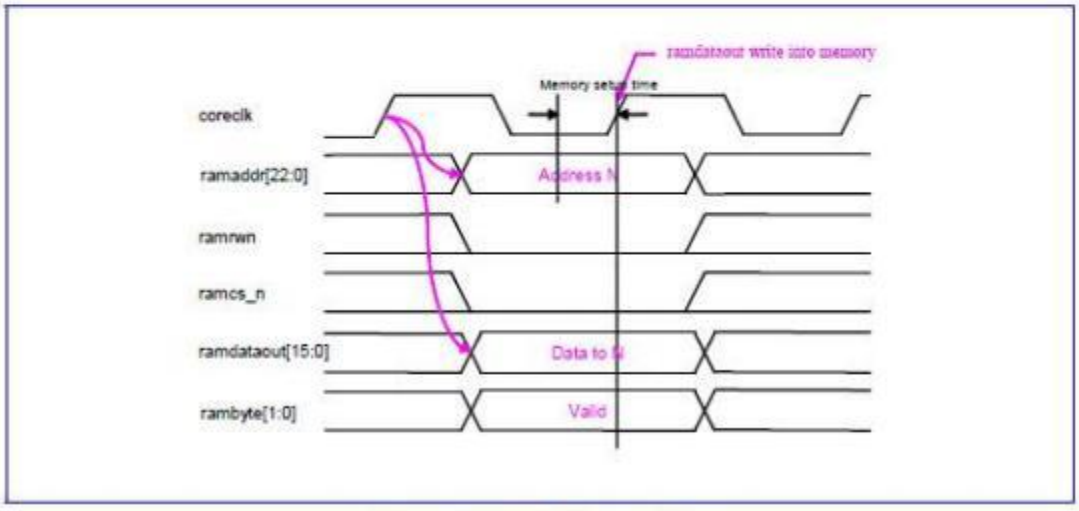


Figure 87: Data Memory Write

上述 XBUS 和 PBUS 总线需要配合 CORE 的 prgbusy 和 rambusy 运行，当相关信号为高，意味着外部忙，core 将不会有指令动作，如果核发出了动作，同时也会发出 fetchabort 信号来指示动作无效

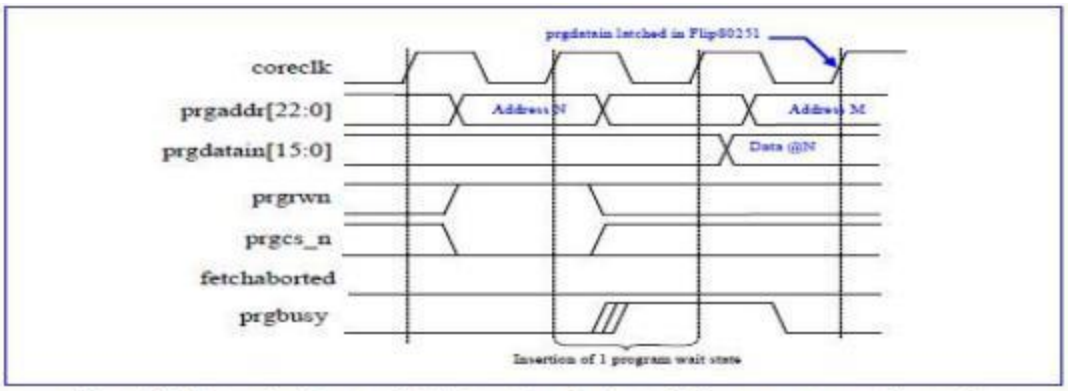


Figure 88: Example of one wait state insertion during a fetch or program read operation

The signal *fetchaborted* is used only for a fetch operation. This signal indicates that the current fetch operation will be aborted. If wait states are required for this access, these wait states insertion must be ended and the signal *prgbsuy* has to be de-asserted.

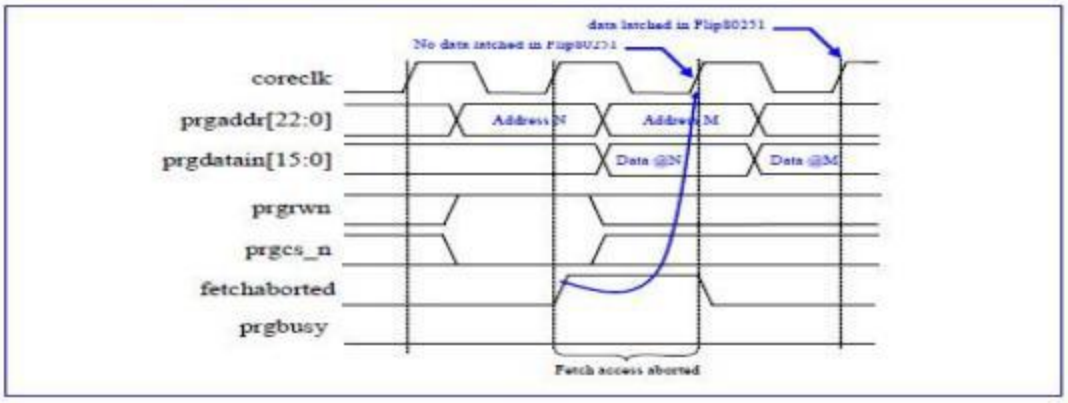


Figure 89: Example of a fetch operation aborted without wait states insertion

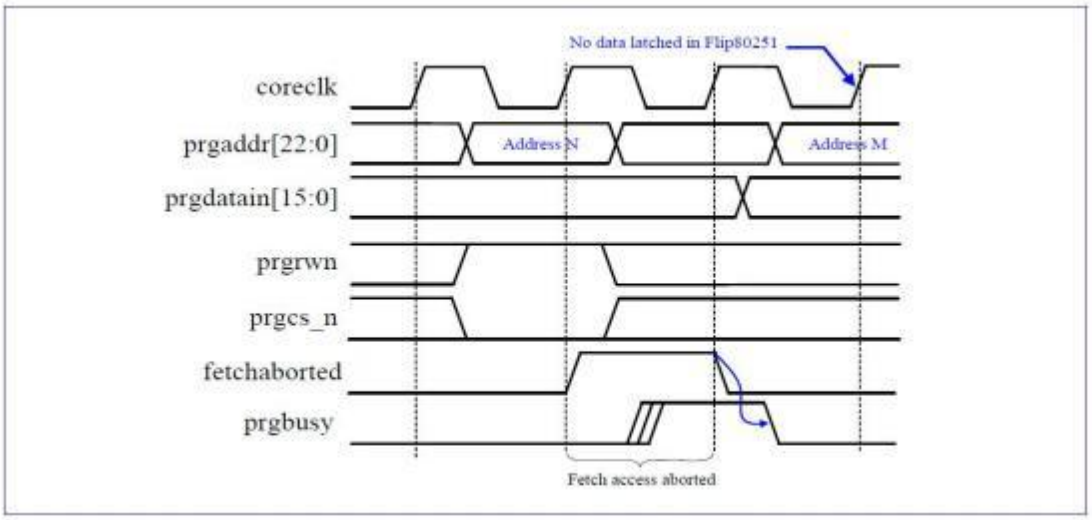


Figure 90: Example of a fetch operation aborted during wait states insertion

问题：为什么会no data latchen,第一拍控制发出，正常情况第二拍发出数据，但主显示放弃及从表示忙，所以第三拍不采样数据，但第拍从已经把 prgbusy

信号拉第，且把数据返回，主也没有abort指示，为什么第四拍不能 latch 数据？？？

答：因为第三拍返回的数据是 ADDRESS N 的数据，而那拍的数据为 abort，所以第四拍不能 latch

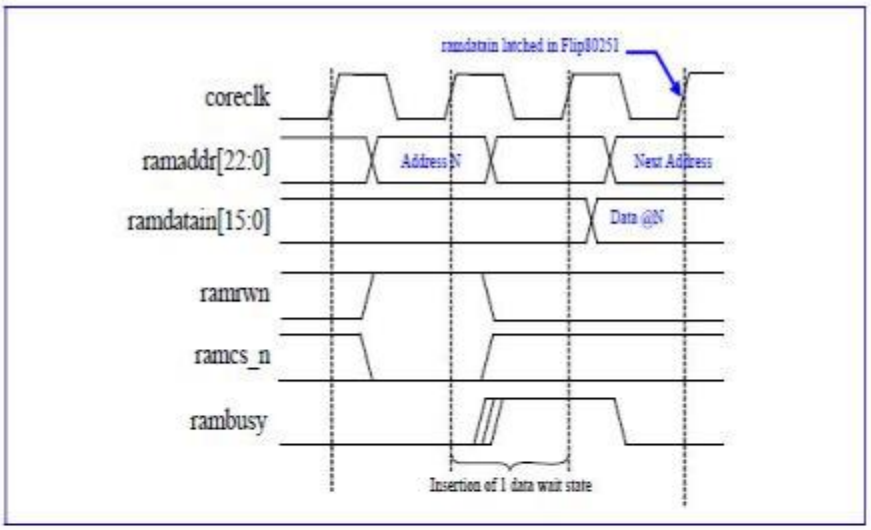


Figure 92: Example of one wait state insertion during data memory read

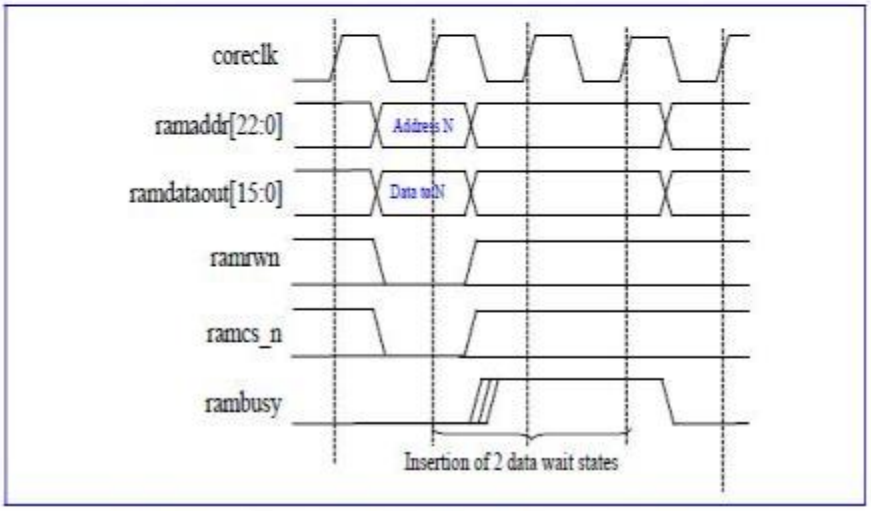
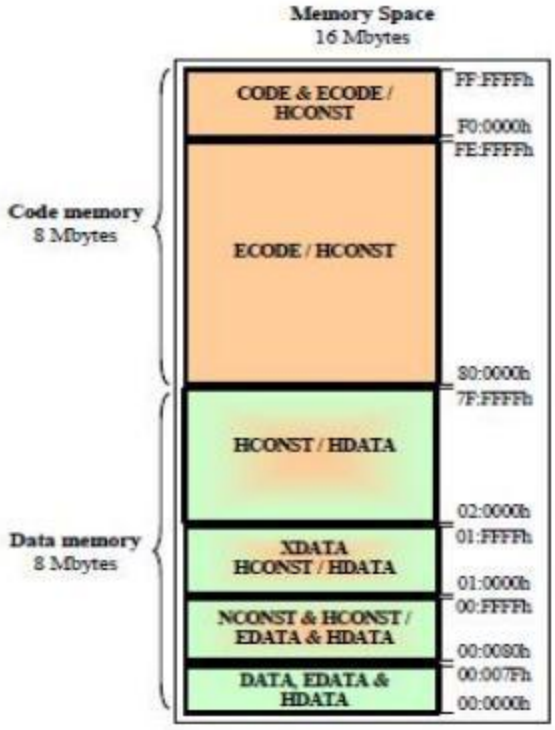


Figure 93: Example of two wait state insertion during data memory write

The Flip80251 memory architecture is described below:



Memory class	Description	Memory location	Memory specifier
DATA	The DATA class refers to the first 128 bytes of internal data memory. Access to data memory is very fast because it can be accessed using an 8-bit address	00:0000h 00:007Fh	data
EDATA	The EDATA class contains variables which must be directed to the RAM space in the first 64 Kbytes of the Flip80251 memory.	00:0000h 00:FFFFh	near
XDATA	The XDATA class is a 64 Kbytes area accessed indirectly through the data pointer (DPTR or DR56). This area is implemented for 80C51 compatibility.	01:0000h 01:FFFFh	xdata
HDATA	The HDATA class contains variables which must be directed to the RAM space in the full address range of the data memory of the Flip80251.	00:0000h 7F:FFFFh	far
NCONST	The NCONST class contains constants which must be directed to ROM space in the first 64 Kbytes of the Flip80251 memory.	00:0000h 00:FFFFh	const near
HCONST	The HCONST class contains constants which must be directed to ROM space in the full address range of the Flip80251.	00:0000h FF:FFFFh	const far / const huge
CODE	The CODE class contains executable program code or constants which must be directed to ROM space in the last 64 Kbytes of the Flip80251.	FF:0000h FF:FFFFh	code
ECODE	The ECODE class contains executable program code which must be directed to ROM space in the full address range of the code memory of the Flip80251.	80:0000h FF:FFFFh	-

Table 5: Memory class table

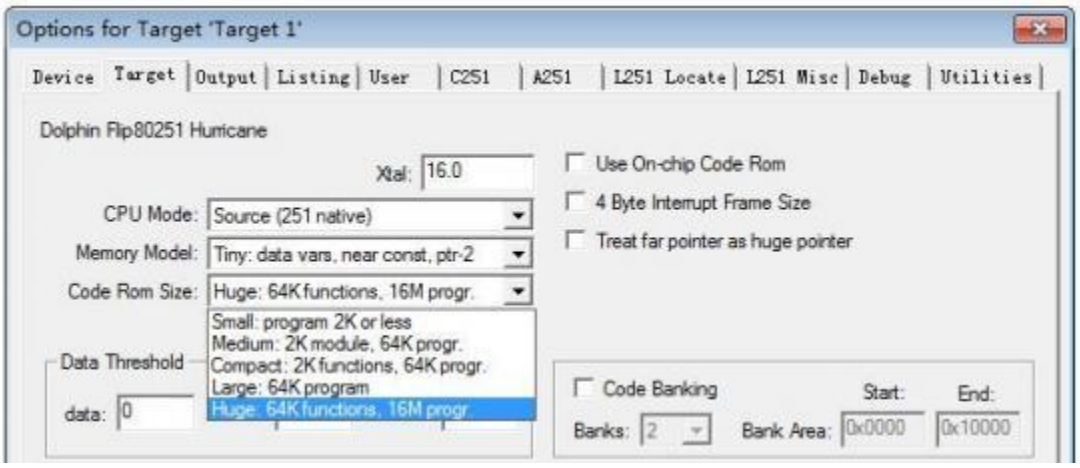
如何配置把程序放在某个指定的空间 ：分两种情况 1.地址在 64K 内；2，程序空间大于 64K，需要在KEIL 中配置

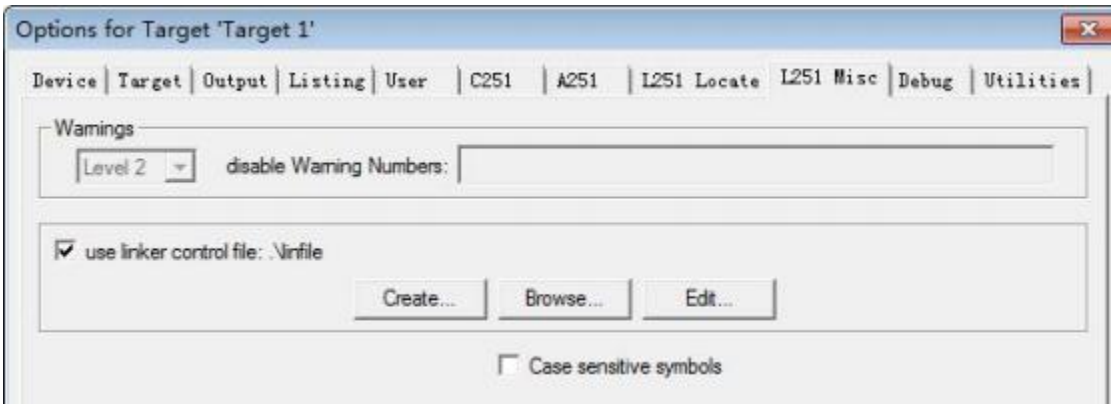
Code Rom Size: Large:64K program 或 Huge:64K function,16M progr;

当要指定程序 myfunc 的位置时，把程序 myfunc 单独写在一个文件中myfile，然后在选项 L251 Misc 中钩上“use linker control file:.\linkfile”,linkfile 为需要 create 的文件，其内容格式为：当为 Huge:64K function,16M progr 模式时：SEGMENTS (?PR?TEST1 (0xFE2000)) 其

中?PR?TEST1 相当于路径；当为 64K program 或其余模式时：SEGMENTS (?PR?*?TEST1 (C:0x2000))

对于?PR?TEST1 或?PR?*?TEST1 可以先不加 link 文件，先生成 map 文件，然后再找到该路径





当选择 Huge:64K function,16M progr 模式时，output 文件类型必须选择 hex386 格式，而非 hex80 格式；扩展线性地址记录(HEX386)

可以按下图方式配置 CODE，DATA 空间范围：这样可以防止编译出来的空间比实际的空间大，如果比实际空间大将会报错

To relocate also the interrupt vectors you should setup the memory layout in in µVision under **Options for Target – L251 Locate – User** classes. The following example shows the setup for:

- Data RAM in address space **0 - 0x3FFF**.
- Code RAM in address space **0xFE0000 - 0xFEFFFF** and **0xFF4000 - 0xFFFFFF**.

Note

- In the example above we assume that the MON251 firmware is stored in address space **0xFF0000 - 0xFF3FFF**.
- **CODE (\$0xFF4000, 0xFF4000 - 0xFFFFFF)** relocates the reset and interrupt vectors to an offset of 0xFF4000. Refer to the L251 User's Guide, Linker Directive for more information.

ARM 知识

Cortex-M 系列比较

Cortex-M comparison table

Feature	Cortex-M0	Cortex-M0+	Cortex-M1	Cortex-M23	Cortex-M3	Cortex-M4	Cortex-M33	Cortex-M35P	Cortex-M7
Instruction Set Architecture	Armv6-M	Armv6-M	Armv6-M	Armv8-M Baseline	Armv7-M	Armv7-M	Armv8-M Mainline	Armv8-M Mainline	Armv7-M
	Thumb, Thumb-2	Thumb, Thumb-2	Thumb, Thumb-2	Thumb, Thumb-2	Thumb, Thumb-2	Thumb, Thumb-2	Thumb, Thumb-2	Thumb, Thumb-2	Thumb, Thumb-2
DMIPS/MHz range*	0.87-1.27	0.95-1.36	0.8	0.98	1.25-1.89	1.25-1.95	1.5	1.5	2.14-3.23
CoreMark®/MHz*	2.33	2.46	1.85	2.64	3.34	3.42	4.02	4.02	5.01
Pipeline Stages	3	2	3	2	3	3	3	3	6
Memory Protection Unit (MPU)	No	Yes (option)	No	Yes (option) (2 x)	Yes (option)	Yes (option)	Yes (option) (2 x)	Yes (option) (2 x)	Yes (option)
Maximum MPU Regions	0	8	0	16	8	8	16	16	16
Trace (ETM or MTB)	No	MTB (option)	No	MTB (option) or ETMv3 (option)	ETMv3 (option)	ETMv3 (option)	MTB (option) and/or ETMv4 (option)	MTB (option) and/or ETMv4 (option)	ETMv4 (option)
Digital Signal Processing (DSP)	No	No	No	No	No	Yes	Yes (option)	Yes (option)	Yes
Floating Point Hardware	No	No	No	No	No	Yes (option SP)	Yes (option SP)	Yes (option SP)	Yes (option SP + DP)
Systick Timer	Yes (option)	Yes (option)	Yes (option)	Yes (2 x)	Yes	Yes	Yes (2 x)	Yes (2 x)	Yes

Built-in Caches	No	No	No	No	No	No	No	Yes (option 2-16kB I-cache	Yes (option 4-64kB I-cache, D-cache)
Tightly Coupled Memory	No	No	Yes	No	No	No	No	No	Yes (option 0-16MB I-TCM/D-TCM)
TrustZone for Armv8-M	No	No	No	Yes (option)	No	No	Yes (option)	Yes (option)	No
Co-processor Interface	No	No	No	No	No	No	Yes (option)	Yes (option)	No
Bus Protocol	AHB Lite	AHB Lite, Fast I/O	AHB Lite	AHB5, Fast I/O	AHB Lite, APB	AHB Lite, APB	AHB5, APB	AHB5, APB	AXI4, AHB Lite, APB, TCM
Wake-up Interrupt Controller Support	Yes	Yes	No	Yes	Yes	Yes	Yes	Yes	Yes
Integrated Interrupt Controller (NVIC)	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Maximum # External Interrupts	32	32	32	240	240	240	480	480	240
Hardware Divide	No	No	No	Yes	Yes	Yes	Yes	Yes	Yes
Single Cycle Multiply	Yes (option)	Yes (option)	No	Yes	Yes	Yes	Yes	Yes	Yes
CMSIS Support	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Dual Core Lock-Step Support	No	No	No	Yes	No	No	Yes	Yes	Yes

*See individual Cortex-M product pages for further information.

SP = Single Precision

DP = Double Precision

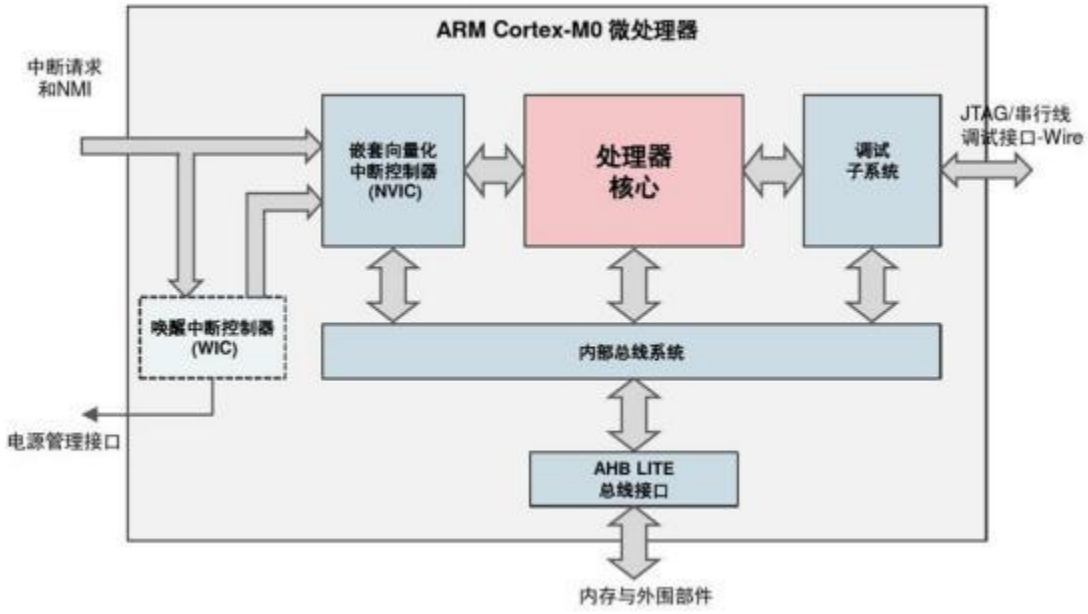
1． Arm 架构

ARM 公司的 Cortex-M0 应用于各种微控制器（MCU）中，并可让研发工程师以 8 位的价位创造 32 位的的效能，并将传统的8 位和 16 位的处理器升级到更高效、更低功耗的 32 位处理器。

Cortex-M0 是 Cortex-M 家族中的 M 系列。最大特点是低功耗的设计。

Cortex-M0 为 32 位、3 级流水线 RISC 处理器，其核心仍为冯.诺依曼结构，是指令和数据共享同一总线的架构。

作为新一代的处理器，Cortex-M0 的设计进行了许多的改革与创新，如系统存储器地址映像(system address map)、改善效率并增强确定性的嵌套向量中断系统(NVIC)与不可屏蔽中断(NMI)、全新的硬件除错单元等等，都带给了使用者全新的体验和更便利、 更有效率的操作。



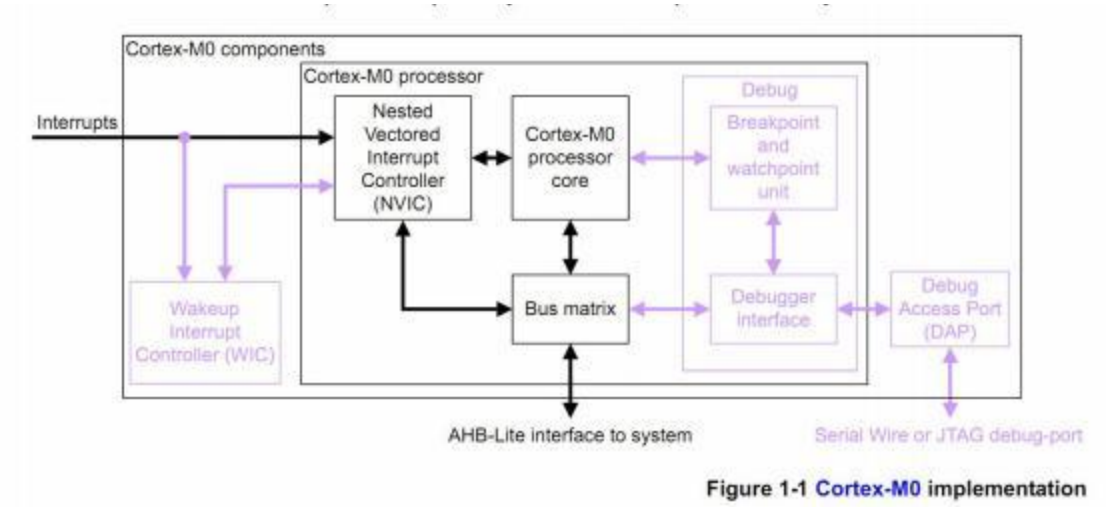


Figure 1-1 Cortex-M0 implementation

对比下面 M0+结构

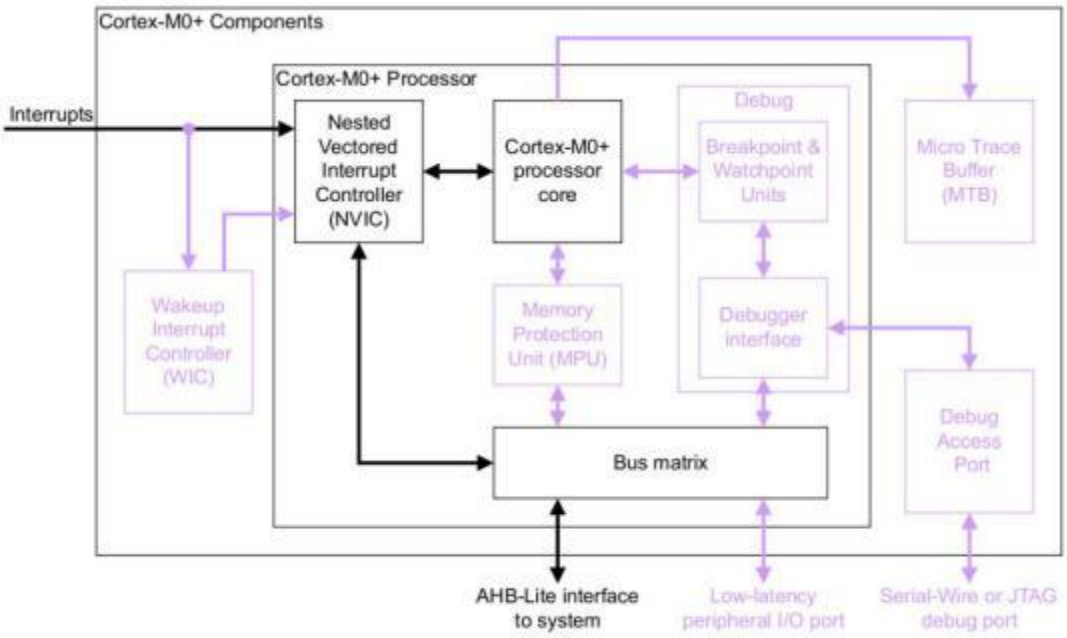


Figure 1-1 Cortex-M0+ implementation

MTB 与 M0+连接关系

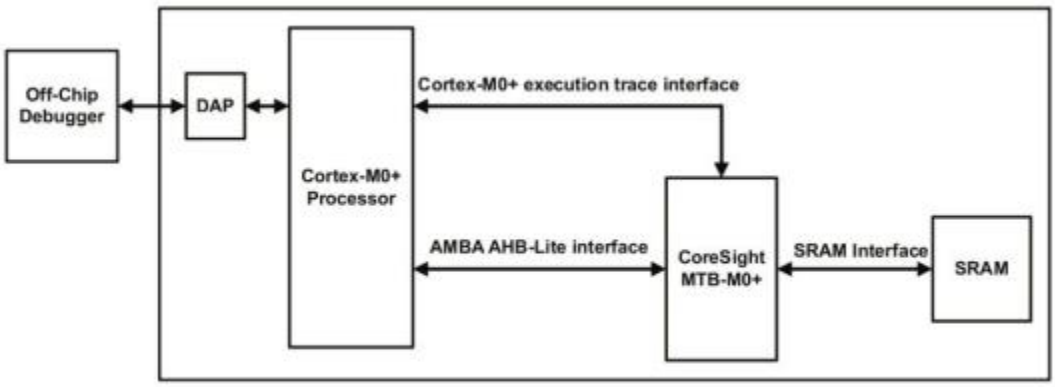
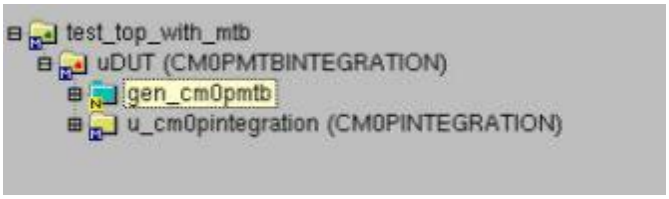


Figure 2-1 MTB system diagram

MTB SRAM can be used for both trace and general purpose storage by the processor.

rtl 集成层次:在实际集成过程, DAP 的层次是在 Processor 里面,MTB 与 PROCESSOR 在同一层次.



mtb 例化

```
CM0PMTB
  #(.RAR(RAR), .AWIDTH(AWIDTH), .USER(USER))
u_cm0pmtb
  (// Outputs
   .HRDATA                      (HRDATAMTB),
   .HREADYOUT                   (HREADYOUTMTB),
   .HRESP                       (HRESPMTB),
   .RAMAD                       (RAMAD[AWIDTH-3:0]),
   .RAMWD                       (RAMWD[31:0]),
   .RAMCS                       (RAMCS),
   .RAMWE                       (RAMWE[3:0]),
   .EDBGRQ                      (EDBGRQMTB),
   .IDLE                        (IDLEMTB),

   // Inputs
   .HCLK                        (HCLKMTB),
   .RESETn                      (PORESETn),
   .HADDR                       (HADDRMTB[31:0]),
   .HBURST                      (3'b000),
   .HMASTLOCK                   (1'b0),
   .HPROT                       (HPROTMTB),
   .HSIZE                       (HSIZEMTB[2:0]),
   .HTRANS                      (HTRANSMTB[1:0]),
   .HWDATA                      (HWDATAMTB[31:0]),
   .HWRITE                      (HWRITEMTB),
   .HSELRAM                     (HSELRAM),
   .HSELSFR                     (HSELSFR),
   .HREADY                      (HREADYMTB),

   .RAMRD                       (RAMRD),
   .IAEXEN                      (IAEXEN),
   .IAEXSEQ                     (IAEXSEQ),
   .IAEX                        (IAEX),
   .ATOMIC                      (ATOMIC),
   .TSTART                      (TSTART),
   .TSTOP                       (TSTOP),
   .NIDEN                       (NIDEN),
   .DBGEN                       (DBGEN),
   .SRAMBASEADDR                (SRAMBASEADDR),
   .ECOREVNUM                   (ECOREVNUM[31:28]));
```

SRAM

AHB

TRACE

MTB 接口描述:

Figure 2-2 shows the interface details.

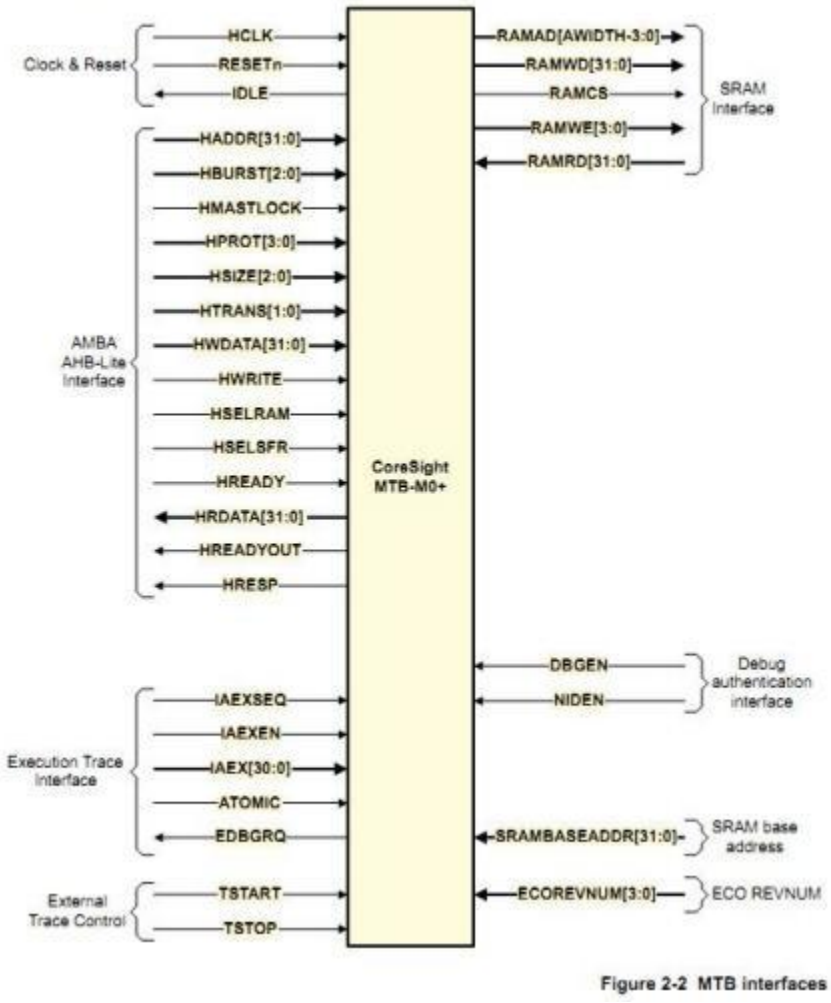


Figure 2-2 MTB interfaces

Table A-6 Debug authentication interface signals

Signal	Direction	Description
DBGEN	Input	Invasive debug enable.
NIDEN	Input	Non-invasive debug enable.

M0+可以有以下几种集成结构:

The Cortex-M0+ processor supports the following levels of hierarchy for integration or implementation:

- Processor component level, CORTEXM0PLUS.
- Processor, DAP and *Wakeup Interrupt Controller* (WIC) integration level, CM0PINTEGRATION.
- Processor, DAP, WIC, and *Micro Trace Buffer* (MTB) integration level, CM0PMTBINTEGRATION.
- Processor, DAP, WIC, MTB, and CTI level, CORTEXM0PLUSINTEGRATIONCS.

M0+集成框图

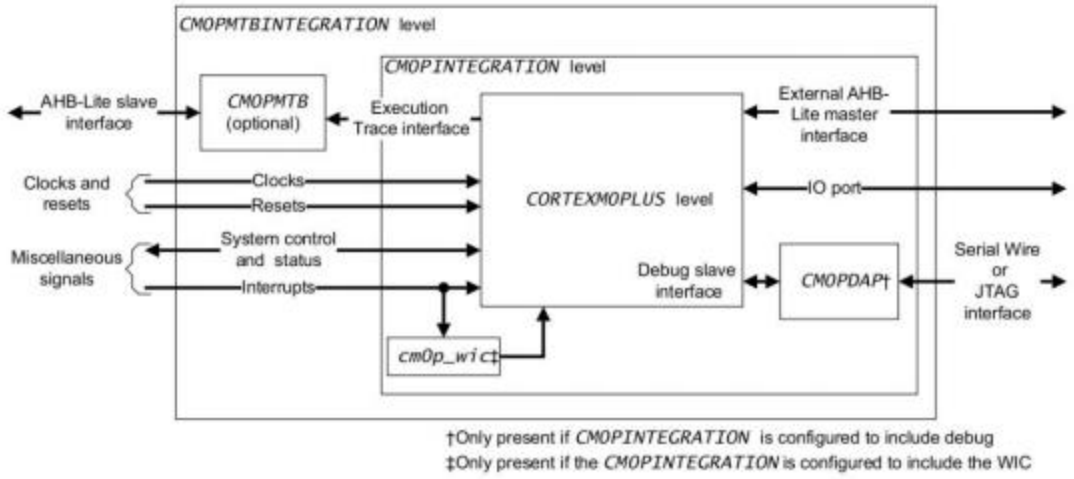
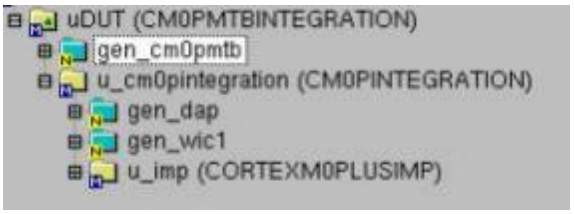
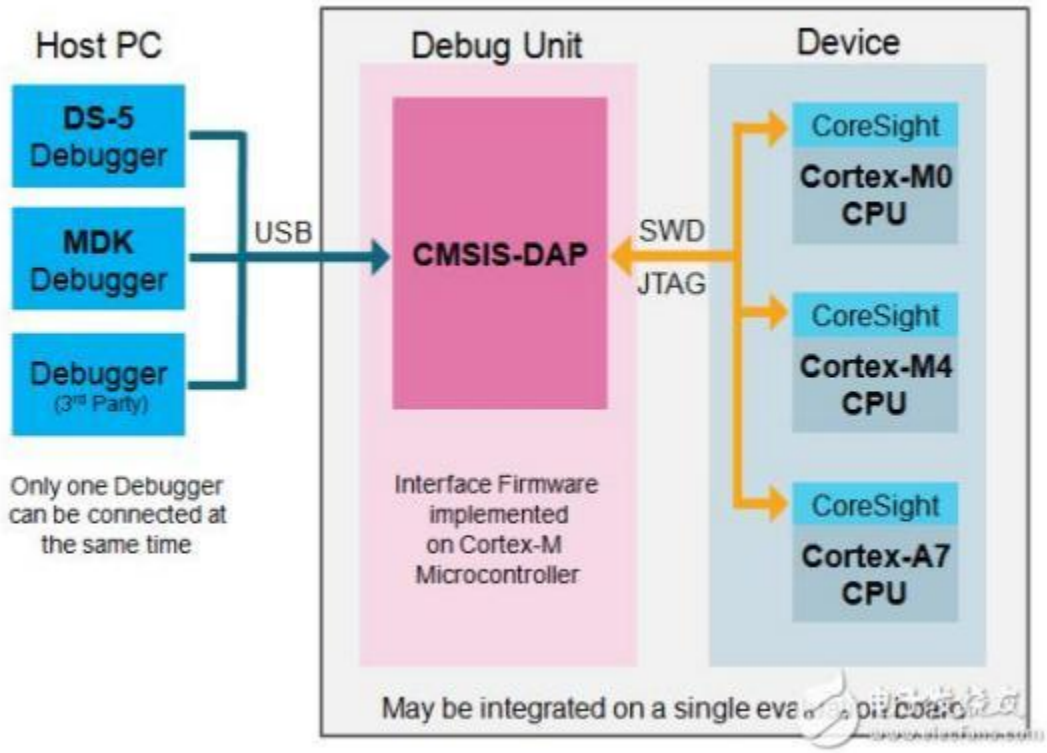


Figure 1-1 Module hierarchy and the main interfaces

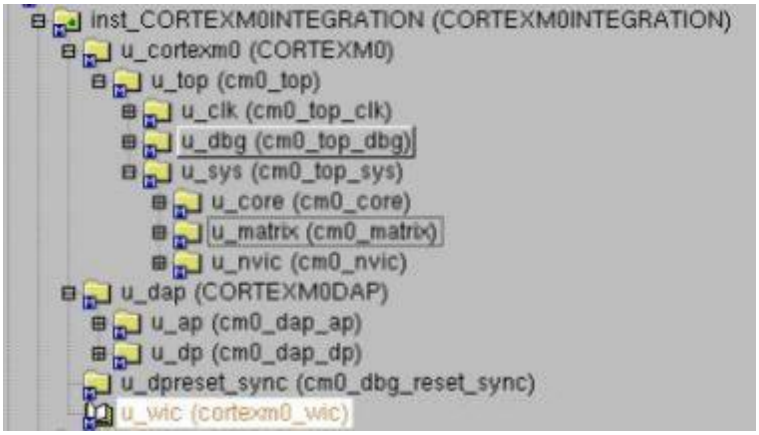
对应 RTL 层次:



对于 MTB(下面的 CoreSight)的使用如下



ARM 提供的 RTL 就是按照上述结构分层.如下:



Cortex-M 系列规格对比

类别	M0	M3	M4	M7
体系结构	ARMv6M（冯诺依曼）	ARMv6M（哈佛）	ARMv6M（哈佛）	ARMv7-M（哈佛）
ISA 支持	Thumb,Thumb-2	Thumb,Thumb-2	Thumb,Thumb-2	Thumb,Thumb-2
DSP 扩展			单周期 16/32 位 MAC 单周期双 16 位 MAC 8/16 位 SIMD 运算 硬件除法（2-12 周期）	单周期 16/32 位 MAC 单周期双 16 位 MAC 8/16 位 SIMD 运算 硬件除法（2-12 周期）
浮点单元			单精度浮点单元 符合 IEEE 754	单和双精度浮点单元 与 IEEE 754 兼容
流水线	3级	3级	3级+分支预测	6级超标量 + 分支预测
DMIPS/MHz	0.9~0.99	1.25~1.50	1.25~1.52	2.14/2.55/3.23
中断	NMI+1-32 物理中断	NMI+1-240 物理中断	NMI+1~240 物理中断	NMI+1~240 物理中断
中断优先级		8~256	8~256	8~256
唤醒中断控制器		最多 240 个	最多 240 个	最多 240 个
内存保护		带有子区域和后台区域的可选 8 区域 MPU	带有子区域和后台区域的可选 8 区域 MPU	可选的 8/16 区域 MPU，带有子区域和背景区域
睡眠模式	集成的 WFI 和 WFE 指令和“退出时睡眠”功能。 睡眠和深度睡眠信号随 ARM 电源管理工具包提供的可选的 Retention 模式	集成的 WFI 和 WFE 指令和“退出时睡眠”功能。 睡眠和深度睡眠信号。随 ARM 电源管理工具包提供的可选保留模式	集成 WFI 和 WFE 指令和“退出时睡眠”功能。 睡眠和深度睡眠信号。 随 ARM 电源管理工具包提供的可选 Retention 模式	集成 WFI 和 WFE 指令以及 Sleep On Exit 功能。 休眠和深度休眠信号。 ARM 电源管理工具包及可选 Retention 模式
增强的指令	硬件单周期 (32x32) 乘法选项	硬件除法（2-12 个周期）和单周期 (32x32) 乘法、饱和数学支持。		
调试	可选 JTAG 和 Serial-Wire 调试端口。最多 4 个断点和 2 个观察点	可选 JTAG 和串行线调试端口。最多 8 个断点和 4 个检测点。	可选 JTAG 和 Serial-Wire 调试端口。最多 8 个断点和 4 个检测点。	可选的 JTAG 和串行线调试端口。最多 8 个断点和 4 个观察点。
跟踪		可选指令跟踪 (ETM)、数据跟踪 (DWT) 和测量跟踪 (ITM)	可选指令跟踪 (ETM)、数据跟踪 (DWT) 和测量跟踪 (ITM)	可选指令跟踪 (ETM)、数据跟踪 (DWT) 和测量跟踪 (ITM)

注:M0+的流水只有 2 级,这样有利于降低功耗.

ARM Cortex-M 架构系列

处理器	ARM 架构	核心架构	Thumb®	Thumb®-2	硬件乘法	硬件除法	饱和运算	DSP 扩展	浮点运算
Cortex-M0	ARMv6-M	冯•诺伊曼	大多数	子集	1 或32个周期	无	无	无	无
Cortex-M0+	ARMv6-M	冯•诺伊曼	大多数	子集	1 或32个周期	无	无	无	无
Cortex-M1	ARMv6-M	冯•诺伊曼	大多数	子集	3 或33 个周期	无	无	无	无
Cortex-M3	ARMv7-M	哈佛	完整	完整	1 个周期	有	有	无	无
Cortex-M4	ARMv7E-M	哈佛	完整	完整	1个周期	有	有	有	可选

流水线使用 3 个阶段，因此指令分 3 个阶段执行。

⑴ 取指从存储器装载一条指令

⑵ 译码识别将要被执行的指令

⑶ 执行处理指令并将结果写会寄存器

无论处理器处于何种状态，程序计数器 R15(PC)总是指向“正在取指”的指令，而不是指向“正在执行”的指令或者正在“译码”的指令。人们一般会习惯性的将正在执行的指令作为参考点，即当前第 1 条指令。所以，PC 总是指向第 3 条指令，或者说 PC 总是指向当前正在执行的指令地址再加 2 条指令的地址

M0 的两种操作模式:线程模式—主程序,处理模式—异常程序

(主程序和异常程序(中断是异常程序的一种))

Cortex-M0 处理器包含两种操作模式和两种状态(见图 3.1)。

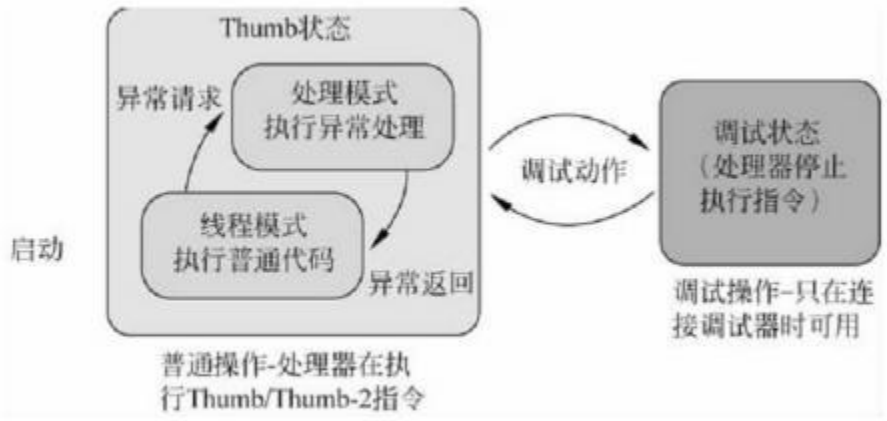


图 3.1　Cortex-M0 处理器的模式和状态

M0 没有特权模式,M3 分特权模式和操作模式

操作模式和特权极别

Cortex-M3 处理器支持两种处理器的操作模式，还支持两级特权操作。

两种操作模式分别为：处理者模式(handler mode，以后不再把 handler 中译——译注)和线程模式（thread mode）。引入两个模式的本意，是用于区别普通应用程序的代码和异常服务例程的代码——包括中断服务例程的代码。

Cortex-M3 的另一个侧面则是特权的分级——特权级和用户级。这可以提供一种存储器访问的保护机制，使得普通的用户程序代码不能意外地，甚至是恶意地执行涉及到要害的操作。处理器支持两种特权级，这也是一个基本的安全模型。

	特权级	用户级
异常handler的代码	handler模式	错误的用法
主应用程序的代码	线程模式	线程模式

图 2.4　Cortex-M3 下的操作模式和特权级别

M3 总线

Cortex-M3 内部有若干个总线接口，以使 CM3 能同时取址和访内（访问内存），它们是：

- 指令存储区总线（两条）
- 系统总线
- 私有外设总线

有两条代码存储区总线负责对代码存储区的访问，分别是 I-Code 总线和 D-Code 总线。前者用于取指，后者用于查表等操作，它们按最佳执行速度进行优化。

系统总线用于访问内存和外设，覆盖的区域包括 SRAM，片上外设，片外 RAM，片外扩展设备，以及系统级存储区的部分空间。

私有外设总线负责一部分私有外设的访问,主要就是访问调试组件。它们也在系统级存储区。

2． 程序入口

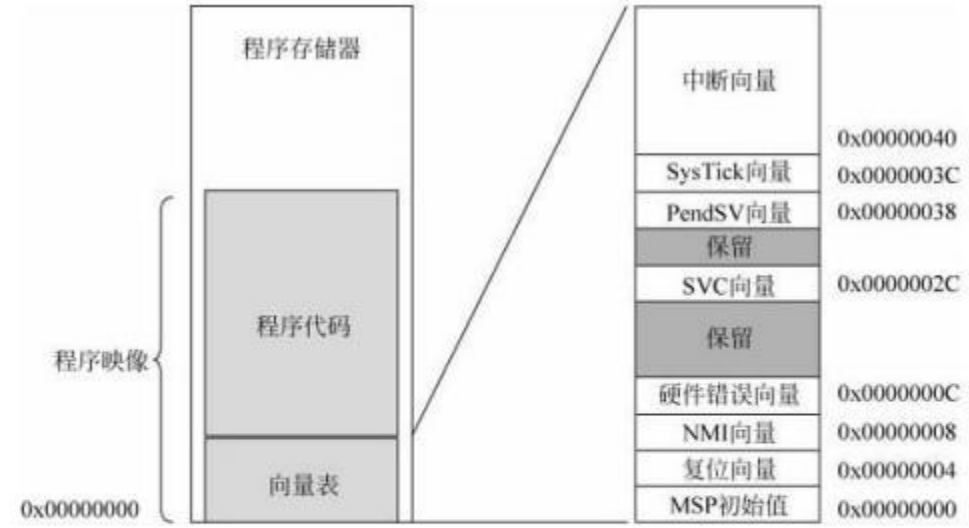


图 3.11 程序映像中的向量表

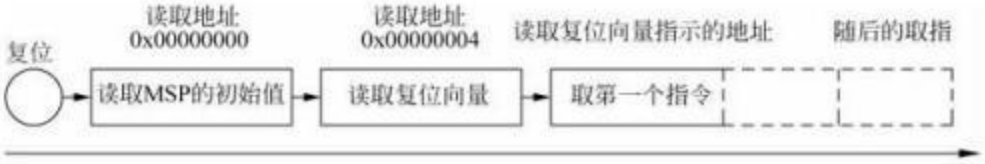


图 3.12 复位流程

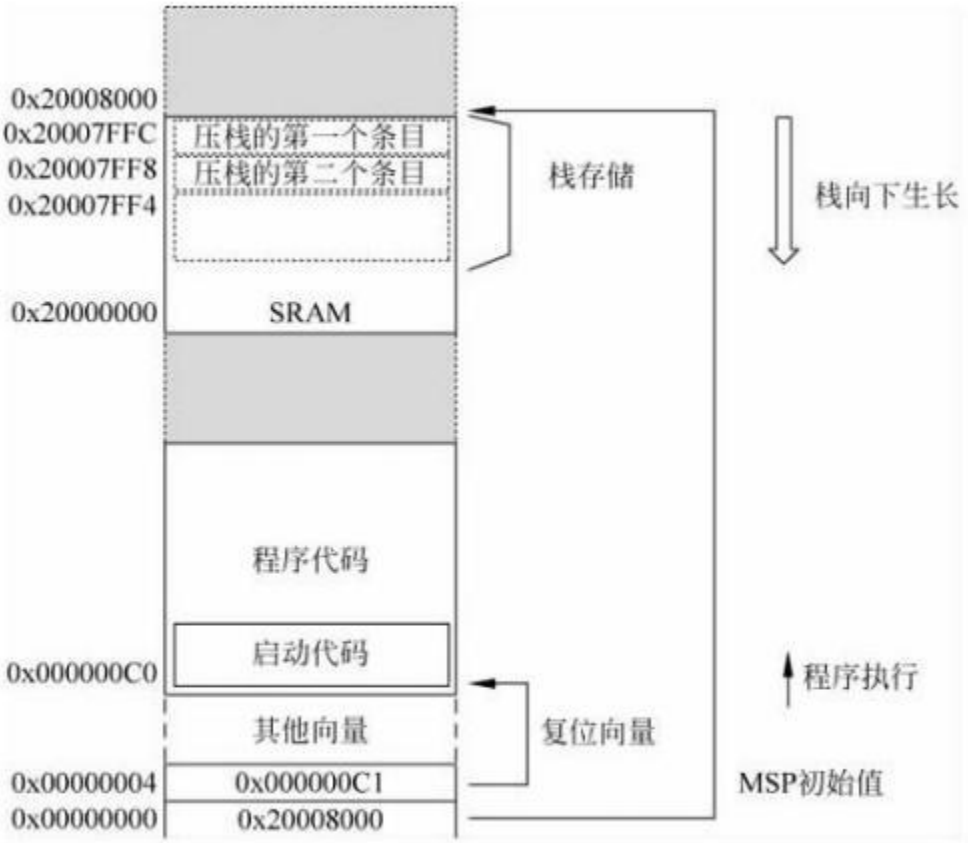
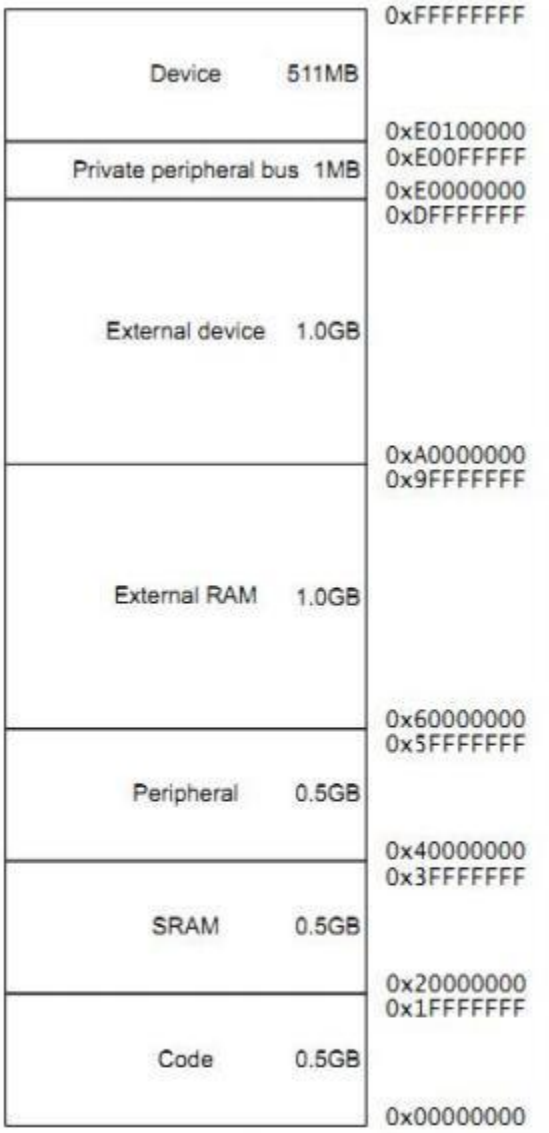


图 3.13　MSP 和 PC 初始化示例

如何初始化 MSP 值?　启动代码可以修改

3． 地址分配



The processor reserves regions of the *Private peripheral bus* (PPB) address range for core peripheral registers, see *About the Cortex-M0 processor and core peripherals* on page 1-3.

0xE000_0000 ~0xE00F_FFFF 用于M0 核处理器寄存器配置,比如调试相关的地址,
ROM Table is at 0xE00F_F003
设计的处理器的外设寄存器地址一般在 0x4000_0000~0x5fff_ffff 区间
Sram 需要支持可以执行程序,主要考虑固件升级时重影射
M0 不支持非字对齐操作

存储器属性
存储器访问属性：可执行(Executable)，可缓冲(Bufferable)，可缓存(Cacheable)，可共享(Shareable)。与 HPORT 一起分析
XN:不可执行

下面是 m0+的存储属性:
关键点在于对于存储的操作顺序不一定与指令的顺序一致,是因为处理器会重新排序(re-order).;如果想对存储空间的操作一定按照指令顺序执行,则在需要的地方使用壁垒指令(DMB,DSB,ISB)

The memory types are:

Normal	The processor can re-order transactions for efficiency, or perform speculative reads.
Device	The processor preserves transaction order relative to other transactions to Device or Strongly-ordered memory.
Strongly-ordered	The processor preserves transaction order relative to all other transactions.

The different ordering requirements for Device and Strongly-ordered memory mean that the memory system can buffer a write to Device memory, but must not buffer a write to Strongly-ordered memory.

The additional memory attributes include.

Shareable	For a shareable memory region, the memory system provides data synchronization between bus masters in a system with multiple bus masters, for example, a processor with a DMA controller. Strongly-ordered memory is always shareable. If multiple bus masters can access a non-shareable memory region, software must ensure data coherency between the bus masters. <This description is required only if the device is likely to be used in systems where memory is shared between multiple processors.>

***Execute Never* (XN)**

Means the processor prevents instruction accesses. A HardFault exception is generated on executing an instruction fetched from an XN region of memory.

2.2.3 Behavior of memory accesses

The behavior of accesses to each region in the memory map is:

Table 2-9 Memory access behavior

Address range	Memory region	Memory type[a]	XN[a]	Description
0x00000000-0x1FFFFFFF	Code	Normal	-	Executable region for program code. You can also put data here.
0x20000000-0x3FFFFFFF	SRAM	Normal	-	Executable region for data. You can also put code here.
0x40000000-0x5FFFFFFF	Peripheral	Device	XN	External device memory.
0x60000000-0x9FFFFFFF	External RAM	Normal	-	Executable region for data.
0xA0000000-0xDFFFFFFF	External device	Device	XN	External device memory.
0xE0000000-0xE00FFFFF	Private Peripheral Bus	Strongly-ordered	XN	This region includes the NVIC, System timer, and System Control Block. Only word accesses can be used in this region.
0xE0100000-0xFFFFFFFF	Device	Device	XN	Vendor specific.

a. See *Memory regions, types and attributes* on page 2-13 for more information.

4. M0 处理器内部寄存器

The address map of the *Private peripheral bus* (PPB) is:

Table 4-1 Core peripheral register regions

Address	Core peripheral	Description
0xE000E008-0xE000E00F	System Control Block	Table 4-10 on page 4-11
0xE000E010-0xE000E01F	Reserved	-
0xE000E010-0xE000E01F	System timer	Table 4-19 on page 4-20
0xE000E100-0xE000E4EF	Nested Vectored Interrupt Controller	Table 4-2 on page 4-3
0xE000ED00-0xE000ED3F	System Control Block	Table 4-10 on page 4-11
0xE000EF00-0xE000EF03	Nested Vectored Interrupt Controller	Table 4-2 on page 4-3

中断相关寄存器:

Table 4-2 NVIC register summary

Address	Name	Type	Reset value	Description
0xE000E100	ISER	RW	0x00000000	*Interrupt Set-enable Register* on page 4-4
0xE000E180	ICER	RW	0x00000000	*Interrupt Clear-enable Register* on page 4-5
0xE000E200	ISPR	RW	0x00000000	*Interrupt Set-pending Register* on page 4-5
0xE000E280	ICPR	RW	0x00000000	*Interrupt Clear-pending Register* on page 4-6
0xE000E400-0xE000E41C	IPR0-7	RW	0x00000000	*Interrupt Priority Registers* on page 4-7

使用方法:

To access the NVIC registers when using CMSIS, use the following functions:

Table 4-3 CMSIS access NVIC functions

CMSIS function	Description
`void NVIC_EnableIRQ(IRQn_Type IRQn)`[a]	Enables an interrupt or exception.
`void NVIC_DisableIRQ(IRQn_Type IRQn)`[a]	Disables an interrupt or exception.
`void NVIC_SetPendingIRQ(IRQn_Type IRQn)`[a]	Sets the pending status of interrupt or exception to 1.
`void NVIC_ClearPendingIRQ(IRQn_Type IRQn)`[a]	Clears the pending status of interrupt or exception to 0.
`uint32_t NVIC_GetPendingIRQ(IRQn_Type IRQn)`[a]	Reads the pending status of interrupt or exception. This function returns non-zero value if the pending status is set to 1.
`void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)`[a]	Sets the priority of an interrupt or exception with configurable priority level to 1.
`uint32_t NVIC_GetPriority(IRQn_Type IRQn)`[a]	Reads the priority of an interrupt or exception with configurable priority level. This function return the current priority level.

a. The input parameter `IRQn` is the IRQ number, see Table 2-11 on page 2-20 for more information.

注意:在使能中断和清除中断是在不同的寄存器对对应位置写 1.

Cortex-M0 的寄存器组中，包含了 13 个 32 位的通用目的寄存器，以及多个特殊寄存器（见图 3.2）。

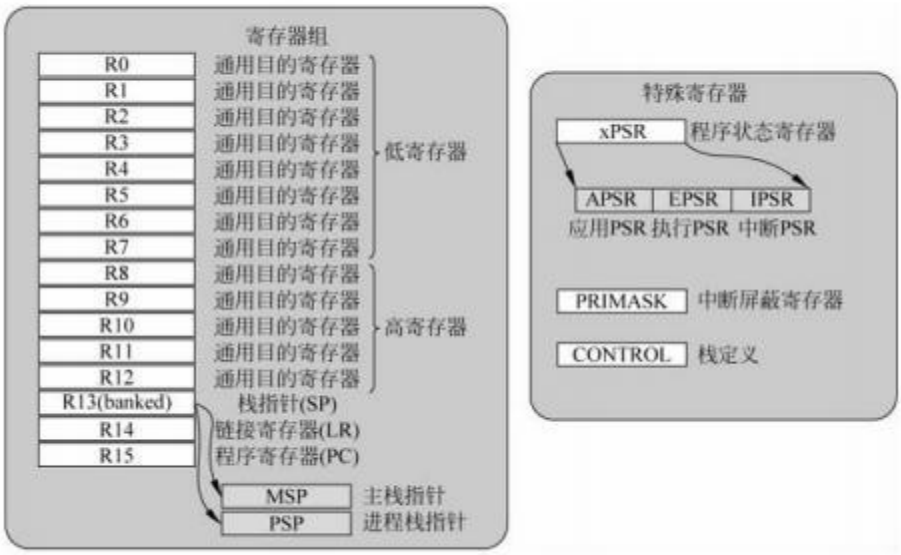


图 3.2　Cortex-M0 的寄存器

5．　M0 中断状态

2.3.1　Exception states

Each exception is in one of the following states:

Inactive　The exception is not active and not pending.

Pending　The exception is waiting to be serviced by the processor.

An interrupt request from a peripheral or from software can change the state of the corresponding interrupt to pending.

Active　An exception that is being serviced by the processor but has not completed.

Note

An exception handler can interrupt the execution of another exception handler. In this case both exceptions are in the active state.

Active and pending

The exception is being serviced by the processor and there is a pending exception from the same source.

Inactive:没响应中断
Pending:响应了中断但还没有进中断处理程序
Active:进入了中断处理程序
Active and pending ：正在中断服务程序,但此中断号又来了一个中断
进入中断程序的时间是固定的.

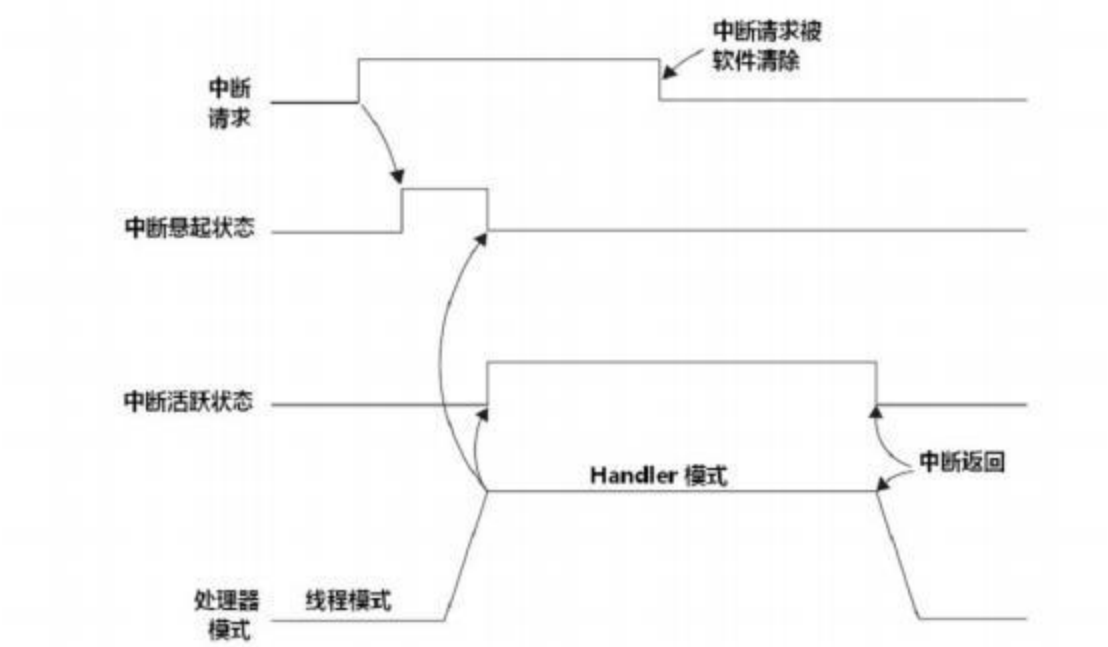


图 7.10 在处理器进入服务例程后对中断活跃状态的设置

6． M0 中断入口地址

中断向量总共有 15+32=47 个,向量表占 47*4byte=188byte,但最低位被 MSP 占用,所以前面有 188byte+4byte=192byte 空间被占用,0~0xbf,这段空间被称为向量表 .但我们在设置向量偏移时,都是以 256byte 为单位来偏移(地址才好对齐,中断向量寄存器的低 7BIT 固定为 0).

M0+支持 4 级中断优先级(每个中断的优先级配置位有 8BIT,但只有最高 2BIT 有效);

在进入中断时,硬件会做 8 个寄存器的现场保护(xpsp,pc,lr,r12,r3~r0),也就是 sp 指针会减去 32byte,如果在进入中断还需要保护其它寄存器,则由编译器生成代码,有软件来进行保护剩下的现场信息.

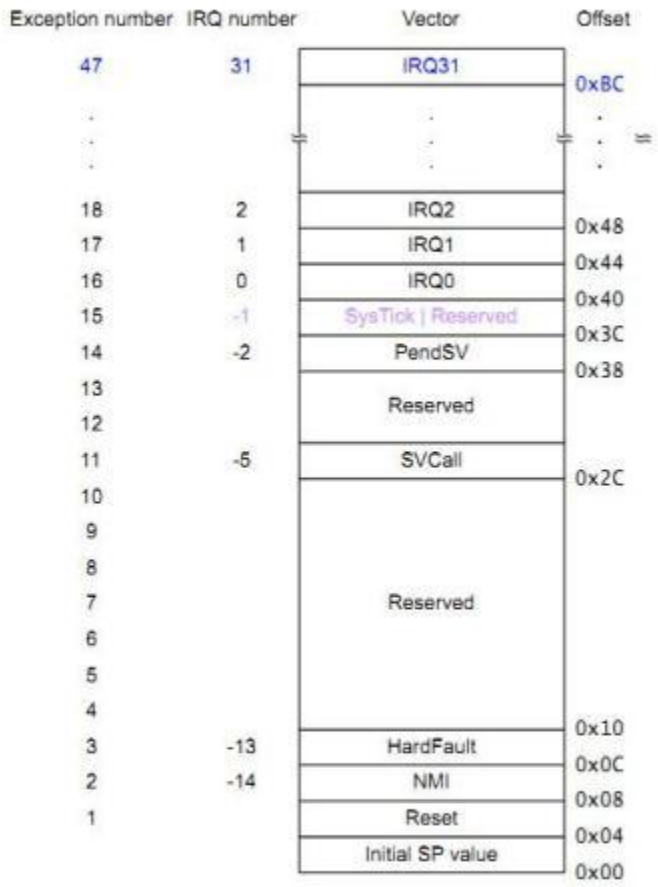


Figure 2-1 Vector table

The vector table is fixed at address 0x00000000.

M0 的中断入口地址的基地址固定在 0x0;M0+核的中断入口地址可以通过寄存器配置(地址为 0XE000ED08)

程序映像的开始处为向量表(见图 3.11),其中包含了异常的起始地址(向量),每个中断向量的地址都等于"异常号×4"。例如,外部 IRQ≠0 的异常类型为 16,因此 IRQ≠0 的向量地址为 16×4=0x40。这些向量的最低位都被置 1,表明异常处理执行时使用 Thumb 指令。向量表的大小由实际使用的中断个数决定。

7． Arm m0 指令集

处理器使用的是 ARMv6-M Thumb 指令集，包括大量的 32 位的使用 Thumb-2 技术的指令。表 7-22 列出了Cortex-M0指令和它们的周期数。周期计数以零等待状态的系统为基准。

指令　目的操作数　源操作数

操　作	描　述	汇编指令	周　期

MOVE	8-bit immediate	MOVS Rd,#<imm>	1
	Lo to Lo	MOVS Rd,Rm	1
	Any to Any	MOV Rd,Rm	1
	Any to PC	MOV PC,Rm	3
Add	3-bit immediate	ADDS Rd,Rn,#<imm>	1
	All registers Lo	ADDS Rd,Rn,Rm	1
Add	Any to Any	ADD Rd,Rd,Rm	1
	Any to PC	ADD PC,PC,Rm	3
	8-bit immediate	ADDS Rd,Rd,#<imm>	1
	With carry	ADCS Rd,Rd,Rm	1
	Immediate to SP	ADD SP,SP,#<imm>	1
	Form address from SP	ADD Rd,SP ,#<imm>	1
	Form address from PC	ADR Rd,<label>	1
Subtract	Lo to Lo	SUBS Rd,Rn,Rm	1
	3-bit immediate	SUBS Rd,Rn,#<imm>	1
	8-bit immediate	SUBS Rd,Rd,#<imm>	1
	With carry	SBCS Rd,Rd,Rm	1

	Immediate from SP	SUB SP, SP, #<imm>	1
	Negate	RSBS Rd, Rn, #0	1
Multiply	Multiply	MULS Rd, Rm, Rd	1 或 32
Compare	Compare	CMP Rn, Rm	1
	Negative	CMP Rn, Rm	1
	Immediate	CMP Rn, #<imm>	1
Logical	AND	ANDS Rd, Rd, Rm	1
	Exclusive OR	EORS Rd, Rd, Rm	1
	OR	ORRS Rd, Rd, Rm	1
	Bit clear	BICS Rd, Rd, Rm	1
	Move NOT	MVNS Rd, Rm	1
	AND test	TST Rn, Rm	1
Shift	Logical shift left by immediate	LSLS Rd, Rm, #<shift>	1
	Logical shift left by	LSLS Rd, Rd, Rs	1

	register		
	Logical shift right by immediate	LSRS Rd, Rm, #<shift>	1
	Logical shift right by register	LSRS Rd, Rd, Rs	1
	Arithmetic shift right	ASRS Rd, Rm, #<shift>	1
	Arithmetic shift right by register	ASRS Rd, Rd, Rs	1
Rotate	Rotate right by register	RORS Rd, Rd, Rs	1
Load	Word, immediate offset	LDR Rd, [Rn, #<imm>]	2
	Halfword, immediate offset	LDRH Rd, [Rn, #<imm>]	2
	Byte, immediate offset	LDRB Rd, [Rn, #<imm>]	2
	Word, register offset	LDR Rd, [Rn, Rm]	2

续表

操　作	描　述	汇编指令	周　期
	Halfword, register offset	LDRH Rd, [Rn, Rm]	2
	Signed halfword, register offset	LDRSH Rd, [Rn, Rm]	2
	Byte, register offset	LDRB Rd, [Rn, Rm]	2
	Signed byte, register offset	LDRSB Rd, [Rn, Rm]	2
	PC-relative	LDR Rd, <label>	2
	SP-relative	LDR Rd, [SP, #<imm>]	2
	Multiple, excluding base	LDM Rn!, {<loreg list>}	1+N
	Multiple, including base	LDM Rn, {<loreg list>}	1+N
Store	Word, immediate offset	STR Rd, [Rn, #<imm>]	2
	Halfword, immediate	STRH Rd, [Rn,	2

	offset	#<imm>]	
	Byte, immediate offset	STRB Rd, [Rn, #<imm>]	2
	Word, register offset	STR Rd, [Rn, Rm]	2
	Halfword, register offset	STRH Rd, [Rn, Rm]	2
	Byte, register offset	STRB Rd, [Rn, Rm]	2
	SP-relative	STR Rd, [SP, #<imm>]	2
	Multiple	STM Rn!, {<loreglist>}	1+N
Push	Push	PUSH {<loreglist>}	1+N
	Push with link register	PUSH {<loreglist>, LR}	1+N
Pop	Pop	POP {<loreglist>}	1+N
	Pop and return	POP {<loreglist>, PC}	4+N
Branch	Conditional	B<cc> <label>	1 或 3

	Unconditional	B<label>	3
	With link	BL<label>	4
	With exchange	BX Rm	3
	With link and exchange	BLX Rm	3
Extend	Signed halfword to word	SXTH Rd, Rm	1
	Signed byte to word	SXTB Rd, Rm	1
	Unsigned halfword	UXTH Rd, Rm	1
	Unsigned byte	UXTB Rd, Rm	1
Reverse	Bytes in word	REV Rd, Rm	1
	Bytes in both halfwords	REV16 Rd, Rm	1
	Signed bottom half word	REVSH Rd, Rm	1
State change	Supervisor Call	SVC #<imm>	-
	Disable interrupts	CPSID i	1

操 作	描 述	汇编指令	周 期

	Enable interrupts	CPS IE i	1
	Read special register	MRS Rd, <specreg>	4
	Write special register	MSR <specreg>, Rn	4
	Breakpoint	BKPT #<imm>	__
Hint	Send event	SEV	1
	Wait for event	WFE	2
	Wait for interrupt	WFI	2
	Yield	YIELD	1
Hint	No operation	NOP	1
Barriers	Instruction synchronization	ISB	4
	Data memory	DMB	4
	Data synchronization	DSB	4

ISO/IEC 的 C 代码不能直接地获取一些 Cortex-MO 的指令。表 7-23 列举了CMS IS 的 C 编译器中提供的部分内部函数用于产生这些指令。如果一个 C 编译器不支持恰当的内部函数，则需要用内嵌汇编来获取有关指令。

表 7-23 CMSIS 内部函数

指 令	CMSIS 内部函数	指 令	CMSIS 内部函数
CPSIE i	void __enable_irq(void)	REV	uint32_t __REV(uint32_t int value)
CPSID i	void __disable_irq(void)	REV16	uint32_t __REV16(uint32_t int value)
ISB	void __ISB(void)	REVSH	uint32_t __REVSH(uint32_t int value)
DSB	void __DSB(void)	SEV	void __SEV(void)
DMB	void __DMB(void)	WFE	void __WFE(void)
NOP	void __NOP(void)	WFI	void __WFI(void)

CMSIS 也提供了几个函数用于获取特殊的寄存器，如表 7-24 所示。

表 7-24 CMSIS 提供的用于获取特殊寄存器的函数

特殊寄存器	获取方式	CMSIS 函数
PRIMASK	读	uint32_t __get_PRIMASK(void)
	写	void __set_PRIMASK(void)
CONTROL	读	Uint32_t __get_CONTROL(uint32_t value)

	写	Void __set_CONTROL(uint32_t value)
MSP	读	Uint32_t __get_MSP(void)
	写	Void __set_MSP(uint32_t TopOfMainStack)
PSP	读	Uint32_t __get_PSP(void)
	写	Void __set_PSP(uint32_t TopOfMainStack)

8． Ready 信号处理

Ahb 总线工作的前提条件是 ready 握手信号为高，所以当核要启动时，ready 信号在复位时默认就是高，对于普通的 slaver，ahb 只是用来配置寄存器，这种行为是当拍完成，所以这种 slaver 的返回 ready 信号可以一直为 1，如果是像flash 控制器这种复杂控制，它当拍是无法完成的，那么它就必须拉低 ready 信号，等完成当前行为再释放 ready 为高，为了流水，需要在完成的当拍把 ready 信号拉高。

9． 流水线知识(转)

本文将讨论处理器的一个重要的基础知识：“流水线 ”。熟悉计算机体系结构的读者一定知道，言及处理器微架构，几乎必谈其流水线。处理器的流水线结构是处理器微架构最基本的一个要素，犹如汽车底盘对于汽车一般具有基石性的作用，它承载并决定了处理器其他微架构的细节。本文将简要介绍处理器的一些常见流水线结构，让您真正读懂处理器流水线。

本文引用地址:

http://www.21ic.com/embed/hardware/processor/201804/51558.html

1 从经典的五级流水线说起

流水线的概念来源于工业制造领域，以汽车装配为例来解释流水线的工作方式，假设装配一辆汽车需要四个步骤：

第一步冲压：制作车身外壳和底盘等部件。

第二步焊接：将冲压成形后的各部件焊接成车身。

第三步涂装：将车身等主要部件清洗、化学处理、打磨、喷漆和烘干。

第四步总装：将各部件(包括发动机和向外采购的零部件)组装成车。

汽车装配则同时对应需要冲压、焊接、涂装和总装四个工人。最简单的方法是一辆汽车依次经过上述四个步骤装配完成之后，下一辆汽车才开始进行装配，最早期的工业制造就是采用的这种原始的方式，即同一时刻只有一辆汽车在装配。不久之后人们发现，某个时段中一辆汽车在进行装配时，其它三个工人都处于闲置状态，显然这是对资源的极大浪费，于是思考出能有效利用资源的新方法，即在第一辆汽车经过冲压进入焊接工序的时候，立刻开始进行第二辆汽车的冲压，而不是等到第一辆汽车经过全部四个工序后才开始，这样在后续生产中就能够保证四个工人一直处于运行状态，不会造成人员的闲置。这样的生产方式就好似流水川流不息，因此被称为流水线。

计算机体系结构教材中被提及最多的经典MIPS 五级流水线如图 1 所示。在此流水线中一条指令的生命周期分为：

取指：

指令取指(Instruction Fetch)是指将指令从存储器中读取出来的过程。

译码：

指令译码(Instruction Decode)是指将存储器中取出的指令进行翻译的过程。经过译码之后得到指令需要的操作数寄存器索引，可以使用此索引从通用寄存器组(Register File，Regfile)中将操作数读出。

执行：

指令译码之后所需要进行的计算类型都已得知，并且已经从通用寄存器组中读取出了所需的操作数，那么接下来便进行指令执行(Instruction Execute)。指令执行是指对指令进行真正运算的过程。譬如，如果指令是一条加法运算指令，则对操作数进行加法操作;如果是减法运算指令，则进行减法操作。

在“执行”阶段的最常见部件为算术逻辑部件运算器(Arithmetic Logical Unit，ALU)，作为实施具体运算的硬件功能单元。

访存：

存储器访问指令往往是指令集中最重要的指令类型之一，访存(Memory Access)是指存储器访问指令将数据从存储器中读出，或者写入存储器的过程。

写回：

写回(Write-Back)是指将指令执行的结果写回通用寄存器组的过程。如果是普通运算指令，该结果值来自于“执行”阶段计算的结果;如果是存储器读指令，该结果来自于“访存”阶段从存储器中读取出来的数据。

在工业制造中采用流水线可以提高单位时间的生产量，同样在处理器中采用流水线设计也有助于提高处理器的性能。以上述的五级流水线为例，由于前一条指令在完成了“取指”进入“译码”阶段后，下一条指令马上就可以进入“取指”阶段，依次类推，如图 2 所示，如果流水线没有停顿，理论上可以取得每个时钟周期都完成一条指令的性能。

图 1 MIPS 五级流水线结构图

图 2 MIPS 五级流水线运行图

2 可不可以不要流水线——流水线和状态机的关系

言及处理器微架构，几乎必谈流水线。那么，我们能否挑战一下权威提出一个有意思的问题：处理器难道就一定需要流水线吗?可否不要流水线呢?

在回答这个问题之前，我们先探讨下流水线的本质：

流水线并不限于处理器设计，在所有的ASIC 电路实现中都广泛采用流水线的思想。流水线本质上可以理解为是一种以面积换性能(Trade Area for Performance)、以空间换时间(Trade Space for Timing)的手段。

譬如，以 5 级流水线为例，其增加了 5 组寄存器，每一个流水线级数内部都有各自的组合逻辑数据通路，彼此之间没有复用资源，因此，其面积开销是比较大的，

但是由于可以让不同的流水线级数同时做不同的事情，而达到流水的效果，提高了性能，优化了时序，增加了吞吐率。

与流水线相对应的另外一种策略是状态机，状态机是流水线的“取反”，同样在所有的ASIC 电路实现中都广泛采用。状态机本质上可以理解为是一种以性能换面积(Trade Performance for Area)、以时间换空间(Trade Timing for Space)的手段。

“流水线”和“状态机”的关系，还有一种说法称之为“展开”和“折叠”的关系。本质上都是一种电路设计时，选择侧重时间(性能)还是空间(面积)的一种取舍。

通过上述分析，假设处理器不采用流水线，而是使用一个状态机来完成，则需要多个时钟周期才能完成一条指令的所有操作，每个时钟周期完成状态机的一个状态(譬如分别为取指、译码、执行、访存和写回)。通过使用状态机，可以省掉上述流水线中的寄存器开销，还可以复用组合逻辑数据通路，因此面积开销比较小，但是每条指令都需要 5 个周期才能完成，吞吐率和性能很差。

谈及此处，就不得不提及 8 位单片机时代的传奇老炮儿 8051 内核，早期原始的 8051 内核微架构就是采用了类似状态机的实现方式而不是流水线。因此，回到最开始我们提出的问题，处理器可否不要流水线，答案是：当然可以，传奇老炮儿 8051 内核就没有流水线。

所以说从功能能上来讲，处理器完全可以不使用流水线，而使用状态机的方式来实现，只不过由于这种方式性能比较差，在现代处理器设计中比较罕见而已。

3 深处种菱浅种稻，不深不浅种荷花——流水线的深度

流水线的级数(又称深度)多少最好呢?要回答这个问题，就需要了解流水线的深浅各自的优劣。此处有一个常见面试题，题目便是：处理器的流水线是否越深越好?在此我们给出答案：

早期的经典流水线是 5 级流水线，分别为取指、译码、执行、访存和写回。现代的处理器往往具有极深的流水线级数，譬如高达十几级，或者二十几级的深度。流水线就像一根黄瓜，切五刀下去得到的每一截长度和切二十到下去得到的每一截长度肯定是不一样的。当流水线的级数越多，那么意味着流水线被切的很细，每一级流水线内容纳的硬件逻辑便越少，熟悉数字同步电路设计的读者应该比较

熟悉，在两级寄存器(每一级流水线由寄存器组成)之间的硬件逻辑越少，则意味能够运行到更高的主频。因此现代的处理器流水线极深主要是由于处理器追求高频的指标所驱使，高端的 ARM Cortex-A 系列由于有十几级的流水线，所以能够运行到高达 2GHz 的主频，而 Intel 的 x86 处理器甚至采用几十级的流水线深度将主频推到 3-4GHz 的高度。主频越高也意味着流水线的吞吐率越高从而性能越高，这是流水线加深的正面意义。

由于每一级流水线都由寄存器组成，那么意味着更多的流水线级数要消耗更多的寄存器，也意味着更多的面积开销。这是流水线加深的负面意义。

同时流水线越深，由于每一级流水线需要进行握手，流水线最后一级的反压信号可能会一直串扰到最前一级造成严重的时序问题，需要使用一些比较高级的技巧来解决此类反压时序问题。这是流水线加深的负面意义。

较深的处理器流水线还有一个问题，由于在流水线的取指令阶段无法得知条件跳转的结果是跳还是不跳，因此只能进行预测，而到了流水线的末端才能够通过实际的运算得知该分支是真的该跳还是不该跳，如果发现真实的结果(譬如该跳)与之前预测的结果(譬如预测为不跳)不相符，则意味着预测失败，需要将所有预取的错误指令流全部丢弃掉，而重新取正确的指令流，这个过程叫做流水线冲刷(Pipeline Flush)，虽然可以使用分支预测器来保证前期的分支预测尽可能的准确，但是也无法做到万无一失。那么，流水线的深度越深，则意味着已经预取了很多的错误指令流，需要将其全部抛弃然后重启，不仅白白的浪费了功耗，还造成了性能的损失。流水线越深则意味着浪费和损失越严重，流水线越浅则浪费和损失越少。这是流水线加深的另一个主要的负面意义。

综上，所谓深处种菱浅种稻，不深不浅种荷花，流水线的不同深度皆有其优缺点，需要根据不同的应用背景合理地进行选择。

由于处理器流水线深浅的不同优劣，根据不同的应用场景，当今处理器的流水线深度在向着两个不同的极端发展，一方面级数越来越深，另一方面又越来越浅，下面我们结合不同的商用处理器例子予以探讨。

4 向上生长——越来越深的流水线

现代的高性能处理器相比最早期的处理器明显存在着流水线越来越深的现象，其驱动因素很简单，那就是追求更高的主频以获取更高的吞吐率和性能。

以最知名的 ARM Cortex-A 系列处理器 IP 为例，Cortex-A7 主打的低功耗前提下的能效比，其流水线级数为8 级;而 Cortex-A15 主打高性能，其流水线深度为 15 级。

当然流水线越来越深也需有其限度，曾有某些商业处理器产品一味地追求极端流水线深度(达到几十级)反而遭遇失败的例子。目前最新的Intel处理器和 ARM 高性能 Cortex-A 系列处理器的流水线深度都在十几级的范围左右。

5 向下生长——越来越浅的流水线

现代低功耗处理器的另外一个趋势也存在着流水线越来越浅的现象，其驱动因素同样很简单，那就是在性能够用的前提下追求极低的功耗。

以最知名的 ARM Cortex-M 系列处理器 IP 为例，2004 年发布的 Cortex-M3 处理器核的流水线级数只有 3 级，2009 年发布的 Cortex-M0 处理器核的流水线级数也只有 3 级，而 2012 年发布的 Cortex-M0+处理器核的流水线级数反而只有 2 级，变得越来越少了，正因为此 ARM 也宣传 Cortex-M0+处理器核为世界上能效比最高的处理器核。

2 级的流水线深度似乎已经浅到底了，读者可能会问，那是不是接下来要发布只有 1 级深度的流水线了?当深度变为 1 之后也就谈不上流水线了，其整体也就变成一个单周期的组合逻辑了，在众多的计算机体系结构教学案例中我们确实见到过很多流水线深度为 1 的处理器核，从功能上来说其仍然可以完成处理器的所有功能，只不过主频相当之低而已。

6 处理器流水线中的反压

随着流水线越深，由于每一级流水线都需要进行握手，流水线最后一级的反压信号可能会一直串扰到最前一级造成严重的反压(Back-pressure)时序问题，需要使用一些比较高级的技巧来解决这些时序问题。在现代处理器设计中，通常有如下若干种方法：

取消握手：此方法能够彻底杜绝反压的发生，时序表现非常好。但是取消握手即意味着流水线中的每一级并不会与其下一级进行握手，因此可能会造成功能错误或者指令丢失。因此这种方法往往需要配合其他的机制，譬如重执行(Replay)，预留大缓存等等。简而言之，此方法比较激进，辅以一系列其他的配置机制，硬件总体的复杂度会比较大。只有在一些非常高级的处理器设计中才会用到。

加入乒乓缓存：加入乒乓缓存(Ping-pong Buffer)是一种用面积换时序的方法，也是解决反压最简单的方法。通过使用乒乓缓存(有两个表项)替换普通的一级流水线(只有一个表项)，可以使得此级流水线向上一级流水线的握手接受信号仅仅需要关注乒乓缓存中是否有一个以上有空的表项即可，而无需将下一级的握手接受信号串扰至上一级。

加入前向旁路缓存：加入前向旁路缓存(Forward Bypass Buffer)也是一种用面积换时序的方法，这是在解决反压时一种非常巧妙的方法。旁路缓存仅有一个表项，由于增加了这一个额外的缓存表项，可以将后向的握手信号时序路径砍断，但是对前向路径不产生影响，因此，可以广泛使用于握手接口。蜂鸟 E200 即于设计中采用此方法，有效地解决了多处反压造成的时序瓶颈。

以上解决反压的技术方法，不仅在处理器设计中能够用到，而且在普通的ASIC电路设计中也会经常用到。

7 处理器流水线中的冲突

处理器的流水线设计中另外一个问题便是流水线中的冲突(Hazards)，主要分为资源冲突和数据冲突。

7.1 流水线中的资源冲突

资源冲突是指流水线中硬件资源的冲突，最常见的是运算单元的冲突，譬如除法器需要多个时钟周期才能完成运算，因此在前一条除法指令运算完成之前，新的除法指令如果也需要除法器则会存在着资源冲突。在处理器的流水线中硬件资源冲突种类还有较多，在此不做一一赘述。解决资源冲突的方法可以通过复制硬件资源或者流水线停顿等待硬件资源的方法解决。

7.2 流水线中的数据冲突

数据冲突是指不同的指令之间的操作数存在数据相关性造成的冲突。常见的数据相关性包括：

WAR(Write-After-Read)相关性，又称先读后写相关性：表示“后序执行的指令需要写回的结果寄存器索引”与“前序执行的指令需要读取的源操作数寄存器索引”相同造成的数据相关性。因此，从理论上来讲，在流水线中“后序指令”一定不能比和它有 WAR 相关性的“前序指令”先执行，否则“后序指令”先写回了结果至通用寄存器组中，“前序指令”再读取操作数时，就会读到错误的数值。

WAW(Write-After-Write)相关性，又称先写后写相关性：表示“后序执行的指令需要写回的结果寄存器索引”与“前序执行的指令需要写回的结果寄存器索引”相同造成的数据相关性。因此，从理论上来讲，在流水线中“后序指令”一定不能比和它有 WAW 相关性的“前序指令”先执行，否则“后序指令”先写回了结果至通用寄存器组中，“前序指令”再写回结果至通用寄存器组中就会将其覆盖。

RAW(Read-After-Write)相关性，又称先写后读相关性：表示“后序执行的指令需要读取的源操作数寄存器索引”与“前序执行的指令需要写回的结果寄存器索引”相同造成的数据相关性。因此，从理论上来讲，在流水线中“后序指令”一定不能比和它有 RAW 相关性的“前序指令”先执行，否则“后序指令”便会从通用寄存器组中读回错误的源操作数。

以上的三种相关性中，RAW 属于真数据相关。

解决数据冲突的常见方法如下：

WAW 和 WAR 可以通过寄存器重命名的方法将相关性去除，从而无需担心其执行顺序。

寄存器重命名技术在 Tomasulo 算法中通过保留站和 ROB(Re-Order Buffer)完成，或者采用纯物理寄存器(而不用ROB)的方式完成。

之所以RAW 称之为真数据相关，是因为其没有办法通过寄存器重命名的方法将相关性去除。一旦产生 RAW 相关性，后序的指令一定要使用和它有 RAW 数据相关性的前序指令执行完成的结果，从而造成流水线的等待停顿。为了能够尽可能的减少流水线停顿带来的性能损失，可以使用“动态调度”的方法。动态调度的思想本质上可以归结于以下方面：

一方面采用数据旁路传播(Data Bypass and Forward)技术尽可能的让前序指令的计算结果更快的旁路传播给后序相关指令的操作数;

另一方面尽可能的让后序相关指令在等待的过程中不阻塞流水线而让其他无关的指令能够继续顺利执行。

早期的 Tomasulo 算法中通过保留站可以达到这两方面的功效，但是保留站由于保存了操作数无法做到很大的深度(否则面积和时序的开销巨大)。

最新的高性能处理器普遍采用在每个运算单元前配置乱序发射队列(Issue Queue)的方式，发射队列仅追踪 RAW 相关性而并不存放操作数，因此可以做的很

深(譬如 16 个表项)。在发射队列中的指令一旦相关性解除之后，再从发射队列中发射出来读取物理寄存器组(Physical Register File)，然后发送给运算单元开始计算。

有关处理器的数据相关性问题和包括动态调度技术在内的解决方法，如果阐述清楚几乎可以单独成书，本文限于篇幅只能提纲挈领式的予以简述，感兴趣的读者可以自行查阅。

本文试图以最通俗化的方式对处理器流水线进行介绍，首先讨论了处理器流水线概述，包括：从经典的五级流水线说起、可不可以不要流水线——流水线和状态机的关系、深处种菱浅种稻，不深不浅种荷花——流水线的深度、向上生长——越来越深的流水线、向下生长——越来越浅的流水线。然后本文介绍了处理器流水线设计的两大难题及其多种解决方法：处理器流水线中的反压、处理器流水线中的冲突。

流水线上多周期指令执行时，流水线是否要暂停？

1、如果每条指令的执行时间长度不一定，像加法 1 周期，乘法 2 周期，除法更多，那在执行多周期指令时，是不是流水线都要暂停等待呢？n 周期指令等同于单周期指令加上 n-1个 NOP？

2、扩展下问题，指令在流水线上，是不是每个流程都要流过，不能跨越？像 MIPS 的 5 流水，为了乘法流水变成 6 级（EX 分解为执行 1，执行 2），那加法也必须分解为 6 级？PS：现在技术条件下，多少级流水最合适？

这种问题最好要先问好应用范围。从复杂程度依次往上：I 上古 8051 没有流水线 < II 信号处理 ARM M4，MIPS 或者其它相当的 DSP 是 4—5 级别流水线（可能有多发射，但一般顺序执行）< III 功耗价格敏感 ARM Cortex A 系列 11-15 级流水线（执行单元 1-5 周期）多发射乱序执行 < IV 通用处理 Intel 处理器 21＋级流水线。

I 不用讨论了。

如果是 II，一般都是顺序发射，顺序执行。这种一般就是加 NOP，因为一来本来就浅，损失不大。加上处理器一般不能动态发射和执行。

对于动态处理器来说，也有办法。比如用“完成”标志。在长指令完成前，下几条的指令先 freeze 在 pipeline 中。这个在移位除法器中尤其有用。因为有时候除法器可能要 3-32 个周期（根据数据）完成，如果一律添加 32 个 NOP 那损失太大了。

因为除法是 6 级那所有其它都变 6 级那是很愚蠢的行为。（那更长的 FPU，DIV 怎么办？）

如果是 III，基本就是在上面动态处理器的过程上插空发射了。而且可以“后发射先完成”。

这几个问题很好，看来是经过思考并深入进去了，值得回答。第一个问题，如果cpu不支持动态指令调度和多发射，那么必须停顿。如果支持动态调度，那么后续不相关的直径依然可以被执行，一直到保留站被塞满并且没有符合条件的指令了，才停顿。

第二个问题，是的不能跨过，或者说如果某一步什么都不用做，那就直接拷贝过去等着，但是不能超前，必须等着。但是你说的是执行过程需要多周期，这个是另外一回事。此时该执行部件一直处于繁忙状态，结果还没有被算出来，但是其他指令比如加法都运行完了，两者没有关联，除法就一直算，直到出结果，判断标志寄存器，算完了就写回，这是个异步过程。一般浮点数单元可以被流水化，因为其很明显的分了多个子步骤比如对阶，相加，规范化，舍入等。但是除法电路比较难以流水，只有部分流水，因为除法涉及到循环计算多次，结果被反馈回来继续算，这样就得独占几乎所有电路模块，或者要么使用一大块关联紧密的组合逻辑那就更不能流水了。所以，浮点算浮点的，加法算加法的，他俩无关联，加法不会被拖慢

10．基础知识

常用方法

1. 编译时找不到 stdio.h 等标准头文件:

   配置 MDK 默认包含路径：进入 options for Target xx->C/C++标签->配置 Include Paths 选项 ，一般是安装盘\Keil\C51\INC

2. 编译时报找不到 BIN40 \ArmAsm

   进工具

   Project->Manag->Components,Environment and

   Books->Folders/Extensions->RealView Folder 选项配置 KEIL 安装路径下的目录

3. KEIL 生成 code 文件并上传

   1.在工程 trunk 目录下建立名为 upcode 的文件夹，在 upcode 文件夹下面分别建立 upcode.bat 文件和 upcode.txt 文件。

   upcode.bat 文件内容如下

c:\Keil\ARM\BIN40\fromelf --vhx --32x1 .\application_test\debug\Pittosporum.axf --output=.\application_test\debug\code （其中 c:\Keil\ARM\BIN40\fromelf 为 KEIL 安装路径下的一个可执行文件,;--vhx --32x1 格式语句;

.\application_test\debug\Pittosporum.axf 为 keil4 生成的文件,通过此文件来生成二进制文件;

--output=.\application_test\debug\code 输出二进制文件名及路径）.

或者在输入下面语句在 run#2 方框中

fromelf --vhx --32x1 ..\output\PRJXXXX.axf --output=..\output_main_rom 是一样的 ,main_rom 就是生成的文件

ftp -s:.\upcode\upcode.txt

upcode.txt 文件内容为：

open 10.70.53.216 //工作站的 IP 地址,

user //修改为自己的工作站用户名

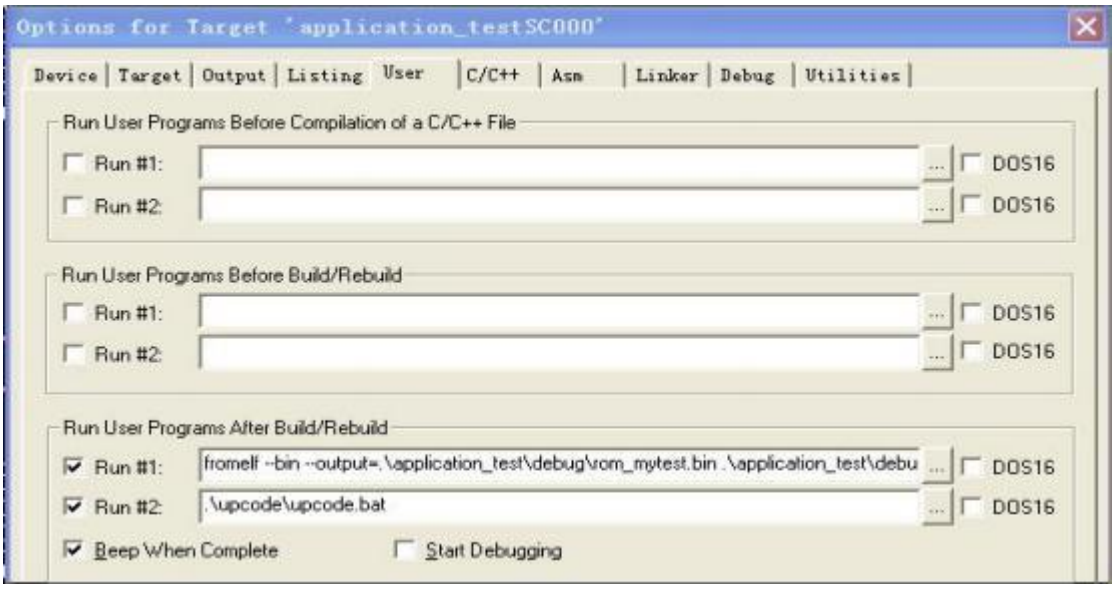
passwd //修改为自己的工作站登陆密码

cd project/xxx/trunk/Digital/FrontEnd/verification/software/sec //修改成上传的目录路径

put .\application_test\debug\code //上传的文件名

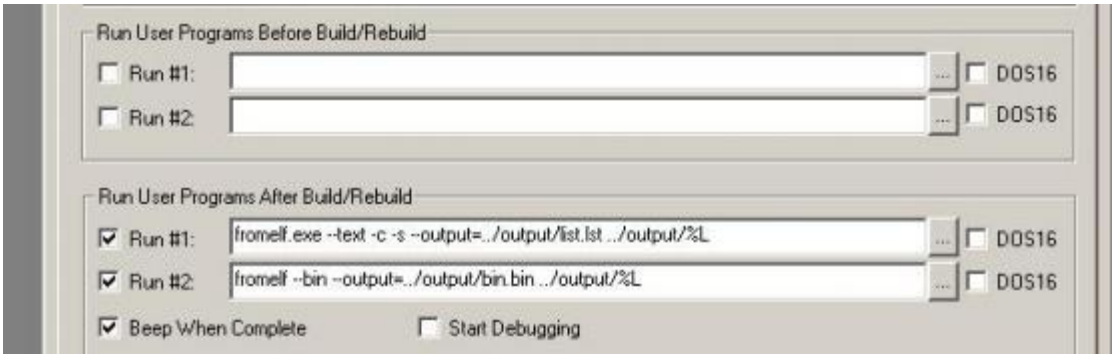
bye

2.打开工程 Target Option 窗口，选择 User 栏，把 Run #2 后面框中的内容改成如图中所示：



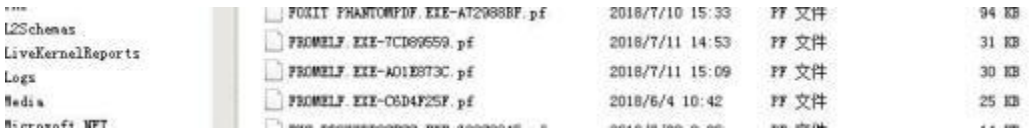
fromelf --bin --output=../output/bin.bin ../output/%L

注意：fromelf.exe 在windows 系统自己内置的一个工具，如下图



fromelf.exe --text -c -s --output=../output/list.lst ../output/%L

在 windows 下搜索到的 fromelf(可能是安装了 keil 工具会自动在 windows 下生成该文件)



函数定位方法，分散加载文件

```
LR_ROM1 0x00000000 {     ; load region size_region
  ER_ROM1 0x00000000 0x00008000 {   ; load address = execution address
```

```
  *.o  (RESET, +First)
  *.o(InRoot$$Sections)
  *.o(exe_at_rom)
  *.o(+RO)
 }

 ROM_EEPROM 0X00008000 0X00001800
  {
  *.o(exe_at_rom_eeprom)
 }

;  ROM_PATCH1 0X00009800 0X00000200
;   {
;    *.o(exe_at_rom_patch1)
;  }
;  ROM_PATCH2 0X00009a00 0X00000200
;   {
;    *.o(exe_at_rom_patch2)
;  }
;  ROM_PATCH3 0X00009c00 0X00000200
;   {
;    *.o(exe_at_rom_patch3)
;  }
;  ROM_PATCH4 0X00009e00 0X00000200
;   {
;    *.o(exe_at_rom_patch4)
;  }

 EEPROMCODE 0x08000000 0X14000
```

```
        {
          *.o(exe_at_eeprom)
        }

        ;this is just an empty area
        NOTHING 0x20000000 EMPTY 0
        {
        }

        ;fpga verification ram,from 0x20000000 to 0x20000440
        FPGA_VERI  +0 EMPTY   0X340
        {

        }
        ;caution:in simulation,this segment can only be used by
modifying memory map after connected to debugger
        RAMCODE +0 0X660
        {
         *.o(exe_at_sram)
        }
        ;user ram,from 0x20000440-0x20000bc0,length=0xbc0,=3008
bytes
        ;ÓÃ» §RAM£¬Î»ÖÃÔÚramÆ«ÒÆµÄ1Kµ½4K£¬Ç° ÃæµÄFPGA_VERI¶ÎÉ¾³ý
ºó£¬Î»ÖÃ½«×Ô¶¯µ÷ÕûÎª RAMÆ«ÒÆµÄOKµ½4K¡£
        RW_RAM1 +0 0x00000dc0  {   ; RW data
         *  (+RW +ZI)
        }
        ;backup for memory test
        ARM_LIB_HEAP 0x20001400 EMPTY 0x100
```

```
        {
        }

        ;STACK,From 0x20001400-0x20001800,1K
        ;Õ»£¬Î»ÖÃÔÚramÆ«ÒÆµÄ5k-6k
        ARM_LIB_STACK 0x20001800 EMPTY -0x300
        {
        }

    }
    LR_ROM_PATCH1 0x00008800   {
      ER_ROM_PATCH1 0X00008800 0X00000200
       {
        *.o(exe_at_rom_patch1)
       }
}
```

通过run#1和分散加载文件可以生成需要的二进制文件；而通过 run#2 可以自动把生成的文件上传到工作站

4. Scatter 基本点：

1. 编译后输出的映像文件中各段是首尾相连的，中间没有空闲的区域，它们的先后关系是根据链接时参数的先后次序决定的 armlinker -file1.o file2.o ……

2. scatter 用于将编译后的映像文件中的特定段加载到多个分散的指定内存区域

3. 有 2 类域 region：执行域（execution region，一般是 ram 区域）和加载域(load region，一般是 rom 区域)

4. 加载域：就是编译之后得到的二进制文件烧写到rom 中的这一段区域，所有的代码 RO、预定义变量 RW、堆栈之类清不清空无关紧要的大片内存区域ZI，都包括在其中

5. 执行域：就是把加载域进行‘解压缩’后的样子。比如：RO 没有变动还是在 ROM 中，RW 被移到了 SRAM 中，而 ZI 被放置在 SDRAM 中

6. scatter 本身并不能对映像实现‘解压缩’，编译器读入 scatter 文件之后会根据其中的各种地址生成启动代码，实现对映像的加载，而这一段代码就是* (InRoot$$Sections)它是__main()的一部分。这就是在汇编启动代码的最后跳转到__main() 而不是跳向main()的原因之一。

7. 起始地址与加载域重合的执行域成为root region，* (InRoot$$Sections)必须放在这个执行域中，否则链接的时候会报错。*(+RO)包含了* (InRoot$$Sections)，所以如果在 root region 中用到了*(+RO)可以不再指定* (InRoot$$Sections)，

5. scatter 语法：

```
ROM_LOAD 0x00000000
 {
     ROM 0x00000000 0x003FFFFF
    {
       vectors.o  (+RO,+FIRST)
       * (InRoot$$Sections)        ; All library sections that must be  in a root region
       *(+RO)
    }
     SRAM 0x00400000 0x003FFFFF
    {
        * (+RW,+ZI)
    }
    SDRAM1 0x41000000 UNINIT
    {
         stack.o (+ZI) ; stack.s 中定义了 top_of_stack 为长度为 1 的 space，指定栈顶地址
    }
```

```
    SDRAM2 +0 UNINIT
    {
        heap.o (+ZI)
    }

}
```

注解：

1. ROM_LOAD 是加载域。这里只有一个，也可以有多个（rom 地址不连续的情况）

2. ROM、SRAM、SDRAM1、SDRAM2 是执行域，有多个。第一个执行域必须和加载域地址重合，因为 ARM 的复位地址就是加载域的起始地址（有 bootstrap 的话加载域址就是 bootstrap 执行完后的跳转地址）

3. vectors.o (+RO, +FIRST) 中断向量表放在最开头

4. ROM 0x00000000 0x003FFFFF; 加载域名 起始地址 最大允许长度；‘最大允许长度’也可以省略，但缺点是编译器不会检查段是否溢出和别的段重叠了。‘起始地址’= +0 表示紧接着上一段开始的连续地址。

5. * (InRoot$$Sections)是复制代码的代码

6. UNINT 关键字表示不进行初始化清零

6. 值得注意的是：

在一个 scatter 文件中，同一个.o 文件不能出现 2 次，即使是在 2 个不同的加载域中也不可以，否则会报错：Ambiguous selectors found for *.o，错误的例子：

```
LOAD1 0x00000000
{
    EXE1
    {
            Init.o
    }
```

```
}
LOAD2 0xFFFF0000
 {
    EXE2
     {
            Init.o
     }
}
```

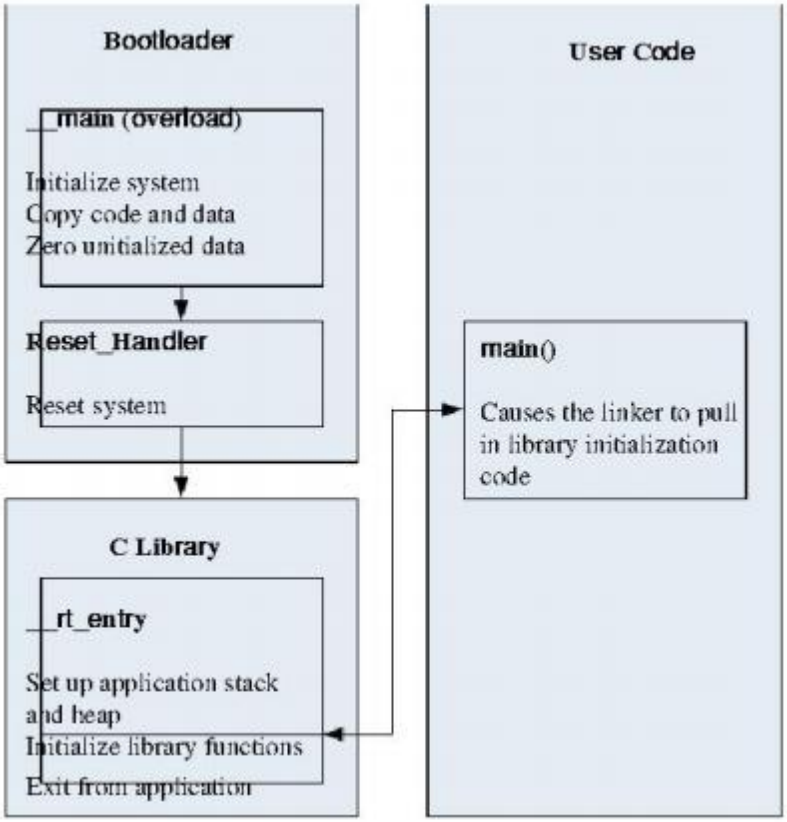
推测是编译器自动生成的 scatter载入代码 InRoot$$Sections不支持把同一 obj搬移 2 次。这就带来一个问题：如果希望把同一段代码（如中断跳转表）载入 2 份拷贝到不同的地址，咋整?一个笨办法是自己写一段代码搬移程序来代替编译器自动生成的搬移代码，但前提是需要搞懂映像文件的组织，增加了工作量。投机一点的方法是在makefile 中把一个.o 文件复制并重新起一个名字，然后把它们传递给armlink.

简单 scatter 分析

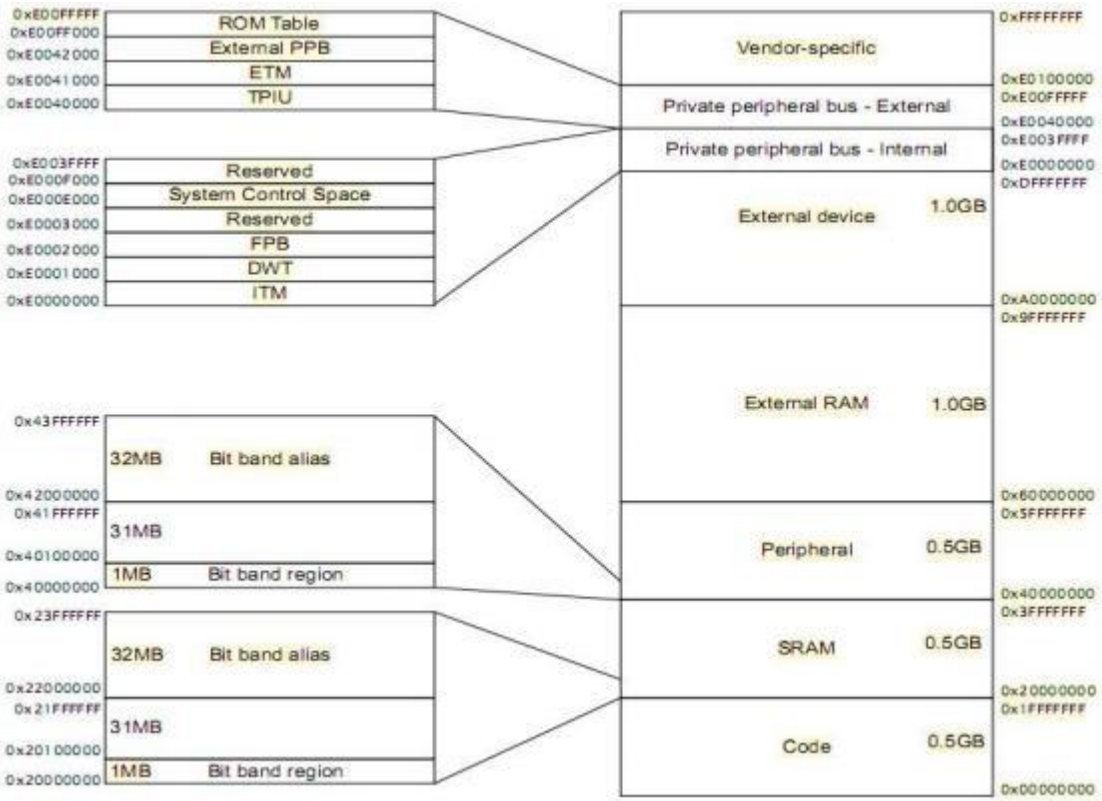
```
LR_IROM1 0x00000800 0x0000C000  {     ; load region size_region
  ER_IROM1 0x00000800 0x0000C000  {   ; load address = execution
address
  *.o  (RESET, +First)
   *.o(InRoot$$Sections)
   *.o(+RO)  //除了下面的代码外，其他代码都放在此空间
  }
  RW_IRAM1 0x20008000 UNINIT 0x00004000  {   ; RW data
   .ANY  (+RW +ZI)
  }


  }
 LR_IROM2 0X00000000 0X00000400  { //程序存放在此地址
```

```
    test0 0X00000000 0X00000400   //执行地址，执行地址等于存放地址，不用搬移
   {
    gpio6_jump.o   //gpio6_jump.o 的代码--需要有一个 gpio6_jump.c 的文件
   }
  }

LR_IROM3 0X00000400 0X00000400  {  //程序存放在此地址
      test1 0X20000400 0X00000400   //执行地址，启动代码会把程序搬移到此地址运行
   {
    gpio7_jump.o   //gpio7_jump.o 的代码--需要有一个 gpio7_jump.c 的文件
   }
}
```

Arm 程序运行过程如下图



	Name	Device Type	XN	Cache	Description
0x00000000-0x1FFFFFFF	Code	Normal	-	WT	Typically ROM or flash memory. Memory required from address 0x0 to support the vector table for system boot code on reset.
0x20000000-0x3FFFFFFF	SRAM	Normal	-	WBWA	SRAM region typically used for on-chip RAM.
0x40000000-0x5FFFFFFF	Peripheral	Device	XN	-	on-chip peripheral address space
0x60000000-0x7FFFFFFF	RAM	Normal	-	WBWA	memory with write-back, write allocate cache attribute for L2/L3 cache support
0x80000000-0x9FFFFFFF	RAM	Normal	-	WT	memory with write-through cache attribute
0xA0000000-0xBFFFFFFF	Device	Device, Shareable	XN	-	shareable device space
0xC0000000-0xDFFFFFFF	Device	Device	XN	-	non-shareable device space
0xE0000000-0xFFFFFFFF	System	-	-	-	system segment including the PPB
+0000000	PPB	SO, (Shareable)	XN	-	1MB region reserved as a Private Peripheral Bus. The PPB supports key resources including the System Control Space, and debug features.
+0100000	Vendor_SYS	-	-	-	Reserved in ARMv6-M



启动文件

启动文件 startup_pittosporum_project.s,里面定义了各种向量，如中断向量，复位向量，存储管理向量，hardfault 向量，调试等各方面的向量，其实就是定义各种情况下的入口地址

Cortex-M 启动流程

在离开复位状态后，SC000 做的第一件事就是读取下列两个 32 位整数的值：

从地址 0x0000,0000 处取出 MSP 的初始值。

从地址 0x0000,0004 处取出 PC 的初始值——这个值是复位向量，LSB 必须是 1。然后从这个值所对应的地址处取指。

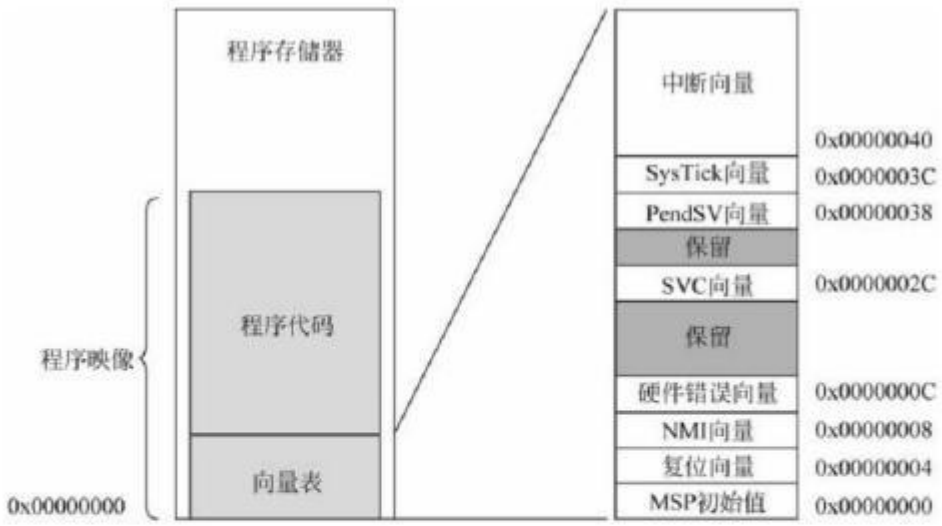


图 3.11 程序映像中的向量表

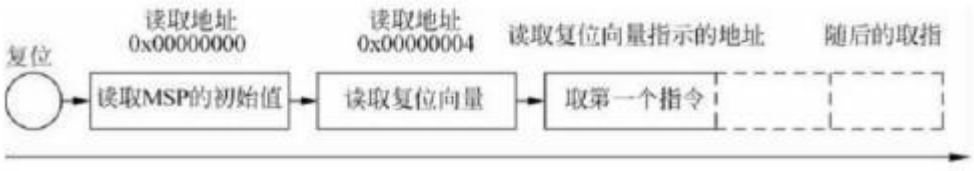


图 3.12 复位流程

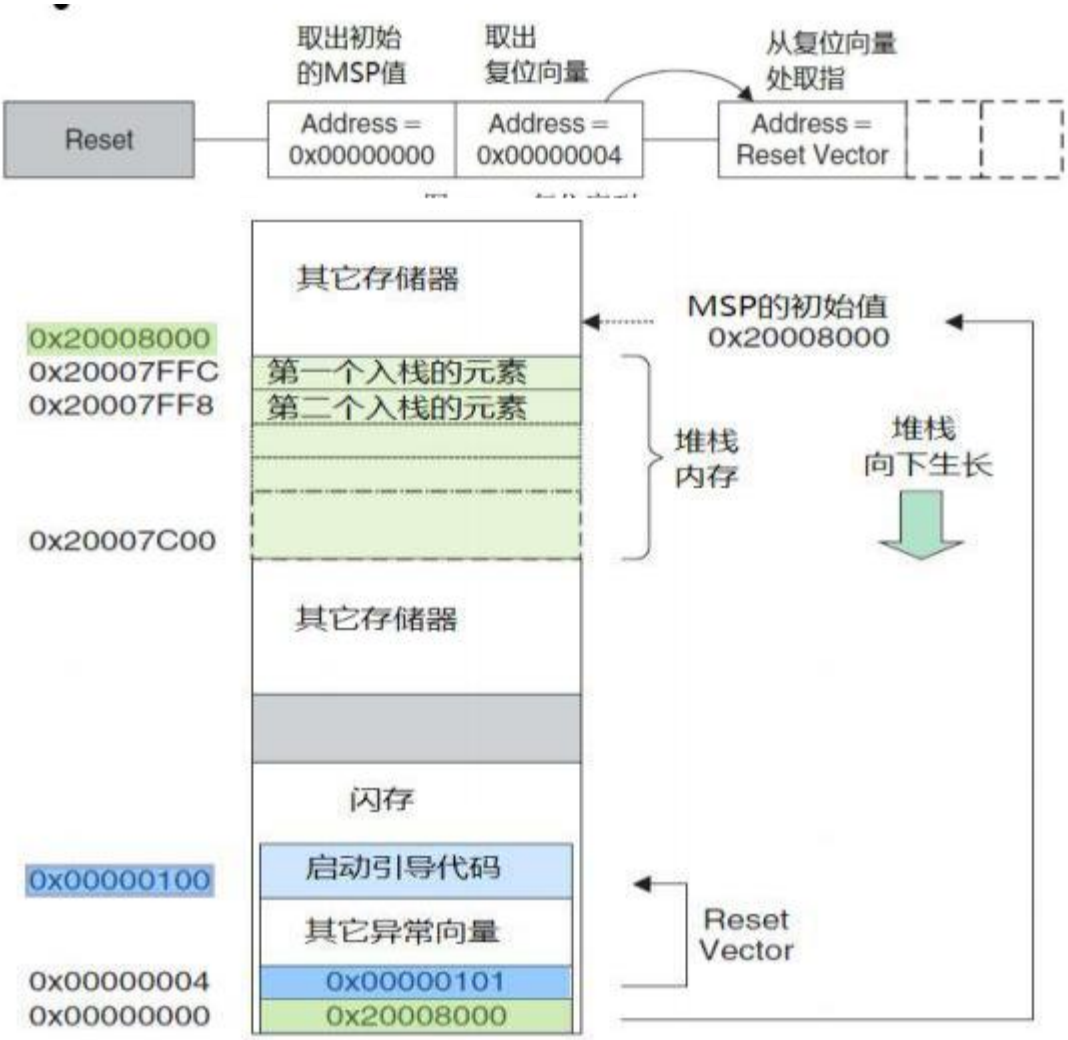


图 3.18　初始 MSP 及 PC 初始化的一个范例

如果 code 地址有偏移,则 04 以上的地址也会偏移.

因为CM0 使用的是向下生长的满栈，所以 MSP 的初始值必须是堆栈内存的末地址加 1。举例来说，如果你的堆栈区域在 0x20007C00 - 0x20007FFF 之间，那么 MSP 的初始值就必须是 0x20008000。

向量表跟随在 MSP 的初始值之后——也就是第 2 个表目。要注意因为 CM3 是在 Thumb 态下执行，所以向量表中的每个数值都必须把 LSB 置 1（也就是奇数）。正是因为这个原因，图 3.18 中使用 0x101 来表达地址 0x100。当 0x100 处的指令得到执行后，就正式开始了程序的执行。在此之前初始化 MSP 是必需的，因为可能第 1 条指令还没执行就会被NMI 或是其它 fault 打断。MSP 初始化好后就已经为它们的服务例程准备好了堆栈。

要理解 Cortex-M0 的启动流程，需要首先大致浏览一下程序映像。通常，Cortex-M0 处理器的程序映像是从地址 0x00000000 开始的。

程序映像的开始处为向量表（见图 3.11），其中包含了异常的起始地址（向量），每个中断向量的地址都等于“异常号×4”。例如，外部 IRQ≠0 的异常类型为 16，因此 IRQ≠0 的向量地址为 16×4＝0x40。这些向量的最低位都被置 1，表明异常处理执行时使用 Thumb 指令。向量表的大小由实际使用的中断个数决定。

向量表中还包含了主栈指针（MSP）的初始值，它存储在向量表的头四个字节。

复位时，处理器首先读取向量表前两个字（即前 8 字节），第一个字为 MSP 的初始值；第二个字为复位向量（见图 3.12），它表示程序执行的起始地址（复位处理）。

例如，如果启动代码位于地址 0x000000C0，就需要在复位向量处写入这个地址，并且将地址的最低位置为 1，以表明当前为 Thumb 代码。因此，地址 0x00000004 处的值被置为 0x000000C1（见图 3.13）。在取得复位向量值以后，处理器将开始从这个地址处执行程序代码。传统的 ARM 处理器（例如 ARM7TDMI）在这方面的处理是不同的，它们往往从地址 0x00000000 处开始执行程序；而且向量表中为跳转指令，Cortex-M 处理器中则是地址值。

程序生成流程：

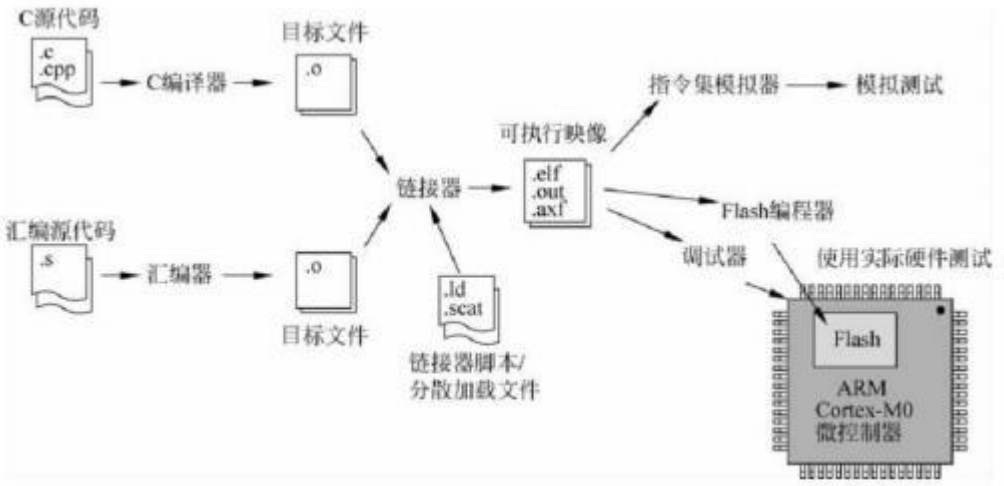


图 4.10　程序生成流程

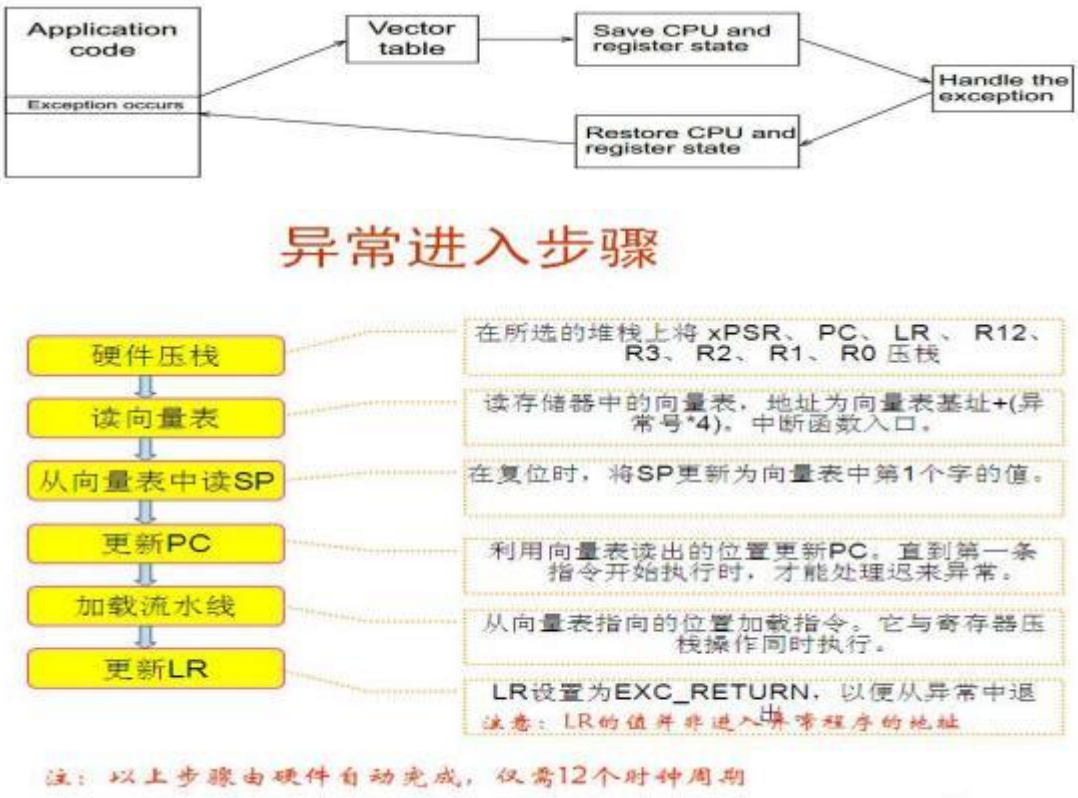
中断响应与处理流程

当 CM0 开始响应一个中断时，会在它看不见的体内奔涌起三股暗流：

入栈：　把 8 个寄存器的值压入栈

取向量：从向量表中找出对应的服务程序入口地址

选择堆栈指针 MSP/PSP，更新堆栈指针 SP，更新连接寄存器 LR，更新程序计数器 PC



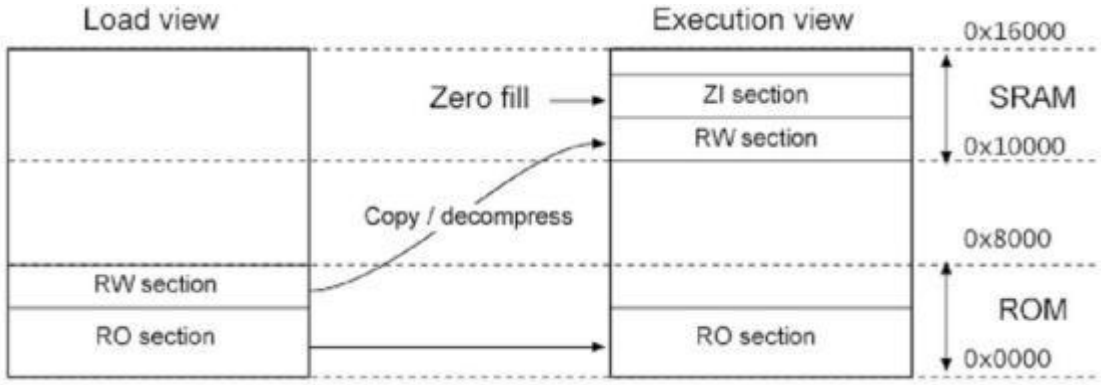
1.响应异常的第一个行动，就是自动保存现场的必要部分：依次把 xPSR, PC, LR, R12 以及 R3 - R0 由硬件自动压入适当的堆栈中：如果当响应异常时，当前的代码正在使用 PSP，则压入 PSP，即使用线程堆栈；否则压入 MSP，使用主堆栈。一旦进入了服务例程，就将一直使用主堆栈。

2.假设入栈开始时，SP 的值为 N，则在入栈后，堆栈内部的变化如下所示。又因为 AHB 接口上的流水线操作本质，地址和数据都在经过一个流水线周期之后才进入。另外，这种入栈在机器的内部，并不是严格按堆栈操作的顺序的——但是机器会保证：正确的寄存器将被保存到正确的位置。

3.CM0 在看不见的内部打乱了入栈的顺序，这是有深层次的原因的。先把 PC 与xPSR 的值保存，就可以更早地启动服务例程指令的预取——因为这需要修改PC；同时，也做到了在早期就可以更新xPSR 中 IPSR 位段的值。

4.为啥袒护 R0 - R3 以及 R12 呢，R4 - R11 就是下等公民？原来，在 ARM 上，有一套的 C 函数调用标准约定（《C/C++ Procedure Call Standard for the ARM Architecture》，AAPCS, Ref5）。个中原因就在它上面：

它使得中断服务例程能用C 语言编写，编译器优先使用被入栈的寄存器来保存中间结果（当然，**如果程序过大也可能要用到 R4 - R11，此时编译器负责生成代码来push它们**。但是，ISR 应该短小精悍，不要让系统如此操心）。



也就是说硬件只会保存 8 个寄存器内容,如果使用了更多寄存器,则由编译器生成汇编代码来保存更多的寄存器.

另外注意,同一个中断不能在没有完成当前中断有响应当前中断源的中端,否则会 fault

嵌套向量中断控制器（NVIC）

更有效地处理异常中断

集成于处理器内部，靠近内核

高效处理系统异常（Exceptions）和中断（Interrupts）

特性

异常优先级设置

“咬尾中断”机制和“晚到异常”处理

完全确定的异常处理时间

异常处理所花销的时钟周期数一样

16 个时钟周期的固定开销

特定寄存器用来权衡延时和预防抖动

只要会 C 就能编写程序

软件控制睡眠模式

ARMv6-M 架构支持为减少功耗而让 Cortex-M0 和系统时钟停止运行的系统睡眠模式

3 段流水线

ARM	Thumb		
PC	PC	取指	指令从存储器中取出
PC-4	PC-2	译码	对指令使用的寄存器进行译码
PC-8	PC-4	执行	从寄存器组中读出寄存器
			执行移位和 ALU 操作
			寄存器写回到寄存器组

图 1-1 指令流水线

Table B1-3 Exception numbers

Exception number　　Exception

1　　Reset　　软件可见地址进入 0x00000004，　0x00000004 地址 放的是复位后的跳转地址

2　　NMI　　软件可见地址进入 0x00000008

3　　HardFault　　软件可见地址进入 0x0000000c

4-10　　Reserved　　软件可见地址进入 0x00000040，后面地址依次+4

11　　SVCall

12-13　　Reserved

14　　PendSV

15　　SysTick, optional

16　　External Interrupt(0)

… …

16 + N			External Interrupt(N)		
	HPROT[3]	HPROT[2]	HPROT[1]	HPROT[0]	
	cacheable	bufferable	privileged	data/opcode	Description
Opcode fetch	–	–	–	0	
Data access	–	–	–	1	
User access	–	–	0	–	
Privileged access	–	–	1	–	
Not bufferable	–	0	–	–	
Bufferable	–	1	–	–	
Not cacheable	0	–	–	–	
Cacheable	1	–	–		

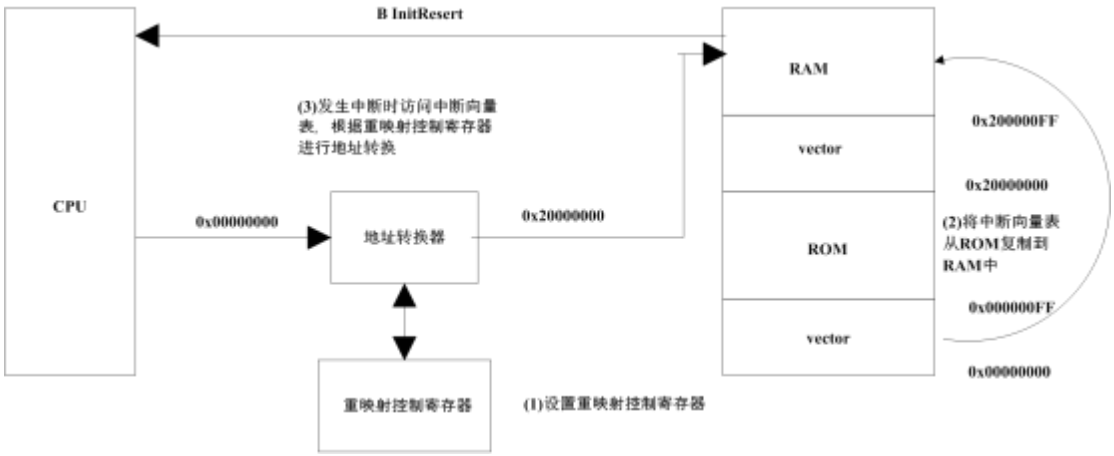
```
instr  = ~HPROT[0]&HPROT[1];
rd     =  HPROT[0]&HPROT[1]&read;
wr     =  HPROT[0]&HPROT[1]&write;
```

HPROT[3:2] 指示指令或数据是从哪里来或读到哪里去，privilegeded 一般与 cacheable 使用，所以 HPROT 出现的值一般为 a,b,f;

Hresp 表示从还回 OKEY or ERROR,0 表示 OKEY;1 表示 ERROR.从设备一般都是还回 0

7. 写 C 程序时,源文件最好分三个文件夹,一个为 include,放头文件,一个为 drive 放各模块的驱动,一个为 main,放主程序

中断向量表的重映射过程：



15.中断重影射

M0 没有 SCB->VTOR 寄存器,M3 以上才有此寄存器,m0 不支持中断向量重映射

```
void SetupVTORToRAM(void)  // 设置向量重映射寄存器
{
UINT32 VTOR;
//new vector stays  in ram 0x20000000
//warning,this address must be 64 words alignment
VTOR = 0X20000000;  // 设置中断向量表的起始地址为 0x20000000
```

```
    Memcpy ((void *)VTOR,  (void *)__TestVectors,
sizeof(__TestVectors));
    //set VTOR register
    SCB->VTOR  | =  SCB_VTOR_TBLBASE_Msk  |  (VTOR &
SCB_VTOR_TBLOFF_Msk);
}
```

定义函数：void *memcpy(void *dest, const void *src, size_t n)

函数说明：memcpy()用来拷贝 src 所指的内存内容前 n 个字节到 dest 所指的内存地址上。**与 strcpy()不同的是,memcpy()会完整的复制n 个字节,不会因为遇到字符串结束'/0'而结束**

返回值： 返回指向 dest 的指针

附加说明：**指针 src 和 dest 所指的内存区域不可重叠**

中断异常处理流程

这里的中断异常是指 Reset、HardFault、NMI 等异常情况的处理。

操作说明

系统运行时，异常可能会随时发生。当一个异常出现以后，SC000 微处理器会执行以下几步操作：

1) 将下一条指令的地址存入相应连接寄存器 LR，以便程序在处理异常返回时能从正确的位置重新开始执行。

2) 将 CPSR (current program status register 当前程序状态寄存器)，复制到相应的 SPSR（saved program status register 程序状态保存寄存器）中。

3) 根据异常类型，强制设置 CPSR 的运行模式位。

4) 强制 PC 从相关的异常向量地址取下一条指令执行，从而跳转到相应的异常处理程序处。

异常处理完毕之后，SC000 微处理器会执行以下几步操作从异常返回：

5） 将连接寄存器 LR 的值减去相应的偏移量后送到 PC 中。

6） 将 SPSR 复制回 CPSR 中。

7） 定义异常中断服务函数;

8） 在中断服务函数中处理异常;

以上步骤中的 1~6 是由 SC000 内核完成的，不需要用户程序参与，步骤 7~8 必须由用户在中断处理函数中实现。为保证在 ARM 处理器发生异常时不至于处于未知状态，在应用程序的设计中，首先要进行异常处理。采用的方式是在异常向量表中的特定位置放置一条跳转指令，跳转到异常处理程序。当 ARM 处理器发生异常时，程序计数器 PC 会被强制设置为对应的异常向量，从而跳转到异常处理程序。当异常处理完成以后，返回到主程序继续执行。可以认为应用程序总是从复位异常处理程序开始执行的，因此复位异常处理程序不需要返回。

汇编中几个常用的段代号,基本跟编译器和处理器没关系:

1.text　　//代码段具有可执行的属性

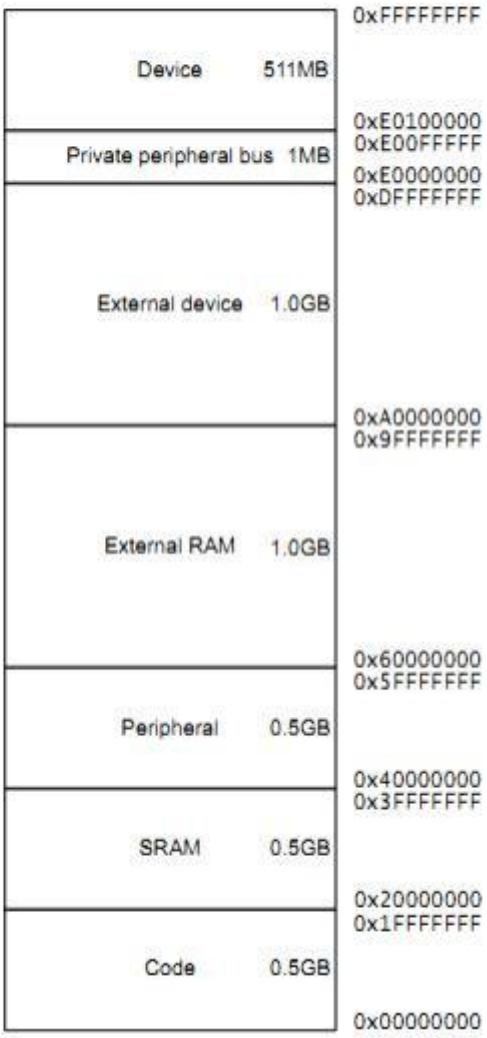
2.data　　//读写数据段,存放可执行的属性

3.const　　//只读数据段(有些编译器不使用此段,将只读数据并入.data 段)

4.bss　　//未初始化的全局变量数据段

ARM 地址分配规则:

This section describes the processor memory map and the behavior of memory accesses. The processor has a fixed memory map that provides up to 4GB of addressable memory. The memory map is:



2.2.3 Behavior of memory accesses

The behavior of accesses to each region in the memory map is:

Table 2-9 Memory access behavior

Address range	Memory region	Memory type[a]	XN[a]	Description
0x00000000-0x1FFFFFFF	Code	Normal	-	Executable region for program code. You can also put data here.
0x20000000-0x3FFFFFFF	SRAM	Normal	-	Executable region for data. You can also put code here.
0x40000000-0x5FFFFFFF	Peripheral	Device	XN	External device memory.
0x60000000-0x9FFFFFFF	External RAM	Normal	-	Executable region for data.
0xA0000000-0xDFFFFFFF	External device	Device	XN	External device memory.
0xE0000000-0xE00FFFFF	Private Peripheral Bus	Strongly-ordered	XN	This region includes the NVIC, System timer, and System Control Block. Only word accesses can be used in this region.
0xE0100000-0xFFFFFFFF	Device	Device	XN	Vendor specific.

中断表

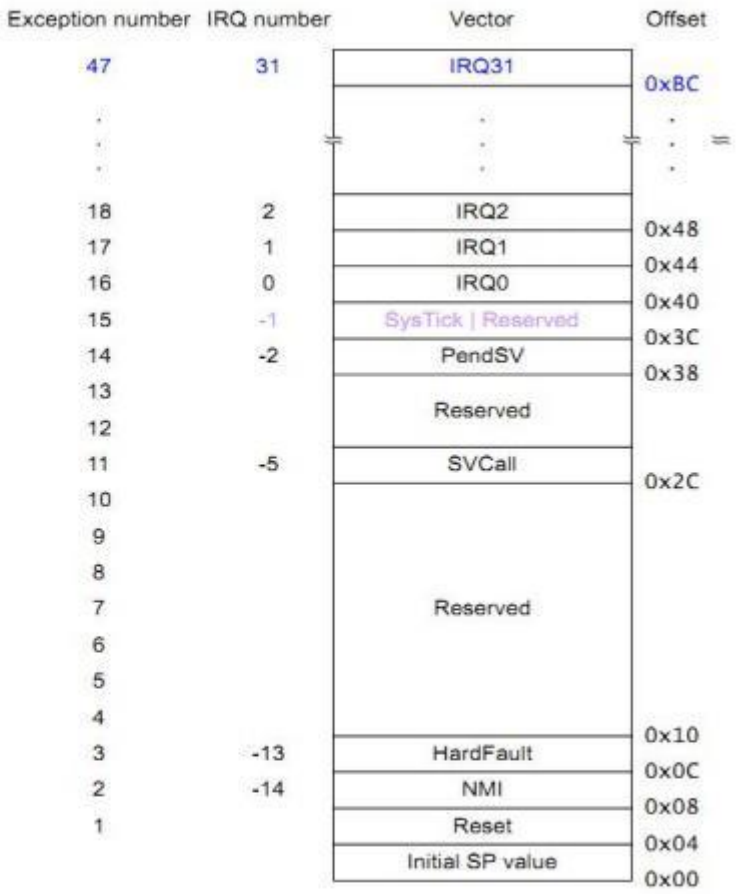


Figure 2-1 Vector table

The vector table is fixed at address `0x00000000`.

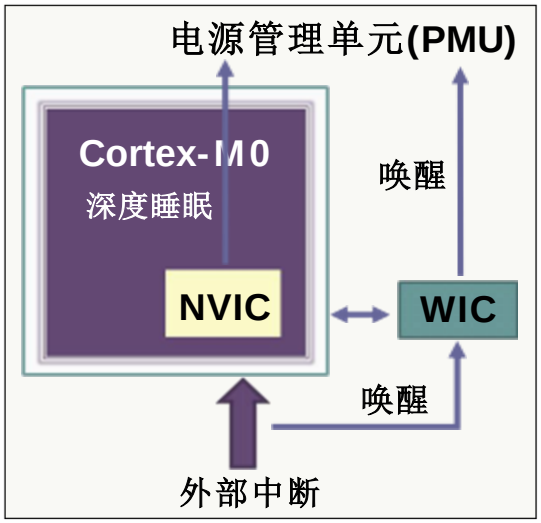
中断进入与唤醒

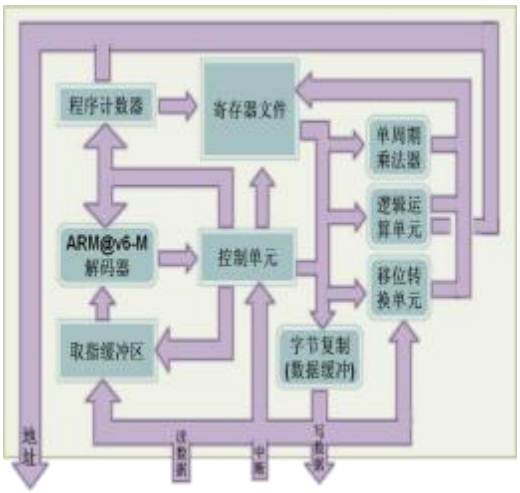
通过指令进入 sleep 模式

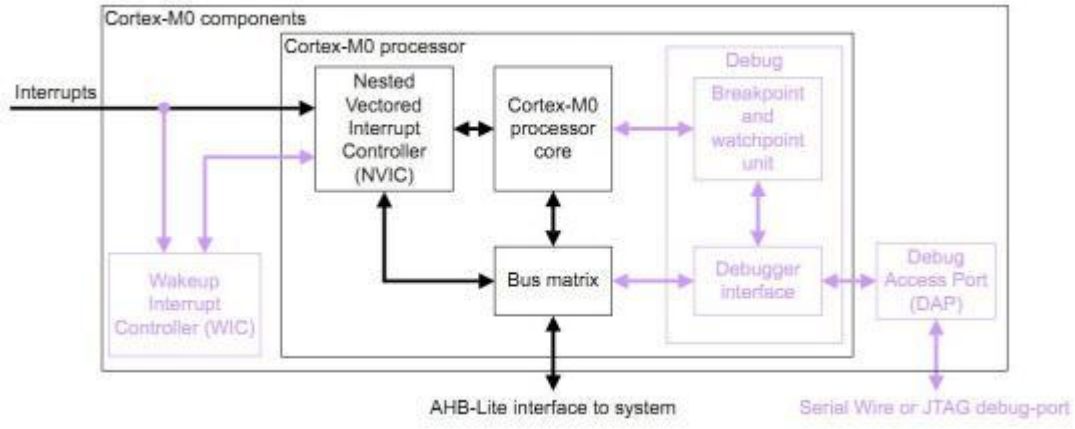
唤醒可以通过内部中断,WFI—wait for interrupt 唤醒

唤醒可以通过外部事件,WFE—wait for event 唤醒

M0 系统架构







软件控制睡眠模式

- 极低的待机功耗(Cortex-M0 内核功耗不到 40μA/MHz)
- 一个低门数中断唤醒控制器（WIC）

Sleep（睡眠）

- 内核时钟停止
- NVIC(嵌套向量中断控制器)继续对中断有效

Deep sleep（深度睡眠）

- WIC 对选择指定中断有效
- Cortex-M0 可进入保持状态

WIC 唤醒信号传递给 PMU

- 内核立即被唤醒

- 立即响应关键事件

嵌套向量中断控制器（NVIC）

- **更有效地处理异常中断**
- 集成于处理器内部，靠近内核
- 高效处理系统异常（Exceptions）和中断（Interrupts）

- 特性

- 异常优先级设置
- “咬尾中断”机制和“晚到异常”处理

完全确定的异常处理时间

- 异常处理所花销的时钟周期数一样
- 16 个时钟周期的固定开销

特定寄存器用来权衡延时和预防抖动

指令执行

- **绝大多数指令占用 2 字节的存储空间**

指令执行占用确定的时间

- 数据处理指令（例如加、移位、逻辑或）1 个时钟周期内完成
- 数据传输（例如加载，存储）需 2 个时钟周期
- 分支转移指令需要 3 个时钟周期

指令操作码都是 32 位

- 处理器的寄存器和算术逻辑单元（ALU ）都是 32 位

与中断相关的寄存器如下：

Table 4-2 NVIC register summary

Address	Name	Type	Reset value	Description
0xE000E100	ISER	RW	0x00000000	*Interrupt Set-enable Register* on page 4-4
0xE000E180	ICER	RW	0x00000000	*Interrupt Clear-enable Register* on page 4-5
0xE000E200	ISPR	RW	0x00000000	*Interrupt Set-pending Register* on page 4-5
0xE000E280	ICPR	RW	0x00000000	*Interrupt Clear-pending Register* on page 4-6
0xE000E400-0xE000E41C	IPR0-7	RW	0x00000000	*Interrupt Priority Registers* on page 4-7

Set 中断和 clear 中断 是通过 2 个地址来操作

有直接的内置函数可用

To access the NVIC registers when using CMSIS, use the following functions:

Table 4-3 CMSIS access NVIC functions

CMSIS function	Description
`void NVIC_EnableIRQ(IRQn_Type IRQn)`[a]	Enables an interrupt or exception.
`void NVIC_DisableIRQ(IRQn_Type IRQn)`[a]	Disables an interrupt or exception.
`void NVIC_SetPendingIRQ(IRQn_Type IRQn)`[a]	Sets the pending status of interrupt or exception to 1.
`void NVIC_ClearPendingIRQ(IRQn_Type IRQn)`[a]	Clears the pending status of interrupt or exception to 0.
`uint32_t NVIC_GetPendingIRQ(IRQn_Type IRQn)`[a]	Reads the pending status of interrupt or exception. This function returns non-zero value if the pending status is set to 1.
`void NVIC_SetPriority(IRQn_Type IRQn, uint32_t priority)`[a]	Sets the priority of an interrupt or exception with configurable priority level to 1.
`uint32_t NVIC_GetPriority(IRQn_Type IRQn)`[a]	Reads the priority of an interrupt or exception with configurable priority level. This function return the current priority level.

a. The input parameter IRQn is the IRQ number, see Table 2-11 on page 2-20 for more information.

In addition, the CMSIS provides a number of functions for NVIC control, including:

Table 4-9 CMSIS functions for NVIC control

CMSIS interrupt control function	Description
`void NVIC_EnableIRQ(IRQn_t IRQn)`	Enable IRQn
`void NVIC_DisableIRQ(IRQn_t IRQn)`	Disable IRQn
`uint32_t NVIC_GetPendingIRQ (IRQn_t IRQn)`	Return true (1) if IRQn is pending
`void NVIC_SetPendingIRQ (IRQn_t IRQn)`	Set IRQn pending
`void NVIC_ClearPendingIRQ (IRQn_t IRQn)`	Clear IRQn pending status
`void NVIC_SetPriority (IRQn_t IRQn, uint32_t priority)`	Set priority for IRQn
`uint32_t NVIC_GetPriority (IRQn_t IRQn)`	Read priority of IRQn
`void NVIC_SystemReset (void)`	Reset the system

系统控制寄存器

Table 4-10 Summary of the SCB registers

Address	Name	Type	Reset value	Description
0xE000ED00	CPUID	RO	0x410CC200	*CPUID Register*
0xE000ED04	ICSR	RW[a]	0x00000000	*Interrupt Control and State Register* on page 4-12
0xE000ED0C	AIRCR	RW[a]	0xFA050000	*Application Interrupt and Reset Control Register* on page 4-15
0xE000ED10	SCR	RW	0x00000000	*System Control Register* on page 4-16
0xE000ED14	CCR	RO	0x00000204	*Configuration and Control Register* on page 4-17
0xE000ED1C	SHPR2	RW	0x00000000	*System Handler Priority Register 2* on page 4-19
0xE000ED20	SHPR3	RW	0x00000000	*System Handler Priority Register 3* on page 4-19

a. See the register description for more information.

M0 有专用的进入 sleep 命令:__wfe(); __wfi();这两条指令在安装目录下 core_cmInstr.h 中定义

Thumb 指令集特点

1. **32 位的操作码**
2. **16 位的指令系统**

寄存器说明

Cortex-M0 的寄存器组中，包含了 13 个 32 位的通用目的寄存器，以及多个特殊寄存器（见图 3.2）。

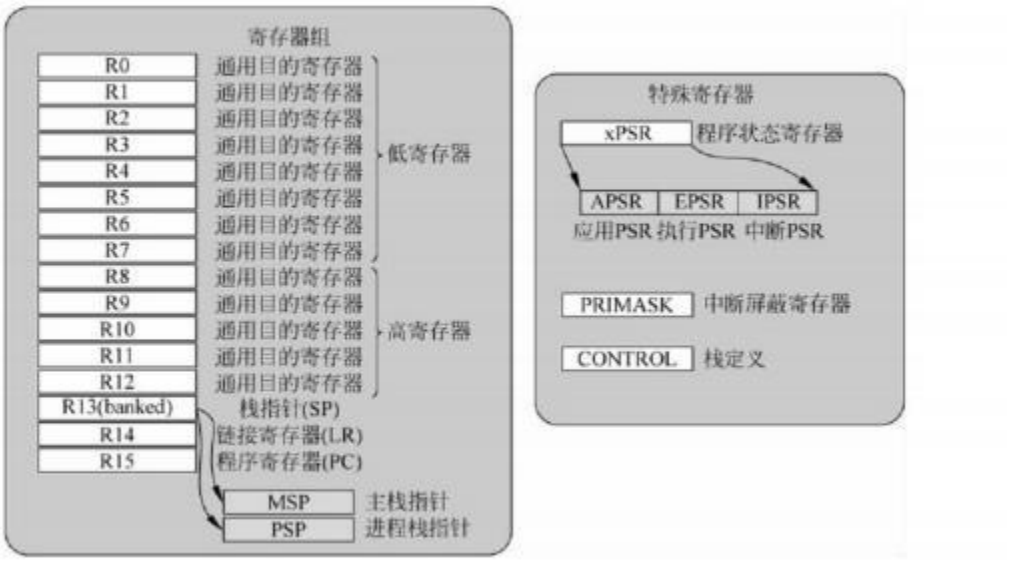


图 3.2 Cortex-M0 的寄存器

数据的解析和控制器的处理过程中，需要处理器内核寄存器的参与。如果需要处理控制器中的数据，这些数据需要先加载到处理器内核的寄存器（寄存器组中的摸个），处理完成后，如果有需必要，它们还会被送到存储器中，这种方式被称为“加载-存储架构”（load-store architecture）。

Cortex-M0 有 13 个 32 位通用寄存器 R0-R12，以及多个特殊寄存器。

寄存器组包含 16 个 32 位寄存器，其中 13 个时通用的，其余三个具有特殊用途。

R0-R12 通用寄存器

R0-R12 为32 位通用寄存器，由于 16 位 thumb 指令集在空间上的限制，许多 Thumb 指令只能操作 R0-R7，它们被称为低寄存器，而想 MOV 之类的一些指令则可以使用全部的寄存器。R0-R12 寄存器的初始化值未定义。

R13,栈指针（SP）

R13 为栈指针，对栈空间进行存取操作（通过 PUSH 和 POP 指令），Cortex-M0 在不同的物理地址上有两个栈指针，一个为主栈指针（MSP），

也被称为 SP_main，一个指针称为进程栈指针（PSP），也为称为 SP_process，只能用在线程模式。可以通过 CONTROL 寄存器，选择使用那个栈指针。

在 ARM 处理器中，由于寄存器是 32 位的，所以 PUSH 和 POP 指令永远是 32 位操作，而且存取地址是 32 位字对齐（32 位对齐）。在处理器上电过程中，中断向量表的头 4 字节会被去除（中断向量表在 0x000000000 地址），然后填充到 MSP，作为 MSP 的初始值，PSP 的初始值为定义。

一般使用操作系统是 PSP 进程栈指针才会被使用，这是因为操作系统内核的栈空间和线程级的应用程序的栈空间是相互独立的。

R14，链接寄存器（LR）

R14 为链接寄存器，用于存储子程序或者函数调用的返回地址。子程序或者函数执行完毕，存储在 LR 中的返回地址将被装在到程序计数器 PC 中，以便调用程序可以继续执行。当发生异常中断时，LR 会提供一个特定值，用于中断返回机制。

尽管 Cortex-M0 处理器的函数返回地址始终时偶数（最低位为 0，因为最小的指令都是 16 位的），LR 的 0 位时可读写的。为了指明当前处于 thumb 状态，一些指令需要函数地址最低位为 1。

R15，程序计数器（PC）

R15 为程序计数器，为可读写。**读操作返回当前正在执行的指令加上 4（这是由流水线的特性决定的）**，而写入 R15 会导致程序跳转执行，这和函数调用不同，链接寄存器不会更新。

Cortex-M0 处理器指令是 16 位对齐的，所以PC 寄存器的最低位必须始终为 0.不过在使用跳转指令 BX 或者 BLX 执行跳转执行时，PC 的最低位应该被置 1，以表明目标分支处于 thumb程序区域。如果试图切换到 Cortex-M0 未知的 ARM 状态，错误异常中断会被触发。

注意，r0~r12 没有初始值。

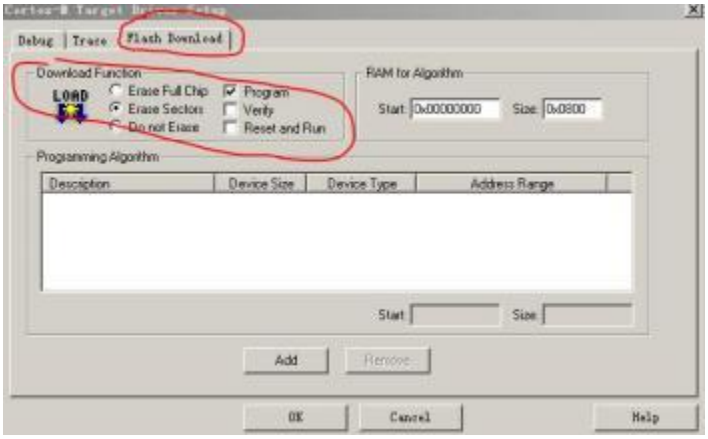
BL 很常用,它在跳转之前会在寄存器 LR(R14)中保存 PC 的当前内容。BL 的经典用法如下：

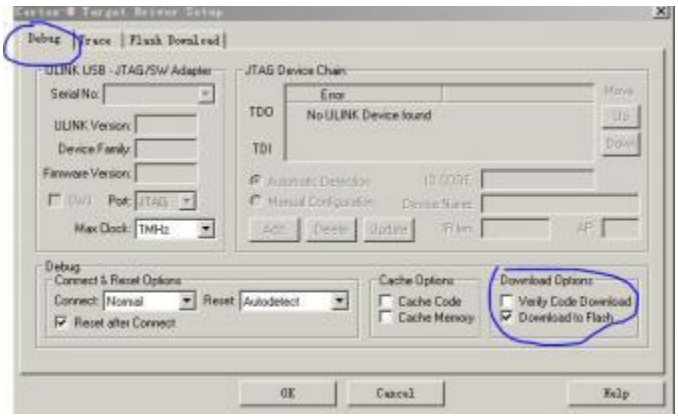
```
bl NEXT          ; 跳转到 NEXT
------

NEXT
……
mov pc, lr       ; 从子程序返回。
```

11．注意点

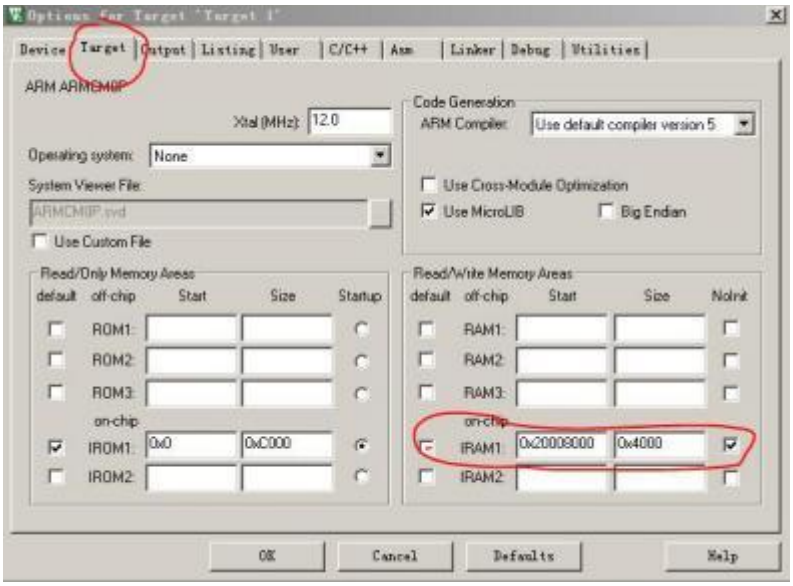
1. 当 keil 在硬件 debug模式下，如果观察窗口的地址是一个不存在的地址，那么只要一点击 debug按钮就会死机，因为它要去读一个不存在的地址，硬件就会挂掉，解决方法是此时配置为软仿状态，把观察窗口地址修改正确，再进行硬件仿真。

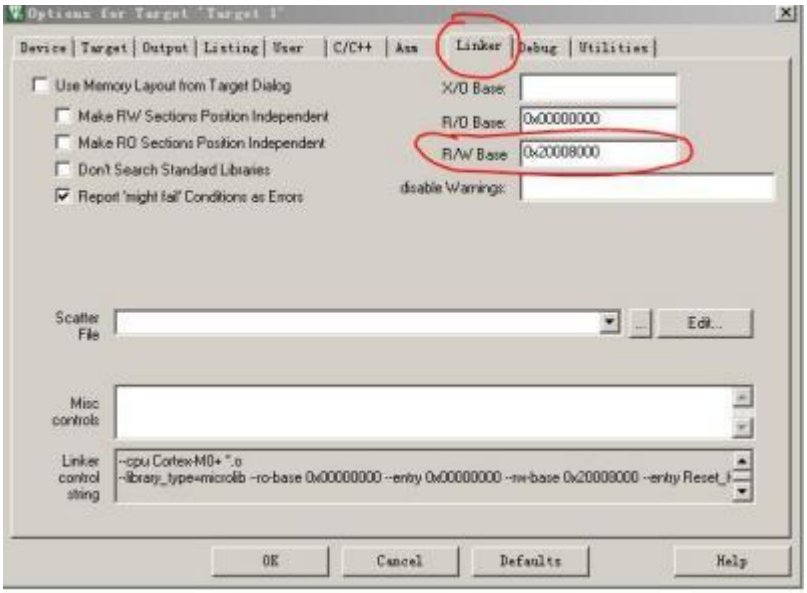
2. Cortex 调试设计为：当点击 debug，程序是下载在 flash 中（keil 必须配置下载到 flash），当退出 debug 状态，程序会继续执行！在 debug 和 flash download 选项当中，都有关于 flash 的配置



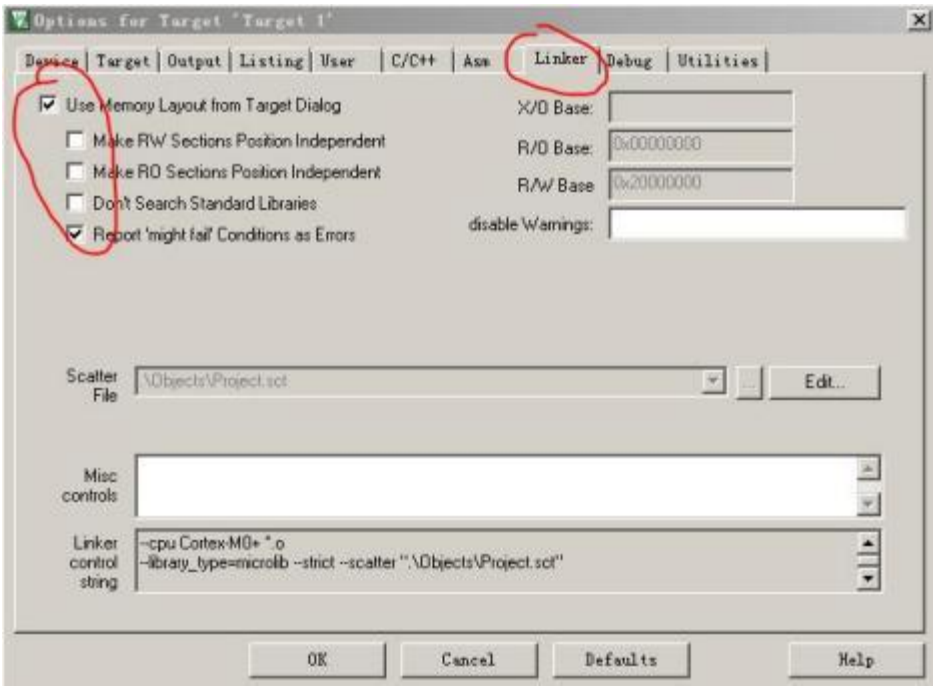


3. 如果通过界面限制变量和堆栈空间，需要配置如下图两个地方





或者配置如下图



上面选项就是根据 Target 对话框中配置来 link.

3. KEIL5 破解成功也会有 32k 代码限制
   以管理员身份开启kei15 进行破解。

4. Comtex0 启动说明

Cpu 固定从地址 0 取数据，此数据为msp，然后从 04 地址取数据，此数据为 cpu 复位地址，即 pc 指针的初始值---第一条程序所放的地址（不是用户的main 函数，是系统的_main 函数），在系统的main 函数里面执行

_scatterload,_scatterload_null,scatterload_zeroinit,_rt_lib_init,rt_lib_init_fp_1,_rt_lib_init_alloca_1,_rt_lib_shutdown,_rt_lib_shutdown_fp_trap_1,_rt_entry,_rt_entry_li,_rt_entry_mian,_rt_exit,_rt_exit_ls,_rt_wxit_exit 等函数，这些函数会初始化根据配置产生的一些 xram 空间，或者程序搬移（分散加载文件可以配置）；然后再执行用户的main 函数。

从工作站仿真角度来看，当初始化 code 区间时，一定要把msp，

reset_handle 放在 code 空间的第一个 word 和第二个 word,其它数据按照分散加载文件放在对应的地址；当用 fpga 验证时，则需要.ini文件来指定这两个地址

5. 从 ahb 总线来看，总线的取指总是提前 pc 执行，因为一般都会预取指令。
6. Keil5 arm 编译程序时，全局变量在编译时就会开辟一个地址给它，而函数中的变量编译时没有，只有在用的时候才分配空间，可能占用的空间为堆栈空间，也就是在 map 文件只会有全局变量信息，而没有函数内部定义的临时变量信息，如果局部变量有很大的数组，需要注意 sp 的大小够不够。Keil 并不会根据程序自动改变堆栈大小；而是根据设置来处理，如下程序；

```
int main(void)
{
  u8 bTxbuf[520];// _at_ 0x20001000  ;
  u8  TvDa;
      for(TvDa=0;TvDa<100;TvDa++)
      bTxbuf[TvDa]=TvDa;
  while(1);
```

}

编译后，sp 的信息是

__initial_sp　　0x20000200　Data　0　startup.o(STACK)

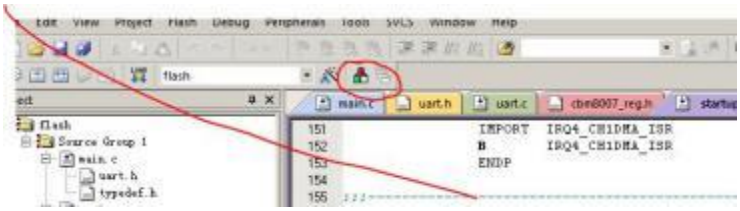
汇编为

```
    4:          u8 bTxbuf[520];// _at_ 0x20001000 ;
    5:          u8      TvDa;
0x00000102 B0FF      SUB       sp,sp,#0x1FC
0x00000104 B083      SUB       sp,sp,#0x0C
    6:                  for(TvDa=0;TvDa<100;TvDa++)
0x00000106 2000      MOVS      r0,#0x00
0x00000108 E003      B         0x00000112
    7:                  bTxbuf[TvDa]=TvDa;
0x0000010A 4669      MOV       r1,sp
0x0000010C 5408      STRB      r0,[r1,r0]
0x0000010E 1C41      ADDS      r1,r0,#1
```

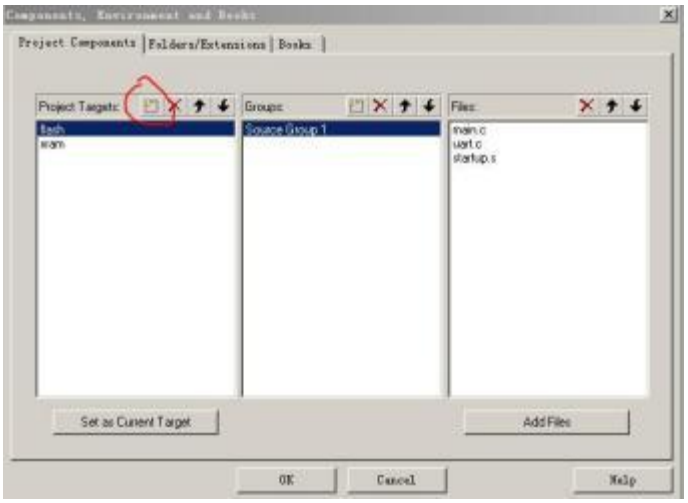
可见使用的变量 bTxbuf[520]地址为 sp-使用的大小，而 sp 的栈顶为 0x200(‘d512),小于 bTxbuf 深度，所以当运行到这个变量时就会进入 hardfault 中断。

12．分别配置

相同的 code 可以根据不同的配置生成不同的代码,配置名如下图可改

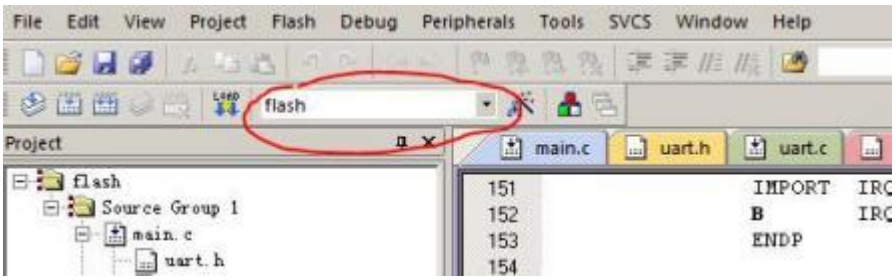


点击红圈按钮,出现如下图

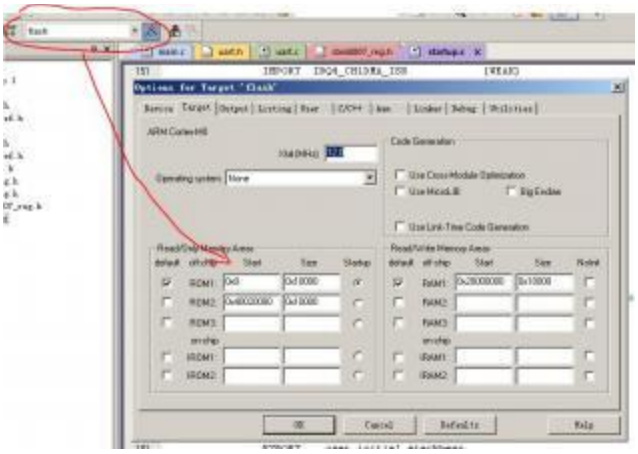


可以增加名称,

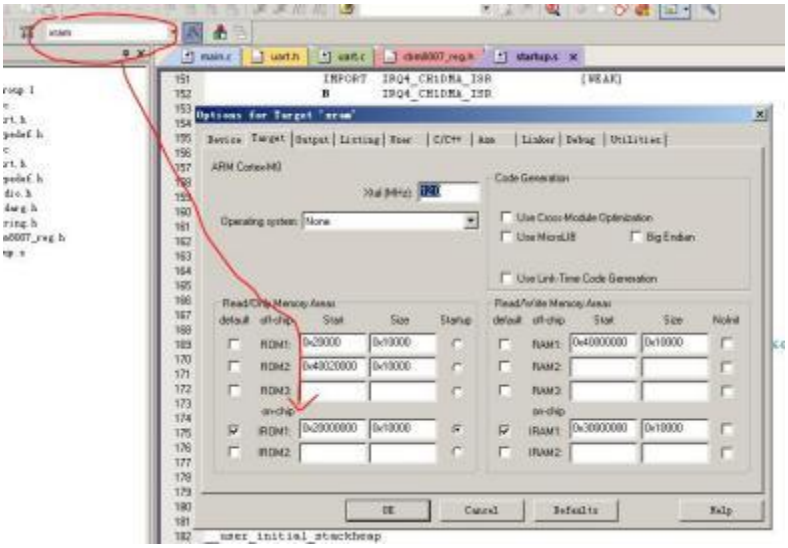
然后选择 target,]



如下配置,进行编译



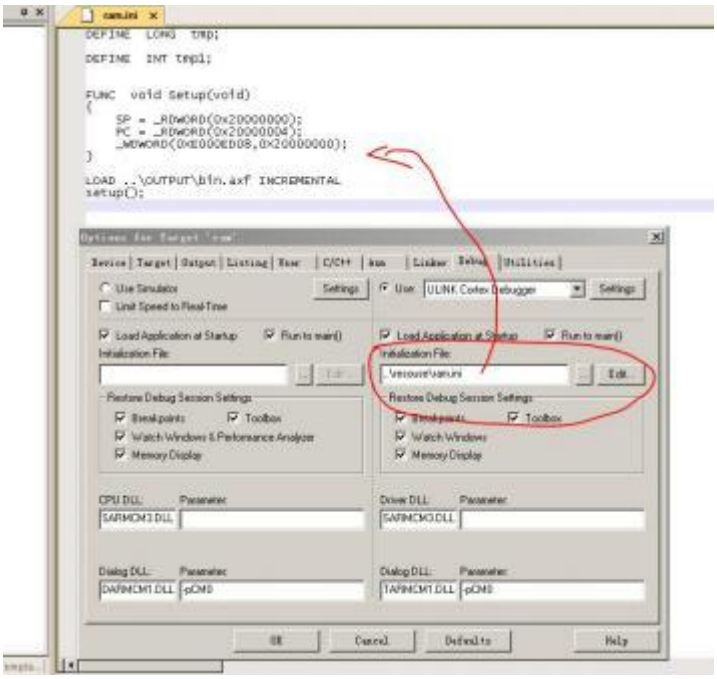
或者选择下图配置编译



他们编译出来的代码不一致,因为起始地址不一致,但功能完全一致.

当使用程序在 xram 空间,仿真需要加入一个xram.ini 的文件,其内容如下:

```
DEFINE  LONG  tmp;
DEFINE  INT tmp1;
FUNC  void Setup (void)
 {
    SP = _RDWORD(0x20000000);  //code 区域第一个 word 是 sp 值
    PC = _RDWORD(0x20000004);  //code 区域第二个 word 是复位值
    _WDWORD(0XE000ED08,0X20000000);   //0XE000ED08 为VECTOR 寄存器地址,意思是中断向量表偏移到0x2000_0000 开始,M0 无此寄存器,M0+才有
 }
LOAD ..\OUTPUT\bin.axf INCREMENTAL  //bin.axf 文件是生成的文件，根据项目需要相应修改
setup ();
```



Option->Debugger->Download 取消勾选 use flash loader

13．KEIL 中启动文件详解（汇编语言）

概述

在嵌入式系统中，启动文件是整个系统非常关键的部分，它会进行一些底层的初始化，构建程序运行必要的环境，比如堆栈初始化，变量初始化等。如果启动文件出现错误，则整个系统就跑不起来，因此研究启动文件非常必要。

在 keil 中，启动文件由汇编代码编写，一般命名为 startup_xxx.s，xxx 为支持的某种芯片，比如可以是 lpc15xx（NXP 的 LPC15xx 系列）、MK60D10（飞思卡尔）、stm32f10x（意法半导体 stm32f10x 系列）等 Cortext-M0/M3/M4 内核芯片。它们的代码格式非常相近，根据启动文件代码由上到下的编写顺序.

可以将其分为以下 5 个典型部分：

1.堆栈空间定义；

2.存放中断向量表；

3.复位中断函数（Reset_Handler）；

4.其它中断异常服务函数，以及弱[WEAK]声明；

5.将堆栈地址传递给库函数，利用库函数初始化堆栈，和库函数自身初始化。

5 个部分具体说明如下：

1.堆栈空间定义

如下图所示，定义了栈大小 Stack_Size = 0X200，即 512 字节；堆大小 Heap_Size = 0X100,256 字节。还定义了三个标号：__initial_sp（栈顶）、__heap_base（堆起始地址）和__heap_limit（堆终止地址），它们的空间由 SPACE 关键字来申请，并记作 Stack_Mem 和 Heap_Mem。

```
26  Stack_Size      EQU     0x00000200
27
28                  AREA    STACK, NOINIT, READWRITE, ALIGN=3
29  Stack_Mem       SPACE   Stack_Size
30  __initial_sp
31
32
33  ; <h> Heap Configuration
34  ;   <o>  Heap Size (in Bytes) <0x0-0xFFFFFFFF:8>
35  ; </h>
36
37  Heap_Size       EQU     0x00000100
38
39                  AREA    HEAP, NOINIT, READWRITE, ALIGN=3
40  __heap_base
41  Heap_Mem        SPACE   Heap_Size
42  __heap_limit
```

通过这些我们可以很容易的知道堆栈的大小，但是它们的绝对地址或者说基地址仅仅从这里是得不到的。编译器编译完工程后，根据生成.bss段（比如未初始化的全局变量）和.data 段（比如初始化的全局变量）的大小以及 RAM 的起始地址，来计算堆栈的基地址。

举个例子：

一个芯片的 RAM 起始地址为 0x0200_0000，RAM 大小为 0x500 字节，程序编译后.bss 段为 0x100 个字节，.data 段为 0x100 个字节。堆栈大小定义如上图。则：

A：堆起始地址 __heap_base==Heap_Mem==0x0200_0200；

B：堆终止地址即栈底 __heap_limit==Stack_Mem==0x0200_0300；

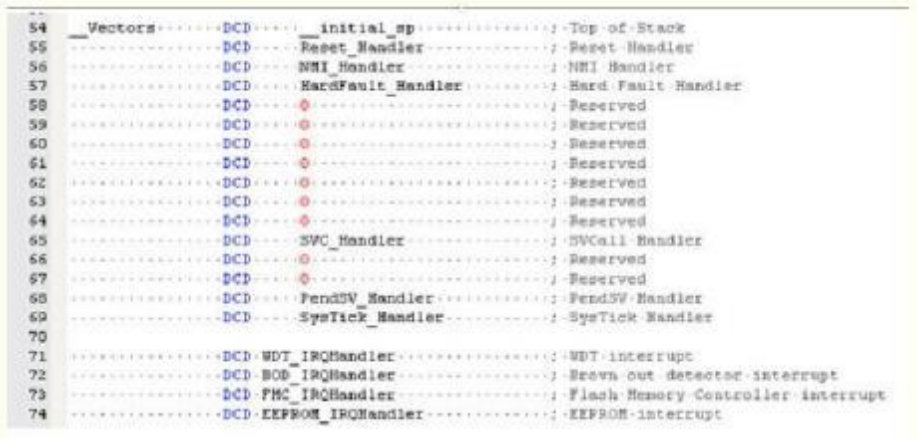
C：栈顶地址 __initial_sp==0x0200_0500（栈是向下生长，栈顶处于 RAM 最大地址处）。

其实，我可以在.map 文件中查看堆栈的大小和基地址，如下图所示：

```
  HEAP                                    0x02000200
  Heap_Mem                                0x02000200
  STACK                                   0x02000300
  Stack_Mem                               0x02000300
  __initial_sp                            0x02000500
```

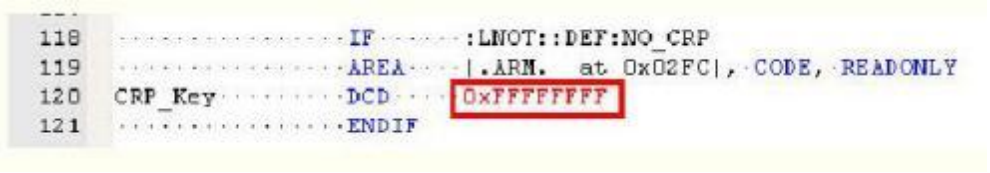
2.存放中断向量表

在启动代码中，会见到许多由 DCD 申请空间存放的一个个函数入口，即中断向量表，如下图所示，只列出了部分。



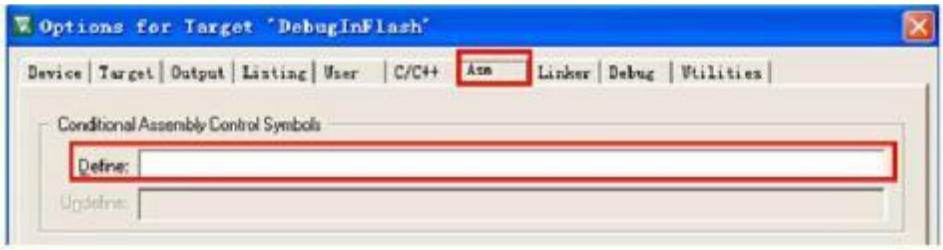
关键字 DCD 代表申请一个字的空间，后面的函数名即为中断服务函数入口地址。另外中断向量表一般存放在Flash 0 地址。

另外，对于 NXP 微控制器，均实现了芯片的加密，加密的设置在向量表的结尾处，具体地址为0x02FC 处。通过在此地址存放不同的值实现是否加密或者加密的等级。加密分为三个等级，CRP1:0x12345678；CRP2:0x87654321；CRP3:0x43218765。至于每个等级的具体说明请参考芯片用户手册。下面说一下加密步骤，以 CRP1 为例：



首先将下图中 0xFFFFFFFF，修改为 0x12345678。

其次，图中 IF :LNOT::DEF:NOCRP 表示如果没有定义宏 NOCRP 则执行下面的代码，那么必须保证汇编中没有定义 NOCRP 宏。即保证下图中Define:一栏中没有定义 NOCRP 即可。



3. 复位中断函数（Reset_Handler）

程序上电后，首先加载 SP 和 PC，ARM 规定从 0 地址处加载 SP，从偏移为 4 的地址（0x00000004）处加载 PC。然后将程序控制权交给程序。我们知道 0 地址处存放__initial_sp，0x00000004 地址处存放 Reset_Handler，加载 PC 后，程序跳转到 Reset_Handler 开始运行。Reset_Handler 函数体如下图所示：

```
133  Reset_Handler   PROC
134                  EXPORT  Reset_Handler             [WEAK]
135                  IMPORT  SystemInit
136                  IMPORT  __main
137                  LDR     R0, =SystemInit
138                  BLX     R0
139                  LDR     R0, =__main
140                  BX      R0
141                  ENDP
```

首先调用 SystemInit 函数来初始化系统的各种时钟，然后调用__main 函数（由 KEIL 微库或者 C 库实现），在__main 函数中：.data 段数据的初始化->.bss 段变量清零->设置堆栈指针->库函数初始化（比如常用的 malloc 函数）->如果必要会设置 main 函数的 argc 和 argv 两个参数->调用用户 main 函数->退出。

4．其它中断异常服务函数，以及弱[WEAK]声明

```
148  NMI_Handler     PROC
149                  EXPORT  NMI_Handler               [WEAK]
150                  B       .
151                  ENDP
152  HardFault_Handler\
153                  PROC
154                  EXPORT  HardFault_Handler         [WEAK]
155                  B       .
156                  ENDP
157  SVC_Handler     PROC
158                  EXPORT  SVC_Handler               [WEAK]
159                  B       .
160                  ENDP
161  PendSV_Handler  PROC
162                  EXPORT  PendSV_Handler            [WEAK]
163                  B       .
164                  ENDP
165  SysTick_Handler PROC
166                  EXPORT  SysTick_Handler           [WEAK]
167                  B       .
168                  ENDP
169  Reserved_IRQHandler PROC
170                  EXPORT  Reserved_IRQHandler       [WEAK]
171                  B       .
172                  ENDP
173
174  Default_Handler PROC
175                  EXPORT  WDT_IRQHandler            [WEAK]
176                  EXPORT  BOD_IRQHandler            [WEAK]
177                  EXPORT  FMC_IRQHandler            [WEAK]
```

如上图所示，这里的中断服务函数是弱声明的（由[WEAK]关键字标注）。所谓弱声明，即：如果用户定义了相同的函数则此处函数失效而使用用户定义的中断服务函数。这样是为了防止用户使能了中断而没有中断服务函数，从而造成程序崩溃。假设使能了中断，而用户又没有定义中断服务函数则会进入默认中断，如下图所示，默认中断为死循环（死循环与程序崩溃不是一个概念）。

```
266  ACMP3_IRQHandler
267  QEI_IRQHandler
268  RTC_ALARM_IRQHandler
269  RTC_WAKE_IRQHandler
270
271                  B       .
```

5.将堆栈地址传递给库函数

第三步骤中，调用__main 函数，然后__main 调用库函数初始化堆栈，但库函数并不知道堆栈的大小，因此我们需要告诉它，具体做法就是传递参数或声明标号。

下图为具体做法，可以看到第一行为：

IF :DEF:__MICROLIB

是条件编译选项，如果定义__MICROLIB，则编译图中红线上面部分，否则编译红线下面部分。那么就分 2 种情况。

```
273 ·····IF·····:DEF:__MICROLIB
274 ·····
275 ·····EXPORT··__initial_sp
276 ·····EXPORT··__heap_base
色填充 ·····EXPORT··__heap_limit
278 ·····
279 ·····ELSE
280 ·····
281 ·····IMPORT··__use_two_region_memory
282 ·····EXPORT··__user_initial_stackheap
283 __user_initial_stackheap
284 
285 ·····LDR·····R0,·=··Heap_Mem
286 ·····LDR·····R1,·=(Stack_Mem·+·Stack_Size)
287 ·····LDR·····R2,·=·(Heap_Mem·+··Heap_Size)
288 ·····LDR·····R3,·=·Stack_Mem
289 ·····BX·····LR
```

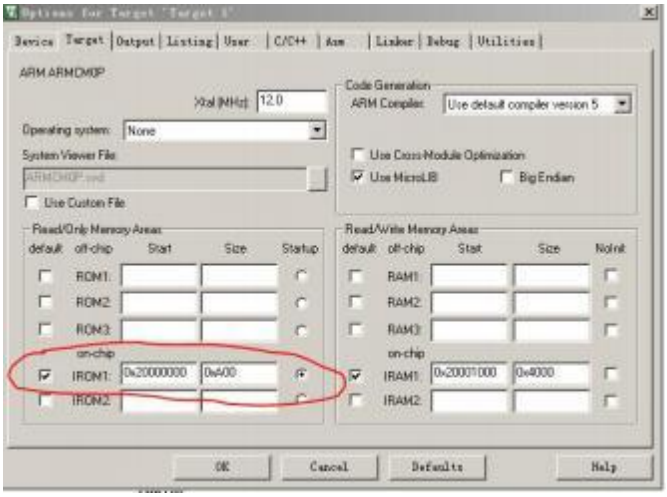
2 种情况的选择可以如下实现：

如果勾选【Options for Target】->【Target】->【Use MicroLIB】，如下图所示。即使用微库，则__MICROLIB 会被定义，编译器编译红线以上代码。用 EXPORT 声明 __initial_sp、__heap_base 和__heap_limit。

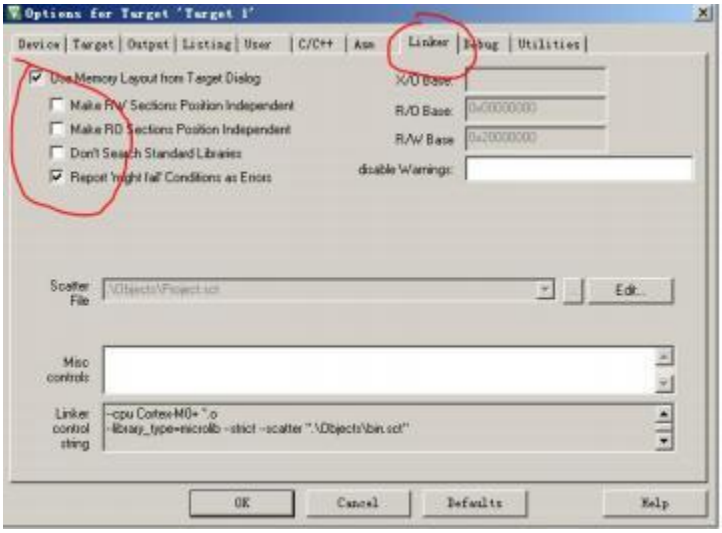
如果不勾选【Use MicroLIB】，则缺省使用 KEIL C 库，上图红线以下会参与编译，KEIL C 库函数会调用__user_initial_stackheap，通过 R0~R3 将堆栈以参数形式传递给KEIL C 库。

14．程序在 ram运行分析

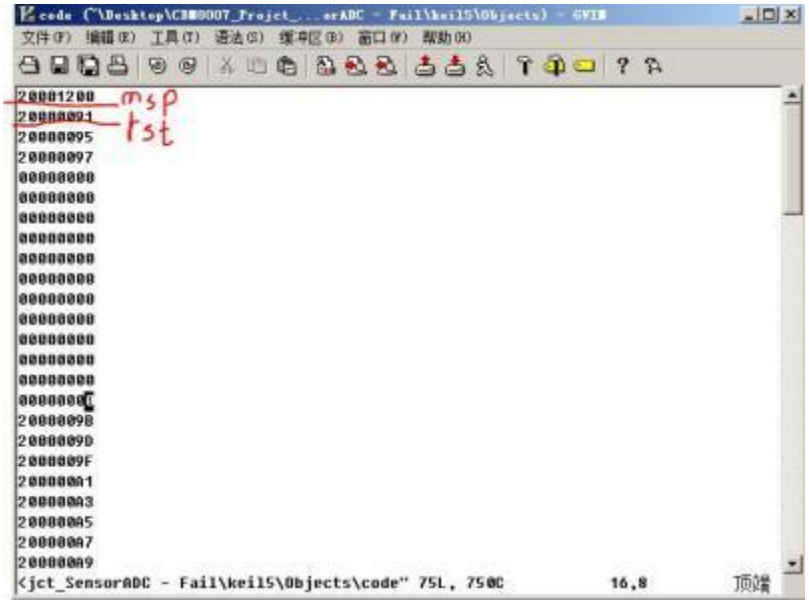
当程序放在 ram 运行时，需要做如下配置，irom 地址要在 ram 空间范围



Link 时如下配置

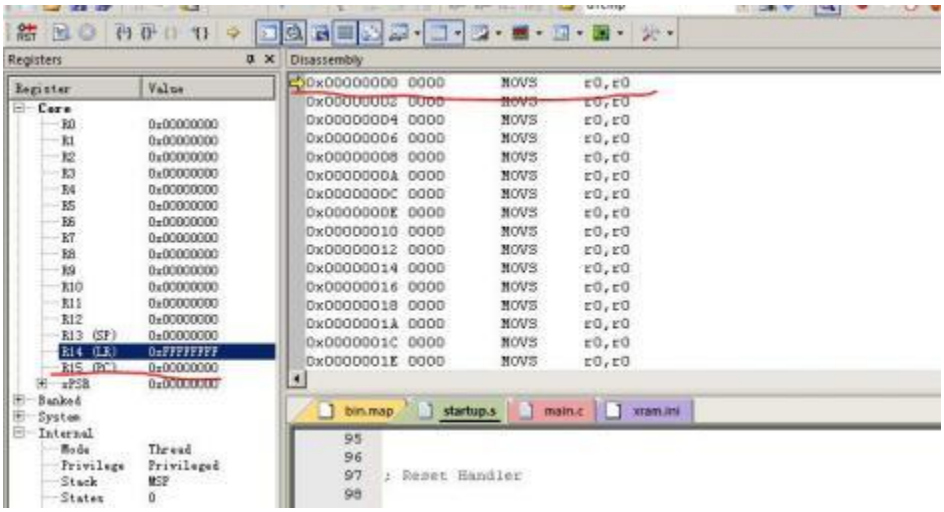


编译之后的代码如下

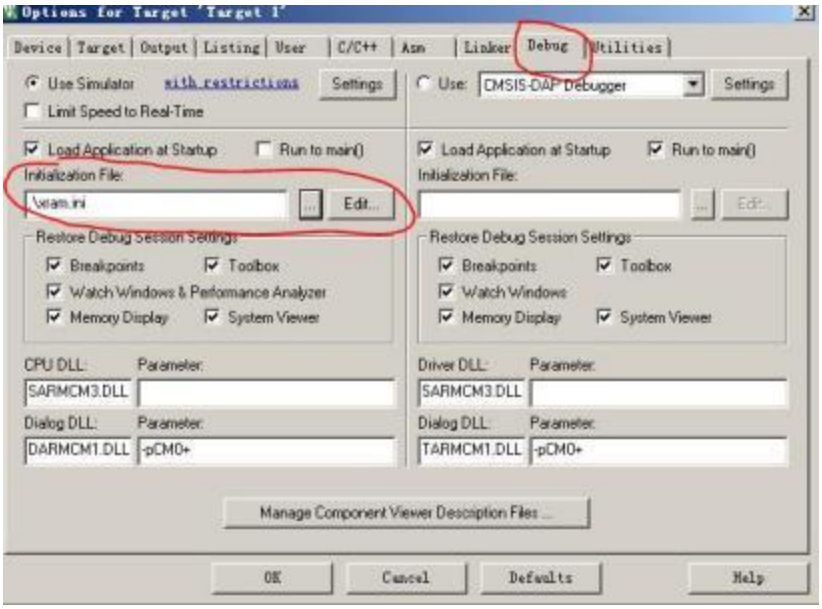


第一行是msp，第二行是复位向量，

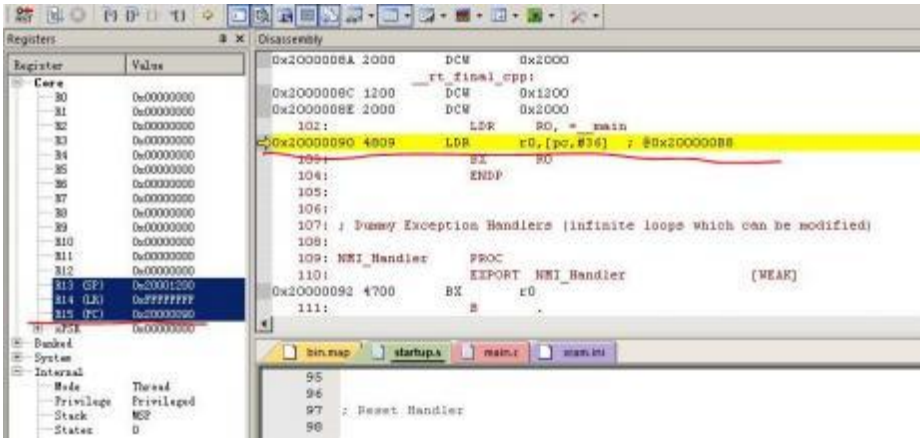
当仿真（软件仿真/硬件仿真）时需要添加一个.ini文件，如果不添加则无法仿真；下面是不添加 ini文件，程序首先会跑道 00 地址，而此处地址在 debug环境中是没有内容的，如下



如果添加了 ini文件，如下



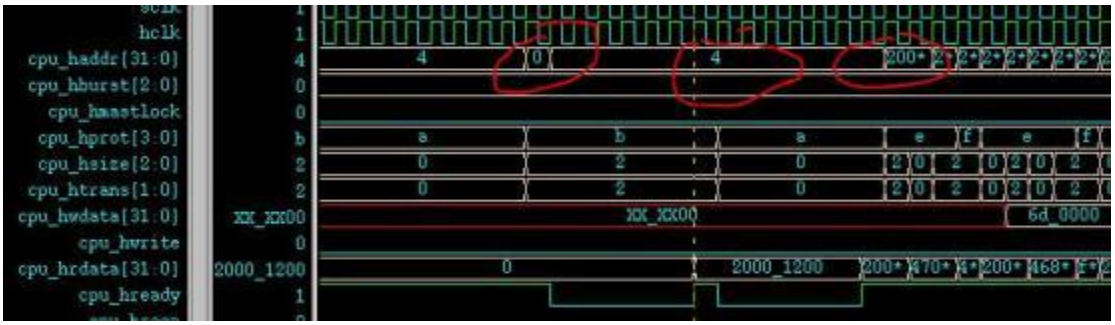
则 debug 会直接跳到启动地址 0x20000090(地址为 word 对齐，所以不会是 0 小 x200000091)



Ini 文件相当于做了 cpu 的如下动作：

1. 从 0 地址读取 msp 地址
2. 从 4 地址读取 reset 地址
3. 跳转到 reset 地址

所以当程序在 xram 运行，程序需要放在ram 中，但程序的头 2 个 word 还是在 code(flash)中。下面是仿真波形，程序预先初始化在 xram,flash 中。



此场景与 load 在 code 区，exec 在 ram 不同（程序放在 code/flash 中，启动时启动代码把主程序搬到xram，然后再在 xram 运行---通过分散加载文件实现）。

由上分析可知，无论程序是在 code/flash 区运行还是在 xram 运行，cpu 总是从 0x0（code/flash 区）地址启动，读取 msp，然后从 0x4 地址读取复位向量，然后再根据复位向量跳转到对应的地址。

15．启动程序分析(转)

启动代码作用一般是：1）堆和栈的初始化；2）向量表定义；3）地址重映射及中断向量表的转移；

；4）设置系统时钟频率；5）中断寄存器的初始化；6）进入 C 应用程序。

```
Stack_Size EQU 0x00000400 ; 定义 Stack_Size 为 0x00000400
AREA STACK, NOINIT, READWRITE, ALIGN=3 ;定义栈，可初始为 0，8 字节对齐
Stack_Mem SPACE Stack_Size ;分配 0x200 个连续字节，并初始化为 0
__initial_sp ;汇编代码地址标号
; <h> Heap Configuration
; <o> Heap Size (in Bytes) <0x0-0xFFFFFFFF:8>
; </h>
Heap_Size EQU 0x00000000 ;定义 Heap_Size 为 0x00000000
AREA HEAP, NOINIT, READWRITE, ALIGN=3 ;定义堆，可初始为 0，8 字节对齐
__heap_base
Heap_Mem SPACE Heap_Size ;分配 0x00 个连续字节，并初始化为 0
```

__heap_limit

PRESERVE8 ; 指定当前文件堆栈 8 字节对齐

THUMB ; 告诉汇编器下面是 32 为的 Thumb 指令，如果需要汇编器将插入位以保证对齐

; Vector Table Mapped to Address 0 at Reset

;以下中断向量表定义

AREA RESET, DATA, READONLY ;定义复位向量段，只读

EXPORT __Vectors ; 定义一个可以在其他文件中使用的全局标号。此处表示中断地址

__Vectors DCD __initial_sp ; Top of Stack 给__initial_sp 分配 4 字节 32 位的地址 0x0

DCD Reset_Handler ; Reset Handler 给标号 Reset Handler 分配地址为 0x00000004

DCD NMI_Handler ; NMI Handler 给标号 NMI Handler 分配地址 0x00000008

DCD HardFault_Handler ; Hard Fault Handler

DCD 0 ; Reserved 这种形式就是保留地址，不给任何标号分配

DCD 0 ; Reserved

DCD 0 ; Reserved

DCD 0 ; Reserved

DCD 0 ; Reserved

DCD 0 ; Reserved

DCD 0 ; Reserved

DCD SVC_Handler

; SVCall Handler

DCD 0 ; Reserved

DCD 0 ; Reserved

DCD PendSV_Handler ; PendSV Handler

DCD SysTick_Handler ; SysTick Handler

; External Interrupts

```
; maximum of 32 External Interrupts are possible

DCD BOD_IRQHandler
DCD WDT_IRQHandler
DCD EINT0_IRQHandler
DCD EINT1_IRQHandler
DCD GPIOP0P1_IRQHandler
DCD GPIOP2P3P4_IRQHandler
DCD PWMA_IRQHandler
DCD PWMB_IRQHandler
DCD TMR0_IRQHandler
DCD TMR1_IRQHandler
DCD TMR2_IRQHandler
DCD TMR3_IRQHandler
DCD UART0_IRQHandler
DCD UART1_IRQHandler
DCD SPI0_IRQHandler
DCD SPI1_IRQHandler
DCD Default_Handler
DCD Default_Handler
DCD I2C_IRQHandler
DCD Default_Handler
DCD Default_Handler
DCD PWRWU_IRQHandler
DCD ADC_IRQHandler
DCD Default_Handler
DCD Default_Handler
; 中断向量表的转移
AREA |.text|, CODE, READONLY ; 代码段定义
```

; Reset Handler

;ENTRY

Reset_Handler PROC ; 标记一个函数的开始 (这里是复位子程序从 0x04 开始

EXPORT Reset_Handler [WEAK] ;将 Reset_Handler 设为外部可引用的标号

; [WEAK] 选项表示当所有的源文件都没有 定义这样一个标号时，

;编译器也不给出错误信息，在多数情况下将该标号置为 0 ，

;若该标号为 B 或 BL 指令引用，则将 B 或 BL 指令置为 NOP 操作。

;EXPORT 提示编译器该标号可以为外部文件引用。

;IMPORT SystemInit [WEAK] ;在自建工程时一般会出现这一行 ,并在复位子程序结束处跳到 MAIN 前

;会先跳到这个初始化程序,如果你的主程序里没有这一初始化程序

;而是在MAIN 中初

始化,将会导致编译器因找不到这一程序而报错

;解决的办法是注解这一行，或加上 [WEAK]

;当然后面跳到 MAIN 之前进初始化函数必须注解掉

IMPORT __main ; 通知编译器要 main 在其他的源文件中定义，但要在当前源文件中引用

LDR R0, =0x50000100 ;送立即数到 R0 这个是寄存器写保护寄存器 (复位为锁定 bit0 为 0)

LDR R1, [R0, #0x10] ;将 0x50000110(rc 校验寄存器)处的值送到 R1

CMP R1, #0x3F ;比较 0x50000110 处的值与 0x3f (默认的中心频率应该是 0x20)

BNE %F0 ; 如果不相等向前搜索标号为 0 的行，(以此实现循环，具体可看局部标号的知识)

;如果相等就解锁 寄存器写保护寄存器 并 将 rc 校验寄存器 设为中心值0x20

LDR R1, =0x59

STR R1, [R0]

LDR R1, =0x16

STR R1, [R0]

LDR R1, =0x88

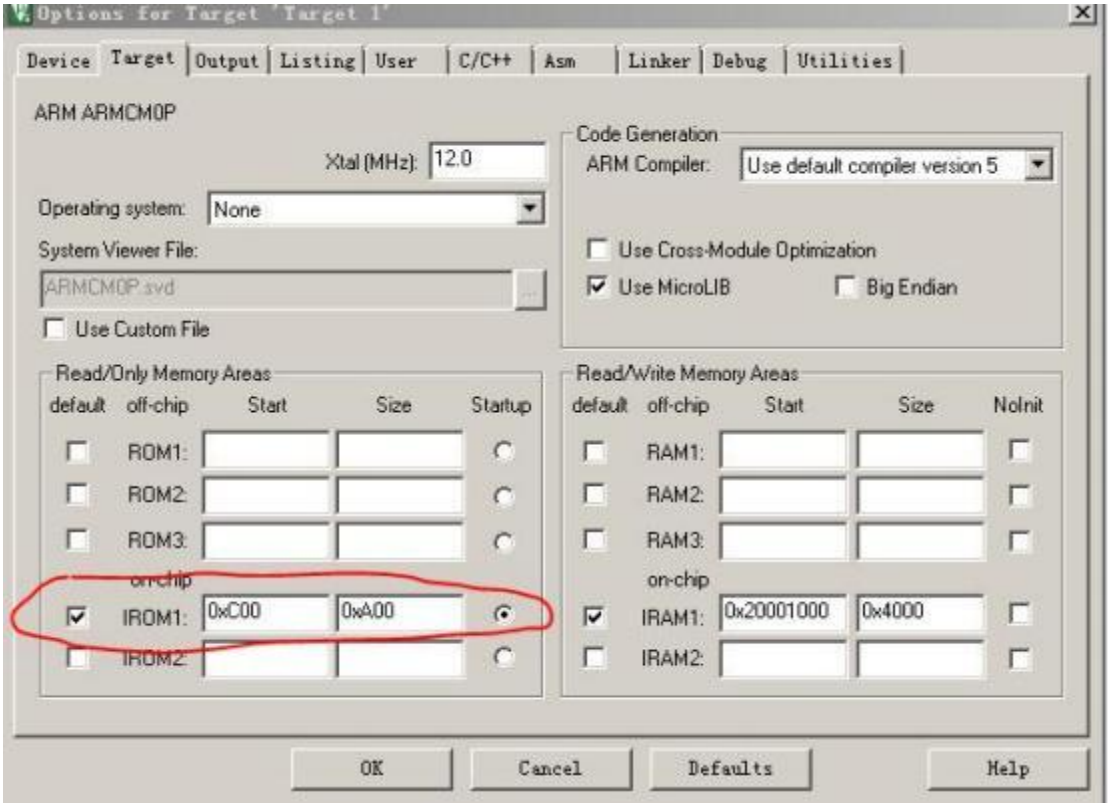
```
STR R1, [R0] ; 向 0x50000100 写入 0x59 0x16 0x88 解锁寄存器写保护寄存器(bit0 为 1)
LDR R1, =0x20 ;加载 0x20 到 R1
STR R1, [R0, #0x10] ;将 R1(0x20 默认中心频率)送到 0x50000110(rc 校验寄存器)
MOVS R1, #0 ;将 0 送到 R1
STR R1, [R0] ;将 0 写入 0x50000100 重锁保护寄存器
0 ;标号
;LDR R0, =SystemInit
;BLX R0 ;这就是前面说的进初始化函数，如果你主程序里没有，必须注解掉。
LDR R0, =__main ; 使用“=”表示 LDR 目前是伪指令不是标准指令。这里是把__main 的地址给 R0。
BX R0 ; BX 是 ARM 指令集和 THUMB 指令集之间程序的跳转
; 也就是跳到 MAIN 处
ENDP ; 复位子程序结束
; Dummy Exception Handlers (infinite loops which can be modified)
NMI_Handler PROC ; 函数的开始
EXPORT NMI_Handler [WEAK] ; EXPORT NMI_Handler 可外部引用 的标号
B . ; 跳到当前行
ENDP ; 函数的结束
HardFault_Handler\
PROC
; "/"是换行的意思
EXPORT HardFault_Handler [WEAK]
B .
ENDP
SVC_Handler PROC
EXPORT SVC_Handler [WEAK]
B .
```

```
ENDP
PendSV_Handler PROC
EXPORT PendSV_Handler [WEAK]
B .
ENDP
SysTick_Handler PROC
EXPORT SysTick_Handler [WEAK]
B .
ENDP
Default_Handler PROC
EXPORT BOD_IRQHandler [WEAK]
EXPORT WDT_IRQHandler [WEAK]
EXPORT EINT0_IRQHandler [WEAK]
EXPORT EINT1_IRQHandler [WEAK]
EXPORT GPIOP0P1_IRQHandler [WEAK]
EXPORT GPIOP2P3P4_IRQHandler [WEAK]
EXPORT PWMA_IRQHandler [WEAK]
EXPORT PWMB_IRQHandler [WEAK]
EXPORT TMR0_IRQHandler [WEAK]
EXPORT TMR1_IRQHandler [WEAK]
EXPORT TMR2_IRQHandler [WEAK]
EXPORT TMR3_IRQHandler [WEAK]
EXPORT UART0_IRQHandler [WEAK]
EXPORT UART1_IRQHandler [WEAK]
EXPORT SPI0_IRQHandler [WEAK]
EXPORT SPI1_IRQHandler [WEAK]
EXPORT I2C_IRQHandler [WEAK]
EXPORT PWRWU_IRQHandler [WEAK]
```

```
EXPORT ADC_IRQHandler [WEAK]


BOD_IRQHandler
WDT_IRQHandler
EINT0_IRQHandler
EINT1_IRQHandler
GPIOP0P1_IRQHandler
GPIOP2P3P4_IRQHandler
PWMA_IRQHandler
PWMB_IRQHandler
TMR0_IRQHandler
TMR1_IRQHandler
TMR2_IRQHandler
TMR3_IRQHandler
UART0_IRQHandler
UART1_IRQHandler
SPI0_IRQHandler
SPI1_IRQHandler
I2C_IRQHandler
PWRWU_IRQHandler
ADC_IRQHandler
B .
ENDP
ALIGN
; User Initial Stack & Heap
; 堆和栈的初始化
IF :DEF:__MICROLIB ;; “DEF”的用法——:DEF:X 就是说 X 定义了则为真，否则为假
EXPORT __initial_sp
```

```
EXPORT __heap_base
EXPORT __heap_limit
ELSE
IMPORT __use_two_region_memory
EXPORT __user_initial_stackheap
__user_initial_stackheap
LDR R0, = Heap_Mem
LDR R1, = (Stack_Mem + Stack_Size)
LDR R2, = (Heap_Mem + Heap_Size)
LDR R3, = Stack_Mem
BX LR
ALIGN
ENDIF
END
```

如下图配置



其产生的.map 文件如下

```
RESET                                0x00000c00   Section      124  startup.o(RESET)
.ARM.Collect$$$$00000000             0x00000c7c   Section        0  entry.o(.ARM.Collect$$$$00000000)
.ARM.Collect$$$$00000001             0x00000c7c   Section        4  entry2.o(.ARM.Collect$$$$00000001)
.ARM.Collect$$$$00000004             0x00000c80   Section        4  entry5.o(.ARM.Collect$$$$00000004)
.ARM.Collect$$$$00000008             0x00000c84   Section        0  entry7b.o(.ARM.Collect$$$$00000008)
.ARM.Collect$$$$0000000A             0x00000c84   Section        0  entry9b.o(.ARM.Collect$$$$0000000A)
.ARM.Collect$$$$0000000B             0x00000c84   Section        8  entry9a.o(.ARM.Collect$$$$0000000B)
.ARM.Collect$$$$0000000D             0x00000c8c   Section        0  entry10a.o(.ARM.Collect$$$$0000000D)
.ARM.Collect$$$$0000000F             0x00000c8c   Section        0  entry11a.o(.ARM.Collect$$$$0000000F)
.ARM.Collect$$$$00002712             0x00000c8c   Section        4  entry2.o(.ARM.Collect$$$$00002712)
__lit__00000000                      0x00000c8c   Data           4  entry2.o(.ARM.Collect$$$$00002712)
.text                                0x00000c90   Section       48  startup.o(.text)
.text                                0x00000cc0   Section       36  init.o(.text)
i.__scatterload_copy                 0x00000ce4   Section       14  handlers.o(i.__scatterload_copy)
i.__scatterload_null                 0x00000cf2   Section        2  handlers.o(i.__scatterload_null)
i.__scatterload_zeroinit             0x00000cf4   Section       14  handlers.o(i.__scatterload_zeroinit)
i.main                               0x00000d02   Section        0  main.o(i.main)
STACK                                0x20001000   Section      512  startup.o(STACK)
```

其中 reset 影射到了 0xc00

```
__Vectors                            0x00000c00   Data           4  startup.o(RESET)
__main                               0x00000c7d   Thumb Code     0  entry.o(.ARM.Collect$$$$00000000)
_main_stk                            0x00000c7d   Thumb Code     0  entry2.o(.ARM.Collect$$$$00000001)
_main_scatterload                    0x00000c81   Thumb Code     0  entry5.o(.ARM.Collect$$$$00000004)
__main_after_scatterload             0x00000c85   Thumb Code     0  entry5.o(.ARM.Collect$$$$00000004)
_main_clock                          0x00000c85   Thumb Code     0  entry7b.o(.ARM.Collect$$$$00000008)
_main_cpp_init                       0x00000c85   Thumb Code     0  entry9b.o(.ARM.Collect$$$$0000000A)
_main_init                           0x00000c85   Thumb Code     0  entry9a.o(.ARM.Collect$$$$0000000B)
__rt_final_cpp                       0x00000c8d   Thumb Code     0  entry10a.o(.ARM.Collect$$$$0000000D)
__rt_final_exit                      0x00000c8d   Thumb Code     0  entry11a.o(.ARM.Collect$$$$0000000F)
Reset_Handler                        0x00000c91   Thumb Code     4  startup.o(.text)
NMI_Handler                          0x00000c95   Thumb Code     2  startup.o(.text)
HardFault_Handler                    0x00000c97   Thumb Code     4  startup.o(.text)
_IRQ_BAK_Handler                     0x00000c9b   Thumb Code     2  startup.o(.text)
```

_vectors 中断向量基地址,所有的中断入口地址为_vectors+offset,如下图 vector 表格,DCD 表示 4 个 BYTE,M0 无 VECTORS 寄存器,所以中断入口地址固定,M0+才有该寄存器,可以配置

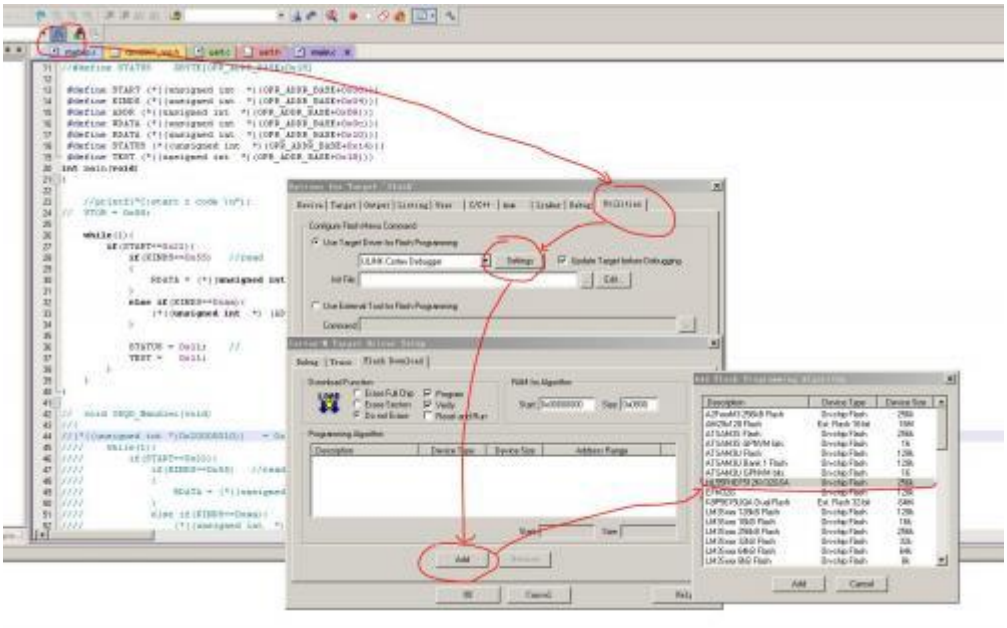
```
; Vector Table Mapped to Address 0 at Reset

                AREA    RESET, DATA, READONLY
                EXPORT  __Vectors

__Vectors       DCD     __initial_sp              ;0 Top of Stack
                DCD     Reset_Handler             ;1 Reset Handler
                DCD     NMI_Handler               ;2 NMI Handler
                DCD     HardFault_Handler         ;3 Hard Fault Handler
                DCD     0                         ;4 Reserved
                DCD     0                         ;5 Reserved
                DCD     0                         ;6 Reserved
                DCD     0                         ;7 Reserved
                DCD     0                         ;8 Reserved
                DCD     0                         ;9 Reserved
                DCD     0                         ;10 Reserved
                DCD     0                         ;11 Reserved
                DCD     0                         ;12 Reserved
                DCD     0                         ;13 Reserved
                DCD     0                         ;14 Reserved
                DCD     0                         ;15 Reserved
;;;-------------------------------------------------------------------------
; External Interrupts  CBM8007
                DCD     _IRQ_BAK_Handler                ; 16:
                DCD     _IRQ_GPIO_Handler               ; 17:
                DCD     _IRQ_UART_Handler ;             ; 18:
                DCD     _IRQ_TIMER0_Handler             ; 19:
                DCD     _IRQ_TIMER1_Handler             ; 20:
                DCD     _IRQ_TIMER2_Handler             ; 21:
                DCD     _IRQ_SPI_Handler                ; 22:
                DCD     _IRQ_FSCV4X_Handler             ; 23:
                DCD     _IRQ_SADC_Handler               ; 24:
                DCD     _IRQ_ADC_Handler                ; 25:
                DCD     _IRQ_IIC_Handler                ; 26:

                DCD     _IRQ_INT0_Handler               ; 27:
```

中断入口地址为 4*(numb+1);如_IRQ_TIMER2_Handler 的入口地址为 (20+1)*4=84(0x54),此处放的跳转指令,跳到对应的中断服务程序,如下为 0x340 地址

```
0x000002ae  0x000002ae  0x00000002  Code  RO     55  i._IRQ_GPIO_Handler  cbm8007xx_isr.o
0x000002b0  0x000002b0  0x00000002  Code  RO     56  i._IRQ_IIC_Handler  cbm8007xx_isr.o
0x000002b2  0x000002b2  0x00000002  PAD
0x000002b4  0x000002b4  0x00000014  Code  RO     57  i._IRQ_INT0_Handler  cbm8007xx_isr.o
0x000002c8  0x000002c8  0x00000014  Code  RO     58  i._IRQ_INT1_Handler  cbm8007xx_isr.o
0x000002dc  0x000002dc  0x00000014  Code  RO     59  i._IRQ_INT2_Handler  cbm8007xx_isr.o
0x000002f0  0x000002f0  0x00000014  Code  RO     60  i._IRQ_INT3_Handler  cbm8007xx_isr.o
0x00000304  0x00000304  0x00000010  Code  RO     61  i._IRQ_SADC_Handler  cbm8007xx_isr.o
0x00000314  0x00000314  0x00000002  Code  RO     62  i._IRQ_SPI_Handler  cbm8007xx_isr.o
0x00000316  0x00000316  0x00000002  PAD
0x00000318  0x00000318  0x00000014  Code  RO     63  i._IRQ_TIMER0_Handler  cbm8007xx_isr.o
0x0000032c  0x0000032c  0x00000014  Code  RO     64  i._IRQ_TIMER1_Handler  cbm8007xx_isr.o
0x00000340  0x00000340  0x00000028  Code  RO     65  i._IRQ_TIMER2_Handler  cbm8007xx_isr.o
0x00000368  0x00000368  0x00000002  Code  RO     66  i._IRQ_UART_Handler  cbm8007xx_isr.o
0x0000036a  0x0000036a  0x0000000e  Code  RO    198  i.__scatterload_copy  mc_p.l(handlers.o)
0x00000378  0x00000378  0x00000002  Code  RO    199  i.__scatterload_null  mc_p.l(handlers.o)
0x0000037a  0x0000037a  0x0000000e  Code  RO    200  i.__scatterload_zeroinit  mc_p.l(handlers.o)
0x00000388  0x00000388  0x0000008c  Code  RO     10  i.main  main.o
0x00000414  0x00000414  0x00000020  Data  RO    196  Region$$Table  anon$$obj.o
```

16．下载程序

当用keil 工具下载程序时，由于不同的芯片操作 flash 的方法不同，所以需要添加一个.FLM 文件在如下位置（把生成的 flm 文件 copy 到安装目录下的一个叫 flash 的文件夹下 ）



上图中HL55HEXXXXXX 文件就是我们根据 flash 的特性用 keil 生成的一个.flm 文件，

.flm 文件就是.axf 文件，只是要求.flm 为后缀

含 program,erase 函数，如下

```
#include "FlashOS.H"        // FlashOS Structures
typedef volatile unsigned long  vu32;
#define M32(adr) (*((vu32 *) (adr)))
// Peripheral Memory Map
#define FLASH_BASE      0x40020000
#define FLASH           ((FLASH_TypeDef*) FLASH_BASE)
// Flash Registers
typedef struct {
  vu32 FOSR;                                    // offset
0x000
```

```
    vu32 FOCR;                                  // offset
0x004
    vu32 TADR;                                  // offset
0x008
    vu32 WRDR;                                  // offset
0x00C
    vu32 OCMR;                                  // offset
0x010
    vu32 RDDR;                                  // offset
0x014
    vu32 CWCR;                                  // offset
0x018
    vu32 FWCR;                                  // offset
0x01C
    vu32 FPSR;                                  // offset
0x020
  } FLASH_TypeDef;

  int Init (unsigned long adr, unsigned long clk, unsigned long
fnc) {

    /* Add your Code */

    return (0);                                 // Finished
without Errors
  }

  int UnInit (unsigned long fnc) {
```

```
  /* Add your Code */
  return (0);                                  // Finished
without Errors
}

int EraseChip (void) {
  FLASH->FOCR =  0xa;                          // Mass Erase
Enabled
  FLASH->OCMR =  0xa;                          // Start Erase
  while (FLASH->OCMR) {
    ;                                          // Reload Read
  }

  return (0);                                  // Finished
without Errors
}

int EraseSector (unsigned long adr) {
  FLASH->FOCR =  0x9;                          // Mass Erase
Enabled
  FLASH->TADR =  adr;
  FLASH->OCMR =  0xa;                          // Start Erase
  while (FLASH->OCMR) {
    ;                                          // Reload Read
  }
  return (0);                                  // Finished
without Errors
}
```

```
int ProgramPage (unsigned long adr, unsigned long sz, unsigned
char *buf)  {

    FLASH->FOCR =  0xc;          // write page enable
    FLASH->TADR =  adr;          // write page address
    FLASH->CWCR =  sz;           // write size, count in bytes
    FLASH->OCMR =  0xa;          // Start write

    while(sz)  {
        FLASH->WRDR = *((unsigned int *)buf);  // Program Word
        buf  += 4;
        sz   -= 4;
    }
    while(FLASH->OCMR);
    return  (0);                 // Finished without Errors
}
```

上面函数是标准接口，固定的函数名，不能改。

"FlashOS.H"文件内容如下

```
#define VERS       1          // Interface Version 1.01
#define UNKNOWN    0          // Unknown
#define ONCHIP     1          // On-chip Flash Memory
#define EXT8BIT    2          // External Flash Device on
8-bit  Bus
#define EXT16BIT   3          // External Flash Device on
16-bit Bus
#define EXT32BIT   4          // External Flash Device on
32-bit Bus
```

```
    #define EXTSPI     5           // External Flash Device on
SPI
    #define SECTOR_NUM 512         // Max Number of Sector Items
    #define PAGE_MAX   65536       // Max Page Size for
Programming
    struct FlashSectors  {
      unsigned long   szSector;    // Sector Size in Bytes
      unsigned long AddrSector;    // Address of Sector
    };

    #define SECTOR_END 0xFFFFFFFF, 0xFFFFFFFF
    struct FlashDevice  {
       unsigned short     Vers;    // Version Number and
Architecture
       char       DevName[128];    // Device Name and
Description
       unsigned short  DevType;    // Device Type: ONCHIP,
EXT8BIT, EXT16BIT, ...
       unsigned long    DevAdr;    // Default Device Start
Address
       unsigned long     szDev;    // Total Size of Device
       unsigned long    szPage;    // Programming Page Size
       unsigned long       Res;    // Reserved for future
Extension
       unsigned char  valEmpty;    // Content of Erased Memory
       unsigned long    toProg;    // Time Out of Program Page
Function
       unsigned long   toErase;    // Time Out of Erase Sector
Function
```

```
    struct FlashSectors sectors[SECTOR_NUM];
};
#define FLASH_DRV_VERS (0x0100+VERS)   // Driver Version, do not modify!
// Flash Programming Functions  (Called by FlashOS)
extern          int  Init         (unsigned long adr,   // Initialize Flash
                                   unsigned long clk,
                                   unsigned long fnc);
extern          int  UnInit       (unsigned long fnc);  // De-initialize Flash
extern          int  BlankCheck   (unsigned long adr,   // Blank Check
                                   unsigned long sz,
                                   unsigned char pat);
extern          int  EraseChip    (void);           // Erase complete Device
extern          int  EraseSector  (unsigned long adr);  // Erase Sector Function
extern       int  ProgramPage  (unsigned long adr, // Program Page Function
                                unsigned long sz,
                                unsigned char *buf);
extern unsigned long Verify       (unsigned long adr,   // Verify Function
                                    unsigned long sz,
                                  unsigned char *buf);
```

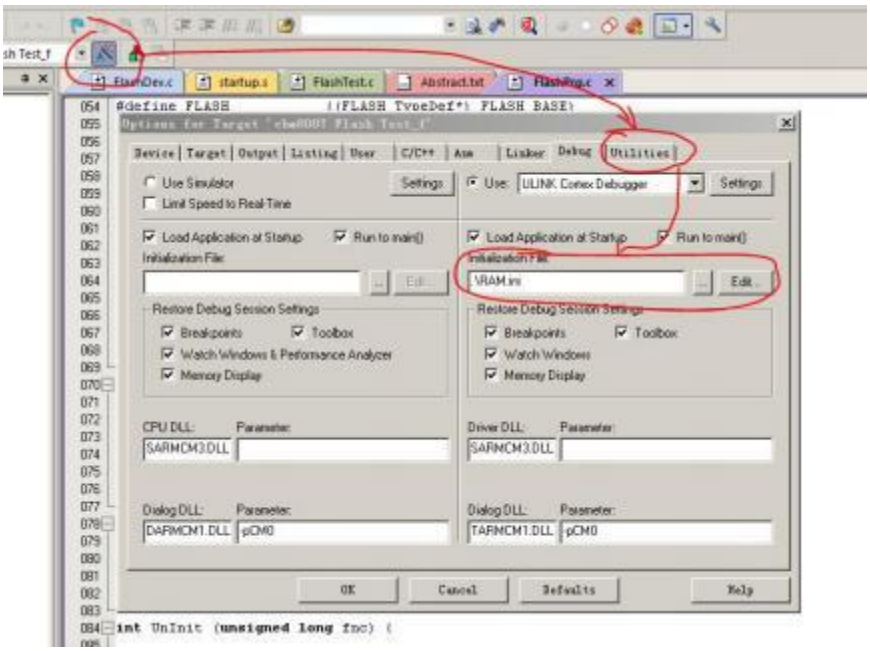
同时需要定义一个 flash 的结构体如下

```
#include "FlashOS.H"        // FlashOS Structures
```

```
    struct FlashDevice const FlashDevice  =  {
       FLASH_DRV_VERS,              // Driver Version, do not
modify!
       "HL55FHEF512KX32GSA",        // Device Name，看上图选择的
名字就是在此设定
       ONCHIP,                      // Device Type
       0x000000,                    // Device Start Address
       0x00040000,           // Device Size in Bytes (fpga
actually 256K, IC is 4MB)
       1024,                        // Programming Page Size
       0,                           // Reserved, must be 0
       0xFF,                        // Initial Content of Erased
Memory
       100,                         // Program Page Timeout 100
mSec
       3000,                        // Erase Sector Timeout 3000
mSec
    // Specify Size and Address of Sectors
       0x01000, 0x000000,           // Sector Size 4kB (64 Sector)
       SECTOR_END
};
```

注意：上述内容在keil 安装目录下，flash 文件夹中有范例。

下载程序时，需要根据情况加入.ini文件，如下图 ：



Flash.ini 文件内容如下：

```
    DEFINE  LONG  tmp;
    DEFINE  INT tmp1;
    FUNC  void Setup (void)
     {
         SP = _RDWORD(0x00000000);    //设置 SP 存放地址
         PC = _RDWORD(0x00000004);    //设置 pc 指针
         _WDWORD(0XE000ED08,0X00000000);// 0XE000ED08 为中断向量
寄存器地址,只有 M0+以上内核才有此寄存器,m0 无此寄存器
     }
    LOAD ..\OUTPUT\bin.axf  INCREMENTAL
setup ();
```

程序放在 xram 空间时.ini 文件为

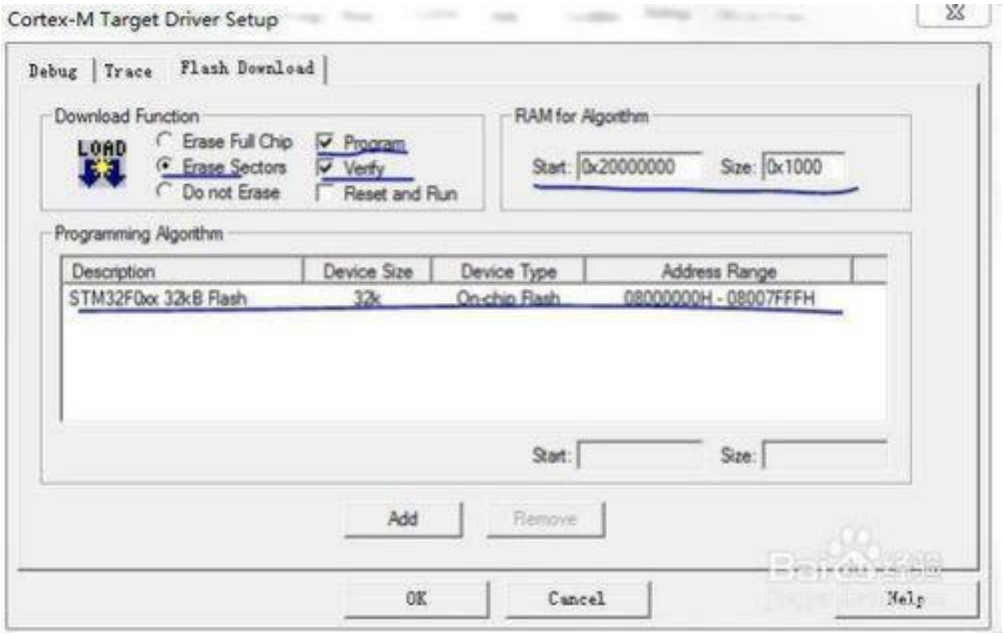
```
    DEFINE  LONG  tmp;
    DEFINE  INT tmp1;
    FUNC  void Setup (void)  //整个地址偏移 0x2000_0000
     {
         SP = _RDWORD(0x20000000);
         PC = _RDWORD(0x20000004);
```

```
        _WDWORD(0XE000ED08,0X20000000);
    }

    LOAD ..\OUTPUT\bin.axf  INCREMENTAL
setup ();
```

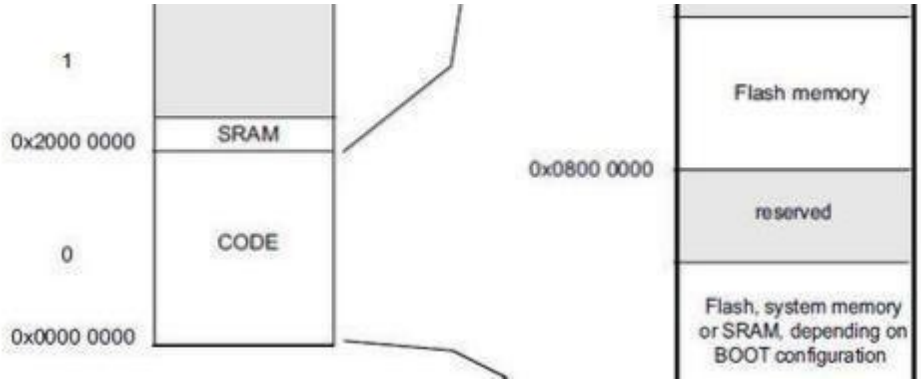
因为程序是固定从 0 地址取 sp，从 4 地址取 pc，上面就是告诉工具从哪个地方获得相关数据

17．下载配置图解(转)

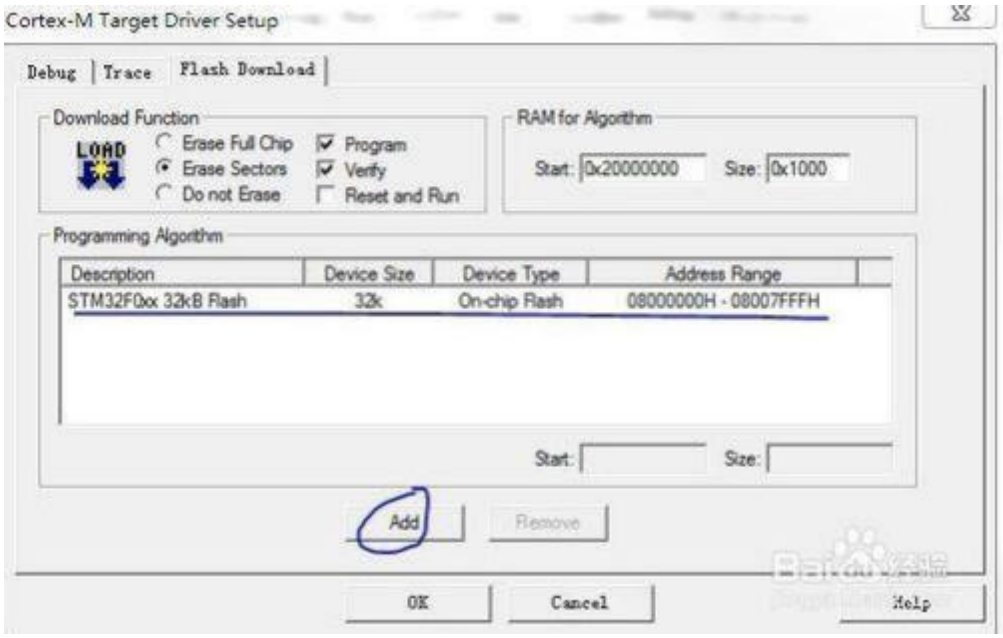
请注意图中标注部分，左上方的Download Function 是告诉下载程序算法需要做的事情。通常情况下选择图示三个即可。擦除，下载和效验。



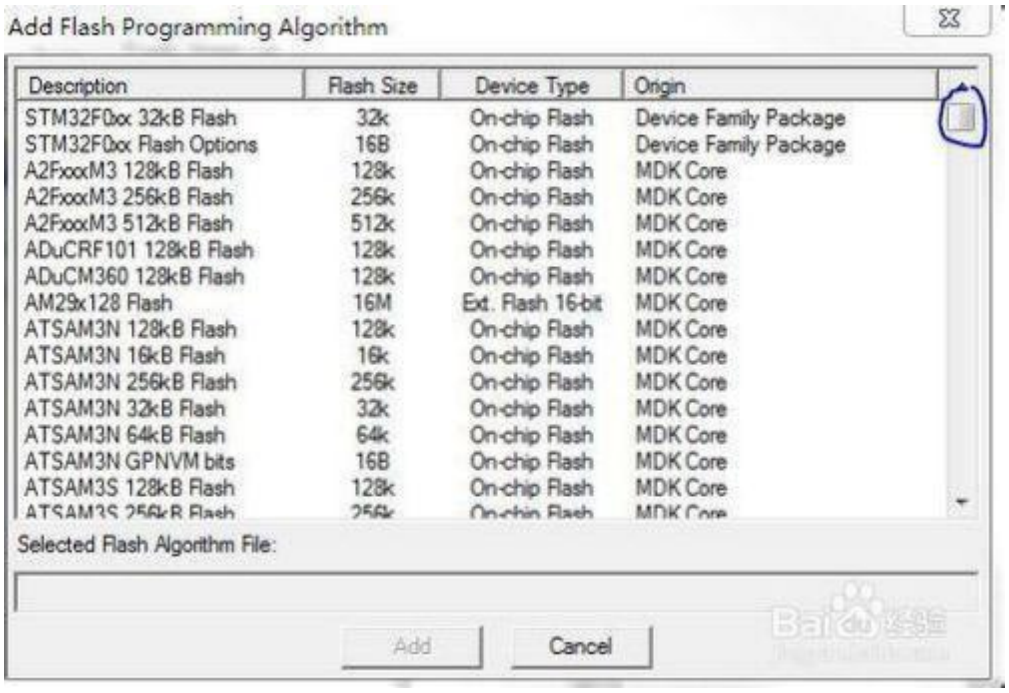
1. 关键的是如何知道 RAM for Algorithm 的起始地址和 size。首先看你前面选择的 MCU，这是你调试的基础，打开其 datasheet，以 STM32F0X 系列为例，如图示部分，SRAM 的地址和大小清晰可见。为什么和SRAM 有关呢？因为Keil首先通过 SW 接口协议将程序下载到 SRAM 中然后才会调用 FLASH 算法将程序烧录。



2. 接下来就需要找到属于该芯片的 flash 算法了，请看下图中的蓝色标注部分，那个算法如何得来的呢？请点击 ADD 按钮，将会弹出一个对话框。



3. 请下拉滚动条，你将会看到很多的算法，找到属于你的算法。



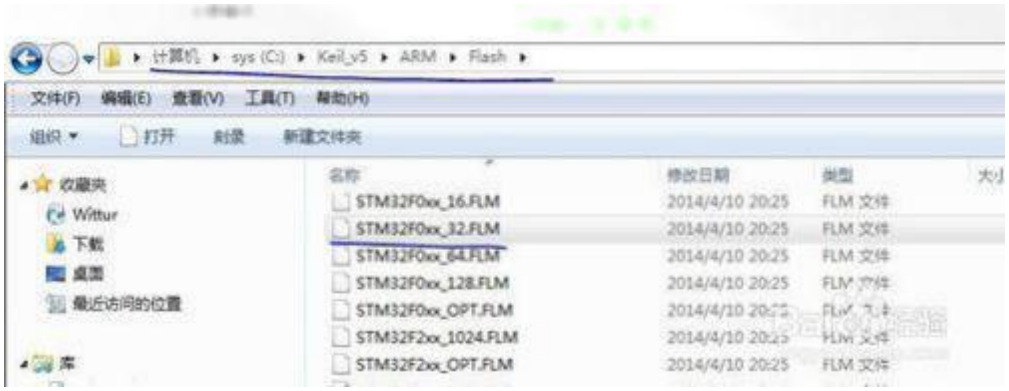
4. 前面提到了通用算法的选择，那么问题来了，这个算法文件如何来的呢？如果你所用的 MCU 不是默认支持的品牌，如何编写属于自己的算法呢？

5. 工具/原料

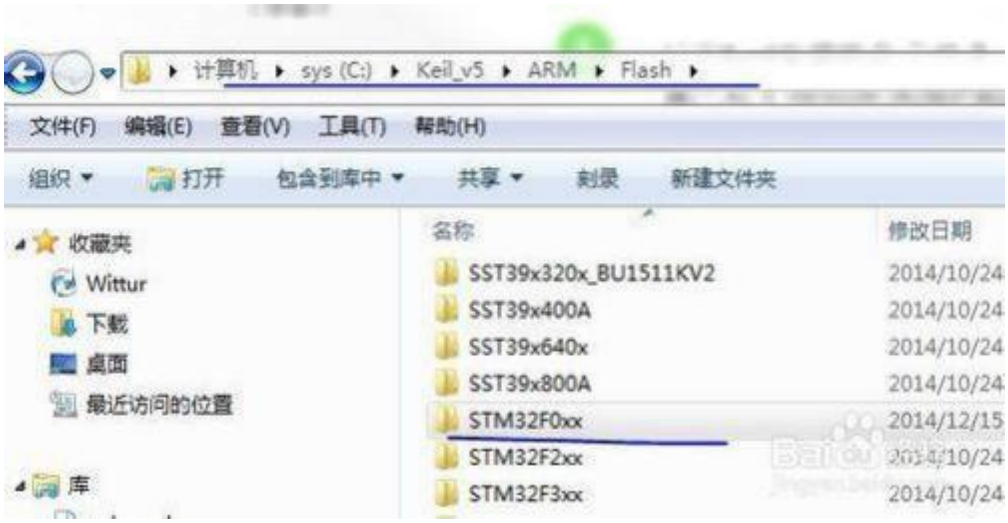
6. Keil uVisionULINK2 仿真器百度经验:jingyan.baidu.com

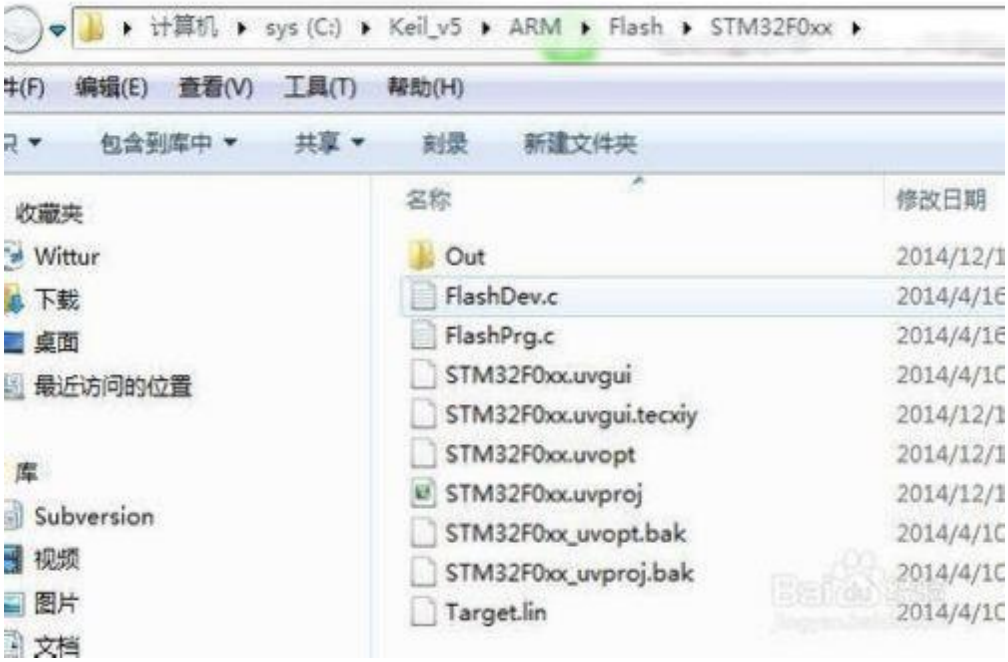
7. 方法/步骤

1 打开Keil 安装所在文件夹，如图示路径，你可以看到算法存储位置，也就是说你通过 ADD 按钮所选择的程序来自于这里，但是这个文件是如何来的呢？

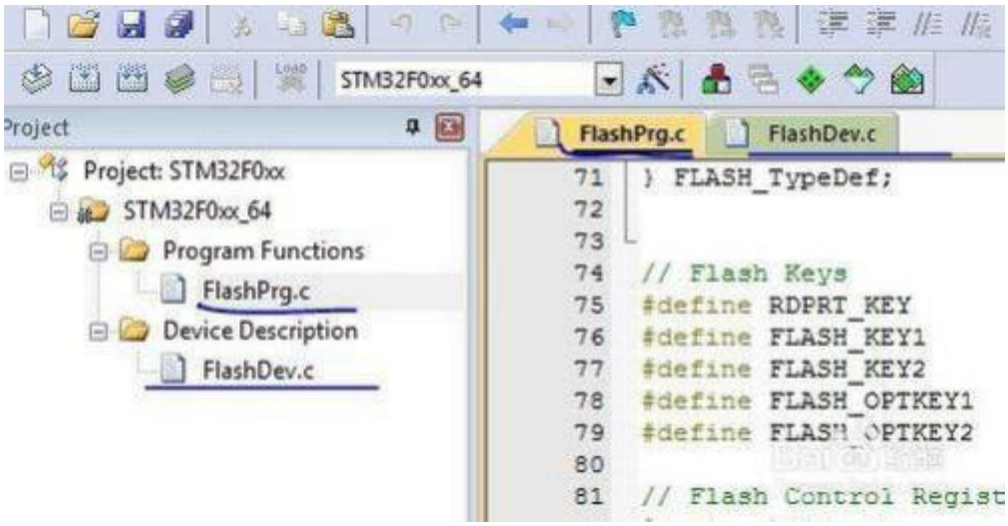


步骤阅读 2 拖动滚动条，可以找到这样一个文件夹，如图示，打开它发现一个Keil 工程，这就是 flash 算法的源码。





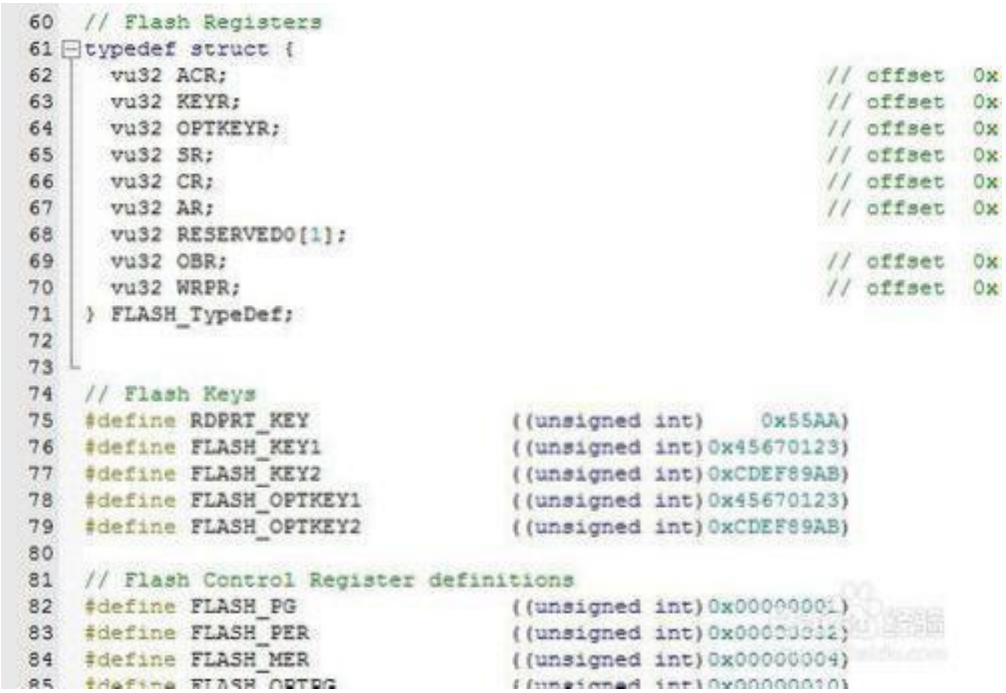
步骤阅读步骤阅读 3 打开这个工程，工程里面主要是两个 C file，注意图中标示。其中FlashDev.c 文件顾名思义是设备Flash 的接口定义，我们可以看到里面有 STM32F0xx 系列的接口。



步骤阅读 4 任意截图一例如下图，这是一个结构体，右边有相应的注释。从注释上看，Device Name 是可以更改，这个也就是你点击 ADD 按钮时所呈现的名字。接下来有 Device start address 和 Device size。继续定义 programming page size，初始化内容填充值，一个 page 下载的 time out，擦除一个 sector 的 timeout。 sector size 等，看起来应该很清晰明白。

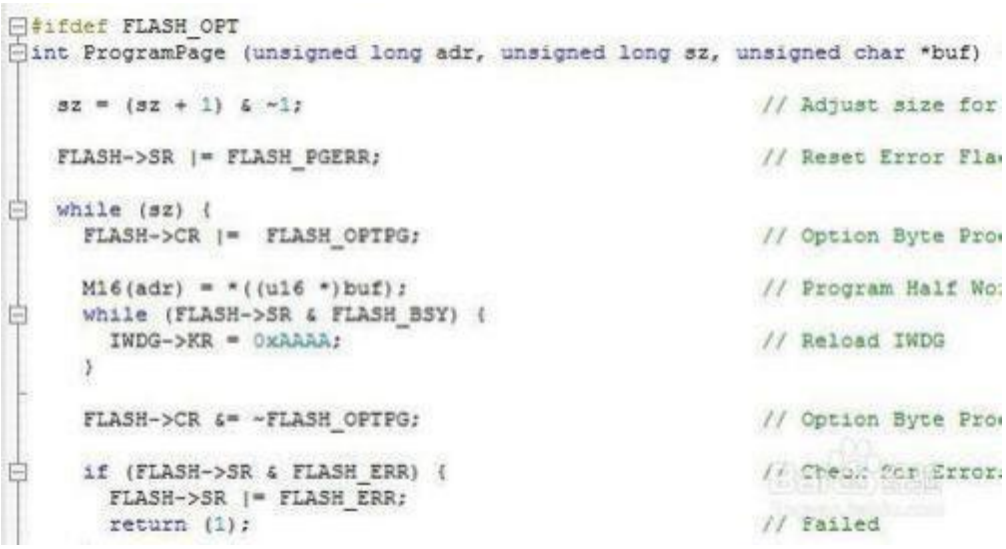
```
#ifdef STM32F0xx_16
struct FlashDevice const FlashDevice  =  {
    FLASH_DRV_VERS,             // Driver Version, do not modify!
    "STM32F0xx 16kB Flash",     // Device Name (16kB)
    ONCHIP,                     // Device Type
    0x08000000,                 // Device Start Address
    0x00004000,                 // Device Size in Bytes (64kB)
    1024,                       // Programming Page Size
    0,                          // Reserved, must be 0
    0xFF,                       // Initial Content of Erased Memory
    100,                        // Program Page Timeout 100 mSec
    6000,                       // Erase Sector Timeout 3000 mSec

// Specify Size and Address of Sectors
    0x0400, 0x000000,           // Sector Size  1kB (16 Sectors)
    SECTOR_END
};
#endif // STM32F0xx_16
```

步骤阅读 5 有了这些接口定义就对 flash 本身有了很清楚的了解，我们在看 FlashPrg.c 文件。这是算法主文件。文件的开头是一些结构体和宏的定义。请仔细阅读你所选择的芯片的 flash 部分寄存器定义，你就可以找到。



```
// Flash Registers
typedef struct {
  vu32 ACR;                                          // offset  0x
  vu32 KEYR;                                         // offset  0x
  vu32 OPTKEYR;                                      // offset  0x
  vu32 SR;                                           // offset  0x
  vu32 CR;                                           // offset  0x
  vu32 AR;                                           // offset  0x
  vu32 RESERVED0[1];
  vu32 OBR;                                          // offset  0x
  vu32 WRPR;                                         // offset  0x
} FLASH_TypeDef;

// Flash Keys
#define RDPRT_KEY               ((unsigned int)     0x55AA)
#define FLASH_KEY1              ((unsigned int)0x45670123)
#define FLASH_KEY2              ((unsigned int)0xCDEF89AB)
#define FLASH_OPTKEY1           ((unsigned int)0x45670123)
#define FLASH_OPTKEY2           ((unsigned int)0xCDEF89AB)

// Flash Control Register definitions
#define FLASH_PG                ((unsigned int)0x00000001)
#define FLASH_PER               ((unsigned int)0x00000002)
#define FLASH_MER               ((unsigned int)0x00000004)
```

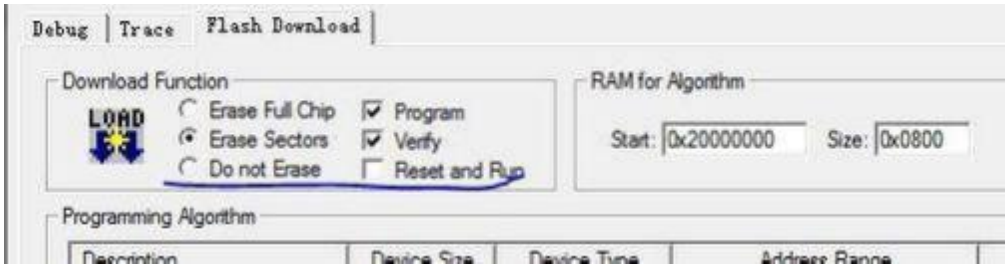
步骤阅读 6 接下来可注意到下面有不同的函数定义，Init()函数很显然是对 flash 的一个初始化，仔细阅读芯片 specification 即可知道。对应的是 UnInit()。你还可以看到 int EraseChip (void)， int EraseSector (unsigned long adr)，int ProgramPage (unsigned long adr, unsigned long sz, unsigned char *buf)， 是不是很熟悉的感觉？



```
#ifdef FLASH_OPT
int ProgramPage (unsigned long adr, unsigned long sz, unsigned char *buf)

  sz = (sz + 1) & ~1;                                // Adjust size for

  FLASH->SR |= FLASH_PGERR;                          // Reset Error Fla

  while (sz) {
    FLASH->CR |=  FLASH_OPTPG;                       // Option Byte Pro

    M16(adr) = *((u16 *)buf);                        // Program Half Wo
    while (FLASH->SR & FLASH_BSY) {
      IWDG->KR = 0xAAAA;                             // Reload IWDG
    }

    FLASH->CR &= ~FLASH_OPTPG;                       // Option Byte Pro

    if (FLASH->SR & FLASH_ERR) {                     // Check for Error
      FLASH->SR |= FLASH_ERR;
      return (1);                                    // Failed
```

步骤阅读 7 让我们再次回顾下图中蓝色标注部分，你可看到其实在下载之初你已经选择过类似的接口，下载过程中将会调用出现在该文件中的程序，

而如果你自己编写，就只要替换其中的代码即可，接口Keil 已经为你准备完毕。如何编写，那么清仔细阅读相应 flash controller 章节的定义。



18．Keil 内存解说

keil 编译后 code，RO-data,RW-data,ZI-data 含义及 mcu 的 flash 实际存储数据

bss 段：

bss 段（bss segment）通常是指用来存放程序中未初始化的全局变量的一块内存区域。

bss 是英文 Block Started by Symbol 的简称。

bss 段属于静态内存分配。

data 段：

数据段（data segment）通常是指用来存放程序中已初始化的全局变量的一块内存区域。

数据段属于静态内存分配。 初始化的内容是变异在rom，上电时从rom读出再写入到该区域中。全局变量是在编译时就分配好的，与静态变量一样

text 段：

代码段（code segment/text segment）通常是指用来存放程序执行代码的一块内存区域。

这部分区域的大小在程序运行前就已经确定，并且内存区域通常属于只读(某些架构也允许代码段为可写，即允许修改程序)。

在代码段中，也有可能包含一些只读的常数变量，例如字符串常量等。

堆（heap）：

堆是用于存放进程运行中被动态分配的内存段，它的大小并不固定，可动态扩张或缩减。

当进程调用malloc 等函数分配内存时，新分配的内存就被动态添加到堆上（堆被扩张）；

当利用free 等函数释放内存时，被释放的内存从堆中被剔除（堆被缩减）。--与 stack对应，不过是由用户来申请内存

栈(stack)：

栈又称堆栈，是用户存放程序临时创建的**局部变量**， ---在map 文件中看不到函数中定义的变量，这些变量在运行过程中自动分配在 stack 中，所以如果定义和的变量大小大于 stack 的大小，则会报内存益出。全局变量/静态变量才能在map 文件看到。

也就是说我们函数括弧“{}”中定义的变量（但不包括 static 声明的变量，static 意味着在数据段中存放变量）。

除此以外，在函数被调用时，其参数也会被压入发起调用的进程栈中，并且待到调用结束后，函数的返回值也会被存放回栈中。

由于栈的先进先出(FIFO)特点，所以栈特别方便用来保存/恢复调用现场。

从这个意义上讲，我们可以把堆栈看成一个寄存、交换临时数据的内存区。

一个程序本质上都是由 bss 段、data 段、text 段三个组成的。

这样的概念，不知道最初来源于哪里的规定，但在当前的计算机程序设计中是很重要的一个基本概念。

而且在嵌入式系统的设计中也非常重要，牵涉到嵌入式系统运行时的内存大小分配，存储单元占用空间大小的问题。

在采用段式内存管理的架构中（比如 intel 的 80x86 系统），bss 段通常是指用来存放程序中未初始化的全局变量的一块内存区域，

一般在初始化时 bss 段部分将会清零。bss 段属于静态内存分配，即程序一开始就将其清零了。

比如，在 C 语言之类的程序编译完成之后，已初始化的全局变量保存在.data 段中，未初始化的全局变量保存在.bss 段中。

text和 data 段都在可执行文件中（在嵌入式系统里一般是固化在镜像文件中），由系统从可执行文件中加载；

而 bss 段不在可执行文件中，由系统初始化。

【例】

两个小程序如下：

程序1：

```
int ar[30000];
void main()
{
    ......
}
```

程序2：

```
int ar[300000] = {1, 2, 3, 4, 5, 6 };
void main()
{
    ......
}
```

发现程序2编译之后所得的.exe文件比程序1的要大得多。当下甚为不解，于是手工编译了一下，并使用了/FAs编译选项来查看了一下其各自的.asm，

发现在程序1.asm中ar的定义如下：

```
_BSS SEGMENT
   ?ar@3PAHADD0493e0HDUP（?）;ar
_BSSENDS
```

而在程序2.asm中，ar被定义为：

```
_DATA SEGMENT
   ?ar@3PAHADD01H  ;ar
        DD02H
        DD03H
        ORG$+1199988
```

_DATA ENDS

区别很明显，一个位于.bss段，而另一个位于.data段，两者的区别在于：

全局的未初始化变量存在于.bss段中，具体体现为一个占位符；

全局的已初始化变量存于.data段中；--从rom读取数据再写入到ram中

而函数内的自动变量都在栈上分配空间；

.bss是不占用.exe文件空间的，其内容由操作系统初始化（清零）；--对此处ram空间写0

.data却需要占用，其内容由程序初始化。因此造成了上述情况。

bss段（未手动初始化的数据）并不给该段的数据分配空间，只是记录数据所需空间的大小；

bss段的大小从可执行文件中得到，然后链接器得到这个大小的内存块，紧跟在数据段后面。

data段（已手动初始化的数据）则为数据分配空间，数据保存在目标文件中；内容放在rom中

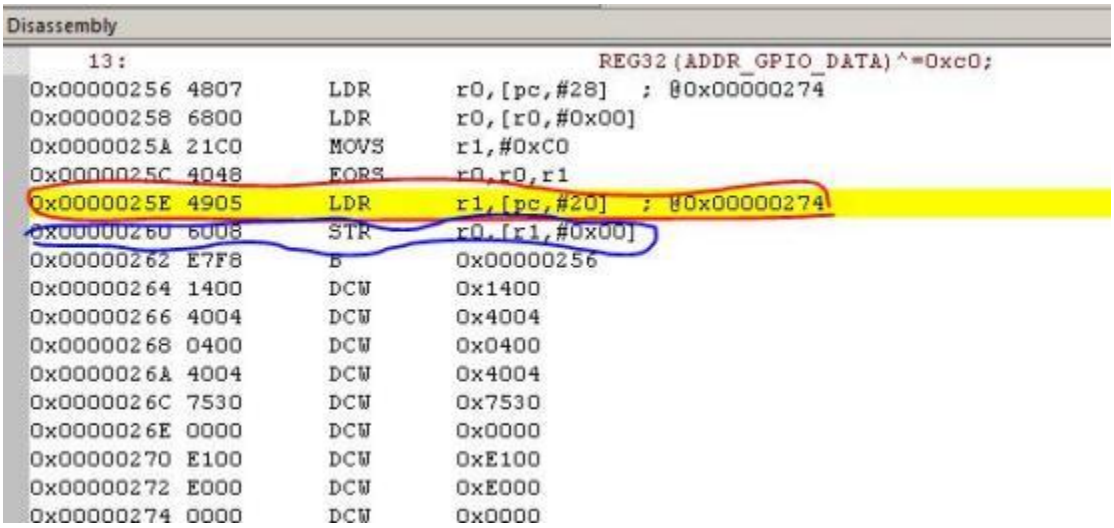
data 段包含经过初始化的全局变量以及它们的值。*当这个内存区进入程序的地址空间后全部清零。*

包含data段和bss段的整个区段此时通常称为数据区

在系统初始化过程中，系统从rom读取数据写入 data 区，而对bss 区直接写 0

中断过程中 r0 问题

在主函数和中断中有对同一个寄存器操作，目的是改变不同的位，但实际上与设计目的不符，因为在中断过程中改变了 r1 寄存器，退出后直接使用了 r1，在下图红色处进入中断，蓝色为下一条是准备把 r1 相关的值写入寄存器中，但此时进入中断响应.



下图是退出中断的程序(中断函数部分)，注意红色部分，此时的 r1 已经修改过的

```
Disassembly
    33:                     REG32(ADDR_GPIO_DATA)^=0x2000;
0x000001F8 4804      LDR        r0,[pc,#16]   ; @0x0000020C
0x000001FA 6800      LDR        r0,[r0,#0x00]
0x000001FC 2101      MOVS       r1,#0x01
0x000001FE 0349      LSLS       r1,r1,#13
0x00000200 4048      EORS       r0,r0,r1
0x00000202 4902      LDR        r1,[pc,#8]    ; @0x0000020C
0x00000204 6008      STR        r0,[r1,#0x00]
    34: }
```

当完全退出该中断然后进入上面蓝色指令,则蓝色指令写给 r0 的值并非在 main 修改过的内容,而是在中断函数修改过的内容

M3 实现上下文切换

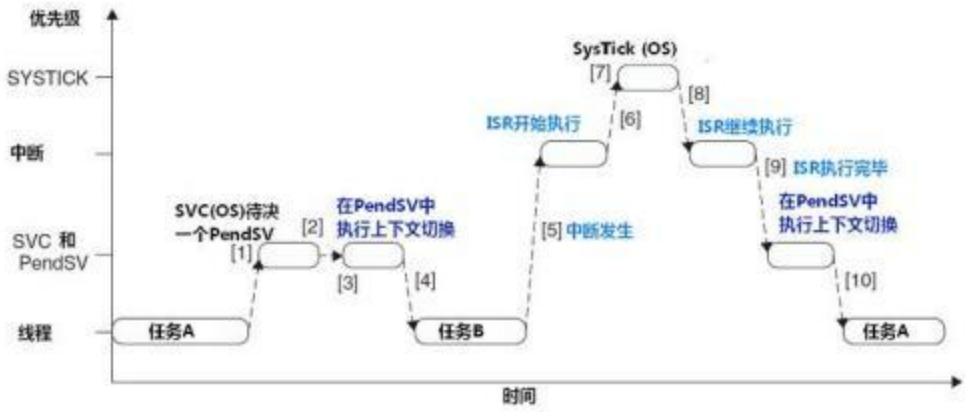


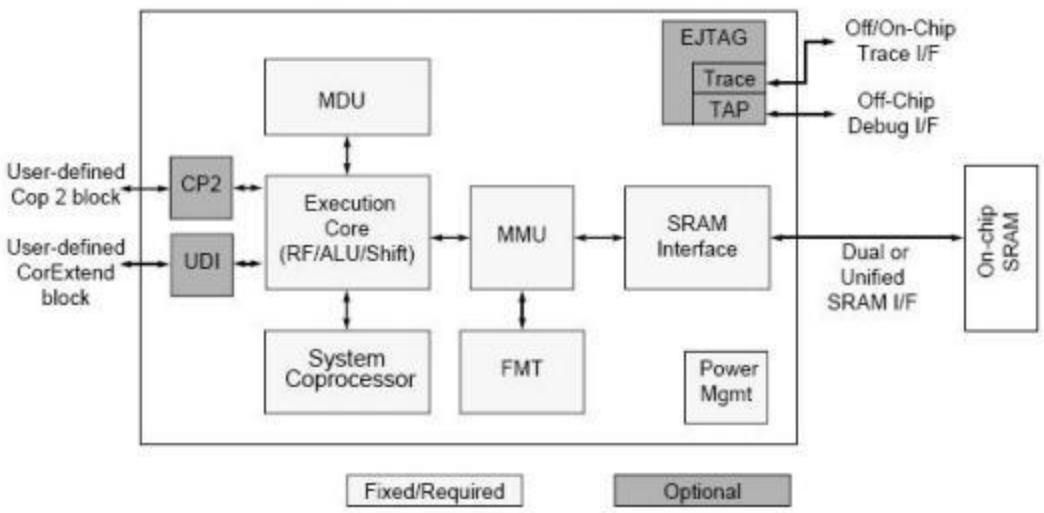
图 7.17 使用 PendSV 控制上下文切换

个中事件的流水账记录如下：

1. 任务 A 呼叫 SVC 来请求任务切换（例如，等待某些工作完成）
2. OS 接收到请求，做好上下文切换的准备，并且悬起一个 PendSV 异常。
3. 当 CPU 退出 SVC 后，它立即进入 PendSV，从而执行上下文切换。
4. 当 PendSV 执行完毕后，将返回到任务 B，同时进入线程模式。
5. 发生了一个中断，并且中断服务程序开始执行
6. 在 ISR 执行过程中，发生 SysTick 异常，并且抢占了该 ISR。
7. OS 执行必要的操作，然后悬起 PendSV 异常以作好上下文切换的准备。
8. 当 SysTick 退出后，回到先前被抢占的 ISR 中，ISR 继续执行
9. ISR 执行完毕并退出后，PendSV 服务例程开始执行，并且在里面执行上下文切换
10. 当 PendSV 执行完毕后，回到任务 A，同时系统再次进入线程模式。

MIPS 知识

Mips 核结构



流水线操作：五级流水线操作

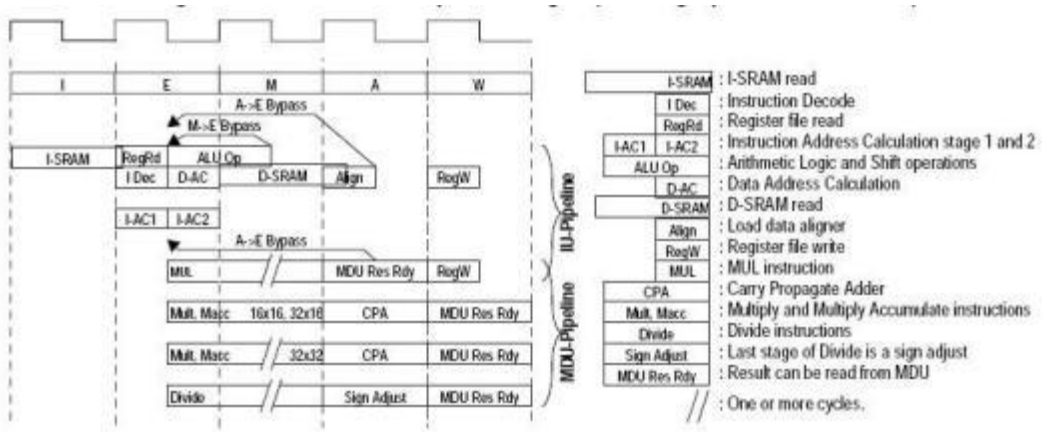
I:I Stage: Instruction Fetch;

E：E Stage: Execution;

M：M Stage: Memory Fetch;

A：A Stage: Align;

W：W Stage: Writeback



Mips 寄存器

1.status 状态寄存器

31 28 27 26 25 24 16 15 8 7 6 5 4 3 2 1 0

| cu0-3|RP|FR|RE| Diag Status| IM7-IM0 KX|SX|UX|KSU|ERL|EXL|IE|

其中 KSU,ERL,EXL,IE 位在这里很重要:

KSU: 模式位 00 -kernel 01--Supervisor 10--User

ERL: error level,0->normal,1->error

EXL: exception level,0->normal,1->exception,异常发生是 EXL 自动置 1

IE: interrupt Enable, 0 -> disable interrupt,1->enable interrupt

(IM 位则可以用于 enbale/disable 具体某个中断,ERL||EXL=1 也使得中断不能响应)

系统所处的模式由 KSU,ERL,EXL 决定:

User mode: KSU = 10 && EXL=0 && ERL=0

Supervisor mode(never used): KSU=01 && EXL=0 && ERL=0

Kernel mode: KSU=00 || EXL=1 || ERL=1

2.cause 寄存器

31 30 29 28 27 16 15 8 7 6 2 1 0

|BD|0 | CE | 0 | IP7 - IP0 |0|Exc code | 0 |

异常发生时 cause 被自动设置

其中:

BD 指示最近发生的异常指令是否在 delay slot 中

CE 发生 coprocessor unusable 异常时的 coprocessor 编号(mips 有 4 个 cp)

IP: interrupt pending, 1->pending,0->no interrupt,CPU 有 6 个中断引脚,加上两个软件中断(最高两个)

Exc code:异常类型,所有的外设中断为 0,系统调用为 8,...

3.EPC 寄存器（PC 写入 EPC 寄存器中）

对一般的异常,EPC 包含:

. 导致异常的指令地址(virtual)

or. if 异常在 delay slot 指令发生,该指令前面那个跳转指令的地址

当 EXL=1 时,处理器不写 EPC

上面 Exc code 的意义

Table 5.13 Cause Register ExcCode Field

1-3	16#00-16#03	- Reserved
4	16#04	AdEL Address error exception (load or instruction fetch)
5	16#05	AdES Address error exception (store)
6	16#06	IBE Bus error exception (instruction fetch)
7	16#07	DBE Bus error exception (data reference: load or store)
8	16#08	Sys Syscall exception
9	16#09	Bp Breakpoint exception
10	16#0a	RI Reserved instruction exception
11	16#0b	CpU Coprocessor Unusable exception
12	16#0c	Ov Arithmetic Overflow exception
13	16#0d	Tr Trap exception
14-15	16#0e-16#0f	- Reserved
16	16#10	IS1 Implementation-Specific Exception 1 (COP2)
17	16#11	CEU CorExtend Unusable
18	16#12	C2E Coprocessor 2 exceptions

19-31 16#13-16#1f - Reserved

MIPS 将存储空间划分为 4 大块--kuseg, kseg0,kseg1 and kseg2.

0xFFFF FFFF

mapped kseg2

0xC000 0000

unmapped uncached kseg1

0xA000 0000

unmapped cached kseg0

0x8000 0000

2G kuseg

0x0000 0000

当开电(Power On)的时候，只有 kseg0 and kseg1 是可以存取的。MIPS CPU 通过 TLB 来 translates all virtual addresses generated by the CPU

MIPS32TM 体系结构含有 32 个通用寄存器（GPR），记为$0-$31。有几个特殊寄存器需要特别注意，$0 寄存器中始终包含零值， $1 寄存器留给汇编器（Assembler）使用作为中间寄存器，$26 和$27 寄存器留作操作系统专用，$28 为全局指针（GP，Globl Pointer），$29 通常用作堆栈指针（SP，Stack Pointer），$30 为帧指针（FP，Frame Pointer），$31 寄存器通常用来存放程序的返回地址（RA，Return Address）。除$0 寄存器外，其余的只是约定俗成的用法，在硬件实现上并没有区别。具体用法可参考附录 B 和文献[1][14]。

MIPS32TM 还含有 3 个特殊寄存器（SPR），分别是 PC 指针、高位寄存器Hi 和低位寄存器Lo。PC 指针中包含当前正在执行指令的地址，也称为程序计数器。Hi 和 Lo 两个寄存器是乘法和除法指令的专用寄存器。

MIPS 体系结构下，程序计数器不是一个寄存器，其实你最好不要去那样想。在一个具有流水线的 CPU 中，程序计数器的值在一个给定的时刻有多个可选值。这一点有点迷惑人。jal 指令的返回地址跟随其后的第二条指令。

寄存器编号	助记符	用法
0	zero	永远返回值为 0
1	at	用做汇编器的暂时变量
2-3	v0, v1	子函数调用返回结果,如果这两个寄存器不够存放需要返回的值，编译器将会通过内存来完成
4-7	a0-a3	子函数调用的参数
8-15	t0-t7	暂时变量，子函数使用时不需要保存与恢复(t0~t9 用法一致)
24-25	t8-t9	依照约定，一个子函数可以不用保存并随便的使用这些寄存器。在作表达式计算时，这些寄存器是非常好的暂时变量。编译器/程序员必须注意的是，当调用一个子函数时，这些寄存器中的值有可能被子函数破坏掉
16-25	s0-s7	子函数寄存器变量。子函数必须保存和恢复使用过的变量在函数返回之前，从而调用函数知道这些寄存器的值没有变化。子函数必须保证当函数返回时这些寄存器的内容必须恢复到函数调用以前的值，或者在子函数里不用这些寄存器或把它们保存在堆栈上并在函数退出时恢复。
26,27	k0,k1	通常被中断或异常处理程序使用作为保存一些系统参数
28	gp	全局指针。一些运行系统维护这个指针来更方便的存取“static“和”extern"变量。

29	sp	堆栈指针
30	s8/fp	第 9 个寄存器变量。子函数可以用来做桢指针
31	ra	子函数的返回地□

C 名字	MIPS 名字	大小(字节)	汇编助记符
longlong	dword	8	"d"代表 ld
int/long	word	4	"w"代表 lw
short	halfword	2	"h"代表 lh
char	byte	1	"b"代表 lb

在函数调用时，MIPS 先把 PC 保留到 RZ，在把 S0，S1 保留，返回函数时，在出栈，MIPS 函数调用的内容由编译器决定，而不是是核硬件来保护现场

异常,中断,初始化：

当一个异常发生,系统的顺序执行被中断时,流水线(PIPELINE)的 CPU 将会有几条指令出于流水线的不同阶段(STAGE).因为我们不想中断处理破坏程序的正常执行,对于没有执行完的指令,我们必需记住它们执行到哪一个阶段,以便在中断处理之后能恢复程序执行.

核根据 STATE，CAUSE 寄存器来判断是什么类型的中断

C 语言开发常用的定义

```
typedef unsigned char  UINT8;
typedef signed char  INT8;
typedef unsigned short  UINT16;
typedef signed short  INT16;
typedef unsigned int  UINT32;
typedef signed int  INT32;
typedef unsigned long long UINT64;
typedef signed long long INT64;
```

```
#define REG8( addr )     (*(volatile UINT8 *) (addr))
#define REG16( addr )     (*(volatile UINT16 *) (addr))
#define REG32( addr )     (*(volatile UINT32 *) (addr))
#define REG64( addr )     (*(volatile UINT64 *) (addr))
```

Mips 开发工程

MIPS 环境采用 MAKEFILE 管理的方式来实现，如下：

```
CC          = sde-gcc
LD          = sde-ld
OBJCOPY     = sde-objcopy
OBJDUMP     = sde-objdump

#=============================================================================
==
# tagert names
#=============================================================================
==
IMAGE_BASENAME =  scd
IMAGENAME = $(IMAGE_BASENAME)

# REC\ELF\MAP\DIS names
SREC = $(IMAGENAME).rec
ELF  = $(IMAGENAME).elf
MAP  = $(IMAGENAME).map
DIS  = $(IMAGENAME).dis
BIN  = $(IMAGENAME).bin
```

```
#=================================================================
==
# dir path
#=================================================================
==
ROOT   = .
TARGET   = $(IMAGE_BASENAME)

#set source dir
LNKDIR      = $(ROOT)/script
OBJDIR      = $(ROOT)/bin/$(TARGET)
MAKDIR   = $(ROOT)/bin/$(TARGET)
INCDIR      = $(ROOT)/include/
LIBDIR     = $(ROOT)/API
SRCDIR      = $(ROOT)/boot \
         $(ROOT)/app/$(TARGET) $(ROOT)/app/$(TARGET)/testcase
MIPS16SRCDIR = $(ROOT)/sha

#set target dir
IMAGE_SREC   = $(OBJDIR)/$(SREC)
IMAGE_DIS    = $(OBJDIR)/$(DIS)
IMAGE_ELF    = $(OBJDIR)/$(ELF)
IMAGE_MAP    = $(OBJDIR)/$(MAP)
IMAGE_BIN  = $(OBJDIR)/$(BIN)

#set clean dir/files
```

```
CLEAN_FILES  = $(OBJDIR)/*.o $(OBJDIR)/*.bin $(OBJDIR)/*.elf
$(OBJDIR)/*.rec  $(OBJDIR)/*.map $(OBJDIR)/*.dis $(LIBDIR)/*.o
$(MIPS16SRCDIR)/*.o

#================================================================
==
# Building Options
#================================================================
==
# Preprocessor Options
DEBUG   = -D_DEBUG_
SIMULATE  = -D_SIMULATE_
REV_OPTS   = '-D_REVMAJ_="$(REVMAJ)"' '-D_REVMIN_="$(REVMIN)"'
P_OPTS  = $(DEBUG) $(SIMULATE) $(REV_OPTS)

# Warning Options
W_OPTS     = -Wimplicit -Wformat

# Debugging Options
D_OPTS  = -g -D_32_

# Directory Options
I_OPTS  = -I $(INCDIR)

# Assembler Options
A_OPTS  = -D_ASSEMBLER_ -DEL

# Optimization Options
O_OPTS  = -G0 -fno-builtin
```

```
O_OPTSNEW  = -Os

# mips16 Options
MIPS16_OPTS= -mips16e

# Machine Options
CPUTUNE  = -mtune=m4k
ENDIAN   = -EL
M_OPTS  = $(CPUTUNE) $(ENDIAN)

# Overall Options
CC_OPTS  = $(M_OPTS) \
     $(P_OPTS) \
     $(I_OPTS) \
     $(O_OPTS) \
     $(W_OPTS) \
     $(D_OPTS) \
     $(MIPS16_OPTS) \
     $(O_OPTSNEW) \

CC_MIPS16_OPTS  = $(M_OPTS) \
     $(P_OPTS) \
     $(I_OPTS) \
     $(O_OPTS) \
     $(W_OPTS) \
     $(D_OPTS) \
     $(MIPS16_OPTS) \
     $(O_OPTSNEW) \
```

```
CC_OPTS_AS = $(CC_OPTS) $(A_OPTS)

# Linker Options
LD_SCRIPT  = $(LNKDIR)/default.xn
LD_OPTS    = -L$(LIBDIR)
LD_MIPS16_OPTS    = -L$(MIPS16SRCDIR)
OFORMAT    = elf32-tradlittlemips

# Objdump Options
OBJDP_OPTS  = -S --disassemble-all



#=================================================================
==
# compiler files
#=================================================================
==
VPATH  = $(SRCDIR)
# source files
SRC_C    = $(foreach dir, $(SRCDIR), $(wildcard $(dir)/*.c))
SRC_S    = $(foreach dir, $(SRCDIR), $(wildcard $(dir)/*.S))
SRC_H    = $(foreach dir, $(SRCDIR) $(INCDIR), $(wildcard $(dir)/*.h))
SRC_LIB_C  = $(foreach dir, $(LIBDIR), $(wildcard $(dir)/*.c))
SRC_LIB_S  = $(foreach dir, $(LIBDIR), $(wildcard $(dir)/*.S))
SRC_MIPS16_C  = $(foreach dir, $(MIPS16SRCDIR), $(wildcard
$(dir)/*.c))
SRC      = $(SRC_S) $(SRC_C)
```

```
# target files
OBJ_C    = $(notdir $(patsubst %.c, %.o, $(SRC_C)))
OBJ_S    = $(notdir $(patsubst %.S, %.o, $(SRC_S)))
OBJ_LIB_C  = $(patsubst %.c, %.o, $(SRC_LIB_C))
OBJ_LIB_S  = $(patsubst %.S, %.o, $(SRC_LIB_S))
OBJ      = $(OBJ_S) $(OBJ_C)
OBJ_ATT  = $(addprefix $(OBJDIR)/,$(OBJ_C))
OBJ_LIB  = $(OBJ_LIB_C) $(OBJ_LIB_S)
OBJ_MIPS16 = $(patsubst %.c, %.o, $(SRC_MIPS16_C))

#=================================================================
==
# target building
#=================================================================
==
.PHONY : release clean dep

# build
all: $(IMAGE_DIS)
 sde-conv -f bin $(IMAGE_ELF) -o $(IMAGE_BIN)

lib: $(OBJ_LIB)
$(IMAGE_DIS) : $(IMAGE_ELF)
 $(OBJDUMP) $(OBJDP_OPTS) $(IMAGE_ELF) > $(IMAGE_DIS)

mips16: $(OBJ_MIPS16)
$(IMAGE_DIS) : $(IMAGE_ELF)
 $(OBJDUMP) $(OBJDP_OPTS) $(IMAGE_ELF) > $(IMAGE_DIS)
```

```
release :
 $(MAKE) clean
 $(MAKE) all

$(IMAGE_SREC) : $(IMAGE_ELF)
 $(OBJCOPY) -O srec $(IMAGE_ELF) $(IMAGE_SREC)

$(IMAGE_ELF) : $(OBJ) $(OBJ_LIB) $(OBJ_MIPS16) $(LD_SCRIPT)
 $(LD) $(LD_OPTS) $(LD_MIPS16_OPTS) -T $(LD_SCRIPT) -EL -o $(IMAGE_ELF)
-Map $(IMAGE_MAP) --oformat $(OFORMAT) $(OBJ_ATT) $(OBJ_LIB)
$(OBJ_MIPS16)

# compile
$(OBJ_C) : %.o : %.c
 $(CC) -c $(CC_OPTS) -o $(OBJDIR)/$@ $<

$(OBJ_S) : %.o : %.S
 $(CC) -c $(CC_OPTS_AS) -o ./boot/$@ $<

$(OBJ_LIB_C) : %.o : %.c
 $(CC) -c $(CC_OPTS) -o $@ $<

$(OBJ_MIPS16) : %.o : %.c
 $(CC) -c $(CC_MIPS16_OPTS) -o $@ $<

$(OBJ_LIB_S) : %.o : %.S
 $(CC) -c $(CC_OPTS_AS) -o $@ $<
```

```
# makefile
$(OBJ) : $(MAKDIR)
$(OBJ_LIB) : $(MAKDIR)
$(OBJ_MIPS16) : $(MAKDIR)

# clean
clean :
 -$(RM) -f $(CLEAN_FILES)
 -$(CD) $(EL_PATH): $(RM) -f $(CLEAN_FILES)

# depend
dep:
 $(CC) $(INCDIR) -M $(SRC) > ./depend.mk

-include ./depend.mk
```

而与 ARM 对应的分散加载文件采用如下方式：文件后缀.xn

```
**************************************************************************
*****/

OUTPUT_ARCH(mips)

/**** Start point ****/
ENTRY(_start)


MEMORY
  {
```

```
  FLASH(rwx): ORIGIN = 0xBFd08000, LENGTH = 576K
  ram(!rw):ORIGIN = 0xb2000000, LENGTH = 24K
 }


SECTIONS
 {
  /**** Code and read-only data ****/


  .text_app 0xBFd08000:
   {
    _ftext_init = ABSOLUTE(.);
    ./boot/Init.o(.text)
    ./boot/sysapi.o  (.text)
    *(.text)
    *(.rodata)
    *(.rodata.*)
    . = ALIGN(8);
/*
    _fdata = ABSOLUTE(.); /* Start of initialised data     */
    *(.data)
    . = ALIGN(8);
*/

    _etext_init = ABSOLUTE(.);
  } >FLASH = 0
```

```
/**** Initialised data ****/

_etext = ABSOLUTE(0xb2000000);

.RAM_SEC  :
AT(_etext)
{
    _sp = ABSOLUTE(. + 0x6000);

 _fdata = ABSOLUTE(.); /* Start of initialised data     */

  *(.data)
  . = ALIGN(8);

  _gp = ABSOLUTE(. + 0x2000); /* Base of small data      */

  *(.lit8)
  *(.lit4)
  *(.sdata)

  . = ALIGN(8);

  _edata  = ABSOLUTE(.); /* End of initialised data     */

} >ram = 0

/**** Uninitialised data ****/

_fbss = ABSOLUTE(.);   /* Start of unitialised data     */
```

```
  .sbss :
  {
    *(.sbss)
    *(.scommon)
  }>ram

  .bss :
  {
    *(.bss)
    *(COMMON)
  }>ram


  _end = . ;
  _freemem = .;

}
```

下面对 linker script 文件里的一些关键点进行如下说明：

如下图所示，第一个红色矩形线框：

MEMORY 是脚本语言里面的一个命令，用来配置内存的大小和位置，

“FLASH(rwx): ORIGIN = 0xBFd08000, LENGTH = 1024K”

上面这条程序是分配一块大小为 1024KB，起始地址为 0xBFd08000 的 FLASH，程序里面的”FLASH”只是一个符号名；”(rwx)”表示分配的这段内存的属性为可读，可写，可执行；关键字 ORIGIN 用于指示内存的起始地址；关键字 LENGTH 用于指示内存的大小。

“ ram(!rw):ORIGIN = 0xb2000000, LENGTH = 24K ”

上面这条程序是分配一块大小为 24KB，起始地址为 0xb2000000 的 RAM，程序里面的

” ram”只是一个符号名；” (!rw)”表示分配的这段内存的属性为不可读，可写；

如下图所示，第二个红色矩形线框：

ABSOLUTE 是一关键字，可以将相对值转化成绝对值，“.”是定位计数器，用于记录当前所在的内存地址。

“_sp = ABSOLUTE(. + 0x6000); ”

上面这条程序是给符号变量”_sp”赋值，将定位计数器的值再加上0x6000，即将当前的内存地址再往后移 24KB，所以_gp 的值为 0xb2006000，这里”_sp”就是堆的顶地址指针，也可以把它理解为 RAM 的大小。

如下图所示，第三个红色矩形线框：

“_gp = ABSOLUTE(. + 0x2000);”

上面这条程序是给符号变量” _gp”赋值，将定位计数器的值再加上0x2000，即将当前的内存地址再往后移 8KB，所以_gp 的值为 0xb2002000，这里”_gp”就是栈的顶地址指针，也可以把它理解为栈的大小。

```
MEMORY
 {

  FLASH(rwx): ORIGIN = 0xBFd08000, LENGTH = 1024K
  ram(!rw):ORIGIN = 0xb2000000, LENGTH = 24K
 }

SECTIONS
{
  /**** Code and read-only data ****/


  .text_app 0xBFd08000:
  {
    _ftext_init = ABSOLUTE(.);
    ./boot/Init.o(.text)
    ./boot/sysapi.o (.text)
    *(.text)
    *(.rodata)
    *(.rodata.*)
    . = ALIGN(8);
/*
    _fdata = ABSOLUTE(.);    /* Start of initialised data        */
    *(.data)
    . = ALIGN(8);
*/

    _etext_init = ABSOLUTE(.);
  } >FLASH = 0

  /**** Initialised data ****/

  _etext = ABSOLUTE(0xb2000000);

  .RAM_SEC  :
  AT(_etext)
  {
    _sp = ABSOLUTE(. + 0x6000);
   _fdata = ABSOLUTE(.);     /* Start of initialised data        */

    *(.data)
    . = ALIGN(8);

    _gp = ABSOLUTE(. + 0x2000); /* Base of small data              */
    *(.lit8)
    *(.lit4)
```

MIPS 初始化文件 .s

```
/****************************** Include
******************************/


#include "define.h"
#include "mipsregs.h"
#include "Marco.h"
#include <mips/asm.h>
#include <mips/regdef.h>
#include <mips/cpu.h>
#include <sys/init.h>
#include <xxxx.h>

/****************************** ROUTINE
******************************/

 .text
 .globl _start
 .ent   _start

_start:
 .set noreorder
 RVECENT(reset,0)      /* Program enter */

.org 0x180
 RVECENT(ExceptionHandle,00)    // 0xbfD0_0180:

.org 0x200
 RVECENT(Software0Interrupt,0)    // 0xbfD0_0200: Software0Interrupt
```

```
 RVECENT(Software1Interrupt,1)    // 0xbfD0_0220: Software1Interrupt
 RVECENT(HardWare0Interrupt,0)    // 0xbfD0_0240: HardWare_Interrupe0
 RVECENT(HardWare1Interrupt,1)    // 0xbfD0_0260: HardWare_Interrupe1
 RVECENT(HardWare2Interrupt,2)    // 0xbfD0_0280: HardWare_Interrupe2
 RVECENT(HardWare3Interrupt,3)    // 0xbfD0_02a0: HardWare_Interrupe3
 RVECENT(HardWare4Interrupt,4)    // 0xbfD0_02c0: HardWare_Interrupe4
 RVECENT(HardWare5Interrupt,5)    // 0xbfD0_02e0: HardWare_Interrupe5


reset:

    CP0_SET_VALUE(Status, 0x10000000)        // CP0 can access,
 CP0_CLR(Status, 0x00000001)    // disable all interrupt
       CP0_SET_Sel(12,1,0x00000020)   // Specify the space between
interrupts is 16#20
      CP0_SET_Sel(15,1,0xBFd08000)   // Set interrupt enter base
address; 16#BFd0_0200
 CP0_SET (Cause,Cause_IV)    // Cause_IV=1,the Interrupt enter addr is
16#200 the 23bit
 CP0_CLR (Status,Status_BEV)    // Status_BEV=0 Normal mode the 22bit
 la   sp, _sp
     CP0_SET (Status,SET_IM)     // enable all interrupt mask
 addiu sp,sp,-24
 sw  ra, 16(sp)
 la  gp, _gp

       jal     main        // for 16e mode
    //   j    main        // for 32bit mode
       nop
```

```
 .text
 .set  noat
 .set noreorder
 .globl ExceptionHandle
 .align 4
ExceptionHandle:
 mfc0 k1, EPC
 move      a0,k1
 jal  Uart1SendByte
 nop
 srl          a0,k1,8
 jal  Uart1SendByte
 nop
 srl          a0,k1,16
 jal  Uart1SendByte
 nop
 srl          a0,k1,24
 jal  Uart1SendByte
 nop
 mfc0 k1, Cause
 move      a0,k1
 jal  Uart1SendByte
 nop
 srl          a0,k1,8
 jal  Uart1SendByte
 nop
 srl          a0,k1,16
 jal  Uart1SendByte
 nop
```

```
srl           a0,k1,24
jal  Uart1SendByte
nop
mfc0 k1, Status
move      a0,k1
jal  Uart1SendByte
nop
srl           a0,k1,8
jal  Uart1SendByte
nop
srl           a0,k1,16
jal  Uart1SendByte
nop
srl           a0,k1,24
jal  Uart1SendByte
nop
      nop
      CP0_CLR(Status,Status_IE)
      wait
eret
.type ExceptionHandle, @function
.size ExceptionHandle,.-ExceptionHandle

/********************************** ENTRY
*****************************************/


.set    reorder
.end   _start
```

```
//=================================================================
==================================
Software0Interrupt:
 INTENTRY(SW0IntHandleAsm,SW0IntHandle)

Software1Interrupt:
 INTENTRY(SW1IntHandleAsm,SW0IntHandle)

HardWare0Interrupt:
 INTENTRY2(HW0IntHandleAsm,HW0IntHandle)

HardWare1Interrupt:
 INTENTRY3(HW1IntHandleAsm,HW1IntHandle)

HardWare2Interrupt:
 INTENTRY4(HW2IntHandleAsm,HW2IntHandle)

HardWare3Interrupt:
 INTENTRY5(HW3IntHandleAsm,HW3IntHandle)

HardWare4Interrupt:
 INTENTRY6(HW4IntHandleAsm,HW4IntHandle)

HardWare5Interrupt:
 INTENTRY7(HW5IntHandleAsm,HW5IntHandle)
```

C 语言开发问题分析

在 MIPS 工具 Eclipse 编译器中，如果对想对某个变量不断赋值，然后不断查询，则需要定义为volatile 类型，否则程序查询时不会一直去读该变量，只有定义为volatile 类型才会有此结果;

volatile UINT8 T100kIntFlag,

while(!T100kIntFlag);

编译后的结果是：

bfc0842a: 6c02 li a0,2

bfc0842c: ec6d or a0,v1

bfc0842e: c580 sb a0,0(a1)

bfc08430: a660 lbu v1,0(a2)

bfc08432: 6a05 li v0,5

bfc08434: ec4b neg a0,v0

bfc08436: ec6c and a0,v1

bfc08438: c680 sb a0,0(a2)

bfc0843a: a560 lbu v1,0(a1)

bfc0843c: 6a10 li v0,16

bfc0843e: ea6d or v0,v1

bfc08440: c540 sb v0,0(a1)

bfc08442: a7c0 lbu a2,0(a3) //变量赋值

bfc08444: 26fe beqz a2,bfc08442 <CGU_TEST_801+0x72>//在此处循环

UINT8 T100kIntFlag,

while(!T100kIntFlag);

编译后的结果是：

bfc08426: 6c02 li a0,2

bfc08428: ec6d or a0,v1

```
bfc0842a: c580        sb a0,0(a1)
bfc0842c: a6e0        lbu a3,0(a2)

bfc0842e: 6a05        li v0,5
bfc08430: ec4b        neg a0,v0
bfc08432: ecec        and a0,a3
bfc08434: c680        sb a0,0(a2)
bfc08436: a560        lbu v1,0(a1)

bfc08438: 6a10        li v0,16
bfc0843a: ea6d        or v0,v1
bfc0843c: c540        sb v0,0(a1)
bfc0843e: 6a00        li v0,0     //立即数赋值
bfc08440: 22ff        beqz v0,bfc08440 <CGU_TEST_801+0x74>  //在此句
循环
```

分析：

```
{
 ...
 while (1)
  {
 if (i) do_something();
 }
}
/* Interrupt service routine. */
void ISR_2(void)
{
 i=1;
}
```

程序的本意是希望 ISR_2 中断产生时，在 main 当中调用 do_something 函数，但是，由于编译器判断在main 函数里面没有修改过 i，因此可能只执行一次对从 i 到某寄存器的读操作，然后每次 if 判断都只使用这个寄存器里面的“i 副本”，导致 do_something 永远也不会被调用。如果变量加上 volatile 修饰，则编译器保证对此变量的读写操作都不会被优化（肯定执行）。此例中 i 也应该如此说明。

一般说来，volatile 用在如下的几个地方：

1、中断服务程序中修改的供其它程序检测的变量需要加volatile；

2、多任务环境下各任务间共享的标志应该加volatile；

3、存储器映射的硬件寄存器通常也要加volatile 说明，因为每次对它的读写都可能由不同意义；

另外，以上这几种情况经常还要同时考虑数据的完整性（相互关联的几个标志读了一半被打断了重写），在 1 中可以通过关中断来实现，2 中可以禁止任务调度，3 中则只能依靠硬件的良好设计了

volatile 总是与优化有关，编译器有一种技术叫做数据流分析，分析程序中的变量在哪里赋值、在哪里使用、在哪里失效，分析结果可以用于常量合并，常量传播等优化，进一步可以死代码消除。但有时这些优化不是程序所需要的，这时可以用volatile 关键字禁止做这些优化，volatile 的字面含义是易变的，它有下面的作用：

1 不会在两个操作之间把 volatile 变量缓存在寄存器中。在多任务、中断、甚至 setjmp 环境下，变量可能被其他的程序改变，编译器自己无法知道，

volatile 就是告诉编译器这种情况。

2 不做常量合并、常量传播等优化，所以像下面的代码：

```
volatile int i = 1;

if (i > 0) ...
```

if 的条件不会当作无条件真。

3 对 volatile 变量的读写不会被优化掉。如果你对一个变量赋值但后面没用到，编译器常常可以省略那个赋值操作，然而对 Memory Mapped IO 的处理是不能这样优化的。

OTHERS

在 win 下打开文件并复制内容

开始菜单 输入 cmd 命令

用命令 cd xxx 打开文件夹,(可以使用 tab 键来自动找到合适文件----- 先输入一个字符)

使用 type 命令打开文件:type file

使用右键 再 mark

选中之后使用 enter 键复制内容

半导体知识

seal ring 是什么

从 layout角度看，seal ring 是一个由 diff, contact, via 和 metal 等 layer 按照一定的规则叠加组成的图形。
从工艺角度看，seal ring 是一种氧化、钝化层结构
从物理角度看，seal ring 是介于芯片（chip）和划片槽（scrible line）之间的（保护）环。

简单按照 pcb 的方式直接理解如下:
seal ring:单块 pcb 边与最外层走线的距离;在画 pcb 开始,需要先画 2 个框,最外层为 pcb 尺寸,次外层就是里面可以走线,布器件的空间,这两个框之间的距离就相当于芯片的 seal ring;

scrible line:画 pcb 会把几个 pcb 拼成一块,在各个 pcb 之间会有锯齿壮把它们连接在一起,这个锯齿的宽度就应该是 scrible line.

seal ring 怎么画
如果不想承担什么风险， 给芯片添加 seal ring 这件事可以交给 foundry，出钱就行。不过多数时候我们宁可自己动手去做这些事情。一般情况下 foundry 都会在 design rule 中清楚的规定 seal ring 的结构及尺寸。常见的 seal ring 会将工艺中所有的层都列出来，并标明 dark or clear，用这里的 dark 和 clear 去对照工艺制程中这些 layer 的 dark 或 clear，如果同为 dark 或 clear 则 seal ring 上面有这层，反之则 seal ring 上没有这层。
design rule 中的 contact 和 via 都是我们常用的方孔，实际上长孔比方孔更好些，foundry 这样建议的。。。
seal ring一般要离芯片 core 有一定的距离，如果芯片面积宽松可以参考 design

rule，因为rule 的规定通常比较大；当然出于成本的考虑，我们宁可根据流片经验自己定义。

seal ring 怎么接

一般的 PSUB 工艺都会将 seal ring 做成 P+的，接到芯片的 ground，而其浮空一般也不会有影响；无特殊情况不能接到 power。

最好不要用metal1 去将 core 和 seal ring 连接在一起，而是用其它上层 metal；如果有diff层连到seal ring, 在靠近 seal ring 的diff上面做一小段 salicide 结构（通常是加 rpo 或 sab），这样对防止 ESD 绝对好处大大的。

如有需要开槽的宽金属连接到 seal ring，请尽量不要在靠近 seal ring 的地方开槽，非开槽不可的话那尽量开的小点，这些都是防止外界的杂质等进入芯片。

seal ring 什么用

seal ring最根本也是最主要的作用就是防止芯片在切割的时候受到机械损伤，以下其它作用可以说是衍生的，搭顺风车的：a) Seal Ring接地，屏蔽芯片外的干扰； b) Seal Ring 可以防止潮气从侧面断口侵入； c) 将切割所产生的静电就近接地，再由切割时邻近的 seal ring共同分摊所产生的电流，将对 die 本体的冲击降到最小。d) ……

1. 工艺角

（Process Corner）PVT(Precess Voltage Temperature)工艺误差

与双极晶体管不同，在不同的晶片之间以及在不同的批次之间，MOSFETs 参数变化很大。为了在一定程度上减轻电路设计任务的困难，工艺工程师们要保证器件的性能在某个范围内，大体上，他们以报废超出这个性能范围的芯片的措施来严格控制预期的参数变化。

detail

通常提供给设计师的性能范围只适用于数字电路并以“工艺角 ”（Process Corner）的形式给出。如图，其思想是：把 NMOS 和 PMOS 晶体管的速度波动范围限制在由四个角所确定的矩形内。这四个角分别是：快 NFET 和快 PFET，慢 NFET 和慢 PFET，快 NFET 和慢 PFET，慢 NFET 和快 PFET。例如，具有较薄的栅氧、较低阈值电压的晶体管，就落在快角附近。

从晶片中提取与每一个角相对应的器件模型时，片上 NMOS 和 PMOS 的测试结构显示出不同的门延时，而这些角的实际选取是为了得到可接受的成品率。

各种工艺角和极限温度条件下对电路进行仿真是决定成品率的基础。

所以我们所说的 ss、tt、ff 分别指的是左下角的 corner，中心、右上角的 corner。

工艺极限 (Process Corner)

如果采用 5-corner model 会有 TT,FF,SS,FS,SF 5 个 corners。如 TT 指 NFET-Typical corner & PFET-Typical corner。其中, Typical 指晶体管驱动电流是一个平均值，FAST 指驱动电流是其最大值，而 SLOW 指驱动电流是其最小值（此电流为 Ids 电流）这是从测量角度解释，也有理解为载流子迁移率(Carrier mobility)的快慢. 载流子迁移率是指在载流子在单位电场作用下的平均漂移速度。至于造成迁移率快慢的因素还需要进一步查找资料。单一器件所测的结果是呈正态分布的，均值在 TT，最小最大限制值为 SS 与 FF。从星空图看 NFET，PFET 所测结果,这 5 种覆盖大约+-3 sigma 即约 99.73% 的范围。对于工艺偏差的情况有很多，比如掺杂浓度，制造时的温度控制，刻蚀程度等，所以造成同一个晶圆上不同区域的情况不同，以及不同晶圆之间不同情况的发生。这种随机性的发生，只有通过统计学的方法才能评估覆盖范围的合理性。

PVT (process, voltage, temperature)

设计除了要满足上述 5 个 corner 外，还需要满足电压与温度等条件，形成的组合称为 PVT (process, voltage, temperature) 条件。电压如：1.0v+10% ,1.0v ,1.0v-10% ; 温度如：-40C, 0C 25C, 125C。设计时设计师还常考虑找到最好最坏情况. 时序分析中将最好的条件(Best Case)定义为速度最快的情况, 而最坏的条件(Worst Case)则相反。最好最坏的定义因不同类型设计而有所不同。最坏的延迟也不都出现在SS[19] 。至于延迟随 PVT 发生怎样的变化，还需要进一步查找资料。

根据不同的仿真需要，会有不同的 PVT 组合。以下列举几种标准 STA 分析条件[16]：

WCS (Worst Case Slow) : slow process, high temperature, lowest voltage

TYP (typical) : typical process, nominal temperature,nominal voltage

BCF (Best Case Fast) : fast process, lowest temperature, high voltage

WCL (Worst Case @ Cold) : slow process, lowest temperature, lowest voltage

在进行功耗分析时，可能是另些组合如：

ML (Maximal Leakage) : fast process, high temperature, high voltage

TL (typical Leakage) : typical process, high temperature, nominal voltage

除此之外，另一个组合条件称为 Scenarios:

Scenarios = Interconnect + operation mode + PVT

噪声(noise)与串扰(crosstalk) 似乎需要另外考虑。

1. 内连线情况(interconnect corner)

制造对互连线造成的影响，如：R_typicalC_typical, R_maxC_max, R_maxC_min, R_minC_min

2. 工作模式 (Operation Mode)

如：function mode, scan mode, sleep mode, standby mode, active mode

PVT

对多种 scenarios 的综合分析，称之为 MMMC (Multi-Mode Multi-Corner) Analysis。

OCV (On-chip Variations)

由于偏差的存在，不同晶圆之间，同一晶圆不同芯片之间，同一芯片不同区域之间情况都是不相同的。造成不同的因素有很多种，这些因素造成的不同主要体现：

1，IR Drop 造成局部不同的供电的差异；

2，晶体管阈值电压的差异；

3，晶体管沟道长度的差异；

4，局部热点形成的温度系数的差异；

5，互连线不同引起的电阻电容的差异。

OCV 可以描述 PVT 在单个芯片所造成的影响。更多的时候，用来考虑长距离走线对时钟路径的影响。在时序分析时引入 derate 参数模拟 OCV 效应，其通过改变时延迟的早晚来影响设计。

三种 STA(Static Timing Analysis)分析方法 [13]：

1，单一模式，用同一条件分析 setup/hold ；

2，WC_BC 模式，用 worst case 计算 setup，用 best case 计算 hold；

3，OCV 模式，计算 setup 用计算 worst case 数据路径，用 best case 计算时钟路径；计算 hold 用 best case 计算数据路径，用 worst case 计算时钟路径；

以上三种方法，在逐步的改进，但显然越来越悲观。比如第三种分析方法，计算 setup 时让数据慢一点到，而时钟快一点到，以压缩 setup 时间。到了计算hold 时，又变成了压缩hold 时间，这样情况显然是不实际的。derate 具体做法是在长路径应用 late 参数，短路径应用 early 参数，就是将原本希望走快的变慢，希望走慢的变快。

除 derate 外，还有一个常用的参数 uncertainty。它定义时钟沿的不确定性，或早或晚，同样压缩了 setup / hold 时间。

锁存器与触发器

锁存器与触发器是静态时序分析中涉及最主要的类型. Latch 是异步单元，即输出在输入改变之后改变。而 FLIP-FLOP 是同步单元，即随时钟信号改变而改变。LATCH 与 FLIP-FLOP 最大差别是 Latch 没有时钟端，可以理解为有一个使能端，所以是电平敏感[8]。Flip-Flop preset 表示在输出产生一个逻辑 1, clear 表示在输出产生一个逻辑 0。 Setup/hold 是分析数据与时钟之间的关系。用时钟沿来采集数据，如果定义用时钟上升沿触发，即用上升沿将数据分割成两部分，前部分为准备时间 setup，后部分为保持时间 hold。说明数据在上升沿来临之前必须准备好，在采集时必须保持一段时间。理解时[removal/recovery]，类似为用

preset/clear 采集时钟数据，用 preset/clear 将时钟分为两部分，前部分为迁移时间，后部分为复原时间。

统计静态时序分析

SSTA (Statistical Static timing analysis)

全局工艺差异(global_process_variations)

也称为片间器件差异(inter-die device variations)，描述同一器件不同芯片间的差异。同一芯片的器件应用同一参数，器件的不同参数是相互独立的，而且每个参数都是呈统计分布的。

局部工艺差异(local_process_variations)

也称为片内器件差异 (intra-die device variations)，描述同一器件在同一芯片不同区域的差异。每个差异也是呈统计分布的。

也就是说对于某一个全局参数，应该细化成多个局部参数，每个局部参数都是呈统计分布的。如果提供的库是基于局部参数差异统计建立起来的，在进行基于 OCV 模型的静态时序分析时，就无需 OCV 参数的设定。

对于互连线差异也是一样的. 决定同一段线的因素有很多种，比如线宽，厚度，介电系数，刻蚀等，但同一因素不同区域是不相同的，各个因素之间也是相互独立的。这些差异同样需要进行统计学概括。

好比中医与西医，西医就是同一种病吃同一种药，而中医则认为同一种病分不同种情况，不同的人也应不同对待。至少感觉上是这样的。

基于这样的观点，同一时序路径可能存在不同种情况的组合，而且每种情况的参数都是呈统计学分布，组合的计算将不是单纯的相加差，而是需要相关性分析与统计学计算。统计方法的引入，改变了传统静态时序分析悲观但不实际的做法。

2. 芯片 signoff 的 Corner 设置

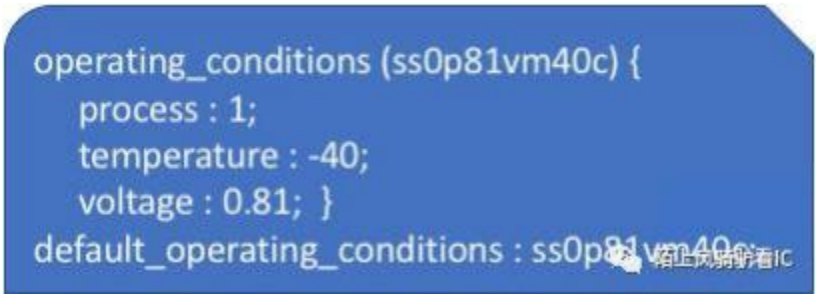
在 STA 星球，用 library PVT、RC corner 跟 OCV 来模拟这些不可控的随机因素。在每个工艺结点，通过大量的建模跟实测，针对每个具体的工艺，foundary 厂都会提供一张推荐的 timingsignoff 表格，建议需要

signoff 的 corner 及各个 corner 需要设置的 ocv 跟 margin。这些 corner 能保证大部分芯片可以承受温度、电压跟工艺偏差，

一个 corner=libraryPVT+ RC corner + OCV，本文将关注于 library PVT。

------OCV（on-chip-variation）也是用来模拟 cell 的 PVT 及线的 RC 变化，与前面两个不同的是，前两者是芯片全局的PVT/RC Corner，OCV 是芯片上内的局部偏差（包括 process 、voltage、temperature、network RC）。比如在 STA 分析 setup 时，并不是用最慢的 library PVT 来 signoff 就是最差情况，对于 capture clock 如果其跑的更快了，那么应该是更悲观的情况，对 RC Corner 也同理；所以在 STA 分析时，要结合 libraray PVT/RC corner/OCV 三者才能 cover 到芯片最坏情况。

PVT 也称为 Operating condition，是 STA 一个基本且重要的概念，在 library 的表头会有 operating condition 的定义，如下图，其中『ss0p81vm40c』是这个 operating condition 的名字，通常这个名字是有意义的，它会标示出该 lib 对应的电压跟温度，如 0p81 对应于 voltage:0.81，m40c 对应于 temperature:-40 度。



P-process：IC 制造工艺本身的不完美，使得制造偏差不可避免，在 library 中会用一个百分比来表示工艺偏差，如 process:1 表示没偏差。在沉积或参杂过程中，杂质浓度密度、氧化层厚度、扩散深度都可能发生偏差，从而导致管子的电阻跟阈值电压发生偏差；光刻过程中由于分辨率的偏差会导致管子的宽长比产生偏差。而这些偏差，都会导致管子性能的差异。

V-voltage：管子的延时取决于饱和电流，而饱和电流取决于供电电压。且不论多电压域芯片，就单电压芯片而言，电池的供电电压本身就在一个范围内变化，再加上片外或片上 voltage regulator 的误差、再加上

IR，一个芯片上的每个管子都可能工作在不同电压下，从而性能也有所差别。

T-temperature：在日常操作中，IC 芯片必须适应温度不恒定的环境，当芯片运行时，由于开关功耗、短路功耗和漏电功耗会使芯片内部的温度发生变化。温度波动对性能的影响通常被认为是线性的，但在深亚微米温度对性能的影响是非线性的。对于一个管子，当温度升高，空穴/电子的移动速度会变慢，使延时增加，而同时温度的升高也会使管子的阈值电压降低，较低的阈值电压意味着更高的电流，因此管子的延时减小。而通常温度升高对空穴/电子移动速度的影响会大于对阈值电压的影响，所以温度升高管子的延时呈增加趋势。但是并不是温度越低管子的延时就越小，晶体管有温度翻转效应，当温度低到某个值之后，随着温度的降低，管子的延时会增加，至于温度翻转点跟具体的工艺相关。

-----上面的 PVT 描述有疑问，既然本文讲的是 library 的 PVT，那么上述 PVT 应该是芯片的生产及工作条件，是全局偏差，比如温度和电压应该是整个芯片可能处于不同的电压温度情况，芯片内部的电压温度不同由 OCV 来 cover

在做 STA 分析时，operating condition 通常通过 lib 读入，如果多个 library 中的 operating condition 不同，通常用第一个读入的 library 中定义的 operating condition。也可以用命令设置/定义 operating condition，不同的工具有不同的实现方式，如：

set_operating_condition。在 debug 环境时可以 report_design 来 check 当前所用的 operating condition。

------对于 PT 而言，如果 PT 读入了多个 operating condition 的库，那个针对每个 cell 的 operating condition 的值是其 link 库的值，并不是第一个读入库的定义 operating condition；由第一个读入库来定义的标准为电压电容单位，transtion 的 pct 定义标准等。

Timing sign-offCorner=library PVT+ RC corner + OCV，PVT 在《巴山夜雨涨秋池，邀君共学 PVT：STA 之 PVT》中。今天来把玩一下 RC corner，这里的 RC 指 gate 跟 network 的寄生参数，寄生参数抽取工具根据电路的物理信息，抽取出电路的

电阻电容值，再以寄生参数文件输入给 STA 工具，常见的寄生参数文件格式为 SPEF。

ICer 都知道在集成电路中是多层走线的，专业术语叫metal layer，不同工艺有不同层 metal layer，任何两层 metal layer 间由介电材料隔离，『走线』通过过孔（VIA）连接。Width 跟 Spacing 是衡量绕线的两条最重要的物理设计规则，它们随着工艺的进步逐步减小。介电材料、绕线材料、线间距、线宽及线的厚度这些物理特性决定了 network 的 RC 值。

Network 电容：

耦合电容：Coupling capacitance=e*T/S

表面电容：Surface capcitance=e*W/H

边缘电容：Fringe capcitance

决定容值的因素：

介电常数：e

线宽：W

线厚：T

线间距：S

介电材料的厚度：H

随着工艺进步，W, S, T 逐代递减，表面电容跟随减小，耦合电容随之增加，耦合电容在总电容中占比增加，当线厚 T 一定时为了减少耦合电容要么增加线间距要么减小介电常数。通常为了减小噪声敏感信号线（如clock net）上的耦合电容，在物理实现时会人为增加对应信号的线宽及线间距，俗称 NDR。要减小介电常数需要从材料入手，从 .18 开始引入 low K 介电材料。

Network 电阻：

R=r/W*T, r 为电阻率，除了跟线宽 W 和线厚 T 相关之外，还跟温度相关，随着温度的上升而增大。

Parameters	Resistance	Surface Capacitance	Coupling Capacitance
温度增加	增加	--	--

线宽减小	增加	减小	--
线厚减小	增加	--	减小
线间距减小	无影响	无影响	增加

由上面的分析可知，Network 的单位电容和单位电阻是不可能同时最大或同时最小的。有了这些铺垫，来看一下不同工艺结点是如何定义 RC corner 的。

90nm 之前，Cell delay 占主导，Network 电容主要是对地电容，STA 只需要两个 RC corner 即可：

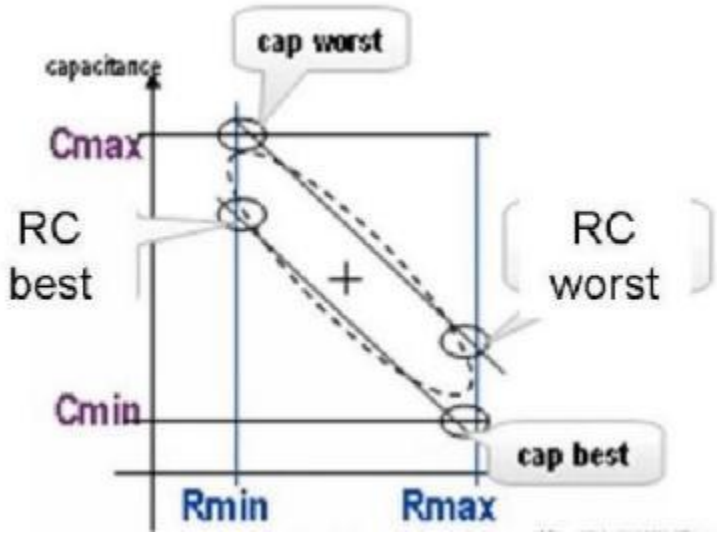
Cbest(Cmin): 电容最小电阻最大

Cworst(Cmax):电容最大电阻最小

90nm 之后，netdelay 的比重越来越大，而且network 的耦合电容不可忽略，所以又增加了两个 RC corner:

RCbest(XTALK corner): 耦合电容最大，（对地电容*电阻）最小

RCworst(Delay corner): 耦合电容最小，（对地电容*电阻）最大



至此总共有两个需要 setup timing sign-off 的 RC corner，有四个需要 hold timing sign-off 的 RC corner：

Setup time sign-off 的 RC corner 是: Cworst / RCworst

Hold time sign-off 的 RC corner 是: Cbest / RCbest / Cworst / RCworst

C-best:

It hasminimum capacitance. So also known as Cmin corner.

Interconnect Resistance is larger than the Typical corner.

This corner results in smallest delay for paths with short nets and can be used for min-path-analysis.

C-worst:

Refers tocorners which results maximum Capacitance. So also known as Cmax corner.

Interconnect resistance is smaller than at typical corner.

This corners results in largest delay for paths with shorts nets and can be used for max-path-analysis.

RC-best:

Refers tothe corners which minimize interconnect RC product. So also known as RC-mincorner.

Typicallycorresponds to smaller etch which increases the trace width. This results insmallest resistance but corresponds to larger than typical capacitance.

Corner has smallest path delay for paths with long interconnects and can be used for min-path-analysis.

RC-worst:

Refers tothe corners which maximize interconnect RC product. So also known as RC-maxcorner.

Typicallycorresponds to larger etch which reduces the trace width. This results inlargest resistance but corresponds to smaller than typical capacitance.

Corner has largest path delay for paths with long interconnects and can be used for max-path-analysis.

------C = a * W *L　R = b *L/W　RC = ab * L^2

所以对于短线来说（L 很小），RC 由于与 L 的平方成正比，非常小，电容起主要作用。对于长线 RC 起主要作用。

Q：还有一种 RC corner 带后缀『_T』，只用于 setup signoff，T 指的是什么？

A：T 代表 tighten，在 rc 的 variation 上的 sigma 分布比不带 T 的更紧，因此只能用于 setup，hold 不推荐。Appleto Apple 地比较，T 的 variation 更小，理论看到的 rc 变化更小，单从 setup 产生violation 的可能性更小。至于悲观还是乐观，除非自己定 criteria，其余的 follow foundry 或者 vendor 的 rule 最重要。

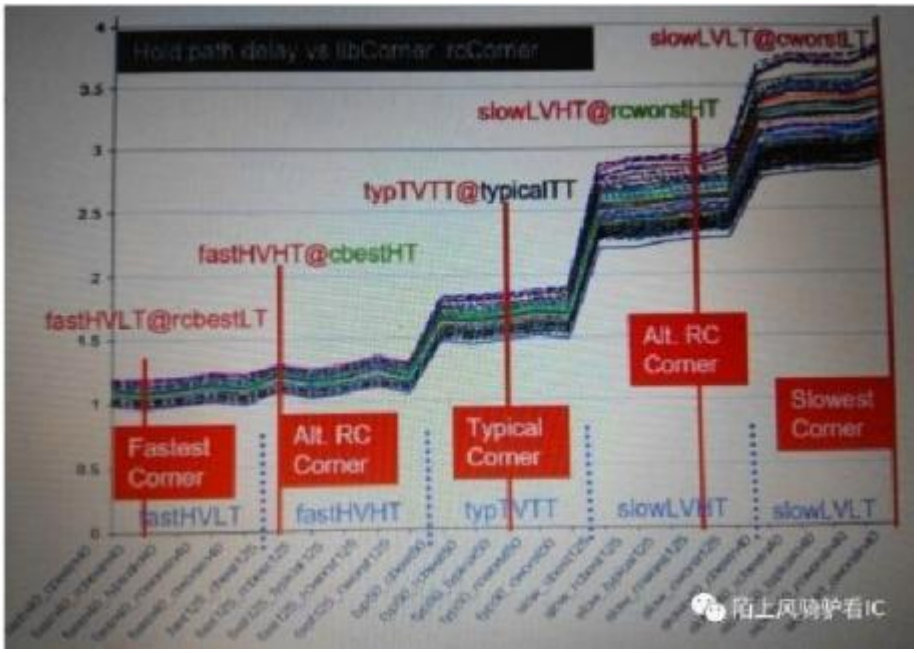
Q：为什么 setup 既需要 sign-offC-corner 又需要 sign-offRC-corner？

A：因为 C-corner 表示『电容』最大/最小，而 RC-corner 是『电容*电阻』最大/最小。通常对于短线而言，电容占主导地位，C-corner 可以 cover RC-corner，但对于长线则电阻占主导地位，C-corner 无法 cover RC-corner，而是反过来

RC-corner 去 cover C-corner。而没人保证一个设计里只有短线没有长线，也没权威对长短的幅度有量化的定义，所以最保险的就是两者分别 sign-off。

Q：为什么 hold 需要 sign-off 所有的 corner？

A：对于 hold 而言，根据其仿真曲线，相互之间都无法完全覆盖，故需要 sign-off 所有 corner。



Q：金属电阻是否跟 density/spacing 相关？为什么方块电阻跟 density/spacing 相关？

A：就电阻本身而言，阻值跟 density 没有关系，只跟金属线自身的宽厚和电阻率相关，

不是每一个程序员都必须玩过linux，只是博主觉得现在的很多服务器都是 linux 系统的，而自己属于那种前端也搞，后台也搞，对框架搭建也感兴趣，但是很多生产上的框架和工具都是安装在服务器上的，而且有不少大公司都要求熟悉在 linux 上开发，因此从个人职业发展有必要去多了解一下linux。

（博主网上查了一下，现在大部分服务器安装的系统主要有：ubuntu server、centos、suse linux enterprise, redhat linux 等）

于是今天博主就在虚拟机上安装了 Ubuntu，并分享整个安装过程给大家。之所以没有安装双系统，是考虑多很多不方便的地方，比如，你在 linux 上开发了一个实例，你想写博客，写博客肯定是在 windows 上方便多了，代码需要从一个系统弄到另外一个系统，这里是双系统的不方便。另外，虚拟机上安装linux 的另外一个好处就是，你可以在 windows 上下载了工具安装包，然后在 linux 上安装。

好了，下面马上附上安装全过程。

VMware Ubuntu 安装详细过程（非常靠谱）--转 验证过可行

一、下载 Ubuntu 镜像文件

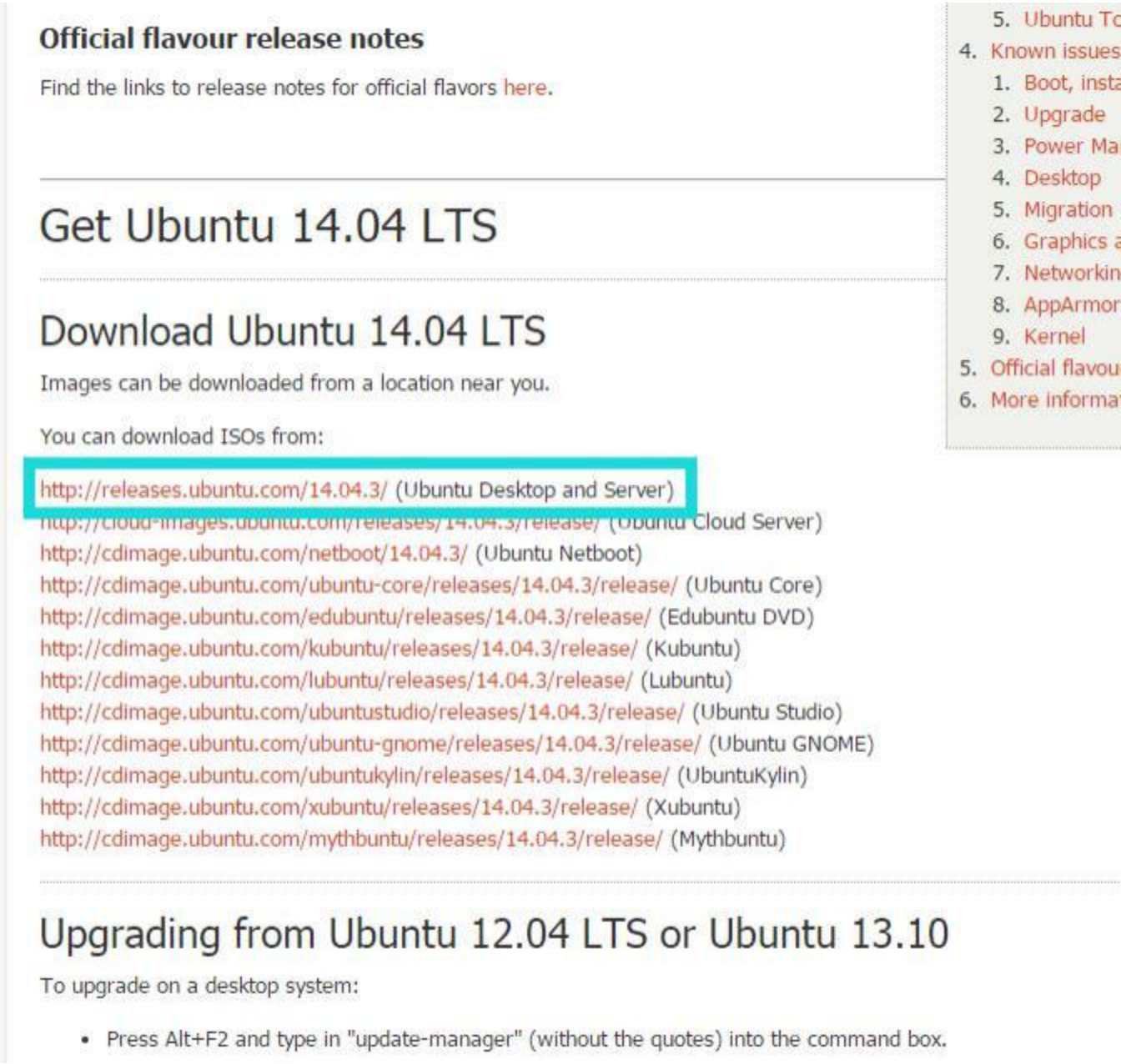
下载地址：http://www.ubuntu.com

打开上面的连接，我们来到如下页面，点击download：

下一步，到如下页面，点击 Ubuntu Desktop：

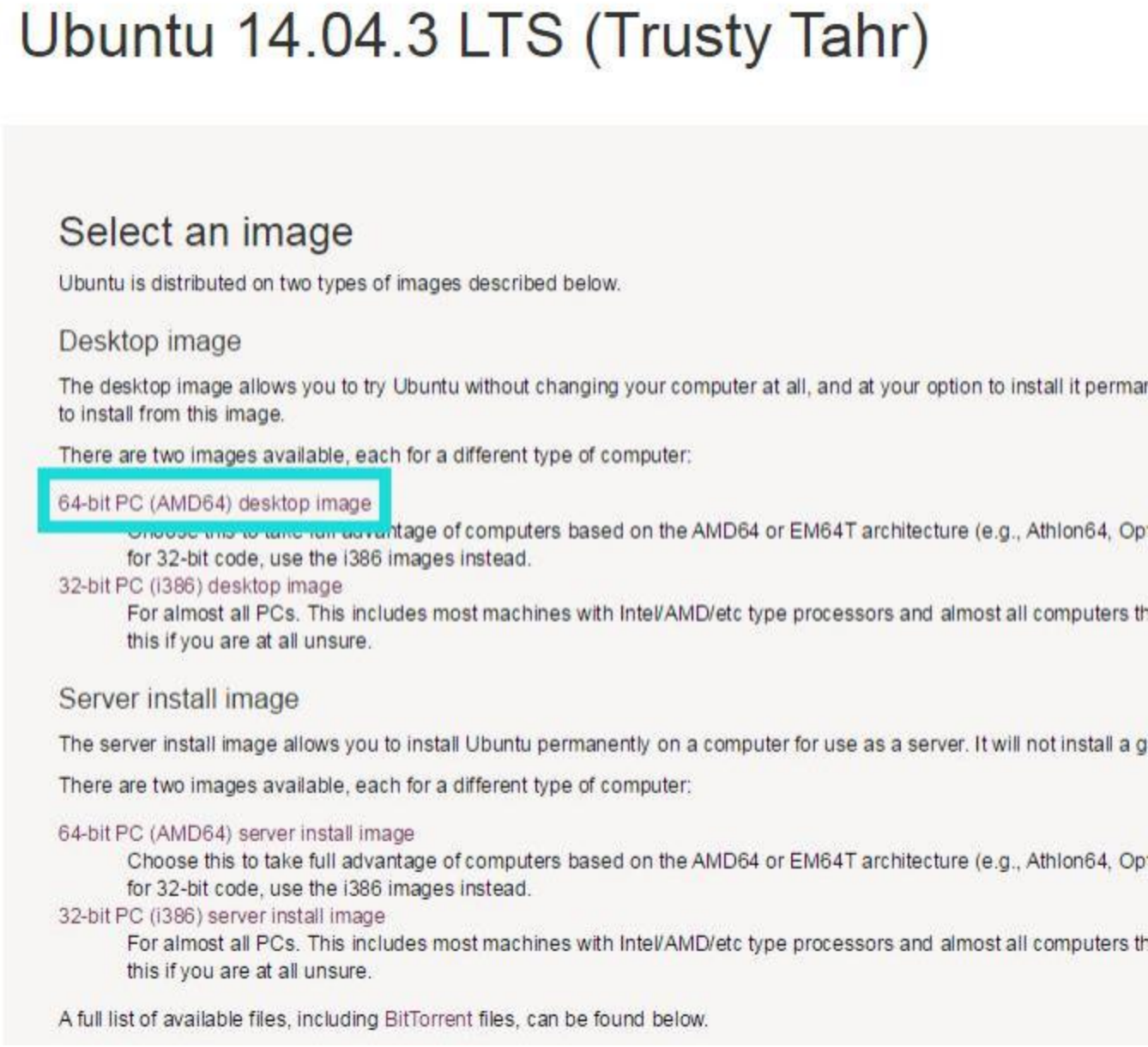
然后选择 Ubuntu 14.04.3 LTS release notes：

然后选择 Ubuntu Desktop and Server：



接下来我们选择桌面版 64-bit PC (AMD64) desktop，博主的电脑是 64 位的。另外 desktop 和 server 版的区别主要是：桌面版面向个人电脑

使用者，可以进行文字处理、网页浏览、多媒体播放和玩游戏。本质上说，这是一个为普通用户所定制的多用途操作系统。另一方面，服务器版旨在充当web 服务器，可用来托管文件、网页及相似内容。



点击上面的连接即可下载，容量大概 1G 左右，时间有点长，接下来我们先看 VMware 的下载安装与在 VMware 上安装 Ubuntu 的过程。

二、VMware 下载安装

VMware 的下载安装就好办了，下载直接百度如下，哈哈：

下载完成即可安装，安装可以全部采取默认，看个人而定，博主的安装目录是自选的。

安装完成之后启动 VMware，需要输入产品密钥（vmware workstation 12），博主随便百度了一个，猿友看你的是否有效：

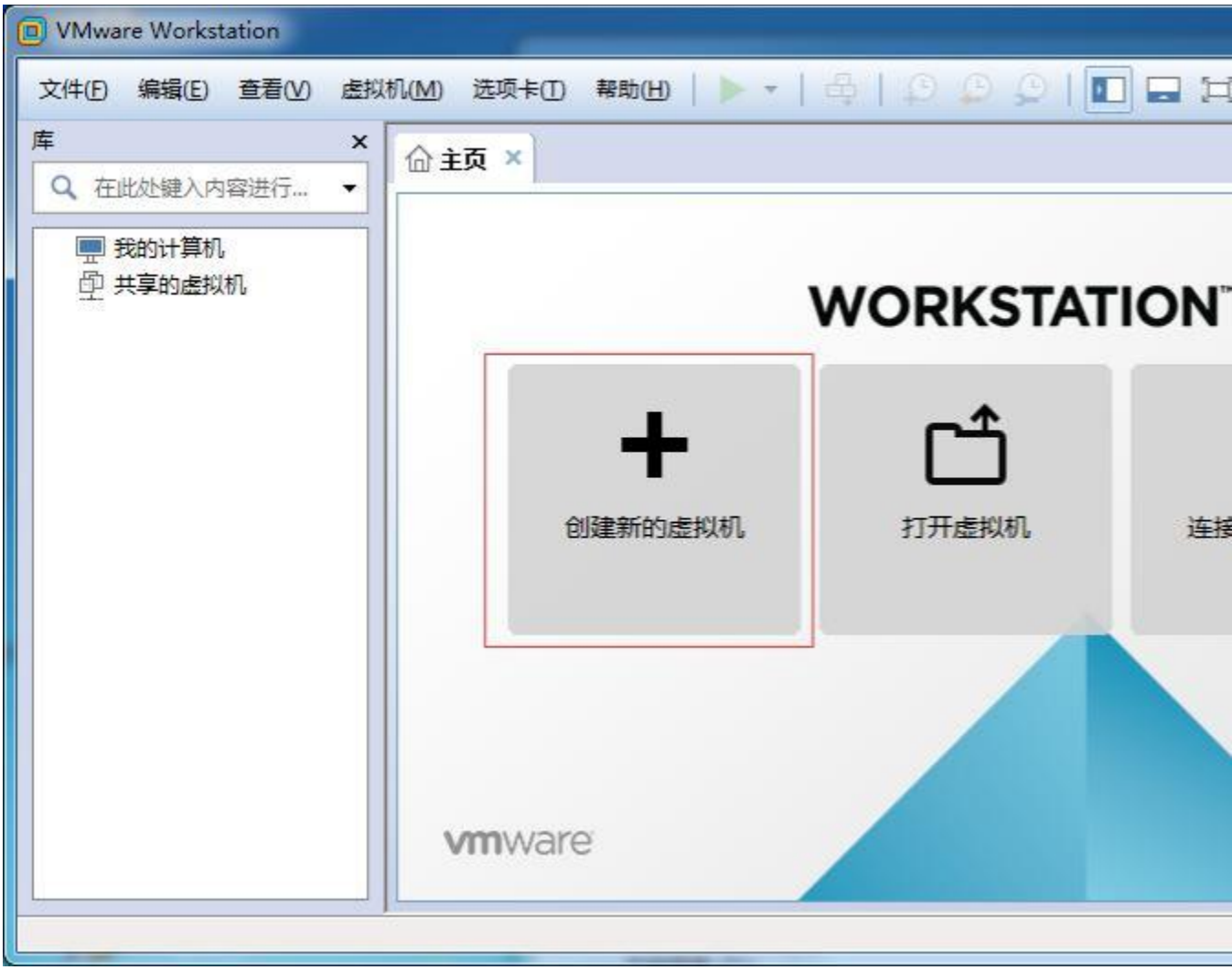
```
5A02H-  243-  49-  7 -3C61N
```

. 1

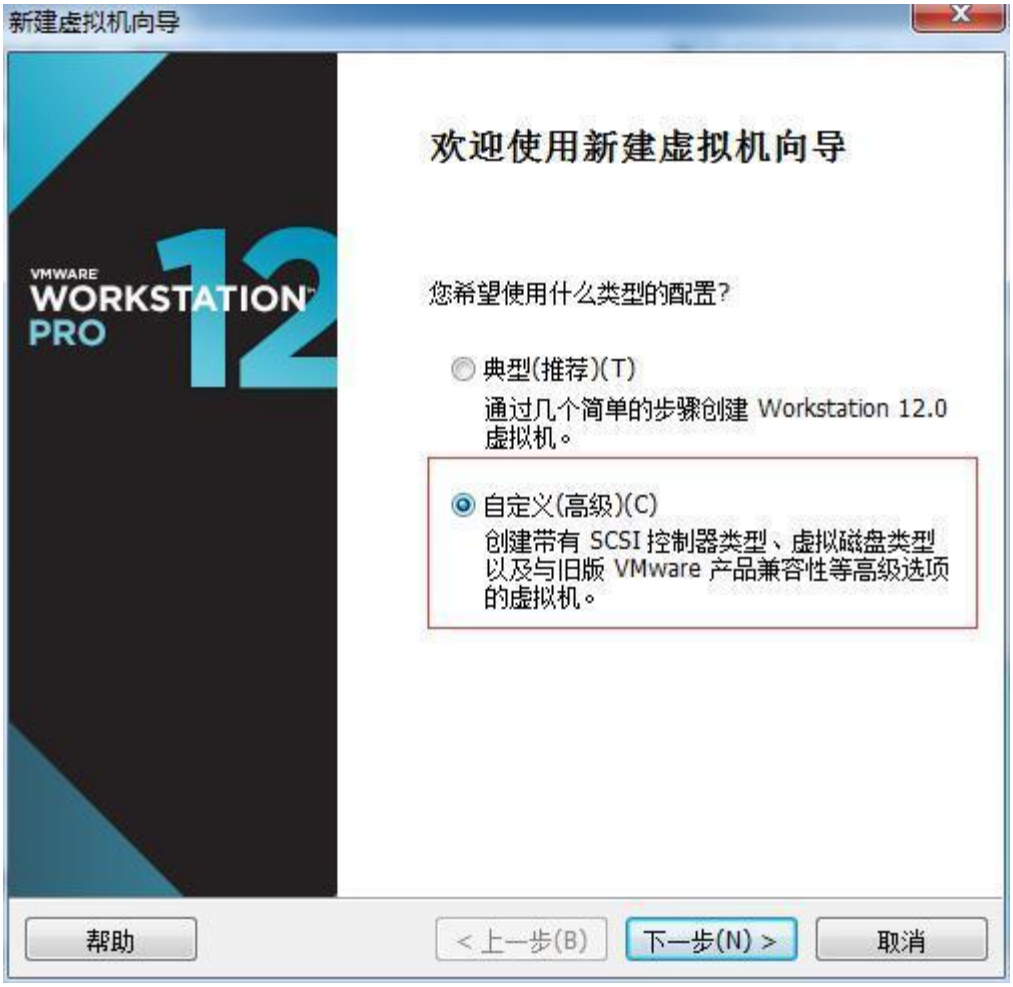
VMware 安装完毕，下面介绍在 VMware 安装 Ubuntu。

三、在 VMware 安装 Ubuntu

1、创建虚拟机


VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T) 帮助(H)
库
在此处键入内容进行...
我的计算机
共享的虚拟机
主页
WORKSTATION
创建新的虚拟机
打开虚拟机
vmware

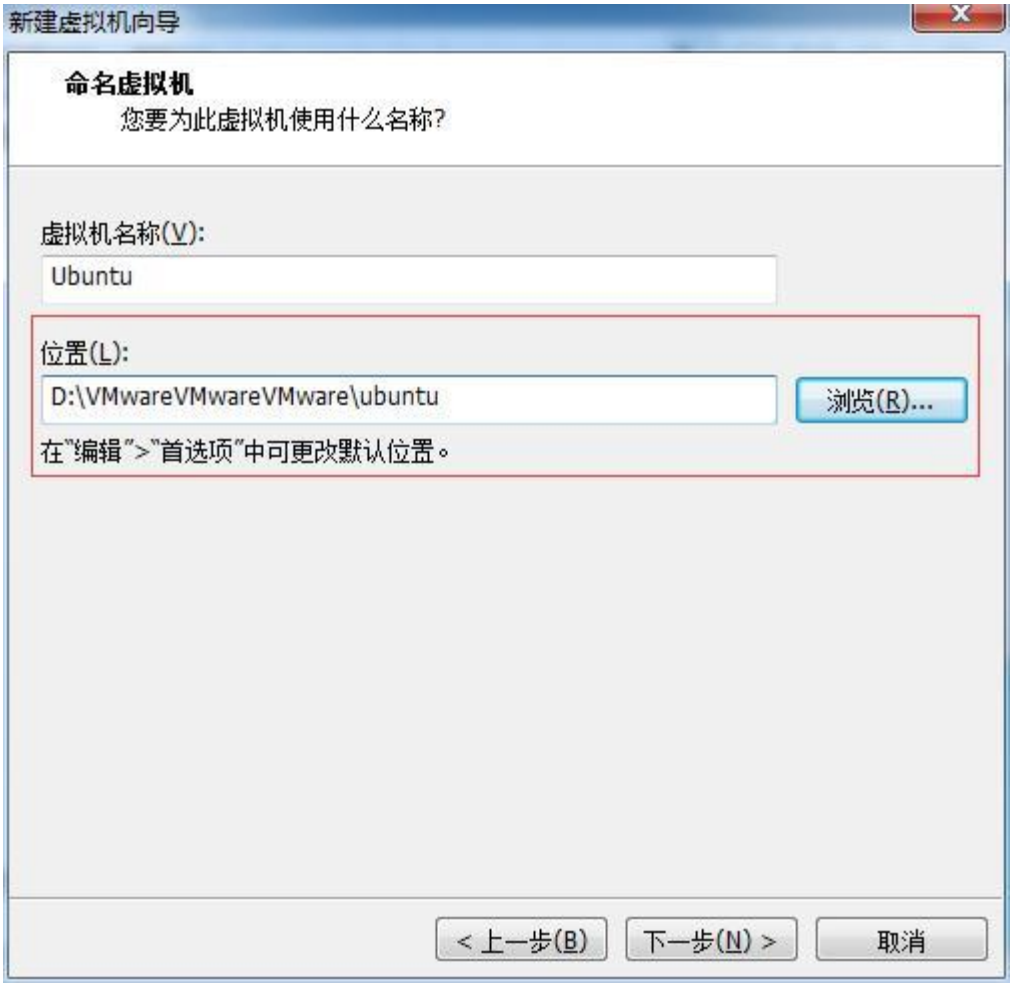
2、 向导选择自定义



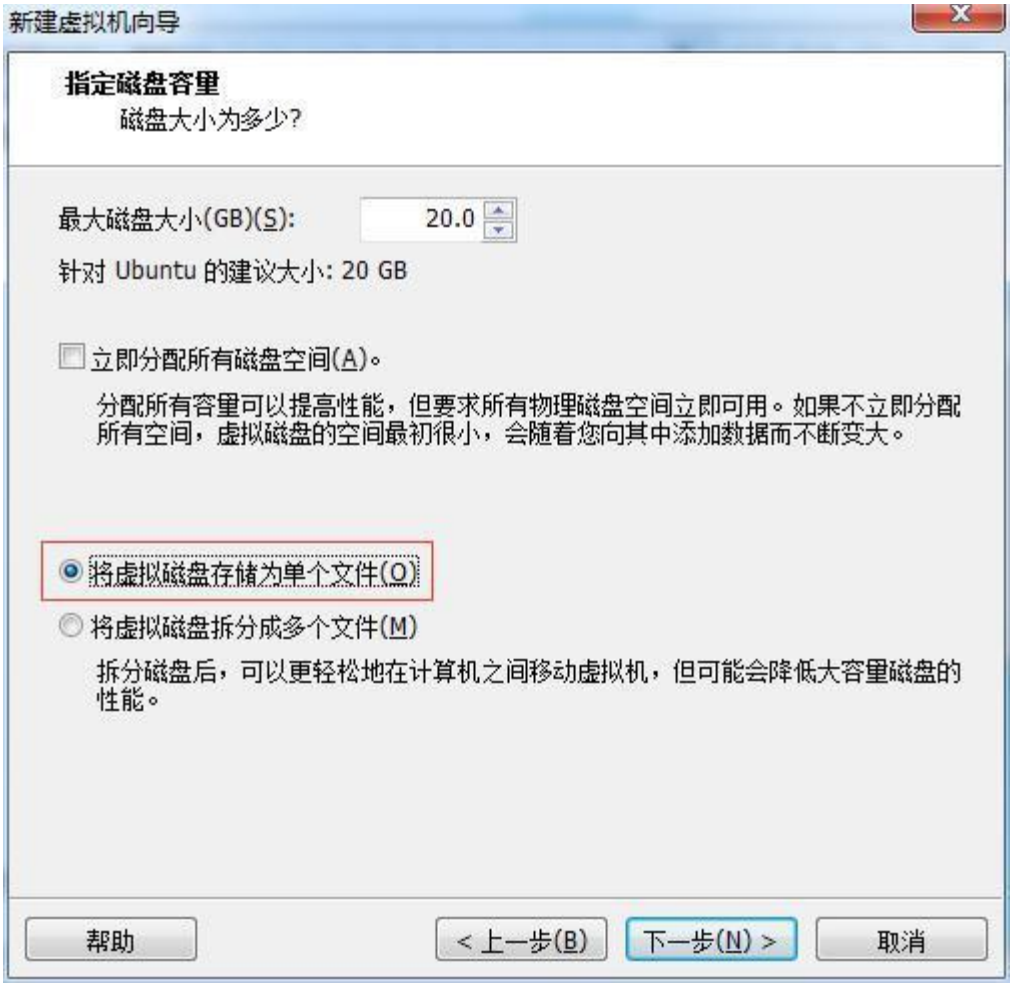
3、然后下一步再下一步，直到这里，稍后再安装系统

4、然后选择 linux，注意这里下面的下拉选择 Ubuntu64，因为我们下载的是 64 位的，如果你的电脑是 32 位的，就选Ubuntu 即可，博主的就是因为Ubuntu，导致后面安装出错了，不过后面还可以设置的。

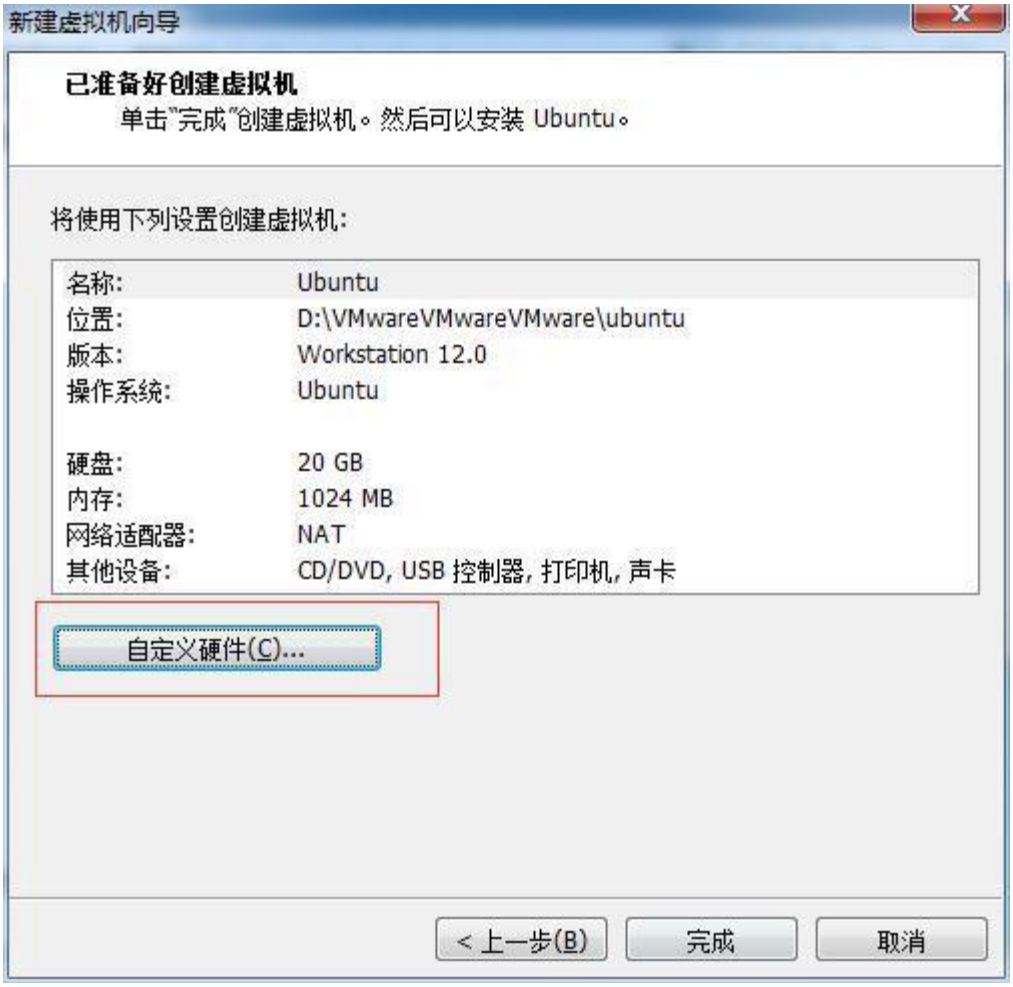
5、选择安装位置，这里必须输入一个已存在的目录，不然后面会报错的



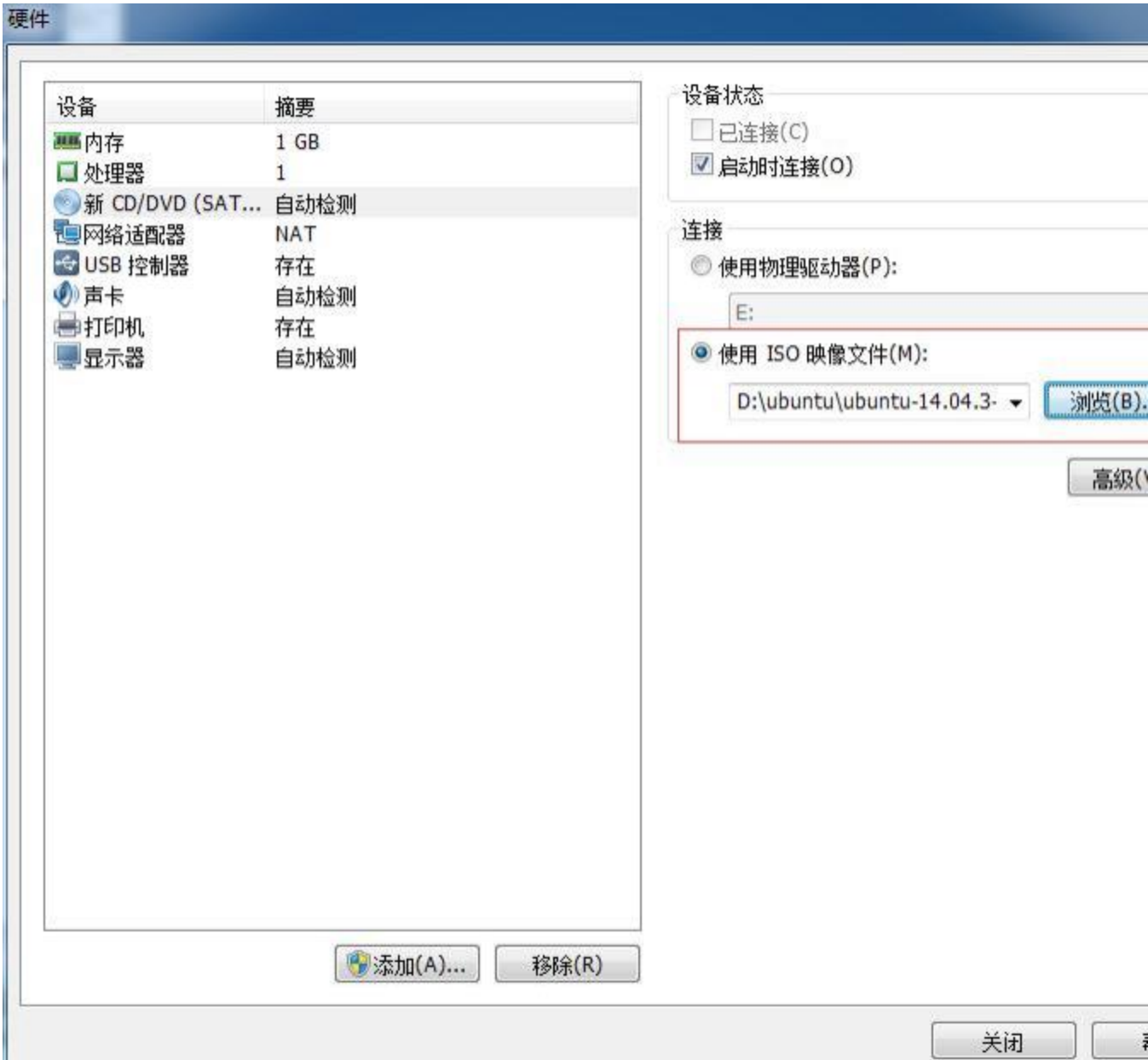
6、后面设置处理器和内存的，电脑配置好的可以试试，否则采用默认的，博主这里是采用默认的，然后下一步…，直到这里，选择将虚拟机存储为单个磁盘：



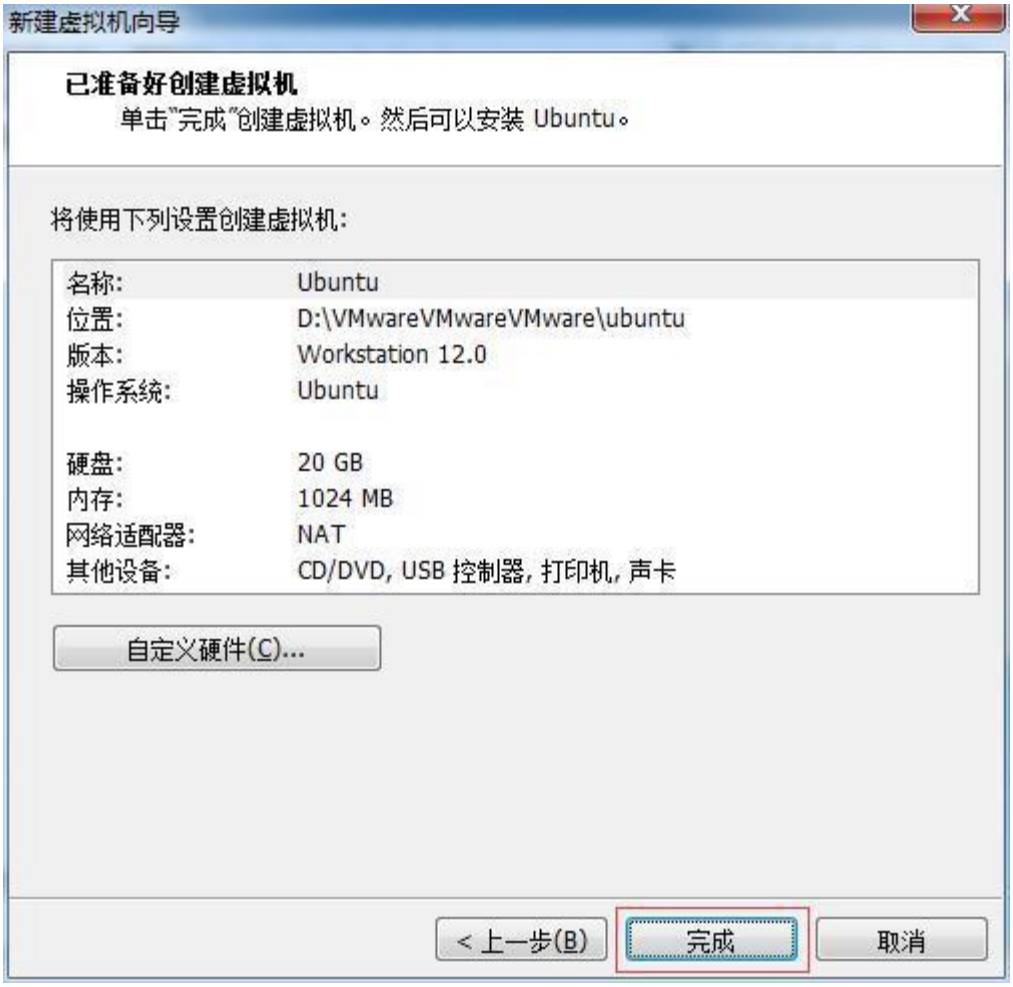
7、然后下一步，到如下页面，点击自定义硬件：



8、然后如下图选择我们第一步下载的 Ubuntu 镜像：



9、然后点击完成，向导设置完成



10、虚拟机已经配置完毕，接下来我们开启虚拟机：

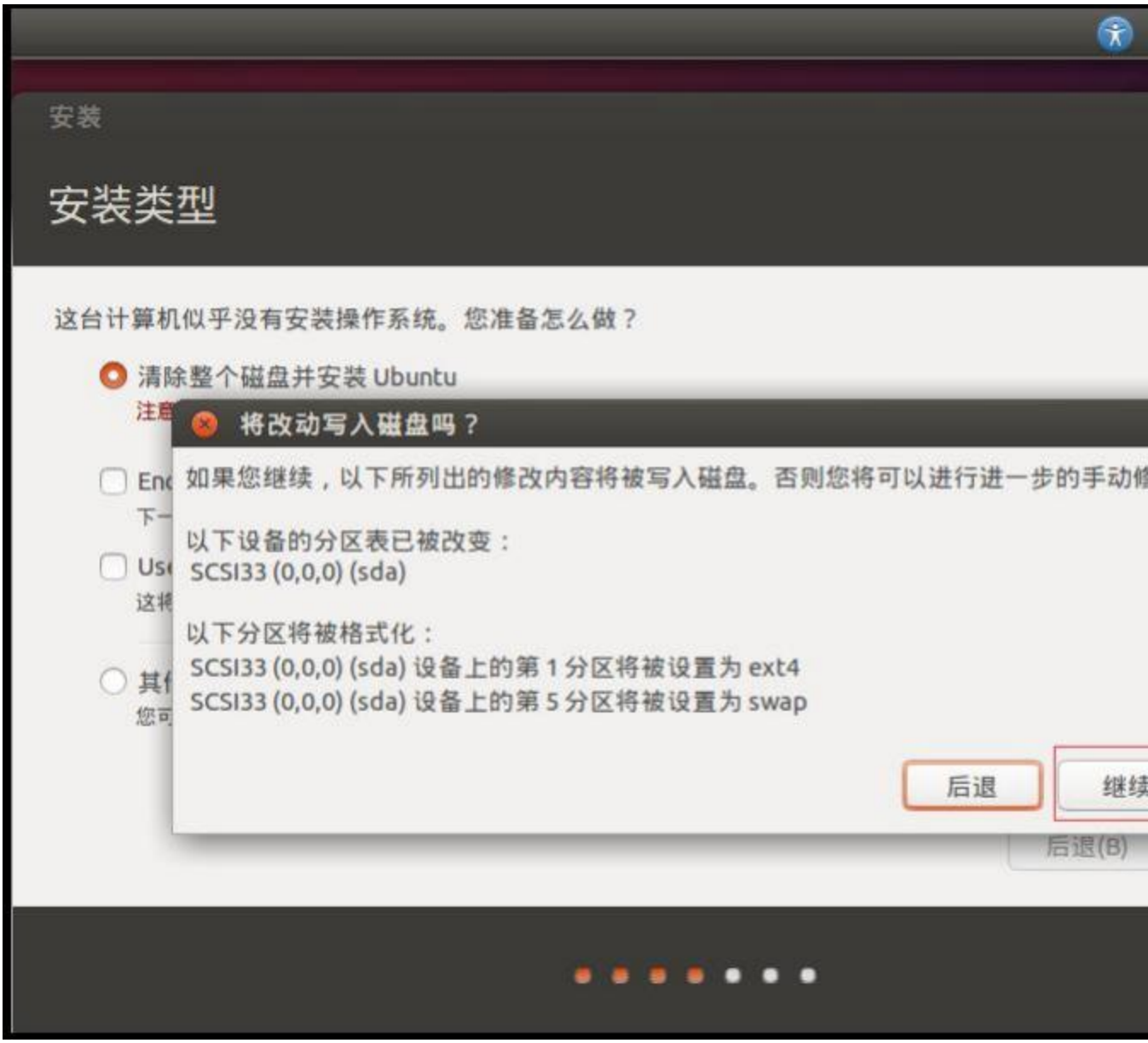
11、然后就会来到如下界面，如果出现错误请看（四、可能遇到的错误），我们选择中文简体点击 Install Ubuntu：

12、到了如下界面，我们点击继续：

安装
准备安装 Ubuntu
要获得最佳的体验，请确定这台计算机：
有至少 6.6 GB 可用的磁盘空间
已经连接到互联网
安装中下载更新
Ubuntu 使用第三方软件播放 Flash、MP3 和其他媒体，并驱动某些图形和无线网络硬件。其中某些
这些软件受到其文档中协议条款的约束。
安装这个第三方软件
Fluendo MP3 插件包含 Fraunhofer IIS 和 Technicolor SA 授权的 MPEG Layer-3 音频解码技术。
退出(Q)
后退(B)

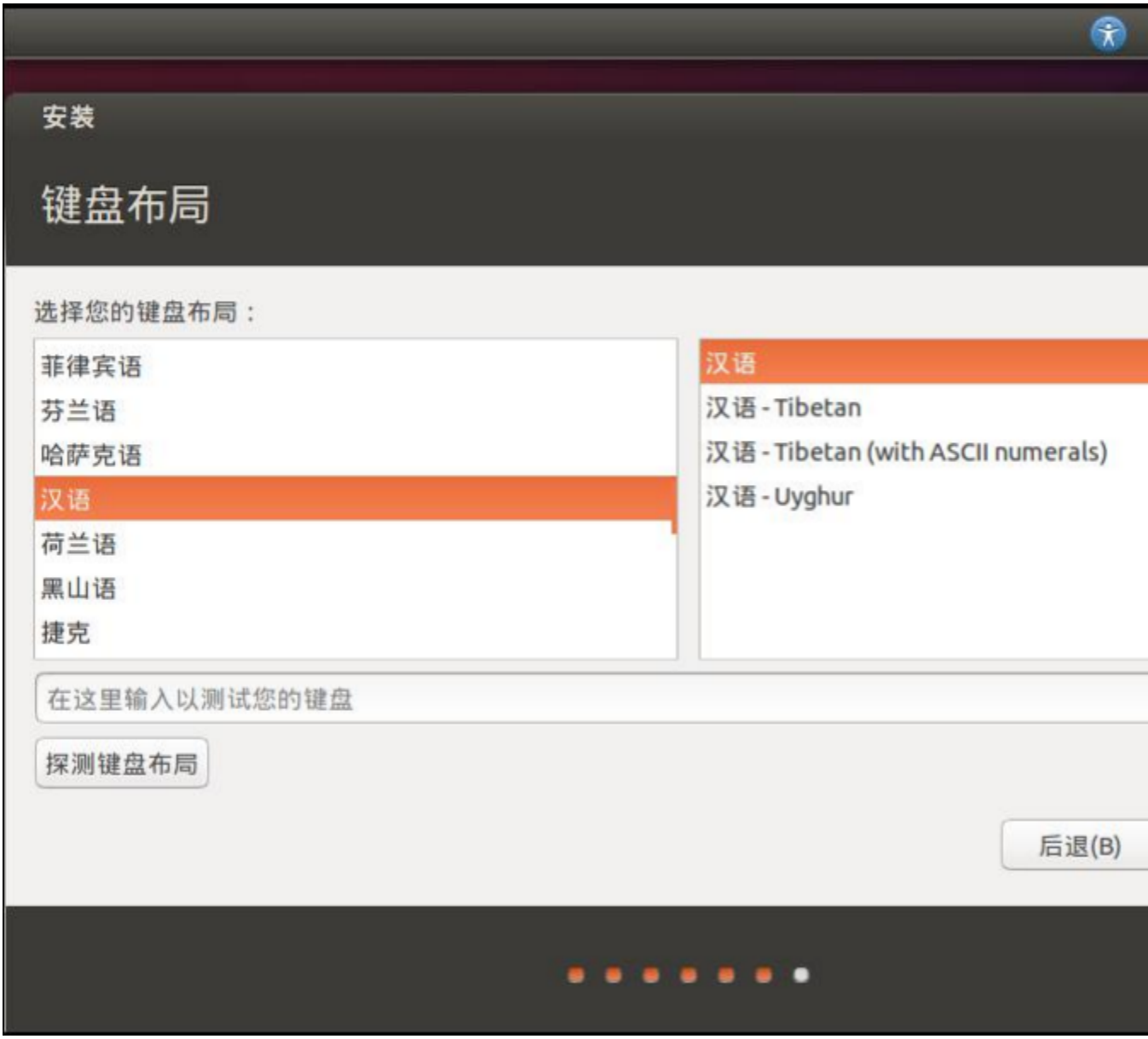
13、然后点击，开始安装：

14、到了这界面，点击继续：



15、如下，输入你的位置，随便输入就好：

16、然后选择汉语，点击继续：



17、设置用户名密码，这里博主选择自动登录

18、下面就开始正式安装了：

19、安装完成之后，会提示重启，点击现在重启

20、重启成功之后，会来到桌面，博主的重启中遇到一些问题，手动重启的，影响不大：关键

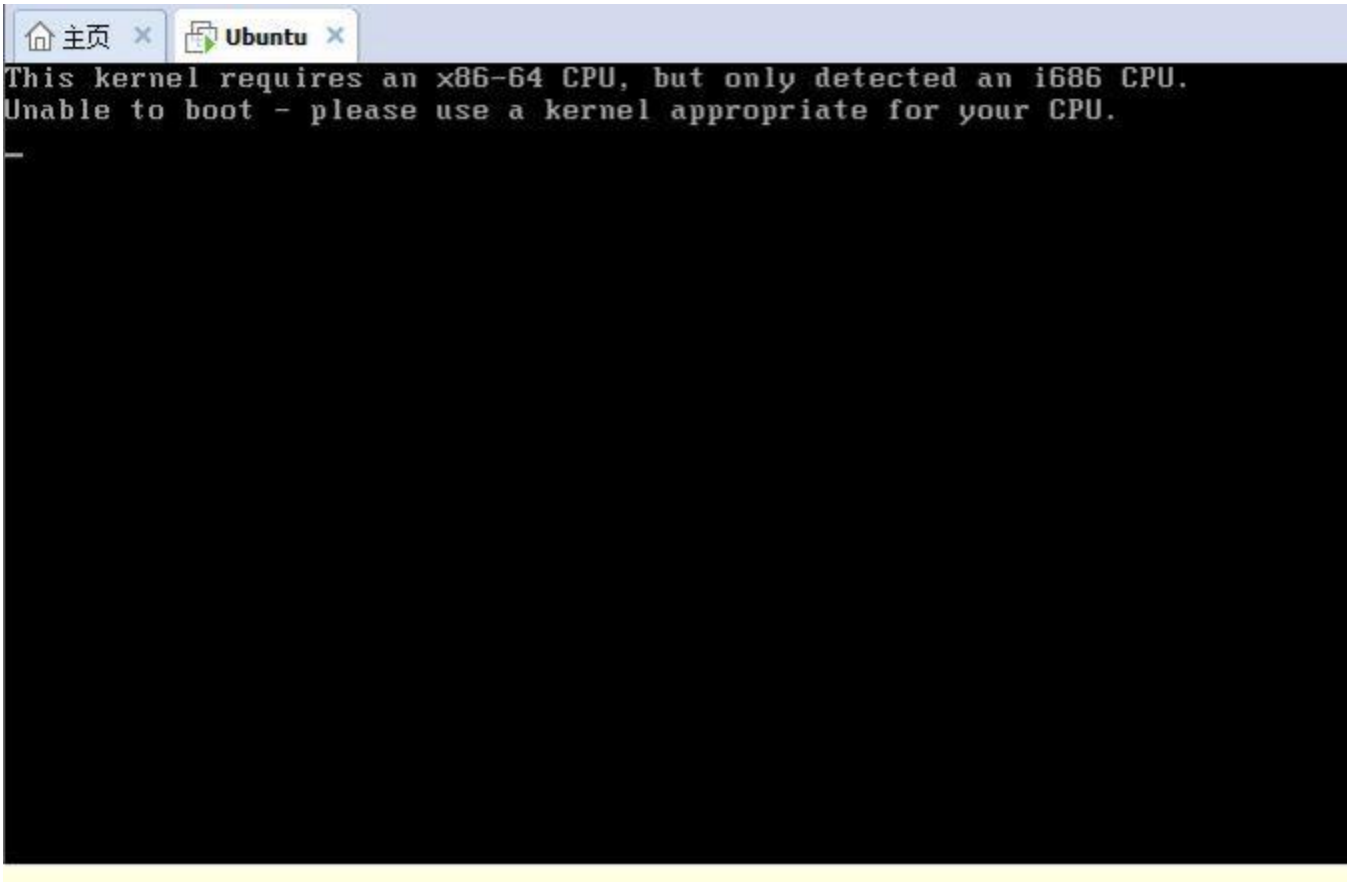
21、我们打开浏览器，输入百度网址，发现可以访问网络：

安装完成！！！！！！！！！！

四、可能遇到的错误

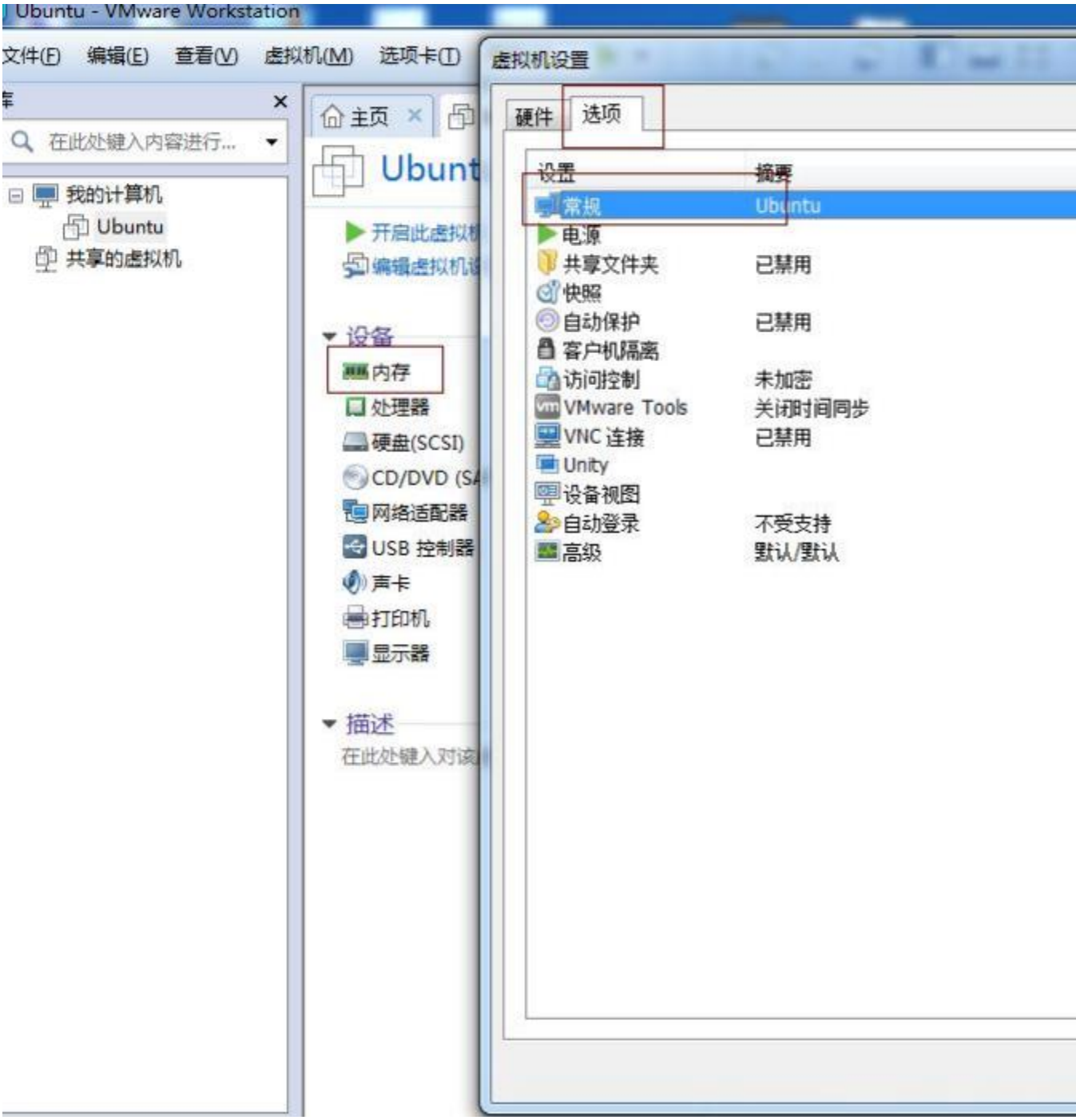
在安装过程中，博主遇到了如下的错误：

4.1、This kernel requires an x86-64 CPU, but only detected an i686 CPU.如下图：

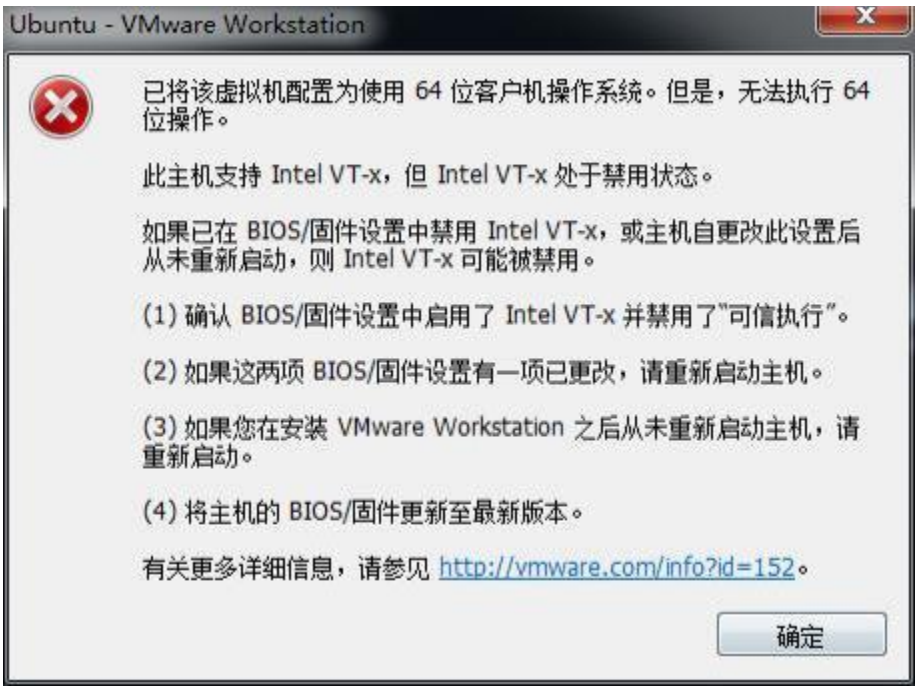


可能原因是：在创建虚拟机向导的时候我们如下页面选择了Ubuntu，而不是 Ubuntu64，但是我下载的镜像是 64 位的，如下图：

解决办法：如下设置回 Ubuntu64 即可，然后继续你没有完成的步骤：


Ubuntu - VMware Workstation
文件(F) 编辑(E) 查看(V) 虚拟机(M) 选项卡(T)
在此处键入内容进行...
我的计算机
Ubuntu
共享的虚拟机
主页
Ubunt
开启此虚拟机
编辑虚拟机设
设备
内存
处理器
硬盘(SCSI)
CD/DVD (SA
网络适配器
USB 控制器
声卡
打印机
显示器
描述
在此处键入对该
虚拟机设置
硬件
选项
设置
摘要
常规
Ubuntu
电源
共享文件夹
已禁用
快照
自动保护
已禁用
客户机隔离
访问控制
未加密
VMware Tools
关闭时间同步
VNC 连接
已禁用
Unity
设备视图
自动登录
不受支持
高级
默认/默认

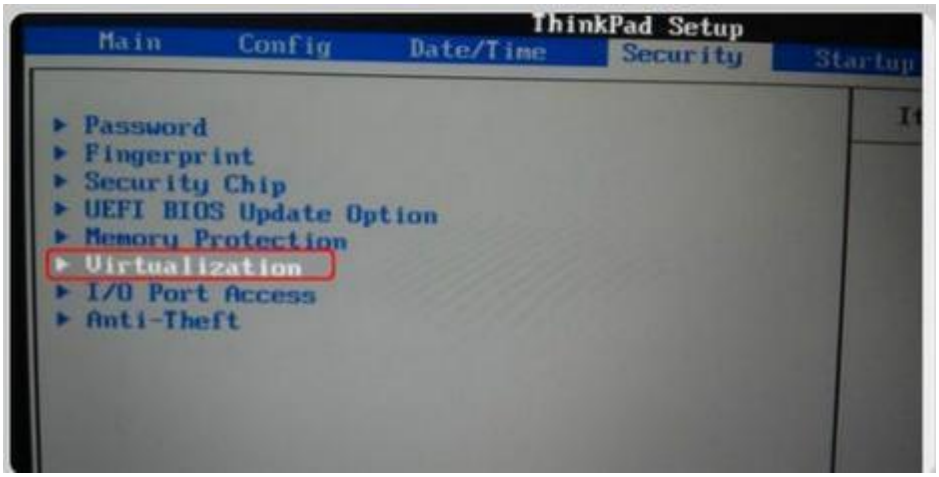
4.2、报如下错误：



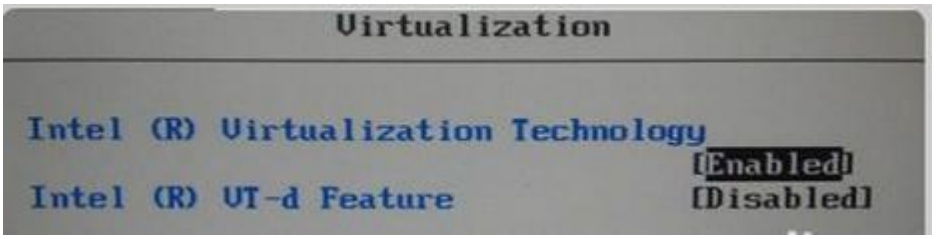
可能原因是：你的电脑没有支持虚拟化

解决办法：重启电脑，进入 bios（博主的进入 bios 的键是 F10），设置虚拟化启用。

进入 bois 后选择 Security 选项：



选择 Virtualization，回车进入，按+键修改俩个选项为 enabled：



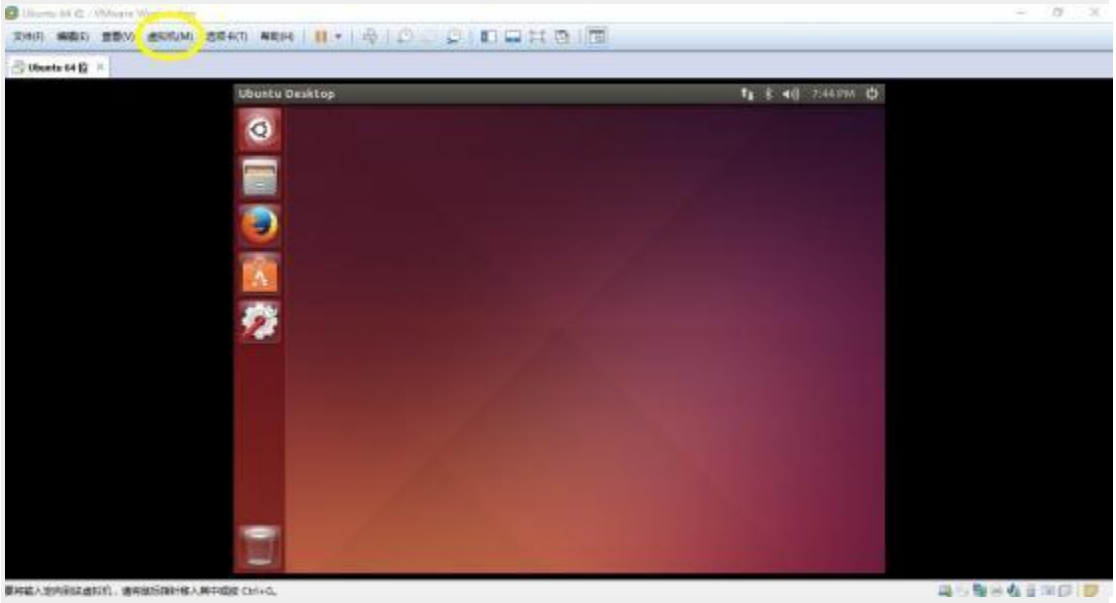
然后按 F10，输入 y 保存退出即可。

设置了启用了虚拟化之后，继续上面没有完成的步骤即可

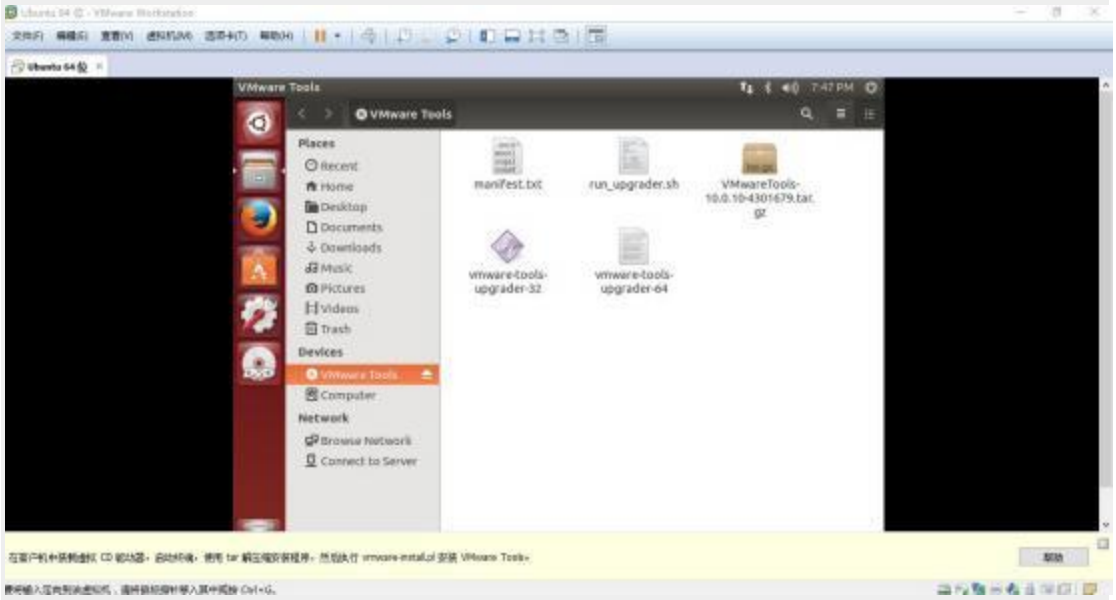
4.3、如果虚拟系统安装成功之后重启失败了（如停留在某个页面时间过长），手动重启 Ubuntu 即可

VMware Tools 安装方法及共享文件夹设置方法—转

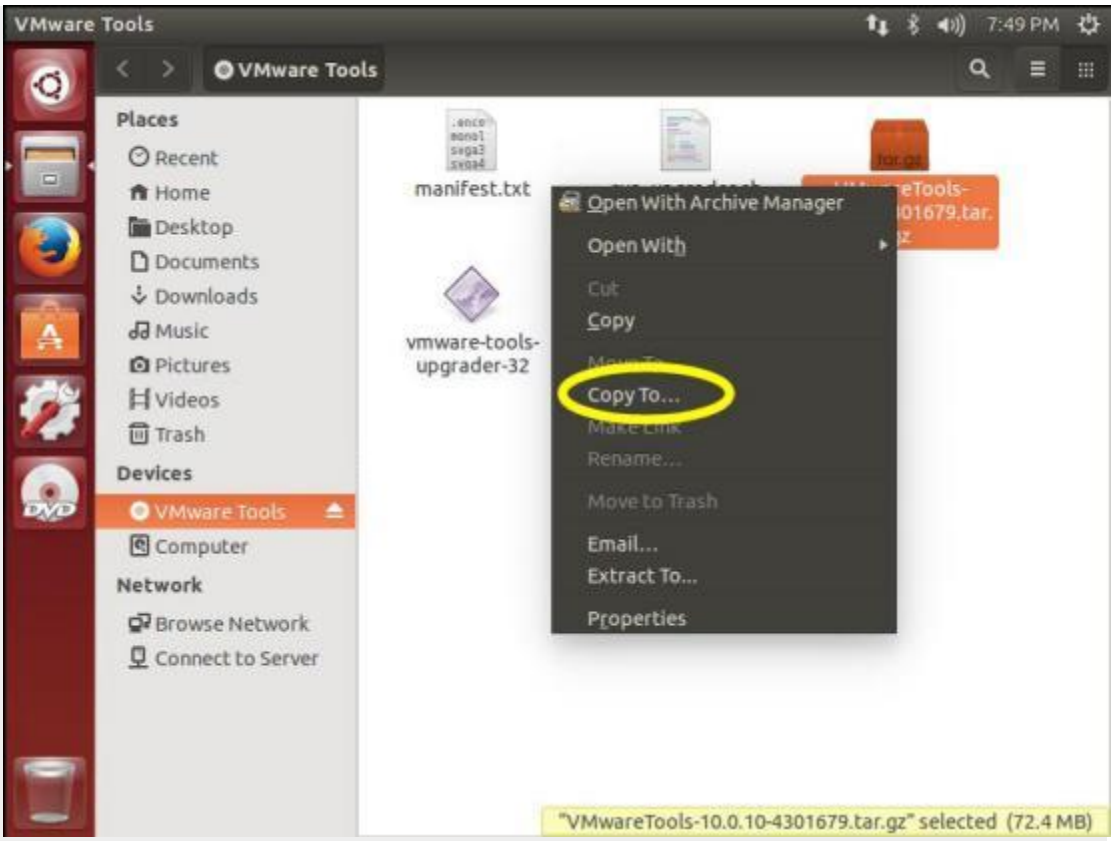
正确安装好 VMware Tools 后，可以实现主机与虚拟机之间的文件共享，
可以设置共享文件夹，以及在主机与虚拟机之间直接进行复制黏贴的操作。
安装方法：
选择"虚拟机"—>"重新安装 VMware Tools"



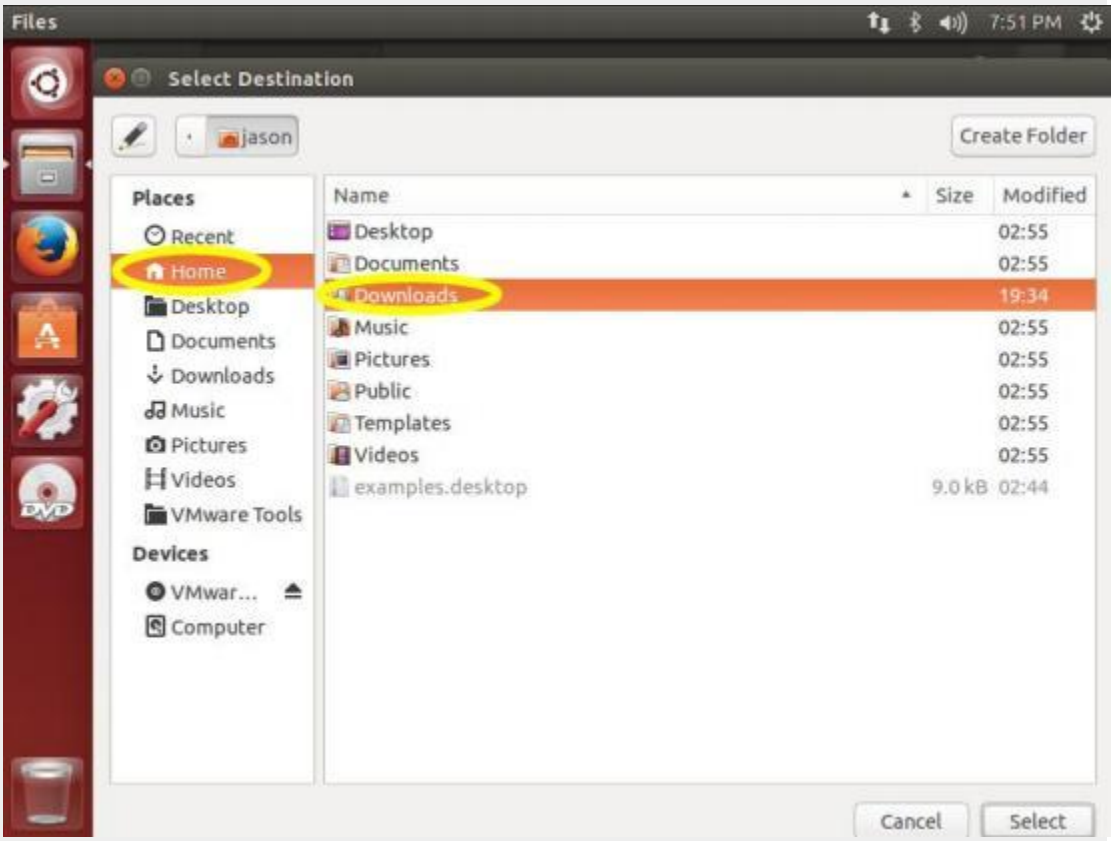
弹出以下界面：



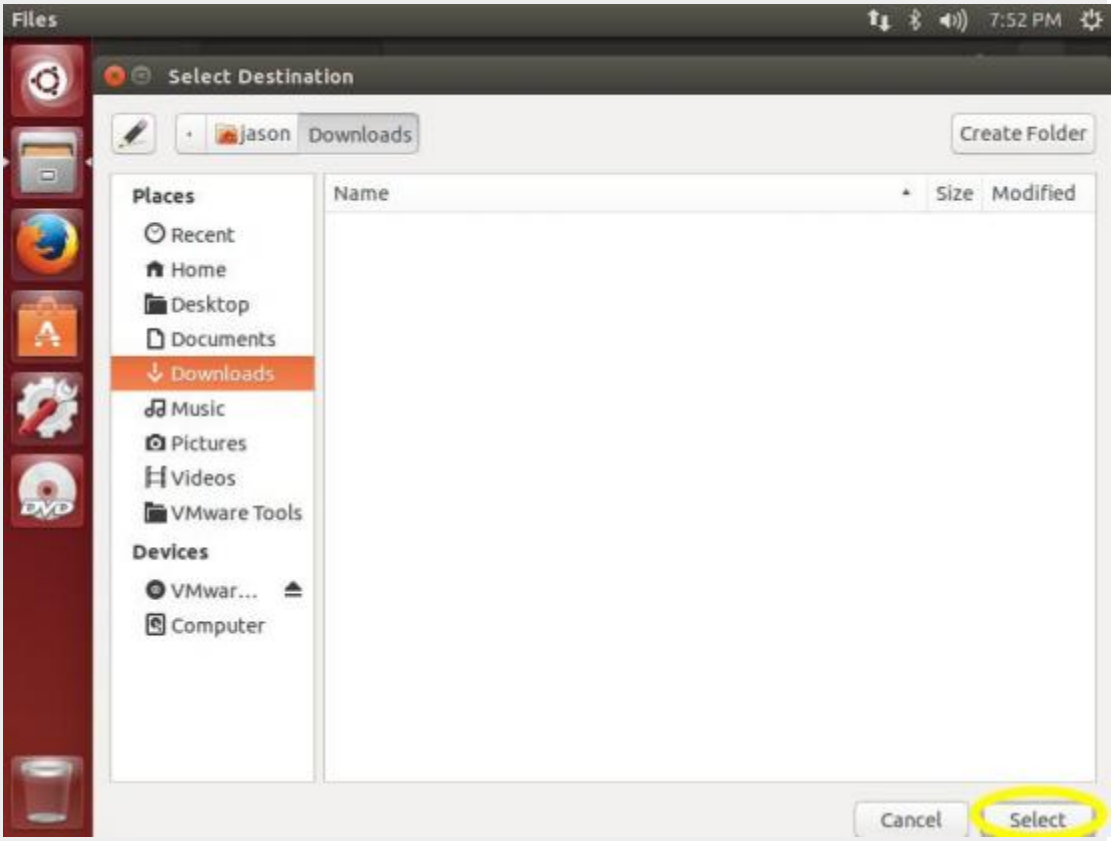
鼠标右键点击 VMwareTools-10.0.10-4301679.tar.gz，选择"Copy To..."



选择"Home"—>"Downloads":

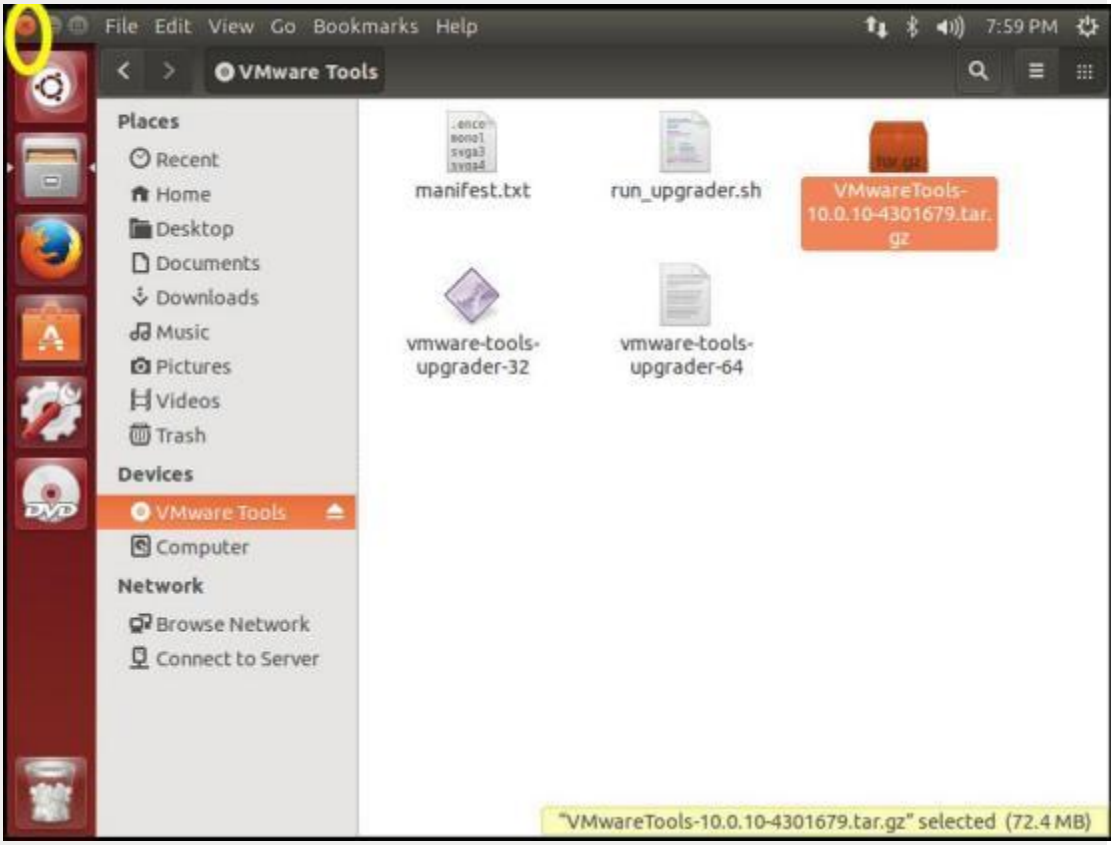


双击"Downloads"，点击右下角的"Select":

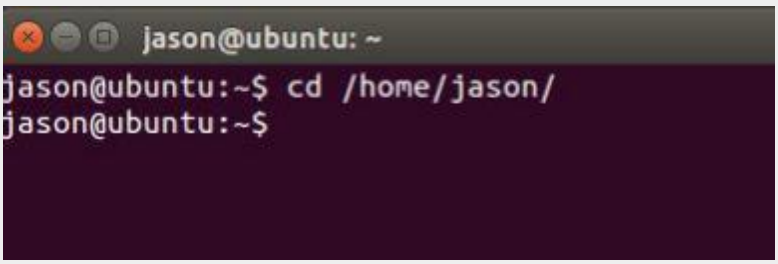


这样，就将 **VMwareTools-10.0.10-4301679.tar.gz** 复制到**"/home/用户/Downloads/"**下

点击左上角的**"X"**，关闭 **Files** 窗口。



接下来，开始解压安装 VMware Tools:
按"Ctrl"+"Alt"+"T",打开终端。
输入 cd /home/ ， 按一次 Tab 键，进入用户目录，我的是/home/jason/，按下回车键。



输入 cd Downloads/ + 回车键
输入 ls + 回车键

发现 **VMwareTools-10.0.10-4301679.tar.gz** 已经被复制到/home/用户名/Downloads/下

接下来，输入 **tar zxf VMwareTools-10.0.10-4301679.tar.gz** ，回车键，解压该文件。

解压完毕后，输入 **sudo ./vmware-tools-distrib/vmware-install.pl** ,回车键，开始安装

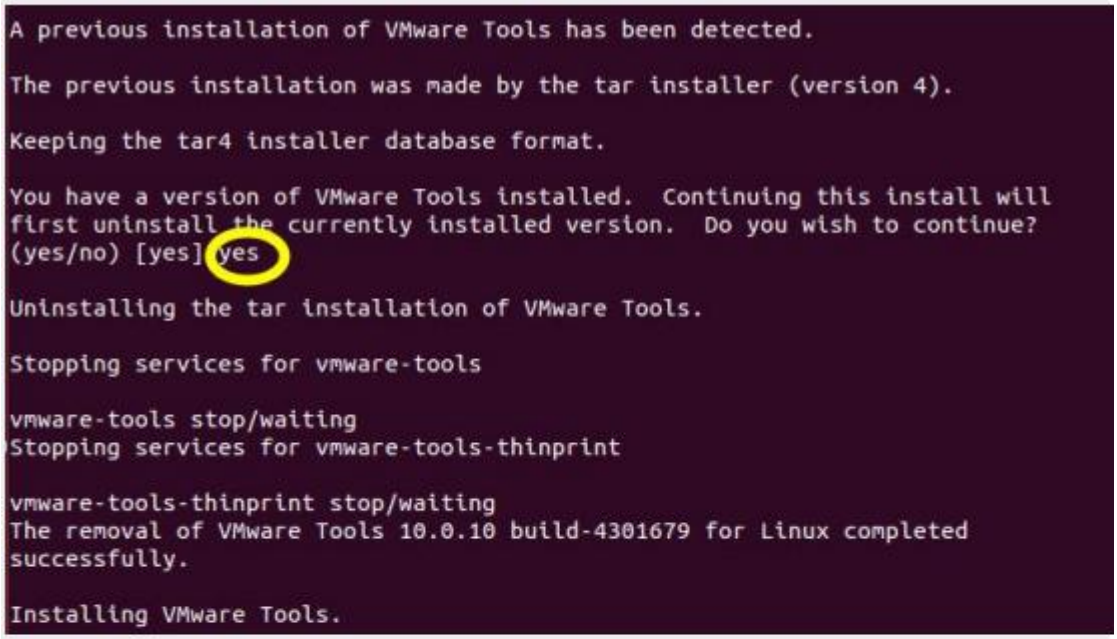
```
jason@ubuntu:~/Downloads$ sudo ./vmware-tools-distrib/vmware-install.pl
[sudo] password for jason:
```

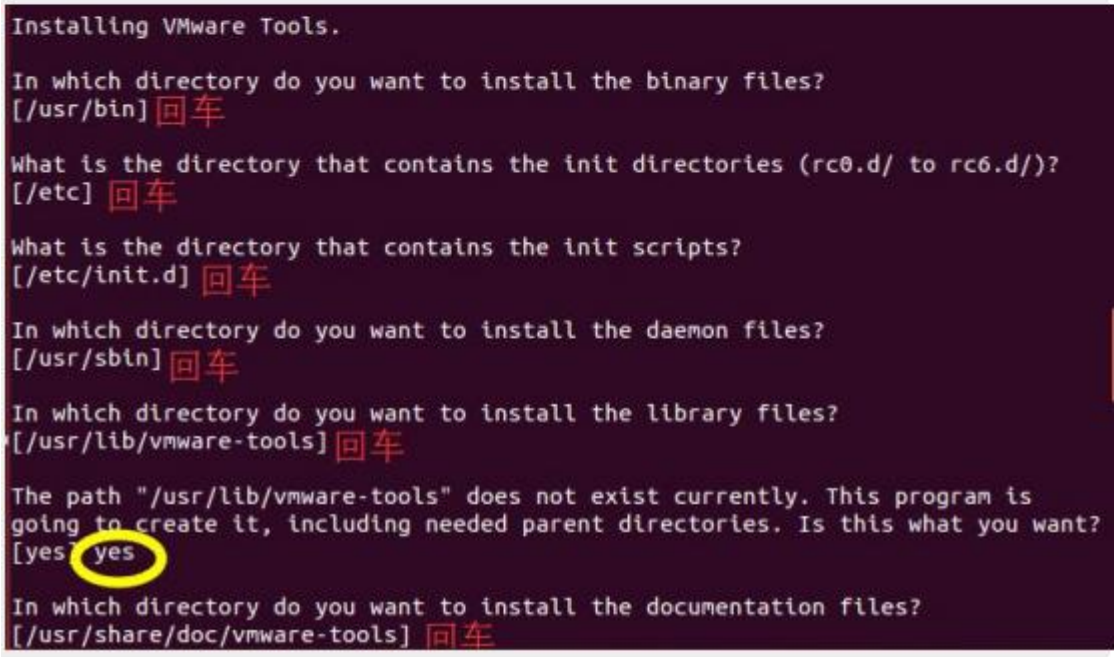
直接输入用户密码，输入过程中不会显示，输入完毕后按回车键即可。

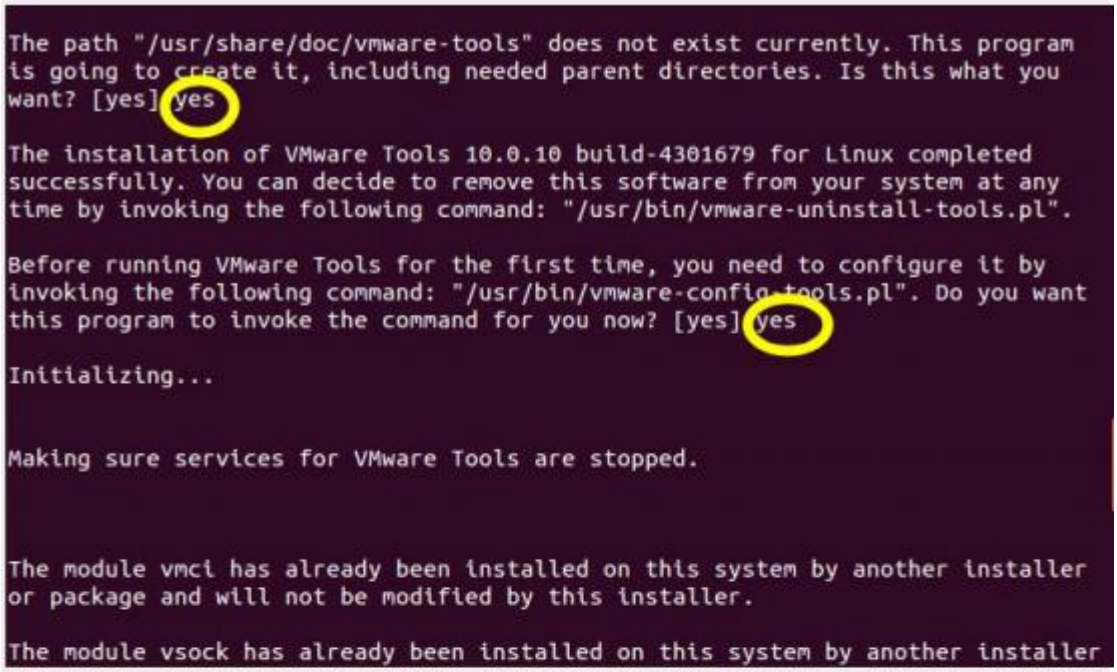
```
jason@ubuntu:~/Downloads$ sudo ./vmware-tools-distrib/vmware-install.pl
[sudo] password for jason:
The installer has detected an existing installation of open-vm-tools on this
system and will not attempt to remove and replace these user-space
applications. It is recommended to use the open-vm-tools packages provided by
the operating system. If you do not want to use the existing installation of
open-vm-tools and attempt to install VMware Tools, you must uninstall the
open-vm-tools packages and re-run this installer.
The packages that need to be removed are:
open-vm-tools
The installer will next check if there are any missing kernel drivers. Type yes
if you want to do this, otherwise type no [yes]
```

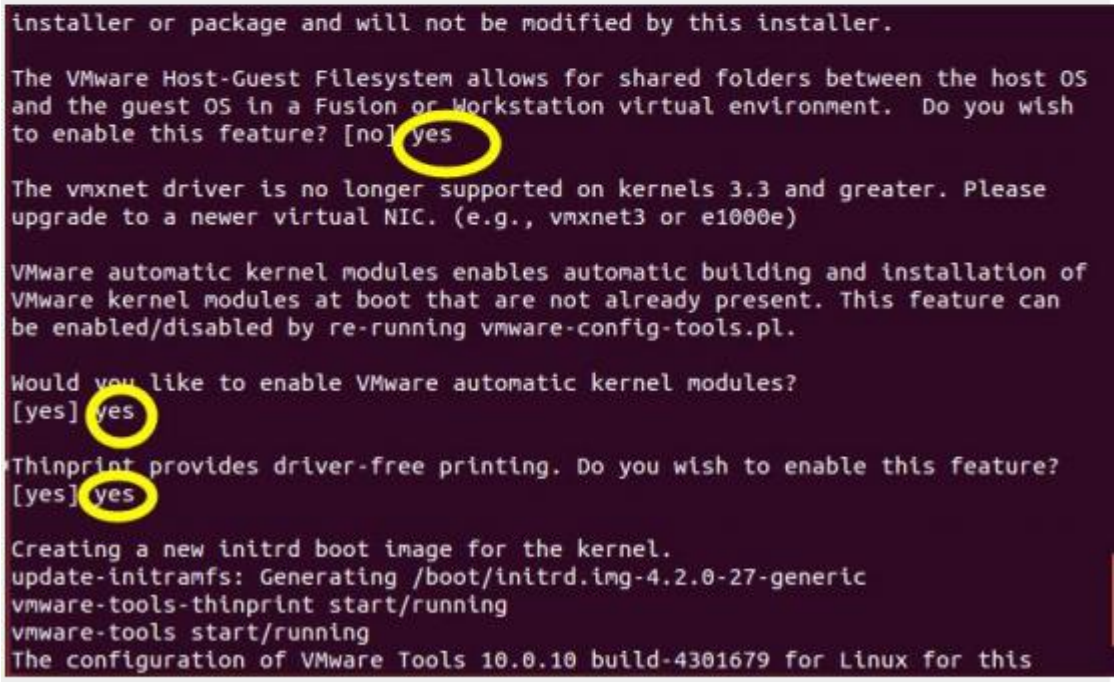
接下来，进入安装步骤，按照截图中的操作进行安装即可：

```
The installer has detected an existing installation of open-vm-tools on this
system and will not attempt to remove and replace these user-space
applications. It is recommended to use the open-vm-tools packages provided by
the operating system. If you do not want to use the existing installation of
open-vm-tools and attempt to install VMware Tools, you must uninstall the
open-vm-tools packages and re-run this installer.
The packages that need to be removed are:
open-vm-tools
The installer will next check if there are any missing kernel drivers. Type yes
if you want to do this, otherwise type no [yes] yes

For kernels >= 4.0.0, open-vm-tools need to be of version 10.0.0 or higher for
VMware Host Guest Filesystem (vmhgfs) to function with VMware Tools. Please
upgrade open-vm-tools and rerun this installer, or uninstall open-vm-tools.
Alternatively, you can disable using the vmhgfs when VMware Tools are
configured.

Enter yes to proceed. [no] yes
```


A previous installation of VMware Tools has been detected.
The previous installation was made by the tar installer (version 4).
Keeping the tar4 installer database format.
You have a version of VMware Tools installed. Continuing this install will
first uninstall the currently installed version. Do you wish to continue?
(yes/no) [yes] yes
Uninstalling the tar installation of VMware Tools.
Stopping services for vmware-tools
vmware-tools stop/waiting
Stopping services for vmware-tools-thinprint
vmware-tools-thinprint stop/waiting
The removal of VMware Tools 10.0.10 build-4301679 for Linux completed
successfully.
Installing VMware Tools.


Installing VMware Tools.
In which directory do you want to install the binary files?
[/usr/bin] 回车
What is the directory that contains the init directories (rc0.d/ to rc6.d/)?
[/etc] 回车
What is the directory that contains the init scripts?
[/etc/init.d] 回车
In which directory do you want to install the daemon files?
[/usr/sbin] 回车
In which directory do you want to install the library files?
[/usr/lib/vmware-tools] 回车
The path "/usr/lib/vmware-tools" does not exist currently. This program is
going to create it, including needed parent directories. Is this what you want?
[yes] yes
In which directory do you want to install the documentation files?
[/usr/share/doc/vmware-tools] 回车


The path "/usr/share/doc/vmware-tools" does not exist currently. This program
is going to create it, including needed parent directories. Is this what you
want? [yes] yes
The installation of VMware Tools 10.0.10 build-4301679 for Linux completed
successfully. You can decide to remove this software from your system at any
time by invoking the following command: "/usr/bin/vmware-uninstall-tools.pl".
Before running VMware Tools for the first time, you need to configure it by
invoking the following command: "/usr/bin/vmware-config-tools.pl". Do you want
this program to invoke the command for you now? [yes] yes
Initializing...
Making sure services for VMware Tools are stopped.
The module vmci has already been installed on this system by another installer
or package and will not be modified by this installer.
The module vsock has already been installed on this system by another installer


installer or package and will not be modified by this installer.
The VMware Host-Guest Filesystem allows for shared folders between the host OS
and the guest OS in a Fusion or Workstation virtual environment. Do you wish
to enable this feature? [no] yes
The vmxnet driver is no longer supported on kernels 3.3 and greater. Please
upgrade to a newer virtual NIC. (e.g., vmxnet3 or e1000e)
VMware automatic kernel modules enables automatic building and installation of
VMware kernel modules at boot that are not already present. This feature can
be enabled/disabled by re-running vmware-config-tools.pl.
Would you like to enable VMware automatic kernel modules?
[yes] yes
Thinprint provides driver-free printing. Do you wish to enable this feature?
[yes] yes
Creating a new initrd boot image for the kernel.
update-initramfs: Generating /boot/initrd.img-4.2.0-27-generic
vmware-tools-thinprint start/running
vmware-tools start/running
The configuration of VMware Tools 10.0.10 build-4301679 for Linux for this

The configuration of VMware Tools 10.0.10 build-4301679 for Linux for this
running kernel completed successfully.
Enjoy,
--the VMware team

到此，**VMware Tools** 便成功安装完成了。执行 **sudo reboot** 重启操作系统。

接下来，设置共享文件夹：

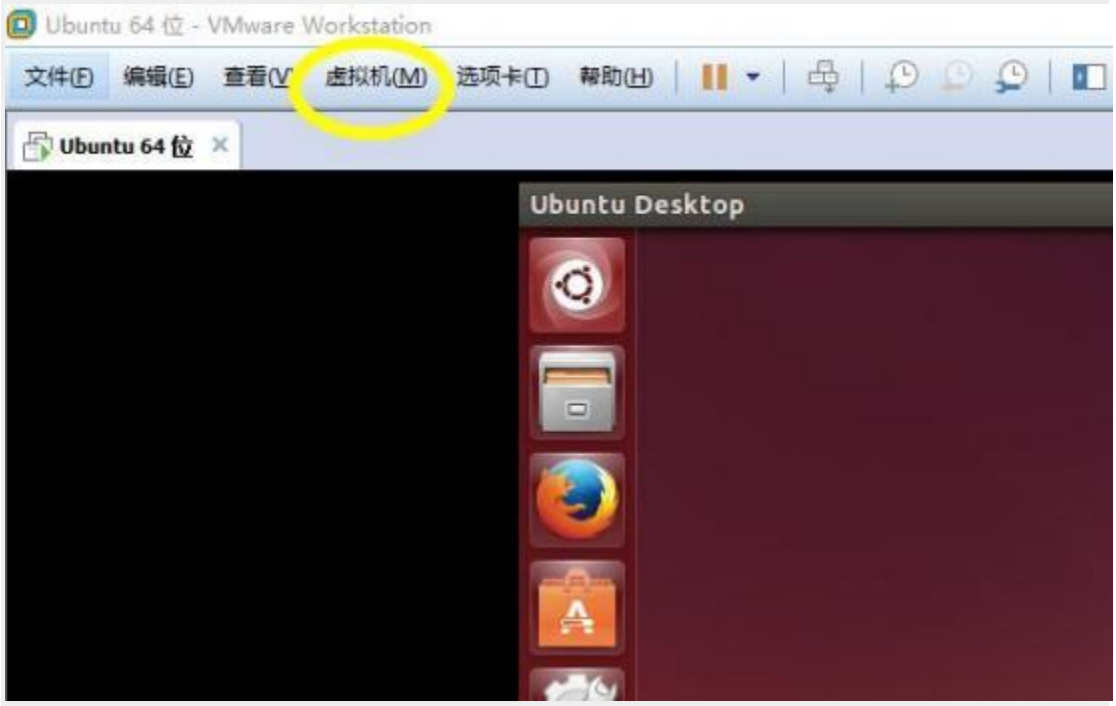
先在主机创建一个文件夹用以虚拟机与主机的文件共享，

我的文件夹路径为 **C:\开发工具**

\ubuntu-14.04.4-desktop-amd64\jasonshare

然后，按照下图进行操作：

选择"虚拟机"—>"设置"

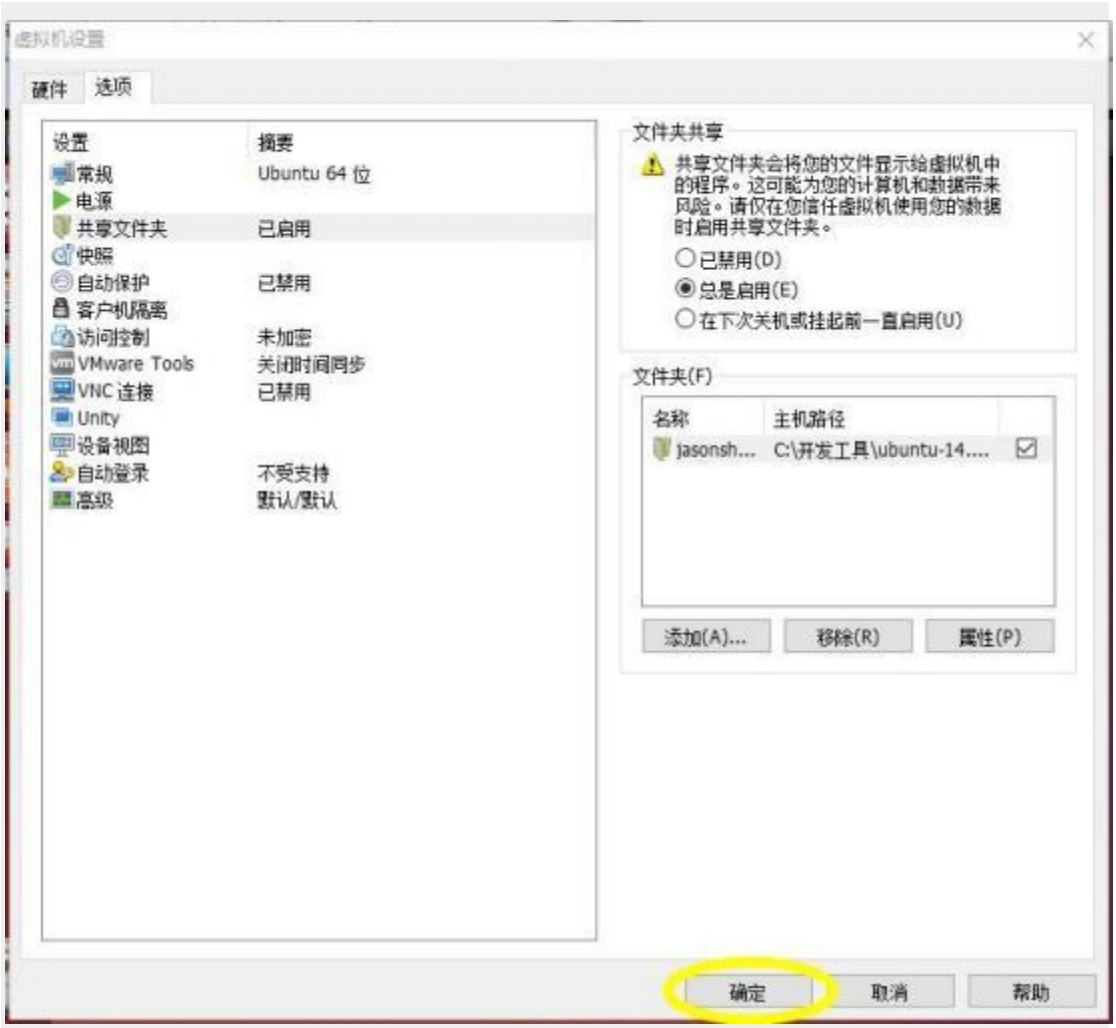


虚拟机设置
1
硬件
选项
设置
摘要
常规
Ubuntu 64 位
2
共享文件夹
已启用
快照
自动保护
已禁用
客户机隔离
访问控制
未加密
VMware Tools
关闭时间同步
VNC 连接
已禁用
Unity
设备视图
自动登录
不受支持
高级
默认/默认
文件夹共享
共享文件夹会将您的文件显示给虚拟机中的程序。这可能为您的计算机和数据带来风险。请仅在您信任虚拟机使用您的数据时启用共享文件夹。
3
已禁用(D)
总是启用(E)
文件夹(F)
名称
主机路径
4
添加(A)...
移除(R)
属性(P)
确定
取消
帮助

点击"浏览"，选择主机中共享文件夹的路径：

咸鱼:东

虚拟机设置
硬件
选项
设置
摘要
常规
Ubuntu 64 位
电源
共享文件夹
快照
自动保护
客户机隔离
访问控制
VMware Too
VNC 连接
Unity
设备视图
自动登录
高级
文件夹共享
共享文件夹会将您的文件显示给虚拟机中的程序。这可能为您的计算机和数据带来风险。请仅在您信任虚拟机使用您的数据
(U)
属性(P)
添加共享文件夹向导
指定共享文件夹属性
指定此共享文件夹的范围。
其他属性
启用此共享(E)
只读(R)
< 上一步(B)
完成
取消
确定
取消
帮助

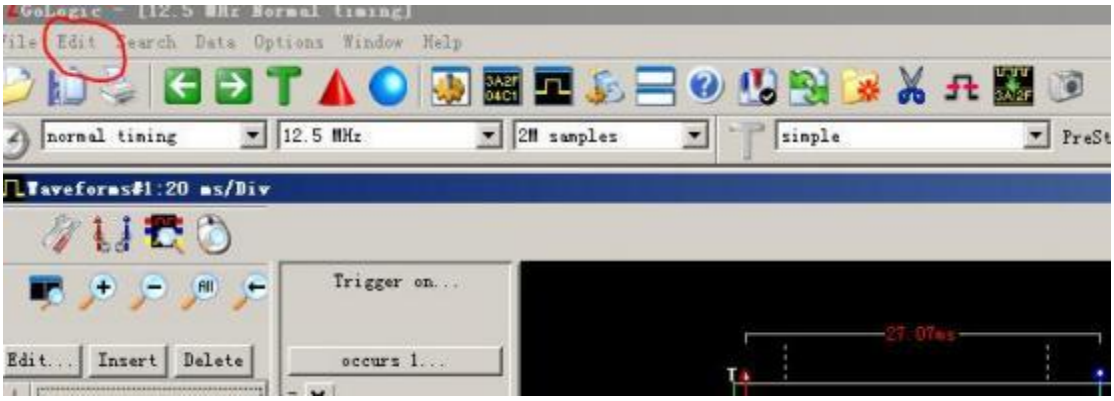


至此，共享文件夹便设置好了。
在虚拟机 ubuntu 中访问主机共享文件夹的方法:
在终端命令行输入 cd /mnt/ + 按下两次 Tab 键 ，回车
即可进入主机中的共享文件夹。

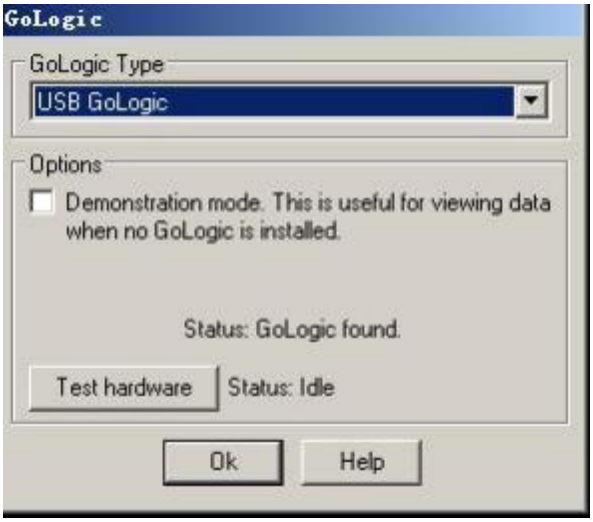
```
jason@ubuntu:~$ cd /mnt/hgfs/jasonshare/
jason@ubuntu:/mnt/hgfs/jasonshare$
```

逻辑分析仪简单使用

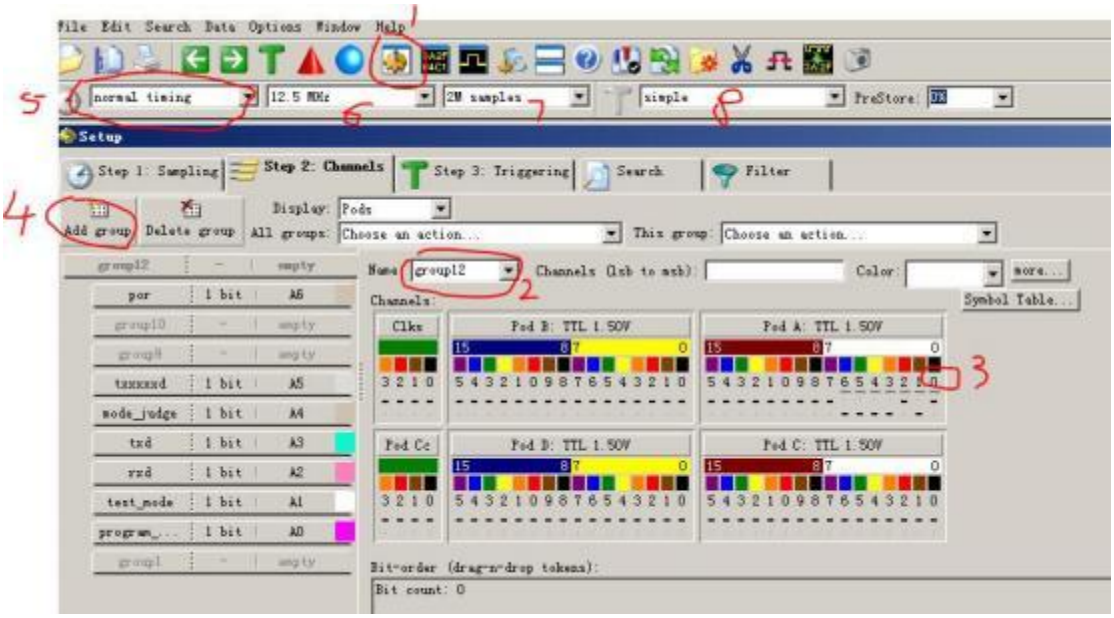
1. 打开逻辑分析仪软件
2. 打开 edit 开关如下图



进入hardware set…… 进入如下图,检测硬件

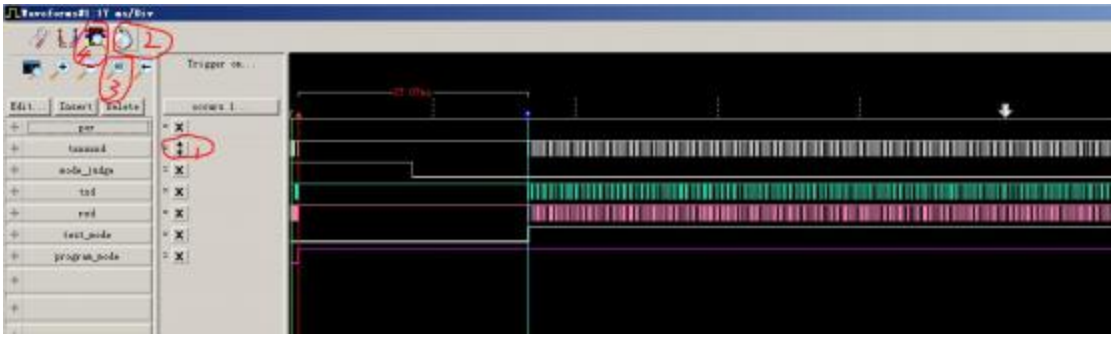


3. 点击 setup进入下图—下图红 1



红 2 表示修改监控信号/波形名字
红 3 选择逻辑分析对应的编号
红 4 添加监控信号/波形

红 5 表示正常时序
红 6 表示采样率
红 7 表示采样深度
红 8 表示采样格式
需要与硬件连接关系一致

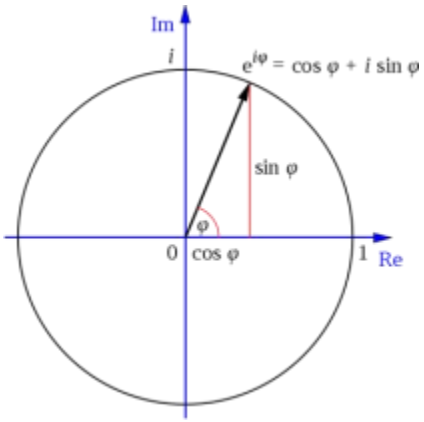
4. 

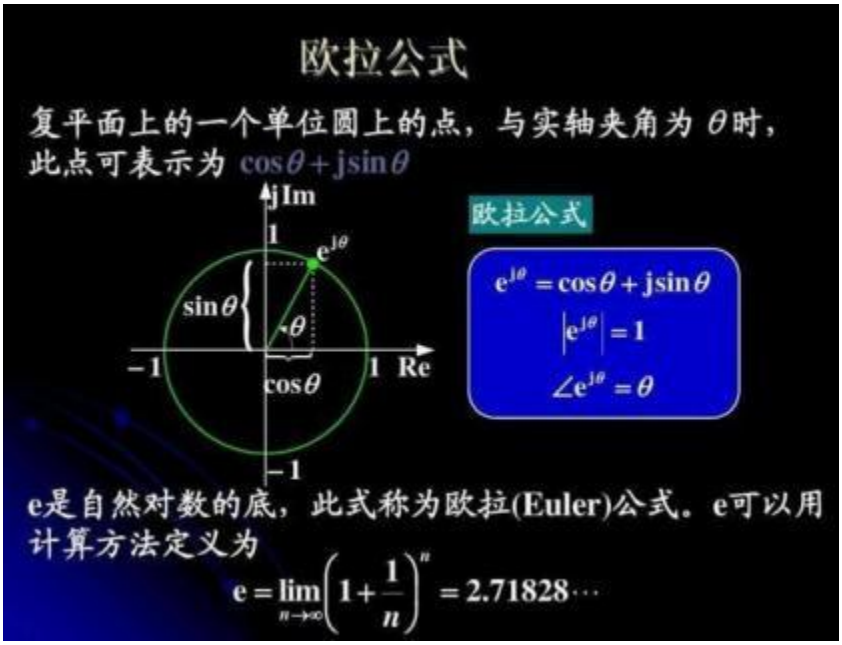
红 1 表示采样触发信号,点击设置
红 2 选择 drag zoom 可以像verdi 一样放大缩小,点击左右键可以测量时间
红 3 全局观察波形
红 4 可移动鼠标放大局部波形

常用电路原理及分析

数学公式

欧拉公式
$e^{ix}=\cos x+i\sin x$





离散傅里叶变换的公式：

$$X(k)=\sum_{n=0}^{N-1}x(n)W_N^{kn} \quad k=0,1,\ldots,N-1.$$

其中

$W_N = e^{-j\frac{2\pi}{N}}$ 为旋转因子

加法电路

```
    localparam A_WIDTH =  16;
    localparam B_WIDTH = 8;
  // Sumation result width should be 1 bit more than biggest widht of adder factor
    localparam C_WIDTH = if (A_WIDTH > B_WIDTH) ? A_WIDTH + 1'b1 : B_WIDTH + 1'b1;

   reg signed [A_WIDTH-1   : 0]       a;
   reg signed [B_WIDTH-1   : 0]       b;

    reg signed [C_WIDTH-1   : 0]       c;
  reg unsigned [C_WIDTH-1   : 0]       c_unsigned;

   always @(*) begin
         c =  {(C_WIDTH-A_WIDTH) {a[A_WIDTH-1]}},a} +
                    {{C_WIDTH-B_WIDTH{b[B_WIDITH-1]}},b};
    end
```

```
always @（*） begin
      c_unsigned =  {~c[C_WIDTH-1],c[C_WIDTH-2:0]};
end
```

仲裁电路

当几个模块同时操作一个模块时,则需要对这些请求进行仲裁排序,常见的就是 cpu 和 dma 同时访问 flash,ram 等 memory;
仲裁电路输入的特点为:

1. 每路请求至少有一个请求信号 req 和一个应答信号 ack;当无ack 应答时,req 一直存在,直到 ack 响应 req 才可以撤消---ack 相当于总线里面的 ready 信号
2. 必须安排优先级

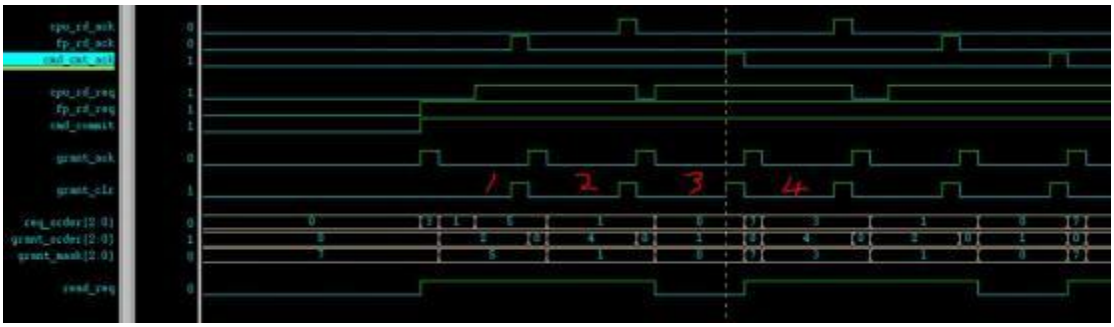
部分代码如下:
Req_order 为所有需要仲裁的输入信号排序,
grant_ack 是真正的请求信号为一个脉冲,也可以理解成同意了一个请求,开始执行对标的操作

Grant_order_nxt 是根据优先级产生的顺序—每次只能一个访问标的
Read_req 是给标的的控制信号,输出, 此信号为电平信号,当有响应时,才可能拉低(如果此时又来一个请求,则不会拉低);所以此处逻辑大致如下
If(req) read_req<=1’b1;else if(ack) read_req<=1’b0;

```
assign      req_order      = {cpu_dma_rd_req, ((fp_wr_req && ~hv_mask) || fp_rd_req), (cmd_commit && (fcmd[3] ? ~hv_mask : 1'b1))} & grant_mask;
assign      grant_order_nxt = req_order[2] ? 3'b100 : req_order[1] ? 3'b010 : req_order[0] ? 3'b001 : 3'b000;
assign      read_req       = (grant_order_nxt[2] && cpu_dma_rd_req)
                          || (grant_order_nxt[1] && fp_rd_req)
                          || (grant_order_nxt[0] && (cmd_commit && (fcmd == 0)));
```

```
always @ (posedge hclk or negedge por_n) begin
    if(~por_n)
        grant_order <= #`UDLY 3'b0;
    else if(grant_ack)
        grant_order <= #`UDLY grant_order_nxt;
    else if(grant_clr)
        grant_order <= #`UDLY 3'b0;
end


always @ (posedge hclk or negedge por_n) begin
    if(~por_n)
        grant_mask <= #`UDLY 3'b111;
    else if(grant_ack)
        grant_mask <= #`UDLY req_order[2] ? {1'b0, grant_mask[1:0]}
                           : req_order[1] ? {grant_mask[2], 1'b0, grant_mask[0]}
                           : req_order[0] ? {grant_mask[2:1], 1'b0}
                           : grant_mask;
    else if(grant_clr)
        grant_mask <= #`UDLY (req_order == 3'b0) ? 3'b111 : grant_mask;
        else if(state[ST_GOON])
        grant_mask <= #`UDLY {1'b1, grant_mask[1:0]};
end
```

grant_clr 为操作完一次标的,pluse 信号
grant_mask 被屏蔽的权限—下次可以执行的标号



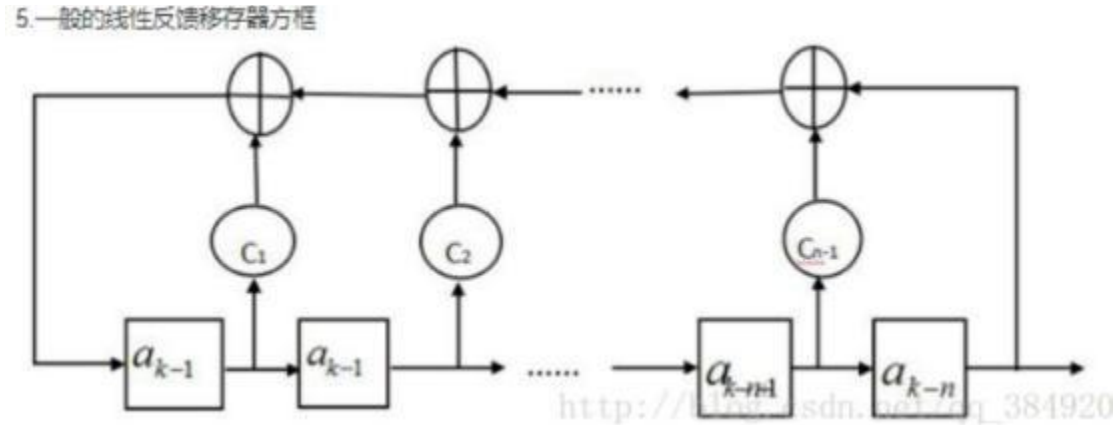
分析上述时序(cpu_rd_req 为 2 号请求,fp_rd_req 为 1 号请求,cmd_commit 为 0 号请求):
第 1 阶段:Fp_rd_req, cmd_commit 同时发生,产生一个 grant_ack,此时把 0,2 号的请求屏蔽 (1 号比 0 号优先级高) —也可以理解成后续可以执行 0 号和 2 号请求;执行 1 号请求,在执行过程中来了 2 号请求,忽略,继续执行完 1 号请求,返回 ack,
第 2 阶段:fp_rd_req 的请求执行完,产生一个新的grant_ack,执行在第 1 阶段的 2 号请求(虽然 0 号比 2 号早,但此时 2 号优先级高于 0 号),屏蔽 0 号请求,执行完 2 号请求,返回 ack
第 3 阶段,虽然 Fp_rd_req, cmd_commit 同时存在,且 2 号优先级高,但此时 0 号在上一个轮回还没执行过,所以必须执行 0 号请求,执行完后返回ack
第 4 阶段:重新下一轮优先级处理

上述方法可以避免某一个请求一直占用总线,在一个周期内,每个请求都有机会执行

伪随机电路

基础知识

伪随机序列包括　序列，　序列是目前　　系统中采用的最基本的　　序列。 是最长线性反馈移位寄存器序列的简称。同时　序列在通信方面有很重要的作用。　序列是由线性反馈移位寄存器产生的。



这是一个典型的线性反馈移位寄存器。通过选取系数的值，其中系数的值只能是 0 或者 1，来确定反馈路径。然后给定寄存器初始值，叫做种子值，然后就可以获得 2^n-1个状态。

常用的反馈系数:

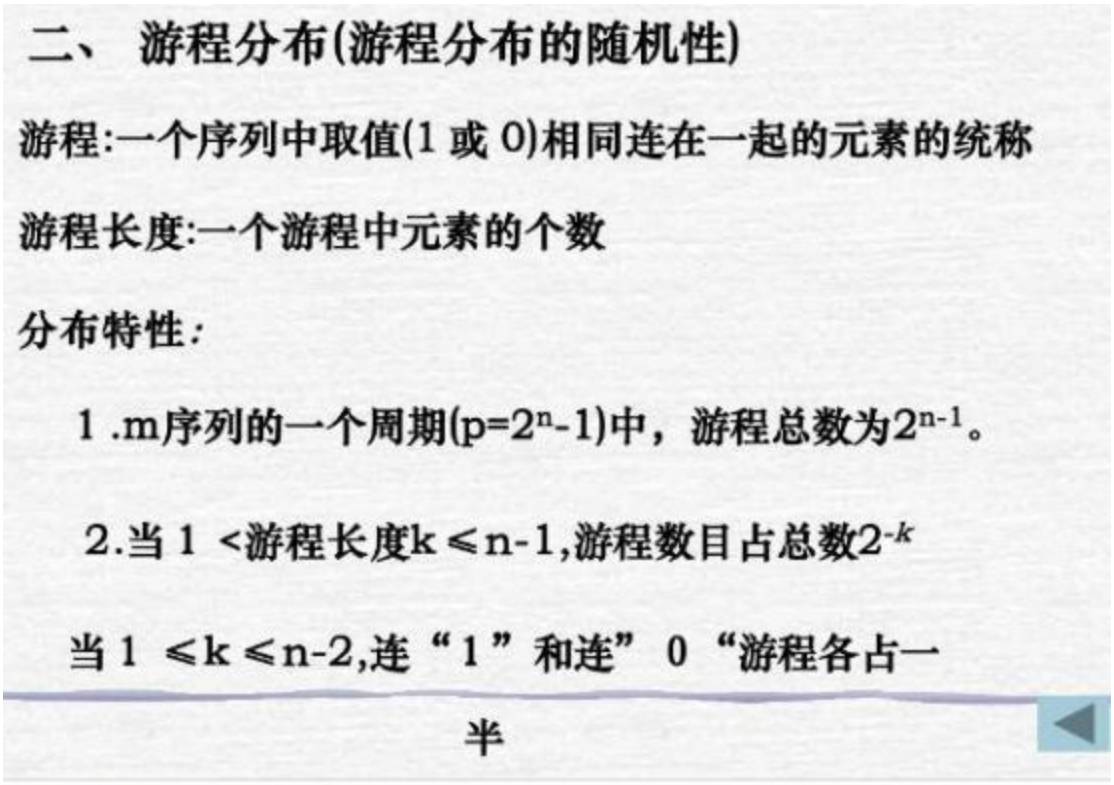
寄存器级数 r	m 序列长度	m 序列产生器反馈系数
3	7	13
4	15	23
5	31	45，67，75
6	63	103，147，155
7	127	203，211，217，235，277，313，325，345，367
8	255	435，453，537，543，545，551，703，747
9	511	1021，1055，1131，1157，1167，1175
10	1023	2011，2033，2157，2443，2745，3471
11	2047	4005，4445，5023，5263，6211，7363
12	4095	10123，11417，12515，13505，14127，15053
13	8191	20033，23261，24633，30741，32535，37505

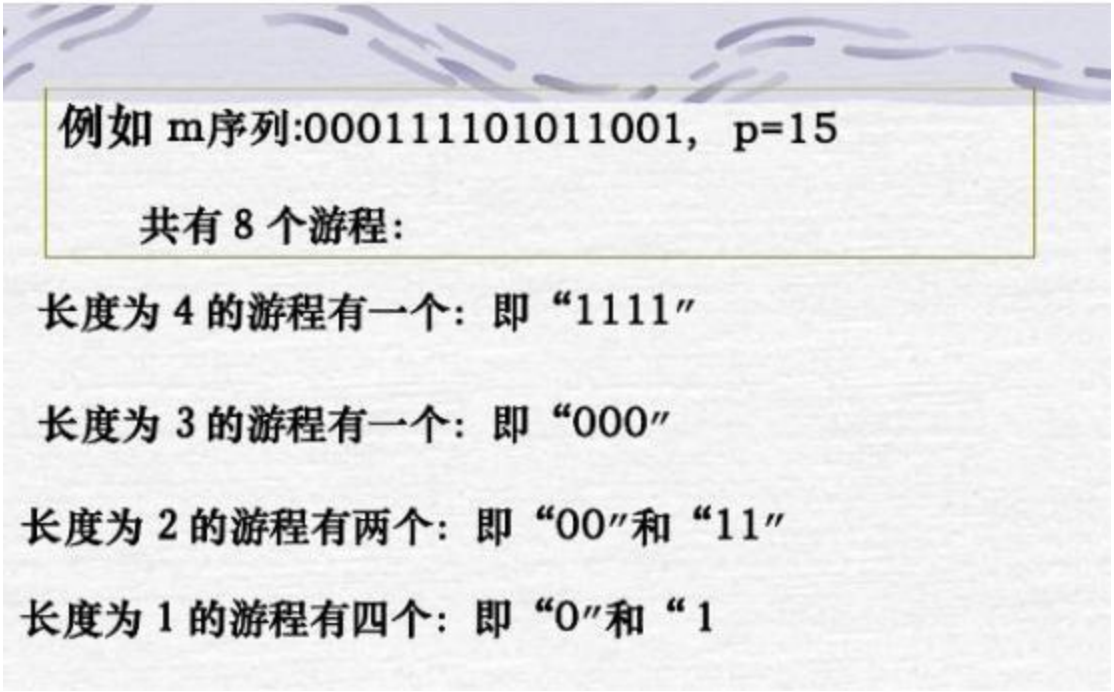
8.2伪随机序列的性质

一、均衡特性(平衡性)

m序列每一周期中在一个周期中 1 与 0 出现的次数基本相等。准确地说 1 的个数比 0 的个数多 1 个。

例如:p=15的m序列中,1的个数为 8,0 的个数为 7。当p足够大时，在一个周期中 1 与 0 出现的次数基本相等。





设计源码

```
module rand_lfs(
    clk_sys,
    rand_clk, //osc800k
```

```
    rst,
    xsel2,
    xadr2,
    xwdata2,
    xrdata2,
    enable,
    out_rand
);
input clk_sys;
input rand_clk;
input rst;
input xsel2;
input[7:0]  xadr2;
input[7:0]  xwdata2;
output[7:0] xrdata2;

input enable;
output out_rand;

parameter ADDR_SEEDL = 3'h0 ;
parameter ADDR_SEEDH = 3'h1 ;
parameter ADDR_FEEDL = 3'h2 ;
parameter ADDR_FEEDH = 3'h3 ;

reg[11:0] register_lfs ;
wire next_register_lfs_0 ;
reg[7:0] seedl ;
reg[3:0] seedh ;
reg[7:0] feedl ;
reg[3:0] feedh ;

wire temp0;
wire temp1;
wire temp2;
wire temp3;
wire temp4;
wire temp5;
wire temp6;
wire temp7;
wire temp8;
wire temp9;
wire temp10;
wire temp11;
```

```
wire seedl_sel = xsel2 & (xadr2[2:0] == ADDR_SEEDL);
wire seedh_sel = xsel2 & (xadr2[2:0] == ADDR_SEEDH);
wire feedl_sel = xsel2 & (xadr2[2:0] == ADDR_FEEDL);
wire feedh_sel = xsel2 & (xadr2[2:0] == ADDR_FEEDH);

wire [7:0] next_seedl = seedl_sel ? xwdata2 : seedl ;
wire [3:0] next_seedh = seedh_sel ? xwdata2 : seedh ;
wire [7:0] next_feedl = feedl_sel ? xwdata2 : feedl ;
wire [3:0] next_feedh = feedh_sel ? xwdata2 : feedh ;
wire [11:0] seed = {seedh,seedl};
wire [11:0] feed = {feedh,feedl};

assign xrdata2 =  seedl_sel ? seedl :
                  seedh_sel ? {4'h0,seedh}:
                  feedl_sel ? feedl :
                  feedh_sel ? {4'h0,feedh} :8'h0;
//--------------------------------
//feed is list :bit[12] must 1
//    feed_data[0]= 13'o10123; => 12'o0123; => 12'h53 ;
//    feed_data[1]= 13'o11417; => 12'o1417; => 12'h30f
//    feed_data[2]= 13'o12515; => 12'o2515; => 12/h54d
//    feed_data[3]= 13'o13505; => 12'o3505; => 12/h745  //default
//    feed_data[4]= 13'o14127; => 12'o4127; => 12/h857
//    feed_data[5]= 13'o15053; => 12'o5053; => 12/ha2b
//--------------------------------
always @(posedge clk_sys or negedge rst)
    if(!rst) begin
        seedl <= 8'h55;
        seedh <= 4'h5;
        feedl <= 8'h45;
        feedh <= 4'h7;
    end
    else begin
        seedl <= next_seedl;
        seedh <= next_seedh;
        feedl <= next_feedl;
        feedh <= next_feedh;
    end

sync_level uSyn_enable(
    .clk            ( rand_clk     ),
    .clk_rst_n      ( rst     ),
    .async_sig      ( enable     ),
    .sync_sig       ( sync_enable)
```

```
);

assign next_register_lfs_0 =  feed[0] ? temp0 :1'b0;   //feed[0] must 1
assign temp0 = feed[1]  ? temp1  ^ register_lfs[0]  : temp1;
assign temp1 = feed[2]  ? temp2  ^ register_lfs[1]  : temp2;
assign temp2 = feed[3]  ? temp3  ^ register_lfs[2]  : temp3;
assign temp3 = feed[4]  ? temp4  ^ register_lfs[3]  : temp4;
assign temp4 = feed[5]  ? temp5  ^ register_lfs[4]  : temp5;
assign temp5 = feed[6]  ? temp6  ^ register_lfs[5]  : temp6;
assign temp6 = feed[7]  ? temp7  ^ register_lfs[6]  : temp7;
assign temp7 = feed[8]  ? temp8  ^ register_lfs[7]  : temp8;
assign temp8 = feed[9]  ? temp9  ^ register_lfs[8]  : temp9;
assign temp9 = feed[10] ? temp10 ^ register_lfs[9]  : temp10;
assign temp10= feed[11] ? temp11 ^ register_lfs[10] : temp11;
assign temp11=register_lfs[11]; //feed[12] must 1
//assign temp11= feed[12] ?register_lfs[11]:1'b0; //feed[12] must 1
assign out_rand = register_lfs[11] ;

always @(posedge rand_clk or negedge rst)
    if(!rst)begin
        register_lfs <= 12'b0;
    end
    else if(!sync_enable) begin
        register_lfs <= seed;
    end
    else begin
        register_lfs <= {register_lfs[10:0],next_register_lfs_0};
    end
endmodule
```

Crc 电路

一 LFSR

线性反馈移位寄存器(LFSR)是内测试电路中最基本的标准模块结构，既用作伪随机测试码产生器，也作为压缩测试结果数据的特征分析器。

一个 n 阶的 LFSR 由 n 个触发器和若干个异或门组成。在实际应用当中，主要用到两种类型的 LFSR，即异或门外接线性反馈移位寄存器（ IE 型 LFSR，图 1）**一用于伪随机数发生器**和异或门内接线性反馈移位寄存器（EE 型 LFSR，图 2）**一用于 crc 效验电路**。其中 g_0 g_1g_2 g_n为’ 0’ 或’ 1’ ，Q_1 Q_2Q_3 Q_n为 LFSR 的输出，M(x)是输入的码字多项式，如 $M(x)=x^4+ x^1+1$，表示输入端的输入顺序为 11001，同样，LFSR 的结构也可以表示为多项式 G(x)，称为生成多项式：

$$G(x)=g_n*x^n+ \cdots+g_1*x^1+g_0;$$

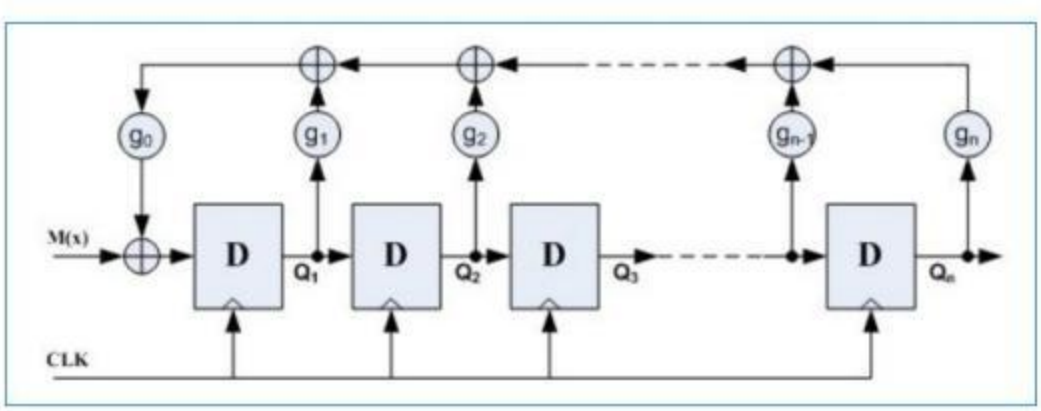


图1 IE型LFSR

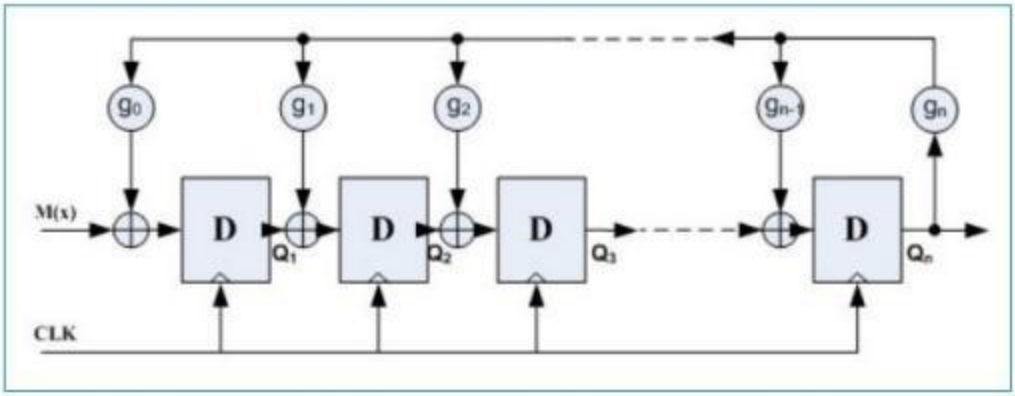


图2 EE型LFSR

二 CRC（*参考博文http:/blog.sina.com.cn/s/blog_468e65190100cxq3.html*）

循环冗余校验码（CRC）的基本原理是：在 K 位信息码后再拼接 R 位的校验码，整个编码长度为N 位，因此，这种编码又叫（N，K）码。对于一个给定的（N，K）码，可以证明存在一个最高次幂为 N-K=R 的多项式 G(x)可以使整个编码被除余数为 0。根据 G(x)可以生成 K 位信息的校验码，而 G(x)叫做这个 CRC 码的生成多项式。校验码的具体生成过程为：假设发送信息用信息多项式 C(X)表示，将 C(x)左移 R 位，则可表示成 C(x)*2 的 R 次方，这样 C(x)的右边就会空出R 位，这就是校验码的位置。通过 C(x)*2 的 R 次方除以生成多项式 G(x)得到的余数就是校验码。

通过 CRC 的生成原理知道 CRC 的检验码生成是通过除法得到，由此联想到可以通过 LFSR 来产生校验码。

假设原信息码子多项式为

$$M(x) = m_{k-1}x^{k-1} + m_{k-2}x^{k-2} + \cdots + m_1 x + m_0$$

生成多项式为

$$G(x) = g_{r-1}x^{r-1} + g_{r-2}x^{r-2} + \cdots g_1 x + g_0$$

那么CRC的码字为 $\frac{M(x)x^r}{G(x)}$ ，使用用LFSR电路来进行实现，将M(x)向左移r位在电路中的意义即为输入完信息码后再输入r个0，所以在电路上的表现就如图5所示。

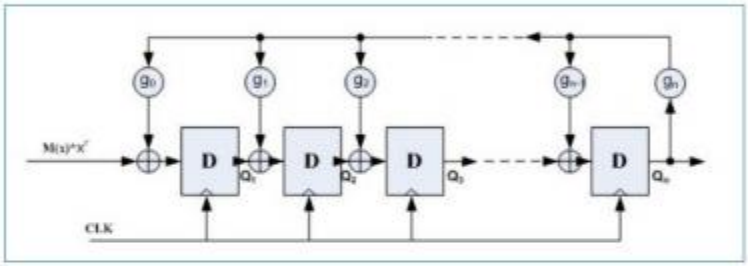


图5 使用LFSR来产生CRE校验码

将这个时刻产生的寄存器输入添加到原信息码的后边就进行完了CRC编码，同样接收端可以使用LFSR来进行CRC检验。

下表中列出了一些见于标准的CRC资料：

名称	生成多项式	简记式*	应用举例
CRC-4	x^4+x+1		ITU G.704
CRC-12	$x^{12}+x^{11}+x^3+x+1$		
CRC-16	$x^{16}+x^{12}+x^2+1$	1005	IBM SDLC
CRC-ITU**	$x^{16}+x^{12}+x^5+1$	1021	ISO HDLC, ITU X.25, V.34/V.41/V.42, PPP-FCS
CRC-32	$x^{32}+x^{26}+x^{23}+\ldots+x^2+x+1$	04C11DB7	ZIP, RAR, IEEE 802 LAN/FDDI, IEEE 1394, PPP-FCS
CRC-32c	$x^{32}+x^{28}+x^{27}+\ldots+x^8+x^6+1$	1EDC6F41	SCTP

* 生成多项式的最高幂次项系数是固定的1，故在简记式中，将最高的1统一去掉了，如04C11DB7实际上是104C11DB7。
** 前称CRC-CCITT。ITU的前身是CCITT。

```
wire [15:0] next_crc16_result  = crc16_start ? 16'hffff :
                                 crc_en ? crc_tmp : crc16_result;

always @(posedge clk r or negedge resetn)
    if(~resetn) begin
```

```
        crc_en          <= 1'b0;
        crc16_result    <= 16'hffff;
    end
    else begin
        crc_en          <= next_crc_en;
        crc16_result    <= next_crc16_result;
    end
integer i;
always@(data_in or crc16_result) begin
    crc_tmp = crc16_result;
    for(i=15;i>=0;i=i-1) begin
        tmp = data_in[i] ^ crc_tmp[15];
        crc_tmp[15] = crc_tmp[14];
        crc_tmp[14] = crc_tmp[13];
        crc_tmp[13] = crc_tmp[12];
        crc_tmp[12] = crc_tmp[11] ^ tmp;
        crc_tmp[11] = crc_tmp[10];
        crc_tmp[10] = crc_tmp[9];
        crc_tmp[9] = crc_tmp[8];
        crc_tmp[8] = crc_tmp[7];
        crc_tmp[7] = crc_tmp[6];
        crc_tmp[6] = crc_tmp[5];
        crc_tmp[5] = crc_tmp[4] ^ tmp;
        crc_tmp[4] = crc_tmp[3];
        crc_tmp[3] = crc_tmp[2];
        crc_tmp[2] = crc_tmp[1];
        crc_tmp[1] = crc_tmp[0];
        crc_tmp[0] = tmp;
    end
end
```

cordic 算法详解

cordic 原理转发

转载自小一休哥的文章：http://blog.csdn.net/qq_39210023/article/details/77456031

目前，学习与开发 FPGA 的程序员们大多使用的是 Verilog HDL 语言（以下简称为 Verilog），关于Verilog 的诸多优点一休哥就不多介绍了，在此，我们将重点放在Verilog 的运算操作上。

我们都知道，在 Verilog 中，运算一般分为逻辑运算（与或非等）与算术运算（加减乘除等）。而在一开始学习Verilog 时，老司机一定会提醒我们，“切记，千万别用‘/’除、‘%’取模（有的也叫取余）和‘**’幂。”这话说的不无道理，因为这三个运算是不可综合的。但，需清楚理解的是，不可综合的具体意思为不能综合为简单的模块，当我们在程序中调用了这些运算时，‘/’除和‘%’取模在Quartus 软件

中是可以综合的，因此可以正常调用运行，但是会消耗一些逻辑资源，而且会产生延时，即这两个运算的处理时间会很长，可能会大于时序控制时钟的单周期时间。此时呢，我们会建议你调用 IP 核来实现运算操作，虽然这样也会消耗许多逻辑资源，但产生的延时相对较小满足了你基本的需求。

问题好像迎刃而解了，可是仔细一想，除了这些运算，我们还剩下什么？对呀，三角函数，反三角函数，对数函数，指数函数呢，这些函数我们在高中就学习了的呀，难道在 FPGA 中就没有用武之地吗？有人会说，查找表呗，首先将某个运算的所有可能的输入与输出对一一罗列出来，然后放进 Rom 中，然后根据输入查表得到输出。这个方法虽然有效的避免了延时问题，却是一个十分消耗资源的方法，不适合资源紧张的设计。那么，就真的没有办法了吗？

答案就是咱们今天的标题了，CORD IC，而且 CORD IC 是一个比较全能的算法，通过这一原理，我们可以实现三角函数，反三角函数，对数函数，指数函数等多种运算。接下来，一休哥就带领大家来学习CORD IC 的原理吧。（题外话：请相信一休哥，本文不会让你感到太多痛苦~）

本文将分三个小部分来展开介绍：

1、CORD IC 的基本原理介绍

2、CORD IC 的具体操作流程介绍

3、CORD IC 的旋转模式——Veri log 仿真

本文涉及到的全部资料链接：

链接：http://pan.baidu.com/s/1gfrJzMj 密码：x92u

一、CORD IC 的基本原理介绍

CORD IC 算法是一个“化繁为简”的算法，将许多复杂的运算转化为一种“仅需要移位和加法”的迭代操作。CORD IC 算法有旋转和向量两个模式，分别可以在圆坐标系、线性坐标系和双曲线坐标系使用，从而可以演算出8 种运算，而结合这 8 种运算也可以衍生出其他许多运算。下表展示了8 种运算在 CORD IC 算法中实现的条件。

	旋转模式	向量模式
圆坐标系	cos & sin	$\tan^{-1}$
线性坐标系	*	/
双曲线坐标系	cosh & sinh	$\tanh^{-1}$

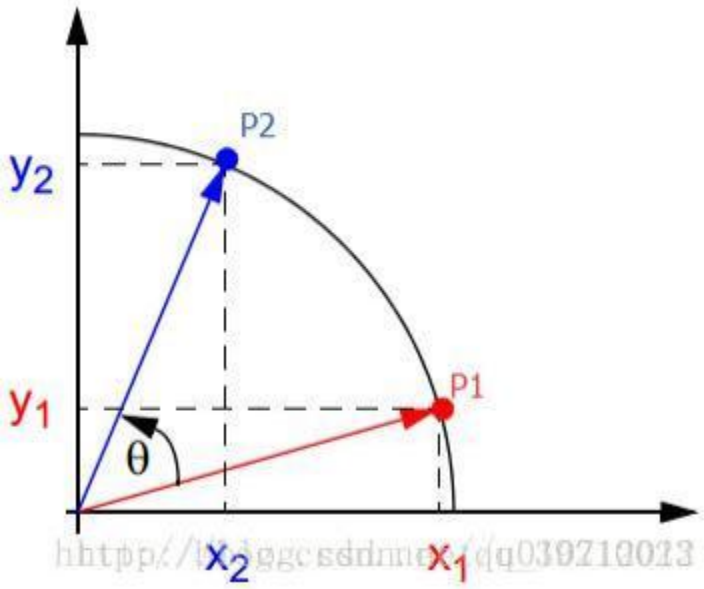
这里写图片描述

首先，我们先从旋转模式下的圆坐标系讲起，这也是 CORD IC 方法最初的用途。

1、CORD IC 的几何原理介绍

假设在 xy 坐标系中有一个点 P1（x1，y1），将 P1 点绕原点旋转 θ 角后得到点 P2（x2，y2）。

这里写图片描述



于是可以得到 P1 和 P2 的关系。

x2 = x1cos θ - y1sin θ = cos θ(x1 - y1tan θ)

y2 = y1cos θ + x1sin θ = cos θ(y1 +x1tan θ)

以上就是CORD IC 的几何原理部分，而我们该如何深入理解这个几何原理的真正含义呢？

从原理中，我们可以知道，当已知一个点 P1 的坐标，并已知该点 P1 旋转的角度 θ， 则可以根据上述公式求得目标点 P2 的坐标。然后，麻烦来了，我们需要用 FPGA 去执行上述运算操作，而 FPGA 的 Verilog 语言根本不支持三角函数运算。因此，我们需要对上述式子进行简化操作，将复杂的运算操作转换为一种单一的“迭代位移”算法。那么，接下来我们介绍优化算法部分。

2、CORD IC 的优化算法介绍

我们先介绍算法的优化原理：将旋转角 θ细化成若干分固定大小的角度 θi，并且规定 θi 满足tan θi=2-i，因此Σ θi 的值在[-99.7°， 99.7°]范围内，如果旋转角 θ超出此范围，则运用简单的三角运算操作即可（加减 π）。

然后我们需要修改几何原理部分的假设，假设在 xy 坐标系中有一个点 P0（x0，y0），将 P0 点绕原点旋转 θ 角后得到点 Pn（xn，yn）。

于是可以得到 P0 和 Pn 的关系。

xn = x0cos θ - y0sin θ = cos θ(x0 - y0tan θ)

yn = y0cos θ + x0sin θ = cos θ(y0 + x0tan θ)

然后，我们将旋转角 θ细化成 θi，由于每次的旋转角度 θi 是固定不变的（因为满足tan θi=2-i），如果一直朝着一个方向旋转则Σ θi 一定会超过 θ（如果 θ在[-99.7°， 99.7°]范围内）。因此我们需要对 θi 设定一个方向值di。如果旋转角已经大于 θ， 则 di 为-1，表示下次旋转为顺时针，即向 θ靠近；如果旋转角已经小于 θ， 则 di 为 1，表示下次旋转为逆时针，即也向 θ靠近。然后我们可以得到每次旋转的角度值 di θi，设角度剩余值为 zi+1，则有 zi+1 = zi - di θi，其中 z0 为 θ。如此随着 i 的增大，角度剩余值 zi+1将会趋近于 0，此时运算结束。（注：可以发现，di 与 zi 的符号位相同）

第一次旋转 θ0，d0 为旋转方向：

x1 = cos θ0(x0 - d0y0tan θ0)

y1 = cos θ0(y0 + d0x0tan θ0)

第二次旋转 θ1，d1 为旋转方向：

x2 = cos θ1(x1 - d1y1tan θ1) = cos θ1cos θ0(x0 - d0y0tan θ0 - d1y0tan θ1 - d1d0 x0tan θ1 tan θ0)

y2 = cos θ1 (y1 + d1x1tan θ1) = cos θ1cos θ0(y0 + d0x0tan θ0 + d1x0tan θ1 - d1d0y0tan θ1 tan θ0)

大家可能已经发现了，在我们旋转的过程中，式子里一直会有tan θ i 和 cos θ i，而每次都可以提取出cos θ i。虽然我们的 FPGA 无法计算 tan θ i，但我们知道 tan θ i = 2-i，因此可以执行和 tan θ i 效果相同的移位操作 2-i 来取代tan θ i。而对于cos θ i，我们可以事先全部提取出来，然后等待迭代结束之后（角度剩余值 zi+1趋近于 0，一般当系统设置最大迭代次数为 16 时 zi+1 已经很小了），然后计算出∏cos θ i 的值即可。

总结一下：

迭代公式有三：

xi+1 = xi - (d i) (y i) (2^-i)，提取了 cos θ i，2-i 等效替换了 tan θ i 之后

y i+1 = y i + (d i) (x i) (2^-i)，提取了 cos θ i，2-i 等效替换了 tan θ i 之后

zi+1 = zi - di θ i

其中 i 从 0 开始迭代，假设当 i = n-1 时，zn 趋近于 0，迭代结束。然后对结果乘上∏cos θ i（ i 从 0 至 n-1），于是得到点 Pn（xn∏cos θ i，yn∏cos θ i），此时的点 Pn 就近似等于之前假设中的点 Pn（xn，yn）了，所以此时的点 Pn 同样满足之前假设得到的公式：

xn∏cos θ i = x0cos θ - y0s in θ

yn∏cos θ i = y0cos θ + x0s in θ

由于 i 从 0 至 n-1，所以上式可以转化成下式：

xn = 1/∏cos θ i (x0cos θ - y0s in θ)，（其中 i 从 0 至 n-1）

yn = 1/∏cos θ i (y0cos θ + x0s in θ)，（其中 i 从 0 至 n-1）

注意：上式中的xn，yn 是经过迭代后的结果，而不是之前假设中的点 Pn（xn，yn）。了解这点是十分重要的。

根据高中学的三角函数关系，可以知道 cos θ i = 1/[(1+tan2 θ i)^0.5] = 1/[(1+2-2i)^0.5]，而 1/[(1+2-2i)^0.5]的极值为 1，因此我们可以得出一个结论：当 i 的次数很大时，∏cos θ i 的值趋于一个常数。

关于如何计算∏cos θ i 的代码如下所示：

```
% 初始化
    16 %迭代次数
    zeros   1 %每次旋转的角度
  _     zeros   1 %每次旋转的角度的余弦值
   zeros   1 %余弦值的N元乘积
 _1  zeros   1 %余弦值的N元乘积的倒数
for i  1:
      2 - i-1
       i  atan
      _    i  cos    i
    if i   1
         i     _    i
        _1 i   1/  i
    else
         i     i-1    _    i
```

_1 i 1/ i

e

n

d

e

n

d

2 256 10

_ _ 10

10

_1 _1 10

这里写图片描述

i	θi（arctan（2^{-i}），单位°）	cosθi	∏cosθi	1/∏cosθi
0	45.0	0.7071067812	0.7071067812	1.414213562
1	26.56505118	0.894427191	0.632455532	1.58113883
2	14.03624347	0.9701425001	0.6135719911	1.629800601
3	7.125016349	0.9922778767	0.6088339125	1.642484066
4	3.576334375	0.9980525785	0.6076482563	1.645688916
5	1.789910608	0.9995120761	0.6073517701	1.646492279
6	0.8951737102	0.999877952	0.6072776441	1.646693254
7	0.4476141709	0.9999694838	0.6072591123	1.646743507
8	0.2238105004	0.9999923707	0.6072544793	1.64675607
9	0.1119056771	0.9999980927	0.6072533211	1.646759211
10	0.05595289189	0.9999995232	0.6072530315	1.646759996
11	0.02797645262	0.9999998808	0.6072529591	1.646760193
12	0.01398822714	0.9999999702	0.607252941	1.646760242
13	0.006994113675	0.9999999925	0.6072529365	1.646760254
14	0.003497056851	0.9999999981	0.6072529354	1.646760257
15	0.001748528427	0.9999999995	0.6072529351	1.646760258

从上表也可以看出，当迭代次数为 16，i=15 时，cos θ i 的值已经非常趋近于 1 了，∏cos θ i 的值则约等 于 0.607253，1/∏cos θ i 为 1.64676。所以当迭代次数等于 16 时，通过迭代得到的点 Pn 坐标已经非常接 近之前假设中的点 Pn。所以，当迭代次数等于 16 时，这个式子是成立的。

xn = 1/∏cos θ i（x0cos θ - y0sin θ），（其中 i 从 0 至 n-1） yn = 1/∏cos θ i（y0cos θ + x0sin θ），（其中 i 从 0 至 n-1）

此时，已知条件有三个 x0、y0 和 θ 。通过 16 次迭代，我们可以得到 xn 和 yn。而式中的∏cos θ i 是个随 i 变化的值，我们可以预先将其值存入系统中。

然后，我们人为设置x0 = ∏cos θ i，y0 = 0，则根据等式，xn = cos θ， yn = sin θ 。其中1/∏cos θ i 的值我们也同样预先存入系统中。如此，我们就实现了正弦和余弦操作了。

带反正切函数的部分设计

下面是带反正切函数的部分设计,正切为正数，负数可以根据基本函数关系变换到正数。

```
module Cordic
(
    clk,
    rst_n,
    Phase,
    enable,
    angletan, //1:反正切算法 0：正余弦函数
    tan_value,//正切值
    Sin,
    Cos,
    Error   //当为反正切算法时，此内容为对应的角度
);

input                       clk;
input                       rst_n;
```

```
input                       enable;
input                       angletan;
input       [31:0]          tan_value;
input       [31:0]          Phase;
output      [31:0]          Sin;
output      [31:0]          Cos;
output      [31:0]          Error;

`define rot0  32'd2949120       //45'C *2^16          tanx0=1        =>
x=45c
`define rot1  32'd1740992       //26.5651'C *2^16     tanx1>>1     =>
x=26.5651 c
`define rot2  32'd919872        //14.0362'C *2^16     tanx2=1>>2     =>
x=14.0362 c
`define rot3  32'd466944        //7.1250'C *2^16
`define rot4  32'd234368        //3.5763'C *2^16
`define rot5  32'd117312        //1.7899'C *2^16
`define rot6  32'd58688         //0.8952'C *2^16
`define rot7  32'd29312         //0.4476'C *2^16
`define rot8  32'd14656         //0.2238'C *2^16
`define rot9  32'd7360          //0.1119'C *2^16
`define rot10 32'd3648          //0.0560'C *2^16
`define rot11 32'd1856          //0.0280'C *2^16
`define rot12 32'd896           //0.0140'C *2^16
`define rot13 32'd448           //0.0070'C *2^16
`define rot14 32'd256           //0.0035'C *2^16
`define rot15 32'd128           //0.0018'C *2^16      tanx15=1>>15
=>x=0.0018 c

parameter Pipeline = 16;
parameter K = 32'h09b74;    //K=0.607253*2^16,32'h09b74,
cosx0*cosx1*....*cosx15 = 0.607253

reg signed  [31:0]      Sin;
reg signed  [31:0]      Cos;
reg signed  [31:0]      Error;
reg signed  [31:0]      x0,   y0,   z0;
reg signed  [31:0]      x1,   y1,   z1;
reg signed  [31:0]      x2,   y2,   z2;
reg signed  [31:0]      x3,   y3,   z3;
reg signed  [31:0]      x4,   y4,   z4;
reg signed  [31:0]      x5,   y5,   z5;
reg signed  [31:0]      x6,   y6,   z6;
reg signed  [31:0]      x7,   y7,   z7;
```

```
reg signed  [31:0]      x8,  y8,  z8;
reg signed  [31:0]      x9,  y9,  z9;
reg signed  [31:0]      x10, y10, z10;
reg signed  [31:0]      x11, y11, z11;
reg signed  [31:0]      x12, y12, z12;
reg signed  [31:0]      x13, y13, z13;
reg signed  [31:0]      x14, y14, z14;
reg signed  [31:0]      x15, y15, z15;
reg signed  [31:0]      x16, y16, z16;
reg         [ 1:0]      Quadrant [Pipeline:0];

always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        x0 <= 1'b0;
        y0 <= 1'b0;
        z0 <= 1'b0;
    end
    else if(enable) begin
        if (angletan)begin
            x0 <= 1<<16;;
            y0 <= tan_value;  //sin0 == 0
            z0 <= 0;
        end
        else begin
            x0 <= K;
            y0 <= 32'd0;  //sin0 == 0
            //z0 <= Phase[15:0] << 16;
            z0 <= Phase[23:0];
        end
    end
end

always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        x1 <= 1'b0;
        y1 <= 1'b0;
        z1 <= 1'b0;
    end
    else if(angletan)begin
        if(y0[31])
```

```
        begin
          x1 <= x0 - y0;
          y1 <= y0 + x0;
          z1 <= z0 - `rot0;
        end
        else
        begin
          x1 <= x0 + y0;
          y1 <= y0 - x0;
          z1 <= z0 + `rot0;
        end
    end
    else begin
        if(z0[31])begin
          x1 <= x0 + y0;
          y1 <= y0 - x0;
          z1 <= z0 + `rot0;
        end
        else  begin
          x1 <= x0 - y0;
          y1 <= y0 + x0;
          z1 <= z0 - `rot0;
        end
    end
end


always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        x2 <= 1'b0;
        y2 <= 1'b0;
        z2 <= 1'b0;
    end
    else if(angletan)begin
        if(y1[31])
        begin
             x2 <= x1 - (y1 >>> 1);
             y2 <= y1 + (x1 >>> 1);
             z2 <= z1 - `rot1;
        end
        else
        begin
```

```
            x2 <= x1 + (y1 >>> 1);
            y2 <= y1 - (x1 >>> 1);
            z2 <= z1 + `rot1;
        end
    end
    else begin
        if(z1[31])
        begin
             x2 <= x1 + (y1 >>> 1);
             y2 <= y1 - (x1 >>> 1);
             z2 <= z1 + `rot1;
        end
        else
        begin
            x2 <= x1 - (y1 >>> 1);
            y2 <= y1 + (x1 >>> 1);
            z2 <= z1 - `rot1;
        end
     end
end

always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        x3 <= 1'b0;
        y3 <= 1'b0;
        z3 <= 1'b0;
    end
    else if(angletan)begin
        if(y2[31])
        begin
             x3 <= x2 - (y2 >>> 2);
             y3 <= y2 + (x2 >>> 2);
             z3 <= z2 - `rot2;
        end
        else
        begin
            x3 <= x2 + (y2 >>> 2);
            y3 <= y2 - (x2 >>> 2);
            z3 <= z2 + `rot2;
        end
    end
     else if(z2[31])
```

```
    begin
        x3 <= x2 + (y2 >>> 2);
        y3 <= y2 - (x2 >>> 2);
        z3 <= z2 + `rot2;
    end
    else
    begin
        x3 <= x2 - (y2 >>> 2);
        y3 <= y2 + (x2 >>> 2);
        z3 <= z2 - `rot2;
    end
end

always @ (posedge clk or negedge rst_n)
begin
     if(!rst_n)
     begin
         x4 <= 1'b0;
         y4 <= 1'b0;
         z4 <= 1'b0;
     end
     else if(angletan)begin
         if(y3[31])
         begin
              x4 <= x3 -    (y3 >>> 3);
              y4 <= y3 +    (x3 >>> 3);
              z4 <= z3 - `rot3;
         end
         else
         begin
              x4 <= x3 +    (y3 >>> 3);
              y4 <= y3 -    (x3 >>> 3);
              z4 <= z3 + `rot3;
         end
    end
     else if(z3[31])
    begin
        x4 <= x3 + (y3 >>> 3);
        y4 <= y3 - (x3 >>> 3);
        z4 <= z3 + `rot3;
    end
    else
    begin
        x4 <= x3 - (y3 >>> 3);
```

```
        y4 <= y3 + (x3 >>> 3);
        z4 <= z3 - `rot3;
    end
end

always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        x5 <= 1'b0;
        y5 <= 1'b0;
        z5 <= 1'b0;
    end
    else if(angletan)begin
        if(y4[31])
        begin
            x5 <= x4 -   (y4 >>> 4);
            y5 <= y4 +   (x4 >>> 4);
            z5 <= z4 - `rot4;
        end
        else
        begin
            x5 <= x4 +   (y4 >>> 4);
            y5 <= y4 -   (x4 >>> 4);
            z5 <= z4 + `rot4;
        end
    end
    else if(z4[31])
    begin
        x5 <= x4 + (y4 >>> 4);
        y5 <= y4 - (x4 >>> 4);
        z5 <= z4 + `rot4;
    end
    else
    begin
        x5 <= x4 - (y4 >>> 4);
        y5 <= y4 + (x4 >>> 4);
        z5 <= z4 - `rot4;
    end
end
.........
```

Perl 生成 code

由于代码重复度很高，可以用 perl脚本实现代码自动生成，如下：

```
#!/usr/bin/perl
#use autodie;

print  ("hell0,world!\n");

my $width_num = 31  ;
my $reg_num = 17  ;
my $end = "\n";
my $success = open FILE, ">cordic_auto_gen.sv";
   if  (!$success){
 print "error opening file \n"
}

print  FILE "module Cordic\n";
print  FILE "(\n";
print  FILE "    clk,\n";;
print  FILE "    rst_n,\n";
print  FILE "    Phase,\n";
print  FILE "    enable,\n";
print  FILE "    angletan,\n";
print  FILE "    tan_value,\n";
print  FILE "    Sin,\n";
print  FILE "    Cos,\n";
print  FILE "    Error\n";
print  FILE ");\n";

print  FILE "parameter Pipeline = 16;\n";
print  FILE "parameter K = 32'h09b74;    //K=0.607253*2^16,32'h09b74,
cosx0*cosx1*....*cosx15 = 0.607253\n";

print  FILE "input                      clk;\n";
print  FILE "input                      rst_n;\n";
print  FILE "input                      enable;\n";
print  FILE "input                      angletan;\n";
print  FILE "input       [ $width_num:0]          tan_value;\n";
print  FILE "input       [ $width_num:0]         Phase;\n";
print  FILE "output      [ $width_num:0]         Sin;\n";
print  FILE "output      [ $width_num:0]         Cos;\n";
print  FILE "output      [ $width_num:0]         Error;\n";
```

```
print  FILE "`define rot0         45*(2<<16)          //tanx0=1
=> x=45c\n";
print  FILE "`define rot1         26.5651*(2<<16)     //tanx1>>1     =>
x=26.5651 c\n";
print  FILE "`define rot2         14.0362*(2<<16)     //tanx2=1>>2
=> x=14.0362 c\n";
print  FILE "`define rot3         7.1250*(2<<16) \n";
print  FILE "`define rot4         3.5763*(2<<16) \n";
print  FILE "`define rot5         1.7899*(2<<16) \n";
print  FILE "`define rot6         0.8952*(2<<16) \n";
print  FILE "`define rot7         0.4476*(2<<16) \n";
print  FILE "`define rot8         0.2238*(2<<16) \n";
print  FILE "`define rot9         0.1119*(2<<16) \n";
print  FILE "`define rot10        0.0560*(2<<16) \n";
print  FILE "`define rot11        0.0280*(2<<16) \n";
print  FILE "`define rot12        0.0140*(2<<16) \n";
print  FILE "`define rot13        0.0070*(2<<16) \n";
print  FILE "`define rot14        0.0035*(2<<16) \n";
print  FILE "`define rot15        0.0018*(2<<16)      // tanx15=1>>15
=>x=0.0018 c\n";

print  FILE "reg signed  [$width_num:0]      Sin;\n";
print  FILE "reg signed  [$width_num:0]      Cos;\n";
print  FILE "reg signed  [$width_num:0]      Error;\n";

my $len = 0;
for($len=0;$len<$reg_num;$len++) {
print  FILE "reg signed  [$width_num:0]      x($len),  y ($len),
z($len);\n";
}

print  FILE "always @ (posedge clk or negedge rst_n)\n";
print  FILE "begin\n";
print  FILE "    if(!rst_n)\n";
print  FILE "    begin\n";
print  FILE "        x0 <= 1'b0;\n    "       ;
print  FILE "        y0 <= 1'b0;\n";
print  FILE "        z0 <= 1'b0;\n";
print  FILE "    end\n";
print  FILE "    else if(enable) begin \n";
print  FILE "        if  (angletan)begin \n";
print  FILE "            x0 <= 1<<16;  \n"    ;
print  FILE "            y0 <= tan_value;  //sin0 == 0\n";
print  FILE "            z0 <= 0;\n";
```

```
print  FILE "        end \n";
print  FILE "        else begin\n";
print  FILE "            x0 <= K; \n     ";
print  FILE "            y0 <= 32'd0;  //sin0 == 0 \n" ;
print  FILE "            //z0 <= Phase[15:0] << 16; \n";
print  FILE "            z0 <= Phase[23:0]; \n";
print  FILE "        end \n";
print  FILE "    end \n";
print  FILE "end \n";


my $len = 1;
my $lenx = 0;
for($len=1;$len<$reg_num;$len++) {
    $lenx = $len - 1;
    print  FILE "always @ (posedge clk or negedge rst_n) \n";
    print  FILE "begin \n";
    print  FILE "    if(!rst_n) \n";
    print  FILE "    begin \n";
    print  FILE "        x$len <= 1'b0 ;       \n"          ;
    print  FILE "        y$len <= 1'b0; \n";
    print  FILE "        z$len <= 1'b0; \n";
    print  FILE "    end \n";
    print  FILE "    else if(angletan) begin \n" ;
    print  FILE "        if(y ($lenx)[31]) begin \n" ;
    print  FILE "          x$len <= x$lenx - y$lenx; \n";
    print  FILE "          y$len <= y$lenx + x$lenx; \n";
    print  FILE "          z$len <= z$lenx - `rot$lenx; \n";
    print  FILE "        end \n " ;
    print  FILE "        else begin \n" ;
    print  FILE "          x$len <= x$lenx + y$lenx; \n";
    print  FILE "          y$len <= y$lenx - x$lenx; \n";
    print  FILE "          z$len <= z$lenx + `rot$lenx; \n";
    print  FILE "        end \n" ;
    print  FILE "    end \n" ;
    print  FILE "    else begin \n";
    print  FILE "        if(z($lenx)[31])begin \n";
    print  FILE "          x$len <= x$lenx + y$lenx; \n";
    print  FILE "          y$len <= y$lenx - x$lenx; \n";
    print  FILE "          z$len <= z$lenx + `rot$lenx; \n";
    print  FILE "        end \n";
    print  FILE "        else  begin \n";
    print  FILE "          x$len <= x$lenx - y$lenx; \n";
    print  FILE "          y$len <= y$lenx + x$lenx; \n";
```

```
    print  FILE "          z$len <= z$lenx - `rot$lenx; \n";
    print  FILE "        end \n";
    print  FILE "    end \n";
    print  FILE "end \n";
}

print  FILE "always @ (posedge clk or negedge rst_n)\n";
print  FILE "begin\n";
print  FILE "    if(!rst_n) begin\n";
print  FILE "        Cos <= 1'b0;\n";
print  FILE "        Sin <= 1'b0;\n";
print  FILE "        Error <= 1'b0;\n";
print  FILE "    end\n";
print  FILE "    else begin\n";
print  FILE "        Error <= z16;\n";
print  FILE "        case(Phase[25:24])\n";
print  FILE "            2'b00: //if the Phase is in first Quadrant,\n";
print  FILE "                begin\n";
print  FILE "                    Cos <= x16;\n";
print  FILE "                    Sin <= y16;\n";
print  FILE "                end\n";
print  FILE "            2'b01: //if the Phase is in second Quadrant,\n";
print  FILE "                begin\n";
print  FILE "                    Cos <= ~(y16) ;//-Sin\n";
print  FILE "                    Sin <= y16;//Cos\n";
print  FILE "                end\n";
print  FILE "            2'b10: //if the Phase is in third Quadrant,t\n";
print  FILE "                begin\n";
print  FILE "                    Cos <= ~(x16) ;//-Cos\n";
print  FILE "                    Sin <= ~(y16) ;//-Sin\n";
print  FILE "                end\n";
print  FILE "            2'b11: //if the Phase is in forth Quadrant,\n";
print  FILE "                begin\n";
print  FILE "                    Cos <= x16;//Sin\n";
print  FILE "                    Sin <= ~(y16) ;//-Cos\n";
print  FILE "                end\n";
print  FILE "        endcase\n";
print  FILE "    end\n";
print  FILE "end\n";
print  FILE "endmodule\n";
```

优化版本

只用一组寄存器及计数器来实现

```
module Cordic
(
    clk,
    rst_n,
    Phase,
    enable,
    angletan, /无此功能
    tan_value, //无此功能
    Quadrant,  //表示哪个象限
    Sin,
    Cos,
    Error
);

parameter Pipeline = 8;  //计算 8 次则放大 8 倍,对应 8 级流水
input                       clk;
input                       rst_n;
input                       enable;
input                       angletan;
input       [Pipeline*2-1:0]          tan_value;
input       [1:0]                     Quadrant;
input       [Pipeline*2-1:0]          Phase;
output      [Pipeline*2-1:0]          Sin;
output      [Pipeline*2-1:0]          Cos;
output      [Pipeline*2-1:0]          Error;

parameter K = 0.607253*('h1<<<Pipeline);
//K=0.607253*2^16,32'h09b74,  cosx0*cosx1*....*cosx15 = 0.607253
//parameter K = 32'h09b74;    //K=0.607253*2^16,32'h09b74,
cosx0*cosx1*....*cosx15 = 0.607253

parameter rot0  = 45*       ('h1<<<Pipeline)          ;//tanx0=1
=> x=45c
parameter rot1  = 26.5651 *('h1<<<Pipeline )   ;// tanx1>>1    =>
x=26.5651 c
parameter rot2  = 14.0362 *('h1<<<Pipeline )   ;// tanx2=1>>2    =>
x=14.0362 c
parameter rot3  = 7.1250  *('h1<<<Pipeline )    ;//
parameter rot4  = 3.5763  *('h1<<<Pipeline )    ;//
parameter rot5  = 1.7899  *('h1<<<Pipeline )    ;//
```

```
parameter rot6  = 0.8952  *('h1<<<Pipeline )    ;//
parameter rot7  = 0.4476  *('h1<<<Pipeline )    ;//
parameter rot8  = 0.2238  *('h1<<<Pipeline )    ;//
parameter rot9  = 0.1119  *('h1<<<Pipeline )    ;//
parameter rot10 = 0.0560  *('h1<<<Pipeline )    ;//
parameter rot11 = 0.0280  *('h1<<<Pipeline )    ;//
parameter rot12 = 0.0140  *('h1<<<Pipeline )    ;//
parameter rot13 = 0.0070  *('h1<<<Pipeline )    ;//
parameter rot14 = 0.0035  *('h1<<<Pipeline )    ;//
parameter rot15 = 0.0018  *('h1<<<Pipeline )    ;// tanx15=1>>15
=>x=0.0018 c


reg signed  [Pipeline*2-1:0]      Sin;
reg signed  [Pipeline*2-1:0]      Cos;
reg signed  [Pipeline*2-1:0]      Error;
reg signed  [Pipeline*2-1:0]      x0,   y0,   z0;
reg [4:0]   step_cnt;


always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        step_cnt <= 5'b0;
    end
    else if(enable) begin
        if(step_cnt!=(Pipeline))begin
            step_cnt <= step_cnt+5'h1;
        end
    end
    else begin
        step_cnt <= 5'b0;
    end
end

always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        x0 <= 1'b0;
        y0 <= 1'b0;
        z0 <= 1'b0;
    end
```

```
else if(!enable) begin
    x0 <= K;
    y0 <= 0;  //sin0 == 0
    z0 <= Phase;
end
else begin
    if(step_cnt==Pipeline)begin
         x0 <= x0 ;
         y0 <= y0 ;
         z0 <= z0 ;
    end
    else begin
        case(step_cnt)
            4'h0: begin
                if(z0[Pipeline*2-1])begin
                  x0 <= x0 + y0;
                  y0 <= y0 - x0;
                end
                else  begin
                  x0 <= x0 - y0;
                  y0 <= y0 + x0;
                end
                if(z0[Pipeline*2-1]) begin
                    z0 <= z0 + rot0;
                end
                else begin
                    z0 <= z0 - rot0;
                end
            end
            4'h1: begin
                if(z0[Pipeline*2-1])begin
                  x0 <= x0 + (y0 >>> 1);
                  y0 <= y0 - (x0 >>> 1);
                end
                else  begin
                  x0 <= x0 - (y0 >>> 1);
                  y0 <= y0 + (x0 >>> 1);
                end
                if(z0[Pipeline*2-1]) begin
                    z0 <= z0 + rot1;
                end
                else begin
                    z0 <= z0 - rot1;
                end
```

```
end
4'h2: begin
    if(z0[Pipeline*2-1])begin
      x0 <= x0 + (y0 >>> 2);
      y0 <= y0 - (x0 >>> 2);
    end
    else  begin
      x0 <= x0 - (y0 >>> 2);
      y0 <= y0 + (x0 >>> 2);
    end
    if(z0[Pipeline*2-1]) begin
        z0 <= z0 + rot2;
    end
    else begin
        z0 <= z0 - rot2;
    end
end
4'h3: begin
    if(z0[Pipeline*2-1])begin
      x0 <= x0 + (y0 >>> 3);
      y0 <= y0 - (x0 >>> 3);
    end
    else  begin
      x0 <= x0 - (y0 >>> 3);
      y0 <= y0 + (x0 >>> 3);
    end
    if(z0[Pipeline*2-1]) begin
        z0 <= z0 + rot3;
    end
    else begin
        z0 <= z0 - rot3;
    end
end
4'h4: begin
    if(z0[Pipeline*2-1])begin
      x0 <= x0 + (y0 >>> 4);
      y0 <= y0 - (x0 >>> 4);
    end
    else  begin
      x0 <= x0 - (y0 >>> 4);
      y0 <= y0 + (x0 >>> 4);
    end
    if(z0[Pipeline*2-1]) begin
        z0 <= z0 + rot4;
```

```
        end
        else begin
            z0 <= z0 - rot4;
        end
    end
    4'h5: begin
        if(z0[Pipeline*2-1])begin
          x0 <= x0 + (y0 >>> 5);
          y0 <= y0 - (x0 >>> 5);
        end
        else  begin
          x0 <= x0 - (y0 >>> 5);
          y0 <= y0 + (x0 >>> 5);
        end
        if(z0[Pipeline*2-1]) begin
            z0 <= z0 + rot5;
        end
        else begin
            z0 <= z0 - rot5;
        end
    end
    4'h6: begin
        if(z0[Pipeline*2-1])begin
          x0 <= x0 + (y0 >>> 6);
          y0 <= y0 - (x0 >>> 6);
        end
        else  begin
          x0 <= x0 - (y0 >>> 6);
          y0 <= y0 + (x0 >>> 6);
        end
        if(z0[Pipeline*2-1]) begin
            z0 <= z0 + rot6;
        end
        else begin
            z0 <= z0 - rot6;
        end
    end
    4'h7: begin
        if(z0[Pipeline*2-1])begin
          x0 <= x0 + (y0 >>> 7);
          y0 <= y0 - (x0 >>> 7);
        end
        else  begin
          x0 <= x0 - (y0 >>> 7);
```

```
            y0 <= y0 +  (x0 >>> 7);
        end
        if(z0[Pipeline*2-1]) begin
            z0 <= z0 + rot7;
        end
        else begin
            z0 <= z0 - rot7;
        end
    end
    4'h8: begin
        if(z0[Pipeline*2-1])begin
          x0 <= x0 +  (y0 >>> 8);
          y0 <= y0 -  (x0 >>> 8);
        end
        else  begin
          x0 <= x0 -  (y0 >>> 8);
          y0 <= y0 +  (x0 >>> 8);
        end
        if(z0[Pipeline*2-1]) begin
            z0 <= z0 + rot8;
        end
        else begin
            z0 <= z0 - rot8;
        end
    end
    4'h9: begin
        if(z0[Pipeline*2-1])begin
          x0 <= x0 +  (y0 >>> 9);
          y0 <= y0 -  (x0 >>> 9);
        end
        else  begin
          x0 <= x0 -  (y0 >>> 9);
          y0 <= y0 +  (x0 >>> 9);
        end
        if(z0[Pipeline*2-1]) begin
            z0 <= z0 + rot9;
        end
        else begin
            z0 <= z0 - rot9;
        end
    end
    4'ha: begin
        if(z0[Pipeline*2-1])begin
          x0 <= x0 +  (y0 >>> 10);
```

```
            y0 <= y0 - (x0 >>> 10);
        end
        else  begin
            x0 <= x0 - (y0 >>> 10);
            y0 <= y0 + (x0 >>> 10);
        end
        if(z0[Pipeline*2-1]) begin
            z0 <= z0 + rot10;
        end
        else begin
            z0 <= z0 - rot10;
        end
    end
    4'hb: begin
        if(z0[Pipeline*2-1])begin
            x0 <= x0 + (y0 >>> 11);
            y0 <= y0 - (x0 >>> 11);
        end
        else  begin
            x0 <= x0 - (y0 >>> 11);
            y0 <= y0 + (x0 >>> 11);
        end
        if(z0[Pipeline*2-1]) begin
            z0 <= z0 + rot11;
        end
        else begin
            z0 <= z0 - rot11;
        end
    end
    4'hc: begin
        if(z0[Pipeline*2-1])begin
            x0 <= x0 + (y0 >>> 12);
            y0 <= y0 - (x0 >>> 12);
        end
        else  begin
            x0 <= x0 - (y0 >>> 12);
            y0 <= y0 + (x0 >>> 12);
        end
        if(z0[Pipeline*2-1]) begin
            z0 <= z0 + rot12;
        end
        else begin
            z0 <= z0 - rot12;
        end
```

```
end
4'hd: begin
    if(z0[Pipeline*2-1])begin
      x0 <= x0 + (y0 >>> 13);
      y0 <= y0 - (x0 >>> 13);
    end
    else  begin
      x0 <= x0 - (y0 >>> 13);
      y0 <= y0 + (x0 >>> 13);
    end
    if(z0[Pipeline*2-1]) begin
        z0 <= z0 + rot13;
    end
    else begin
        z0 <= z0 - rot13;
    end
end
4'he: begin
    if(z0[Pipeline*2-1])begin
      x0 <= x0 + (y0 >>> 14);
      y0 <= y0 - (x0 >>> 14);
    end
    else  begin
      x0 <= x0 - (y0 >>> 14);
      y0 <= y0 + (x0 >>> 14);
    end
    if(z0[Pipeline*2-1]) begin
        z0 <= z0 + rot14;
    end
    else begin
        z0 <= z0 - rot14;
    end
end
4'hf: begin
    if(z0[Pipeline*2-1])begin
      x0 <= x0 + (y0 >>> 15);
      y0 <= y0 - (x0 >>> 15);
    end
    else  begin
      x0 <= x0 - (y0 >>> 15);
      y0 <= y0 + (x0 >>> 15);
    end
    if(z0[Pipeline*2-1]) begin
        z0 <= z0 + rot15;
```

```
                    end
                    else begin
                        z0 <= z0 - rot15;
                    end
                end
                default:begin
                    x0 <= x0;
                    y0 <= y0;
                    z0 <= z0;
                end
            endcase
        end
    end
end


always @ (posedge clk or negedge rst_n)
begin
    if(!rst_n)
    begin
        Cos <= 1'b0;
        Sin <= 1'b0;
        Error <= 1'b0;
    end
    else
    begin
        if(step_cnt==(Pipeline*+1))begin
            Error <= z0;
            case(Quadrant)
                2'b00: //if the Phase is in first Quadrant,the
Sin(X)=Sin(A),Cos(X)=Cos(A)
                    begin
                        Cos <= x0;
                        Sin <= y0;
                    end
                2'b01: //if the Phase is in second Quadrant,the
Sin(X)=Sin(A+90)=CosA,Cos(X)=Cos(A+90)=-SinA
                    begin
                        Cos <= ~(y0) ;//-Sin
                        Sin <= y0;//Cos
                    end
                2'b10: //if the Phase is in third Quadrant,the
Sin(X)=Sin(A+180)=-SinA,Cos(X)=Cos(A+180)=-CosA
                    begin
```

```
                        Cos <= ~(x0) ;//-Cos
                        Sin <= ~(y0) ;//-Sin
                    end
                2'b11: //if the Phase is in forth Quadrant,the
Sin(X)=Sin(A+270)=-CosA,Cos(X)=Cos(A+270)=SinA
                    begin
                        Cos <= x0;//Sin
                        Sin <= ~(y0) ;//-Cos
                    end
            endcase
        end
    end
end
endmodule
```

用于验证的 cos,sin 模型

实际是根据rtl转化过来,不能用于模型,但因为验证正确,可以作为模型.用于计算 fft 系数
支持负数处理,但可以全部用正数,负角度可以转化为正角度,输入范围为(0~90度)

```
task cos_sin(input real angle, output  bit signed [31:0]
cos_result,output  bit signed [31:0] sin_result);
    real  shift16 ;
    bit signed [31:0] x[16] ,y[16] ,z[16] ;
    bit[31:0] rot[16];
    rot[00] =  32'd2949120;
    rot[01] =  32'd1740992;
    rot[02] =  32'd919872 ;
    rot[03] =  32'd466944 ;
    rot[04] =  32'd234368 ;
    rot[05] =  32'd117312 ;
    rot[06] =  32'd58688  ;
    rot[07] =  32'd29312  ;
    rot[08] =  32'd14656  ;
    rot[09] =  32'd7360   ;
    rot[10] =  32'd3648   ;
    rot[11] =  32'd1856   ;
    rot[12] =  32'd896    ;
    rot[13] =  32'd448    ;
    rot[14] =  32'd256    ;
    rot[15] =  32'd128    ;
    shift16 = angle* (32'h1<<<16);
    x[0] = 32'h09b74;
```

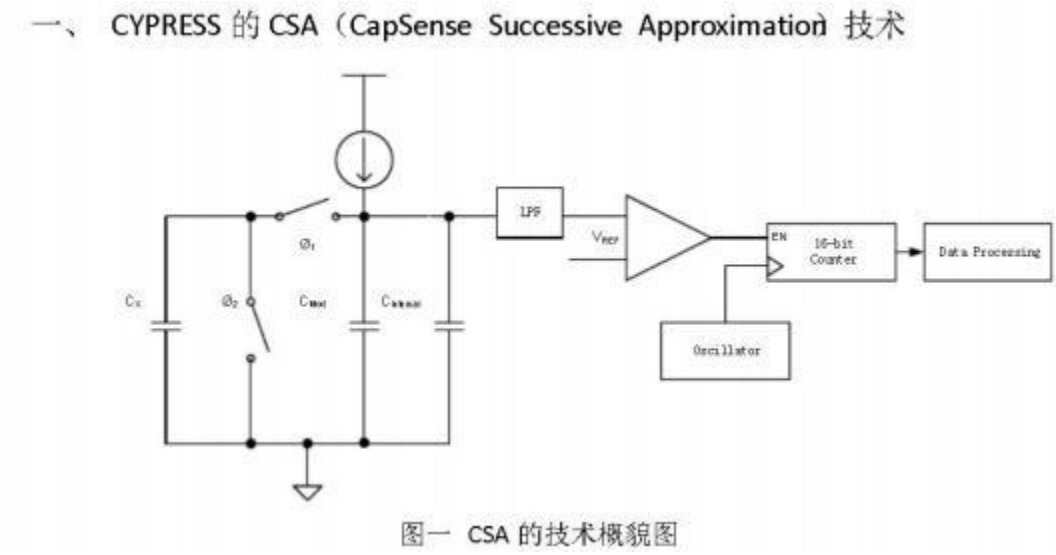
```
    y[0] = 32'd0;
    z[0] = shift16;
    for(int i=1;i<17;i++)begin
       if(z[i-1][31])begin
          x[i] = x[i-1] + (y[i-1]>>>(i-1));
          y[i] = y[i-1] - (x[i-1]>>>(i-1));
          z[i] = z[i-1] + rot[i-1];
        end
        else  begin
          x[i] = x[i-1] - (y[i-1]>>>(i-1));
          y[i] = y[i-1] + (x[i-1]>>>(i-1));
          z[i] = z[i-1] - rot[i-1];
        end
    end
    cos_result = x[15];
    sin_result = y[15];
    print_decimal(cos_result,16);
    print_decimal(sin_result,16);
endtask
```

deci 表示用多少bit 来表示小数,下面是 16bit 表示整数,16bit 表示小数

```
task print_decimal(input[31:0] datain,input int deci);//16 bit decimal
    int datatemp;
    string sig[2];
    bit get_sig;
    sig[1] ="-";
    sig[0] ="";
    datatemp =0;
    get_sig= datain[31];
    if(get_sig) datain= ~datain+32'h1;
    for(int i=0;i<deci;i++)begin
        datatemp = datain[deci-1-i]*(5000000>>i) +datatemp;
    end
    if(datatemp>=1000000)
        $display ("%0h changto is   %s%0d.%0d",
datain,get_sig?sig[1]:sig[0], datain>>deci,datatemp);
    else if(datatemp>=100000)
        $display ("%0h changto is   %s%0d.0%0d",
datain,get_sig?sig[1]:sig[0], datain>>deci,datatemp);
    else if(datatemp>=10000)
        $display ("%0h changto is   %s%0d.00%0d",
datain,get_sig?sig[1]:sig[0], datain>>deci,datatemp);
    else if(datatemp>=1000)
```

```
        $display ("%0h changto is   %s%0d.000%0d",
datain,get_sig?sig[1]:sig[0],  datain>>deci,datatemp);
    else if(datatemp>=100)
        $display ("%0h changto is   %s%0d.0000%0d",
datain,get_sig?sig[1]:sig[0],  datain>>deci,datatemp);
    else if(datatemp>=10)
        $display ("%0h changto is   %s%0d.00000%0d",
datain,get_sig?sig[1]:sig[0],  datain>>deci,datatemp);
    else
        $display ("%0h changto
is   %s%0d.000000%0d",datain,get_sig?sig[1]:sig[0],
datain>>deci,datatemp);
endtask
```

几种触控技术分析

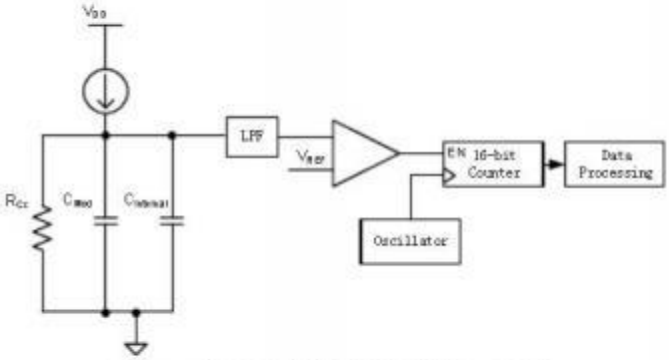


图一 CSA 的技术概貌图

CSA 工作原理如下所示：

1)、校准阶段

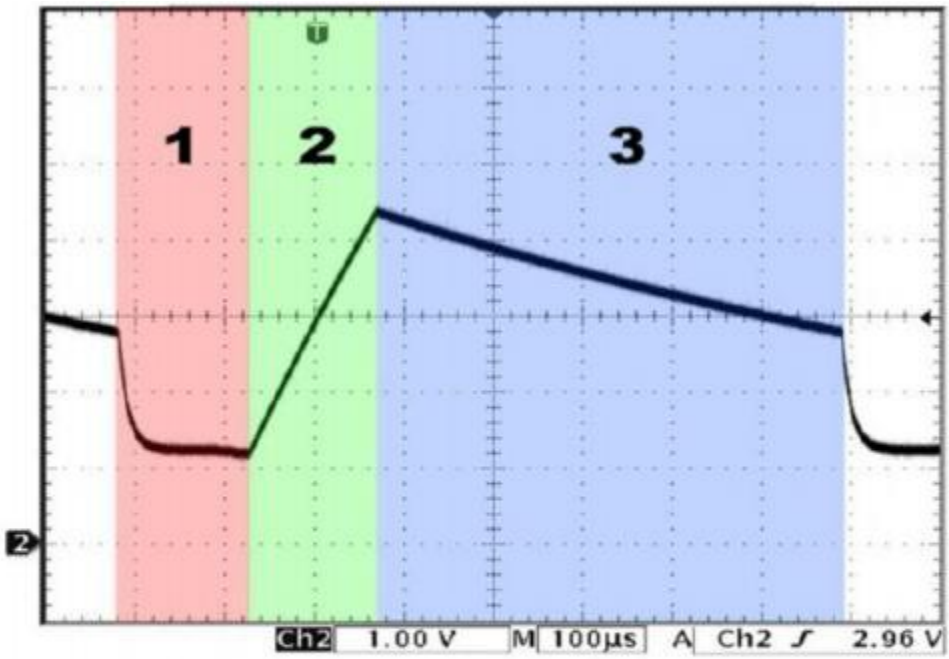
1、把Φ1 和Φ2 这两个时钟交替开启，那样感应电容的作用就像一个电阻，CMOD 电容上的电压就会稳定在一个固定值 V_{Start} = I/fC，等效电路如下图所示。



图二 电容采样阶段的等效电路图

2、把开关Φ1 断开一固定时间，CMOD 上的电压就会线性上升，把 CMOD 上的电压和 VREF 进行比较，当 CMOD 上电压小于 VREF 时，计数器开始计数；当 CMOD 上电压大于 VREF 时，停止计数。

3、当计数器的输出为 0 时，说明 Vstart 电压高于 VREF，IDAC 电流过大；如果计数器的输出值远大于 0，那么说明 Vstart 电压过低，IDAC 电流过小。下一步就会依据逐次逼近算法来设置 IDAC 电流，目标是把 IDAC 设置在一个适当的值，使得 Vstart 电压略低于 VREF，计数器的输出略大于 0。这时 IDAC 设置完毕，校准完成。

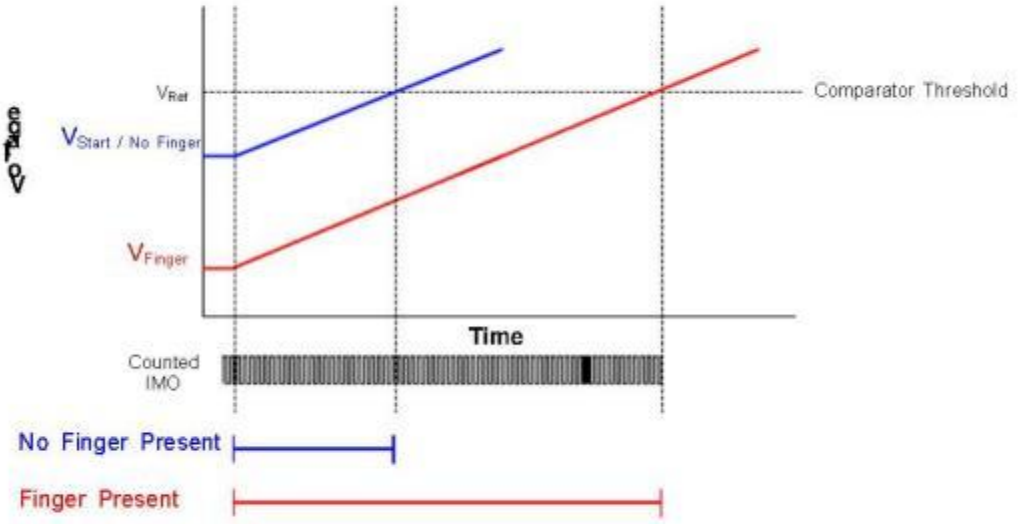


图三 CMOD 上的电压波形图

CMOD 上的电压如上图所示，第一阶段为Φ1 和Φ2 交替开启阶段，CMOD 电压稳定在 I/fc；第二阶段为Φ1 断开，CMOD 上电压直线上升阶段；第三阶段为数据处理阶段，CSA 模块空闲，CMOD 上电压下降是由于外接示波器漏电导致。

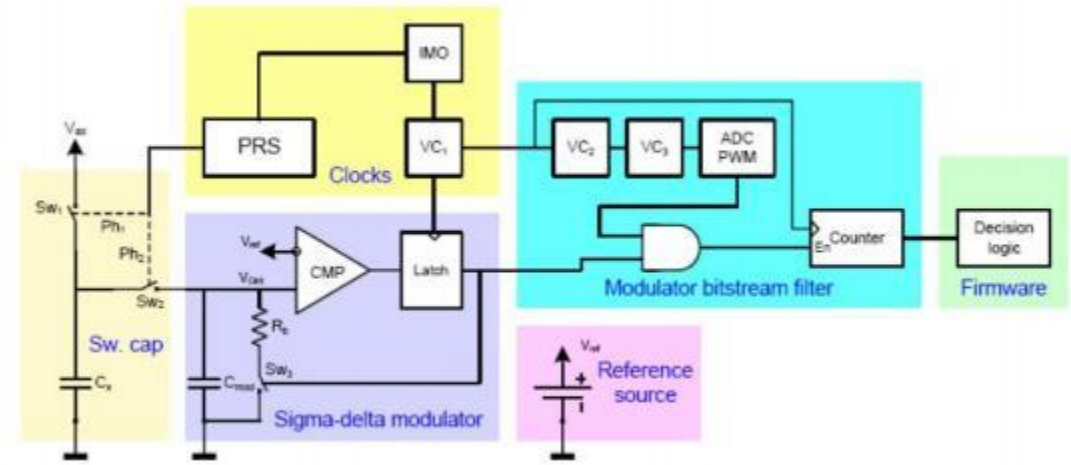
2)、检测阶段

当有手指触摸时，CX 电容变大，VStart = I/fC电压变低，计数器的计数值会变大，那么就可以据此判定触摸的发生。



图四 触摸发生前后的计数值对比

二、 CYPRESS 的 CSD（CapSense Sigma Delta）技术

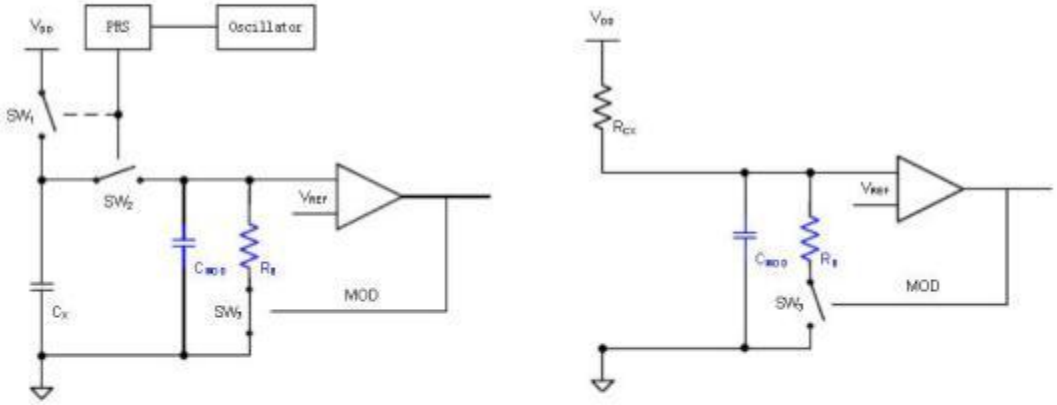


图五 CSD 技术的系统框图

从上图可以看出，在 CSD 技术框图中，主要有一下几个模块：开关电容、时钟、sigma-delta 调制器、滤波器。

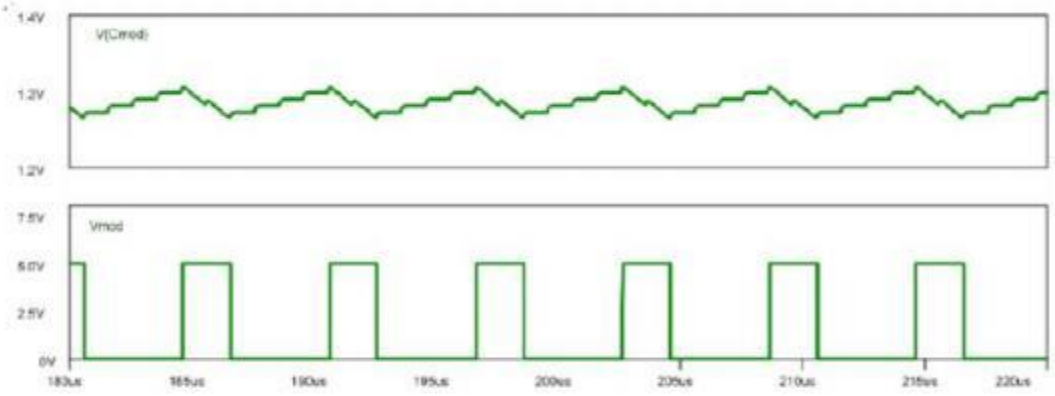
CSD 结构的工作原理如下所示。

步骤 1：开关 SW1 和 SW2 交替打开，开关电容电路等效为一个电阻，向 CMOD 电容充电：CMOD 上电压达到 VREF 时，比较器翻转，SW3 闭合，CMOD 开始通过 RB 对地放电，CMOD 电压开始下降，当 CMOD 上电压低于 VREF 时，SW3 断开。CMOD 又开始充电，如此周期往复。



图六 调制器的等效工作原理图

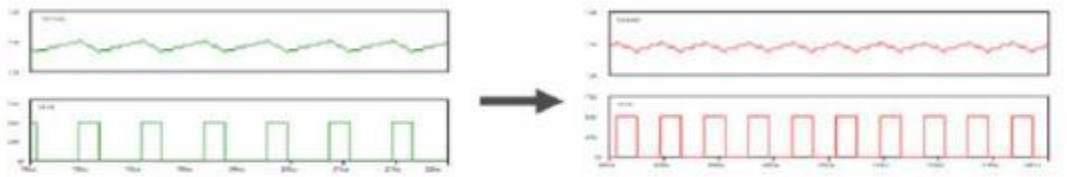
在图中，等效电阻 RCX 大小为 1/fc。



图七 CMOD 以及比较器的输出波形图

步骤 2：测量输出的占空比。将比较器的输出信号滤波后，进行占空比的测量。

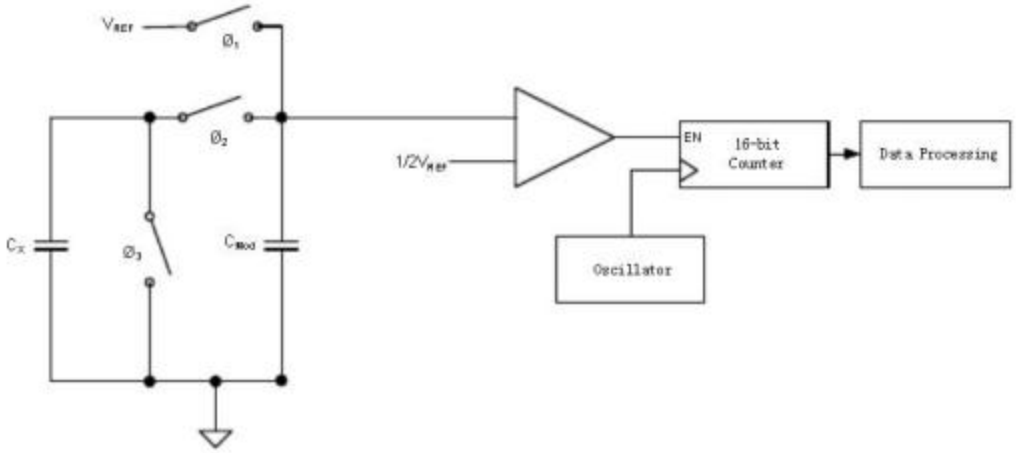
- Increase in C_x reduces R_{CX} causing more current to charge C_{Mod}.
- More current results in C_{Mod} charging faster.
- More current results in slower discharge times.



图八 触摸前后的波形比对

触摸后电容变小?冲放电更快?

三、 SIGMA 的 CDC（Cap Digital Conversion）技术



图九 SIGMA 的 CDC 技术原理图

SIGMA 的 CDC 技术原理图如上图所示，其工作原理为：
步骤 1：开关Φ1 导通，CMOD 上电压升至 VREF。
步骤 2：开关Φ2 和Φ3 交替导通，CX 开关部分电路等效电阻为 R= 1/fC 这时 CMOD 上电压开始下降，当下降到 1/2VREF 时，计数结束。
当有手指触摸时，电容 CX 变大，等效电阻 R 变小，计数值变少。

四、 总结

原理比较：

1 CSA 技术的触摸计数值为：初始电压 VStart = I/f ，结束电压 VREF，充电电流大小为 I 对 CMOD 进行充电。

2 CSD 技术充电电流的等效电阻值为 I/f ，放电电阻为固定阻值 RB，计数值的多少取决于计数器开启时间的长短。

3 SIGMA 的 CDC 技术的初始电压为 VREF，结束电压为 1/2VREF，等效放电电阻为 I/f 。计数值的多少可以通过 CMOD 大小来调节。

精度比较：

1 CSA 技术的精度可以通过 CMOD 的电容大小来调节分辨率。但是实际上 CMOD 的大小受限于 电容的大小和计数时间的长短等因素，分辨率的提高还是会有限制。

2 CSD 技术的精度的提高只需要简单的增加计数的时间即可，通过加长计数时间，CSD 可以做到很高精度。

3 SIGMA 的 CDC 技术的精度提高也和 CSA 类似，也是通过改变 CMOD 电容来改变分辨率的。

三者都可以做到高精度，只是 CSD 的精度设置不需要改变硬件电路就可以实现，这会在有些应用上带来方便。比如可以在初期检测有无手指触摸的时候用更低的精度，后期检测具体触摸电容改变大小时用更高的精度。

抗干扰能力比较：

与 CSR（CapSense Relaxation Oscillator）技术相比，CSA、CSD、CDC 都在开关电容电路与外部调制电容之间组成了低阻通路，具有更强的抗干扰能力。

CSA 技术中的电容 只在阶段一连接内部总线，计数器技术阶段 与内部总线断开，这大大提升了 CSA 系统的抗干扰能力。

CDC 技术类似，第一阶段电容 会与电容 CMOD 断开，第二阶段基准电压 VREF 会与电容 CMOD 断开，所以这会提升系统对电容 和基准电压 VREF 的抗干扰能力。

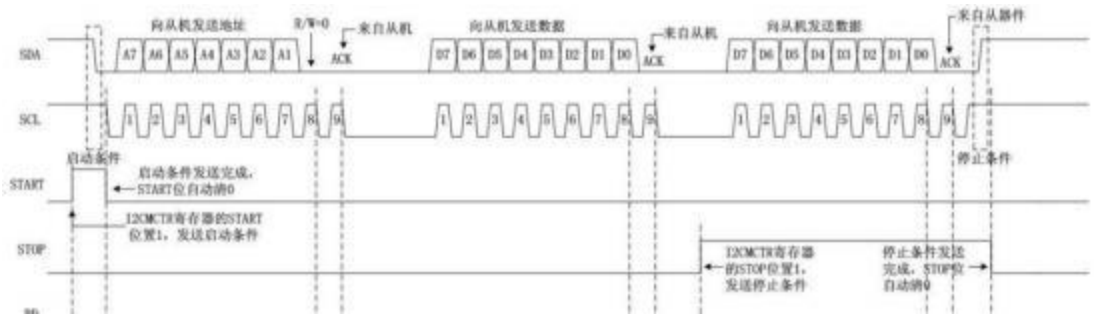
CSD 技术抗干扰能力的提升，具有更加灵活多样的处理方法，其可以在开关电容部分加入伪随机时钟（PRS），提高系统的抗中频噪声能力。也可以通过设置不同的计数时间，来设置不同的抗噪声能力。

三者与 CSR 技术相比，都有更强的抗干扰能力，只是 CSD 技术对于抗干扰可以有更灵活多样的处理方式。

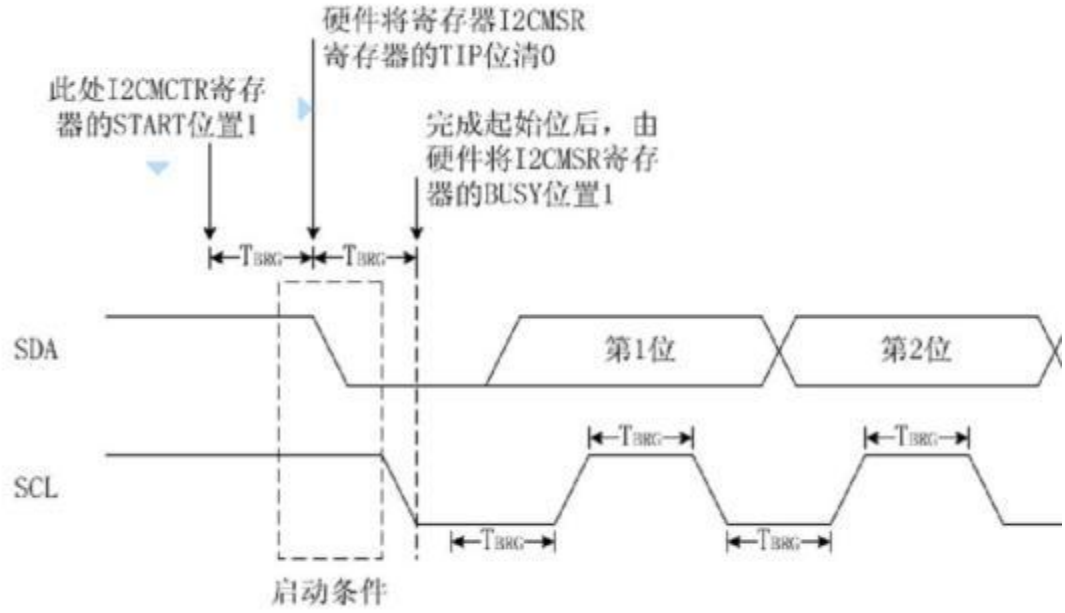
通讯设计注意点(i2c, uart, spi 等)

I2c master 电路

时序：

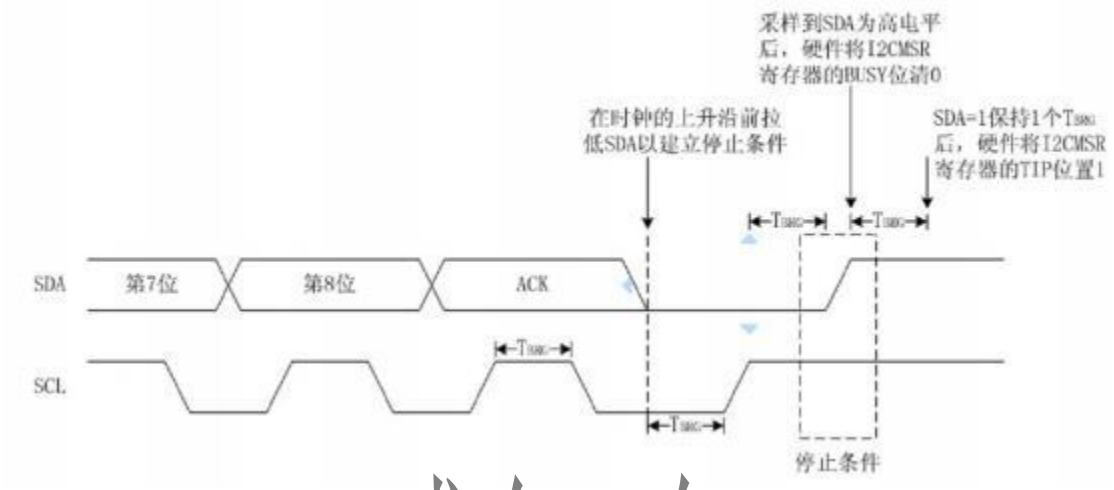


连续发送数据时不需要 start,stop 信号,这两个信号表示贞信号的开始和结束.
启动波形



设计波形

花了 5 拍时钟,正常设计
停止波形



咸鱼:东

5 拍时钟,正常设计
设计时序

发送数据时,比 5 个时钟多了红蓝间隔的时间,此处时间是用来检测从设备是否拉低了时钟(主要是外部来的信号需要做滤波处理),也就是当主机要把时钟拉高时,需要检测从设备是否拉住了时钟,如果拉住了,则不能发送数据!等从机释放后再发送数据.当数据线被从设备拉低,则重新传输数据---此为 i2c 协议规定

注意:由于 i2c 要求总线上要上拉电阻,所以设计时,当需要输出高电平,则把 io 口设置为输入态,当要输出低电平,则把 io 口设置为输出态,而输出数据可以固定为 0;